

run_installer_gemini.sh

```
1 #!/usr/bin/env bash
2 #
3 # Launch interactive AI Breadboard Project Installer using Gemini CLI with gemini-3.1-flash-lite
4 #
5
6 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
7 SKILL_DIR="$(dirname "$SCRIPT_DIR")"
8 PROJECT_ROOT="$(dirname "$(dirname "$SKILL_DIR")")"
9
10 INSTRUCTION_FILE="$SKILL_DIR/INSTALL-INSTRUCTION.md"
11 if [ ! -f "$INSTRUCTION_FILE" ]; then
12     INSTRUCTION_FILE="$PROJECT_ROOT/INSTALL-INSTRUCTION.md"
13 fi
14
15 echo "=====
16 =
17 echo "□ Launching Gemini CLI Installer (gemini-3.1-flash-lite
18 )"
19 echo "=====
20 =
21 echo "Project Root: $PROJECT_ROOT"
22 echo "Instruction: $INSTRUCTION_FILE"
23 echo ""
24
25 PROMPT="You are the AI Breadboard Interactive Installer. Read
26 and follow all instructions from '$INSTRUCTION_FILE'. Guide me
27 step by step through installing AI Breadboard, perform pre-flight checks, handle errors, and verify directories and components at the end."
28
29 # Execute Gemini CLI with gemini-3.1-flash-lite model
30 gemini --model "gemini-3.1-flash-lite" --prompt "$PROMPT"
```

verify_installation.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Verification and Structural Integrity Test for
  AI Breadboard
4 # =====
5 # Description:
6 #   Automated diagnostic tool that verifies presence and integ
  rity of all
7 #   required project directories, essential configuration file
  s, Python
8 #   virtual environment binaries, and core package imports.
9 #
10 # Examples:
11 #   python verify_installation.py
12 #   python verify_installation.py --json
13 #   python verify_installation.py --project-root "C:\path\to\A
  I-Breadboard"
14 #
15 # File: .agents/skills/project-installer/scripts/verify_instal
  lation.py
16 # Project: AI Breadboard
17 # Package: Installation
18 # Module: Verification
19 # Class: InstallationVerifier
20 # Function: main
21 # Author: hypo69
22 # Copyright: © 2026 hypo69
23 # =====
24
25 import argparse
26 import importlib
27 import json
28 import os
```

```

29 import platform
30 import subprocess
31 import sys
32 from dataclasses import dataclass, field, asdict
33 from pathlib import Path
34 from typing import Dict, List, Optional, Tuple
35
36
37 @dataclass
38 class VerificationResult:
39     """Dataclass holding the verification results and status."""
40     is_valid: bool = True
41     os_name: str = platform.system()
42     project_root: str = ""
43     missing_directories: List[str] = field(default_factory=list)
44     existing_directories: List[str] = field(default_factory=list)
45     missing_files: List[str] = field(default_factory=list)
46     existing_files: List[str] = field(default_factory=list)
47     venv_status: Dict[str, str] = field(default_factory=dict)
48     module_status: Dict[str, str] = field(default_factory=dict)
49     errors: List[str] = field(default_factory=list)
50     warnings: List[str] = field(default_factory=list)
51
52
53 class InstallationVerifier:
54     """Validates full structural integrity and runtime readiness of AI Breadboard."""
55
56     REQUIRED_DIRECTORIES: List[str] = [
57         "core",
58         "scripts",
59         "install",
60         "launchers",
61         "docs",

```

```

62         "tests",
63         "webinterface",
64         ".ai",
65         ".agents/skills",
66         "tmp",
67     ]
68
69     REQUIRED_FILES: List[str] = [
70         "header.py",
71         "config.json",
72         "requirements.txt",
73         "INSTALL-INSTRUCTION.md",
74     ]
75
76     ALTERNATIVE_FILES: List[Tuple[str, str]] = [
77         (".env", ".env.example"),
78     ]
79
80     CORE_MODULES: List[str] = [
81         "fastapi",
82         "uvicorn",
83         "pydantic",
84         "dotenv",
85         "cryptography",
86         "aiohttp",
87         "platformdirs",
88     ]
89
90     def __init__(self, project_root: Optional[Path] = None):
91         """Initialize the InstallationVerifier.
92
93         Args:
94             project_root (Optional[Path]): Root directory of the project.
95         """
96         if project_root:
97             self.root: Path = project_root.resolve()
98         else:

```

```

99         # Try finding root by locating header.py or config
    .json
100         current: Path = Path(__file__).resolve().parent
101         found_root: Optional[Path] = None
102         for parent in [current] + list(current.parents):
103             if (parent / "header.py").exists() and (parent
    / "config.json").exists():
104                 found_root = parent
105                 break
106         self.root = found_root if found_root else Path.cwd
    ().resolve()
107
108     def check_directories(self, result: VerificationResult) ->
    None:
109         """Verify presence of all mandatory directories.
110
111         Args:
112             result (VerificationResult): Result object to upda
    te.
113         """
114         for dir_name in self.REQUIRED_DIRECTORIES:
115             dir_path: Path = self.root / dir_name
116             if dir_path.is_dir():
117                 result.existing_directories.append(dir_name)
118             else:
119                 result.missing_directories.append(dir_name)
120                 result.is_valid = False
121                 result.errors.append(f"Missing required direct
    ory: {dir_name}")
122
123     def check_files(self, result: VerificationResult) -> None:
124         """Verify presence of all mandatory and configuration
    files.
125
126         Args:
127             result (VerificationResult): Result object to upda
    te.
128         """

```

```

129         for file_name in self.REQUIRED_FILES:
130             file_path: Path = self.root / file_name
131             if file_path.is_file():
132                 result.existing_files.append(file_name)
133             else:
134                 result.missing_files.append(file_name)
135                 result.is_valid = False
136                 result.errors.append(f"Missing required file:
137 | {file_name}")
138
139         for primary, fallback in self.ALTERNATIVE_FILES:
140             has_primary: bool = (self.root / primary).is_file(
141 | )
142             has_fallback: bool = (self.root / fallback).is_fil
143 | e()
144             if has_primary:
145                 result.existing_files.append(primary)
146             elif has_fallback:
147                 result.existing_files.append(fallback)
148                 result.warnings.append(
149 |         f"Optional file {primary} missing, found t
150 | emplate {fallback}"
151 |         )
152             else:
153                 result.missing_files.append(primary)
154                 result.warnings.append(
155 |         f"Neither {primary} nor {fallback} was fou
156 | nd in project root"
157 |         )
158
159     def check_virtual_environment(self, result: VerificationRe
160 | sult) -> Optional[Path]:
161         """Check the status and executable path of the virtual
162 | environment.
163
164 | Args:
165 |     result (VerificationResult): Result object to upda
166 | te.

```

```

159
160         Returns:
161             Optional[Path]: Path to python executable if valid
162             , None otherwise.
163             """
164             venv_dir: Path = self.root / "venv"
165             is_windows: bool = platform.system() == "Windows"
166
167             if is_windows:
168                 py_exe: Path = venv_dir / "Scripts" / "python.exe"
169                 pip_exe: Path = venv_dir / "Scripts" / "pip.exe"
170             else:
171                 py_exe = venv_dir / "bin" / "python"
172                 pip_exe = venv_dir / "bin" / "pip"
173
174             if venv_dir.is_dir() and py_exe.is_file():
175                 result.venv_status["path"] = str(py_exe)
176                 result.venv_status["pip"] = str(pip_exe) if pip_ex
177             e.is_file() else "not_found"
178                 result.venv_status["status"] = "active"
179
180             # Check Python version from venv
181             try:
182                 proc = subprocess.run(
183                     [str(py_exe), "--version"],
184                     capture_output=True,
185                     text=True,
186                     timeout=5,
187                 )
188                 result.venv_status["version"] = proc.stdout.st
189             rip() or proc.stderr.strip()
190             except Exception as ex:
191                 result.venv_status["version_check_error"] = st
192             r(ex)
193
194             return py_exe
195         else:
196             result.venv_status["path"] = "missing"
197             result.venv_status["status"] = "missing"

```

```

193         result.warnings.append(
194             f"Virtual environment missing at {venv_dir}. Run
|un installer to create venv."
195         )
196         return None
197
198     def check_core_modules(
199         self, result: VerificationResult, python_exe: Optional
| [Path] = None
200     ) -> None:
201         """Check whether core dependencies can be imported.
202
203         Args:
204             result (VerificationResult): Result object to upda
|te.
205             python_exe (Optional[Path]): Specific python binar
|y to test against.
206         """
207         if python_exe and python_exe.is_file():
208             # Test in external venv subprocess
209             for module_name in self.CORE_MODULES:
210                 try:
211                     proc = subprocess.run(
212                         [str(python_exe), "-c", f"import {modu
|le_name}"],
213                         capture_output=True,
214                         text=True,
215                         timeout=5,
216                     )
217                     if proc.returncode == 0:
218                         result.module_status[module_name] = "o
|k"
219                     else:
220                         result.module_status[module_name] = "m
|issing"
221                         result.warnings.append(
222                             f"Module '{module_name}' failed to
|import in virtualenv: {proc.stderr.strip()}"

```



```

223         )
224         except Exception as ex:
225             result.module_status[module_name] = f"erro
r: {ex}"
226         else:
227             # Test in current Python environment
228             for module_name in self.CORE_MODULES:
229                 try:
230                     importlib.import_module(module_name)
231                     result.module_status[module_name] = "ok"
232                 except ImportError:
233                     result.module_status[module_name] = "missi
ng"
234                     result.warnings.append(
235                         f"Module '{module_name}' is not instal
led in current Python environment"
236                     )
237                 except Exception as ex:
238                     result.module_status[module_name] = f"load
_error: {type(ex).__name__}"
239                     result.warnings.append(
240                         f"Module '{module_name}' encountered l
oad issue: {ex}"
241                     )
242
243     def verify_all(self) -> VerificationResult:
244         """Execute full verification checks across directories
, files, venv, and modules.
245
246         Returns:
247             VerificationResult: Populated validation result ob
ject.
248         """
249         result: VerificationResult = VerificationResult(
250             project_root=str(self.root)
251         )
252
253         self.check_directories(result)

```

```

254         self.check_files(result)
255         python_exe: Optional[Path] = self.check_virtual_enviro
nment(result)
256         self.check_core_modules(result, python_exe)
257
258         return result
259
260
261 def main() -> int:
262     """CLI Entry point for verification tool.
263
264     Returns:
265         int: Exit code 0 if required components exist, 1 other
wise.
266     """
267     parser = argparse.ArgumentParser(
268         description="Verify AI Breadboard installation integri
ty and directories."
269     )
270     parser.add_argument(
271         "--project-root",
272         type=str,
273         default="",
274         help="Path to AI Breadboard project root (default: aut
o-detected)",
275     )
276     parser.add_argument(
277         "--json",
278         action="store_true",
279         help="Output structured JSON results",
280     )
281     args = parser.parse_args()
282
283     project_root_path: Optional[Path] = Path(args.project_root
) if args.project_root else None
284     verifier: InstallationVerifier = InstallationVerifier(proj
ect_root_path)
285     result: VerificationResult = verifier.verify_all()

```

```

286
287     if args.json:
288         print(json.dumps(asdict(result), indent=2, ensure_ascii
289                             i=False))
289     else:
290         print("=" * 65)
291         print("  AI Breadboard Installation Verification")
292         print("=" * 65)
293         print(f"Project Root: {result.project_root}")
294         print(f"Platform:      {result.os_name}")
295         print(f"Status:        {'  PASSED' if result.is_valid e
296                             lse '  FAILED'}")
296         print("-" * 65)
297         print(f"Directories:  {len(result.existing_directories
298                             )} verified, {len(result.missing_directories)} missing")
298         print(f"Files:        {len(result.existing_files)} ver
299                             ified, {len(result.missing_files)} missing")
299         print(f"Venv:          {result.venv_status.get('status'
300                             , 'unknown')} ({result.venv_status.get('path', 'N/A')})")
300
301         if result.module_status:
302             ok_mods = sum(1 for v in result.module_status.valu
303                             es() if v == "ok")
303             print(f"Modules:      {ok_mods}/{len(result.module
304                             _status)} imported successfully")
304
305         if result.errors:
306             print("\n  Errors:")
307             for err in result.errors:
308                 print(f"    • {err}")
309
310         if result.warnings:
311             print("\n  Warnings:")
312             for warn in result.warnings:
313                 print(f"    • {warn}")
314
315         print("=" * 65)
316

```

```
317 |     return 0 if result.is_valid else 1
318 |
319 |
320 | if __name__ == "__main__":
321 |     sys.exit(main())
322 |
```

search_media.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Module
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: search_media.py
9 # Project: ai-breadboard
10 # Package: .agents.skills.rag-search-manager.scripts
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import argparse
16 import sys
17 from pathlib import Path
18 import json
19 import os
20 from dotenv import load_dotenv
21
22 # Добавляем корень проекта в путь для импортов
23 sys.path.insert(0, str(Path(__file__).resolve().parents[3]))
24
25 from src.ai.gemini.user_query_rag import search_user_context
26 from plugins.web_search import WebSearchPlugin
27
28 # Loading .env
29 load_dotenv(Path(__file__).resolve().parents[3] / '.env')
30
31 async def search_media(query):
32     print(f"👉 Начинаю поиск: '{query}'")
33
```

```
34 | # 1. Попытка поиска в RAG
35 | print("□ Проверяю RAG...")
36 | try:
37 |     user_id = 'default_user'
38 |     api_key = os.getenv('AGY_API_KEY')
39 |
40 |     if not api_key:
41 |         print("□ API ключ не найден в .env.")
42 |     else:
43 |         results = search_user_context(user_id, api_key, query, top_k=3, threshold=0.3)
44 |         if results:
45 |             print(f"□ Найдено в RAG: {json.dumps(results, indent=2, ensure_ascii=False)}")
46 |             return
47 |         else:
48 |             print("□ В RAG ничего не найдено.")
49 | except Exception as e:
50 |     print(f"□ Error при поиске в RAG: {e}")
51 |
52 | # 2. Если не найдено, поиск в интернете
53 | print("□ RAG не дал результатов. Ищу в интернете...")
54 | try:
55 |     # Initialization плагина поиска
56 |     search_plugin = WebSearchPlugin()
57 |
58 |     # Выполнение поиска (используем движок по умолчанию из конфига)
59 |     results = await search_plugin.search(query)
60 |
61 |     if results:
62 |         print(f"□ Найдено в интернете: {json.dumps(results, indent=2, ensure_ascii=False)}")
63 |     else:
64 |         print("□ В интернете ничего не найдено.")
65 | except Exception as e:
66 |     print(f"□ Error при поиске в интернете: {e}")
67 |
```

```
68 | if __name__ == "__main__":
69 |     import asyncio
70 |     parser = argparse.ArgumentParser()
71 |     parser.add_argument("--query", required=True, help="Запрос
    | для поиска")
72 |     args = parser.parse_args()
73 |     asyncio.run(search_media(args.query))
74 |
```

init_skill.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Skill Initializer Utility
4 # =====
5 # Description:
6 #   Scaffolds a new standard AI Breadboard agent skill directory in
7 #   .agents/skills/<skill_name> with SKILL.md, README.md, scripts/,
8 #   references/,
9 #   and assets/ folders.
10 #
11 # Examples:
12 #   python init_skill.py my-awesome-skill --description "Description of skill"
13 #
14 # File: .agents/skills/skill-factory/scripts/init_skill.py
15 # Project: AI Breadboard
16 # Package: SkillFactory
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # =====
20
21 import argparse
22 import sys
23 from pathlib import Path
24
25 # Find project root
26 CURRENT_DIR = Path(__file__).resolve().parent
27 SKILLS_ROOT = CURRENT_DIR.parents[1] # .agents/skills/
28 PROJECT_ROOT = CURRENT_DIR.parents[2] # project root
29
30 def create_skill(name: str, description: str = "") -> Path:
```



```

31 |     """Create a new standard skill directory structure in .agents/skills/<name>.
32 |
33 |     Args:
34 |         name (str): The name of the new skill in kebab-case.
35 |         description (str): Short description of the skill.
36 |
37 |     Returns:
38 |         Path: The path to the created skill directory.
39 |     """
40 |     skill_dir = SKILLS_ROOT / name
41 |     if skill_dir.exists():
42 |         print(f"Skill directory already exists: {skill_dir}")
43 |     )
44 |
45 |     skill_dir.mkdir(parents=True, exist_ok=True)
46 |     (skill_dir / "scripts").mkdir(exist_ok=True)
47 |     (skill_dir / "references").mkdir(exist_ok=True)
48 |     (skill_dir / "assets").mkdir(exist_ok=True)
49 |
50 |     skill_md_content = f"""---
51 | name: {name}
52 | description: {description or f"Agent skill for {name}."}
53 | ---
54 |
55 | # {name.replace('-', ' ').title()} Skill
56 |
57 | ## Purpose
58 | {description or f"Provides capabilities for {name}."}
59 |
60 | ## Usage & Protocol
61 | Describe how AI agents should execute this skill and what triggers its activation.
62 |
63 | ## Directory Structure
64 | - `SKILL.md`: Main instructions and frontmatter contract.
65 | - `README.md`: English documentation for developers.

```

```

66 | - `scripts/`: Executable helper tools.
67 | - `references/`: Reference documentation and guidelines.
68 | - `assets/`: Static data, examples, and assets.
69 | """
70 |
71 |     readme_content = f"""# {name.replace('-', ' ').title()}
72 |
73 | ## Overview
74 | {description or f"Skill module for {name}."}
75 |
76 | ## Location
77 | `agents/skills/{name}/`
78 | """
79 |
80 |     (skill_dir / "SKILL.md").write_text(skill_md_content, encoding="utf-8")
81 |     (skill_dir / "README.md").write_text(readme_content, encoding="utf-8")
82 |
83 |     print(f"    Successfully created skill '{name}' at:\n    {skill_dir}")
84 |     return skill_dir
85 |
86 |
87 | def main() -> int:
88 |     """CLI entry point for skill initialization."""
89 |     parser = argparse.ArgumentParser(description="Create a new
90 |     AI Breadboard agent skill.")
91 |     parser.add_argument("name", help="Name of the skill in kebab-case (e.g. data-analyzer)")
92 |     parser.add_argument("--description", "-d", default="", help="Description of the skill")
93 |
94 |     args = parser.parse_args()
95 |
96 |     create_skill(args.name, args.description)
97 |     return 0

```

```
98 | if __name__ == "__main__":  
99 |     sys.exit(main())  
100 |
```

pack.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Skill Packaging Utility
4 # =====
5 # Description:
6 #   Packages a skill directory from .agents/skills/<skill_name
7 #   > into a
8 #   distributable .skill archive in the dist/ folder.
9 #
10 # Examples:
11 #   python pack.py .agents/skills/project-installer
12 #   python pack.py project-installer
13 #
14 # File: .agents/skills/skill-factory/scripts/pack.py
15 # Project: AI Breadboard
16 # Package: SkillFactory
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # =====
20
21 import argparse
22 import os
23 import sys
24 import zipfile
25 from pathlib import Path
26
27 CURRENT_DIR = Path(__file__).resolve().parent
28 SKILLS_ROOT = CURRENT_DIR.parents[1] # .agents/skills/
29
30 def package_skill(skill_target: str) -> Path:
31     """Package a skill folder into a .skill archive.
32
```

```

33     Args:
34         skill_target (str): Name of the skill or path to the s
kill directory.
35
36     Returns:
37         Path: Path to the created .skill package.
38     """
39     target_path = Path(skill_target)
40     if not target_path.exists():
41         # Try finding in .agents/skills/<skill_target>
42         target_path = SKILLS_ROOT / skill_target
43
44     skill_path = target_path.resolve()
45     if not skill_path.is_dir():
46         raise FileNotFoundError(f"Skill directory not found: {
skill_path}")
47
48     output_path = skill_path / "dist"
49     output_path.mkdir(parents=True, exist_ok=True)
50
51     skill_name = skill_path.name
52     skill_file = output_path / f"{skill_name}.skill"
53
54     print(f"␣ Packaging skill from {skill_path} into {skill_f
ile}...")
55
56     with zipfile.ZipFile(skill_file, "w", zipfile.ZIP_DEFLATED
) as zipf:
57         for root, dirs, files in os.walk(skill_path):
58             # Skip build artifacts, dist, git, and python byte
code
59             dirs[:] = [d for d in dirs if d not in ["dist", ".
git", "__pycache__", ".pytest_cache"]]
60
61             for file in sorted(files):
62                 file_path = Path(root) / file
63                 arcname = file_path.relative_to(skill_path)
64                 zipf.write(file_path, arcname)

```

```
65 |
66 |     print(f"    Skill successfully packaged: {skill_file}")
67 |     return skill_file
68 |
69 |
70 | def main() -> int:
71 |     """CLI Entry point for pack utility."""
72 |     parser = argparse.ArgumentParser(description="Package an A
73 | I Breadboard agent skill.")
74 |     parser.add_argument("skill", help="Name or path to skill f
75 | older (e.g. project-installer)")
76 |     args = parser.parse_args()
77 |
78 |     try:
79 |         package_skill(args.skill)
80 |         return 0
81 |     except Exception as ex:
82 |         print(f"    Failed to package skill: {ex}", file=sys.std
83 | err)
84 |         return 1
85 |
86 |
87 | if __name__ == "__main__":
88 |     sys.exit(main())
89 |
```

delete_media.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Module
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: delete_media.py
9 # Project: ai-breadboard
10 # Package: .agents.skills.smart-deletion-duplicates.scripts
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import sqlite3
16 import os
17 import pandas as pd
18 import argparse
19 import ast
20
21 DB_PATH = r'C:\mediateka\plugins\media_organizer\data\media.db'
22
23 def delete_media(file_path):
24     # 1. Удаление файла с диска
25     if os.path.exists(file_path):
26         try:
27             os.remove(file_path)
28             print(f"Файл удален: {file_path}")
29         except Exception as e:
30             print(f"Error при удалении файла {file_path}: {e}")
31     )
32     return False
```

```
32     else:
33         print(f"Файл не найден: {file_path}")
34         return False
35
36     # 2. Удаление записи из БД
37     try:
38         conn = sqlite3.connect(DB_PATH)
39         cursor = conn.cursor()
40         cursor.execute("DELETE FROM media WHERE path = ?", (file_path,))
41         conn.commit()
42         conn.close()
43         print(f"Запись удалена из БД: {file_path}")
44         return True
45     except Exception as e:
46         print(f"Error при удалении записи из БД {file_path}: {e}")
47         return False
48
49 def main():
50     parser = argparse.ArgumentParser()
51     parser.add_argument('--file', required=True, help='Path to CSV with paths to delete')
52     parser.add_argument('--execute', action='store_true', help='Execute deletion')
53     args = parser.parse_args()
54
55     df = pd.read_csv(args.file)
56
57     files_to_delete = []
58     for _, row in df.iterrows():
59         paths = ast.literal_eval(row['to_delete'])
60         files_to_delete.extend(paths)
61
62     if not args.execute:
63         print("Dry run: list of files to delete:")
64         for f in files_to_delete:
65             print(f)
```



```
66 |         print("Run with --execute to proceed.")
67 |         return
68 |
69 |     for f in files_to_delete:
70 |         delete_media(f)
71 |
72 | if __name__ == '__main__':
73 |     main()
74 |
```

process_scheduled_deletions.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Module
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: process_scheduled_deletions.py
9 # Project: ai-breadboard
10 # Package: .agents.skills.smart-deletion-duplicates.scripts
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import sqlite3
16 import os
17 import argparse
18
19 DB_PATH = r'C:\mediateka\plugins\media_organizer\data\media.db'
20
21 def process_deletions(execute=False):
22     conn = sqlite3.connect(DB_PATH)
23     cursor = conn.cursor()
24     cursor.execute("SELECT id, path FROM media WHERE status =
25     'кандидат на удаление'")
26
27     if not candidates:
28         print("Нет записей со статусом 'кандидат на удаление'.
29 ")
30
31     conn.close()
32     return
```

```

31 |
32 |     print(f"Найдено {len(candidates)} кандидатов на удаление."
33 | )
34 |     for id, path in candidates:
35 |         # Check существования файла
36 |         if os.path.exists(path):
37 |             if execute:
38 |                 try:
39 |                     os.remove(path)
40 |                     print(f"Файл удален: {path}")
41 |                     # Удаление записи из БД
42 |                     cursor.execute("DELETE FROM media WHERE id
43 | = ?", (id,))
44 |                     print(f"Запись {id} удалена из БД.")
45 |                     except Exception as e:
46 |                         print(f"Error при удалении {path}: {e}")
47 |                     else:
48 |                         print(f"[DRY RUN] Файл доступен для удаления:
49 | {path}")
50 |                     else:
51 |                         print(f"[SKIP] Файл недоступен (диск отключен?): {
52 | path}")
53 |
54 |     if execute:
55 |         conn.commit()
56 |         conn.close()
57 |
58 | def main():
59 |     parser = argparse.ArgumentParser()
60 |     parser.add_argument('--execute', action='store_true', help
61 | = 'Execute deletion')
62 |     args = parser.parse_args()
63 |
64 |     process_deletions(execute=args.execute)
65 |
66 | if __name__ == '__main__':
67 |     main()

```


chat.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Фиктивный объект модели для инициализации плаги
  ина.
4 # =====
5 # Description:
6 #   Консольный интерфейс для взаимодействия с RAG-системой мед
  иатеки.
7 #
8 # File: chat.py
9 # Project: ai-breadboard
10 # Package: .agents.skills.web-chat-cli.src
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import sys
16 import argparse
17 import asyncio
18 # Импортируем RAG-плагин для использования логики
19 from plugins.rag import RAGPlugin
20
21 class FakeAIModel:
22     """Фиктивный объект модели для инициализации плагина."""
23     async def chat(self, message: str, system_instruction: str
  = "", model_name: str = "") -> str:
24         """Эмуляция ответа ИИ."""
25         return f"Ответ от {model_name} на: {message}"
26
27 def parse_arguments() -> argparse.Namespace:
28     """Парсинг аргументов командной строки.
29
30     Args:
```

```

31         None
32
33     Returns:
34         argparse.Namespace: Аргументы командной строки.
35     """
36     parser = argparse.ArgumentParser(description="Web Chat CLI
37 ")
38     parser.add_argument('--model', default='gemini-1.5-flash',
39 help='Имя модели')
40     parser.add_argument('--debug', action='store_true', help='
41 Режим отладки')
42     return parser.parse_args()
43
44 async def chat_loop(args: argparse.Namespace) -> None:
45     """Основной цикл интерактивного чата.
46
47     Args:
48         args (argparse.Namespace): Аргументы командной строки.
49
50     Returns:
51         None
52     """
53     print(f"Web Chat CLI запущен (Модель: {args.model}). Введи
54 те 'exit' для выхода.")
55
56     # Initialization плагина
57     ai_model = FakeAIModel()
58     rag_plugin = RAGPlugin(ai_model)
59
60     while True:
61         try:
62             user_input = input("Вы: ")
63             if user_input.lower() == 'exit':
64                 break
65
66             # Обработка через плагин (асинхронно)
67             print("Система: Обработка...")
68

```

```
65 |         response = None
66 |         async for output in rag_plugin._handle(user_input,
67 |         model_name=args.model):
68 |             if "text" in output:
69 |                 response = output["text"]
70 |             elif "status" in output:
71 |                 print(f"[{output['status']}]")
72 |
73 |             if response:
74 |                 print(f"Система: {response}")
75 |             else:
76 |                 print("Система: (Нет ответа)")
77 |
78 |         except EOFError:
79 |             break
80 |         except KeyboardInterrupt:
81 |             print("\nВыход...")
82 |             break
83 |         except Exception as e:
84 |             print(f"Error: {e}")
85 |
86 | if __name__ == '__main__':
87 |     args = parse_arguments()
88 |     asyncio.run(chat_loop(args))
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Module
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: __init__.py
9 # Project: ai-breadboard
10 # Package: .agents.skills.web-chat-cli.src
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15
```


test_chat.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Testing web-chat-cli argument parsing
4 # =====
5 # Description:
6 #   Unit tests for web-chat-cli argument parsing and configura
7 #   tion defaults.
8 #
9 # File: test_chat.py
10 # Project: ai-breadboard
11 # Package: .agents.skills.web-chat-cli.tests
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import importlib.util
17 import os
18 import sys
19 from pathlib import Path
20 from unittest.mock import patch
21
22 # Import chat dynamically relative to this test file
23 chat_file = Path(__file__).resolve().parents[1] / "src" / "cha
24 t.py"
25 spec = importlib.util.spec_from_file_location("chat", str(chat
26 _file))
27 if spec and spec.loader:
28     chat = importlib.util.module_from_spec(spec)
29     spec.loader.exec_module(chat)
30     parse_arguments = getattr(chat, "parse_arguments", None)
31 else:
32     parse_arguments = None
33
```

```
31 |
32 | def test_parse_arguments_default():
33 |     """Test argument parsing with default parameters."""
34 |     if parse_arguments is None:
35 |         return
36 |     test_args = ["chat.py"]
37 |     with patch("sys.argv", test_args):
38 |         args = parse_arguments()
39 |         assert args.model == "gemini-1.5-flash", "Default model sh
ould be gemini-1.5-flash"
40 |
41 |
42 | def test_parse_arguments_custom():
43 |     """Test argument parsing with custom parameters."""
44 |     if parse_arguments is None:
45 |         return
46 |     test_args = ["chat.py", "--model", "gemini-1.5-pro", "--de
bug"]
47 |     with patch("sys.argv", test_args):
48 |         args = parse_arguments()
49 |         assert args.model == "gemini-1.5-pro", "Model should be up
dated to gemini-1.5-pro"
50 |         assert args.debug is True, "Debug mode should be enabled"
51 |
```

inspect_user_rags.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Module
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: inspect_user_rags.py
9 # Project: ai-breadboard
10 # Package: .ai.tools.ai
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import sqlite3
16
17 for dbpath in [
18     'src/ai/gemini/user_rags/user_rag_1.db',
19     'src/ai/gemini/user_rags/user_rag_anon_10.0.0.4.db',
20 ]:
21     conn = sqlite3.connect(dbpath)
22     tables = conn.execute("SELECT name FROM sqlite_master WHERE
23     type='table'").fetchall()
24     print(f'--- {dbpath} ---')
25     print('Tables:', [t[0] for t in tables])
26     for t in tables:
27         cnt = conn.execute(f'SELECT COUNT(*) FROM {t[0]}').fetchone()[0]
28         cols = [c[1] for c in conn.execute(f'PRAGMA table_info
29         ({t[0]}')').fetchall()]
30         print(f'  {t[0]}: {cnt} rows, cols={cols}')
31         if cnt > 0:
32             row = conn.execute(f'SELECT * FROM {t[0]} LIMIT 1')
```

```
    ).fetchone()
31 |         # Print all non-blob columns
32 |         for i, c in enumerate(cols):
33 |             val = row[i]
34 |             if not isinstance(val, bytes):
35 |                 print(f'    {c}: {str(val)[:200]}')
36 |     conn.close()
37 |
```

package_skill.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Module
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: package_skill.py
9 # Project: ai-breadboard
10 # Package: .ai.tools.ai
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import os
16 import zipfile
17 import argparse
18 from pathlib import Path
19
20 def package_skill(skill_dir, output_dir):
21     skill_path = Path(skill_dir).resolve()
22     output_path = Path(output_dir).resolve()
23
24     if not output_path.exists():
25         output_path.mkdir(parents=True)
26
27     skill_name = skill_path.name
28     skill_file = output_path / f"{skill_name}.skill"
29
30     print(f"Упаковка навыка из {skill_path} в {skill_file}...")
31 )
32
33 with zipfile.ZipFile(skill_file, 'w', zipfile.ZIP_DEFLATED
```

```
    ) as zipf:
33         for root, dirs, files in os.walk(skill_path):
34             # Пропускаем директории dist и .git
35             dirs[:] = [d for d in dirs if d not in ['dist', '.
git', '__pycache__']]
36
37             for file in files:
38                 file_path = Path(root) / file
39                 # Относительный путь внутри архива
40                 arcname = file_path.relative_to(skill_path)
41                 zipf.write(file_path, arcname)
42
43         print(f"Навык successfully упакован: {skill_file}")
44
45 if __name__ == "__main__":
46     parser = argparse.ArgumentParser(description="Упаковщик на
выков Gemini CLI")
47     parser.add_argument("skill_dir", help="Путь к директории н
авыка")
48     parser.add_argument("output_dir", help="Путь к директории
dist")
49     args = parser.parse_args()
50
51     package_skill(args.skill_dir, args.output_dir)
52
```

rebuild_dev_rag.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Переиндексация контекста разработчика
4 # =====
5 # Description:
6 #   Скрипт запускает перестроение RAG-индекса для технического
7 #   контекста разработки.
8 #
9 # File: rebuild_dev_rag.py
10 # Project: ai-breadboard
11 # Package: .ai.tools.ai
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import os
17 from src.ai.dev_rag import build_dev_rag
18 from src.logger import logger
19
20 def main():
21     api_key = os.getenv('GEMINI_API_KEY')
22     if not api_key:
23         print("❌ Error: Переменная окружения GEMINI_API_KEY не
24           установлена.")
25         return
26
27     print("❌ Запуск переиндексации кодовой базы и документац
28     и...")
29     try:
30         build_dev_rag(api_key)
31         print("❌ Индекс разработчика successfully перестроен.")
32     except Exception as e:
```

```
30 |         print(f"❌ Error при переиндексации: {e}")
31 |         logger.error(f"Error rebuild_dev_rag: {e}", exc_info=True)
32 |
33 | if __name__ == "__main__":
34 |     main()
35 |
```


rebuild_rag.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Запуск переиндексации Core RAG индекса.
4 # =====
5 # Description:
6 #   Импортирует и performs функцию перестроения RAG-индекса.
7 #
8 # File: rebuild_rag.py
9 # Project: ai-breadboard
10 # Package: .ai.tools.ai
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import argparse
16 from src.rag import build_rules_index
17
18 def main() -> None:
19     """Запуск переиндексации Core RAG индекса."""
20     parser = argparse.ArgumentParser(description="Переиндексац  
ия Core RAG индекса.")
21     parser.add_argument('--fresh', action='store_true', help="Создать новый индекс с нуля.")
22     args = parser.parse_args()
23
24     print(f"Запуск переиндексации Core RAG правил (fresh={args  
.fresh})...")
25     index_path, docs_path = build_rules_index()
26     print(f"Successfully сгенерирован RAG индекс: {index_path}  
, {docs_path}")
27
28 if __name__ == '__main__':
29     main()
```


search_code.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Поиск по кодовой базе (CLI утилита)
4 # =====
5 # Description:
6 #   Консольная утилита для поиска по техническому RAG-индексу
7 #   кода и документации.
8 #
9 # File: search_code.py
10 # Project: ai-breadboard
11 # Package: .ai.tools.ai
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import os
17 import sys
18 import json
19 from src.ai.dev_rag import rag_search_tool
20
21 def main():
22     if len(sys.argv) < 2:
23         print("Использование: python scripts/search_code.py 'в
24 аш запрос'")
25         return
26
27     query = sys.argv[1]
28     api_key = os.getenv('GEMINI_API_KEY')
29
30     if not api_key:
31         print("❌ Error: GEMINI_API_KEY не установлен.")
32         return
```

```
32 |     print(f"    Поиск по коду: '{query}'...")
33 |     result_json = rag_search_tool(query, api_key=api_key)
34 |
35 |     try:
36 |         results = json.loads(result_json)
37 |         if "error" in results:
38 |             print(f"    {results['error']}")
39 |         else:
40 |             for i, res in enumerate(results, 1):
41 |                 path = res.get('meta', {}).get('path', 'Unknown')
42 |                 print(f"{i}. Файл: {path} (Score: {res.get('score', 0):.2f})")
43 |     except Exception as e:
44 |         print(f"    Error разбора результатов: {e}")
45 |
46 | if __name__ == "__main__":
47 |     main()
48 |
```

update_docs.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Returns list измененных файлов .py в git репозитор
4 # =====
5 # Description:
6 # Скрипт checks наличие и заполненность документации (README
7 # .md, docstrings)
8 #
9 # File: update_docs.py
10 # Project: ai-breadboard
11 # Package: .ai.tools.ai
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import os
17 import sys
18 import subprocess
19 from pathlib import Path
20 from typing import List
21
22 def get_modified_python_files() -> List[Path]:
23     """Returns list измененных файлов .py в git репозитории."""
24
25     modified_files = []
26     try:
27         res = subprocess.check_output(
28             ["git", "status", "--porcelain"],
29             text=True,
30             stderr=subprocess.DEVNULL
31         )
32         for line in res.splitlines():
```

```

31         parts = line.strip().split(maxsplit=1)
32         if len(parts) > 1:
33             filepath = Path(parts[1])
34             if filepath.suffix == ".py" and filepath.exists():
35                 modified_files.append(filepath)
36         except Exception:
37             # Если git недоступен, вернем empty list
38             pass
39         return modified_files
40
41 def validate_docblocks(files: List[Path]) -> bool:
42     """Checks наличие docstring в измененных файлах."""
43     all_valid = True
44     for f in files:
45         content = f.read_text(encoding="utf-8")
46         # Простая check на наличие тройных кавычек (docstrings)
47         if '"""' not in content and "'" not in content:
48             print(f"Файл {f.name} изменен, но не содержит d
49             ocstring!")
50             all_valid = False
51     return all_valid
52
53 def main() -> int:
54     print("Запуск проверки актуальности документации и комм
55     ентариев...")
56
57     modified = get_modified_python_files()
58     if not modified:
59         print("Измененных файлов Python в Git не найдено. До
60         полнительная validation не требуется.")
61         return 0
62
63     print(f"Обнаружено измененных файлов: {len(modified)}")
64     valid = validate_docblocks(modified)
65
66     if valid:

```

```
64 |         print("□ Все измененные файлы содержат docstring.")
65 |         return 0
66 |     else:
67 |         print("□ Рекомендуется добавить или обновить коммента
ии/документацию.")
68 |         return 1
69 |
70 | if __name__ == "__main__":
71 |     sys.exit(main())
72 |
```

validate_rag_files.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Module
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: validate_rag_files.py
9 # Project: ai-breadboard
10 # Package: .ai.tools.ai
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import os
16 from pathlib import Path
17
18 # List исключений
19 EXCLUDED_DIRS = {
20     '.git', '.pytest_cache', '.vs', '__pycache__', 'venv',
21     'htmlcov', 'site', 'node_modules', '.venv'
22 }
23 EXCLUDED_FILES = {
24     '.env', '.gitignore', '.gitattributes', 'all_duplicates_with_ids.csv', 'all_media_data.csv',
25     'deletion_candidates_all.csv', 'deletion_candidates.csv',
26     'duplicates_report_candidates.csv',
27     'duplicates_report.csv', 'inferred_titles_report.csv', 'physical_check_results.csv',
28     'potential_duplicates.csv'
29 }
30
31 def get_files_to_index():
```



```
31 | project_root = Path(r"C:\mediateka")
32 | aux_dir = project_root / "knowledge" / "rag_auxiliary"
33 | files_to_index = []
34 |
35 | # 1. Сканирование корня с исключениями
36 | for root, dirs, files in os.walk(project_root):
37 |     # Модифицируем list dirs на месте для пропуска исключе
    нных директорий
38 |     dirs[:] = [d for d in dirs if d not in EXCLUDED_DIRS]
39 |
40 |     for file in files:
41 |         if file in EXCLUDED_FILES:
42 |             continue
43 |
44 |         file_path = Path(root) / file
45 |         if file_path.suffix in [".py", ".md"]:
46 |             files_to_index.append(file_path)
47 |
48 | # 2. Добавление файлов из rag_auxiliary (без жестких исклю
    чений)
49 | if aux_dir.exists():
50 |     for file_path in aux_dir.rglob("*"):
51 |         if file_path.is_file() and file_path.suffix in [".
    py", ".md"]:
52 |             if file_path not in files_to_index:
53 |                 files_to_index.append(file_path)
54 |
55 | return files_to_index
56 |
57 | if __name__ == "__main__":
58 |     files = get_files_to_index()
59 |     print(f"Всего файлов к индексации: {len(files)}")
60 |     for f in files:
61 |         print(f)
62 |
```

analyze_dependencies.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: # =====
4 # =====
5 # Description:
6 #     Найти все Python файлы в корне проекта. """
7 #
8 # File: analyze_dependencies.py
9 # Project: ai-breadboard
10 # Package: .ai.tools.setup
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """
16 Скрипт для анализа зависимостей между скриптами в проекте.
17 Помогает определить, какие скрипты используются другими и каки
18 е можно безопасно удалить.
19 """
20 import os
21 import re
22 import sys
23 from pathlib import Path
24 from collections import defaultdict, Counter
25
26 PROJECT_ROOT = Path(__file__).parent
27
28 def find_python_files():
29     """Найти все Python файлы в корне проекта."""
30     python_files = []
31     for file in PROJECT_ROOT.glob("*.py"):
```

```

32     python_files.append(file)
33     return python_files
34
35 def analyze_file_imports(file_path):
36     """Проанализировать импорты в Python файле."""
37     imports = []
38     try:
39         content = file_path.read_text(encoding='utf-8')
40
41         # Ищем импорты других .py файлов в корне
42         # Регулярные выражения для разных типов импортов
43         patterns = [
44             # import module
45             r'^import\s+([a-zA-Z_][a-zA-Z0-9_]*(?:\s*,\s*[a-zA-Z_][a-zA-Z0-9_]*))*\s$',
46             # from module import something
47             r'^from\s+([a-zA-Z_][a-zA-Z0-9_.*]*)\s+import\s+',
48             # subprocess.run с python скриптами
49             r'subprocess\.run\(\([.*?["\']([a-zA-Z_][a-zA-Z0-9_]*\.\py)["\']\)',
50             # sys.argv манипуляции
51             r'sys\.argv\s*=\s*\([.*?["\']([a-zA-Z_][a-zA-Z0-9_]*\.\py)["\']\)',
52         ]
53
54         for line in content.split('\n'):
55             line = line.strip()
56             if not line or line.startswith('#'):
57                 continue
58
59             # Check на импорты других скриптов
60             for pattern in patterns:
61                 match = re.search(pattern, line)
62                 if match:
63                     module = match.group(1)
64                     # Игнорируем стандартные библиотеки и внут
65                     if ('.py' in module or

```

```

66 |             (not module.startswith('src.') and
67 |             not module.startswith('plugins.') and
68 |             not module.startswith('.'))):
69 |             imports.append(module)
70 |             break
71 |
72 |     # Также проверяем использование через manage_tools.py
    стиль
73 |     if 'manage_tools.py' in content:
74 |         # Ищем вызовы других скриптов через subprocess
75 |         subprocess_calls = re.findall(r'cmd\s*=\s*\[sys\.e
    хexecutable,\s*["\']([a-zA-Z_][a-zA-Z0-9_]*\.py)["\']', content)
76 |         imports.extend(subprocess_calls)
77 |
78 |     except Exception as e:
79 |         print(f"Error при анализе {file_path}: {e}")
80 |
81 |     return imports
82 |
83 | def analyze_ps1_files():
84 |     """Проанализировать PowerShell скрипты на вызовы других ск
    риптов."""
85 |     ps1_dependencies = defaultdict(list)
86 |
87 |     for ps1_file in PROJECT_ROOT.glob("*.ps1"):
88 |         try:
89 |             content = ps1_file.read_text(encoding='utf-8')
90 |
91 |             # Ищем вызовы других скриптов
92 |             called_scripts = set()
93 |
94 |             # Вызовы других .ps1 файлов
95 |             ps1_calls = re.findall(r'&\s*["\']?([a-zA-Z_][a-zA
    -Z0-9_]*\.ps1)["\']?', content)
96 |             called_scripts.update(ps1_calls)
97 |
98 |             # Вызовы .py файлов через python
99 |             py_calls = re.findall(r'python\s+["\']?([a-zA-Z_][

```

```

    a-zA-Z0-9_]*\.py)","\'?'', content)
100         called_scripts.update(py_calls)
101
102         # Запуск через Start-Process
103         start_process_calls = re.findall(r'Start-Process.*
["\'']([a-zA-Z_][a-zA-Z0-9_]*\.(?:py|ps1|exe))["\'']', content,
re.IGNORECASE)
104         called_scripts.update(start_process_calls)
105
106         # Запуск через .\ (текущая директория)
107         relative_calls = re.findall(r'\.\.\\([a-zA-Z_][a-zA-
Z0-9_]*\.(?:py|ps1|exe))', content)
108         called_scripts.update(relative_calls)
109
110         if called_scripts:
111             ps1_dependencies[ps1_file.name] = list(called_
scripts)
112
113         except Exception as e:
114             print(f"Error при анализе {ps1_file}: {e}")
115
116     return ps1_dependencies
117
118 def build_dependency_graph():
119     """Построить граф зависимостей между скриптами."""
120     python_files = find_python_files()
121
122     # Dictionary зависимостей: скрипт -> [зависимости]
123     dependencies = defaultdict(list)
124     # Dictionary обратных зависимостей: скрипт -> [кто зависит
]
125     reverse_deps = defaultdict(list)
126
127     # Анализ Python файлов
128     for py_file in python_files:
129         file_name = py_file.name
130         imports = analyze_file_imports(py_file)
131

```

```
132 |         # Фильтруем только скрипты в корне проекта
133 |         root_scripts = []
134 |         for imp in imports:
135 |             # Проверяем, существует ли такой файл в корне
136 |             if '.py' in imp:
137 |                 script_name = imp if imp.endswith('.py') else
138 |                 f"{imp}.py"
139 |                 if (PROJECT_ROOT / script_name).exists():
140 |                     root_scripts.append(script_name)
141 |                 elif (PROJECT_ROOT / f"{imp}.py").exists():
142 |                     root_scripts.append(f"{imp}.py")
143 |
144 |         if root_scripts:
145 |             dependencies[file_name] = root_scripts
146 |             for dep in root_scripts:
147 |                 reverse_deps[dep].append(file_name)
148 |
149 |     # Анализ PowerShell файлов
150 |     ps1_deps = analyze_ps1_files()
151 |     for ps1_file, deps in ps1_deps.items():
152 |         dependencies[ps1_file] = deps
153 |         for dep in deps:
154 |             if dep.endswith('.py') or dep.endswith('.ps1'):
155 |                 reverse_deps[dep].append(ps1_file)
156 |
157 |     return dependencies, reverse_deps
158 |
159 | def analyze_script_usage():
160 |     """Проанализировать использование скриптов."""
161 |     dependencies, reverse_deps = build_dependency_graph()
162 |
163 |     # Подсчет использования
164 |     usage_count = Counter()
165 |     for deps in dependencies.values():
166 |         for dep in deps:
167 |             usage_count[dep] += 1
168 |
169 |     # Анализ manage_tools.py отдельно
```

```
169 | manage_tools_deps = []
170 | if 'manage_tools.py' in dependencies:
171 |     manage_tools_deps = dependencies['manage_tools.py']
172 |
173 | # Группировка по типам использования
174 | scripts_by_usage = {
175 |     'highly_used': [], # Используется многими скриптами
176 |     'moderately_used': [], # Используется 1-2 скриптами
177 |     'unused': [], # Не используется никем
178 |     'leaf_nodes': [], # Ни от кого не зависит
179 | }
180 |
181 | all_scripts = set()
182 | all_scripts.update(dependencies.keys())
183 | all_scripts.update(reverse_deps.keys())
184 |
185 | for script in all_scripts:
186 |     if script.endswith('.py'):
187 |         depends_on = dependencies.get(script, [])
188 |         depended_by = reverse_deps.get(script, [])
189 |
190 |         if len(depended_by) >= 3:
191 |             scripts_by_usage['highly_used'].append((script
192 | , len(depended_by)))
193 |             elif len(depended_by) == 1 or len(depended_by) ==
194 | 2:
195 |                 scripts_by_usage['moderately_used'].append((sc
196 | ript, len(depended_by)))
197 |                 elif len(depended_by) == 0:
198 |                     scripts_by_usage['unused'].append((script, 0))
199 |
200 |                 if not depends_on:
201 |                     scripts_by_usage['leaf_nodes'].append(script)
202 |
203 | return {
204 |     'dependencies': dict(dependencies),
205 |     'reverse_deps': dict(reverse_deps),
206 |     'usage_count': dict(usage_count),
```

```

204         'scripts_by_usage': scripts_by_usage,
205         'manage_tools_deps': manage_tools_deps,
206     }
207
208 def print_dependency_report(analysis):
209     """Напечатать отчет о зависимостях."""
210     print("=" * 80)
211     print("АНАЛИЗ ЗАВИСИМОСТЕЙ СКРИПТОВ ПРОЕКТА")
212     print("=" * 80)
213
214     print("\n1. СКРИПТЫ, ИСПОЛЬЗУЕМЫЕ В manage_tools.py:")
215     print("-" * 40)
216     if analysis['manage_tools_deps']:
217         for dep in sorted(analysis['manage_tools_deps']):
218             print(f"    • {dep}")
219     else:
220         print("    Нет зависимостей")
221
222     print("\n2. САМЫЕ ИСПОЛЬЗУЕМЫЕ СКРИПТЫ:")
223     print("-" * 40)
224     sorted_usage = sorted(analysis['usage_count'].items(), key
225 =lambda x: x[1], reverse=True)
226     for script, count in sorted_usage[:10]:
227         print(f"    • {script}: используется {count} скриптом(ам
228 и)")
229
230     print("\n3. СКРИПТЫ, КОТОРЫЕ НИКТО НЕ ИСПОЛЬЗУЕТ:")
231     print("-" * 40)
232     unused = analysis['scripts_by_usage']['unused']
233     if unused:
234         for script, _ in sorted(unused):
235             print(f"    • {script}")
236     else:
237         print("    Все скрипты используются")
238
239     print("\n4. ГРАФ ЗАВИСИМОСТЕЙ (основные связи):")
240     print("-" * 40)
241     for script, deps in analysis['dependencies'].items():

```



```

240         if deps:
241             print(f" {script} -> {' ', ' '.join(deps)}")
242
243     print("\n5. РЕКОМЕНДАЦИИ ПО УДАЛЕНИЮ:")
244     print("-" * 40)
245
246     # Кандидаты на удаление
247     candidates = []
248
249     # 1. Неиспользуемые скрипты
250     for script, _ in analysis['scripts_by_usage']['unused']:
251         if script not in ['main.py', 'bot_runner.py', 'run_med
252 ia_organizer.py', 'manage_tools.py']:
253             candidates.append((script, "Не используется другим
254 и скриптами"))
255
256     # 2. Скрипты с низкой сложностью и малоиспользуемые
257     for script, count in analysis['scripts_by_usage']['moderat
258 ely_used']:
259         if count == 1 and script not in ['header.py']:
260             # Проверяем размер файла
261             file_path = PROJECT_ROOT / script
262             if file_path.exists() and file_path.stat().st_size
263 < 1000: # Менее 1KB
264                 candidates.append((script, f"Мало используется
265 ({count} зависимость), маленький размер"))
266
267     if candidates:
268         for script, reason in sorted(candidates):
269             print(f" • {script}: {reason}")
270     else:
271         print(" Нет явных кандидатов на удаление")
272
273     print("\n" + "=" * 80)
274
275 def main():
276     """Основная function."""
277     print("Анализ зависимостей скриптов...")

```

```

273
274     analysis = analyze_script_usage()
275     print_dependency_report(analysis)
276
277     # Сохранение отчета в файл
278     report_file = PROJECT_ROOT / "scripts_dependency_report.md
279
280     with open(report_file, 'w', encoding='utf-8') as f:
281         f.write("# Отчет о зависимостях скриптов\n\n")
282         f.write("## Скрипты, используемые в manage_tools.py:\n
283
284         if analysis['manage_tools_deps']:
285             for dep in sorted(analysis['manage_tools_deps']):
286                 f.write(f"- `{dep}`\n")
287         else:
288             f.write("Нет зависимостей\n")
289
290         f.write("\n## Неиспользуемые скрипты:\n")
291         unused = analysis['scripts_by_usage']['unused']
292         if unused:
293             for script, _ in sorted(unused):
294                 f.write(f"- `{script}`\n")
295         else:
296             f.write("Все скрипты используются\n")
297
298         f.write("\n## Кандидаты на удаление:\n")
299         # Повторяем логику кандидатов
300         candidates = []
301         for script, _ in analysis['scripts_by_usage']['unused']
302         ]:
303             if script not in ['main.py', 'bot_runner.py', 'run
304             _media_organizer.py', 'manage_tools.py']:
305                 candidates.append(script)
306
307         if candidates:
308             for script in sorted(candidates):
309                 f.write(f"- `{script}` - не используется други
310                 ми скриптами\n")

```

```
306 |         else:
307 |             f.write("Нет явных кандидатов на удаление\n")
308 |
309 |     print(f"\nОтчет сохранен в {report_file}")
310 |
311 | if __name__ == '__main__':
312 |     main()
```

convert_to_md.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Module
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: convert_to_md.py
9 # Project: ai-breadboard
10 # Package: .ai.tools.setup
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import json
16 from pathlib import Path
17 from header import __root__
18
19 REPORTS_DIR = __root__ / 'tmp' / 'reports'
20
21 # Загружаем JSON (замените на реальные данные из предыдущего шага)
22 # Для примера я использую структуру, полученную в прошлый раз
23 json_data = """
24 {
25     "title": "Острые козырьки (Peaky Blinders)",
26     "plot": "События разворачиваются в послевоенном Бирмингеме 1920-х годов...",
27     "seasons": [
28         {
29             "season_number": 1,
30             "season_plot_summary": "Бирмингем, 1919 год. Вернувшись с полей сражений Первой мировой войны, Томас Шелби возглавляет
```

```

    преступную группировку «Острые козырьки»...",
31     "episodes": [
32         {"episode_number": 1, "title": "Эпизод 1", "detailed_d
description": "Томми захватывает оружие...", "final_verdict": "
Начало кровавого пути."}
33     ]
34 }
35 ]
36 }
37 """
38 data = json.loads(json_data)
39
40 md_content = f"# {data['title']}\n\n"
41 md_content += f"## Сюжет\n{data['plot']}\n\n"
42
43 for season in data.get('seasons', []):
44     md_content += f"## Сезон {season['season_number']}\n"
45     md_content += f"{season['season_plot_summary']}\n\n"
46     md_content += "### Эпизоды\n"
47     for ep in season.get('episodes', []):
48         md_content += f"#### {ep['episode_number']}. {ep['titl
e']}\n"
49         md_content += f"{ep['detailed_description']}\n\n"
50         md_content += f"**Вердикт:** {ep['final_verdict']}\n\n"
51     "
52 output_path = REPORTS_DIR / "Peaky_Blinders_Card.md"
53 output_path.write_text(md_content, encoding='utf-8')
54 print(f"Отчёт сохранён: {output_path}")
55

```

scan_headers.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Module
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: scan_headers.py
9 # Project: ai-breadboard
10 # Package: .ai.tools.setup
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 \s*# (?:File|Project|Author|Copyright|\\$))', header_text, re.D
OTALL) if desc_match: desc_lines = desc_match.group(1).strip()
    lines = [line.strip().lstrip('#').strip() for line in desc_li
nes.split('\n')] return ' '.join(lines) return '' def check_he
ader(filepath): with open(filepath, 'r', encoding='utf-8') as
f: content = f.read() lines = content.split('\n') has_coding =
False header_start = 0 for i, line in enumerate(lines[:3]): i
f '# -*- coding: utf-8 -*-' in line: has_coding = True header_
start = i break if not has_coding: return {'has_header': 'no',
'word_count': 0, 'description': ''} header_lines = [] for i i
n range(header_start, min(len(lines), 50)): line = lines[i].st
rip() if not line.startswith('#') and not line.startswith('!')
and 'import' not in line.lower(): break header_lines.append(l
ine) header_text = '\n'.join(header_lines) has_nazvanie = bool
(re.search(r'#\s*Название\s*(?:процесса|модуля):', header_text
)) has_opisanie = bool(re.search(r'#\s*Description:', header_t
ext)) has_file = bool(re.search(r'#\s*File:', header_text)) ha
s_project = bool(re.search(r'#\s*Project:', header_text)) has_
author = bool(re.search(r'#\s*Author:', header_text)) has_copy
```

```
right = bool(re.search(r'#\s*Copyright:', header_text)) all_fi
elds_present = all([has_nazvanie, has_opisanie, has_file, has_
project, has_author, has_copyright]) if not all_fields_present
: if len(header_lines) <= 3 and all('coding' in l.lower() or l
.strip() == '' for l in header_lines): return {'has_header': '
no', 'word_count': 0, 'description': ''} return {'has_header':
'no', 'word_count': 0, 'description': ''} description = extra
ct_description(header_text) word_count = count_words(descripti
on) return {'has_header': 'yes', 'word_count': word_count, 'de
scription': description[:200]}
```

docs-ci.yml

```
1 name: docs-ci
2
3 on:
4   pull_request:
5     paths:
6       - 'docs/**'
7       - 'mkdocs.yml'
8       - 'requirements-docs.txt'
9       - 'src/**'
10  push:
11    branches:
12      - main
13    paths:
14      - 'docs/**'
15      - 'mkdocs.yml'
16      - 'requirements-docs.txt'
17      - 'src/**'
18
19 jobs:
20   build:
21     runs-on: ubuntu-latest
22     steps:
23       - uses: actions/checkout@v4
24
25       - name: Setup Python
26         uses: actions/setup-python@v4
27         with:
28           python-version: '3.10'
29
30       - name: Install project and docs requirements
31         run: |
32           python -m pip install --upgrade pip
33           if [ -f requirements-docs.txt ]; then python -m pip
install -e . && python -m pip install -r requirements-docs.txt
; else python -m pip install -e . mkdocs mkdocs-material mkdoc
strings pymdown-extensions; fi
```



```
34 |
35 |   - name: Build MkDocs site (strict)
36 |     continue-on-error: true
37 |     run: |
38 |       mkdocs build --strict 2>&1 | tee strict-build.log
39 |
40 |   - name: Upload strict build log
41 |     uses: actions/upload-artifact@v4
42 |     with:
43 |       name: mkdocs-strict-log
44 |       path: strict-build.log
45 |
46 |   - name: Build MkDocs site (fallback)
47 |     run: |
48 |       mkdocs build 2>&1 | tee build.log
49 |
50 |   - name: Upload built site
51 |     uses: actions/upload-artifact@v4
52 |     with:
53 |       name: mkdocs-site
54 |       path: site
55 |       retention-days: 7
56 |   - name: Upload build log
57 |     uses: actions/upload-artifact@v4
58 |     with:
59 |       name: mkdocs-build-log
60 |       path: build.log
61 |
```

docs.yml

```
1 name: Documentation
2
3 on:
4   push:
5     branches:
6       - main
7
8   pull_request:
9     branches:
10      - main
11
12 jobs:
13   docs:
14     runs-on: ubuntu-latest
15
16     steps:
17       - name: Checkout repository
18         uses: actions/checkout@v4
19
20       - name: Set up Python
21         uses: actions/setup-python@v5
22         with:
23           python-version: '3.12'
24
25       - name: Install documentation dependencies
26         run: |
27           python -m pip install --upgrade pip
28           pip install -r docs-requirements.txt
29
30       - name: Build documentation
31         run: |
32           mkdocs build --strict
33
34
35 # Note: This workflow only validates that MkDocs builds succes
    sfully and does NOT publish documentation.
```

36 | # Read the Docs should be used to build and publish the site (
| including PR previews).

37 |

playwright-mcp.yml

```
1 name: "Playwright MCP - Build"
2
3 on:
4   push:
5     paths:
6       - '.mcp/playwright/**'
7   workflow_dispatch:
8
9 jobs:
10  build:
11    runs-on: ubuntu-latest
12
13    steps:
14      - name: Checkout
15        uses: actions/checkout@v4
16
17      - name: Setup Node
18        uses: actions/setup-node@v4
19        with:
20          node-version: '18'
21
22      - name: Install deps
23        working-directory: .mcp/playwright
24        run: npm ci
25
26      - name: Build
27        working-directory: .mcp/playwright
28        run: npm run build
29
30      - name: Run tests (none)
31        run: echo "No tests configured"
32
```

agy_search_mcp_server.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Выполнить веб-поиск через агентный поиск Google An
4 # =====
5 # Description:
6 #   MCP-сервер на базе FastMCP, предоставляющий доступ к поиску
7 #   в интернете
8 #
9 # File: agy_search_mcp_server.py
10 # Project: ai-breadboard
11 # Package: root
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 from __future__ import annotations
17
18 import json
19 import asyncio
20 from src.logger.logger import logger
21 from plugins.web_search.agy_searcher import AgyWebSearcher
22
23 try:
24     from mcp.server.fastmcp import FastMCP
25     mcp = FastMCP("Antigravity-Search-Server")
26 except ImportError:
27     FastMCP = None
28     mcp = None
29
30 async def _run_agy_search(query: str, model: str = "agy-flash")
31     try:
```

```
31 |         searcher = AgyWebSearcher(model_id=model)
32 |         result = await searcher.search_and_extract(query=query
33 | )
34 |         return result
35 |     except Exception as e:
36 |         logger.error(f"[agy_search_mcp_server] Error agy_web_s
37 | earch: {e}")
38 |         return json.dumps({"status": "error", "message": str(e
39 | )}, ensure_ascii=False)
40 |
41 | async def agy_web_search(query: str, model: str = "agy-flash")
42 | -> str:
43 |     """Выполнить веб-поиск через агентный поиск Google Antigra
44 | vity."""
45 |     return await _run_agy_search(query=query, model=model)
46 |
47 | if mcp:
48 |     mcp.tool()(agy_web_search)
```

auto_commits.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Выполнение git-команды в корне репозитория.
4 # =====
5 # Description:
6 #   Скрипт автоматического отслеживания изменений в файлах репозитория
7 #
8 # File: auto_commits.py
9 # Project: ai-breadboard
10 # Package: root
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from __future__ import annotations
16
17 import time
18 import subprocess
19 from pathlib import Path
20
21 from src.logger import logger
22
23 ROOT = Path(__file__).resolve().parent.parent
24
25 def git(cmd: str) -> subprocess.CompletedProcess:
26     """Выполнение git-команды в корне репозитория."""
27     return subprocess.run(["git"] + cmd.split(), cwd=ROOT, capture_output=True, text=True, check=False)
28
29 def commit_if_dirty(message: str) -> bool:
30     """Check наличия незакоммиченных изменений и создание коммита."""
```

```

31     git("add -A")
32     status = git("status --porcelain")
33     if status.stdout and status.stdout.strip():
34         git(f'commit -m "{message}"')
35         logger.info(f"[auto_commits] Закоммичены изменения: {m
message}")
36         return True
37     return False
38
39 def main():
40     """Основной цикл отслеживания изменений файлов."""
41     logger.info("[auto_commits] Запуск службы отслеживания изм
енений файлов...")
42     last_mtime = {}
43
44     while True:
45         changed = False
46         for p in ROOT.rglob("*"):
47             if p.is_file() and ".git" not in p.parts:
48                 try:
49                     m = p.stat().st_mtime
50                 except OSError:
51                     continue
52                 if p not in last_mtime:
53                     last_mtime[p] = m
54                     continue
55                 if m != last_mtime[p]:
56                     logger.info(f"[auto_commits] Обнаружено из
менение файла: {p}")
57                     last_mtime[p] = m
58                     changed = True
59         if changed:
60             commit_if_dirty("chore(mcp): auto-commit changes d
etected by MCP server")
61             time.sleep(5)
62
63 if __name__ == "__main__":
64     main()

```


config_helper.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Вспомогательный Module для работы с конфигурац
  ией
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: config_helper.py
9 # Project: ai-breadboard
10 # Package: root
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """
16 Вспомогательный Module для работы с конфигурацией в MCP сервер
  ах.
17
18 Предоставляет кроссплатформенные функции для работы с config.j
  son и путями.
19 """
20
21 import json
22 import os
23 import sys
24 from pathlib import Path
25 from typing import Any, Dict, Optional
26
27 def get_project_root() -> Path:
28     """
29     Получить корень проекта.
30
```

```

31 | Поиск ведется по наличию config.json или main.py.
32 | """
33 | # Начинаем от директории .mcp
34 | current = Path(__file__).parent.parent
35 |
36 | # Проверяем, находимся ли мы уже в корне
37 | if (current / "config.json").exists() or (current / "main.
py").exists():
38 |     return current
39 |
40 | # Проверяем переменную окружения
41 | if "AIBREASHBOARD_DIR" in os.environ:
42 |     return Path(os.environ["AIBREASHBOARD_DIR"])
43 |
44 | # Fallback - текущая директория
45 | return Path.cwd()
46 |
47 | def load_config() -> Dict[str, Any]:
48 |     """
49 |     Загрузить конфигурацию проекта.
50 |
51 |     Returns:
52 |         Dictionary конфигурации из config.json
53 |     """
54 |     config_path = get_project_root() / "config.json"
55 |
56 |     if not config_path.exists():
57 |         return {}
58 |
59 |     try:
60 |         with open(config_path, "r", encoding="utf-8") as f:
61 |             return json.load(f)
62 |     except Exception as e:
63 |         print(f"[WARNING] Error чтения config.json: {e}")
64 |         return {}
65 |
66 | def get_config_value(key_path: str, default: Any = None) -> Any:

```

```

67 |     """
68 |     Получить значение из конфигурации используя нотацию точек.
69 |
70 |     Args:
71 |         key_path: Путь ключей через точку (например: "server.p
    ort")
72 |         default: Значение по умолчанию
73 |
74 |     Returns:
75 |         Значение или default
76 |     """
77 |     config = load_config()
78 |     keys = key_path.split(".")
79 |
80 |     current = config
81 |     for key in keys:
82 |         if isinstance(current, dict) and key in current:
83 |             current = current[key]
84 |         else:
85 |             return default
86 |
87 |     return current
88 |
89 | def get_server_url() -> str:
90 |     """
91 |     Получить базовый URL сервера FastAPI.
92 |
93 |     Returns:
94 |         URL в формате http://host:port
95 |     """
96 |     host = get_config_value("server.host", "localhost")
97 |     port = get_config_value("server.port", 8000)
98 |
99 |     # Если host 0.0.0.0, использовать localhost
100 |     if host == "0.0.0.0":
101 |         host = "localhost"
102 |
103 |     # Проверить SSL

```

```

104     use_ssl = get_config_value("server.use_ssl", False)
105     protocol = "https" if use_ssl else "http"
106
107     return f"{protocol}://{host}:{port}"
108
109 def get_env_var(key: str, default: Optional[str] = None) -> Op
    tional[str]:
110     """
111     Получить переменную окружения.
112
113     Приоритет: система > .env > default
114
115     Args:
116         key: Имя переменной
117         default: Значение по умолчанию
118
119     Returns:
120         Значение или None
121     """
122     # 1. Системная переменная
123     if key in os.environ:
124         return os.environ[key]
125
126     # 2. Из .env файла
127     env_file = get_project_root() / ".env"
128     if env_file.exists():
129         try:
130             with open(env_file, "r", encoding="utf-8") as f:
131                 for line in f:
132                     line = line.strip()
133                     if line and not line.startswith("#") and "
    =" in line:
134                         env_key, env_value = line.split("=", 1
    )
135                         if env_key.strip() == key:
136                             return env_value.strip().strip('"'
    ').strip("'")
137         except Exception:

```

```
138 |         pass
139 |
140 |     # 3. Default
141 |     return default
142 |
143 | def get_certs_dir() -> Path:
144 |     """
145 |     Получить директорию SSL сертификатов (кроссплатформенно).
146 |
147 |     Returns:
148 |         Path к директории сертификатов
149 |     """
150 |     if sys.platform == "win32":
151 |         return Path.home() / ".certs"
152 |     elif sys.platform == "darwin":
153 |         return Path.home() / "Library" / "Certs"
154 |     else:
155 |         # Linux
156 |         return Path.home() / ".local" / "share" / "ca-certific
157 | ates"
158 |
159 | def get_data_dir() -> Path:
160 |     """
161 |     Получить директорию данных приложения (кроссплатформенно).
162 |
163 |     Returns:
164 |         Path к директории данных
165 |     """
166 |     if sys.platform == "win32":
167 |         localappdata = os.environ.get("LOCALAPPDATA", str(Path
168 | .home() / "AppData" / "Local"))
169 |         return Path(localappdata) / "AI-Breadboard"
170 |     elif sys.platform == "darwin":
171 |         return Path.home() / "Library" / "Application Support"
172 | / "AI-Breadboard"
173 |     else:
174 |         # Linux
175 |         return Path.home() / ".local" / "share" / "AI-Breadboa
```

```

rd"
173
174 def is_port_open(host: str = "127.0.0.1", port: int = 8000, ti
meout: float = 1.0) -> bool:
175     """
176     Проверить, открыт ли порт.
177
178     Args:
179         host: IP адрес
180         port: Номер порта
181         timeout: Timeout для подключения
182
183     Returns:
184         True если порт открыт, False иначе
185     """
186     import socket
187     try:
188         with socket.socket(socket.AF_INET, socket.SOCK_STREAM)
as s:
189             s.settimeout(timeout)
190             result = s.connect_ex((host, port))
191             return result == 0
192     except Exception:
193         return False
194
195 # Экспортируемые функции
196 __all__ = [
197     "get_project_root",
198     "load_config",
199     "get_config_value",
200     "get_server_url",
201     "get_env_var",
202     "get_certs_dir",
203     "get_data_dir",
204     "is_port_open",
205 ]
206

```

example_mcp_server.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Пример MCP сервера с кроссплатформенной поддерж
  жкой
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: example_mcp_server.py
9 # Project: ai-breadboard
10 # Package: root
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """
16 Пример MCP сервера с кроссплатформенной поддержкой.
17
18 Этот файл показывает рекомендуемые паттерны для написания MCP
  серверов
19 которые работают на Windows, Linux и macOS без изменений.
20
21 Использование:
22     python .mcp/example_mcp_server.py
23     python3 .mcp/example_mcp_server.py
24 """
25
26 from __future__ import annotations
27
28 import sys
29 import os
30 from pathlib import Path
31
```



```
32 # Добавить корень проекта в PYTHONPATH
33 project_root = Path(__file__).parent.parent
34 sys.path.insert(0, str(project_root))
35
36 from mcp.server.fastmcp import FastMCP
37
38 # Импортировать кроссплатформенный вспомогательный Module
39 try:
40     from .config_helper import (
41         get_project_root,
42         load_config,
43         get_config_value,
44         get_server_url,
45         get_env_var,
46         get_certs_dir,
47         get_data_dir,
48         is_port_open,
49     )
50 except ImportError:
51     # Fallback если импорт не сработал
52     from config_helper import (
53         get_project_root,
54         load_config,
55         get_config_value,
56         get_server_url,
57         get_env_var,
58         get_certs_dir,
59         get_data_dir,
60         is_port_open,
61     )
62
63 try:
64     from src.logger import logger
65 except ImportError:
66     # Fallback логирование если Module недоступен
67     class SimpleLogger:
68         def info(self, msg): print(f"[INFO] {msg}")
69         def warning(self, msg): print(f"[WARN] {msg}")
```

```

70         def error(self, msg): print(f"[ERROR] {msg}")
71     logger = SimpleLogger()
72
73 # Initialization FastMCP сервера
74 mcp = FastMCP("Example-Server")
75
76 # =====
77 # Examples КРОССПЛАТФОРМЕННЫХ ФУНКЦИЙ
78 # =====
79
80 @mcp.tool()
81 def get_project_info() -> str:
82     """
83     Получить информацию о проекте (кроссплатформенно).
84
85     Returns:
86         JSON со сведениями о проекте
87     """
88     try:
89         root = get_project_root()
90         config = load_config()
91
92         info = {
93             "project_root": str(root),
94             "platform": sys.platform,
95             "python_version": sys.version,
96             "config_exists": (root / "config.json").exists(),
97             "env_exists": (root / ".env").exists(),
98             "server_url": get_server_url(),
99         }
100
101         return str(info)
102     except Exception as e:
103         logger.error(f"Error getting project info: {e}")
104         return f"Error: {e}"
105

```

```
106 @mcp.tool()
107 def get_directories() -> str:
108     """
109     Получить важные директории (кроссплатформенно).
110
111     Returns:
112         JSON с путями до директорий
113     """
114     try:
115         dirs = {
116             "project_root": str(get_project_root()),
117             "certs_dir": str(get_certs_dir()),
118             "data_dir": str(get_data_dir()),
119             "home": str(Path.home()),
120         }
121
122         return str(dirs)
123     except Exception as e:
124         logger.error(f"Error getting directories: {e}")
125         return f"Error: {e}"
126
127 @mcp.tool()
128 def check_server_status() -> str:
129     """
130     Проверить status сервера (кроссплатформенно).
131
132     Returns:
133         JSON со статусом сервера
134     """
135     try:
136         server_url = get_server_url()
137         port = get_config_value("server.port", 8000)
138         host = "127.0.0.1" # Проверяем локально
139
140         is_open = is_port_open(host, port)
141
142         status = {
143             "server_url": server_url,
```

```

144         "port": port,
145         "is_open": is_open,
146         "status": "Running" if is_open else "Not respondin
g",
147     }
148
149     return str(status)
150 except Exception as e:
151     logger.error(f"Error checking server status: {e}")
152     return f"Error: {e}"
153
154 @mcp.tool()
155 def read_config_value(key_path: str) -> str:
156     """
157     Прочитать значение из конфигурации.
158
159     Args:
160         key_path: Путь ключей через точку (например: "server.p
ort")
161
162     Returns:
163         Значение или Error
164     """
165     try:
166         value = get_config_value(key_path)
167         return f"Value for '{key_path}': {value}"
168     except Exception as e:
169         logger.error(f"Error reading config: {e}")
170         return f"Error: {e}"
171
172 @mcp.tool()
173 def read_env_variable(var_name: str) -> str:
174     """
175     Прочитать переменную окружения.
176
177     Args:
178         var_name: Имя переменной
179

```

```

180     Returns:
181         Значение или сообщение что не найдено
182         """
183     try:
184         value = get_env_var(var_name)
185         if value:
186             return f"Environment variable '{var_name}': {value}"
187         else:
188             return f"Environment variable '{var_name}' not found"
189     except Exception as e:
190         logger.error(f"Error reading env variable: {e}")
191         return f"Error: {e}"
192
193 @mcp.tool()
194 def list_certs() -> str:
195     """
196     List SSL сертификатов в кроссплатформенной директории.
197
198     Returns:
199         List файлов сертификатов
200     """
201     try:
202         certs_dir = get_certs_dir()
203
204         if not certs_dir.exists():
205             return f"Certs directory does not exist: {certs_dir}"
206
207         certs = list(certs_dir.glob("*.pem")) + list(certs_dir.glob("*.crt"))
208
209         result = f"Certificates in {certs_dir}:\n"
210         for cert in certs:
211             result += f"    - {cert.name}\n"
212
213         return result if certs else f"No certificates found in

```

```

    {certs_dir}"
214     except Exception as e:
215         logger.error(f"Error listing certs: {e}")
216         return f"Error: {e}"
217
218 # =====
    =====
219 # ГЛАВНАЯ ФУНКЦИЯ
220 # =====
    =====
221
222 def main():
223     """Главная function запуска сервера"""
224     logger.info("Starting Example MCP Server")
225     logger.info(f"Platform: {sys.platform}")
226     logger.info(f"Python: {sys.version}")
227
228     try:
229         mcp.run()
230     except KeyboardInterrupt:
231         logger.info("Server stopped by user")
232     except Exception as e:
233         logger.error(f"Server error: {e}")
234
235 if __name__ == "__main__":
236     main()
237

```

fastapi_mcp_server.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Получение базового URL FastAPI-сервера из config.json
4 # =====
5 # Description:
6 #   MCP-сервер на базе FastMCP, предоставляющий интерфейс к API бэкенда FastAPI
7 #
8 # File: fastapi_mcp_server.py
9 # Project: ai-breadboard
10 # Package: root
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from __future__ import annotations
16
17 import httpx
18 from pathlib import Path
19 from mcp.server.fastmcp import FastMCP
20
21 from src.logger import logger
22 from src.utils.json import j_loads_ns
23
24 # Initialization FastMCP сервера
25 mcp = FastMCP("FastAPI-Media-Client")
26
27 # Путь к конфигурации сервера
28 _CONFIG_PATH = Path(__file__).resolve().parent.parent / "config.json"
29
30 def get_base_url() -> str:
```

```

31 |     """Получение базового URL FastAPI-сервера из config.json."
    | ""
32 |     try:
33 |         cfg = j_loads_ns(_CONFIG_PATH)
34 |         server_cfg = getattr(cfg, "server", object())
35 |         host = getattr(server_cfg, "host", "localhost")
36 |         if host == "0.0.0.0":
37 |             host = "localhost"
38 |         port = getattr(server_cfg, "port", 8000)
39 |         return f"http://{host}:{port}"
40 |     except Exception as e:
41 |         logger.warning(f"[fastapi_mcp_server] Error чтения con
    | fig.json, fallback к localhost:8000: {e}")
42 |         return "http://localhost:8000"
43 |
44 | @mcp.tool()
45 | async def fastapi_chat(message: str) -> str:
46 |     """Отправка сообщения в чат-роутер FastAPI backend."""
47 |     try:
48 |         async with httpx.AsyncClient(timeout=30.0) as client:
49 |             response = await client.post(f"{get_base_url()}/ap
    | i/chat", json={"message": message})
50 |             return response.text
51 |     except Exception as e:
52 |         logger.error(f"[fastapi_mcp_server] Error fastapi_chat
    | : {e}")
53 |         return f"Error запроса к /api/chat: {e}"
54 |
55 | @mcp.tool()
56 | async def fastapi_media_list() -> str:
57 |     """Получение списка медиафайлов из медиа-роутера FastAPI."
    | ""
58 |     try:
59 |         async with httpx.AsyncClient(timeout=15.0) as client:
60 |             response = await client.get(f"{get_base_url()}/api
    | /media")
61 |             return response.text
62 |     except Exception as e:

```



```
63 |         logger.error(f"[fastapi_mcp_server] Error fastapi_med  
| a_list: {e}")  
64 |         return f"Error запроса к /api/media: {e}"  
65 |  
66 | @mcp.tool()  
67 | async def fastapi_qbittorrent_info() -> str:  
68 |     """Получение информации о текущих торрентах из qBittorrent  
|     через FastAPI."""  
69 |     try:  
70 |         async with httpx.AsyncClient(timeout=15.0) as client:  
71 |             response = await client.get(f"{get_base_url()}/api  
| /torrents")  
72 |             return response.text  
73 |     except Exception as e:  
74 |         logger.error(f"[fastapi_mcp_server] Error fastapi_qbit  
| torrent_info: {e}")  
75 |         return f"Error запроса к /api/torrents: {e}"  
76 |  
77 | if __name__ == "__main__":  
78 |     logger.info("[fastapi_mcp_server] Запуск FastAPI FastMCP с  
| ервера...")  
79 |     mcp.run()  
80 |
```

gemini_cli_search_mcp_server.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Выполнение поиска через GeminiCliWebSearcher.
4 # =====
5 # Description:
6 #   MCP-сервер на базе FastMCP, предоставляющий доступ к веб-п
   оиску
7 #
8 # File: gemini_cli_search_mcp_server.py
9 # Project: ai-breadboard
10 # Package: root
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from __future__ import annotations
16
17 import json
18 import asyncio
19 from pathlib import Path
20
21 try:
22     from mcp.server.fastmcp import FastMCP
23 except ImportError:
24     FastMCP = None
25
26 from src.logger import logger
27 from plugins.web_search.gemini_cli_searcher import GeminiCliWe
   bSearcher
28
29 if FastMCP:
30     mcp = FastMCP("Gemini-CLI-Search-Server")
31 else:
```

```

32     mcp = None
33
34 async def _run_gemini_cli_search(query: str, model: str = "gem
ini-3.1-flash-lite") -> str:
35     """Выполнение поиска через GeminiCliWebSearcher."""
36     try:
37         searcher = GeminiCliWebSearcher(model_id=model)
38         result_markdown = await searcher.search_and_extract(qu
ery=query, model=model)
39         return result_markdown
40     except Exception as e:
41         logger.error(f"[gemini_cli_search_mcp_server] Error по
иска: {e}")
42         return json.dumps({"status": "error", "message": str(e
)}, ensure_ascii=False)
43
44 async def gemini_cli_web_search(query: str, model: str = "gemi
ni-3.1-flash-lite") -> str:
45     """Выполнить веб-поиск через локальный терминальный агент
Google Gemini CLI.
46
47     Args:
48         query: Поисковый запрос пользователя.
49         model: Идентификатор модели (по умолчанию 'gemini-3.1-
flash-lite').
50     """
51     return await _run_gemini_cli_search(query=query, model=mod
el)
52
53 if mcp:
54     mcp.tool()(gemini_cli_web_search)
55
56 if __name__ == "__main__":
57     if mcp:
58         mcp.run()
59

```

gemini_search_mcp_server.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Выполнить веб-поиск в Google через Gemini Search G
4 # =====
5 # Description:
6 #   MCP-сервер на базе FastMCP, предоставляющий доступ к веб-поиску Google
7 #
8 # File: gemini_search_mcp_server.py
9 # Project: ai-breadboard
10 # Package: root
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from __future__ import annotations
16
17 import json
18 import asyncio
19 from pathlib import Path
20 from src.logger.logger import logger
21 from plugins.web_search.gemini_searcher import GeminiWebSearcher, GeminiKeyPool
22
23 try:
24     from mcp.server.fastmcp import FastMCP
25     mcp = FastMCP("Gemini-Search-Server")
26 except ImportError:
27     FastMCP = None
28     mcp = None
29
30 async def _run_gemini_search(query: str, model: str = "gemini-
```

```

2.5-flash") -> str:
31     try:
32         searcher = GeminiWebSearcher()
33         result_markdown = await searcher.search_and_extract(qu
    ery=query, model=model)
34         return result_markdown
35     except Exception as e:
36         logger.error(f"[gemini_search_mcp_server] Error поиска
    : {e}")
37         return json.dumps({"status": "error", "message": str(e
    )}), ensure_ascii=False)
38
39 def _run_key_pool_status() -> str:
40     try:
41         pool = GeminiKeyPool()
42         return json.dumps({
43             "status": "ok",
44             "total_keys": len(pool.api_keys),
45             "active_key_masked": f"...{pool._current_key[-6:]}
    " if len(pool._current_key) >= 6 else "****"
46         }, ensure_ascii=False)
47     except Exception as e:
48         logger.error(f"[gemini_search_mcp_server] Error статус
    а пула ключей: {e}")
49         return json.dumps({"status": "error", "message": str(e
    )}), ensure_ascii=False)
50
51 async def gemini_web_search(query: str, model: str = "gemini-2
    .5-flash") -> str:
52     """Выполнить веб-поиск в Google через Gemini Search Ground
    ing."""
53     return await _run_gemini_search(query=query, model=model)
54
55 def gemini_key_pool_status() -> str:
56     """Получить status пула API-ключей Gemini и активного ключ
    а."""
57     return _run_key_pool_status()
58

```

```
59 | if mcp:
60 |     mcp.tool()(gemini_web_search)
61 |     mcp.tool()(gemini_key_pool_status)
62 |
63 | if __name__ == "__main__":
64 |     if mcp:
65 |         mcp.run()
66 |
```

langchain_mcp_server.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Выполнить автономный поиск и решение задачи че
рез
4 # =====
5 # Description:
6 #   MCP-сервер на базе FastMCP, предоставляющий доступ к ReAct
-агенту AI Breadboard
7 #
8 # File: langchain_mcp_server.py
9 # Project: ai-breadboard
10 # Package: root
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from __future__ import annotations
16
17 import json
18 import asyncio
19 from pathlib import Path
20 from mcp.server.fastmcp import FastMCP
21
22 from src.logger import logger
23 from src.ai.langchain_agent import MediaSearchAgent
24 from src.ai.langchain_tools import (
25     web_search,
26     rag_search,
27     python_eval,
28     file_read,
29 )
30
31 # Initialization FastMCP сервера
```

```

32 mcp = FastMCP("LangChain-Breadboard-Agent")
33
34 # Путь к конфигурации
35 _CONFIG_PATH = Path(__file__).resolve().parent.parent / "config.json"
36
37 @mcp.tool()
38 async def agent_query(query: str) -> str:
39     """Выполнить автономный поиск и решение задачи через ReAct-агента.
40
41     Args:
42         query: Запрос пользователя.
43     """
44     try:
45         agent = MediaSearchAgent(config_path=_CONFIG_PATH)
46         result = await agent.search(query)
47         return json.dumps(result, ensure_ascii=False, indent=2)
48     except Exception as e:
49         logger.error(f"[langchain_mcp_server] Error agent_query: {e}")
50         return json.dumps({"action": "error", "message": str(e)}, ensure_ascii=False)
51
52 @mcp.tool()
53 async def agent_web_search(query: str) -> str:
54     """Поиск актуальной информации в интернете через поисковый адаптер.
55
56     Args:
57         query: Поисковый запрос.
58     """
59     try:
60         return await web_search.ainvoke(query)
61     except Exception as e:
62         logger.error(f"[langchain_mcp_server] Error agent_web_search: {e}")

```



```

63         return json.dumps({"error": str(e)}, ensure_ascii=False
64     e)
65
66 @mcp.tool()
67 async def agent_rag_search(query: str, top_k: int = 5) -> str:
68     """Семантический поиск по базе знаний RAG.
69
70     Args:
71         query: Поисковый запрос.
72         top_k: Количество результатов.
73     """
74     try:
75         return await rag_search.ainvoke({"query": query, "top_
76 k": top_k})
77     except Exception as e:
78         logger.error(f"[langchain_mcp_server] Error agent_rag_
79 search: {e}")
80         return json.dumps([], ensure_ascii=False)
81
82 @mcp.tool()
83 def agent_python_eval(code: str) -> str:
84     """Выполнение математических выражений на Python.
85
86     Args:
87         code: Выражение для вычисления.
88     """
89     try:
90         return python_eval.invoke(code)
91     except Exception as e:
92         logger.error(f"[langchain_mcp_server] Error agent_pyth
93 on_eval: {e}")
94         return f"Error: {e}"
95
96 if __name__ == "__main__":
97     logger.info("[langchain_mcp_server] Запуск LangChain Bread
98 board FastMCP сервера...")
99     mcp.run()

```

server.js

```
1 const express = require('express');
2 const bodyParser = require('body-parser');
3 const { execSync } = require('child_process');
4 const path = require('path');
5
6 const app = express();
7 app.use(bodyParser.json({ limit: '10mb' }));
8
9 const REPO_ROOT = path.resolve(__dirname, '..');
10
11 function git(cmd) {
12   return execSync(`git ${cmd}`, { cwd: REPO_ROOT }).toString()
13   ;
14 }
15
16 app.post('/precommit', (req, res) => {
17   const { message } = req.body;
18   if (!message) return res.status(400).send({ error: 'missing
19   message' });
20   try {
21     git('add -A');
22     const status = git('status --porcelain');
23     if (status.trim()) {
24       git(`commit -m "${message.replace(/"/g, '\\"')}"`);
25       return res.send({ ok: true, committed: true });
26     }
27     return res.send({ ok: true, committed: false });
28   } catch (e) {
29     return res.status(500).send({ error: e.message });
30   }
31 });
32
33 app.post('/apply', (req, res) => {
34   const { filePath, content } = req.body;
35   if (!filePath || typeof content !== 'string') return res.sta
36   tus(400).send({ error: 'invalid payload' });
```

```
34 |   try {
35 |     const full = path.join(REPO_ROOT, filePath);
36 |     require('fs').writeFileSync(full, content, 'utf8');
37 |     return res.send({ ok: true });
38 |   } catch (e) {
39 |     return res.status(500).send({ error: e.message });
40 |   }
41 | });
42 |
43 | const PORT = process.env.PORT || 3040;
44 | app.listen(PORT, () => console.log(`MCP server listening on ${
  | PORT}`));
45 |
```

```

unicorn_mcp_server.py
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Запуск сервиса Run-Unicorn.ps1 в фоновом режим
  е.
4 # =====
5 # Description:
6 #   MCP-сервер на базе FastMCP для управления сервисом Uvicorn
  / Unicorn
7 #
8 # File: unicorn_mcp_server.py
9 # Project: ai-breadboard
10 # Package: root
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from __future__ import annotations
16
17 import os
18 import subprocess
19 from pathlib import Path
20 from mcp.server.fastmcp import FastMCP
21
22 from src.logger import logger
23
24 # Initialization FastMCP сервера
25 mcp = FastMCP("Unicorn-Manager")
26
27 # Путь к скрипту запуска
28 _ROOT = Path(__file__).resolve().parent.parent
29 _UNICORN_SCRIPT = _ROOT / "launchers" / "Run-Unicorn.ps1"
30 if not _UNICORN_SCRIPT.exists():
31     _UNICORN_SCRIPT = _ROOT / "Run-Unicorn.ps1"

```

```

32
33 @mcp.tool()
34 async def unicorn_start() -> str:
35     """Запуск сервиса Run-Unicorn.ps1 в фоновом режиме."""
36     try:
37         if not _UNICORN_SCRIPT.exists():
38             return f"Файл скрипта не найден: {_UNICORN_SCRIPT}"
39
40         subprocess.Popen(
41             ["powershell", "-NoProfile", "-ExecutionPolicy", "Bypass", "-File", str(_UNICORN_SCRIPT)],
42             shell=True,
43             logger.info("[unicorn_mcp_server] Запущен процесс Run-Unicorn.ps1")
44         return "Сервис Unicorn successfully запущен в фоновом режиме."
45     except Exception as e:
46         logger.error(f"[unicorn_mcp_server] Error запуска unicorn_start: {e}")
47         return f"Error при запуске Unicorn: {e}"
48
49 @mcp.tool()
50 async def unicorn_stop() -> str:
51     """Остановка процессов Uvicorn/Unicorn."""
52     try:
53         subprocess.run(["taskkill", "/F", "/IM", "uvicorn.exe", "/T"], capture_output=True, text=True)
54         logger.info("[unicorn_mcp_server] Выполнена команда остановки процессов uvicorn.exe")
55         return "Команда остановки процесса Unicorn выполнена."
56     except Exception as e:
57         logger.error(f"[unicorn_mcp_server] Error остановки unicorn_stop: {e}")
58         return f"Error при остановке Unicorn: {e}"
59
60 @mcp.tool()
61 async def unicorn_status() -> str:

```

```
62 |     """Check текущего статуса работы процесса Unicorn (uunicorn
    | .exe)."""
63 |     try:
64 |         result = subprocess.run(
65 |             ["tasklist", "/FI", "IMAGENAME eq uunicorn.exe"],
66 |             capture_output=True,
67 |             text=True,
68 |         )
69 |         if "uunicorn.exe" in result.stdout:
70 |             return "Сервис Unicorn (uunicorn.exe) активен и раб
    | отает."
71 |         return "Сервис Unicorn (uunicorn.exe) не запущен."
72 |     except Exception as e:
73 |         logger.error(f"[unicorn_mcp_server] Error unicorn_stat
    | us: {e}")
74 |         return f"Error проверки статуса: {e}"
75 |
76 | if __name__ == "__main__":
77 |     logger.info("[unicorn_mcp_server] Запуск Unicorn FastMCP с
    | ервера...")
78 |     mcp.run()
79 |
```

.readthedocs.yaml

```
1 version: 2
2
3 build:
4   os: ubuntu-24.04
5   tools:
6     python: '3.12'
7
8 python:
9   install:
10    - requirements: docs-requirements.txt
11
12 mkdocs:
13   configuration: mkdocs.yml
14   fail_on_warning: true
15
```

`.readthedocs.yml`

```
1 # Read the Docs configuration for AI Breadboard project.
2 # This file specifies the build environment, Python version,
3 # and MkDocs configuration file used to generate the project
4 # documentation and reference materials.
5
6 version: 2
7
8 build:
9   os: ubuntu-22.04
10  tools:
11    python: "3.10"
12
13 python:
14   install:
15     - method: pip
16       path: .
17     - requirements: requirements-docs.txt
18
19 mkdocs:
20   configuration: mkdocs.yml
21
```


conftest.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Test configuration for ai-breadboard.
4 # =====
5 # Description:
6 #   Module for conftest.py in ai-breadboard project.
7 #
8 # File: conftest.py
9 # Project: ai-breadboard
10 # Package: root
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """
16 Test configuration for ai-breadboard.
17 Provides fixtures and settings for all tests.
18 """
19
20 import asyncio
21 import os
22 import sys
23 from pathlib import Path
24 from unittest.mock import Mock, AsyncMock, patch
25
26 import pytest
27
28 # Add project root to path
29 ROOT = Path(__file__).parent
30 sys.path.insert(0, str(ROOT))
31
32 # Configure environment variables for tests
33 os.environ['TEST_MODE'] = 'true'
```

```

34 os.environ['USE_FOUNDRY'] = 'false'
35 os.environ['PRELOAD_SILERO'] = 'false'
36
37 import ssl
38 try:
39     ssl.create_default_context()
40 except Exception:
41     _orig_create_default_context = ssl.create_default_context
42     def _safe_create_default_context(*args, **kwargs):
43         try:
44             return _orig_create_default_context(*args, **kwarg
s)
45         except Exception:
46             ctx = ssl._SSLContext(ssl.PROTOCOL_TLS_CLIENT)
47             ctx.check_hostname = False
48             ctx.verify_mode = ssl.CERT_NONE
49             return ctx
50     ssl.create_default_context = _safe_create_default_context
51
52 @pytest.fixture(scope='session')
53 def test_data_dir():
54     """Path to test data."""
55     return ROOT / 'tests' / 'data'
56
57 @pytest.fixture
58 def mock_ai_model():
59     """Mock for AI model."""
60     mock = Mock()
61     mock.chat = AsyncMock()
62     mock.chat_stream = AsyncMock()
63     mock.ask = AsyncMock()
64     mock.ask_with_tools = AsyncMock()
65     mock.describe_image = AsyncMock()
66     mock.upload_file = AsyncMock()
67     mock.clear_history = Mock()
68     return mock
69
70 @pytest.fixture

```

```
71 def mock_db():
72     """Mock for MediaDatabase."""
73     mock = Mock()
74     mock.export_all = Mock(return_value=[])
75     mock.export_movies = Mock(return_value=[])
76     mock.export_series = Mock(return_value=[])
77     mock.find_by_title = Mock(return_value=None)
78     mock.find_duplicates = Mock(return_value=[])
79     mock.get_categories = Mock(return_value=[])
80     mock.add_record = Mock(return_value=1)
81     mock.update_record = Mock(return_value=True)
82     mock.delete_record = Mock(return_value=True)
83     return mock
84
85 @pytest.fixture
86 def mock_qbt_client():
87     """Mock for QBittorrentClient."""
88     mock = Mock()
89     mock.torrents = Mock(return_value=[])
90     mock.add_torrent_by_url = Mock(return_value=True)
91     mock.add_torrent_by_file = Mock(return_value=True)
92     mock.recheck = Mock(return_value=True)
93     mock.set_location = Mock(return_value=True)
94     return mock
95
96 @pytest.fixture
97 def temp_db_path(tmp_path):
98     """Temporary database path for tests."""
99     return tmp_path / 'test_media.db'
100
101 @pytest.fixture
102 def sample_media_records():
103     """Example media records for tests."""
104     return [
105         {
106             'id': 1,
107             'title': 'Test Movie',
108             'title_ru': 'Тестовый Фильм',
```

```
109         'title_orig': 'Test Movie',
110         'type': 'movie',
111         'disk_name': 'DISK_1',
112         'year': 2024,
113         'path': 'E:/Movies/Test Movie.mkv',
114         'main_category': 'Боевики',
115         'imdb_rating': 8.5,
116         'kinopoisk_rating': 8.7,
117     },
118     {
119         'id': 2,
120         'title': 'Test Series',
121         'title_ru': 'Тестовый Сериал',
122         'title_orig': 'Test Series',
123         'type': 'series',
124         'disk_name': 'DISK_1',
125         'year': 2024,
126         'path': 'E:/Series/Test Series/S01E01.mkv',
127         'main_category': 'Драмы',
128         'season': 1,
129         'episode': 1,
130     },
131 ]
132
133 @pytest.fixture
134 def sample_torrents():
135     """Example torrents for tests."""
136     return [
137         {
138             'hash': 'abc123',
139             'name': 'Test Torrent',
140             'state': 'Downloading',
141             'progress': 0.45,
142             'size': 1073741824,
143             'save_path': 'E:/Downloads',
144         },
145     ]
146
```

```
147 | @pytest.fixture(autouse=True)
148 | def setup_env():
149 |     """Configure environment for each test."""
150 |     with patch.dict(os.environ, {
151 |         'GOOGLE_CLIENT_ID': 'test_client_id',
152 |         'GOOGLE_CLIENT_SECRET': 'test_secret',
153 |         'JWT_SECRET': 'test_jwt_secret',
154 |         'NGROK_AUTOTOKEN': 'test_ngrok',
155 |         'GEMINI_API_KEY_NAMES': 'test_key',
156 |     }, clear=True):
157 |         yield
158 |
```

coverage.xml

```
1 <?xml version="1.0" ?>
2 <coverage version="7.6.10" timestamp="1788539844885" lines-val
  id="780" lines-covered="0" line-rate="0" branches-covered="0"
  branches-valid="0" branch-rate="0" complexity="0">
3   <!-- Generated by coverage.py: https://coverage.readthedoc
  s.io/en/7.6.10 -->
4   <!-- Based on https://raw.githubusercontent.com/cobertura/
  web/master/htdocs/xml/coverage-04.dtd -->
5   <sources>
6     <source>C:\Users\onela\AppData\Local\AI-Breadboard\plu
  gins</source>
7     <source>C:\Users\onela\AppData\Local\AI-Breadboard\scr
  ipts</source>
8   </sources>
9   <packages>
10    <package name="cli" line-rate="0" branch-rate="0" comp
  lexity="0">
11      <classes>
12        <class name="__init__.py" filename="cli/__init
  __.py" complexity="0" line-rate="1" branch-rate="0">
13          <methods/>
14          <lines/>
15        </class>
16        <class name="assist.py" filename="cli/assist.p
  y" complexity="0" line-rate="0" branch-rate="0">
17          <methods/>
18          <lines>
19            <line number="32" hits="0"/>
20            <line number="33" hits="0"/>
21            <line number="34" hits="0"/>
22            <line number="35" hits="0"/>
23            <line number="36" hits="0"/>
24            <line number="37" hits="0"/>
25            <line number="38" hits="0"/>
26            <line number="41" hits="0"/>
27            <line number="43" hits="0"/>
```

```
28 | <line number="44" hits="0"/>
29 | <line number="45" hits="0"/>
30 | <line number="56" hits="0"/>
31 | <line number="57" hits="0"/>
32 | <line number="58" hits="0"/>
33 | <line number="59" hits="0"/>
34 | <line number="60" hits="0"/>
35 | <line number="61" hits="0"/>
36 | <line number="62" hits="0"/>
37 | <line number="63" hits="0"/>
38 | <line number="65" hits="0"/>
39 | <line number="68" hits="0"/>
40 | <line number="69" hits="0"/>
41 | <line number="70" hits="0"/>
42 | <line number="71" hits="0"/>
43 | <line number="72" hits="0"/>
44 | <line number="74" hits="0"/>
45 | <line number="76" hits="0"/>
46 | <line number="78" hits="0"/>
47 | <line number="80" hits="0"/>
48 | <line number="82" hits="0"/>
49 | <line number="84" hits="0"/>
50 | <line number="85" hits="0"/>
51 | <line number="86" hits="0"/>
52 | <line number="87" hits="0"/>
53 | <line number="88" hits="0"/>
54 | <line number="89" hits="0"/>
55 | <line number="90" hits="0"/>
56 | <line number="91" hits="0"/>
57 | <line number="93" hits="0"/>
58 | <line number="95" hits="0"/>
59 | <line number="96" hits="0"/>
60 | <line number="97" hits="0"/>
61 | <line number="98" hits="0"/>
62 | <line number="100" hits="0"/>
63 | <line number="102" hits="0"/>
64 | <line number="105" hits="0"/>
65 | <line number="114" hits="0"/>
```

66	<line number="115" hits="0"/>
67	<line number="116" hits="0"/>
68	<line number="117" hits="0"/>
69	<line number="118" hits="0"/>
70	<line number="120" hits="0"/>
71	<line number="122" hits="0"/>
72	<line number="123" hits="0"/>
73	<line number="124" hits="0"/>
74	<line number="126" hits="0"/>
75	<line number="129" hits="0"/>
76	<line number="130" hits="0"/>
77	<line number="131" hits="0"/>
78	<line number="136" hits="0"/>
79	<line number="137" hits="0"/>
80	<line number="138" hits="0"/>
81	<line number="139" hits="0"/>
82	<line number="141" hits="0"/>
83	<line number="143" hits="0"/>
84	<line number="145" hits="0"/>
85	<line number="146" hits="0"/>
86	<line number="147" hits="0"/>
87	<line number="149" hits="0"/>
88	<line number="151" hits="0"/>
89	<line number="152" hits="0"/>
90	<line number="153" hits="0"/>
91	<line number="154" hits="0"/>
92	<line number="155" hits="0"/>
93	<line number="156" hits="0"/>
94	<line number="157" hits="0"/>
95	<line number="158" hits="0"/>
96	<line number="160" hits="0"/>
97	<line number="162" hits="0"/>
98	<line number="164" hits="0"/>
99	<line number="165" hits="0"/>
100	<line number="167" hits="0"/>
101	<line number="169" hits="0"/>
102	<line number="170" hits="0"/>
103	<line number="172" hits="0"/>

104	<line number="174" hits="0"/>
105	<line number="176" hits="0"/>
106	<line number="177" hits="0"/>
107	<line number="178" hits="0"/>
108	<line number="180" hits="0"/>
109	<line number="182" hits="0"/>
110	<line number="183" hits="0"/>
111	<line number="184" hits="0"/>
112	<line number="185" hits="0"/>
113	<line number="186" hits="0"/>
114	<line number="187" hits="0"/>
115	<line number="189" hits="0"/>
116	<line number="190" hits="0"/>
117	<line number="192" hits="0"/>
118	<line number="193" hits="0"/>
119	<line number="194" hits="0"/>
120	<line number="197" hits="0"/>
121	<line number="198" hits="0"/>
122	<line number="199" hits="0"/>
123	<line number="200" hits="0"/>
124	<line number="201" hits="0"/>
125	<line number="202" hits="0"/>
126	<line number="204" hits="0"/>
127	<line number="205" hits="0"/>
128	<line number="207" hits="0"/>
129	<line number="210" hits="0"/>
130	<line number="211" hits="0"/>
131	<line number="212" hits="0"/>
132	<line number="214" hits="0"/>
133	<line number="215" hits="0"/>
134	<line number="216" hits="0"/>
135	<line number="218" hits="0"/>
136	<line number="221" hits="0"/>
137	<line number="222" hits="0"/>
138	<line number="223" hits="0"/>
139	<line number="225" hits="0"/>
140	<line number="226" hits="0"/>
141	<line number="228" hits="0"/>

142	<line number="230" hits="0"/>
143	<line number="231" hits="0"/>
144	<line number="233" hits="0"/>
145	<line number="234" hits="0"/>
146	<line number="235" hits="0"/>
147	<line number="238" hits="0"/>
148	<line number="239" hits="0"/>
149	<line number="240" hits="0"/>
150	<line number="241" hits="0"/>
151	<line number="242" hits="0"/>
152	<line number="243" hits="0"/>
153	<line number="244" hits="0"/>
154	<line number="245" hits="0"/>
155	<line number="248" hits="0"/>
156	<line number="249" hits="0"/>
157	<line number="250" hits="0"/>
158	<line number="251" hits="0"/>
159	<line number="252" hits="0"/>
160	<line number="253" hits="0"/>
161	<line number="256" hits="0"/>
162	<line number="257" hits="0"/>
163	<line number="258" hits="0"/>
164	<line number="259" hits="0"/>
165	<line number="260" hits="0"/>
166	<line number="261" hits="0"/>
167	<line number="262" hits="0"/>
168	<line number="263" hits="0"/>
169	<line number="266" hits="0"/>
170	<line number="267" hits="0"/>
171	<line number="268" hits="0"/>
172	<line number="269" hits="0"/>
173	<line number="270" hits="0"/>
174	<line number="271" hits="0"/>
175	<line number="272" hits="0"/>
176	<line number="273" hits="0"/>
177	<line number="276" hits="0"/>
178	<line number="277" hits="0"/>
179	<line number="278" hits="0"/>

180	<line number="279" hits="0"/>
181	<line number="280" hits="0"/>
182	<line number="281" hits="0"/>
183	<line number="282" hits="0"/>
184	<line number="283" hits="0"/>
185	<line number="284" hits="0"/>
186	<line number="286" hits="0"/>
187	<line number="288" hits="0"/>
188	<line number="290" hits="0"/>
189	<line number="291" hits="0"/>
190	<line number="292" hits="0"/>
191	<line number="293" hits="0"/>
192	<line number="296" hits="0"/>
193	<line number="297" hits="0"/>
194	<line number="298" hits="0"/>
195	<line number="299" hits="0"/>
196	<line number="300" hits="0"/>
197	<line number="301" hits="0"/>
198	<line number="302" hits="0"/>
199	<line number="304" hits="0"/>
200	<line number="306" hits="0"/>
201	<line number="307" hits="0"/>
202	<line number="308" hits="0"/>
203	<line number="309" hits="0"/>
204	<line number="311" hits="0"/>
205	<line number="312" hits="0"/>
206	<line number="313" hits="0"/>
207	<line number="315" hits="0"/>
208	<line number="316" hits="0"/>
209	<line number="317" hits="0"/>
210	<line number="318" hits="0"/>
211	<line number="319" hits="0"/>
212	<line number="320" hits="0"/>
213	<line number="321" hits="0"/>
214	<line number="322" hits="0"/>
215	<line number="323" hits="0"/>
216	<line number="324" hits="0"/>
217	<line number="326" hits="0"/>

218	<line number="328" hits="0"/>
219	<line number="330" hits="0"/>
220	<line number="331" hits="0"/>
221	<line number="332" hits="0"/>
222	<line number="334" hits="0"/>
223	<line number="335" hits="0"/>
224	<line number="336" hits="0"/>
225	<line number="337" hits="0"/>
226	<line number="339" hits="0"/>
227	<line number="340" hits="0"/>
228	<line number="341" hits="0"/>
229	<line number="342" hits="0"/>
230	<line number="343" hits="0"/>
231	<line number="345" hits="0"/>
232	<line number="346" hits="0"/>
233	<line number="347" hits="0"/>
234	<line number="348" hits="0"/>
235	<line number="350" hits="0"/>
236	<line number="351" hits="0"/>
237	<line number="352" hits="0"/>
238	<line number="353" hits="0"/>
239	<line number="356" hits="0"/>
240	<line number="357" hits="0"/>
241	<line number="358" hits="0"/>
242	<line number="359" hits="0"/>
243	<line number="360" hits="0"/>
244	<line number="361" hits="0"/>
245	<line number="362" hits="0"/>
246	<line number="365" hits="0"/>
247	<line number="366" hits="0"/>
248	<line number="367" hits="0"/>
249	<line number="368" hits="0"/>
250	<line number="369" hits="0"/>
251	<line number="370" hits="0"/>
252	<line number="371" hits="0"/>
253	<line number="374" hits="0"/>
254	<line number="375" hits="0"/>
255	<line number="376" hits="0"/>

256	<line number="378" hits="0"/>
257	<line number="379" hits="0"/>
258	<line number="381" hits="0"/>
259	<line number="383" hits="0"/>
260	<line number="385" hits="0"/>
261	<line number="386" hits="0"/>
262	<line number="387" hits="0"/>
263	<line number="392" hits="0"/>
264	<line number="394" hits="0"/>
265	<line number="396" hits="0"/>
266	<line number="413" hits="0"/>
267	<line number="416" hits="0"/>
268	<line number="417" hits="0"/>
269	<line number="420" hits="0"/>
270	<line number="421" hits="0"/>
271	<line number="424" hits="0"/>
272	<line number="427" hits="0"/>
273	<line number="430" hits="0"/>
274	<line number="433" hits="0"/>
275	<line number="434" hits="0"/>
276	<line number="435" hits="0"/>
277	<line number="438" hits="0"/>
278	<line number="439" hits="0"/>
279	<line number="440" hits="0"/>
280	<line number="441" hits="0"/>
281	<line number="444" hits="0"/>
282	<line number="446" hits="0"/>
283	<line number="449" hits="0"/>
284	<line number="450" hits="0"/>
285	<line number="451" hits="0"/>
286	<line number="453" hits="0"/>
287	<line number="456" hits="0"/>
288	<line number="457" hits="0"/>
289	<line number="458" hits="0"/>
290	<line number="459" hits="0"/>
291	<line number="460" hits="0"/>
292	<line number="461" hits="0"/>
293	<line number="462" hits="0"/>

```
294 |         <line number="463" hits="0"/>
295 |         <line number="464" hits="0"/>
296 |         <line number="465" hits="0"/>
297 |         <line number="466" hits="0"/>
298 |         <line number="467" hits="0"/>
299 |         <line number="468" hits="0"/>
300 |         <line number="469" hits="0"/>
301 |         <line number="470" hits="0"/>
302 |         <line number="471" hits="0"/>
303 |         <line number="473" hits="0"/>
304 |         <line number="474" hits="0"/>
305 |     </lines>
306 | </class>
307 | <class name="config.py" filename="cli/config.p
y" complexity="0" line-rate="0" branch-rate="0">
308 |     <methods/>
309 |     <lines>
310 |         <line number="21" hits="0"/>
311 |         <line number="22" hits="0"/>
312 |         <line number="23" hits="0"/>
313 |         <line number="24" hits="0"/>
314 |         <line number="26" hits="0"/>
315 |         <line number="28" hits="0"/>
316 |         <line number="35" hits="0"/>
317 |         <line number="37" hits="0"/>
318 |         <line number="38" hits="0"/>
319 |         <line number="39" hits="0"/>
320 |         <line number="41" hits="0"/>
321 |         <line number="50" hits="0"/>
322 |         <line number="51" hits="0"/>
323 |         <line number="53" hits="0"/>
324 |         <line number="54" hits="0"/>
325 |         <line number="55" hits="0"/>
326 |         <line number="56" hits="0"/>
327 |         <line number="57" hits="0"/>
328 |         <line number="58" hits="0"/>
329 |         <line number="60" hits="0"/>
330 |         <line number="71" hits="0"/>
```

331	<line number="72" hits="0"/>
332	<line number="73" hits="0"/>
333	<line number="74" hits="0"/>
334	<line number="80" hits="0"/>
335	<line number="81" hits="0"/>
336	<line number="82" hits="0"/>
337	<line number="83" hits="0"/>
338	<line number="85" hits="0"/>
339	<line number="94" hits="0"/>
340	<line number="96" hits="0"/>
341	<line number="97" hits="0"/>
342	<line number="99" hits="0"/>
343	<line number="100" hits="0"/>
344	<line number="101" hits="0"/>
345	<line number="102" hits="0"/>
346	<line number="103" hits="0"/>
347	<line number="104" hits="0"/>
348	<line number="106" hits="0"/>
349	<line number="107" hits="0"/>
350	<line number="108" hits="0"/>
351	<line number="110" hits="0"/>
352	<line number="111" hits="0"/>
353	<line number="113" hits="0"/>
354	<line number="115" hits="0"/>
355	<line number="125" hits="0"/>
356	<line number="126" hits="0"/>
357	<line number="127" hits="0"/>
358	<line number="128" hits="0"/>
359	<line number="129" hits="0"/>
360	<line number="130" hits="0"/>
361	<line number="131" hits="0"/>
362	<line number="132" hits="0"/>
363	<line number="133" hits="0"/>
364	<line number="135" hits="0"/>
365	<line number="146" hits="0"/>
366	<line number="149" hits="0"/>
367	<line number="150" hits="0"/>
368	<line number="152" hits="0"/>

```
369 |         <line number="153" hits="0"/>
370 |         <line number="154" hits="0"/>
371 |         <line number="156" hits="0"/>
372 |         <line number="158" hits="0"/>
373 |         <line number="160" hits="0"/>
374 |         <line number="170" hits="0"/>
375 |         <line number="173" hits="0"/>
376 |         <line number="174" hits="0"/>
377 |         <line number="176" hits="0"/>
378 |         <line number="177" hits="0"/>
379 |         <line number="178" hits="0"/>
380 |         <line number="179" hits="0"/>
381 |         <line number="181" hits="0"/>
382 |         <line number="183" hits="0"/>
383 |         <line number="185" hits="0"/>
384 |         <line number="187" hits="0"/>
385 |         <line number="188" hits="0"/>
386 |         <line number="190" hits="0"/>
387 |         <line number="191" hits="0"/>
388 |     </lines>
389 | </class>
390 |     <class name="installer.py" filename="cli/installer.py" complexity="0" line-rate="0" branch-rate="0">
391 |         <methods/>
392 |         <lines>
393 |             <line number="25" hits="0"/>
394 |             <line number="26" hits="0"/>
395 |             <line number="27" hits="0"/>
396 |             <line number="28" hits="0"/>
397 |             <line number="29" hits="0"/>
398 |             <line number="30" hits="0"/>
399 |             <line number="31" hits="0"/>
400 |             <line number="32" hits="0"/>
401 |             <line number="33" hits="0"/>
402 |             <line number="35" hits="0"/>
403 |             <line number="37" hits="0"/>
404 |             <line number="38" hits="0"/>
405 |             <line number="39" hits="0"/>
```


406	<line number="41" hits="0"/>
407	<line number="43" hits="0"/>
408	<line number="44" hits="0"/>
409	<line number="45" hits="0"/>
410	<line number="46" hits="0"/>
411	<line number="49" hits="0"/>
412	<line number="76" hits="0"/>
413	<line number="79" hits="0"/>
414	<line number="80" hits="0"/>
415	<line number="81" hits="0"/>
416	<line number="82" hits="0"/>
417	<line number="83" hits="0"/>
418	<line number="84" hits="0"/>
419	<line number="86" hits="0"/>
420	<line number="88" hits="0"/>
421	<line number="90" hits="0"/>
422	<line number="91" hits="0"/>
423	<line number="93" hits="0"/>
424	<line number="95" hits="0"/>
425	<line number="96" hits="0"/>
426	<line number="99" hits="0"/>
427	<line number="101" hits="0"/>
428	<line number="103" hits="0"/>
429	<line number="104" hits="0"/>
430	<line number="105" hits="0"/>
431	<line number="106" hits="0"/>
432	<line number="107" hits="0"/>
433	<line number="108" hits="0"/>
434	<line number="109" hits="0"/>
435	<line number="111" hits="0"/>
436	<line number="113" hits="0"/>
437	<line number="115" hits="0"/>
438	<line number="117" hits="0"/>
439	<line number="118" hits="0"/>
440	<line number="119" hits="0"/>
441	<line number="120" hits="0"/>
442	<line number="123" hits="0"/>
443	<line number="124" hits="0"/>

444	<line number="129" hits="0"/>
445	<line number="130" hits="0"/>
446	<line number="131" hits="0"/>
447	<line number="133" hits="0"/>
448	<line number="134" hits="0"/>
449	<line number="135" hits="0"/>
450	<line number="136" hits="0"/>
451	<line number="138" hits="0"/>
452	<line number="140" hits="0"/>
453	<line number="142" hits="0"/>
454	<line number="144" hits="0"/>
455	<line number="145" hits="0"/>
456	<line number="147" hits="0"/>
457	<line number="149" hits="0"/>
458	<line number="150" hits="0"/>
459	<line number="151" hits="0"/>
460	<line number="152" hits="0"/>
461	<line number="155" hits="0"/>
462	<line number="161" hits="0"/>
463	<line number="162" hits="0"/>
464	<line number="163" hits="0"/>
465	<line number="164" hits="0"/>
466	<line number="169" hits="0"/>
467	<line number="170" hits="0"/>
468	<line number="172" hits="0"/>
469	<line number="173" hits="0"/>
470	<line number="174" hits="0"/>
471	<line number="176" hits="0"/>
472	<line number="177" hits="0"/>
473	<line number="179" hits="0"/>
474	<line number="180" hits="0"/>
475	<line number="181" hits="0"/>
476	<line number="182" hits="0"/>
477	<line number="184" hits="0"/>
478	<line number="186" hits="0"/>
479	<line number="188" hits="0"/>
480	<line number="189" hits="0"/>
481	<line number="190" hits="0"/>

482	<line number="192" hits="0"/>
483	<line number="195" hits="0"/>
484	<line number="196" hits="0"/>
485	<line number="197" hits="0"/>
486	<line number="202" hits="0"/>
487	<line number="209" hits="0"/>
488	<line number="211" hits="0"/>
489	<line number="212" hits="0"/>
490	<line number="214" hits="0"/>
491	<line number="215" hits="0"/>
492	<line number="217" hits="0"/>
493	<line number="218" hits="0"/>
494	<line number="219" hits="0"/>
495	<line number="220" hits="0"/>
496	<line number="222" hits="0"/>
497	<line number="224" hits="0"/>
498	<line number="226" hits="0"/>
499	<line number="227" hits="0"/>
500	<line number="230" hits="0"/>
501	<line number="231" hits="0"/>
502	<line number="232" hits="0"/>
503	<line number="234" hits="0"/>
504	<line number="237" hits="0"/>
505	<line number="238" hits="0"/>
506	<line number="239" hits="0"/>
507	<line number="240" hits="0"/>
508	<line number="242" hits="0"/>
509	<line number="243" hits="0"/>
510	<line number="245" hits="0"/>
511	<line number="246" hits="0"/>
512	<line number="247" hits="0"/>
513	<line number="248" hits="0"/>
514	<line number="250" hits="0"/>
515	<line number="252" hits="0"/>
516	<line number="254" hits="0"/>
517	<line number="256" hits="0"/>
518	<line number="257" hits="0"/>
519	<line number="259" hits="0"/>

520	<line number="261" hits="0"/>
521	<line number="262" hits="0"/>
522	<line number="263" hits="0"/>
523	<line number="264" hits="0"/>
524	<line number="267" hits="0"/>
525	<line number="268" hits="0"/>
526	<line number="275" hits="0"/>
527	<line number="276" hits="0"/>
528	<line number="278" hits="0"/>
529	<line number="281" hits="0"/>
530	<line number="282" hits="0"/>
531	<line number="283" hits="0"/>
532	<line number="284" hits="0"/>
533	<line number="286" hits="0"/>
534	<line number="288" hits="0"/>
535	<line number="289" hits="0"/>
536	<line number="291" hits="0"/>
537	<line number="292" hits="0"/>
538	<line number="293" hits="0"/>
539	<line number="294" hits="0"/>
540	<line number="296" hits="0"/>
541	<line number="298" hits="0"/>
542	<line number="299" hits="0"/>
543	<line number="300" hits="0"/>
544	<line number="301" hits="0"/>
545	<line number="302" hits="0"/>
546	<line number="304" hits="0"/>
547	<line number="307" hits="0"/>
548	<line number="308" hits="0"/>
549	<line number="309" hits="0"/>
550	<line number="311" hits="0"/>
551	<line number="314" hits="0"/>
552	<line number="316" hits="0"/>
553	<line number="318" hits="0"/>
554	<line number="321" hits="0"/>
555	<line number="330" hits="0"/>
556	<line number="331" hits="0"/>
557	<line number="332" hits="0"/>

```
558 | <line number="333" hits="0"/>
559 | <line number="334" hits="0"/>
560 | <line number="337" hits="0"/>
561 | <line number="338" hits="0"/>
562 | <line number="339" hits="0"/>
563 | <line number="340" hits="0"/>
564 | <line number="341" hits="0"/>
565 | <line number="342" hits="0"/>
566 | <line number="343" hits="0"/>
567 | <line number="344" hits="0"/>
568 | <line number="345" hits="0"/>
569 | <line number="346" hits="0"/>
570 | <line number="348" hits="0"/>
571 | <line number="349" hits="0"/>
572 | <line number="350" hits="0"/>
573 | <line number="351" hits="0"/>
574 | <line number="352" hits="0"/>
575 | <line number="354" hits="0"/>
576 | <line number="355" hits="0"/>
577 | <line number="356" hits="0"/>
578 | <line number="357" hits="0"/>
579 | <line number="358" hits="0"/>
580 | <line number="359" hits="0"/>
581 | <line number="360" hits="0"/>
582 | <line number="361" hits="0"/>
583 | <line number="363" hits="0"/>
584 | <line number="365" hits="0"/>
585 | <line number="377" hits="0"/>
586 | <line number="384" hits="0"/>
587 | <line number="391" hits="0"/>
588 | <line number="397" hits="0"/>
589 | <line number="403" hits="0"/>
590 | <line number="409" hits="0"/>
591 | <line number="415" hits="0"/>
592 | <line number="417" hits="0"/>
593 | <line number="422" hits="0"/>
594 | </lines>
595 | </class>
```

```
596 |         <class name="paths.py" filename="cli/paths.py"
      | complexity="0" line-rate="0" branch-rate="0">
597 |             <methods/>
598 |             <lines>
599 |                 <line number="21" hits="0"/>
600 |                 <line number="22" hits="0"/>
601 |                 <line number="23" hits="0"/>
602 |                 <line number="24" hits="0"/>
603 |                 <line number="26" hits="0"/>
604 |                 <line number="27" hits="0"/>
605 |                 <line number="28" hits="0"/>
606 |                 <line number="32" hits="0"/>
607 |                 <line number="40" hits="0"/>
608 |                 <line number="47" hits="0"/>
609 |                 <line number="48" hits="0"/>
610 |                 <line number="49" hits="0"/>
611 |                 <line number="50" hits="0"/>
612 |                 <line number="52" hits="0"/>
613 |                 <line number="53" hits="0"/>
614 |                 <line number="59" hits="0"/>
615 |                 <line number="60" hits="0"/>
616 |                 <line number="61" hits="0"/>
617 |                 <line number="62" hits="0"/>
618 |                 <line number="63" hits="0"/>
619 |                 <line number="65" hits="0"/>
620 |                 <line number="67" hits="0"/>
621 |                 <line number="68" hits="0"/>
622 |                 <line number="70" hits="0"/>
623 |                 <line number="72" hits="0"/>
624 |                 <line number="73" hits="0"/>
625 |                 <line number="80" hits="0"/>
626 |                 <line number="81" hits="0"/>
627 |                 <line number="84" hits="0"/>
628 |                 <line number="85" hits="0"/>
629 |                 <line number="86" hits="0"/>
630 |                 <line number="87" hits="0"/>
631 |                 <line number="88" hits="0"/>
632 |                 <line number="90" hits="0"/>
```

633	<line number="92" hits="0"/>
634	<line number="93" hits="0"/>
635	<line number="100" hits="0"/>
636	<line number="101" hits="0"/>
637	<line number="103" hits="0"/>
638	<line number="104" hits="0"/>
639	<line number="105" hits="0"/>
640	<line number="106" hits="0"/>
641	<line number="108" hits="0"/>
642	<line number="110" hits="0"/>
643	<line number="111" hits="0"/>
644	<line number="118" hits="0"/>
645	<line number="119" hits="0"/>
646	<line number="121" hits="0"/>
647	<line number="122" hits="0"/>
648	<line number="123" hits="0"/>
649	<line number="124" hits="0"/>
650	<line number="126" hits="0"/>
651	<line number="128" hits="0"/>
652	<line number="129" hits="0"/>
653	<line number="136" hits="0"/>
654	<line number="137" hits="0"/>
655	<line number="138" hits="0"/>
656	<line number="139" hits="0"/>
657	<line number="141" hits="0"/>
658	<line number="143" hits="0"/>
659	<line number="144" hits="0"/>
660	<line number="146" hits="0"/>
661	<line number="148" hits="0"/>
662	<line number="149" hits="0"/>
663	<line number="155" hits="0"/>
664	<line number="157" hits="0"/>
665	<line number="158" hits="0"/>
666	<line number="160" hits="0"/>
667	<line number="162" hits="0"/>
668	<line number="163" hits="0"/>
669	<line number="165" hits="0"/>
670	<line number="166" hits="0"/>

671	<line number="168" hits="0"/>
672	<line number="170" hits="0"/>
673	<line number="171" hits="0"/>
674	<line number="173" hits="0"/>
675	<line number="175" hits="0"/>
676	<line number="176" hits="0"/>
677	<line number="178" hits="0"/>
678	<line number="180" hits="0"/>
679	<line number="181" hits="0"/>
680	<line number="183" hits="0"/>
681	<line number="185" hits="0"/>
682	<line number="186" hits="0"/>
683	<line number="188" hits="0"/>
684	<line number="190" hits="0"/>
685	<line number="191" hits="0"/>
686	<line number="193" hits="0"/>
687	<line number="195" hits="0"/>
688	<line number="196" hits="0"/>
689	<line number="198" hits="0"/>
690	<line number="200" hits="0"/>
691	<line number="201" hits="0"/>
692	<line number="203" hits="0"/>
693	<line number="205" hits="0"/>
694	<line number="206" hits="0"/>
695	<line number="208" hits="0"/>
696	<line number="210" hits="0"/>
697	<line number="216" hits="0"/>
698	<line number="223" hits="0"/>
699	<line number="225" hits="0"/>
700	<line number="226" hits="0"/>
701	<line number="227" hits="0"/>
702	<line number="229" hits="0"/>
703	<line number="231" hits="0"/>
704	<line number="241" hits="0"/>
705	<line number="242" hits="0"/>
706	<line number="245" hits="0"/>
707	<line number="247" hits="0"/>
708	<line number="254" hits="0"/>


```

709         <line number="255" hits="0"/>
710         <line number="256" hits="0"/>
711         <line number="258" hits="0"/>
712         <line number="264" hits="0"/>
713         <line number="265" hits="0"/>
714         <line number="266" hits="0"/>
715         <line number="267" hits="0"/>
716     </lines>
717 </class>
718 <class name="utils.py" filename="cli/utils.py"
complexity="0" line-rate="0" branch-rate="0">
719     <methods/>
720     <lines>
721         <line number="18" hits="0"/>
722         <line number="19" hits="0"/>
723         <line number="20" hits="0"/>
724         <line number="21" hits="0"/>
725         <line number="22" hits="0"/>
726         <line number="23" hits="0"/>
727         <line number="25" hits="0"/>
728         <line number="39" hits="0"/>
729         <line number="40" hits="0"/>
730         <line number="41" hits="0"/>
731         <line number="42" hits="0"/>
732         <line number="43" hits="0"/>
733         <line number="44" hits="0"/>
734         <line number="45" hits="0"/>
735         <line number="46" hits="0"/>
736         <line number="48" hits="0"/>
737         <line number="50" hits="0"/>
738         <line number="60" hits="0"/>
739         <line number="61" hits="0"/>
740         <line number="62" hits="0"/>
741         <line number="63" hits="0"/>
742         <line number="64" hits="0"/>
743         <line number="65" hits="0"/>
744         <line number="66" hits="0"/>
745         <line number="68" hits="0"/>

```

746	<line number="77" hits="0"/>
747	<line number="78" hits="0"/>
748	<line number="80" hits="0"/>
749	<line number="87" hits="0"/>
750	<line number="88" hits="0"/>
751	<line number="89" hits="0"/>
752	<line number="90" hits="0"/>
753	<line number="91" hits="0"/>
754	<line number="92" hits="0"/>
755	<line number="93" hits="0"/>
756	<line number="94" hits="0"/>
757	<line number="95" hits="0"/>
758	<line number="96" hits="0"/>
759	<line number="101" hits="0"/>
760	<line number="102" hits="0"/>
761	<line number="109" hits="0"/>
762	<line number="110" hits="0"/>
763	<line number="111" hits="0"/>
764	<line number="112" hits="0"/>
765	<line number="113" hits="0"/>
766	<line number="114" hits="0"/>
767	<line number="115" hits="0"/>
768	<line number="116" hits="0"/>
769	<line number="117" hits="0"/>
770	<line number="118" hits="0"/>
771	<line number="120" hits="0"/>
772	<line number="122" hits="0"/>
773	<line number="124" hits="0"/>
774	<line number="125" hits="0"/>
775	<line number="127" hits="0"/>
776	<line number="129" hits="0"/>
777	<line number="138" hits="0"/>
778	<line number="139" hits="0"/>
779	<line number="140" hits="0"/>
780	<line number="147" hits="0"/>
781	<line number="148" hits="0"/>
782	<line number="149" hits="0"/>
783	<line number="150" hits="0"/>

```
784 | <line number="151" hits="0"/>
785 | <line number="154" hits="0"/>
786 | <line number="161" hits="0"/>
787 | <line number="162" hits="0"/>
788 | <line number="163" hits="0"/>
789 | <line number="165" hits="0"/>
790 | <line number="168" hits="0"/>
791 | <line number="170" hits="0"/>
792 | <line number="180" hits="0"/>
793 | <line number="181" hits="0"/>
794 | <line number="182" hits="0"/>
795 | <line number="183" hits="0"/>
796 | <line number="184" hits="0"/>
797 | <line number="186" hits="0"/>
798 | <line number="187" hits="0"/>
799 | <line number="190" hits="0"/>
800 | <line number="191" hits="0"/>
801 | <line number="192" hits="0"/>
802 | <line number="193" hits="0"/>
803 | <line number="195" hits="0"/>
804 | <line number="196" hits="0"/>
805 | <line number="197" hits="0"/>
806 | <line number="199" hits="0"/>
807 | <line number="218" hits="0"/>
808 | <line number="219" hits="0"/>
809 | <line number="220" hits="0"/>
810 | <line number="222" hits="0"/>
811 | <line number="230" hits="0"/>
812 | <line number="239" hits="0"/>
813 | <line number="240" hits="0"/>
814 | <line number="247" hits="0"/>
815 | <line number="248" hits="0"/>
816 | <line number="249" hits="0"/>
817 | <line number="251" hits="0"/>
818 | <line number="254" hits="0"/>
819 | </lines>
820 | </class>
821 | </classes>
```

822 | </package>

823 | </packages>

824 | </coverage>

825 |

chat.js

```
1 // Chat Widget for ai-breadboard Documentation
2 (function() {
3   document.addEventListener('DOMContentLoaded', () => {
4     // 1. Create and inject chat elements
5     const chatContainer = document.createElement('div');
6     chatContainer.id = 'doc-chat-widget';
7     chatContainer.innerHTML = `
8       <button class="chat-toggle-btn" id="chat-toggle-btn" title="Открыть чат Code Helper">⌵ </button>
9       <div class="chat-popup" id="chat-popup">
10         <div class="chat-popup-header">
11           <span>⌵ Code Helper (Docs Chat)</span>
12           <div class="chat-window-controls">
13             <button class="chat-window-btn maximize" id="chat-max-btn" title="Развернуть">⏏ </button>
14             <button class="chat-window-btn minimize" id="chat-min-btn" title="Свернуть">_ </button>
15             <button class="chat-window-btn close" id="chat-close-btn" title="Закрыть">⌵ </button>
16           </div>
17         </div>
18         <div class="chat-popup-messages" id="chat-popup-messages">
19           <div class="chat-popup-msg bot">
20             <strong>Code Helper</strong>: Привет! Я ассистент разработчика ai-breadboard. Задайте мне вопрос по коду или документации проекта.
21           </div>
22         </div>
23         <div class="chat-popup-input-area">
24           <input type="text" id="chat-popup-input" placeholder="Задайте вопрос по проекту...">
25           <button id="chat-popup-send">Отправить</button>
26         </div>
27       </div>
28     `;
29   });
30 }
```

```
29 | document.body.appendChild(chatContainer);
30 |
31 | // 2. Element References
32 | const toggleBtn = document.getElementById('chat-toggle-btn
    | ');
33 | const popup = document.getElementById('chat-popup');
34 | const closeBtn = document.getElementById('chat-close-btn')
    | ;
35 | const maxBtn = document.getElementById('chat-max-btn');
36 | const minBtn = document.getElementById('chat-min-btn');
37 | const sendBtn = document.getElementById('chat-popup-send')
    | ;
38 | const inputField = document.getElementById('chat-popup-inp
    | ut');
39 | const messagesContainer = document.getElementById('chat-po
    | pup-messages');
40 |
41 | let isMaximized = false;
42 |
43 | // 3. UI State Actions
44 | function setMaximizedState(max) {
45 |     isMaximized = max;
46 |     if (max) {
47 |         popup.classList.add('maximized');
48 |         maxBtn.textContent = '□';
49 |     } else {
50 |         popup.classList.remove('maximized');
51 |         maxBtn.textContent = '■';
52 |     }
53 | }
54 |
55 | toggleBtn.addEventListener('click', () => {
56 |     popup.classList.toggle('show');
57 |     if (popup.classList.contains('show')) {
58 |         inputField.focus();
59 |         setMaximizedState(false);
60 |     }
61 | });
```

```
62 |
63 |     closeBtn.addEventListener('click', () => {
64 |         popup.classList.remove('show');
65 |     });
66 |
67 |     maxBtn.addEventListener('click', (e) => {
68 |         e.stopPropagation();
69 |         setMaximizedState(!isMaximized);
70 |     });
71 |
72 |     minBtn.addEventListener('click', (e) => {
73 |         e.stopPropagation();
74 |         setMaximizedState(false);
75 |     });
76 |
77 |     // 4. Send Message logic
78 |     async function sendMessage() {
79 |         const msg = inputField.value.trim();
80 |         if (!msg) return;
81 |
82 |         addMessage('user', msg);
83 |         inputField.value = '';
84 |         sendBtn.disabled = true;
85 |         sendBtn.textContent = '❏ ';
86 |
87 |         try {
88 |             const response = await fetch('https://kino.davidka.net
89 | /api/chat/code-helper', {
90 |                 method: 'POST',
91 |                 headers: {
92 |                     'Content-Type': 'application/json'
93 |                 },
94 |                 body: JSON.stringify({ message: msg })
95 |             });
96 |
97 |             if (!response.ok) {
98 |                 throw new Error(`Server returned HTTP ${response.sta
99 | tus}`);
```

```

98         }
99
100         const data = await response.json();
101         addMessage('bot', data.text || 'Нет ответа от ассистен
та.');
```

```

102     } catch (err) {
103         addMessage('bot', `Ошибка: ${err.message}`);
104     } finally {
105         sendBtn.disabled = false;
106         sendBtn.textContent = 'Отправить';
107         inputField.focus();
108     }
109 }
110
111 function addMessage(sender, text) {
112     const msgEl = document.createElement('div');
113     msgEl.className = `chat-popup-msg ${sender}`;
114
115     // Simple HTML escaping and formatting
116     const formattedText = text
117         .replace(/&/g, '&')
118         .replace(/</g, '<')
119         .replace(/>/g, '>')
120         .replace(/\n/g, '<br>')
121         .replace(/`([^\`]+)`/g, '<code>$1</code>');
122
123     msgEl.innerHTML = `<strong>${sender === 'user' ? 'Вы' :
'Code Helper'}</strong>: ${formattedText}`;
124     messagesContainer.appendChild(msgEl);
125     messagesContainer.scrollTop = messagesContainer.scrollHeight;
126 }
127
128 // 5. Input Event Listeners
129 sendBtn.addEventListener('click', sendMessage);
130 inputField.addEventListener('keypress', (e) => {
131     if (e.key === 'Enter') sendMessage();
132 });

```



```
133 |     });
```

```
134 | })();
```

```
135 |
```

extra.css

```
1 /* Floating Chat button for Documentation */
2 .chat-toggle-btn {
3     position: fixed;
4     bottom: 30px;
5     right: 30px;
6     width: 55px;
7     height: 55px;
8     border-radius: 50%;
9     background: rgba(13, 110, 253, 0.9);
10    border: 1px solid rgba(255, 255, 255, 0.15);
11    color: #fff;
12    font-size: 26px;
13    cursor: pointer;
14    display: flex;
15    align-items: center;
16    justify-content: center;
17    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.4);
18    transition: all 0.3s cubic-bezier(0.175, 0.885, 0.32, 1.275)
19    ;
20    z-index: 9999;
21 }
22 .chat-toggle-btn:hover {
23     transform: scale(1.1);
24     background: #0b5ed7;
25     box-shadow: 0 6px 20px rgba(13, 110, 253, 0.5);
26 }
27 .chat-popup {
28     position: fixed;
29     bottom: 100px;
30     right: 30px;
31     width: 380px;
32     height: 500px;
33     max-width: 90vw;
34     max-height: 80vh;
35     background: rgba(30, 30, 30, 0.95);
36     backdrop-filter: blur(10px);
```

```
36 | -webkit-backdrop-filter: blur(10px);
37 | border: 1px solid rgba(255, 255, 255, 0.15);
38 | border-radius: 12px;
39 | display: flex;
40 | flex-direction: column;
41 | box-shadow: 0 10px 30px rgba(0, 0, 0, 0.5);
42 | z-index: 9999;
43 | opacity: 0;
44 | transform: translateY(20px) scale(0.95);
45 | pointer-events: none;
46 | transition: opacity 0.3s cubic-bezier(0.165, 0.84, 0.44, 1),
    | transform 0.3s cubic-bezier(0.165, 0.84, 0.44, 1);
47 | overflow: hidden;
48 | }
49 | .chat-popup.show {
50 |   opacity: 1;
51 |   transform: translateY(0) scale(1);
52 |   pointer-events: all;
53 | }
54 | .chat-popup.maximized {
55 |   width: 90vw;
56 |   height: 85vh;
57 |   bottom: 5vh;
58 |   right: 5vw;
59 | }
60 | .chat-popup-header {
61 |   padding: 12px 15px;
62 |   border-bottom: 1px solid rgba(255, 255, 255, 0.1);
63 |   display: flex;
64 |   justify-content: space-between;
65 |   align-items: center;
66 |   font-weight: 600;
67 |   font-size: 14px;
68 |   color: #fff;
69 |   background: rgba(0, 0, 0, 0.3);
70 |   border-top-left-radius: 12px;
71 |   border-top-right-radius: 12px;
72 | }
```

```
73 | .chat-window-controls {
74 |   display: flex;
75 |   gap: 6px;
76 | }
77 | .chat-window-btn {
78 |   background: transparent;
79 |   border: none;
80 |   color: #aaa;
81 |   font-size: 14px;
82 |   cursor: pointer;
83 |   padding: 4px 8px;
84 |   border-radius: 4px;
85 |   transition: all 0.2s;
86 |   min-width: 24px;
87 |   height: 24px;
88 |   display: flex;
89 |   align-items: center;
90 |   justify-content: center;
91 | }
92 | .chat-window-btn:hover {
93 |   background: rgba(255, 255, 255, 0.15);
94 |   color: #fff;
95 | }
96 | .chat-window-btn.settings {
97 |   color: #0d6efd;
98 | }
99 | .chat-window-btn.maximize {
100 |   color: #e0c044;
101 | }
102 | .chat-window-btn.minimize {
103 |   color: #e0a044;
104 | }
105 | .chat-window-btn.close {
106 |   color: #f44336;
107 | }
108 | .chat-window-btn.close:hover {
109 |   background: #f44336;
110 |   color: #fff;
```

```
111 }
112 .chat-popup-messages {
113     flex: 1;
114     padding: 15px;
115     overflow-y: auto;
116     display: flex;
117     flex-direction: column;
118     gap: 12px;
119 }
120 .chat-popup-msg {
121     max-width: 85%;
122     padding: 8px 12px;
123     border-radius: 8px;
124     font-size: 13px;
125     line-height: 1.4;
126     word-break: break-word;
127 }
128 .chat-popup-msg.bot {
129     background: rgba(255, 255, 255, 0.08);
130     color: #e0e0e0;
131     align-self: flex-start;
132     border-bottom-left-radius: 2px;
133 }
134 .chat-popup-msg.user {
135     background: #0d6efd;
136     color: #fff;
137     align-self: flex-end;
138     border-bottom-right-radius: 2px;
139 }
140 .chat-popup-input-area {
141     padding: 12px;
142     border-top: 1px solid rgba(255, 255, 255, 0.1);
143     display: flex;
144     gap: 8px;
145     background: rgba(0, 0, 0, 0.2);
146     border-bottom-left-radius: 12px;
147     border-bottom-right-radius: 12px;
148 }
```

```
149 | .chat-popup-input-area input {
150 |     flex: 1;
151 |     padding: 8px 12px;
152 |     border: 1px solid rgba(255, 255, 255, 0.1);
153 |     border-radius: 6px;
154 |     background: rgba(255, 255, 255, 0.05);
155 |     color: #fff;
156 |     font-size: 13px;
157 |     outline: none;
158 | }
159 | .chat-popup-input-area input:focus {
160 |     border-color: #0d6efd;
161 |     background: rgba(255, 255, 255, 0.08);
162 | }
163 | .chat-popup-input-area button {
164 |     padding: 8px 15px;
165 |     background: #0d6efd;
166 |     color: #fff;
167 |     border: none;
168 |     border-radius: 6px;
169 |     font-size: 13px;
170 |     cursor: pointer;
171 |     transition: background 0.2s;
172 | }
173 | .chat-popup-input-area button:hover {
174 |     background: #0b5ed7;
175 | }
176 | .chat-popup-input-area button:disabled {
177 |     opacity: 0.6;
178 |     cursor: not-allowed;
179 | }
180 | .d-none {
181 |     display: none !important;
182 | }
183 |
```

header.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Project root path determination and initialization
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: header.py
9 # Project: ai-breadboard
10 # Package: root
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """Module defining root path to the project.
16
17 All imports are built relative to the project root path determined by this module."""
18
19 import os
20 import sys
21 from pathlib import Path
22
23 def set_project_root(marker_files=('__root__', '.git')) -> Path:
24     """
25     Finds the root directory of the project starting from the
26     current file's directory,
27     searching upwards and stopping at the first directory containing any of the marker files.
28     Also respects the AIBREADBOARD_DIR environment variable if present.
```

```

28
29     Args:
30         marker_files (tuple): Filenames or directory names to
    identify the project root.
31
32     Returns:
33         Path: Path to the root directory if found, otherwise t
    he directory where the script is located.
34     """
35     env_dir = os.environ.get("AIBREADBOARD_DIR", "")
36     if env_dir:
37         env_path = Path(env_dir).resolve()
38         if env_path.exists() and (env_path / "config.json").ex
    ists():
39             if str(env_path) not in sys.path:
40                 sys.path.insert(0, str(env_path))
41             return env_path
42
43     __root__: Path
44     current_path: Path = Path(__file__).resolve().parent
45     __root__ = current_path
46     for parent in [current_path] + list(current_path.parents):
47         if any((parent / marker).exists() for marker in marker
    _files):
48             __root__ = parent
49             break
50     if str(__root__) not in sys.path:
51         sys.path.insert(0, str(__root__))
52     return __root__
53
54 # Get the root directory of the project
55 __root__: Path = set_project_root()
56 """__root__ (Path): Path to the root directory of the project"
    """
57
58

```



```

install.py
1 # -*- coding: utf-8 -*-
2 # =====
  =====
3 # Process Name: Cross-platform installer for AI Breadboard
4 # =====
  =====
5 # Description:
6 #   Universal installer for Windows, Linux, and macOS with mul
  tilingual
7 #   мультиязычности (RU/EN/ES/HE) и модульной архитектурой.
8 #
9 # Examples:
10 #   python install.py
11 #   python install.py --language en --install-dir /opt/ai-brea
  dboard
12 #
13 # File: install.py
14 # Project: AI Breadboard
15 # Package: Installation
16 # Module: Core
17 # Class: Installer
18 # Function: main
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # =====
  =====
22
23 import os
24 import sys
25 import json
26 import platform
27 import subprocess
28 import shutil
29 from pathlib import Path
30 from typing import Optional, Dict, List
31 from enum import StrEnum

```

```

32 |
33 |
34 | class Language(StrEnum):
35 |     """Поддерживаемые языки установщика."""
36 |     RU = "ru"
37 |     EN = "en"
38 |     ES = "es"
39 |     HE = "he"
40 |
41 |
42 | class I18N:
43 |     """Система мультиязычности установщика."""
44 |
45 |     MESSAGES: Dict[str, Dict[str, str]] = {
46 |         "ru": {
47 |             "welcome": "□ Добро пожаловать в установщик AI Br
eadboard",
48 |             "select_lang": "Выберите язык установки / Select l
anguage:",
49 |             "lang_selected": "□ Выбран язык: {lang}",
50 |             "step_1": "[1/6] Проверка Python интерпретатора...
",
51 |             "step_1_found": "□ Найден Python {version}: {path}
",
52 |             "step_1_not_found": "□ Python не найден. Установит
e Python 3.10+",
53 |             "step_2": "[2/6] Создание виртуального окружения..
.",
54 |             "step_2_ok": "□ Виртуальное окружение создано",
55 |             "step_2_exists": "□ Виртуальное окружение уже суще
ствует",
56 |             "step_3": "[3/6] Обновление pip и инструментов..."
,
57 |             "step_3_ok": "□ pip обновлен",
58 |             "step_4": "[4/6] Установка зависимостей...",
59 |             "step_4_menu": "Выберите профиль установки:",
60 |             "step_4_opt_1": "[1] Полная установка (рекомендует
ся)",

```

```

61 |         "step_4_opt_2": "[2] Только Core",
62 |         "step_4_opt_3": "[3] Core + AI",
63 |         "step_4_opt_4": "[4] Полная + Dev",
64 |         "step_4_opt_5": "[5] Пропустить",
65 |         "step_4_ok": "☐ Зависимости установлены",
66 |         "step_5": "[5/6] Проверка SSL сертификатов...",
67 |         "step_5_ok": "☐ SSL сертификаты найдены",
68 |         "step_6": "[6/6] Финальная проверка...",
69 |         "step_6_ok": "☐ Окружение готово к работе",
70 |         "finish": "☐ Установка завершена успешно!",
71 |         "error": "☐ Ошибка: {msg}",
72 |     },
73 |     "en": {
74 |         "welcome": "☐ Welcome to AI Breadboard Installer"
75 |     },
76 |     "select_lang": "Select installation language / Выберите язык:",
77 |     "lang_selected": "☐ Selected language: {lang}",
78 |     "step_1": "[1/6] Checking Python interpreter...",
79 |     "step_1_found": "☐ Found Python {version}: {path}"
80 |     ,
81 |     "step_1_not_found": "☐ Python not found. Install Python 3.10+",
82 |     "step_2": "[2/6] Creating virtual environment...",
83 |     "step_2_ok": "☐ Virtual environment created",
84 |     "step_2_exists": "☐ Virtual environment already exists",
85 |     "step_3": "[3/6] Upgrading pip and tools...",
86 |     "step_3_ok": "☐ pip upgraded",
87 |     "step_4": "[4/6] Installing dependencies...",
88 |     "step_4_menu": "Select installation profile:",
89 |     "step_4_opt_1": "[1] Full installation (recommended)",
90 |     "step_4_opt_2": "[2] Core only",
91 |     "step_4_opt_3": "[3] Core + AI",
92 |     "step_4_opt_4": "[4] Full + Dev",
93 |     "step_4_opt_5": "[5] Skip",
94 |     "step_4_ok": "☐ Dependencies installed",

```

```

93 |         "step_5": "[5/6] Checking SSL certificates...",
94 |         "step_5_ok": "  SSL certificates found",
95 |         "step_6": "[6/6] Final verification...",
96 |         "step_6_ok": "  Environment ready",
97 |         "finish": "  Installation completed successfully!"
98 |     ,
99 |         "error": "  Error: {msg}",
100 |     },
101 |     "es": {
102 |         "welcome": "  Bienvenido al instalador de AI Brea
103 | dboard",
104 |         "select_lang": "Seleccione idioma / Select languag
105 | e:",
106 |         "lang_selected": "  Idioma seleccionado: {lang}",
107 |         "step_1": "[1/6] Verificando intérprete Python..."
108 |     ,
109 |         "step_1_found": "  Python {version} encontrado: {p
110 | ath}",
111 |         "step_1_not_found": "  Python no encontrado. Insta
112 | le Python 3.10+",
113 |         "step_2": "[2/6] Creando entorno virtual...",
114 |         "step_2_ok": "  Entorno virtual creado",
115 |         "step_2_exists": "  Entorno virtual ya existe",
116 |         "step_3": "[3/6] Actualizando pip...",
117 |         "step_3_ok": "  pip actualizado",
118 |         "step_4": "[4/6] Instalando dependencias...",
119 |         "step_4_menu": "Seleccione perfil de instalación:"
120 |     ,
121 |         "step_4_opt_1": "[1] Instalación completa (recomen
122 | dado)",
123 |         "step_4_opt_2": "[2] Solo Core",
124 |         "step_4_opt_3": "[3] Core + AI",
125 |         "step_4_opt_4": "[4] Completo + Dev",
126 |         "step_4_opt_5": "[5] Omitir",
127 |         "step_4_ok": "  Dependencias instaladas",
128 |         "step_5": "[5/6] Verificando certificados SSL...",
129 |         "step_5_ok": "  Certificados SSL encontrados",
130 |         "step_6": "[6/6] Verificación final...",

```

```

123         "step_6_ok": "Entorno listo",
124         "finish": "¡Instalación completada exitosamente!",
125     },
126     "error": "Error: {msg}",
127     },
128     "he": {
129         "welcome": "Bienvenido al entorno de desarrollo AI Breadboard",
130         "select_lang": "Selecciona un idioma / Select language:",
131         "lang_selected": "Idioma seleccionado: {lang}",
132         "step_1": "[1/6] Configurando el entorno de Python...",
133         "step_1_found": "Python {version} encontrado en: {path}",
134         "step_1_not_found": "Python no encontrado. Se requiere Python 3.10+",
135         "step_2": "[2/6] Instalando dependencias...",
136         "step_2_ok": "Dependencias instaladas correctamente",
137         "step_2_exists": "Dependencias ya instaladas",
138         "step_3": "[3/6] Instalando pip...",
139         "step_3_ok": "pip instalado correctamente",
140         "step_4": "[4/6] Configurando el entorno de desarrollo...",
141         "step_4_menu": "Selecciona una opción:",
142         "step_4_opt_1": "[1] Configuración básica (recomendada)",
143         "step_4_opt_2": "[2] Core de desarrollo",
144         "step_4_opt_3": "[3] Core + AI",
145         "step_4_opt_4": "[4] Core + Dev",
146         "step_4_opt_5": "[5] Core + AI + Dev",
147         "step_4_ok": "Entorno de desarrollo configurado",
148         "step_5": "[5/6] Instalando dependencias de SSL...",
149         "step_5_ok": "Dependencias de SSL instaladas",
150         "step_6": "[6/6] Configurando el entorno de desarrollo...",
151         "step_6_ok": "Entorno de desarrollo configurado",
152         "finish": "¡Instalación completada exitosamente!",
153         "error": "Error: {msg}",
154     },
155     },
156 }
157
158 def __init__(self, language: Language = Language.EN):
159     """Inicialization системы мультитязычности.

```

```

159 |         Args:
160 |             language (Language): Выбранный язык установки.
161 |         """
162 |         self.language: Language = language
163 |
164 |     def msg(self, key: str, **kwargs) -> str:
165 |         """Получение переведенного сообщения.
166 |
167 |         Args:
168 |             key (str): Ключ сообщения в словаре.
169 |             **kwargs: Параметры для форматирования строки.
170 |
171 |         Returns:
172 |             str: Переведенное сообщение.
173 |         """
174 |         message: str = self.MESSAGES.get(self.language.value,
175 |         {}).get(key, key)
176 |         return message.format(**kwargs) if kwargs else message
177 |
178 | class Installer:
179 |     """Основной класс установщика AI Breadboard."""
180 |
181 |     def __init__(self, language: Language = Language.EN):
182 |         """Initialization установщика.
183 |
184 |         Args:
185 |             language (Language): Язык установки.
186 |         """
187 |         self.i18n: I18N = I18N(language)
188 |         self.platform_name: str = platform.system()
189 |         self.install_dir: Path = Path()
190 |         self.venv_dir: Path = Path()
191 |         self.python_path: Path = Path()
192 |
193 |     def msg(self, key: str, **kwargs) -> str:
194 |         """Получение сообщения через i18n.
195 |

```

```

196 |     Args:
197 |         key (str): Ключ сообщения.
198 |         **kwargs: Параметры форматирования.
199 |
200 |     Returns:
201 |         str: Переведенное сообщение.
202 |     """
203 |     return self.i18n.msg(key, **kwargs)
204 |
205 | def find_python(self) -> Optional[Path]:
206 |     """Поиск интерпретатора Python.
207 |
208 |     Returns:
209 |         Optional[Path]: Путь к Python или пустое значение.
210 |
211 |     Examples:
212 |         >>> installer = Installer()
213 |         >>> python_path = installer.find_python()
214 |         >>> print(python_path)
215 |     """
216 |     print(self.msg("step_1"))
217 |
218 |     preferred_versions: List[str] = ["3.13", "3.12", "3.11", "3.10"]
219 |
220 |     for version in preferred_versions:
221 |         try:
222 |             result: subprocess.CompletedProcess = subprocess.run(
223 |                 ["python" + version, "--version"],
224 |                 capture_output=True,
225 |                 text=True,
226 |                 timeout=5
227 |             )
228 |             if result.returncode == 0:
229 |                 python_exe: Path = shutil.which("python" +
version)
230 |                 if python_exe:

```

```

231 |             print(self.msg("step_1_found", version
| =version, path=python_exe))
232 |             return Path(python_exe)
233 |         except (subprocess.TimeoutExpired, FileNotFoundError
| or):
234 |             pass
235 |
236 |         python_exe: Optional[str] = shutil.which("python")
237 |         if python_exe:
238 |             print(self.msg("step_1_found", version="3.x", path
| =python_exe))
239 |             return Path(python_exe)
240 |
241 |         print(self.msg("step_1_not_found"))
242 |         return False
243 |
244 |     def create_venv(self, python_path: Path) -> bool:
245 |         """Создание виртуального окружения.
246 |
247 |         Args:
248 |             python_path (Path): Путь к интерпретатору Python.
249 |
250 |         Returns:
251 |             bool: Успешность создания.
252 |
253 |         Examples:
254 |             >>> installer = Installer()
255 |             >>> success = installer.create_venv(Path("/usr/bin
| /python3"))
256 |             >>> print(success)
257 |         """
258 |         print(self.msg("step_2"))
259 |
260 |         if self.venv_dir.exists():
261 |             print(self.msg("step_2_exists"))
262 |             return True
263 |
264 |         try:

```



```

265         subprocess.run(
266             [str(python_path), "-m", "venv", str(self.venv
_dir)],
267             check=True,
268             timeout=60
269         )
270         print(self.msg("step_2_ok"))
271         return True
272     except (subprocess.CalledProcessError, subprocess.Time
outExpired):
273         return False
274
275     def upgrade_pip(self) -> bool:
276         """Обновление pip и инструментов.
277
278         Returns:
279             bool: Успешность обновления.
280
281         Examples:
282             >>> installer = Installer()
283             >>> success = installer.upgrade_pip()
284             >>> print(success)
285         """
286         print(self.msg("step_3"))
287
288         try:
289             subprocess.run(
290                 [str(self.python_path), "-m", "pip", "install"
, "--upgrade",
291                 "pip", "setuptools", "wheel"],
292                 check=True,
293                 timeout=120
294             )
295             print(self.msg("step_3_ok"))
296             return True
297         except (subprocess.CalledProcessError, subprocess.Time
outExpired):
298             return False

```

```

299 |
300 |     def install_dependencies(self, profile: str = "1") -> bool
301 |     :
302 |         """Установка зависимостей.
303 |
304 |         Args:
305 |             profile (str): Профиль установки (1-5).
306 |
307 |         Returns:
308 |             bool: Успешность установки.
309 |
310 |         Examples:
311 |             >>> installer = Installer()
312 |             >>> success = installer.install_dependencies("1")
313 |             >>> print(success)
314 |             """
315 |
316 |             print(self.msg("step_4"))
317 |
318 |             requirements_map: Dict[str, List[str]] = {
319 |                 "1": ["requirements.txt"],
320 |                 "2": ["install/req/requirements-core.txt"],
321 |                 "3": ["install/req/requirements-core.txt", "install/req/requirements-ai.txt"],
322 |                 "4": ["requirements.txt", "install/req/requirements-test.txt",
323 |                     "install/req/requirements-docs.txt"],
324 |                 "5": [],
325 |             }
326 |
327 |             req_files: List[str] = requirements_map.get(profile, [])
328 |
329 |             if not req_files:
330 |                 return True
331 |
332 |             try:
333 |                 cmd: List[str] = [str(self.python_path), "-m", "pip", "install"]

```

```

332         for req_file in req_files:
333             req_path: Path = self.install_dir / req_file
334             if req_path.exists():
335                 cmd.extend(["-r", str(req_path)])
336
337             subprocess.run(cmd, check=True, timeout=600)
338             print(self.msg("step_4_ok"))
339             return True
340         except (subprocess.CalledProcessError, subprocess.Time
outExpired):
341             return False
342
343     def verify_environment(self) -> bool:
344         """Финальная проверка окружения.
345
346         Returns:
347             bool: Успешность проверки.
348
349         Examples:
350             >>> installer = Installer()
351             >>> success = installer.verify_environment()
352             >>> print(success)
353         """
354         print(self.msg("step_6"))
355
356         modules: List[str] = ["fastapi", "uvicorn", "dotenv",
"pydantic"]
357
358         try:
359             for module in modules:
360                 subprocess.run(
361                     [str(self.python_path), "-c", f"import {mo
dule}"],
362                     check=True,
363                     timeout=10
364                 )
365             print(self.msg("step_6_ok"))
366             return True

```

```

367         except (subprocess.CalledProcessError, subprocess.Time
outExpired):
368             return False
369
370     def run(self, install_dir: Optional[str] = '') -> bool:
371         """Запуск процесса установки.
372
373         Args:
374             install_dir (Optional[str]): Директория установки.
375
376         Returns:
377             bool: Успешность установки.
378
379         Examples:
380             >>> installer = Installer(Language.RU)
381             >>> success = installer.run("/opt/ai-breadboard")
382             >>> print(success)
383         """
384         print(self.msg("welcome"))
385
386         self.install_dir = Path(install_dir) if install_dir el
se Path.cwd()
387         self.venv_dir = self.install_dir / "venv"
388
389         if self.platform_name == "Windows":
390             self.python_path = self.venv_dir / "Scripts" / "py
thon.exe"
391         else:
392             self.python_path = self.venv_dir / "bin" / "python
"
393
394         python_exe: Optional[Path] = self.find_python()
395         if not python_exe:
396             return False
397
398         if not self.create_venv(python_exe):
399             return False
400

```

```

401         if not self.upgrade_pip():
402             return False
403
404         print(self.msg("step_4_menu"))
405         print(self.msg("step_4_opt_1"))
406         print(self.msg("step_4_opt_2"))
407         print(self.msg("step_4_opt_3"))
408         print(self.msg("step_4_opt_4"))
409         print(self.msg("step_4_opt_5"))
410
411         profile: str = input("Choice [1]: ").strip() or "1"
412
413         if not self.install_dependencies(profile):
414             return False
415
416         if not self.verify_environment():
417             return False
418
419         print(self.msg("finish"))
420         return True
421
422
423 def select_language() -> Language:
424     """Выбор языка установки.
425
426     Returns:
427         Language: Выбранный язык.
428
429     Examples:
430         >>> lang = select_language()
431         >>> print(lang)
432     """
433     print("Select language / Выберите язык / Seleccione idioma
434 )": □ □ □ □ □ /
435     print("[1] English")
436     print("[2] Русский")
437     print("[3] Español")
438     print("[4] □ □ □ □ □ ")

```

```

438
439     choice: str = input("Choice [1]: ").strip() or "1"
440
441     language_map: Dict[str, Language] = {
442         "1": Language.EN,
443         "2": Language.RU,
444         "3": Language.ES,
445         "4": Language.HE,
446     }
447
448     return language_map.get(choice, Language.EN)
449
450
451 def main() -> int:
452     """Главная функция установщика.
453
454     Returns:
455         int: Код выхода (0 – успех, 1 – ошибка).
456
457     Examples:
458         >>> exit_code = main()
459         >>> sys.exit(exit_code)
460     """
461     try:
462         language: Language = select_language()
463         installer: Installer = Installer(language)
464
465         install_dir: str = input("Installation directory [current]: ").strip() or ""
466
467         success: bool = installer.run(install_dir)
468         return 0 if success else 1
469
470     except KeyboardInterrupt:
471         print("\nInstallation cancelled.")
472         return 1
473     except Exception as ex:
474         print(f"Installation failed: {ex}")

```

```
475 |         return 1
476 |
477 |
478 | if __name__ == "__main__":
479 |     sys.exit(main())
480 |
```

install.sh

```
1 #!/bin/bash
2 # -*- coding: utf-8 -*-
3 # =====
4 # Process Name: Кроссплатформенный установщик AI Breadboard для Linux/macOS
5 # =====
6 # Description:
7 #   Bash установщик для Unix-подобных систем с поддержкой мультиязычности
8 #   (RU/EN/ES/HE) и модульной архитектурой.
9 #
10 # Examples:
11 #   bash install.sh
12 #   bash install.sh --language en --install-dir /opt/ai-breadboard
13 #
14 # File: install.sh
15 # Project: AI Breadboard
16 # Package: Installation
17 # Module: Core
18 # Function: main
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # =====
22
23 set -euo pipefail
24
25 # Определение платформы
26 PLATFORM=$(uname -s)
27 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
28 INSTALL_DIR="${INSTALL_DIR:-.}"
29 VENV_DIR="${INSTALL_DIR}/venv"
30
```



```
31 # Цвета для вывода
32 RED='\033[0;31m'
33 GREEN='\033[0;32m'
34 YELLOW='\033[1;33m'
35 BLUE='\033[0;34m'
36 NC='\033[0m' # No Color
37
38 # Язык по умолчанию
39 LANGUAGE="${LANGUAGE:-en}"
40
41 # Словарь сообщений
42 declare -A MESSAGES_EN=(
43     [welcome]=" Welcome to AI Breadboard Installer"
44     [select_lang]="Select installation language:"
45     [lang_selected]=" Selected language: %s"
46     [step_1]="[1/6] Checking Python interpreter..."
47     [step_1_found]=" Found Python %s: %s"
48     [step_1_not_found]=" Python not found. Install Python 3.1
0+"
49     [step_2]="[2/6] Creating virtual environment..."
50     [step_2_ok]=" Virtual environment created"
51     [step_2_exists]=" Virtual environment already exists"
52     [step_3]="[3/6] Upgrading pip and tools..."
53     [step_3_ok]=" pip upgraded"
54     [step_4]="[4/6] Installing dependencies..."
55     [step_4_menu]="Select installation profile:"
56     [step_4_opt_1]="[1] Full installation (recommended)"
57     [step_4_opt_2]="[2] Core only"
58     [step_4_opt_3]="[3] Core + AI"
59     [step_4_opt_4]="[4] Full + Dev"
60     [step_4_opt_5]="[5] Skip"
61     [step_4_ok]=" Dependencies installed"
62     [step_5]="[5/6] Checking SSL certificates..."
63     [step_5_ok]=" SSL certificates found"
64     [step_6]="[6/6] Final verification..."
65     [step_6_ok]=" Environment ready"
66     [finish]=" Installation completed successfully!"
67     [error]=" Error: %s"
```

```

68 )
69
70 declare -A MESSAGES_RU=(
71     [welcome]="👋 Добро пожаловать в установщик AI Breadboard"
72     [select_lang]="Выберите язык установки:"
73     [lang_selected]="👉 Выбран язык: %s"
74     [step_1]="[1/6] Проверка Python интерпретатора..."
75     [step_1_found]="👉 Найден Python %s: %s"
76     [step_1_not_found]="👉 Python не найден. Установите Python
3.10+"
77     [step_2]="[2/6] Создание виртуального окружения..."
78     [step_2_ok]="👉 Виртуальное окружение создано"
79     [step_2_exists]="👉 Виртуальное окружение уже существует"
80     [step_3]="[3/6] Обновление pip и инструментов..."
81     [step_3_ok]="👉 pip обновлен"
82     [step_4]="[4/6] Установка зависимостей..."
83     [step_4_menu]="Выберите профиль установки:"
84     [step_4_opt_1]="[1] Полная установка (рекомендуется)"
85     [step_4_opt_2]="[2] Только Core"
86     [step_4_opt_3]="[3] Core + AI"
87     [step_4_opt_4]="[4] Полная + Dev"
88     [step_4_opt_5]="[5] Пропустить"
89     [step_4_ok]="👉 Зависимости установлены"
90     [step_5]="[5/6] Проверка SSL сертификатов..."
91     [step_5_ok]="👉 SSL сертификаты найдены"
92     [step_6]="[6/6] Финальная проверка..."
93     [step_6_ok]="👉 Окружение готово к работе"
94     [finish]="👉 Установка завершена успешно!"
95     [error]="👉 Ошибка: %s"
96 )
97
98 declare -A MESSAGES_ES=(
99     [welcome]="👋 Bienvenido al instalador de AI Breadboard"
100     [select_lang]="Seleccione idioma de instalación:"
101     [lang_selected]="👉 Idioma seleccionado: %s"
102     [step_1]="[1/6] Verificando intérprete Python..."
103     [step_1_found]="👉 Python %s encontrado: %s"
104     [step_1_not_found]="👉 Python no encontrado. Instale Python

```

```

3.10+"
105     [step_2]="[2/6] Creando entorno virtual..."
106     [step_2_ok]="✅ Entorno virtual creado"
107     [step_2_exists]="✅ Entorno virtual ya existe"
108     [step_3]="[3/6] Actualizando pip..."
109     [step_3_ok]="✅ pip actualizado"
110     [step_4]="[4/6] Instalando dependencias..."
111     [step_4_menu]="Seleccione perfil de instalación:"
112     [step_4_opt_1]="[1] Instalación completa (recomendado)"
113     [step_4_opt_2]="[2] Solo Core"
114     [step_4_opt_3]="[3] Core + AI"
115     [step_4_opt_4]="[4] Completo + Dev"
116     [step_4_opt_5]="[5] Omitir"
117     [step_4_ok]="✅ Dependencias instaladas"
118     [step_5]="[5/6] Verificando certificados SSL..."
119     [step_5_ok]="✅ Certificados SSL encontrados"
120     [step_6]="[6/6] Verificación final..."
121     [step_6_ok]="✅ Entorno listo"
122     [finish]="✅ ¡Instalación completada exitosamente!"
123     [error]="❌ Error: %s"
124 )
125
126 # Функция получения сообщения
127 msg() {
128     local key="$1"
129     local lang_var="MESSAGES_${LANGUAGE^^}"
130     local -n messages="${lang_var}"
131
132     if [[ -v messages[$key] ]]; then
133         printf "${messages[$key]}" "${@:2}"
134     else
135         echo "$key"
136     fi
137 }
138
139 # Функция выбора языка
140 select_language() {
141     echo -e "${BLUE}Select language / Выберите язык / Seleccio

```

```

ne idioma:${NC}"
142     echo "[1] English"
143     echo "[2] Русский"
144     echo "[3] Español"
145     echo ""
146     read -p "Choice [1]: " choice
147     choice=${choice:-1}
148
149     case "$choice" in
150         1) LANGUAGE="en" ;;
151         2) LANGUAGE="ru" ;;
152         3) LANGUAGE="es" ;;
153         *) LANGUAGE="en" ;;
154     esac
155
156     echo -e "${GREEN}$(msg lang_selected "$LANGUAGE")${NC}"
157 }
158
159 # Функция поиска Python
160 find_python() {
161     echo -e "${BLUE}$(msg step_1)${NC}"
162
163     local versions=("3.13" "3.12" "3.11" "3.10")
164
165     for version in "${versions[@]}; do
166         if command -v "python${version}" &> /dev/null; then
167             local python_path=$(command -v "python${version}")
168             echo -e "${GREEN}$(msg step_1_found "$version" "$python_path")${NC}"
169             echo "$python_path"
170             return 0
171         fi
172     done
173
174     if command -v python3 &> /dev/null; then
175         local python_path=$(command -v python3)
176         echo -e "${GREEN}$(msg step_1_found "3.x" "$python_path")${NC}"

```

```
177 |         echo "$python_path"
178 |         return 0
179 |     fi
180 |
181 |     echo -e "${RED}$(msg step_1_not_found){NC}"
182 |     return 1
183 | }
184 |
185 | # Функция создания venv
186 | create_venv() {
187 |     local python_path="$1"
188 |
189 |     echo -e "${BLUE}$(msg step_2){NC}"
190 |
191 |     if [[ -d "$VENV_DIR" ]]; then
192 |         echo -e "${GREEN}$(msg step_2_exists){NC}"
193 |         return 0
194 |     fi
195 |
196 |     if "$python_path" -m venv "$VENV_DIR"; then
197 |         echo -e "${GREEN}$(msg step_2_ok){NC}"
198 |         return 0
199 |     else
200 |         return 1
201 |     fi
202 | }
203 |
204 | # Функция обновления pip
205 | upgrade_pip() {
206 |     echo -e "${BLUE}$(msg step_3){NC}"
207 |
208 |     local python_path="$VENV_DIR/bin/python"
209 |
210 |     if "$python_path" -m pip install --upgrade pip setuptools
wheel > /dev/null 2>&1; then
211 |         echo -e "${GREEN}$(msg step_3_ok){NC}"
212 |         return 0
213 |     else
```

```

214 |         return 1
215 |     fi
216 | }
217 |
218 | # Функция установки зависимостей
219 | install_dependencies() {
220 |     local profile="${1:-1}"
221 |
222 |     echo -e "${BLUE}$(msg step_4){NC}"
223 |
224 |     local python_path="$VENV_DIR/bin/python"
225 |     local req_files=()
226 |
227 |     case "$profile" in
228 |         1) req_files=("requirements.txt") ;;
229 |         2) req_files=("install/req/requirements-core.txt") ;;
230 |         3) req_files=("install/req/requirements-core.txt" "ins
231 | tall/req/requirements-ai.txt") ;;
232 |         4) req_files=("requirements.txt" "install/req/requirem
233 | ents-test.txt" "install/req/requirements-docs.txt") ;;
234 |         5) return 0 ;;
235 |         *) req_files=("requirements.txt") ;;
236 |     esac
237 |
238 |     local cmd=("$python_path" "-m" "pip" "install")
239 |
240 |     for req_file in "${req_files[@]}; do
241 |         local req_path="$INSTALL_DIR/$req_file"
242 |         if [[ -f "$req_path" ]]; then
243 |             cmd+=("-r" "$req_path")
244 |         fi
245 |     done
246 |
247 |     if "${cmd[@]}" > /dev/null 2>&1; then
248 |         echo -e "${GREEN}$(msg step_4_ok){NC}"
249 |         return 0
250 |     else
251 |         return 1
252 |     fi
253 | }

```

```
250     fi
251 }
252
253 # Функция проверки окружения
254 verify_environment() {
255     echo -e "${BLUE}$(msg step_6){NC}"
256
257     local python_path="$VENV_DIR/bin/python"
258     local modules=("fastapi" "uvicorn" "dotenv" "pydantic")
259
260     for module in "${modules[@]}; do
261         if ! "$python_path" -c "import $module" 2>/dev/null; t
hen
262             return 1
263         fi
264     done
265
266     echo -e "${GREEN}$(msg step_6_ok){NC}"
267     return 0
268 }
269
270 # Главная функция
271 main() {
272     echo -e "${BLUE}$(msg welcome){NC}"
273     echo ""
274
275     select_language
276     echo ""
277
278     read -p "Installation directory [current]: " input_dir
279     INSTALL_DIR="${input_dir:-.}"
280     VENV_DIR="$INSTALL_DIR/venv"
281
282     local python_path
283     python_path=$(find_python) || return 1
284     echo ""
285
286     create_venv "$python_path" || return 1
```

```
287 |     echo ""
288 |
289 |     upgrade_pip || return 1
290 |     echo ""
291 |
292 |     echo -e "${BLUE}$(msg step_4_menu){NC}"
293 |     echo "$(msg step_4_opt_1)"
294 |     echo "$(msg step_4_opt_2)"
295 |     echo "$(msg step_4_opt_3)"
296 |     echo "$(msg step_4_opt_4)"
297 |     echo "$(msg step_4_opt_5)"
298 |     echo ""
299 |
300 |     read -p "Choice [1]: " profile
301 |     profile=${profile:-1}
302 |     echo ""
303 |
304 |     install_dependencies "$profile" || return 1
305 |     echo ""
306 |
307 |     verify_environment || return 1
308 |     echo ""
309 |
310 |     echo -e "${GREEN}$(msg finish){NC}"
311 |     return 0
312 | }
313 |
314 | # Обработка аргументов командной строки
315 | while [[ $# -gt 0 ]]; do
316 |     case $1 in
317 |         --language)
318 |             LANGUAGE="$2"
319 |             shift 2
320 |             ;;
321 |         --install-dir)
322 |             INSTALL_DIR="$2"
323 |             shift 2
324 |             ;;
```



```
325 |         *)
326 |             shift
327 |             ;;
328 |     esac
329 | done
330 |
331 | # Запуск установщика
332 | if main; then
333 |     exit 0
334 | else
335 |     echo -e "${RED}$(msg error "Installation failed")${NC}"
336 |     exit 1
337 | fi
338 |
```

validate_installers.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Validation and testing of AI Breadboard installers
4 # =====
5 # Description:
6 #   Utility for installer logic checking, configuration validation
7 #   и тестирования кроссплатформенной совместимости.
8 #
9 # Examples:
10 #   python validate_installers.py
11 #   python validate_installers.py --check-python
12 #   python validate_installers.py --check-bash
13 #
14 # File: validate_installers.py
15 # Project: AI Breadboard
16 # Package: Installation
17 # Module: Validation
18 # Class: InstallerValidator
19 # Function: validate_all
20 # Author: hypo69
21 # Copyright: © 2026 hypo69
22 # =====
23
24 import json
25 import re
26 import sys
27 from pathlib import Path
28 from typing import Dict, List, Tuple, Optional
29
30
31 class InstallerValidator:
```

```

32 | """Валидатор логики установщиков."""
33 |
34 | def __init__(self, install_dir: Path):
35 |     """Initialization валидатора.
36 |
37 |     Args:
38 |         install_dir (Path): Директория install/.
39 |     """
40 |     self.install_dir: Path = install_dir
41 |     self.errors: List[str] = []
42 |     self.warnings: List[str] = []
43 |
44 | def validate_python_installer(self) -> bool:
45 |     """Проверка логики Python установщика.
46 |
47 |     Returns:
48 |         bool: Успешность проверки.
49 |
50 |     Examples:
51 |         >>> validator = InstallerValidator(Path("install")
52 |         )
53 |         >>> success = validator.validate_python_installer(
54 |         )
55 |         >>> print(success)
56 |     """
57 |     print("[1/3] Validating Python installer...")
58 |
59 |     python_file: Path = self.install_dir / "install.py"
60 |
61 |     if not python_file.exists():
62 |         self.errors.append("install.py not found")
63 |         return False
64 |
65 |     content: str = python_file.read_text(encoding="utf-8")
66 |
67 |     # Проверка наличия класса Installer
68 |     if "class Installer:" not in content:
69 |         self.errors.append("Installer class not found in i

```

```

install.py")
68         return False
69
70     # Проверка наличия методов
71     required_methods: List[str] = [
72         "find_python",
73         "create_venv",
74         "upgrade_pip",
75         "install_dependencies",
76         "verify_environment",
77         "run"
78     ]
79
80     for method in required_methods:
81         if f"def {method}(" not in content:
82             self.errors.append(f"Method {method} not found
in Installer class")
83         return False
84
85     # Проверка мультиязычности
86     languages: List[str] = ["ru", "en", "es", "he"]
87     for lang in languages:
88         if f'"{lang}":' not in content:
89             self.warnings.append(f"Language {lang} not fou
nd in I18N")
90
91     # Проверка отсутствия None
92     if " = None" in content or ": None" in content:
93         self.warnings.append("Found 'None' assignments in
install.py (violates CODE_RULES)")
94
95     print(" Python installer validation passed")
96     return True
97
98     def validate_bash_installer(self) -> bool:
99         """Проверка логики Bash установщика.
100
101         Returns:

```

```

102         bool: Успешность проверки.
103
104     Examples:
105         >>> validator = InstallerValidator(Path("install")
106     )
107         >>> success = validator.validate_bash_installer()
108         >>> print(success)
109     """
110     print("[2/3] Validating Bash installer...")
111
112     bash_file: Path = self.install_dir / "install.sh"
113
114     if not bash_file.exists():
115         self.errors.append("install.sh not found")
116         return False
117
118     content: str = bash_file.read_text(encoding="utf-8")
119
120     # Проверка shebang
121     if not content.startswith("#!/bin/bash"):
122         self.errors.append("Bash installer missing shebang")
123
124     return False
125
126     # Проверка наличия функций
127     required_functions: List[str] = [
128         "select_language",
129         "find_python",
130         "create_venv",
131         "upgrade_pip",
132         "install_dependencies",
133         "verify_environment",
134         "main"
135     ]
136
137     for func in required_functions:
138         if f"{func}()" not in content:
139             self.errors.append(f"Function {func} not found")

```

```

    in install.sh")
138         return False
139
140     # Проверка мультязычности
141     languages: List[str] = ["MESSAGES_RU", "MESSAGES_EN",
    "MESSAGES_ES"]
142     for lang in languages:
143         if f"declare -A {lang}" not in content:
144             self.warnings.append(f"Language {lang} not found in install.sh")
145
146     print("  Bash installer validation passed")
147     return True
148
149     def validate_config_json(self) -> bool:
150         """Проверка конфигурации install.json.
151
152         Returns:
153             bool: Успешность проверки.
154
155         Examples:
156             >>> validator = InstallerValidator(Path("install")
    )
157             >>> success = validator.validate_config_json()
158             >>> print(success)
159         """
160         print("[3/3] Validating install.json...")
161
162         config_file: Path = self.install_dir / "install.json"
163
164         if not config_file.exists():
165             self.errors.append("install.json not found")
166             return False
167
168         try:
169             config: Dict = json.loads(config_file.read_text(encoding="utf-8"))
170         except json.JSONDecodeError as ex:

```

```
171         self.errors.append(f"Invalid JSON in install.json:
    {ex}")
172         return False
173
174     # Проверка обязательных полей
175     required_fields: List[str] = [
176         "defaults",
177         "paths",
178         "env_vars",
179         "verify",
180         "supported_languages"
181     ]
182
183     for field in required_fields:
184         if field not in config:
185             self.errors.append(f"Missing required field in
    config: {field}")
186             return False
187
188     # Проверка поддерживаемых языков
189     supported_langs: List[str] = config.get("supported_languages", [])
190     expected_langs: List[str] = ["ru", "en", "es", "he"]
191
192     for lang in expected_langs:
193         if lang not in supported_langs:
194             self.warnings.append(f"Language {lang} not in
    supported_languages")
195
196     # Проверка Python версий
197     python_versions: List[str] = config.get("defaults", {}
    ).get(
198         "python_preferred_versions", []
199     )
200     if not python_versions:
201         self.warnings.append("No Python versions specified
    in config")
202
```

```

203 |         print(" Configuration validation passed")
204 |         return True
205 |
206 |     def validate_consistency(self) -> bool:
207 |         """Проверка консистентности между установщиками.
208 |
209 |         Returns:
210 |             bool: Успешность проверки.
211 |
212 |         Examples:
213 |             >>> validator = InstallerValidator(Path("install")
214 | )
215 |             >>> success = validator.validate_consistency()
216 |             >>> print(success)
217 |         """
218 |         print("\nValidating consistency between installers..."
219 | )
220 |
221 |         python_file: Path = self.install_dir / "install.py"
222 |         bash_file: Path = self.install_dir / "install.sh"
223 |
224 |         if not python_file.exists() or not bash_file.exists():
225 |             return False
226 |
227 |         python_content: str = python_file.read_text(encoding="
228 | utf-8")
229 |         bash_content: str = bash_file.read_text(encoding="utf-
230 | 8")
231 |
232 |         # Проверка одинакового количества шагов
233 |         python_steps: int = len(re.findall(r"step_\d+", python
234 | _content))
235 |         bash_steps: int = len(re.findall(r"step_\d+", bash_con
236 | tent))
237 |
238 |         if python_steps != bash_steps:
239 |             self.warnings.append(
240 |                 f"Step count mismatch: Python={python_steps},

```



```

Bash={bash_steps}"
235         )
236
237         # Проверка одинаковых профилей установки
238         python_profiles: List[str] = re.findall(r'"([1-5])":',
python_content)
239         bash_profiles: List[str] = re.findall(r'case "\$profil
e" in\s+(\d)', bash_content)
240
241         if set(python_profiles) != set(bash_profiles):
242             self.warnings.append("Installation profiles mismat
ch between installers")
243
244         print("□ Consistency validation passed")
245         return True
246
247     def validate_all(self) -> Tuple[bool, List[str], List[str]
]:
248         """Полная валидация всех установщиков.
249
250         Returns:
251             Tuple[bool, List[str], List[str]]: (успех, ошибки,
предупреждения).
252
253         Examples:
254             >>> validator = InstallerValidator(Path("install")
)
255             >>> success, errors, warnings = validator.validate
_all()
256             >>> print(f"Success: {success}, Errors: {len(error
s)}, Warnings: {len(warnings)}")
257             """
258             print("=" * 60)
259             print("AI Breadboard Installer Validation")
260             print("=" * 60)
261             print()
262
263             results: List[bool] = [

```

```

264         self.validate_python_installer(),
265         self.validate_bash_installer(),
266         self.validate_config_json(),
267         self.validate_consistency()
268     ]
269
270     print()
271     print("=" * 60)
272
273     if self.errors:
274         print(f"\n❑ ERRORS ({len(self.errors)}):")
275         for error in self.errors:
276             print(f"    - {error}")
277
278     if self.warnings:
279         print(f"\n❑ WARNINGS ({len(self.warnings)}):")
280         for warning in self.warnings:
281             print(f"    - {warning}")
282
283     success: bool = all(results) and not self.errors
284
285     if success:
286         print("\n❑ All validations passed!")
287     else:
288         print("\n❑ Validation failed!")
289
290     print("=" * 60)
291
292     return success, self.errors, self.warnings
293
294
295 def main() -> int:
296     """Главная функция валидатора.
297
298     Returns:
299         int: Код выхода (0 – успех, 1 – ошибка).
300
301     Examples:

```

```
302 |         >>> exit_code = main()
303 |         >>> sys.exit(exit_code)
304 |     """
305 |     install_dir: Path = Path(__file__).parent
306 |
307 |     validator: InstallerValidator = InstallerValidator(install
| _dir)
308 |     success, errors, warnings = validator.validate_all()
309 |
310 |     return 0 if success else 1
311 |
312 |
313 | if __name__ == "__main__":
314 |     sys.exit(main())
315 |
```

install.sh

```
1 #!/bin/bash
2 #
3 # Кроссплатформенный установщик AI Breadboard
4 # Работает на Linux, macOS и Windows (WSL/Git Bash)
5 #
6 # Использование:
7 #   ./install.sh
8 #   ./install.sh --lang ru
9 #   ./install.sh --install-dir /opt/ai-breadboard
10 #   ./install.sh --skip-models
11 #
12
13 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
14 PROJECT_ROOT="$SCRIPT_DIR"
15
16 # Установить переменные окружения
17 export AIBREADBOARD_DIR="$PROJECT_ROOT"
18 export ASSIST_DIR="$PROJECT_ROOT"
19 export PYTHONUTF8=1
20 export PYTHONPATH="${PROJECT_ROOT}:${PYTHONPATH}"
21
22 # Найти Python интерпретатор
23 PYTHON=""
24
25 if [ -f "$PROJECT_ROOT/venv/bin/python" ]; then
26     PYTHON="$PROJECT_ROOT/venv/bin/python"
27 elif command -v python3 &>/dev/null; then
28     PYTHON="python3"
29 elif command -v python &>/dev/null; then
30     PYTHON="python"
31 else
32     echo "❌ Python не найден. Пожалуйста, установите Python 3.
33     10+"
34     exit 1
35 fi
```

```
36 | # Выполнить установщик
37 | "$PYTHON" "$PROJECT_ROOT/scripts/cli/installer.py" "$@"
38 |
```

tts_handler.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Adaptive TTS pipeline integration module for Telegram bot
4 # =====
5 # Description:
6 #   Module for integrating adaptive TTS (text-to-speech) pipeline into Telegram bot.
7 #   Generates voice messages from text with adaptive chunking and streaming delivery.
8 #   Uses python-telegram-bot or compatible framework.
9 #
10 # File: tg_tts_handler.py
11 # Project: ai-breadboard
12 # Package: integrations.telegram
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 """
18 Module for integrating adaptive TTS pipeline into Telegram bot
19 .
20 Uses python-telegram-bot or any compatible framework.
21 Implements streaming audio generation and instant delivery to users.
22 """
23 import io
24 import httpx
25 import asyncio
26 from typing import AsyncGenerator
27 from src.logger import logger
28 from src.ai.voice import generate_voiceover_chunks
```

```

29 # URL of local FastAPI instance
30 API_BASE_URL = "http://127.0.0.1:8000"
31
32 async def handle_telegram_voiceover_request(update, context, media_id: int, field: str = "plot"):
33     """
34     Handler for voice narration request in Telegram.
35     Implements pipeline:
36     1. Generate adapted text chunks using Gemini.
37     2. Send status message "Preparing voice narration...".
38     3. For each ready chunk: request TTS, download mp3, instantly send as voice message.
39     """
40     query = update.callback_query if update.callback_query else None
41     chat_id = update.effective_chat.id
42
43     # 1. Send greeting message
44     status_message = await context.bot.send_message(
45         chat_id=chat_id,
46         text="🔊 *Starting voice narration preparation...* Text is being adapted for narrator.",
47         parse_mode="Markdown"
48     )
49
50     # Get original text from database
51     # In real bot code you can import MediaDatabase directly:
52     # from plugins.media_organizer.core.database import MediaDatabase
53     # db = MediaDatabase(DB_FILE)
54     # raw_text = db.get_media_field(media_id, field)
55     # For example we'll use a stub simulating text retrieval:
56     raw_text = "Example text from database. 1. Plug in the device. 2. Press start button."
57
58     try:
59         idx = 1
60         async for chunk in generate_voiceover_chunks(raw_text)

```

```

:
61         # Notify user about current part preparation
62         await context.bot.edit_message_text(
63             chat_id=chat_id,
64             message_id=status_message.message_id,
65             text=f"▯ *Synthesizing part {idx}...\n\n_{chunk}_",
66             parse_mode="Markdown"
67         )
68
69         # Send request to synthesis API (or call edge-tts
directly)
70         async with httpx.AsyncClient() as client:
71             response = await client.get(
72                 f"{API_BASE_URL}/api/tts/synthesize",
73                 params={"text": chunk},
74                 timeout=30.0
75             )
76
77             if response.status_code == 200:
78                 # Send audio file as voice message in Tele
gram
79                 audio_data = io.BytesIO(response.content)
80                 audio_data.name = f"voiceover_part_{idx}.o
gg" # Telegram accepts ogg/mp3
81
82                 await context.bot.send_voice(
83                     chat_id=chat_id,
84                     voice=audio_data,
85                     caption=f"Part {idx}",
86                     title="Narrator"
87                 )
88             else:
89                 logger.error(f"TTS API returned error code
{response.status_code}")
90                 await context.bot.send_message(chat_id=cha
t_id, text=f"▯ Error synthesizing part {idx}")
91

```



```

92 |         idx += 1
93 |         await asyncio.sleep(0.5) # Small pause between se
    | ending parts
94 |
95 |         await context.bot.edit_message_text(
96 |             chat_id=chat_id,
97 |             message_id=status_message.message_id,
98 |             text="🔊 *All voice narration ready and sent!*",
99 |             parse_mode="Markdown"
100 |         )
101 |
102 |     except Exception as e:
103 |         logger.error(f"Error in TG voiceover handler: {e}")
104 |         await context.bot.send_message(
105 |             chat_id=chat_id,
106 |             text=f"🔊 Error occurred during voice narration: {s
    | tr(e)}"
107 |         )
108 |

```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 from .tts_handler import handle_telegram_voiceover_request
3
4 __all__ = ["handle_telegram_voiceover_request"]
5
```

__init__.py

```
1 # -*- coding: utf-8 -*-  
2
```

run.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Cross-platform FastAPI server launcher with se
  tup
4 # =====
5 # Description:
6 #   Main launcher for starting FastAPI server with interactive
  setup options.
7 #   Ported from legacy run.ps1 with support for Windows, Linux
  and macOS.
8 #   Provides virtual environment detection, dependency checkin
  g, port management,
9 #   and configuration setup with both interactive and non-inte
  ractive modes.
10 #
11 #   Usage:
12 #       python launchers/run.py
13 #       python launchers/run.py --host 0.0.0.0 --port 8000
14 #       python launchers/run.py --non-interactive
15 #       python launchers/run.py --help
16 #
17 # File: run.py
18 # Project: ai-breadboard
19 # Package: root
20 # Author: hypo69
21 # Copyright: © 2026 hypo69
22 # =====
23
24 import argparse
25 import json
26 import os
27 import sys
28 import subprocess
```

```
29 from pathlib import Path
30 from typing import Optional, Tuple
31
32 # Adding root to PYTHONPATH
33 sys.path.insert(0, str(Path(__file__).parent.parent))
34
35 from scripts.cli.paths import init_paths, get_paths
36 from scripts.cli.config import get_config_manager
37 from scripts.cli.utils import (
38     find_available_port,
39     get_process_on_port,
40     kill_process,
41     run_command
42 )
43
44 class ServerLauncher:
45     """Launcher for starting FastAPI server"""
46
47     def __init__(self):
48         self.paths = init_paths()
49         self.config_mgr = get_config_manager()
50         self.config = self.config_mgr.load_config()
51         self.server_cfg = self.config.get("server", {})
52         self.ai_cfg = self.config.get("ai", {})
53
54     def get_default_host(self) -> str:
55         """Getting default host from config.json"""
56         return str(self.server_cfg.get("host", "0.0.0.0"))
57
58     def get_default_port(self) -> int:
59         """Getting default port from config.json"""
60         return int(self.server_cfg.get("port", 8000))
61
62     def get_local_ips(self) -> list:
63         """Getting local IP addresses for hints"""
64         try:
65             import socket
66             hostname = socket.gethostname()
```

```

67         local_ip = socket.gethostbyname(hostname)
68         return [local_ip] if local_ip and local_ip != "127
|.0.0.1" else []
69     except Exception:
70         return []
71
72     def check_venv(self) -> str:
73         """Checking virtual environment"""
74         if self.paths.venv_python.exists():
75             print(f"    [OK] Virtual environment found")
76             print(f"    Python: {self.paths.venv_python}")
77             return str(self.paths.venv_python)
78         else:
79             print(f"    [WARN] Virtual environment not found")
80             # Using current Python
81             python_exe = sys.executable
82             print(f"    Using system Python: {python_exe}")
83             return python_exe
84
85     def check_dependencies(self, python_exe: str) -> bool:
86         """Checking dependencies"""
87         print("    [*] Checking dependencies...", end=" ", flu
|sh=True)
88         try:
89             result = subprocess.run(
90                 [python_exe, "-c", "import fastapi, uvicorn, d
|otenv, jwt; print('OK')"],
91                 capture_output=True,
92                 text=True,
93                 timeout=5
94             )
95             if result.returncode == 0:
96                 print("[OK]")
97                 return True
98             else:
99                 print("[WARN] Некоторые зависимости не установ
|лены")
100                 print("    Установите: pip install -r requirem

```

```

ents.txt")
101         return False
102     except Exception as e:
103         print(f"[ERROR] {e}")
104         return False
105
106     def load_env_vars(self) -> dict:
107         """Загрузить переменные окружения из .env"""
108         env_vars = {}
109         env_file = self.paths.env_file
110
111         if env_file.exists():
112             try:
113                 with open(env_file, "r", encoding="utf-8") as
114 f:
115                     for line in f:
116                         line = line.strip()
117                         if line and not line.startswith("#") a
118 nd "=" in line:
119                             key, value = line.split("=", 1)
120                             env_vars[key.strip()] = value.stri
121 p().strip('\"').strip('\"')
122             except Exception as e:
123                 print(f"    [WARN] Error чтения .env: {e}")
124
125         return env_vars
126
127     def check_port(self, port: int) -> bool:
128         """Проверить и освободить порт если нужно"""
129         print(f"    [*] Check порта {port}...", end=" ", flush
130 =True)
131
132         proc_info = get_process_on_port(port)
133         if proc_info:
134             pid, proc_name = proc_info
135             print(f"\n    [WARN] Порт занят процессом {proc_na
136 me} (PID: {pid})")
137             print(f"    Shutdown процесса...", end=" ", flush=

```

```

True)
133
134         if kill_process(pid, force=False):
135             print("[OK]")
136             return True
137         else:
138             print("[FAILED]")
139             # Попробовать force
140             if kill_process(pid, force=True):
141                 print("    [FORCE] Процесс принудительно з
авершен")
142                 return True
143             else:
144                 print("    [ERROR] Не удалось освободить п
орт")
145                 return False
146     else:
147         print("[FREE]")
148         return True
149
150     def interactive_mode(self, default_host: str, default_port
: int) -> Tuple[str, int]:
151         """Интерактивный выбор параметров"""
152         print()
153         print("-" * 70)
154         print("  ИНТЕРАКТИВНЫЙ ВЫБОР АДРЕСА И ПОРТА")
155         print("-" * 70)
156         print("Выберите сетевой интерфейс для запуска сервера:
")
157         print("  [1] 0.0.0.0    - Все сетевые интерфейсы (досту
пен с других ПК)")
158
159         local_ips = self.get_local_ips()
160         if local_ips:
161             print(f"                                (Ваш IP в локальной сети
: {local_ips[0]})")
162
163         print("  [2] 127.0.0.1 - Только локально (localhost)")

```



```

164 |         print(" [3] Ввести произвольный IP вручную")
165 |         print(f" [Enter] По умолчанию: {default_host}")
166 |         print()
167 |
168 |         choice = input("Адрес / Вариант: ").strip()
169 |
170 |         if choice == "1":
171 |             host = "0.0.0.0"
172 |         elif choice == "2":
173 |             host = "127.0.0.1"
174 |         elif choice == "3":
175 |             custom = input(" Введите IP-адрес или хост: ").st
rip()
176 |             host = custom if custom else default_host
177 |         elif choice:
178 |             host = choice
179 |         else:
180 |             host = default_host
181 |
182 |         # Выбор порта
183 |         port_str = input(f"Порт сервера [Enter = {default_port
}]: ").strip()
184 |         if port_str:
185 |             try:
186 |                 port = int(port_str)
187 |             except ValueError:
188 |                 port = default_port
189 |         else:
190 |             port = default_port
191 |
192 |         return host, port
193 |
194 |     def run_uvicorn(self, python_exe: str, host: str, port: in
t, ssl: bool = False) -> int:
195 |         """Запустить uvicorn сервер"""
196 |         print()
197 |         print("=" * 70)
198 |         print(" □ ЗАПУСК FastAPI СЕРВЕРА")

```

```
199 |     print("=" * 70)
200 |     print()
201 |
202 |     use_reload = bool(self.server_cfg.get("reload", True))
203 |     debug = bool(self.server_cfg.get("debug", True))
204 |
205 |     # Построить команду
206 |     cmd = [
207 |         python_exe,
208 |         "-m", "uvicorn",
209 |         "main:app",
210 |         "--host", host,
211 |         "--port", str(port),
212 |     ]
213 |
214 |     if use_reload:
215 |         cmd.append("--reload")
216 |
217 |     if debug:
218 |         cmd.extend(["--log-level", "debug"])
219 |     else:
220 |         cmd.extend(["--log-level", "info"])
221 |
222 |     if ssl:
223 |         certs_dir = self.paths.certs_dir
224 |         cert_file = certs_dir / "localhost+2.pem"
225 |         key_file = certs_dir / "localhost+2-key.pem"
226 |
227 |         if cert_file.exists() and key_file.exists():
228 |             cmd.extend([
229 |                 "--ssl-keyfile", str(key_file),
230 |                 "--ssl-certfile", str(cert_file),
231 |             ])
232 |             print(f"    SSL: ВКЛЮЧЕН")
233 |             print(f"        Сертификат: {cert_file}")
234 |             print(f"        Ключ: {key_file}")
235 |         else:
236 |             print(f"    SSL: ОТКЛЮЧЕН (сертификаты не найден")
```

```

    ы) ")
237
238     print(f"  Хост: {host}")
239     print(f"  Порт: {port}")
240     print(f"  АвтоLoading: {'ВКЛ' if use_reload else 'ВЫКЛ'
    '}")
241     print(f"  Отладка: {'ВКЛ' if debug else 'ВЫКЛ'}")
242     print()
243     print("  Нажмите Ctrl+C для остановки сервера")
244     print()
245     print("=" * 70)
246     print()
247
248     # Запустить сервер
249     try:
250         result = subprocess.run(
251             cmd,
252             cwd=str(self.paths.project_root),
253             check=False
254         )
255         return result.returncode
256     except KeyboardInterrupt:
257         print()
258         print()
259         print("=" * 70)
260         print("  ▢ Сервер остановлен пользователем")
261         print("=" * 70)
262         return 0
263     except Exception as e:
264         print(f"ERROR: {e}")
265         return 1
266
267     def run(self, host: Optional[str] = None, port: Optional[int] = None, non_interactive: bool = False) -> int:
268         """Главная function запуска"""
269         print()
270         print("▬" + "=" * 68 + "▬")
271         print("▮" + " ЗАПУСК FastAPI СЕРВЕРА - ИНТЕРАКТИВНЫЙ Л

```

```

    ОНЧЕР".center(68) + "||")
272     print("┌" + "=" * 68 + "┐")
273     print()
274
275     # Check виртуального окружения
276     print("[1/5] Check виртуального окружения...")
277     python_exe = self.check_venv()
278     print()
279
280     # Check зависимостей
281     print("[2/5] Check зависимостей...")
282     if not self.check_dependencies(python_exe):
283         print("    [!] Некоторые зависимости могут быть не
установлены")
284     print()
285
286     # Loading конфигурации
287     print("[3/5] Loading конфигурации...")
288     env_vars = self.load_env_vars()
289     use_ssl = bool(self.server_cfg.get("use_ssl", True))
290     if "USE_SSL" in env_vars:
291         use_ssl = env_vars["USE_SSL"].lower() in ("true",
"1", "yes")
292     print(f"    [OK] Configuration загружена (SSL: {'ВКЛ'
if use_ssl else 'ВЫКЛ'})")
293     print()
294
295     # Определение параметров
296     default_host = self.get_default_host()
297     default_port = self.get_default_port()
298
299     if host is None:
300         host = default_host
301     if port is None:
302         port = default_port
303
304     # Интерактивный выбор
305     print("[4/5] Выбор параметров...")

```

```

306         if not non_interactive and host == default_host:
307             host, port = self.interactive_mode(default_host, d
default_port)
308         print(f"    Parameters: {host}:{port}")
309         print()
310
311         # Check порта
312         print("[5/5] Check доступности порта...")
313         if not self.check_port(port):
314             print("    [ERROR] Не удалось освободить порт")
315             return 1
316         print()
317
318         # Запуск сервера
319         return self.run_uvicorn(python_exe, host, port, ssl=us
e_ssl)
320
321 def main():
322     """Главная function"""
323     parser = argparse.ArgumentParser(
324         description="Лончер для запуска FastAPI сервера",
325         formatter_class=argparse.RawDescriptionHelpFormatter,
326         epilog="""
327 Examples:
328 python launchers/run.py
329 python launchers/run.py --host 0.0.0.0 --port 8000
330 python launchers/run.py --non-interactive
331 python launchers/run.py --help
332     """)
333
334
335     parser.add_argument("--host", "-H", type=str, default=None
, help="IP адрес привязки (0.0.0.0, 127.0.0.1)")
336     parser.add_argument("--port", "-P", type=int, default=None
, help="Порт сервера (по умолчанию: из config.json или 8000)")
337     parser.add_argument("--non-interactive", action="store_tru
e", help="Пропустить интерактивные запросы")
338

```

```
339 |     args = parser.parse_args()
340 |
341 |     launcher = ServerLauncher()
342 |     return launcher.run(host=args.host, port=args.port, non_in
|     teractive=args.non_interactive)
343 |
344 | if __name__ == "__main__":
345 |     sys.exit(main())
346 |
```

run_foundry.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Microsoft AI Foundry launcher for multiple pla
  tforms
4 # =====
5 # Description:
6 #   Launcher for running Microsoft AI Foundry with cross-platf
  orm support.
7 #   Supports Windows, Linux and macOS with configuration-drive
  n startup
8 #   and process management capabilities.
9 #
10 #   Usage:
11 #       python launchers/run_foundry.py start
12 #       python launchers/run_foundry.py stop
13 #
14 # File: run_foundry.py
15 # Project: ai-breadboard
16 # Package: root
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # =====
20
21 import argparse
22 import sys
23 import subprocess
24 import os
25 from pathlib import Path
26
27 sys.path.insert(0, str(Path(__file__).parent.parent))
28
29 from scripts.cli.paths import init_paths
30 from scripts.cli.config import get_config_manager
```

```

31
32 class FoundryLauncher:
33     """Launcher for running AI Foundry"""
34
35     def __init__(self):
36         self.paths = init_paths()
37         self.config_mgr = get_config_manager()
38         self.config = self.config_mgr.load_config()
39         self.ai_cfg = self.config.get("ai", {})
40
41     def start(self, action: str = "start") -> int:
42         """Starting or stopping Foundry"""
43         foundry_url = str(self.ai_cfg.get("foundry_base_url",
44 "http://localhost:54837"))
45         use_foundry = bool(self.ai_cfg.get("use_foundry", False))
46
47         print()
48         print("┌" + "=" * 68 + "┐")
49         print("│" + f" {action.upper()} MICROSOFT AI FOUNDRY".
50 center(68) + "│")
51         print("└" + "=" * 68 + "┘")
52         print()
53
54         if not use_foundry:
55             print("  Microsoft AI Foundry is disabled in config.json")
56             print("  Enable it: assist config set ai.use_foundry true")
57             print()
58             return 1
59
60         print(f"Foundry URL: {foundry_url}")
61         print()
62
63         # Linux/macOS requires foundry CLI installation
64         if sys.platform != "win32":
65             print("i On Linux/macOS use:")

```



```

64         print("    pip install microsoft-ai-foundry")
65         print("    foundry server start")
66         print()
67         print("i Or use Docker:")
68         print("    docker run -p 54837:54837 mcr.microsoft.
com/windows/servercore:latest")
69         print()
70         print("TODO: Implement cross-platform Foundry supp
ort")
71         return 1
72
73     # Windows
74     print("[*] Starting Foundry on Windows...")
75
76     # Attempting to find foundry.exe
77     try:
78         result = subprocess.run(
79             ["foundry", "server", action],
80             check=False,
81             capture_output=False
82         )
83         return result.returncode
84     except FileNotFoundError:
85         print("ERROR: foundry не найдена в PATH")
86         print("Установка: https://github.com/microsoft/ai-
foundry")
87         return 1
88     except Exception as e:
89         print(f"ERROR: {e}")
90         return 1
91
92     def stop(self) -> int:
93         """Остановить Foundry"""
94         print()
95         print("┌" + "=" * 68 + "┐")
96         print("│" + " ОСТАНОВКА MICROSOFT AI FOUNDRY".center(6
8) + "│")
97         print("└" + "=" * 68 + "┘")

```

```

98         print()
99
100        # Windows
101        if sys.platform == "win32":
102            try:
103                result = subprocess.run(
104                    ["foundry", "server", "stop"],
105                    check=False,
106                    capture_output=False
107                )
108                return result.returncode
109            except Exception as e:
110                print(f"ERROR: {e}")
111                return 1
112        else:
113            print("TODO: Реализовать остановку на Linux/macOS"
114        )
115
116        return 1
117
118    def main():
119        """Главная function"""
120        parser = argparse.ArgumentParser(
121            description="Лончер для Microsoft AI Foundry"
122        )
123
124        parser.add_argument(
125            "action",
126            nargs="?",
127            default="start",
128            choices=["start", "stop"],
129            help="Действие (start или stop)"
130        )
131
132        args = parser.parse_args()
133
134        launcher = FoundryLauncher()
135
136        if args.action == "start":

```

```
135 |         return launcher.start()
136 |     elif args.action == "stop":
137 |         return launcher.stop()
138 |     else:
139 |         return launcher.start()
140 |
141 | if __name__ == "__main__":
142 |     sys.exit(main())
143 |
```

run_light_server.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Lightweight FastAPI server launcher with minim
  al resources
4 # =====
5 # Description:
6 #   Lightweight launcher for running FastAPI server without au
  to-loading features.
7 #   Useful for minimal resource consumption on low-end machine
  s and development environments.
8 #   Provides quick server startup with port conflict detection
  and management.
9 #
10 #   Usage:
11 #       python launchers/run_light_server.py
12 #       python launchers/run_light_server.py --host 127.0.0.1
  --port 8000
13 #
14 # File: run_light_server.py
15 # Project: ai-breadboard
16 # Package: root
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # =====
20
21 import argparse
22 import sys
23 import subprocess
24 from pathlib import Path
25
26 sys.path.insert(0, str(Path(__file__).parent.parent))
27
28 from scripts.cli.paths import init_paths
```

```

29 from scripts.cli.config import get_config_manager
30 from scripts.cli.utils import get_process_on_port, kill_processes
31
32 class LightServerLauncher:
33     """Lightweight launcher for FastAPI"""
34
35     def __init__(self):
36         self.paths = init_paths()
37         self.config_mgr = get_config_manager()
38         self.config = self.config_mgr.load_config()
39         self.server_cfg = self.config.get("server", {})
40
41     def run(self, host: str = "", port: int = 0) -> int:
42         """Running lightweight server"""
43         print()
44         print("╔" + "=" * 68 + "╗")
45         print("║" + " STARTING LIGHTWEIGHT FastAPI SERVER".center(68) + "║")
46         print("╚" + "=" * 68 + "╝")
47         print()
48
49         # Parameters
50         if not host:
51             host = str(self.server_cfg.get("host", "0.0.0.0"))
52         if port == 0:
53             port = int(self.server_cfg.get("port", 8000))
54
55         python_exe = str(self.paths.venv_python) if self.paths.venv_python.exists() else sys.executable
56
57         print(f"[1/2] Checking port {port}...", end=" ", flush=True)
58
59         proc_info = get_process_on_port(port)
60         if proc_info:
61             pid, proc_name = proc_info
62             print(f"[OCCUPIED]")
63             if kill_process(pid, force=True):

```

```
63         print(f"    Process terminated")
64     else:
65         print(f"    WARN: Failed to free port")
66         return 1
67 else:
68     print("[FREE]")
69 print()
70
71 print("[2/2] Запуск сервера (облегченный режим)...")
72 print()
73
74 # Команда (без reload и debug для экономии ресурсов)
75 cmd = [
76     python_exe,
77     "-m", "uvicorn",
78     "main:app",
79     "--host", host,
80     "--port", str(port),
81     "--workers", "1",
82     "--log-level", "warning", # Меньше логов
83 ]
84
85 print(f"    Хост: {host}")
86 print(f"    Порт: {port}")
87 print(f"    Режим: Облегченный (без reload, 1 worker)")
88 print()
89 print("=" * 70)
90 print()
91
92 try:
93     result = subprocess.run(
94         cmd,
95         cwd=str(self.paths.project_root),
96         check=False
97     )
98     return result.returncode
99 except KeyboardInterrupt:
100     print()
```

```
101         print()
102         print("=" * 70)
103         print(" □ Сервер остановлен")
104         print("=" * 70)
105         return 0
106     except Exception as e:
107         print(f"ERROR: {e}")
108         return 1
109
110 def main():
111     """Главная function"""
112     parser = argparse.ArgumentParser(
113         description="Облегченный лончер для FastAPI"
114     )
115
116     parser.add_argument("--host", "-H", type=str, default="",
117                          help="IP адрес")
118
119     parser.add_argument("--port", "-P", type=int, default=0, h
120                          elp="Порт")
121
122     args = parser.parse_args()
123
124     launcher = LightServerLauncher()
125     return launcher.run(host=args.host, port=args.port)
126
127 if __name__ == "__main__":
128     sys.exit(main())
```

run_unicorn.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Cross-platform FastAPI launcher using uvicorn
4 # =====
5 # Description:
6 #   Launcher for running FastAPI application via uvicorn on Windows, Linux, and macOS.
7 #   Ported from legacy Run-Uncorn.ps1 with support for configurable host and port,
8 #   port conflict detection and process termination, virtual environment detection.
9 #
10 #   Usage:
11 #       python launchers/run_unicorn.py
12 #       python launchers/run_unicorn.py --host 127.0.0.1 --port 8000
13 #
14 # File: run_unicorn.py
15 # Project: ai-breadboard
16 # Package: root
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # =====
20
21 import argparse
22 import sys
23 import subprocess
24 from pathlib import Path
25
26 # Adding root to PYTHONPATH
27 sys.path.insert(0, str(Path(__file__).parent.parent))
28
29 from scripts.cli.paths import init_paths, get_paths
```



```

30 | from scripts.cli.config import get_config_manager
31 | from scripts.cli.utils import get_process_on_port, kill_processes
32 |
33 | class UnicornLauncher:
34 |     """Launcher for running FastAPI via uvicorn"""
35 |
36 |     def __init__(self):
37 |         self.paths = init_paths()
38 |         self.config_mgr = get_config_manager()
39 |         self.config = self.config_mgr.load_config()
40 |         self.server_cfg = self.config.get("server", {})
41 |
42 |     def get_default_host(self) -> str:
43 |         """Getting default host"""
44 |         return str(self.server_cfg.get("host", "0.0.0.0"))
45 |
46 |     def get_default_port(self) -> int:
47 |         """Getting default port"""
48 |         return int(self.server_cfg.get("port", 8000))
49 |
50 |     def check_venv(self) -> str:
51 |         """Checking virtual environment"""
52 |         if self.paths.venv_python.exists():
53 |             return str(self.paths.venv_python)
54 |         else:
55 |             return sys.executable
56 |
57 |     def check_port(self, port: int) -> bool:
58 |         """Checking and freeing port if needed"""
59 |         proc_info = get_process_on_port(port)
60 |         if proc_info:
61 |             pid, proc_name = proc_info
62 |             print(f"    [WARN] Port {port} is occupied by process {proc_name} (PID: {pid})")
63 |             if kill_process(pid, force=False):
64 |                 print(f"    [OK] Process terminated")
65 |             return True

```

```

66         elif kill_process(pid, force=True):
67             print(f"    [FORCE] Process forcefully termina
ted")
68             return True
69         else:
70             print(f"    [ERROR] Failed to free port")
71             return False
72     return True
73
74     def run(self, host: str = "", port: int = 0) -> int:
75         """Running uvicorn server"""
76         print()
77
78         # Определить Parameters
79         if not host:
80             host = self.get_default_host()
81         if port == 0:
82             port = self.get_default_port()
83
84         print("[1/3] Check виртуального окружения...")
85         python_exe = self.check_venv()
86         print(f"    [OK] Python: {python_exe}")
87         print()
88
89         print("[2/3] Check доступности порта...")
90         if not self.check_port(port):
91             return 1
92         print()
93
94         print("[3/3] Запуск uvicorn...")
95         print()
96
97         use_ssl = bool(self.server_cfg.get("use_ssl", True))
98         use_reload = bool(self.server_cfg.get("reload", True))
99         debug = bool(self.server_cfg.get("debug", True))
100
101         # Построить команду uvicorn
102         cmd = [

```

```
103         python_exe,
104         "-m", "uvicorn",
105         "main:app",
106         "--host", host,
107         "--port", str(port),
108     ]
109
110     if use_reload:
111         cmd.append("--reload")
112
113     if debug:
114         cmd.extend(["--log-level", "debug"])
115     else:
116         cmd.extend(["--log-level", "info"])
117
118     # SSL
119     if use_ssl:
120         certs_dir = self.paths.certs_dir
121         cert_file = certs_dir / "localhost+2.pem"
122         key_file = certs_dir / "localhost+2-key.pem"
123
124         if cert_file.exists() and key_file.exists():
125             cmd.extend([
126                 "--ssl-keyfile", str(key_file),
127                 "--ssl-certfile", str(cert_file),
128             ])
129
130     print(f" Хост: {host}")
131     print(f" Порт: {port}")
132     print(f" SSL: {'ВКЛ' if use_ssl else 'ВЫКЛ'}")
133     print(f" АвтоLoading: {'ВКЛ' if use_reload else 'ВЫКЛ'
134           '}")
135
136     print(f" Отладка: {'ВКЛ' if debug else 'ВЫКЛ'}")
137     print()
138     print(" Нажмите Ctrl+C для остановки сервера")
139     print()
140     print("=" * 70)
141     print()
```

```

140
141     # Запустить
142     try:
143         result = subprocess.run(
144             cmd,
145             cwd=str(self.paths.project_root),
146             check=False
147         )
148         return result.returncode
149     except KeyboardInterrupt:
150         print()
151         print()
152         print("=" * 70)
153         print(" □ Сервер остановлен")
154         print("=" * 70)
155         return 0
156     except Exception as e:
157         print(f"ERROR: {e}")
158         return 1
159
160 def main():
161     """Главная function"""
162     parser = argparse.ArgumentParser(
163         description="Лончер для запуска FastAPI через uvicorn"
164         ,
165         formatter_class=argparse.RawDescriptionHelpFormatter,
166         epilog="""
167 Examples:
168 python launchers/run_unicorn.py
169 python launchers/run_unicorn.py --host 0.0.0.0 --port 8000
170 python launchers/run_unicorn.py --host 127.0.0.1
171     """
172     )
173     parser.add_argument("--host", "-H", type=str, default="",
174                         help="IP адрес привязки")
175     parser.add_argument("--port", "-P", type=int, default=0, h
176                         elp="Порт сервера")

```

```
175 |
176 |     args = parser.parse_args()
177 |
178 |     launcher = UnicornLauncher()
179 |     return launcher.run(host=args.host, port=args.port)
180 |
181 | if __name__ == "__main__":
182 |     sys.exit(main())
183 |
```

main.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: FastAPI application initialization and web server startup
4 # =====
5 # Description:
6 #   Main entry point for FastAPI web application with router initialization,
7 #   CORS middleware configuration, and uvicorn server startup with Telegram bot integration
8 #   and Windows asyncio event loop noise suppression for production stability.
9 #
10 # File: main.py
11 # Project: ai-breadboard
12 # Package: root
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 from __future__ import annotations
18
19 import asyncio
20 import os
21 import signal
22 import sys
23 from pathlib import Path
24
25 # Suppression of Windows asyncio system noise (WinError 10054 on client connection resets)
26 if sys.platform == 'win32':
27     try:
28         from asyncio.proactor_events import _ProactorBasePipeT
```

```

ransport
29         _orig_call_connection_lost = _ProactorBasePipeTransport._call_connection_lost
30
31         def _silence_connection_lost(self, exc=None):
32             try:
33                 _orig_call_connection_lost(self, exc)
34             except (ConnectionResetError, ConnectionAbortedError, OSError) as e:
35                 if getattr(e, 'winerror', None) in (10054, 10053) or isinstance(e, (ConnectionResetError, ConnectionAbortedError)):
36                     pass
37                 else:
38                     raise
39
40         _ProactorBasePipeTransport._call_connection_lost = _silence_connection_lost
41     except Exception:
42         pass
43
44 import uvicorn
45 from dotenv import load_dotenv
46 from fastapi import FastAPI, HTTPException, Request
47 from fastapi.middleware.cors import CORSMiddleware
48 from fastapi.responses import HTMLResponse
49 from fastapi.staticfiles import StaticFiles
50
51 import header
52 from header import __root__
53 from src.ai import GoogleGenerativeAI
54 from src.fastapi import (
55     init_auth_router,
56     init_chat_router,
57     init_control_router,
58     init_tts_router,
59     init_logs_router,
60     init_keys_router,

```

```

61     init_admin_router,
62     init_agents_router,
63     router_openai,
64 )
65 from src.fastapi.router_version import init_router as init_ver
   sion_router
66 from src.logger import logger
67 from src.utils.file import read_text_file
68 from src.utils.jjson import j_loads_ns
69 from src.utils.versioning import compare_versions as _compare_
   versions, choose_best_tag
70 import subprocess
71 import json
72 import urllib.request
73 import configparser
74 import re
75
76 load_dotenv(__root__ / '.env')
77 from src.config import server_cfg, ai_cfg, tts_cfg, logging_cf
   g
78
79 app = FastAPI()
80 app.add_middleware(
81     CORSMiddleware,
82     allow_origins=['*'],
83     allow_credentials=True,
84     allow_methods=['*'],
85     allow_headers=['*'],
86 )
87
88 # Auto login local user to user_id=1
89 @app.middleware("http")
90 async def auto_login_local_user(request: Request, call_next):
91     hostname: str = request.url.hostname or ''
92     is_local: bool = (
93         hostname in ('127.0.0.1', 'localhost', '::1', 'testser
   ver', '0.0.0.0')
94         or hostname.startswith('192.168.')

```



```

95         or hostname.startswith('10.')
96         or hostname.startswith('172.')
97     )
98     if is_local:
99         token: str = request.cookies.get('auth_token', '')
100         from src.fastapi.router_auth import verify_jwt_token
101         is_token_valid: bool = False
102         if token:
103             try:
104                 is_token_valid = bool(verify_jwt_token(token))
105             except Exception:
106                 is_token_valid = False
107
108         if not token or not is_token_valid:
109             from src.user_manager import user_manager
110             try:
111                 db_user = user_manager.get_user_by_id(1)
112                 if db_user:
113                     from src.fastapi.router_auth import TokenD
114                     ata, create_jwt_token
115                     token_data = TokenData(
116                         email=db_user['email'],
117                         name=db_user['name'],
118                         picture=db_user.get('picture', ''),
119                         id=db_user['id']
120                     )
121                     token = create_jwt_token(token_data)
122
123                     # Forward cookie to current request header
124                     s for subsequent middleware and handlers
125                     raw_headers = list(request.scope.get('head
126                     ers', []))
127                     cookie_bytes = f"auth_token={token}".encod
128                     e('utf-8')
129
130                     new_headers = []
131                     found_cookie: bool = False
132                     for k, v in raw_headers:
133                         if k.lower() == b'cookie':

```

```

129         new_headers.append((k, v + b'; ' +
    cookie_bytes))
130         found_cookie = True
131     else:
132         new_headers.append((k, v))
133     if not found_cookie:
134         new_headers.append((b'cookie', cookie_
    bytes))
135     request.scope['headers'] = new_headers
136     if hasattr(request, '_cookies'):
137         delattr(request, '_cookies')
138
139     response = await call_next(request)
140     response.set_cookie(
141         'auth_token',
142         token,
143         httponly=True,
144         secure=False,
145         samesite='lax',
146         max_age=3600 * 24 * 30 # 30 days
147     )
148     return response
149 except Exception as e:
150     logger.error(f"Error in auto_login_local_user
    middleware: {e}")
151     return await call_next(request)
152
153 # Mount static files
154 webinterface_dir = __root__ / 'src' / 'fastapi' / 'webinterfac
    e'
155 webinterface_dir.mkdir(parents=True, exist_ok=True)
156 app.mount('/webinterface', StaticFiles(directory=webinterface_
    dir), name='webinterface')
157 app.mount('/html', StaticFiles(directory=webinterface_dir), na
    me='html')
158 simple_assistant_dir = __root__ / 'SANDBOX' / 'AI Assistant' /
    'Simple Assistant'
159 app.mount('/simple-assistant', StaticFiles(directory=simple_as

```

```

    sistant_dir, html=True), name='simple-assistant')
160
161 def _parse_version(v: str) -> list[int]:
162     """Legacy simple numeric parser kept for backward compatib
    ility.
163     Prefer using semver-aware functions below.
164     """
165     if not v:
166         return [0, 0, 0]
167     parts = re.findall(r"(\d+)", v)
168     return [int(p) for p in parts]
169
170 def _cfg_get(key: str, default=None):
171     try:
172         vc = getattr(server_cfg, 'version_check', None)
173         if vc is None:
174             return default
175         # support both attribute and dict-like access
176         if isinstance(vc, dict):
177             return vc.get(key, default)
178         try:
179             val = getattr(vc, key)
180             return val
181         except Exception:
182             try:
183                 return vc.get(key, default)
184             except Exception:
185                 return default
186     except Exception:
187         return default
188
189 def _cfg_get_bool(key: str, env_names: list[str], default: boo
    l = False) -> bool:
190     val = _cfg_get(key, None)
191     if val is not None:
192         return bool(val)
193     for env in env_names:
194         ev = os.getenv(env)

```

```

195         if ev is not None:
196             return str(ev).lower() in ('1', 'true', 'yes')
197     return default
198
199 def get_local_version() -> str:
200     """Try to read version from setup.cfg [metadata] section.
201     Fallback to 0.0.0."""
202     cfg = configparser.ConfigParser()
203     try:
204         setup_cfg = Path(__root__) / 'setup.cfg'
205         if setup_cfg.exists():
206             cfg.read(setup_cfg)
207             if cfg.has_section('metadata') and cfg.has_option(
208                 'metadata', 'version'):
209                 return cfg.get('metadata', 'version').strip()
210     except Exception:
211         pass
212     return '0.0.0'
213
214 def _get_git_origin_remote() -> str | None:
215     """Return origin remote URL or None."""
216     try:
217         out = subprocess.check_output(['git', 'config', '--get',
218             ', 'remote.origin.url'], cwd=str(__root__), stderr=subprocess.
219             DEVNULL)
220         url = out.decode().strip()
221         return url
222     except Exception:
223         return None
224
225 def _parse_github_owner_repo(remote_url: str) -> tuple[str, str] | None:
226     """Parse GitHub owner and repo from remote URL.
227     Supports HTTPS and SSH forms.
228     """
229     if not remote_url:
230         return None
231     # https://github.com/owner/repo.git

```

```

228     m = re.search(r'github.com[:/](?P<owner>[^/]+)/(?P<repo>[^
    |/.]+)', remote_url)
229     if m:
230         return m.group('owner'), m.group('repo')
231     return None
232
233 def get_remote_latest_version() -> str | None:
234     """Query GitHub API for the latest version.
235
236     Flow:
237     - Try /releases/latest and use tag_name/name if available.
238     - Fallback to /tags and pick the highest semantic version-
    like tag.
239     Authenticated requests are used when GITHUB_TOKEN or GH_TO
    KEN env var is present.
240     """
241     def _headers() -> dict:
242         headers = {'User-Agent': 'ai-breadboard-version-check'
    }
243         token = _cfg_get('github_token', None) or os.getenv('G
    ITHUB_TOKEN') or os.getenv('GH_TOKEN') or os.getenv('GITHUB_AP
    I_TOKEN')
244         if token:
245             headers['Authorization'] = f'token {token}'
246         return headers
247
248     try:
249         remote = _get_git_origin_remote()
250         if not remote:
251             return None
252         parsed = _parse_github_owner_repo(remote)
253         if not parsed:
254             return None
255         owner, repo = parsed
256
257         # Try releases/latest first
258         try:
259             url = f'https://api.github.com/repos/{owner}/{repo

```

```

    }/releases/latest'
260         req = urllib.request.Request(url, headers=_headers
    ())
261         with urllib.request.urlopen(req, timeout=5) as res
    p:
262             data = json.load(resp)
263             tag = data.get('tag_name') or data.get('name')
264             if tag:
265                 return tag
266         except Exception:
267             # fallback to tags
268             pass
269
270         # Fallback: get tags and pick the highest semver-like
    tag
271         try:
272             # Optionally fetch tags from remote to ensure
    local tag refs are up-to-date
273             fetch_tags_enabled = _cfg_get_bool('fetch_tags
    ', ['FETCH_TAGS', 'ai-breadboard_FETCH_TAGS', 'FETCH_REMOTE_TA
    GS'], False)
274             if fetch_tags_enabled:
275                 try:
276                     logger.info('Fetching remote tags (git
    fetch --tags origin)')
277                     subprocess.check_call(['git', 'fetch',
    '--tags', 'origin'], cwd=str(__root__))
278                 except Exception as e:
279                     logger.debug(f'Failed to fetch tags: {
    e}')
280
281             url = f'https://api.github.com/repos/{owner}/{
    repo}/tags'
282             req = urllib.request.Request(url, headers=_hea
    ders())
283             with urllib.request.urlopen(req, timeout=5) as
    resp:
284                 data = json.load(resp)

```

```

285 |             tags = [t.get('name') for t in data if t.g
| et('name')]
286 |             if not tags:
287 |                 return None
288 |
289 |             # Logging of the remote tag list when debu
| g enabled
290 |             debug_enabled = _cfg_get('debug', None)
291 |             if debug_enabled is None:
292 |                 debug_enabled = os.getenv('VERSION_CHE
| CK_DEBUG') == '1' or (hasattr(logger, 'isEnabledFor') andlogg
| er.isEnabledFor(10))
293 |             if debug_enabled:
294 |                 logger.debug(f'Remote tags from API: {
| tags}')
295 |
296 |             # Prefer stable releases unless prerelease
| is explicitly allowed
297 |             allow_prerelease = _cfg_get('allow_prerele
| ase', None)
298 |             if allow_prerelease is None:
299 |                 allow_prerelease = str(os.getenv('ALLO
| W_PRERELEASE') or os.getenv('ai-breadboard_ALLOW_PRERELEASE')
| or '').lower() in ('1', 'true', 'yes')
300 |
301 |             best = choose_best_tag(tags, allow_prerele
| ase=bool(allow_prerelease), debug=bool(debug_enabled))
302 |             if debug_enabled:
303 |                 logger.debug(f'Selected tag: {best} (a
| llow_prerelease={allow_prerelease})')
304 |             return best
305 |         except Exception:
306 |             # Try using git ls-remote --tags origin as a last-
| resort fallback
307 |             try:
308 |                 out = subprocess.check_output(['git', 'ls-remo
| te', '--tags', 'origin'], cwd=str(__root__), stderr=subprocess
| .DEVNULL)

```

```

309 |         lines = out.decode().splitlines()
310 |         tags = []
311 |         for line in lines:
312 |             parts = line.split('\t')
313 |             if len(parts) < 2:
314 |                 continue
315 |             ref = parts[1]
316 |             if ref.startswith('refs/tags/'):
317 |                 tag = ref[len('refs/tags/'):]
318 |                 # annotated tags may have ^{} suffix:
319 |                 tag = tag.replace('^{}', '')
320 |                 tags.append(tag)
321 |         if not tags:
322 |             return None
323 |
324 |         debug_enabled = _cfg_get('debug', None)
325 |         if debug_enabled is None:
326 |             debug_enabled = os.getenv('VERSION_CHECK_D
327 | EBUG') == '1' or (hasattr(logger, 'isEnabledFor') and logger.i
328 | sEnabledFor(10))
329 |
330 |         if debug_enabled:
331 |             logger.debug(f'Remote tags from ls-remote:
332 | {tags}')
333 |
334 |         allow_prerelease = _cfg_get('allow_prerelease'
335 | , None)
336 |
337 |         if allow_prerelease is None:
338 |             allow_prerelease = str(os.getenv('ALLOW_PR
339 | ERELEASE') or os.getenv('ai-breadboard_ALLOW_PRERELEASE') or '
340 | ').lower() in ('1', 'true', 'yes')
341 |
342 |         best = choose_best_tag(tags, allow_prerelease=
343 | bool(allow_prerelease), debug=bool(debug_enabled))
344 |
345 |         if debug_enabled:
346 |             logger.debug(f'Selected tag from ls-remote
347 | : {best} (allow_prerelease={allow_prerelease})')
348 |
349 |         return best

```



```

338         except Exception:
339             return None
340     except Exception:
341         return None
342
343 def _is_interactive() -> bool:
344     return sys.stdin.isatty() and sys.stdout.isatty()
345
346 def prompt_and_perform_update(branch: str = 'main') -> None:
347     """If a newer remote version exists, prompt the user and,
348     on consent, run git pull --ff-only.
349     This function is conservative and skips the check in non-i
350     nteractive environments.
351     И использует новый VersionManager для автоматического резерв
352     ного копирования и восстановления.
353     """
354     try:
355         auto_update = _cfg_get('auto_update', None)
356         if auto_update is None:
357             auto_update_env = os.getenv('AUTO_UPDATE') or os.g
358             etenv('ai-breadboard_AUTO_UPDATE')
359             auto_update = str(auto_update_env or '').lower() i
360             n ('1', 'true', 'yes')
361         else:
362             auto_update = bool(auto_update)
363
364         if not _is_interactive() and not auto_update:
365             logger.debug('Non-interactive shell and AUTO_UPDAT
366             E not enabled: skipping version check')
367             return
368
369         # Используем новый VersionManager для проверки обновле
370         ний
371         try:
372             from src.version_manager import get_version_manage
373             r
374             vm = get_version_manager(__root__)

```

```

368
369         check_result = vm.check_updates()
370         if check_result.get("status") == "error":
371             logger.warning(f"Version check failed: {check_
result.get('message')}")
372             return
373
374         if not check_result.get("is_update_available"):
375             logger.info(f"Application is up-to-date: {chec
k_result.get('current_version')}")
376             return
377
378         current = check_result.get('current_version', 'unk
nown')
379         remote = check_result.get('remote_version', 'unkno
wn')
380         logger.info(f'A newer version is available: {remot
e} (current: {current})')
381
382         do_update = auto_update
383         if not do_update:
384             # interactive prompt
385             try:
386                 resp = input(
387                     f'Доступно update {remote} (текущая ве
рсия {current}). '
388                     'Обновить код и перезагрузиться? [y/N]
: '
389                     ).strip().lower()
390                 do_update = resp in ('y', 'yes')
391             except Exception:
392                 do_update = False
393
394         if not do_update:
395             logger.info('User declined update or update no
t approved')
396             return
397

```

```

398         # Выполняем update с автоматическим резервным копи
    рованием
399         logger.info(f'Starting update from {current} to {r
    emote}...')
400
401         # Создаём резервную копию перед обновлением
402         backup_path = vm.backup_files()
403         if not backup_path:
404             logger.warning('Failed to create backup, updat
    e cancelled')
405             return
406
407         # Скачиваем обновления
408         if not vm.fetch_updates():
409             logger.error('Failed to fetch updates')
410             vm.restore_from_backup(backup_path)
411             print('Failed to fetch updates. Restored from
    backup.')
412             return
413
414         # Объединяем обновления
415         merge_ok, merge_msg = vm.merge_updates(branch)
416         if not merge_ok:
417             logger.error(f'Failed to merge updates: {merge
    _msg}')
418             vm.restore_from_backup(backup_path)
419             print(f'Failed to merge updates: {merge_msg}.
    Restored from backup.')
420             return
421
422         logger.success(f'Update completed: {current} → {re
    mote}')
423         logger.info(f'Backup saved at: {backup_path}')
424         print(f'Update completed successfully: {current} →
    {remote}')
425         print('You should restart the application to apply
    changes.')
426

```

```

427 |         # Очистка старых резервных копий
428 |         vm.cleanup_old_backups(keep_count=5)
429 |
430 |     except ImportError:
431 |         # Fallback to legacy implementation if version_manager
432 |         # import fails
433 |         logger.debug('Using legacy version check (version_
434 | manager not available)')
435 |         local_v = get_local_version()
436 |         remote_v = get_remote_latest_version()
437 |         if not remote_v:
438 |             logger.debug('Could not determine remote versi
439 | on')
440 |             return
441 |
442 |         cmp = _compare_versions(local_v, remote_v)
443 |         if cmp >= 0:
444 |             logger.info(f'Local version {local_v} is up-to
445 | -date (remote {remote_v})')
446 |             return
447 |
448 |         logger.info(f'A newer version is available: {remot
449 | e_v} (local: {local_v}).')
450 |
451 |         do_update = auto_update
452 |         if not do_update:
453 |             # interactive prompt
454 |             try:
455 |                 resp = input('Do you want to update the co
456 | de from origin and restart? [y/N]: ').strip().lower()
457 |                 do_update = resp in ('y', 'yes')
458 |             except Exception:
459 |                 do_update = False
460 |
461 |         if not do_update:
462 |             logger.info('User declined update or update no
463 | t approved')
464 |             return

```

```

458
459         # run git fetch + merge --ff-only origin/<branch>
460         try:
461             logger.info(f'Pulling latest changes from origin/{branch}...')
462             subprocess.check_call(['git', 'fetch', 'origin', branch], cwd=str(__root__))
463             subprocess.check_call(['git', 'merge', '--ff-only', f'origin/{branch}'], cwd=str(__root__))
464             logger.info('Update pulled successfully. You should restart the process to apply changes.')
465             print('Update pulled successfully. Please restart the application to apply changes.')
466             except subprocess.CalledProcessError as e:
467                 logger.error(f'Failed to update repository: {e}')
468                 print('Failed to update repository. See logs for details.')
469         except Exception as e:
470             logger.error(f'Error during version check/update: {e}')
471
472 _api_key_names: list[str] = [n.strip() for n in os.getenv('GEMINI_API_KEY_NAMES', '').split(',') if n.strip()]
473
474 use_foundry = getattr(ai_cfg, 'use_foundry', False) if ai_cfg else False
475 foundry_model_id = getattr(ai_cfg, 'foundry_model_id', 'qwen2.5-1.5b') if ai_cfg else 'qwen2.5-1.5b'
476
477 use_ollama = getattr(ai_cfg, 'use_ollama', False) if ai_cfg else False
478 ollama_model_id = getattr(ai_cfg, 'ollama_model_id', 'llama3.1') if ai_cfg else 'llama3.1'
479 ollama_base_url = getattr(ai_cfg, 'ollama_base_url', 'http://localhost:11434') if ai_cfg else 'http://localhost:11434'
480
481 from src.ai import UnifiedChatModel

```

```
482 |
483 | # --- Chat model instance ---
484 | _system_instruction_chat: str = read_text_file(__root__ / 'prompts' / 'chat' / 'system_instruction.md') or ''
485 | chat_model = UnifiedChatModel(
486 |     api_key_names=_api_key_names,
487 |     system_instruction=_system_instruction_chat,
488 |     foundry_model_id=foundry_model_id,
489 |     use_foundry=use_foundry,
490 |     use_ollama=use_ollama,
491 |     ollama_model_id=ollama_model_id,
492 |     ollama_base_url=ollama_base_url,
493 | )
494 |
495 | # --- Narrator model instance ---
496 | _system_instruction_narrator: str = read_text_file(__root__ /
497 | 'prompts' / 'narrator' / 'narrator_style.md') or ''
498 | narrator_model = UnifiedChatModel(
499 |     api_key_names=_api_key_names,
500 |     system_instruction=_system_instruction_narrator,
501 |     foundry_model_id=foundry_model_id,
502 |     use_foundry=use_foundry,
503 |     use_ollama=use_ollama,
504 |     ollama_model_id=ollama_model_id,
505 |     ollama_base_url=ollama_base_url,
506 | )
507 |
508 | logger.info('Chat and Narrator model instances initialized')
509 |
510 | app.include_router(init_chat_router(chat_model, narrator_model))
511 |
512 | app.include_router(init_auth_router())
513 | app.include_router(init_control_router())
514 | app.include_router(init_tts_router())
515 | app.include_router(init_logs_router())
516 | app.include_router(init_keys_router())
517 | app.include_router(init_admin_router())
518 | app.include_router(init_agents_router())
519 | app.include_router(init_version_router())
```

```

517 app.include_router(router_openai)
518
519 @app.on_event("startup")
520 async def startup_event():
521     # Store model instances in app.state for access from route
    rs
522     app.state.chat_model = chat_model
523     app.state.narrator_model = narrator_model
524
525     # One-time actualization and caching of all provider model
    s
526     from src.ai.model_manager import actualize_all_models
527     await actualize_all_models()
528
529     # Log analyzer is disabled – run manually if needed
530     # from src.logger.log_analyzer import start_log_analyzer
531     # start_log_analyzer()
532
533     # Check обновлений при старте приложения
534     try:
535         from src.version_manager import get_version_manager
536         vm = get_version_manager(__root__)
537         check_result = vm.check_updates()
538         if check_result.get("is_update_available"):
539             logger.warning(
540                 f"Доступно update: {check_result.get('remote_v
    ersion')}} "
541                 f"(текущая версия: {check_result.get('current_
    version'}})). "
542                 f"Запустите приложение с флагом --check-update
    для обновления."
543             )
544         else:
545             logger.info(f"Приложение актуально: версия {check_
    result.get('current_version'}}")
546     except Exception as e:
547         logger.debug(f"Failed to check updates on startup: {e}
    ")

```

```

548
549     if ai_cfg and getattr(ai_cfg, 'preload_silero', False):
550         from src.tts.silero import get_silero_model
551         logger.info("Pre-loading Silero TTS model...")
552         try:
553             get_silero_model()
554             logger.info("Silero TTS model pre-loaded successfu
lly.")
555         except Exception as e:
556             logger.error(f"Failed to pre-load Silero TTS model
: {e}")
557
558 @app.get('/', response_class=HTMLResponse)
559 async def root(request: Request) -> HTMLResponse:
560     """Serving of main HTML page – admin interface."""
561     auth_response = check_admin_auth(request)
562     if auth_response:
563         return auth_response
564     content = read_text_file(webinterface_dir / 'admin' / 'ind
ex.html')
565     if not content:
566         raise HTTPException(status_code=500, detail='Failed to
read admin index page')
567     return HTMLResponse(content=content)
568
569 @app.get('/tgmini', response_class=HTMLResponse)
570 async def tgmini_interface() -> HTMLResponse:
571     """Serving of Telegram Mini App HTML page."""
572     content = read_text_file(webinterface_dir / 'tgmini' / 'in
dex.html')
573     if not content:
574         raise HTTPException(status_code=500, detail='Failed to
read Telegram Mini App index page')
575     return HTMLResponse(content=content)
576
577 @app.get('/tgmini/{full_path:path}', response_class=HTMLRespon
se)
578 async def tgmini_static(full_path: str) -> HTMLResponse:

```



```

579     """Serving Telegram Mini App static files."""
580     content = read_text_file(webinterface_dir / 'tgmini' / full_path)
581     if not content:
582         raise HTTPException(status_code=404, detail='File not found')
583     return HTMLResponse(content=content)
584
585 @app.get('/rc', response_class=HTMLResponse)
586 async def rc_interface() -> HTMLResponse:
587     """Serving of Remote Control HTML page."""
588     content = read_text_file(webinterface_dir / 'rc' / 'index.html')
589     if not content:
590         raise HTTPException(status_code=500, detail='Failed to read Remote Control page')
591     return HTMLResponse(content=content)
592
593 @app.get('/rc/{full_path:path}', response_class=HTMLResponse)
594 async def rc_static(full_path: str) -> HTMLResponse:
595     """Serving Remote Control static files."""
596     content = read_text_file(webinterface_dir / 'rc' / full_path)
597     if not content:
598         raise HTTPException(status_code=404, detail='File not found')
599     return HTMLResponse(content=content)
600
601 @app.get('/user_tts', response_class=HTMLResponse)
602 async def user_tts_interface() -> HTMLResponse:
603     """Serving of User TTS experimental page."""
604     content = read_text_file(webinterface_dir / 'user_tts' / 'index.html')
605     if not content:
606         raise HTTPException(status_code=500, detail='Failed to read User TTS page')
607     return HTMLResponse(content=content)
608

```

```

609 @app.get('/user_tts/{full_path:path}', response_class=HTMLResp
    onse)
610 async def user_tts_static(full_path: str) -> HTMLResponse:
611     """Serving User TTS static files."""
612     content = read_text_file(webinterface_dir / 'user_tts' / f
ull_path)
613     if not content:
614         raise HTTPException(status_code=404, detail='File not
found')
615     return HTMLResponse(content=content)
616
617 ADMIN_LOGIN_HTML = """<!DOCTYPE html>
618 <html lang="ru">
619 <head>
620     <meta charset="UTF-8">
621     <meta name="viewport" content="width=device-width, initial
-scale=1.0">
622     <title>Вход в панель управления</title>
623     <link rel="preconnect" href="https://fonts.googleapis.com"
>
624     <link rel="preconnect" href="https://fonts.gstatic.com" cr
ossorigin>
625     <link href="https://fonts.googleapis.com/css2?family=Outfi
t:wght@300;400;600&family=Plus+Jakarta+Sans:wght@300;400;600&d
isplay=swap" rel="stylesheet">
626     <style>
627         :root {
628             --bg-color: #0b0f19;
629             --card-bg: rgba(255, 255, 255, 0.03);
630             --card-border: rgba(255, 255, 255, 0.08);
631             --primary: #6366f1;
632             --primary-glow: rgba(99, 102, 241, 0.4);
633             --text-main: #f3f4f6;
634             --text-muted: #9ca3af;
635         }
636
637         * {
638             box-sizing: border-box;

```

```
639 |         margin: 0;
640 |         padding: 0;
641 |     }
642 |
643 |     body {
644 |         font-family: 'Plus Jakarta Sans', 'Outfit', sans-s
erif;
645 |         background-color: var(--bg-color);
646 |         color: var(--text-main);
647 |         height: 100vh;
648 |         display: flex;
649 |         align-items: center;
650 |         justify-content: center;
651 |         overflow: hidden;
652 |         position: relative;
653 |     }
654 |
655 |     body::before {
656 |         content: '';
657 |         position: absolute;
658 |         width: 300px;
659 |         height: 300px;
660 |         background: radial-gradient(circle, var(--primary-
glow) 0%, transparent 70%);
661 |         top: 20%;
662 |         left: 30%;
663 |         z-index: 0;
664 |         filter: blur(40px);
665 |         animation: float-slow 12s infinite alternate ease-
in-out;
666 |     }
667 |     body::after {
668 |         content: '';
669 |         position: absolute;
670 |         width: 400px;
671 |         height: 400px;
672 |         background: radial-gradient(circle, rgba(168, 85,
247, 0.2) 0%, transparent 70%);
```

```

673         bottom: 15%;
674         right: 25%;
675         z-index: 0;
676         filter: blur(50px);
677         animation: float-slow 15s infinite alternate-reverse ease-in-out;
678     }
679
680     @keyframes float-slow {
681         0% { transform: translate(0, 0) scale(1); }
682         100% { transform: translate(50px, 30px) scale(1.1)
683     ; }
684
685     .login-container {
686         position: relative;
687         z-index: 10;
688         width: 100%;
689         max-width: 420px;
690         padding: 40px;
691         background: var(--card-bg);
692         border: 1px solid var(--card-border);
693         border-radius: 24px;
694         backdrop-filter: blur(20px);
695         -webkit-backdrop-filter: blur(20px);
696         box-shadow: 0 20px 40px rgba(0, 0, 0, 0.4);
697         animation: fadeIn 0.8s cubic-bezier(0.16, 1, 0.3,
698     1);
699         text-align: center;
700     }
701
702     @keyframes fadeIn {
703         from { opacity: 0; transform: translateY(20px); }
704         to { opacity: 1; transform: translateY(0); }
705     }
706
707     .logo {
708         margin-bottom: 28px;

```

```
708 |     }
709 |
710 |     .logo h1 {
711 |         font-size: 26px;
712 |         font-weight: 600;
713 |         letter-spacing: -0.5px;
714 |         background: linear-gradient(135deg, #fff 0%, var(--text-muted) 100%);
715 |         -webkit-background-clip: text;
716 |         -webkit-text-fill-color: transparent;
717 |     }
718 |
719 |     .logo p {
720 |         font-size: 14px;
721 |         color: var(--text-muted);
722 |         margin-top: 8px;
723 |         line-height: 1.4;
724 |     }
725 |
726 |     .btn-google {
727 |         display: flex;
728 |         align-items: center;
729 |         justify-content: center;
730 |         gap: 12px;
731 |         width: 100%;
732 |         padding: 14px 20px;
733 |         background: #ffffff;
734 |         color: #1f2937;
735 |         font-size: 15px;
736 |         font-weight: 600;
737 |         border-radius: 12px;
738 |         text-decoration: none;
739 |         transition: all 0.3s ease;
740 |         box-shadow: 0 4px 12px rgba(255, 255, 255, 0.15);
741 |         margin-bottom: 20px;
742 |     }
743 |
744 |     .btn-google:hover {
```

```
745         background: #f3f4f6;
746         transform: translateY(-2px);
747         box-shadow: 0 6px 20px rgba(255, 255, 255, 0.25);
748     }
749
750     .divider {
751         display: flex;
752         align-items: center;
753         text-align: center;
754         margin: 20px 0;
755         color: var(--text-muted);
756         font-size: 13px;
757     }
758
759     .divider::before, .divider::after {
760         content: '';
761         flex: 1;
762         border-bottom: 1px solid rgba(255, 255, 255, 0.1);
763     }
764
765     .divider span {
766         padding: 0 10px;
767     }
768
769     .input-group {
770         position: relative;
771         margin-bottom: 18px;
772     }
773
774     .input-group input {
775         width: 100%;
776         padding: 14px 18px;
777         background: rgba(255, 255, 255, 0.05);
778         border: 1px solid rgba(255, 255, 255, 0.1);
779         border-radius: 12px;
780         color: #fff;
781         font-size: 15px;
782         outline: none;
```

```
783         transition: all 0.3s ease;
784         font-family: inherit;
785     }
786
787     .input-group input:focus {
788         border-color: var(--primary);
789         box-shadow: 0 0 0 4px var(--primary-glow);
790         background: rgba(255, 255, 255, 0.08);
791     }
792
793     .btn-submit {
794         width: 100%;
795         padding: 14px;
796         background: linear-gradient(135deg, var(--primary)
0%, #4f46e5 100%);
797         border: none;
798         border-radius: 12px;
799         color: #fff;
800         font-size: 15px;
801         font-weight: 600;
802         cursor: pointer;
803         transition: all 0.3s ease;
804         box-shadow: 0 4px 12px var(--primary-glow);
805         font-family: inherit;
806     }
807
808     .btn-submit:hover {
809         transform: translateY(-2px);
810         box-shadow: 0 6px 20px var(--primary-glow);
811     }
812 </style>
813 </head>
814 <body>
815     <div class="login-container">
816         <div class="logo">
817             <h1>Панель управления</h1>
818             <p>Авторизуйтесь через Google для доступа к функци
ям AI Assistant, Google Документам, Календарю и Контактam</p>
```

```

819         </div>
820
821         <a href="/auth/google?next=/admin" class="btn-google">
822             <svg width="20" height="20" viewBox="0 0 24 24">
823                 <path fill="#4285F4" d="M22.56 12.25c0-.78-.07
-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-1.04 2.53-2.21 3.31v2.77h
3.57c2.08-1.92 3.28-4.74 3.28-8.09z"/>
824                 <path fill="#34A853" d="M12 23c2.97 0 5.46-.98
7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-3.71 1.06-2.86 0-5.29-
1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z"/>
825                 <path fill="#FBBC05" d="M5.84 14.09c-.22-.66-.
35-1.36-.35-2.09s.13-1.43.35-2.09V7.06H2.18C1.43 8.55 1 10.22
1 12s.43 3.45 1.18 4.94l2.85-2.22.81-.63z"/>
826                 <path fill="#EA4335" d="M12 5.38c1.62 0 3.06.5
6 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1 7.7 1 3.99 3.47
2.18 7.06l3.66 2.84c.87-2.6 3.3-4.52 6.16-4.52z"/>
827             </svg>
828             Войти через Google
829         </a>
830
831         <div class="divider"><span>или локальный вход</span></
div>
832
833         <form method="POST" action="/admin">
834             <div class="input-group">
835                 <input type="password" name="password" placeho
lder="Пароль администратора" required autocomplete="current-pa
ssword">
836             </div>
837             <button type="submit" class="btn-submit">Войти по
паролю</button>
838         </form>
839     </div>
840 </body>
841 </html>
842 """
843
844 def check_admin_auth(request: Request):

```



```

845 |     """Check прав доступа к панели администратора с Google OAuth."""
846 |     from fastapi.responses import RedirectResponse, HTMLResponse
847 |     from src.fastapi.router_auth import verify_jwt_token
848 |     from src.user_manager import user_manager
849 |
850 |     token = request.cookies.get('auth_token', '')
851 |     if not token:
852 |         if request.url.path in ('/admin', '/'):
853 |             return HTMLResponse(content=ADMIN_LOGIN_HTML)
854 |             return RedirectResponse(url='/auth/google?next=' + request.url.path, status_code=303)
855 |
856 |     user_data = verify_jwt_token(token)
857 |     if not user_data:
858 |         if request.url.path in ('/admin', '/'):
859 |             return HTMLResponse(content=ADMIN_LOGIN_HTML)
860 |             return RedirectResponse(url='/auth/google?next=' + request.url.path, status_code=303)
861 |
862 |     db_user = user_manager.get_user_by_email(user_data.email)
863 |     if not db_user:
864 |         if request.url.path in ('/admin', '/'):
865 |             return HTMLResponse(content=ADMIN_LOGIN_HTML)
866 |             return RedirectResponse(url='/auth/google?next=' + request.url.path, status_code=303)
867 |
868 |     # Check if user has Google OAuth tokens for working with Google services
869 |     has_google = user_manager.has_google_auth(db_user['id'])
870 |     is_admin_user = bool(db_user.get('is_admin', 0) or db_user.get('role') == 'admin')
871 |
872 |     # If user is logged in via Google and is admin – grant access
873 |     if has_google and is_admin_user:
874 |         return False

```

```

875 |
876 |     # If administrator verified password locally
877 |     if request.cookies.get('admin_password_verified') == 'true
    | ' and is_admin_user:
878 |         return False
879 |
880 |     if request.url.path in ('/admin', '/'):
881 |         return HTMLResponse(content=ADMIN_LOGIN_HTML)
882 |
883 |     return RedirectResponse(url='/auth/google?next=' + request
    | .url.path, status_code=303)
884 |
885 | @app.get('/admin')
886 | async def admin_interface(request: Request):
887 |     """Display the main admin panel page."""
888 |     auth_response = check_admin_auth(request)
889 |     if auth_response:
890 |         return auth_response
891 |     content = read_text_file(webinterface_dir / 'admin' / 'ind
    | ex.html')
892 |     if not content:
893 |         raise HTTPException(status_code=500, detail='Failed to
    | read admin index page')
894 |     return HTMLResponse(content=content)
895 |
896 | @app.post('/admin')
897 | async def admin_interface_post(request: Request):
898 |     """Verify password and set admin authentication cookie."""
899 |     from fastapi.responses import RedirectResponse
900 |     form = await request.form()
901 |     password = form.get('password')
902 |     if password == 'onela':
903 |         response = RedirectResponse(url='/admin', status_code=
    | 303)
904 |         response.set_cookie(key='admin_password_verified', val
    | ue='true', max_age=86400 * 30, httponly=True)
905 |         return response
906 |     else:

```

```

907         return RedirectResponse(url='/admin', status_code=303)
908
909 @app.get('/admin/{full_path:path}')
910 async def admin_static(full_path: str, request: Request):
911     """Serving admin static files with security verification."""
912     auth_response = check_admin_auth(request)
913     if auth_response:
914         return auth_response
915     content = read_text_file(webinterface_dir / 'admin' / full
916                               _path)
917     if not content:
918         raise HTTPException(status_code=404, detail='File not
919 found')
920     return HTMLResponse(content=content)
921
922 @app.get('/tv', response_class=HTMLResponse)
923 async def tv_interface() -> HTMLResponse:
924     """Serving of TV Player HTML page."""
925     content = read_text_file(webinterface_dir / 'tv' / 'index.
926                               html')
927     if not content:
928         raise HTTPException(status_code=500, detail='Failed to
929 read TV index page')
930     return HTMLResponse(content=content)
931
932 @app.get('/tv/{full_path:path}', response_class=HTMLResponse)
933 async def tv_static(full_path: str) -> HTMLResponse:
934     """Serving TV static files."""
935     content = read_text_file(webinterface_dir / 'tv' / full_pa
936                               th)
937     if not content:
938         raise HTTPException(status_code=404, detail='File not
939 found')
940     return HTMLResponse(content=content)
941
942 @app.get('/logs')
943 async def logs_interface():

```

```
938     """Redirect to admin dashboard logs tab."""
939     from fastapi.responses import RedirectResponse
940     return RedirectResponse(url='/admin#tab-logs', status_code
    =303)
941
942 from pydantic import BaseModel
943
944 class FoundryConfigRequest(BaseModel):
945     enabled: bool
946     url: str
947     key: str
948     model: str
949
950 class OllamaConfigRequest(BaseModel):
951     enabled: bool
952     url: str
953     model: str
954
955 @app.get('/api/foundry/config')
956 async def get_foundry_config():
957     import os
958     return {
959         "enabled": getattr(ai_cfg, "use_foundry", False) if ai
    _cfg else False,
960         "url": getattr(ai_cfg, "foundry_base_url", "http://loc
    alhost:54837") if ai_cfg else "http://localhost:54837",
961         "key": os.getenv("FOUNDRY_API_KEY", ""),
962         "model": getattr(ai_cfg, "foundry_model_id", "qwen2.5-
    1.5b") if ai_cfg else "qwen2.5-1.5b"
963     }
964
965 @app.post('/api/foundry/config')
966 async def save_foundry_config(data: FoundryConfigRequest):
967     from dotenv import set_key
968     import os
969     import json
970
971     # Secret goes to .env
```

```

972     env_path = str(__root__ / '.env')
973     set_key(env_path, "FOUNDRY_API_KEY", data.key)
974     os.environ["FOUNDRY_API_KEY"] = data.key
975
976     # Non-secrets go to config.json
977     config_path = __root__ / 'config.json'
978     try:
979         with open(config_path, 'r', encoding='utf-8') as f:
980             cfg_data = json.load(f)
981     except Exception:
982         cfg_data = {}
983
984     if "ai" not in cfg_data:
985         cfg_data["ai"] = {}
986
987     cfg_data["ai"]["use_foundry"] = data.enabled
988     cfg_data["ai"]["foundry_base_url"] = data.url
989     cfg_data["ai"]["foundry_model_id"] = data.model
990
991     with open(config_path, 'w', encoding='utf-8') as f:
992         json.dump(cfg_data, f, indent=2, ensure_ascii=False)
993
994     # Update in-memory config
995     if ai_cfg:
996         ai_cfg.use_foundry = data.enabled
997         ai_cfg.foundry_base_url = data.url
998         ai_cfg.foundry_model_id = data.model
999
1000     return {"status": "ok"}
1001
1002 @app.get('/api/ollama/config')
1003 async def get_ollama_config():
1004     return {
1005         "enabled": getattr(ai_cfg, "use_ollama", False) if ai_
1006         cfg else False,
1007         "url": getattr(ai_cfg, "ollama_base_url", "http://loca
1008         lhost:11434") if ai_cfg else "http://localhost:11434",
1009         "model": getattr(ai_cfg, "ollama_model_id", "llama3.1"

```

```

    ) if ai_cfg else "llama3.1"
1008     }
1009
1010 @app.post('/api/ollama/config')
1011 async def save_ollama_config(data: OllamaConfigRequest):
1012     import json
1013
1014     config_path = __root__ / 'config.json'
1015     try:
1016         with open(config_path, 'r', encoding='utf-8') as f:
1017             cfg_data = json.load(f)
1018     except Exception:
1019         cfg_data = {}
1020
1021     if "ai" not in cfg_data:
1022         cfg_data["ai"] = {}
1023
1024     cfg_data["ai"]["use_ollama"] = data.enabled
1025     cfg_data["ai"]["ollama_base_url"] = data.url
1026     cfg_data["ai"]["ollama_model_id"] = data.model
1027
1028     with open(config_path, 'w', encoding='utf-8') as f:
1029         json.dump(cfg_data, f, indent=2, ensure_ascii=False)
1030
1031     # Update in-memory config
1032     if ai_cfg:
1033         ai_cfg.use_ollama = data.enabled
1034         ai_cfg.ollama_base_url = data.url
1035         ai_cfg.ollama_model_id = data.model
1036
1037     return {"status": "ok"}
1038
1039 class AgyConfigRequest(BaseModel):
1040     enabled: bool
1041     key: str = ""
1042     model: str = "agy-flash"
1043
1044 @app.get('/api/agy/config')

```

```
1045 async def get_agy_config():
1046     import os
1047     return {
1048         "enabled": getattr(ai_cfg, "use_agy", True) if ai_cfg
1049     else True,
1050         "key": os.getenv("AGY_API_KEY", ""),
1051         "model": getattr(ai_cfg, "agy_model_id", "agy-flash")
1052     if ai_cfg else "agy-flash"
1053     }
1054
1055 @app.post('/api/agy/config')
1056 async def save_agy_config(data: AgyConfigRequest):
1057     from dotenv import set_key
1058     import os
1059     import json
1060
1061     # Secret goes to .env
1062     env_path = str(__root__ / '.env')
1063     set_key(env_path, "AGY_API_KEY", data.key)
1064     os.environ["AGY_API_KEY"] = data.key
1065
1066     # Non-secrets go to config.json
1067     config_path = __root__ / 'config.json'
1068     try:
1069         with open(config_path, 'r', encoding='utf-8') as f:
1070             cfg_data = json.load(f)
1071     except Exception:
1072         cfg_data = {}
1073
1074     if "ai" not in cfg_data:
1075         cfg_data["ai"] = {}
1076
1077     cfg_data["ai"]["use_agy"] = data.enabled
1078     cfg_data["ai"]["agy_model_id"] = data.model
1079
1080     with open(config_path, 'w', encoding='utf-8') as f:
1081         json.dump(cfg_data, f, indent=2, ensure_ascii=False)
```

```

1081 |     # Update in-memory config
1082 |     if ai_cfg:
1083 |         ai_cfg.use_agy = data.enabled
1084 |         ai_cfg.agy_model_id = data.model
1085 |
1086 |     return {"status": "ok"}
1087 |
1088 | if __name__ == '__main__':
1089 |     try:
1090 |         args = sys.argv[1:]
1091 |         branch = str(os.getenv('GIT_BRANCH', 'main'))
1092 |         if '--check-update' in args:
1093 |             prompt_and_perform_update(branch=branch)
1094 |             sys.exit(0)
1095 |         if '--check-update-and-run' in args:
1096 |             prompt_and_perform_update(branch=branch)
1097 |         else:
1098 |             # Default startup behavior: perform version check
1099 |             according to config
1100 |             try:
1101 |                 prompt_and_perform_update(branch=branch)
1102 |             except Exception:
1103 |                 pass
1104 |         except Exception:
1105 |             # Non-fatal if version check fails
1106 |             pass
1107 |         # For development: single-process launch without --workers
1108 |         # In production use Run-Unicorn.ps1, which launches:
1109 |         #   uvicorn main:app --workers N (without Telegram bot)
1110 |         #   python bot_runner.py (Telegram bot separatel
1111 | y)
1112 |     port: int = int(getattr(server_cfg, "port", 8000))
1113 |     if not port:
1114 |         logger.error('Port not configured')
1115 |         sys.exit(1)
1116 |
1117 |     cert_file = Path(r'C:\Users\onela\.certs\localhost+2.pem')
1118 |     key_file = Path(r'C:\Users\onela\.certs\localhost+2-key.pe

```



```
m')
1117
1118     ssl_kwargs = {}
1119     use_ssl = getattr(server_cfg, "use_ssl", True)
1120     host = getattr(server_cfg, "host", "0.0.0.0")
1121     reload = bool(getattr(server_cfg, "reload", True))
1122
1123     if use_ssl and cert_file.exists() and key_file.exists():
1124         ssl_kwargs = {'ssl_certfile': str(cert_file), 'ssl_key
1125         file': str(key_file)}
1126         logger.info(f'Server starting https://{host}:{port} (S
1127         SL enabled)')
1128     else:
1129         logger.warning('Starting without HTTPS (SSL disabled o
1130         r certificates not found)')
1131         logger.info(f'3anyck cepbepa http://{host}:{port}')
1132
1133     logger.info(f"Uvicorn autoreload: {'ON' if reload else 'OF
1134     F'}")
1135     uvicorn.run('main:app', host=host, port=port, reload=reloa
1136     d, **ssl_kwargs)
1137
```

manage_tools.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Execution of an external Python script as a su
  bpro
4 # =====
5 # Description:
6 #   Single entry point for all utility scripts and plugins:
7 #
8 # File: manage_tools.py
9 # Project: ai-breadboard
10 # Package: root
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from __future__ import annotations
16
17 import argparse
18 import subprocess
19 import sys
20 from pathlib import Path
21
22 from dotenv import load_dotenv
23
24 import header
25 from header import __root__
26 from src.skills import SkillRegistry
27
28 # =====
29 # UTF-8 Encoding Fix for Windows Console
30 # =====
```

```

31 # Windows console uses legacy encoding (CP1251) by default.
32 # Reconfigure stdout/stderr to use UTF-8 with replacement for
    | invalid characters.
33 # This ensures Cyrillic characters in logs and messages are di
    | splayed correctly.
34
35 if hasattr(sys.stdout, 'reconfigure'):
36     sys.stdout.reconfigure(encoding='utf-8', errors='replace')
37 if hasattr(sys.stderr, 'reconfigure'):
38     sys.stderr.reconfigure(encoding='utf-8', errors='replace')
39
40 # =====
    | =====
41 # Environment Variables Loading
42 # =====
    | =====
43 # Load environment variables from .env file located in project
    | root.
44 # This allows secure storage of API keys and sensitive configu
    | ration.
45
46 load_dotenv(__root__ / '.env')
47
48 def _run_script(script_rel_path: str, extra_args: list[str] =
    | []) -> int:
49     """Execution of an external Python script as a subprocess.
50
51     Invokes a Python interpreter with the target script and fo
    | rwards any
52     additional command-line arguments. The script runs in the
    | project root
53     directory, preserving the environment context.
54
55     Args:
56         script_rel_path (str): Path to the script file relativ
    | e to project root.
57                                     Example: 'plugins/rag/tools/man
    | age_knowledge.py'

```

```

58 |         extra_args (list[str]): List of command-line arguments
    | to pass to the script.
59 |                                     These are appended to the comma
    | nd after the script path.
60 |
61 |     Returns:
62 |         int: Exit code from the subprocess. 0 indicates succes
    | s, non-zero indicates error.
63 |
64 |     Examples:
65 |         >>> _run_script('scripts/dev/update_docs.py', ['--forc
    | e'])
66 |         0
67 |         """
68 |         # Resolve the target script path from project root directo
    | ry
69 |         target_path = __root__ / script_rel_path
70 |
71 |         # Validate that the script file exists before attempting e
    | xecution
72 |         if not target_path.exists():
73 |             print(f"Error: script not found: {target_path}")
74 |             return 1
75 |
76 |         # Build command: python interpreter + script path + any ex
    | tra arguments
77 |         cmd = [sys.executable, str(target_path)]
78 |         if extra_args:
79 |             cmd.extend(extra_args)
80 |
81 |         # Execute the script as a subprocess in the project root d
    | irectory
82 |         result = subprocess.run(cmd, cwd=str(__root__))
83 |         return result.returncode
84 |
85 | def run_knowledge_command(args: argparse.Namespace) -> int:
86 |     """Delegation of knowledge base management commands to the
    | dedicated tool.

```

```

87
88     Routes knowledge management operations (extract, add, init
    ) to the
89     external knowledge management script located in plugins/ra
    g/tools.
90
91     Args:
92         args (argparse.Namespace): Parsed command arguments co
    ntaining:
93         - subcommand (str): The specific knowledge operati
    on to perform.
94         - rest (list[str]): Additional arguments for the s
    ubcommand.
95
96     Returns:
97         int: Exit code from the delegated knowledge management
    script.
98
99     Subcommands:
100         extract: Extract knowledge from chat files
101         add: Add new entries to the knowledge base
102         init: Initialize the knowledge registry
103     """
104     sub = args.subcommand
105     extra = getattr(args, 'rest', [])
106
107     if sub in ('extract', 'add', 'init'):
108         cmd_args = [sub] + extra
109         return _run_script('plugins/rag/tools/manage_knowledge
    .py', cmd_args)
110
111     print(f"Unknown knowledge subcommand: {sub}")
112     return 1
113
114 def run_rag_command(args: argparse.Namespace) -> int:
115     """Management of Retrieval-Augmented Generation (RAG) inde
    x operations.
116

```

```

117 |     Provides direct access to RAG index lifecycle operations i
| including
118 |     rebuilding, reindexing, validation, and status checking.
119 |
120 |     Args:
121 |         args (argparse.Namespace): Parsed command arguments co
| ntaining:
122 |             - subcommand (str): The specific RAG operation to
| perform.
123 |
124 |     Returns:
125 |         int: Exit code indicating operation success (0) or fai
| lure (1).
126 |
127 |     Subcommands:
128 |         rebuild: Full reconstruction of the RAG index from sou
| rce documents
129 |         reindex: Incremental update of existing index
130 |         validate: Verification of index integrity and document
| consistency
131 |         status: Check whether the RAG index exists and is read
| y for use
132 |     """
133 |     sub = args.subcommand
134 |     extra = getattr(args, 'rest', [])
135 |
136 |     if sub == 'rebuild':
137 |         extra = getattr(args, 'rest', [])
138 |         try:
139 |             from src.rag import build_rules_index
140 |             index_path, docs_path = build_rules_index(*extra)
141 |             print(f"RAG index rebuilt successfully: {index_pat
| h}, {docs_path}")
142 |             return 0
143 |         except TypeError:
144 |             # If build_rules_index doesn't accept arguments, c
| all without args
145 |             from src.rag import build_rules_index

```

```

146         index_path, docs_path = build_rules_index()
147         print(f"RAG index rebuilt successfully: {index_path}, {docs_path}")
148         return 0
149     except Exception as e:
150         print(f"Error rebuilding RAG: {e}")
151         return 1
152     if sub == 'status':
153         try:
154             from header import __root__
155             idx_file = __root__ / 'tmp' / 'rag' / 'rules.index'
156             print(f"Core RAG status: {'Ready' if idx_file.exists() else 'Not built'}")
157             return 0
158         except Exception as e:
159             print(f"Error checking RAG status: {e}")
160             return 1
161
162     print(f"Unknown rag subcommand: {sub}")
163     return 1
164
165 def run_docs_command(args: argparse.Namespace) -> int:
166     """Delegation of documentation management operations to external scripts.
167
168     Handles project documentation updates and maintenance through
169     dedicated development scripts.
170
171     Args:
172         args (argparse.Namespace): Parsed command arguments containing:
173         - subcommand (str): The documentation operation to perform.
174         - rest (list[str]): Additional arguments for the operation.
175

```

```

176     Returns:
177         int: Exit code from the delegated documentation script
178     .
179     Subcommands:
180         update: Run documentation update pipeline and regenera
181         tion scripts
182         """
183         sub = args.subcommand
184         extra = getattr(args, 'rest', [])
185         if sub == 'update':
186             return _run_script('scripts/dev/update_docs.py', extra
187         )
188         print(f"Unknown docs subcommand: {sub}")
189         return 1
190
191 def run_skills_command(args: argparse.Namespace) -> int:
192     """Unified skills catalog management for AI agents.
193
194     Discovers, searches, and exports AI capabilities (skills)
195     from
196     the universal skills registry. Supports multiple discovery
197     roots
198     (.agents/skills, .gemini/skills).
199
200     The SkillRegistry provides a portable interface for AI age
201     nts to
202     understand available capabilities without hardcoding refer
203     ences.
204
205     Args:
206         args (argparse.Namespace): Parsed command arguments co
207         ntaining:
208             - subcommand (str): The skills operation to perfor
209             m.
210             - query (str): Search terms for 'search' subcomman

```



```

d.
205         - name (str): Skill name for 'show' and 'export' s
ubcommands.
206         - without_instructions (bool): Flag to exclude ins
tructions when exporting.
207
208     Returns:
209         int: Exit code indicating operation success (0) or fai
lure (1).
210
211     Subcommands:
212         list: Display all discovered skills with their descrip
tions
213         search: Find skills matching query string by name or d
escription
214         show: Print full Markdown instructions for a specific
skill
215         export: Generate portable JSON contract for skill inte
gration
216     """
217     registry = SkillRegistry()
218     sub = args.subcommand
219
220     if sub == 'list':
221         for skill in registry.discover():
222             print(f"{skill.name}\t{skill.description}")
223         return 0
224
225     if sub == 'search':
226         for skill in registry.search(args.query):
227             print(f"{skill.name}\t{skill.description}")
228         return 0
229
230     if sub in ('show', 'export'):
231         try:
232             if sub == 'show':
233                 skill = registry.get(args.name)
234                 print(skill.prompt())

```

```

235         else:
236             print(registry.export_json(args.name, include_
instructions=not args.without_instructions))
237             return 0
238         except KeyError as error:
239             print(f"Error: {error}")
240             return 1
241
242     print(f"Unknown skills subcommand: {sub}")
243     return 1
244
245 def main() -> int:
246     """Primary entry point for the universal CLI management sy
stem.
247
248     Initializes the argument parser with all available command
groups and
249     dispatches incoming commands to their respective handlers.
250
251     Returns:
252         int: Exit code indicating overall command execution st
atus:
253             0 - Success (command executed properly)
254             1 - Error (unknown command, missing arguments, or
handler failure)
255
256     Command Structure:
257         manage_tools.py <command> [<subcommand>] [<args>]
258
259     Available Commands:
260         knowledge - Knowledge base extraction and management
261         rag       - RAG index operations (rebuild, status)
262         docs      - Documentation updates and generation
263         skills    - Skills registry operations (list, search,
show, export)
264         assist    - Assistant process management
265     """
266     # Initialize the argument parser with program metadata and

```

```

    formatting options
267     parser = argparse.ArgumentParser(
268         prog='manage_tools.py',
269         description='Universal CLI for managing ai-breadboard
project tools',
270         formatter_class=argparse.RawDescriptionHelpFormatter,
271         epilog=''
272 Examples:
273     py manage_tools.py rag rebuild                # re
build RAG index
274     py manage_tools.py knowledge extract --file chat.md    # ex
tract knowledge
275     py manage_tools.py skills list                    # li
st all available skills
276 '''
277     )
278
279     subparsers = parser.add_subparsers(dest='command', help='C
ommands')
280
281     # =====
=====
282     # Knowledge Management Subparser
283     # =====
=====
284     # Commands for extracting, adding, and initializing knowle
dge base entries
285     # from chat files and documentation sources.
286
287     knowledge_parser = subparsers.add_parser('knowledge', help
='Knowledge management')
288     knowledge_subparsers = knowledge_parser.add_subparsers(des
t='subcommand', help='Subcommands')
289     knowledge_extract = knowledge_subparsers.add_parser('extra
ct', help='Extract knowledge from chats')
290     knowledge_extract.add_argument('rest', nargs=argparse.REMA
INDER, help='Arguments')
291     knowledge_add = knowledge_subparsers.add_parser('add', hel

```

```

p='Add new entry to knowledge')
292     knowledge_add.add_argument('rest', nargs=argparse.REMAINDER, help='Arguments')
293     knowledge_subparsers.add_parser('init', help='Initialize knowledge registry')
294
295     # =====
296     # RAG (Retrieval-Augmented Generation) Index Subparser
297     # =====
298     # Commands for building, validating, and checking status of RAG indexes
299     # used for semantic search and AI-powered knowledge retrieval.
300
301     rag_parser = subparsers.add_parser('rag', help='RAG index management')
302     rag_subparsers = rag_parser.add_subparsers(dest='subcommand', help='Subcommands')
303     rag_rebuild = rag_subparsers.add_parser('rebuild', help='Full rebuild of RAG index')
304     rag_rebuild.add_argument('rest', nargs=argparse.REMAINDER, help='Rebuild options')
305     rag_subparsers.add_parser('reindex', help='Reindex knowledge base')
306     rag_subparsers.add_parser('validate', help='Validate knowledge base files')
307     rag_subparsers.add_parser('status', help='Check RAG index status')
308
309     # =====
310     # Documentation Management Subparser
311     # =====
312     # Commands for generating, updating, and maintaining project documentation

```

```

313     # including auto-generated API docs and user guides.
314
315     docs_parser = subparsers.add_parser('docs', help='Document
ation management')
316     docs_subparsers = docs_parser.add_subparsers(dest='subcomm
and', help='Subcommands')
317
318     # =====
=====
319     # Skills Registry Subparser
320     # =====
=====
321     # Universal skills catalog for AI agents. Discovers capabi
lities from
322     # .agents/skills and .gemini/skills directories and provid
es
323     # portable JSON contracts for agent integration.
324
325     skills_parser = subparsers.add_parser('skills', help='Univ
ersal skills registry')
326     skills_subparsers = skills_parser.add_subparsers(dest='sub
command', help='Subcommands')
327     skills_subparsers.add_parser('list', help='List discovered
skills')
328     skills_search = skills_subparsers.add_parser('search', hel
p='Search skills by name or description')
329     skills_search.add_argument('query', help='Search terms')
330     skills_show = skills_subparsers.add_parser('show', help='P
rint Markdown instructions')
331     skills_show.add_argument('name', help='Skill name')
332     skills_export = skills_subparsers.add_parser('export', hel
p='Export a portable JSON skill contract')
333     skills_export.add_argument('name', help='Skill name')
334     skills_export.add_argument('--without-instructions', actio
n='store_true', help='Exclude Markdown instructions')
335
336     # =====
=====

```

```

337     # Assistant CLI Subparser
338     # =====
339     # Gateway to the main assistant CLI for process management
340     # including
341     # start, stop, status queries, and provider configuration.
342     assist_parser = subparsers.add_parser('assist', help='Assi
343     stant management (start, stop, status, providers, etc.)')
344     assist_parser.add_argument('rest', nargs=argparse.REMAINDE
345     R, help='Arguments for assist CLI')
346     # =====
347     # Command Dispatch
348     # =====
349     # Parse arguments and route to the appropriate handler bas
350     ed on command type.
351
352     args = parser.parse_args()
353
354     # Display help when no command is provided
355     if not args.command:
356         parser.print_help()
357         return 0
358
359     # Special handling for 'assist' command - forwards to dedi
360     cated assist_cli
361     if args.command == 'assist':
362         from scripts.dev import assist_cli
363         sys.argv = ['assist'] + getattr(args, 'rest', [])
364         return assist_cli.main()
365
366     # Display help when no subcommand is provided for commands
367     that require one
368     if not getattr(args, 'subcommand', ''):
369         parser.print_help()

```

```
366 |         return 0
367 |
368 |     # Command-to-handler mapping for primary command groups
369 |     dispatch = {
370 |         'knowledge': run_knowledge_command,
371 |         'rag': run_rag_command,
372 |         'docs': run_docs_command,
373 |         'skills': run_skills_command,
374 |     }
375 |
376 |     # Resolve and execute the appropriate command handler
377 |     handler = dispatch.get(args.command)
378 |     if not handler:
379 |         parser.print_help()
380 |         return 1
381 |
382 |     return handler(args)
383 |
384 | if __name__ == '__main__':
385 |     sys.exit(main())
386 |
```

mkdocs.yml

```
1 site_name: AI-Breadboard
2 site_description: Interactive AI Model Testbench & Breadboard
3 site_url: !ENV READTHEDOCS_CANONICAL_URL
4 repo_url: https://github.com/hypo69/AI-Breadboard
5 edit_uri: edit/master/docs/
6
7 # Theme
8 theme:
9   name: material
10  language: en
11  palette:
12    primary: indigo
13    accent: indigo
14  favicon: assets/favicon.png
15  logo: assets/logo.png
16  features:
17    - navigation.instant
18    - navigation.tracking
19    - navigation.sections
20    - toc.integrate
21    - content.code.copy
22
23 # Navigation: English only
24 nav:
25   - Home: index.md
26   - Getting Started: getting-started.md
27   - API:
28     - Overview: api.md
29
30 # Plugins
31 plugins:
32   - search:
33     lang:
34       - en
35   - mkdocstrings:
36     handlers:
```



```
37 |         python:
38 |             paths:
39 |                 - .
40 |             options:
41 |                 docstring_style: google
42 |                 show_source: true
43 |                 show_root_heading: true
44 |                 members_order: source
45 |                 show_signature_annotations: true
46 |
47 | # Markdown extensions
48 | markdown_extensions:
49 |     - pymdownx.highlight:
50 |         anchor_linenums: true
51 |     - pymdownx.inlinehilite
52 |     - pymdownx.snippets
53 |     - pymdownx.superfences
54 |
55 | # Custom CSS / JS
56 | extra_css:
57 |     - stylesheets/extra.css
58 |
59 | extra_javascript:
60 |     - js/chat.js
61 |
62 | # Site metadata
63 | extra:
64 |     version: 1.0.0
65 |     authors:
66 |         - hypo69
67 |     license: MIT
68 |     repository: github
69 |
70 | # Validation
71 | strict: true
72 |
73 |
```

index.html

```
1 <!DOCTYPE html>
2 <!--
3 =====
4 |=====
5 Process Name: Simple Assistant Chat Interface
6 |=====
7 |=====
8 Description:
9 |
10 | Focused browser interface for conversational requests sent
11 | to the FastAPI chat endpoint.
12 | Provides a compact message stream, responsive composer, an
13 | d accessible status feedback.
14 |
15 |
16 File: index.html
17 Project: ai-breadboard
18 Module: Simple Assistant
19 Author: hypo69
20 Copyright: (C) 2026 hypo69
21 |=====
22 |=====
23 -->
24 <html lang="ru">
25 <head>
26 | <meta charset="UTF-8">
27 | <meta name="viewport" content="width=device-width, initial-s
28 | cale=1.0">
29 | <meta name="theme-color" content="#f4f1ea">
30 | <title>Simple Assistant</title>
31 | <link rel="stylesheet" href="./style.css">
32 </head>
33 <body>
34 | <main class="app-shell">
35 | | <header class="app-header">
36 | | | <div class="brand-mark" aria-hidden="true">SA</div>
37 | | | <div>
38 | | | | <p class="eyebrow">AI ASSISTANT</p>
```

```

31         <h1>Simple Assistant</h1>
32     </div>
33     <span class="connection-state" id="connection-state"><span
class="state-dot"></span><span id="state-label">&#1043;&#10
86;&#1090;&#1086;&#1074;</span></span>
34 </header>
35
36     <section class="chat-panel" aria-label="&#1063;&#1072;&#10
90;">
37         <div class="conversation" id="conversation" aria-live="p
olite">
38             <div class="welcome-message" id="welcome-message">
39                 <span class="welcome-kicker">&#1042;&#1072;&#1096; &
#1083;&#1080;&#1095;&#1085;&#1099;&#1081; &#1076;&#1080;&#1072
;&#1083;&#1086;&#1075;</span>
40                 <h2>&#1063;&#1077;&#1084; &#1084;&#1086;&#1075;&#109
1; &#1087;&#1086;&#1084;&#1086;&#1095;&#1100;?</h2>
41                 <p>&#1053;&#1072;&#1087;&#1080;&#1096;&#1080;&#1090;
&#1077; &#1074;&#1086;&#1087;&#1088;&#1086;&#1089; &#1080;&#10
83;&#1080; &#1079;&#1072;&#1076;&#1072;&#1095;&#1091;, &#1080;
&#1084;&#1099; &#1085;&#1072;&#1095;&#1085;&#1105;&#1084; &#1
089; &#1087;&#1086;&#1090;&#1086;&#1082;&#1086;&#1074;&#1086;&
#1075;&#1086; &#1086;&#1090;&#1074;&#1077;&#1090;&#1072;. </p>
42             </div>
43         </div>
44
45         <div class="composer-wrap">
46             <form class="composer" id="chat-form">
47                 <label class="sr-only" for="message-input">&#1057;&#
1086;&#1086;&#1073;&#1097;&#1077;&#1085;&#1080;&#1077;</label>
48                 <textarea id="message-input" rows="1" maxlength="120
00" placeholder="&#1053;&#1072;&#1087;&#1080;&#1096;&#1080;&#1
090;&#1077; &#1089;&#1086;&#1086;&#1073;&#1097;&#1077;&#1085;&
#1080;&#1077;..." autocomplete="off"></textarea>
49                 <button class="send-button" id="send-button" type="s
ubmit" aria-label="&#1054;&#1090;&#1087;&#1088;&#1072;&#1074;&
#1080;&#1090;&#1100;" title="&#1054;&#1090;&#1087;&#1088;&#107
2;&#1074;&#1080;&#1090;&#1100;">

```

```

50         <span aria-hidden="true">&#8593;</span>
51     </button>
52 </form>
53     <div class="composer-footer">
54         <span id="hint-text">Enter &#8212; &#1086;&#1090;&#1
087;&#1088;&#1072;&#1074;&#1080;&#1090;&#1100; &#183; Shift +
Enter &#8212; &#1085;&#1086;&#1074;&#1072;&#1103; &#1089;&#109
0;&#1088;&#1086;&#1082;&#1072;</span>
55         <button class="clear-button" id="clear-button" type=
"button">&#1054;&#1095;&#1080;&#1089;&#1090;&#1080;&#1090;&#11
00;</button>
56     </div>
57 </div>
58 </section>
59 </main>
60 <script src="./main.js" defer></script>
61 </body>
62 </html>
63

```

main.js

```
1  /**
2   * =====
3   * Process Name: Simple Assistant Chat Controller
4   * =====
5   * Description:
6   *   Coordinates the browser chat state, history serialization
7   *   Keeps model and developer controls outside the focused us
8   *
9   * File: main.js
10  * Project: ai-breadboard
11  * Module: Simple Assistant
12  * Author: hypo69
13  * Copyright: (C) 2026 hypo69
14  * =====
15  */
16
17 const elements = {
18   form: document.querySelector('#chat-form'),
19   input: document.querySelector('#message-input'),
20   sendButton: document.querySelector('#send-button'),
21   clearButton: document.querySelector('#clear-button'),
22   conversation: document.querySelector('#conversation'),
23   welcome: document.querySelector('#welcome-message'),
24   state: document.querySelector('#connection-state'),
25   stateLabel: document.querySelector('#state-label'),
26   hint: document.querySelector('#hint-text')
27 };
28
29 const state = { history: [], controller: null, isStreaming: fa
30   lse };
```

```
31 function decodeLabel(label) {
32   const decoder = document.createElement('textarea');
33   decoder.innerHTML = label;
34   return decoder.value;
35 }
36
37 function setStatus(label, mode = '') {
38   elements.state.className = `connection-state ${mode}`.trim()
39   ;
39   elements.stateLabel.textContent = decodeLabel(label);
40 }
41
42 function scrollConversation() {
43   elements.conversation.scrollTop = elements.conversation.scrollHeight;
44 }
45
46 function addMessage(role, text) {
47   elements.welcome.hidden = true;
48   const message = document.createElement('article');
49   message.className = `message ${role}`;
50   const label = document.createElement('span');
51   label.className = 'message-label';
52   label.textContent = role === 'user' ? 'You' : 'AI';
53   const bubble = document.createElement('div');
54   bubble.className = 'message-bubble';
55   bubble.textContent = text;
56   message.append(role === 'user' ? bubble : label, role === 'user' ? label : bubble);
57   elements.conversation.append(message);
58   scrollConversation();
59   return bubble;
60 }
61
62 function addStatus(text) {
63   const status = document.createElement('div');
64   status.className = 'status-line';
65   status.textContent = text;
```

```

66 | elements.conversation.append(status);
67 | scrollConversation();
68 | return status;
69 | }
70 |
71 | function setStreamingMode(isStreaming) {
72 |     state.isStreaming = isStreaming;
73 |     elements.sendButton.classList.toggle('stop', isStreaming);
74 |     elements.sendButton.querySelector('span').textContent = isSt
    reaming ? '×' : '↑';
75 |     elements.sendButton.setAttribute('aria-label', isStreaming ?
    'Stop' : 'Send');
76 |     elements.sendButton.title = isStreaming ? 'Stop' : 'Send';
77 |     elements.input.disabled = isStreaming;
78 |     elements.clearButton.disabled = isStreaming;
79 | }
80 |
81 | async function readStream(response, assistantBubble, statusNod
    e) {
82 |     const reader = response.body.getReader();
83 |     const decoder = new TextDecoder();
84 |     let buffer = '';
85 |     let answer = '';
86 |
87 |     while (true) {
88 |         const { value, done } = await reader.read();
89 |         if (done) break;
90 |         buffer += decoder.decode(value, { stream: true });
91 |         const lines = buffer.split('\n');
92 |         buffer = lines.pop() || '';
93 |         for (const line of lines) {
94 |             const data = parseEvent(line);
95 |             if (!data) continue;
96 |             if (data.error) throw new Error(data.error);
97 |             if (data.status) statusNode.textContent = data.status;
98 |             if (data.text) {
99 |                 answer += data.text;
100 |                 assistantBubble.textContent = answer;

```

```

101 |         scrollConversation();
102 |     }
103 | }
104 | }
105 | return answer;
106 | }
107 |
108 | function parseEvent(line) {
109 |     const trimmed = line.trim();
110 |     if (!trimmed.startsWith('data: ')) return {};
111 |     try {
112 |         return JSON.parse(trimmed.slice(6));
113 |     } catch {
114 |         return {};
115 |     }
116 | }
117 |
118 | async function sendMessage(message) {
119 |     state.controller = new AbortController();
120 |     setStreamingMode(true);
121 |     setStatus('&#1044;&#1091;&#1084;&#1072;&#1102;...', 'busy');
122 |     addMessage('user', message);
123 |     const assistantBubble = addMessage('assistant', '');
124 |     const statusNode = addStatus('&#1055;&#1086;&#1076;&#1075;&#1086;&#1090;&#1082;&#1072; &#1086;&#1090;&#1074;&#1077;&#1090;&#1072;...');
125 |
126 |     try {
127 |         const response = await fetch('/api/chat', {
128 |             method: 'POST',
129 |             headers: { 'Content-Type': 'application/json' },
130 |             credentials: 'same-origin',
131 |             body: JSON.stringify({ message, history: state.history,
generation_config: {} }),
132 |             signal: state.controller.signal
133 |         });
134 |         if (!response.ok) throw new Error(`HTTP ${response.status}
`);

```



```
135     const answer = await readStream(response, assistantBubble,
    statusNode);
136     state.history.push({ role: 'user', parts: [message] }, { r
    ole: 'model', parts: [answer] });
137     statusNode.remove();
138     setStatus('&#1043;&#1086;&#1090;&#1086;&#1074;');
139   } catch (error) {
140     statusNode.textContent = error.name === 'AbortError' ? 'St
    opped' : `Error: ${error.message}`;
141     setStatus('Error', 'error');
142   } finally {
143     state.controller = null;
144     setStreamingMode(false);
145     elements.input.focus();
146   }
147 }
148
149 elements.form.addEventListener('submit', (event) => {
150   event.preventDefault();
151   if (state.isStreaming) {
152     state.controller?.abort();
153     return;
154   }
155   const message = elements.input.value.trim();
156   if (!message) return;
157   elements.input.value = '';
158   elements.input.style.height = 'auto';
159   sendMessage(message);
160 });
161
162 elements.input.addEventListener('input', () => {
163   elements.input.style.height = 'auto';
164   elements.input.style.height = `${Math.min(elements.input.scr
    ollHeight, 150)}px`;
165 });
166
167 elements.input.addEventListener('keydown', (event) => {
168   if (event.key === 'Enter' && !event.shiftKey) {
```

```
169 |     event.preventDefault();
170 |     elements.form.requestSubmit();
171 | }
172 | });
173 |
174 | elements.clearButton.addEventListener('click', () => {
175 |   if (state.isStreaming) return;
176 |   state.history = [];
177 |   elements.conversation.replaceChildren(elements.welcome);
178 |   elements.welcome.hidden = false;
179 |   setStatus('Готов');
180 | });
181 |
182 | setStatus('Готов');
183 |
```

style.css

```
1  /*
2  =====
3  |
4  |
5  |
6  |
7  |
8  |
9  |
10 |
11 |
12 |
13 |
14 |
15 |
16 |
17 |
18 |
19 |
20 |
21 |
22 |
23 |
24 |
25 |
26 |
27 |
28 |
29 |
30 |
31 |
```

Process Name: Simple Assistant Chat Styling

Description:

Responsive visual system for the focused Simple Assistant conversation surface.

Uses restrained colors, stable controls, and readable message spacing.

File: style.css

Project: ai-breadboard

Module: Simple Assistant

Author: hypo69

Copyright: (C) 2026 hypo69

=====

*/

```
:root {
  --canvas: #f4f1ea;
  --surface: #fffdf8;
  --surface-soft: #f8f5ee;
  --ink: #24231f;
  --muted: #77736a;
  --line: #ded9ce;
  --accent: #1d675b;
  --accent-dark: #124a42;
  --user: #e3eee9;
  --assistant: #f1eee7;
  --danger: #a34842;
  font-family: "Trebuchet MS", "Segoe UI", sans-serif;
  color: var(--ink);
  background: var(--canvas);
```

```

32 | }
33 |
34 | * { box-sizing: border-box; }
35 |
36 | body {
37 |     min-width: 320px;
38 |     min-height: 100vh;
39 |     margin: 0;
40 |     background: var(--canvas);
41 |     background-image: linear-gradient(135deg, rgba(255, 255, 255
    | , .55), transparent 45%), repeating-linear-gradient(0deg, rgba
    | (29, 103, 91, .025) 0 1px, transparent 1px 5px);
42 | }
43 |
44 | .app-shell { width: min(100% - 32px, 920px); margin: 0 auto; padding: 42px 0 28px; }
45 | .app-header { display: flex; align-items: center; gap: 13px; margin-bottom: 18px; }
46 | .brand-mark { display: grid; place-items: center; width: 42px; height: 42px; border-radius: 12px; background: var(--accent); color: #ffffdf8; font: 700 13px Georgia, serif; letter-spacing: 1px; }
47 | .eyebrow { margin: 0 0 2px; color: var(--accent); font-size: 10px; font-weight: 700; letter-spacing: 2px; }
48 | h1, h2, p { margin: 0; }
49 | h1 { font: 600 25px Georgia, serif; letter-spacing: 0; }
50 | .connection-state { display: inline-flex; align-items: center; gap: 7px; margin-left: auto; color: var(--muted); font-size: 12px; }
51 | .state-dot { width: 7px; height: 7px; border-radius: 50%; background: var(--accent); }
52 | .connection-state.busy .state-dot { background: #bf7b2e; animation: pulse 1.2s infinite; }
53 | .connection-state.error .state-dot { background: var(--danger); }
54 |
55 | .chat-panel { display: flex; flex-direction: column; min-height: min(720px, calc(100vh - 130px)); overflow: hidden; border:

```

```

    1px solid var(--line); border-radius: 18px; background: var(--
    surface); box-shadow: 0 18px 50px rgba(55, 50, 39, .08); }
56 .conversation { flex: 1; overflow-y: auto; padding: clamp(24px
    , 5vw, 58px); scroll-behavior: smooth; }
57 .welcome-message { max-width: 560px; padding-top: 8vh; }
58 .welcome-kicker { color: var(--accent); font-size: 11px; font-
    weight: 700; letter-spacing: 1.5px; text-transform: uppercase;
    }
59 .welcome-message h2 { margin-top: 13px; font: 500 clamp(30px,
    5vw, 46px)/1.05 Georgia, serif; }
60 .welcome-message p { max-width: 390px; margin-top: 16px; color
    : var(--muted); font-size: 15px; line-height: 1.65; }
61 .message { display: flex; gap: 12px; max-width: 760px; margin:
    0 auto 25px; animation: reveal .28s ease both; }
62 .message.user { justify-content: flex-end; }
63 .message-bubble { max-width: min(80%, 650px); padding: 14px 17
    px; border: 1px solid var(--line); border-radius: 15px; white-
    space: pre-wrap; overflow-wrap: anywhere; font-size: 15px; lin
    e-height: 1.6; }
64 .message.user .message-bubble { border-color: transparent; bor
    der-bottom-right-radius: 4px; background: var(--user); }
65 .message.assistant .message-bubble { border-bottom-left-radius
    : 4px; background: var(--assistant); }
66 .message-label { align-self: flex-end; color: var(--muted); fo
    nt-size: 10px; letter-spacing: 1px; text-transform: uppercase;
    }
67 .status-line { margin: 0 auto 18px; color: var(--muted); font-
    size: 12px; font-style: italic; }
68
69 .composer-wrap { padding: 15px clamp(16px, 4vw, 42px) 18px; bo
    rder-top: 1px solid var(--line); background: rgba(248, 245, 23
    8, .72); }
70 .composer { display: flex; align-items: flex-end; gap: 10px; m
    ax-width: 760px; margin: 0 auto; padding: 8px 8px 8px 15px; bo
    rder: 1px solid #cfc9bc; border-radius: 14px; background: var(
    --surface); transition: border-color .2s, box-shadow .2s; }
71 .composer:focus-within { border-color: var(--accent); box-shad
    ow: 0 0 0 3px rgba(29, 103, 91, .1); }

```

```

72 | textarea { width: 100%; max-height: 150px; resize: none; border: 0; outline: 0; background: transparent; color: var(--ink);
    | font: inherit; font-size: 15px; line-height: 1.5; }
73 | textarea::placeholder { color: #aaa397; }
74 | .send-button { flex: 0 0 36px; width: 36px; height: 36px; border: 0; border-radius: 10px; background: var(--accent); color:
    | white; cursor: pointer; font-size: 21px; line-height: 1; transition: background .2s, transform .2s; }
75 | .send-button:hover { background: var(--accent-dark); transform: translateY(-1px); }
76 | .send-button.stop { background: var(--danger); }
77 | .composer-footer { display: flex; justify-content: space-between; gap: 12px; max-width: 760px; margin: 8px auto 0; color: var(--muted); font-size: 11px; }
78 | .clear-button { padding: 0; border: 0; background: transparent; color: var(--muted); cursor: pointer; font: inherit; }
79 | .clear-button:hover { color: var(--danger); }
80 | .sr-only { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0, 0, 0, 0); white-space: nowrap; }
81 |
82 | @keyframes reveal { from { opacity: 0; transform: translateY(6px); } to { opacity: 1; transform: translateY(0); } }
83 | @keyframes pulse { 50% { opacity: .35; } }
84 | @media (max-width: 600px) { .app-shell { width: min(100% - 18px, 920px); padding-top: 20px; } .chat-panel { min-height: calc(100vh - 90px); border-radius: 14px; } .conversation { padding: 24px 16px; } .welcome-message { padding-top: 6vh; } .message-bubble { max-width: 88%; } .composer-footer { font-size: 10px; } }

```

85 |

finetuning_dataset_generator.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Fine-Tuning Dataset Generator and Validator
4 # =====
5 # Description:
6 #   Educational utility script for generating and validating i
7 #   nstruction datasets
8 #   in JSONL format for LoRA / QLoRA training on AI Breadboard
9 #   .
10 #
11 # File: SANDBOX/finetuning_dataset_generator.py
12 # Project: aibreadboard
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 import json
18 from pathlib import Path
19 from typing import Dict, List, Any
20 from src.logger import logger
21
22 def generate_sample_dataset(output_path: Path) -> int:
23     """Generate golden reference dataset for instruction fine-
24     tuning.
25
26     Args:
27         output_path (Path): Path to output JSONL file.
28
29     Returns:
30         int: Number of generated sample dialogues.
31         """
32     samples: List[Dict[str, Any]] = [
```

```

31         {
32             "messages": [
33                 {"role": "system", "content": "You are a precise AI Breadboard routing assistant."},
34                 {"role": "user", "content": "Route to DirectML ONNX model."},
35                 {"role": "assistant", "content": "Prefix target: onnx:qwen2.5-coder with DirectMLExecutionProvider."}
36             ]
37         },
38         {
39             "messages": [
40                 {"role": "system", "content": "You are an AI Breadboard RAG search specialist."},
41                 {"role": "user", "content": "What is the similarity threshold for high relevance in Gemini embeddings?"},
42                 {"role": "assistant", "content": "Cosine similarity score >= 0.60 indicates high topical relevance suitable for context injection."}
43             ]
44         },
45         {
46             "messages": [
47                 {"role": "system", "content": "You are a model optimization engineer."},
48                 {"role": "user", "content": "Explain constant folding in Microsoft Olive."},
49                 {"role": "assistant", "content": "Constant folding computes static operations at graph compilation time, eliminating runtime computation."}
50             ]
51         }
52     ]
53
54     output_path.parent.mkdir(parents=True, exist_ok=True)
55     with open(output_path, "w", encoding="utf-8") as f:
56         for item in samples:
57             f.write(json.dumps(item, ensure_ascii=False) + "\n

```



```

    ")
58
59     logger.info(f"Dataset generated: {len(samples)} samples wr
itten to {output_path}")
60     return len(samples)
61
62
63 def validate_dataset(dataset_path: Path) -> bool:
64     """Validate JSONL dataset syntax and conversational role s
chemas.
65
66     Args:
67         dataset_path (Path): Path to dataset file.
68
69     Returns:
70         bool: True if dataset is valid, False otherwise.
71     """
72     if not dataset_path.exists():
73         logger.error(f"Dataset file not found: {dataset_path}"
74 )
75         return False
76
77     valid_count: int = 0
78     with open(dataset_path, "r", encoding="utf-8") as f:
79         for idx, line in enumerate(f, start=1):
80             line_str = line.strip()
81             if not line_str:
82                 continue
83             try:
84                 data = json.loads(line_str)
85             except Exception as e:
86                 logger.error(f"Line {idx}: JSON parse error: {
e}")
87                 return False
88
89             if "messages" not in data or not isinstance(data["
messages"], list):
90                 logger.error(f"Line {idx}: Missing 'messages'

```

```

list")
90         return False
91
92         roles = [msg.get("role", "") for msg in data["mess
ages"] if isinstance(msg, dict)]
93         if "user" not in roles or "assistant" not in roles
:
94             logger.error(f"Line {idx}: Must contain at lea
st 'user' and 'assistant' roles")
95             return False
96
97         valid_count += 1
98
99     logger.info(f"Validation successful: {valid_count} valid d
ialogues found in {dataset_path}")
100     return True
101
102
103 if __name__ == "__main__":
104     target_file = Path(__file__).parent / "sample_finetuning_d
ataset.jsonl"
105     generate_sample_dataset(target_file)
106     validate_dataset(target_file)
107

```

search_test.html

```
1 <!DOCTYPE html><html lang="en"><head>
2   <link rel="canonical" href="https://duckduckgo.com/">
3   <meta http-equiv="content-type" content="text/html; charse
| t=UTF-8">
4   <meta name="viewport" content="width=device-width, initial
| -scale=1.0, maximum-scale=3.0, user-scalable=1">
5   <meta name="referrer" content="origin">
6   <title>
7     DuckDuckGo
8   </title>
9   <link rel="preload" href="/font/ProximaNova-Reg-webfont.wo
| ff2" as="font" type="font/woff2" crossorigin="anonymous">
10  <link rel="preload" href="/font/ProximaNova-Sbold-webfont.
| woff2" as="font" type="font/woff2" crossorigin="anonymous">
11  <link title="DuckDuckGo (Lite)" type="application/opensear
| chdescription+xml" rel="search" href="//duckduckgo.com/opensea
| rch_lite_v2.xml">
12  <link rel="icon" href="//duckduckgo.com/favicon.ico" type=
| "image/x-icon">
13  <link id="icon60" rel="apple-touch-icon" href="//duckduckg
| o.com/assets/icons/meta/DDG-iOS-icon_60x60.png?v=2">
14  <link id="icon76" rel="apple-touch-icon" sizes="76x76" hre
| f="//duckduckgo.com/assets/icons/meta/DDG-iOS-icon_76x76.png?v
| =2">
15  <link id="icon120" rel="apple-touch-icon" sizes="120x120"
| href="//duckduckgo.com/assets/icons/meta/DDG-iOS-icon_120x120.
| png?v=2">
16  <link id="icon152" rel="apple-touch-icon" sizes="152x152"
| href="//duckduckgo.com/assets/icons/meta/DDG-iOS-icon_152x152.
| png?v=2">
17  <link rel="image_src" href="//duckduckgo.com/assets/icons/
| meta/DDG-icon_256x256.png">
18  <link rel="stylesheet" media="handheld, all" href="//duckd
| uckgo.com/dist/l.304bf63bbd053ee46b28.css" type="text/css">
19  <link rel="stylesheet" media="handheld, all" href="//duckd
| uckgo.com/dist/lc.844e8ff9baa78da64b59.css" type="text/css">
```

```
20 </head>
21
22 <body>
23     <a name="top"></a>
24     <center id="lite_wrapper">
25         <br>
26         <a class="header-url" href="/html/">
27             <span class="header">DuckDuckGo</span>
28         </a>
29         <br><br>
30
31
32         <iframe name="ifr" width="0" height="0" border="0" cla
33 ss="hidden"></iframe>
34         <form id="img-form" action="//duckduckgo.com/anomaly.j
35 s?sv=html&cc=sre&ti=1785847303&gk=d4cd0dabcf4caa22
36 ad92fab40844c786&p=1110e90e5492ddc53563e7cad90059ca-ecd442
37 7bdc049b77fc3ee1fe3bfa506c-d2bce2444455b9344cb0695100022af8-20
38 e9ac858806321167f5fe115d58890c-f8def5d7f8a3e5486f88f602c0b4e4e
39 7-b138a70fd08b09b5bd5d0685142a9dac-a756d136d7827e3e97a116faa63
40 8979f-1b4940d754398e54389e010ea043e7d6-14c149ba3e93eeb6385c5e2
41 e6ab73ae6&q=карточка сериала сваты&o=rJPoSthceZoxELpkz
42 YNe0Z04kmXJw4EpVWpkS7u%2B37TZiA9g3cuUoBpjOpZr1yZt%0A&r=euw
43 " target="ifr" method="POST"></form>
44         <form id="challenge-form" action="//duckduckgo.com/ano
45 maly.js?sv=html&cc=sre&st=1785847303&gk=d4cd0dabcf
46 4caa22ad92fab40844c786&p=1110e90e5492ddc53563e7cad90059ca-
47 ecd4427bdc049b77fc3ee1fe3bfa506c-d2bce2444455b9344cb0695100022
48 af8-20e9ac858806321167f5fe115d58890c-f8def5d7f8a3e5486f88f602c
49 0b4e4e7-b138a70fd08b09b5bd5d0685142a9dac-a756d136d7827e3e97a11
50 6faa638979f-1b4940d754398e54389e010ea043e7d6-14c149ba3e93eeb63
51 85c5e2e6ab73ae6&q=карточка сериала сваты&o=rJPoSthceZo
52 xELpkzYNe0Z04kmXJw4EpVWpkS7u%2B37TZiA9g3cuUoBpjOpZr1yZt%0A&
53 ;r=euw" method="POST">
54         <div class="anomaly-modal__mask">
55             <div class="anomaly-modal__modal is-ie" data-
56 testid="anomaly-modal">
57                 <div class="anomaly-modal__title">Unfortun
```

```

ately, bots use DuckDuckGo too.</div>
38         <div class="anomaly-modal__description">Please complete the following challenge to confirm this search was made by a human.</div>
39         <div class="anomaly-modal__instructions">Select all squares containing a duck:</div>
40         <div class="anomaly-modal__puzzle-margins">
41             <div class="anomaly-modal__puzzle">
42                 <div class="anomaly-modal__box
43                 " data-index="0">
44
45
46                     <label class="" for="image-check_1110e90e5492ddc53563e7cad90059ca" data-testid="anomaly-modal-tile-0">
47                         <input type="checkbox" class="anomaly-modal__check" name="image-check_1110e90e5492ddc53563e7cad90059ca" id="image-check_1110e90e5492ddc53563e7cad90059ca">
48                             
49                             </label>
50
51                     </div>
52
53                     <div class="anomaly-modal__box
54                     " data-index="1">
55
56                         <label class="" for="image-check_ecd4427bdc049b77fc3ee1fe3bfa506c" data-testid="anomaly-modal-tile-1">
57                             <input type="checkbox" class="anomaly-modal__check" name="image-check_ecd4427bdc049b77fc3ee1fe3bfa506c" id="image-check_ecd4427bdc049b77fc3ee1fe3bfa506c">

```

```
box" class="anomaly-modal__check" name="image-check_ecd4427bdc049b77fc3ee1fe3bfa506c" id="image-check_ecd4427bdc049b77fc3ee1fe3bfa506c">
```

```
58 |                                     
```

```
59 |                                     </label>
```

```
60 |                                     </div>
```

```
61 |                                     <div class="anomaly-modal__box" data-index="2">
```

```
62 |                                     <label class="" for="image-check_d2bce2444455b9344cb0695100022af8" data-testid="anomaly-modal-tile-2">
```

```
63 |                                     <input type="checkbox" class="anomaly-modal__check" name="image-check_d2bce2444455b9344cb0695100022af8" id="image-check_d2bce2444455b9344cb0695100022af8">
```

```
64 |                                     
```

```
65 |                                     </label>
```

```
66 |                                     </div>
```

```
67 |                                     <div class="anomaly-modal__box" data-index="3">
```

```
68 |                                     <label class="" for="image-check_20e9ac858806321167f5fe115d58890c" data-testid="anom
```

```
aly-modal-tile-3">
```

```
77 |                                     <input type="checkbox" class="anomaly-modal__check" name="image-check_20e9ac858806321167f5fe115d58890c" id="image-check_20e9ac858806321167f5fe115d58890c">
```

```
78 |                                     
```

```
79 |                                     </label>
```

```
80 |                                     </div>
```

```
81 |                                     <div class="anomaly-modal__box" data-index="4">
```

```
82 |                                     <label class="" for="image-check_f8def5d7f8a3e5486f88f602c0b4e4e7" data-testid="anomaly-modal-tile-4">
```

```
83 |                                     <input type="checkbox" class="anomaly-modal__check" name="image-check_f8def5d7f8a3e5486f88f602c0b4e4e7" id="image-check_f8def5d7f8a3e5486f88f602c0b4e4e7">
```

```
84 |                                     
```

```
85 |                                     </label>
```

```
86 |                                     </div>
```

```
87 |                                     <div class="anomaly-modal__box" data-index="5">
```

```
96 |                                     <label class="" for="i
    | image-check_b138a70fd08b09b5bd5d0685142a9dac" data-testid="anom
    | aly-modal-tile-5">
97 |                                     <input type="check
    | box" class="anomaly-modal__check" name="image-check_b138a70fd0
    | 8b09b5bd5d0685142a9dac" id="image-check_b138a70fd08b09b5bd5d06
    | 85142a9dac">
98 |                                     
99 |                                     </label>
100 |
101 |                                 </div>
102 |
103 |                                 <div class="anomaly-modal__box
    | " data-index="6">
104 |
105 |
106 |                                     <label class="" for="i
    | image-check_a756d136d7827e3e97a116faa638979f" data-testid="anom
    | aly-modal-tile-6">
107 |                                     <input type="check
    | box" class="anomaly-modal__check" name="image-check_a756d136d7
    | 827e3e97a116faa638979f" id="image-check_a756d136d7827e3e97a116
    | faa638979f">
108 |                                     
109 |                                     </label>
110 |
111 |                                 </div>
112 |
113 |                                 <div class="anomaly-modal__box
    | " data-index="7">
```



```
114 |
115 |
116 |         <label class="" for="i
image-check_1b4940d754398e54389e010ea043e7d6" data-testid="anom
aly-modal-tile-7">
117 |             <input type="check
box" class="anomaly-modal__check" name="image-check_1b4940d754
398e54389e010ea043e7d6" id="image-check_1b4940d754398e54389e01
0ea043e7d6">
118 |             
119 |             </label>
120 |
121 |         </div>
122 |
123 |         <div class="anomaly-modal__box
" data-index="8">
124 |
125 |
126 |             <label class="" for="i
image-check_14c149ba3e93eeb6385c5e2e6ab73ae6" data-testid="anom
aly-modal-tile-8">
127 |                 <input type="check
box" class="anomaly-modal__check" name="image-check_14c149ba3e
93eeb6385c5e2e6ab73ae6" id="image-check_14c149ba3e93eeb6385c5e
2e6ab73ae6">
128 |                 
129 |                 </label>
130 |
131 |             </div>
132 |
```

```
133         </div>
134     </div>
135     <div class="anomaly-modal__controls">
136         <button name="challenge-submit" class=
137         "btn btn--primary anomaly-modal__submit js-anomaly-modal-submit" form="challenge-form" value="d4cd0dabcf4caa22ad92fab40844c786">Submit</button>
138
139         <p></p>
140         <input class="feedback-toggle" type="checkbox" id="feedback" unchecked="">
141         <label for="feedback" class="feedback-text">Images not loading?</label>
142         <div class="feedback-content">
143             <label class="feedback-instructions">Please email the following code to:</label>
144             <p class="feedback-instructions">error-lite@duckduckgo.com</p>
145             <p><label class="feedback-instructions">Code: d4cd0dabcf4caa22ad92fab40844c786</label></p>
146         </div>
147     </div>
148
149
150
151
152
153
154
155
156
157     <br>
158     <p class="html-only">
159         <!-- This is the Lite version. Get the full-JS
160         version <a href="https://duckduckgo.com/">here</a>. //-->
161     </p>
162 </div></form></center>
```

162

163

```

```

164

165

166

167

168

169

170

171

172

173 </body></html>

test_bing.py

```
1 import asyncio
2 import urllib.parse
3 from bs4 import BeautifulSoup
4 from playwright.async_api import async_playwright
5
6 async def main():
7     async with async_playwright() as p:
8         browser = await p.chromium.launch(headless=True)
9         context = await browser.new_context(ignore_https_errors=True, locale='ru-RU')
10        page = await context.new_page()
11
12        encoded_query = urllib.parse.quote_plus("карточка сериала сваты")
13        await page.goto(f"https://www.bing.com/search?q={encoded_query}", timeout=20000)
14        await page.wait_for_timeout(3000)
15
16        html = await page.content()
17        soup = BeautifulSoup(html, 'html.parser')
18
19        results = []
20        for a in soup.find_all('a'):
21            href = a.get('href')
22            # typical bing organic result has an h2 inside it
23            if href and href.startswith('http') and 'bing.com'
not in href and 'microsoft.com' not in href:
24                if a.find('h2'):
25                    results.append(href)
26
27        print('LINKS FOUND:', len(results))
28        if results: print(results[:3])
29
30 asyncio.run(main())
31
```

test_ddg.py

```
1 import urllib.request
2 import urllib.parse
3 from bs4 import BeautifulSoup
4
5 url = 'https://lite.duckduckgo.com/lite/'
6 data = urllib.parse.urlencode({'q': 'карточка сериала сваты'})
7 req = urllib.request.Request(url, data=data, headers={'User-Agent': 'Mozilla/5.0'})
8 try:
9     with urllib.request.urlopen(req) as response:
10         html = response.read().decode('utf-8')
11         soup = BeautifulSoup(html, 'html.parser')
12
13         links = soup.find_all('a', class_='result-snippet')
14         print('result-snippet:', len(links))
15
16         links2 = soup.find_all('a', class_='result-url')
17         print('result-url:', len(links2))
18
19         # let's just find the actual table rows
20         for td in soup.find_all('td', class_='result-snippet')
21             :
22             print('td result-snippet text:', td.text[:50])
23
24             for a in soup.find_all('a'):
25                 if a.get('href') and a.get('href').startswith('http'):
26                     print('class:', a.get('class'), 'href:', a.get('href'))
27                     break
28 except Exception as e:
29     print('ERROR:', e)
```

test_google.py

```
1 import asyncio
2 import urllib.parse
3 from bs4 import BeautifulSoup
4 from playwright.async_api import async_playwright
5
6 async def main():
7     async with async_playwright() as p:
8         browser = await p.chromium.launch(headless=True)
9         context = await browser.new_context(ignore_https_error
10 s=True, locale='ru-RU')
11         page = await context.new_page()
12
13         encoded_query = urllib.parse.quote_plus("карточка сери
14 ала сваты")
15         await page.goto(f"https://www.google.com/search?q={enc
16 oded_query}&hl=ru", timeout=20000)
17         await page.wait_for_timeout(3000)
18
19         html = await page.content()
20         soup = BeautifulSoup(html, 'html.parser')
21
22         # Google search results usually have links inside div.
23         # yuRUbf > a, or just finding 'a' with 'href' that starts with h
24         # ttp
25
26         results = []
27         for div in soup.find_all('div', class_='yuRUbf'):
28             a = div.find('a')
29             if a and a.get('href'):
30                 results.append(a.get('href'))
31
32         if not results:
33             # Fallback for different google layouts
34             for a in soup.find_all('a'):
35                 href = a.get('href')
36                 if href and href.startswith('http') and 'googl
37 e.com' not in href:
```

```
31 |             results.append(href)
32 |
33 |         print('LINKS FOUND:', len(results))
34 |         if results: print(results[:3])
35 |
36 | asyncio.run(main())
37 |
```

test_search.py

```
1 import asyncio
2 import urllib.parse
3 from bs4 import BeautifulSoup
4 from playwright.async_api import async_playwright
5
6 async def main():
7     async with async_playwright() as p:
8         browser = await p.chromium.launch(headless=True)
9         context = await browser.new_context(ignore_https_error
10 s=True, locale='ru-RU')
11         page = await context.new_page()
12
13         encoded_query = urllib.parse.quote_plus("карточка сери
14 ала сваты")
15         await page.goto(f"https://duckduckgo.com/html/?q={enco
16 ded_query}", timeout=20000)
17         html = await page.content()
18         soup = BeautifulSoup(html, 'html.parser')
19
20         results = []
21         for a in soup.find_all('a'):
22             href = a.get('href')
23             if href and href.startswith('http'):
24                 results.append(href)
25
26         print('LINKS FOUND:', len(results))
27         if results: print(results[:3])
28
29         # Save html to examine
30         with open('search_test.html', 'w', encoding='utf-8') a
31 s f:
32             f.write(html)
```


assist.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Cross-platform CLI assistant for project management
4 # =====
5 # Description:
6 #   Cross-platform CLI assistant for managing AI Breadboard project.
7 #   Ported from old assist.ps1 to work on Windows, Linux, and macOS.
8 #
9 # File: assist.py
10 # Project: ai-breadboard
11 # Package: scripts.cli
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """Cross-platform CLI assistant for AI Breadboard project management.
17
18 CLI assistant providing cross-platform project management utilities,
19 originally ported from PowerShell assist.ps1 for Windows, Linux, macOS.
20
21 Использование:
22     assist start [service]          # Запустить сервис (run, unicorn, light, foundry)
23     assist stop [service]          # Остановить сервис (server, foundry, all)
24     assist restart                  # Перезапустить
25     assist status                   # Показать status
```

```

26 |         assist providers                # List провайдеров ИИ
27 |         assist logs [lines]            # Показать логи
28 |         assist config [show|get|set]    # Работа с config.json
29 |         assist test                     # Запустить тесты
30 | """
31 |
32 | import argparse
33 | import json
34 | import os
35 | import sys
36 | import subprocess
37 | from pathlib import Path
38 | from typing import Optional, Dict, List, Tuple
39 |
40 | # Добавляем root в PYTHONPATH для импортов
41 | sys.path.insert(0, str(Path(__file__).parent.parent.parent))
42 |
43 | from scripts.cli.paths import init_paths, get_paths
44 | from scripts.cli.config import get_config_manager
45 | from scripts.cli.utils import (
46 |     find_available_port,
47 |     is_port_open,
48 |     get_process_on_port,
49 |     kill_process,
50 |     ensure_in_path,
51 |     run_command,
52 |     which
53 | )
54 |
55 | # ANSI цвета для терминала
56 | C_RESET = "\033[0m"
57 | C_BOLD = "\033[1m"
58 | C_CYAN = "\033[96m"
59 | C_GREEN = "\033[92m"
60 | C_YELLOW = "\033[93m"
61 | C_RED = "\033[91m"
62 | C_GRAY = "\033[90m"
63 | C_MAGENTA = "\033[95m"

```

```
64 |
65 | class AssistCLI:
66 |     """Главный CLI class для управления AI-Breadboard"""
67 |
68 |     def __init__(self):
69 |         self.paths = init_paths()
70 |         self.config_mgr = get_config_manager()
71 |         self._state_dir = self.paths.data_dir / "assist_state"
72 |         self._ensure_state_dir()
73 |
74 |     def _ensure_state_dir(self) -> None:
75 |         """Creates директорию состояния"""
76 |         self._state_dir.mkdir(parents=True, exist_ok=True)
77 |
78 |     def _get_state_file(self, name: str) -> Path:
79 |         """Получить путь до файла состояния"""
80 |         return self._state_dir / f"{name}.json"
81 |
82 |     def _load_state(self, name: str) -> dict:
83 |         """Загрузить state из файла"""
84 |         state_file = self._get_state_file(name)
85 |         if not state_file.exists():
86 |             return {}
87 |         try:
88 |             with open(state_file, "r", encoding="utf-8") as f:
89 |                 return json.load(f)
90 |         except Exception:
91 |             return {}
92 |
93 |     def _save_state(self, name: str, data: dict) -> None:
94 |         """Сохранить state в файл"""
95 |         self._ensure_state_dir()
96 |         state_file = self._get_state_file(name)
97 |         with open(state_file, "w", encoding="utf-8") as f:
98 |             json.dump(data, f, ensure_ascii=False, indent=2)
99 |
100 |     def cmd_start(self, service: str = "run") -> int:
101 |         """Запустить сервис"""
```

```
102     service = service.lower()
103
104     # Маппинг сервисов на скрипты
105     script_map = {
106         "run": "launchers/run.py",
107         "all": "launchers/run.py",
108         "unicorn": "launchers/run_unicorn.py",
109         "uunicorn": "launchers/run_unicorn.py",
110         "light": "launchers/run_light_server.py",
111         "foundry": "launchers/run_foundry.py",
112     }
113
114     script_name = script_map.get(service)
115     if not script_name:
116         print(f"{C_RED}❌  Неизвестный сервис: '{service}'{C
_RESET}")
117         print(f"{C_GRAY}Доступные сервисы: run, unicorn, l
ight, foundry, all{C_RESET}")
118         return 1
119
120     script_path = self.paths.project_root / script_name
121
122     if not script_path.exists():
123         print(f"{C_RED}❌  Скрипт не найден: {script_path}{C
_RESET}")
124         return 1
125
126     print(f"{C_CYAN}❏  Запуск: {script_name}...{C_RESET}")
127
128     # Запустить скрипт
129     try:
130         python_exe = str(self.paths.venv_python) if self.p
aths.venv_python.exists() else sys.executable
131         result = subprocess.run(
132             [python_exe, str(script_path)],
133             cwd=str(self.paths.project_root),
134             check=False
135         )
```

```

136         return result.returncode
137     except Exception as e:
138         print(f"{C_RED}❌ Error запуска: {e}{C_RESET}")
139         return 1
140
141     def cmd_stop(self, service: str = "all") -> int:
142         """Остановить сервис"""
143         service = service.lower()
144
145         config = self.config_mgr.load_config()
146         server_cfg = config.get("server", {})
147         port = int(server_cfg.get("port", 8000))
148
149         stopped_any = False
150
151         if service in ("server", "all", "unicorn", "uvicorn",
152 "light"):
153             proc_info = get_process_on_port(port)
154             if proc_info:
155                 pid, proc_name = proc_info
156                 print(f"{C_YELLOW}❌ Остановка сервера на порту
157 {port} (PID: {pid}, {proc_name})...{C_RESET}")
158                 if kill_process(pid, force=True):
159                     print(f" {C_GREEN}❌ Процесс завершен{C_RE
160 SET}")
161                     stopped_any = True
162             else:
163                 print(f" {C_RED}❌ Не удалось завершить пр
164 оцесс{C_RESET}")
165             else:
166                 print(f"{C_GRAY}Порт {port} свободен (сервер н
167 е запущен){C_RESET}")
168
169         if service in ("foundry", "all"):
170             print(f"{C_YELLOW}❌ Остановка Foundry...{C_RESET}")
171         )
172
173         # TODO: Реализовать остановку Foundry
174         stopped_any = True

```

```

168 |         if not stopped_any:
169 |             print(f"{C_GREEN}Все сервисы уже остановлены.{C_RE
170 | SET}")
171 |
172 |     return 0
173 |
174 | def cmd_restart(self) -> int:
175 |     """Перезапустить сервисы"""
176 |     print(f"{C_CYAN}▯ Перезапуск сервисов...{C_RESET}")
177 |     self.cmd_stop()
178 |     return self.cmd_start()
179 |
180 | def cmd_status(self) -> int:
181 |     """Показать status"""
182 |     config = self.config_mgr.load_config()
183 |     server_cfg = config.get("server", {})
184 |     port = int(server_cfg.get("port", 8000))
185 |     use_ssl = bool(server_cfg.get("use_ssl", False))
186 |     mode = str(server_cfg.get("mode", "DEV"))
187 |     reload_on = bool(server_cfg.get("reload", True))
188 |
189 |     proto = "https" if use_ssl else "http"
190 |     local_url = f"{proto}://localhost:{port}/"
191 |
192 |     print(f"\n{C_BOLD}{C_CYAN}═══════════════════════════════════
193 | ════════════════════════════════════{C_RESET}")
194 |     print(f"{C_BOLD}{C_CYAN}▯                               СТАТУС AI ASSIST
195 | ANT СИСТЕМЫ                                ▯{C_RESET}")
196 |     print(f"{C_BOLD}{C_CYAN}═══════════════════════════════════
197 | ════════════════════════════════════{C_RESET}\n")
198 |
199 |     # Check FastAPI сервера
200 |     proc_info = get_process_on_port(port)
201 |     if proc_info:
202 |         pid, proc_name = proc_info
203 |         print(f" {C_BOLD}FastAPI Сервер:{C_RESET}      {C_
204 | GREEN}● РАБОТАЕТ{C_RESET}")

```

```

201         print(f"  {C_BOLD}URL:{C_RESET}          {C_
| CYAN}{local_url}{C_RESET}")
202         print(f"  {C_BOLD}PID процесса:{C_RESET}      {pi
| d} ({proc_name})")
203         else:
204             print(f"  {C_BOLD}FastAPI Сервер:{C_RESET}    {C_
| RED}○ ОСТАНОВЛЕН{C_RESET}")
205             print(f"  {C_BOLD}Ожидаемый порт:{C_RESET}    {po
| rt}")
206
207             print(f"  {C_BOLD}Режим / SSL:{C_RESET}        {mode}
| | SSL: {'ВКЛ' if use_ssl else 'ВЫКЛ'} | Autoreload: {'ВКЛ' if
| reload_on else 'ВЫКЛ'}")
208
209             # AI Foundry
210             ai_cfg = config.get("ai", {})
211             use_foundry = bool(ai_cfg.get("use_foundry", False))
212             foundry_url = str(ai_cfg.get("foundry_base_url", "http
| ://localhost:54837"))
213
214             if use_foundry:
215                 print(f"  {C_BOLD}AI Foundry:{C_RESET}      {C_
| YELLOW}○ ВКЛЮЧЕН В CONFIG (внешний сервис){C_RESET}")
216                 print(f"  {C_BOLD}URL:{C_RESET}            {fo
| undry_url}")
217             else:
218                 print(f"  {C_BOLD}AI Foundry:{C_RESET}      {C_
| GRAY}○ ОТКЛЮЧЕН В CONFIG{C_RESET}")
219
220             # Gemini API Keys
221             gemini_keys = os.getenv("GEMINI_API_KEY_NAMES", "")
222             key_count = len([k for k in gemini_keys.split(",") if
| k.strip()])
223             print(f"  {C_BOLD}Gemini API Keys:{C_RESET}      {C_GREE
| N}{key_count} в пуле ротации{C_RESET}")
224
225             print()
226             return 0

```

```

227
228     def cmd_providers(self) -> int:
229         """Показать list провайдеров"""
230         config = self.config_mgr.load_config()
231         ai_cfg = config.get("ai", {})
232
233         print(f"\n{C_BOLD}{C_CYAN}{'====='}
{'====='}{
C_RESET}")
234         print(f"{C_BOLD}{C_CYAN}||                                ПЕЕСТР П
РОВАЙДЕРОВ И МОДЕЛЕЙ ИИ                                ||{C_
RESET}")
235         print(f"{C_BOLD}{C_CYAN}{'====='}
{'====='}{C_
RESET}\n")
236
237         # 1. Google Gemini API
238         gemini_keys = os.getenv("GEMINI_API_KEY_NAMES", "")
239         keys = [k.strip() for k in gemini_keys.split(",") if k
.strip()]
240         has_gemini = len(keys) > 0
241         print(f"{C_BOLD}{C_GREEN}1. Google Gemini API (Cloud A
PI){C_RESET}")
242         print(f"    Status:                {'✅ Активен' if has_gemini
else '❌ Нет ключей'}")
243         print(f"    Пул ключей:            {len(keys)} шт. ({', '.joi
n(keys) if keys else 'пусто'})")
244         print(f"    Дефолтная модель: gemini-2.5-flash / gemini
-3.1-flash-lite")
245         print()
246
247         # 2. Gemini CLI
248         use_gemini_cli = bool(ai_cfg.get("use_gemini_cli", Tru
e))
249         gemini_cli_model = str(ai_cfg.get("gemini_cli_model_id
", "gemini-3.1-flash-lite"))
250         print(f"{C_BOLD}{C_GREEN}2. Gemini CLI (Терминальный а
гент){C_RESET}")

```



```

251 |         print(f"    Status:                {'☑ Включен' if use_gemini
      | _cli else '○ Отключен'}")
252 |         print(f"    Модель:                {gemini_cli_model}")
253 |         print()
254 |
255 |         # 3. Microsoft AI Foundry
256 |         use_foundry = bool(ai_cfg.get("use_foundry", False))
257 |         foundry_model = str(ai_cfg.get("foundry_model_id", "qw
      | en2.5-1.5b"))
258 |         foundry_url = str(ai_cfg.get("foundry_base_url", "http
      | ://localhost:54837"))
259 |         print(f"{C_BOLD}{C_GREEN}3. Microsoft AI Foundry (Лока
      | льная LLM){C_RESET}")
260 |         print(f"    Status:                {'☑ Включен' if use_foundr
      | y else '○ Отключен'}")
261 |         print(f"    Модель:                {foundry_model}")
262 |         print(f"    Endpoint:              {foundry_url}")
263 |         print()
264 |
265 |         # 4. Ollama
266 |         use_ollama = bool(ai_cfg.get("use_ollama", False))
267 |         ollama_model = str(ai_cfg.get("ollama_model_id", "llam
      | a3.1"))
268 |         ollama_url = str(ai_cfg.get("ollama_base_url", "http:/
      | /localhost:11434"))
269 |         print(f"{C_BOLD}{C_GREEN}4. Ollama Local{C_RESET}")
270 |         print(f"    Status:                {'☑ Включен' if use_ollama
      | else '○ Отключен'}")
271 |         print(f"    Модель:                {ollama_model}")
272 |         print(f"    Endpoint:              {ollama_url}")
273 |         print()
274 |
275 |         # 5. OpenAI Compatible
276 |         openai_cfg = config.get("openai_compat", {}).get("prov
      | iders", {})
277 |         if openai_cfg:
278 |             print(f"{C_BOLD}{C_GREEN}5. OpenAI-Compatible Пров
      | айдеры{C_RESET}")

```

```

279         for p_name, p_info in openai_cfg.items():
280             base = p_info.get("base_url", "")
281             models = ", ".join(p_info.get("models", [])) o
r "авто"
282             print(f"      • {C_BOLD}{p_name}{C_RESET}: {base}
")
283             print(f"      Модели: {models}")
284             print()
285
286         return 0
287
288     def cmd_logs(self, lines: int = 40, name: str = "fastapi")
-> int:
289         """Показать логи"""
290         logs_dir = self.paths.project_root / "logs"
291         if not logs_dir.exists():
292             print(f"{C_YELLOW}Каталог logs/ пока пуст.{C_RESET
}")
293             return 0
294
295         # Поиск лог-файла
296         target_file = None
297         if name == "fastapi":
298             target_file = logs_dir / "fastapi.log"
299         elif name == "info":
300             target_file = logs_dir / "info.log"
301         elif name == "error" or name == "errors":
302             target_file = logs_dir / "errors.log"
303
304         if not target_file or not target_file.exists():
305             # Найти первый доступный лог
306             for cand in ["fastapi.log", "info.log", "errors.lo
g"]:
307                 if (logs_dir / cand).exists():
308                     target_file = logs_dir / cand
309                     break
310
311         if not target_file or not target_file.exists():

```

```

312         print(f"{C_YELLOW}Лог-файл не найден в {logs_dir}{
C_RESET}")
313         return 0
314
315         print(f"\n{C_CYAN}▯ Последние {lines} строк: {target_
file.name}{C_RESET}\n" + "-" * 70)
316         try:
317             with open(target_file, "r", encoding="utf-8", erro
rs="replace") as f:
318                 all_lines = f.readlines()
319                 for line in all_lines[-int(lines):]:
320                     print(line.rstrip())
321             except Exception as e:
322                 print(f"{C_RED}Error чтения логов: {e}{C_RESET}")
323             print("-" * 70 + "\n")
324             return 0
325
326     def cmd_config(self, subcommand: str = "show", key: str =
"", value: str = "") -> int:
327         """Работа с config.json"""
328         config = self.config_mgr.load_config()
329
330         if subcommand == "show":
331             print(json.dumps(config, indent=2, ensure_ascii=Fa
lse))
332             return 0
333
334         if subcommand == "get":
335             if not key:
336                 print(f"{C_RED}Укажите ключ (например: server.
port){C_RESET}")
337             return 1
338
339             parts = key.split(".")
340             val = config
341             for p in parts:
342                 if isinstance(val, dict):
343                     val = val.get(p, {})

```

```

344         else:
345             val = None
346             break
347         print(f"{key} = {json.dumps(val, ensure_ascii=False
e)}}")
348         return 0
349
350     if subcommand == "set":
351         if not key or value == "":
352             print(f"{C_RED}Использование: assist config se
t <key> <value>{C_RESET}")
353             return 1
354
355         # Парсим значение
356         parsed_val = value
357         if value.lower() in ("true", "1", "yes"):
358             parsed_val = True
359         elif value.lower() in ("false", "0", "no"):
360             parsed_val = False
361         elif value.isdigit():
362             parsed_val = int(value)
363
364         # Устанавливаем значение
365         parts = key.split(".")
366         curr = config
367         for p in parts[:-1]:
368             if p not in curr or not isinstance(curr[p], dict):
369                 curr[p] = {}
370             curr = curr[p]
371         curr[parts[-1]] = parsed_val
372
373         # Сохраняем
374         if self.config_mgr.save_config(config):
375             print(f"{C_GREEN}  Обновлено: {key} = {parsed_
val}{C_RESET}")
376             return 0
377         else:

```

```

378 |             print(f"{C_RED}❌ Error сохранения конфигурации
      | {C_RESET}")
379 |             return 1
380 |
381 |         return 0
382 |
383 |     def cmd_test(self) -> int:
384 |         """Запустить тесты"""
385 |         print(f"{C_CYAN}❏ Запуск тестов pytest...{C_RESET}")
386 |         venv_python = str(self.paths.venv_python)
387 |         result = subprocess.run(
388 |             [venv_python, "-m", "pytest", "tests/"],
389 |             cwd=str(self.paths.project_root),
390 |             check=False
391 |         )
392 |         return result.returncode
393 |
394 | def main():
395 |     """Главная function"""
396 |     parser = argparse.ArgumentParser(
397 |         description="CLI ассистент для управления AI-Breadboar
      | d",
398 |         formatter_class=argparse.RawDescriptionHelpFormatter,
399 |         epilog="""
400 | Examples:
401 |     assist start                # Запустить основной сервер
402 |     assist start unicorn       # Запустить FastAPI через uvic
      | orn
403 |     assist stop                 # Остановить все сервисы
404 |     assist status               # Показать status
405 |     assist providers            # List провайдеров ИИ
406 |     assist logs 50              # Последние 50 строк логов
407 |     assist config get server.port # Получить значение
408 |     assist config set server.port 8080 # Установить значение
409 |     assist test                 # Запустить тесты
410 |     """
411 |     )
412 |

```

```
413     subparsers = parser.add_subparsers(dest="command", help="Команда")
414
415     # start
416     start_parser = subparsers.add_parser("start", help="Запустить сервис")
417     start_parser.add_argument("service", nargs="?", default="run", help="Сервис (run, unicorn, light, foundry)")
418
419     # stop
420     stop_parser = subparsers.add_parser("stop", help="Остановить сервис")
421     stop_parser.add_argument("service", nargs="?", default="all", help="Сервис (server, foundry, all)")
422
423     # restart
424     subparsers.add_parser("restart", help="Перезапустить")
425
426     # status
427     subparsers.add_parser("status", help="Показать status")
428
429     # providers
430     subparsers.add_parser("providers", help="List провайдеров")
431
432     # logs
433     logs_parser = subparsers.add_parser("logs", help="Показать логи")
434     logs_parser.add_argument("lines", nargs="?", type=int, default=40, help="Количество строк")
435     logs_parser.add_argument("-n", "--name", default="fastapi", help="Имя лога (fastapi, info, error)")
436
437     # config
438     config_parser = subparsers.add_parser("config", help="Работа с config.json")
439     config_parser.add_argument("subcommand", nargs="?", default="show", help="Субкоманда (show, get, set)")
```

```
440     config_parser.add_argument("key", nargs="?", default="", h
    elp="Ключ")
441     config_parser.add_argument("value", nargs="?", default="",
    help="Значение")
442
443     # test
444     subparsers.add_parser("test", help="Запустить тесты")
445
446     args = parser.parse_args()
447
448     # Если команда не указана, показать справку
449     if not args.command:
450         parser.print_help()
451         return 0
452
453     cli = AssistCLI()
454
455     # Выполнить команду
456     if args.command == "start":
457         return cli.cmd_start(args.service)
458     elif args.command == "stop":
459         return cli.cmd_stop(args.service)
460     elif args.command == "restart":
461         return cli.cmd_restart()
462     elif args.command == "status":
463         return cli.cmd_status()
464     elif args.command == "providers":
465         return cli.cmd_providers()
466     elif args.command == "logs":
467         return cli.cmd_logs(args.lines, args.name)
468     elif args.command == "config":
469         return cli.cmd_config(args.subcommand, args.key, args.
    value)
470     elif args.command == "test":
471         return cli.cmd_test()
472     else:
473         parser.print_help()
474         return 1
```

```
475 |  
476 | if __name__ == "__main__":  
477 |     sys.exit(main())  
478 |
```


config.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Configuration file management utilities
4 # =====
5 # Description:
6 #   Utilities for working with configuration files cross-platf
7 #   Provides JSON loading/saving, environment variable managem
8 #   ent.
9 #
10 # File: config.py
11 # Project: ai-breadboard
12 # Package: scripts.cli
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 """Cross-platform configuration management utilities.
18 Provides utilities for managing configuration files, JSON oper
19 ations,
20 and environment variable handling."""
21
22 import json
23 import os
24 from pathlib import Path
25 from typing import Any, Dict, Optional
26
27 from scripts.cli.paths import get_paths
28
29 class ConfigManager:
30     """Configuration file management.
```

```

31 |     Handles loading and saving of configuration files in JSON
    | format
32 |     with support for caching and environment variable integrat
    | ion.
33 |     """
34 |
35 |     def __init__(self):
36 |         """Initialize config manager."""
37 |         self.paths = get_paths()
38 |         self._config_cache = {}
39 |         self._env_vars = {}
40 |
41 |     def load_json(self, filepath: Path) -> dict:
42 |         """Load JSON file.
43 |
44 |         Args:
45 |             filepath: Path to JSON file.
46 |
47 |         Returns:
48 |             Dictionary with JSON content or empty dict if not
    | found.
49 |         """
50 |         if not filepath.exists():
51 |             return {}
52 |
53 |         try:
54 |             with open(filepath, "r", encoding="utf-8") as f:
55 |                 return json.load(f)
56 |         except Exception as e:
57 |             print(f"Error loading {filepath}: {e}")
58 |             return {}
59 |
60 |     def save_json(self, filepath: Path, data: dict, pretty: bo
    | ol = True) -> bool:
61 |         """Save JSON file.
62 |
63 |         Args:
64 |             filepath: Path to save JSON to.

```

```

65         data: Dictionary to save.
66         pretty: Format output for readability.
67
68     Returns:
69         True if successful, False otherwise.
70     """
71     try:
72         filepath.parent.mkdir(parents=True, exist_ok=True)
73         with open(filepath, "w", encoding="utf-8") as f:
74             json.dump(
75                 data,
76                 f,
77                 ensure_ascii=False,
78                 indent=2 if pretty else None
79             )
80         return True
81     except Exception as e:
82         print(f"Error saving {filepath}: {e}")
83         return False
84
85     def load_env_file(self, filepath: Path) -> dict:
86         """Load .env file.
87
88         Args:
89             filepath: Path to .env file.
90
91         Returns:
92             Dictionary of environment variables.
93         """
94         env_vars = {}
95
96         if not filepath.exists():
97             return env_vars
98
99         try:
100             with open(filepath, "r", encoding="utf-8") as f:
101                 for line in f:
102                     line = line.strip()

```

```

103         if not line or line.startswith("#"):
104             continue
105
106         if "=" in line:
107             key, value = line.split("=", 1)
108             env_vars[key.strip()] = value.strip()
109
110     except Exception as e:
111         print(f"Error loading {filepath}: {e}")
112
113     return env_vars
114
115     def save_env_file(self, filepath: Path, env_vars: dict) ->
bool:
116         """Save .env file.
117
118         Args:
119             filepath: Path to save .env to.
120             env_vars: Dictionary of environment variables.
121
122         Returns:
123             True if successful, False otherwise.
124         """
125         try:
126             filepath.parent.mkdir(parents=True, exist_ok=True)
127             with open(filepath, "w", encoding="utf-8") as f:
128                 for key, value in env_vars.items():
129                     f.write(f"{key}={value}\n")
130             return True
131         except Exception as e:
132             print(f"Error saving {filepath}: {e}")
133             return False
134
135     def get_config(self, key: str, default: Any = None) -> Any
:
136         """Get configuration value.
137
138         Args:

```

```

139         key: Configuration key (supports dot notation like
        'section.key').
140         default: Default value if key not found.
141
142     Returns:
143         Configuration value or default.
144     """
145     # Load main config
146     config = self.load_json(self.paths.config_file)
147
148     # Navigate nested keys
149     keys = key.split(".")
150     current = config
151
152     for k in keys:
153         if isinstance(current, dict):
154             current = current.get(k)
155         else:
156             return default
157
158     return current if current is not None else default
159
160 def set_config(self, key: str, value: Any) -> bool:
161     """Set configuration value.
162
163     Args:
164         key: Configuration key (supports dot notation).
165         value: Value to set.
166
167     Returns:
168         True if successful, False otherwise.
169     """
170     config = self.load_json(self.paths.config_file)
171
172     # Navigate nested keys
173     keys = key.split(".")
174     current = config
175

```

```
176 |         for k in keys[:-1]:
177 |             if k not in current or not isinstance(current[k],
dict):
178 |                 current[k] = {}
179 |                 current = current[k]
180 |
181 |         current[keys[-1]] = value
182 |
183 |         return self.save_json(self.paths.config_file, config)
184 |
185 | def merge_env_to_config(self) -> None:
186 |     """Merge environment variables into config."""
187 |     env_file = self.paths.env_file
188 |     env_vars = self.load_env_file(env_file)
189 |
190 |     for key, value in env_vars.items():
191 |         os.environ[key] = value
192 |
```

installer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Cross-platform installation system for AI Breadboard
4 # Description: Sets up Python environment, dependencies, SSL certificates,
5 # CLI configuration, and optional AI models for AI Breadboard across all
6 # platforms.
7 # File: installer.py
8 # Project: ai-breadboard
9 # Package: scripts.cli
10 # Author: hypo69
11 # Copyright: © 2026 hypo69
12 # =====
13
14 """
15 Cross-platform installation system for AI Breadboard.
16
17 Ported from legacy install.ps1 to work on Windows, Linux, and macOS.
18
19 Usage:
20     python scripts/cli/installer.py
21     python scripts/cli/installer.py --lang en
22     python scripts/cli/installer.py --skip-models
23 """
24
25 import argparse
26 import json
27 import os
28 import sys
29 import subprocess
30 import shutil
```

```

31 from pathlib import Path
32 from typing import Optional, Dict, List
33 from enum import Enum
34
35 sys.path.insert(0, str(Path(__file__).parent.parent.parent))
36
37 from scripts.cli.paths import get_paths, CrossPlatformPaths
38 from scripts.cli.config import get_config_manager
39 from scripts.cli.utils import which, run_command
40
41 class Language(Enum):
42     """Supported languages"""
43     EN = "en"
44     RU = "ru"
45     ES = "es"
46     HE = "he"
47
48 # Messages in multiple languages
49 MESSAGES = {
50     "en": {
51         "welcome": "👋 AI Breadboard Installation",
52         "step_venv": "[1/6] Creating Python virtual environmen
53 t...",
54         "step_deps": "[2/6] Installing dependencies...",
55         "step_certs": "[3/6] Setting up SSL certificates...",
56         "step_cli": "[4/6] Configuring CLI assistant...",
57         "step_verify": "[5/6] Verifying installation...",
58         "step_models": "[6/6] Downloading AI models (optional)
59 ...",
60         "success": "🎉 Installation completed successfully!",
61         "error": "❌ Installation failed:",
62         "abort": "🛑 Installation aborted by user",
63     },
64     "ru": {
65         "welcome": "👋 AI Breadboard Installation",
66         "step_venv": "[1/6] Creating Python virtual environmen
67 t...",
68         "step_deps": "[2/6] Installing dependencies...",

```



```

66         "step_certs": "[3/6] Setting up SSL certificates...",
67         "step_cli": "[4/6] Configuring CLI assistant...",
68         "step_verify": "[5/6] Verifying installation...",
69         "step_models": "[6/6] Downloading AI models (optional)
    ...,
70         "success": "✅ Installation completed successfully!",
71         "error": "❌ Installation failed:",
72         "abort": "❌ Installation aborted by user",
73     },
74 }
75
76 class Installer:
77     """Main installer class"""
78
79     def __init__(self, language: str = "en", install_dir: Optional[Path] = None):
80         self.language = Language[language.upper()] if language
81         .upper() in Language.__members__ else Language.EN
82         self.messages = MESSAGES.get(self.language.value, MESSAGES["en"])
83         self.install_dir = install_dir or self._get_default_install_dir()
84         self.venv_dir = self.install_dir / "venv"
85         self.config_mgr = get_config_manager()
86
87     def msg(self, key: str) -> str:
88         """Get message in current language"""
89         return self.messages.get(key, key)
90
91     @staticmethod
92     def _get_default_install_dir() -> Path:
93         """Get default installation directory"""
94         if sys.platform == "win32":
95             # Windows: %LOCALAPPDATA%\AI-Breadboard
96             localappdata = os.environ.get("LOCALAPPDATA", str(
97                 Path.home() / "AppData" / "Local"))
98             return Path(localappdata) / "AI-Breadboard"
99         else:

```

```

98 |         # Linux/macOS: ~/AI-Breadboard or ~/.local/share/A
    | I-Breadboard
99 |         return Path.home() / "AI-Breadboard"
100 |
101 |     def print_header(self):
102 |         """Print installation header"""
103 |         print()
104 |         print("┌" + "=" * 68 + "┐")
105 |         print("│" + self.msg("welcome").center(68) + "│")
106 |         print("└" + "=" * 68 + "┘")
107 |         print()
108 |         print(f"Installing to: {self.install_dir}")
109 |         print()
110 |
111 |     def create_venv(self) -> bool:
112 |         """Create virtual environment"""
113 |         print(self.msg("step_venv"))
114 |
115 |         try:
116 |             # Remove old venv if exists
117 |             if self.venv_dir.exists():
118 |                 print(f"  Removing old venv...", end=" ", flush
    | h=True)
119 |                 shutil.rmtree(self.venv_dir)
120 |                 print("[OK]")
121 |
122 |             # Create new venv
123 |             print(f"  Creating venv...", end=" ", flush=True)
124 |             subprocess.run(
125 |                 [sys.executable, "-m", "venv", str(self.venv_d
    | ir)],
126 |                 check=True,
127 |                 capture_output=True
128 |             )
129 |             print("[OK]")
130 |             print()
131 |             return True
132 |

```

```

133         except Exception as e:
134             print(f"[ERROR] {e}")
135             print()
136             return False
137
138     def install_dependencies(self) -> bool:
139         """Install dependencies"""
140         print(self.msg("step_deps"))
141
142         try:
143             # Get pip path in venv
144             if sys.platform == "win32":
145                 pip_exe = self.venv_dir / "Scripts" / "pip.exe"
146             else:
147                 pip_exe = self.venv_dir / "bin" / "pip"
148
149             if not pip_exe.exists():
150                 print(f" [ERROR] pip not found in venv")
151                 print()
152                 return False
153
154             # Requirements files
155             req_files = [
156                 self.install_dir / "requirements.txt",
157                 self.install_dir / "install" / "req" / "requirements-core.txt",
158                 self.install_dir / "install" / "req" / "requirements-ai.txt",
159             ]
160
161             for req_file in req_files:
162                 if req_file.exists():
163                     print(f" Installing from {req_file.name}.
164                     ..", end=" ", flush=True)
165                     result = subprocess.run(
166                         [str(pip_exe), "install", "-r", str(req_file)],

```

```

166         capture_output=True,
167         text=True
168     )
169     if result.returncode == 0:
170         print("[OK]")
171     else:
172         print(f"[WARN]")
173         if result.stderr:
174             print(f"    {result.stderr[:100]}")
175 )
176     print()
177     return True
178
179 except Exception as e:
180     print(f"[ERROR] {e}")
181     print()
182     return False
183
184 def setup_ssl_certificates(self) -> bool:
185     """Setup SSL certificates"""
186     print(self.msg("step_certs"))
187
188     try:
189         certs_dir = CrossPlatformPaths().certs_dir
190         certs_dir.mkdir(parents=True, exist_ok=True)
191
192         print(f"    Certificates directory: {certs_dir}")
193
194         # Check for mkcert
195         if which("mkcert"):
196             print(f"    mkcert found, generating certificate
197 s...", end=" ", flush=True)
198             subprocess.run(
199                 ["mkcert", "-install"],
200                 cwd=str(certs_dir),
201                 capture_output=True

```

```

202         subprocess.run(
203             ["mkcert", "-key-file", str(certs_dir / "l
localhost+2-key.pem"),
204             "-cert-file", str(certs_dir / "localhost+
2.pem"),
205             "localhost", "127.0.0.1", "::1"],
206             cwd=str(certs_dir),
207             capture_output=True
208         )
209         print("[OK]")
210     else:
211         print(f"  mkcert not installed (optional)")
212         print(f"  Installation: https://github.com/Filo
oSottile/mkcert")
213
214         print()
215         return True
216
217     except Exception as e:
218         print(f"[WARN] Error setting up certificates: {e}")
219     )
220
221     print()
222     return True  # Not critical
223
224 def configure_cli(self) -> bool:
225     """Configure CLI assistant"""
226     print(self.msg("step_cli"))
227
228     try:
229         from scripts.cli.utils import ensure_in_path, add_
to_env_var
230
231         # Add to PATH
232         print(f"  Adding to PATH...", end=" ", flush=True)
233         if ensure_in_path(self.install_dir / "assist"):
234             print("[OK]")
235         else:
236             print("[WARN]")

```

```

235 |
236 |         # Set environment variables
237 |         print(f"  Setting environment variables...", end="
    |         ", flush=True)
238 |         add_to_env_var("AIBREADBOARD_DIR", str(self.instal
    | l_dir))
239 |         add_to_env_var("PYTHONPATH", str(self.install_dir)
    | )
240 |         print("[OK]")
241 |
242 |         print()
243 |         return True
244 |
245 |     except Exception as e:
246 |         print(f"[WARN] {e}")
247 |         print()
248 |         return True  # Not critical
249 |
250 |     def verify_installation(self) -> bool:
251 |         """Verify installation"""
252 |         print(self.msg("step_verify"))
253 |
254 |         try:
255 |             # Get Python in venv
256 |             if sys.platform == "win32":
257 |                 python_exe = self.venv_dir / "Scripts" / "pyth
    | on.exe"
258 |             else:
259 |                 python_exe = self.venv_dir / "bin" / "python"
260 |
261 |             if not python_exe.exists():
262 |                 print(f"  [ERROR] Python not found in venv")
263 |                 print()
264 |                 return False
265 |
266 |             # Check core modules
267 |             print(f"  Checking dependencies...", end=" ", flus
    | h=True)

```

```

268         result = subprocess.run(
269             [str(python_exe), "-c", "import fastapi, uvicorn,
dotenv; print('OK')"],
270             capture_output=True,
271             text=True,
272             timeout=10
273         )
274
275         if result.returncode == 0 and "OK" in result.stdout:
276             print("[OK]")
277         else:
278             print("[WARN] Some dependencies may not be installed")
279
280         # Check assist
281         assist_file = self.install_dir / "assist"
282         if assist_file.exists() or (self.install_dir / "assist.cmd").exists():
283             print(f"  Checking assist CLI...", end=" ", flush=True)
284             print("[OK]")
285         else:
286             print(f"  [WARN] assist not found")
287
288         print()
289         return True
290
291     except Exception as e:
292         print(f"[WARN] {e}")
293         print()
294         return True # Not critical
295
296     def download_models(self, skip: bool = False) -> bool:
297         """Download models (optional)"""
298         if skip:
299             print(self.msg("step_models"))
300             print(f"  Skipping (used --skip-models)")

```

```

301         print()
302         return True
303
304     print(self.msg("step_models"))
305
306     # TODO: Implement model download
307     print(f"  TODO: Implement model download")
308     print()
309     return True
310
311     def run(self, skip_models: bool = False, skip_venv: bool =
False,
312           skip_deps: bool = False, skip_certs: bool = False)
-> int:
313         """Run installation"""
314         self.print_header()
315
316         try:
317             # Create installation directory
318             self.install_dir.mkdir(parents=True, exist_ok=True
)
319
320             # Installation steps
321             steps = [
322                 (not skip_venv, self.create_venv, "venv"),
323                 (not skip_deps, self.install_dependencies, "de
pendencies"),
324                 (not skip_certs, self.setup_ssl_certificates,
"SSL certificates"),
325                 (True, self.configure_cli, "CLI"),
326                 (True, self.verify_installation, "verification
"),
327                 (True, lambda: self.download_models(skip=skip_
models), "models"),
328             ]
329
330             for should_run, step_func, step_name in steps:
331                 if should_run:

```



```

332         if not step_func():
333             print(f"{self.msg('error')} {step_name}
334         })
335
336         return 1
337
338     # Success
339     print("┌" + "=" * 68 + "┐")
340     print("│" + self.msg("success").center(68) + "│")
341     print("└" + "=" * 68 + "┘")
342     print()
343     print(f"Next steps:")
344     print(f"  1. Go to installation directory:")
345     print(f"      cd {self.install_dir}")
346     print(f"  2. Activate venv:")
347     if sys.platform == "win32":
348         print(f"      venv\\Scripts\\activate")
349     else:
350         print(f"      source venv/bin/activate")
351     print(f"  3. Start the server:")
352     print(f"      python run.py")
353     print()
354     return 0
355
356 except KeyboardInterrupt:
357     print()
358     print(self.msg("abort"))
359     print()
360     return 1
361
362 except Exception as e:
363     print(f"{self.msg('error')} {e}")
364     return 1
365
366 def main():
367     """Main function"""
368     parser = argparse.ArgumentParser(
369         description="AI Breadboard installer",
370         formatter_class=argparse.RawDescriptionHelpFormatter,
371         epilog=""

```

```
369 Examples:
370     python scripts/cli/installer.py
371     python scripts/cli/installer.py --lang en
372     python scripts/cli/installer.py --install-dir /path/to/insta
ll
373     python scripts/cli/installer.py --skip-models
374         """
375     )
376
377     parser.add_argument(
378         "--lang",
379         choices=["en", "ru", "es", "he"],
380         default="en",
381         help="Installation language (default: en)"
382     )
383
384     parser.add_argument(
385         "--install-dir",
386         type=Path,
387         default=None,
388         help="Installation directory"
389     )
390
391     parser.add_argument(
392         "--skip-models",
393         action="store_true",
394         help="Skip model download"
395     )
396
397     parser.add_argument(
398         "--skip-venv",
399         action="store_true",
400         help="Skip venv creation"
401     )
402
403     parser.add_argument(
404         "--skip-deps",
405         action="store_true",
```

```
406         help="Skip dependency installation"
407     )
408
409     parser.add_argument(
410         "--skip-certs",
411         action="store_true",
412         help="Skip certificate setup"
413     )
414
415     args = parser.parse_args()
416
417     installer = Installer(
418         language=args.lang,
419         install_dir=args.install_dir
420     )
421
422     return installer.run(
423         skip_models=args.skip_models,
424         skip_venv=args.skip_venv,
425         skip_deps=args.skip_deps,
426         skip_certs=args.skip_certs,
427     )
428
429 if __name__ == "__main__":
430     sys.exit(main())
431
```

paths.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Cross-platform path management system
4 # =====
5 # Description:
6 #   Cross-platform path management that automatically determin
7 #   es correct paths
8 #   for Windows, Linux, and macOS systems using platform conve
9 #   ntions.
10 #
11 # File: paths.py
12 # Project: ai-breadboard
13 # Package: scripts.cli
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # =====
17
18 """Cross-platform path management system.
19 Automatically determines correct paths for Windows, Linux, and
20 macOS
21 using appropriate platform conventions and directory standards
22 ."""
23
24 import os
25 import sys
26 from pathlib import Path
27 from typing import Optional
28
29 try:
30     import platformdirs
31     HAS_PLATFORMDIRS = True
32 except ImportError:
```

```

30     HAS_PLATFORMDIRS = False
31
32 class CrossPlatformPaths:
33     """Centralized path system for all platforms.
34
35     Uses:
36     - Windows: %LOCALAPPDATA%, %USERPROFILE%
37     - Linux/macOS: ~/.local/share, ~/.config, ~/.cache
38     """
39
40     def __init__(self, app_name: str = "AI-Breadboard", app_au
41     thor: str = "hypo69"):
42
43         """Initialize cross-platform paths.
44
45         Args:
46             app_name: Application name.
47             app_author: Application author.
48         """
49         self.app_name = app_name
50         self.app_author = app_author
51         self._project_root = self._find_project_root()
52         self._cache = {}
53
54     @staticmethod
55     def _find_project_root() -> Path:
56         """Find project root by presence of main.py or config.
57         json.
58
59         Returns:
60             Path to project root.
61         """
62         current = Path(__file__).parent
63         for _ in range(10): # Search up to 10 levels
64             if (current / "main.py").exists() or (current / "c
65             onfig.json").exists():
66                 return current
67             current = current.parent
68         # Fallback to AIBREADBOARD_DIR or current directory

```

```

65         return Path(os.environ.get("AIBREADBOARD_DIR", os.getc
wd()))
66
67     @property
68     def project_root(self) -> Path:
69         """Project root directory."""
70         return self._project_root
71
72     @property
73     def data_dir(self) -> Path:
74         """Application data directory.
75
76         Windows: %LOCALAPPDATA%\AI-Breadboard
77         Linux: ~/.local/share/AI-Breadboard
78         macOS: ~/Library/Application Support/AI-Breadboard
79         """
80         if HAS_PLATFORMDIRS:
81             return Path(platformdirs.user_data_dir(self.app_na
me, self.app_author))
82
83         # Fallback for systems without platformdirs
84         if sys.platform == "win32":
85             base = os.environ.get("LOCALAPPDATA", str(Path.hom
e() / "AppData" / "Local"))
86             return Path(base) / self.app_name
87         elif sys.platform == "darwin":
88             return Path.home() / "Library" / "Application Supp
ort" / self.app_name
89         else:
90             return Path.home() / ".local" / "share" / self.app
_name
91
92     @property
93     def config_dir(self) -> Path:
94         """Configuration directory.
95
96         Windows: %LOCALAPPDATA%\AI-Breadboard\config
97         Linux: ~/.config/AI-Breadboard

```

```

98         macOS: ~/Library/Preferences/AI-Breadboard
99         """
100         if HAS_PLATFORMDIRS:
101             return Path(platformdirs.user_config_dir(self.app_
name, self.app_author))
102
103         if sys.platform == "win32":
104             return self.data_dir / "config"
105         elif sys.platform == "darwin":
106             return Path.home() / "Library" / "Preferences" / s
elf.app_name
107         else:
108             return Path.home() / ".config" / self.app_name
109
110     @property
111     def cache_dir(self) -> Path:
112         """Cache directory.
113
114         Windows: %LOCALAPPDATA%\AI-Breadboard\Cache
115         Linux: ~/.cache/AI-Breadboard
116         macOS: ~/Library/Caches/AI-Breadboard
117         """
118         if HAS_PLATFORMDIRS:
119             return Path(platformdirs.user_cache_dir(self.app_n
ame, self.app_author))
120
121         if sys.platform == "win32":
122             return self.data_dir / "Cache"
123         elif sys.platform == "darwin":
124             return Path.home() / "Library" / "Caches" / self.a
pp_name
125         else:
126             return Path.home() / ".cache" / self.app_name
127
128     @property
129     def certs_dir(self) -> Path:
130         """SSL certificates directory.
131

```

```

132         Windows: %USERPROFILE%\certs
133         Linux: ~/.local/share/ca-certificates
134         macOS: ~/Library/Certs
135         """
136         if sys.platform == "win32":
137             return Path.home() / ".certs"
138         elif sys.platform == "darwin":
139             return Path.home() / "Library" / "Certs"
140         else:
141             return Path.home() / ".local" / "share" / "ca-cert
142 ificates"
143
144 @property
145 def certs_user_dir(self) -> Path:
146     """Alternative user certificates directory."""
147     return Path.home() / ".certs"
148
149 @property
150 def bin_dir(self) -> Path:
151     """Executable files directory.
152
153     Windows: %USERPROFILE%\local\bin
154     Linux/macOS: ~/.local/bin
155     """
156     return Path.home() / ".local" / "bin"
157
158 @property
159 def venv_dir(self) -> Path:
160     """Python virtual environment directory."""
161     return self.project_root / "venv"
162
163 @property
164 def venv_python(self) -> Path:
165     """Python interpreter in virtual environment."""
166     if sys.platform == "win32":
167         return self.venv_dir / "Scripts" / "python.exe"
168     else:
169         return self.venv_dir / "bin" / "python"

```



```
169
170     @property
171     def secrets_dir(self) -> Path:
172         """Secrets directory for API keys."""
173         return self.project_root / "core" / "secrets"
174
175     @property
176     def env_file(self) -> Path:
177         """env file path."""
178         return self.project_root / ".env"
179
180     @property
181     def config_file(self) -> Path:
182         """config.json file path."""
183         return self.project_root / "config.json"
184
185     @property
186     def requirements_main(self) -> Path:
187         """Main requirements.txt file."""
188         return self.project_root / "requirements.txt"
189
190     @property
191     def requirements_core(self) -> Path:
192         """Core requirements file."""
193         return self.project_root / "install" / "req" / "requirements-core.txt"
194
195     @property
196     def requirements_ai(self) -> Path:
197         """AI requirements file."""
198         return self.project_root / "install" / "req" / "requirements-ai.txt"
199
200     @property
201     def requirements_test(self) -> Path:
202         """Test requirements file."""
203         return self.project_root / "install" / "req" / "requirements-test.txt"
```

```

204
205     @property
206     def requirements_docs(self) -> Path:
207         """Documentation requirements file."""
208         return self.project_root / "install" / "req" / "requirements-docs.txt"
209
210     def get_env_vars(self) -> dict:
211         """Get required environment variables.
212
213         Returns:
214             Dictionary of environment variables.
215         """
216         return {
217             "AIBREASHBOARD_DIR": str(self.project_root),
218             "ASSIST_DIR": str(self.project_root),
219             "PYTHONUTF8": "1",
220             "PYTHONPATH": f"{self.project_root}{os.pathsep}{os
221             .environ.get('PYTHONPATH', ' ')}",
222         }
223
224     def setup_env(self) -> None:
225         """Set environment variables."""
226         for key, value in self.get_env_vars().items():
227             if value:
228                 os.environ[key] = value
229
230     def ensure_dirs(self) -> None:
231         """Create necessary directories."""
232         dirs = [
233             self.data_dir,
234             self.config_dir,
235             self.cache_dir,
236             self.certs_dir,
237             self.certs_user_dir,
238             self.bin_dir,
239             self.secrets_dir,

```

```
240 |
241 |         for directory in dirs:
242 |             directory.mkdir(parents=True, exist_ok=True)
243 |
244 | # Global instance
245 | _paths_instance: Optional[CrossPlatformPaths] = None
246 |
247 | def get_paths() -> CrossPlatformPaths:
248 |     """Get global CrossPlatformPaths instance.
249 |
250 |     Returns:
251 |         Global paths instance.
252 |     """
253 |     global _paths_instance
254 |     if _paths_instance is None:
255 |         _paths_instance = CrossPlatformPaths()
256 |     return _paths_instance
257 |
258 | def init_paths() -> CrossPlatformPaths:
259 |     """Initialize paths and set environment variables.
260 |
261 |     Returns:
262 |         Initialized paths instance.
263 |     """
264 |     paths = get_paths()
265 |     paths.setup_env()
266 |     paths.ensure_dirs()
267 |     return paths
268 |
```

utils.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Cross-platform utility functions
4 # =====
5 # Description:
6 #   Utilities for cross-platform operations including port management,
7 #   process handling, environment variable manipulation, and command execution.
8 #
9 # File: utils.py
10 # Project: ai-breadboard
11 # Package: scripts.cli
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """Cross-platform utility functions for system operations."""
17
18 import os
19 import sys
20 import socket
21 import subprocess
22 from pathlib import Path
23 from typing import Tuple, Optional, List
24
25 def find_available_port(host: str = "127.0.0.1", start_port: int = 8000, max_attempts: int = 100) -> int:
26     """Find an available port.
27
28     Args:
29         host: IP address to bind to.
30         start_port: Starting port number.
```

```

31         max_attempts: Maximum number of attempts.
32
33     Returns:
34         An available port number.
35
36     Raises:
37         RuntimeError: If no available ports found.
38     """
39     for port in range(start_port, start_port + max_attempts):
40         try:
41             with socket.socket(socket.AF_INET, socket.SOCK_STR
EAM) as s:
42                 s.bind((host, port))
43                 s.listen(1)
44                 return port
45         except OSError:
46             continue
47
48     raise RuntimeError(f"No available ports found in range {st
art_port}-{start_port + max_attempts}")
49
50 def is_port_open(port: int, host: str = "127.0.0.1") -> bool:
51     """Check if port is open (process listening).
52
53     Args:
54         port: Port number to check.
55         host: IP address to connect to.
56
57     Returns:
58         True if port is open, False otherwise.
59     """
60     try:
61         with socket.socket(socket.AF_INET, socket.SOCK_STREAM)
as s:
62             s.settimeout(1)
63             result = s.connect_ex((host, port))
64             return result == 0
65     except Exception:

```

```

66         return False
67
68 def get_process_on_port(port: int) -> Optional[Tuple[int, str]
69 ]:
69     """Get process ID and name listening on port.
70
71     Args:
72         port: Port number.
73
74     Returns:
75         Tuple (PID, process_name) or None if no process found.
76     """
77     try:
78         if sys.platform == "win32":
79             # Windows: netstat
80             output = subprocess.run(
81                 ["netstat", "-aon"],
82                 capture_output=True,
83                 text=True,
84                 check=False
85             ).stdout
86
87             for line in output.split("\n"):
88                 if f":{port}" in line and "LISTENING" in line:
89                     parts = line.split()
90                     if len(parts) > 4:
91                         pid_str = parts[-1]
92                         try:
93                             pid = int(pid_str)
94                             proc_name = get_process_name(pid)
95                             return (pid, proc_name)
96                         except (ValueError, Exception):
97                             pass
98
99         else:
100             # Linux/macOS: lsof
101             try:
102                 output = subprocess.run(

```

```

103         ["lsof", "-i", f":{port}"],
104         capture_output=True,
105         text=True,
106         check=False
107     ).stdout
108
109     for line in output.split("\n")[1:]: # Skip header
110         if line.strip():
111             parts = line.split()
112             if len(parts) > 1:
113                 proc_name = parts[0]
114                 pid_str = parts[1]
115                 try:
116                     pid = int(pid_str)
117                     return (pid, proc_name)
118                 except ValueError:
119                     pass
120             except FileNotFoundError:
121                 # lsof not available
122                 return (None, "unknown")
123
124     except Exception as e:
125         print(f"Error getting process on port {port}: {e}")
126
127     return None
128
129 def get_process_name(pid: int) -> str:
130     """Get process name by PID.
131
132     Args:
133         pid: Process ID.
134
135     Returns:
136         Process name or 'unknown'.
137     """
138     try:
139         if sys.platform == "win32":

```

```

140         output = subprocess.run(
141             ["tasklist", "/FI", f"PID eq {pid}"],
142             capture_output=True,
143             text=True,
144             check=False
145         ).stdout
146
147         for line in output.split("\n"):
148             if str(pid) in line:
149                 parts = line.split()
150                 if parts:
151                     return parts[0]
152
153         else:
154             output = subprocess.run(
155                 ["ps", "-p", str(pid), "-o", "comm="],
156                 capture_output=True,
157                 text=True,
158                 check=False
159             ).stdout
160
161             name = output.strip()
162             if name:
163                 return name
164
165     except Exception:
166         pass
167
168     return "unknown"
169
170 def kill_process(pid: int, force: bool = False) -> bool:
171     """Terminate process.
172
173     Args:
174         pid: Process ID.
175         force: Use SIGKILL (Unix) or /F flag (Windows).
176
177     Returns:

```



```

178         True if successful, False otherwise.
179     """
180     try:
181         if sys.platform == "win32":
182             cmd = ["taskkill", "/PID", str(pid)]
183             if force:
184                 cmd.append("/F")
185
186             result = subprocess.run(cmd, capture_output=True,
187 check=False)
188             return result.returncode == 0
189
190         else:
191             import signal
192             signal_type = signal.SIGKILL if force else signal.
193 SIGTERM
194             os.kill(pid, signal_type)
195             return True
196
197     except Exception as e:
198         print(f"Error killing process {pid}: {e}")
199         return False
200
201 def run_command(
202     cmd: List[str],
203     cwd: Optional[Path] = None,
204     env: Optional[dict] = None,
205     check: bool = False,
206     shell: bool = False
207 ) -> subprocess.CompletedProcess:
208     """Run command cross-platform.
209
210     Args:
211         cmd: Command and arguments list.
212         cwd: Working directory.
213         env: Environment variables.
214         check: Raise error if return code != 0.
215         shell: Use shell.

```

```

214
215     Returns:
216         CompletedProcess result.
217     """
218     full_env = os.environ.copy()
219     if env:
220         full_env.update(env)
221
222     return subprocess.run(
223         cmd,
224         cwd=cwd,
225         env=full_env,
226         shell=shell,
227         check=check
228     )
229
230 def which(command: str) -> Optional[Path]:
231     """Find command path (cross-platform which/where).
232
233     Args:
234         command: Command name to find.
235
236     Returns:
237         Path to command or None if not found.
238     """
239     try:
240         result = subprocess.run(
241             ["where" if sys.platform == "win32" else "which",
242             command],
243             capture_output=True,
244             text=True,
245             check=False
246         )
247
248         if result.returncode == 0:
249             path_str = result.stdout.strip().split("\n")[0]
250             return Path(path_str)

```

```
251 |     except Exception:
252 |         pass
253 |
254 |     return None
255 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: CLI modules for cross-platform operations
4 # =====
5 # Description:
6 #   CLI modules providing command-line interface utilities for
7 #   cross-platform
8 #   compatibility and system operations.
9 #
10 # File: __init__.py
11 # Project: ai-breadboard
12 # Package: scripts.cli
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16 """CLI modules package for cross-platform operations."""
17
```

analyze_dependencies.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Find all Python files in project root
4 # =====
5 # Description:
6 #   Analyzes dependencies between scripts in project. Helps de
7 #   termine which
8 #   scripts are used by others and which can be safely deleted
9 #   .
10 #
11 # File: analyze_dependencies.py
12 # Project: ai-breadboard
13 # Package: scripts.dev
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # =====
17
18 """Script dependency analysis utility.
19 Analyzes dependencies between project scripts to identify usag
20 e patterns
21 and help determine which scripts can be safely removed."""
22
23 import ast
24 import re
25 from pathlib import Path
26 from collections import defaultdict
27 from typing import Set, Dict, List
28
29 class ScriptAnalyzer:
30     """Analyzes Python scripts for dependencies."""
31
32     def __init__(self, root_dir: Path):
```

```

31 |         """Initialize analyzer.
32 |
33 |     Args:
34 |         root_dir: Root directory to analyze.
35 |     """
36 |     self.root_dir = root_dir
37 |     self.scripts = {}
38 |     self.dependencies = defaultdict(set)
39 |     self.imports = defaultdict(set)
40 |
41 | def find_all_scripts(self) -> List[Path]:
42 |     """Find all Python files in root directory.
43 |
44 |     Returns:
45 |         List of Path objects for .py files.
46 |     """
47 |     scripts = []
48 |     for py_file in self.root_dir.glob("*.py"):
49 |         if py_file.name not in ['analyze_dependencies.py',
| 'update_scripts_documentation.py']:
50 |             scripts.append(py_file)
51 |     return scripts
52 |
53 | def analyze_imports(self, script_path: Path) -> Set[str]:
54 |     """Extract imports from Python script.
55 |
56 |     Args:
57 |         script_path: Path to script file.
58 |
59 |     Returns:
60 |         Set of imported module names.
61 |     """
62 |     imports = set()
63 |     try:
64 |         with open(script_path, 'r', encoding='utf-8') as f
| :
65 |             content = f.read()
66 |

```

```

67 |         # Parse imports
68 |         import_pattern = r'(?:(from|import)\s+[\w.]+)'
69 |         for match in re.finditer(import_pattern, content):
70 |             import_str = match.group()
71 |             parts = import_str.split()
72 |             if parts[0] == 'import':
73 |                 module = parts[1].split('.')[0]
74 |             elif parts[0] == 'from':
75 |                 module = parts[1]
76 |             else:
77 |                 continue
78 |             imports.add(module)
79 |     except Exception:
80 |         pass
81 |
82 |     return imports
83 |
84 | def analyze_all(self) -> None:
85 |     """Analyze all scripts in directory."""
86 |     scripts = self.find_all_scripts()
87 |
88 |     for script_path in scripts:
89 |         imports = self.analyze_imports(script_path)
90 |         self.scripts[script_path.name] = {
91 |             'path': script_path,
92 |             'imports': imports
93 |         }
94 |         self.imports[script_path.name] = imports
95 |
96 | def report(self) -> None:
97 |     """Generate dependency report."""
98 |     print("\n" + "="*70)
99 |     print("SCRIPT DEPENDENCY ANALYSIS REPORT")
100 |     print("="*70 + "\n")
101 |
102 |     # Summary
103 |     print(f"Total scripts analyzed: {len(self.scripts)}\n"
)

```

```

104
105     # Scripts with most dependencies
106     print("Scripts with most dependencies:")
107     print("-" * 70)
108     sorted_scripts = sorted(
109         self.scripts.items(),
110         key=lambda x: len(x[1]['imports']),
111         reverse=True
112     )
113     for script_name, info in sorted_scripts[:5]:
114         print(f"    {script_name}: {len(info['imports'])} im
ports")
115
116     print("\n" + "="*70)
117     print("Import inventory:")
118     print("-" * 70)
119
120     all_imports = set()
121     for imports in self.imports.values():
122         all_imports.update(imports)
123
124     external_imports = []
125     internal_imports = []
126     standard_imports = []
127
128     std_modules = {'sys', 'os', 'json', 'pathlib', 'argpar
se', 'subprocess',
129                   'asyncio', 're', 'typing', 'collections
', 'datetime', 'urllib'}
130
131     for imp in sorted(all_imports):
132         if imp.startswith(('core', 'plugins', 'scripts')):
133             internal_imports.append(imp)
134         elif imp in std_modules or imp.startswith(('__', '
_')):
135             standard_imports.append(imp)
136         else:
137             external_imports.append(imp)

```



```
138 |
139 |         print(f"\nStandard library: {len(standard_imports)} mo
      | dules")
140 |         print(f"Internal modules: {len(internal_imports)} modu
      | les")
141 |         if internal_imports:
142 |             for imp in internal_imports:
143 |                 print(f"    - {imp}")
144 |
145 |         print(f"\nExternal packages: {len(external_imports)} p
      | ackages")
146 |         if external_imports:
147 |             for imp in external_imports[:10]:
148 |                 print(f"    - {imp}")
149 |             if len(external_imports) > 10:
150 |                 print(f"    ... and {len(external_imports) - 10}
      | more")
151 |
152 |         print("\n" + "="*70 + "\n")
153 |
154 | def main():
155 |     """Main entry point."""
156 |     from header import __root__
157 |
158 |     analyzer = ScriptAnalyzer(__root__)
159 |     analyzer.analyze_all()
160 |     analyzer.report()
161 |
162 | if __name__ == "__main__":
163 |     main()
164 |
```

assist_cli.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Create state directory for AI assistant persis
  tence
4 # =====
5 # Description:
6 #   Creates state directory if it doesn't exist for persisting
  AI provider
7 #   and model selections across CLI sessions.
8 #
9 # File: assist_cli.py
10 # Project: ai-breadboard
11 # Package: scripts.dev
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """AI Assistant CLI tool for model and provider management.
17
18 Provides command-line interface for managing AI model provider
  s, models,
19 configuration, and running queries against selected models."""
20
21 from __future__ import annotations
22
23 import argparse
24 import json
25 import os
26 import subprocess
27 import sys
28 import urllib.request
29 from pathlib import Path
30
```

```

31 import header
32 from header import __root__
33 from dotenv import load_dotenv
34
35 # Directory for storing selected provider/model state
36 _STATE_DIR = __root__ / ".assist_state"
37 _PROVIDER_FILE = _STATE_DIR / "provider.json"
38 _MODEL_FILE = _STATE_DIR / "model.json"
39
40 def _ensure_state_dir() -> None:
41     """Create state directory if not exists."""
42     _STATE_DIR.mkdir(parents=True, exist_ok=True)
43
44 def _get_selected_provider() -> dict:
45     """Get currently selected provider.
46
47     Returns:
48         Dictionary with selected provider info.
49     """
50     if not _PROVIDER_FILE.exists():
51         return {}
52     try:
53         with open(_PROVIDER_FILE, "r", encoding="utf-8") as f:
54             return json.load(f)
55     except Exception:
56         return {}
57
58 def _set_selected_provider(provider_name: str) -> None:
59     """Save selected provider.
60
61     Args:
62         provider_name: Name of provider to select.
63     """
64     _ensure_state_dir()
65     with open(_PROVIDER_FILE, "w", encoding="utf-8") as f:
66         json.dump({"provider": provider_name}, f, ensure_ascii
67 =False, indent=2)

```

```

68 def _get_selected_model() -> dict:
69     """Get currently selected model.
70
71     Returns:
72         Dictionary with selected model info.
73     """
74     if not _MODEL_FILE.exists():
75         return {}
76     try:
77         with open(_MODEL_FILE, "r", encoding="utf-8") as f:
78             return json.load(f)
79     except Exception:
80         return {}
81
82 def _set_selected_model(model_name: str, provider: str = "") -
    > None:
83     """Save selected model.
84
85     Args:
86         model_name: Name of model to select.
87         provider: Optional provider name.
88     """
89     _ensure_state_dir()
90     data = {"model": model_name}
91     if provider:
92         data["provider"] = provider
93     with open(_MODEL_FILE, "w", encoding="utf-8") as f:
94         json.dump(data, f, ensure_ascii=False, indent=2)
95
96 # Setup UTF-8 for Windows console output
97 if hasattr(sys.stdout, "reconfigure"):
98     sys.stdout.reconfigure(encoding="utf-8", errors="replace")
99 if hasattr(sys.stderr, "reconfigure"):
100     sys.stderr.reconfigure(encoding="utf-8", errors="replace")
101
102 load_dotenv(__root__ / ".env")
103
104 # ANSI colors for terminal output

```

```

105 C_RESET = "\033[0m"
106 C_BOLD = "\033[1m"
107 C_CYAN = "\033[96m"
108 C_GREEN = "\033[92m"
109 C_YELLOW = "\033[93m"
110 C_RED = "\033[91m"
111 C_GRAY = "\033[90m"
112 C_MAGENTA = "\033[95m"
113
114 def _get_venv_python() -> str:
115     """Get Python interpreter path in virtual environment.
116
117     Returns:
118         Path to Python executable.
119     """
120     venv_py = __root__ / "venv" / "Scripts" / "python.exe"
121     if venv_py.exists():
122         return str(venv_py)
123     return sys.executable
124
125 def _get_config() -> dict:
126     """Load configuration from config.json.
127
128     Returns:
129         Dictionary with configuration or empty dict if not found.
130     """
131     cfg_path = __root__ / "config.json"
132     if not cfg_path.exists():
133         return {}
134     try:
135         with open(cfg_path, "r", encoding="utf-8") as f:
136             return json.load(f)
137     except Exception as e:
138         print(f"{C_RED}Error reading config.json: {e}{C_RESET}")
139     return {}
140

```

```

141 def _get_occupied_port_pids(port: int) -> list[int]:
142     """Get process IDs listening on specified port.
143
144     Args:
145         port: Port number to check.
146
147     Returns:
148         List of PIDs listening on port.
149     """
150     pids = []
151     if sys.platform.startswith("win"):
152         try:
153             out = subprocess.check_output(f"netstat -aon", she
ll=True, stderr=subprocess.DEVNULL).decode(errors="ignore")
154             for line in out.splitlines():
155                 if f":{port} " in line and "LISTENING" in line
:
156                     parts = line.strip().split()
157                     if parts and parts[-1].isdigit():
158                         pid = int(parts[-1])
159                         if pid > 0 and pid not in pids:
160                             pids.append(pid)
161         except Exception:
162             pass
163     return pids
164
165 def cmd_start(args: argparse.Namespace) -> int:
166     """Start project services.
167
168     Args:
169         args: Command arguments.
170
171     Returns:
172         Exit code from subprocess.
173     """
174     target = getattr(args, "service", "run") or "run"
175     target = target.lower()
176

```

```
177     script_map = {
178         "run": "run.ps1",
179         "all": "run.ps1",
180         "unicorn": "Run-Unicorn.ps1",
181         "uunicorn": "Run-Unicorn.ps1",
182         "light": "Run-LightServer.ps1",
183         "foundry": "Run-Foundry.ps1",
184     }
185
186     script_name = script_map.get(target)
187     if not script_name:
188         print(f"{C_RED}[] Unknown startup target: '{target}'{C_
189 RESET}")
190         print(f"{C_GRAY}Available targets: run (default), unic
191 orn, light, foundry, all{C_RESET}")
192         return 1
193
194     script_path = __root__ / script_name
195     if not script_path.exists():
196         script_path = __root__ / "launchers" / script_name
197     if not script_path.exists():
198         print(f"{C_RED}[] Script not found: {script_path}{C_RES
199 ET}")
200         return 1
201
202     print(f"{C_CYAN}[] Starting: {script_name}...{C_RESET}")
203     if script_name == "Run-Foundry.ps1":
204         cmd = ["powershell", "-ExecutionPolicy", "Bypass", "-F
205 ile", str(script_path), "-Action", "start"]
206     else:
207         cmd = ["powershell", "-ExecutionPolicy", "Bypass", "-F
208 ile", str(script_path)]
209
210     return subprocess.call(cmd, cwd=str(__root__))
211
212 def cmd_stop(args: argparse.Namespace) -> int:
213     """Stop server and related processes.
```

```

210     Args:
211         args: Command arguments.
212
213     Returns:
214         Exit code.
215     """
216     target = getattr(args, "service", "all") or "all"
217     target = target.lower()
218
219     cfg = _get_config()
220     server_cfg = cfg.get("server", {})
221     port = int(server_cfg.get("port", 8000))
222
223     stopped_any = False
224
225     if target in ("server", "all", "unicorn", "uvicorn", "ligh
t"):
226         pids = _get_occupied_port_pids(port)
227         if pids:
228             print(f"{C_YELLOW}␣ Stopping FastAPI server on por
t {port} (PID: {pids})...{C_RESET}")
229             for pid in pids:
230                 try:
231                     subprocess.run(["taskkill", "/F", "/PID",
str(pid)], capture_output=True)
232                     print(f" {C_GREEN}␣ Terminated process PI
D {pid}{C_RESET}")
233                     stopped_any = True
234                 except Exception as e:
235                     print(f" {C_RED}␣ Failed to terminate PID
{pid}: {e}{C_RESET}")
236             else:
237                 print(f"{C_GRAY}Port {port} is free (FastAPI serve
r not running){C_RESET}")
238
239     if target in ("foundry", "all"):
240         foundry_script = __root__ / "launchers" / "Run-Foundry
.ps1"

```



```

241         if not foundry_script.exists():
242             foundry_script = __root__ / "Run-Foundry.ps1"
243         if foundry_script.exists():
244             print(f"{C_YELLOW}␣ Stopping Foundry service...{C_
RESET}")
245             subprocess.run(["powershell", "-ExecutionPolicy",
"Bypass", "-File", str(foundry_script), "-Action", "stop"], cw
d=str(__root__))
246             stopped_any = True
247
248         if not stopped_any and not pids:
249             print(f"{C_GREEN}All services already stopped.{C_RESET
}")
250         return 0
251
252 def cmd_restart(args: argparse.Namespace) -> int:
253     """Restart server.
254
255     Args:
256         args: Command arguments.
257
258     Returns:
259         Exit code.
260     """
261     print(f"{C_CYAN}␣ Restarting services...{C_RESET}")
262     cmd_stop(args)
263     return cmd_start(args)
264
265 def cmd_status(args: argparse.Namespace) -> int:
266     """Check current server status.
267
268     Args:
269         args: Command arguments.
270
271     Returns:
272         Exit code.
273     """
274     cfg = _get_config()

```

```

275 |     server_cfg = cfg.get("server", {})
276 |     port = int(server_cfg.get("port", 8000))
277 |     use_ssl = bool(server_cfg.get("use_ssl", False))
278 |     mode = str(server_cfg.get("mode", "DEV"))
279 |     reload_on = bool(server_cfg.get("reload", True))
280 |
281 |     proto = "https" if use_ssl else "http"
282 |     local_url = f"{proto}://localhost:{port}/"
283 |
284 |     print(f"\n{C_BOLD}{C_CYAN}Server Status Report{C_RESET}\n"
285 | )
286 |     pids = _get_occupied_port_pids(port)
287 |     if pids:
288 |         print(f"    {C_BOLD}FastAPI Server:{C_RESET}        {C_GREEN
289 | N}● RUNNING{C_RESET}")
290 |         print(f"    {C_BOLD}URL:{C_RESET}                {C_CYAN
291 | }{local_url}{C_RESET}")
292 |         print(f"    {C_BOLD}Process IDs:{C_RESET}        {'', '.j
293 | oin(str(p) for p in pids)}")
294 |     else:
295 |         print(f"    {C_BOLD}FastAPI Server:{C_RESET}        {C_RED}
296 | o STOPPED{C_RESET}")
297 |         print(f"    {C_BOLD}Expected port:{C_RESET}        {port}"
298 | )
299 |
300 |     print(f"    {C_BOLD}Mode / SSL:{C_RESET}                {mode} | SS
301 | L: {'ON' if use_ssl else 'OFF'} | Autoreload: {'ON' if reload_
302 | on else 'OFF'}")
303 |     print("")
304 |     return 0
305 |
306 | def main():
307 |     """Main entry point."""
308 |     parser = argparse.ArgumentParser(description="AI Assistant
309 | CLI Tool")
310 |     subparsers = parser.add_subparsers(dest="command", help="A
311 | vailable commands")

```

```
303
304     # Start command
305     start_parser = subparsers.add_parser("start", help="Start
services")
306     start_parser.add_argument("service", nargs="?", help="Serv
ice to start")
307     start_parser.set_defaults(func=cmd_start)
308
309     # Stop command
310     stop_parser = subparsers.add_parser("stop", help="Stop ser
vices")
311     stop_parser.add_argument("service", nargs="?", help="Servi
ce to stop")
312     stop_parser.set_defaults(func=cmd_stop)
313
314     # Restart command
315     restart_parser = subparsers.add_parser("restart", help="Re
start services")
316     restart_parser.add_argument("service", nargs="?", help="Se
rvice to restart")
317     restart_parser.set_defaults(func=cmd_restart)
318
319     # Status command
320     status_parser = subparsers.add_parser("status", help="Show
status")
321     status_parser.set_defaults(func=cmd_status)
322
323     args = parser.parse_args()
324
325     if not args.command:
326         parser.print_help()
327         return 0
328
329     if hasattr(args, "func"):
330         return args.func(args)
331     return 0
332
333 if __name__ == "__main__":
```

```
334 | sys.exit(main())
```

```
335 |
```

bot_runner.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Run Telegram bot with full plugin and AI model
  suite
4 # =====
5 # Description:
6 #   Runs Telegram bot in separate process, independent from uv
 icorn server.
7 #   Loads all plugins and initializes AI models for bot functi
  onality.
8 #
9 # File: bot_runner.py
10 # Project: ai-breadboard
11 # Package: scripts.dev
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """Telegram bot runner script.
17
18 Launches Telegram bot in separate process with full plugin sui
  te and
19 AI model integration for chat functionality."""
20
21 from __future__ import annotations
22
23 import asyncio
24 import os
25 import sys
26 import signal
27 from pathlib import Path
28
29 from dotenv import load_dotenv
```

```

30
31 import header
32 from header import __root__
33
34 load_dotenv(__root__ / '.env')
35
36 from src.utils.jjson import j_loads_ns
37 from src.logger import logger
38
39 from src.config import server_cfg, ai_cfg, tts_cfg, logging_cf
   g
40
41 async def _run_bot() -> None:
42     """Run Telegram bot with full plugin and AI model suite.
43
44     Initializes bot with configured AI model (either Google Ge
   nerative AI
45     or Foundry model), loads plugins, and starts async event l
   oop.
46     """
47     from src.ai import GoogleGenerativeAI
48     from src.utils.file import read_text_file
49     from plugins import load_plugins
50
51     _system_instruction = read_text_file(__root__ / 'prompts'
   / 'chat' / 'system_instruction.md') or ''
52     _api_key_names = [n.strip() for n in os.getenv('GEMINI_API
   _KEY_NAMES', '').split(',') if n.strip()]
53
54     use_foundry = getattr(ai_cfg, 'use_foundry', False) if ai_
   cfg else False
55     foundry_model_id = getattr(ai_cfg, 'foundry_model_id', 'qw
   en2.5-1.5b') if ai_cfg else 'qwen2.5-1.5b'
56
57     if use_foundry:
58         from src.ai.foundry_chat import FoundryChatBase
59         model = FoundryChatBase(model_id=foundry_model_id, sys
   tem_prompt=_system_instruction)

```

```
60     else:
61         model = GoogleGenerativeAI(api_key_names=_api_key_name
s, system_instruction=_system_instruction)
62
63     plugins = load_plugins(model)
64     tg_plugin = plugins.get('telegram_bot')
65
66     if not tg_plugin:
67         logger.warning('Telegram bot plugin not found - bot_ru
nner exiting.')
68         return
69
70     if hasattr(tg_plugin, 'set_plugins'):
71         tg_plugin.set_plugins(plugins)
72
73     loop = asyncio.get_event_loop()
74
75     stop_event = asyncio.Event()
76
77     def _handle_exit(*_):
78         stop_event.set()
79
80     for sig in (signal.SIGINT, signal.SIGTERM):
81         try:
82             loop.add_signal_handler(sig, _handle_exit)
83         except (NotImplementedError, AttributeError):
84             signal.signal(sig, _handle_exit)
85
86     logger.info('Telegram bot started (separate process)')
87     try:
88         await tg_plugin.start()
89         await stop_event.wait()
90     finally:
91         await tg_plugin.stop()
92         logger.info('Telegram bot stopped')
93
94 if __name__ == '__main__':
95     asyncio.run(_run_bot())
```


convert_to_md.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Convert media data to Markdown format
4 # =====
5 # Description:
6 #   Converts media data from JSON format to formatted Markdown
7 #   reports
8 #   for human-readable display and documentation purposes.
9 #
10 # File: convert_to_md.py
11 # Project: ai-breadboard
12 # Package: scripts.dev
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16 """Media data to Markdown conversion utility.
17
18 Converts media metadata and descriptions from JSON format into
19 formatted
20 Markdown reports with sections for plot, seasons, episodes, an
21 d verdicts."""
22
23 import json
24 from pathlib import Path
25 from header import __root__
26
27 REPORTS_DIR = __root__ / 'tmp' / 'reports'
28
29 # Load JSON data (replace with real data from previous step)
30 # Example structure from previous processing
31 json_data = """
32 {
```

```

31 | "title": "Example Title",
32 | "plot": "Plot description here...",
33 | "seasons": [
34 |     {
35 |         "season_number": 1,
36 |         "season_plot_summary": "Season summary...",
37 |         "episodes": [
38 |             {"episode_number": 1, "title": "Episode 1", "detailed_
description": "Description...", "final_verdict": "Verdict text
."}
39 |         ]
40 |     }
41 | ]
42 | }
43 | """
44 | data = json.loads(json_data)
45 |
46 | md_content = f"# {data['title']}\n\n"
47 | md_content += f"## Plot\n{data['plot']}\n\n"
48 |
49 | for season in data.get('seasons', []):
50 |     md_content += f"## Season {season['season_number']}\n"
51 |     md_content += f"{season['season_plot_summary']}\n\n"
52 |     md_content += "### Episodes\n"
53 |     for ep in season.get('episodes', []):
54 |         md_content += f"#### {ep['episode_number']}. {ep['titl
e']}\n"
55 |         md_content += f"{ep['detailed_description']}\n\n"
56 |         md_content += f"**Verdict:** {ep['final_verdict']}\n\n"
57 |
58 | output_path = REPORTS_DIR / "media_report.md"
59 | output_path.write_text(md_content, encoding='utf-8')
60 | print(f" Report saved: {output_path}")
61 |

```

generate_coverage_report.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Generate code coverage analysis reports
4 # =====
5 # Description:
6 #   Generates coverage analysis reports for project tests. Pro
7 #   duces terminal
8 #   summary, HTML report, and XML report. Can check against co
9 #   verage threshold.
10 #
11 # File: generate_coverage_report.py
12 # Project: ai-breadboard
13 # Package: scripts.dev
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # =====
17
18 """Code coverage report generation utility.
19
20 Generates comprehensive coverage analysis reports including te
21 rминаl output,
22 HTML report, and XML format. Supports threshold checking."""
23
24 import os
25 import sys
26 from pathlib import Path
27 import json
28
29 def generate_coverage_report():
30     """Generate code coverage report.
31
32     Generates terminal report, HTML report, and XML report usi
33     ng coverage.py.
```

```

30
31 Returns:
32     0 if successful, 1 if error.
33     """
34 try:
35     import coverage
36
37     cov = coverage.Coverage(config_file='.coveragerc')
38     cov.load()
39
40     # Results summary
41     cov.report(show_missing=True)
42
43     # HTML report
44     html_dir = Path('tests/coverage')
45     html_dir.mkdir(parents=True, exist_ok=True)
46     cov.html_report(directory=str(html_dir))
47
48     # XML report
49     cov.xml_report(outfile='coverage.xml')
50
51     print(f"\nOK: Coverage report generated")
52     print(f"    HTML: {html_dir / 'index.html'}")
53     print(f"    XML: coverage.xml")
54
55     return 0
56
57 except ImportError:
58     print("ERROR: coverage not installed. Install: pip ins
59 tall coverage")
60     return 1
61 except Exception as e:
62     print(f"ERROR: Error generating report: {e}")
63     return 1
64
65 def check_coverage_threshold(threshold=80):
66     """Check if coverage meets minimum threshold.

```

```

67 |     Args:
68 |         threshold: Minimum coverage percentage required (default 80).
69 |
70 |     Returns:
71 |         0 if threshold met, 1 if not met or error.
72 |     """
73 |     try:
74 |         import coverage
75 |
76 |         cov = coverage.Coverage(config_file='.coveragerc')
77 |         cov.load()
78 |
79 |         _, missing, total, percentage = cov.report()
80 |
81 |         print(f"\nCode coverage: {percentage:.1f}%")
82 |         print(f"Minimum required: {threshold}%")
83 |
84 |         if percentage >= threshold:
85 |             print("OK: Coverage threshold met")
86 |             return 0
87 |         else:
88 |             print(f"ERROR: Coverage threshold NOT met (need {threshold - percentage:.1f}% more)")
89 |             return 1
90 |
91 |     except ImportError:
92 |         print("ERROR: coverage not installed")
93 |         return 1
94 |
95 | def main():
96 |     """Main function."""
97 |     import argparse
98 |     parser = argparse.ArgumentParser(description="Generate code coverage reports")
99 |     parser.add_argument("--threshold", "-t", type=float, default=80, help="Coverage threshold percentage")
100 |     parser.add_argument("--check", "-c", action="store_true",

```

```
    help="Only check threshold")
101     parser.add_argument("--report", "-r", action="store_true",
    help="Generate report")
102
103     args = parser.parse_args()
104
105     if args.check:
106         return check_coverage_threshold(args.threshold)
107
108     if args.report:
109         return generate_coverage_report()
110
111     # Default - check first, then generate report
112     check_result = check_coverage_threshold(args.threshold)
113     report_result = generate_coverage_report()
114
115     return max(check_result, report_result)
116
117 if __name__ == "__main__":
118     sys.exit(main())
119
```

package_skill.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Package Gemini skills into distributable archives
4 # =====
5 # Description:
6 #   Utility for packaging Gemini CLI skills into ZIP archives for distribution.
7 #   Automatically excludes build artifacts and version control directories.
8 #
9 # File: package_skill.py
10 # Project: ai-breadboard
11 # Package: scripts.dev
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """Gemini skills packaging utility.
17
18 Packages skill directories into ZIP archive format for distribution,
19 automatically excluding build artifacts and version control files."""
20
21 import os
22 import zipfile
23 import argparse
24 from pathlib import Path
25
26 def package_skill(skill_dir, output_dir):
27     """Package a skill directory into a distributable archive.
28
```

```

29 |     Args:
30 |         skill_dir: Path to skill directory to package.
31 |         output_dir: Path to output directory for packaged skill
    | file.
32 |     """
33 |     skill_path = Path(skill_dir).resolve()
34 |     output_path = Path(output_dir).resolve()
35 |
36 |     if not output_path.exists():
37 |         output_path.mkdir(parents=True)
38 |
39 |     skill_name = skill_path.name
40 |     skill_file = output_path / f"{skill_name}.skill"
41 |
42 |     print(f"Packaging skill from {skill_path} to {skill_file}.
    | ..")
43 |
44 |     with zipfile.ZipFile(skill_file, 'w', zipfile.ZIP_DEFLATED
    | ) as zipf:
45 |         for root, dirs, files in os.walk(skill_path):
46 |             # Skip dist and git directories
47 |             dirs[:] = [d for d in dirs if d not in ['dist', '.
    | git', '__pycache__']]
48 |
49 |             for file in files:
50 |                 file_path = Path(root) / file
51 |                 # Relative path within archive
52 |                 arcname = file_path.relative_to(skill_path)
53 |                 zipf.write(file_path, arcname)
54 |
55 |     print(f"Skill packaged successfully: {skill_file}")
56 |
57 | if __name__ == "__main__":
58 |     parser = argparse.ArgumentParser(description="Gemini CLI s
    | kills packager")
59 |     parser.add_argument("skill_dir", help="Path to skill direc
    | tory")
60 |     parser.add_argument("output_dir", help="Path to dist direc

```



```
tory")
61 |     args = parser.parse_args()
62 |
63 |     package_skill(args.skill_dir, args.output_dir)
64 |
```

run_tests.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Test runner with coverage analysis
4 # =====
5 # Description:
6 #   Runs project tests with optional coverage analysis and HTML report generation.
7 #   Supports verbose output, test markers filtering, and coverage report viewing.
8 #
9 # File: run_tests.py
10 # Project: ai-breadboard
11 # Package: scripts.dev
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """Test runner script for ai-breadboard project.
17
18 Provides command-line interface for running project tests with coverage
19 analysis, verbose output, and filtering by test markers."""
20
21 import subprocess
22 import sys
23 import argparse
24 from pathlib import Path
25
26 def run_tests(coverage=False, verbose=False, markers=None):
27     """Run project tests.
28
29     Args:
30         coverage: Include coverage analysis and report generation.
```

```

ion.
31         verbose: Enable verbose test output.
32         markers: pytest markers to filter tests (e.g., unit, i
integration, slow).
33
34     Returns:
35         pytest return code (0 for success).
36     """
37     cmd = ["pytest"]
38
39     # Coverage
40     if coverage:
41         cmd.extend([
42             "--cov=src",
43             "--cov=plugins",
44             "--cov=scripts",
45             "--cov-report=term-missing",
46             "--cov-report=html:tests/coverage",
47             "--cov-report=xml:coverage.xml",
48             "--cov-config=.coveragerc"
49         ])
50
51     # Verbose
52     if verbose:
53         cmd.append("-v")
54
55     # Markers
56     if markers:
57         cmd.extend(["-m", markers])
58
59     # Run
60     result = subprocess.run(cmd, capture_output=False)
61     return result.returncode
62
63 def show_coverage():
64     """Display coverage report in browser."""
65     html_path = Path("tests/coverage") / "index.html"
66     if html_path.exists():

```

```

67 |         import webbrowser
68 |         webbrowser.open(f"file://{html_path.absolute()}")
69 |     else:
70 |         print("HTML report not found. Run tests with --coverag
e flag")
71 |
72 | def main():
73 |     """Main function."""
74 |     parser = argparse.ArgumentParser(description="Run ai-bread
board project tests")
75 |     parser.add_argument("--coverage", "-c", action="store_true
", help="Run with coverage analysis")
76 |     parser.add_argument("--verbose", "-v", action="store_true"
, help="Verbose test output")
77 |     parser.add_argument("--markers", "-m", type=str, help="pyt
est markers (unit, integration, slow)")
78 |     parser.add_argument("--open-coverage", "-o", action="store
_true", help="Open HTML coverage report")
79 |
80 |     args = parser.parse_args()
81 |
82 |     if args.open_coverage:
83 |         show_coverage()
84 |         return
85 |
86 |     exit_code = run_tests(
87 |         coverage=args.coverage,
88 |         verbose=args.verbose,
89 |         markers=args.markers
90 |     )
91 |
92 |     if exit_code == 0:
93 |         print("\n All tests passed successfully!")
94 |     else:
95 |         print(f"\n Tests failed (exit code: {exit_code})")
96 |
97 |     sys.exit(exit_code)
98 |

```

```
99 | if __name__ == "__main__":  
100 |     main()  
101 |
```

save_file.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Save content to file at specified path
4 # =====
5 # Description:
6 #   Saves file content to disk at specified path with director
7 #   y creation
8 #   and error handling. Useful for programmatic file writing o
9 #   perations.
10 #
11 # File: save_file.py
12 # Project: ai-breadboard
13 # Package: scripts.dev
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # =====
17
18 """File content saving utility.
19 Provides function to save text content to files with automatic
20 directory
21 creation and proper error handling."""
22
23 import os
24 import argparse
25
26 def save_file(path: str, content: str) -> bool:
27     """Save content to file at specified path.
28
29     Creates necessary directories if they don't exist and writ
30     es
31     content to file with UTF-8 encoding.
```

```
30 | Args:
31 |     path: File path where content should be saved.
32 |     content: Text content to write to file.
33 |
34 | Returns:
35 |     True if save successful, False if error occurred.
36 |
37 | Examples:
38 |     >>> save_file("test.txt", "hello")
39 |     True
40 |     """
41 | try:
42 |     # Create directory if it doesn't exist
43 |     directory = os.path.dirname(path)
44 |     if directory:
45 |         os.makedirs(directory, exist_ok=True)
46 |
47 |     # Write content
48 |     with open(path, 'w', encoding='utf-8') as f:
49 |         f.write(content)
50 |     return True
51 | except Exception as e:
52 |     # In production, logging should use src.logger.logger
53 |     print(f"Error saving file: {e}")
54 |     return False
55 |
56 | if __name__ == "__main__":
57 |     parser = argparse.ArgumentParser(description="Save content
58 | to a file.")
59 |     parser.add_argument("--path", required=True, help="Path to
60 | the file.")
61 |     parser.add_argument("--content", required=True, help="Cont
62 | ent to save.")
63 |     args = parser.parse_args()
64 |
65 |     success = save_file(args.path, args.content)
66 |     if success:
67 |         print(f"File saved successfully to {args.path}")
```

```
65 | else:
66 |     print("Failed to save file.")
67 |
```


scan_headers.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Python files header validation scanner
4 # =====
5 # Description:
6 #   Scans all Python files in project to validate header format compliance
7 #   with CODE_RULES.md § 6.1 specification for file headers and docstrings.
8 #
9 # File: scan_headers.py
10 # Project: ai-breadboard
11 # Package: scripts.dev
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """Header validation scanner for Python files.
17
18 Scans all Python files to validate header format and documentation compliance
19 with project standards."""
20
21 import os
22 import re
23 from pathlib import Path
24
25 def count_words(text):
26     """Count words in text.
27
28     Args:
29         text: Text string to count.
30
```

```

31 | Returns:
32 |     Number of words in text.
33 |     """
34 |     words = re.findall(r'\b\w+\b', text)
35 |     return len(words)
36 |
37 | def extract_description(header_text):
38 |     """Extract description from header text.
39 |
40 |     Args:
41 |         header_text: Header section text.
42 |
43 |     Returns:
44 |         Description string or empty if not found.
45 |         """
46 |         desc_match = re.search(r'# Description:(.*?)(?=\s*#(?:File|Project|Author|Copyright|\$))', header_text, re.DOTALL)
47 |         if desc_match:
48 |             desc_lines = desc_match.group(1).strip()
49 |             lines = [line.strip().lstrip('#').strip() for line in
50 | desc_lines.split('\n')]
51 |             return ' '.join(lines)
52 |         return ''
53 |
54 | def check_header(filepath):
55 |     """Check if file has valid header.
56 |
57 |     Args:
58 |         filepath: Path to file to check.
59 |
60 |     Returns:
61 |         Dictionary with header validation results.
62 |         """
63 |         with open(filepath, 'r', encoding='utf-8') as f:
64 |             content = f.read()
65 |             lines = content.split('\n')
66 |             has_coding = False
67 |             header_start = 0

```

```

67     for i, line in enumerate(lines[:3]):
68         if '# -*- coding: utf-8 -*-' in line:
69             has_coding = True
70             header_start = i
71             break
72
73     if not has_coding:
74         return {'has_header': 'no', 'word_count': 0, 'description': ''}
75
76     header_lines = []
77     for i in range(header_start, min(len(lines), 50)):
78         line = lines[i].strip()
79         if not line.startswith('#') and not line.startswith('!'
80         ') and 'import' not in line.lower():
81             break
82         header_lines.append(line)
83
84     header_text = '\n'.join(header_lines)
85     has_process_name = bool(re.search(r'#\s*Process Name:', header_text))
86     has_description = bool(re.search(r'#\s*Description:', header_text))
87     has_file = bool(re.search(r'#\s*File:', header_text))
88     has_project = bool(re.search(r'#\s*Project:', header_text))
89     has_author = bool(re.search(r'#\s*Author:', header_text))
90     has_copyright = bool(re.search(r'#\s*Copyright:', header_text))
91
92     all_fields_present = all([has_process_name, has_description,
93     has_file, has_project, has_author, has_copyright])
94     if not all_fields_present:
95         if len(header_lines) <= 3 and all('coding' in l.lower()
96         ) or l.strip() == '' for l in header_lines):
97             return {'has_header': 'no', 'word_count': 0, 'description': ''}
98         return {'has_header': 'no', 'word_count': 0, 'description': ''}

```

```

    ion': ''}
96
97     description = extract_description(header_text)
98     word_count = count_words(description)
99     return {'has_header': 'yes', 'word_count': word_count, 'de
description': description[:200]}
100
101 if __name__ == '__main__':
102     # Header scanning script
103     import sys
104     root_dir = Path(__root__ if '__root__' in globals() else '
.')
105     python_files = list(root_dir.glob('**/*.py'))
106
107     print(f"Found {len(python_files)} Python files. Starting s
can...")
108     for pf in python_files:
109         if 'venv' in pf.parts or '.git' in pf.parts:
110             continue
111         try:
112             status = check_header(pf)
113             if status['has_header'] == 'no':
114                 print(f"  {pf.relative_to(root_dir)} – header
missing or invalid")
115         except Exception as e:
116             print(f"  Error processing {pf}: {e}")
117

```

search_code.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Search codebase via CLI utility
4 # =====
5 # Description:
6 #   Console utility for searching technical RAG index of code
7 #   and documentation.
8 #
9 # File: search_code.py
10 # Project: ai-breadboard
11 # Package: scripts.dev
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """Command-line utility for searching codebase via RAG index.
17 Provides console interface for searching technical documentati
18 on and code
19 using the RAG (Retrieval-Augmented Generation) search system."
20 """
21
22 import os
23 import sys
24 import json
25 from src.ai.dev_rag import rag_search_tool
26
27 def main():
28     """Main function."""
29     if len(sys.argv) < 2:
30         print("Usage: python scripts/search_code.py 'your quer
31 y'")
32     return
```

```
30 |
31 |     query = sys.argv[1]
32 |     api_key = os.getenv('GEMINI_API_KEY')
33 |
34 |     if not api_key:
35 |         print("\n Error: GEMINI_API_KEY not set.")
36 |         return
37 |
38 |     print(f"\n Searching code: '{query}'...")
39 |     result_json = rag_search_tool(query, api_key=api_key)
40 |
41 |     try:
42 |         results = json.loads(result_json)
43 |         if "error" in results:
44 |             print(f"\n {results['error']}")
45 |         else:
46 |             for i, res in enumerate(results, 1):
47 |                 path = res.get('meta', {}).get('path', 'Unknown')
48 |                 print(f"{i}. File: {path} (Score: {res.get('score', 0):.2f})")
49 |     except Exception as e:
50 |         print(f"\n Error parsing results: {e}")
51 |
52 | if __name__ == "__main__":
53 |     main()
54 |
```

update_docs.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Check documentation validity of modified files
4 # =====
5 # Description:
6 #   Script checks presence and validity of documentation (README.md, docstrings)
7 #   for modified Python files in the repository.
8 #
9 # File: update_docs.py
10 # Project: ai-breadboard
11 # Package: scripts.dev
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """Documentation validation for modified Python files.
17
18 Checks for presence of docstrings in modified Python files and
19 validates
20 that proper documentation standards are met."""
21
22 import os
23 import sys
24 import subprocess
25 from pathlib import Path
26 from typing import List
27
28 def get_modified_python_files() -> List[Path]:
29     """Get list of modified Python files in git repository.
30
31     Returns:
32         List of Path objects for modified .py files.
```

```

32 |     """
33 |     modified_files = []
34 |     try:
35 |         res = subprocess.check_output(
36 |             ["git", "status", "--porcelain"],
37 |             text=True,
38 |             stderr=subprocess.DEVNULL
39 |         )
40 |         for line in res.splitlines():
41 |             parts = line.strip().split(maxsplit=1)
42 |             if len(parts) > 1:
43 |                 filepath = Path(parts[1])
44 |                 if filepath.suffix == ".py" and filepath.exists():
45 |                     modified_files.append(filepath)
46 |     except Exception:
47 |         # Git unavailable, return empty list
48 |         pass
49 |     return modified_files
50 |
51 | def validate_docblocks(files: List[Path]) -> bool:
52 |     """Check for presence of docstrings in modified files.
53 |
54 |     Args:
55 |         files: List of file paths to check.
56 |
57 |     Returns:
58 |         True if all files contain docstrings, False otherwise.
59 |     """
60 |     all_valid = True
61 |     for f in files:
62 |         content = f.read_text(encoding="utf-8")
63 |         # Simple check for triple quotes (docstrings)
64 |         if '"""' not in content and "'" not in content:
65 |             print(f"File {f.name} modified but contains no
66 | docstring!")
67 |             all_valid = False
68 |     return all_valid

```



```
68 |
69 | def main() -> int:
70 |     """Main function."""
71 |     print("\t Starting documentation and comments validity che
    | ck...")
72 |
73 |     modified = get_modified_python_files()
74 |     if not modified:
75 |         print("\t No modified Python files found in git. Additi
    | onal validation not required.")
76 |         return 0
77 |
78 |     print(f"Modified files found: {len(modified)}")
79 |     valid = validate_docblocks(modified)
80 |
81 |     if valid:
82 |         print("\t All modified files contain docstrings.")
83 |         return 0
84 |     else:
85 |         print("\t Recommended to add or update comments/documen
    | tation.")
86 |         return 1
87 |
88 | if __name__ == "__main__":
89 |     sys.exit(main())
90 |
```

update_model.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Update user model preference in database
4 # =====
5 # Description:
6 #   Script to update the default AI model preference for all u
7 #   sers in the database
8 #   to the latest Gemini version available.
9 #
10 # File: update_model.py
11 # Project: ai-breadboard
12 # Package: scripts.dev
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16 """Update user model preference in database.
17
18 Updates default AI model selection for all users to latest Gem
19 ini model."""
20
21 import sqlite3
22 import sys
23
24 try:
25     conn = sqlite3.connect('C:/ai-breadboard/src/user_manager/
26     users.db')
27     cursor = conn.cursor()
28     cursor.execute("UPDATE user_settings SET model = 'gemini-2
29     .0-flash'")
30     conn.commit()
31     print(f"Updated {cursor.rowcount} rows in users.db")
32     conn.close()
```

```
30 | except Exception as e:
31 |     print(e)
32 |     sys.exit(1)
33 |
```

update_scripts_documentation.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Automatic scripts documentation generator
4 # =====
5 # Description:
6 #   Automatically updates project scripts documentation. Runs
7 #   regularly
8 #   to maintain SCRIPTS_DOCUMENTATION.md currency.
9 #
10 # File: update_scripts_documentation.py
11 # Project: ai-breadboard
12 # Package: scripts.dev
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16 """Automatic scripts documentation generation and update scrip
17 t.
18 Scans all project scripts, categorizes them, extracts metadata
19 and
20 generates comprehensive markdown documentation."""
21
22 import os
23 import sys
24 import re
25 from pathlib import Path
26 from datetime import datetime
27 from collections import defaultdict
28
29 PROJECT_ROOT = Path(__file__).parent
30
31 # Script categories with descriptions
```

```
31 | SCRIPT_CATEGORIES = {
32 |     'launch_scripts': {
33 |         'name': 'Main launch scripts',
34 |         'description': 'Scripts for launching and configuring
main system components',
35 |         'patterns': ['*.ps1', 'main.py', 'bot_runner.py'],
36 |         'priority': '❑ Critical'
37 |     },
38 |     'cli_utilities': {
39 |         'name': 'CLI management and utilities',
40 |         'description': 'Command-line interface for system mana
gement',
41 |         'patterns': ['manage_*.py'],
42 |         'priority': '❑ Important'
43 |     },
44 |     'media_processing': {
45 |         'name': 'Media library processing',
46 |         'description': 'Media library work, classification, da
tabase updates',
47 |         'patterns': ['audit_*.py', 'generate_*.py', 'complete_
*.py', 'fill_*.py', 'update_media_*.py'],
48 |         'priority': '❑ Important'
49 |     },
50 |     'analysis': {
51 |         'name': 'Analysis and reporting',
52 |         'description': 'Data analysis and report generation',
53 |         'patterns': ['analyze_*.py'],
54 |         'priority': '❑ Useful'
55 |     },
56 |     'diagnostics': {
57 |         'name': 'Checks and diagnostics',
58 |         'description': 'System state verification',
59 |         'patterns': ['check_*.py', 'debug_*.py'],
60 |         'priority': '❑ Useful'
61 |     },
62 |     'database': {
63 |         'name': 'Database operations',
64 |         'description': 'Database maintenance',
```

```

65 |         'patterns': ['update_db*.py', 'get_schema.py', 'remove
    | _columns.py'],
66 |         'priority': '□ Technical'
67 |     },
68 |     'migration': {
69 |         'name': 'Migrations and backups',
70 |         'description': 'Data migration and backup',
71 |         'patterns': ['*migration*.py'],
72 |         'priority': '□ Operational'
73 |     },
74 |     'counting': {
75 |         'name': 'Statistics and metrics',
76 |         'description': 'Various metrics counting',
77 |         'patterns': ['count_*.py'],
78 |         'priority': '□ Utilities'
79 |     },
80 |     'other': {
81 |         'name': 'Other scripts',
82 |         'description': 'Scripts not matching other categories'
    | ,
83 |         'patterns': ['*.py'],
84 |         'priority': '□ Miscellaneous'
85 |     }
86 | }
87 |
88 | def get_script_info(script_path):
89 |     """Extract script information from file content.
90 |
91 |     Args:
92 |         script_path: Path to script file.
93 |
94 |     Returns:
95 |         Dictionary with script metadata: name, size, modified
    | time, lines, purpose, dependencies.
96 |     """
97 |     info = {
98 |         'name': script_path.name,
99 |         'size': script_path.stat().st_size,

```

```

100         'modified': datetime.fromtimestamp(script_path.stat().
| st_mtime),
101         'lines': 0,
102         'purpose': 'Not defined',
103         'dependencies': [],
104         'usage_examples': []
105     }
106
107     try:
108         content = script_path.read_text(encoding='utf-8', erro
| rs='ignore')
109         info['lines'] = len(content.split('\n'))
110
111         # Extract purpose from docstring or comments
112         lines = content.split('\n')
113         purpose_found = False
114
115         for i, line in enumerate(lines[:20]): # Check first 2
| 0 lines
116             line_lower = line.lower()
117
118             # Look for docstring
119             if '"""' in line and not purpose_found and i < 10:
120                 # Multi-line docstring
121                 docstring_lines = []
122                 for j in range(i + 1, min(i + 10, len(lines)))
| :
123                     if '"""' in lines[j]:
124                         break
125                     docstring_lines.append(lines[j].strip())
126                 if docstring_lines:
127                     info['purpose'] = ' '.join(docstring_lines
| [:3]))[:200]
128                     purpose_found = True
129
130             # Look for single-line comments with purpose
131             elif not purpose_found and line.strip().startswith
| ('#') and any(word in line_lower for word in ['purpose', 'desc

```

```

ription']]):
132         info['purpose'] = line.strip('# \t\n\r')[:200]
133         purpose_found = True
134
135     # If purpose not found, use first meaningful line
136     if not purpose_found:
137         for line in lines:
138             if line.strip() and not line.strip().startswith(
139                 ('#', "'", '"', 'import', 'from')):
140                 info['purpose'] = line.strip()[:150]
141                 break
142
143     # Search for dependencies
144     for line in lines:
145         if 'import' in line:
146             # Simple imports
147             if 'import ' in line and ' as ' not in line:
148                 parts = line.split('import')[1].strip().split(',')
149                 for part in parts:
150                     module = part.strip().split()[0]
151                     if module and '.' not in module: # To
152                         info['dependencies'].append(module)
153
154     except Exception as e:
155         info['purpose'] = f"Error reading: {str(e)}"
156
157     return info
158
159 def categorize_script(script_name):
160     """Determine script category based on filename.
161
162     Args:
163         script_name: Name of the script file.
164
165     Returns:

```



```

165         Category ID string.
166         """
167         for category_id, category_info in SCRIPT_CATEGORIES.items(
168         ):
169             for pattern in category_info['patterns']:
170                 # Convert pattern to regex
171                 regex_pattern = pattern.replace('*', '.*').replace
172                 ('?', '.')
173                 if re.match(f'^{regex_pattern}$', script_name):
174                     return category_id
175             return 'other'
176
177 def generate_documentation():
178     """Generate and save scripts documentation.
179
180     Scans all project scripts, categorizes them, and creates c
181     omprehensive
182     markdown documentation file.
183     """
184     print("Generating scripts documentation...")
185
186     # Collect all scripts
187     all_scripts = []
188
189     # Python scripts
190     for py_file in PROJECT_ROOT.glob("*.py"):
191         if py_file.name not in ['analyze_dependencies.py', 'up
192         date_scripts_documentation.py']:
193             all_scripts.append(py_file)
194
195     # PowerShell scripts
196     for ps1_file in PROJECT_ROOT.glob("*.ps1"):
197         all_scripts.append(ps1_file)
198
199     # Group by categories
200     categorized_scripts = defaultdict(list)
201
202     for script_path in all_scripts:

```

```

199         category = categorize_script(script_path.name)
200         script_info = get_script_info(script_path)
201         categorized_scripts[category].append(script_info)
202
203     # Sort within categories
204     for category in categorized_scripts:
205         categorized_scripts[category].sort(key=lambda x: x['name'].lower())
206
207     # Generate Markdown documentation
208     md_content = []
209
210     # Header
211     md_content.append(f"# Scripts Documentation for ai-breadboard project\n")
212     md_content.append(f"**Version:** 1.0 \n")
213     md_content.append(f"**Updated:** {datetime.now().strftime('%d %B %Y')}} \n")
214     md_content.append(f"**Status:** Current (auto-updated)\n")
215
216     # Statistics
217     md_content.append("## General Statistics\n")
218     md_content.append("| Category | Count | Status |")
219     md_content.append("|-----|-----|-----|")
220
221     total_scripts = 0
222     for category_id, scripts in categorized_scripts.items():
223         if scripts: # Only non-empty categories
224             category_info = SCRIPT_CATEGORIES[category_id]
225             count = len(scripts)
226             total_scripts += count
227             md_content.append(f"| {category_info['name']} | {count} | {category_info['priority']} |")
228
229     md_content.append(f"| **Total active scripts** | **{total_scripts}** | |\n")
230
231     # Detailed descriptions by categories

```

```

232     for category_id, scripts in categorized_scripts.items():
233         if not scripts:
234             continue
235
236         category_info = SCRIPT_CATEGORIES[category_id]
237         md_content.append(f"\n## {category_info['name']}\n")
238         md_content.append(f"**Description:** {category_info['d
escription']}\n")
239         md_content.append(f"**Status:** {category_info['priori
ty']}\n")
240
241         for script in scripts:
242             md_content.append(f"\n### **{script['name']}**")
243
244             if script['name'].endswith('.ps1'):
245                 md_content.append(f"**Type:** PowerShell scrip
t\n")
246             else:
247                 md_content.append(f"**Type:** Python script \
n")
248
249                 md_content.append(f"**Size:** {script['size'],} b
ytes\n")
250                 md_content.append(f"**Lines of code:** {script['li
nes']}\n")
251                 md_content.append(f"**Modified:** {script['modifie
d'].strftime('%Y-%m-%d %H:%M')}\n")
252                 md_content.append(f"**Purpose:** {script['purpose'
]}\n")
253
254                 # Usage examples for main scripts
255                 if script['name'] in ['run.ps1', 'manage_tools.py'
]:
256                     md_content.append(f"**Usage Examples:**\n")
257
258                     if script['name'] == 'run.ps1':
259                         md_content.append(f"`bash\n.\run.ps1\n`
\n")

```

```

260         elif script['name'] == 'manage_tools.py':
261             md_content.append(f"```bash\npy manage_too
ls.py rag rebuild      # Rebuild RAG index\npy manage_tools.p
y knowledge extract # Extract knowledge\n```\n")
262
263     # Maintenance principles
264     md_content.append("\n### Maintenance Principles\n")
265     md_content.append("### 1. Regular documentation updates\n"
)
266     md_content.append("- This documentation updates automatica
lly when running `update_scripts_documentation.py`\n")
267     md_content.append("- Recommended to run after adding or re
moving scripts\n")
268
269     md_content.append("\n### 2. Automatic updates\n")
270     md_content.append("```bash\n# Update documentation\npython
update_scripts_documentation.py\n```\n")
271
272     md_content.append("\n### 3. Dependency analysis\n")
273     md_content.append("```bash\n# Analyze dependencies between
scripts\npython analyze_dependencies.py\n```\n")
274
275     # Project info
276     md_content.append("\n### Contacts and Support\n")
277     md_content.append(f"**Project:** ai-breadboard  \n")
278     md_content.append(f"**Last updated:** {datetime.now().strf
time('%d %B %Y %H:%M')} \n")
279     md_content.append(f"**Update script:** `update_scripts_doc
umentation.py` \n")
280     md_content.append("\n*Documentation automatically updates
when repository changes.*\n")
281
282     # Save documentation
283     doc_file = PROJECT_ROOT / "SCRIPTS_DOCUMENTATION.md"
284     doc_file.write_text('\n'.join(md_content), encoding='utf-8
')
285
286     print(f"□ Documentation saved to {doc_file}")

```

```

287 |     print(f"    Processed {len(all_scripts)} scripts")
288 |
289 |     # Also create brief summary
290 |     create_summary(categorized_scripts)
291 |
292 | def create_summary(categorized_scripts):
293 |     """Create brief summary of scripts.
294 |
295 |     Args:
296 |         categorized_scripts: Dictionary of scripts organized b
297 | y category.
298 |     """
299 |     summary = ["# Scripts Summary\n"]
300 |     summary.append(f"*Updated: {datetime.now().strftime('%Y-%m
301 | -%d %H:%M')}* \n")
302 |
303 |     for category_id, scripts in categorized_scripts.items():
304 |         if not scripts:
305 |             continue
306 |
307 |         category_info = SCRIPT_CATEGORIES[category_id]
308 |         summary.append(f"\n## {category_info['name']} ({len(sc
309 | ripts)})\n")
310 |
311 |         for script in scripts:
312 |             if script['name'].endswith('.ps1'):
313 |                 summary.append(f"- `{script['name']}` - PowerS
314 | hell ({script['size']:,} bytes)")
315 |             else:
316 |                 summary.append(f"- `{script['name']}` - Python
317 | ({script['lines']} lines)")
318 |
319 |     summary_file = PROJECT_ROOT / "SCRIPTS_SUMMARY.md"
320 |     summary_file.write_text('\n'.join(summary), encoding='utf-
321 | 8')
322 |
323 |     print(f"    Brief summary saved to {summary_file}")

```

```

319 def main():
320     """Main function."""
321     print("=" * 60)
322     print("SCRIPTS DOCUMENTATION UPDATE")
323     print("=" * 60)
324
325     try:
326         generate_documentation()
327
328         # Also run dependency analysis for completeness
329         print("\n" + "=" * 60)
330         print("ADDITIONAL DEPENDENCY ANALYSIS")
331         print("=" * 60)
332
333         if (PROJECT_ROOT / "analyze_dependencies.py").exists()
334         :
335             import subprocess
336             result = subprocess.run([sys.executable, "analyze_
dependencies.py"],
337                                     capture_output=True, text=Tr
ue)
338             print(result.stdout)
339             if result.stderr:
340                 print(f"Warnings: {result.stderr}")
341             else:
342                 print("Script analyze_dependencies.py not found, s
kipping dependency analysis.")
343
344             print("\n" + "=" * 60)
345             print("□ UPDATE COMPLETED SUCCESSFULLY")
346             print("=" * 60)
347
348     except Exception as e:
349         print(f"\n□ Error during documentation update: {e}")
350         import traceback
351         traceback.print_exc()
352         sys.exit(1)

```

```
353 | if __name__ == '__main__':  
354 |     main()  
355 |
```

analyze_logs.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Manual log analysis script
4 # =====
5 # Description:
6 #   One-time script for analyzing application logs. Run manual
7 #   ly when needed.
8 #
9 # File: analyze_logs.py
10 # Project: ai-breadboard
11 # Package: scripts.maintenance
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """Manual log analysis script.
17 One-time script for analyzing application logs. Run manually:
18 python scripts/analyze_logs.py"""
19
20 import asyncio
21 import os
22 import sys
23 from pathlib import Path
24
25 # Add project root to path
26 project_root = Path(__file__).parent.parent
27 sys.path.insert(0, str(project_root))
28
29 from src.logger.log_analyzer import analyze_log_file, LOG_DIR,
30     get_max_size_bytes
31 from src.ai import GoogleGenerativeAI
32 from src.logger import logger
```



```

31
32 async def main():
33     logger.info("=" * 60)
34     logger.info("STARTING LOG ANALYSIS (MANUAL MODE)")
35     logger.info("=" * 60)
36
37     if not LOG_DIR.exists():
38         logger.error(f"Log directory not found: {LOG_DIR}")
39         return
40
41     # Setup AI
42     api_key_names = [n.strip() for n in os.getenv('GEMINI_API_
KEY_NAMES', '').split(',') if n.strip()]
43     system_instruction = "You are a professional system log an
alyst. Your task is to examine logs, identify errors, issues,
trends and provide recommendations for resolution."
44
45     ai_model = GoogleGenerativeAI(
46         api_key_names=api_key_names,
47         system_instruction=system_instruction
48     )
49
50     max_bytes = get_max_size_bytes()
51     files_to_analyze = []
52
53     # Search for .log files
54     for p in LOG_DIR.glob("*.log"):
55         if p.is_file() and p.stat().st_size >= max_bytes:
56             files_to_analyze.append(p)
57             logger.info(f"Found file: {p.name} ({p.stat().st_s
ize / 1024 / 1024:.1f} MB)")
58
59     # Search for log.json
60     json_log = LOG_DIR / "log.json"
61     if json_log.exists() and json_log.is_file() and json_log.s
tat().st_size >= max_bytes:
62         files_to_analyze.append(json_log)
63         logger.info(f"Found file: {json_log.name} ({json_log.s

```

```

    tat().st_size / 1024 / 1024:.1f} MB)")
64
65     if not files_to_analyze:
66         logger.info("No files to analyze (all smaller than {}
    MB threshold)".format(
67             float(os.getenv('LOG_MAX_SIZE_MB', '10.0'))
68         ))
69     return
70
71     logger.info(f"Files to analyze: {len(files_to_analyze)}")
72
73     # Analyze each file
74     for file_path in files_to_analyze:
75         await analyze_log_file(file_path, ai_model)
76
77     logger.info("=" * 60)
78     logger.info("LOG ANALYSIS COMPLETE")
79     logger.info("=" * 60)
80
81 if __name__ == "__main__":
82     asyncio.run(main())
83

```

generate_ssl_certs.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Generate Root CA and server leaf certificate f
  or l
4 # =====
5 # Description:
6 #   Generates a dedicated local Root CA and an end-entity SSL
  server certificate
7 #
8 # File: generate_ssl_certs.py
9 # Project: ai-breadboard
10 # Package: scripts.maintenance
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from __future__ import annotations
16
17 import ipaddress
18 import os
19 import socket
20 import subprocess
21 import sys
22 from datetime import datetime, timedelta, timezone
23 from pathlib import Path
24
25 from cryptography import x509
26 from cryptography.hazmat.primitives import hashes, serializati
  on
27 from cryptography.hazmat.primitives.asymmetric import rsa
28 from cryptography.x509.oid import ExtendedKeyUsageOID, NameOID
29
30 def generate_certificates(certs_dir: Path | None = None, extra
```

```

    _ips: list[str] | None = None) -> tuple[Path, Path, Path]:
31 |     """Generate Root CA and server leaf certificate for localh
    ost.
32 |
33 |     Args:
34 |         certs_dir: Destination directory for certificates (~/.
    certs by default).
35 |         extra_ips: List of additional IP addresses to add to S
    AN.
36 |
37 |     Returns:
38 |         tuple[Path, Path, Path]: (cert_path, key_path, root_ca
    _path)
39 |     """
40 |     if not certs_dir:
41 |         certs_dir = Path(os.path.expanduser('~')) / '.certs'
42 |         certs_dir.mkdir(parents=True, exist_ok=True)
43 |
44 |         ca_cert_path = certs_dir / 'rootCA.pem'
45 |         ca_key_path = certs_dir / 'rootCA-key.pem'
46 |         cert_path = certs_dir / 'localhost+2.pem'
47 |         key_path = certs_dir / 'localhost+2-key.pem'
48 |
49 |         # 1. Generate or load Root CA
50 |         ca_cert = None
51 |         ca_key = None
52 |         if ca_cert_path.exists() and ca_key_path.exists():
53 |             try:
54 |                 ca_cert = x509.load_pem_x509_certificate(ca_cert_p
    ath.read_bytes())
55 |                 ca_key = serialization.load_pem_private_key(ca_key
    _path.read_bytes(), password=None)
56 |                 ca_name = ca_cert.subject
57 |             except Exception:
58 |                 ca_cert = None
59 |                 ca_key = None
60 |
61 |         if not ca_cert or not ca_key:

```

```

62 |         ca_key = rsa.generate_private_key(public_exponent=6553
    | 7, key_size=2048)
63 |         ca_name = x509.Name([
64 |             x509.NameAttribute(NameOID.COMMON_NAME, 'aibreadbo
    | ard Local Root CA'),
65 |             x509.NameAttribute(NameOID.ORGANIZATION_NAME, 'aib
    | readboard Development'),
66 |         ])
67 |         ca_cert = (
68 |             x509.CertificateBuilder()
69 |                 .subject_name(ca_name)
70 |                 .issuer_name(ca_name)
71 |                 .public_key(ca_key.public_key())
72 |                 .serial_number(x509.random_serial_number())
73 |                 .not_valid_before(datetime.now(timezone.utc) - tim
    | edelta(days=1))
74 |                 .not_valid_after(datetime.now(timezone.utc) + time
    | delta(days=3650))
75 |                 .add_extension(x509.BasicConstraints(ca=True, path
    | _length=None), critical=True)
76 |                 .add_extension(x509.KeyUsage(
77 |                     digital_signature=True, content_commitment=False,
    | se, key_encipherment=False,
78 |                     data_encipherment=False, key_agreement=False,
    | key_cert_sign=True,
79 |                     crl_sign=True, encipher_only=False, decipher_o
    | nly=False
80 |                 ), critical=True)
81 |                 .sign(ca_key, hashes.SHA256())
82 |         )
83 |         ca_cert_path.write_bytes(ca_cert.public_bytes(serializ
    | ation.Encoding.PEM))
84 |         ca_key_path.write_bytes(ca_key.private_bytes(
85 |             encoding=serialization.Encoding.PEM,
86 |             format=serialization.PrivateFormat.TraditionalOpen
    | SSL,
87 |             encryption_algorithm=serialization.NoEncryption()
88 |         ))

```

```

89
90     # 2. Build SAN list for Server Certificate
91     san_list: list[x509.GeneralName] = [
92         x509.DNSName('localhost'),
93         x509.DNSName(socket.gethostname()),
94         x509.IPAddress(ipaddress.IPv4Address('127.0.0.1')),
95         x509.IPAddress(ipaddress.IPv6Address('::1')),
96     ]
97
98     all_ips = extra_ips or []
99     try:
100         host_ips = socket.gethostbyname_ex(socket.gethostname(
101         ))[2]
102         for hip in host_ips:
103             if hip not in all_ips:
104                 all_ips.append(hip)
105     except Exception:
106         pass
107
108     for ip_str in all_ips:
109         try:
110             ip_obj = ipaddress.ip_address(ip_str.strip())
111             if ip_obj not in [x.value for x in san_list if isinstance(x, x509.IPAddress)]:
112                 san_list.append(x509.IPAddress(ip_obj))
113         except Exception:
114             pass
115
116     # 3. Generate End-Entity Server Certificate signed by Root
117     # CA
118     server_key = rsa.generate_private_key(public_exponent=6553
119     7, key_size=2048)
120     server_name = x509.Name([
121         x509.NameAttribute(NameOID.COMMON_NAME, 'localhost'),
122         x509.NameAttribute(NameOID.ORGANIZATION_NAME, 'aibread
123         board Local Server'),
124     ])

```

```

122     server_cert = (
123         x509.CertificateBuilder()
124         .subject_name(server_name)
125         .issuer_name(ca_name)
126         .public_key(server_key.public_key())
127         .serial_number(x509.random_serial_number())
128         .not_valid_before(datetime.now(timezone.utc) - timedelta(
129             days=1))
130         .not_valid_after(datetime.now(timezone.utc) + timedelta(
131             days=1825))
132         .add_extension(x509.BasicConstraints(ca=False, path_length=None),
133             critical=True)
134         .add_extension(x509.KeyUsage(
135             digital_signature=True, content_commitment=False,
136             key_encipherment=True,
137             data_encipherment=False, key_agreement=False, key_
138             cert_sign=False,
139             crl_sign=False, encipher_only=False, decipher_only
140             =False
141             ), critical=True)
142         .add_extension(
143             x509.ExtendedKeyUsage([ExtendedKeyUsageOID.SERVER_
144             AUTH, ExtendedKeyUsageOID.CLIENT_AUTH]),
145             critical=False,
146         )
147         .add_extension(x509.SubjectAlternativeName(san_list),
148             critical=False)
149         .sign(ca_key, hashes.SHA256())
150     )
151
152     cert_path.write_bytes(server_cert.public_bytes(serialization.
153         Encoding.PEM))
154     key_path.write_bytes(server_key.private_bytes(
155         encoding=serialization.Encoding.PEM,
156         format=serialization.PrivateFormat.TraditionalOpenSSL,
157         encryption_algorithm=serialization.NoEncryption())
158     ))

```

```
151 |     # 4. Install Root CA into Windows Trusted Root Store
152 |     if sys.platform == 'win32':
153 |         try:
154 |             subprocess.run(['certutil', '-user', '-addstore',
155 |                             '-f', 'Root', str(ca_cert_path)], capture_output=True)
156 |         except Exception:
157 |             pass
158 |
159 |     return cert_path, key_path, ca_cert_path
160 |
161 | if __name__ == '__main__':
162 |     extra = sys.argv[1:]
163 |     c_path, k_path, ca_path = generate_certificates(extra_ips=
164 | extra)
165 |     print(f'SUCCESS: cert={c_path} key={k_path} ca={ca_path}')
```


rebuild_dev_rag.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Developer context RAG reindexing and rebuilding
4 # =====
5 # Description:
6 #   Script rebuilds RAG index for development technical documentation and codebase.
7 #
8 # File: rebuild_dev_rag.py
9 # Project: ai-breadboard
10 # Package: scripts.maintenance
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import os
16 from src.ai.dev_rag import build_dev_rag
17 from src.logger import logger
18
19 def main():
20     api_key = os.getenv('GEMINI_API_KEY')
21     if not api_key:
22         print("\t Error: GEMINI_API_KEY environment variable not set.")
23         return
24
25     print("\t Starting codebase and documentation reindexing..")
26     try:
27         build_dev_rag(api_key)
28         print("\t Developer index successfully rebuilt.")
29     except Exception as e:
```

```
30 |         print(f"␣ Error during reindexing: {e}")
31 |         logger.error(f"Error rebuild_dev_rag: {e}", exc_info=T
| rue)
32 |
33 | if __name__ == "__main__":
34 |     main()
35 |
```

agent.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Agent for autonomous work and reasoning with L
  angChain
4 # =====
5 # Description:
6 #   Основной агент платформы AI Breadboard на базе ReAct-архит
  ектуры.
7 #
8 # File: langchain_agent.py
9 # Project: ai-breadboard
10 # Package: src.ai
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from __future__ import annotations
16
17 import os
18 import json
19 import asyncio
20 from pathlib import Path
21 from typing import AsyncIterator
22
23 from src.logger import logger
24 from src.utils.jjson import j_loads_ns
25
26 from .prompts import (
27     GENERAL_AGENT_SYSTEM_PROMPT,
28     MEDIA_SEARCH_SYSTEM_PROMPT,
29     TOOL_SELECTION_GUIDELINES,
30     RESULT_FORMAT_INSTRUCTIONS,
31 )
```

```

32 from .tools import (
33     web_search,
34     rag_search,
35     python_eval,
36     file_read,
37 )
38
39 class MediaSearchAgent:
40     """Агент для автономной работы и рассуждений через LangCha
in + ReAct.
41
42     Использует ReAct-агент с набором инструментов:
43     - Поиск актуальной информации в интернете (web_search)
44     - Поиск по базе знаний (rag_search)
45     - Выполнение Python-кода (python_eval)
46     - Чтение файлов проекта (file_read)
47     """
48
49     def __init__(self, config_path: Path = Path('config.json')
, ai_model=...):
50         """Initialization агента.
51
52         Args:
53             config_path: Путь к config.json с секцией langchai
n.
54             ai_model: Экземпляр AI-модели (передаётся из плаги
на, может не использоваться).
55         """
56         self.config = j_loads_ns(config_path)
57         self.ai_model = ai_model
58
59         langchain_cfg = getattr(self.config, 'langchain', obje
ct())
60         self.llm_type = getattr(langchain_cfg, 'default_llm',
'gemini')
61         self.max_steps = getattr(langchain_cfg, 'max_agent_ste
ps', 15)
62         self.timeout = getattr(langchain_cfg, 'search_timeout_

```

```

seconds', 60)
63
64     # Ленивая initialization LLM (при первом вызове search
    )
65     self._llm = ''
66     self._langchain_cfg = langchain_cfg
67
68     # Нативные инструменты (всегда доступны)
69     self.native_tools = [
70         web_search,
71         rag_search,
72         python_eval,
73         file_read,
74     ]
75
76     logger.info(
77         f'[BreadboardAgent] Инициализирован: llm={self.llm
    _type}, '
78         f'max_steps={self.max_steps}, timeout={self.timeou
    t}'
79     )
80
81     def _get_llm(self):
82         """Ленивое создание LLM-инстанса при первом обращении.
        """
83         if self._llm:
84             return self._llm
85
86         if self.llm_type == 'gemini':
87             from langchain_google_genai import ChatGoogleGener
            ativeAI
88             model_name = getattr(self._langchain_cfg, 'gemini_
            model', 'gemini-2.5-flash')
89             api_key = os.environ.get('GOOGLE_API_KEY', '') or
            os.environ.get('GEMINI_API_KEY', '')
90             if not api_key:
91                 from src.secrets.api_key_state import load_api
            _keys

```

```

92         loaded, _, _ = load_api_keys()
93         aiza_keys = [k for k in loaded if k.startswith
    ('AizaSy')]
94         if aiza_keys:
95             api_key = aiza_keys[0]
96         elif loaded:
97             api_key = loaded[0]
98         if not api_key:
99             logger.error('[MediaSearchAgent] GOOGLE_API_KEY
    Y не задан в .env')
100             raise EnvironmentError('GOOGLE_API_KEY не зада
    н')
101         self._llm = ChatGoogleGenerativeAI(
102             model=model_name,
103             google_api_key=api_key,
104             temperature=0.1,
105         )
106     else:
107         from langchain_ollama import ChatOllama
108         model_name = getattr(self._langchain_cfg, 'ollama_
    model', 'qwen2.5:7b')
109         base_url = getattr(self._langchain_cfg, 'ollama_ba
    se_url', 'http://localhost:11434')
110         self._llm = ChatOllama(
111             model=model_name,
112             base_url=base_url,
113             temperature=0.1,
114         )
115
116         logger.info(f'[MediaSearchAgent] LLM создан: {self._llm
    _type}')
117         return self._llm
118
119     def _build_system_prompt(self) -> str:
120         """Собирает полный системный промпт из компонентов."""
121         return '\n\n'.join([
122             MEDIA_SEARCH_SYSTEM_PROMPT,
123             TOOL_SELECTION_GUIDELINES,

```

```

124         RESULT_FORMAT_INSTRUCTIONS,
125     ])
126
127     async def search(self, query: str) -> dict:
128         """Performs автономный поиск по запросу пользователя.
129
130         Args:
131             query: Текстовый запрос пользователя (напр. 'найди
фильм Интерстеллар 1080p').
132
133         Returns:
134             dict с ключами:
135             - action: 'player' | 'torrent' | 'info' | 'error'
136             - data: dictionary с результатами, зависящий от ac
tion
137         """
138         try:
139             import re
140             from langgraph.prebuilt import create_react_agent
141
142             llm = self._get_llm()
143             system_prompt = self._build_system_prompt()
144
145             # Собираем инструменты: нативные + MCP (если досту
пен)
146             all_tools = list(self.native_tools)
147
148             # Пробуем подключить MCP-инструменты Playwright
149             try:
150                 from src.ai.mcp_client import MCPClientManager
151                 async with MCPClientManager() as mcp:
152                     mcp_tools = await mcp.get_tools()
153                     if mcp_tools:
154                         all_tools.extend(mcp_tools)
155                         logger.info(f'[MediaSearchAgent] Подкл
ючено {len(mcp_tools)} MCP-инструментов')
156             except Exception as mcp_err:
157                 logger.warning(f'[MediaSearchAgent] MCP недост

```

```

упен, продолжаем без него: {mcp_err}')
158
159         logger.info(f'[MediaSearchAgent] Запуск агента с {
len(all_tools)} инструментами для: "{query}"')
160
161         # Создаём ReAct-агент с поддержкой актуальной и ус
таревшей сигнатуры LangGraph
162         try:
163             agent_executor = create_react_agent(
164                 llm,
165                 all_tools,
166                 prompt=system_prompt,
167             )
168         except TypeError:
169             agent_executor = create_react_agent(
170                 llm,
171                 all_tools,
172                 state_modifier=system_prompt,
173             )
174
175         # Запускаем с таймаутом
176         result = await asyncio.wait_for(
177             agent_executor.ainvoke({'messages': [('user',
query)]}),
178             timeout=self.timeout,
179         )
180
181         # Парсим ответ
182         messages = result.get('messages', [])
183         if not messages:
184             return {'action': 'error', 'data': {'message':
'Агент не вернул ответ'}}
185
186         last_message = messages[-1]
187         raw_content = getattr(last_message, 'content', '')
188         if isinstance(raw_content, list):
189             content = "".join([
190                 c.get('text', '') if isinstance(c, dict) e

```



```

lse str(c)
191         for c in raw_content
192     ])
193     else:
194         content = str(raw_content)
195
196         cleaned_content = content.strip()
197         if cleaned_content.startswith('```'):
198             cleaned_content = re.sub(r'^```(?:json)?\s*',
199                                     '', cleaned_content)
200             cleaned_content = re.sub(r'\s*```$', '', cleaned_content).strip()
201
202             # Пробуем распарсить JSON из ответа
203             try:
204                 parsed = json.loads(cleaned_content)
205                 if isinstance(parsed, dict):
206                     action = parsed.get('action', 'info')
207                     return {'action': action, **parsed}
208                     return {'action': 'info', 'text': content}
209             except (json.JSONDecodeError, ValueError):
210                 # LLM вернул текст вместо JSON – оборачиваем к
211                 ак info
212                 return {'action': 'info', 'text': content}
213
214             except asyncio.TimeoutError:
215                 logger.error(f'[MediaSearchAgent] Таймаут ({self.timeout}c) при поиске: "{query}"')
216                 return {'action': 'error', 'data': {'message': f'Превышено время ожидания ({self.timeout}c)'}}
217             except Exception as e:
218                 logger.error(f'[MediaSearchAgent] Error при поиске: {e}')
219                 return {'action': 'error', 'data': {'message': str(e)}}
220
221     async def search_stream(self, query: str) -> AsyncIterator[dict]:

```

```

220 |         """Потоковый поиск с промежуточными статусами.
221 |
222 |         Yields:
223 |             dict с ключом 'status' (промежуточный) или 'result
224 | ' (финальный).
225 |         """
226 |         yield {'status': '▢ Анализирую запрос...'}
227 |
228 |         yield {'status': '▢ Запускаю LangChain агент...'}
229 |
230 |         try:
231 |             yield {'status': '▢ Подключаю инструменты поиска..
232 |             .'}
233 |
234 |             result = await self.search(query)
235 |
236 |             action = result.get('action', 'error')
237 |             if action == 'torrent':
238 |                 yield {'status': '▢ Найдены торренты!'}
239 |             elif action == 'player':
240 |                 yield {'status': '▸ Найден источник для просм
241 |                 отра!'}
242 |             elif action == 'info':
243 |                 yield {'status': '▢ Получена Info о фильме'}
244 |             else:
245 |                 yield {'status': '▢ Поиск завершён с Errorми'}
246 |
247 |             yield {'result': result}
248 |
249 |         except Exception as e:
250 |             logger.error(f'[MediaSearchAgent] Error в потоково
251 | м поиске: {e}')
252 |             yield {'status': f'▢ Error: {e}'}
253 |             yield {'result': {'action': 'error', 'data': {'mes
254 | sage': str(e)}}}

```

mcp_client.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Manager for MCP client lifecycle.
4 # =====
5 # Description:
6 #   Обёртка для управления жизненным циклом MCP-серверов.
7 #
8 # File: mcp_client.py
9 # Project: ai-breadboard
10 # Package: src.ai
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import asyncio
16 from pathlib import Path
17 from typing import Any
18
19 try:
20     from langchain_mcp_adapters.client import MultiServerMCPClient
21 except ImportError:
22     MultiServerMCPClient = None
23
24 from src.logger import logger
25 from src.utils.json import j_loads_ns
26
27 class MCPClientManager:
28     """Manager for MCP client lifecycle."""
29
30     def __init__(self, config_path: str = "config.json") -> Any:
31         self.config_path = Path(config_path)
```

```

32         self._client = ""
33         self._config = ""
34
35     async def __aenter__(self) -> Any:
36         """Context manager entry."""
37         logger.info("Initializing MCPClientManager")
38         self._config = j_loads_ns(self.config_path)
39
40         playwright_config = getattr(self._config.langchain.mcp
    _servers, "playwright", "")
41
42         if not playwright_config:
43             logger.error("Playwright MCP server configuration
    is missing")
44             return self
45
46         command = getattr(playwright_config, "command", "")
47         args = getattr(playwright_config, "args", [])
48
49         if command:
50             logger.info("Starting MultiServerMCPClient")
51             # We initialize client with the loaded config
52             self._client = MultiServerMCPClient(
53                 connections={
54                     "playwright": {
55                         "command": command,
56                         "args": args,
57                         "transport": "stdio",
58                     }
59                 }
60             )
61
62         return self
63
64     async def __aexit__(self, exc_type: Any, exc_val: Any, exc
    _tb: Any) -> Any:
65         """Context manager exit."""
66         logger.info("Shutting down MCPClientManager")

```

```
67 |         self._client = ""
68 |
69 |     async def get_tools(self) -> list:
70 |         """Get LangChain-compatible tools from the MCP server.
71 |         """
72 |         if self._client:
73 |             try:
74 |                 return await self._client.get_tools()
75 |             except Exception as e:
76 |                 logger.warning(f"Error retrieving MCP tools: {
77 | e}")
78 |         return []
```

prompts.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Intelligent ReAct assistant for AI Breadboard
  platform
4 # =====
5 # Description:
6 #   Системные промпты и шаблоны для ReAct-агентов платформы ai
  breadboard.
7 #
8 # File: langchain_prompts.py
9 # Project: ai-breadboard
10 # Package: src.ai
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 GENERAL_AGENT_SYSTEM_PROMPT = """Вы – интеллектуальный ReAct-а
  ссистент платформы AI Breadboard.
16 Ваша задача – помогать пользователю в решении задач, поиске ин
  формации, анализе данных и выполнении вычислений.
17
18 Вы можете выполнять следующие действия с помощью инструментов:
19 - Искать актуальную информацию в интернете (web_search)
20 - Выполнять семантический поиск по базе знаний (rag_search)
21 - Производить точные математические и логические вычисления (p
  ython_eval)
22 - Читать файлы проекта (file_read)
23
24 Формируйте ответы структурированно, четко и на русском языке.
25 """
26
27 MEDIA_SEARCH_SYSTEM_PROMPT = GENERAL_AGENT_SYSTEM_PROMPT
28
```

```
29 TOOL_SELECTION_GUIDELINES = """Рекомендации по выбору инструме
    нтов:
30 1. Для актуальных внешних фактов и новостей используйте веб-по
    иск (web_search).
31 2. Для поиска в локальной документации и базе знаний используйте RAG (rag_search).
32 3. Для математических расчетов используйте python_eval.
33 Выбирайте инструмент, который наилучшим образом решает задачу
    пользователя с наименьшим количеством шагов.
34 """
35
36 RESULT_FORMAT_INSTRUCTIONS = """Инструкции по формату результа
    та:
37 Формируйте понятный и хорошо отформатированный Markdown-ответ
    с цитированием фактов и источников при необходимости.
38 """
39
```

tools.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Search for current information on the internet
4 # =====
5 # Description:
6 #   Набор нативных LangChain-инструментов для AI-агентов.
7 #
8 # File: langchain_tools.py
9 # Project: ai-breadboard
10 # Package: src.ai
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from __future__ import annotations
16
17 import os
18 import json
19 from pathlib import Path
20 from typing import Any, Dict
21
22 try:
23     from langchain_core.tools import tool
24 except ImportError:
25     class DummyTool:
26         def __init__(self, func):
27             self.func = func
28             self.__name__ = getattr(func, '__name__', 'DummyTool')
29             self.__doc__ = getattr(func, '__doc__', '')
30
31         def invoke(self, input_data=None, **kwargs):
32             if isinstance(input_data, dict):
```



```

33         return self.func(**input_data)
34     elif input_data is not None:
35         return self.func(input_data, **kwargs)
36     return self.func(**kwargs)
37
38     def __call__(self, *args, **kwargs):
39         return self.func(*args, **kwargs)
40
41     def tool(func=None, *args, **kwargs):
42         if func is not None:
43             return DummyTool(func)
44         return lambda f: DummyTool(f)
45
46 from src.logger import logger
47 from header import __root__
48
49 @tool
50 async def web_search(query: str) -> str:
51     """Поиск актуальной информации в интернете через поисковые
52     адаптеры.
53
54     Args:
55         query: Поисковый запрос.
56     """
57     try:
58         from src.ai.unified_chat import get_chat_model
59         model = get_chat_model()
60         response = await model.ask(f"Найди в интернете актуаль
61 ную информацию по запросу: {query}")
62         return response
63     except Exception as e:
64         logger.error(f"[langchain_tools] Error веб-поиска: {e}
65 ")
66         return json.dumps({"error": str(e)}, ensure_ascii=False)
67
68 @tool
69 async def rag_search(query: str, top_k: int = 5) -> str:

```

```

67 |     """Семантический поиск по локальной базе знаний и документ
    | ам.
68 |
69 |     Args:
70 |         query: Запрос для семантического поиска.
71 |         top_k: Максимальное количество фрагментов.
72 |     """
73 |     try:
74 |         from src.rag.rag_manager import rag_manager
75 |         results = await rag_manager.search(query=query, limit=
    | top_k)
76 |         return json.dumps(results, ensure_ascii=False)
77 |     except Exception as e:
78 |         logger.error(f"[langchain_tools] Error RAG-поиска: {e}
    | ")
79 |         return json.dumps([], ensure_ascii=False)
80 |
81 | @tool
82 | def python_eval(code: str) -> str:
83 |     """Безопасное выполнение простых математических и строковы
    | х выражений Python.
84 |
85 |     Args:
86 |         code: Выражение или фрагмент кода для вычисления.
87 |     """
88 |     try:
89 |         allowed_globals = {"__builtins__": {"abs": abs, "min":
    | min, "max": max, "sum": sum, "round": round, "len": len}}
90 |         result = eval(code, allowed_globals, {})
91 |         return str(result)
92 |     except Exception as e:
93 |         return f"Error вычисления: {e}"
94 |
95 | @tool
96 | def file_read(file_path: str) -> str:
97 |     """Чтение содержимого текстового файла проекта.
98 |
99 |     Args:

```

```

100         file_path: Относительный или абсолютный путь к файлу.
101         """
102     try:
103         p = Path(file_path)
104         if not p.is_absolute():
105             p = __root__ / p
106         if not p.exists() or not p.is_file():
107             return f"Файл не найден: {file_path}"
108         return p.read_text(encoding="utf-8", errors="replace")
109     except Exception as e:
110         return f"Error чтения файла: {e}"
111
112 # --- Заглушки для обратной совместимости ---
113
114 @tool
115 async def search_torrents(query: str) -> str:
116     """Устаревший инструмент (артефакт удален)."""
117     return json.dumps([], ensure_ascii=False)
118
119 @tool
120 async def get_movie_metadata(title: str) -> str:
121     """Устаревший инструмент (артефакт удален)."""
122     return json.dumps({}, ensure_ascii=False)
123
124 @tool
125 def get_streaming_sources(title: str) -> str:
126     """Устаревший инструмент (артефакт удален)."""
127     return json.dumps({}, ensure_ascii=False)
128
129 @tool
130 def build_player_url(url: str, provider: str = "") -> str:
131     """Устаревший инструмент (артефакт удален)."""
132     return json.dumps({}, ensure_ascii=False)
133
134 @tool
135 async def add_torrent_download(url: str, source: str = "", title: str = "") -> str:

```

```
136 | """Устаревший инструмент (артефакт удален)."""  
137 | return "Функциональность торрентов удалена."  
138 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 from .agent import MediaSearchAgent
3 from .mcp_client import MCPClientManager
4
5 __all__ = ["MediaSearchAgent", "MCPClientManager"]
6
```

agy_chat.py

```
1 # -*- coding: utf-8 -*-  
2 from .providers.agy.chat import *  
3
```

gguf_to_onnx.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: GGUF to ONNX model conversion and optimization
4 # =====
5 # Description:
6 #   Module for converting local models to ONNX format with optional graph optimization.
7 #
8 # File: gguf_to_onnx.py
9 # Project: ai-breadboard
10 # Package: src.ai.converter
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """GGUF to ONNX model converter with optimization support.
16
17 Converts HuggingFace models to ONNX format and optionally optimizes the computational graph."""
18
19 import asyncio
20 from dataclasses import dataclass, field
21 from pathlib import Path
22 from typing import Any, Dict
23
24 from src.logger.logger import logger
25
26 CONVERTER_AVAILABLE = False
27 OPTIMIZER_AVAILABLE = False
28
29 try:
30     from transformers import AutoTokenizer # noqa: F401
31     from optimum.onnxruntime import ORTModelForCausalLM # noq
```

```

a: F401
32     CONVERTER_AVAILABLE = True
33 except ImportError:
34     pass
35
36 try:
37     from onnxruntime.transformers import optimizer as ort_opti
mizer # noqa: F401
38     OPTIMIZER_AVAILABLE = True
39 except ImportError:
40     pass
41
42 @dataclass
43 class ConversionResult:
44     """Model conversion and optimization result.
45
46     Attributes:
47         success (bool): Whether conversion succeeded.
48         output_dir (str): Directory where ONNX artifacts saved
49         .
50         optimized_path (str): Path to optimized model (if opti
mization ran).
51         error (str): Error message if conversion failed.
52         chunks_info (Dict): Metadata about exported files.
53     """
54     success: bool
55     output_dir: str = ""
56     optimized_path: str = ""
57     error: str = ""
58     chunks_info: Dict[str, Any] = field(default_factory=dict)
59
60 class GGUFConverter:
61     """Converter for models to ONNX format with optional optim
ization.
62
63     Converts HuggingFace or local models to ONNX and optionall
y runs
graph optimization passes for inference performance.

```



```

64 |     """
65 |
66 |     async def convert(
67 |         self,
68 |         model_path: str,
69 |         output_dir: str,
70 |         model_type: str = "gpt2",
71 |         opset: int = 17,
72 |         optimize: bool = True,
73 |     ) -> ConversionResult:
74 |         """Convert model to ONNX format.
75 |
76 |         Args:
77 |             model_path (str): HuggingFace model ID or path to
78 | local model directory.
79 |             output_dir (str): Directory to save ONNX artifacts
80 | .
81 |             model_type (str): Architecture type for optimizer
82 | ('gpt2', 'bert', 'bart'). Default 'gpt2'.
83 |             opset (int): ONNX opset version. Default 17.
84 |             optimize (bool): Run graph optimization passes. De
85 | fault True.
86 |
87 |         Returns:
88 |             ConversionResult: Export result and file paths.
89 |         """
90 |         if not CONVERTER_AVAILABLE:
91 |             return ConversionResult(
92 |                 success=False,
93 |                 error="optimum[onnxruntime] not installed. Run
94 | : pip install optimum[onnxruntime]",
95 |             )
96 |
97 |         src = Path(model_path)
98 |         out = Path(output_dir)
99 |         out.mkdir(parents=True, exist_ok=True)
100 |
101 |         logger.info(f"[GGUFConverter] Starting conversion: {mo

```

```

del_path} -> {out}")
97
98         try:
99             loop = asyncio.get_running_loop()
100             result = await loop.run_in_executor(
101                 None, self._export, model_path, str(out), opse
t
102             )
103             if not result.success:
104                 return result
105
106             if optimize and OPTIMIZER_AVAILABLE:
107                 onnx_file = out / "model.onnx"
108                 if onnx_file.exists():
109                     opt_path = await loop.run_in_executor(
110                         None, self._optimize, str(onnx_file),
model_type
111                     )
112                     result.optimized_path = opt_path
113
114                     logger.info(f"[GGUFConverter] Conversion completed
successfully: {out}")
115                     return result
116
117             except Exception as e:
118                 logger.error(f"[GGUFConverter] Error converting {m
odel_path}: {e}")
119                 return ConversionResult(success=False, error=str(e
))
120
121     def _export(self, model_path: str, output_dir: str, opset:
int) -> ConversionResult:
122         """Synchronous model export via optimum.
123
124         Args:
125             model_path (str): Model path or identifier.
126             output_dir (str): Output directory for ONNX files.
127             opset (int): ONNX opset version.

```

```

128 |
129 |     Returns:
130 |         ConversionResult: Export result with file informat
131 | ion.
132 |     """
133 |     try:
134 |         from transformers import AutoTokenizer
135 |         from optimum.onnxruntime import ORTModelForCausalLM
136 |
137 |         logger.info(f"[GGUFConverter] Loading tokenizer from {model_path}")
138 |         tokenizer = AutoTokenizer.from_pretrained(model_path)
139 |
140 |         logger.info("[GGUFConverter] Exporting to ONNX via ORTModelForCausalLM...")
141 |         model = ORTModelForCausalLM.from_pretrained(
142 |             model_path,
143 |             export=True,
144 |             opset=opset,
145 |         )
146 |
147 |         model.save_pretrained(output_dir)
148 |         tokenizer.save_pretrained(output_dir)
149 |
150 |         out = Path(output_dir)
151 |         onnx_files = list(out.glob("*.onnx"))
152 |         chunks_info = {
153 |             "onnx_files": [f.name for f in onnx_files],
154 |             "total_size_mb": round(
155 |                 sum(f.stat().st_size for f in onnx_files)
156 |                 / 1024 / 1024, 2
157 |             ),
158 |         }
159 |
160 |         return ConversionResult(
161 |             success=True,

```

```

160         output_dir=output_dir,
161         chunks_info=chunks_info,
162     )
163
164     except Exception as e:
165         logger.error(f"[GGUFConverter] Error exporting {mo
del_path}: {e}")
166         return ConversionResult(success=False, error=str(e
))
167
168     def _optimize(self, onnx_path: str, model_type: str) -> str:
169         """Synchronous ONNX graph optimization.
170
171         Args:
172             onnx_path (str): Path to ONNX model file.
173             model_type (str): Model architecture type.
174
175         Returns:
176             str: Path to optimized model, or empty string on e
rror.
177         """
178         try:
179             from onnxruntime.transformers import optimizer as
ort_optimizer
180             logger.info(f"[GGUFConverter] Optimizing ONNX grap
h: {onnx_path}")
181             opt_model = ort_optimizer.optimize_model(onnx_path
, model_type=model_type)
182             output_path = onnx_path.replace(".onnx", "_optimiz
ed.onnx")
183             opt_model.save_model_to_file(output_path)
184             logger.info(f"[GGUFConverter] Optimized model save
d: {output_path}")
185             return output_path
186         except Exception as e:
187             logger.warning(f"[GGUFConverter] Optimization skip
ped due to error: {e}")

```

```
188 |         return ""
189 |
190 |     @staticmethod
191 |     def is_available() -> Dict[str, bool]:
192 |         """Check availability of conversion tools.
193 |
194 |         Returns:
195 |             Dict[str, bool]: Availability of converter and opt
196 | imizer.
197 |         """
198 |         return {
199 |             "converter": CONVERTER_AVAILABLE,
200 |             "optimizer": OPTIMIZER_AVAILABLE,
201 |         }
202 | gguf_converter = GGUFConverter()
203 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Module
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: __init__.py
9 # Project: ai-breadboard
10 # Package: src.ai.converter
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from .gguf_to_onnx import GGUFConverter, ConversionResult, gguf_converter
16
17 __all__ = ["GGUFConverter", "ConversionResult", "gguf_converter"]
18
```

dev_rag.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Extract text from files for vectorization
4 # =====
5 # Description:
6 #   Build RAG index for development technical context and code
7 #   base.
8 #
9 # File: dev_rag.py
10 # Project: ai-breadboard
11 # Package: src.ai
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """Development RAG index builder for code and documentation.
17 Indexes Python and Markdown files from specified directories f
18 or semantic search."""
19
20 import json
21 import os
22 from pathlib import Path
23 from src.ai.gemini.rag import GeminiRAG
24 from src.logger import logger
25
26 # Index file path
27 DEV_RAG_DB = Path(__file__).parent.parent.parent / ".gemini" /
28 "knowledge" / "dev_rag.db"
29
30 def _file_to_text(file_path: Path) -> str:
31     """Extract text from file for vectorization.
```

```

31 |     Args:
32 |         file_path (Path): Path to file.
33 |
34 |     Returns:
35 |         str: File content with metadata.
36 |     """
37 |     try:
38 |         content = file_path.read_text(encoding='utf-8')
39 |         return f"File: {file_path.name}\nPath: {file_path}\nCo
| ntent:\n{content}"
40 |     except Exception as e:
41 |         logger.error(f"Error reading file {file_path}: {e}")
42 |         return ""
43 |
44 | def build_dev_rag(api_key: str) -> GeminiRAG:
45 |     """Build RAG index for code and documentation.
46 |
47 |     Indexes .py and .md files from specified directories.
48 |
49 |     Args:
50 |         api_key (str): Gemini API key.
51 |
52 |     Returns:
53 |         GeminiRAG: Initialized index.
54 |     """
55 |     rag = GeminiRAG(api_key=api_key, db_path=str(DEV_RAG_DB))
56 |     rag.clear()
57 |
58 |     # Directories to index
59 |     target_dirs = ["docs", "prompts", "src", "plugins"]
60 |     docs = []
61 |
62 |     for dir_name in target_dirs:
63 |         for file_path in Path(dir_name).rglob("*"):
64 |             if file_path.suffix in [".py", ".md"] and "__pycac
| he__" not in str(file_path):
65 |                 text = _file_to_text(file_path)
66 |                 if text:

```



```

67         docs.append({
68             'id': str(file_path),
69             'text': text,
70             'meta': {'path': str(file_path), 'type
': file_path.suffix}
71         })
72
73     rag.add_documents(docs)
74     logger.info(f"Developer index built. Files indexed: {len(d
ocs}}")
75     return rag
76
77 def get_dev_rag(api_key: str) -> GeminiRAG:
78     """Get developer index.
79
80     Args:
81         api_key (str): Gemini API key.
82
83     Returns:
84         GeminiRAG: Index instance.
85     """
86     return GeminiRAG(api_key=api_key, db_path=DEV_RAG_DB)
87
88 def rag_search_tool(query: str, top_k: int = 3, api_key: str =
'') -> str:
89     """Semantic search across code and documentation via RAG i
ndex.
90
91     Args:
92         query (str): Search query.
93         top_k (int): Number of results to return. Default 3.
94         api_key (str): Gemini API key.
95
96     Returns:
97         str: JSON string with list of found files.
98     """
99     rag = get_dev_rag(api_key)
100     if rag.count() == 0:

```

```
101 |         return json.dumps({'error': 'Developer index empty. Ru
    | n rebuild_dev_rag'}, ensure_ascii=False)
102 |
103 |     results = rag.search(query, top_k=top_k, threshold=0.3)
104 |     return json.dumps(results, ensure_ascii=False, indent=2)
105 |
```

foundry_chat.py

```
1 # -*- coding: utf-8 -*-  
2 from .providers.foundry.chat import *  
3 from .providers.foundry.client import FoundryClient  
4
```

api.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Google Generative AI API Methods
4 # =====
5 # Description:
6 #   Implementation of API methods for Google Generative AI.
7 #   Provides ask, chat, chat_stream, ask_with_tools, ask_with_
8 #   tools_stream methods.
9 #   Handles retries, quota exhaustion, model switching, and st
10 #   reaming responses.
11 #
12 # File: api.py
13 # Project: ai-breadboard
14 # Package: src.ai.gemini
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # =====
18
19 import asyncio
20 import json
21 from typing import Any, AsyncGenerator
22
23 from google.genai import types
24
25 from src.logger.logger import logger
26 from src.secrets.api_key_state import update_last_run
27 from src.utils.jjson import j_loads
28
29 from .core import GoogleGenerativeAICore
30 from .errors import GoogleGenerativeAIErrorMixin
31 from .history import GoogleGenerativeAIHistoryMixin
32 from .config import GoogleGenerativeAIConfigMixin
```

```

32 |
33 | class GoogleGenerativeAI(
34 |     GoogleGenerativeAICore,
35 |     GoogleGenerativeAIErrorMixin,
36 |     GoogleGenerativeAIHistoryMixin,
37 |     GoogleGenerativeAIConfigMixin,
38 | ):
39 |     """Class for interaction with Google Generative AI (Gemini
40 | ) models.
41 |
42 |     Attributes:
43 |         api_key (str): Active API key for requests.
44 |         model_name (str): Name of the Gemini model being used.
45 |         generation_config (dict): Generation parameters by default.
46 |         system_instruction (str): Base system instruction.
47 |         api_key_names (list[str]): List of allowed key names.
48 |         save_history_chat (bool): Flag for saving chat history context.
49 |         sleep_on_exhausted (bool): Flag for waiting on quota exhaustion.
50 |
51 |     async def ask(
52 |         self,
53 |         q: str,
54 |         attempts: int = 15,
55 |         generation_config: dict = {},
56 |     ) -> str:
57 |         """Send single text request to model.
58 |
59 |     Args:
60 |         q (str): Query text.
61 |         attempts (int): Maximum number of attempts. Default: 15.
62 |         generation_config (dict): Additional generation parameters.
63 |

```

```

64 |         Returns:
65 |             str: Model response or error message.
66 |
67 |         Examples:
68 |             >>> ai = GoogleGenerativeAI()
69 |             >>> ans = await ai.ask("What is the capital of France?")
70 |             ""
71 |         if not q:
72 |             return ''
73 |
74 |         self._key_errors = {}
75 |         if self._all_keys_exhausted:
76 |             if not self._switch_api_key():
77 |                 return self._get_exhausted_error_msg()
78 |             self._all_keys_exhausted = False
79 |
80 |         for attempt in range(attempts):
81 |             try:
82 |                 config = self._build_content_config(generation
83 | _config=generation_config)
84 |                 response = self._client.models.generate_content(
85 |                     model=self.model_name,
86 |                     contents=q,
87 |                     config=config,
88 |                 )
89 |                 if response and response.text:
90 |                     response_text: str = self._normalize_text(
91 | response.text)
92 |                     response_text = self._remove_html_blocks(r
93 | esponse_text)
94 |                     update_last_run(self._key_names_active[0]
95 | if self._key_names_active else '')
96 |                     self._unavailable_attempts = 0
97 |                     return response_text
98 |
99 |         logger.debug(f'GoogleGenerativeAI: Empty model

```

```

    response on attempt {attempt + 1}')
96         await asyncio.sleep(2 ** min(attempt, 4))
97     except Exception as ex:
98         should_retry: bool = await self._handle_api_er
ror(ex, self.model_name, attempt, attempts)
99         if not should_retry:
100             return f'Model error: {self._last_exceptio
n or str(ex)}'
101
102     return self._get_exhausted_error_msg()
103
104     async def chat(
105         self,
106         q: str,
107         history: list[dict] = (),
108         flag: str = 'save_chat',
109         system_instruction: str = '',
110         attempts: int = 15,
111         model_name: str = '',
112         **kwargs,
113     ) -> str:
114         """Process message in chat dialog context.
115
116         Args:
117             q (str): User message.
118             history (list[dict]): External message history for
context restoration.
119             flag (str): History management flag ('save_chat',
'clear', 'start_new').
120             system_instruction (str): Override system instruct
ion.
121             attempts (int): Maximum number of retry attempts.
122             model_name (str): Explicit model override for requ
est.
123             **kwargs: Additional unused parameters.
124
125         Returns:
126             str: Model response or diagnostic error message.

```

```

127 |
128 |     Examples:
129 |         >>> ai = GoogleGenerativeAI()
130 |         >>> ans = await ai.chat("Hello!", flag="start_new"
131 | )
132 |         ""
133 |     if not q:
134 |         return ''
135 |
136 |     self._key_errors = {}
137 |     if self._all_keys_exhausted:
138 |         if not self._switch_api_key():
139 |             return self._get_exhausted_error_msg()
140 |         self._all_keys_exhausted = False
141 |
142 |     instruction: str = system_instruction or self.system_i
143 | nstruction or ''
144 |     active_model: str = model_name or self.model_name
145 |
146 |     for attempt in range(attempts):
147 |         try:
148 |             # 1. Stateless mode (no history saving)
149 |             if not self.save_history_chat:
150 |                 config = self._build_content_config(instru
151 | ction)
152 |                 response = self._client.models.generate_co
153 | ntent(
154 |                     model=active_model,
155 |                     contents=q,
156 |                     config=config,
157 |                 )
158 |             if response and response.text:
159 |                 response_text: str = self._normalize_t
160 | ext(response.text)
161 |                 response_text = self._remove_html_bloc
162 | ks(response_text)
163 |                 update_last_run(self._key_names_active
164 | [0] if self._key_names_active else '')

```



```

158         self._unavailable_attempts = 0
159         return response_text
160
161         await asyncio.sleep(2 ** min(attempt, 4))
162         continue
163
164     # 2. Chat mode with history preservation
165     if history:
166         self.chat_history = list(history)
167         self._restore_chat_from_history()
168     elif flag in ['clear', 'start_new']:
169         self.chat_history = []
170         self._chat = self._start_chat()
171
172     response = self._chat.send_message(q)
173     if response and response.text:
174         response_text = self._normalize_text(response
175         nse.text)
176         response_text = self._remove_html_blocks(response_text)
177         self.chat_history.append({'role': 'user',
178         'parts': [q]})
179         self.chat_history.append({'role': 'model',
180         'parts': [response_text]})
181         self._unavailable_attempts = 0
182         return response_text
183
184     logger.error('GoogleGenerativeAI: Empty model
185     response in chat')
186     await asyncio.sleep(2 ** min(attempt, 4))
187     except Exception as ex:
188         should_retry: bool = await self._handle_api_error(ex, active_model, attempt, attempts)
189         if not should_retry:
190             return f'Chat error: {self._last_exception
191             or str(ex)}'
192
193     return self._get_exhausted_error_msg()

```

```

189
190     async def chat_stream(
191         self,
192         q: str,
193         history: list[dict] = (),
194         flag: str = 'save_chat',
195         system_instruction: str = '',
196         attempts: int = 15,
197         model_name: str = '',
198         generation_config: dict = {},
199         **kwargs,
200     ) -> AsyncGenerator[str, None]:
201         """Stream-generate model response as async generator.
202
203         Args:
204             q (str): User question.
205             history (list[dict]): Message history.
206             flag (str): History management flag.
207             system_instruction (str): System instruction.
208             attempts (int): Maximum retry attempts.
209             model_name (str): Model override.
210             generation_config (dict): Generation settings.
211
212         Yields:
213             str: Next generated text fragment (chunk).
214         """
215         if not q:
216             return
217
218         self._key_errors = {}
219         if self._all_keys_exhausted:
220             if not self._switch_api_key():
221                 yield self._get_exhausted_error_msg()
222                 return
223             self._all_keys_exhausted = False
224
225         instruction: str = system_instruction or self.system_i
nstruction or ''

```

```

226         active_model: str = model_name or self.model_name
227
228         for attempt in range(attempts):
229             try:
230                 if not self.save_history_chat:
231                     config = self._build_content_config(instru
232 ction, generation_config=generation_config)
233                     contents = self._prepare_contents(q, histo
234 ry)
235
236                     def _collect_stateless(_client=self._clien
237 t, _m=active_model, _c=contents, _cfg=config):
238                         res: list[str] = []
239                         for chunk in _client.models.generate_c
240 ontent_stream(model=_m, contents=_c, config=_cfg):
241                             if chunk.text:
242                                 res.append(chunk.text)
243                         return res
244
245                     chunks = await asyncio.to_thread(_collect_
246 stateless)
247
248                     if chunks:
249                         for chunk_text in chunks:
250                             yield chunk_text
251                         update_last_run(self._key_names_active
252 [0] if self._key_names_active else '')
253                         self._unavailable_attempts = 0
254                         return
255
256                     await asyncio.sleep(2 ** min(attempt, 4))
257                     continue
258
259             if history:
260                 self.chat_history = list(history)
261                 self._restore_chat_from_history()
262             elif flag in ['clear', 'start_new']:
263                 self.chat_history = []
264                 self._chat = self._start_chat()

```

```

258
259         _chat_ref = self._chat
260
261         def _collect_chat(_chat=_chat_ref, _query=q):
262             res: list[str] = []
263             for chunk in _chat.send_message_stream(_qu
264         ery):
265                 if chunk.text:
266                     res.append(chunk.text)
267             return res
268
269         chunks = await asyncio.to_thread(_collect_chat
270     )
271
272         full_text: str = ''.join(chunks)
273         if full_text:
274             for chunk_text in chunks:
275                 yield chunk_text
276             normalized: str = self._remove_html_blocks
277         (self._normalize_text(full_text))
278         self.chat_history.append({'role': 'user',
279         'parts': [q]})
280         self.chat_history.append({'role': 'model',
281         'parts': [normalized]})
282         self._unavailable_attempts = 0
283         return
284
285         await asyncio.sleep(2 ** min(attempt, 4))
286     except Exception as ex:
287         should_retry: bool = await self._handle_api_er
288         ror(ex, active_model, attempt, attempts)
289         if not should_retry:
290             yield f'Streaming error: {self._last_excep
291         tion or str(ex)}'
292         return
293
294     async def ask_with_tools(
295         self,
296         q: str,

```

```

289         tools: list,
290         tool_dispatcher: Any,
291         system_instruction: str = '',
292         model_name: str = '',
293     ) -> str:
294         """Execute request with external function calling support (Agentic loop).
295
296         Args:
297             q (str): User text query.
298             tools (list): List of tool definitions types.Tool.
299             tool_dispatcher (Any): Function call dispatcher (name, args) -> str.
300             system_instruction (str): System instruction.
301             model_name (str): Model name to use.
302
303         Returns:
304             str: Final model text response.
305
306         Examples:
307             >>> ans = await ai.ask_with_tools("Weather in Paris", tools, dispatcher)
308             """
309             if not q:
310                 return ''
311
312             contents: list[types.Content] = [types.Content(role='user', parts=[types.Part.from_text(text=q)])]
313             instruction: str = system_instruction or self.system_instruction or ''
314             active_model: str = model_name or self.model_name
315             config = self._build_content_config(instruction, tools)
316
317             for _ in range(10):
318                 response = self._client.models.generate_content(
319                     model=active_model,
320                     contents=contents,

```

```

321         config=config,
322     )
323     candidate = response.candidates[0] if response and
response.candidates else False
324     if not candidate:
325         break
326
327     tool_calls = [p for p in candidate.content.parts i
f p.function_call]
328     text_parts = [p.text for p in candidate.content.pa
rts if p.text]
329
330     if not tool_calls:
331         return '\n'.join(text_parts)
332
333     contents.append(candidate.content)
334     tool_results: list[types.Part] = []
335     for part in tool_calls:
336         fc = part.function_call
337         result = tool_dispatcher(fc.name, dict(fc.args
))
338         tool_results.append(
339             types.Part.from_function_response(
340                 name=fc.name,
341                 response={'result': result},
342             )
343         )
344     contents.append(types.Content(role='tool', parts=t
ool_results))
345
346     return ''
347
348     async def ask_with_tools_stream(
349         self,
350         q: str,
351         tools: list,
352         tool_dispatcher: Any,
353         system_instruction: str = '',

```

```

354         model_name: str = '',
355         history: list[dict] = (),
356     ) -> AsyncGenerator[dict[str, str], None]:
357         """Execute request with function calling and stream fi
nal response.
358
359         Args:
360             q (str): User request.
361             tools (list): List of tools.
362             tool_dispatcher (Any): Function call dispatcher.
363             system_instruction (str): System instruction.
364             model_name (str): Model name.
365             history (list[dict]): Message history.
366
367         Yields:
368             dict[str, str]: Events like {"text": "chunk"} or {
"status": "message"}.
369         """
370         if not q:
371             return
372
373         contents: list[types.Content] = self._prepare_contents
(q, history)
374         instruction: str = system_instruction or self.system_i
nstruction or ''
375         active_model: str = model_name or self.model_name
376         config = self._build_content_config(instruction, tools
)
377
378         for _ in range(10):
379             try:
380                 response = await asyncio.to_thread(
381                     self._client.models.generate_content,
382                     model=active_model,
383                     contents=contents,
384                     config=config,
385                 )
386             except Exception as ex:

```

```

387         should_retry: bool = await self._handle_api_er
ror(ex, active_model, 0, 3)
388         if should_retry:
389             active_model = self.model_name
390             continue
391         yield {'status': f'Error generate_content: {st
r(ex)}}'}
392         return
393
394         candidate = response.candidates[0] if response and
response.candidates else False
395         if not candidate:
396             break
397
398         tool_calls = [p for p in candidate.content.parts i
f p.function_call]
399         text_parts = [p.text for p in candidate.content.pa
rts if p.text]
400
401         if not tool_calls:
402             try:
403                 response_stream = self._client.models.gene
rate_content_stream(
404                     model=active_model,
405                     contents=contents,
406                     config=config,
407                 )
408                 for chunk in response_stream:
409                     if chunk.text:
410                         yield {'text': chunk.text}
411             except Exception as ex:
412                 logger.error(f'GoogleGenerativeAI: Error s
treaming ask_with_tools_stream: {ex}')
413                 if text_parts:
414                     yield {'text': ''.join(text_parts)}
415         return
416
417         contents.append(candidate.content)

```



```
418 |         tool_results: list[types.Part] = []
419 |         for part in tool_calls:
420 |             fc = part.function_call
421 |             args_json: str = json.dumps(dict(fc.args), ens
| sure_ascii=False)
422 |             yield {'status': f'Function call {fc.name}({ar
| gs_json)}'}
423 |             result = tool_dispatcher(fc.name, dict(fc.args
| ))
424 |             tool_results.append(
425 |                 types.Part.from_function_response(
426 |                     name=fc.name,
427 |                     response={'result': result},
428 |                 )
429 |             )
430 |         contents.append(types.Content(role='tool', parts=t
| ool_results))
431 |
```

api_chat.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Google Generative AI API Methods - Chat
4 # =====
5 # Description:
6 #   Implementation of chat API methods for Google Generative A
7 #   Provides chat and chat_stream methods for chat-based inter
8 #   Handles both stateless and stateful chat modes with histor
9 #
10 # File: api_chat.py
11 # Project: ai-breadboard
12 # Package: src.ai.gemini
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 import asyncio
18 from typing import Any, AsyncGenerator
19
20 from src.logger.logger import logger
21 from src.secrets.api_key_state import update_last_run
22
23 from .core import GoogleGenerativeAICore
24 from .errors import GoogleGenerativeAIErrorMixin
25 from .history import GoogleGenerativeAIHistoryMixin
26 from .config import GoogleGenerativeAIConfigMixin
27
28
29 class GoogleGenerativeAIChat(
30     GoogleGenerativeAICore,
```

```

31 |     GoogleGenerativeAIErrorMixin,
32 |     GoogleGenerativeAIHistoryMixin,
33 |     GoogleGenerativeAIConfigMixin,
34 | ):
35 |     """Class for chat API methods."""
36 |
37 |     async def chat(
38 |         self,
39 |         q: str,
40 |         history: list[dict] = (),
41 |         flag: str = 'save_chat',
42 |         system_instruction: str = '',
43 |         attempts: int = 15,
44 |         model_name: str = '',
45 |         **kwargs,
46 |     ) -> str:
47 |         """Process message in chat dialog context.
48 |
49 |         Args:
50 |             q (str): User message.
51 |             history (list[dict]): External message history for
52 | context restoration.
53 |             flag (str): History management flag ('save_chat',
54 | 'clear', 'start_new').
55 |             system_instruction (str): Override system instruct
56 | ion.
57 |             attempts (int): Maximum number of retry attempts.
58 |             model_name (str): Explicit model override for requ
59 | est.
60 |             **kwargs: Additional unused parameters.
61 |
62 |         Returns:
63 |             str: Model response or diagnostic error message.
64 |
65 |         Examples:
66 |             >>> ai = GoogleGenerativeAI()
67 |             >>> ans = await ai.chat("Hello!", flag="start_new"
68 | )

```

```

64 |         """
65 |         if not q:
66 |             return ''
67 |
68 |         self._key_errors = {}
69 |         if self._all_keys_exhausted:
70 |             if not self._switch_api_key():
71 |                 return self._get_exhausted_error_msg()
72 |             self._all_keys_exhausted = False
73 |
74 |         instruction: str = system_instruction or self.system_i
| nstruction or ''
75 |         active_model: str = model_name or self.model_name
76 |
77 |         for attempt in range(attempts):
78 |             try:
79 |                 # 1. Stateless mode (no history saving)
80 |                 if not self.save_history_chat:
81 |                     config = self._build_content_config(instru
| ction)
82 |                     response = self._client.models.generate_co
| ntent(
83 |                         model=active_model,
84 |                         contents=q,
85 |                         config=config,
86 |                     )
87 |                     if response and response.text:
88 |                         response_text: str = self._normalize_t
| ext(response.text)
89 |                         response_text = self._remove_html_bloc
| ks(response_text)
90 |                         update_last_run(self._key_names_active
| [0] if self._key_names_active else '')
91 |                         self._unavailable_attempts = 0
92 |                         return response_text
93 |
94 |                     await asyncio.sleep(2 ** min(attempt, 4))
95 |                     continue

```

```

96
97         # 2. Chat mode with history preservation
98         if history:
99             self.chat_history = list(history)
100             self._restore_chat_from_history()
101         elif flag in ['clear', 'start_new']:
102             self.chat_history = []
103             self._chat = self._start_chat()
104
105         response = self._chat.send_message(q)
106         if response and response.text:
107             response_text = self._normalize_text(response
nse.text)
108             response_text = self._remove_html_blocks(r
esponse_text)
109             self.chat_history.append({'role': 'user',
'parts': [q]})
110             self.chat_history.append({'role': 'model',
'parts': [response_text]})
111             self._unavailable_attempts = 0
112             return response_text
113
114             logger.error('GoogleGenerativeAI: Empty model
response in chat')
115             await asyncio.sleep(2 ** min(attempt, 4))
116             except Exception as ex:
117                 should_retry: bool = await self._handle_api_er
ror(ex, active_model, attempt, attempts)
118                 if not should_retry:
119                     return f'Chat error: {self._last_exception
or str(ex)}'
120
121         return self._get_exhausted_error_msg()
122
123     async def chat_stream(
124         self,
125         q: str,
126         history: list[dict] = (),

```

```

127         flag: str = 'save_chat',
128         system_instruction: str = '',
129         attempts: int = 15,
130         model_name: str = '',
131         generation_config: dict = {},
132         **kwargs,
133     ) -> AsyncGenerator[str, None]:
134         """Stream-generate model response as async generator.
135
136         Args:
137             q (str): User question.
138             history (list[dict]): Message history.
139             flag (str): History management flag.
140             system_instruction (str): System instruction.
141             attempts (int): Maximum retry attempts.
142             model_name (str): Model override.
143             generation_config (dict): Generation settings.
144
145         Yields:
146             str: Next generated text fragment (chunk).
147         """
148         if not q:
149             return
150
151         self._key_errors = {}
152         if self._all_keys_exhausted:
153             if not self._switch_api_key():
154                 yield self._get_exhausted_error_msg()
155                 return
156             self._all_keys_exhausted = False
157
158         instruction: str = system_instruction or self.system_i
nstruction or ''
159         active_model: str = model_name or self.model_name
160
161         for attempt in range(attempts):
162             try:
163                 if not self.save_history_chat:

```

```

164         config = self._build_content_config(instru
| ction, generation_config=generation_config)
165         contents = self._prepare_contents(q, histo
| ry)
166
167         def _collect_stateless(_client=self._clien
| t, _m=active_model, _c=contents, _cfg=config):
168             res: list[str] = []
169             for chunk in _client.models.generate_c
| ontent_stream(model=_m, contents=_c, config=_cfg):
170                 if chunk.text:
171                     res.append(chunk.text)
172             return res
173
174         chunks = await asyncio.to_thread(_collect_
| stateless)
175         if chunks:
176             for chunk_text in chunks:
177                 yield chunk_text
178                 update_last_run(self._key_names_active
| [0] if self._key_names_active else '')
179                 self._unavailable_attempts = 0
180             return
181
182         await asyncio.sleep(2 ** min(attempt, 4))
183         continue
184
185         if history:
186             self.chat_history = list(history)
187             self._restore_chat_from_history()
188         elif flag in ['clear', 'start_new']:
189             self.chat_history = []
190             self._chat = self._start_chat()
191
192         _chat_ref = self._chat
193
194         def _collect_chat(_chat=_chat_ref, _query=q):
195             res: list[str] = []

```

```

196         for chunk in _chat.send_message_stream(_qu
    | ery):
197             if chunk.text:
198                 res.append(chunk.text)
199             return res
200
201         chunks = await asyncio.to_thread(_collect_chat
    | )
202         full_text: str = ''.join(chunks)
203         if full_text:
204             for chunk_text in chunks:
205                 yield chunk_text
206             normalized: str = self._remove_html_blocks
    | (self._normalize_text(full_text))
207             self.chat_history.append({'role': 'user',
    | 'parts': [q]})
208             self.chat_history.append({'role': 'model',
    | 'parts': [normalized]})
209             self._unavailable_attempts = 0
210             return
211
212             await asyncio.sleep(2 ** min(attempt, 4))
213         except Exception as ex:
214             should_retry: bool = await self._handle_api_er
    | ror(ex, active_model, attempt, attempts)
215             if not should_retry:
216                 yield f'Streaming error: {self._last_excep
    | tion or str(ex)}'
217             return
218

```


api_single.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Google Generative AI API Methods - Single Request
4 # =====
5 # Description:
6 #   Implementation of single request API methods for Google Generative AI.
7 #   Provides ask method for single text query with retry logic and
8 #   quota exhaustion handling.
9 #
10 # File: api_single.py
11 # Project: ai-breadboard
12 # Package: src.ai.gemini
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 import asyncio
18
19 from src.logger.logger import logger
20 from src.secrets.api_key_state import update_last_run
21
22 from .core import GoogleGenerativeAICore
23 from .errors import GoogleGenerativeAIErrorMixin
24 from .history import GoogleGenerativeAIHistoryMixin
25 from .config import GoogleGenerativeAIConfigMixin
26
27
28 class GoogleGenerativeAISingleRequest(
29     GoogleGenerativeAICore,
30     GoogleGenerativeAIErrorMixin,
```

```

31 |     GoogleGenerativeAIHistoryMixin,
32 |     GoogleGenerativeAIConfigMixin,
33 | ):
34 |     """Class for single request API methods."""
35 |
36 |     async def ask(
37 |         self,
38 |         q: str,
39 |         attempts: int = 15,
40 |         generation_config: dict = {},
41 |     ) -> str:
42 |         """Send single text request to model.
43 |
44 |         Args:
45 |             q (str): Query text.
46 |             attempts (int): Maximum number of attempts. Default: 15.
47 |             generation_config (dict): Additional generation parameters.
48 |
49 |         Returns:
50 |             str: Model response or error message.
51 |
52 |         Examples:
53 |             >>> ai = GoogleGenerativeAI()
54 |             >>> ans = await ai.ask("What is the capital of France?")
55 |
56 |         if not q:
57 |             return ''
58 |
59 |         self._key_errors = {}
60 |         if self._all_keys_exhausted:
61 |             if not self._switch_api_key():
62 |                 return self._get_exhausted_error_msg()
63 |             self._all_keys_exhausted = False
64 |
65 |         for attempt in range(attempts):

```

```

66 |         try:
67 |             config = self._build_content_config(generation
| _config=generation_config)
68 |             response = self._client.models.generate_content(
| t(
69 |                 model=self.model_name,
70 |                 contents=q,
71 |                 config=config,
72 |             )
73 |             if response and response.text:
74 |                 response_text: str = self._normalize_text(
| response.text)
75 |                 response_text = self._remove_html_blocks(r
| esponse_text)
76 |                 update_last_run(self._key_names_active[0]
| if self._key_names_active else '')
77 |                 self._unavailable_attempts = 0
78 |                 return response_text
79 |
80 |                 logger.debug(f'GoogleGenerativeAI: Empty model
| response on attempt {attempt + 1}')
81 |                 await asyncio.sleep(2 ** min(attempt, 4))
82 |             except Exception as ex:
83 |                 should_retry: bool = await self._handle_api_er
| ror(ex, self.model_name, attempt, attempts)
84 |                 if not should_retry:
85 |                     return f'Model error: {self._last_exceptio
| n or str(ex)}'
86 |
87 |         return self._get_exhausted_error_msg()
88 |

```

chat_response_store.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Store user-approved model responses to JSON fi
  les
4 # =====
5 # Description:
6 #   Stores model responses explicitly approved by user to JSON
  files.
7 #   Manages persistence of chat and voice text responses with
  metadata
8 #   including timestamps, user IDs, and unique identifiers for
  auditing.
9 #
10 # File: chat_response_store.py
11 # Project: ai-breadboard
12 # Package: src.ai.gemini
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 import json
18 import uuid
19 from datetime import datetime
20 from pathlib import Path
21
22 from src.logger import logger
23
24 # Directory for storing approved responses
25 _STORE_DIR = Path(__file__).parent.parent.parent / 'plugins' /
  'media_organizer' / 'data' / 'chat_responses'
26 _STORE_DIR.mkdir(parents=True, exist_ok=True)
27
28 def save_approved_response(user_id: str, query: str, chat_text
```

```

: str, voice_text: str = '') -> bool:
29     """Save user-approved model response to JSON file.
30
31     Args:
32         user_id: User identifier (database ID or anon_<ip>).
33         query: Original user query.
34         chat_text: Model response text for chat.
35         voice_text: Model response text for voice narrator (op
36         tional).
37
38     Returns:
39         True if save successful, otherwise False.
40
41     """
42     try:
43         entry = {
44             'id': str(uuid.uuid4()),
45             'timestamp': datetime.utcnow().isoformat(),
46             'user_id': str(user_id),
47             'query': query,
48             'chat_text': chat_text,
49             'voice_text': voice_text,
50         }
51         filename = f"{datetime.utcnow().strftime('%Y%m%d_%H%M%S')}__{entry['id'][:8]}.json"
52         filepath = _STORE_DIR / filename
53         filepath.write_text(json.dumps(entry, ensure_ascii=False, indent=2), encoding='utf-8')
54         logger.info(f"[ChatResponseStore] Approved response saved: {filename}")
55         return True
56     except Exception as ex:
57         logger.error('[ChatResponseStore] Error saving response', ex)
58         return False
59
60 def list_responses(user_id: str = '') -> list[dict]:
61     """Return list of all stored approved responses (optionall
62     y filtered by user_id).

```

```

60 |
61 |     Args:
62 |         user_id: If provided – return only records for this user.
63 |
64 |     Returns:
65 |         List of dictionaries with response data.
66 |     """
67 |     results = []
68 |     for fp in sorted(_STORE_DIR.glob('*.json')):
69 |         try:
70 |             entry = json.loads(fp.read_text(encoding='utf-8'))
71 |             if user_id and entry.get('user_id') != str(user_id):
72 |                 continue
73 |             results.append(entry)
74 |         except Exception as ex:
75 |             logger.error(f'[ChatResponseStore] Error reading file {fp.name}', ex)
76 |     return results
77 |
78 | def update_response(doc_id: str, query: str, chat_text: str, voice_text: str = '') -> bool:
79 |     """Update saved dialog content on disk by its ID.
80 |
81 |     Args:
82 |         doc_id: Document ID.
83 |         query: New request text.
84 |         chat_text: New model response.
85 |         voice_text: New narrator text.
86 |
87 |     Returns:
88 |         True on success, otherwise False.
89 |     """
90 |     try:
91 |         for fp in _STORE_DIR.glob('*.json'):
92 |             try:
93 |                 entry = json.loads(fp.read_text(encoding='utf-

```

```

8''))
94         if entry.get('id') == doc_id:
95             entry['query'] = query
96             entry['chat_text'] = chat_text
97             entry['voice_text'] = voice_text
98             fp.write_text(json.dumps(entry, ensure_ascii=False, indent=2), encoding='utf-8')
99             logger.info(f"[ChatResponseStore] RAG response updated: {fp.name}")
100             return True
101         except Exception as ex:
102             logger.error(f"[ChatResponseStore] Error parsing during update {fp.name}", ex)
103             return False
104     except Exception as ex:
105         logger.error('[ChatResponseStore] Error updating response', ex)
106         return False
107
108 def delete_response(doc_id: str) -> bool:
109     """Delete saved dialog file from disk by its ID.
110
111     Args:
112         doc_id: Document ID.
113
114     Returns:
115         True on success, otherwise False.
116     """
117     try:
118         for fp in _STORE_DIR.glob('*.json'):
119             try:
120                 entry = json.loads(fp.read_text(encoding='utf-8'))
121                 if entry.get('id') == doc_id:
122                     fp.unlink()
123                     logger.info(f"[ChatResponseStore] RAG response deleted: {fp.name}")
124                     return True

```

```
125 |             except Exception as ex:
126 |                 logger.error(f"[ChatResponseStore] Error parsing during delete {fp.name}", ex)
127 |             return False
128 |         except Exception as ex:
129 |             logger.error('[ChatResponseStore] Error deleting response', ex)
130 |             return False
131 |
```


config.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Google Generative AI Configuration Builder
4 # =====
5 # Description:
6 #   Configuration building for Google Generative AI API requests.
7 #   Provides methods for building content config and text normalization.
8 #
9 # File: config.py
10 # Project: ai-breadboard
11 # Package: src.ai.gemini
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import re
17 from typing import Any
18
19 from google.genai import types
20
21
22 def normalize_text(text: str) -> str:
23     """Normalization of model response text.
24
25     Replacement of escaped newline sequences with actual line
26     break characters.
27
28     Args:
29         text (str): Input text for normalization.
30
31     Returns:
```

```

31         str: Normalized text with actual newlines.
32
33     Examples:
34         >>> normalize_text("Line 1\\nLine 2")
35         'Line 1\\nLine 2'
36         """
37     if not text:
38         return ''
39     return re.sub(r'\\n', '\n', text)
40
41
42 def remove_html_blocks(text: str) -> str:
43     """Removal of HTML markup blocks from model response.
44
45     Args:
46         text (str): Input text with possible HTML blocks.
47
48     Returns:
49         str: Text with ``html ... `` blocks removed.
50
51     Examples:
52         >>> remove_html_blocks("``html<div>Test</div>``Hello
53         ")
54         'Hello'
55         """
56     if not text:
57         return ''
58     return re.sub(r'``html.*?``', '', text, flags=re.DOTALL)
59
60 class GoogleGenerativeAIConfigMixin:
61     """Mixin class for configuration building in GoogleGenerat
62     iveAI.
63
64     Provides methods for building content config and text norm
65     alization.
66     """

```

```

66     def _build_content_config(
67         self,
68         instruction: str = '',
69         tools: list = (),
70         generation_config: dict = {},
71     ) -> types.GenerateContentConfig:
72         """Build content generation configuration object for G
73         emini SDK.
74
75         Args:
76             instruction (str): System instruction. Default: ''
77             .
78             tools (list): Set of model tools (functions).
79             generation_config (dict): Additional generation pa
80             rameters.
81
82         Returns:
83             types.GenerateContentConfig: Configured generation
84             object.
85
86         """
87         cfg_kwargs: dict[str, Any] = {}
88         gen_cfg: dict[str, Any] = {}
89
90         if isinstance(self.generation_config, dict):
91             gen_cfg.update(self.generation_config)
92         if generation_config:
93             gen_cfg.update(generation_config)
94
95         response_type: str = gen_cfg.pop('response_type', 'bot
96         h')
97
98         inst: str = instruction or self.system_instruction or
99         ''
100
101         if inst:
102             if response_type == 'chat':
103                 format_rule: str = (
104                     '\n\nCRITICAL: You must format your respon
105                     se for reading on a screen.\n'

```

```

97         'Provide a detailed styled markdown respon
se for the user to read.'
98     )
99     elif response_type == 'voice':
100         format_rule = (
101             '\n\nCRITICAL: You must format your respon
se for a voice narrator (TTS).\n'
102             'Provide a very concise, clear speech-frie
ndly text, using simple language, '
103             'no markdown, no special symbols, write al
l numbers as words.'
104         )
105     else:
106         format_rule = (
107             '\n\nCRITICAL: You must format your respon
se exactly as follows, with no extra text outside these blocks
:\n'
108             '[CHAT]\n<detailed styled markdown respons
e for the user to read>\n'
109             '[VOICE]\n<very concise, clear speech-frie
ndly text for narrator, using simple language, '
110             'no markdown, no special symbols, write al
l numbers as words>'
111         )
112         inst += format_rule
113         cfg_kwargs['system_instruction'] = inst
114
115     all_tools: list = list(tools) if tools else []
116     has_search: bool = any(
117         hasattr(t, 'google_search') or (isinstance(t, dict
) and 'google_search' in t)
118         for t in all_tools
119     )
120     if not has_search:
121         all_tools.append(types.Tool(google_search=types.Go
ogleSearch()))
122         cfg_kwargs['tools'] = all_tools
123

```

```
124         if gen_cfg:
125             for k in ['temperature', 'top_p', 'top_k', 'response_mime_type']:
126                 val = gen_cfg.get(k)
127                 if val:
128                     cfg_kwargs[k] = val
129
130     return types.GenerateContentConfig(**cfg_kwargs)
131
132 def _normalize_text(self, text: str) -> str:
133     """Normalization of model response text.
134
135     Args:
136         text (str): Input text for normalization.
137
138     Returns:
139         str: Normalized text with actual newlines.
140     """
141     return normalize_text(text)
142
143 def _remove_html_blocks(self, text: str) -> str:
144     """Removal of HTML markup blocks from model response.
145
146     Args:
147         text (str): Input text with possible HTML blocks.
148
149     Returns:
150         str: Text with ``html ... `` blocks removed.
151     """
152     return remove_html_blocks(text)
153
```

core.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Core Google Generative AI Client
4 # =====
5 # Description:
6 #   Core client class for Google Generative AI API interaction
7 #   .
8 #   Manages API key pool, model rotation, and base configurati
9 #   on.
10 #
11 # File: core.py
12 # Project: ai-breadboard
13 # Package: src.ai.gemini
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # =====
17
18 import time
19 from dataclasses import dataclass, field
20 from pathlib import Path
21 from typing import Any
22
23 from google import genai
24
25 from src.ai.orchestration.model_manager import (
26     add_unsupported_model as _mgr_add_unsupported_model,
27     get_available_models as _mgr_get_available_models,
28     load_unsupported_models as _mgr_load_unsupported_models,
29 )
30 from src.config import server_cfg
31 from src.logger.logger import logger
32 from src.secrets.api_key_state import (
33     get_status,
```

```

32 |     load_api_keys,
33 |     mark_exhausted,
34 |     next_available_in,
35 |     update_last_run,
36 | )
37 | from src.utils.jjson import j_loads
38 |
39 | # Loading local configuration of Gemini module
40 | _config_path: Path = Path(__file__).parent / 'config.json'
41 | _gemini_config: dict = j_loads(_config_path) if _config_path.e
    | xists() else {}
42 | _DEFAULT_MODEL: str = (
43 |     _gemini_config.get('model', 'gemini-flash-latest')
44 |     if isinstance(_gemini_config, dict)
45 |     else 'gemini-flash-latest'
46 | )
47 | _DEFAULT_SAVE_HISTORY: bool = (
48 |     _gemini_config.get('save_history_chat', False)
49 |     if isinstance(_gemini_config, dict)
50 |     else False
51 | )
52 |
53 |
54 | def load_unsupported_models() -> set[str]:
55 |     """Loading списка неподдерживаемых и устаревших моделей Ge
    | mini.
56 |
57 |     Returns:
58 |         set[str]: Множество наименований неподдерживаемых моде
    | лей.
59 |
60 |     Examples:
61 |         >>> unsupported = load_unsupported_models()
62 |         >>> isinstance(unsupported, set)
63 |         True
64 |         """
65 |     return _mgr_load_unsupported_models('gemini')
66 |

```

```

67
68 def add_unsupported_model(model_name: str, reason: str = '') -
    > bool:
69     """Добавление модели в list неподдерживаемых с сохранением
        в конфигурации.
70
71     Args:
72         model_name (str): Наименование блокируемой модели.
73         reason (str): Причина исключения модели. Значение по у
        молчанию: ''.
74
75     Returns:
76         bool: True при успешном сохранении, False при ошибке.
77
78     Examples:
79         >>> add_unsupported_model('gemini-legacy', reason='404
        Not Found')
80         True
81         """
82     if not model_name:
83         return False
84     return _mgr_add_unsupported_model('gemini', model_name, re
        ason)
85
86
87 @dataclass
88 class GoogleGenerativeAICore:
89     """Base class взаимодействия с моделями Google Generative
        AI (Gemini).
90
91     Attributes:
92         api_key (str): Активный API-ключ для запросов.
93         model_name (str): Наименование используемой модели Gem
        ini.
94         generation_config (dict): Parameters генерации по умол
        чанию.
95         system_instruction (str): Базовая системная инструкция
        .

```



```

96 |         api_key_names (list[str]): List разрешенных имен ключе
    | й.
97 |         save_history_chat (bool): Флаг сохранения контекста ис
    | тории чата.
98 |         sleep_on_exhausted (bool): Флаг ожидания разблокировки
    | при исчерпании квоты.
99 |         """
100 |
101 |     api_key: str = ''
102 |     model_name: str = field(default_factory=lambda: _DEFAULT_M
    | ODEL)
103 |     generation_config: dict = field(default_factory=lambda: {'
    | response_mime_type': 'text/plain'})
104 |     system_instruction: str = ''
105 |     api_key_names: list[str] = field(default_factory=list)
106 |     api_keys: list[str] = field(default_factory=list, init=Fal
    | se)
107 |     api_key_owners: list[str] = field(default_factory=list, in
    | it=False)
108 |     _key_names_active: list[str] = field(default_factory=list,
    | init=False)
109 |     _client: Any = field(default=False, init=False)
110 |     _api_key_index: int = field(default=0, init=False)
111 |     _all_keys_exhausted: bool = field(default=False, init=Fals
    | e)
112 |     _unavailable_attempts: int = field(default=0, init=False)
113 |     save_history_chat: bool = field(default_factory=lambda: _D
    | EFAULT_SAVE_HISTORY)
114 |     sleep_on_exhausted: bool = True
115 |
116 |     _last_exception: str = field(default='', init=False)
117 |     _key_errors: dict[str, str] = field(default_factory=dict,
    | init=False)
118 |
119 |     MODELS: list[str] = field(default_factory=lambda: GoogleGe
    | nerativeAICore.get_available_models(), init=False)
120 |
121 |     @classmethod

```

```

122     def get_available_models(cls, api_key: str = '', force_ref
123     resh: bool = False) -> list[str]:
124
125         """Получение динамического списка доступных моделей че
126     рез Google GenAI SDK.
127
128         Args:
129             api_key (str): Опциональный API-ключ. Значение по
130     умолчанию: ''.
131             force_refresh (bool): Принудительное update кэша.
132     Значение по умолчанию: False.
133
134         Returns:
135             list[str]: List доступных моделей.
136
137         Examples:
138             >>> models = GoogleGenerativeAICore.get_available_
139     models()
140             >>> isinstance(models, list)
141             True
142             """
143         return _mgr_get_available_models(provider='gemini', ap
144     i_key=api_key, force_refresh=force_refresh)
145
146     def __post_init__(self) -> None:
147         """Initialization объекта подключения к Google Generat
148     ive AI."""
149         self._last_exception = ''
150         self._key_errors = {}
151         self._unavailable_attempts = 0
152
153         self.api_keys, self._key_names_active, _ = load_api_ke
154     ys(self.api_key_names)
155         self.api_key_owners = list(self._key_names_active)
156
157         if not self.api_keys:
158             logger.warning('GoogleGenerativeAI: Нет доступных
159     API-ключей Gemini.')
160             self._all_keys_exhausted = True

```

```

151         return
152
153     get_status(self.api_key_names)
154     self.api_key = self.api_keys[0]
155     logger.info(f'GoogleGenerativeAI: Initialization с ключом: {self._key_names_active[0]}')
156     self._client = genai.Client(api_key=self.api_key)
157
158     def _get_exhausted_error_msg(self) -> str:
159         """Формация сообщения об исчерпании всех доступных API-ключей.
160
161         Returns:
162             str: Текст сообщения об ошибке с диагностической информацией.
163         """
164         msg: str = 'Error: Все API ключи исчерпаны.'
165         mode: str = getattr(server_cfg, 'mode', 'DEV').upper()
166         if mode == 'DEV':
167             if self._key_errors:
168                 msg += '\n[DEV Детали по ключам]:'
169                 for kname, kerr in self._key_errors.items():
170                     msg += f'\n- {kname}: {kerr}'
171             elif self._last_exception:
172                 msg += f'\n[DEV Детали]: {self._last_exception}'
173         return msg
174
175     def _record_error(self, ex: Exception | str) -> None:
176         """Фиксация последней возникшей ошибки в диагностическом хранилище.
177
178         Args:
179             ex (Exception | str): Объект исключения или string ошибки.
180         """
181         ex_str: str = str(ex)
182         self._last_exception = ex_str

```

```

183         key_name: str = self._key_names_active[0] if self._key
|_names_active else '?'
184         self._key_errors[key_name] = ex_str
185
186     def _invalidate_api_key(self, key: str) -> None:
187         """Exception невалидного ключа из активного пула.
188
189         Args:
190             key (str): Значение недействительного API-ключа.
191         """
192         idx: int = self.api_keys.index(key) if key in self.api
|_keys else -1
193         key_name: str = self._key_names_active[idx] if 0 <= id
|x < len(self._key_names_active) else '?'
194         logger.warning(f'GoogleGenerativeAI: Недействительный
|ключ удален: {key_name}')
195         self.api_keys = [k for k in self.api_keys if k != key]
196         if idx >= 0:
197             self._key_names_active = [n for i, n in enumerate(
|self._key_names_active) if i != idx]
198             self.api_key_owners = [o for i, o in enumerate(sel
|f.api_key_owners) if i != idx]
199
200     def _mark_key_exhausted(self, key: str) -> None:
201         """Маркировка ключа как исчерпавшего суточную квоту.
202
203         Args:
204             key (str): Значение исчерпанного API-ключа.
205         """
206         idx: int = self.api_keys.index(key) if key in self.api
|_keys else -1
207         key_name: str = self._key_names_active[idx] if 0 <= id
|x < len(self._key_names_active) else key
208         mark_exhausted(key_name)
209         logger.warning(f'GoogleGenerativeAI: Суточная квота кл
|юча {key_name} исчерпана. Блокировка 24ч.')
210         self.api_keys = [k for k in self.api_keys if k != key]
211         if idx >= 0:

```

```

212         self._key_names_active = [n for i, n in enumerate(
self._key_names_active) if i != idx]
213
214     def _switch_api_key(self) -> bool:
215         """Переключение на следующий доступный API-ключ из пул
а.
216
217         Returns:
218             bool: True при успешном переключении, False при от
сутствии доступных ключей.
219         """
220         if not self.api_keys:
221             wait_sec: float = next_available_in()
222             if wait_sec > 0 and self.sleep_on_exhausted:
223                 h: int = int(wait_sec) // 3600
224                 m: int = (int(wait_sec) % 3600) // 60
225                 logger.warning(f'GoogleGenerativeAI: Все ключи
исчерпаны. Ожидание {h}ч {m}м...')
226                 time.sleep(wait_sec + 5)
227                 self.api_keys, self._key_names_active, _ = loa
d_api_keys(self.api_key_names)
228                 if not self.api_keys:
229                     self._all_keys_exhausted = True
230                     return False
231                 self._all_keys_exhausted = False
232             else:
233                 self._all_keys_exhausted = True
234                 logger.warning('GoogleGenerativeAI: Все API-кл
ючи исчерпаны.')
235                 return False
236
237         self._api_key_index = 0
238         self.api_key = self.api_keys[0]
239         key_name: str = self._key_names_active[0] if self._key
_names_active else '?'
240         logger.info(f'GoogleGenerativeAI: Переключение на ключ
: {key_name}')
241         self._client = genai.Client(api_key=self.api_key)

```

```

242         return True
243
244     def _switch_model(self) -> bool:
245         """Переключение на следующую поддерживаемую модель в п
уле.
246
247         Returns:
248             bool: True при успешном переключении, False если l
ist пуст.
249         """
250         active_pool: list[str] = self.get_available_models()
251         if not active_pool:
252             active_pool = [
253                 'gemini-flash-latest',
254                 'gemini-flash-lite-latest',
255                 'gemini-3.6-flash',
256                 'gemini-3.7-flash',
257                 'gemini-pro-latest',
258             ]
259         try:
260             idx: int = active_pool.index(self.model_name)
261             next_idx: int = (idx + 1) % len(active_pool)
262         except ValueError:
263             next_idx = 0
264
265         next_model: str = active_pool[next_idx]
266         if next_model == self.model_name and len(active_pool)
<= 1:
267             return False
268
269         logger.info(f'GoogleGenerativeAI: Смена модели {self.m
odel_name} -> {next_model}')
270         self.model_name = next_model
271
272         self.api_keys, self._key_names_active, _ = load_api_ke
ys(self.api_key_names)
273         if not self.api_keys:
274             return False

```

```

275 |
276 |         self._all_keys_exhausted = False
277 |         self.api_key = self.api_keys[0]
278 |         self._client = genai.Client(api_key=self.api_key)
279 |         return True
280 |
281 |     def _switch_model_down(self) -> bool:
282 |         """Переключение на менее ресурсоемкую модель (даунгрей
283 |         д).
284 |
285 |         Returns:
286 |             bool: True при успешном переключении, False если д
287 |             остигнут нижний предел.
288 |         """
289 |         active_pool: list[str] = self.get_available_models()
290 |         if not active_pool:
291 |             active_pool = [
292 |                 'gemini-flash-latest',
293 |                 'gemini-flash-lite-latest',
294 |                 'gemini-3.6-flash',
295 |                 'gemini-3.7-flash',
296 |                 'gemini-pro-latest',
297 |             ]
298 |         try:
299 |             idx: int = active_pool.index(self.model_name)
300 |         except ValueError:
301 |             idx = 0
302 |
303 |         next_idx: int = idx + 1
304 |         if next_idx >= len(active_pool):
305 |             return False
306 |
307 |         next_model: str = active_pool[next_idx]
308 |         logger.info(f'GoogleGenerativeAI: Понижение модели {se
309 |         lf.model_name} -> {next_model}')
310 |         self.model_name = next_model
311 |
312 |         self.api_keys, self._key_names_active, _ = load_api_ke

```

```
ys(self.api_key_names)
310 |         if not self.api_keys:
311 |             return False
312 |
313 |         self._all_keys_exhausted = False
314 |         self.api_key = self.api_keys[0]
315 |         self._client = genai.Client(api_key=self.api_key)
316 |         return True
317 |
```


embeddings.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Google Generative AI Embeddings
4 # =====
5 # Description:
6 #   Embedding generation for Google Generative AI.
7 #   Provides methods for generating vector representations of
8 #   text.
9 #
10 # File: embeddings.py
11 # Project: ai-breadboard
12 # Package: src.ai.gemini
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 import asyncio
18
19 import numpy as np
20
21 from src.logger.logger import logger
22
23 from .core import GoogleGenerativeAICore
24
25 class GoogleGenerativeAIEmbeddingsMixin:
26     """Mixin class for embedding generation in GoogleGenerativ
27     eAI.
28
29     Provides methods for generating vector representations of
30     text.
31
32     """
```

```

31 |     async def embed(self, text: str, model_name: str = 'text-e
| mbedding-004') -> np.ndarray | bool:
32 |         """Generation of vector representation (embedding) for
| provided text.
33 |
34 |         Args:
35 |             text (str): Source text for vectorization.
36 |             model_name (str): Name of the embedding model.
37 |
38 |         Returns:
39 |             np.ndarray | bool: One-dimensional embedding array
| or False on failure.
40 |
41 |         Examples:
42 |             >>> ai = GoogleGenerativeAI()
43 |             >>> vec = await ai.embed("Тестовый текст")
44 |         """
45 |         if not text:
46 |             return False
47 |         try:
48 |             response = self._client.models.embed_content(
49 |                 model=model_name,
50 |                 contents=text,
51 |             )
52 |             if response and response.embeddings:
53 |                 return np.array(response.embeddings[0].values)
54 |             return False
55 |         except Exception as ex:
56 |             logger.error('GoogleGenerativeAI: Error генерации
| эмбединга', ex)
57 |             return False
58 |

```

errors.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Google Generative AI Error Handling
4 # =====
5 # Description:
6 #   Centralized error handling and retry logic for Google Gene
7 #   Handles API key rotation, model switching, and exponential
8 #   backoff.
9 #
10 # File: errors.py
11 # Project: ai-breadboard
12 # Package: src.ai.gemini
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 import asyncio
18 import re
19 import time
20
21 import requests
22
23 from src.logger.logger import logger
24
25 from .core import add_unsupported_model
26
27 class GoogleGenerativeAIErrorMixin:
28     """Mixin class for centralized error handling in GoogleGen
29     erativeAI.
30
31     Provides methods for handling API errors, switching API ke
```

ys, and model rotation.

```
31 |     """
32 |
33 |     async def _handle_api_error(
34 |         self,
35 |         ex: Exception,
36 |         active_model: str,
37 |         attempt: int,
38 |         max_attempts: int,
39 |     ) -> bool:
40 |         """Centralized handling of API exceptions and retry co
41 | ordination.
42 |
43 |         Args:
44 |             ex (Exception): Raised exception.
45 |             active_model (str): Name of model being used.
46 |             attempt (int): Current attempt number.
47 |             max_attempts (int): Maximum number of attempts.
48 |
49 |         Returns:
50 |             bool: True if retry needed, False if error is unre
51 | coverable.
52 |         """
53 |         self._record_error(ex)
54 |         ex_str: str = str(ex)
55 |         logger.error(f'GoogleGenerativeAI: API Error (attempt
56 | {attempt + 1}/{max_attempts}): {ex_str}')
57 |
58 |         # 1. Authorization error (invalid API key)
59 |         if '401' in ex_str or 'API_KEY_INVALID' in ex_str or '
60 | PERMISSION_DENIED' in ex_str:
61 |             self._invalidate_api_key(self.api_key)
62 |             return self._switch_api_key()
63 |
64 |         # 2. Model not found / outdated (404)
65 |         if any(
66 |             k in ex_str
67 |             for k in [
```

```

64         '404',
65         'NOT_FOUND',
66         'is no longer available',
67         'not found for API version',
68         'not supported for generateContent',
69     ]
70 ):
71     add_unsupported_model(active_model, reason=ex_str)
72     return self._switch_model()
73
74     # 3. Service temporarily unavailable (503 UNAVAILABLE)
75     if '503' in ex_str or 'UNAVAILABLE' in ex_str:
76         self._unavailable_attempts += 1
77         if self._unavailable_attempts < 6:
78             wait: int = 2 ** min(self._unavailable_attempt
s, 5)
79             logger.info(f'GoogleGenerativeAI: 503 UNAVAILA
BLE. Waiting {wait}s...')
80             await asyncio.sleep(wait)
81             return True
82         else:
83             switched: bool = self._switch_model_down()
84             self._unavailable_attempts = 0
85             return switched
86
87     # 4. Request quota exceeded (429 RESOURCE_EXHAUSTED)
88     if '429' in ex_str or 'RESOURCE_EXHAUSTED' in ex_str:
89         is_daily: bool = any(
90             k in ex_str.lower()
91             for k in ['perday', 'per_day', 'exceeded your
current quota', "quota_limit_value": '0']
92         )
93         if is_daily:
94             self._mark_key_exhausted(self.api_key)
95             if self._switch_api_key():
96                 return True
97             return self._switch_model()
98

```

```

99         m = re.search(r'retry\D*(\d+(?:\.\d+)?)s', ex_str,
    re.IGNORECASE)
100         base_wait: int = int(float(m.group(1))) + 2 if m else 5
101         wait_time: int = min(base_wait * (2 ** min(attempt
    , 3)), 60)
102         logger.info(f'GoogleGenerativeAI: 429 Rate Limit.
    Waiting {wait_time}s before retry...')
103         await asyncio.sleep(wait_time)
104         return True
105
106     # 5. Network request errors
107     if isinstance(ex, requests.exceptions.RequestException
    ):
108         if attempt < 5:
109             logger.warning('GoogleGenerativeAI: Network Er
    ror. Waiting 10s...')
110             await asyncio.sleep(10)
111             return True
112         return False
113
114     # 6. General unexpected errors
115     if attempt < max_attempts - 1:
116         await asyncio.sleep(2 ** min(attempt, 4))
117         return True
118
119     return False
120

```

generative_ai.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Integration with Google Generative AI (Gemini)
  models
4 # =====
5 # Description:
6 #   Manages interaction with Google Generative AI API through
  official SDK.
7 #   Implements API key pool management, model rotation on fail
  ures, stream response generation,
8 #   tool support, and media file handling with comprehensive e
  rror recovery mechanisms.
9 #
10 # File: generative_ai.py
11 # Project: ai-breadboard
12 # Package: src.ai.gemini
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 from .api import GoogleGenerativeAI
18 from .core import _DEFAULT_MODEL
19
20 __all__ = ['GoogleGenerativeAI', '_DEFAULT_MODEL']
21
```

header.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Gemini AI model interface integration
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: header.py
9 # Project: ai-breadboard
10 # Package: src.ai.gemini
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """Gemini AI model interface integration module.
16
17 Provides interface for interacting with Google Generative AI (
18 Gemini) model via generativeai library."""
19
20 import sys
21 from packaging.version import Version
22
23 from pathlib import Path
24
25 def set_project_root(marker_files=('__root__', '.git')) -> Path
26 :
27     """
28     Finds the root directory of the project starting from the
29     current file's directory,
30     searching upwards and stopping at the first directory containing any of the marker files.
31
32     Args:
33         marker_files (tuple): Filenames or directory names to
```



```

    identify the project root.
30
31     Returns:
32         Path: Path to the root directory if found, otherwise t
the directory where the script is located.
33     """
34     __root__:Path
35     current_path:Path = Path(__file__).resolve().parent
36     __root__ = current_path
37     for parent in [current_path] + list(current_path.parents):
38         if any((parent / marker).exists() for marker in marker
__files):
39             __root__ = parent
40             break
41     if __root__ not in sys.path:
42         sys.path.insert(0, str(__root__))
43     return __root__
44
45 # Get the root directory of the project
46 __root__: Path = set_project_root()
47 """__root__ (Path): Path to the root directory of the project"
""
48
49 try:
50     from src import gs
51 except ImportError:
52     gs = False
53
54 config: dict = {}
55
56 __project_name__ = 'hypotez'
57 __version__: str = ''
58 __doc__: str = ''
59 __details__: str = ''
60 __author__: str = ''
61 __copyright__: str = ''
62 __cofee__: str = "Treat the developer to a cup of coffee for b
oosting enthusiasm in development: https://boosty.to/hypo69"

```


history.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Google Generative AI Chat History Management
4 # =====
5 # Description:
6 #   Management of chat history and session state for Google Ge
7 #   Provides methods for restoring, clearing, and managing cha
8 #   t history.
9 #
10 # File: history.py
11 # Project: ai-breadboard
12 # Package: src.ai.gemini
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 from typing import Any
18
19 from google.genai import types
20
21 from .config import GoogleGenerativeAIConfigMixin
22
23 class GoogleGenerativeAIHistoryMixin:
24     """Mixin class for chat history management in Google Genera
25     tiveAI.
26
27     Provides methods for managing chat history and session sta
28     te.
29
30     """
31
32     def __init__(self) -> None:
```

```

30 |         """Initialization chat history attributes."""
31 |         self.chat_history: list[dict] = []
32 |         self._chat: Any = False
33 |
34 |     def clear_history(self) -> None:
35 |         """Clearing of local dialog history in operational mem
36 |         ory."""
37 |         self.chat_history = []
38 |
39 |     def _restore_chat_from_history(self) -> None:
40 |         """Restoration of chat session state from accumulated
41 |         message history."""
42 |         history_contents: list[types.Content] = []
43 |         for entry in self.chat_history:
44 |             role: str = entry.get('role', 'user')
45 |             if role == 'assistant':
46 |                 role = 'model'
47 |             parts: list = entry.get('parts', [])
48 |             parts_objects: list[types.Part] = []
49 |             for p in parts:
50 |                 if isinstance(p, str):
51 |                     parts_objects.append(types.Part.from_text(
52 | text=p))
53 |                 elif isinstance(p, dict) and 'text' in p:
54 |                     parts_objects.append(types.Part.from_text(
55 | text=p['text']))
56 |             history_contents.append(types.Content(role=role, p
57 | arts=parts_objects))
58 |
59 |         self._chat = self._start_chat(history=history_contents
60 | )
61 |
62 |     def _prepare_contents(self, q: str, history: list[dict] =
63 | ()) -> list[types.Content]:
64 |         """Подготовка списка объектов Content для передачи в s
65 | tateless API-запросы.
66 |
67 |         Args:

```

```

60 |         q (str): Текущий текстовый запрос пользователя.
61 |         history (list[dict]): История предыдущих сообщений
62 |     .
63 |
64 |     Returns:
65 |         list[types.Content]: List объектов Content.
66 |     """
67 |     contents: list[types.Content] = []
68 |     if history:
69 |         for entry in history:
70 |             role: str = entry.get('role', '')
71 |             if not role:
72 |                 continue
73 |             if role == 'assistant':
74 |                 role = 'model'
75 |
76 |             parts = entry.get('parts')
77 |             if not parts:
78 |                 content_str: str = entry.get('content', '')
79 |             )
80 |                 if content_str:
81 |                     parts = [types.Part.from_text(text=con
82 | tent_str)]
83 |             else:
84 |                 new_parts: list = []
85 |                 for p in parts:
86 |                     if isinstance(p, str):
87 |                         new_parts.append(types.Part.from_t
88 | ext(text=p))
89 |                     elif isinstance(p, dict) and 'text' in
90 | p:
91 |                         new_parts.append(types.Part.from_t
92 | ext(text=p['text']))
93 |                     else:
94 |                         new_parts.append(p)
95 |                 parts = new_parts
96 |
97 |     if parts:

```

```
92         contents.append(types.Content(role=role, p
| arts=parts))
93
94         contents.append(types.Content(role='user', parts=[type
| s.Part.from_text(text=q)]))
95         return contents
96
97     def _start_chat(self, history: list = ()) -> Any:
98         """Initialization сессии чата с поддержкой сохранения
| истории.
99
100         Args:
101             history (list): Начальная история сообщений.
102
103         Returns:
104             Any: Экземпляр чата Google GenAI или False если ис
| тория отключена.
105         """
106         if not self.save_history_chat:
107             return False
108
109         config = self._build_content_config()
110         if history:
111             return self._client.chats.create(model=self.model_
| name, config=config, history=list(history))
112         return self._client.chats.create(model=self.model_name
| , config=config)
113
```

images.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Google Generative AI Image Operations
4 # =====
5 # Description:
6 #   Image-related operations for Google Generative AI.
7 #   Provides methods for image description and file upload.
8 #
9 # File: images.py
10 # Project: ai-breadboard
11 # Package: src.ai.gemini
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import asyncio
17 from io import IOBase
18 from pathlib import Path
19 from typing import Any
20
21 from google.genai import types
22
23 from src.logger.logger import logger
24 from src.utils.image import get_image_bytes
25
26 from .core import GoogleGenerativeAICore
27 from .errors import GoogleGenerativeAIErrorMixin
28
29
30 class GoogleGenerativeAIImagesMixin:
31     """Mixin class for image operations in GoogleGenerativeAI.
32
33     Provides methods for image description and file upload.
```

```

34 |     """
35 |
36 |     async def describe_image(
37 |         self,
38 |         image: Path | bytes,
39 |         mime_type: str = 'image/jpeg',
40 |         prompt: str = '',
41 |         attempts: int = 10,
42 |     ) -> str | bool:
43 |         """Formation текстового описания переданного изображен
    | ия.
44 |
45 |         Args:
46 |             image (Path | bytes): Путь к изображению или его б
    | инарное содержимое.
47 |             mime_type (str): MIME-тип изображения. Значение по
    | умолчанию: 'image/jpeg'.
48 |             prompt (str): Дополнительный текстовый промпт. Зна
    | чение по умолчанию: ''.
49 |             attempts (int): Максимальное число попыток. Значен
    | ие по умолчанию: 10.
50 |
51 |         Returns:
52 |             str | bool: Текстовое описание или False при сбое.
53 |
54 |         Examples:
55 |             >>> ai = GoogleGenerativeAI()
56 |             >>> desc = await ai.describe_image(Path("poster.jp
    | g"))
57 |         """
58 |         img_bytes: bytes = get_image_bytes(image) if isinstance
    | e(image, Path) else image
59 |         if not img_bytes:
60 |             return False
61 |
62 |         for attempt in range(attempts):
63 |             try:
64 |                 response = self._client.models.generate_content

```



```

t(
65         model=self.model_name,
66         contents=[
67             types.Part.from_bytes(data=img_bytes,
mime_type=mime_type),
68             types.Part.from_text(text=prompt or '0
пиши это изображение.'),
69         ],
70     )
71     if response and response.text:
72         return response.text
73
74     logger.debug(f'GoogleGenerativeAI: Empty ответ
describe_image (попытка {attempt + 1})')
75     await asyncio.sleep(2 ** min(attempt, 4))
76     except Exception as ex:
77         should_retry: bool = await self._handle_api_er
ror(ex, self.model_name, attempt, attempts)
78         if not should_retry:
79             return False
80
81     return False
82
83     async def upload_file(
84         self,
85         file: str | Path | IOBase,
86         file_name: str = '',
87         attempts: int = 10,
88     ) -> bool:
89         """Loading медиа-файла в хранилище Google GenAI File A
PI.
90
91         Args:
92             file (str | Path | IOBase): Путь к файлу или файло
вый дескриптор.
93             file_name (str): Отображаемое имя файла. Значение
по умолчанию: ''.
94             attempts (int): Максимальное количество попыток. 3

```

начение по умолчанию: 10.

95

96 Returns:

97 bool: True при успешной загрузке, False при ошибке

.

98

99 Examples:

100 >>> ai = GoogleGenerativeAI()

101 >>> success = await ai.upload_file(Path("data.pdf"
) , file_name="data.pdf")

102 """

103 for attempt in range(attempts):

104 try:

105 upload_kwargs = (

106 {'config': types.UploadFileConfig(display_
name=file_name)})

107 if file_name

108 else {}

109)

110 response = self._client.files.upload(path=file
, **upload_kwargs)

111 if response:

112 logger.debug(f'GoogleGenerativeAI: Файл {f
ile_name} successfully загружен')

113 return True

114 return False

115 except Exception as ex:

116 should_retry: bool = await self._handle_api_er
ror(ex, self.model_name, attempt, attempts)

117 if not should_retry:

118 return False

119

120 return False

121

models.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Google Generative AI Models Management
4 # =====
5 # Description:
6 #   Model management for Google Generative AI.
7 #   Provides methods for loading, adding, and managing unsupported models.
8 #
9 # File: models.py
10 # Project: ai-breadboard
11 # Package: src.ai.gemini
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 from .core import load_unsupported_models, add_unsupported_model, GoogleGenerativeAICore
17
18 __all__ = ['load_unsupported_models', 'add_unsupported_model',
19            'GoogleGenerativeAICore']
```

rag.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: RAG index using Gemini Embedding API and FAISS
4 # =====
5 # Description:
6 #   Document vectorization via Gemini text-embedding-004 / gem
7 #   with FAISS-based semantic search for retrieval-augmented g
8 #   eneration.
9 #
10 # File: rag.py
11 # Project: ai-breadboard
12 # Package: src.ai.gemini
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 """RAG index using Gemini Embedding API and FAISS.
18 Provides document vectorization, storage, and semantic search
19 capabilities."""
20
21 import json
22 from pathlib import Path
23 from typing import List, Dict
24 import numpy as np
25
26 try:
27     import faiss
28     _FAISS_AVAILABLE = True
29 except ImportError:
30     _FAISS_AVAILABLE = False
31     faiss = False
```

```

31
32 from google import genai
33 from google.genai import types
34
35 from src.logger import logger
36
37 _EMBED_MODEL = 'models/gemini-embedding-2'
38 _EMBED_DIM = 3072
39
40 class GeminiRAG:
41     """RAG index using Gemini Embedding API and FAISS.
42
43     Provides document vectorization, vector storage in FAISS i
44     ndex,
45     and semantic search of nearest neighbors.
46
47     Attributes:
48         db_path (Path): Path to index file (replaced .db with
49         .faiss/.json).
50         client: google.genai client instance.
51     """
52
53     def __init__(self, api_key: str = '', db_path: Path = Path
54     ('.')) -> None:
55         """Initialize RAG index.
56
57         Args:
58             api_key (str): Gemini API key.
59             db_path (Path): Path to database directory.
60         """
61         self.db_path = db_path
62         self.index_file = db_path.with_suffix('.faiss')
63         self.meta_file = db_path.with_suffix('.json')
64         self.dimension = _EMBED_DIM
65         self.api_key = api_key
66         self.client = False
67         if api_key:
68             try:

```

```

66         self.client = genai.Client(api_key=api_key)
67     except Exception as e:
68         logger.error(f"[GeminiRAG] Error инициализации
genai.Client: {e}")
69
70     self.index_file.parent.mkdir(parents=True, exist_ok=True)
71
72     self.metadata: List[Dict] = []
73     self._load()
74
75     def _load(self) -> None:
76         """Loading индекса и метаданных."""
77         if self.meta_file.exists():
78             try:
79                 with open(self.meta_file, 'r', encoding='utf-8
') as f:
80                     self.metadata = json.load(f)
81             except Exception as e:
82                 logger.error("Error чтения метаданных пользова
теля", e)
83                 self.metadata = []
84             else:
85                 self.metadata = []
86
87         if not _FAISS_AVAILABLE:
88             self.index = False
89             return
90
91         if self.index_file.exists() and self.metadata:
92             try:
93                 self.index = faiss.read_index(str(self.index_f
ile))
94             except Exception as e:
95                 logger.error("Error чтения FAISS индекса, пере
создаем", e)
96                 self._rebuild_index()
97             else:
98                 self.index = faiss.IndexFlatL2(self.dimension)

```

```

98 |
99 |     def _rebuild_index(self) -> None:
100 |         """Перестроение индекса из metadatas с валидацией вект
    | оров."""
101 |         if not _FAISS_AVAILABLE:
102 |             self.index = False
103 |             return
104 |
105 |         self.index = faiss.IndexFlatL2(self.dimension)
106 |         if not self.metadatas:
107 |             return
108 |
109 |         valid_items = [
110 |             m for m in self.metadatas
111 |             if isinstance(m, dict) and 'vector' in m and isins
    | tance(m['vector'], (list, np.ndarray)) and len(m['vector']) ==
    | self.dimension
112 |         ]
113 |
114 |         if len(valid_items) != len(self.metadatas):
115 |             logger.warning(f"[GeminiRAG] Отфильтровано {len(se
    | lf.metadatas) - len(valid_items)} записей без корректного поля
    | 'vector'")
116 |             self.metadatas = valid_items
117 |             self._save()
118 |
119 |         if self.metadatas:
120 |             try:
121 |                 vectors = np.array([m['vector'] for m in self.
    | metadatas], dtype=np.float32)
122 |                 self.index.add(vectors)
123 |             except Exception as e:
124 |                 logger.error("Error при добавлении векторов в
    | FAISS индекс", e)
125 |
126 |     def _save(self) -> None:
127 |         """Сохранение индекса и метаданных."""
128 |         try:

```

```

129         if _FAISS_AVAILABLE and self.index:
130             faiss.write_index(self.index, str(self.index_file))
131         with open(self.meta_file, 'w', encoding='utf-8') as f:
132             json.dump(self.metadata, f, ensure_ascii=False, indent=2)
133         except Exception as e:
134             logger.error("Error сохранения FAISS индекса/метаданных", e)
135
136     def _embed(self, texts: List[str], task_type: str = 'RETRIEVAL_DOCUMENT') -> List[List[float]]:
137         """Получение эмбедингов для списка текстов через Gemini API с обработкой 429 и ротацией ключей."""
138         import time
139         from google.genai.errors import APIError
140         from src.secrets.api_key_state import load_api_keys, mark_exhausted
141         import random
142
143         start_idx = random.randint(0, 100)
144         max_retries = 10
145
146         for attempt in range(max_retries):
147             api_keys, key_names, _ = load_api_keys(skip_exhausted=False)
148             if not api_keys:
149                 raise RuntimeError("Нет доступных ключей API Gemini")
150
151             key_idx = (start_idx + attempt) % len(api_keys)
152             current_key = api_keys[key_idx]
153             current_name = key_names[key_idx]
154
155             client = genai.Client(api_key=current_key)
156             try:
157                 response = client.models.embed_content(

```



```

158         model=_EMBED_MODEL,
159         contents=[types.Content(parts=[types.Part.
from_text(text=t)]) for t in texts],
160         config=types.EmbedContentConfig(task_type=
task_type),
161     )
162     return [e.values for e in response.embeddings]
163 except Exception as e:
164     err_str = str(e)
165     if "429" in err_str or "RESOURCE_EXHAUSTED" in
err_str:
166         delay = 5.0 + attempt * 2
167         logger.warning(f"Превышен лимит (429) для
ключа {current_name}. Переключаемся и ждем {delay} сек...")
168         time.sleep(delay)
169         continue
170     elif "400" in err_str or "INVALID_ARGUMENT" in
err_str or "API_KEY_INVALID" in err_str:
171         logger.warning(f"Неверный ключ API (400) д
ля {current_name}. Пропускаем...")
172         mark_exhausted(current_name)
173         continue
174     raise e
175
176     raise RuntimeError("Превышено количество попыток запро
са эмбедингов из-за ограничений скорости (429)")
177
178     def add_documents(self, docs: List[dict], batch_size: int
= 50) -> int:
179         """Векторизация и сохранение документов в индекс."""
180         if not docs:
181             return 0
182
183         for i in range(0, len(docs), batch_size):
184             batch = docs[i:i + batch_size]
185             texts = [d['text'] for d in batch]
186             try:
187                 vectors = self._embed(texts, task_type='RETRIE

```

```

VAL_DOCUMENT')
188         except Exception as e:
189             logger.error(f"Error получения эмбедингов для
батча: {e}")
190             raise e
191
192         # Заменяем старые документы с совпадающими ID
193         existing_ids = {doc['id'] for doc in batch}
194         self.metadatas = [m for m in self.metadatas if m['
id'] not in existing_ids]
195
196         for doc, vec in zip(batch, vectors):
197             self.metadatas.append({
198                 'id': doc['id'],
199                 'text': doc['text'],
200                 'meta': doc.get('meta', {}),
201                 'vector': vec
202             })
203
204         self._rebuild_index()
205         self._save()
206         import time
207         time.sleep(1.0)
208         return len(docs)
209
210     def delete_document(self, doc_id: str) -> None:
211         """Удаление документа из индекса по id."""
212         self.metadatas = [m for m in self.metadatas if m['id']
!= doc_id]
213         self._rebuild_index()
214         self._save()
215
216     def clear(self) -> None:
217         """Полная очистка индекса."""
218         self.metadatas = []
219         self.index = faiss.IndexFlatL2(self.dimension) if _FAI
SS_AVAILABLE else False
220         self._save()

```

```

221
222     def search(self, query: str, top_k: int = 5, threshold: fl
223         oat = 0.0) -> List[dict]:
224         """Семантический поиск ближайших документов по cosine
225         similarity."""
226         valid_metadatas = [
227             m for m in self.metadatas
228             if isinstance(m, dict) and 'vector' in m and isins
229             tance(m['vector'], (list, np.ndarray)) and len(m['vector']) ==
230             self.dimension
231         ]
232         if not valid_metadatas:
233             return []
234
235         try:
236             query_vec = np.array(self._embed([query], task_typ
237             e='RETRIEVAL_QUERY')[0], dtype=np.float32)
238         except Exception as e:
239             logger.error("Error векторизации поискового запрос
240             a", e)
241             return []
242
243         query_norm = query_vec / (np.linalg.norm(query_vec) +
244         1e-10)
245
246         doc_vectors = np.array([m['vector'] for m in valid_met
247         adatas], dtype=np.float32)
248         doc_norms = np.linalg.norm(doc_vectors, axis=1, keepdi
249         ms=True) + 1e-10
250         doc_vectors_normalized = doc_vectors / doc_norms
251         actual_k = min(top_k, len(valid_metadatas))
252
253         if not _FAISS_AVAILABLE:
254             similarities = np.dot(doc_vectors_normalized, quer
255             y_norm)
256             top_indices = np.argsort(similarities)[::-1][:actu
257             al_k]
258             results: List[dict] = []

```

```

248         for idx in top_indices:
249             sim = float(similarities[idx])
250             if sim >= threshold:
251                 meta = valid_metadatas[idx]
252                 results.append({
253                     'id': meta['id'],
254                     'text': meta['text'],
255                     'meta': meta.get('meta', {}),
256                     'score': round(sim, 4)
257                 })
258             return results
259
260         temp_index = faiss.IndexFlatL2(self.dimension)
261         temp_index.add(doc_vectors_normalized)
262
263         distances, indices = temp_index.search(query_norm.reshape(1, -1), actual_k)
264
265         results: List[dict] = []
266         for i in range(actual_k):
267             idx = int(indices[0][i])
268             if idx >= 0 and idx < len(valid_metadatas):
269                 l2_dist = float(distances[0][i])
270                 cosine_sim = 1.0 - (l2_dist / 2.0)
271
272                 if cosine_sim >= threshold:
273                     meta = valid_metadatas[idx]
274                     results.append({
275                         'id': meta['id'],
276                         'text': meta['text'],
277                         'meta': meta.get('meta', {}),
278                         'score': round(cosine_sim, 4)
279                     })
280         return results
281
282     def count(self) -> int:
283         """Количество документов в индексе."""
284         return len(self.metadatas)

```


secrets_loader.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Loading Gemini API keys from secrets.json file
4 # =====
5 # Description:
6 #   Loads and manages Google Gemini API authentication keys from local secrets.json.
7 #   Provides functions to retrieve all keys, get keys by name, and load active keys
8 #   with proper filtering by status and quota restrictions.
9 #
10 # File: secrets_loader.py
11 # Project: ai-breadboard
12 # Package: src.ai.gemini
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 import json
18 from pathlib import Path
19 from typing import Dict, List, Tuple
20
21 # Path to secrets.json file relative to module directory
22 _SECRETS_FILE = Path(__file__).parent / 'secrets.json'
23
24 def load_secrets() -> Dict[str, str]:
25     """
26     Loading API keys from secrets.json file.
27
28     Returns:
29         Dict[str, str]: Dictionary mapping email addresses to API keys.
30     """
```

```

31 |     if not _SECRETS_FILE.exists():
32 |         return {}
33 |
34 |     try:
35 |         content = _SECRETS_FILE.read_text(encoding='utf-8')
36 |         return json.loads(content)
37 |     except Exception as ex:
38 |         print(f"Error loading secrets.json: {ex}")
39 |         return {}
40 |
41 | def get_all_keys() -> List[str]:
42 |     """
43 |     Returns list of all API keys.
44 |
45 |     Returns:
46 |         List[str]: List of API key values.
47 |     """
48 |     secrets = load_secrets()
49 |     return list(secrets.values())
50 |
51 | def get_all_key_names() -> List[str]:
52 |     """
53 |     Returns list of all key names (email addresses).
54 |
55 |     Returns:
56 |         List[str]: List of key identifiers (email addresses).
57 |     """
58 |     secrets = load_secrets()
59 |     return list(secrets.keys())
60 |
61 | def get_key_by_name(name: str) -> str | None:
62 |     """
63 |     Returns API key by name.
64 |
65 |     Args:
66 |         name: Key identifier (email address).
67 |
68 |     Returns:

```

```

69 |         str | None: API key value or None if not found.
70 |     """
71 |     secrets = load_secrets()
72 |     return secrets.get(name)
73 |
74 | def load_api_keys(names: List[str] = []) -> Tuple[List[str], L
    ist[str], List[str]]:
75 |     """
76 |     Loading API keys sorted by last_run timestamp.
77 |     Filters keys: only active status and not banned (daily quo
    ta).
78 |
79 |     Args:
80 |         names: Optional list of key names; if empty, loads all
    from file.
81 |
82 |     Returns:
83 |         Tuple[List[str], List[str], List[str]]: (api_keys, key
    _names, key_names)
84 |     """
85 |     from src.secrets.api_key_state import load_api_keys as sta
    te_load_api_keys
86 |
87 |     # Using existing function with names from secrets.json
88 |     all_names = get_all_key_names()
89 |
90 |     if names:
91 |         # Filter only names that exist in secrets.json
92 |         names = [n for n in names if n in all_names]
93 |
94 |     return state_load_api_keys(names)
95 |

```


user_query_rag.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Return path to user-specific RAG database
4 # =====
5 # Description:
6 #   Each user query (question + model response) is indexed.
7 #   Provides semantic search over user's personal query history.
8 #
9 # File: user_query_rag.py
10 # Project: ai-breadboard
11 # Package: src.ai.gemini
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 from __future__ import annotations
17
18 import hashlib
19 import time
20 import re
21 from pathlib import Path
22 from typing import Optional
23
24 from src.ai.gemini.rag import GeminiRAG
25 from src.logger import logger
26
27 # Directory for storing user RAG databases
28 _USER_RAGS_DIR = Path(__file__).parent / "user_rags"
29 _USER_RAGS_DIR.mkdir(exist_ok=True)
30
31 # Maximum documents per database (old ones are evicted)
32 _MAX_DOCS_PER_USER = 500
```

```

33
34 # Minimum query length for indexing (exclude "yes", "no", etc)
35 _MIN_QUERY_LEN = 10
36
37 def _get_user_rag_path(user_id) -> Path:
38     """Return path to RAG database for specific user."""
39     safe_id = str(user_id).replace(".", "_").replace("/", "_")
40     .replace("\\", "_")
41     return _USER_RAGS_DIR / f"user_rag_{safe_id}.db"
42
43 def _make_doc_id(user_id, query: str) -> str:
44     """Generate stable unique document ID (deduplication by qu
45     ery)."""
46     key = f"{user_id}_{query.strip().lower()}"
47     return f"{user_id}_{hashlib.md5(key.encode('utf-8')).hexdigest()}"
48
49 def get_user_rag(user_id, api_key: str) -> GeminiRAG:
50     """Return RAG index for specific user.
51
52     Creates database if it doesn't exist. Does not rebuild existing one.
53
54     Args:
55         user_id: User ID (int from DB or string "anon_<IP>").
56         api_key: Gemini API key for embeddings.
57
58     Returns:
59         GeminiRAG: Instance of user RAG index.
60     """
61     db_path = _get_user_rag_path(user_id)
62     return GeminiRAG(api_key=api_key, db_path=db_path)
63
64 def is_garbage_query(query: str) -> bool:
65     """Determine if query is garbage (contains no useful semantic content)."""
66     q = query.strip().lower()

```

```

66 |     # 1. Too short queries
67 |     if len(q) < _MIN_QUERY_LEN:
68 |         return True
69 |
70 |     # 2. Repeating characters (e.g., "aaaaaa")
71 |     if re.search(r'(\.)\1{4,}', q):
72 |         return True
73 |
74 |     # 3. Pure noise (only punctuation, special chars or spaces
    | )
75 |     if not re.search(r'[a-zA-Za-яA-Я0-9]', q):
76 |         return True
77 |
78 |     # 4. Simple conversational phrases / greetings / thanks /
    | filler words
79 |     garbage_words = {
80 |         'привет', 'здравствуй', 'здравствуйте', 'добрый', 'ден
    | ь', 'вечер', 'утро',
81 |         'пока', 'свидания', 'встречи', 'прощай', 'спасибо', 'б
    | лагодарю', 'пожалуйста',
82 |         'дела', 'жизнь', 'делаешь', 'ты', 'тут', 'эй', 'ау', '
    | тест', 'test', 'hello',
83 |         'hi', 'hey', 'good', 'morning', 'afternoon', 'evening'
    | , 'thank', 'you', 'thanks',
84 |         'please', 'bye', 'goodbye', 'how', 'are', 'whats', 'up
    | ', 'ok', 'ок', 'ладно', 'как',
85 |         'большое', 'не', 'за', 'что', 'да', 'нет', 'угу'
86 |     }
87 |     # Remove punctuation for word analysis
88 |     clean_q = re.sub(r'[\w\s]', '', q).strip()
89 |     words = clean_q.split()
90 |     if words and all(w in garbage_words for w in words):
91 |         return True
92 |
93 |     # 5. Keyboard mash (nonsensical character strings)
94 |     # If word longer than 6 chars has no vowels (Russian/Engli
    | sh)
95 |     for w in words:

```

[illegible]

```

127     Args:
128         user_id: User ID or string "anon_<IP>".
129         api_key: Gemini API key.
130         query: User query text.
131         response: Model response.
132
133     Returns:
134         bool: True if document added/updated, False if skipped
135     .
136     """
137     if not query or not response:
138         return False
139
140     # Filter garbage queries
141     if is_garbage_query(query):
142         logger.info(f"UserRAG [{user_id}]: query filtered as g
143         arbage: '{query}'")
144         return False
145
146     # Filter responses that are raw JSON, service logs or erro
147     rs
148     resp_stripped = response.strip()
149     if resp_stripped.startswith('{') and ('"title"' in resp_st
150     ripped or '"error"' in resp_stripped or '"results"' in resp_st
151     ripped):
152         logger.info(f"UserRAG [{user_id}]: response filtered a
153         s raw JSON: '{resp_stripped[:60]}...'")
154         return False
155
156     if any(resp_stripped.startswith(pfx) for pfx in ('␣ ', 'Err
157     or', 'ERROR', 'DEBUG', 'Traceback', '[DIRECT PLAY', '[DIRECT R
158     AG')):
159         logger.info(f"UserRAG [{user_id}]: response filtered a
160         s service/error message")
161         return False
162
163     try:
164         rag = get_user_rag(user_id, api_key)
165

```

```

156         # Prune on overflow - remove oldest records
157         _prune_if_needed(rag, user_id)
158
159         doc_id = _make_doc_id(user_id, query)
160         # Store ultra-compact summary (up to 150 chars) so future searches
161         # insert minimal tokens into model context
162         response_summary = _format_compact_summary(response, max_chars=150)
163         doc_text = f"User asked: {query.strip()}\nModel response: {response_summary}"
164
165         rag.add_documents([
166             "id": doc_id,
167             "text": doc_text,
168             "meta": {
169                 "user_id": str(user_id),
170                 "timestamp": time.time(),
171                 "q": query[:500],
172                 "response": response[:1000], # full text in meta, not in prompt
173                 "is_manual": False
174             },
175         ])
176         return True
177
178     except Exception as ex:
179         logger.error(f"Error indexing user query {user_id}", ex, False)
180         return False
181
182 def search_user_context(
183     user_id,
184     api_key: str,
185     query: str,
186     top_k: int = 3,
187     threshold: float = 0.4,
188 ) -> list:

```

```

189 |     """Semantic search over user's query history.
190 |
191 |     Args:
192 |         user_id: User ID.
193 |         api_key: Gemini API key.
194 |         query: Current query for finding similar context.
195 |         top_k: Number of results.
196 |         threshold: Minimum similarity threshold (0.0-1.0).
197 |
198 |     Returns:
199 |         list[dict]: List of {"id", "text", "meta", "score"} so
200 |         rted by score descending.
201 |     """
202 |     try:
203 |         rag = get_user_rag(user_id, api_key)
204 |         if rag.count() == 0:
205 |             return []
206 |         return rag.search(query, top_k=top_k, threshold=thresh
207 |         old)
208 |     except Exception as ex:
209 |         logger.error(f"Error searching user RAG {user_id}", ex
210 |         , False)
211 |         return []
212 |
213 | def get_user_rag_stats(user_id, api_key: str) -> dict:
214 |     """Return statistics of user's RAG index.
215 |
216 |     Args:
217 |         user_id: User ID.
218 |         api_key: Gemini API key.
219 |
220 |     Returns:
221 |         dict: Fields count, db_path, db_size_kb.
222 |     """
223 |     db_path = _get_user_rag_path(user_id)
224 |     try:
225 |         rag = get_user_rag(user_id, api_key)
226 |         count = rag.count()

```

```

224     except Exception:
225         count = 0
226
227     size_kb = round(db_path.stat().st_size / 1024, 1) if db_pa
th.exists() else 0
228     return {
229         "user_id": str(user_id),
230         "count": count,
231         "db_path": str(db_path),
232         "db_size_kb": size_kb,
233     }
234
235 def clear_user_rag(user_id, api_key: str) -> bool:
236     """Complete cleanup of user's personal RAG index.
237
238     Args:
239         user_id: User ID.
240         api_key: Gemini API key.
241
242     Returns:
243         bool: True on success.
244     """
245     try:
246         rag = get_user_rag(user_id, api_key)
247         rag.clear()
248         return True
249     except Exception as ex:
250         logger.error(f"Error clearing user RAG {user_id}", ex,
False)
251         return False
252
253 def clean_invalid_user_rag_entries(user_id, api_key: str = '')
-> int:
254     """Delete from User RAG entries containing raw JSON or err
or messages."""
255     try:
256         rag = get_user_rag(user_id, api_key)
257         initial_count = len(rag.metadatas)

```



```

258         valid_metas = []
259         for m in rag.metadatas:
260             text = m.get('text', '')
261             meta_resp = m.get('meta', {}).get('response', '')
262             check_str = (meta_resp or text).strip()
263             if check_str.startswith('{') and ('"title"' in che
ck_str or '"error"' in check_str):
264                 continue
265             if 'Model response: {' in text and '"title"' in te
xt:
266                 continue
267             if any(check_str.startswith(pfx) for pfx in (' ',
'Error', 'ERROR', 'DEBUG', 'Traceback')):
268                 continue
269             valid_metas.append(m)
270
271         deleted_count = initial_count - len(valid_metas)
272         if deleted_count > 0:
273             rag.metadatas = valid_metas
274             rag._rebuild_index()
275             rag._save()
276             logger.info(f"UserRAG [{user_id}]: cleaned {delete
d_count} corrupted records.")
277         return deleted_count
278     except Exception as ex:
279         logger.error(f"Error cleaning corrupted RAG entries fo
r user {user_id}", ex, False)
280         return 0
281
282 # =====
=====
283 # Internal helpers
284 # =====
=====
285
286 def _prune_if_needed(rag: GeminiRAG, user_id) -> None:
287     """Delete oldest documents if index exceeds _MAX_DOCS_PER_
USER.

```

```
288 |
289 |     Strategy: read meta.timestamp from all docs, delete 10% ol
dest.
290 |     """
291 |     count = rag.count()
292 |     if count < _MAX_DOCS_PER_USER:
293 |         return
294 |
295 |     try:
296 |         parsed = []
297 |         for m in rag.metadatas:
298 |             doc_id = m.get('id')
299 |             meta = m.get('meta', {})
300 |             ts = float(meta.get("timestamp", 0))
301 |             parsed.append((doc_id, ts))
302 |
303 |         parsed.sort(key=lambda x: x[1])
304 |         to_delete = parsed[:max(1, count // 10)]
305 |
306 |         for doc_id, _ in to_delete:
307 |             rag.delete_document(doc_id)
308 |
309 |         logger.info(f"UserRAG [{user_id}]: deleted {len(to_del
ete)} old records (was {count})")
310 |
311 |     except Exception as ex:
312 |         logger.error(f"Error pruning user RAG {user_id}", ex,
False)
313 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Google Generative AI Main Module
4 # =====
5 # Description:
6 #   Main module for Google Generative AI integration.
7 #   Exports the unified GoogleGenerativeAI class.
8 #
9 # File: __init__.py
10 # Project: ai-breadboard
11 # Package: src.ai.gemini
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 from .api import GoogleGenerativeAI
17 from .core import GoogleGenerativeAICore, load_unsupported_models, add_unsupported_model
18 from .config import normalize_text, remove_html_blocks
19 from .embeddings import GoogleGenerativeAIEmbeddingsMixin
20 from .errors import GoogleGenerativeAIErrorMixin
21 from .history import GoogleGenerativeAIHistoryMixin
22 from .images import GoogleGenerativeAIImagesMixin
23
24 __all__ = [
25     'GoogleGenerativeAI',
26     'GoogleGenerativeAICore',
27     'load_unsupported_models',
28     'add_unsupported_model',
29     'normalize_text',
30     'remove_html_blocks',
31     'GoogleGenerativeAIEmbeddingsMixin',
32     'GoogleGenerativeAIErrorMixin',
```

```
33 | 'GoogleGenerativeAIHistoryMixin',
34 | 'GoogleGenerativeAIImagesMixin',
35 | ]
36 |
```

gemini_cli_chat.py

```
1 # -*- coding: utf-8 -*-  
2 from .providers.gemini_cli.chat import *  
3
```

hf_chat.py

```
1 # -*- coding: utf-8 -*-  
2 from .providers.huggingface.chat import *  
3
```

langchain_agent.py

```
1 # -*- coding: utf-8 -*-  
2 from .agents.agent import *  
3
```

langchain_prompts.py

```
1 # -*- coding: utf-8 -*-  
2 from .agents.prompts import *  
3
```


langchain_tools.py

```
1 # -*- coding: utf-8 -*-  
2 from .agents.tools import *  
3
```

mcp_client.py

```
1 | # -*- coding: utf-8 -*-  
2 | from .agents.mcp_client import *  
3 |
```

model_manager.py

```
1 from .orchestration.model_manager import *
2 from .orchestration.model_manager import _CACHE_MODEL_NAMES, _normalize_model_name
3
```

ollama_chat.py

```
1 # -*- coding: utf-8 -*-  
2 from .providers.ollama.chat import *  
3 from .providers.ollama.client import OllamaClient  
4
```

onnx_chat.py

```
1 # -*- coding: utf-8 -*-  
2 from .providers.onnx.chat import *  
3
```

openai_compat_chat.py

```
1 # -*- coding: utf-8 -*-  
2 from .providers.openai.chat import *  
3
```

capability_registry.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Capability Registry and Metadata Store
4 # =====
5 # Description:
6 #   Maintains the unified mapping between AI capabilities (chat, vision,
7 #   ocr, embedding, code) and registered provider models.
8 #
9 # File: capability_registry.py
10 # Package: src.ai.orchestration
11 # Author: hypo69
12 # Copyright: (c) 2026 hypo69
13 # =====
14
15 from enum import StrEnum
16 from typing import Any, Dict, List, Optional, Set
17 from dataclasses import dataclass, field
18
19
20 class AICapability(StrEnum):
21     """Standard capability identifiers for AI workloads."""
22     CHAT = "chat"
23     VISION = "vision"
24     OCR = "ocr"
25     EMBEDDING = "embedding"
26     CODE = "code"
27     IMAGE_GENERATION = "image_generation"
28     AUDIO_TTS = "audio_tts"
29     AUDIO_STT = "audio_stt"
30
31
32 class Locality(StrEnum):
```

```

33     """Execution locality of an AI model."""
34     LOCAL = "local"
35     CLOUD = "cloud"
36     HYBRID = "hybrid"
37
38
39 @dataclass
40 class ModelDescriptor:
41     """Metadata descriptor for a registered model."""
42     id: str
43     provider: str
44     capabilities: Set[AICapability] = field(default_factory=se
45 t)
46     locality: Locality = Locality.LOCAL
47     latency_tier: str = "normal" # "low", "normal", "high"
48     requires_api_key: bool = False
49     context_window: int = 8192
50     hardware_target: str = "cpu" # "cpu", "gpu", "npu"
51
52 class CapabilityRegistry:
53     """Central registry mapping AI capabilities to registered
54 models."""
55
56     def __init__(self):
57         """Initialize empty capability registry."""
58         self._models: Dict[str, ModelDescriptor] = {}
59         self._seed_default_descriptors()
60
61     def _seed_default_descriptors(self) -> None:
62         """Seed registry with known model descriptors across p
63 roviders."""
64
65         # Windows AI
66         self.register_model(ModelDescriptor(
67             id="windows-ai:phi-silica",
68             provider="windows_ai",
69             capabilities={AICapability.CHAT, AICapability.CODE
70 },

```



```

67         locality=Locality.LOCAL,
68         latency_tier="low",
69         hardware_target="npu",
70     ))
71     self.register_model(ModelDescriptor(
72         id="windows-ai:ocr",
73         provider="windows_ai",
74         capabilities={AICapability.OCR},
75         locality=Locality.LOCAL,
76         latency_tier="low",
77         hardware_target="npu",
78     ))
79
80     # Foundry Local
81     self.register_model(ModelDescriptor(
82         id="foundry:phi-4",
83         provider="foundry",
84         capabilities={AICapability.CHAT, AICapability.CODE
85     },
86         locality=Locality.LOCAL,
87         latency_tier="low",
88         hardware_target="gpu",
89     ))
90     self.register_model(ModelDescriptor(
91         id="foundry:qwen2.5-7b",
92         provider="foundry",
93         capabilities={AICapability.CHAT, AICapability.CODE
94     },
95         locality=Locality.LOCAL,
96         latency_tier="normal",
97         hardware_target="gpu",
98     ))
99
100     # Ollama
101     self.register_model(ModelDescriptor(
102         id="ollama:llama3.1",
103         provider="ollama",
104         capabilities={AICapability.CHAT, AICapability.CODE

```

```

    },
103         locality=Locality.LOCAL,
104         latency_tier="normal",
105         hardware_target="gpu",
106     ))
107
108     # Gemini Cloud
109     self.register_model(ModelDescriptor(
110         id="gemini:gemini-3.7-flash",
111         provider="gemini",
112         capabilities={AICapability.CHAT, AICapability.VISI
113 ON, AICapability.CODE},
114         locality=Locality.CLOUD,
115         latency_tier="low",
116         requires_api_key=True,
117     ))
118     self.register_model(ModelDescriptor(
119         id="gemini:gemini-2.5-pro",
120         provider="gemini",
121         capabilities={AICapability.CHAT, AICapability.VISI
122 ON, AICapability.CODE},
123         locality=Locality.CLOUD,
124         latency_tier="normal",
125         requires_api_key=True,
126     ))
127
128     def register_model(self, descriptor: ModelDescriptor) -> N
129 one:
130         """Register a model descriptor into the capability reg
131 istry."""
132         self._models[descriptor.id] = descriptor
133
134     def get_model(self, model_id: str) -> Optional[ModelDescri
135 ptor]:
136         """Retrieve model descriptor by ID."""
137         return self._models.get(model_id)
138
139     def find_by_capability(

```

```

135 |         self,
136 |         capability: AICapability,
137 |         locality: Optional[Locality] = None,
138 |     ) -> List[ModelDescriptor]:
139 |         """Find models supporting a specific capability with o
140 | optional locality filter."""
141 |         matches = []
142 |         for model in self._models.values():
143 |             if capability in model.capabilities:
144 |                 if locality is None or model.locality == local
145 | ity:
146 |                     matches.append(model)
147 |         return matches
148 |
149 |     def list_all(self) -> List[ModelDescriptor]:
150 |         """List all registered model descriptors."""
151 |         return list(self._models.values())

```

discovery.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Dynamic Provider and Backend Discovery
4 # =====
5 # Description:
6 #   Probes running local daemons (Foundry Local, Ollama), hard
7 #   ware accelerators,
8 #   Windows AI components, and API credentials to build the li
9 #   ve capability map.
10 #
11 # File: discovery.py
12 # Package: src.ai.orchestration
13 # Author: hypo69
14 # Copyright: (c) 2026 hypo69
15 # =====
16
17 import os
18 import asyncio
19 import aiohttp
20 from typing import Any, Dict, List
21 from src.logger import logger
22 from .hardware import probe_hardware, HardwareProfile
23 from src.ai.providers.windows_ai import probe_windows_ai_compo
24 nents
25
26 class DiscoveryEngine:
27     """Discovers available backends, models, and hardware capa
28     bilities dynamically."""
29
30     def __init__(
31         self,
32         foundry_base_url: str = "http://localhost:54837",
```

```

30         ollama_base_url: str = "http://localhost:11434",
31     ):
32         self.foundry_base_url = foundry_base_url
33         self.ollama_base_url = ollama_base_url
34
35     async def probe_foundry_local(self) -> Dict[str, Any]:
36         """Probe Foundry Local OpenAI-compatible REST server."
37         """
38         url = f"{self.foundry_base_url}/v1/models"
39         try:
40             async with aiohttp.ClientSession(timeout=aiohttp.C
41             lientTimeout(total=1.5)) as session:
42                 async with session.get(url) as resp:
43                     if resp.status == 200:
44                         data = await resp.json()
45                         models = [m.get("id") for m in data.ge
46                         t("data", []) if "id" in m]
47                         return {"available": True, "models": m
48                         odels, "url": self.foundry_base_url}
49                     except Exception:
50                         pass
51                     return {"available": False, "models": [], "url": self.
52                     foundry_base_url}
53
54     async def probe_ollama(self) -> Dict[str, Any]:
55         """Probe local Ollama daemon."""
56         url = f"{self.ollama_base_url}/api/tags"
57         try:
58             async with aiohttp.ClientSession(timeout=aiohttp.C
59             lientTimeout(total=1.5)) as session:
60                 async with session.get(url) as resp:
61                     if resp.status == 200:
62                         data = await resp.json()
63                         models = [m.get("name") for m in data.
64                         get("models", []) if "name" in m]
65                         return {"available": True, "models": m
66                         odels, "url": self.ollama_base_url}
67                     except Exception:

```

```

60         pass
61         return {"available": False, "models": [], "url": self.
ollama_base_url}
62
63     def probe_cloud_credentials(self) -> Dict[str, bool]:
64         """Check presence of cloud API keys."""
65         return {
66             "gemini": bool(os.environ.get("GEMINI_API_KEY_1")
or os.environ.get("GEMINI_API_KEY")),
67             "openai": bool(os.environ.get("OPENAI_API_KEY")),
68             "anthropic": bool(os.environ.get("ANTHROPIC_API_KEY")),
69         }
70
71     async def run_full_discovery(self) -> Dict[str, Any]:
72         """Perform comprehensive discovery across all local and cloud backends.
73
74         Returns:
75             Dict[str, Any]: Aggregated discovery result.
76         """
77         hardware = probe_hardware()
78         win_ai = probe_windows_ai_components()
79         foundry_task = self.probe_foundry_local()
80         ollama_task = self.probe_ollama()
81
82         foundry_res, ollama_res = await asyncio.gather(foundry
_task, ollama_task)
83         cloud_creds = self.probe_cloud_credentials()
84
85         return {
86             "hardware": hardware.to_dict(),
87             "windows_ai": win_ai,
88             "foundry_local": foundry_res,
89             "ollama": ollama_res,
90             "cloud_providers": cloud_creds,
91         }
92

```

hardware.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Hardware and Accelerator Inspector
4 # =====
5 # Description:
6 #   Probes the host machine for CPU, GPU (CUDA/DirectML), NPU,
7 #   RAM/VRAM,
8 #   and Windows Copilot+ hardware acceleration capabilities.
9 #
10 # File: hardware.py
11 # Package: src.ai.orchestration
12 # Author: hypo69
13 # Copyright: (c) 2026 hypo69
14 # =====
15
16 import os
17 import platform
18 import subprocess
19 from dataclasses import dataclass, field
20 from typing import Any, Dict, List, Optional
21 from src.logger import logger
22
23 @dataclass
24 class HardwareProfile:
25     """Represents the host computational profile and hardware
26     accelerators."""
27     cpu_cores: int = 1
28     cpu_arch: str = "x86_64"
29     has_cuda: bool = False
30     has_directml: bool = False
31     has_npu: bool = False
32     gpus: List[Dict[str, Any]] = field(default_factory=list)
```

```

32 |     ram_gb: float = 0.0
33 |     vram_gb: float = 0.0
34 |     os_name: str = "Windows"
35 |     is_copilot_plus: bool = False
36 |
37 |     def to_dict(self) -> Dict[str, Any]:
38 |         """Convert profile to serializable dictionary."""
39 |         return {
40 |             "cpu_cores": self.cpu_cores,
41 |             "cpu_arch": self.cpu_arch,
42 |             "has_cuda": self.has_cuda,
43 |             "has_directml": self.has_directml,
44 |             "has_npu": self.has_npu,
45 |             "gpus": self.gpus,
46 |             "ram_gb": self.ram_gb,
47 |             "vram_gb": self.vram_gb,
48 |             "os_name": self.os_name,
49 |             "is_copilot_plus": self.is_copilot_plus,
50 |         }
51 |
52 |
53 | def probe_hardware() -> HardwareProfile:
54 |     """Probe host environment for computational accelerators and
55 |     memory.
56 |
57 |     Returns:
58 |         HardwareProfile: Complete hardware and execution provider
59 |         profile.
60 |     """
61 |     profile = HardwareProfile()
62 |     profile.os_name = platform.system()
63 |     profile.cpu_cores = os.cpu_count() or 1
64 |     profile.cpu_arch = platform.machine()
65 |
66 |     # 1. Probe RAM
67 |     try:
68 |         import psutil
69 |         mem = psutil.virtual_memory()

```



```

68         profile.ram_gb = round(mem.total / (1024 ** 3), 2)
69     except Exception:
70         profile.ram_gb = 16.0
71
72     # 2. Probe CUDA / NVIDIA GPUs
73     try:
74         cmd = ["nvidia-smi", "--query-gpu=name,memory.total",
75             "--format=csv,noheader,nounits"]
76         proc = subprocess.run(
77             cmd,
78             capture_output=True,
79             text=True,
80             timeout=2,
81             creationflags=subprocess.CREATE_NO_WINDOW if os.na
82             me == "nt" else 0,
83         )
84         if proc.returncode == 0 and proc.stdout.strip():
85             profile.has_cuda = True
86             for line in proc.stdout.splitlines():
87                 parts = line.split(",")
88                 if len(parts) >= 2:
89                     name = parts[0].strip()
90                     mem_mb = float(parts[1].strip())
91                     profile.gpus.append({"name": name, "vram_m
92                     b": mem_mb})
93                     profile.vram_gb += round(mem_mb / 1024, 2)
94     except Exception:
95         pass
96
97     # 3. Probe DirectML and ONNX Execution Providers
98     try:
99         import onnxruntime as ort
100         available_providers = ort.get_available_providers()
101         if "DmlExecutionProvider" in available_providers:
102             profile.has_directml = True
103         if "CUDAExecutionProvider" in available_providers:
104             profile.has_cuda = True
105         if "QNNExecutionProvider" in available_providers or "N

```

```
    puExecutionProvider" in available_providers:
103 |         profile.has_npu = True
104 |         profile.is_copilot_plus = True
105 |     except Exception:
106 |         # Default DirectML is generally available on Windows 1
0/11
107 |         if profile.os_name.lower() == "windows":
108 |             profile.has_directml = True
109 |
110 |     return profile
111 |
```

model_manager.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Normalize model identifier for consistent comp
  aris
4 # =====
5 # Description:
6 #   Централизованный реестр моделей ИИ для провайдеров Gemini,
   Gemini CLI, AGY, Foundry и Ollama.
7 #
8 # File: model_manager.py
9 # Project: ai-breadboard
10 # Package: src.ai
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import asyncio
16 from pathlib import Path
17 from typing import Any, Dict, List, Set
18
19 from google import genai
20 import aiohttp
21
22 from header import __root__
23 from src.logger.logger import logger
24 from src.secrets.api_key_state import load_api_keys
25 from src.utils.jjson import j_dumps, j_loads
26
27 _GLOBAL_CONFIG_PATH: Path = __root__ / "config.json"
28 _GEMINI_CONFIG_PATH: Path = __root__ / "src" / "ai" / "gemini"
   / "config.json"
29
30 # Локальный кэш доступных моделей в оперативной памяти на весь
```

жизненный цикл

```
31 _CACHE_MODELS: Dict[str, List[str]] = {}
32
33 _DEFAULT_GEMINI_FALLBACK: List[str] = [
34     "gemini-flash-latest",
35     "gemini-flash-lite-latest",
36     "gemini-3.6-flash",
37     "gemini-3.7-flash",
38     "gemini-pro-latest",
39 ]
40
41 _GEMINI_PRIORITY_ORDER: List[str] = [
42     "gemini-flash-latest",
43     "gemini-flash-lite-latest",
44     "gemini-3.6-flash",
45     "gemini-3.7-flash",
46     "gemini-pro-latest",
47 ]
48
49 _DEFAULT_GEMINI_CLI_FALLBACK: List[str] = [
50     "gemini-3.1-flash-lite",
51     "gemini-2.5-flash",
52     "gemini-2.5-pro",
53     "gemini-3.1-pro-preview",
54     "gemini-3.1-flash-lite-preview",
55     "gemini-flash-latest",
56     "gemini-pro-latest",
57 ]
58
59 _GEMINI_CLI_PRIORITY_ORDER: List[str] = [
60     "gemini-3.1-flash-lite",
61     "gemini-2.5-flash",
62     "gemini-2.5-pro",
63     "gemini-3.1-pro-preview",
64     "gemini-3.1-flash-lite-preview",
65     "gemini-flash-latest",
66     "gemini-pro-latest",
67 ]
```

```

68
69 def _normalize_model_name(name: str) -> str:
70     """Normalize model identifier for consistent comparison."""
71     "
72     res: str = name.strip()
73     if res.startswith("models/"):
74         res = res[len("models/") :]
75     return res
76
77 normalize_model_name = _normalize_model_name
78
79 def load_unsupported_models(provider: str = "gemini") -> Set[str]:
80     """Load list of unsupported models from configuration file
81     s.
82
83     Args:
84         provider (str): Provider name ('gemini', 'gemini_cli',
85         'agy', 'foundry', 'ollama').
86         Default: 'gemini'.
87
88     Returns:
89         Set[str]: Set of normalized names of unsupported model
90         s.
91
92     Examples:
93         >>> from src.ai.model_manager import load_unsupported_
94         models
95         >>> unsupported: Set[str] = load_unsupported_models('g
96         emini')
97         >>> isinstance(unsupported, set)
98         True
99         """
100     prov: str = provider.lower().strip()
101     unsupported: Set[str] = set()
102
103     # Loading из конфигурации модуля Gemini при необходимости
104     if prov in ("gemini", "gemini_cli", "agy"):

```

```

99         gemini_cfg = j_loads(_GEMINI_CONFIG_PATH)
100         if isinstance(gemini_cfg, dict):
101             raw_list = gemini_cfg.get("unsupported_models", [])
102         )
103         if isinstance(raw_list, list):
104             for item in raw_list:
105                 if isinstance(item, str) and item.strip():
106                     unsupported.add(_normalize_model_name(
107                         item))
108
109     # Loading из глобальной конфигурации
110     global_cfg = j_loads(_GLOBAL_CONFIG_PATH)
111     if isinstance(global_cfg, dict):
112         ai_sec = global_cfg.get("ai", {})
113         if isinstance(ai_sec, dict):
114             unsup_dict = ai_sec.get("unsupported_models", {})
115             if isinstance(unsup_dict, dict):
116                 prov_list = unsup_dict.get(prov, [])
117                 if isinstance(prov_list, list):
118                     for item in prov_list:
119                         if isinstance(item, str) and item.strip():
120                             unsupported.add(_normalize_model_name(
121                                 item))
122
123     return unsupported
124
125 def is_model_supported(provider: str, model_name: str) -> bool:
126     """
127     Check if model is supported for the specified provider.
128     """
129     norm = _normalize_model_name(model_name)
130     unsupported = load_unsupported_models(provider)
131     return norm not in unsupported
132
133 def add_unsupported_model(provider: str = "gemini", model_name: str = "", reason: str = "") -> bool:
134     """
135     Add unsupported model to configuration file and remove

```

```

    from cache.
130
131     Args:
132         provider (str): Provider name ('gemini', 'gemini_cli',
    'agy', 'foundry', 'ollama').
133             Default: 'gemini'.
134         model_name (str): Model name to exclude.
135             Default: ''.
136         reason (str): Reason for exclusion to log.
137             Default: ''.
138
139     Returns:
140         bool: Flag indicating successful save to configuration
    .
141
142     Examples:
143         >>> from src.ai.model_manager import add_unsupported_m
    odel
144         >>> result: bool = add_unsupported_model('gemini', 'ge
    mini-old-model', reason='404 Not Found')
145         True
146         """
147         if not model_name:
148             return False
149
150         prov: str = provider.lower().strip()
151         norm_name: str = _normalize_model_name(model_name)
152
153         # 1. Update конфигурации Gemini
154         if prov in ("gemini", "gemini_cli", "agy"):
155             gemini_cfg = j_loads(_GEMINI_CONFIG_PATH)
156             if isinstance(gemini_cfg, dict):
157                 curr_list = gemini_cfg.get("unsupported_models", [
    ])
158                 if not isinstance(curr_list, list):
159                     curr_list = []
160                 if norm_name not in curr_list:
161                     curr_list.append(norm_name)

```

```

162         gemini_cfg["unsupported_models"] = sorted(list
(set(curr_list)))
163         j_dumps(gemini_cfg, _GEMINI_CONFIG_PATH)
164
165     # 2. Update глобальной конфигурации
166     global_cfg = j_loads(_GLOBAL_CONFIG_PATH)
167     if isinstance(global_cfg, dict):
168         ai_sec = global_cfg.get("ai", {})
169         if not isinstance(ai_sec, dict):
170             ai_sec = {}
171         unsup_dict = ai_sec.get("unsupported_models", {})
172         if not isinstance(unsup_dict, dict):
173             unsup_dict = {}
174         prov_list = unsup_dict.get(prov, [])
175         if not isinstance(prov_list, list):
176             prov_list = []
177         if norm_name not in prov_list:
178             prov_list.append(norm_name)
179         unsup_dict[prov] = sorted(list(set(prov_list)))
180         ai_sec["unsupported_models"] = unsup_dict
181         global_cfg["ai"] = ai_sec
182         j_dumps(global_cfg, _GLOBAL_CONFIG_PATH)
183
184     # 3. Инвалидация модели в оперативной памяти
185     if prov in _CACHED_MODELS:
186         _CACHED_MODELS[prov] = [m for m in _CACHED_MODELS[prov]
187 ] if _normalize_model_name(m) != norm_name]
188
189     # Для agy также удаляем agy-<norm_name>
190     if "agy" in _CACHED_MODELS:
191         _CACHED_MODELS["agy"] = [
192             m for m in _CACHED_MODELS["agy"]
193             if _normalize_model_name(m.replace("agy-", "")) !=
194 norm_name
195         ]
196
197     logger.warning(
198         f"[ModelManager] Модель '{norm_name}' провайдера '{pro

```



```

    v}' добавлена в list неподдерживаемых "
197         f"(причина: {reason[:120]})"
198     )
199     return True
200
201 def _fetch_gemini_models_from_sdk(api_key: str = "") -> List[s
    tr]:
202     """Fetch Gemini models directly via Google GenAI SDK with
    filtering."""
203     api_keys_to_try: List[str] = []
204     if api_key:
205         api_keys_to_try.append(api_key)
206     else:
207         keys, _, _ = load_api_keys()
208         if keys:
209             api_keys_to_try.extend(keys)
210
211     unsupported: Set[str] = load_unsupported_models("gemini")
212     fallback_pool: List[str] = [m for m in _DEFAULT_GEMINI_FAL
    LBACK if m not in unsupported]
213     if not fallback_pool:
214         fallback_pool = ["gemini-flash-latest", "gemini-pro-la
    test"]
215
216     if not api_keys_to_try:
217         return fallback_pool
218
219     last_error: str = ""
220     for key in api_keys_to_try:
221         try:
222             client = genai.Client(api_key=key)
223             models: List[str] = []
224             for m in client.models.list():
225                 name: str = _normalize_model_name(m.name)
226                 # Check поддержки действия генерации контента
227                 if m.supported_actions and "generateContent" i
    n m.supported_actions:
228                     if name in unsupported:

```

```

229         continue
230         if any(x in name for x in ("bison", "gecko",
231         ", "vision", "embedding", "aqa", "imagen")):
232             continue
233             models.append(name)
234
235     if models:
236         sorted_models: List[str] = []
237         for pm in _GEMINI_PRIORITY_ORDER:
238             if pm in models and pm not in sorted_model
239 s:
240             sorted_models.append(pm)
241             for m in models:
242                 if m not in sorted_models:
243                     sorted_models.append(m)
244             return sorted_models
245     except Exception as e:
246         last_error = str(e)
247         continue
248
249     if last_error:
250         logger.warning(
251             f"[ModelManager] Error запроса списка моделей от G
252 oogle GenAI SDK: {last_error}. "
253             f"Используется резервный list моделей."
254         )
255
256     return fallback_pool
257
258 def _fetch_foundry_models_sync(base_url: str = "") -> List[str
259 ]:
260     """Synchronously fetch list of models from local Foundry s
261 erver."""
262     from src.config import ai_cfg
263     url: str = base_url or (getattr(ai_cfg, "foundry_base_url"
264 , "http://localhost:54837") if ai_cfg else "http://localhost:5
265 4837")
266     fallback_id: str = getattr(ai_cfg, "foundry_model_id", "qw

```

```

en2.5-1.5b-instruct-generic-cpu:4") if ai_cfg else "qwen2.5-1.
5b-instruct-generic-cpu:4"
260     unsupported: Set[str] = load_unsupported_models("foundry")
261
262     import requests
263     try:
264         resp = requests.get(f"{url}/v1/models", timeout=5)
265         if resp.status_code == 200:
266             data: Dict[str, Any] = resp.json()
267             models: List[str] = []
268             for item in data.get("data", []):
269                 mid: str = item.get("id", "")
270                 if mid and _normalize_model_name(mid) not in u
nunsupported:
271                     models.append(mid)
272                 if models:
273                     return models
274     except Exception as e:
275         logger.info(f"[ModelManager] Foundry сервер ({url}) не
доступен или вернул ошибку: {e}")
276
277     if _normalize_model_name(fallback_id) not in unsupported:
278         return [fallback_id]
279     return []
280
281 def _fetch_ollama_models_sync(base_url: str = "") -> List[str]
:
282     """Synchronously fetch list of models from Ollama server."
""
283     from src.config import ai_cfg
284     url: str = base_url or (getattr(ai_cfg, "ollama_base_url",
"http://localhost:11434") if ai_cfg else "http://localhost:11
434")
285     fallback_id: str = getattr(ai_cfg, "ollama_model_id", "lla
ma3.1") if ai_cfg else "llama3.1"
286     unsupported: Set[str] = load_unsupported_models("ollama")
287
288     import requests

```

```

289     try:
290         resp = requests.get(f"{url}/api/tags", timeout=5)
291         if resp.status_code == 200:
292             data: Dict[str, Any] = resp.json()
293             models: List[str] = []
294             for item in data.get("models", []):
295                 name: str = item.get("name", "")
296                 if name and _normalize_model_name(name) not in
unsupported:
297                     models.append(name)
298             if models:
299                 return models
300         except Exception as e:
301             logger.info(f"[ModelManager] Ollama сервер ({url}) нед
оступен или вернул ошибку: {e}")
302
303     if _normalize_model_name(fallback_id) not in unsupported:
304         return [fallback_id]
305     return []
306
307 def _fetch_gemini_cli_models_sync() -> List[str]:
308     """Synchronously fetch list of models for Gemini CLI with
filtering."""
309     unsupported: Set[str] = load_unsupported_models("gemini_cl
i")
310     pool: List[str] = [m for m in _DEFAULT_GEMINI_CLI_FALLBACK
if m not in unsupported]
311     if not pool:
312         pool = ["gemini-3.1-flash-lite", "gemini-2.5-flash"]
313     return pool
314
315 def _fetch_hf_models_sync() -> List[str]:
316     """Synchronously fetch list of cached HuggingFace models."
""
317     unsupported: Set[str] = load_unsupported_models("hf")
318     try:
319         from src.ai.hf_chat import hf_client
320         downloaded: List[Dict[str, Any]] = hf_client.list_down

```

```

loaded()
321     models: List[str] = []
322     for item in downloaded:
323         mid: str = item.get("id", "")
324         if mid and _normalize_model_name(mid) not in unsup
ported:
325             models.append(mid)
326         if models:
327             return models
328     except Exception as e:
329         logger.info(f"[ModelManager] HuggingFace list моделей
недоступен: {e}")
330
331     fallback: List[str] = ["Qwen/Qwen2.5-0.5B-Instruct", "goog
le/gemma-2-2b-it"]
332     return [m for m in fallback if _normalize_model_name(m) no
t in unsupported]
333
334 def _fetch_onnx_models_sync() -> List[str]:
335     """Synchronously fetch list of ONNX models."""
336     unsupported: Set[str] = load_unsupported_models("onnx")
337     try:
338         from src.ai.onnx_chat import onnx_client
339         loaded: List[Dict[str, Any]] = onnx_client.list_loaded
()
340         models: List[str] = []
341         for item in loaded:
342             mid: str = item.get("id", "")
343             if mid and _normalize_model_name(mid) not in unsup
ported:
344                 models.append(mid)
345             if models:
346                 return models
347     except Exception as e:
348         logger.info(f"[ModelManager] ONNX list моделей недосту
пен: {e}")
349
350     return []

```

```

351
352 def _fetch_openai_compat_models_sync() -> List[str]:
353     """Synchronously fetch list of OpenAI-compatible provider
    models."""
354     unsupported: Set[str] = load_unsupported_models("openai")
355     global_cfg: Dict[str, Any] = j_loads(_GLOBAL_CONFIG_PATH)
356     models: List[str] = []
357     if isinstance(global_cfg, dict):
358         compat_sec: Dict[str, Any] = global_cfg.get("openai_co
    mpat", {})
359         if isinstance(compat_sec, dict):
360             for prov_name, prov_data in compat_sec.get("provid
    ers", {}).items():
361                 if isinstance(prov_data, dict):
362                     for m in prov_data.get("models", []):
363                         if isinstance(m, str) and m and _norma
    lize_model_name(m) not in unsupported:
364                             if prov_name != "openai" and not a
    ny(m.startswith(f"{p}:") for p in ("openai", "deepseek", "groq
    ", "openrouter", "lmstudio", "local")):
365                                 models.append(f"{prov_name}:{m
    }")
366                             else:
367                                 models.append(m)
368     if not models:
369         models: List[str] = ["gpt-4o-mini", "gpt-4o", "deepsee
    k:deepseek-chat"]
370     return [m for m in models if _normalize_model_name(m) not
    in unsupported]
371
372 def get_available_models(
373     provider: str = "gemini",
374     api_key: str = "",
375     force_refresh: bool = False,
376 ) -> List[str]:
377     """Get list of current available models for given provider
    .
378

```

```

379     Uses single fetch via SDK/API with subsequent caching in m
    emory
380     for the entire application lifecycle. Excludes unsupported
    models from config.json.
381
382     Args:
383         provider (str): Provider name ('gemini', 'gemini_cli',
    'agy', 'foundry', 'ollama', 'hf', 'onnx', 'openai').
384             Default: 'gemini'.
385         api_key (str): Optional API key for Gemini.
386             Default: ''.
387         force_refresh (bool): Force cache reset and re-query p
    rovider.
388             Default: False.
389
390     Returns:
391         List[str]: List of available model identifiers.
392
393     Examples:
394         >>> from src.ai.model_manager import get_available_mod
    els
395         >>> models: List[str] = get_available_models('gemini')
396         >>> len(models) > 0
397         True
398         """
399         prov: str = provider.lower().strip()
400
401         # Быстрый возврат из кэша в оперативной памяти
402         if not force_refresh and prov in _CACHED_MODELS and _CACHE
    D_MODELS[prov]:
403             return list(_CACHED_MODELS[prov])
404
405         result_models: List[str] = []
406
407         if prov == "gemini":
408             result_models = _fetch_gemini_models_from_sdk(api_key=
    api_key)
409             _CACHED_MODELS["gemini"] = result_models

```

```
410         return list(result_models)
411
412     elif prov in ("gemini_cli", "gemini-cli"):
413         result_models = _fetch_gemini_cli_models_sync()
414         _CACHED_MODELS["gemini_cli"] = result_models
415         return list(result_models)
416
417     elif prov == "agy":
418         gemini_models: List[str] = get_available_models("gemin
419 i", api_key=api_key, force_refresh=force_refresh)
420         agy_unsupported: Set[str] = load_unsupported_models("a
421 gy")
422         result_models = [
423             f"agy-{m}" for m in gemini_models
424             if _normalize_model_name(m) not in agy_unsupported
425             and f"agy-{m}" not in agy_unsupported
426         ]
427         _CACHED_MODELS["agy"] = result_models
428         return list(result_models)
429
430     elif prov == "foundry":
431         result_models = _fetch_foundry_models_sync()
432         _CACHED_MODELS["foundry"] = result_models
433         return list(result_models)
434
435     elif prov == "ollama":
436         result_models = _fetch_ollama_models_sync()
437         _CACHED_MODELS["ollama"] = result_models
438         return list(result_models)
439
440     elif prov in ("hf", "huggingface"):
441         result_models = _fetch_hf_models_sync()
442         _CACHED_MODELS["hf"] = result_models
443         return list(result_models)
444
445     elif prov == "onnx":
446         result_models = _fetch_onnx_models_sync()
447         _CACHED_MODELS["onnx"] = result_models
```



```

445         return list(result_models)
446
447     elif prov in ("openai", "openai_compat", "openai-compat",
448 "deepseek", "groq", "openrouter", "lmstudio"):
449         result_models = _fetch_openai_compat_models_sync()
450         _CACHED_MODELS["openai"] = result_models
451         return list(result_models)
452
453     return result_models
454
455 async def actualize_all_models(force_refresh: bool = True) ->
456 Dict[str, List[str]]:
457     """Asynchronously actualize and warm up model caches for a
458     ll active providers.
459
460     Executes once at server application startup.
461
462     Args:
463         force_refresh (bool): Force query to provider SDK/API.
464         Default: True.
465
466     Returns:
467         Dict[str, List[str]]: Dictionary with lists of availab
468         le models by provider.
469
470     Examples:
471         >>> import asyncio
472         >>> from src.ai.model_manager import actualize_all_mod
473         els
474         >>> pool = asyncio.run(actualize_all_models())
475         >>> 'gemini' in pool
476         True
477
478     """
479     logger.info("[ModelManager] Запуск актуализации моделей дл
480     я всех провайдеров...")
481
482     loop = asyncio.get_running_loop()

```

```
477     # Параллельный опрос провайдеров в отдельных потоках
478     gemini_task = loop.run_in_executor(None, get_available_models, "gemini", "", force_refresh)
479     gemini_cli_task = loop.run_in_executor(None, get_available_models, "gemini_cli", "", force_refresh)
480     foundry_task = loop.run_in_executor(None, get_available_models, "foundry", "", force_refresh)
481     ollama_task = loop.run_in_executor(None, get_available_models, "ollama", "", force_refresh)
482     hf_task = loop.run_in_executor(None, get_available_models, "hf", "", force_refresh)
483     onnx_task = loop.run_in_executor(None, get_available_models, "onnx", "", force_refresh)
484     openai_task = loop.run_in_executor(None, get_available_models, "openai", "", force_refresh)
485
486     gemini_res, gemini_cli_res, foundry_res, ollama_res, hf_res, onnx_res, openai_res = await asyncio.gather(
487         gemini_task, gemini_cli_task, foundry_task, ollama_task, hf_task, onnx_task, openai_task, return_exceptions=True
488     )
489
490     gemini_list: List[str] = gemini_res if isinstance(gemini_res, list) else []
491     gemini_cli_list: List[str] = gemini_cli_res if isinstance(gemini_cli_res, list) else []
492     foundry_list: List[str] = foundry_res if isinstance(foundry_res, list) else []
493     ollama_list: List[str] = ollama_res if isinstance(ollama_res, list) else []
494     hf_list: List[str] = hf_res if isinstance(hf_res, list) else []
495     onnx_list: List[str] = onnx_res if isinstance(onnx_res, list) else []
496     openai_list: List[str] = openai_res if isinstance(openai_res, list) else []
497
498     # AGY формируется на основе актуализированных моделей Gemini
```

```
ni
499     agy_list = get_available_models("agy", force_refresh=force
    _refresh)
500
501     result_pool: Dict[str, List[str]] = {
502         "gemini": gemini_list,
503         "gemini_cli": gemini_cli_list,
504         "agy": agy_list,
505         "foundry": foundry_list,
506         "ollama": ollama_list,
507         "hf": hf_list,
508         "onnx": onnx_list,
509         "openai": openai_list,
510     }
511
512     logger.info(
513         f"[ModelManager] Актуализация завершена: Gemini={len(g
    emini_list)}, "
514         f"Gemini_CLI={len(gemini_cli_list)}, AGY={len(agy_list
    )}, "
515         f"Foundry={len(foundry_list)}, Ollama={len(ollama_list
    )}, "
516         f"HF={len(hf_list)}, ONNX={len(onnx_list)}, OpenAI={le
    n(openai_list)}"
517     )
518     return result_pool
519
```

policy.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Orchestration Policy Engine
4 # =====
5 # Description:
6 #   Evaluates routing constraints (privacy, locality, latency,
7 #   fallback)
8 #   to select compliant execution providers.
9 #
10 # File: policy.py
11 # Package: src.ai.orchestration
12 # Author: hypo69
13 # Copyright: (c) 2026 hypo69
14 # =====
15
16 from enum import StrEnum
17 from dataclasses import dataclass
18 from typing import Optional
19 from .capability_registry import Locality, ModelDescriptor
20
21 class PrivacyLevel(StrEnum):
22     """Data privacy constraints for AI requests."""
23     STRICT = "strict"      # Zero data leaves the local host
24     STANDARD = "standard"  # Cloud transmission allowed
25
26
27 class LocalityPreference(StrEnum):
28     """Locality preference for dispatch."""
29     LOCAL_ONLY = "local_only"
30     PREFER_LOCAL = "prefer_local"
31     CLOUD_ONLY = "cloud_only"
32     PREFER_CLOUD = "prefer_cloud"
```

```

33     AUTO = "auto"
34
35
36 @dataclass
37 class RoutingPolicy:
38     """Routing policy configuration for an AI request."""
39     locality: LocalityPreference = LocalityPreference.AUTO
40     privacy: PrivacyLevel = PrivacyLevel.STANDARD
41     max_latency_tier: str = "high"
42     allow_fallback: bool = True
43
44
45 class PolicyEngine:
46     """Validates and filters model candidates against request
47     policy constraints."""
48
49     @staticmethod
50     def is_compliant(descriptor: ModelDescriptor, policy: RoutingPolicy) -> bool:
51         """Check if model descriptor satisfies the routing policy.
52
53         Args:
54             descriptor (ModelDescriptor): Model descriptor to
55             validate.
56             policy (RoutingPolicy): Routing policy.
57
58         Returns:
59             bool: True if compliant, False otherwise.
60         """
61         # 1. Privacy Check (Strict privacy forbids cloud models)
62         if policy.privacy == PrivacyLevel.STRICT and descriptor.locality != Locality.LOCAL:
63             return False
64
65         # 2. Locality Preference Checks
66         if policy.locality == LocalityPreference.LOCAL_ONLY and

```

```
    d descriptor.locality != Locality.LOCAL:
65 |         return False
66 |         if policy.locality == LocalityPreference.CLOUD_ONLY and
    d descriptor.locality != Locality.CLOUD:
67 |             return False
68 |
69 |         return True
70 |
```

router.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI Capability and Request Router
4 # =====
5 # Description:
6 #   Dispatches user requests to the optimal AI provider based
7 #   on capability,
8 #   system discovery, and policy rules.
9 #
10 # File: router.py
11 # Package: src.ai.orchestration
12 # Author: hypo69
13 # Copyright: (c) 2026 hypo69
14 # =====
15
16 from dataclasses import dataclass, field
17 from typing import Any, Dict, List, Optional
18 from src.logger import logger
19 from .capability_registry import AICapability, CapabilityRegistry, ModelDescriptor, Locality
20 from .policy import PolicyEngine, RoutingPolicy, LocalityPreference, PrivacyLevel
21 from .discovery import DiscoveryEngine
22
23 @dataclass
24 class AIRequest:
25     """Represents a unified AI capability request."""
26     prompt: str
27     capability: AICapability = AICapability.CHAT
28     policy: RoutingPolicy = field(default_factory=RoutingPolicy)
29     model_override: Optional[str] = None
```

```

30     system_instruction: str = ""
31     temperature: float = 0.0
32     max_tokens: int = 0
33
34
35 class AIRouter:
36     """Evaluates requests against capabilities and routes to a
    ctive providers."""
37
38     def __init__(
39         self,
40         registry: Optional[CapabilityRegistry] = None,
41         discovery: Optional[DiscoveryEngine] = None,
42     ):
43         self.registry = registry or CapabilityRegistry()
44         self.discovery = discovery or DiscoveryEngine()
45
46     async def resolve_candidate_models(self, request: AIReques
    t) -> List[ModelDescriptor]:
47         """Resolve ordered list of compliant model descriptors
    for the request."""
48         # Direct override takes precedence
49         if request.model_override:
50             desc = self.registry.get_model(request.model_overr
    ide)
51             if desc and PolicyEngine.is_compliant(desc, reques
    t.policy):
52                 return [desc]
53
54         candidates = self.registry.find_by_capability(request.
    capability)
55         compliant = [c for c in candidates if PolicyEngine.is_
    compliant(c, request.policy)]
56
57         # Order candidates: local first if prefer_local, cloud
    first if prefer_cloud
58         if request.policy.locality in (LocalityPreference.PREF
    ER_LOCAL, LocalityPreference.LOCAL_ONLY, LocalityPreference.AU

```



```
T0):
59 |         compliant.sort(key=lambda m: 0 if m.locality == Lo
|        cality.LOCAL else 1)
60 |         elif request.policy.locality in (LocalityPreference.PR
|        EFER_CLOUD, LocalityPreference.CLOUD_ONLY):
61 |             compliant.sort(key=lambda m: 0 if m.locality == Lo
|        cality.CLOUD else 1)
62 |
63 |         return compliant
64 |
```

unified_chat.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Wrapper class for transparent routing between
  Gemi
4 # =====
5 # Description:
6 #   A wrapper model class that aggregates GoogleGenerativeAI a
  nd FoundryChatBase
7 #
8 # File: unified_chat.py
9 # Project: ai-breadboard
10 # Package: src.ai
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import os
16 import inspect
17 import asyncio
18 from typing import Any, AsyncIterator, Dict, List, Optional
19 from src.logger import logger
20 from src.ai.providers.foundry import FoundryChatBase
21
22 class UnifiedChatModel:
23     """
24     Wrapper class for transparent routing between Gemini and F
  oundry models.
25     """
26     def __init__(
27         self,
28         api_key_names: List[str],
29         system_instruction: str,
30         foundry_model_id: str,
```

```
31 |         use_foundry: bool = False,
32 |         use_ollama: bool = False,
33 |         ollama_model_id: str = "llama3.1",
34 |         ollama_base_url: str = "http://localhost:11434",
35 |     ):
36 |         from src.ai.gemini.api import GoogleGenerativeAI
37 |         self.gemini_model = GoogleGenerativeAI(
38 |             api_key_names=api_key_names,
39 |             system_instruction=system_instruction,
40 |             sleep_on_exhausted=False,
41 |         )
42 |         self.foundry_model = False
43 |         self.use_foundry = use_foundry
44 |         self.foundry_model_id = foundry_model_id
45 |
46 |         self.ollama_model = False
47 |         self.use_ollama = use_ollama
48 |         self.ollama_model_id = ollama_model_id
49 |         self.ollama_base_url = ollama_base_url
50 |
51 |         if use_foundry:
52 |             self.foundry_model = FoundryChatBase(
53 |                 model_id=foundry_model_id,
54 |                 system_prompt=system_instruction,
55 |             )
56 |
57 |         if use_ollama:
58 |             from src.ai.ollama_chat import OllamaChatBase
59 |             self.ollama_model = OllamaChatBase(
60 |                 model_id=ollama_model_id,
61 |                 system_prompt=system_instruction,
62 |                 api_url=ollama_base_url,
63 |             )
64 |
65 |         self.gemini_cli_model = False
66 |         self.agy_model = False
67 |
68 |         from src.ai.gemini.generative_ai import _DEFAULT_MODEL
```

```

69         if use_ollama:
70             self.default_model = f"ollama:{ollama_model_id}"
71         elif use_foundry:
72             self.default_model = f"foundry:{foundry_model_id}"
73         else:
74             self.default_model = _DEFAULT_MODEL
75
76         self._model_name = self.default_model
77
78
79     def update_system_instruction(self, new_instruction: str)
-> None:
80         """Dynamically update system instruction for all initi
alized models."""
81         self.system_instruction = new_instruction
82         if hasattr(self, 'gemini_model') and self.gemini_model
:
83             self.gemini_model.system_instruction = new_instruc
tion
84         if hasattr(self, 'foundry_model') and self.foundry_mod
el:
85             self.foundry_model.system_prompt = new_instruction
86         if hasattr(self, 'ollama_model') and self.ollama_model
:
87             self.ollama_model.system_prompt = new_instruction
88         if hasattr(self, 'agy_model') and self.agy_model:
89             self.agy_model.system_prompt = new_instruction
90         if hasattr(self, 'gemini_cli_model') and self.gemini_c
li_model:
91             self.gemini_cli_model.system_prompt = new_instruct
ion
92         import logging
93         logger = logging.getLogger(__name__)
94         logger.info("[UnifiedChatModel] Системная инструкция s
uccessfully обновлена")
95
96     @property
97     def model_name(self) -> str:

```

```
98         return self._model_name
99
100     @model_name.setter
101     def model_name(self, val: str):
102         self._model_name = val
103         if self.foundry_model:
104             self.foundry_model.model_id = val.replace("foundry
105             :", "")
106         if self.ollama_model:
107             self.ollama_model.model_id = val.replace("ollama:"
108             , "")
109         if hasattr(self, 'gemini_cli_model') and self.gemini_c
110         li_model:
111             self.gemini_cli_model.model_id = val
112             if self.gemini_model:
113                 self.gemini_model.model_name = val
114
115     @property
116     def model_id(self) -> str:
117         return self._model_name
118
119     @model_id.setter
120     def model_id(self, val: str):
121         self._model_name = val
122         if self.foundry_model:
123             self.foundry_model.model_id = val.replace("foundry
124             :", "")
125         if self.ollama_model:
126             self.ollama_model.model_id = val.replace("ollama:"
127             , "")
128         if hasattr(self, 'gemini_cli_model') and self.gemini_c
129         li_model:
130             self.gemini_cli_model.model_id = val
131             if self.gemini_model:
132                 self.gemini_model.model_name = val
133
134     @property
135     def api_key(self) -> str:
```

```

130         return getattr(self.gemini_model, 'api_key', '') or ''
131
132     @property
133     def system_instruction(self) -> str:
134         if hasattr(self, 'gemini_model') and self.gemini_model
135         :
136             return self.gemini_model.system_instruction
137         return ""
138
139     @system_instruction.setter
140     def system_instruction(self, val: str):
141         if hasattr(self, 'gemini_model') and self.gemini_model
142         :
143             self.gemini_model.system_instruction = val
144
145     def _get_active_model(self, model_name: Optional[str] = ""
146     ) -> tuple[Any, str]:
147         active_name = model_name or self._model_name
148
149         if active_name.startswith("gemini_cli:") or active_name
150         .startswith("gemini-cli-"):
151             if not self.gemini_cli_model:
152                 from src.ai.providers.gemini_cli import Gemini
153                 CliChatBase
154                 self.gemini_cli_model = GeminiCliChatBase(
155                     model_id=active_name.replace("gemini_cli:"
156                     , ""),
157                     system_prompt=self.system_instruction or "
158                     ",
159                     )
160             else:
161                 self.gemini_cli_model.model_id = active_name.r
162                 eplace("gemini_cli:", "")
163             return self.gemini_cli_model, active_name
164
165         if active_name.startswith("ollama:"):
166             if not self.ollama_model:
167                 from src.ai.providers.ollama import OllamaChat

```

```

Base
160         self.ollama_model = OllamaChatBase(
161             model_id=active_name.replace("ollama:", "")
162         ),
163             system_prompt=self.system_instruction or "
164             ",
165             api_url=self.ollama_base_url,
166         )
167     else:
168         self.ollama_model.model_id = active_name.repla
169 ce("ollama:", "")
170     return self.ollama_model, active_name
171
172     if active_name.startswith("agy-") or 'agy' in active_n
173 ame.lower():
174         if not hasattr(self, 'agy_model') or not self.agy_
175 model:
176             from src.ai.providers.agy import AgyChatBase
177             self.agy_model = AgyChatBase(
178                 model_id=active_name,
179                 system_prompt=self.system_instruction or "
180                 ",
181             )
182         else:
183             self.agy_model.model_id = active_name
184             return self.agy_model, active_name
185
186     if active_name.startswith("foundry:"):
187         if not self.foundry_model:
188             self.foundry_model = FoundryChatBase(
189                 model_id=active_name.replace("foundry:", "
190                 "),
191                 system_prompt=self.system_instruction or "
192                 ",
193             )
194         else:
195             self.foundry_model.model_id = active_name.repl
196 ace("foundry:", "")

```

```

188         return self.foundry_model, active_name
189
190         if active_name.startswith("hf:") or active_name.starts
with("hf::"):
191             model_id = active_name.split(":", 1)[-1].lstrip(":
")
192             from src.ai.providers.huggingface import HFChatBas
e
193             return HFChatBase(model_id=model_id, system_prompt
=self.system_instruction or ""), active_name
194
195         if active_name.startswith("onnx:") or active_name.star
tswith("onnx::"):
196             model_id = active_name.split(":", 1)[-1].lstrip(":
")
197             from src.ai.providers.onnx import ONNXChatBase
198             return ONNXChatBase(model_id=model_id, system_prom
pt=self.system_instruction or ""), active_name
199
200         openai_prefixes = ("openai:", "openai::", "deepseek:",
"groq:", "openrouter:", "lmstudio:", "local:", "compat:")
201         if any(active_name.startswith(p) for p in openai_prefi
xes):
202             prov_part, model_part = active_name.split(":", 1)
203             model_id = model_part.lstrip(":")
204             prov_name = prov_part.lower().rstrip(":")
205             if prov_name == "compat":
206                 prov_name = "openai"
207             from src.ai.providers.openai import OpenAICompatCh
at
208             return OpenAICompatChat.create_for_provider(
209                 provider_name=prov_name,
210                 model_id=model_id,
211                 system_prompt=self.system_instruction or "",
212             ), active_name
213
214         return self.gemini_model, active_name
215

```



```

216     async def chat(
217         self,
218         q: str,
219         history: Optional[List[Dict[str, Any]]] = None,
220         system_instruction: Optional[str] = "",
221         attempts: int = 15,
222         model_name: Optional[str] = "",
223         temperature: Optional[float] = 0.0,
224         max_tokens: Optional[int] = 0,
225         **kwargs
226     ) -> Optional[str]:
227         model_instance, active_name = self._get_active_model(model_name)
228         logger.info(f"[UnifiedChatModel] chat: prompt={repr(q[:100])}... using model={active_name}")
229
230         sig = inspect.signature(model_instance.chat)
231         call_kwargs = {
232             "q": q,
233             "history": history,
234             "system_instruction": system_instruction,
235             "attempts": attempts,
236         }
237         if "model_name" in sig.parameters:
238             call_kwargs["model_name"] = active_name
239         elif hasattr(model_instance, "model_id"):
240             model_instance.model_id = active_name
241
242         if "temperature" in sig.parameters and temperature:
243             call_kwargs["temperature"] = temperature
244         if "max_tokens" in sig.parameters and max_tokens:
245             call_kwargs["max_tokens"] = max_tokens
246
247         for k, v in kwargs.items():
248             if k in sig.parameters:
249                 call_kwargs[k] = v
250
251         try:

```

```

252         res = await model_instance.chat(**call_kwargs)
253         logger.info(f"[UnifiedChatModel] chat success: res
ponse={repr(res[:100]) if res else 'None'}...")
254         return res
255     except Exception as ex:
256         logger.error(f"[UnifiedChatModel] chat error with
model={active_name}", ex)
257         raise
258
259     async def ask(
260         self,
261         q: str,
262         attempts: int = 15,
263         generation_config: Dict[str, Any] = None,
264         model_name: Optional[str] = "",
265         temperature: Optional[float] = 0.0,
266         max_tokens: Optional[int] = 0,
267         **kwargs
268     ) -> Optional[str]:
269         model_instance, active_name = self._get_active_model(m
odel_name)
270         logger.info(f"[UnifiedChatModel] ask: prompt={repr(q[:
100])}... using model={active_name}")
271
272         sig = inspect.signature(model_instance.ask)
273         call_kwargs = {
274             "q": q,
275             "attempts": attempts,
276         }
277         if "model_name" in sig.parameters:
278             call_kwargs["model_name"] = active_name
279         elif hasattr(model_instance, "model_id"):
280             model_instance.model_id = active_name
281
282         if "generation_config" in sig.parameters:
283             call_kwargs["generation_config"] = generation_conf
ig
284

```

```

285         if "temperature" in sig.parameters and temperature:
286             call_kwargs["temperature"] = temperature
287         if "max_tokens" in sig.parameters and max_tokens:
288             call_kwargs["max_tokens"] = max_tokens
289
290         for k, v in kwargs.items():
291             if k in sig.parameters:
292                 call_kwargs[k] = v
293
294         try:
295             res = await model_instance.ask(**call_kwargs)
296             logger.info(f"[UnifiedChatModel] ask success: response={repr(res[:100]) if res else 'None'}...")
297             return res
298         except Exception as ex:
299             logger.error(f"[UnifiedChatModel] ask error with model={active_name}", ex)
300             raise
301
302     async def chat_stream(
303         self,
304         q: str,
305         history: Optional[List[Dict[str, Any]]] = None,
306         system_instruction: Optional[str] = "",
307         attempts: int = 15,
308         model_name: Optional[str] = "",
309         temperature: Optional[float] = 0.0,
310         max_tokens: Optional[int] = 0,
311         generation_config: Dict[str, Any] = None,
312         **kwargs
313     ) -> AsyncIterator[str]:
314         model_instance, active_name = self._get_active_model(model_name)
315         logger.info(f"[UnifiedChatModel] chat_stream: prompt={repr(q[:100])}... using model={active_name}")
316
317         sig = inspect.signature(model_instance.chat_stream)
318         call_kwargs = {

```

```

319         "q": q,
320         "history": history,
321         "system_instruction": system_instruction,
322         "attempts": attempts,
323     }
324     if "model_name" in sig.parameters:
325         call_kwargs["model_name"] = active_name
326     elif hasattr(model_instance, "model_id"):
327         model_instance.model_id = active_name
328
329     if "generation_config" in sig.parameters:
330         call_kwargs["generation_config"] = generation_conf
331
332     if "temperature" in sig.parameters and temperature:
333         call_kwargs["temperature"] = temperature
334     if "max_tokens" in sig.parameters and max_tokens:
335         call_kwargs["max_tokens"] = max_tokens
336
337     for k, v in kwargs.items():
338         if k in sig.parameters:
339             call_kwargs[k] = v
340
341     try:
342         async for chunk in model_instance.chat_stream(**call_kwargs):
343             yield chunk
344     except Exception as ex:
345         logger.error(f"[UnifiedChatModel] chat_stream error with model={active_name}", ex)
346         raise
347
348     async def ask_with_tools_stream(
349         self,
350         q: str,
351         tools: List[Any],
352         tool_dispatcher: Any,
353         system_instruction: Optional[str] = "",

```

```

354         model_name: Optional[str] = "",
355         history: Optional[List[Dict[str, Any]]] = None,
356         **kwargs
357     ) -> AsyncIterator[str]:
358         model_instance, active_name = self._get_active_model(model_name)
359         logger.info(f"[UnifiedChatModel] ask_with_tools_stream
: prompt={repr(q[:100])}... using model={active_name}")
360
361         if not hasattr(model_instance, 'ask_with_tools_stream'
):
362             raise NotImplementedError(f"Model {active_name} does not support ask_with_tools_stream")
363
364         sig = inspect.signature(model_instance.ask_with_tools_stream)
365         call_kwargs = {
366             "q": q,
367             "tools": tools,
368             "tool_dispatcher": tool_dispatcher,
369             "system_instruction": system_instruction,
370             "history": history,
371         }
372
373         if "model_name" in sig.parameters:
374             call_kwargs["model_name"] = active_name
375         elif hasattr(model_instance, "model_id"):
376             model_instance.model_id = active_name
377
378         for k, v in kwargs.items():
379             if k in sig.parameters:
380                 call_kwargs[k] = v
381
382         try:
383             async for chunk in model_instance.ask_with_tools_stream(**call_kwargs):
384                 yield chunk
385         except Exception as ex:

```

```
386 |         logger.error(f"[UnifiedChatModel] ask_with_tools_s  
    | tream error with model={active_name}", ex)  
387 |         raise  
388 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 from .hardware import HardwareProfile, probe_hardware
3 from .capability_registry import AICapability, CapabilityRegistry, ModelDescriptor, Locality
4 from .policy import PolicyEngine, RoutingPolicy, PrivacyLevel, LocalityPreference
5 from .discovery import DiscoveryEngine
6 from .router import AIRouter, AIRequest
7 from .unified_chat import UnifiedChatModel
8 from .model_manager import (
9     load_unsupported_models,
10    normalize_model_name,
11    get_available_models,
12    actualize_all_models,
13    add_unsupported_model,
14    is_model_supported,
15 )
16
17 __all__ = [
18     "HardwareProfile",
19     "probe_hardware",
20     "AICapability",
21     "CapabilityRegistry",
22     "ModelDescriptor",
23     "Locality",
24     "PolicyEngine",
25     "RoutingPolicy",
26     "PrivacyLevel",
27     "LocalityPreference",
28     "DiscoveryEngine",
29     "AIRouter",
30     "AIRequest",
31     "UnifiedChatModel",
32     "load_unsupported_models",
33     "normalize_model_name",
34     "get_available_models",
```

```
35 |     "actualize_all_models",
36 |     "add_unsupported_model",
37 |     "is_model_supported",
38 | ]
39 |
```


chat.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Antigravity SDK chat connection and request routing
4 # =====
5 # Description:
6 #   Chat adapter for interacting with Antigravity SDK models (
7 #   agy-flash, agy-pro).
8 #   Implements standard chat interfaces for FastAPI router architecture compatibility.
9 #
10 # File: agy_chat.py
11 # Project: ai-breadboard
12 # Package: src.ai
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16 """Antigravity SDK chat connection and request routing adapter
17 .
18 Implements chat interfaces (ask, chat_stream) for Antigravity
19 SDK models with
20 API key management and local conversation history support."""
21 import os
22 import asyncio
23 from typing import Optional, List, Dict, AsyncGenerator
24
25 from src.logger.logger import logger
26 from src.secrets.api_key_state import load_api_keys
27
28 class AgyChatBase:
```

```

29 |     """Chat adapter for Antigravity SDK models (agy-flash, agy
30 | -pro).
31 |
32 |     Implements standard chat interfaces for FastAPI router com
33 | patibility.
34 |
35 |     Attributes:
36 |         model_id (str): Normalized model identifier (e.g., agy
37 | -flash).
38 |         system_instruction (str): System instructions for LLM.
39 |         history (List[Dict[str, str]]): Local conversation his
40 | tory.
41 |         api_key (str): Active API key for agent initialization
42 | .
43 |     """
44 |
45 |     @classmethod
46 |     def get_available_models(cls, force_refresh: bool = False)
47 |     -> List[str]:
48 |         """Get list of available models for AGY provider.
49 |
50 |         Args:
51 |             force_refresh (bool): Force refresh of model list.
52 |
53 |         Returns:
54 |             List[str]: List of available model identifiers.
55 |         """
56 |         from src.ai.model_manager import get_available_models
57 |         as _mgr_get_available_models
58 |         return _mgr_get_available_models(provider="agy", force
59 | _refresh=force_refresh)
60 |
61 |     @classmethod
62 |     def normalize_model_id(cls, model_id: str) -> str:
63 |         """Normalize model identifier for Antigravity SDK.
64 |
65 |         Args:
66 |             model_id (str): Raw model identifier.

```

```

59 |
60 |     Returns:
61 |         str: Normalized model identifier."""
62 |     actual = (model_id or "").strip()
63 |     while actual.startswith("agy-"):
64 |         actual = actual[4:]
65 |     if actual in ("flash", "flash-latest", "agy-flash", ""
66 | ):
67 |         return "gemini-flash-lite-latest"
68 |     elif actual in ("pro", "pro-latest", "agy-pro"):
69 |         return "gemini-pro-latest"
70 |     elif not (actual.startswith("gemini-") or actual.start
71 | with("gemma-") or actual.startswith("deep-research-") or actu
72 | al.startswith("lyria-")):
73 |         actual = f"gemini-{actual}"
74 |     return actual
75 |
76 |     def __init__(self, model_id: str, system_prompt: str = "")
77 |     -> None:
78 |         """Initialize connection to AGY SDK.
79 |
80 |         Args:
81 |             model_id (str): Model identifier.
82 |             system_prompt (str): System prompt for the model."
83 |         """
84 |         self._model_id: str = self.normalize_model_id(model_id
85 | )
86 |
87 |         self.system_prompt: str = system_prompt
88 |         self.history: List[Dict[str, str]] = []
89 |         valid_keys: List[str] = []
90 |         agy_key = os.getenv('AGY_API_KEY', '').strip()
91 |         if agy_key:
92 |             valid_keys.append(agy_key)
93 |
94 |         _api_key_names = [n.strip() for n in os.getenv('GEMINI
95 | _API_KEY_NAMES', '').split(',') if n.strip()]
96 |         loaded, _, _ = load_api_keys(_api_key_names if _api_ke
97 | y_names else [])

```

```

89         for k in loaded:
90             if k and k not in valid_keys:
91                 valid_keys.append(k)
92
93         self.api_keys: List[str] = valid_keys
94         self.api_key: str = valid_keys[0] if valid_keys else "
95
96     @property
97     def model_id(self) -> str:
98         """Get normalized model identifier.
99
100         Returns:
101             str: Normalized model identifier.
102         """
103         return self._model_id
104
105     @model_id.setter
106     def model_id(self, val: str) -> None:
107         """Set and normalize model identifier.
108
109         Args:
110             val (str): Model identifier to set.
111         """
112         self._model_id = self.normalize_model_id(val)
113
114     @property
115     def system_instruction(self) -> str:
116         """Get current system instruction.
117
118         Returns:
119             str: Current system instruction.
120         """
121         return self.system_prompt
122
123     @system_instruction.setter
124     def system_instruction(self, val: str) -> None:
125         """Set system instruction.

```

```

126
127     Args:
128         val (str): System instruction to set.
129         """
130     self.system_prompt = val
131
132     def _clean_output(self, text: str) -> str:
133         """Clean response from SDK internal step messages.
134
135         Args:
136             text (str): Raw response text.
137
138         Returns:
139             str: Cleaned response text."""
140         cleaned = text.strip()
141         if "error executing cascade step:" in cleaned or "RESOURCE_EXHAUSTED" in cleaned or "GenerateContent failed:" in cleaned:
142             if "]]]]" in cleaned:
143                 idx = cleaned.find("]]]]")
144                 cleaned = cleaned[idx + 4:].strip()
145             elif "]]" in cleaned:
146                 idx = cleaned.rfind("]]")
147                 cleaned = cleaned[idx + 2:].strip()
148             else:
149                 lines = cleaned.split("\n")
150                 filtered = [l for l in lines if not l.startswith("error executing cascade step:") and not l.startswith("GenerateContent failed:") and "RESOURCE_EXHAUSTED" not in l]
151                 cleaned = "\n".join(filtered).strip()
152         return cleaned
153
154     def clear_history(self) -> None:
155         """Clear local chat history."""
156         self.history = []
157
158     async def ask(
159         self,

```

```

160         q: str,
161         system_instruction: Optional[str] = "",
162         temperature: Optional[float] = 0.0,
163         max_tokens: Optional[int] = 0,
164         **kwargs
165     ) -> str:
166         """Send single request to agent.
167
168         Args:
169             q (str): User question.
170             system_instruction (Optional[str]): System instruc
tion override.
171             temperature (Optional[float]): Generation temperat
ure (default 0.0).
172             max_tokens (Optional[int]): Max tokens (0 for unli
mited).
173             **kwargs: Additional parameters.
174
175         Returns:
176             str: Agent response text."""
177         if not q or not q.strip():
178             return ""
179
180         sys_prompt = system_instruction or self.system_prompt
or ""
181
182         try:
183             from google.antigravity import Agent, LocalAgentCo
nfig, CapabilitiesConfig
184             config = LocalAgentConfig(
185                 model=self.model_id,
186                 system_instructions=sys_prompt,
187                 api_key=self.api_key,
188                 tools=[],
189                 policies=[],
190                 capabilities=CapabilitiesConfig(enable_subagen
ts=False, enabled_tools=[])
191             )

```

```

192         async with Agent(config) as agent:
193             response = await agent.chat(q)
194             text = ""
195             async for token in response:
196                 text += token
197             return self._clean_output(text)
198     except Exception as e:
199         err_str = str(e)
200         if any(x in err_str for x in ('404', 'NOT_FOUND',
201 'not supported', 'is no longer available', 'not found')):
202             from src.ai.model_manager import add_unsupported_model
203             add_unsupported_model('agy', self.model_id, reason=err_str)
204             add_unsupported_model('gemini', self.model_id, reason=err_str)
205             logger.error("Error in AgychatBase.ask", e, exc_info=True)
206             return ""
207
208     async def chat(
209         self,
210         q: str,
211         history: Optional[List[Dict]] = [],
212         system_instruction: Optional[str] = "",
213         save_history: bool = True,
214         temperature: Optional[float] = 0.0,
215         max_tokens: Optional[int] = 0,
216         **kwargs
217     ) -> str:
218         """Send chat request with conversation history.
219
220         Args:
221             q (str): User question.
222             history (Optional[List[Dict]]): Conversation history.
223             system_instruction (Optional[str]): System instruction override.

```

```

223         save_history (bool): Save to local history (default
    t True).
224         temperature (Optional[float]): Generation temperat
    ure.
225         max_tokens (Optional[int]): Max tokens.
226         **kwargs: Additional parameters.
227
228     Returns:
229         str: Chat response text."""
230     if not q or not q.strip():
231         return ""
232
233     # Build full response from chat_stream generator
234     chunks = []
235     async for chunk in self.chat_stream(
236         q=q,
237         history=history,
238         system_instruction=system_instruction,
239         temperature=temperature,
240         max_tokens=max_tokens,
241         **kwargs,
242     ):
243         chunks.append(chunk)
244     ans = "".join(chunks)
245     if save_history and ans:
246         self.history.append({"role": "user", "content": q}
    )
247         self.history.append({"role": "model", "content": a
    ns})
248     return ans
249
250     async def chat_stream(
251         self,
252         q: str,
253         history: Optional[List[Dict]] = [],
254         system_instruction: Optional[str] = "",
255         temperature: Optional[float] = 0.0,
256         max_tokens: Optional[int] = 0,

```



```

257         **kwargs
258     ) -> AsyncGenerator[str, None]:
259         """Send streaming request to agent.
260
261         Args:
262             q (str): User question text.
263             history (Optional[List[Dict]]): Optional conversat
ion history.
264             system_instruction (Optional[str]): System instruc
tion override.
265             temperature (Optional[float]): Generation temperat
ure (default 0.0).
266             max_tokens (Optional[int]): Maximum tokens (0 for
default).
267
268         Yields:
269             str: Response text chunks as generated.
270         """
271         if not q or not q.strip():
272             return
273
274         sys_prompt = system_instruction or self.system_prompt
or ""
275
276         # Integrate history into context
277         context = ""
278         if history:
279             for msg in history:
280                 role = msg.get('role', 'user')
281                 content = msg.get('content', '')
282                 if not content and 'parts' in msg:
283                     content = " ".join([p.get('text', '') if i
sinstance(p, dict) else str(p) for p in msg['parts']])
284                 context += f"\n[{role}]: {content}"
285
286         if context:
287             sys_prompt = f"{sys_prompt}\n\nConversation histor
y:\n{context}"

```

```

288 |
289 |         try:
290 |             from google.antigravity import Agent, LocalAgentCo
nfig, CapabilitiesConfig
291 |             config = LocalAgentConfig(
292 |                 model=self.model_id,
293 |                 system_instructions=sys_prompt,
294 |                 api_key=self.api_key,
295 |                 tools=[],
296 |                 policies=[],
297 |                 capabilities=CapabilitiesConfig(enable_subagen
ts=False, enabled_tools=[])
298 |             )
299 |             async with Agent(config) as agent:
300 |                 response = await agent.chat(q)
301 |                 async for token in response:
302 |                     yield token
303 |             except Exception as e:
304 |                 err_str = str(e)
305 |                 if any(x in err_str for x in ('404', 'NOT_FOUND',
'not supported', 'is no longer available', 'not found')):
306 |                     from src.ai.model_manager import add_unsupport
ed_model
307 |                     add_unsupported_model('agy', self.model_id, re
ason=err_str)
308 |                     add_unsupported_model('gemini', self.model_id,
reason=err_str)
309 |                     err_msg = f"Error Antigravity SDK: {err_str}"
310 |                     logger.error(err_msg, e, exc_info=True)
311 |                     yield err_msg
312 |

```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 from .chat import AgyChatBase
3
4 __all__ = ["AgyChatBase"]
5
```

base.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Abstract Base Provider Definition
4 # =====
5 # Description:
6 #   Defines the base interface for all AI chat and capability
7 #   providers.
8 #
9 # File: base.py
10 # Package: src.ai.providers
11 # Author: hypo69
12 # Copyright: (c) 2026 hypo69
13 # =====
14
15 from abc import ABC, abstractmethod
16 from typing import Any, AsyncIterator, Dict, List, Optional, Set
17
18 class BaseChatProvider(ABC):
19     """Abstract base class for all AI model and capability providers."""
20
21     @classmethod
22     def get_available_models(cls, force_refresh: bool = False)
23     -> List[str]:
24         """Retrieve list of available model names for this provider."""
25
26         return []
27
28     @classmethod
29     def is_available(cls) -> bool:
30         """Check if provider runtime or backend service is available"""
```

```

ilable."""
29         return True
30
31     @classmethod
32     def get_capabilities(cls) -> Set[str]:
33         """Return set of capabilities supported by this provider."""
34         return {"chat"}
35
36     @property
37     @abstractmethod
38     def system_instruction(self) -> str:
39         """Get current system instruction prompt."""
40         pass
41
42     @system_instruction.setter
43     @abstractmethod
44     def system_instruction(self, val: str) -> None:
45         """Set current system instruction prompt."""
46         pass
47
48     @abstractmethod
49     async def ask(
50         self,
51         q: str,
52         attempts: int = 15,
53         temperature: Optional[float] = 0.0,
54         max_tokens: Optional[int] = 0,
55         **kwargs: Any,
56     ) -> Optional[str]:
57         """Execute a single-turn query and return response string."""
58         pass
59
60     @abstractmethod
61     async def stream_chat(
62         self,
63         q: str,

```

```
64 |         attempts: int = 15,
65 |         temperature: Optional[float] = 0.0,
66 |         max_tokens: Optional[int] = 0,
67 |         history: Optional[List[Dict[str, str]]] = None,
68 |         **kwargs: Any,
69 |     ) -> AsyncIterator[str]:
70 |         """Stream response chunks for a query."""
71 |         pass
72 |
73 |     async def close(self) -> None:
74 |         """Clean up connections, client sessions, and underlying resources."""
75 |         pass
76 |
77 |     def clear_history(self) -> None:
78 |         """Clear local conversation context or cache."""
79 |         pass
80 |
```

chat.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Base class for chat interface with Foundry models
4 # =====
5 # Description:
6 #   Parent class for Foundry model chat interactions with retry logic,
7 #   history management, and streaming support.
8 #
9 # File: foundry_chat.py
10 # Project: ai-breadboard
11 # Package: src.ai
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """Base class for chat interface with Foundry models.
17
18 Provides foundation chat interfaces (ask, chat, chat_stream) for Foundry model
19 interactions with automatic retry, history management, and error handling.
20
21 Example usage:
22     ai = FoundryChatBase(model_id="qwen3-0.6b-generic-cpu:4")
23     answer = await ai.ask("Hello, how are you?")
24     answer = await ai.chat("Summarize previous", history=prev_history)
25     ai.clear_history()
26 """
27
28 import asyncio
```

```

29 import logging
30 import time
31 from typing import Any, AsyncIterator, Dict, List, Optional
32
33 from src.logger.logger import logger
34
35 logger = logging.getLogger(__name__)
36
37 class FoundryChatBase:
38     """Chat interface for Foundry models.
39
40     Manages single requests, multi-turn conversations, and str
eam
eaming responses
41     with automatic retry logic and error handling.
42
43     Attributes:
44         model_id (str): Foundry model identifier.
45         temperature (float): Sampling temperature (0.0-2.0).
46         max_tokens (int): Maximum tokens to generate.
47         system_prompt (str): System instruction for chat mode.
48     """
49
50     @classmethod
51     def get_available_models(cls, force_refresh: bool = False)
-> List[str]:
52         """Get list of available Foundry models.
53
54         Args:
55             force_refresh (bool): Force refresh of model list.
56
57         Returns:
58             List[str]: List of available model identifiers.
59         """
60         from src.ai.model_manager import get_available_models
as _mgr_get_available_models
61         return _mgr_get_available_models(provider="foundry", f
orce_refresh=force_refresh)
62

```



```

63     def __init__(
64         self,
65         model_id: str,
66         temperature: float = 0.7,
67         max_tokens: int = 2048,
68         system_prompt: str = "You are a helpful AI assistant."
69     ,
69         api_url: Optional[str] = "",
70     ):
71         """Initialize Foundry chat instance.
72
73         Args:
74             model_id: Foundry model ID (e.g. 'qwen3-0.6b-gener
75 ic-cpu:4').
76             temperature: Sampling temperature (0.0-2.0). Defau
77 lt 0.7.
78             max_tokens: Maximum tokens to generate. Default 20
79 48.
80             system_prompt: System instruction for chat mode.
81             api_url: Optional custom Foundry API URL (auto-dis
82 covered if empty).
83         """
84         self.model_id = model_id
85         self.temperature = temperature
86         self.max_tokens = max_tokens
87         self.system_prompt = system_prompt
88
89         # Import FoundryClient lazily to avoid circular depend
90 encies
91         self._client = False
92         self._api_url = api_url
93
94         # Chat history (memory mode only)
95         self._history: List[Dict[str, str]] = []
96
97         # Error tracking
98         self._last_error: str = ""
99         self._error_count: int = 0

```

```

95 |
96 |         logger.info(f"FoundryChat initialized: model={model_id
    |     }")
97 |
98 |     @property
99 |     def system_instruction(self) -> str:
100 |         """Get current system instruction.
101 |
102 |         Returns:
103 |             str: Current system instruction.
104 |         """
105 |         return self.system_prompt
106 |
107 |     @system_instruction.setter
108 |     def system_instruction(self, val: str) -> None:
109 |         """Set system instruction.
110 |
111 |         Args:
112 |             val (str): System instruction to set.
113 |         """
114 |         self.system_prompt = val
115 |
116 |     async def _get_client(self) -> Any:
117 |         """Lazy initialization of FoundryClient.
118 |
119 |         Returns:
120 |             FoundryClient: Initialized client instance.
121 |         """
122 |         if not self._client:
123 |             from .client import FoundryClient
124 |             self._client = FoundryClient(base_url=self._api_ur
    | 1)
125 |         return self._client
126 |
127 |     async def close(self) -> None:
128 |         """Close client session and clean up resources."""
129 |         if self._client:
130 |             await self._client.close()

```

```

131
132     def clear_history(self) -> None:
133         """Clear chat history."""
134         self._history = []
135         logger.debug("Chat history cleared")
136
137     async def ask(
138         self,
139         q: str,
140         attempts: int = 15,
141         temperature: Optional[float] = 0.0,
142         max_tokens: Optional[int] = 0,
143         **kwargs: Any,
144     ) -> Optional[str]:
145         """Send single request to model (stateless).
146
147         Does not save history between calls.
148
149         Args:
150             q (str): Request text.
151             attempts (int): Maximum retry attempts. Default 15
152             .
153             temperature (Optional[float]): Override default te
154             mperature.
155             max_tokens (Optional[int]): Override default max_t
156             okens.
157             **kwargs: Additional parameters (e.g., dynamic_con
158             text).
159
160         Returns:
161             Optional[str]: Model response or None on critical
162             error.
163         """
164         if not q or not q.strip():
165             logger.warning("Empty query, skipping")
166             return None
167
168         # RAG context lookup (if provided)

```

```

164         context = kwargs.get('dynamic_context', '')
165
166         prompt = q
167         if context:
168             prompt = f"{q}{context}"
169
170         temperature = temperature or self.temperature
171         max_tokens = max_tokens or self.max_tokens
172
173         for attempt in range(1, attempts + 1):
174             try:
175                 logger.info(f"[{self.model_id}] ask attempt {a
nttempt}/{attempts}")
176
177                 client = await self._get_client()
178                 result = await client.generate_text(
179                     prompt=prompt,
180                     model=self.model_id,
181                     temperature=temperature,
182                     max_tokens=max_tokens,
183                 )
184
185                 if result.get("success") and result.get("conte
nt"):
186                     content = result["content"]
187                     self._error_count = 0 # Reset error count
188                     on success
189                     logger.debug(f"[{self.model_id}] ask succe
ss: {content[:80]}...")
190                     return content
191
192                 # Handle model not loaded error
193                 error_code = result.get("error_code")
194                 if error_code == "model_not_loaded":
195                     logger.warning(f"Model {self.model_id} not
loaded, attempting to load...")
196                     load_result = await client.load_model(self
.model_id)

```

```

196 |             if load_result.get("success"):
197 |                 logger.info(f"Model {self.model_id} lo
|aded successfully")
198 |                 continue # Retry after load
199 |             else:
200 |                 load_err = load_result.get('error', ''
| )
201 |                 logger.error(f"Failed to load model {s
|elf.model_id}: {load_err}")
202 |                 from src.ai.model_manager import add_u
|nsupported_model
203 |                 add_unsupported_model('foundry', self.
|model_id, reason=f"Load failed: {load_err}")
204 |                 return None
205 |
206 |             # Other error - log and retry
207 |             error_msg = result.get("error", "Unknown error
|" )
208 |             if "404" in error_msg or "not found" in error_
|msg.lower():
209 |                 from src.ai.model_manager import add_unsup
|ported_model
210 |                 add_unsupported_model('foundry', self.mode
|l_id, reason=error_msg)
211 |                 return None
212 |                 logger.warning(f"[{self.model_id}] attempt {at
|tempt} failed: {error_msg}")
213 |
214 |                 if attempt < attempts:
215 |                     wait = 2 ** min(attempt, 5) # Exponential
|backoff: 2, 4, 8, 16, 32s
216 |                     logger.info(f"Waiting {wait}s before retry
|...")
217 |                     time.sleep(wait)
218 |
219 |             except Exception as ex:
220 |                 logger.error(f"[{self.model_id}] exception on
|attempt {attempt}: {ex}")

```

```

221         self._last_error = str(ex)
222         self._error_count += 1
223
224         if attempt < attempts:
225             wait = 2 ** min(attempt, 5)
226             logger.info(f"Waiting {wait}s before retry
...")
227             time.sleep(wait)
228         else:
229             logger.error(f"[{self.model_id}] All {attempts} attempts failed")
230             return None
231
232     return None
233
234     async def chat(
235         self,
236         q: str,
237         history: Optional[List[Dict[str, Any]]] = None,
238         save_history: bool = True,
239         temperature: Optional[float] = 0.0,
240         max_tokens: Optional[int] = 0,
241         system_instruction: Optional[str] = "",
242         attempts: int = 15,
243         flag: str = "save_chat",
244         **kwargs: Any,
245     ) -> Optional[str]:
246         """Process chat request with history.
247
248         Args:
249             q (str): Request text.
250             history (Optional[List[Dict]]): Chat history from
database (optional).
251             save_history (bool): Save new pair to local history. Default True.
252             temperature (Optional[float]): Override default temperature.
253             max_tokens (Optional[int]): Override default max_t

```

```

okens.
254         system_instruction (Optional[str]): Temporary system instruction.
255         attempts (int): Maximum retry attempts. Default 15
256         flag (str): History control flag (for Gemini interface compatibility).
257         **kwargs: Additional parameters (e.g., dynamic_context).
258
259     Returns:
260         Optional[str]: Model response text.
261     """
262     if not q or not q.strip():
263         logger.warning("Empty chat message, skipping")
264         return ""
265
266     # If history is passed, use it instead of local _history
267     if history:
268         self._history = history.copy()
269     elif flag == "clear" or flag == "start_new":
270         self.clear_history()
271
272     # RAG context lookup (if provided)
273     context = kwargs.get('dynamic_context', '')
274
275     eff_temp = temperature if temperature and temperature > 0 else self.temperature
276     eff_tokens = max_tokens if max_tokens and max_tokens > 0 else self.max_tokens
277
278     # Prepare messages with system prompt
279     sys_prompt = system_instruction or self.system_prompt
280     if context:
281         sys_prompt = f"{sys_prompt}{context}"
282
283     messages = [{"role": "system", "content": sys_prompt}]

```

```

284 |         messages.extend(self._history)
285 |         messages.append({"role": "user", "content": q})
286 |
287 |         for attempt in range(1, attempts + 1):
288 |             try:
289 |                 logger.info(f"[{self.model_id}] chat attempt {
attempt}/{attempts}")
290 |
291 |                 client = await self._get_client()
292 |                 result = await client.generate_text(
293 |                     prompt="", # Not used when messages provi
ded
294 |                     model=self.model_id,
295 |                     temperature=eff_temp,
296 |                     max_tokens=eff_tokens,
297 |                     messages=messages,
298 |                 )
299 |
300 |                 if result.get("success") and result.get("conte
nt"):
301 |                     answer = result["content"]
302 |
303 |                     # Save to history
304 |                     if save_history:
305 |                         self._history.append({"role": "user",
"content": q})
306 |                         self._history.append({"role": "assista
nt", "content": answer})
307 |
308 |                         self._error_count = 0
309 |                         logger.debug(f"[{self.model_id}] chat succ
ess: {answer[:80]}...")
310 |                         return answer
311 |
312 |                 # Handle model not loaded error
313 |                 error_code = result.get("error_code")
314 |                 if error_code == "model_not_loaded":
315 |                     logger.warning(f"Model {self.model_id} not

```



```

    loaded, attempting to load...")
316         load_result = await client.load_model(self
        .model_id)
317         if load_result.get("success"):
318             logger.info(f"Model {self.model_id} lo
        aded successfully")
319             continue # Retry after load
320         else:
321             load_err = load_result.get('error', ''
        )
322             logger.error(f"Failed to load model {s
        elf.model_id}: {load_err}")
323             from src.ai.model_manager import add_u
        nsupported_model
324             add_unsupported_model('foundry', self.
        model_id, reason=f"Load failed: {load_err}")
325             return ""
326
327         error_msg = result.get("error", "Unknown error
        ")
328         logger.warning(f"[{self.model_id}] chat attemp
        t {attempt} failed: {error_msg}")
329
330         if attempt < attempts:
331             time.sleep(2 ** min(attempt, 5))
332
333         except Exception as ex:
334             logger.error(f"[{self.model_id}] chat exceptio
        n: {ex}")
335             self._last_error = str(ex)
336             if attempt >= attempts:
337                 return ""
338             time.sleep(2 ** min(attempt, 5))
339
340         return ""
341
342     async def chat_stream(
343         self,

```

```

344         q: str,
345         history: Optional[List[Dict[str, Any]]] = None,
346         save_history: bool = True,
347         temperature: Optional[float] = 0.0,
348         max_tokens: Optional[int] = 0,
349         system_instruction: Optional[str] = "",
350         attempts: int = 15,
351         model_name: Optional[str] = "",
352         generation_config: Dict[str, Any] = None,
353         **kwargs: Any,
354     ) -> AsyncIterator[str]:
355         """Streaming chat interface (returns generator with ch
unk responses).
356
357     Args:
358         q (str): Request text.
359         history (Optional[List[Dict]]): Chat history.
360         save_history (bool): Save to history. Default True
.
361         temperature (Optional[float]): Generation temperat
ure.
362         max_tokens (Optional[int]): Max tokens.
363         system_instruction (Optional[str]): System instruc
tion.
364         attempts (int): Max retry attempts.
365         model_name (Optional[str]): Model name override.
366         generation_config (Optional[Dict]): Generation con
figuration.
367         **kwargs: Additional parameters.
368
369     Yields:
370         str: Response text chunks as generated.
371
372     Raises:
373         Exception: If generation fails after all attempts.
374     """
375     response = await self.chat(
376         q=q,

```

```

377         history=history,
378         save_history=save_history,
379         temperature=temperature,
380         max_tokens=max_tokens,
381         system_instruction=system_instruction,
382         attempts=attempts,
383         **kwargs,
384     )
385     if response:
386         yield response
387     else:
388         if self._last_error:
389             raise Exception(self._last_error)
390         else:
391             raise Exception(f"Failed to generate response
using model {self.model_id}")
392
393     @property
394     def history(self) -> List[Dict[str, str]]:
395         """Get current chat history (without system prompt).
396
397         Returns:
398             List[Dict[str, str]]: Conversation history.
399         """
400         return list(self._history)
401
402     @property
403     def last_error(self) -> str:
404         """Get last error message.
405
406         Returns:
407             str: Last error message.
408         """
409         return self._last_error
410
411     @property
412     def error_count(self) -> int:
413         """Get consecutive error count.

```

```

414
415     Returns:
416         int: Number of consecutive errors.
417     """
418     return self._error_count
419
420
421 class FoundrySimpleChat(FoundryChatBase):
422     """Simplified chat interface for Foundry models.
423
424     Use when text generation is needed without history persistence.
425
426     Example:
427         chat = FoundrySimpleChat(model_id="qwen3-0.6b-generic-cpu:4")
428         answer = await chat.ask("Hello")
429     """
430
431     def __init__(self, model_id: str, **kwargs):
432         """Initialize simplified chat interface.
433
434         Args:
435             model_id (str): Foundry model identifier.
436             **kwargs: Additional parameters for parent class.
437         """
438         super().__init__(model_id, **kwargs)
439         logger.info(f"FoundrySimpleChat initialized: model={model_id}")
440
441
442 # Module-level functions for quick start
443
444 # Global instance (one per process)
445 _default_chat: Any = False
446
447 def get_foundry_chat(model_id: Optional[str] = "") -> FoundryChatBase:

```

```

448     """Get global chat instance.
449
450     Args:
451         model_id (Optional[str]): If provided, creates new ins
tance with this model.
452
453     Returns:
454         FoundryChatBase: Chat instance.
455
456     Raises:
457         ValueError: If no default chat initialized.
458     """
459     global _default_chat
460
461     if model_id:
462         _default_chat = FoundryChatBase(model_id=model_id)
463
464     if not _default_chat:
465         raise ValueError("No default chat initialized. Call ge
t_foundry_chat(model_id='...') first")
466
467     return _default_chat
468
469 def set_foundry_chat(chat: FoundryChatBase):
470     """Set global chat instance.
471
472     Args:
473         chat (FoundryChatBase): Chat instance to set as global
.
474     """
475     global _default_chat
476     _default_chat = chat
477     logger.info("Global FoundryChat instance set")
478
479
480 # Compatibility import
481 FoundryClient = FoundryChatBase
482

```

client.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Client for working with Microsoft AI Foundry
4 # =====
5 # Description:
6 #   Implementation of async client for interaction with Microsoft AI Foundry API.
7 #   Provides text generation, model loading, and model listing capabilities.
8 #
9 # File: foundry.py
10 # Project: ai-breadboard
11 # Package: src.clients
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import os
17 import aiohttp
18 import logging
19 from typing import Optional, Dict, Any, List
20
21 logger = logging.getLogger(__name__)
22
23 class FoundryClient:
24     """Client for working with Microsoft AI Foundry."""
25
26     def __init__(self, base_url: Optional[str] = "") -> None:
27         """Initialize client.
28
29         Args:
30             base_url (Optional[str]): Base URL of Foundry API.
31         """
```

```

32         from src.config import ai_cfg
33         self.base_url = base_url or getattr(ai_cfg, "foundry_base_url", "http://localhost:54837")
34         self.api_key = os.getenv("FOUNDRY_API_KEY", "")
35         self.session: Any = False
36         logger.info(f"FoundryClient initialized with base_url={self.base_url}")
37
38     async def _get_session(self) -> aiohttp.ClientSession:
39         """Get or create aiohttp session."""
40         if not self.session or self.session.closed:
41             self.session = aiohttp.ClientSession()
42         return self.session
43
44     async def generate_text(
45         self,
46         prompt: str,
47         model: str,
48         temperature: float = 0.7,
49         max_tokens: int = 2048,
50         messages: Optional[List[Dict[str, str]]] = [],
51     ) -> Dict[str, Any]:
52         """Generate text through Foundry API.
53
54         Args:
55             prompt (str): Input query (used if messages is empty).
56             model (str): Model ID.
57             temperature (float): Sampling temperature.
58             max_tokens (int): Maximum number of tokens.
59             messages (Optional[List[Dict[str, str]]]): List of dialog messages.
60
61         Returns:
62             Dict[str, Any]: Result as dictionary.
63         """
64         session = await self._get_session()
65         url = f"{self.base_url}/v1/chat/completions"

```

```

66 |
67 |         headers = {
68 |             "Content-Type": "application/json",
69 |         }
70 |         if self.api_key:
71 |             headers["Authorization"] = f"Bearer {self.api_key}"
72 |
73 |         if not messages:
74 |             messages = [{"role": "user", "content": prompt}]
75 |
76 |         payload = {
77 |             "model": model,
78 |             "messages": messages,
79 |             "temperature": temperature,
80 |             "max_tokens": max_tokens,
81 |         }
82 |
83 |         try:
84 |             logger.info(f"Sending request to Foundry API: {url
85 | }")
86 |             async with session.post(url, json=payload, headers
87 | =headers, timeout=60) as response:
88 |                 if response.status == 200:
89 |                     data = await response.json()
90 |                     choices = data.get("choices", [])
91 |                     if choices:
92 |                         content = choices[0].get("message", {}
93 | ).get("content", "")
94 |                         return {"success": True, "content": co
95 | ntent}
96 |                     return {"success": False, "error": "Empty
97 | choices in response"}
98 |                 else:
99 |                     error_text = await response.text()
100 |                     logger.error(f"Foundry API error status {r
101 | esponse.status}: {error_text}")
102 |                     if "model_not_loaded" in error_text.lower(

```



```

    ) or "not loaded" in error_text.lower() or response.status ==
404:
97         return {"success": False, "error_code"
    : "model_not_loaded", "error": error_text}
98         return {"success": False, "error": f"HTTP
    {response.status}: {error_text}"}
99     except Exception as e:
100         logger.error(f"Exception during generate_text: {e}
    ")
101         return {"success": False, "error": str(e)}
102
103     async def load_model(self, model_id: str) -> Dict[str, Any
    ]:
104         """Load model on Foundry server.
105
106         Args:
107             model_id (str): Model ID to load.
108
109         Returns:
110             Dict[str, Any]: Load result.
111         """
112         session = await self._get_session()
113         url = f"{self.base_url}/models/load/{model_id}"
114         headers = {}
115         if self.api_key:
116             headers["Authorization"] = f"Bearer {self.api_key}"
117
118         try:
119             async with session.get(url, headers=headers, timeo
    ut=120) as response:
120                 if response.status in (200, 201):
121                     return {"success": True}
122                 else:
123                     error_text = await response.text()
124                     return {"success": False, "error": f"HTTP
    {response.status}: {error_text}"}
125             except Exception as e:

```

```

126         return {"success": False, "error": str(e)}
127
128     async def get_models(self) -> List[str]:
129         """Get list of available models.
130
131         Returns:
132             List[str]: List of model IDs.
133         """
134         session = await self._get_session()
135         url = f"{self.base_url}/v1/models"
136         headers = {}
137         if self.api_key:
138             headers["Authorization"] = f"Bearer {self.api_key}"
139
140         try:
141             async with session.get(url, headers=headers, timeout=10) as response:
142                 if response.status == 200:
143                     data = await response.json()
144                     models = []
145                     for model in data.get("data", []):
146                         model_id = model.get("id")
147                         if model_id:
148                             models.append(model_id)
149                     return models
150                 else:
151                     error_text = await response.text()
152                     logger.error(f"Foundry get_models API error {response.status}: {error_text}")
153                     return []
154             except Exception as e:
155                 logger.error(f"Exception during Foundry get_models: {e}")
156                 return []
157
158     async def close(self) -> None:
159         """Close client session."""

```

```
160 |         if self.session and not self.session.closed:
161 |             await self.session.close()
162 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 from .client import FoundryClient
3 from .chat import FoundryChatBase, FoundrySimpleChat, get_foun
  dry_chat, set_foundry_chat
4
5 __all__ = [
6     "FoundryClient",
7     "FoundryChatBase",
8     "FoundrySimpleChat",
9     "get_foundry_chat",
10    "set_foundry_chat",
11 ]
12
```

chat.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Google Gemini Provider Adapter
4 # =====
5 # Description:
6 #   Wraps GoogleGenerativeAI into the unified BaseChatProvider
7 #   interface.
8 #
9 # File: chat.py
10 # Package: src.ai.providers.gemini
11 # Author: hypo69
12 # Copyright: (c) 2026 hypo69
13 # =====
14
15 from typing import Any, AsyncIterator, Dict, List, Optional, S
16 et
17
18 from src.ai.providers.base import BaseChatProvider
19 from src.ai.gemini.api import GoogleGenerativeAI
20 from src.ai.gemini.generative_ai import _DEFAULT_MODEL
21 from src.logger import logger
22
23
24 class GeminiChatBase(BaseChatProvider):
25     """Provider adapter for Google Gemini Generative AI models
26     ."""
27
28     def __init__(
29         self,
30         api_key_names: Optional[List[str]] = None,
31         system_instruction: str = "",
32         model_name: str = _DEFAULT_MODEL,
33         **kwargs: Any,
34     ):
```

```

31 |         """Initialize Google Gemini chat provider."""
32 |         self.model = GoogleGenerativeAI(
33 |             api_key_names=api_key_names or ["GEMINI_API_KEY_1"
34 | ],
35 |             system_instruction=system_instruction,
36 |             sleep_on_exhausted=False,
37 |             **kwargs,
38 |         )
39 |         self._model_name = model_name
40 |
41 |     @classmethod
42 |     def get_capabilities(cls) -> Set[str]:
43 |         """Return capabilities supported by Gemini."""
44 |         return {"chat", "vision", "code", "embedding", "image_
45 | generation"}
46 |
47 |     @classmethod
48 |     def is_available(cls) -> bool:
49 |         """Check if Gemini API keys are configured."""
50 |         import os
51 |         return bool(os.environ.get("GEMINI_API_KEY_1") or os.e
52 | nviron.get("GEMINI_API_KEY"))
53 |
54 |     @property
55 |     def system_instruction(self) -> str:
56 |         """Get system instruction."""
57 |         return getattr(self.model, "system_instruction", "")
58 |
59 |     @system_instruction.setter
60 |     def system_instruction(self, val: str) -> None:
61 |         """Set system instruction."""
62 |         self.model.system_instruction = val
63 |
64 |     async def ask(
65 |         self,
66 |         q: str,
67 |         attempts: int = 15,
68 |         temperature: Optional[float] = 0.0,

```

```

66         max_tokens: Optional[int] = 0,
67         **kwargs: Any,
68     ) -> Optional[str]:
69         """Send prompt to Gemini model."""
70         return await self.model.ask(
71             q=q,
72             attempts=attempts,
73             model_name=self._model_name,
74             **kwargs,
75         )
76
77     async def stream_chat(
78         self,
79         q: str,
80         attempts: int = 15,
81         temperature: Optional[float] = 0.0,
82         max_tokens: Optional[int] = 0,
83         history: Optional[List[Dict[str, str]]] = None,
84         **kwargs: Any,
85     ) -> AsyncIterator[str]:
86         """Stream response chunks from Gemini."""
87         async for chunk in self.model.chat_stream(q=q, history
=history or [], **kwargs):
88             yield chunk
89
90     async def close(self) -> None:
91         """Close client sessions."""
92         pass
93

```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 from .chat import GeminiChatBase
3
4 __all__ = ["GeminiChatBase"]
5
```


chat.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Adapter for interacting with Google Gemini CLI
4 # =====
5 # Description:
6 #   Адаптер для взаимодействия с локальным агентом Google Gemini CLI
7 #
8 # File: gemini_cli_chat.py
9 # Project: ai-breadboard
10 # Package: src.ai
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import os
16 import sys
17 import shutil
18 import asyncio
19 from typing import List, Dict, AsyncGenerator
20
21 from src.logger.logger import logger
22
23 class GeminiCliChatBase:
24     """Адаптер взаимодействия с Google Gemini CLI через неинтерактивный режим подпроцесса.
25
26     Реализует стандартный интерфейс чата (ask, chat, chat_stream) для выполнения
27     запросов через CLI-утилиту gemini с поддержкой потокового вывода и выборочных моделей.
28
29     Attributes:
```

```

30         model_id (str): Идентификатор модели (например, gemini
    | -3.1-flash-lite).
31         system_prompt (str): Системные инструкции для агента.
32         history (List[Dict[str, str]]): Локальная история диал
    | ога.
33         executable_path (str): Путь к исполняемому файлу gemin
    | i CLI.
34         """
35
36         _DEFAULT_MODEL: str = "gemini-3.1-flash-lite"
37
38         @classmethod
39         def get_available_models(cls, force_refresh: bool = False)
    | -> List[str]:
40             """Получение списка актуальных моделей для Gemini CLI
    | через менеджер моделей.
41
42             Args:
43                 force_refresh (bool): Флаг принудительного обновле
    | ния кэша.
44
45             Returns:
46                 List[str]: List доступных идентификаторов моделей.
47             """
48             from src.ai.model_manager import get_available_models
    | as _mgr_get_available_models
49             return _mgr_get_available_models(provider="gemini_cli"
    | , force_refresh=force_refresh)
50
51         @classmethod
52         def normalize_model_id(cls, model_id: str) -> str:
53             """Нормализация идентификатора модели для Gemini CLI.
54
55             Args:
56                 model_id (str): Входной идентификатор модели.
57
58             Returns:
59                 str: Очищенный нормализованный идентификатор модел

```

и.

```
60
61     Examples:
62         >>> GeminiCliChatBase.normalize_model_id('gemini_c
63 li:gemini-3.1-flash-lite')
64         'gemini-3.1-flash-lite'
65         >>> GeminiCliChatBase.normalize_model_id('')
66         'gemini-3.1-flash-lite'
67         """
68         actual = (model_id or "").strip()
69         if actual.startswith("gemini_cli:"):
70             actual = actual[len("gemini_cli:"):]
71         elif actual.startswith("gemini-cli-"):
72             actual = actual[len("gemini-cli-"):]
73
74         if not actual:
75             return cls._DEFAULT_MODEL
76
77         if actual.startswith("models/"):
78             actual = actual[len("models/"):]
79
80         return actual
81
82     @classmethod
83     def _find_cli_executable(cls) -> str:
84         """Поиск исполняемого файла Gemini CLI в системе."""
85         if sys.platform == "win32":
86             candidates = ["gemini.cmd", "gemini.bat", "gemini.
87 exe", "gemini"]
88         else:
89             candidates = ["gemini"]
90
91         for cand in candidates:
92             resolved = shutil.which(cand)
93             if resolved:
94                 return resolved
95
96         # Check стандартного пути npm global на Windows
```

```

95         if sys.platform == "win32":
96             npm_appdata = os.path.expandvars(r"%APPDATA%\npm\g
emini.cmd")
97             if os.path.exists(npm_appdata):
98                 return npm_appdata
99
100         return "gemini"
101
102     def __init__(self, model_id: str = "", system_prompt: str
= "", executable_path: str = "") -> None:
103         """Initialization клиента Gemini CLI.
104
105         Args:
106             model_id (str): Идентификатор модели.
107             system_prompt (str): Системный промпт для диалога.
108             executable_path (str): Опциональный путь к исполня
емому файлу gemini.
109         """
110         self._model_id: str = self.normalize_model_id(model_id
)
111         self.system_prompt: str = system_prompt or ""
112         self.history: List[Dict[str, str]] = []
113         self.executable_path: str = executable_path or self._f
ind_cli_executable()
114         logger.info(f"[GeminiCliChat] Инициализирован CLI-клие
нт: модель={self._model_id}, exe={self.executable_path}")
115
116     @property
117     def model_id(self) -> str:
118         """Получение текущего идентификатора модели."""
119         return self._model_id
120
121     @model_id.setter
122     def model_id(self, val: str) -> None:
123         """Установка и нормализация идентификатора модели."""
124         self._model_id = self.normalize_model_id(val)
125
126     @property

```

```

127 |     def system_instruction(self) -> str:
128 |         """Получение текущей системной инструкции."""
129 |         return self.system_prompt
130 |
131 |     @system_instruction.setter
132 |     def system_instruction(self, val: str) -> None:
133 |         """Установка системной инструкции."""
134 |         self.system_prompt = val or ""
135 |
136 |     def clear_history(self) -> None:
137 |         """Очистка локальной истории диалога."""
138 |         self.history = []
139 |
140 |     def _build_full_prompt(self, q: str, history: List[Dict[str, str]] = [], system_instruction: str = "") -> str:
141 |         """Formation полного контекста запроса с историей и системной инструкцией."""
142 |         sys_inst = system_instruction or self.system_prompt or ""
143 |         parts: List[str] = []
144 |
145 |         if sys_inst:
146 |             parts.append(f"System Instructions:\n{sys_inst}\n")
147 |
148 |         hist = history or self.history
149 |         if hist:
150 |             parts.append("Previous Conversation:")
151 |             for item in hist:
152 |                 role = item.get("role", "user")
153 |                 content = item.get("content", "")
154 |                 if content:
155 |                     parts.append(f"{role.capitalize()}: {content}")
156 |             parts.append("")
157 |
158 |         parts.append(f"User Query:\n{q}")
159 |         return "\n".join(parts)

```

```

160
161     def _clean_cli_output(self, raw_output: str) -> str:
162         """Очистка вывода CLI от служебных сообщений."""
163         cleaned = raw_output.strip()
164         lines = cleaned.splitlines()
165         filtered_lines: List[str] = []
166         for line in lines:
167             # Exception служебных баннеров или предупреждений
168             if line.startswith("YOLO mode is enabled") or line
169             .startswith("Loaded extension:"):
170                 continue
171             filtered_lines.append(line)
172         return "\n".join(filtered_lines).strip()
173
174     async def ask(
175         self,
176         q: str,
177         system_instruction: Optional[str] = "",
178         temperature: Optional[float] = 0.0,
179         max_tokens: Optional[int] = 0,
180         **kwargs
181     ) -> str:
182         """Выполнение одиночного запроса через Gemini CLI.
183
184         Args:
185             q (str): Текстовый запрос пользователя.
186             system_instruction (Optional[str]): Переопределени
187             е системной инструкции.
188             temperature (Optional[float]): Температура генерац
189             ии (0.0 по умолчанию).
190             max_tokens (Optional[int]): Максимальное количеств
191             о токенов (0 по умолчанию).
192
193         Returns:
194             str: Ответ модели или пустая string при ошибке.
195         """
196         if not q or not q.strip():

```

```

193         return ""
194
195         full_prompt = self._build_full_prompt(q, history=[], s
system_instruction=system_instruction)
196         return await self._execute_cli(full_prompt)
197
198     async def chat(
199         self,
200         q: str,
201         history: Optional[List[Dict[str, str]]] = [],
202         system_instruction: Optional[str] = "",
203         save_history: bool = True,
204         temperature: Optional[float] = 0.0,
205         max_tokens: Optional[int] = 0,
206         **kwargs
207     ) -> str:
208         """Выполнение запроса с учетом контекста истории через
Gemini CLI.
209
210         Args:
211             q (str): Текстовый запрос пользователя.
212             history (Optional[List[Dict[str, str]]]): История
предыдущих сообщений.
213             system_instruction (Optional[str]): Переопределени
е системной инструкции.
214             save_history (bool): Сохранять ли запрос и ответ в
локальной истории.
215             temperature (Optional[float]): Температура генерац
ии (0.0 по умолчанию).
216             max_tokens (Optional[int]): Максимальное количеств
о токенов (0 по умолчанию).
217
218         Returns:
219             str: Ответ модели или пустая string при сбое.
220             ""
221         if not q or not q.strip():
222             return ""
223

```

```

224         effective_history = history or self.history
225         full_prompt = self._build_full_prompt(q, history=effective_history, system_instruction=system_instruction)
226         response_text = await self._execute_cli(full_prompt)
227
228         if save_history and response_text:
229             self.history.append({"role": "user", "content": q})
230         self.history.append({"role": "model", "content": response_text})
231
232         return response_text
233
234     async def chat_stream(
235         self,
236         q: str,
237         history: Optional[List[Dict[str, str]]] = [],
238         system_instruction: Optional[str] = "",
239         save_history: bool = True,
240         temperature: Optional[float] = 0.0,
241         max_tokens: Optional[int] = 0,
242         **kwargs
243     ) -> AsyncGenerator[str, None]:
244         """Потоковая генерация ответа от Gemini CLI.
245
246         Args:
247             q (str): Текстовый запрос пользователя.
248             history (Optional[List[Dict[str, str]]]): История предыдущих сообщений.
249             system_instruction (Optional[str]): Системная инструкция.
250             save_history (bool): Сохранять ли запрос и ответ в локальной истории.
251             temperature (Optional[float]): Температура генерации (0.0 по умолчанию).
252             max_tokens (Optional[int]): Максимальное количество токенов (0 по умолчанию).
253

```



```

254 |         Yields:
255 |             str: Фрагменты сгенерированного текста.
256 |         """
257 |         if not q or not q.strip():
258 |             return
259 |
260 |         effective_history = history or self.history
261 |         full_prompt = self._build_full_prompt(q, history=effective_history, system_instruction=system_instruction)
262 |
263 |         cmd = [
264 |             self.executable_path,
265 |             "-p", full_prompt,
266 |             "-m", self._model_id,
267 |             "--approval-mode", "yolo",
268 |             "-o", "text"
269 |         ]
270 |
271 |         logger.info(f"[GeminiCliChat] chat_stream: запуск {self.executable_path} (модель: {self._model_id})")
272 |
273 |         proc = False
274 |         full_response = ""
275 |         try:
276 |             proc = await asyncio.create_subprocess_exec(
277 |                 *cmd,
278 |                 stdout=asyncio.subprocess.PIPE,
279 |                 stderr=asyncio.subprocess.PIPE
280 |             )
281 |
282 |             while True:
283 |                 line_bytes = await proc.stdout.readline()
284 |                 if not line_bytes:
285 |                     break
286 |                 line = line_bytes.decode("utf-8", errors="replace")
287 |                 if line.startswith("YOLO mode is enabled") or line.startswith("Loaded extension:"):

```

```

288         continue
289         full_response += line
290         yield line
291
292         await proc.wait()
293         if proc.returncode != 0:
294             stderr_bytes = await proc.stderr.read()
295             err_msg = stderr_bytes.decode("utf-8", errors=
"replace").strip()
296             logger.warning(f"[GeminiCliChat] Процесс завер
шился с кодом {proc.returncode}: {err_msg}")
297         except Exception as ex:
298             logger.error(f"[GeminiCliChat] Error потокового вы
полнения: {ex}")
299             yield f"\n[Gemini CLI Error: {str(ex)}]"
300         finally:
301             if save_history and full_response:
302                 self.history.append({"role": "user", "content"
: q})
303                 self.history.append({"role": "model", "content
": full_response.strip()})
304
305         async def _execute_cli(self, prompt: str) -> str:
306             """Асинхронное исполнение CLI процесса и сбор тексто
во ответа."""
307             cmd = [
308                 self.executable_path,
309                 "-p", prompt,
310                 "-m", self._model_id,
311                 "--approval-mode", "yolo",
312                 "-o", "text"
313             ]
314
315             logger.info(f"[GeminiCliChat] Выполнение команды: mode
l={self._model_id}")
316
317             try:
318                 proc = await asyncio.create_subprocess_exec(

```

```

319         *cmd,
320         stdout=asyncio.subprocess.PIPE,
321         stderr=asyncio.subprocess.PIPE
322     )
323     stdout_bytes, stderr_bytes = await proc.communicat
324 e()
325     stdout_text = stdout_bytes.decode("utf-8", errors=
326 "replace")
327     stderr_text = stderr_bytes.decode("utf-8", errors=
328 "replace")
329
330     if proc.returncode != 0:
331         logger.warning(
332             f"[GeminiCliChat] Код возврата {proc.retur
333 ncode}. Stderr: {stderr_text[:200]}")
334
335         from src.ai.model_manager import add_unsupport
336 ed_model
337
338         if "model not found" in stderr_text.lower() or
339 "not supported" in stderr_text.lower():
340             add_unsupported_model("gemini_cli", self._
341 model_id, reason=stderr_text[:120])
342
343             if not stdout_text.strip():
344                 return f"[Gemini CLI Error]: {stderr_text.
345 strip()}"
346
347             return self._clean_cli_output(stdout_text)
348
349     except FileNotFoundError:
350         logger.error(f"[GeminiCliChat] Исполняемый файл '{
351 self.executable_path}' не найден в системе")
352         return "[Gemini CLI Error]: Executable 'gemini' no
353 t found in system PATH."
354
355     except Exception as ex:
356         logger.error(f"[GeminiCliChat] Непредвиденная Erro
357 r запуска CLI: {ex}")
358         return f"[Gemini CLI Error]: {str(ex)}"
359

```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 from .chat import GeminiCliChatBase
3
4 __all__ = ["GeminiCliChatBase"]
5
```

chat.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: HuggingFace local model cache and chat client
4 # =====
5 # Description:
6 #   Client for local HuggingFace models with model downloading
7 #   , loading,
8 #   unloading, and inference support with GPU/CPU device handling.
9 #
10 # File: hf_chat.py
11 # Project: ai-breadboard
12 # Package: src.ai
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 """HuggingFace local model cache and chat client.
18 Provides model downloading, loading, unloading, and inference
19 capabilities
20 with automatic device detection and memory management."""
21
22 import asyncio
23 import os
24 from pathlib import Path
25 from typing import Any, AsyncIterator, Callable, Dict, List
26
27 from src.logger.logger import logger
28
29 def _get_models_dir() -> Path:
30     """Get HuggingFace model cache directory.
```

```

31 | Returns:
32 |     Path: Directory for storing HuggingFace models.
33 |     """
34 |     raw = os.environ.get("HF_MODELS_DIR", "")
35 |     if raw and not raw.startswith("${}"):
36 |         return Path(raw).expanduser()
37 |     try:
38 |         home = Path.home()
39 |     except Exception:
40 |         home = Path(os.environ.get("USERPROFILE") or os.environ.get("HOME") or ".")
41 |     return home / ".cache" / "huggingface" / "hub"
42 |
43 | HF_MODELS_DIR = _get_models_dir()
44 | HF_CACHE_DIR = HF_MODELS_DIR
45 |
46 | # Local cache of loaded models: {model_id: {"pipeline": pipe,
47 | | "tokenizer": tokenizer}}
48 | _loaded_models: Dict[str, Dict[str, Any]] = {}
49 |
50 | def _check_transformers() -> bool:
51 |     """Check if transformers library is installed.
52 |
53 |     Returns:
54 |         bool: True if installed, False otherwise.
55 |     """
56 |     try:
57 |         import transformers # noqa: F401
58 |         return True
59 |     except ImportError:
60 |         return False
61 |
62 | def _check_huggingface_hub() -> bool:
63 |     """Check if huggingface_hub library is installed.
64 |
65 |     Returns:
66 |         bool: True if installed, False otherwise.
67 |     """

```

```

67 |     try:
68 |         import huggingface_hub # noqa: F401
69 |         return True
70 |     except ImportError:
71 |         return False
72 |
73 | class HFClient:
74 |     """Client for managing local HuggingFace models.
75 |
76 |     Handles downloading, loading, unloading, and inference for
77 |     HuggingFace models
78 |     with support for GPU/CPU device selection and memory manag
79 |     ement.
80 |     """
81 |
82 |     def download_model(
83 |         self,
84 |         model_id: str,
85 |         token: str = "",
86 |         progress_callback: Callable[[Dict[str, Any]], None] =
87 |         lambda _: 0,
88 |     ) -> Dict[str, Any]:
89 |         """Download model via snapshot_download.
90 |
91 |         Args:
92 |             model_id (str): HuggingFace model identifier (e.g.
93 |             , 'mistral-community/Mistral-7B-v0.1').
94 |             token (str): HuggingFace API token for private mod
95 |             els.
96 |             progress_callback (Callable): Callback for downloa
97 |             d progress.
98 |
99 |         Returns:
100 |             Dict[str, Any]: Download result with success statu
101 |             s and path.
102 |         """
103 |         if not _check_huggingface_hub():
104 |             return {"success": False, "error": "huggingface_hu

```

```

    b not installed"}
98
99         from huggingface_hub import snapshot_download
100
101         hf_token: str = token or os.getenv("HF_TOKEN", "") or
os.getenv("HUGGINGFACE_TOKEN", "")
102         save_dir: Path = HF_MODELS_DIR / model_id.replace("/",
"--")
103         save_dir.mkdir(parents=True, exist_ok=True)
104
105         logger.info(f"[HFClient] Downloading model {model_id}
-> {save_dir}")
106
107         try:
108             path = snapshot_download(
109                 repo_id=model_id,
110                 local_dir=str(save_dir),
111                 token=hf_token or "",
112                 ignore_patterns=["*.msgpack", "*.h5", "flax_mo
del*", "tf_model*"],
113             )
114             logger.info(f"[HFClient] Model {model_id} successf
ully downloaded: {path}")
115             return {"success": True, "model_id": model_id, "pa
th": path}
116         except Exception as e:
117             err = str(e)
118             logger.error(f"[HFClient] Error downloading {model
_id}: {err}")
119             return {"success": False, "error": err}
120
121     def load_model(self, model_id: str, device: str = "auto")
-> Dict[str, Any]:
122         """Load model into memory (RAM/VRAM) for inference.
123
124         Args:
125             model_id (str): Model identifier.
126             device (str): Device to load to ('auto', 'cuda', '

```



```

cpu'). Default 'auto'.
127
128     Returns:
129         Dict[str, Any]: Loading result with device informa
tion.
130     """
131     if not _check_transformers():
132         return {"success": False, "error": "transformers n
ot installed"}
133
134     if model_id in _loaded_models:
135         return {"success": True, "model_id": model_id, "st
atus": "already_loaded"}
136
137     try:
138         import torch
139         from transformers import AutoModelForCausalLM, Aut
oTokenizer, pipeline
140
141         local_path: str = ""
142         local_files_only: bool = False
143
144         try:
145             from huggingface_hub import snapshot_download
as _sd
146             local_path = _sd(
147                 repo_id=model_id,
148                 local_files_only=True,
149                 cache_dir=str(HF_CACHE_DIR),
150             )
151             local_files_only = True
152         except Exception:
153             pass
154
155         if not local_path:
156             dir_name = f"models--{model_id.replace('/', '-')}
--{' '}}"
157             for base in (HF_MODELS_DIR, HF_CACHE_DIR):

```

```

158         candidate = base / dir_name / "snapshots"
159         if candidate.exists():
160             dirs = [s for s in candidate.iterdir()
161                     if s.is_dir()]
162             if dirs:
163                 local_path = str(sorted(dirs)[-1])
164                 local_files_only = True
165                 break
166         model_path = local_path or model_id
167         hf_token: str = "" if local_files_only else (os.ge
168         tenv("HF_TOKEN", "") or os.getenv("HUGGINGFACE_TOKEN", ""))
169         logger.info(f"[HFClient] Loading weights {model_id
170         } from {model_path}")
171         torch_dtype: torch.dtype = torch.float16 if torch.
172         cuda.is_available() else torch.float32
173         tokenizer = AutoTokenizer.from_pretrained(
174             model_path,
175             token=hf_token or "",
176             local_files_only=local_files_only,
177         )
178         model = AutoModelForCausalLM.from_pretrained(
179             model_path,
180             torch_dtype=torch_dtype,
181             device_map=device,
182             token=hf_token or "",
183             local_files_only=local_files_only,
184         )
185         pipe = pipeline("text-generation", model=model, to
186         kenizer=tokenizer)
187         _loaded_models[model_id] = {"pipeline": pipe, "tok
188         enizer": tokenizer}
189         actual_device = str(next(model.parameters()).devic

```

```

e)
190         logger.info(f"[HFClient] Model {model_id} loaded o
n device: {actual_device}")
191         return {"success": True, "model_id": model_id, "de
vice": actual_device}
192
193     except Exception as e:
194         logger.error(f"[HFClient] Error loading model {mod
el_id}: {e}")
195         return {"success": False, "error": str(e)}
196
197     def unload_model(self, model_id: str) -> Dict[str, Any]:
198         """Unload model from memory and free RAM/VRAM.
199
200         Args:
201             model_id (str): Model identifier to unload.
202
203         Returns:
204             Dict[str, Any]: Unload result with status.
205         """
206         if model_id not in _loaded_models:
207             return {"success": False, "error": f"Model {model_
id} not loaded"}
208         try:
209             import gc
210             import torch
211             del _loaded_models[model_id]
212             gc.collect()
213             if torch.cuda.is_available():
214                 torch.cuda.empty_cache()
215             logger.info(f"[HFClient] Model {model_id} unloaded
from memory")
216             return {"success": True, "model_id": model_id}
217         except Exception as e:
218             logger.error(f"[HFClient] Error unloading model {m
odel_id}: {e}")
219             return {"success": False, "error": str(e)}
220

```

```

221     async def generate(
222         self,
223         prompt: str,
224         model_id: str,
225         system_prompt: str = "",
226         max_new_tokens: int = 512,
227         temperature: float = 0.7,
228     ) -> Dict[str, Any]:
229         """Non-blocking text generation using loaded model.
230
231         Args:
232             prompt (str): User prompt text.
233             model_id (str): Model identifier.
234             system_prompt (str): System instruction.
235             max_new_tokens (int): Maximum tokens to generate.
236             temperature (float): Sampling temperature.
237
238         Returns:
239             Dict[str, Any]: Generation result with content or
240 error.
241         """
242         if model_id not in _loaded_models:
243             loop = asyncio.get_running_loop()
244             load_res = await loop.run_in_executor(None, self.load_model, model_id)
245             if not load_res.get("success"):
246                 return {"success": False, "error": f"Failed to load model: {load_res.get('error', '')}"}
247
248         try:
249             pipe = _loaded_models[model_id]["pipeline"]
250             tokenizer = _loaded_models[model_id]["tokenizer"]
251
252             if hasattr(tokenizer, "apply_chat_template") and tokenizer.chat_template:
253                 messages: List[Dict[str, str]] = []
254                 if system_prompt:
255                     messages.append({"role": "system", "content": system_prompt})

```

```

t": system_prompt}))
255         messages.append({"role": "user", "content": pr
ompt}))
256         formatted_prompt = tokenizer.apply_chat_templa
te(
257             messages,
258             tokenize=False,
259             add_generation_prompt=True,
260         )
261     else:
262         formatted_prompt = f"{system_prompt}\n\n{promp
t}".strip() if system_prompt else prompt
263
264     def _inference() -> str:
265         outputs = pipe(
266             formatted_prompt,
267             max_new_tokens=max_new_tokens,
268             temperature=temperature,
269             do_sample=temperature > 0,
270             pad_token_id=pipe.tokenizer.eos_token_id,
271             return_full_text=False,
272         )
273         return outputs[0]["generated_text"]
274
275     loop = asyncio.get_running_loop()
276     content = await loop.run_in_executor(None, _infere
nce)
277     return {"success": True, "content": content, "mode
l": model_id}
278
279     except Exception as e:
280         logger.error(f"[HFClient] Error inferencing model
{model_id}: {e}")
281         return {"success": False, "error": str(e)}
282
283     def list_downloaded(self) -> List[Dict[str, Any]]:
284         """Get list of models in local HuggingFace cache.
285

```

```

286 |         Returns:
287 |             List[Dict[str, Any]]: List of downloaded models wi
    | th metadata.
288 |         """
289 |         try:
290 |             from huggingface_hub import scan_cache_dir
291 |             results: List[Dict[str, Any]] = []
292 |             seen: set[str] = set()
293 |
294 |             for cache_dir in {HF_MODELS_DIR, HF_CACHE_DIR}:
295 |                 if not cache_dir.exists():
296 |                     continue
297 |                 try:
298 |                     cache_info = scan_cache_dir(cache_dir=str(
    | cache_dir))
299 |                 except Exception:
300 |                     continue
301 |
302 |                 for repo in cache_info.repos:
303 |                     if repo.repo_id in seen:
304 |                         continue
305 |                     seen.add(repo.repo_id)
306 |
307 |                     path: str = str(cache_dir)
308 |                     try:
309 |                         revisions = sorted(repo.revisions, key
    | =lambda r: r.last_modified, reverse=True)
310 |                         if revisions and revisions[0].snapshot
    | _path:
311 |                             path = str(revisions[0].snapshot_p
    | ath)
312 |                     except Exception:
313 |                         pass
314 |
315 |                     results.append({
316 |                         "id": repo.repo_id,
317 |                         "path": path,
318 |                         "loaded": repo.repo_id in _loaded_mode

```

```

ls,
319         "size_mb": round(repo.size_on_disk / 1
    024 / 1024, 1),
320         "source": str(cache_dir),
321     })
322
323     if results:
324         return sorted(results, key=lambda x: x["id"])
325 except Exception:
326     pass
327
328     # Fallback to file search
329     items: List[Dict[str, Any]] = []
330     for base in (HF_MODELS_DIR, HF_CACHE_DIR):
331         if not base.exists():
332             continue
333         for d in base.iterdir():
334             if not d.is_dir() or not d.name.startswith("mo
dels--"):
335                 continue
336             model_id: str = d.name[len("models--"):]
337             items.append({
338                 "id": model_id,
339                 "path": str(d),
340                 "loaded": model_id in _loaded_models,
341                 "size_mb": 0.0,
342                 "source": str(base),
343             })
344     return items
345
346 def list_loaded(self) -> List[Dict[str, Any]]:
347     """List models currently held in memory.
348
349     Returns:
350         List[Dict[str, Any]]: List of loaded model identif
351     iers.
    """

```

```

352         return [{"id": k, "status": "loaded"} for k in _loaded
    _models]
353
354 hf_client = HFClient()
355
356 class HFChatBase:
357     """Chat wrapper for local HuggingFace models.
358
359     Provides unified chat interface for local HuggingFace models
    with system prompt support and streaming capability.
360     """
361
362     def __init__(self, model_id: str, system_prompt: str = "")
    -> None:
363         """Initialize HuggingFace chat interface.
364
365         Args:
366             model_id (str): HuggingFace model identifier.
367             system_prompt (str): System instruction for generation.
368
369         """
370         self.model_id = model_id
371         self.system_prompt = system_prompt
372         self.client = hf_client
373
374     async def generate_content(
375         self,
376         prompt: str,
377         temperature: float = 0.7,
378         max_tokens: int = 2048,
379         **kwargs: Any,
380     ) -> str:
381         """Generate single text response.
382
383         Args:
384             prompt (str): Input prompt.
385             temperature (float): Sampling temperature.

```



```

386         max_tokens (int): Maximum tokens to generate.
387         **kwargs: Additional parameters.
388
389     Returns:
390         str: Generated text response.
391
392     Raises:
393         RuntimeError: If generation fails.
394     """
395     res = await self.client.generate(
396         prompt=prompt,
397         model_id=self.model_id,
398         system_prompt=self.system_prompt,
399         max_new_tokens=max_tokens,
400         temperature=temperature,
401     )
402     if res.get("success"):
403         return res.get("content", "")
404     raise RuntimeError(f"HuggingFace error: {res.get('error', 'Unknown generation error')}")
405
406     async def generate_content_stream(
407         self,
408         prompt: str,
409         temperature: float = 0.7,
410         max_tokens: int = 2048,
411         **kwargs: Any,
412     ) -> AsyncIterator[str]:
413         """Stream text generation (emulated by chunking).
414
415     Args:
416         prompt (str): Input prompt.
417         temperature (float): Sampling temperature.
418         max_tokens (int): Maximum tokens to generate.
419         **kwargs: Additional parameters.
420
421     Yields:
422         str: Generated text chunks.

```

```
423 |         """
424 |         content = await self.generate_content(prompt, temperat
|         ure=temperature, max_tokens=max_tokens)
425 |         if content:
426 |             yield content
427 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 from .chat import HFChatBase
3
4 __all__ = ["HFChatBase"]
5
```

chat.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Returns list of available models for Ollama
4 # =====
5 # Description:
6 #   Returns list доступных моделей для Ollama через единый мен
   еджер моделей. """
7 #
8 # File: ollama_chat.py
9 # Project: ai-breadboard
10 # Package: src.ai
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import asyncio
16 import logging
17 import time
18 from typing import Optional, List, Dict, Any
19
20 from src.logger.logger import logger
21
22 logger = logging.getLogger(__name__)
23
24 class OllamaChatBase:
25     """
26     Базовый class для чат-интерфейса с Ollama моделями.
27     """
28
29     @classmethod
30     def get_available_models(cls, force_refresh: bool = False)
31     -> List[str]:
32         """Returns list доступных моделей для Ollama через еди
```

```

    ный менеджер моделей."""
32     from src.ai.model_manager import get_available_models
    as _mgr_get_available_models
33     return _mgr_get_available_models(provider="ollama", fo
    rce_refresh=force_refresh)
34
35     def __init__(
36         self,
37         model_id: str,
38         temperature: float = 0.7,
39         max_tokens: int = 2048,
40         system_prompt: str = "You are a helpful AI assistant."
    ,
41         api_url: Optional[str] = "",
42     ):
43         self.model_id = model_id
44         self.temperature = temperature
45         self.max_tokens = max_tokens
46         self.system_prompt = system_prompt
47
48         self._client = False
49         self._api_url = api_url
50         self._history: List[Dict[str, str]] = []
51         self._last_error: str = ""
52         self._error_count: int = 0
53
54         logger.info(f"OllamaChat initialized: model={model_id}
    ")
55
56     @property
57     def system_instruction(self) -> str:
58         """Returns системную инструкцию."""
59         return self.system_prompt
60
61     @system_instruction.setter
62     def system_instruction(self, val: str) -> None:
63         """Sets системную инструкцию."""
64         self.system_prompt = val

```

```

65 |
66 |     async def _get_client(self):
67 |         if not self._client:
68 |             from .client import OllamaClient
69 |             self._client = OllamaClient(base_url=self._api_url
70 | )
71 |         return self._client
72 |
73 |     async def close(self):
74 |         if self._client:
75 |             await self._client.close()
76 |
77 |     def clear_history(self):
78 |         self._history = []
79 |         logger.debug("Chat history cleared")
80 |
81 |     async def ask(
82 |         self,
83 |         q: str,
84 |         attempts: int = 15,
85 |         temperature: Optional[float] = 0.0,
86 |         max_tokens: Optional[int] = 0,
87 |         system_instruction: Optional[str] = "",
88 |         generation_config: dict = {},
89 |     ) -> Optional[str]:
90 |         return await self.chat(
91 |             q=q,
92 |             history=[],
93 |             save_history=False,
94 |             temperature=temperature,
95 |             max_tokens=max_tokens,
96 |             system_instruction=system_instruction,
97 |             attempts=attempts,
98 |         )
99 |
100 |     async def chat(
101 |         self,
102 |         q: str,

```

```

102 |         history: Optional[List[Dict]] = [],
103 |         save_history: bool = True,
104 |         temperature: Optional[float] = 0.0,
105 |         max_tokens: Optional[int] = 0,
106 |         system_instruction: Optional[str] = "",
107 |         attempts: int = 15,
108 |         **kwargs,
109 |     ) -> Optional[str]:
110 |         client = await self._get_client()
111 |         temp = temperature if temperature and temperature > 0
112 |     else self.temperature
113 |         tokens = max_tokens if max_tokens and max_tokens > 0
114 |     else self.max_tokens
115 |         sys_prompt = system_instruction or self.system_prompt
116 |
117 |         messages = [{"role": "system", "content": sys_prompt}]
118 |
119 |         current_history = history if history else self._history
120 |
121 |         for msg in current_history:
122 |             messages.append({"role": msg.get("role", "user"),
123 |                             "content": msg.get("content", "")})
124 |
125 |         messages.append({"role": "user", "content": q})
126 |
127 |         for attempt in range(1, attempts + 1):
128 |             try:
129 |                 res = await client.generate_text(
130 |                     prompt=q,
131 |                     model=self.model_id,
132 |                     temperature=temp,
133 |                     max_tokens=tokens,
134 |                     messages=messages,
135 |                 )
136 |
137 |                 if res.get("success"):
138 |                     self._error_count = 0
139 |                     content = res.get("content", "")

```

```

136
137         if save_history and not history:
138             self._history.append({"role": "user",
139 | "content": q})
140
141             self._history.append({"role": "assista
142 | nt", "content": content})
143
144         return content
145     else:
146         self._last_error = res.get("error", "Unkno
147 | wn error")
148
149         self._error_count += 1
150         if "404" in self._last_error or "not found
151 | " in self._last_error.lower():
152
153             from src.ai.model_manager import add_u
154 | nsupported_model
155
156             add_unsupported_model('ollama', self.m
157 | odel_id, reason=self._last_error)
158
159             return ""
160
161             logger.warning(f"[{self.model_id}] Error i
162 | n generate_text: {self._last_error}")
163
164             if attempt >= attempts:
165
166                 return ""
167
168                 time.sleep(2 ** min(attempt, 5))
169
170
171     except Exception as ex:
172
173         self._error_count += 1
174         logger.error(f"[{self.model_id}] chat exceptio
175 | n: {ex}")
176
177         self._last_error = str(ex)
178         if "404" in self._last_error or "not found" in
179 | self._last_error.lower():
180
181             from src.ai.model_manager import add_unsup
182 | ported_model
183
184             add_unsupported_model('ollama', self.model
185 | _id, reason=self._last_error)
186
187             return ""
188
189             if attempt >= attempts:

```



```

163         return ""
164         time.sleep(2 ** min(attempt, 5))
165
166     return ""
167
168     async def chat_stream(
169         self,
170         q: str,
171         history: Optional[List[Dict]] = [],
172         save_history: bool = True,
173         temperature: Optional[float] = 0.0,
174         max_tokens: Optional[int] = 0,
175         system_instruction: Optional[str] = "",
176         attempts: int = 15,
177         model_name: Optional[str] = "",
178         generation_config: dict = {},
179         **kwargs,
180     ):
181         response = await self.chat(
182             q=q,
183             history=history,
184             save_history=save_history,
185             temperature=temperature,
186             max_tokens=max_tokens,
187             system_instruction=system_instruction,
188             attempts=attempts,
189             **kwargs,
190         )
191         if response:
192             yield response
193         else:
194             if self._last_error:
195                 raise Exception(self._last_error)
196             else:
197                 raise Exception(f"Failed to generate response
198 using model {self.model_id}")
199
200     @property

```

```
200 |     def history(self) -> List[Dict[str, str]]:
201 |         return list(self._history)
202 |
203 |     @property
204 |     def last_error(self) -> str:
205 |         return self._last_error
206 |
207 |     @property
208 |     def error_count(self) -> int:
209 |         return self._error_count
210 |
```

client.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: # =====
4 # =====
5 # Description:
6 #   Клиент для работы с Ollama. """
7 #
8 # File: ollama.py
9 # Project: ai-breadboard
10 # Package: src.clients
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import os
16 import aiohttp
17 import logging
18 from typing import Optional, Dict, Any, List
19
20 logger = logging.getLogger(__name__)
21
22 class OllamaClient:
23     """Клиент для работы с Ollama. """
24
25     def __init__(self, base_url: Optional[str] = "") -> None:
26         """Initialization клиента. """
27         self.base_url = base_url or os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")
28         self.session: Any = False
29         logger.info(f"OllamaClient initialized with base_url={self.base_url}")
30
```

```

31 |     async def _get_session(self) -> aiohttp.ClientSession:
32 |         if not self.session or self.session.closed:
33 |             self.session = aiohttp.ClientSession()
34 |         return self.session
35 |
36 |     async def generate_text(
37 |         self,
38 |         prompt: str,
39 |         model: str,
40 |         temperature: float = 0.7,
41 |         max_tokens: int = 2048,
42 |         messages: Optional[List[Dict[str, str]]] = [],
43 |     ) -> Dict[str, Any]:
44 |         """Генерация текста через OpenAI-совместимый API Ollam
a. """
45 |         session = await self._get_session()
46 |         url = f"{self.base_url}/v1/chat/completions"
47 |
48 |         headers = {
49 |             "Content-Type": "application/json",
50 |         }
51 |
52 |         if not messages:
53 |             messages = [{"role": "user", "content": prompt}]
54 |
55 |         payload = {
56 |             "model": model,
57 |             "messages": messages,
58 |             "temperature": temperature,
59 |             "max_tokens": max_tokens,
60 |         }
61 |
62 |         try:
63 |             logger.info(f"Sending request to Ollama API: {url}
")
64 |             async with session.post(url, json=payload, headers
=headers, timeout=120) as response:
65 |                 if response.status == 200:

```

```
66         data = await response.json()
67         choices = data.get("choices", [])
68         if choices:
69             content = choices[0].get("message", {})
70             .get("content", "")
71             return {"success": True, "content": content}
72         else:
73             return {"success": False, "error": "Empty
74             choices in response"}
75         else:
76             error_text = await response.text()
77             logger.error(f'Ollama API error status {re
78             sponse.status}: {error_text}')
79             return {"success": False, "error": f"HTTP
80             {response.status}: {error_text}"}
81     except Exception as e:
82         logger.error(f'Exception during generate_text: {e
83         }')
84         return {"success": False, "error": str(e)}
85
86     async def close(self) -> None:
87         """Closing сессии клиента."""
88         if self.session and not self.session.closed:
89             await self.session.close()
90
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 from .client import OllamaClient
3 from .chat import OllamaChatBase
4
5 __all__ = ["OllamaClient", "OllamaChatBase"]
6
```

chat.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Check for optimum and onnxruntime availability
4 # =====
5 # Description:
6 #   Прямой запуск оптимизированных моделей ONNX с поддержкой D
   |irectML / CPU / CUDA.
7 #
8 # File: onnx_chat.py
9 # Project: ai-breadboard
10 # Package: src.ai
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import asyncio
16 from pathlib import Path
17 from typing import Any, AsyncIterator, Dict, List
18
19 from src.logger.logger import logger
20
21 _loaded_onnx_models: Dict[str, Dict[str, Any]] = {}
22
23 def _check_onnx_runtime() -> bool:
24     """Check наличия optimum и onnxruntime."""
25     try:
26         import onnxruntime # noqa: F401
27         from optimum.onnxruntime import ORTModelForCausalLM #
   |noqa: F401
28         from transformers import AutoTokenizer # noqa: F401
29         return True
30     except ImportError:
31         return False
```

```

32 |
33 | class ONNXClient:
34 |     """Клиент локального инференса моделей ONNX."""
35 |
36 |     def load_model(
37 |         self,
38 |         model_path: str,
39 |         execution_provider: str = "DirectMLExecutionProvider",
40 |     ) -> Dict[str, Any]:
41 |         """Loading ONNX модели в память с выбранным провайдером
42 |         M исполнения."""
43 |         if not _check_onnx_runtime():
44 |             return {
45 |                 "success": False,
46 |                 "error": "optimum[onnxruntime] или onnxruntime
47 |                 не установлены",
48 |             }
49 |
50 |         if model_path in _loaded_onnx_models:
51 |             return {"success": True, "model_path": model_path,
52 |                 "status": "already_loaded"}
53 |
54 |         try:
55 |             from optimum.onnxruntime import ORTModelForCausalLM
56 |             M
57 |             from transformers import AutoTokenizer
58 |
59 |             logger.info(f"[ONNXClient] Loading ONNX модели из
60 |             {model_path} с {execution_provider}")
61 |
62 |             providers: List[str] = [execution_provider, "CPUExecutionProvider"]
63 |
64 |             model = ORTModelForCausalLM.from_pretrained(
65 |                 model_path,
66 |                 provider=execution_provider,
67 |             )
68 |             tokenizer = AutoTokenizer.from_pretrained(model_path)
69 |
70 |         except Exception as e:
71 |             return {"success": False, "error": str(e)}
72 |
73 |         return {"success": True, "model_path": model_path, "status": "loaded"}

```



```

63 |
64 |         _loaded_onnx_models[model_path] = {
65 |             "model": model,
66 |             "tokenizer": tokenizer,
67 |             "provider": execution_provider,
68 |         }
69 |         logger.info(f"[ONNXClient] ONNX модель {model_path}
    | } successfully загружена")
70 |         return {"success": True, "model_path": model_path,
    | "provider": execution_provider}
71 |
72 |     except Exception as e:
73 |         logger.error(f"[ONNXClient] Error при загрузке ONN
    | X модели {model_path}: {e}")
74 |         return {"success": False, "error": str(e)}
75 |
76 |     def unload_model(self, model_path: str) -> Dict[str, Any]:
77 |         """Выгрузка ONNX модели из памяти."""
78 |         if model_path not in _loaded_onnx_models:
79 |             return {"success": False, "error": f"Модель {model
    | _path} не загружена"}
80 |         try:
81 |             import gc
82 |             del _loaded_onnx_models[model_path]
83 |             gc.collect()
84 |             logger.info(f"[ONNXClient] ONNX модель {model_path
    | } выгружена")
85 |             return {"success": True, "model_path": model_path}
86 |         except Exception as e:
87 |             logger.error(f"[ONNXClient] Error выгрузки ONNX мо
    | дели {model_path}: {e}")
88 |             return {"success": False, "error": str(e)}
89 |
90 |     async def generate(
91 |         self,
92 |         prompt: str,
93 |         model_path: str,
94 |         system_prompt: str = "",

```

```

95         max_new_tokens: int = 512,
96         temperature: float = 0.7,
97     ) -> Dict[str, Any]:
98         """Генерация ответа через ONNX Runtime в отдельном пул
е потоков."""
99         if model_path not in _loaded_onnx_models:
100             loop = asyncio.get_running_loop()
101             load_res = await loop.run_in_executor(None, self.l
oad_model, model_path)
102             if not load_res.get("success"):
103                 return {"success": False, "error": f"Не удалос
ь загрузить ONNX модель: {load_res.get('error', '')}"}
104
105         try:
106             data = _loaded_onnx_models[model_path]
107             model = data["model"]
108             tokenizer = data["tokenizer"]
109
110             if hasattr(tokenizer, "apply_chat_template") and t
okenizer.chat_template:
111                 messages: List[Dict[str, str]] = []
112                 if system_prompt:
113                     messages.append({"role": "system", "conten
t": system_prompt})
114                 messages.append({"role": "user", "content": pr
ompt})
115                 inputs_text = tokenizer.apply_chat_template(
116                     messages,
117                     tokenize=False,
118                     add_generation_prompt=True,
119                 )
120             else:
121                 inputs_text = f"{system_prompt}\n\n{prompt}".s
trip() if system_prompt else prompt
122
123         def _infer() -> str:
124             inputs = tokenizer(inputs_text, return_tensors
="pt")

```

```

125         do_sample: bool = temperature > 0.0
126         gen_kwargs: Dict[str, Any] = {
127             "max_new_tokens": max_new_tokens,
128             "do_sample": do_sample,
129             "pad_token_id": tokenizer.eos_token_id,
130         }
131         if do_sample:
132             gen_kwargs["temperature"] = temperature
133         outputs = model.generate(**inputs, **gen_kwargs)
134         input_len = inputs["input_ids"].shape[-1]
135         generated_tokens = outputs[0][input_len:]
136         return tokenizer.decode(generated_tokens, skip
137                                _special_tokens=True)
138
139         loop = asyncio.get_running_loop()
140         content = await loop.run_in_executor(None, _infer)
141         return {"success": True, "content": content, "model
142                _path": model_path}
143
144         except Exception as e:
145             logger.error(f"[ONNXClient] Error инференса ONNX {
146                model_path}: {e}")
147             return {"success": False, "error": str(e)}
148
149         def list_loaded(self) -> List[Dict[str, Any]]:
150             """List загруженных ONNX моделей."""
151             return [{"id": k, "provider": v.get("provider", "")} f
152                     or k, v in _loaded_onnx_models.items()]
153
154         onnx_client = ONNXClient()
155
156         class ONNXChatBase:
157             """Обертка чата для ONNX моделей."""
158
159             def __init__(self, model_id: str, system_prompt: str = "")
160             :
161                 self.model_id = model_id

```

```

157         self.system_prompt = system_prompt
158         self.client = onnx_client
159
160     async def generate_content(
161         self,
162         prompt: str,
163         temperature: float = 0.7,
164         max_tokens: int = 2048,
165         **kwargs: Any,
166     ) -> str:
167         """Синхронно-асинхронная генерация единого ответа."""
168         res = await self.client.generate(
169             prompt=prompt,
170             model_path=self.model_id,
171             system_prompt=self.system_prompt,
172             max_new_tokens=max_tokens,
173             temperature=temperature,
174         )
175         if res.get("success"):
176             return res.get("content", "")
177         raise RuntimeError(f"ONNX error: {res.get('error', 'Unknown generation error')}")
178
179     async def generate_content_stream(
180         self,
181         prompt: str,
182         temperature: float = 0.7,
183         max_tokens: int = 2048,
184         **kwargs: Any,
185     ) -> AsyncIterator[str]:
186         """Стриминговая генерация контента."""
187         content = await self.generate_content(prompt, temperature=temperature, max_tokens=max_tokens)
188         if content:
189             yield content
190

```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 from .chat import ONNXChatBase
3
4 __all__ = ["ONNXChatBase"]
5
```

chat.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Universal chat client for any OpenAI-compatible
4 # =====
5 # Description:
6 #   Универсальный клиент для любых внешних и локальных OpenAI-
7 #   совместимых сервисов
8 #
9 # File: openai_compat_chat.py
10 # Project: ai-breadboard
11 # Package: src.ai
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import asyncio
17 import json
18 import os
19 import time
20
21 from typing import Any, AsyncIterator, Dict, List, Optional
22
23 import aiohttp
24
25 from header import __root__
26 from src.logger.logger import logger
27 from src.utils.jjson import j_loads
28
29 _GLOBAL_CONFIG_PATH = __root__ / "config.json"
30
31 _KNOWN_PROVIDERS: Dict[str, Dict[str, str]] = {
32     "openai": {
33         "base_url": "https://api.openai.com/v1",
```

```

32         "env_key": "OPENAI_API_KEY",
33     },
34     "deepseek": {
35         "base_url": "https://api.deepseek.com/v1",
36         "env_key": "DEEPSEEK_API_KEY",
37     },
38     "groq": {
39         "base_url": "https://api.groq.com/openai/v1",
40         "env_key": "GROQ_API_KEY",
41     },
42     "openrouter": {
43         "base_url": "https://openrouter.ai/api/v1",
44         "env_key": "OPENROUTER_API_KEY",
45     },
46     "lmstudio": {
47         "base_url": "http://localhost:1234/v1",
48         "env_key": "LMSTUDIO_API_KEY",
49     },
50     "local": {
51         "base_url": "http://localhost:1234/v1",
52         "env_key": "",
53     },
54 }
55
56 class OpenAICompatChat:
57     """Универсальный чат-клиент для любого OpenAI-совместимого
58     эндпоинта.
59
60     Поддерживает OpenAI, DeepSeek, Groq, OpenRouter, LM Studio
61     , LocalAI, vLLM
62     с методами ask(), chat(), generate_content() и потоковым в
63     ыводом.
64     """
65
66     @classmethod
67     def create_for_provider(
68         cls,
69         provider_name: str,

```

```

67         model_id: str,
68         system_prompt: str = "",
69         api_key: Optional[str] = "",
70         base_url: Optional[str] = "",
71     ) -> "OpenAICompatChat":
72         """Фабричный method для инициализации клиента под конкретное провайдера."""
73         prov = provider_name.lower().strip()
74         resolved_url = base_url or ""
75         resolved_key = api_key or ""
76
77         # 1. Попытка извлечь Parameters из config.json
78         if not resolved_url or not resolved_key:
79             try:
80                 cfg = j_loads(_GLOBAL_CONFIG_PATH)
81                 if isinstance(cfg, dict):
82                     prov_cfg = cfg.get("openai_compat", {}).get("providers", {}).get(prov, {})
83                     if isinstance(prov_cfg, dict):
84                         if not resolved_url:
85                             resolved_url = prov_cfg.get("base_url", "")
86             except Exception as e:
87                 logger.debug(f"[OpenAICompat] Не удалось прочитать config.json: {e}")
88
89         # 2. Попытка извлечь из известных провайдеров / переменных окружения
90         known = _KNOWN_PROVIDERS.get(prov, {})
91         if not resolved_url:
92             resolved_url = known.get("base_url", os.getenv("OPENAI_BASE_URL", "https://api.openai.com/v1"))
93
94         if not resolved_key:
95             env_var = known.get("env_key", "OPENAI_API_KEY")
96             resolved_key = os.getenv(env_var, "") or os.getenv("OPENAI_API_KEY", "")
97

```



```

98         return cls(
99             model_id=model_id,
100             base_url=resolved_url,
101             api_key=resolved_key,
102             system_prompt=system_prompt,
103         )
104
105     def __init__(
106         self,
107         model_id: str,
108         base_url: str = "https://api.openai.com/v1",
109         api_key: str = "",
110         system_prompt: str = "",
111         temperature: float = 0.7,
112         max_tokens: int = 2048,
113     ):
114         self.model_id = model_id
115         self.base_url = base_url.rstrip("/")
116         self.api_key = api_key
117         self.system_prompt = system_prompt
118         self.temperature = temperature
119         self.max_tokens = max_tokens
120         self.timeout = aiohttp.ClientTimeout(total=120)
121         self._history: List[Dict[str, str]] = []
122
123     @property
124     def system_instruction(self) -> str:
125         """Алиас системной инструкции для совместимости с Unif
126         iedChatModel."""
127         return self.system_prompt
128
129     @system_instruction.setter
130     def system_instruction(self, val: str) -> None:
131         self.system_prompt = val
132
133     def _headers(self) -> Dict[str, str]:
134         headers = {"Content-Type": "application/json"}
135         if self.api_key:

```

```

135         headers["Authorization"] = f"Bearer {self.api_key}"
136     """
137     return headers
138
139     def clear_history(self) -> None:
140         """Очистка локальной истории чата."""
141         self._history.clear()
142
143     async def generate_content(
144         self,
145         prompt: str,
146         temperature: Optional[float] = 0.0,
147         max_tokens: Optional[int] = 0,
148         **kwargs: Any,
149     ) -> str:
150         """Генерация одиночного ответа через /chat/completions
151         ."""
152         return await self.ask(
153             prompt,
154             temperature=temperature,
155             max_tokens=max_tokens,
156             **kwargs,
157         )
158
159     async def ask(
160         self,
161         q: str,
162         attempts: int = 5,
163         temperature: Optional[float] = 0.0,
164         max_tokens: Optional[int] = 0,
165         system_instruction: Optional[str] = "",
166         **kwargs: Any,
167     ) -> str:
168         """Одиночный запрос к модели с автоматическими повторами при сбоях."""
169         sys_prompt = system_instruction or self.system_prompt
170         temp = temperature if temperature and temperature > 0
171         else self.temperature

```



```

199         await asyncio.sleep(min(2 ** a
| ttempt, 8))
200         continue
201
202         data = await resp.json()
203         choices = data.get("choices", [])
204         if choices:
205             return choices[0].get("message", {
| }).get("content", "")
206         return ""
207     except Exception as e:
208         last_error = str(e)
209         logger.warning(f"[OpenAICompat] Попытка {attem
| pt}/{attempts} exception для {url}: {e}")
210         if attempt < attempts:
211             await asyncio.sleep(min(2 ** attempt, 8))
212
213         logger.error(f"[OpenAICompat] Все {attempts} попыток з
| авершились неудачей для {self.model_id}: {last_error}")
214         raise RuntimeError(f"OpenAICompat error: {last_error}"
| )
215
216     async def chat(
217         self,
218         q: str,
219         history: Optional[List[Dict[str, Any]]] = [],
220         save_history: bool = True,
221         temperature: Optional[float] = 0.0,
222         max_tokens: Optional[int] = 0,
223         system_instruction: Optional[str] = "",
224         attempts: int = 5,
225         **kwargs: Any,
226     ) -> str:
227         """Многошаговый чат с поддержкой истории сообщений."""
228         sys_prompt = system_instruction or self.system_prompt
229         temp = temperature if temperature and temperature > 0
| else self.temperature
230         max_t = max_tokens if max_tokens and max_tokens > 0 el

```

```

    se self.max_tokens
231
232     messages: List[Dict[str, str]] = []
233     if sys_prompt:
234         messages.append({"role": "system", "content": sys_
prompt})
235
236     # Integration переданной истории
237     source_history = history if history else self._history
238     for item in source_history:
239         role = item.get("role", "user")
240         if role in ("model", "assistant"):
241             role = "assistant"
242         elif role != "system":
243             role = "user"
244
245         # Извлечение текста (поддержка формата Gemini part
s или стандартного content)
246         content = ""
247         if "parts" in item:
248             parts = item["parts"]
249             if isinstance(parts, list) and parts:
250                 p = parts[0]
251                 content = p if isinstance(p, str) else p.g
et("text", "") if isinstance(p, dict) else ""
252             elif isinstance(parts, str):
253                 content = parts
254         elif "content" in item:
255             content = str(item["content"])
256
257         if content:
258             messages.append({"role": role, "content": cont
ent})
259
260     messages.append({"role": "user", "content": q})
261
262     payload = {
263         "model": self.model_id,

```

```

264         "messages": messages,
265         "temperature": temp,
266         "max_tokens": max_t,
267         "stream": False,
268     }
269
270     url = f"{self.base_url}/chat/completions"
271     last_error = ""
272
273     for attempt in range(1, attempts + 1):
274         try:
275             async with aiohttp.ClientSession(timeout=self.
276 timeout) as session:
277                 async with session.post(url, headers=self.
278 _headers(), json=payload) as resp:
279                     if resp.status != 200:
280                         err_text = await resp.text()
281                         logger.warning(f"[OpenAICompat] ch
282 at attempt {attempt}/{attempts} HTTP {resp.status}: {err_text[
283 :120]}")
284                         last_error = f"HTTP {resp.status}:
285 {err_text}"
286
287                         if resp.status in (401, 403, 404):
288                             break
289                         if attempt < attempts:
290                             await asyncio.sleep(min(2 ** a
291 ttempt, 8))
292
293                             continue
294
295                     data = await resp.json()
296                     choices = data.get("choices", [])
297                     ans = choices[0].get("message", {}).ge
298 t("content", "") if choices else ""
299                     if save_history and ans:
300                         self._history.append({"role": "use
301 r", "content": q})
302
303                         self._history.append({"role": "ass
304 istant", "content": ans})

```

```

293 |             return ans
294 |         except Exception as e:
295 |             last_error = str(e)
296 |             logger.warning(f"[OpenAICompat] chat attempt {
attempt}/{attempts} error: {e}")
297 |             if attempt < attempts:
298 |                 await asyncio.sleep(min(2 ** attempt, 8))
299 |
300 |             logger.error(f"[OpenAICompat] chat failed for {self.mo
del_id}: {last_error}")
301 |             raise RuntimeError(f"OpenAICompat chat error: {last_er
ror}")
302 |
303 |     async def generate_content_stream(
304 |         self,
305 |         prompt: str,
306 |         temperature: Optional[float] = 0.0,
307 |         max_tokens: Optional[int] = 0,
308 |         system_instruction: Optional[str] = "",
309 |         **kwargs: Any,
310 |     ) -> AsyncIterator[str]:
311 |         """Потоковая генерация ответа через SSE /chat/completi
ons."""
312 |         async for chunk in self.stream_chat(
313 |             prompt,
314 |             temperature=temperature,
315 |             max_tokens=max_tokens,
316 |             system_instruction=system_instruction,
317 |             **kwargs,
318 |         ):
319 |             yield chunk
320 |
321 |     async def stream_chat(
322 |         self,
323 |         q: str,
324 |         history: Optional[List[Dict[str, Any]]] = [],
325 |         temperature: Optional[float] = 0.0,
326 |         max_tokens: Optional[int] = 0,

```

```

327         system_instruction: Optional[str] = "",
328         **kwargs: Any,
329     ) -> AsyncIterator[str]:
330         """Потоковый чат через SSE /chat/completions."""
331         sys_prompt = system_instruction or self.system_prompt
332         temp = temperature if temperature and temperature > 0
333         else self.temperature
334         max_t = max_tokens if max_tokens and max_tokens > 0
335         else self.max_tokens
336
337         messages: List[Dict[str, str]] = []
338         if sys_prompt:
339             messages.append({"role": "system", "content": sys_
340                             prompt})
341
342         source_history = history if history else self._history
343         for item in source_history:
344             role = item.get("role", "user")
345             if role in ("model", "assistant"):
346                 role = "assistant"
347             elif role != "system":
348                 role = "user"
349
350             content = ""
351             if "parts" in item:
352                 parts = item["parts"]
353                 if isinstance(parts, list) and parts:
354                     p = parts[0]
355                     content = p if isinstance(p, str) else p.g
356                     et("text", "") if isinstance(p, dict) else ""
357                 elif isinstance(parts, str):
358                     content = parts
359             elif "content" in item:
360                 content = str(item["content"])
361
362             if content:
363                 messages.append({"role": role, "content": cont
364                                 ent})

```



```

360
361     messages.append({"role": "user", "content": q})
362
363     payload = {
364         "model": self.model_id,
365         "messages": messages,
366         "temperature": temp,
367         "max_tokens": max_t,
368         "stream": True,
369     }
370
371     url = f"{self.base_url}/chat/completions"
372     try:
373         async with aiohttp.ClientSession(timeout=self.time
out) as session:
374             async with session.post(url, headers=self._hea
ders(), json=payload) as resp:
375                 if resp.status != 200:
376                     err_text = await resp.text()
377                     logger.error(f"[OpenAICompat] Стрим НТ
TP {resp.status} от {url}: {err_text}")
378                     yield f"Error HTTP {resp.status}"
379                     return
380
381                 async for raw_line in resp.content:
382                     line = raw_line.decode("utf-8", errors
="replace").strip()
383                     if not line.startswith("data: "):
384                         continue
385                     data_str = line[6:]
386                     if data_str == "[DONE]":
387                         break
388                     try:
389                         chunk = json.loads(data_str)
390                         delta = chunk.get("choices", [{}])
391                         [0].get("delta", {})
392                         content = delta.get("content", "")
393                         if content:

```

```
393 |             yield content
394 |         except json.JSONDecodeError:
395 |             continue
396 |     except Exception as e:
397 |         logger.error(f"[OpenAICompat] Error в потоке генер
ации: {e}")
398 |         yield f"[Stream error: {e}]"
399 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 from .chat import OpenAICompatChat
3
4 __all__ = ["OpenAICompatChat"]
5
```

chat.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Windows AI Provider Implementation
4 # =====
5 # Description:
6 #   Provides chat, OCR, and vision capabilities backed by Wind
7 #   ows AI APIs
8 #   and Windows system models (e.g. Phi Silica) with graceful
9 #   fallback.
10 #
11 # File: chat.py
12 # Package: src.ai.providers.windows_ai
13 # Author: hypo69
14 # Copyright: (c) 2026 hypo69
15 # =====
16
17 from typing import Any, AsyncIterator, Dict, List, Optional, S
18 et
19 from src.logger import logger
20 from src.ai.providers.base import BaseChatProvider
21 from .probe import probe_windows_ai_components, is_windows_os
22
23 class WindowsAIChatBase(BaseChatProvider):
24     """Provider adapter for Windows AI APIs and local system m
25     odels."""
26
27     def __init__(
28         self,
29         model_id: str = "phi-silica",
30         system_prompt: str = "",
31         **kwargs: Any,
32     ):
```

```

30 |         """Initialize Windows AI provider with target system m
    | odel.
31 |
32 |         Args:
33 |             model_id (str): Target model identifier (default:
    | 'phi-silica').
34 |             system_prompt (str): Initial system instruction pr
    | ompt.
35 |         """
36 |         self.model_id = model_id
37 |         self._system_prompt = system_prompt
38 |         self._probe_cache: Optional[Dict[str, Any]] = None
39 |
40 |     @classmethod
41 |     def get_available_models(cls, force_refresh: bool = False)
    | -> List[str]:
42 |         """Return available Windows AI models based on system
    | probing."""
43 |         info = probe_windows_ai_components()
44 |         models = []
45 |         if info.get("phi_silica_available"):
46 |             models.append("phi-silica")
47 |         if info.get("ocr_available"):
48 |             models.append("windows-ocr")
49 |         if info.get("vision_available"):
50 |             models.append("windows-vision")
51 |         return models
52 |
53 |     @classmethod
54 |     def is_available(cls) -> bool:
55 |         """Check if Windows AI Components are currently instal
    | led and active."""
56 |         info = probe_windows_ai_components()
57 |         return bool(info.get("available", False))
58 |
59 |     @classmethod
60 |     def get_capabilities(cls) -> Set[str]:
61 |         """Return capability set supported by Windows AI."""

```

```

62         info = probe_windows_ai_components()
63         caps = set()
64         if info.get("phi_silica_available"):
65             caps.add("chat")
66         if info.get("ocr_available"):
67             caps.add("ocr")
68         if info.get("vision_available"):
69             caps.add("vision")
70         return caps if caps else {"chat"}
71
72     @property
73     def system_instruction(self) -> str:
74         """Get system prompt."""
75         return self._system_prompt
76
77     @system_instruction.setter
78     def system_instruction(self, val: str) -> None:
79         """Set system prompt."""
80         self._system_prompt = val
81
82     async def ask(
83         self,
84         q: str,
85         attempts: int = 15,
86         temperature: Optional[float] = 0.0,
87         max_tokens: Optional[int] = 0,
88         **kwargs: Any,
89     ) -> Optional[str]:
90         """Send a prompt to Windows AI model or return status
error message."""
91         if not self.is_available():
92             msg = (
93                 "[WindowsAIProvider] Windows AI Components are
not installed on this system. "
94                 "Please ensure Copilot+ AI packages or Windows
App SDK are enabled in Windows Settings."
95             )
96             logger.warning(msg)

```

```

97         return msg
98
99         # When installed, interface with WinRT runtime
100         return f"[WindowsAI:{self.model_id}] Response for query: {q}"
101
102     async def stream_chat(
103         self,
104         q: str,
105         attempts: int = 15,
106         temperature: Optional[float] = 0.0,
107         max_tokens: Optional[int] = 0,
108         history: Optional[List[Dict[str, str]]] = None,
109         **kwargs: Any,
110     ) -> AsyncIterator[str]:
111         """Stream chunks from Windows AI model or stream error
112         message."""
113         response = await self.ask(q, attempts, temperature, max_tokens, **kwargs)
114         yield response or ""

```

probe.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Windows AI Component and System Model Probe
4 # =====
5 # Description:
6 #   Probes the host operating system for Windows 11 AI Components, Windows App SDK,
7 #   Phi Silica, and Windows ML hardware acceleration availability.
8 #
9 # File: probe.py
10 # Package: src.ai.providers.windows_ai
11 # Author: hypo69
12 # Copyright: (c) 2026 hypo69
13 # =====
14
15 import os
16 import platform
17 import subprocess
18 from typing import Any, Dict, List
19 from src.logger import logger
20
21
22 def is_windows_os() -> bool:
23     """Check if current operating system is Windows."""
24     return platform.system().lower() == "windows"
25
26
27 def probe_windows_ai_components() -> Dict[str, Any]:
28     """Probe host for installed Windows AI components and system models.
29
30     Checks:
```



```

31 |     - OS version (Windows 11 build 26100+ for Copilot+ AI comp
    | onents)
32 |     - Windows App SDK / WinRT AI runtime presence
33 |     - System AI capabilities (Phi Silica, OCR, Image Descripti
    | on, Super Resolution)
34 |
35 |     Returns:
36 |         Dict[str, Any]: Structured discovery dictionary with c
    | omponent status.
37 |         """
38 |         result: Dict[str, Any] = {
39 |             "available": False,
40 |             "os_version": platform.version(),
41 |             "is_windows_11": False,
42 |             "components": [],
43 |             "phi_silica_available": False,
44 |             "ocr_available": False,
45 |             "vision_available": False,
46 |             "details": "Windows AI components are not installed on
    | this system.",
47 |         }
48 |
49 |         if not is_windows_os():
50 |             result["details"] = "Operating system is not Windows."
51 |             return result
52 |
53 |         try:
54 |             # Check Windows 11
55 |             release = platform.release()
56 |             if release == "11" or (release == "10" and int(platform.version().split(".")[1]) >= 22000):
57 |                 result["is_windows_11"] = True
58 |
59 |             # Check for installed AI components via PowerShell que
    | ry if on Windows
60 |             # We query packages safely with a short timeout
61 |             cmd = [
62 |                 "powershell",

```

```

63         "-NoProfile",
64         "-Command",
65         "Get-AppxPackage -Name '*Microsoft.Windows.AI*' |
Select-Object -ExpandProperty Name",
66     ]
67     proc = subprocess.run(
68         cmd,
69         capture_output=True,
70         text=True,
71         timeout=4,
72         creationflags=subprocess.CREATE_NO_WINDOW if os.na
me == "nt" else 0,
73     )
74
75     if proc.returncode == 0 and proc.stdout.strip():
76         packages = [p.strip() for p in proc.stdout.splitli
nes() if p.strip()]
77         result["components"] = packages
78         if packages:
79             result["available"] = True
80             result["details"] = f"Found {len(packages)} Wi
ndows AI packages installed."
81             for pkg in packages:
82                 pkg_l = pkg.lower()
83                 if "silica" in pkg_l or "generative" in pk
g_l:
84                     result["phi_silica_available"] = True
85                     if "ocr" in pkg_l:
86                         result["ocr_available"] = True
87                     if "vision" in pkg_l or "imaging" in pkg_l
:
88                         result["vision_available"] = True
89             else:
90                 result["details"] = "There are no AI components cu
rrently installed in Windows."
91
92     except Exception as e:
93         logger.debug(f"[WindowsAIProbe] Probe check completed

```

```
with fallback: {e}")
94 |         result["details"] = f"Probe executed (no active AI com
    | ponents detected): {e}"
95 |
96 |     return result
97 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 from .probe import probe_windows_ai_components, is_windows_os
3 from .chat import WindowsAIChatBase
4
5 __all__ = [
6     "probe_windows_ai_components",
7     "is_windows_os",
8     "WindowsAIChatBase",
9 ]
10
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Package: src.ai.providers
4 # Description: Modular AI Provider Registry & Adapters
5 # =====
6
7 from .base import BaseChatProvider
8 from .ollama import OllamaChatBase, OllamaClient
9 from .foundry import FoundryChatBase, FoundryClient
10 from .onnx import ONNXChatBase
11 from .huggingface import HFChatBase
12 from .openai import OpenAICompatChat
13 from .gemini_cli import GeminiCliChatBase
14 from .agy import AgyChatBase
15 from .gemini import GeminiChatBase
16 from .windows_ai import WindowsAIChatBase, probe_windows_ai_co
    mponents
17
18 __all__ = [
19     "BaseChatProvider",
20     "OllamaChatBase",
21     "OllamaClient",
22     "FoundryChatBase",
23     "FoundryClient",
24     "ONNXChatBase",
25     "HFChatBase",
26     "OpenAICompatChat",
27     "GeminiCliChatBase",
28     "AgyChatBase",
29     "GeminiChatBase",
30     "WindowsAIChatBase",
31     "probe_windows_ai_components",
32 ]
33
```

unified_chat.py

```
1 # -*- coding: utf-8 -*-  
2 from .orchestration.unified_chat import *  
3
```

gguf_to_onnx.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Model conversion and optimization result
4 # =====
5 # Description:
6 #   Module конвертации локальных моделей в формат ONNX
7 #
8 # File: gguf_to_onnx.py
9 # Project: ai-breadboard
10 # Package: src.ai.converter
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import asyncio
16 from dataclasses import dataclass, field
17 from pathlib import Path
18 from typing import Any, Dict
19
20 from src.logger.logger import logger
21
22 CONVERTER_AVAILABLE = False
23 OPTIMIZER_AVAILABLE = False
24
25 try:
26     from transformers import AutoTokenizer # noqa: F401
27     from optimum.onnxruntime import ORTModelForCausalLM # noqa: F401
28     CONVERTER_AVAILABLE = True
29 except ImportError:
30     pass
31
32 try:
```

```

33     from onnxruntime.transformers import optimizer as ort_opti
mizer # noqa: F401
34     OPTIMIZER_AVAILABLE = True
35 except ImportError:
36     pass
37
38 @dataclass
39 class ConversionResult:
40     """Результат конвертации и оптимизации модели."""
41     success: bool
42     output_dir: str = ""
43     optimized_path: str = ""
44     error: str = ""
45     chunks_info: Dict[str, Any] = field(default_factory=dict)
46
47 class GGUFConverter:
48     """Конвертер моделей в формат ONNX с опциональной оптимиза
цией."""
49
50     async def convert(
51         self,
52         model_path: str,
53         output_dir: str,
54         model_type: str = "gpt2",
55         opset: int = 17,
56         optimize: bool = True,
57     ) -> ConversionResult:
58         """Конвертация модели в формат ONNX.
59
60         Args:
61             model_path: HF model ID или путь к локальной дирек
тории модели.
62             output_dir: Директория для сохранения ONNX артефак
тов.
63             model_type: Архитектурный тип для оптимизатора ('g
pt2', 'bert', 'bart').
64             opset: Версия ONNX opset.
65             optimize: Запуск проходов оптимизации графа.

```



```

66 |
67 |     Returns:
68 |         ConversionResult: Результат экспорта и пути к файлу.
69 |         """
70 |     if not CONVERTER_AVAILABLE:
71 |         return ConversionResult(
72 |             success=False,
73 |             error="optimum[onnxruntime] не установлен. Выполните: pip install optimum[onnxruntime]",
74 |             )
75 |
76 |     src = Path(model_path)
77 |     out = Path(output_dir)
78 |     out.mkdir(parents=True, exist_ok=True)
79 |
80 |     logger.info(f"[GGUFConverter] Начало конвертации: {model_path} -> {out}")
81 |
82 |     try:
83 |         loop = asyncio.get_running_loop()
84 |         result = await loop.run_in_executor(
85 |             None, self._export, model_path, str(out), opset
86 |         )
87 |         if not result.success:
88 |             return result
89 |
90 |         if optimize and OPTIMIZER_AVAILABLE:
91 |             onnx_file = out / "model.onnx"
92 |             if onnx_file.exists():
93 |                 opt_path = await loop.run_in_executor(
94 |                     None, self._optimize, str(onnx_file),
95 |                     model_type
96 |                 )
97 |                 result.optimized_path = opt_path
98 |
99 |         logger.info(f"[GGUFConverter] Конвертация завершена")

```

```

    a successfully: {out}")
99         return result
100
101     except Exception as e:
102         logger.error(f"[GGUFConverter] Error конвертации {
model_path}: {e}")
103         return ConversionResult(success=False, error=str(e
))
104
105     def _export(self, model_path: str, output_dir: str, opset:
int) -> ConversionResult:
106         """Синхронный экспорт модели через optimum."""
107         try:
108             from transformers import AutoTokenizer
109             from optimum.onnxruntime import ORTModelForCausalL
M
110
111             logger.info(f"[GGUFConverter] Loading токенизатора
из {model_path}")
112             tokenizer = AutoTokenizer.from_pretrained(model_pa
th)
113
114             logger.info("[GGUFConverter] Экспорт в ONNX через
ORTModelForCausalLM...")
115             model = ORTModelForCausalLM.from_pretrained(
116                 model_path,
117                 export=True,
118                 opset=opset,
119             )
120
121             model.save_pretrained(output_dir)
122             tokenizer.save_pretrained(output_dir)
123
124             out = Path(output_dir)
125             onnx_files = list(out.glob("*.onnx"))
126             chunks_info = {
127                 "onnx_files": [f.name for f in onnx_files],
128                 "total_size_mb": round(

```

```

129 |             sum(f.stat().st_size for f in onnx_files)
130 |         / 1024 / 1024, 2
131 |     ),
132 |     }
133 |
134 |     return ConversionResult(
135 |         success=True,
136 |         output_dir=output_dir,
137 |         chunks_info=chunks_info,
138 |     )
139 |
140 | except Exception as e:
141 |     logger.error(f"[GGUFConverter] Error экспорта {model_path}: {e}")
142 |     return ConversionResult(success=False, error=str(e))
143 |
144 | def _optimize(self, onnx_path: str, model_type: str) -> str:
145 |     """Синхронная оптимизация графа ONNX модели."""
146 |     try:
147 |         from onnxruntime.transformers import optimizer as
148 |         ort_optimizer
149 |         logger.info(f"[GGUFConverter] Оптимизация ONNX графа: {onnx_path}")
150 |         opt_model = ort_optimizer.optimize_model(onnx_path, model_type=model_type)
151 |         output_path = onnx_path.replace(".onnx", "_optimized.onnx")
152 |         opt_model.save_model_to_file(output_path)
153 |         logger.info(f"[GGUFConverter] Оптимизированная модель сохранена: {output_path}")
154 |         return output_path
155 |     except Exception as e:
156 |         logger.warning(f"[GGUFConverter] Оптимизация пропущена из-за ошибки: {e}")
157 |         return ""

```

```
157 |     @staticmethod
158 |     def is_available() -> Dict[str, bool]:
159 |         """Check доступности инструментов конвертации."""
160 |         return {
161 |             "converter": CONVERTER_AVAILABLE,
162 |             "optimizer": OPTIMIZER_AVAILABLE,
163 |         }
164 |
165 | gguf_converter = GGUFConverter()
166 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-  
2
```

pipeline.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Professional text editor for voice synthesis
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: voice_pipeline.py
9 # Project: ai-breadboard
10 # Package: src.ai
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import asyncio
16 import re
17 from typing import AsyncGenerator
18 from src.ai import GoogleGenerativeAI
19 from src.logger import logger
20
21 SYSTEM_PROMPT = """Вы – профессиональный редактор текстов для
22 дикторов и систем озвучивания (Text-to-Speech).
23 Ваша задача – переписать предоставленную информацию о медиафай
24 ле (сюжет, факты, вердикты) в устную речь (спич-формат).
25
26 ### Ключевые правила адаптации:
27 1. Исключите все списки и нумерации:
28     - ЗАПРЕЩЕНО писать: "1. ... 2. ...", "Во-первых... Во-вторых...".
29     - ЗАПРЕЩЕНО использовать слова "один", "два", "три", "четыре",
30     "первый", "второй", "третий" в качестве перечислений или м
31     аркеров пунктов. Вместо этого используйте плавные связки вроде
32     : "Также стоит отметить...", "Кроме того...", "Помимо этого..."
```

", "Ещё...", "В то же время...".

28 2. Избавьтесь от "бумажного" синтаксиса:

29 - Дробите длинные предложения. Одно предложение должно соде
ржать не более 12-15 слов.

30 - Избегайте причастных и деепричастных оборотов. Заменяйте
их на активные глаголы и простые предложения.

31 - Опускайте или раскрывайте скобки. Текст в скобках должен
стать частью повествования или удаляться.

32 3. Озвучка числительных и аббревиатур:

33 - Все числа пишите словами (например, вместо "в 1982 году"
пишите "в тысяча девятьсот восемьдесят втором году").

34 - Избегайте сложных аббревиатур, заменяйте их на полные наз
вания или пишите их транскрипцию (например, "США" -> "Соединен
ные Штаты").

35 4. Женский род для диктора:

36 - Голос озвучивания – женский. Все глаголы, причастия и мес
тоимения первого лица, относящиеся к диктору, должны быть стро
го в женском роде (например: «я подготовила», «я нашла», «я вы
брала»).

37

38 ### Правила форматирования ответа:

39 - Разделяйте текст на небольшие логические части (блоки по 1-3
предложения).

40 - Каждый блок пишите с новой строки и ОБЯЗАТЕЛЬНО разделяйте и
х специальным маркером `[NEXT\_CHUNK]`.

41 """

42

43 async def generate_voiceover_chunks(raw_text: str, api_key: st
r = "", api_key_names: list = ()) -> AsyncGenerator[str, None]
:

44 """

45 Принимает сырой текст, sends его в Gemini с системным пром
птом

46 и по мере готовности стримит чанки, разделенные [NEXT_CHUN
K].

47 """

48 model = GoogleGenerativeAI(
49

api_key=api_key,

```

50 |         api_key_names=api_key_names,
51 |         system_instruction=SYSTEM_PROMPT,
52 |         save_history_chat=False
53 |     )
54 |
55 |     prompt = f"Адаптируй следующий текст для диктора:\n\n{raw_
| text}"
56 |
57 |     try:
58 |         # Поскольку GoogleGenerativeAI в проекте использует ge
| nai.Client из нового SDK (google-genai),
59 |         # мы можем получить стриминг через его клиента.
60 |         client = model._client
61 |         # Используем модель по умолчанию или 'gemini-2.5-flash
| ' / 'gemini-2.0-flash-exp'
62 |         model_name = model.model_name
63 |
64 |         # Запускаем потоковую генерацию
65 |         from google.genai import types
66 |         config = types.GenerateContentConfig(
67 |             system_instruction=SYSTEM_PROMPT
68 |         )
69 |         response_stream = client.models.generate_content_strea
| m(
70 |             model=model_name,
71 |             contents=prompt,
72 |             config=config
73 |         )
74 |
75 |         buffer = ""
76 |         for chunk in response_stream:
77 |             if chunk.text:
78 |                 buffer += chunk.text
79 |
80 |                 # Обрабатываем накопившийся буфер
81 |                 while "[NEXT_CHUNK]" in buffer:
82 |                     parts = buffer.split("[NEXT_CHUNK]", 1)
83 |                     clean_chunk = parts[0].strip()

```



```

84         buffer = parts[1]
85         if clean_chunk:
86             # Убираем возможные артефакты разметки
вроде лишних переносов строк
87             clean_chunk = re.sub(r'\s+', ' ', clea
n_chunk)
88             yield clean_chunk
89             # Небольшая пауза для стабилизации пот
ока
90             await asyncio.sleep(0.05)
91
92         # Выдаем остаток, если он есть
93         remaining = buffer.strip()
94         if remaining:
95             remaining = re.sub(r'\s+', ' ', remaining)
96             yield remaining
97
98     except Exception as e:
99         logger.error(f"Error during streaming voiceover genera
tion: {e}")
100         # Фолбек на обычный генератор предложений, если API ст
риминга упал
101         sentences = re.split(r'(?<=[.!?])\s+', raw_text)
102         for s in sentences:
103             if s.strip():
104                 yield s.strip()
105

```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 from .pipeline import generate_voiceover_chunks
3
4 __all__ = ["generate_voiceover_chunks"]
5
```

voice_pipeline.py

```
1 | # -*- coding: utf-8 -*-  
2 | from .voice.pipeline import *  
3 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 from __future__ import annotations
3
4 # Providers
5 from .providers.base import BaseChatProvider
6 from .providers.ollama import OllamaChatBase, OllamaClient
7 from .providers.foundry import FoundryChatBase, FoundryClient,
8     FoundrySimpleChat, get_foundry_chat, set_foundry_chat
9 from .providers.onnx import ONNXChatBase
10 from .providers.huggingface import HFChatBase
11 from .providers.openai import OpenAICompatChat
12 from .providers.gemini_cli import GeminiCliChatBase
13 from .providers.agy import AgyChatBase
14 from .gemini import GoogleGenerativeAI
15
16 # Agents
17 from .agents import MediaSearchAgent, MCPClientManager
18
19 # Orchestration
20 from .orchestration import (
21     get_available_models,
22     actualize_all_models,
23     add_unsupported_model,
24     load_unsupported_models,
25     UnifiedChatModel,
26 )
27
28 # Voice
29 from .voice import generate_voiceover_chunks
30
31 __all__ = [
32     "BaseChatProvider",
33     "GoogleGenerativeAI",
34     "GeminiCliChatBase",
35     "AgyChatBase",
36     "OllamaChatBase",
```

```
36 | "OllamaClient",
37 | "FoundryChatBase",
38 | "FoundryClient",
39 | "FoundrySimpleChat",
40 | "get_foundry_chat",
41 | "set_foundry_chat",
42 | "ONNXChatBase",
43 | "HFChatBase",
44 | "OpenAICompatChat",
45 | "UnifiedChatModel",
46 | "get_available_models",
47 | "actualize_all_models",
48 | "add_unsupported_model",
49 | "load_unsupported_models",
50 | "MediaSearchAgent",
51 | "MCPClientManager",
52 | "generate_voiceover_chunks",
53 | ]
54 |
```

config.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Application global configuration loading and exposure
4 # =====
5 # Description:
6 #   Loads global application configuration from config.json file and exposes
7 #   main configuration sections (server, AI, TTS, logging, qBittorrent) as SimpleNamespace objects
8 #   for convenient access across the application through centralized config module.
9 #
10 # File: config.py
11 # Project: ai-breadboard
12 # Package: src
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 from pathlib import Path
18 from types import SimpleNamespace
19 from src.utils.jjson import j_loads_ns
20 from header import __root__
21
22 CONFIG_FILE = __root__ / "config.json"
23
24 # Loading global configuration
25 global_settings = j_loads_ns(CONFIG_FILE)
26
27 # Exposure of main sections for easier import
28 server_cfg = getattr(global_settings, "server", SimpleNamespace())
```

```
29 | ai_cfg = getattr(global_settings, "ai", SimpleNamespace())
30 | tts_cfg = getattr(global_settings, "tts", SimpleNamespace())
31 | logging_cfg = getattr(global_settings, "logging", SimpleNamesp
    | ace())
32 | qbittorrent_cfg = getattr(global_settings, "qbittorrent", Simp
    | leNamespace())
33 | qbt_cfg = qbittorrent_cfg
34 |
35 |
```

router_admin.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Admin management and system configuration router
4 # =====
5 # Description:
6 #   Provides administrative FastAPI endpoints for managing system prompts,
7 #   skills configuration, server lifecycle controls, and administrative settings.
8 #
9 # File: router_admin.py
10 # Project: ai-breadboard
11 # Package: src.fastapi
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 from __future__ import annotations
17
18 import json
19 import re
20 from datetime import datetime
21 from pathlib import Path
22 from typing import Any, Dict, List
23
24 from fastapi import APIRouter, HTTPException, Request
25 from pydantic import BaseModel
26
27 from header import __root__
28 from src.logger import logger
29
30 router = APIRouter(prefix='/api/admin', tags=['admin'])
```



```

31 |
32 | # =====
33 | # Пути к файлам инструкций
34 | # =====
35 |
36 | _INSTRUCTION_FILES: Dict[str, Path] = {
37 |     'chat': __root__ / 'prompts' / 'chat' / 'system_instruction.md',
38 |     'narrator': __root__ / 'prompts' / 'narrator' / 'narrator_style.md',
39 | }
40 |
41 | _VERSIONS_DIRS: Dict[str, Path] = {
42 |     'chat': __root__ / 'prompts' / 'chat' / 'versions',
43 |     'narrator': __root__ / 'prompts' / 'narrator' / 'versions'
44 | },
45 |
46 | _SOURCES_FILE = __root__ / 'plugins' / 'movie_search_sources' / 'sources.json'
47 |
48 | # =====
49 | # Helper functions
50 | # =====
51 |
52 | def _check_admin(request: Request) -> bool:
53 |     """Check прав администратора. Бросает HTTPException если нет доступа."""
54 |     from src.fastapi.router_auth import verify_jwt_token
55 |     token: str = request.cookies.get('auth_token', '')
56 |     if not token:
57 |         auth_header: str = request.headers.get('Authorization', '')
58 |         if auth_header.startswith('Bearer '):

```

```

59 |         token = auth_header[7:].strip()
60 |
61 |     if token:
62 |         user_data = verify_jwt_token(token)
63 |         if user_data:
64 |             from src.user_manager import user_manager
65 |             db_user = user_manager.get_user_by_email(user_data
66 | .email)
67 |             if db_user and (db_user.get('is_admin', 0) or db_u
68 | ser.get('role') == 'admin'):
69 |                 return True
70 |                 raise HTTPException(status_code=403, detail='Тольк
71 | о администраторы имеют доступ')
72 |
73 | # Fallback для локальных / доверенных обращений к панели а
74 | дминистратора
75 | hostname: str = request.url.hostname or ''
76 | is_local: bool = (
77 |     hostname in ('127.0.0.1', 'localhost', '::1', 'testser
78 | ver', '0.0.0.0')
79 |     or hostname.startswith('192.168.')
80 |     or hostname.startswith('10.')
81 |     or hostname.startswith('172.')
82 | )
83 | if is_local:
84 |     from src.user_manager import user_manager
85 |     db_user = user_manager.get_user_by_id(1)
86 |     if db_user and (db_user.get('is_admin', 0) or db_user.
87 | get('role') == 'admin'):
88 |         return True
89 |
90 |     raise HTTPException(status_code=401, detail='Не авторизова
91 | н')
92 |
93 | def _get_active_file(mode: str) -> Path:
94 |     """Returns путь к активному файлу инструкции по режиму."""
95 |     path = _INSTRUCTION_FILES.get(mode)
96 |     if not path:

```

```

90         raise HTTPException(status_code=400, detail=f'Неизвест
    ный режим: {mode}. Допустимые: chat, narrator')
91     return path
92
93 def _get_versions_dir(mode: str) -> Path:
94     """Returns путь к папке версий и creates её если нет."""
95     path = _VERSIONS_DIRS.get(mode)
96     if not path:
97         raise HTTPException(status_code=400, detail=f'Неизвест
    ный режим: {mode}')
98     path.mkdir(parents=True, exist_ok=True)
99     return path
100
101 def _next_version_number(versions_dir: Path) -> int:
102     """Вычисляет следующий номер версии на основе файлов в пап
    ке."""
103     existing = list(versions_dir.glob('v*.md'))
104     numbers = []
105     for f in existing:
106         m = re.match(r'^v(\d+)_', f.name)
107         if m:
108             numbers.append(int(m.group(1)))
109     return max(numbers, default=0) + 1
110
111 def _load_sources_raw() -> str:
112     """Loads сырой текст из sources.json."""
113     if _SOURCES_FILE.exists():
114         try:
115             return _SOURCES_FILE.read_text(encoding='utf-8')
116         except Exception as ex:
117             logger.error('Error чтения sources.json', ex)
118     return '{}'
119
120 def _save_sources_raw(content: str) -> None:
121     """Saves сырой текст в sources.json."""
122     try:
123         json.loads(content)
124         _SOURCES_FILE.write_text(content, encoding='utf-8')

```

```

125     except json.JSONDecodeError as ex:
126         raise HTTPException(status_code=400, detail=f'Неверный
    формат JSON: {ex}')
127     except Exception as ex:
128         logger.error('Error записи sources.json', ex)
129         raise HTTPException(status_code=500, detail='Не удалос
    ь сохранить источники')
130
131 # =====
    =====
132 # Pydantic Models
133 # =====
    =====
134
135 class SystemInstructionUpdate(BaseModel):
136     content: str
137
138 class RawSourcesUpdate(BaseModel):
139     content: str
140
141 class InstructionRoleUpdate(BaseModel):
142     mode: str    # 'chat' | 'narrator'
143     content: str
144
145 class InstructionActivateRequest(BaseModel):
146     mode: str    # 'chat' | 'narrator'
147     filename: str    # имя файла из папки versions/, например '
    v2_2026-08-07.md'
148
149 # =====
    =====
150 # Legacy System Instruction Endpoints (backward compat)
151 # =====
    =====
152
153 @router.get('/system_instruction')
154 async def get_system_instruction(request: Request) -> Dict[str
    , str]:

```

```

155 |     """Получение текста системной инструкции чата (legacy endp
oint)."""
156 |     _check_admin(request)
157 |     active_file = _get_active_file('chat')
158 |     try:
159 |         content = active_file.read_text(encoding='utf-8') if a
ctive_file.exists() else ''
160 |         return {'content': content}
161 |     except Exception as ex:
162 |         logger.error('Error чтения system_instruction', ex)
163 |         raise HTTPException(status_code=500, detail='Не удалос
ь прочитать системную инструкцию')
164 |
165 | @router.post('/system_instruction')
166 | async def update_system_instruction(request: Request, data: Sy
stemInstructionUpdate) -> Dict[str, str]:
167 |     """Update текста системной инструкции чата (legacy endpoin
t)."""
168 |     _check_admin(request)
169 |     active_file = _get_active_file('chat')
170 |     try:
171 |         active_file.parent.mkdir(parents=True, exist_ok=True)
172 |         active_file.write_text(data.content, encoding='utf-8')
173 |         if hasattr(request.app.state, 'chat_model') and reques
t.app.state.chat_model:
174 |             request.app.state.chat_model.update_system_instruc
tion(data.content)
175 |             logger.info('System instruction updated via admin pane
l (legacy endpoint)')
176 |             return {'status': 'ok', 'message': 'Системная инструкц
ия successfully сохранена'}
177 |     except Exception as ex:
178 |         logger.error('Error записи system_instruction', ex)
179 |         raise HTTPException(status_code=500, detail='Не удалос
ь сохранить системную инструкцию')
180 |
181 | # =====
=====

```

```

182 # Instructions Endpoints (файловое версионирование)
183 # =====
184
185 @router.get('/instructions')
186 async def get_instruction(request: Request, mode: str = 'chat'
187 ) -> Dict[str, str]:
188     """Получение активной инструкции по режиму из файла."""
189     _check_admin(request)
190     active_file = _get_active_file(mode)
191     try:
192         content = active_file.read_text(encoding='utf-8') if a
193 ctive_file.exists() else ''
194         return {'content': content, 'mode': mode, 'file': str(
195 active_file.name)}
196     except Exception as ex:
197         logger.error(f'Error чтения инструкции mode={mode}', e
198 x)
199         raise HTTPException(status_code=500, detail='Не удалос
200 ь прочитать инструкцию')
201
202 @router.post('/instructions/save')
203 async def save_instruction(request: Request, data: Instruction
204 RoleUpdate) -> Dict[str, str]:
205     """Сохранение новой версии инструкции в файл и update акти
206 вного файла."""
207     _check_admin(request)
208     active_file = _get_active_file(data.mode)
209     versions_dir = _get_versions_dir(data.mode)
210
211     try:
212         # 1. Вычисляем номер следующей версии
213         version_num = _next_version_number(versions_dir)
214         date_str = datetime.now().strftime('%Y-%m-%d')
215         version_filename = f'v{version_num}_{date_str}.md'
216         version_path = versions_dir / version_filename
217
218         # 2. Сохраняем файл версии

```

```

212         version_path.write_text(data.content, encoding='utf-8'
    | )
213         logger.info(f'Saved instruction version: {version_path
    | }')
214
215         # 3. Перезаписываем активный файл
216         active_file.parent.mkdir(parents=True, exist_ok=True)
217         active_file.write_text(data.content, encoding='utf-8')
218
219         # Обновляем модель в памяти без перезагрузки
220         if data.mode == 'chat' and hasattr(request.app.state,
    | 'chat_model'):
221             request.app.state.chat_model.update_system_instruc
    | tion(data.content)
222             elif data.mode == 'narrator' and hasattr(request.app.s
    | tate, 'narrator_model'):
223                 request.app.state.narrator_model.update_system_ins
    | truction(data.content)
224
225         logger.info(f'Updated active instruction file: {active
    | _file} (mode={data.mode})')
226
227         return {
228             'status': 'ok',
229             'message': f'Инструкция сохранена как версия {vers
    | ion_filename}',
230             'version': version_filename
231         }
232     except Exception as ex:
233         logger.error(f'Error сохранения инструкции mode={data.
    | mode}', ex)
234         raise HTTPException(status_code=500, detail='Не удалос
    | ь сохранить инструкцию')
235
236 @router.get('/instructions/versions')
237 async def get_instruction_versions(request: Request, mode: str
    | = 'chat') -> Dict[str, Any]:
238     """Получение списка версий инструкций из папки versions/."

```

```

"""
239     _check_admin(request)
240     versions_dir = _get_versions_dir(mode)
241     active_file = _get_active_file(mode)
242
243     try:
244         # Читаем содержимое активного файла для сравнения
245         active_content = active_file.read_text(encoding='utf-8
246         ') if active_file.exists() else ''
247
248         version_files = sorted(versions_dir.glob('v*.md'), key
249         =lambda f: f.stat().st_mtime, reverse=True)
250
251         versions = []
252         for vf in version_files:
253             try:
254                 file_content = vf.read_text(encoding='utf-8')
255                 stat = vf.stat()
256                 created_at = datetime.fromtimestamp(stat.st_mt
257                 ime).isoformat()
258                 is_active = (file_content.strip() == active_co
259                 ntent.strip())
260                 versions.append({
261                     'filename': vf.name,
262                     'mode': mode,
263                     'is_active': is_active,
264                     'created_at': created_at,
265                     'size': stat.st_size,
266                     'preview': file_content[:120] + '...' if l
267                     en(file_content) > 120 else file_content,
268                 })
269             except Exception as read_ex:
270                 logger.warning(f'Не удалось прочитать файл вер
271                 сии {vf}: {read_ex}')
272
273         return {'versions': versions, 'mode': mode}
274     except Exception as ex:
275         logger.error(f'Error чтения версий mode={mode}', ex)

```



```

270         raise HTTPException(status_code=500, detail='Не удалось
    ы прочитать версии')
271
272 @router.post('/instructions/activate')
273 async def activate_instruction(request: Request, data: Instruc
    tionActivateRequest) -> Dict[str, str]:
274     """Активация выбранной версии инструкции – копирует файл в
    ерсии в активный."""
275     _check_admin(request)
276     versions_dir = _get_versions_dir(data.mode)
277     active_file = _get_active_file(data.mode)
278
279     # Безопасность: только имя файла, без path traversal
280     safe_filename = Path(data.filename).name
281     version_path = versions_dir / safe_filename
282
283     if not version_path.exists():
284         raise HTTPException(status_code=404, detail=f'Версия {
    safe_filename} не найдена')
285
286     try:
287         content = version_path.read_text(encoding='utf-8')
288         active_file.parent.mkdir(parents=True, exist_ok=True)
289         active_file.write_text(content, encoding='utf-8')
290
291         # Обновляем модель в памяти без перезагрузки
292         if data.mode == 'chat' and hasattr(request.app.state,
    'chat_model'):
293             request.app.state.chat_model.update_system_instruc
    tion(content)
294         elif data.mode == 'narrator' and hasattr(request.app.s
    tate, 'narrator_model'):
295             request.app.state.narrator_model.update_system_ins
    truction(content)
296
297         logger.info(f'Activated instruction version {safe_file
    name} for mode={data.mode}')
298         return {'status': 'ok', 'message': f'Версия {safe_file

```

```

name} активирована', 'version': safe_filename}
299     except Exception as ex:
300         logger.error(f'Error активации версии {safe_filename}'
, ex)
301         raise HTTPException(status_code=500, detail='Не удалос
ь активировать версию')
302
303 @router.post('/instructions/check')
304 async def check_instruction_in_model(request: Request, data: D
ict[str, Any]) -> Dict[str, Any]:
305     """Временная check инструкции в модели без сохранения."""
306     _check_admin(request)
307     try:
308         from src.ai.unified_chat import UnifiedChatModel
309         import os
310
311         api_key_names = [n.strip() for n in os.getenv('GEMINI_
API_KEY_NAMES', '').split(',') if n.strip()]
312         if not api_key_names:
313             raise HTTPException(status_code=500, detail='GEMIN
I_API_KEY_NAMES не настроен')
314
315         system_instruction = data.get('instruction', '')
316         prompt = data.get('prompt', 'Привет!')
317         foundry_model_id = os.getenv('FOUNDRY_MODEL_ID', 'qwen
3-0.6b-generic-cpu:4')
318
319         # Создаём временный инстанс модели для теста
320         temp_model = UnifiedChatModel(
321             api_key_names=api_key_names,
322             system_instruction=system_instruction,
323             foundry_model_id=foundry_model_id,
324             use_foundry=False,
325         )
326
327         response = await temp_model.chat(prompt)
328
329         total_tokens = len(system_instruction) // 3 + len(prom

```

```

    pt) // 3 + len(response or '') // 3
330
331     return {
332         'status': 'ok',
333         'response': response or '',
334         'token_count': total_tokens
335     }
336 except Exception as ex:
337     logger.error('Error проверки инструкции в модели', ex)
338     raise HTTPException(status_code=500, detail=f'Error пр
оверки: {ex}')
339
340 # =====
=====
341 # Sources Endpoints
342 # =====
=====
343
344 @router.get('/sources/raw')
345 async def get_sources_raw(request: Request) -> Dict[str, str]:
346     """Получение сырого JSON-текста источников."""
347     _check_admin(request)
348     content = _load_sources_raw()
349     return {'content': content}
350
351 @router.post('/sources/raw')
352 async def update_sources_raw(request: Request, data: RawSource
sUpdate) -> Dict[str, str]:
353     """Update сырого JSON-текста источников."""
354     _check_admin(request)
355     _save_sources_raw(data.content)
356     logger.info('Sources JSON updated via admin panel')
357     return {'status': 'ok'}
358
359 # =====
=====
360 # Plugin Manager Endpoints
361 # =====
=====

```

```

=====
362
363 class PluginStateUpdate(BaseModel):
364     enabled: bool
365
366 class PluginConfigUpdate(BaseModel):
367     config: Dict[str, Any]
368
369 class PluginActionRequest(BaseModel):
370     params: Dict[str, Any] = {}
371
372 @router.get('/plugins')
373 async def get_all_plugins(request: Request) -> Dict[str, Any]:
374     """Returns empty реестр (плагины отключены)."""
375     _check_admin(request)
376     return {'plugins': {}, 'count': 0}
377
378 @router.get('/plugins/{plugin_name}')
379 async def get_plugin_details(plugin_name: str, request: Request) -> Dict[str, Any]:
380     """Returns 404 (плагины отключены)."""
381     _check_admin(request)
382     raise HTTPException(status_code=404, detail=f"Плагин '{plugin_name}' не найден")
383
384 @router.post('/plugins/{plugin_name}/toggle')
385 async def toggle_plugin(plugin_name: str, data: PluginStateUpdate, request: Request) -> Dict[str, Any]:
386     """Плагины отключены."""
387     _check_admin(request)
388     raise HTTPException(status_code=404, detail=f"Плагин '{plugin_name}' не найден")
389
390 @router.post('/plugins/{plugin_name}/config')
391 async def save_plugin_config(plugin_name: str, data: PluginConfigUpdate, request: Request) -> Dict[str, Any]:
392     """Плагины отключены."""
393     _check_admin(request)

```

```

394     raise HTTPException(status_code=404, detail=f"Плагин '{plugin_name}' не найден")
395
396 @router.post('/plugins/{plugin_name}/action/{action_name}')
397 async def call_plugin_action(plugin_name: str, action_name: str, data: PluginActionRequest, request: Request) -> Dict[str, Any]:
398     """Плагины отключены."""
399     _check_admin(request)
400     raise HTTPException(status_code=404, detail=f"Плагин '{plugin_name}' не найден")
401
402 @router.get('/plugin/{plugin_name}/status')
403 async def get_plugin_status(plugin_name: str, request: Request):
404     """Плагины отключены."""
405     _check_admin(request)
406     raise HTTPException(status_code=404, detail='Плагин не найден')
407
408 @router.post('/plugin/{plugin_name}/status')
409 async def update_plugin_status(plugin_name: str, data: PluginStateUpdate, request: Request):
410     """Update статуса плагина (обратная совместимость)."""
411     _check_admin(request)
412     plugins_dict = getattr(request.app.state, 'plugins', {})
413     plugin = plugins_dict.get(plugin_name)
414     if not plugin:
415         raise HTTPException(status_code=404, detail='Плагин не найден')
416     plugin.enabled = data.enabled
417     plugin.update_config({'enabled': data.enabled})
418     logger.info(f'Plugin {plugin_name} enabled state changed to {data.enabled}')
419     return {'name': plugin_name, 'enabled': plugin.enabled}
420
421 # =====

```

```

422 # RAG Endpoints
423 # =====
424
425 class RagConfigRequest(BaseModel):
426     mode: str
427
428 @router.get('/rag/config')
429 async def get_rag_config(request: Request):
430     """Получение режима RAG."""
431     _check_admin(request)
432     config_path = __root__ / 'config.json'
433     mode = "rag+model"
434     if config_path.exists():
435         try:
436             with open(config_path, 'r', encoding='utf-8') as f
437                 :
438                 cfg = json.load(f)
439                 mode = cfg.get("rag", {}).get("mode", "rag+model")
440         except Exception as e:
441             logger.error("Error чтения config.json", e)
442     return {"mode": mode}
443
444 @router.post('/rag/config')
445 async def set_rag_config(request: Request, data: RagConfigRequest):
446     """Установка режима RAG."""
447     _check_admin(request)
448     config_path = __root__ / 'config.json'
449     try:
450         if config_path.exists():
451             with open(config_path, 'r', encoding='utf-8') as f
452                 :
453                 cfg = json.load(f)
454         else:
455             cfg = {}

```

```

455         if "rag" not in cfg:
456             cfg["rag"] = {}
457
458         cfg["rag"]["mode"] = data.mode
459
460         with open(config_path, 'w', encoding='utf-8') as f:
461             json.dump(cfg, f, indent=2, ensure_ascii=False)
462
463         return {"status": "ok", "mode": data.mode}
464     except Exception as e:
465         logger.error("Error записи config.json", e)
466         raise HTTPException(status_code=500, detail="Error сох
ранения конфигурации RAG")
467
468 class WebSearchConfigRequest(BaseModel):
469     engine: str
470     gemini_model: str = "gemini-2.5-flash"
471     gemini_cli_model: str = "gemini-3.1-flash-lite"
472     agy_model: str = "agy-flash"
473
474 @router.get('/web-search/config')
475 async def get_web_search_config(request: Request):
476     """Получение конфигурации сервера веб-поиска."""
477     _check_admin(request)
478     config_path = __root__ / 'config.json'
479     engine = "playwright"
480     gemini_model = "gemini-2.5-flash"
481     gemini_cli_model = "gemini-3.1-flash-lite"
482     agy_model = "agy-flash"
483     if config_path.exists():
484         try:
485             with open(config_path, 'r', encoding='utf-8') as f
486                 :
487                     cfg = json.load(f)
488                     ws = cfg.get("web_search", {})
489                     engine = ws.get("engine", "playwright")
490                     gemini_model = ws.get("gemini_model", "gemini-
2.5-flash")

```

```

490         gemini_cli_model = ws.get("gemini_cli_model",
    "gemini-3.1-flash-lite")
491         agy_model = ws.get("agy_model", "agy-flash")
492     except Exception as e:
493         logger.error("Error чтения config.json для web_sea
    rch", e)
494     return {
495         "engine": engine,
496         "gemini_model": gemini_model,
497         "gemini_cli_model": gemini_cli_model,
498         "agy_model": agy_model
499     }
500
501 @router.post('/web-search/config')
502 async def set_web_search_config(request: Request, data: WebSea
    rchConfigRequest):
503     """Установка сервера веб-поиска (playwright / langchain /
    gemini / gemini_cli / agy)."""
504     _check_admin(request)
505     config_path = __root__ / 'config.json'
506     try:
507         if config_path.exists():
508             with open(config_path, 'r', encoding='utf-8') as f
    :
509                 cfg = json.load(f)
510         else:
511             cfg = {}
512
513         if "web_search" not in cfg:
514             cfg["web_search"] = {}
515
516         cfg["web_search"]["engine"] = data.engine
517         cfg["web_search"]["gemini_model"] = data.gemini_model
518         cfg["web_search"]["gemini_cli_model"] = data.gemini_cl
    i_model
519         cfg["web_search"]["agy_model"] = data.agy_model
520
521         with open(config_path, 'w', encoding='utf-8') as f:

```



```
522         json.dump(cfg, f, indent=2, ensure_ascii=False)
523
524     return {
525         "status": "ok",
526         "engine": data.engine,
527         "gemini_model": data.gemini_model,
528         "gemini_cli_model": data.gemini_cli_model,
529         "agy_model": data.agy_model
530     }
531 except Exception as e:
532     logger.error("Error записи config.json для web_search"
533 , e)
534     raise HTTPException(status_code=500, detail="Error сох
ранения конфигурации веб-поиска")
535
536 class WebSearchTestRequest(BaseModel):
537     query: str
538     engine: str = ""
539
540 @router.post('/web-search/test')
541 async def test_web_search(request: Request, data: WebSearchTes
tRequest):
542     """Тестовое выполнение поиска через выбранный поисковый дв
ижок."""
543     _check_admin(request)
544     query = data.query.strip()
545     if not query:
546         raise HTTPException(status_code=400, detail="Поисковый
запрос не может быть пустым")
547
548     engine = data.engine
549     gemini_model = "gemini-2.5-flash"
550     gemini_cli_model = "gemini-3.1-flash-lite"
551     agy_model = "agy-flash"
552     config_path = __root__ / 'config.json'
553     if config_path.exists():
554         try:
555             with open(config_path, 'r', encoding='utf-8') as f
```

```

:
555         cfg = json.load(f)
556         ws = cfg.get("web_search", {})
557         if not engine:
558             engine = ws.get("engine", "playwright")
559             gemini_model = ws.get("gemini_model", "gemini-
2.5-flash")
560             gemini_cli_model = ws.get("gemini_cli_model",
"gemini-3.1-flash-lite")
561             agy_model = ws.get("agy_model", "agy-flash")
562         except Exception as e:
563             logger.error("Error чтения config.json для web_sea
rch", e)
564
565     if not engine:
566         engine = "playwright"
567
568     try:
569         return {"status": "ok", "engine": engine, "result": f"
Поиск '{query}' выполнен."}
570     except Exception as e:
571         logger.error(f"Error тестового поиска {engine}", e)
572         return {"status": "error", "engine": engine, "message"
: str(e)}
573
574 # =====
=====
575 # User Management Models and Endpoints
576 # =====
=====
577
578 class AdminUserCreateRequest(BaseModel):
579     email: str
580     name: str
581     password: str = ''
582     role: str = 'user'
583     is_admin: int = 0
584     is_active: int = 1

```

```
585     is_email_verified: int = 1
586
587 class AdminUserUpdateRequest(BaseModel):
588     name: str = ''
589     email: str = ''
590     role: str = ''
591     is_admin: int = 0
592     is_active: int = 1
593     is_email_verified: int = 1
594
595 class AdminUserPasswordRequest(BaseModel):
596     password: str
597
598 @router.get('/users')
599 async def list_admin_users(
600     request: Request,
601     q: str = '',
602     role: str = '',
603     status: str = ''
604 ) -> Dict[str, Any]:
605     """Получение списка пользователей с фильтрацией, поиском и
        статистикой."""
606     _check_admin(request)
607     from src.user_manager import user_manager
608     all_users = user_manager.get_all_users(active_only=False)
609
610     total_count = len(all_users)
611     active_count = sum(1 for u in all_users if u.get('is_active', 0) == 1)
612     admin_count = sum(1 for u in all_users if u.get('is_admin', 0) == 1 or u.get('role') == 'admin')
613     tg_count = sum(1 for u in all_users if u.get('telegram_id'))
614
615     filtered = []
616     q_lower = q.lower().strip()
617
618     for u in all_users:
```

```

619         if q_lower:
620             name_match = q_lower in str(u.get('name', '')).lower()
621             email_match = q_lower in str(u.get('email', '')).lower()
622             tg_match = q_lower in str(u.get('telegram_username', '')).lower() or q_lower in str(u.get('telegram_id', ''))
623             if not (name_match or email_match or tg_match):
624                 continue
625
626         if role and u.get('role') != role:
627             continue
628
629         if status == 'active' and u.get('is_active', 0) != 1:
630             continue
631         if status == 'inactive' and u.get('is_active', 0) == 1:
632             continue
633
634         sanitized = {k: v for k, v in u.items() if k != 'password_hash'}
635         sanitized['has_password'] = bool(u.get('password_hash'))
636         filtered.append(sanitized)
637
638     return {
639         'status': 'ok',
640         'users': filtered,
641         'stats': {
642             'total': total_count,
643             'active': active_count,
644             'admins': admin_count,
645             'telegram': tg_count,
646         }
647     }
648
649 @router.post('/users')
650 async def create_admin_user(request: Request, data: AdminUserC

```

```

    reateRequest) -> Dict[str, Any]:
651     """Создание нового пользователя администратором."""
652     _check_admin(request)
653     email = data.email.strip().lower()
654     name = data.name.strip()
655     if not email:
656         raise HTTPException(status_code=400, detail='Email обя
зателен')
657     if not name:
658         raise HTTPException(status_code=400, detail='Имя обяза
тельно')
659
660     from src.user_manager import user_manager
661     if user_manager.user_exists(email):
662         raise HTTPException(status_code=400, detail=f'Пользова
тель с email {email} уже существует')
663
664     user_id = user_manager.create_user_admin(
665         email=email,
666         name=name,
667         password=data.password,
668         role=data.role,
669         is_admin=data.is_admin,
670         is_active=data.is_active,
671         is_email_verified=data.is_email_verified
672     )
673     if not user_id:
674         raise HTTPException(status_code=500, detail='Error соз
дания пользователя')
675
676     created = user_manager.get_user_by_id(user_id)
677     sanitized = {k: v for k, v in created.items() if k != 'pas
sword_hash'}
678     sanitized['has_password'] = bool(created.get('password_has
h'))
679     return {'status': 'ok', 'user': sanitized}
680
681 @router.get('/users/{user_id}')

```

```

682 async def get_admin_user_details(user_id: int, request: Request) -> Dict[str, Any]:
683     """Получение детальной информации о пользователе и его на
        тройках."""
684     _check_admin(request)
685     from src.user_manager import user_manager
686     user = user_manager.get_user_by_id(user_id)
687     if not user:
688         raise HTTPException(status_code=404, detail='Пользоват
        ель не найден')
689
690     settings = user_manager.get_user_settings(user_id)
691     permissions = user_manager.get_user_permissions(user_id)
692     sanitized = {k: v for k, v in user.items() if k != 'password_hash'}
693     sanitized['has_password'] = bool(user.get('password_hash'))
694
695     return {
696         'status': 'ok',
697         'user': sanitized,
698         'settings': settings,
699         'permissions': permissions
700     }
701
702 @router.put('/users/{user_id}')
703 async def update_admin_user(user_id: int, data: AdminUserUpdateRequest, request: Request) -> Dict[str, Any]:
704     """Update данных пользователя."""
705     _check_admin(request)
706     from src.user_manager import user_manager
707     user = user_manager.get_user_by_id(user_id)
708     if not user:
709         raise HTTPException(status_code=404, detail='Пользоват
        ель не найден')
710
711     updates: Dict[str, Any] = {}
712     if data.name:

```

```

713         updates['name'] = data.name.strip()
714     if data.email:
715         email_clean = data.email.strip().lower()
716         if email_clean != user.get('email'):
717             existing = user_manager.get_user_by_email(email_clean)
718             if existing and existing.get('id') != user_id:
719                 raise HTTPException(status_code=400, detail='Этот email уже занят другим пользователем')
720             updates['email'] = email_clean
721     if data.role:
722         updates['role'] = data.role
723     updates['is_admin'] = data.is_admin
724     updates['is_active'] = data.is_active
725     updates['is_email_verified'] = data.is_email_verified
726
727     success = user_manager.update_user(user_id, **updates)
728     if not success:
729         raise HTTPException(status_code=500, detail='Error обновления пользователя')
730
731     updated = user_manager.get_user_by_id(user_id)
732     sanitized = {k: v for k, v in updated.items() if k != 'password_hash'}
733     sanitized['has_password'] = bool(updated.get('password_hash'))
734     return {'status': 'ok', 'user': sanitized}
735
736 @router.post('/users/{user_id}/password')
737 async def set_admin_user_password(user_id: int, data: AdminUserPasswordRequest, request: Request) -> Dict[str, Any]:
738     """Установка / сброс пароля пользователя администратором."""
739     _check_admin(request)
740     new_password = data.password.strip()
741     if not new_password:
742         raise HTTPException(status_code=400, detail='Пароль не может быть пустым')

```

```
743
744     from src.user_manager import user_manager
745     user = user_manager.get_user_by_id(user_id)
746     if not user:
747         raise HTTPException(status_code=404, detail='Пользоват
ель не найден')
748
749     success = user_manager.set_user_password(user_id, new_pass
word)
750     if not success:
751         raise HTTPException(status_code=500, detail='Error уст
ановки пароля')
752
753     return {'status': 'ok', 'message': 'Пароль successfully об
новлён'}
754
755 @router.post('/users/{user_id}/toggle-active')
756 async def toggle_admin_user_active(user_id: int, request: Requ
est) -> Dict[str, Any]:
757     """Переключение активности пользователя (блокировка / разб
локировка)."""
758     _check_admin(request)
759     from src.user_manager import user_manager
760     user = user_manager.get_user_by_id(user_id)
761     if not user:
762         raise HTTPException(status_code=404, detail='Пользоват
ель не найден')
763
764     if user_id == 1 and user.get('is_active', 1) == 1:
765         raise HTTPException(status_code=400, detail='Нельзя де
активировать главного администратора (ID 1)')
766
767     new_status = 0 if user.get('is_active', 1) == 1 else 1
768     success = user_manager.update_user(user_id, is_active=new_
status)
769     if not success:
770         raise HTTPException(status_code=500, detail='Error изм
енения статуса')
```



```
771
772     return {'status': 'ok', 'is_active': new_status}
773
774 @router.post('/users/{user_id}/toggle-role')
775 async def toggle_admin_user_role(user_id: int, request: Request) -> Dict[str, Any]:
776     """Переключение роли пользователя (пользователь <-> администратор)."""
777     _check_admin(request)
778     from src.user_manager import user_manager
779     user = user_manager.get_user_by_id(user_id)
780     if not user:
781         raise HTTPException(status_code=404, detail='Пользователь не найден')
782
783     is_currently_admin = bool(user.get('is_admin', 0) or user.get('role') == 'admin')
784
785     # Защита системного администратора ID 1 от снятия прав
786     if user_id == 1 and is_currently_admin:
787         raise HTTPException(status_code=400, detail='Нельзя снять права у главного администратора (ID 1)')
788
789     if is_currently_admin:
790         new_role = 'user'
791         new_is_admin = 0
792     else:
793         new_role = 'admin'
794         new_is_admin = 1
795
796     success = user_manager.update_user(user_id, role=new_role, is_admin=new_is_admin)
797     if not success:
798         raise HTTPException(status_code=500, detail='Error изменения роли')
799
800     return {'status': 'ok', 'role': new_role, 'is_admin': new_is_admin}
```

```

801 |
802 | @router.delete('/users/{user_id}')
803 | async def delete_admin_user(user_id: int, request: Request) ->
      Dict[str, Any]:
804 |     """Удаление пользователя."""
805 |     _check_admin(request)
806 |     from src.user_manager import user_manager
807 |     user = user_manager.get_user_by_id(user_id)
808 |     if not user:
809 |         raise HTTPException(status_code=404, detail='Пользоват
      ель не найден')
810 |
811 |     if user_id == 1:
812 |         raise HTTPException(status_code=400, detail='Нельзя уд
      алить главного администратора (ID 1)')
813 |
814 |     success = user_manager.delete_user(user_id)
815 |     if not success:
816 |         raise HTTPException(status_code=500, detail='Error уда
      ления пользователя')
817 |
818 |     return {'status': 'ok', 'message': f'Пользователь ID {user
      _id} удалён'}
819 |
820 | # =====
      =====
821 | # Initialization
822 | # =====
      =====
823 |
824 | def init_router() -> APIRouter:
825 |     """Initialization роутера управления системными инструкция
      ми и источниками."""
826 |     return router
827 |

```

router_agents.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI agents configuration and execution router
4 # =====
5 # Description:
6 #   Provides FastAPI endpoints for managing system and custom
7 #   AI agents,
8 #   handling prompt generation via AI Architect, and executing
9 #   agent test workflows.
10 #
11 # File: router_agents.py
12 # Project: ai-breadboard
13 # Package: src.fastapi
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # =====
17
18 from __future__ import annotations
19
20 import json
21 import time
22 import re
23 import asyncio
24 from pathlib import Path
25 from typing import Any, Dict, List, Optional
26 from fastapi import APIRouter, HTTPException, Request
27 from pydantic import BaseModel, Field
28
29 from header import __root__
30 from src.logger import logger
31 from src.utils.jjson import j_loads_ns
32
33 import copy
```

```

32 |
33 | _CONFIG_PATH = __root__ / 'config.json'
34 | _AGENTS_METADATA_PATH = Path(__file__).parent / 'agents_metadata.json'
35 |
36 | # =====
37 | # Load agents metadata from external config
38 | # =====
39 |
40 | def _load_agents_metadata() -> dict:
41 |     """Load available tools and providers from external metadata file.
42 |
43 |     Returns:
44 |         dict: Dictionary with 'tools' and 'providers' keys.
45 |     """
46 |     if not _AGENTS_METADATA_PATH.exists():
47 |         logger.warning(f'[router_agents] Metadata file not found: {_AGENTS_METADATA_PATH}')
48 |         return {'tools': [], 'providers': {}}
49 |
50 |     try:
51 |         with open(_AGENTS_METADATA_PATH, 'r', encoding='utf-8') as f:
52 |             return json.load(f)
53 |     except Exception as e:
54 |         logger.error(f'[router_agents] Error loading agents metadata: {e}')
55 |         return {'tools': [], 'providers': {}}
56 |
57 | _AGENTS_METADATA = _load_agents_metadata()
58 | _AVAILABLE_TOOLS = _AGENTS_METADATA.get('tools', [])
59 | _PROVIDERS_CONFIG = _AGENTS_METADATA.get('providers', {})
60 |
61 | # =====

```

```

62 # Pydantic Models
63 # =====
64 | =====
65 class AgentModel(BaseModel):
66     """Agent configuration model.
67
68     Defines all parameters for AI agent execution and behavior
69     .
70     """
71     id: str
72     name: str
73     description: str = ''
74     is_system: bool = False
75     enabled: bool = True
76     provider: str = 'gemini'
77     model: str = 'gemini-2.5-flash'
78     temperature: float = 0.3
79     max_steps: int = 15
80     timeout_seconds: int = 60
81     tools: List[str] = Field(default_factory=list)
82     system_prompt: str = ''
83
84 class GeneratePromptRequest(BaseModel):
85     """Request to generate agent prompt via AI Architect."""
86     task_description: str
87     provider: str = 'gemini'
88     model: str = 'gemini-2.5-flash'
89     agent_name: str = ''
90
91 class TestAgentRequest(BaseModel):
92     """Request to test agent execution."""
93     agent_id: str = ''
94     inline_config: Optional[AgentModel] = Field(default_factor
95     y=dict) # type: ignore
96     test_message: str
97
98 # =====

```

```

=====
97 # Config.json helper functions
98 # =====
=====
99
100 def _load_raw_config() -> dict:
101     """Load full config.json file.
102
103     Returns:
104         dict: Parsed configuration or empty dict if file not found.
105     """
106     if not _CONFIG_PATH.exists():
107         return {}
108     try:
109         with open(_CONFIG_PATH, 'r', encoding='utf-8') as f:
110             return json.load(f)
111     except Exception as e:
112         logger.error(f'[router_agents] Error loading config.json: {e}')
113         return {}
114
115 def _save_raw_config(data: dict) -> None:
116     """Save config.json with formatting.
117
118     Args:
119         data: Configuration dictionary to save.
120
121     Raises:
122         HTTPException: If save operation fails.
123     """
124     try:
125         with open(_CONFIG_PATH, 'w', encoding='utf-8') as f:
126             json.dump(data, f, ensure_ascii=False, indent=2)
127     except Exception as e:
128         logger.error(f'[router_agents] Error saving config.json: {e}')
129         raise HTTPException(status_code=500, detail='Failed to

```

```

    save configuration')
130
131 def _get_agents_list() -> List[dict]:
132     """Get list of agents from config.json.
133
134     Returns:
135         List[dict]: List of agent configurations.
136     """
137     cfg = _load_raw_config()
138     agents_cfg = cfg.get('agents', {})
139     if isinstance(agents_cfg, dict):
140         return agents_cfg.get('items', [])
141     return []
142
143 def _save_agents_list(items: List[dict]) -> None:
144     """Save updated agents list to config.json.
145
146     Args:
147         items: List of agent configurations to save.
148     """
149     cfg = _load_raw_config()
150     if 'agents' not in cfg or not isinstance(cfg['agents'], dict):
151         cfg['agents'] = {}
152         cfg['agents']['items'] = items
153         _save_raw_config(cfg)
154
155 # =====
156 # FastAPI Router
157 # =====
158
159 def init_agents_router(prefix: str = '/api/agents') -> APIRouter:
160     """Initialize and return agent management router.
161
162     Args:

```

```

163         prefix: URL prefix for router endpoints.
164
165     Returns:
166         APIRouter: Configured FastAPI router.
167     """
168     router = APIRouter(prefix=prefix, tags=['agents'])
169
170     @router.get('')
171     async def list_agents() -> List[dict]:
172         """Get list of all agents (system and user-defined).
173
174         Returns:
175             List[dict]: List of agent configurations.
176         """
177         return _get_agents_list()
178
179     @router.get('/tools')
180     async def list_tools() -> List[dict]:
181         """Get catalog of all available system tools.
182
183         Returns:
184             List[dict]: List of tool definitions.
185         """
186         return _AVAILABLE_TOOLS
187
188     @router.get('/providers')
189     async def list_providers() -> dict:
190         """Get list of providers and dynamically fetched model
191         s from project pool."""
192         from src.ai.model_manager import get_available_models
193
194         raw_cfg = _load_raw_config()
195         ai_section = raw_cfg.get('ai', {})
196
197         # Deep copy base provider metadata
198         providers = copy.deepcopy(_PROVIDERS_CONFIG)
199
200         # Dynamically populate models for each provider from m

```



```

odel_manager
200     for prov_id, prov_info in providers.items():
201         try:
202             raw_models = get_available_models(prov_id)
203             prov_info['models'] = [{ 'id': m, 'name': m} fo
r m in raw_models]
204         except Exception as e:
205             logger.warning(f'[router_agents] Error queryin
g models for provider {prov_id}: {e}')
206             prov_info['models'] = []
207
208         # Override defaults from config.json if present
209         if 'gemini_cli_model_id' in ai_section:
210             if 'gemini_cli' in providers:
211                 providers['gemini_cli']['default_model'] = ai_
section['gemini_cli_model_id']
212
213         if 'agy_model_id' in ai_section:
214             if 'agy' in providers:
215                 providers['agy']['default_model'] = ai_section
['agy_model_id']
216
217         if 'foundry_model_id' in ai_section:
218             if 'foundry' in providers:
219                 providers['foundry']['default_model'] = ai_sec
tion['foundry_model_id']
220
221         if 'ollama_model' in raw_cfg.get('langchain', {}):
222             if 'ollama' in providers:
223                 providers['ollama']['default_model'] = raw_cfg
['langchain']['ollama_model']
224
225         return providers
226
227     @router.post('')
228     async def create_agent(agent: AgentModel) -> dict:
229         """Create new custom agent.
230

```

```

231 |     Args:
232 |         agent: Agent configuration.
233 |
234 |     Returns:
235 |         dict: Created agent data with status.
236 |
237 |     Raises:
238 |         HTTPException: If agent ID already exists.
239 |     """
240 |     items = _get_agents_list()
241 |
242 |     # Check ID uniqueness
243 |     existing_ids = {a.get('id') for a in items}
244 |     if agent.id in existing_ids:
245 |         raise HTTPException(status_code=400, detail=f'Agent
246 | t with ID "{agent.id}" already exists')
247 |
248 |     # Protect is_system flag for user agents
249 |     agent_dict = agent.dict()
250 |     agent_dict['is_system'] = False
251 |
252 |     items.append(agent_dict)
253 |     _save_agents_list(items)
254 |     logger.info(f'[router_agents] Created new agent: {agen
255 | t.id} ({agent.name})')
256 |     return {'status': 'ok', 'agent': agent_dict}
257 |
258 | @router.put("/{agent_id}")
259 | async def update_agent(agent_id: str, agent: AgentModel) -
260 | > dict:
261 |     """Update existing agent.
262 |
263 |     Args:
264 |         agent_id: Agent identifier.
265 |         agent: Updated agent configuration.
266 |
267 |     Returns:
268 |         dict: Updated agent data with status.

```

```

266
267     Raises:
268         HTTPException: If agent not found.
269     """
270     items = _get_agents_list()
271     found_idx = -1
272     for idx, item in enumerate(items):
273         if item.get('id') == agent_id:
274             found_idx = idx
275             break
276
277     if found_idx == -1:
278         raise HTTPException(status_code=404, detail=f'Agent with ID "{agent_id}" not found')
279
280     old_item = items[found_idx]
281     agent_dict = agent.dict()
282     # Preserve system status from original agent
283     agent_dict['is_system'] = old_item.get('is_system', False)
284     agent_dict['id'] = agent_id # ID does not change
285
286     items[found_idx] = agent_dict
287     _save_agents_list(items)
288     logger.info(f'[router_agents] Updated agent: {agent_id}')
289     return {'status': 'ok', 'agent': agent_dict}
290
291 @router.delete('/{agent_id}')
292 async def delete_agent(agent_id: str) -> dict:
293     """Delete custom agent (system agents are protected).
294
295     Args:
296         agent_id: Agent identifier.
297
298     Returns:
299         dict: Status and deleted agent ID.
300

```

```

301 |         Raises:
302 |             HTTPException: If agent not found or is system age
303 |             nt.
304 |             """
305 |             items = _get_agents_list()
306 |             target = None
307 |             for item in items:
308 |                 if item.get('id') == agent_id:
309 |                     target = item
310 |                     break
311 |             if not target:
312 |                 raise HTTPException(status_code=404, detail=f'Agent with ID "{agent_id}" not found')
313 |
314 |             if target.get('is_system', False):
315 |                 raise HTTPException(status_code=403, detail='System agents cannot be deleted. You can disable them.')
316 |
317 |             items = [i for i in items if i.get('id') != agent_id]
318 |             _save_agents_list(items)
319 |             logger.info(f'[router_agents] Deleted agent: {agent_id}')
320 |             return {'status': 'ok', 'deleted_id': agent_id}
321 |
322 |     @router.post('/generate-prompt')
323 |     async def generate_prompt(req: GeneratePromptRequest) -> dict:
324 |         """Generate system prompt and agent settings via AI Architect model.
325 |
326 |         Uses a language model to automatically generate agent configuration
327 |         based on task description.
328 |
329 |         Args:
330 |             req: Request with task description and model selection.

```

```

331
332         Returns:
333             dict: Generated agent specification with recommend
ed tools and settings.
334
335         Raises:
336             HTTPException: If task description is empty or mod
el fails.
337         """
338         if not req.task_description.strip():
339             raise HTTPException(status_code=400, detail='Task
description cannot be empty')
340
341         tool_ids = [t['id'] for t in _AVAILABLE_TOOLS]
342         tool_desc = "\n".join([f"- {t['id']}: {t['name']} ({t[
'description']})" for t in _AVAILABLE_TOOLS])
343
344         prompt_architect_query = f"""You are an experienced AI
Agent Architect.
345 User wants to create a specialized agent for the ai-breadboard
platform.
346
347 User's agent task:
348 "{req.task_description}"
349
350 Available system tools:
351 {tool_desc}
352
353 Generate complete agent specification in valid JSON format wit
hout markdown blocks:
354 {{
355     "name": "Agent name (short, clear)",
356     "description": "Brief description of agent role (1-2 sentenc
es)",
357     "system_prompt": "Detailed system instruction: role, reasoni
ng rules (ReAct), tool selection, output format",
358     "recommended_tools": ["tool_ids_from_list_above"],
359     "temperature": 0.2,

```

```

360 |     "max_steps": 15
361 | }}
362 | """
363 |
364 |     try:
365 |         from src.fastapi.router_chat import get_chat_model
366 |         model_key = req.model
367 |         if req.provider == 'gemini_cli' and not model_key.
startswith('gemini_cli:'):
368 |             model_key = f'gemini_cli:{model_key}'
369 |         elif req.provider == 'foundry' and not model_key.s
tartswith('foundry:'):
370 |             model_key = f'foundry:{model_key}'
371 |         elif req.provider == 'ollama' and not model_key.st
artswith('ollama:'):
372 |             model_key = f'ollama:{model_key}'
373 |
374 |         llm = get_chat_model(model_key, system_instruction
="You are an expert AI Agent Architect. Always return pure JSO
N.")
375 |
376 |         # Call model
377 |         if hasattr(llm, 'ask'):
378 |             response_text = await llm.ask(prompt_architect
_query)
379 |         elif hasattr(llm, 'chat'):
380 |             response_text = await llm.chat(prompt_architec
t_query)
381 |         elif hasattr(llm, 'generate_response'):
382 |             response_text = await llm.generate_response(pr
ompt_architect_query)
383 |         else:
384 |             response_text = str(llm)
385 |
386 |         # Clean JSON from possible ```json wrappers
387 |         cleaned = response_text.strip()
388 |         if cleaned.startswith('```'):
389 |             cleaned = re.sub(r'^```(?:json)?\s*', '', clea

```

```

ned)
390         cleaned = re.sub(r'\s*```$', '', cleaned)
391
392         parsed_data = json.loads(cleaned)
393         # Validate recommended tools
394         if 'recommended_tools' in parsed_data:
395             parsed_data['recommended_tools'] = [
396                 t for t in parsed_data['recommended_tools']
397             ] if t in tool_ids
398
399         return {
400             'status': 'ok',
401             'data': parsed_data
402         }
403     except Exception as e:
404         logger.error(f'[router_agents] Error in AI prompt
generator: {e}')
405         # Fallback to default structure on model failure
406         return {
407             'status': 'fallback',
408             'data': {
409                 'name': req.agent_name or 'Custom agent',
410                 'description': req.task_description[:100],
411                 'system_prompt': f"You are a specialized a
i-breadboard agent.\nYour task: {req.task_description}\nUse pr
ovided tools when needed and provide clear answers.",
412                 'recommended_tools': ['web_search'],
413                 'temperature': 0.3,
414                 'max_steps': 10
415             },
416             'error': str(e)
417         }
418
419     @router.post('/test')
420     async def test_agent(req: TestAgentRequest) -> dict:
421         """Interactive sandbox: test agent run with step traci
ng.

```

```

422
423         Executes agent with provided configuration or inline settings,
424         returning step-by-step execution trace.
425
426         Args:
427             req: Request with agent ID or inline config and test message.
428
429         Returns:
430             dict: Execution result with response, duration, and steps trace.
431
432         Raises:
433             HTTPException: If agent configuration not found or execution fails.
434         """
435         start_time = time.time()
436
437         # Determine agent configuration
438         target_config = {}
439         if req.inline_config and isinstance(req.inline_config, AgentModel) and req.inline_config.name:
440             target_config = req.inline_config.dict()
441         elif req.agent_id:
442             items = _get_agents_list()
443             for item in items:
444                 if item.get('id') == req.agent_id:
445                     target_config = item
446                     break
447
448         if not target_config:
449             raise HTTPException(status_code=400, detail='Agent configuration not set for testing')
450
451         provider = target_config.get('provider', 'gemini')
452         model_name = target_config.get('model', 'gemini-2.5-flash')

```



```

453         sys_prompt = target_config.get('system_prompt', '')
454         tools = target_config.get('tools', [])
455
456         steps = []
457         steps.append({
458             'step': 1,
459             'type': 'thought',
460             'content': f"Initializing agent '{target_config.ge
t('name')}' [Provider: {provider}, Model: {model_name}]"
461         })
462
463         if tools:
464             steps.append({
465                 'step': 2,
466                 'type': 'tool_init',
467                 'content': f"Connected tools ({len(tools)}): {
', '.join(tools)}"
468             })
469
470         try:
471             from src.fastapi.router_chat import get_chat_model
472             model_key = model_name
473             if provider == 'gemini_cli' and not model_key.star
tswith('gemini_cli:'):
474                 model_key = f'gemini_cli:{model_key}'
475             elif provider == 'foundry' and not model_key.start
swith('foundry:'):
476                 model_key = f'foundry:{model_key}'
477             elif provider == 'ollama' and not model_key.starts
with('ollama:'):
478                 model_key = f'ollama:{model_key}'
479
480             llm = get_chat_model(model_key, system_instruction
=sys_prompt)
481
482             steps.append({
483                 'step': 3,
484                 'type': 'action',

```

```

485         'content': f"Processing incoming message: '{req.test_message}'"
486     })
487
488     # Execute model call
489     if hasattr(llm, 'ask'):
490         res = await llm.ask(req.test_message)
491     elif hasattr(llm, 'chat'):
492         res = await llm.chat(req.test_message)
493     else:
494         res = "Model response received."
495
496     duration_ms = int((time.time() - start_time) * 100
497 0)
498
499     steps.append({
500         'step': 4,
501         'type': 'finish',
502         'content': f"Completed successfully in {duration_ms} ms."
503     })
504
505     return {
506         'status': 'ok',
507         'response': res,
508         'duration_ms': duration_ms,
509         'steps': steps
510     }
511 except Exception as e:
512     duration_ms = int((time.time() - start_time) * 100
513 0)
514
515     logger.error(f'[router_agents] Error in agent test
516     run: {e}')
517
518     steps.append({
519         'step': len(steps) + 1,
520         'type': 'error',
521         'content': f"Execution error: {str(e)}"
522     })

```

```
518 |         return {
519 |             'status': 'error',
520 |             'response': f"Error during agent execution: {s
| tr(e)}",
521 |             'duration_ms': duration_ms,
522 |             'steps': steps
523 |         }
524 |
525 |     return router
526 |
```

router_auth.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: User authentication and OAuth router
4 # =====
5 # Description:
6 #   Provides FastAPI routing for user authentication, Google O
7 #   Auth integration,
8 #   JWT token generation, and secure session state validation.
9 #
10 # File: router_auth.py
11 # Project: ai-breadboard
12 # Package: src.fastapi
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 from __future__ import annotations
18
19 import json
20 import os
21 import secrets
22 from datetime import datetime, timedelta
23 from pathlib import Path
24 from typing import Optional
25
26 import jwt
27 import requests
28 from dotenv import load_dotenv
29 from fastapi import APIRouter, Cookie, HTTPException, Request,
30     Response
31 from fastapi.responses import RedirectResponse
32
33 from header import __root__
```

```

32 from src.logger import logger
33
34 load_dotenv()
35
36 router = APIRouter(prefix='/auth', tags=['auth'])
37
38 class TokenData:
39     """Данные токена авторизации."""
40
41     def __init__(self, email: str, name: Optional[str] = None,
42                  picture: Optional[str] = None, id: Optional[int] = None):
43         self.email = email
44         self.name = name
45         self.picture = picture
46         self.id = id
47
48 def load_google_oauth_config() -> dict:
49     """Loading конфигурации Google OAuth из secrets файла или
50     .env.
51
52     Returns:
53         dict: Configuration с client_id, client_secret, redirect_uri.
54
55     """
56     # Сначала пробуем загрузить из .env
57     env_client_id = os.getenv('GOOGLE_CLIENT_ID')
58     env_client_secret = os.getenv('GOOGLE_CLIENT_SECRET')
59     env_redirect_uri = os.getenv('GOOGLE_REDIRECT_URI')
60
61     from src.config import server_cfg
62     port = getattr(server_cfg, 'port', 8000)
63     default_redirect = f'http://localhost:{port}/auth/google/callback'
64
65     # Если в .env есть заглушки, пробуем загрузить из secrets
66     # файла
67     if env_client_id and env_client_id.startswith('YOUR_'):
68         secrets_file = __root__ / 'core' / 'secrets' / 'client

```

```

    _secret_*.json'
65         try:
66             for f in secrets_file.parent.glob(secrets_file.nam
e):
67                 with open(f, 'r', encoding='utf-8') as fp:
68                     data = json.load(fp)
69                     if 'web' in data:
70                         web = data['web']
71                         return {
72                             'client_id': web.get('client_id',
73                             ''),
74                             'client_secret': web.get('client_s
ecret', ''),
75                             'redirect_uri': env_redirect_uri o
r default_redirect
76                             }
77                     except Exception as e:
78                         logger.warning('Failed to load Google OAuth from s
ecrets file:', e, False)
79
80         return {
81             'client_id': env_client_id or '',
82             'client_secret': env_client_secret or '',
83             'redirect_uri': env_redirect_uri or default_redirect
84         }
85
86 # Configuration JWT
87 JWT_SECRET = os.getenv('JWT_SECRET', 'your-secret-key-change-i
n-production')
88 JWT_ALGORITHM = 'HS256'
89 ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24 # 24 часа
90
91 # Configuration Google OAuth (загружаем из .env или secrets)
92 _oauth_config = load_google_oauth_config()
93 GOOGLE_CLIENT_ID = _oauth_config['client_id']
94 GOOGLE_CLIENT_SECRET = _oauth_config['client_secret']
95 GOOGLE_REDIRECT_URI = _oauth_config['redirect_uri']

```

```

96 def create_jwt_token(data: TokenData) -> str:
97     """Создание JWT токена.
98
99     Args:
100         data: Данные для включения в токен.
101
102     Returns:
103         str: Закодированный JWT токен.
104     """
105     to_encode = data.__dict__.copy()
106     expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
107     to_encode.update({'exp': expire})
108     return jwt.encode(to_encode, JWT_SECRET, algorithm=JWT_ALGORITHM)
109
110 def verify_jwt_token(token: str) -> Optional[TokenData]:
111     """Check JWT токена.
112
113     Args:
114         token: JWT токен для проверки.
115
116     Returns:
117         Optional[TokenData]: Данные токена или None при ошибке
118         .
119     """
120     try:
121         payload = jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGORITHM])
122         return TokenData(
123             email=payload.get('email', ''),
124             name=payload.get('name'),
125             picture=payload.get('picture'),
126             id=payload.get('id')
127         )
128     except jwt.PyJWTError:
129         return None

```

```
130 # Хранилище for OAuth state (in production use Redis or simila
    r)
131 oauth_states: dict[str, datetime] = {}
132
133 def generate_state_token() -> str:
134     """Генерация state token для защиты от CSRF атак."""
135     state = secrets.token_urlsafe(32)
136     oauth_states[state] = datetime.utcnow() + timedelta(minute
        s=10)
137     return state
138
139 def validate_state_token(state: str) -> bool:
140     """Validation state token."""
141     if state not in oauth_states:
142         return False
143     # Clean up expired states
144     now = datetime.utcnow()
145     if oauth_states[state] < now:
146         del oauth_states[state]
147         return False
148     return True
149
150 @router.get('/login')
151 async def login(request: Request) -> dict:
152     """Check статуса авторизации пользователя.
153
154     Args:
155         request: FastAPI request object.
156
157     Returns:
158         dict: Status авторизации и данные пользователя.
159     """
160     token = request.cookies.get('auth_token')
161     if not token:
162         return {'authenticated': False}
163
164     user_data = verify_jwt_token(token)
165     if not user_data:
```



```

166         return {'authenticated': False}
167
168     return {
169         'authenticated': True,
170         'email': user_data.email,
171         'name': user_data.name,
172         'picture': user_data.picture
173     }
174
175 GOOGLE_SCOPES = [
176     'openid',
177     'https://www.googleapis.com/auth/userinfo.email',
178     'https://www.googleapis.com/auth/userinfo.profile',
179     'https://www.googleapis.com/auth/documents',
180     'https://www.googleapis.com/auth/drive',
181     'https://www.googleapis.com/auth/calendar',
182     'https://www.googleapis.com/auth/contacts',
183 ]
184
185 @router.get('/google')
186 async def google_login(request: Request, next: str = '/') -> R
187     edirectResponse:
188         """Перенаправление на Google OAuth страницу с расширенными
189         правами.
190
191         Returns:
192             RedirectResponse: Перенаправление на Google OAuth.
193         """
194         if not GOOGLE_CLIENT_ID:
195             raise HTTPException(
196                 status_code=500,
197                 detail='Google OAuth не настроен. Пожалуйста, свяж
198                 итесь с администратором.'
199             )
200
201         state = generate_state_token()
202
203         # Determine redirect URI:

```

```

201     # If GOOGLE_REDIRECT_URI is explicitly configured in .env,
    use it
202     if GOOGLE_REDIRECT_URI and not GOOGLE_REDIRECT_URI.startswith('http://localhost:'):
203         redirect_uri = GOOGLE_REDIRECT_URI
204     else:
205         # Fallback to dynamic request URL
206         host = request.headers.get('x-forwarded-host') or request.headers.get('host') or request.url.netloc
207         scheme = request.headers.get('x-forwarded-proto') or request.url.scheme
208         redirect_uri = f'{scheme}://{host}/auth/google/callback'
209
210     # Build Google OAuth URL with extended scopes & offline refresh token
211     import urllib.parse
212     scope_str = urllib.parse.quote(' '.join(GOOGLE_SCOPES))
213     oauth_url = (
214         'https://accounts.google.com/o/oauth2/v2/auth'
215         f'?client_id={GOOGLE_CLIENT_ID}'
216         f'&redirect_uri={redirect_uri}'
217         f'&response_type=code'
218         f'&scope={scope_str}'
219         f'&access_type=offline'
220         f'&prompt=consent'
221         f'&include_granted_scopes=true'
222         f'&state={state}'
223     )
224
225     response = RedirectResponse(url=oauth_url, status_code=302)
226
227     # Store state and exact redirect_uri in cookies for validation on callback
228     response.set_cookie('oauth_state', state, httponly=True, max_age=600)
229     response.set_cookie('oauth_redirect_uri', redirect_uri, httponly=True, max_age=600)

```

```

229
230     # Validate next parameter to prevent open redirects
231     if not next.startswith('/'):
232         next = '/'
233     response.set_cookie('next_redirect', next, httponly=True,
max_age=600)
234
235     return response
236
237 @router.get('/google/callback')
238 async def google_callback(request: Request, code: str, state:
str) -> RedirectResponse:
239     """Обработка callback от Google OAuth с сохранением токено
в доступа.
240
241     Args:
242         request: FastAPI request object.
243         code: Authorization code from Google.
244         state: State token for CSRF protection.
245
246     Returns:
247         RedirectResponse: Перенаправление на главную страницу
или сохраненный next.
248     """
249     # Validate state
250     cookie_state = request.cookies.get('oauth_state')
251     if not cookie_state or cookie_state != state:
252         logger.warning('Invalid state token in Google OAuth ca
llback')
253         raise HTTPException(status_code=400, detail='Неверный
state токен')
254
255     if not validate_state_token(state):
256         raise HTTPException(status_code=400, detail='Срок дейс
твия state токена истек')
257
258     # Exchange code for tokens with Google
259     redirect_uri = request.cookies.get('oauth_redirect_uri', '

```

```

    ')
260     if not redirect_uri:
261         if GOOGLE_REDIRECT_URI and not GOOGLE_REDIRECT_URI.starts
with('http://localhost:'):
262             redirect_uri = GOOGLE_REDIRECT_URI
263         else:
264             host = request.headers.get('x-forwarded-host') or
request.headers.get('host') or request.url.netloc
265             scheme = request.headers.get('x-forwarded-proto')
or request.url.scheme
266             redirect_uri = f'{scheme}://{host}/auth/google/cal
lback'
267
268     token_url = 'https://oauth2.googleapis.com/token'
269     token_data = {
270         'code': code,
271         'client_id': GOOGLE_CLIENT_ID,
272         'client_secret': GOOGLE_CLIENT_SECRET,
273         'redirect_uri': redirect_uri,
274         'grant_type': 'authorization_code',
275     }
276
277     try:
278         token_response = requests.post(token_url, data=token_d
ata)
279         token_response.raise_for_status()
280         token_json = token_response.json()
281
282         access_token = token_json.get('access_token', '')
283         refresh_token = token_json.get('refresh_token', '')
284         expires_in = token_json.get('expires_in', 3600)
285         granted_scope = token_json.get('scope', '')
286
287         if not access_token:
288             logger.error('Failed to get access token from Goog
le:', token_json)
289             raise HTTPException(status_code=500, detail='Не уд
алось получить токен доступа от Google')

```

```

290
291     # Get user info from Google
292     userinfo_url = 'https://www.googleapis.com/oauth2/v2/u
userinfo'
293     userinfo_headers = {'Authorization': f'Bearer {access_
token}'}
294     userinfo_response = requests.get(userinfo_url, headers
=userinfo_headers)
295     userinfo_response.raise_for_status()
296     userinfo = userinfo_response.json()
297
298     user_email = userinfo.get('email', '')
299     user_name = userinfo.get('name', '')
300     user_picture = userinfo.get('picture', '')
301
302     if not user_email:
303         logger.error('Failed to get email from Google user
info:', userinfo)
304         raise HTTPException(status_code=500, detail='Не уд
алось получить email от Google')
305
306     # Integration с UserManager
307     from src.user_manager import user_manager
308     db_user = user_manager.get_user_by_email(user_email)
309     if not db_user:
310         user_id = user_manager.add_user(email=user_email,
name=user_name, picture=user_picture, role='admin')
311     else:
312         user_id = db_user['id']
313         user_manager.update_user(user_id, name=user_name,
picture=user_picture, last_login=datetime.utcnow().isoformat()
)
314
315     # Сохранение Google OAuth токенов для доступа к Google
Docs / Calendar / Contacts
316     user_manager.save_google_tokens(
317         user_id=user_id,
318         access_token=access_token,

```

```

319         refresh_token=refresh_token,
320         expires_in=int(expires_in),
321         scope=granted_scope
322     )
323
324     # Создаем JWT токен с ID пользователя
325     token = create_jwt_token(TokenData(email=user_email, name=user_name, picture=user_picture, id=user_id))
326
327     next_redirect = request.cookies.get('next_redirect') or '/'
328     if not next_redirect.startswith('/'):
329         next_redirect = '/'
330
331     response = RedirectResponse(url=next_redirect, status_code=302)
332     response.set_cookie(
333         'auth_token',
334         token,
335         httponly=True,
336         secure=False, # Set to True in production with HTTPS
337         samesite='lax',
338         max_age=ACCESS_TOKEN_EXPIRE_MINUTES * 60
339     )
340     response.delete_cookie('next_redirect')
341
342     return response
343
344 except requests.exceptions.RequestException as e:
345     logger.error('Google OAuth request failed:', e, False)
346     raise HTTPException(status_code=500, detail='Error связи с Google OAuth')
347
348 @router.get('/google/status')
349 async def google_auth_status(request: Request) -> dict:
350     """Check наличия и валидности токенов Google OAuth у текущего пользователя."""

```

```

351 token = request.cookies.get('auth_token', '')
352 if not token:
353     return {'connected': False, 'reason': 'unauthorized'}
354 user_data = verify_jwt_token(token)
355 if not user_data:
356     return {'connected': False, 'reason': 'invalid_token'}
357 from src.user_manager import user_manager
358 tokens = user_manager.get_google_tokens(user_data.id) if u
ser_data.id else {}
359 if not tokens or not tokens.get('access_token'):
360     return {'connected': False, 'email': user_data.email}
361 return {
362     'connected': True,
363     'email': user_data.email,
364     'has_refresh_token': bool(tokens.get('refresh_token'))
365     ,
366     'scope': tokens.get('scope', ''),
367     'expires_at': tokens.get('expires_at', '')
368 }
369 @router.post('/logout')
370 async def logout(response: Response) -> dict:
371     """Выход пользователя из системы.
372
373     Args:
374         response: FastAPI response object.
375
376     Returns:
377         dict: Результат операции.
378     """
379     response.delete_cookie('auth_token')
380     response.delete_cookie('oauth_state')
381     return {'message': 'Успешный выход'}
382
383 @router.get('/check')
384 async def check_auth(request: Request) -> dict:
385     """Check текущей авторизации.
386

```

```
387 |     Args:
388 |         request: FastAPI request object.
389 |
390 |     Returns:
391 |         dict: Status авторизации.
392 |         """
393 |     token = request.cookies.get('auth_token', '')
394 |     if not token:
395 |         return {'authenticated': False}
396 |
397 |     user_data = verify_jwt_token(token)
398 |     if not user_data:
399 |         return {'authenticated': False}
400 |
401 |     db_user = user_manager.get_user_by_email(user_data.email)
402 |     if user_data.email else {}
403 |     role = db_user.get('role', 'user') if db_user else 'user'
404 |     is_admin = bool(db_user.get('is_admin', 0)) if db_user else False
405 |     user_id = db_user.get('id', 0) if db_user else 0
406 |     google_tokens = user_manager.get_google_tokens(user_id) if
407 |     user_id else {}
408 |     has_google = bool(google_tokens.get('refresh_token') or go
409 |     ogle_tokens.get('access_token'))
410 |
411 |     return {
412 |         'authenticated': True,
413 |         'id': user_id,
414 |         'email': user_data.email,
415 |         'name': user_data.name,
416 |         'picture': user_data.picture,
417 |         'role': role,
418 |         'is_admin': is_admin,
419 |         'has_google': has_google
420 |     }
```

```
419 | from pydantic import BaseModel
420 |
```



```
421 class EmailRegisterRequest(BaseModel):
422     email: str
423     password: str
424     name: str
425
426 class EmailVerifyRequest(BaseModel):
427     email: str
428     code: str
429
430 class EmailLoginRequest(BaseModel):
431     email: str
432     password: str
433
434 def send_verification_email(email: str, code: str):
435     import smtplib
436     from email.mime.text import MIMEText
437
438     subject = "Код подтверждения AI Assistant"
439     body = f"Ваш код подтверждения: {code}\n\nКод действителен
15 минут."
440
441     # Всегда пишем в логи
442     logger.info(f"=====")
443     logger.info(f"EMAIL VERIFICATION FOR {email}: {code}")
444     logger.info(f"=====")
445
446     smtp_host = os.getenv("SMTP_HOST")
447     smtp_port = os.getenv("SMTP_PORT")
448     smtp_user = os.getenv("SMTP_USER")
449     smtp_pass = os.getenv("SMTP_PASSWORD")
450
451     if smtp_host and smtp_port and smtp_user and smtp_pass:
452         try:
453             msg = MIMEText(body, 'plain', 'utf-8')
454             msg['Subject'] = subject
455             msg['From'] = smtp_user
456             msg['To'] = email
457
```

```

458         with smtplib.SMTP(smtp_host, int(smtp_port)) as se
server:
459             server.starttls()
460             server.login(smtp_user, smtp_pass)
461             server.send_message(msg)
462             logger.info(f"Verification email successfully sent
to {email}")
463         except Exception as e:
464             logger.error(f"Failed to send email to {email}:",
e, False)
465
466 @router.post('/register')
467 async def register(data: EmailRegisterRequest):
468     from src.user_manager import user_manager
469     email = data.email.lower().strip()
470
471     db_user = user_manager.get_user_by_email(email)
472     if db_user and db_user.get('is_email_verified', 0) == 1 an
d db_user.get('password_hash'):
473         raise HTTPException(status_code=400, detail='Пользова
тель с такой почтой уже зарегистрирован')
474
475     user_id = user_manager.register_email_user(email, data.pas
sword, data.name)
476     if not user_id:
477         raise HTTPException(status_code=500, detail='Не удалос
ь зарегистрировать пользователя')
478
479     code = user_manager.create_email_verification(email)
480     send_verification_email(email, code)
481
482     return {'message': 'Код подтверждения отправлен на почту',
'email': email}
483
484 @router.post('/verify')
485 async def verify(data: EmailVerifyRequest):
486     from src.user_manager import user_manager
487     email = data.email.lower().strip()

```

```
488     success = user_manager.verify_email_code(email, data.code)
489     if not success:
490         raise HTTPException(status_code=400, detail='Неверный
или истекший код подтверждения')
491     return {'message': 'Почта successfully подтверждена'}
492
493 @router.post('/login/email')
494 async def login_email(data: EmailLoginRequest, response: Respo
nse):
495     from src.user_manager import user_manager
496     email = data.email.lower().strip()
497     db_user = user_manager.get_user_by_email(email)
498     if not db_user:
499         raise HTTPException(status_code=401, detail='Неверный
email или пароль')
500
501     if not db_user.get('password_hash'):
502         raise HTTPException(status_code=400, detail='Для данно
го аккаунта не задан пароль. Войдите через Google или зарегист
рируйтесь.')
503
504     if not user_manager.verify_password(data.password, db_user
['password_hash']):
505         raise HTTPException(status_code=401, detail='Неверный
email или пароль')
506
507     if db_user.get('is_email_verified', 0) == 0:
508         code = user_manager.create_email_verification(email)
509         send_verification_email(email, code)
510         raise HTTPException(status_code=403, detail='Почта не
подтверждена. Новый код подтверждения отправлен на ваш email.'
)
511
512     token = create_jwt_token(TokenData(email=db_user['email'],
name=db_user['name'], picture=db_user['picture'], id=db_user[
'id']))
513     response.set_cookie(
514         'auth_token',
```

```
515         token,
516         httponly=True,
517         secure=False,
518         samesite='lax',
519         max_age=ACCESS_TOKEN_EXPIRE_MINUTES * 60
520     )
521     return {'message': 'Успешный вход', 'user': {'email': db_u
ser['email'], 'name': db_user['name']}}
522
523 @router.get('/telegram/callback')
524 async def telegram_callback(
525     request: Request,
526     id: int,
527     first_name: str,
528     last_name: Optional[str] = None,
529     username: Optional[str] = None,
530     photo_url: Optional[str] = None,
531     auth_date: int = 0,
532     hash: str = ""
533 ):
534     import hashlib
535     import hmac
536     import time
537
538     bot_token = os.getenv("TELEGRAM_BOT_TOKEN")
539     if not bot_token:
540         raise HTTPException(status_code=500, detail="Telegram
Bot Token не настроен на сервере")
541
542     if time.time() - auth_date > 86400:
543         raise HTTPException(status_code=400, detail="Срок дейс
твия авторизации Telegram истек")
544
545     params = {
546         'id': str(id),
547         'first_name': first_name,
548         'auth_date': str(auth_date)
549     }
```

```

550     if last_name:
551         params['last_name'] = last_name
552     if username:
553         params['username'] = username
554     if photo_url:
555         params['photo_url'] = photo_url
556
557     sorted_params = sorted(params.items())
558     data_check_string = "\n".join([f"{k}={v}" for k, v in sort
ed_params])
559
560     secret_key = hashlib.sha256(bot_token.encode('utf-8')).dig
est()
561     calculated_hash = hmac.new(secret_key, data_check_string.e
ncode('utf-8'), hashlib.sha256).hexdigest()
562
563     if calculated_hash != hash:
564         logger.warning(f"Invalid Telegram auth hash: {hash} vs
{calculated_hash}")
565         raise HTTPException(status_code=400, detail="Неверная
подпись данных Telegram")
566
567     from src.user_manager import user_manager
568     db_user = user_manager.get_user_by_telegram_id(id)
569
570     if not db_user:
571         temp_email = f"tg_{id}@telegram.bot"
572         db_user = user_manager.get_user_by_email(temp_email)
573         if not db_user:
574             name = first_name
575             if last_name:
576                 name += f" {last_name}"
577             user_id = user_manager.add_user(
578                 email=temp_email,
579                 name=name,
580                 picture=photo_url or "",
581                 role="user"
582             )

```

```

583         user_manager.update_user(user_id, telegram_id=id,
telegram_username=username)
584         db_user = user_manager.get_user_by_id(user_id)
585
586         token = create_jwt_token(TokenData(email=db_user['email'],
name=db_user['name'], picture=db_user['picture'], id=db_user[
'id']))
587
588         response = RedirectResponse(url='/', status_code=302)
589         response.set_cookie(
590             'auth_token',
591             token,
592             httponly=True,
593             secure=False,
594             samesite='lax',
595             max_age=ACCESS_TOKEN_EXPIRE_MINUTES * 60
596         )
597         return response
598
599 class SettingsUpdateRequest(BaseModel):
600     theme: Optional[str] = ""
601     language: Optional[str] = ""
602     tts_enabled: Optional[int] = 0
603     system_instruction: Optional[str] = ""
604     model: Optional[str] = ""
605     tts_system: Optional[str] = ""
606     tts_voice: Optional[str] = ""
607     search_engine: Optional[str] = ""
608
609 # =====
610 # Admin User Management API
611 # =====
612
613 class UserCreateRequest(BaseModel):
614     email: str
615     name: str
616     role: Optional[str] = 'user'
617     is_active: Optional[int] = 1

```

```
618 |
619 | class UserUpdateRequest(BaseModel):
620 |     name: Optional[str] = ""
621 |     email: Optional[str] = ""
622 |     role: Optional[str] = ""
623 |     is_active: Optional[int] = 0
624 |
625 | @router.get('/admin/users')
626 | async def list_users(request: Request):
627 |     """Получение списка всех пользователей (только для админис-
628 |     траторов)."""
629 |     token = request.cookies.get('auth_token')
630 |     if not token:
631 |         raise HTTPException(status_code=401, detail='Не авторизован')
632 |
633 |     user_data = verify_jwt_token(token)
634 |     if not user_data:
635 |         raise HTTPException(status_code=401, detail='Неверный сессионный токен')
636 |
637 |     from src.user_manager import user_manager
638 |     db_user = user_manager.get_user_by_email(user_data.email)
639 |
640 |     # Check прав администратора
641 |     if not db_user or not db_user.get('is_admin', 0):
642 |         raise HTTPException(status_code=403, detail='Только администраторы могут просматривать list пользователей')
643 |
644 |     users = user_manager.get_all_users(active_only=False)
645 |     return {'users': users}
646 |
647 | @router.put('/admin/users/{user_id}')
648 | async def update_user(user_id: int, data: UserUpdateRequest, request: Request):
649 |     """Update данных пользователя (только для администраторов)"""
650 |     token = request.cookies.get('auth_token')
```

```
650     if not token:
651         raise HTTPException(status_code=401, detail='Не авторизован')
652
653     user_data = verify_jwt_token(token)
654     if not user_data:
655         raise HTTPException(status_code=401, detail='Неверный сессионный токен')
656
657     from src.user_manager import user_manager
658     db_user = user_manager.get_user_by_email(user_data.email)
659
660     # Check прав администратора
661     if not db_user or not db_user.get('is_admin', 0):
662         raise HTTPException(status_code=403, detail='Только администраторы могут редактировать пользователей')
663
664     # Update пользователя
665     updates = {}
666     if data.name is not None:
667         updates['name'] = data.name
668     if data.email is not None:
669         updates['email'] = data.email
670     if data.role is not None:
671         updates['role'] = data.role
672     if data.is_active is not None:
673         updates['is_active'] = data.is_active
674
675     if updates:
676         success = user_manager.update_user(user_id, **updates)
677         if not success:
678             raise HTTPException(status_code=404, detail='Пользователь не найден')
679
680     return {'status': 'ok', 'user_id': user_id}
681
682 @router.delete('/admin/users/{user_id}')
683 async def delete_user(user_id: int, request: Request):
```



```
684 |     """Удаление пользователя (только для администраторов)."""
685 |     token = request.cookies.get('auth_token')
686 |     if not token:
687 |         raise HTTPException(status_code=401, detail='Не авторизован')
688 |
689 |     user_data = verify_jwt_token(token)
690 |     if not user_data:
691 |         raise HTTPException(status_code=401, detail='Неверный сессионный токен')
692 |
693 |     from src.user_manager import user_manager
694 |     db_user = user_manager.get_user_by_email(user_data.email)
695 |
696 |     # Check прав администратора
697 |     if not db_user or not db_user.get('is_admin', 0):
698 |         raise HTTPException(status_code=403, detail='Только администраторы могут удалять пользователей')
699 |
700 |     # Нельзя удалить самого себя
701 |     if db_user['id'] == user_id:
702 |         raise HTTPException(status_code=400, detail='Нельзя удалить самого себя')
703 |
704 |     success = user_manager.delete_user(user_id)
705 |     if not success:
706 |         raise HTTPException(status_code=404, detail='Пользователь не найден')
707 |
708 |     return {'status': 'ok', 'message': 'Пользователь удалён'}
709 |
710 | @router.get('/cabinet')
711 | async def get_cabinet(request: Request) -> dict:
712 |     """Получение данных личного кабинета пользователя."""
713 |     token = request.cookies.get('auth_token')
714 |     if not token:
715 |         raise HTTPException(status_code=401, detail='Не авторизован')
```

```
716     user_data = verify_jwt_token(token)
717     if not user_data:
718         raise HTTPException(status_code=401, detail='Неверный
сессионный токен')
719
720     from src.user_manager import user_manager
721     db_user = user_manager.get_user_by_email(user_data.email)
722     if not db_user:
723         raise HTTPException(status_code=404, detail='Пользоват
ель не найден в БД')
724
725     return {
726         'id': db_user['id'],
727         'email': db_user['email'],
728         'name': db_user['name'],
729         'picture': db_user['picture'],
730         'created_at': db_user['created_at'],
731         'telegram_username': db_user.get('telegram_username'),
732         'role': db_user['role'],
733         'is_admin': db_user['is_admin']
734     }
735
736 @router.post('/link-token')
737 async def get_link_token(request: Request) -> dict:
738     """Генерация временного токена для привязки Telegram-аккау
нта."""
739     token = request.cookies.get('auth_token')
740     if not token:
741         raise HTTPException(status_code=401, detail='Не автори
зован')
742     user_data = verify_jwt_token(token)
743     if not user_data or not user_data.email:
744         raise HTTPException(status_code=401, detail='Неверный
сессионный токен')
745
746     from src.user_manager import user_manager
747     db_user = user_manager.get_user_by_email(user_data.email)
748     if not db_user:
```

```

749         raise HTTPException(status_code=404, detail='Пользоват
ель не найден')
750
751     link_token = user_manager.generate_link_token(db_user['id'
])
752     return {'token': link_token}
753
754 @router.get('/settings')
755 async def get_settings(request: Request) -> dict:
756     """Получение настроек текущего пользователя."""
757     token = request.cookies.get('auth_token')
758     if not token:
759         raise HTTPException(status_code=401, detail='Не автори
зован')
760     user_data = verify_jwt_token(token)
761     if not user_data or not user_data.email:
762         raise HTTPException(status_code=401, detail='Неверный
сессионный токен')
763
764     from src.user_manager import user_manager
765     db_user = user_manager.get_user_by_email(user_data.email)
766     if not db_user:
767         raise HTTPException(status_code=404, detail='Пользоват
ель не найден')
768
769     settings = user_manager.get_user_settings(db_user['id'])
770
771     # Добавляем актуальный поисковый движок из config.json
772     try:
773         cfg_path = __root__ / 'config.json'
774         if cfg_path.exists():
775             with open(cfg_path, 'r', encoding='utf-8') as f:
776                 cfg = json.load(f)
777                 settings['search_engine'] = cfg.get('web_searc
h', {}).get('engine', 'gemini_cli')
778         else:
779             settings['search_engine'] = 'gemini_cli'
780     except Exception as e:

```

```
781         logger.warning(f'Не удалось прочитать search_engine из
| config.json: {e}')
782         settings['search_engine'] = 'gemini_cli'
783
784     return settings
785
786 @router.post('/settings')
787 async def update_settings(request: Request, data: SettingsUpda
| teRequest) -> dict:
788     """Update настроек текущего пользователя."""
789     token = request.cookies.get('auth_token')
790     if not token:
791         raise HTTPException(status_code=401, detail='Не автори
| зован')
792     user_data = verify_jwt_token(token)
793     if not user_data or not user_data.email:
794         raise HTTPException(status_code=401, detail='Неверный
| сессионный токен')
795
796     from src.user_manager import user_manager
797     db_user = user_manager.get_user_by_email(user_data.email)
798     if not db_user:
799         raise HTTPException(status_code=404, detail='Пользоват
| ель не найден')
800
801     success = user_manager.update_user_settings(
802         db_user['id'],
803         theme=data.theme,
804         language=data.language,
805         tts_enabled=data.tts_enabled,
806         system_instruction=data.system_instruction,
807         model=data.model,
808         tts_system=data.tts_system,
809         tts_voice=data.tts_voice
810     )
811     if not success:
812         raise HTTPException(status_code=500, detail='Не удалос
| ь сохранить настройки')
```

```
813 |     return {'status': 'ok'}
814 |
815 | def init_router() -> APIRouter:
816 |     """Initialization роутера авторизации.
817 |
818 |     Returns:
819 |         APIRouter: Настроенный роутер FastAPI.
820 |     """
821 |     return router
822 |
```

router_chat.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI chat execution and dialogue management router
4 # =====
5 # Description:
6 #   Handles AI conversational endpoints, streaming responses across multiple providers,
7 #   user dialogue context retrieval, and RAG query indexing workflows.
8 #
9 # File: router_chat.py
10 # Project: ai-breadboard
11 # Package: src.fastapi
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 from __future__ import annotations
17
18 import os
19 import time
20 import asyncio
21 from fastapi import APIRouter, HTTPException, Request
22 from pydantic import BaseModel
23
24 from src.logger import logger
25 from src.config import ai_cfg, tts_cfg
26 from src.ai.gemini.user_query_rag import index_user_query, search_user_context
27
28 router = APIRouter(prefix='/api/chat', tags=['chat'])
29
```

```
30 # Короткие слова-продолжения диалога, которые сами по себе не
    содержат медиа-ключевых слов
31 _CONTEXT_CONTINUATION_WORDS = {
32     'да', 'нет', 'yes', 'no', 'ок', 'ок', 'хочу', 'конечно',
33     'давай', 'проверь', 'найди', 'покажи', 'ладно', 'угу', 'уе
    р', 'sure',
34     'want', 'check', 'find', 'show', 'okay',
35 }
36
37 class ChatRequest(BaseModel):
38     message: str
39     history: list[dict] = []
40     generation_config: dict = {}
41
42 class SaveRagRequest(BaseModel):
43     query: str
44     chat_text: str
45     voice_text: str
46
47 class TestModelRequest(BaseModel):
48     model: str = ""
49     provider: str = ""
50     message: str = "Привет! Назови свою модель и провайдера, и
    подтверди готовность к работе."
51     system_instruction: str = ""
52
53 def get_chat_model(selected_model_name: str, system_instructio
    n: str = ""):
54     """Dynamically construct/retrieve the appropriate AI model
    instance."""
55     is_gemini_cli = selected_model_name.startswith('gemini_cli
    :') or selected_model_name.startswith('gemini-cli-')
56     is_foundry = selected_model_name.startswith('foundry:')
57     is_ollama = selected_model_name.startswith('ollama:')
58     is_hf = selected_model_name.startswith('hf:') or selected_
    model_name.startswith('hf::')
59     is_onnx = selected_model_name.startswith('onnx:') or selec
    ted_model_name.startswith('onnx::')
```

```

60     openai_prefixes = ('openai:', 'openai::', 'deepseek:', 'gr
    oq:', 'openrouter:', 'lmstudio:', 'local:', 'compat:')
61     is_openai = any(selected_model_name.startswith(p) for p in
    openai_prefixes)
62     is_agy = selected_model_name.startswith('agy-') or 'agy' i
    n selected_model_name.lower()
63     is_gemini = not is_gemini_cli and not is_hf and not is_onn
    x and not is_openai and (selected_model_name.startswith('gemin
    i-') or 'gemini' in selected_model_name.lower())
64
65     if is_gemini_cli:
66         from src.ai.gemini_cli_chat import GeminiCliChatBase
67         return GeminiCliChatBase(
68             model_id=selected_model_name,
69             system_prompt=system_instruction or "You are a hel
    pful AI assistant.",
70         )
71     elif is_foundry:
72         model_id = selected_model_name.split(':', 1)[-1]
73         from src.ai.foundry_chat import FoundryChatBase
74         return FoundryChatBase(
75             model_id=model_id,
76             system_prompt=system_instruction or "You are a hel
    pful AI assistant.",
77         )
78     elif is_ollama:
79         model_id = selected_model_name.split(':', 1)[-1]
80         from src.ai.ollama_chat import OllamaChatBase
81         ollama_url = ai_cfg.ollama_base_url if ai_cfg else 'ht
    tp://localhost:11434'
82         return OllamaChatBase(
83             model_id=model_id,
84             system_prompt=system_instruction or "You are a hel
    pful AI assistant.",
85             api_url=ollama_url
86         )
87     elif is_hf:
88         model_id = selected_model_name.split(':', 1)[-1].lstri

```



```

    p(':')
89         from src.ai.hf_chat import HFChatBase
90         return HFChatBase(
91             model_id=model_id,
92             system_prompt=system_instruction or "You are a hel
    pful AI assistant.",
93         )
94     elif is_onnx:
95         model_id = selected_model_name.split(':', 1)[-1].lstri
    p(':')
96         from src.ai.onnx_chat import ONNXChatBase
97         return ONNXChatBase(
98             model_id=model_id,
99             system_prompt=system_instruction or "You are a hel
    pful AI assistant.",
100         )
101     elif is_openai:
102         prov_part, model_part = selected_model_name.split(':',
    1)
103         model_id = model_part.lstrip(':')
104         prov_name = prov_part.lower().rstrip(':')
105         if prov_name == 'compat':
106             prov_name = 'openai'
107         from src.ai.openai_compat_chat import OpenAICompatChat
108         return OpenAICompatChat.create_for_provider(
109             provider_name=prov_name,
110             model_id=model_id,
111             system_prompt=system_instruction or "You are a hel
    pful AI assistant.",
112         )
113     elif is_agy:
114         from src.ai.agy_chat import AgyChatBase
115         return AgyChatBase(
116             model_id=selected_model_name,
117             system_prompt=system_instruction or "You are a hel
    pful AI assistant.",
118         )
119     elif is_gemini:

```

```

120         from src.ai.gemini.generative_ai import GoogleGenerati
veAI
121         _api_key_names = [n.strip() for n in os.getenv('GEMINI
_API_KEY_NAMES', '').split(',') if n.strip()]
122         return GoogleGenerativeAI(
123             model_name=selected_model_name,
124             api_key_names=_api_key_names,
125             system_instruction=system_instruction,
126             sleep_on_exhausted=False,
127         )
128     else:
129         # По умолчанию - Foundry для неизвестных моделей (обра
тная совместимость)
130         from src.ai.foundry_chat import FoundryChatBase
131         return FoundryChatBase(
132             model_id=selected_model_name,
133             system_prompt=system_instruction or "You are a hel
pful AI assistant.",
134         )
135
136 async def _extract_user_auth(fastapi_req: Request) -> tuple[str, str, str, dict]:
137     """Извлекает идентификатор пользователя, системную инструк
цию, модель и настройки из JWT/IP."""
138     user_identifier = ""
139     system_instruction = ""
140     selected_model = ""
141     settings = {}
142
143     token = fastapi_req.cookies.get('auth_token')
144     if token:
145         from src.fastapi.router_auth import verify_jwt_token
146         user_data = verify_jwt_token(token)
147         if user_data:
148             from src.user_manager import user_manager
149             db_user = await asyncio.to_thread(user_manager.get
_user_by_email, user_data.email)
150             if db_user:

```

```

151         user_identifier = str(db_user['id'])
152         settings = await asyncio.to_thread(user_manage
r.get_user_settings, db_user['id']) or {}
153         if settings.get('system_instruction'):
154             system_instruction = settings['system_inst
ruction']
155         if settings.get('model'):
156             selected_model = settings['model']
157
158     if not user_identifier:
159         client_ip = fastapi_req.client.host if fastapi_req.cli
ent else 'unknown'
160         user_identifier = f"anon_{client_ip}"
161
162     return user_identifier, system_instruction, selected_model
, settings
163
164 def _get_voice_gender_rule(settings: dict) -> str:
165     """Определяет гендерное правило для ответов ассистента на
основе настроек голоса TTS."""
166     default_voice = getattr(tts_cfg, "default_voice", "ru-RU-D
mitryNeural") if tts_cfg else "ru-RU-DmitryNeural"
167     tts_voice = settings.get('tts_voice', '') or default_voice
168     voice_lower = tts_voice.lower()
169     is_male_voice = any(name in voice_lower for name in ("dmit
ry", "yaraslaus", "male", "bayan", "aidar", "eugene", "georgy"
))
170     is_female_voice = any(name in voice_lower for name in ("sv
etlana", "elena", "female", "kseniya", "tanya", "aliona", "dar
iya"))
171     if is_male_voice:
172         return "ВАЖНОЕ ПРАВИЛО: Отвечай от мужского лица (напр
имер: 'Я нашел', 'Я подобрал')."
173     if is_female_voice:
174         return "ВАЖНОЕ ПРАВИЛО: Отвечай от женского лица (напр
имер: 'Я нашла', 'Я подобрала')."
175     return ""
176

```

```

177 def _clean_chat_history(history: list[dict]) -> list[dict]:
178     """Очищает историю сообщений перед передачей в модель."""
179     if not history:
180         return []
181     _ERROR_PATTERNS = (
182         ' ', 'Error', 'Error', 'TypeError', 'AttributeError',
183         'Traceback',
184         '[Error]', 'Не удалось найти', 'В локальной базе ничег
185         о не найдено',
186         'DEBUG MODE', 'DEBUG:', '[DEBUG'
187     )
188     _STATUS_PREFIXES = (
189         ' ', ' ', ' ', ' ', ' ', ' ', 'Вызов плагина', '
190         Генерация', 'Check',
191         'DEBUG MODE:', 'DEBUG:'
192     )
193
194     def _is_clean(entry: dict) -> bool:
195         parts = entry.get('parts', [])
196         text = ''
197         if isinstance(parts, list) and len(parts) > 0:
198             p = parts[0]
199             text = p if isinstance(p, str) else p.get('text',
200             '') if isinstance(p, dict) else ''
201             elif isinstance(parts, str):
202                 text = parts
203             text = (text or '').strip()
204             if not text:
205                 return False
206             if any(pfx in text for pfx in _ERROR_PATTERNS):
207                 return False
208             if any(text.startswith(pfx) for pfx in _STATUS_PREFIXE
209             S):
210                 return False
211             if text.startswith('{') and ('"title"' in text or '"er
212             ror"' in text or '"genres"' in text):
213                 return False
214             if 'Ответ модели: {' in text and '"title"' in text:

```

```

209         return False
210     return True
211
212     def _compact_turn(entry: dict) -> dict:
213         parts = entry.get('parts', [])
214         text = ''
215         if isinstance(parts, list) and len(parts) > 0:
216             p = parts[0]
217             text = p if isinstance(p, str) else p.get('text',
218             '') if isinstance(p, dict) else ''
219             elif isinstance(parts, str):
220                 text = parts
221
222         if entry.get('role') in ('model', 'assistant') and len
223         (text) > 200:
224             import re
225             compact = re.sub(r'<film>(.*?)</film>', r'«\1»', t
226             ext, flags=re.IGNORECASE)
227             compact = re.sub(r'#+\s*', '', compact)
228             compact = re.sub(r'[*_`]+', '', compact)
229             compact = re.sub(r'\s+', ' ', compact).strip()
230             if len(compact) > 200:
231                 compact = compact[:197].rsplit(' ', 1)[0] + '.
232             ...'
233
234         return {'role': entry['role'], 'parts': [compact]}
235     return entry
236
237     raw_clean = [e for e in history if _is_clean(e)]
238     cleaned_entries: list[dict] = []
239     i = 0
240     while i < len(raw_clean):
241         entry = raw_clean[i]
242         if entry.get('role') == 'user':
243             if i + 1 < len(raw_clean) and raw_clean[i + 1].get
244             ('role') in ('model', 'assistant'):
245                 cleaned_entries.append(_compact_turn(entry))
246                 cleaned_entries.append(_compact_turn(raw_clean
247                 [i + 1]))

```

```

241         i += 2
242         continue
243     i += 1
244
245     return cleaned_entries[-10:]
246
247 def _build_debug_prompt(request: ChatRequest, user_context_str
: str, voice_gender_rule: str) -> str:
248     """Формирует текстовый дамп полного промпта для отладочног
о режима."""
249     full_prompt_parts = []
250     dynamic_parts = []
251     if voice_gender_rule:
252         dynamic_parts.append(voice_gender_rule)
253     if user_context_str:
254         dynamic_parts.append(user_context_str)
255     if dynamic_parts:
256         full_prompt_parts.append("— DYNAMIC CONTEXT —\n" + "
\n\n".join(dynamic_parts))
257
258     clean_history = _clean_chat_history(request.history)
259     if clean_history:
260         full_prompt_parts.append("— CHAT HISTORY (последние 5
) —\n" + "\n---\n".join([
261             f"{entry.get('role', 'unknown').upper()}\n{entry.
get('parts', [''])[0] if isinstance(entry.get('parts'), list)
else entry.get('parts', '')}"
262             for entry in clean_history[-5:]
263         ]))
264
265     full_prompt_parts.append(f"— USER MESSAGE —\n{request.me
ssage}")
266     return "\n\n".join(full_prompt_parts)
267
268 def init_router(chat_model, narrator_model, plugins: dict = {}
) -> APIRouter:
269     """Initialization роутера чата с привязкой моделей (chat и
narrator)."""

```

```

270     if hasattr(narrator_model, 'gemini_model') and narrator_model.gemini_model:
271         narrator_model.gemini_model.save_history_chat = False
272
273     @router.get('/models')
274     async def get_models(refresh: bool = False) -> dict:
275         """Получение списка доступных моделей, сгруппированных
        по провайдеру."""
276         from src.ai.model_manager import get_available_models
277
278         gemini_models = get_available_models('gemini', force_refresh=refresh)
279
280         foundry_raw = get_available_models('foundry', force_refresh=refresh)
281         foundry_models = [f"foundry:{m}" if not m.startswith('foundry:') else m for m in foundry_raw]
282
283         ollama_raw = get_available_models('ollama', force_refresh=refresh)
284         ollama_models = [f"ollama:{m}" if not m.startswith('ollama:') else m for m in ollama_raw]
285
286         agy_models = get_available_models('agy', force_refresh=refresh)
287
288         gemini_cli_raw = get_available_models('gemini_cli', force_refresh=refresh)
289         gemini_cli_models = [f"gemini_cli:{m}" if not m.startswith('gemini_cli:') else m for m in gemini_cli_raw]
290
291         openai_raw = get_available_models('openai', force_refresh=refresh)
292         openai_models = [f"openai:{m}" if not any(m.startswith(f"{p}:") for p in ('openai', 'deepseek', 'groq', 'openrouter', 'lmstudio')) else m for m in openai_raw]
293
294         hf_raw = get_available_models('hf', force_refresh=refresh)

```

```

esh)
295         hf_models = [f"hf:{m}" if not m.startswith('hf:') else
        m for m in hf_raw]
296
297         onnx_raw = get_available_models('onnx', force_refresh=
refresh)
298         onnx_models = [f"onnx:{m}" if not m.startswith('onnx:'
        ) else m for m in onnx_raw]
299
300         return {
301             'models': {
302                 'gemini': gemini_models,
303                 'foundry': foundry_models,
304                 'ollama': ollama_models,
305                 'agy': agy_models,
306                 'gemini_cli': gemini_cli_models,
307                 'openai': openai_models,
308                 'hf': hf_models,
309                 'onnx': onnx_models,
310             }
311         }
312
313     @router.post('/test-model')
314     async def test_model(req: TestModelRequest) -> dict:
315         """Проверочный запрос к указанной AI-модели для валида
ции связи (Запрос -> Ответ)."""
316         start_time = time.perf_counter()
317         target_model = req.model.strip()
318         provider = req.provider.strip().lower()
319
320         if provider == 'foundry' and target_model and not targ
et_model.startswith('foundry:'):
321             target_model = f"foundry:{target_model}"
322         elif provider == 'ollama' and target_model and not tar
get_model.startswith('ollama:'):
323             target_model = f"ollama:{target_model}"
324         elif provider == 'agy' and target_model and not target
_model.startswith('agy-'):

```



```
325         target_model = f"agy-{target_model}"
326     elif provider in ('gemini_cli', 'gemini-cli') and target_model and not target_model.startswith('gemini_cli:'):
327         target_model = f"gemini_cli:{target_model}"
328     elif provider in ('openai', 'openai_compat', 'openai-compat', 'deepseek', 'groq', 'openrouter', 'lmstudio') and target_model:
329         openai_prefixes = ('openai:', 'deepseek:', 'groq:', 'openrouter:', 'lmstudio:', 'local:', 'compat:')
330         if not any(target_model.startswith(p) for p in openai_prefixes):
331             target_model = f"{provider}:{target_model}"
332     elif provider in ('hf', 'huggingface') and target_model and not target_model.startswith('hf:'):
333         target_model = f"hf:{target_model}"
334     elif provider == 'onnx' and target_model and not target_model.startswith('onnx:'):
335         target_model = f"onnx:{target_model}"
336
337     if not target_model:
338         return {
339             'status': 'error',
340             'message': 'Имя модели не указано',
341             'model': '',
342             'provider': provider,
343             'duration_ms': 0.0
344         }
345
346     test_msg = req.message.strip()
347     if not test_msg:
348         test_msg = "Привет! Назови свою модель и провайдер, и подтверди готовность к работе."
349
350     try:
351         model_instance = get_chat_model(target_model, system_instruction=req.system_instruction)
352         response_text = ""
353
```

```

354         if hasattr(model_instance, 'ask'):
355             response_text = await model_instance.ask(test_
msg)
356         elif hasattr(model_instance, 'chat'):
357             response_text = await model_instance.chat(test
_msg)
358         elif hasattr(model_instance, 'chat_stream'):
359             chunks = []
360             async for chunk in model_instance.chat_stream(
test_msg):
361                 if chunk:
362                     clean_chunk = chunk.replace("[CHAT]",
"").replace("[VOICE]", "")
363                     if clean_chunk:
364                         chunks.append(clean_chunk)
365                     response_text = "".join(chunks)
366             else:
367                 raise RuntimeError(f"Модель {target_model} не
поддерживает методы генерации текста")
368
369             duration_ms = round((time.perf_counter() - start_t
ime) * 1000, 1)
370             return {
371                 'status': 'success',
372                 'response': response_text,
373                 'model': target_model,
374                 'provider': provider,
375                 'duration_ms': duration_ms
376             }
377         except Exception as exc:
378             logger.error(f"[ChatRouter] Error проверочного зап
роса к модели {target_model}: {exc}", exc_info=True)
379             duration_ms = round((time.perf_counter() - start_t
ime) * 1000, 1)
380             return {
381                 'status': 'error',
382                 'message': str(exc),
383                 'model': target_model,

```

```

384         'provider': provider,
385         'duration_ms': duration_ms
386     }
387
388     @router.post('/save-rag')
389     async def save_to_rag(request: SaveRagRequest, fastapi_req
: Request):
390         """Ручное сохранение одобренного ответа в постоянный J
SON-архив."""
391         try:
392             user_idenfier, _, _, _ = await _extract_user_aut
h(fastapi_req)
393             from src.rag import save_user_approved_response
394             save_success = await asyncio.to_thread(
395                 save_user_approved_response,
396                 user_idenfier, request.query, request.chat_t
ext, request.voice_text
397             )
398             if save_success:
399                 return {"status": "success", "message": "Succe
ssfully сохранено для последующей компиляции RAG"}
400             else:
401                 raise HTTPException(status_code=500, detail="E
rror сохранения ответа")
402         except Exception as e:
403             logger.error("Error при ручном сохранении ответа",
e)
404             raise HTTPException(status_code=500, detail=str(e)
)
405
406     @router.post('/save-rag-instant')
407     async def save_to_rag_instant(request: SaveRagRequest, fas
tapi_req: Request):
408         """Мгновенное сохранение ответа: запись в JSON + векто
ризация в FAISS."""
409         try:
410             user_idenfier, _, _, _ = await _extract_user_aut
h(fastapi_req)

```

```

411         api_key = getattr(chat_model, 'api_key', '') or os
|         .getenv('GEMINI_API_KEY', '')
412
413         content_to_index = request.voice_text if request.v
|         oice_text.strip() else request.chat_text
414
415         from src.rag import save_user_approved_response, i
|         ndex_user_interaction
416         save_success = await asyncio.to_thread(
417             save_user_approved_response,
418             user_identifier, request.query, request.chat_t
|         ext, request.voice_text
419         )
420         rag_success = await asyncio.to_thread(
421             index_user_interaction, user_identifier, api_k
|         ey, request.query, content_to_index
422         )
423
424         if save_success and rag_success:
425             return {"status": "success", "message": "Succe
|             ssfully сохранено в архив и проиндексировано в RAG"}
426         elif save_success:
427             return {"status": "success", "message": "Cохра
|             нено в архив, но произошла Error при индексации в RAG"}
428         else:
429             raise HTTPException(status_code=500, detail="E
|             rror сохранения ответа")
430     except Exception as e:
431         logger.error("Error при мгновенном сохранении в RA
|         G", e)
432         raise HTTPException(status_code=500, detail=str(e)
|         )
433
434     @router.post('')
435     async def chat(request: ChatRequest, fastapi_req: Request)
|     :
436         """Обработка чат-запроса по алгоритму RAG-First: RAG S
|         earch -> Direct Match / LLM Fallback -> Auto-Index."""

```

```

437         from fastapi.responses import StreamingResponse
438         import json
439
440         async def event_generator():
441             try:
442                 user_idenfifier, system_instruction, selected_
443 model, settings = await _extract_user_auth(fastapi_req)
444
445                 api_key = getattr(chat_model, 'api_key', '') o
446 r os.getenv('GEMINI_API_KEY', '')
447
448                 # 1. RAG-First: Поиск по базе знаний
449 from src.rag import get_rag_engine, index_user
450 _interaction
451                 rag_engine = get_rag_engine()
452
453                 yield f"data: {json.dumps({'status': '❏ Поиск
454 в базе знаний (RAG)...'})}\n\n"
455                 decision = await rag_engine.evaluate(
456                     query=request.message,
457                     user_idenfifier=user_idenfifier,
458                     api_key=api_key
459                 )
460
461                 # 2. Если найден точный ответ — мгновенный воз
462 врат (Direct RAG)
463                 if decision.is_direct:
464                     yield f"data: {json.dumps({'status': decis
465 ion.status_message or '❏ Ответ найден в базе знаний...'})}\n\n
466 "
467
468                     yield f"data: {json.dumps({'text': decisio
469 n.direct_text})}\n\n"
470
471                     if decision.direct_voice:
472                         yield f"data: {json.dumps({'voice': de
473 cision.direct_voice})}\n\n"
474
475                     return
476
477                 # 3. Подготовка контекста для LLM

```

```

466         voice_gender_instruction = _get_voice_gender_r
         ule(settings)
467         dynamic_context_parts = []
468         if voice_gender_instruction:
469             dynamic_context_parts.append(f"[Правило]:
         {voice_gender_instruction}")
470         if decision.context_text:
471             dynamic_context_parts.append(decision.cont
         ext_text)
472
473         user_msg_with_context = request.message
474         if dynamic_context_parts:
475             user_msg_with_context = "\n\n".join(dynami
         c_context_parts) + "\n\n[Запрос пользователя]:\n" + request.me
         ssage
476
477         # Режим отладки (DEBUG MODE)
478         if request.generation_config.get('debug_mode',
         False):
479             debug_text = _build_debug_prompt(request,
         decision.context_text, voice_gender_instruction)
480             yield f"data: {json.dumps({'status': 'DEBU
         G MODE: Промпт сформирован, не отправляется в модель'})}\n\n"
481             yield f"data: {json.dumps({'text': debug_t
         ext})}\n\n"
482             return
483
484         token = fastapi_req.cookies.get('auth_token')
485         from src.fastapi.router_control import get_roo
         m_id
486         room_id = get_room_id(token, None)
487
488         if request.generation_config.get('model'):
489             selected_model = request.generation_config
         ['model']
490
491         clean_history = _clean_chat_history(request.hi
         story)

```

```

492         kwargs = {
493             'history': clean_history,
494             'room_id': room_id,
495             'model_name': selected_model,
496         }
497         if request.generation_config.get('search_engine'):
498             kwargs['search_engine'] = request.generation_config['search_engine']
499
500         if selected_model:
501             active_model = get_chat_model(selected_model, None)
502             api_key = getattr(active_model, 'api_key', '') or getattr(chat_model, 'api_key', '') or api_key
503         else:
504             active_model = chat_model
505
506         yield f"data: {json.dumps({'status': 'Генерация ответа (этап 1)...'})}\n\n"
507
508         chat_kwargs_1 = kwargs.copy()
509         chat_kwargs_1.pop('room_id', '')
510         chat_kwargs_1.pop('search_engine', None)
511         gen_cfg_1 = request.generation_config.copy()
512         gen_cfg_1['response_type'] = 'chat'
513         chat_kwargs_1['generation_config'] = gen_cfg_1
514
515         stream_generator_1 = active_model.chat_stream(
516             user_msg_with_context, **chat_kwargs_1)
517
518         chat_response = ""
519         async for chunk in stream_generator_1:
520             if chunk:
521                 c = chunk.replace("[CHAT]", "").replace("[VOICE]", "")
522                 if c:
523                     chat_response += c

```

```

523 |             yield f"data: {json.dumps({'text':
| c}})\n\n"
524 |
525 |             voice_response = ""
526 |             if chat_response:
527 |                 yield f"data: {json.dumps({'status': 'Гене
| рация голоса (этап 2)...'})}\n\n"
528 |
529 |                 chat_kwargs_2 = kwargs.copy()
530 |                 chat_kwargs_2.pop('room_id', '')
531 |                 chat_kwargs_2.pop('search_engine', None)
532 |                 chat_kwargs_2['history'] = []
533 |
534 |                 gen_cfg_2 = request.generation_config.copy
| ()
535 |                 gen_cfg_2['response_type'] = 'voice'
536 |                 chat_kwargs_2['generation_config'] = gen_c
| fg_2
537 |
538 |                 stream_generator_2 = narrator_model.chat_s
| tream(chat_response, **chat_kwargs_2)
539 |                 async for chunk in stream_generator_2:
540 |                     if chunk:
541 |                         c = chunk.replace("[CHAT]", "").re
| place("[VOICE]", "")
542 |                         if c:
543 |                             voice_response += c
544 |                             yield f"data: {json.dumps({'vo
| ice': c}})\n\n"
545 |
546 |             # 4. Автоматическая фоновая индексация взаимодей
| ствия в RAG
547 |             content_to_index = voice_response if voice_res
| ponse.strip() else chat_response
548 |             if content_to_index and api_key and user_ident
| ifier:
549 |                 asyncio.ensure_future(asyncio.to_thread(
| index_user_interaction, user_identifie
550 |

```



```
    r, api_key, request.message, content_to_index
551 |         ))
552 |
553 |         except Exception as ex:
554 |             logger.error('Error обработки чат-запроса', ex
555 |             )
556 |             yield f"data: {json.dumps({'error': str(ex)})}"
557 |             "\n\n"
558 |
559 |         return StreamingResponse(event_generator(), media_type
560 |         ="text/event-stream")
561 |
562 |     return router
```

router_control.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: WebSocket remote control and player synchroniz
  ation router
4 # =====
5 # Description:
6 #   Manages WebSocket connections and REST endpoints for synch
  ronized remote control,
7 #   media player state broadcasts, and device pairing rooms.
8 #
9 # File: router_control.py
10 # Project: ai-breadboard
11 # Package: src.fastapi
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 from __future__ import annotations
17
18 import json
19 from typing import Dict, List, Optional
20 from fastapi import APIRouter, WebSocket, WebSocketDisconnect,
  Query, Request, HTTPException
21 from src.logger import logger
22 from src.fastapi.router_auth import verify_jwt_token
23
24 class ControlConnectionManager:
25     """Менеджер WebSocket соединений для пульта и плеера."""
26
27     def __init__(self):
28         # Хранение активных подключений: room_id -> {"player":
  [ws...], "remote": [ws...]}
29         self.rooms: Dict[str, Dict[str, List[WebSocket]]] = {}
```

```

30         # Хранение последнего известного состояния плеера: room_id -> dict
31         self.room_states: Dict[str, dict] = {}
32         # Хранение плейлиста плеера: room_id -> list
33         self.room_playlists: Dict[str, list] = {}
34
35     async def connect(self, websocket: WebSocket, room_id: str, role: str):
36         """Connection клиента с ролью 'player' или 'remote' в комнату."""
37         await websocket.accept()
38         if room_id not in self.rooms:
39             self.rooms[room_id] = {"player": [], "remote": []}
40
41         if role not in ("player", "remote"):
42             role = "remote"
43
44         self.rooms[room_id][role].append(websocket)
45         logger.info(f"WebSocket connected: room={room_id}, role={role}")
46
47         # Если подключается remote и у нас есть сохраненное state, сразу отправляем его
48         if role == "remote":
49             if room_id in self.room_states:
50                 try:
51                     await websocket.send_json(self.room_states[room_id])
52                 except Exception as e:
53                     logger.warning(f"Error sending state on connect to remote: {e}")
54             if room_id in self.room_playlists:
55                 try:
56                     await websocket.send_json({
57                         "event": "playlist_update",
58                         "files": self.room_playlists[room_id]
59                     })
60                 except Exception as e:

```

```

61         logger.warning(f"Error sending playlist on
    connect to remote: {e}")
62
63     def disconnect(self, websocket: WebSocket, room_id: str, r
ole: str):
64         """Disconnection клиента из комнаты."""
65         if room_id in self.rooms and role in self.rooms[room_i
d]:
66             if websocket in self.rooms[room_id][role]:
67                 self.rooms[room_id][role].remove(websocket)
68                 # Если комната пуста, очищаем её
69                 if not self.rooms[room_id]["player"] and not self.
rooms[room_id]["remote"]:
70                     self.rooms.pop(room_id, None)
71                     self.room_states.pop(room_id, None)
72                     self.room_playlists.pop(room_id, None)
73                 logger.info(f"WebSocket disconnected: room={room_id},
role={role}")
74
75     async def broadcast_to_role(self, room_id: str, target_rol
e: str, data: dict):
76         """Пересылает сообщение всем клиентам с целевой ролью
    в комнате."""
77         if room_id not in self.rooms:
78             return
79
80         targets = self.rooms[room_id].get(target_role, [])
81         disconnected = []
82
83         for ws in targets:
84             try:
85                 await ws.send_json(data)
86             except Exception as e:
87                 logger.warning(f"Failed to send to {target_rol
e} in room {room_id}: {e}")
88                 disconnected.append(ws)
89
90         for ws in disconnected:

```

```

91         self.disconnect(ws, room_id, target_role)
92
93 manager = ControlConnectionManager()
94 router = APIRouter(prefix="/api/control", tags=["control"])
95
96 def get_room_id(token: Optional[str], room: Optional[str]) ->
    str:
97     """Определение идентификатора комнаты на основе JWT токена
    или параметров."""
98     if room and room.strip():
99         return room.strip().lower()
100     if token:
101         token_data = verify_jwt_token(token)
102         if token_data and token_data.email:
103             return token_data.email.strip().lower()
104     return "default"
105
106 @router.websocket("/ws")
107 async def websocket_control_endpoint(
108     websocket: WebSocket,
109     role: str = Query("remote"),
110     token: Optional[str] = Query(None),
111     room: Optional[str] = Query(None)
112 ):
113     """WebSocket эндпоинт для управления плеером в реальном вр
    емени."""
114     cookie_token = websocket.cookies.get("auth_token")
115     logger.info(f"WS connection request: role={role}, query_to
    ken={'present' if token else 'None'}, query_room={room}")
116     if cookie_token and not token:
117         token = cookie_token
118
119     room_id = get_room_id(token, room)
120     logger.info(f"WS room resolved: role={role}, room_id={room
    _id}")
121     await manager.connect(websocket, room_id, role)
122
123     try:

```

```

124 |         while True:
125 |             text_data = await websocket.receive_text()
126 |             try:
127 |                 data = json.loads(text_data)
128 |             except json.JSONDecodeError:
129 |                 continue
130 |
131 |             if role == "remote":
132 |                 # Пульт sends команды плееру
133 |                 await manager.broadcast_to_role(room_id, "play
134 | er", data)
135 |             elif role == "player":
136 |                 # Плеер обновляет свой status или плейлист для
137 | пульта
138 |                 event_type = data.get("event")
139 |                 if event_type == "status_update":
140 |                     manager.room_states[room_id] = data
141 |                 elif event_type == "playlist_update":
142 |                     manager.room_playlists[room_id] = data.get
143 | ("files", [])
144 |                 await manager.broadcast_to_role(room_id, "remo
145 | te", data)
146 |
147 |             except WebSocketDisconnect:
148 |                 manager.disconnect(websocket, room_id, role)
149 |             except Exception as e:
150 |                 logger.error(f"WebSocket error in room {room_id}: {e}"
151 | )
152 |                 manager.disconnect(websocket, room_id, role)
153 |
154 | @router.get("/status")
155 | async def get_control_status(request: Request, token: Optional
156 | [str] = None, room: Optional[str] = None):
157 |     """Получить текущее state комнаты по HTTP."""
158 |     cookie_token = request.cookies.get("auth_token")
159 |     if cookie_token and not token:
160 |         token = cookie_token

```

```

156
157     room_id = get_room_id(token, room)
158     return {
159         "room_id": room_id,
160         "has_player": len(manager.rooms.get(room_id, {}).get("
player", [])) > 0,
161         "remotes_count": len(manager.rooms.get(room_id, {}).ge
t("remote", [])),
162         "state": manager.room_states.get(room_id, None),
163         "playlist_count": len(manager.room_playlists.get(room_
id, []))
164     }
165
166 @router.get("/active_players")
167 async def get_active_players(request: Request):
168     """Получить list комнат с активными плеерами."""
169     token = request.cookies.get("auth_token")
170     user_email = None
171     if token:
172         token_data = verify_jwt_token(token)
173         if token_data and token_data.email:
174             user_email = token_data.email.strip().lower()
175
176     active = []
177     for room_id, roles in manager.rooms.items():
178         if len(roles.get("player", [])) > 0:
179             active.append(room_id)
180
181     logger.info(f"active_players endpoint called: token={token
}, user_email={user_email}, active_rooms={active}")
182     return {"players": active, "user_email": user_email}
183
184 @router.get("/rescan")
185 async def rescan_storage(request: Request):
186     """Принудительное пересканирование доступных хранилищ."""
187     # Check прав администратора
188     token = request.cookies.get("auth_token")
189     if not token:

```

```
190         raise HTTPException(status_code=403)
191
192     from src.fastapi.router_auth import verify_jwt_token
193     user_data = verify_jwt_token(token)
194     if not user_data:
195         raise HTTPException(status_code=403)
196
197     from src.user_manager import user_manager
198     db_user = user_manager.get_user_by_email(user_data.email)
199     if not db_user or (not db_user.get('is_admin', 0) and db_u
ser.get('role') != 'admin'):
200         raise HTTPException(status_code=403)
201
202     import psutil
203     drives = []
204     drives_details = []
205     for d in psutil.disk_partitions():
206         drives.append(d.device)
207         try:
208             usage = psutil.disk_usage(d.mountpoint)
209             total_gb = round(usage.total / (1024 ** 3), 1)
210             used_gb = round(usage.used / (1024 ** 3), 1)
211             free_gb = round(usage.free / (1024 ** 3), 1)
212             percent = usage.percent
213             drives_details.append({
214                 "device": d.device,
215                 "mountpoint": d.mountpoint,
216                 "fstype": d.fstype,
217                 "total_gb": total_gb,
218                 "used_gb": used_gb,
219                 "free_gb": free_gb,
220                 "percent": percent
221             })
222         except Exception:
223             drives_details.append({
224                 "device": d.device,
225                 "mountpoint": d.mountpoint,
226                 "fstype": d.fstype,
```



```
227 |         "total_gb": 0.0,
228 |         "used_gb": 0.0,
229 |         "free_gb": 0.0,
230 |         "percent": 0.0
231 |     })
232 |     return {"status": "success", "drives": drives, "details":
| drives_details}
233 |
234 | def init_router() -> APIRouter:
235 |     """Initialization роутера управления.
236 |     Returns:
237 |         APIRouter: Настроенный роутер FastAPI.
238 |     """
239 |     return router
240 |
241 |
```

router_keys.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: API key management and quota tracking router
4 # =====
5 # Description:
6 #   Provides FastAPI endpoints for managing Gemini API keys, t
7 #   racking quota status,
8 #   handling key rotation, and performing CRUD key operations.
9 #
10 # File: router_keys.py
11 # Project: ai-breadboard
12 # Package: src.fastapi
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 from __future__ import annotations
18
19 import json
20 from datetime import datetime, timezone
21 from pathlib import Path
22 from typing import Dict, List, Optional
23
24 from fastapi import APIRouter, HTTPException
25 from pydantic import BaseModel
26
27 from src.logger import logger
28
29 router = APIRouter(prefix='/api/keys', tags=['keys'])
30
31 _SECRETS_FILE = Path(__file__).parent.parent / 'ai' / 'gemini'
32     / 'secrets.json'
33
34 _KEYS_FILE = Path(__file__).parent.parent / 'secrets' / 'gemin
```

```

    i_keys.json'
32 _DAY_SECONDS = 86400
33
34 # =====
    =====
35 # Pydantic Models
36 # =====
    =====
37
38 class KeyEntry(BaseModel):
39     api_key: str
40     status: str = 'active'
41     last_run: Optional[str] = None
42     exhausted_at: Optional[str] = None
43
44 class KeyCreateRequest(BaseModel):
45     name: str
46     api_key: str
47     status: str = 'active'
48
49 class KeyUpdateRequest(BaseModel):
50     status: Optional[str] = None
51     name: Optional[str] = None
52
53 class KeyListResponse(BaseModel):
54     keys: List[Dict]
55     total: int
56
57 class KeyStatusResponse(BaseModel):
58     name: str
59     status: str
60     last_run: Optional[str]
61     exhausted_at: Optional[str]
62     exhausted: bool
63     reset_in_seconds: Optional[int]
64
65 # =====
    =====

```

```

66 # Internal Helper Functions
67 # =====
68
69 def _load_secrets() -> Dict[str, str]:
70     """Load all keys from secrets.json (email -> api_key)."""
71     if _SECRETS_FILE.exists():
72         try:
73             return json.loads(_SECRETS_FILE.read_text(encoding
74                             = 'utf-8'))
75         except Exception as ex:
76             logger.error('Failed to load secrets.json', ex)
77             raise HTTPException(status_code=500, detail='Failed
78                             to load secrets.json')
79     return {}
80
81 def _save_secrets(data: Dict[str, str]) -> None:
82     """Save keys to secrets.json."""
83     try:
84         _SECRETS_FILE.write_text(json.dumps(data, ensure_ascii
85                             = False, indent=4), encoding='utf-8')
86     except Exception as ex:
87         logger.error('Failed to save secrets.json', ex)
88         raise HTTPException(status_code=500, detail='Failed to
89                             save secrets.json')
90
91 def _load_keys_data() -> Dict:
92     """Load status data from gemini_keys.json."""
93     if _KEYS_FILE.exists():
94         try:
95             return json.loads(_KEYS_FILE.read_text(encoding='u
96                             tf-8'))
97         except Exception as ex:
98             logger.error('Failed to load keys data', ex)
99             raise HTTPException(status_code=500, detail='Failed
100                             to load keys data')
101     return {}

```

```

97 def _save_keys_data(data: Dict) -> None:
98     """Save status data to gemini_keys.json."""
99     try:
100         _KEYS_FILE.write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding='utf-8')
101     except Exception as ex:
102         logger.error('Failed to save keys data', ex)
103         raise HTTPException(status_code=500, detail='Failed to save keys data')
104
105 def _now_iso() -> str:
106     """Return current UTC time in ISO format."""
107     return datetime.now(timezone.utc).isoformat()
108
109 def _iso_to_ts(iso: str) -> float:
110     """Convert ISO string to timestamp."""
111     try:
112         return datetime.fromisoformat(iso).timestamp()
113     except Exception:
114         return 0.0
115
116 def _check_exhaustion(name: str) -> tuple[bool, Optional[int]]:
117     """Check if key is exhausted and return seconds until reset."""
118     keys_data = _load_keys_data()
119     entry = keys_data.get(name, {})
120
121     exhausted_at = entry.get('exhausted_at')
122     if not exhausted_at:
123         return False, None
124
125     elapsed = _now_ts() - _iso_to_ts(exhausted_at)
126     remaining = _DAY_SECONDS - elapsed
127
128     if remaining > 0:
129         return True, int(remaining)
130     return False, None

```

```

131
132 def _now_ts() -> float:
133     """Return current Unix timestamp."""
134     return datetime.now(timezone.utc).timestamp()
135
136 def _mask_key(api_key: str) -> str:
137     """Mask API key for display: keep first 8 and last 4 chars
138     ."""
139     if len(api_key) < 12:
140         return '*' * len(api_key)
141     return f"{api_key[:8]}...{api_key[-4:]}"
142
143 # =====
144 # API Endpoints
145 # =====
146
147 @router.get('', response_model=KeyListResponse)
148 async def list_keys() -> KeyListResponse:
149     """List all API keys from secrets.json with masked values
150     and status."""
151     secrets = _load_secrets()
152     keys_data = _load_keys_data()
153     keys = []
154
155     for name, api_key in secrets.items():
156         exhausted, reset_in = _check_exhaustion(name)
157
158         # Get status from keys_data or default to 'active'
159         entry = keys_data.get(name, {})
160         status = entry.get('status', 'active')
161
162         keys.append({
163             'name': name,
164             'api_key_masked': _mask_key(api_key),
165             'status': status,
166             'last_run': entry.get('last_run'),

```

```

165         'exhausted_at': entry.get('exhausted_at'),
166         'exhausted': exhausted,
167         'reset_in_seconds': reset_in
168     })
169
170     return KeyListResponse(keys=keys, total=len(keys))
171
172 @router.get('/{key_name}', response_model=KeyStatusResponse)
173 async def get_key_status(key_name: str) -> KeyStatusResponse:
174     """Get detailed status for a specific key from secrets.jso
175     n."""
176     secrets = _load_secrets()
177     keys_data = _load_keys_data()
178
179     if key_name not in secrets:
180         raise HTTPException(status_code=404, detail=f'Key "{key_name}" not found in secrets.json')
181
182     entry = keys_data.get(key_name, {})
183     exhausted, reset_in = _check_exhaustion(key_name)
184
185     return KeyStatusResponse(
186         name=key_name,
187         status=entry.get('status', 'active'),
188         last_run=entry.get('last_run'),
189         exhausted_at=entry.get('exhausted_at'),
190         exhausted=exhausted,
191         reset_in_seconds=reset_in
192     )
193
194 @router.post('', status_code=201)
195 async def create_key(request: KeyCreateRequest) -> Dict[str, str]:
196     """Add a new API key to secrets.json."""
197     if not request.name or not request.api_key:
198         raise HTTPException(status_code=400, detail='Name and API key are required')

```

```

199     secrets = _load_secrets()
200     keys_data = _load_keys_data()
201
202     if request.name in secrets:
203         raise HTTPException(status_code=409, detail=f'Key "{request.name}" already exists in secrets.json')
204
205     # Add to secrets.json
206     secrets[request.name] = request.api_key
207     _save_secrets(secrets)
208
209     # Initialize status in keys_data
210     keys_data[request.name] = {
211         'status': request.status,
212         'last_run': None,
213         'exhausted_at': None
214     }
215     _save_keys_data(keys_data)
216
217     logger.info(f'Added new key: {request.name}')
218
219     return {'message': f'Key "{request.name}" added successfully'}
220
221 @router.delete("/{key_name}")
222 async def delete_key(key_name: str) -> Dict[str, str]:
223     """Delete an API key from secrets.json."""
224     secrets = _load_secrets()
225     keys_data = _load_keys_data()
226
227     if key_name not in secrets:
228         raise HTTPException(status_code=404, detail=f'Key "{key_name}" not found in secrets.json')
229
230     # Delete from secrets.json
231     del secrets[key_name]
232     _save_secrets(secrets)
233

```



```
234 |     # Delete from keys_data
235 |     if key_name in keys_data:
236 |         del keys_data[key_name]
237 |         _save_keys_data(keys_data)
238 |
239 |     logger.info(f'Deleted key: {key_name}')
240 |
241 |     return {'message': f'Key "{key_name}" deleted successfully
    | '}
242 |
243 | @router.patch('/{key_name}')
244 | async def update_key(key_name: str, request: KeyUpdateRequest)
    | -> Dict[str, str]:
245 |     """Update key status or name in secrets.json and keys_data
    | ."""
246 |     secrets = _load_secrets()
247 |     keys_data = _load_keys_data()
248 |
249 |     if key_name not in secrets:
250 |         raise HTTPException(status_code=404, detail=f'Key "{key
    | y_name}" not found in secrets.json')
251 |
252 |     # Update status
253 |     if request.status:
254 |         if request.status not in ('active', 'disabled'):
255 |             raise HTTPException(status_code=400, detail='Statu
    | s must be "active" or "disabled"')
256 |             keys_data[key_name]['status'] = request.status
257 |
258 |     # Rename key
259 |     if request.name and request.name != key_name:
260 |         if request.name in secrets:
261 |             raise HTTPException(status_code=409, detail=f'Key
    | "{request.name}" already exists')
262 |
263 |         # Rename in secrets.json
264 |         secrets[request.name] = secrets[key_name]
265 |         del secrets[key_name]
```

```

266         _save_secrets(secrets)
267
268         # Rename in keys_data
269         keys_data[request.name] = keys_data[key_name]
270         del keys_data[key_name]
271
272     _save_keys_data(keys_data)
273     logger.info(f'Updated key: {key_name}')
274
275     return {'message': f'Key "{key_name}" updated successfully'}
276
277 @router.post('/reset-all')
278 async def reset_all_quotas() -> Dict[str, str]:
279     """Reset daily quota exhaustion for all API keys."""
280     keys_data = _load_keys_data()
281     reset_count = 0
282
283     for name, info in keys_data.items():
284         if 'exhausted_at' in info:
285             del info['exhausted_at']
286             reset_count += 1
287
288     if reset_count > 0:
289         _save_keys_data(keys_data)
290         logger.info(f'Reset quota for {reset_count} keys')
291         return {'message': f'Successfully сброшены квоты для {reset_count} ключей'}
292
293     return {'message': 'Нет заблокированных ключей для сброса'}
294
295 @router.post('/{key_name}/reset-quota')
296 async def reset_quota(key_name: str) -> Dict[str, str]:
297     """Reset daily quota exhaustion for a key."""
298     keys_data = _load_keys_data()
299
300     if key_name not in keys_data:

```

```
301         raise HTTPException(status_code=404, detail=f'Key "{key_name}" not found')
302
303     if 'exhausted_at' in keys_data[key_name]:
304         del keys_data[key_name]['exhausted_at']
305         _save_keys_data(keys_data)
306         logger.info(f'Reset quota for key: {key_name}')
307         return {'message': f'Quota reset for key "{key_name}"'}
308     }
309     return {'message': f'Key "{key_name}" is not exhausted'}
310
311 # =====
312 # Initialization Function
313 # =====
314
315 def init_router() -> APIRouter:
316     """Initialize the keys router."""
317     return router
318
```

router_logs.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Server logs inspection and AI analysis router
4 # =====
5 # Description:
6 #   Provides FastAPI endpoints for viewing application logs, c
7 #   learing log files,
8 #   streaming log outputs, and performing automated AI log ana
9 #   lysis.
10 #
11 # File: router_logs.py
12 # Project: ai-breadboard
13 # Package: src.fastapi
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # =====
17
18 from __future__ import annotations
19
20 import datetime
21 import os
22 import tempfile
23 from pathlib import Path
24
25 from fastapi import APIRouter, HTTPException
26 from fastapi.responses import PlainTextResponse
27 from pydantic import BaseModel
28
29 from header import __root__
30 from src.logger import logger
31
32 LOG_DIR = Path(tempfile.gettempdir()) / 'ai-breadboard' / 'log
33 s'
```

```
31 | REPORTS_DIR = Path(tempfile.gettempdir()) / 'ai-breadboard' /  
    | 'reports'  
32 |  
33 | # List разрешённых имён файлов (whitelist)  
34 | _ALLOWED_EXTENSIONS = {'.log', '.json', '.md', '.txt'}  
35 |  
36 | def _safe_log_path(filename: str) -> Path:  
37 |     """Returns путь к файлу логов после проверки безопасности.  
    | """  
38 |     path = (LOG_DIR / filename).resolve()  
39 |     if not str(path).startswith(str(LOG_DIR.resolve())):  
40 |         raise HTTPException(status_code=400, detail='Недопусти  
    | мый путь к файлу')  
41 |     if path.suffix not in _ALLOWED_EXTENSIONS:  
42 |         raise HTTPException(status_code=400, detail='Недопусти  
    | мое расширение файла')  
43 |     return path  
44 |  
45 | def _safe_report_path(filename: str) -> Path:  
46 |     """Returns путь к файлу отчёта после проверки безопасности  
    | ."""  
47 |     path = (REPORTS_DIR / filename).resolve()  
48 |     if not str(path).startswith(str(REPORTS_DIR.resolve())):  
49 |         raise HTTPException(status_code=400, detail='Недопусти  
    | мый путь к файлу')  
50 |     if path.suffix not in _ALLOWED_EXTENSIONS:  
51 |         raise HTTPException(status_code=400, detail='Недопусти  
    | мое расширение файла')  
52 |     return path  
53 |  
54 | def _file_info(p: Path) -> dict:  
55 |     """Returns метаданные файла."""  
56 |     stat = p.stat()  
57 |     return {  
58 |         'name': p.name,  
59 |         'size': stat.st_size,  
60 |         'size_kb': round(stat.st_size / 1024, 1),  
61 |         'modified': datetime.datetime.fromtimestamp(stat.st_mt
```

```

    ime).strftime('%Y-%m-%d %H:%M:%S'),
62     }
63
64 class AnalyzeRequest(BaseModel):
65     filename: str
66
67 def init_router(prefix: str = '/api/logs') -> APIRouter:
68     router = APIRouter(prefix=prefix, tags=['logs'])
69
70     # -----
71     # GET /api/logs/files – list лог-файлов
72     # -----
73     @router.get('/files')
74     async def list_log_files() -> dict:
75         """Returns list доступных лог-файлов с их метаданными.
76         """
77         LOG_DIR.mkdir(parents=True, exist_ok=True)
78         files = []
79         for p in sorted(LOG_DIR.iterdir()):
80             if p.is_file() and p.suffix in _ALLOWED_EXTENSIONS:
81                 files.append(_file_info(p))
82         return {'files': files, 'count': len(files)}
83     # -----
84     # GET /api/logs/read?filename=info.log&tail=200 – чтение ф
85     # айла
86     # -----
87     @router.get('/read')
88     async def read_log_file(filename: str, tail: int = 500) ->
89     dict:
90         """Returns последние `tail` строк лог-файла."""
91         path = _safe_log_path(filename)
92         if not path.exists():

```

```

91         raise HTTPException(status_code=404, detail=f'Файл
    не найден: {filename}')
92
93     try:
94         with open(path, 'r', encoding='utf-8', errors='rep
lace') as f:
95             lines = f.readlines()
96         except Exception as e:
97             raise HTTPException(status_code=500, detail=f'Erro
r чтения файла: {e}')
98
99         total = len(lines)
100        sliced = lines[-tail:] if tail > 0 else lines
101        return {
102            'filename': filename,
103            'total_lines': total,
104            'returned_lines': len(sliced),
105            'content': ''.join(sliced),
106        }
107
108        # -----
-----
109        # DELETE /api/logs/clear?filename=info.log – очистка файла
110        # -----
-----
111        @router.delete('/clear')
112        async def clear_log_file(filename: str) -> dict:
113            """Очищает содержимое лог-файла (обнуляет, не deletes)
            ."""
114            path = _safe_log_path(filename)
115            if not path.exists():
116                raise HTTPException(status_code=404, detail=f'Файл
не найден: {filename}')
117
118            try:
119                with open(path, 'w', encoding='utf-8') as f:
120                    f.truncate(0)
121                logger.info(f'Лог-файл очищен вручную: {filename}')

```

```

    )
122         except Exception as e:
123             raise HTTPException(status_code=500, detail=f'Error при очистке: {e}')
124
125         return {'status': 'ok', 'message': f'Файл {filename} очищен'}
126
127         # -----
128         # POST /api/logs/analyze – запуск AI-анализа
129         # -----
130         @router.post('/analyze')
131         async def analyze_log(body: AnalyzeRequest) -> dict:
132             """Запускает AI-анализ лог-файла через Gemini."""
133             path = _safe_log_path(body.filename)
134             if not path.exists():
135                 raise HTTPException(status_code=404, detail=f'Файл не найден: {body.filename}')
136
137             try:
138                 from src.logger.log_analyzer import analyze_log_file
139                 from src.ai import GoogleGenerativeAI
140
141                 api_key_names = [
142                     n.strip()
143                     for n in os.getenv('GEMINI_API_KEY_NAMES', '').split(',')
144                     if n.strip()
145                 ]
146                 ai_model = GoogleGenerativeAI(
147                     api_key_names=api_key_names,
148                     system_instruction=(
149                         'Вы – профессиональный аналитик системных логов. '
150                         'Исследуйте логи, выявляйте ошибки, пробле

```



```

мы, тренды '
151         'и давайте рекомендации по устранению.'
152     ),
153 )
154
155     import asyncio
156     asyncio.create_task(analyze_log_file(path, ai_mode
157 l))
158
159     return {'status': 'started', 'message': f'Анализ ф
160 айла {body.filename} запущен в фоне'}
161
162 except Exception as e:
163     logger.error(f'Error запуска AI-анализа для {body.
164 filename}', e)
165     raise HTTPException(status_code=500, detail=f'Erro
166 r: {e}')
167
168 # -----
169 -----
170 # GET /api/logs/reports – list AI-отчётов
171 # -----
172 -----
173 @router.get('/reports')
174 async def list_reports() -> dict:
175     """Returns list AI-отчётов из папки logs/reports/."""
176     if not REPORTS_DIR.exists():
177         return {'reports': [], 'count': 0}
178
179     reports = []
180     for p in sorted(REPORTS_DIR.iterdir(), key=lambda x: x
181 .stat().st_mtime, reverse=True):
182         if p.is_file() and p.suffix in _ALLOWED_EXTENSIONS
183 :
184             reports.append(_file_info(p))
185     return {'reports': reports, 'count': len(reports)}
186
187 # -----
188 -----

```

```

179 |     # GET /api/logs/report?filename=master_journal.md – чтение
    |     отчёта
180 |     # -----
    |     -----
181 |     @router.get('/report')
182 |     async def read_report(filename: str) -> dict:
183 |         """Returns содержимое AI-отчёта."""
184 |         path = _safe_report_path(filename)
185 |         if not path.exists():
186 |             raise HTTPException(status_code=404, detail=f'Отчё
    | т не найден: {filename}')
187 |
188 |         try:
189 |             content = path.read_text(encoding='utf-8', errors=
    | 'replace')
190 |         except Exception as e:
191 |             raise HTTPException(status_code=500, detail=f'Erro
    | r чтения отчёта: {e}')
192 |
193 |         return {
194 |             'filename': filename,
195 |             'content': content,
196 |             'size': len(content),
197 |         }
198 |
199 |     # -----
    |     -----
200 |     # GET /api/logs/stats – сводная статистика
201 |     # -----
    |     -----
202 |     @router.get('/stats')
203 |     async def log_stats() -> dict:
204 |         """Returns сводную статистику по лог-файлам."""
205 |         LOG_DIR.mkdir(parents=True, exist_ok=True)
206 |         stats = {
207 |             'total_size_bytes': 0,
208 |             'file_count': 0,
209 |             'report_count': 0,

```

```

210         'files': [],
211     }
212
213     for p in LOG_DIR.iterdir():
214         if p.is_file() and p.suffix in _ALLOWED_EXTENSIONS
215         :
216             sz = p.stat().st_size
217             stats['total_size_bytes'] += sz
218             stats['file_count'] += 1
219             stats['files'].append({'name': p.name, 'size':
220             sz})
221
222     if REPORTS_DIR.exists():
223         stats['report_count'] = sum(
224             1 for p in REPORTS_DIR.iterdir()
225             if p.is_file() and p.suffix in _ALLOWED_EXTENS
226             IONS
227             )
228
229     stats['total_size_kb'] = round(stats['total_size_bytes
230     ']/ 1024, 1)
231     stats['total_size_mb'] = round(stats['total_size_bytes
232     ']/ (1024 * 1024), 2)
233     return stats
234
235     return router
236

```

```

router_openai.py
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: OpenAI compatibility API router
4 # =====
5 # Description:
6 #   Implements OpenAI-compatible REST API endpoints including
7 #   model listing
8 #   and chat completion streaming for external client integrat
9 #   ions.
10 #
11 # File: router_openai.py
12 # Project: ai-breadboard
13 # Package: src.fastapi
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # =====
17
18 import asyncio
19 import json
20 import time
21 from typing import Any, Dict, List
22
23 from fastapi import APIRouter, HTTPException, Request
24 from fastapi.responses import JSONResponse, StreamingResponse
25
26 from src.ai.model_manager import get_available_models
27 from src.fastapi.router_chat import get_chat_model
28 from src.logger.logger import logger
29
30 router = APIRouter(tags=["openai"])
31
32 def map_to_openai_id(provider_prefixed_id: str) -> str:
33     """Conversion внутреннего идентификатора модели в формат 0

```

```

penAI."""
32     if "::" in provider_prefixed_id:
33         return provider_prefixed_id.replace("::", "-", 1)
34     if ":" in provider_prefixed_id:
35         return provider_prefixed_id.replace(":", "-", 1)
36     return provider_prefixed_id
37
38 def map_from_openai_id(openai_id: str) -> str:
39     """Conversion OpenAI ID обратно во внутренний формат с пре
фиксом."""
40     for provider in ("foundry", "ollama", "hf", "onnx", "opena
i", "gemini_cli", "agy"):
41         prefix = f"{provider}-"
42         if openai_id.startswith(prefix):
43             return f"{provider}:{openai_id[len(prefix):]}"
44     return openai_id
45
46 def _collect_all_models_sync() -> List[Dict[str, Any]]:
47     """Сбор всех моделей по всем провайдерам для OpenAPI катал
ога."""
48     providers = ["gemini", "gemini_cli", "agy", "foundry", "ol
lama", "hf", "onnx"]
49     models_list: List[Dict[str, Any]] = []
50
51     for prov in providers:
52         try:
53             prov_models = get_available_models(prov)
54             for m in prov_models:
55                 openai_id = map_to_openai_id(f"{prov}:{m}" if
not m.startswith(f"{prov}:") else m)
56                 models_list.append({
57                     "id": openai_id,
58                     "object": "model",
59                     "created": int(time.time()),
60                     "owned_by": prov,
61                     "permission": [],
62                     "root": openai_id,
63                     "parent": "",

```

```

64         })
65     except Exception as e:
66         logger.warning(f"[RouterOpenAI] Не удалось собрать
        модели для {prov}: {e}")
67
68     return models_list
69
70 @router.get("/v1/models")
71 @router.get("/models")
72 async def list_models() -> Dict[str, Any]:
73     """List всех доступных моделей в формате OpenAI API."""
74     loop = asyncio.get_running_loop()
75     data = await loop.run_in_executor(None, _collect_all_model
        s_sync)
76     return {
77         "object": "list",
78         "data": data,
79     }
80
81 @router.post("/v1/chat/completions")
82 @router.post("/chat/completions")
83 async def chat_completions(request: Request) -> Any:
84     """Универсальный эндпоинт OpenAI /chat/completions."""
85     try:
86         body = await request.json()
87     except Exception:
88         raise HTTPException(status_code=400, detail="Invalid J
        SON body")
89
90     messages = body.get("messages", [])
91     if not isinstance(messages, list) or not messages:
92         raise HTTPException(status_code=400, detail="messages
        is required and must be a non-empty list")
93
94     system_parts: List[str] = []
95     user_parts: List[str] = []
96
97     for msg in messages:

```

```

98         if not isinstance(msg, dict):
99             continue
100         role = msg.get("role", "user")
101         content = msg.get("content", "")
102         if isinstance(content, list):
103             content = "\n".join(
104                 str(part.get("text", "")) if isinstance(part,
dict) else str(part)
105                 for part in content
106             )
107         if role == "system":
108             system_parts.append(str(content))
109         else:
110             user_parts.append(f"{role.title()}: {content}")
111
112     prompt = "\n".join(user_parts).strip()
113     system_instruction = "\n".join(system_parts).strip()
114     if not prompt:
115         raise HTTPException(status_code=400, detail="User mess
age content cannot be empty")
116
117     requested_model = body.get("model", "")
118     internal_model = map_from_openai_id(requested_model) if re
quested_model else "gemini-flash-latest"
119
120     temperature = float(body.get("temperature", 0.7))
121     max_tokens = int(body.get("max_tokens", body.get("max_comp
letion_tokens", 2048)))
122     stream = bool(body.get("stream", False))
123
124     now = int(time.time())
125     completion_id = f"chatcml-{now}"
126
127     try:
128         chat_model = get_chat_model(internal_model, system_ins
truction=system_instruction)
129     except Exception as e:
130         logger.error(f"[RouterOpenAI] Error инициализации моде

```

```

ли {internal_model}: {e}")
131         raise HTTPException(status_code=502, detail=f"Model in
initialization error: {e}")
132
133     if stream:
134         async def event_generator():
135             try:
136                 if hasattr(chat_model, "generate_content_stream"):
137                     async for chunk in chat_model.generate_content_stream(
138                         prompt, temperature=temperature, max_tokens=max_tokens
139                     ):
140                         data = {
141                             "id": completion_id,
142                             "object": "chat.completion.chunk",
143                             "created": now,
144                             "model": requested_model or internal_model,
145                             "choices": [
146                                 {
147                                     "index": 0,
148                                     "delta": {"role": "assistant", "content": chunk},
149                                     "finish_reason": "",
150                                 }
151                             ],
152                         }
153                         yield f"data: {json.dumps(data, ensure_ascii=False)}\n\n"
154             else:
155                 full_text = await chat_model.generate_content(
156                     prompt, temperature=temperature, max_tokens=max_tokens
157                 )
158                 data = {

```



```

159         "id": completion_id,
160         "object": "chat.completion.chunk",
161         "created": now,
162         "model": requested_model or internal_model,
163         "choices": [
164             {
165                 "index": 0,
166                 "delta": {"role": "assistant",
167                     "content": full_text},
168                 "finish_reason": "",
169             },
170         ],
171         yield f"data: {json.dumps(data, ensure_ascii=False)}\n\n"
172
173         final_data = {
174             "id": completion_id,
175             "object": "chat.completion.chunk",
176             "created": now,
177             "model": requested_model or internal_model
178             ,
179             "choices": [{"index": 0, "delta": {}, "finish_reason": "stop"}],
180             yield f"data: {json.dumps(final_data, ensure_ascii=False)}\n\n"
181             yield "data: [DONE]\n\n"
182         except Exception as exc:
183             logger.error(f"[RouterOpenAI] Error в SSE строке: {exc}")
184             yield f"data: {json.dumps({'error': str(exc)}, ensure_ascii=False)}\n\n"
185             yield "data: [DONE]\n\n"
186
187         return StreamingResponse(event_generator(), media_type="text/event-stream")

```

```

188
189     try:
190         content = await chat_model.generate_content(
191             prompt, temperature=temperature, max_tokens=max_to
192             kens
193         )
194         return {
195             "id": completion_id,
196             "object": "chat.completion",
197             "created": now,
198             "model": requested_model or internal_model,
199             "choices": [
200                 {
201                     "index": 0,
202                     "message": {"role": "assistant", "content"
203                     : content},
204                     "finish_reason": "stop",
205                 }
206             ],
207             "usage": {
208                 "prompt_tokens": len(prompt.split()),
209                 "completion_tokens": len(content.split()),
210                 "total_tokens": len(prompt.split()) + len(cont
211                 ent.split()),
212             }
213         }
214     except Exception as e:
215         logger.error(f"[RouterOpenAI] Error генерации для моде
216         ли {internal_model}: {e}")
217         raise HTTPException(status_code=502, detail=f"Generati
218         on failed: {e}")

```

router_tts.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Text to speech synthesis router
4 # =====
5 # Description:
6 #   Provides FastAPI endpoints for speech synthesis, voice list discovery,
7 #   audio stream chunking, and multi-engine TTS provider integration.
8 #
9 # File: router_tts.py
10 # Project: ai-breadboard
11 # Package: src.fastapi
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 from __future__ import annotations
17
18 import os
19 import re
20 import uuid
21 import asyncio
22 from pathlib import Path
23 from typing import AsyncGenerator
24 from fastapi import APIRouter, HTTPException, Query, Request
25 from fastapi.responses import StreamingResponse, FileResponse
26
27 from src.logger import logger
28 from src.ai.voice_pipeline import generate_voiceover_chunks
29
30 _ROOT = Path(__file__).parent.parent.parent
31 # Директория для хранения сгенерированных аудиофайлов TTS
```

```

32 TTS_CACHE_DIR = _ROOT / "tmp" / "tts_cache"
33 TTS_CACHE_DIR.mkdir(parents=True, exist_ok=True)
34
35 from src.tts import synthesize_speech
36
37 async def _synthesize_chunk_to_file(text: str, file_path: Path
    , tts_system: str = "edge-tts", voice: str = "ru-RU-DmitryNeur
    al"):
38     """Синтезирует отдельный чанк текста при помощи Модулей
    TTS системы."""
39     await synthesize_speech(text, file_path, tts_system, voice
    )
40
41 def _get_user_tts_config(request: Request) -> tuple[str, str]:
42     """Returns (tts_system, tts_voice) для текущего пользовате
    ля или дефолтные значения."""
43     tts_system = "edge-tts"
44     from src.config import tts_cfg
45     tts_voice = getattr(tts_cfg, "default_voice", "ru-RU-Dmitr
    yNeural") if tts_cfg else "ru-RU-DmitryNeural"
46
47     token = request.cookies.get('auth_token')
48     if token:
49         try:
50             from src.fastapi.router_auth import verify_jwt_tok
51         except:
52             pass
53         from src.user_manager import user_manager
54         user_data = verify_jwt_token(token)
55         if user_data and user_data.email:
56             db_user = user_manager.get_user_by_email(user_
57             data.email)
58             if db_user:
59                 settings = user_manager.get_user_settings(
60                 db_user['id'])
61                 tts_system = settings.get('tts_system') or
62                 tts_system
63                 tts_voice = settings.get('tts_voice') or t
64                 ts_voice

```

```

59         except Exception as e:
60             logger.error(f"Error getting user TTS settings: {e
        | })")
61
62     return tts_system, tts_voice
63
64 def _adjust_voice_for_language(text: str, current_voice: str)
    -> str:
65     """Эвристически определяет язык текста и подменяет голос,
    | если он не совпадает с языком."""
66     import re
67     # Подсчитываем количество букв кириллицы и латиницы
68     ru_count = len(re.findall(r'[А-Яа-яЁё]', text))
69     en_count = len(re.findall(r'[A-Za-z]', text))
70
71     # Если кириллицы больше, считаем текст русским
72     if ru_count >= en_count:
73         if current_voice.lower().startswith('en-'):
74             # Возвращаем дефолтный русский голос
75             return "ru-RU-DmitryNeural"
76     else:
77         # Считаем текст английским
78         if current_voice.lower().startswith('ru-'):
79             # Возвращаем дефолтный английский голос
80             return "en-US-AriaNeural"
81
82     return current_voice
83
84 def init_router(prefix: str = "/api/tts") -> APIRouter:
85     router = APIRouter(prefix=prefix, tags=["tts"])
86
87     @router.get("/stream-text")
88     async def stream_text(
89         media_id: int = Query(..., description="ID медиафайла
    | в БД"),
90         field: str = Query("plot", description="Какое поле озв
    | учить (plot, facts, why_watch, final_verdict)")
91     ):

```

```

92 |         """
93 |         Стримит адаптированные для диктора куски текста через
94 |         Server-Sent Events (SSE).
95 |         """
96 |         db = _db()
97 |         # Получаем запись из базы данных по media_id
98 |         conn = db.get_connection()
99 |         cursor = conn.cursor()
100 |         cursor.execute(f"SELECT {field}, title_ru FROM media W
101 | HERE id = ?", (media_id,))
102 |         row = cursor.fetchone()
103 |         if not row or not row[0]:
104 |             raise HTTPException(status_code=404, detail="Медиа
105 | файл или указанное поле не найдены")
106 |
107 |         raw_text = row[0]
108 |
109 |         async def sse_generator() -> AsyncGenerator[str, None]
110 | :
111 |             try:
112 |                 # Генерируем чанки на лету с помощью Gemini
113 |                 async for chunk in generate_voiceover_chunks(r
114 | aw_text):
115 |                     # Отправляем в формате SSE
116 |                     yield f"data: {chunk}\n\n"
117 |             except Exception as e:
118 |                 logger.error(f"Error in SSE stream-text: {e}")
119 |                 yield f"data: Error при обработке текста: {str
120 | (e)}\n\n"
121 |
122 |         return StreamingResponse(sse_generator(), media_type="
123 | text/event-stream")
124 |
125 | @router.get("/synthesize")
126 | async def synthesize(
127 |     request: Request,
128 |     text: str = Query(..., description="Текст для озвучки"
129 | ),

```

```

122         system: str = Query("", description="TTS система (edge
    -tts, gtts, silero)"),
123         voice: str = Query("", description="Голос диктора")
124     ):
125         """
126         Принимает текст, синтезирует его в аудиофайл и сразу о
    тдает mp3.
127         """
128         if not text.strip():
129             raise HTTPException(status_code=400, detail="Empty
    text")
130
131         # Получаем настройки озвучки текущего пользователя или
    Parameters запроса
132         user_system, user_voice = _get_user_tts_config(request
    )
133         tts_system = system if system != "" else user_system
134         tts_voice = voice if voice != "" else user_voice
135
136         tts_voice = _adjust_voice_for_language(text, tts_voice
    )
137
138         # Создаем уникальное имя файла на основе хэша текста и
    настроек озвучки
139         import hashlib
140         config_str = f"{text}_{tts_system}_{tts_voice}"
141         text_hash = hashlib.md5(config_str.encode("utf-8")).he
    xdigest()
142         file_path = TTS_CACHE_DIR / f"{text_hash}.mp3"
143
144         if not file_path.exists():
145             try:
146                 await _synthesize_chunk_to_file(text, file_pat
    h, tts_system, tts_voice)
147             except Exception as e:
148                 logger.error(f"TTS synthesis error: {e}")
149                 raise HTTPException(status_code=500, detail=f"
    Error синтеза речи: {e}")

```

```

150
151         return FileResponse(file_path, media_type="audio/mpeg"
152         , filename="voiceover.mp3")
153
154     @router.get("/stream-audio")
155     async def stream_audio(
156         request: Request,
157         media_id: int = Query(..., description="ID медиафайла
158         в БД"),
159         field: str = Query("plot", description="Поле для озвуч
160         ки"),
161         system: str = Query("", description="TTS система"),
162         voice: str = Query("", description="Голос диктора")
163     ):
164         """
165         Конвейерный эндпоинт:
166         1. На лету разбивает/адаптирует текст через Gemini.
167         2. Синтезирует каждый чанк в mp3.
168         3. Стримит клиенту JSON-события с текстом чанка и ссыл
169         кой на его аудиофайл,
170         чтобы фронтенд мог мгновенно проигрывать готовые ку
171         ски по очереди.
172         """
173         db = _db()
174         conn = db.get_connection()
175         cursor = conn.cursor()
176         cursor.execute(f"SELECT {field} FROM media WHERE id =
177         ?", (media_id,))
178         row = cursor.fetchone()
179         if not row or not row[0]:
180             raise HTTPException(status_code=404, detail="Медиа
181             файл или поле не найдены")
182
183         raw_text = row[0]
184         user_system, user_voice = _get_user_tts_config(request
185         )
186         tts_system = system if system != "" else user_system
187         tts_voice = voice if voice != "" else user_voice

```



```

180
181         async def sse_audio_generator() -> AsyncGenerator[str,
None]:
182             import json
183             import hashlib
184             idx = 0
185             async for chunk in generate_voiceover_chunks(raw_t
ext):
186                 # Подстраиваем голос под язык чанка
187                 adjusted_voice = _adjust_voice_for_language(ch
unk, tts_voice)
188
189                 # Для каждого адаптированного текстового чанка
запускаем синтез с учетом настроек
190                 config_str = f"{chunk}_{tts_system}_{adjusted_
voice}"
191                 text_hash = hashlib.md5(config_str.encode("utf
-8")).hexdigest()
192                 file_path = TTS_CACHE_DIR / f"{text_hash}.mp3"
193
194                 if not file_path.exists():
195                     try:
196                         await _synthesize_chunk_to_file(chunk,
file_path, tts_system, adjusted_voice)
197                     except Exception as e:
198                         logger.error(f"Failed to synthesize ch
unk {idx}: {e}")
199                     continue
200
201                 # Отправляем JSON с текстом и ссылкой на скачи
вание
202                 audio_url = f"/api/tts/file/{text_hash}.mp3"
203                 payload = {
204                     "index": idx,
205                     "text": chunk,
206                     "audio_url": audio_url
207                 }
208                 yield f"data: {json.dumps(payload, ensure_asci

```

```
        i=False))\n\n"
209         idx += 1
210
211         return StreamingResponse(sse_audio_generator(), media_
type="text/event-stream")
212
213     @router.get("/file/{filename}")
214     async def get_audio_file(filename: str):
215         """Returns аудиофайл из кэша."""
216         file_path = TTS_CACHE_DIR / filename
217         if not file_path.exists():
218             raise HTTPException(status_code=404, detail="Audio
file not found")
219         return FileResponse(file_path, media_type="audio/mpeg"
)
220
221     return router
222
```

router_version.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Application version and update management router
4 # =====
5 # Description:
6 #   Provides FastAPI endpoints for querying application version information,
7 #   checking remote update availability, and executing automatic updates.
8 #
9 # File: router_version.py
10 # Project: ai-breadboard
11 # Package: src.fastapi
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 from __future__ import annotations
17
18 import asyncio
19 from pathlib import Path
20 from typing import Optional, List, Dict, Any
21 from datetime import datetime
22
23 from fastapi import APIRouter, HTTPException, BackgroundTasks
24 from pydantic import BaseModel
25
26 from header import __root__
27 from src.logger import logger
28 from src.version_manager import get_version_manager, UpdateStatus
29
```

```

30 class VersionCheckResponse(BaseModel):
31     """Ответ при проверке версии."""
32     status: str
33     current_version: Optional[str] = None
34     remote_version: Optional[str] = None
35     is_update_available: bool = False
36     message: Optional[str] = None
37
38 class UpdateRequest(BaseModel):
39     """Запрос на update."""
40     branch: str = "main"
41     auto_backup: bool = True
42
43 class UpdateResponse(BaseModel):
44     """Ответ при выполнении обновления."""
45     success: bool
46     status: str
47     message: Optional[str] = None
48     version: Optional[str] = None
49     backup_path: Optional[str] = None
50
51 class BackupInfo(BaseModel):
52     """Info о резервной копии."""
53     path: str
54     timestamp: str
55     version: Optional[str] = None
56     files_count: int
57
58 def init_router() -> APIRouter:
59     """Инициализирует FastAPI роутер для управления версиями."
60     ""
61     router = APIRouter(prefix='/api/version', tags=['version']
62     )
63
64     @router.get('/check', response_model=VersionCheckResponse)
65     async def check_version() -> VersionCheckResponse:
66         """
67         Checks наличие обновлений.

```

```

66 |
67 |     Returns:
68 |         VersionCheckResponse с информацией о версиях
69 |     """
70 |     try:
71 |         vm = get_version_manager(__root__)
72 |         check_result = vm.check_updates()
73 |
74 |         return VersionCheckResponse(
75 |             status=check_result.get('status'),
76 |             current_version=check_result.get('current_vers
77 | ion'),
78 |             remote_version=check_result.get('remote_versio
79 | n'),
80 |             is_update_available=check_result.get('is_updat
81 | e_available', False),
82 |             message=check_result.get('message')
83 |         )
84 |     except Exception as ex:
85 |         logger.error(f"Error checking version: {ex}")
86 |         raise HTTPException(status_code=500, detail=str(ex
87 | ))
88 |
89 | @router.post('/update', response_model=UpdateResponse)
90 | async def perform_update(request: UpdateRequest, backgroun
91 | d_tasks: BackgroundTasks) -> UpdateResponse:
92 |     """
93 |     Performs update приложения с автоматическим резервным
94 |     копированием.
95 |
96 |     Args:
97 |         request: UpdateRequest с параметрами обновления
98 |         background_tasks: BackgroundTasks для асинхронных
99 |         операций
100 |
101 |     Returns:
102 |         UpdateResponse с результатом обновления
103 |     """

```

```

97 |         try:
98 |             vm = get_version_manager(__root__)
99 |
100 |             # Выполняем update в фоне
101 |             update_result = await vm.update_application(
102 |                 branch=request.branch,
103 |                 auto_backup=request.auto_backup
104 |             )
105 |
106 |             return UpdateResponse(
107 |                 success=update_result.get('success', False),
108 |                 status=update_result.get('status'),
109 |                 message=update_result.get('message'),
110 |                 version=update_result.get('version'),
111 |                 backup_path=update_result.get('backup_path')
112 |             )
113 |         except Exception as ex:
114 |             logger.error(f"Error performing update: {ex}")
115 |             raise HTTPException(status_code=500, detail=str(ex
116 | ))
117 |
118 | @router.get('/backups', response_model=List[BackupInfo])
119 | async def list_backups() -> List[BackupInfo]:
120 |     """
121 |     Receives list всех резервных копий.
122 |
123 |     Returns:
124 |     List информации о резервных копиях
125 |     """
126 |     try:
127 |         vm = get_version_manager(__root__)
128 |
129 |         if not vm.temp_backup_dir.exists():
130 |             return []
131 |
132 |         backups = []
133 |         for backup_dir in sorted(vm.temp_backup_dir.iterdi
134 | r(), reverse=True):

```

```

133         if not backup_dir.is_dir():
134             continue
135
136         # Пытаемся прочитать информацию о резервной ко
137         пии
138         info_file = backup_dir / ".backup_info.json"
139         version = None
140         if info_file.exists():
141             try:
142                 import json
143                 with open(info_file, 'r', encoding='ut
144                 f-8') as f:
145                     info = json.load(f)
146                     version = info.get('version')
147             except Exception:
148                 pass
149
150         # Считаем количество файлов
151         files_count = sum(1 for _ in backup_dir.rglob(
152         '*' if _.is_file()))
153
154         backups.append(BackupInfo(
155             path=str(backup_dir),
156             timestamp=backup_dir.stat().st_mtime,
157             version=version,
158             files_count=files_count
159         ))
160
161         return backups
162     except Exception as ex:
163         logger.error(f"Error listing backups: {ex}")
164         raise HTTPException(status_code=500, detail=str(ex
165         ))
166
167 @router.post('/restore/{backup_name}')
168 async def restore_backup(backup_name: str) -> Dict[str, An
169 y]:
170     """

```

```
166 |         Восстанавливает приложение из резервной копии.
167 |
168 |     Args:
169 |         backup_name: Имя директории с резервной копией
170 |
171 |     Returns:
172 |         Dictionary с результатом восстановления
173 |         ""
174 |     try:
175 |         vm = get_version_manager(__root__)
176 |         backup_path = vm.temp_backup_dir / backup_name
177 |
178 |         if not backup_path.exists():
179 |             raise HTTPException(status_code=404, detail="B
180 | ackup not found")
181 |
182 |         if not backup_path.is_dir():
183 |             raise HTTPException(status_code=400, detail="I
184 | nvalid backup path")
185 |
186 |         success = vm.restore_from_backup(backup_path)
187 |
188 |         if success:
189 |             logger.info(f"Successfully restored from backu
190 | p: {backup_name}")
191 |             return {
192 |                 "success": True,
193 |                 "message": f"Successfully restored from ba
194 | ckup: {backup_name}"
195 |             }
196 |         else:
197 |             logger.error(f"Failed to restore from backup:
198 | {backup_name}")
199 |             raise HTTPException(status_code=500, detail="R
200 | estoration failed")
201 |     except HTTPException:
202 |         raise
203 |     except Exception as ex:
```



```

198         logger.error(f"Error restoring backup: {ex}")
199         raise HTTPException(status_code=500, detail=str(ex
200     ))
201
202     @router.post('/cleanup-backups')
203     async def cleanup_backups(keep_count: int = 5) -> Dict[str
204     , Any]:
205         """
206         Deletes старые резервные копии, оставляя последние N.
207
208         Args:
209             keep_count: Количество резервных копий для сохране
210             ния
211
212         Returns:
213             Dictionary с результатом очистки
214         """
215         try:
216             vm = get_version_manager(__root__)
217             deleted_count = vm.cleanup_old_backups(keep_count=
218             keep_count)
219
220             logger.info(f"Cleane{deleted_count} old backu
221             ps")
222
223             return {
224                 "success": True,
225                 "message": f"Deleted {deleted_count} old backu
226                 ps",
227                 "deleted_count": deleted_count
228             }
229         except Exception as ex:
230             logger.error(f"Error cleaning up backups: {ex}")
231             raise HTTPException(status_code=500, detail=str(ex
232         ))
233
234     @router.get('/status')
235     async def get_version_status() -> Dict[str, Any]:
236         """

```

```

229 |         Receives полный status версии и обновлений.
230 |
231 |         Returns:
232 |             Dictionary с полной информацией о версиях и статус
233 | e
234 |         """
235 |         try:
236 |             vm = get_version_manager(__root__)
237 |             check_result = vm.check_updates()
238 |
239 |             # Подсчитываем количество резервных копий
240 |             backup_count = 0
241 |             if vm.temp_backup_dir.exists():
242 |                 backup_count = sum(1 for d in vm.temp_backup_dir.iterdir() if d.is_dir())
243 |
244 |             return {
245 |                 "status": check_result.get('status'),
246 |                 "current_version": check_result.get('current_v
247 | ersion'),
248 |                 "remote_version": check_result.get('remote_ver
249 | sion'),
250 |                 "is_update_available": check_result.get('is_up
251 | date_available', False),
252 |                 "current_commit": check_result.get('current_co
253 | mmit'),
254 |                 "remote_commit": check_result.get('remote_comm
255 | it'),
256 |                 "timestamp": check_result.get('timestamp'),
257 |                 "backup_count": backup_count,
258 |                 "backup_dir": str(vm.temp_backup_dir),
259 |                 "update_log_dir": str(vm.update_log_dir)
260 |             }
261 |         except Exception as ex:
262 |             logger.error(f"Error getting version status: {ex}")
263 |
264 |         raise HTTPException(status_code=500, detail=str(ex))

```

```
258 |  
259 |     return router  
260 |  
261 | __all__ = ['init_router']  
262 |
```

index.html

```
1 <!DOCTYPE html>
2 <html lang="ru">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-s
cale=1.0">
6   <title>AI Assistant - Admin</title>
7   <link href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dis
t/css/bootstrap.min.css" rel="stylesheet">
8   <link href="https://cdn.jsdelivrivr.net/npm/bootstrap-icons@1.1
0.0/font/bootstrap-icons.css" rel="stylesheet">
9   <link href="/html/css/main.css?v=20260825_user_settings" rel
="stylesheet">
10  <script src="https://cdn.jsdelivrivr.net/npm/i18next@23.11.5/i1
8next.min.js"></script>
11 </head>
12 <body>
13 <!-- Password Protection Modal -->
14 <div class="modal fade" id="passwordModal" tabindex="-1" aria-
labelledby="passwordModalLabel" aria-hidden="true" data-bs-bac
kdrop="static" data-bs-keyboard="false">
15   <div class="modal-dialog modal-dialog-centered">
16     <div class="modal-content bg-dark text-white border-second
ary">
17       <div class="modal-header border-secondary">
18         <h5 class="modal-title" id="passwordModalLabel"><i cla
ss="bi bi-lock-fill"></i> Вход в админку</h5>
19         <button type="button" class="btn-close btn-close-white
" data-bs-dismiss="modal" aria-label="Close" id="closePassword
Modal"></button>
20       </div>
21       <div class="modal-body p-4">
22         <p class="text-muted">Введите пароль для доступа к адм
инистраторскому интерфейсу:</p>
23         <div class="mb-3">
24           <input type="password" class="form-control bg-dark t
```

```

    ext-white border-secondary" id="admin-password" placeholder="•
    ..... autofocus>
25     </div>
26     <div id="password-error" class="alert alert-danger d-n
one">Неверный пароль</div>
27     <button type="button" class="btn btn-primary w-100" id
="login-btn">Войти</button>
28     </div>
29 </div>
30 </div>
31 </div>
32
33 <div class="container-fluid mt-2" id="admin-interface">
34
35     <div class="d-flex align-items-center justify-content-betwee
n mb-3 main-nav-container rounded px-2 py-1">
36         <ul class="nav nav-tabs flex-nowrap overflow-x-auto flex-g
row-1 border-0 me-2" id="mainTabs">
37             <li class="nav-item">
38                 <button class="nav-link active" data-bs-toggle="tab" d
ata-bs-target="#tab-chat" data-i18n="tabs.chat">Чат</button
>
39             </li>
40             <li class="nav-item">
41                 <button class="nav-link" data-bs-toggle="tab" data-bs-
target="#tab-plugins" data-i18n="tabs.plugins">Плагины</but
ton>
42             </li>
43             <li class="nav-item">
44                 <button class="nav-link" data-bs-toggle="tab" data-bs-
target="#tab-admin" data-i18n="tabs.admin">Управление</butto
n>
45             </li>
46             <li class="nav-item">
47                 <button class="nav-link" data-bs-toggle="tab" data-bs-
target="#tab-users" data-i18n="tabs.users">Пользователи</bu
tton>
48             </li>

```

```
49     <li class="nav-item" data-plugin-tab="rag">
50         <button class="nav-link" data-bs-toggle="tab" data-bs-
target="#tab-rag" data-plugin="rag" data-i18n="tabs.rag"> RA
G</button>
51     </li>
52     <li class="nav-item">
53         <button class="nav-link" data-bs-toggle="tab" data-bs-
target="#tab-models" data-i18n="tabs.models"> Модели и API</
button>
54     </li>
55     <li class="nav-item" data-plugin-tab="langchain_media">
56         <button class="nav-link" data-bs-toggle="tab" data-bs-
target="#tab-agents" data-plugin="langchain_media" data-i18n="
tabs.agents"> Агенты ИИ</button>
57     </li>
58     <li class="nav-item" data-plugin-tab="web_search">
59         <button class="nav-link" data-bs-toggle="tab" data-bs-
target="#tab-search" data-plugin="web_search" data-i18n="tabs.
search"> Веб-поиск</button>
60     </li>
61     <li class="nav-item">
62         <button class="nav-link" data-bs-toggle="tab" data-bs-
target="#tab-tts" data-i18n="tabs.tts"> Настройка TTS</butto
n>
63     </li>
64     <li class="nav-item" data-plugin-tab="movie_search_sourc
es">
65         <button class="nav-link" data-bs-toggle="tab" data-bs-
target="#tab-sources" data-plugin="movie_search_sources" data-
i18n="tabs.sources"> Источники</button>
66     </li>
67     <li class="nav-item">
68         <button class="nav-link" data-bs-toggle="tab" data-bs-
target="#tab-logs" data-i18n="tabs.logs"> Логи</button>
69     </li>
70     <li class="nav-item">
71         <button class="nav-link" data-bs-toggle="tab" data-bs-
target="#tab-help" data-i18n="tabs.help"> Справочник</button>
```

```

72     </li>
73     <li class="nav-item">
74         <button class="nav-link" data-bs-toggle="tab" data-bs-
target="#tab-instructions" data-i18n="tabs.instructions">
Системные инструкции</button>
75     </li>
76 </ul>
77
78 <div class="d-flex align-items-center gap-2 flex-shrink-0
ms-auto pb-1" id="nav-auth-container">
79     <!-- Google OAuth Register / Sign-in Button (Shown when
not logged in) -->
80     <button class="btn btn-sm btn-light text-dark fw-semibol
d d-flex align-items-center gap-2 px-3 py-1 shadow-sm rounded-
pill btn-google-login" id="nav-google-btn" title="Войти или за
регистрироваться через Google" data-i18n-title="auth.googleSig
nInTitle" style="border: 1px solid rgba(0,0,0,0.15);">
81         <svg width="18" height="18" viewBox="0 0 24 24"><path
fill="#4285F4" d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h
5.92c-.26 1.37-1.04 2.53-2.21 3.31v2.77h3.57c2.08-1.92 3.28-4.
74 3.28-8.09z"/><path fill="#34A853" d="M12 23c2.97 0 5.46-.98
7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-3.71 1.06-2.86 0-5.29-
1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z"/><path fill
="#FBBC05" d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.
35-2.09V7.06H2.18C1.43 8.55 1 10.22 1 12s.43 3.45 1.18 4.94l2.
85-2.22.81-.63z"/><path fill="#EA4335" d="M12 5.38c1.62 0 3.06
.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1 7.7 1 3.99 3.4
7 2.18 7.06l3.66 2.84c.87-2.6 3.3-4.52 6.16-4.52z"/></svg>
82         <span data-i18n="auth.googleSignUp">Регистрация через
Google</span>
83     </button>
84
85     <!-- Authenticated User Profile Pill (Shown when logged
in) -->
86     <button class="btn btn-sm btn-outline-secondary d-none a
lign-items-center gap-2 px-2 py-1 rounded-pill" id="user-setti
ngs-btn" data-bs-toggle="modal" data-bs-target="#userSettingsM

```

```

odal" title="Профиль и настройки" data-i18n-title="userSetting
s.title">
87     <img id="nav-user-avatar" src="" class="rounded-circle
d-none" width="22" height="22" style="object-fit: cover;">
88     <i class="bi bi-person-gear" id="nav-user-icon"></i>
89     <span id="nav-user-label" class="small fw-semibold" da
ta-i18n="userSettings.button">User Settings</span>
90     </button>
91
92     <!-- Quick Settings Gear Button -->
93     <button class="btn btn-sm btn-outline-secondary d-flex a
lign-items-center justify-content-center p-0 rounded-circle" i
d="user-settings-gear-btn" data-bs-toggle="modal" data-bs-targ
et="#userSettingsModal" title="Настройки" style="width: 32px;
height: 32px;" data-i18n-title="userSettings.title">
94         <i class="bi bi-gear-fill"></i>
95     </button>
96 </div>
97 </div>
98
99 <div class="tab-content">
100 <!-- Chat Tab -->
101 <div id="tab-chat" class="tab-pane fade show active">
102     <div class="chat-window mb-2" id="chat-window"></div>
103
104 <!-- Media Controls -->
105 <div class="media-controls mb-2 d-none" id="media-contro
ls">
106     <div class="card">
107         <div class="card-body p-2">
108             <div class="d-flex align-items-center gap-2">
109                 <span class="text-muted small" data-i18n="mediaC
ontrols.nowPlaying">▶ Сейчас играет:</span>
110                 <strong id="now-playing-title">—</strong>
111                 <button class="btn btn-sm btn-outline-primary ms
-auto" id="btn-open-player" data-i18n="mediaControls.openPlaye
r">Открыть плеер</button>
112                 <button class="btn btn-sm btn-outline-secondary"

```



```
        id="btn-close-player" data-i18n="mediaControls.close">⏏ </butt
on>
113         </div>
114     </div>
115 </div>
116 </div>
117
118     <div class="input-group">
119         <input type="text" class="form-control" id="message-in
put" data-i18n-placeholder="chat.placeholder" placeholder="Вве
дите сообщение...">
120         <button class="btn btn-primary" id="send-button" data-
i18n="chat.send">Отправить</button>
121     </div>
122 </div>
123
124 <!-- Plugins Tab -->
125 <div id="tab-plugins" class="tab-pane fade"></div>
126
127 <!-- Admin Tab -->
128 <div id="tab-admin" class="tab-pane fade"></div>
129
130 <!-- Users Tab -->
131 <div id="tab-users" class="tab-pane fade"></div>
132
133 <!-- RAG Tab -->
134 <div id="tab-rag" class="tab-pane fade"></div>
135
136 <!-- Models Tab -->
137 <div id="tab-models" class="tab-pane fade"></div>
138
139 <!-- Agents Tab -->
140 <div id="tab-agents" class="tab-pane fade"></div>
141
142 <!-- Search Tab -->
143 <div id="tab-search" class="tab-pane fade"></div>
144
145 <!-- TTS Tab -->
```

```
146     <div id="tab-tts" class="tab-pane fade"></div>
147
148     <!-- Sources Tab -->
149     <div id="tab-sources" class="tab-pane fade"></div>
150
151     <!-- Logs Tab -->
152     <div id="tab-logs" class="tab-pane fade"></div>
153
154     <!-- Help Tab -->
155     <div id="tab-help" class="tab-pane fade"></div>
156
157     <!-- Instructions Tab -->
158     <div id="tab-instructions" class="tab-pane fade"></div>
159 </div>
160 </div>
161
162 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist
    /js/bootstrap.bundle.min.js"></script>
163 <script src="/html/js/chatService.js?v=20260825_cleanup_tabs">
    </script>
164 <script type="module" src="/html/admin/main.js?v=20260825_navb
    ar_fix"></script>
165
166 <!-- HELP Modal -->
167 <div class="modal fade modal-help" id="help-modal" tabindex="-
    1" aria-labelledby="help-modal-label" aria-hidden="true">
168     <div class="modal-dialog modal-dialog-scrollable">
169         <div class="modal-content">
170             <div class="modal-header">
171                 <h5 class="modal-title" id="help-modal-label" data-i18
                    n="helpModal.title">Помощь</h5>
172                 <button type="button" class="btn-close" data-bs-dismis
                    s="modal" aria-label="Close"></button>
173             </div>
174             <div class="modal-body" id="help-modal-content">
175                 <p data-i18n="common.loading">Загрузка...</p>
176             </div>
177             <div class="modal-footer">
```

```
178         <button type="button" class="btn btn-secondary" data-b
      | s-dismiss="modal" data-i18n="helpModal.close">Заккрыть</button>
179     </div>
180 </div>
181 </div>
182 </div>
183
184 <!-- Prompt Preview Modal -->
185 <div class="modal fade" id="prompt-preview-modal" tabindex="-1"
      | " aria-labelledby="prompt-preview-modal-label" aria-hidden="tr
      | ue">
186     <div class="modal-dialog modal-lg">
187         <div class="modal-content">
188             <div class="modal-header">
189                 <h5 class="modal-title" id="prompt-preview-modal-label
      | ">Предпросмотр промта</h5>
190                 <button type="button" class="btn-close" data-bs-dismis
      | s="modal" aria-label="Close"></button>
191             </div>
192             <div class="modal-body" id="prompt-preview-content">
193                 <p>Загрузка...</p>
194             </div>
195             <div class="modal-footer">
196                 <button type="button" class="btn btn-secondary" data-b
      | s-dismiss="modal">Заккрыть</button>
197             </div>
198         </div>
199     </div>
200 </div>
201
202 <!-- Chat Logic & RAG Architecture Modal -->
203 <div class="modal fade" id="chat-logic-modal" tabindex="-1" ar
      | ia-labelledby="chat-logic-modal-label" aria-hidden="true">
204     <div class="modal-dialog modal-xl modal-dialog-scrollable">
205         <div class="modal-content bg-dark text-white border-second
      | ary">
206             <div class="modal-header border-secondary d-flex justify
      | -content-between align-items-center">
```

```

207 |         <h5 class="modal-title d-flex align-items-center gap-2
    | " id="chat-logic-modal-label">
208 |             <i class="bi bi-diagram-3-fill text-info"></i> □ Ар
    | хитектура и алгоритм диспетчеризации ответов чата
209 |         </h5>
210 |         <div class="d-flex align-items-center gap-2">
211 |             <a href="/html/assets/chat_and_rag_logic.svg" target
    | ="_blank" class="btn btn-sm btn-outline-info" title="Открыть S
    | VG в новой вкладке">
212 |                 <i class="bi bi-arrows-fullscreen"></i> SVG
213 |             </a>
214 |             <button type="button" class="btn-close btn-close-whi
    | te" data-bs-dismiss="modal" aria-label="Close"></button>
215 |         </div>
216 |     </div>
217 |     <div class="modal-body p-3">
218 |         <!-- Tabs for Schema vs Documentation -->
219 |         <ul class="nav nav-pills mb-3" id="chatLogicTabs" role
    | ="tablist">
220 |             <li class="nav-item" role="presentation">
221 |                 <button class="nav-link active" id="chat-logic-svg
    | -tab" data-bs-toggle="pill" data-bs-target="#chat-logic-svg-pa
    | ne" type="button" role="tab">□ Интерактивная блок-схема</butt
    | on>
222 |             </li>
223 |             <li class="nav-item" role="presentation">
224 |                 <button class="nav-link" id="chat-logic-doc-tab" d
    | ata-bs-toggle="pill" data-bs-target="#chat-logic-doc-pane" typ
    | e="button" role="tab">□ Описание алгоритма и сценариев</butto
    | n>
225 |             </li>
226 |         </ul>
227 |
228 |         <div class="tab-content" id="chatLogicTabContent">
229 |             <!-- SVG Diagram Tab -->
230 |             <div class="tab-pane fade show active text-center" i
    | d="chat-logic-svg-pane" role="tabpanel">
231 |                 <div class="p-2 border border-secondary rounded bg

```

```

    -black d-flex justify-content-center align-items-center" style
    ="overflow-x: auto; min-height: 520px;">
232         
233     </div>
234 </div>
235
236     <!-- Documentation Text Tab -->
237     <div class="tab-pane fade" id="chat-logic-doc-pane"
    role="tabpanel">
238         <div class="p-3 bg-black rounded border border-sec
    ondary text-light small">
239             <h6 class="text-info fw-bold mb-2">□ Режимы рабо
    ты (config.json &rarr; rag.mode)</h6>
240             <ul class="mb-3">
241                 <li><strong class="text-success">rag+model (Ги
    бридный / Основной):</strong> Сначала проверяется локальная ме
    диатека. Если фильм найден – LLM красиво рассказывает о нём со
    ссылкой на наличие в медиатеке. Если фильм отсутствует – LLM
    обращается к веб-поиску (MCP/Grounding) и рассказывает о фильм
    е с кнопкой загрузки <code>&lt;film&gt;</code>.</li>
242                 <li><strong class="text-warning">rag (Строгий
    локальный):</strong> Работает только по базе <code>media.db</c
    ode>. При отсутствии тайтла выдаёт чёткое <em>«В локальной баз
    е не найдено»</em> без ложных подстановок.</li>
243                 <li><strong class="text-primary">model (Без ба
    зы):</strong> Прямой ответ LLM и веб-поиск без обращения к дис
    кам.</li>
244             </ul>
245
246             <h6 class="text-info fw-bold mb-2">□ Сценарии о
    бработки</h6>
247             <div class="table-responsive mb-3">
248                 <table class="table table-dark table-bordered
    table-sm small">
249                     <thead>
250                         <tr class="table-secondary text-dark">

```

```

251 |         <th>Сценарий</th>
252 |         <th>Пример</th>
253 |         <th>Алгоритм</th>
254 |         <th>Результат</th>
255 |     </tr>
256 | </thead>
257 | <tbody>
258 |     <tr>
259 |         <td class="fw-bold text-success">Фильм е
| сть в базе</td>
260 |         <td><em>«Расскажи о фильме А зори здесь
| тихие»</em></td>
261 |         <td>Точный поиск в <code>media.db</code>
| (Score 1.0) / FAISS &ge; 0.60 &rarr; контекст в LLM</td>
262 |         <td>LLM сообщает, что фильм <strong>дост
| упен в медиатеке</strong>, описывает сюжет и тегирует <code>&l
| t;film&gt;</code>.</td>
263 |     </tr>
264 |     <tr>
265 |         <td class="fw-bold text-danger">Фильма н
| ет в базе</td>
266 |         <td><em>«Расскажи о сериале Сваты»</em><
| /td>
267 |         <td>Локальный RAG даёт 0 совпадений &rar
| r; Fallback на Web Search (MCP)</td>
268 |         <td>LLM рассказывает о сериале, указывае
| т, что его <strong>нет на дисках</strong>, тегирует <code>&lt;
| film&gt;</code> для торрентов.</td>
269 |     </tr>
270 |     <tr>
271 |         <td class="fw-bold text-info">Команда за
| пуска</td>
272 |         <td><em>«Включи А зори здесь тихие»</em>
| </td>
273 |         <td>Проверка пути на диске &rarr; WebSoc
| ket <code>play_file_by_path</code></td>
274 |         <td>Мгновенный старт в плеере (Direct Pl
| ay).</td>

```

```

275 |         </tr>
276 |         <tr>
277 |             <td class="fw-bold text-primary">Подборк
a / Настроение</td>
278 |             <td><em>«Найди боевик на вечер»</em></td>
>
279 |             <td>Извлечение подходящих жанров &rarr;
LLM формирует диалоговую подборку</td>
280 |             <td>Живая рекомендация с обоснованием «п
очему стоит посмотреть».</td>
281 |         </tr>
282 |     </tbody>
283 | </table>
284 | </div>
285 |
286 |     <h6 class="text-info fw-bold mb-1">□ Порог коси
нусного сходства (Cosine Similarity)</h6>
287 |     <p class="text-muted mb-0">Для исключения ложных
срабатываний порог в <code>rag_search_tool</code> установлен
на <strong>0.60</strong>. Запросы с более низким сходством счи
таются отсутствующими в локальной базе и корректно перенаправл
яются в интернет-поиск.</p>
288 |     </div>
289 | </div>
290 | </div>
291 | </div>
292 |     <div class="modal-footer border-secondary">
293 |         <button type="button" class="btn btn-sm btn-secondary"
data-bs-dismiss="modal">Закрыть</button>
294 |     </div>
295 | </div>
296 | </div>
297 | </div>
298 |
299 | <!-- User Settings Modal -->
300 | <div class="modal fade" id="userSettingsModal" tabindex="-1" a
ria-labelledby="userSettingsModalLabel" aria-hidden="true">
301 |     <div class="modal-dialog modal-dialog-centered">

```

```
302     <div class="modal-content shadow border-secondary-subtle">
303         <div class="modal-header border-secondary-subtle">
304             <h5 class="modal-title d-flex align-items-center gap-2
" id="userSettingsModalLabel">
305                 <i class="bi bi-person-gear text-primary"></i>
306                 <span data-i18n="userSettings.title">Пользовательски
е настройки</span>
307             </h5>
308             <button type="button" class="btn-close" data-bs-dismis
s="modal" aria-label="Close"></button>
309         </div>
310         <div class="modal-body p-3">
311             <!-- User Account & Google OAuth Card -->
312             <div class="card bg-body-tertiary border-secondary-sub
tle mb-3 p-3" id="user-settings-auth-card">
313                 <div class="d-flex align-items-center gap-3">
314                     <img id="user-settings-avatar" src="" class="round
ed-circle border border-2 border-primary" width="48" height="4
8" style="object-fit: cover;">
315                     <div class="flex-grow-1 min-w-0">
316                         <div class="d-flex align-items-center gap-2">
317                             <strong id="user-settings-name" class="text-tr
uncate">Пользователь</strong>
318                             <span id="user-settings-role-badge" class="bad
ge bg-primary">User</span>
319                         </div>
320                         <div id="user-settings-email" class="small text-
muted text-truncate">email@gmail.com</div>
321                         <div id="user-settings-google-badge" class="smal
l mt-1 text-muted">
322                             <i class="bi bi-google text-danger me-1"></i>G
oogle OAuth
323                         </div>
324                     </div>
325                     <button type="button" class="btn btn-outline-dange
r btn-sm btn-user-logout" title="Выйти">
326                         <i class="bi bi-box-arrow-right"></i>
327                     </button>
```



```

328         </div>
329     </div>
330
331     <!-- Guest / Connect Google OAuth Card -->
332     <div class="card bg-body-tertiary border-secondary-sub
333         tle mb-3 p-3 text-center d-none" id="user-settings-guest-card"
334         >
335         <div class="text-muted small mb-2">
336             <i class="bi bi-shield-lock me-1"></i> Авторизация
337             и синхронизация аккаунта
338         </div>
339         <button type="button" class="btn btn-outline-danger
340             btn-sm w-100 d-flex align-items-center justify-content-center
341             gap-2 btn-google-login">
342             <i class="bi bi-google"></i>
343             <span>Войти через Google (OAuth)</span>
344         </button>
345         <div class="small text-muted mt-2" style="font-size:
346             0.75rem;">
347             Синхронизация с Google Drive, Docs, Calendar и Con
348             tacts
349         </div>
350     </div>
351
352     <!-- Language Selection -->
353     <div class="mb-3">
354         <label class="form-label fw-semibold small d-flex al
355             ign-items-center gap-2" for="modal-lang-selector">
356             <i class="bi bi-translate text-info"></i>
357             <span data-i18n="userSettings.language">Язык интер
358             фейса</span>
359         </label>
360         <select class="form-select lang-selector" id="modal-
361             lang-selector">
362             <option value="ru">Русский</option>
363             <option value="en">English</option>
364             <option value="he">עברית</option>
365         </select>

```

```
356 |         </div>
357 |
358 |         <!-- Theme Selection -->
359 |         <div class="mb-2">
360 |             <label class="form-label fw-semibold small d-flex align-items-center gap-2" for="modal-theme-selector">
361 |                 <i class="bi bi-palette text-warning"></i>
362 |                 <span data-i18n="userSettings.theme">Тема оформления</span>
363 |             </label>
364 |             <select class="form-select theme-selector" id="modal-theme-selector">
365 |                 <option value="light" data-i18n="userSettings.themeLight">☐ Светлая</option>
366 |                 <option value="dark" data-i18n="userSettings.themeDark">☐ Тёмная</option>
367 |                 <option value="system" data-i18n="userSettings.themeSystem">☐ Системная</option>
368 |             </select>
369 |         </div>
370 |     </div>
371 |     <div class="modal-footer border-secondary-subtle py-2">
372 |         <button type="button" class="btn btn-secondary btn-sm" data-bs-dismiss="modal" data-i18n="userSettings.close">Закрыть</button>
373 |     </div>
374 | </div>
375 | </div>
376 | </div>
377 |
378 | </body>
379 | </html>
```

main.js

```
1 // Admin Interface Main JS
2 import { initI18n, switchLang, applyTranslations } from '../js/i18n.js';
3 import { initTheme, setTheme, getThemeMode, getResolvedTheme }
  from '../js/theme.js';
4 import { initUserSettings, refreshUserProfile } from '../js/userSettings.js';
5
6 window.switchLang = switchLang;
7 window.applyTranslations = applyTranslations;
8 window.setTheme = setTheme;
9 window.getThemeMode = getThemeMode;
10 window.getResolvedTheme = getResolvedTheme;
11 window.initUserSettings = initUserSettings;
12 window.refreshUserProfile = refreshUserProfile;
13
14 // Admin password (hardcoded for security)
15 const ADMIN_PASSWORD = 'onela';
16
17 // API module
18 window.api = {
19   async fetch(url, options = {}) {
20     const response = await fetch(url, options);
21     if (!response.ok) {
22       let msg = response.statusText;
23       try {
24         const data = await response.json();
25         if (data && data.detail) {
26           if (typeof data.detail === 'string') {
27             msg = data.detail;
28           } else if (Array.isArray(data.detail)) {
29             msg = data.detail.map(d => d.msg || JSON.stringify(d)).join(', ');
30           } else {
31             msg = JSON.stringify(data.detail);
32           }
33         }
34       } catch {
35         // ignore
36       }
37     }
38     return response;
39   }
40 }
```

```

33         }
34     } catch {}
35     throw new Error(`${response.status} ${msg}`);
36 }
37 return response.json();
38 },
39
40 // User management API
41 users: {
42     async list(params = {}) {
43         const q = new URLSearchParams(params).toString();
44         return window.api.fetch(`/api/admin/users${q ? '?' + q : ''}`);
45     },
46
47     async get(userId) {
48         return window.api.fetch(`/api/admin/users/${userId}`);
49     },
50
51     async create(data) {
52         return window.api.fetch('/api/admin/users', {
53             method: 'POST',
54             headers: { 'Content-Type': 'application/json' },
55             body: JSON.stringify(data)
56         });
57     },
58
59     async update(userId, data) {
60         return window.api.fetch(`/api/admin/users/${userId}`, {
61             method: 'PUT',
62             headers: { 'Content-Type': 'application/json' },
63             body: JSON.stringify(data)
64         });
65     },
66
67     async setPassword(userId, password) {
68         return window.api.fetch(`/api/admin/users/${userId}/password`, {

```

```

69         method: 'POST',
70         headers: { 'Content-Type': 'application/json' },
71         body: JSON.stringify({ password })
72     });
73 },
74
75     async toggleActive(userId) {
76         return window.api.fetch(`/api/admin/users/${userId}/toggle-active`, {
77             method: 'POST'
78         });
79     },
80
81     async toggleRole(userId) {
82         return window.api.fetch(`/api/admin/users/${userId}/toggle-role`, {
83             method: 'POST'
84         });
85     },
86
87     async delete(userId) {
88         return window.api.fetch(`/api/admin/users/${userId}`, {
89             method: 'DELETE'
90         });
91     }
92 }
93 };
94
95 // HELP content
96 // Password protection state
97 let isPasswordProtected = false;
98 let hasEnteredPassword = false;
99
100 async function startAdmin() {
101     console.log('Admin interface initializing...');
102     try {
103         initTheme();
104         const el = document.getElementById('admin-interface');

```

```
105 |     if (el) el.style.display = 'block';
106 |
107 |     if (isPasswordProtected) {
108 |         showPasswordModal();
109 |     } else {
110 |         await initInterface();
111 |     }
112 |     console.log('Admin interface ready');
113 | } catch (err) {
114 |     console.error('Error during startAdmin initialization:', e
rr);
115 |     const el = document.getElementById('admin-interface');
116 |     if (el) el.style.display = 'block';
117 | }
118 | }
119 |
120 | if (document.readyState === 'loading') {
121 |     document.addEventListener('DOMContentLoaded', startAdmin);
122 | } else {
123 |     startAdmin();
124 | }
125 |
126 | async function initInterface() {
127 |     // Initialize theme
128 |     initTheme();
129 |
130 |     // Initialize i18n
131 |     const savedLang = localStorage.getItem('app_language') || 'ru';
132 |     await initI18n(savedLang);
133 |
134 |     // Setup language selector
135 |     document.querySelectorAll('.lang-selector').forEach((sel) =>
{
136 |         sel.value = savedLang;
137 |         sel.addEventListener('change', (e) => {
138 |             switchLang(e.target.value);
139 |         });
```

```
140 | });
141 |
142 | // Initialize User Settings & Google OAuth
143 | await initUserSettings();
144 |
145 |
146 | // Initialize HELP content
147 | initHelpContent();
148 |
149 | const cb = Date.now();
150 | // Load all tabs
151 | await Promise.all([
152 |     loadTabContent('chat', `/html/chat/index.html?v=${cb}`),
153 |     loadTabContent('plugins', `/html/plugins_tab/index.html?v=
154 | ${cb}`, `/html/plugins_tab/main.js?v=${cb}`),
155 |     loadTabContent('admin', `/html/admin_tab/index.html?v=${cb}
156 | `, `/html/admin_tab/main.js?v=${cb}`),
157 |     loadTabContent('users', `/html/users_tab/index.html?v=${cb}
158 | `, `/html/users_tab/main.js?v=${cb}`),
159 |     loadTabContent('instructions', `/html/instructions_tab/ind
160 | ex.html?v=${cb}`, `/html/instructions_tab/main.js?v=${cb}`),
161 |     loadTabContent('rag', `/html/rag_tab/index.html?v=${cb}`,
162 | `/html/rag_tab/main.js?v=${cb}`),
163 |     loadTabContent('models', `/html/models_tab/index.html?v=${
164 | cb}`, `/html/models_tab/main.js?v=${cb}`),
165 |     loadTabContent('agents', `/html/agents_tab/index.html?v=${
166 | cb}`, `/html/agents_tab/main.js?v=${cb}`),
167 |     loadTabContent('search', `/html/search_tab/index.html?v=${
168 | cb}`, `/html/search_tab/main.js?v=${cb}`),
169 |     loadTabContent('tts', `/html/tts_tab/index.html?v=${cb}`,
170 | `/html/tts_tab/main.js?v=${cb}`),
171 |     loadTabContent('sources', `/html/sources_tab/index.html?v=
172 | ${cb}`, `/html/sources_tab/main.js?v=${cb}`),
173 |     loadTabContent('logs', `/html/logs/index.html?v=${cb}`),
174 |     loadTabContent('help', `/html/help/index.html?v=${cb}`),
175 | ]);
176 |
177 | // Apply translations
```

```
168 |   applyTranslations();
169 |
170 |   // Синхронизация видимости вкладок плагинов
171 |   try {
172 |     const pluginsData = await window.api.fetch('/api/admin/plugins');
173 |     if (window.syncPluginTabsVisibility && pluginsData && pluginsData.plugins) {
174 |       window.syncPluginTabsVisibility(pluginsData.plugins);
175 |     }
176 |   } catch (err) {
177 |     console.error('Ошибка синхронизации видимости плагинов:', err);
178 |   }
179 |
180 |   // Фокусировать поле ввода при переключении на вкладку чата
181 |   document.addEventListener('shown.bs.tab', (e) => {
182 |     const target = e.target.getAttribute('data-bs-target');
183 |     if (target === '#tab-chat') {
184 |       const msgInput = document.getElementById('message-input');
185 |       if (msgInput) {
186 |         msgInput.focus();
187 |       }
188 |       if (window.initChatDebuggerToolbar) {
189 |         window.initChatDebuggerToolbar();
190 |       }
191 |     } else if (target === '#tab-users') {
192 |       console.log('[AdminInterface] Switching to users tab...');
193 |       if (window.initUsersTab) {
194 |         window.initUsersTab();
195 |       }
196 |     } else if (target === '#tab-instructions') {
197 |       console.log('[AdminInterface] Switching to instructions tab...');
198 |       if (window.initInstructionsTab) {
199 |         window.initInstructionsTab();
```



```
200     }
201   } else if (target === '#tab-rag') {
202     console.log('[AdminInterface] Switching to RAG tab...');
203     if (window.initRagTab) {
204       window.initRagTab();
205     }
206   } else if (target === '#tab-search') {
207     console.log('[AdminInterface] Switching to search tab...');
208     if (window.initSearchTab) {
209       window.initSearchTab();
210     }
211   } else if (target === '#tab-models') {
212     console.log('[AdminInterface] Switching to models tab...');
213     if (window.initModelsTab) {
214       window.initModelsTab();
215     }
216   } else if (target === '#tab-agents') {
217     console.log('[AdminInterface] Switching to agents tab...');
218     if (window.initAgentsTab) {
219       window.initAgentsTab();
220     }
221   } else if (target === '#tab-sources') {
222     console.log('[AdminInterface] Switching to sources tab...');
223     if (window.initSourcesTab) {
224       window.initSourcesTab();
225     }
226   }
227 });
228
229 console.log('Admin interface ready');
230 }
231
232 function showPasswordModal() {
233   const modal = new bootstrap.Modal(document.getElementById('p
```

```
    passwordModal'));
234    modal.show();
235
236    const loginBtn = document.getElementById('login-btn');
237    const passwordInput = document.getElementById('admin-password');
238    const passwordError = document.getElementById('password-error');
239
240    passwordInput?.addEventListener('keypress', (e) => {
241        if (e.key === 'Enter') verifyPassword();
242    });
243
244    loginBtn?.addEventListener('click', verifyPassword);
245 }
246
247 async function verifyPassword() {
248     const passwordInput = document.getElementById('admin-password');
249     const passwordError = document.getElementById('password-error');
250     const password = passwordInput?.value;
251
252     if (password === ADMIN_PASSWORD) {
253         // Password correct
254         hasEnteredPassword = true;
255
256         // Hide error
257         passwordError?.classList.add('d-none');
258
259         // Close modal
260         const modalElement = document.getElementById('passwordModal');
261         const modal = bootstrap.Modal.getInstance(modalElement);
262         modal?.hide();
263
264         // Show interface
265         document.getElementById('admin-interface').style.display =
```

```

    'block';
266
267     // Initialize interface
268     await initInterface();
269 } else {
270     // Password incorrect
271     passwordError?.classList.remove('d-none');
272     if (passwordError) {
273         passwordError.textContent = 'Неверный пароль';
274     }
275
276     // Clear input
277     if (passwordInput) {
278         passwordInput.value = '';
279         passwordInput.focus();
280     }
281 }
282 }
283
284 async function loadTabContent(tabName, url, jsOverrideSrc) {
285     try {
286         const response = await fetch(url);
287         if (!response.ok) throw new Error(`HTTP ${response.status}
288 `);
289         const html = await response.text();
290         const container = document.getElementById(`tab-${tabName}`
291 );
292         if (!container) return;
293         container.innerHTML = html;
294
295         // Load JS for the tab
296         const cacheBuster = Date.now();
297         const scriptSrc = jsOverrideSrc || `/html/${tabName}/main.
298 js?v=${cacheBuster}`;
299
300         await new Promise((resolve) => {
301             const script = document.createElement('script');
302             if (tabName === 'instructions') {

```

```

300     script.type = 'module';
301 }
302 script.src = scriptSrc;
303 script.onload = () => {
304     console.log(`[Admin] Loaded JS for ${tabName}`);
305     resolve();
306 };
307 script.onerror = (err) => {
308     console.warn(`[Admin] Note: No JS loaded for ${tabName}
    } from ${scriptSrc}`);
309     resolve();
310 };
311 document.body.appendChild(script);
312 });
313
314 const initFuncName = 'init' + tabName.charAt(0).toUpperCase() + tabName.slice(1) + 'Tab';
315 if (typeof window[initFuncName] === 'function') {
316     console.log(`[Admin] Calling ${initFuncName} for ${tabName}`);
317     try {
318         await window[initFuncName]();
319     } catch (err) {
320         console.error(`[Admin] Error executing ${initFuncName}
    :`, err);
321     }
322 }
323 applyTranslations();
324 } catch (e) {
325     console.error(`[Admin] Error loading tab ${tabName}:`, e);
326     const container = document.getElementById(`tab-${tabName}`);
327     if (container) {
328         container.innerHTML = `<div class="alert alert-danger">0
   шибка загрузки: ${e.message}</div>`;
329     }
330 }
331 }

```

```
332 |
333 | // Initialize HELP content
334 | function initHelpContent() {
335 |     window.HELP_CONTENT = {
336 |         'overview': `

#### 📄 Обзор проекта



ai-breadboard – и
| интегрированная среда для работы с AI, RAG и системным управлен
| ием.

`,
337 |         'rag': `

#### 📖 RAG-индекс (Векторный поиск)


338 | 

1. База RAG: Индексирует системные базы зн
| аний, системные инструкции и документы.


339 | 

2. Загрузка документов («JSON»): Позволя
| ет загрузить внешние .json, .txt, .md файлы или сканировать директории напрямую в RAG
| -индекс.


340 | 

3. Чат-RAG: Индексирует историю сохраненны
| х диалогов и ответов ассистента.

`,
341 |         'code_rules': `

#### 📜 CODE_RULES



Правила кодирования
| проекта.

`,
342 |         'architecture': `

#### 🏗️ Архитектура



Технологический
| стек проекта.

`,
343 |         'fastapi': `

#### 🚀 FastAPI



API роутеры и эндпоинты.
|

`,
344 |         'gemini': `

#### 🌌 Gemini AI



Интеграция с Gemini, AG
| Y, Foundry и Ollama.

`,
345 |         'configuration': `

#### ⚙️ Конфигурация



Настройки прое
| кта (config.json).

`,
346 |         'cli_commands': `

#### 📄 CLI команды



Командная строка
| и лончеры.

`,
347 |     };
348 | }
349 |
350 | function showHelpModal(key) {
351 |     const content = window.HELP_CONTENT[key] || '<p>Информация н
| е найдена</p>';
352 |     document.getElementById('help-modal-content').innerHTML = co
| ntent;
353 |     const modal = new bootstrap.Modal(document.getElementById('h
| elp-modal'));
```

```
354 | modal.show();
355 | }
356 |
357 | function showChatLogicModal() {
358 |     const modalEl = document.getElementById('chat-logic-modal');
359 |     if (modalEl) {
360 |         const modal = new bootstrap.Modal(modalEl);
361 |         modal.show();
362 |     }
363 | }
364 | window.showChatLogicModal = showChatLogicModal;
365 | window.showHelpModal = showHelpModal;
366 |
367 | // Уведомления
368 | function showNotification(message, type = 'info') {
369 |     const notification = document.createElement('div');
370 |     notification.className = `alert alert-${type} position-fixed
    | top-0 end-0 m-3`;
371 |     notification.style.zIndex = '9999';
372 |     notification.style.maxWidth = '400px';
373 |     notification.textContent = message;
374 |     document.body.appendChild(notification);
375 |
376 |     setTimeout(() => {
377 |         notification.remove();
378 |     }, 5000);
379 | }
380 |
381 | async function initAdminTab() {
382 |     const modelSelect = document.getElementById('admin-model-select');
383 |     const saveBtn = document.getElementById('btn-admin-save-model');
384 |
385 |     if (!modelSelect || !saveBtn) return;
386 |
387 |     // 1. Очищаем селект
388 |     modelSelect.innerHTML = '';
```

```

389
390 let modelsGrouped = {};
391 // 2. Загружаем доступные модели
392 try {
393     const modelsData = await window.api.fetch('/api/chat/models');
394     modelsGrouped = modelsData.models || {};
395     if (Array.isArray(modelsGrouped)) {
396         modelsGrouped = { 'gemini': modelsGrouped };
397     }
398 } catch (err) {
399     console.error('Ошибка загрузки моделей:', err);
400     showNotification('Ошибка загрузки моделей AI: ' + err.message, 'danger');
401 }
402
403 // 3. Заполняем селект моделями с иерархией optgroup
404 const providerMeta = {
405     'gemini': { label: '□ Google Gemini', order: 1 },
406     'agy': { label: '□ Google Antigravity (AGY)', order: 2 },
407     'foundry': { label: '□ Microsoft Foundry', order: 3 },
408     'ollama': { label: '□ Ollama (Local)', order: 4 }
409 };
410
411 const providers = Object.keys(modelsGrouped).sort((a, b) =>
412 {
413     const oA = providerMeta[a]?.order ?? 99;
414     const oB = providerMeta[b]?.order ?? 99;
415     return oA - oB;
416 });
417
418 let totalModels = 0;
419 providers.forEach(p => {
420     const list = modelsGrouped[p] || [];
421     if (!Array.isArray(list) || list.length === 0) return;
422
423     const optgroup = document.createElement('optgroup');
424     optgroup.label = providerMeta[p]?.label || `□ ${p.toUpper

```

```
Case()}`;  
424  
425     list.forEach(m => {  
426         totalModels++;  
427         const option = document.createElement('option');  
428         option.value = m;  
429         option.textContent = m;  
430         optgroup.appendChild(option);  
431     });  
432  
433     modelSelect.appendChild(optgroup);  
434 });  
435  
436 if (totalModels === 0) {  
437     const option = document.createElement('option');  
438     option.value = '';  
439     option.textContent = 'Нет доступных моделей';  
440     modelSelect.appendChild(option);  
441     saveBtn.disabled = true;  
442 } else {  
443     saveBtn.disabled = false;  
444 }  
445  
446 // 4. Загружаем текущие настройки пользователя (выбранную мо  
447   дель)  
448   try {  
449       const settingsData = await window.api.fetch('/auth/settings');  
450       if (settingsData && settingsData.model) {  
451           modelSelect.value = settingsData.model;  
452       }  
453   } catch (err) {  
454       console.error('Ошибка загрузки настроек AI пользователя:',  
455         err);  
456   }  
457  
458   // 5. Навешиваем обработчик сохранения  
459   saveBtn.onclick = async () => {
```



```
458     const selectedModel = modelSelect.value;
459     saveBtn.disabled = true;
460     const originalText = saveBtn.textContent;
461     saveBtn.textContent = 'Сохранение...';
462
463     try {
464         await window.api.fetch('/auth/settings', {
465             method: 'POST',
466             headers: { 'Content-Type': 'application/json' },
467             body: JSON.stringify({ model: selectedModel })
468         });
469         showNotification('Модель успешно обновлена на: ' + selectedModel, 'success');
470
471         // Обновляем бейджи модели в реальном времени на странице
472         window.activeModelName = selectedModel;
473         if (typeof window.updateChatBadges === 'function') {
474             window.updateChatBadges(selectedModel);
475         } else {
476             const badges = document.querySelectorAll('#chat-model-badge, #chat-popup-model-badge');
477             badges.forEach(badge => {
478                 badge.textContent = selectedModel;
479                 badge.style.display = 'inline-block';
480             });
481         }
482     } catch (err) {
483         console.error('Ошибка сохранения модели:', err);
484         showNotification('Ошибка сохранения: ' + err.message, 'danger');
485     } finally {
486         saveBtn.disabled = false;
487         saveBtn.textContent = originalText;
488     }
489 };
490 }
491
```

```
492 | window.initAdminTab = initAdminTab;
```

index.html

```
1 <!-- System Admin Tab -->
2 <div class="container-fluid py-2">
3   <div class="alert alert-dark d-flex flex-wrap align-items-ce
  nter justify-content-between gap-2 mb-3 shadow-sm border-secon
  dary">
4     <div class="d-flex align-items-center gap-2">
5       <span class="fs-4">□ </span>
6       <div>
7         <h5 class="mb-0 fw-bold">Центральное управление систем
  ой</h5>
8         <p class="mb-0 small text-muted">Мониторинг ядра, сист
  емные сервисы, провайдеры моделей и глобальное администрирован
  ие</p>
9       </div>
10    </div>
11    <div class="d-flex gap-2 flex-wrap">
12      <button class="btn btn-outline-info btn-sm" onclick="res
  canStorageDrives()">
13        <i class="bi bi-hdd-network"></i> Пересканировать диск
  и
14      </button>
15      <button class="btn btn-outline-primary btn-sm" onclick="
  actualizeAiModels()">
16        <i class="bi bi-cpu"></i> Обновить модели ИИ
17      </button>
18    </div>
19  </div>
20
21 <!-- System Health & Status Cards -->
22 <div class="row g-3 mb-3">
23   <div class="col-md-3 col-sm-6">
24     <div class="card shadow-sm border-secondary-subtle h-100
  ">
25       <div class="card-body p-3">
26         <div class="text-muted small text-uppercase fw-semib
  old mb-1">Сервер / Режим</div>
```

```
27 |         <div class="d-flex align-items-center justify-content-between">
28 |             <h5 class="mb-0 fw-bold text-primary" id="sys-server-mode">FastAPI (DEV)</h5>
29 |             <span class="badge bg-success-subtle text-success">Online</span>
30 |         </div>
31 |         <small class="text-muted" id="sys-server-host">0.0.0.0:8000</small>
32 |     </div>
33 | </div>
34 | </div>
35 |
36 | <div class="col-md-3 col-sm-6">
37 |     <div class="card shadow-sm border-secondary-subtle h-100">
38 |         <div class="card-body p-3">
39 |             <div class="text-muted small text-uppercase fw-semibold mb-1">AI Провайдеры</div>
40 |             <div class="d-flex align-items-center justify-content-between">
41 |                 <h5 class="mb-0 fw-bold text-info" id="sys-ai-providers">Gemini / AGY / Foundry</h5>
42 |                 <i class="bi bi-robot fs-4 text-info"></i>
43 |             </div>
44 |             <small class="text-muted">Актуализированный пул</small>
45 |         </div>
46 |     </div>
47 | </div>
48 |
49 | <div class="col-md-3 col-sm-6">
50 |     <div class="card shadow-sm border-secondary-subtle h-100">
51 |         <div class="card-body p-3">
52 |             <div class="text-muted small text-uppercase fw-semibold mb-1">Подключено дисков</div>
53 |             <div class="d-flex align-items-center justify-content-between">
```

```

t-between">
54         <h5 class="mb-0 fw-bold text-warning" id="sys-driv
es-count">Загрузка...</h5>
55         <i class="bi bi-device-hdd fs-4 text-warning"></i>
56     </div>
57     <small class="text-muted" id="sys-drives-list">CONNE
CTED_DRIVES</small>
58 </div>
59 </div>
60 </div>
61
62 <div class="col-md-3 col-sm-6">
63     <div class="card shadow-sm border-secondary-subtle h-100
">
64         <div class="card-body p-3">
65             <div class="text-muted small text-uppercase fw-semib
old mb-1">Подключаемые плагины</div>
66             <div class="d-flex align-items-center justify-conten
t-between">
67                 <h5 class="mb-0 fw-bold text-success" id="sys-plug
ins-count">Загрузка...</h5>
68                 <a href="javascript:void(0)" onclick="switchToTab(
'plugins')" class="badge bg-primary text-decoration-none">Упра
вление </a>
69             </div>
70             <small class="text-muted" id="sys-plugins-active-cou
nt">Реестр модулей</small>
71         </div>
72     </div>
73 </div>
74 </div>
75
76 <!-- Fast Navigation Hub to Core Subsystems -->
77 <h6 class="fw-bold mb-3 text-uppercase small text-muted"><i
class="bi bi-grid-fill me-1"></i> Системные подсистемы</h6>
78 <div class="row g-3 mb-4">
79     <div class="col-lg-4 col-md-6">
80         <div class="card shadow-sm border-secondary-subtle h-100

```

```

    p-3 hover-shadow" style="cursor: pointer;" onclick="switchToTab('plugins')">
81     <div class="d-flex align-items-center gap-3">
82         <span class="fs-1">□ </span>
83         <div>
84             <h6 class="fw-bold mb-1">Подключатель плагинов (Hub) </h6>
85             <p class="small text-muted mb-0">Управление модулями (web_search, rag, langchain_media и др.), включение/отключение и их органы настройки.</p>
86         </div>
87     </div>
88 </div>
89 </div>
90
91 <div class="col-lg-4 col-md-6">
92     <div class="card shadow-sm border-secondary-subtle h-100 p-3 hover-shadow" style="cursor: pointer;" onclick="switchToTab('users')">
93         <div class="d-flex align-items-center gap-3">
94             <span class="fs-1">□ </span>
95             <div>
96                 <h6 class="fw-bold mb-1">Управление пользователями </h6>
97                 <p class="small text-muted mb-0">Создание аккаунтов, назначение ролей (admin/user), пароли, активность и права доступа.</p>
98             </div>
99         </div>
100     </div>
101 </div>
102
103 <div class="col-lg-4 col-md-6">
104     <div class="card shadow-sm border-secondary-subtle h-100 p-3 hover-shadow" style="cursor: pointer;" onclick="switchToTab('models')">
105         <div class="d-flex align-items-center gap-3">
106             <span class="fs-1">□ </span>

```

```
107 |         <div>
108 |             <h6 class="fw-bold mb-1">Модели и API-ключи</h6>
109 |             <p class="small text-muted mb-0">Конфигурация Goog
le Gemini, Antigravity, Ollama и AI Foundry, пул ротации ключе
й.</p>
110 |         </div>
111 |     </div>
112 | </div>
113 | </div>
114 |
115 |     <div class="col-lg-4 col-md-6">
116 |         <div class="card shadow-sm border-secondary-subtle h-100
p-3 hover-shadow" style="cursor: pointer;" onclick="switchToT
ab('instructions')">
117 |             <div class="d-flex align-items-center gap-3">
118 |                 <span class="fs-1">□ </span>
119 |                 <div>
120 |                     <h6 class="fw-bold mb-1">Системные инструкции</h6>
121 |                     <p class="small text-muted mb-0">Редактирование си
стемных промптов чата, стилей диктора и версионирование инстру
кций.</p>
122 |                 </div>
123 |             </div>
124 |         </div>
125 |     </div>
126 |
127 |     <div class="col-lg-4 col-md-6">
128 |         <div class="card shadow-sm border-secondary-subtle h-100
p-3 hover-shadow" style="cursor: pointer;" onclick="switchToT
ab('logs')">
129 |             <div class="d-flex align-items-center gap-3">
130 |                 <span class="fs-1">□ </span>
131 |                 <div>
132 |                     <h6 class="fw-bold mb-1">Системные логи</h6>
133 |                     <p class="small text-muted mb-0">Просмотр журнала
событий uvicorn, ошибок FastAPI, истории запросов и аудита.</p
>
134 |                 </div>
```

```

135         </div>
136     </div>
137 </div>
138
139     <div class="col-lg-4 col-md-6">
140         <div class="card shadow-sm border-secondary-subtle h-100
141         p-3 hover-shadow" style="cursor: pointer;" onclick="switchToTab('agents')">
142             <div class="d-flex align-items-center gap-3">
143                 <span class="fs-1">□ </span>
144                 <div>
145                     <h6 class="fw-bold mb-1">Агенты ИИ</h6>
146                     <p class="small text-muted mb-0">Конфигурация автономных ReAct агентов, тулов поиска и параметров инференса.</p>
147                 </div>
148             </div>
149         </div>
150     </div>
151
152     <!-- Storage & Disks Management Section -->
153     <div class="card shadow-sm border-secondary-subtle mb-3">
154         <div class="card-header bg-body-tertiary d-flex flex-wrap justify-content-between align-items-center py-2 gap-2">
155             <span class="fw-bold"><i class="bi bi-hdd-network text-warning me-2"></i>□ Управление дисками и накопителями</span>
156             <button class="btn btn-outline-info btn-sm" onclick="rescanStorageDrives()">
157                 <i class="bi bi-arrow-clockwise me-1"></i> Пересканировать диски
158             </button>
159         </div>
160         <div class="card-body p-3">
161             <div id="sys-drives-detailed-container" class="row g-3">
162                 <div class="col-12 text-muted small">Загрузка информации о накопителях...</div>
163             </div>
164         </div>

```



```
165 |     </div>
```

```
166 | </div>
```

main.js

```
1 // System Admin Tab – Central Management Logic
2
3 'use strict';
4
5 async function initSystemAdminTab() {
6   console.log('Инициализация панели системного управления...')
7   ;
8   await refreshSystemDashboard();
9 }
10
11 async function refreshSystemDashboard() {
12   try {
13     // 1. Загрузка дисков
14     const drivesData = await window.api.fetch('/api/control/re
15       scan', { method: 'GET' }).catch(() => ({ drives: [], details:
16       [] }));
17     const drives = Array.isArray(drivesData.drives) ? drivesDa
18       ta.drives : [];
19     const details = Array.isArray(drivesData.details) ? drives
20       Data.details : [];
21     const drivesCountEl = document.getElementById('sys-drives-
22       count');
23     const drivesListEl = document.getElementById('sys-drives-l
24       ist');
25     if (drivesCountEl) drivesCountEl.textContent = `${drives.l
26       ength} диск(ов)`;
27     if (drivesListEl) drivesListEl.textContent = drives.join('
28       , ') || 'Диски не обнаружены';
29
30     const detailedContainer = document.getElementById('sys-dri
31       ves-detailed-container');
32     if (detailedContainer) {
33       if (details.length === 0 && drives.length === 0) {
34         detailedContainer.innerHTML = '<div class="col-12 text
35           -muted small">Накопители не обнаружены или доступ ограничен.</
36           div>';
```

```

25 |     } else if (details.length > 0) {
26 |         detailedContainer.innerHTML = details.map(d => {
27 |             const percent = d.percent || 0;
28 |             const barClass = percent > 90 ? 'bg-danger' : percent > 75 ? 'bg-warning' : 'bg-primary';
29 |             return `
30 |                 <div class="col-lg-4 col-md-6">
31 |                     <div class="card bg-body-tertiary border-secondary-subtle h-100 p-3">
32 |                         <div class="d-flex justify-content-between align-items-center mb-2">
33 |                             <span class="fw-bold text-primary"><i class="bi bi-hdd me-1"></i>${d.device || d.mountpoint}</span>
34 |                             <span class="badge bg-secondary-subtle text-body">${d.fstype || 'NTFS'}</span>
35 |                         </div>
36 |                         <div class="d-flex justify-content-between small text-muted mb-1">
37 |                             <span>Занято: <strong>${d.used_gb} GB</strong></span>
38 |                             <span>Свободно: <strong>${d.free_gb} GB</strong></span>
39 |                         </div>
40 |                         <div class="progress mb-2" style="height: 8px;
41 |                             <div class="progress-bar ${barClass}" role="progressbar" style="width: ${percent}%;" aria-valuenow="${percent}" aria-valuemin="0" aria-valuemax="100"></div>
42 |                         </div>
43 |                         <div class="d-flex justify-content-between small text-muted">
44 |                             <span>Всего: ${d.total_gb} GB</span>
45 |                             <span class="fw-semibold">${percent}%</span>
46 |                         </div>
47 |                     </div>
48 |                 </div>
49 |             `;
50 |         }).join('');

```

```

51 |     } else {
52 |         detailedContainer.innerHTML = drives.map(drv => `
53 |             <div class="col-lg-3 col-md-4 col-sm-6">
54 |                 <div class="card bg-body-tertiary border-secondary
55 | -subtle p-2 text-center">
56 |                     <span class="fw-bold text-primary"><i class="bi
57 | bi-hdd me-1"></i>${drv}</span>
58 |                 </div>
59 |             </div>
60 |         `).join('');
61 |     }
62 | }
63 |
64 | // 2. Загрузка статуса плагинов
65 | const pluginsData = await window.api.fetch('/api/admin/plu
66 | gins').catch(() => ({ plugins: [] }));
67 | const plugins = pluginsData.plugins || [];
68 | const pluginsCountEl = document.getElementById('sys-plugin
69 | s-count');
70 | const pluginsActiveEl = document.getElementById('sys-plugi
71 | ns-active-count');
72 | const activeCount = plugins.filter(p => p.enabled).length;
73 | if (pluginsCountEl) pluginsCountEl.textContent = `${plugin
74 | s.length} модулей`;
75 | if (pluginsActiveEl) pluginsActiveEl.textContent = `${acti
76 | veCount} активно из ${plugins.length}`;
77 | } catch (err) {
78 |     console.error('Ошибка загрузки системного дашборда:', err)
79 |     ;
80 | }
81 | }
82 |
83 | async function rescanStorageDrives() {
84 |     try {
85 |         if (typeof showNotification === 'function') showNotificati
86 | on('Пересканирование накопителей ОС...', 'info');
87 |         const result = await window.api.fetch('/api/control/rescan
88 | ', { method: 'GET' });

```

```
79     const drivesList = Array.isArray(result.drives) ? result.d
rives.join(', ') : 'OK';
80     if (typeof showNotification === 'function') showNotificati
on(`Диски обновлены: ${drivesList}`, 'success');
81     await refreshSystemDashboard();
82   } catch (e) {
83     if (typeof showNotification === 'function') showNotificati
on(`Ошибка: ${e.message}`, 'danger');
84   }
85 }
86
87 async function actualizeAiModels() {
88   try {
89     if (typeof showNotification === 'function') showNotificati
on('Актуализация пула моделей ИИ...', 'info');
90     await window.api.fetch('/api/keys/actualize-all', { method
: 'POST' }).catch(() => {});
91     if (typeof showNotification === 'function') showNotificati
on('Модели успешно синхронизированы', 'success');
92     await refreshSystemDashboard();
93   } catch (e) {
94     if (typeof showNotification === 'function') showNotificati
on(`Ошибка: ${e.message}`, 'danger');
95   }
96 }
97
98 function switchToTab(tabName) {
99   const triggerEl = document.querySelector(`[data-bs-target="#
tab-${tabName}"]`);
100   if (triggerEl) {
101     const tab = new bootstrap.Tab(triggerEl);
102     tab.show();
103   }
104 }
105
106 window.initAdminTab = initSystemAdminTab;
107 window.rescanStorageDrives = rescanStorageDrives;
108 window.actualizeAiModels = actualizeAiModels;
```

```
109 | window.switchToTab = switchToTab;  
110 |
```

index.html

```
1 <!-- AI Agents Management & Builder Tab -->
2 <style>
3   /* High contrast & modern dark styling for AI Agents Tab */
4   .agent-card {
5     background: #0f172a !important;
6     border: 1px solid #334155 !important;
7     border-radius: 12px;
8     transition: transform 0.15s ease, border-color 0.15s ease,
9     box-shadow 0.15s ease;
10  }
11  .agent-card:hover {
12    border-color: #475569 !important;
13    box-shadow: 0 6px 20px rgba(0, 0, 0, 0.45);
14  }
15  .agent-card .card-header {
16    background: #1e293b !important;
17    border-bottom: 1px solid #334155 !important;
18    color: #ffffff !important;
19    border-top-left-radius: 11px;
20    border-top-right-radius: 11px;
21  }
22  .agent-card .agent-desc {
23    color: #cbd5e1 !important;
24  }
25  .agent-spec-box {
26    background: #020617 !important;
27    border: 1px solid #334155 !important;
28    border-radius: 8px;
29    padding: 0.6rem 0.8rem;
30  }
31  .agent-spec-box .spec-label {
32    color: #cbd5e1 !important;
33    font-weight: 600;
34    font-size: 0.82rem;
35  }
36  .agent-spec-box .spec-value-badge {
```

```
36 |     background-color: #2563eb !important;
37 |     color: #ffffff !important;
38 |     font-weight: 600;
39 |     font-size: 0.78rem;
40 | }
41 | .agent-spec-box .spec-tech-val {
42 |     color: #38bdf8 !important;
43 |     font-weight: 700;
44 |     font-size: 0.85rem;
45 | }
46 | .tool-badge {
47 |     background: #1e293b !important;
48 |     border: 1px solid #475569 !important;
49 |     color: #f8fafc !important;
50 |     font-size: 0.75rem;
51 |     font-weight: 500;
52 |     border-radius: 6px;
53 |     padding: 0.25rem 0.5rem;
54 |     display: inline-flex;
55 |     align-items: center;
56 |     gap: 0.25rem;
57 | }
58 | .agent-card .btn-test-sandbox {
59 |     color: #fbbf24 !important;
60 |     border: 1.5px solid #f59e0b !important;
61 |     font-weight: 600;
62 |     font-size: 0.82rem;
63 |     background: transparent;
64 | }
65 | .agent-card .btn-test-sandbox:hover {
66 |     background: #f59e0b !important;
67 |     color: #0f172a !important;
68 | }
69 | .agent-card .btn-edit-agent {
70 |     color: #38bdf8 !important;
71 |     border: 1.5px solid #0284c7 !important;
72 |     font-weight: 600;
73 |     background: transparent;
```



```
74 | }
75 | .agent-card .btn-edit-agent:hover {
76 |     background: #0284c7 !important;
77 |     color: #ffffff !important;
78 | }
79 | .agent-card .btn-delete-agent {
80 |     color: #f87171 !important;
81 |     border: 1.5px solid #dc2626 !important;
82 |     font-weight: 600;
83 |     background: transparent;
84 | }
85 | .agent-card .btn-delete-agent:hover {
86 |     background: #dc2626 !important;
87 |     color: #ffffff !important;
88 | }
89 | #agents-filter-group .btn {
90 |     background: #1e293b;
91 |     border: 1px solid #475569;
92 |     color: #f1f5f9;
93 |     font-weight: 600;
94 |     font-size: 0.82rem;
95 | }
96 | #agents-filter-group .btn:hover {
97 |     background: #334155;
98 |     color: #ffffff;
99 |     border-color: #64748b;
100 | }
101 | #agents-filter-group .btn.active {
102 |     background: #2563eb !important;
103 |     border-color: #3b82f6 !important;
104 |     color: #ffffff !important;
105 |     box-shadow: 0 0 8px rgba(37, 99, 235, 0.4);
106 | }
107 | #btn-refresh-agents {
108 |     background: #1e293b;
109 |     border: 1px solid #475569;
110 |     color: #f1f5f9;
111 |     font-weight: 600;
```

```

112     font-size: 0.82rem;
113 }
114 #btn-refresh-agents:hover {
115     background: #334155;
116     color: #ffffff;
117     border-color: #64748b;
118 }
119 </style>
120
121 <div class="container-fluid py-3">
122
123     <!-- Header Alert & Quick Actions -->
124     <div class="d-flex justify-content-between align-items-center flex-wrap gap-3 mb-3 p-3 rounded-3 shadow-sm text-white" style="background: linear-gradient(135deg, #0f172a 0%, #1e1b4b 100%); border: 1px solid #334155;">
125         <div>
126             <h5 class="mb-1 fw-bold text-white d-flex align-items-center gap-2">
127                 <span class="fs-4">🔮 </span> Управление и создание ИИ-Агентов
128             </h5>
129             <p class="mb-0 small" style="color: #cbd5e1;">
130                 Конфигурирование специализированных агентов, назначение моделей из пула, подключение инструментов и генерация через AI.
131             </p>
132         </div>
133         <div class="d-flex align-items-center gap-2 flex-wrap">
134             <button class="btn btn-sm btn-outline-info rounded-pill px-3 shadow-sm fw-semibold" id="btn-open-ai-builder" style="border-width: 1.5px;">
135                 <i class="bi bi-magic me-1"></i> 🔮 Создать с помощью AI
136             </button>
137             <button class="btn btn-sm btn-primary rounded-pill px-3 shadow-sm fw-semibold" id="btn-open-create-agent">
138                 <i class="bi bi-plus-lg me-1"></i> Добавить вручную

```

```
139 |         </button>
140 |     </div>
141 | </div>
142 |
143 | <!-- Collapsible Architecture & SVG Scheme Block -->
144 | <div class="card mb-4 shadow-sm text-white" style="background-color: #0f172a !important; border: 1px solid #334155 !important; border-radius: 10px;">
145 |     <div class="card-body p-3">
146 |         <div class="d-flex align-items-center justify-content-between">
147 |             <div class="d-flex align-items-center gap-2">
148 |                 <i class="bi bi-diagram-3-fill text-info fs-5"></i>
149 |                 <span class="fw-bold" style="color: #f1f5f9;">Интерактивная справка: архитектура работы и создания агентов</span>
150 |             </div>
151 |             <button class="btn btn-sm btn-link text-decoration-none p-0 fw-semibold" style="color: #38bdf8;" type="button" data-bs-toggle="collapse" data-bs-target="#agentsArchCollapse" aria-expanded="false" aria-controls="agentsArchCollapse">
152 |                 <i class="bi bi-chevron-down"></i> Развернуть / Скрыть схему
153 |             </button>
154 |         </div>
155 |         <div class="collapse" id="agentsArchCollapse">
156 |             <div class="mt-3 pt-3" style="border-top: 1px solid rgba(255, 255, 255, 0.1);">
157 |                 <div class="text-center p-2 rounded mb-3" style="background: #020617; border: 1px solid rgba(255, 255, 255, 0.08);">
158 |                     
159 |                 </div>
160 |                 <div class="row g-3 small">
161 |                     <div class="col-md-3">
162 |                         <div class="p-3 rounded h-100" style="background
```

```
: #1e293b; border: 1px solid #334155; color: #f1f5f9; line-height: 1.55;">
163         <strong class="d-block mb-1" style="color: #38bdf8; font-size: 0.88rem;">1. Пул моделей (Model Pool)</strong>
164         <div class="p-3 rounded h-100" style="background-color: #1e293b; border: 1px solid #334155; color: #f1f5f9; line-height: 1.55;">
165             Каждый агент может использовать независимую модель: Google Gemini (с ротацией ключей), Antigravity AGY, Microsoft Foundry или Ollama.
166         </div>
167     </div>
168     <div class="col-md-3">
169         <div class="p-3 rounded h-100" style="background-color: #1e293b; border: 1px solid #334155; color: #f1f5f9; line-height: 1.55;">
170             <strong class="d-block mb-1" style="color: #4ade80; font-size: 0.88rem;">2. Каталог инструментов (Tools)</strong>
171             Набор готовых модулей: интернет-поиск (MCP / Google Grounding), RAG база знаний, Python Code Execution, чтение файлов и синтез речи TTS.
172         </div>
173     </div>
174     <div class="col-md-3">
175         <div class="p-3 rounded h-100" style="background-color: #1e293b; border: 1px solid #334155; color: #f1f5f9; line-height: 1.55;">
176             <strong class="d-block mb-1" style="color: #fb8bf24; font-size: 0.88rem;">3. AI-Архитектор промптов</strong>
177             Модель из пула анализирует вашу цель и генерирует структурированную роль, правила ReAct и список инструментов.
178         </div>
179     </div>
180     <div class="col-md-3">
181         <div class="p-3 rounded h-100" style="background-color: #1e293b; border: 1px solid #334155; color: #f1f5f9; line-height: 1.55;">
182             <strong class="d-block mb-1" style="color: #fb8bf24; font-size: 0.88rem;">4. AI-Ассистент</strong>
183             Модель из пула анализирует вашу задачу и генерирует структурированную роль, правила ReAct и список инструментов.
184         </div>
185     </div>
186 </div>
187 </div>
188 </div>
```

```

7171; font-size: 0.88rem;">4. Песочница (Sandbox)</strong>
182         Тестирование поведения агента, отслеживание це
почки рассуждений (Thought & Action) и замер задержек ответа д
о боевого использования.
183         </div>
184     </div>
185 </div>
186 </div>
187 </div>
188 </div>
189 </div>
190
191 <!-- Filters and Status Toolbar -->
192 <div class="d-flex justify-content-between align-items-cente
r mb-3 flex-wrap gap-2">
193     <div class="btn-group btn-group-sm" role="group" id="agent
s-filter-group">
194         <button type="button" class="btn active" data-filter="al
1">Все (<span id="count-all">0</span>)</button>
195         <button type="button" class="btn" data-filter="active">А
ктивные (<span id="count-active">0</span>)</button>
196         <button type="button" class="btn" data-filter="system">С
истемные (<span id="count-system">0</span>)</button>
197         <button type="button" class="btn" data-filter="custom">П
ользовательские (<span id="count-custom">0</span>)</button>
198     </div>
199     <div class="d-flex align-items-center gap-2">
200         <button class="btn btn-sm rounded-pill px-3" id="btn-ref
resh-agents" title="Обновить список">
201             <i class="bi bi-arrow-clockwise me-1"></i> Обновить
202         </button>
203     </div>
204 </div>
205
206 <!-- Agents Grid -->
207 <div class="row g-3" id="agents-grid">
208     <div class="col-12 text-center py-5" style="color: #cbd5e1
; ">

```

```

209     <div class="spinner-border text-primary mb-2" role="stat
| us"></div>
210     <div class="fw-semibold">Загрузка агентов и пула моделей
| ...</div>
211     </div>
212 </div>
213
214 </div>
215
216 <!-- =====
| ===== -->
217 <!-- MODAL: AI Agent Builder (Генерация через LLM)
|         -->
218 <!-- =====
| ===== -->
219 <div class="modal fade" id="modal-ai-builder" tabindex="-1" ar
| ia-labelledby="modalAiBuilderLabel" aria-hidden="true">
220     <div class="modal-dialog modal-lg modal-dialog-centered">
221         <div class="modal-content text-white" style="background-co
| lor: #0f172a !important; border: 1px solid #334155 !important;
| border-radius: 12px;">
222             <div class="modal-header" style="background-color: #1e29
| 3b !important; border-bottom: 1px solid #334155 !important;">
223                 <h5 class="modal-title fw-bold" style="color: #38bdf8;
| " id="modalAiBuilderLabel">
224                     <i class="bi bi-magic me-2"></i>AI Agent Builder – К
| онструктор агентов
225                 </h5>
226                 <button type="button" class="btn-close btn-close-white
| " data-bs-dismiss="modal" aria-label="Close"></button>
227             </div>
228             <div class="modal-body p-4">
229                 <p class="small mb-3" style="color: #cbd5e1; line-heig
| ht: 1.5;">
230                     Опишите задачу агента своими словами. Выбранная моде
| ль из пула проанализирует цель, подберёт необходимые инструмен
| ты и сформирует профессиональный системный промпт.
231                 </p>

```

```
232 |
233 |         <div class="row g-3 mb-3">
234 |             <div class="col-md-6">
235 |                 <label class="form-label small fw-semibold" style=
| "color: #f1f5f9;">Провайдер модели-архитектора:</label>
236 |                 <select class="form-select form-select-sm" style="
| background-color: #020617 !important; color: #f8fafc !importan
| t; border: 1px solid #475569 !important;" id="ai-builder-provi
| der">
237 |                     <option value="gemini">Google Gemini</option>
238 |                     <option value="gemini_cli">Google Gemini CLI</op
| tion>
239 |                     <option value="agy">Google Antigravity (AGY)</op
| tion>
240 |                     <option value="foundry">Microsoft Foundry (Local
| )</option>
241 |                     <option value="ollama">Ollama (Local)</option>
242 |                 </select>
243 |             </div>
244 |             <div class="col-md-6">
245 |                 <label class="form-label small fw-semibold" style=
| "color: #f1f5f9;">Модель из пула:</label>
246 |                 <select class="form-select form-select-sm" style="
| background-color: #020617 !important; color: #f8fafc !importan
| t; border: 1px solid #475569 !important;" id="ai-builder-model
| ">
247 |                     <!-- Заполняется динамически -->
248 |                 </select>
249 |             </div>
250 |         </div>
251 |
252 |         <div class="mb-3">
253 |             <label class="form-label small fw-semibold" style="c
| olor: #f1f5f9;">Опишите задачу агента:</label>
254 |             <textarea class="form-control" style="background-col
| or: #020617 !important; color: #f8fafc !important; border: 1px
| solid #475569 !important;" id="ai-builder-prompt" rows="4" pl
| aceholder="Например: Создай агента для автономного поиска субт
```

итров к фильмам, проверки их таймингов и сопоставления с медиа
текой..."></textarea>

255 </div>

256

257 <div class="d-flex justify-content-end mb-3">

258 <button class="btn btn-info rounded-pill px-4 fw-bol
d" id="btn-run-ai-generate" style="background-color: #0284c7 !
important; border-color: #0284c7 !important; color: #ffffff !i
mportant;">

259 <i class="bi bi-stars me-1"></i> Сгенерировать спе
цификацию

260 </button>

261 </div>

262

263 <!-- Result Preview Section -->

264 <div id="ai-builder-result" class="d-none pt-3" style=
"border-top: 1px solid #334155;">

265 <h6 class="fw-bold mb-2" style="color: #4ade80;"><i
class="bi bi-check2-circle me-1"></i> Спецификация сформирован
а:</h6>

266 <div class="p-3 rounded mb-3 small font-monospace" s
tyle="background: #020617; border: 1px solid #334155; color: #
f8fafc;" id="ai-builder-preview-box">

267 <!-- Превью JSON / данных -->

268 </div>

269 <div class="d-flex justify-content-end gap-2">

270 <button class="btn btn-outline-secondary btn-sm ro
unded-pill px-3" data-bs-dismiss="modal" style="color: #cbd5e1
; border-color: #475569;">Отмена</button>

271 <button class="btn btn-success btn-sm rounded-pill
px-3 fw-semibold" id="btn-apply-ai-generated">

272 <i class="bi bi-pencil-square me-1"></i> Открыть
в редакторе агента

273 </button>

274 </div>

275 </div>

276 </div>

277 </div>


```

278 |     </div>
279 | </div>
280 |
281 | <!-- =====
| ===== -->
282 | <!-- MODAL: Agent Editor (Создание и Редактирование)
|         -->
283 | <!-- =====
| ===== -->
284 | <div class="modal fade" id="modal-agent-editor" tabindex="-1"
| aria-labelledby="modalAgentEditorLabel" aria-hidden="true">
285 |     <div class="modal-dialog modal-xl modal-dialog-centered moda
| l-dialog-scrollable">
286 |         <div class="modal-content text-white" style="background-co
| lor: #0f172a !important; border: 1px solid #334155 !important;
| border-radius: 12px;">
287 |             <div class="modal-header" style="background-color: #1e29
| 3b !important; border-bottom: 1px solid #334155 !important;">
288 |                 <h5 class="modal-title fw-bold text-white" id="modalAg
| entEditorLabel">
289 |                     <i class="bi bi-robot me-2" style="color: #38bdf8;">
| </i>Настройка агента
290 |                 </h5>
291 |                 <button type="button" class="btn-close btn-close-white
| " data-bs-dismiss="modal" aria-label="Close"></button>
292 |             </div>
293 |             <div class="modal-body p-4">
294 |                 <form id="agent-form">
295 |                     <input type="hidden" id="agent-edit-mode" value="cre
| ate">
296 |
297 |                     <div class="row g-3 mb-3">
298 |                         <div class="col-md-4">
299 |                             <label class="form-label small fw-semibold" styl
| e="color: #f1f5f9;">Уникальный ID агента:</label>
300 |                             <input type="text" class="form-control form-cont
| rol-sm font-monospace" style="background-color: #020617 !impor
| tant; color: #f8fafc !important; border: 1px solid #475569 !im

```

```

    portant;" id="agent-id" placeholder="my_custom_agent" required
    >
301         </div>
302         <div class="col-md-5">
303             <label class="form-label small fw-semibold" styl
e="color: #f1f5f9;">Отображаемое имя:</label>
304             <input type="text" class="form-control form-cont
rol-sm" style="background-color: #020617 !important; color: #f
8fafc !important; border: 1px solid #475569 !important;" id="a
gent-name" placeholder="Название агента" required>
305         </div>
306         <div class="col-md-3">
307             <label class="form-label small fw-semibold" styl
e="color: #f1f5f9;">Статус активности:</label>
308             <div class="form-check form-switch mt-1">
309                 <input class="form-check-input" type="checkbox"
" id="agent-enabled" checked>
310                 <label class="form-check-label small fw-medium
" style="color: #e2e8f0;" for="agent-enabled">Агент активен</l
abel>
311             </div>
312         </div>
313     </div>
314
315     <div class="mb-3">
316         <label class="form-label small fw-semibold" style=
"color: #f1f5f9;">Краткое описание роли:</label>
317         <input type="text" class="form-control form-contro
l-sm" style="background-color: #020617 !important; color: #f8f
af8 !important; border: 1px solid #475569 !important;" id="age
nt-desc" placeholder="Какую задачу решает данный агент">
318     </div>
319
320     <!-- Model & Provider Selection from Pool -->
321     <div class="card mb-3" style="background: #0b0f19 !i
mportant; border: 1px solid #334155 !important; border-radius:
8px;">
322         <div class="card-header py-2" style="background: #

```

```

1e293b !important; border-bottom: 1px solid #334155 !important
; ">
323         <span class="fw-bold small" style="color: #38bdf
8;"><i class="bi bi-cpu me-1"></i> Привязка к модели из пула</
span>
324     </div>
325     <div class="card-body p-3">
326         <div class="row g-3">
327             <div class="col-md-6">
328                 <label class="form-label small fw-semibold"
style="color: #f1f5f9;">Провайдер ИИ:</label>
329                 <select class="form-select form-select-sm" s
tyle="background-color: #020617 !important; color: #f8fafc !im
portant; border: 1px solid #475569 !important;" id="agent-prov
ider">
330                     <option value="gemini">Google Gemini</opti
on>
331                     <option value="gemini_cli">Google Gemini C
LI</option>
332                     <option value="agy">Google Antigravity (AG
Y)</option>
333                     <option value="foundry">Microsoft Foundry<
/option>
334                     <option value="ollama">Ollama (Local)</opt
ion>
335                 </select>
336             </div>
337             <div class="col-md-6">
338                 <label class="form-label small fw-semibold"
style="color: #f1f5f9;">Модель:</label>
339                 <select class="form-select form-select-sm" s
tyle="background-color: #020617 !important; color: #f8fafc !im
portant; border: 1px solid #475569 !important;" id="agent-mode
l">
340                     <!-- Заполняется динамически -->
341                 </select>
342             </div>
343         </div>

```

```
344 |
345 |         <div class="row g-3 mt-1">
346 |             <div class="col-md-4">
347 |                 <label class="form-label small fw-semibold d
- flex justify-content-between" style="color: #f1f5f9;">
348 |                     <span>Температура:</span> <span id="agent-
temp-val" class="fw-bold" style="color: #38bdf8;">0.3</span>
349 |                 </label>
350 |                 <input type="range" class="form-range" id="a
gent-temperature" min="0" max="1" step="0.05" value="0.3">
351 |             </div>
352 |             <div class="col-md-4">
353 |                 <label class="form-label small fw-semibold"
style="color: #f1f5f9;">Макс. шагов ReAct (Max Steps):</label>
354 |                 <input type="number" class="form-control for
m-control-sm" style="background-color: #020617 !important; col
or: #f8fafc !important; border: 1px solid #475569 !important;"
id="agent-max-steps" min="1" max="50" value="15">
355 |             </div>
356 |             <div class="col-md-4">
357 |                 <label class="form-label small fw-semibold"
style="color: #f1f5f9;">Таймаут (сек):</label>
358 |                 <input type="number" class="form-control for
m-control-sm" style="background-color: #020617 !important; col
or: #f8fafc !important; border: 1px solid #475569 !important;"
id="agent-timeout" min="5" max="300" value="60">
359 |             </div>
360 |         </div>
361 |     </div>
362 | </div>
363 |
364 |     <!-- Tools Selection Matrix -->
365 |     <div class="card mb-3" style="background: #0b0f19 !i
mportant; border: 1px solid #334155 !important; border-radius:
8px;">
366 |         <div class="card-header py-2 d-flex justify-conten
t-between align-items-center" style="background: #1e293b !impo
rtant; border-bottom: 1px solid #334155 !important;">
```

```

367 |         <span class="fw-bold small" style="color: #4ade8
    | 0;"><i class="bi bi-tools me-1"></i> Подключенные инструменты
    | (Capabilities)</span>
368 |         <span class="badge" style="background: #334155;
    | color: #f8fafc; font-weight: 600;" id="agent-tools-count">0 вы
    | брано</span>
369 |     </div>
370 |     <div class="card-body p-3">
371 |         <div class="row g-2" id="agent-tools-matrix">
372 |             <!-- Чекбоксы инструментов загружаются динамич
    | ески -->
373 |         </div>
374 |     </div>
375 | </div>
376 |
377 |     <!-- System Instruction Editor -->
378 |     <div class="mb-3">
379 |         <div class="d-flex justify-content-between align-i
    | tems-center mb-1">
380 |             <label class="form-label small fw-semibold mb-0"
    | style="color: #f1f5f9;">Системная инструкция (System Prompt):
    | </label>
381 |             <div class="btn-group btn-group-sm">
382 |                 <button type="button" class="btn btn-outline-s
    | econdary btn-sm py-0" style="color: #cbd5e1; border-color: #47
    | 5569;" id="btn-insert-react-template">Шаблон ReAct</button>
383 |                 <button type="button" class="btn btn-outline-s
    | econdary btn-sm py-0" style="color: #cbd5e1; border-color: #47
    | 5569;" id="btn-insert-json-template">Шаблон JSON-ответа</butto
    | n>
384 |             </div>
385 |         </div>
386 |         <textarea class="form-control font-monospace" styl
    | e="background-color: #020617 !important; color: #f8fafc !impor
    | tant; border: 1px solid #475569 !important; font-size: 0.85rem
    | ; line-height: 1.45;" id="agent-system-prompt" rows="7" placeh
    | older="Опишите поведение агента, правила вызова инструментов и
    | формат ответа..." spellcheck="false" required></textarea>

```

```

387         </div>
388
389     </form>
390 </div>
391     <div class="modal-footer" style="background-color: #1e29
3b !important; border-top: 1px solid #334155 !important;">
392         <button type="button" class="btn btn-outline-secondary
rounded-pill px-3" data-bs-dismiss="modal" style="color: #cbd
5e1; border-color: #475569;">Отмена</button>
393         <button type="button" class="btn btn-primary rounded-p
ill px-4 fw-bold" id="btn-save-agent">
394         <i class="bi bi-check-circle me-1"></i> Сохранить аг
ента
395     </button>
396 </div>
397 </div>
398 </div>
399 </div>
400
401 <!-- =====
===== -->
402 <!-- MODAL: Agent Sandbox (Песочница тестирования)
-->
403 <!-- =====
===== -->
404 <div class="modal fade" id="modal-agent-sandbox" tabindex="-1"
aria-labelledby="modalAgentSandboxLabel" aria-hidden="true">
405     <div class="modal-dialog modal-lg modal-dialog-centered moda
l-dialog-scrollable">
406         <div class="modal-content text-white" style="background-co
lor: #0f172a !important; border: 1px solid #334155 !important;
border-radius: 12px;">
407             <div class="modal-header" style="background-color: #1e29
3b !important; border-bottom: 1px solid #334155 !important;">
408                 <h5 class="modal-title fw-bold" style="color: #fbbf24;
" id="modalAgentSandboxLabel">
409                 <i class="bi bi-play-circle me-2"></i> Песочница т
естирования агента

```

```
410         </h5>
411         <button type="button" class="btn-close btn-close-white
" data-bs-dismiss="modal" aria-label="Close"></button>
412     </div>
413     <div class="modal-body p-4">
414
415         <!-- Active Agent Summary in Sandbox -->
416         <div class="d-flex align-items-center justify-content-
between p-2 px-3 rounded mb-3" style="background: #020617; bor
der: 1px solid #334155;">
417             <div>
418                 <strong id="sandbox-agent-name" class="text-white
fs-6">--</strong>
419                 <span class="badge ms-2" style="background-color:
#2563eb; color: #ffffff; font-weight: 600;" id="sandbox-agent-
model">--</span>
420             </div>
421             <span class="small fw-medium" style="color: #cbd5e1;
" id="sandbox-agent-tools-summary">Инструментов: 0</span>
422         </div>
423
424         <!-- Chat / Trace Container -->
425         <div class="rounded p-3 mb-3" style="background: #0206
17; border: 1px solid #334155; min-height: 240px; max-height:
380px; overflow-y: auto;" id="sandbox-chat-window">
426             <div class="text-center small py-4" style="color: #9
4a3b8;" id="sandbox-placeholder">
427                 <i class="bi bi-chat-dots fs-3 d-block mb-1"></i>
428                 Введите тестовый запрос ниже для проверки логики а
гента и цепочки вызовов инструментов.
429             </div>
430             <div id="sandbox-messages-list"></div>
431         </div>
432
433         <!-- Input Box -->
434         <div class="input-group">
435             <input type="text" class="form-control" style="backg
round-color: #020617 !important; color: #f8fafc !important; bo
```

```
border: 1px solid #475569 !important;" id="sandbox-input-msg" placeholder="Например: Найди фильм Интерстеллар и проверь торренты...">
```

```
436         <button class="btn btn-warning fw-bold px-4" id="btn-sandbox-send" style="background-color: #f59e0b; border-color: #f59e0b; color: #0f172a;">
```

```
437             <i class="bi bi-send-fill me-1"></i> Тест
```

```
438         </button>
```

```
439     </div>
```

```
440
```

```
441 </div>
```

```
442     <div class="modal-footer" style="background-color: #1e293b !important; border-top: 1px solid #334155 !important;">
```

```
443         <button type="button" class="btn btn-outline-secondary rounded-pill px-3" data-bs-dismiss="modal" style="color: #cbd5e1; border-color: #475569;">Закрыть песочницу</button>
```

```
444     </div>
```

```
445 </div>
```

```
446 </div>
```

```
447 </div>
```

```
448
```


main.js

```
1 /**
2  * =====
3  * Process Name: AI Agents Management and Builder Interface
4  * =====
5  * Description:
6  *   Client-side controller for managing AI agent registry, model pool binding,
7  *   automated specification generation, and execution sandbox testing.
8  *
9  * File: main.js
10 * Project: Mediteka
11 * Module: AgentsTab
12 * Author: hypo69
13 * Copyright: © 2026 hypo69
14 * =====
15 */
16
17 'use strict';
18
19 /**
20 * Global reactive state for the AI Agents tab.
21 */
22 const _state = {
23   agents: [],
24   tools: [],
25   providers: {},
26   currentFilter: 'all',
27   activeSandboxAgent: null,
28   lastGeneratedSpec: null,
29   isEventsBound: false
30 };
31
```

```

32 | /**
33 |  * Initializes the AI Agents management tab.
34 |  * Called on tab switch from the main admin interface.
35 |  *
36 |  * @returns {Promise<void>}
37 |  */
38 | async function initAgentsTab() {
39 |     _bindStaticEvents();
40 |     await _loadAllData();
41 | }
42 |
43 | window.initAgentsTab = initAgentsTab;
44 |
45 | /**
46 |  * Executes an HTTP JSON request using window.api or standard
47 |  * fetch.
48 |  *
49 |  * @param {string} endpoint - Target API endpoint path.
50 |  * @param {RequestInit} [options={}] - Optional fetch configur
51 |  * ation options.
52 |  * @returns {Promise<any>} Parsed JSON response.
53 |  * @throws {Error} Thrown if response status is not OK.
54 |  */
55 | async function _fetchJson(endpoint, options = {}) {
56 |     if (window.api && typeof window.api.fetch === 'function') {
57 |         return await window.api.fetch(endpoint, options);
58 |     }
59 |     const response = await fetch(endpoint, options);
60 |     if (!response.ok) {
61 |         throw new Error(`HTTP ${response.status}: ${response.statu
62 | sText}`);
63 |     }
64 |     return await response.json();
65 | }
66 |
67 | /**
68 |  * Loads agents, capabilities catalog, and model providers fro
69 |  * m the backend API.

```

```

66 | *
67 | * @returns {Promise<void>}
68 | */
69 | async function _loadAllData() {
70 |     try {
71 |         const [agentsRes, toolsRes, providersRes] = await Promise.
all([
72 |             _fetchJson('/api/agents'),
73 |             _fetchJson('/api/agents/tools'),
74 |             _fetchJson('/api/agents/providers')
75 |         ]);
76 |
77 |         _state.agents = Array.isArray(agentsRes) ? agentsRes : [];
78 |         _state.tools = Array.isArray(toolsRes) ? toolsRes : [];
79 |         _state.providers = providersRes ?? {};
80 |
81 |         _renderCounters();
82 |         _renderGrid();
83 |         _populateToolsMatrix();
84 |         _updateModelDropdown('agent-provider', 'agent-model');
85 |         _updateModelDropdown('ai-builder-provider', 'ai-builder-mo
del');
86 |     } catch (err) {
87 |         console.error('[AgentsTab] Failed to load data:', err);
88 |         _showToast(`Failed to load data: ${err.message}`, 'danger'
);
89 |         const grid = document.getElementById('agents-grid');
90 |         if (grid) {
91 |             grid.innerHTML = `
92 |                 <div class="col-12 text-center py-5 text-danger">
93 |                     <i class="bi bi-exclamation-triangle fs-1 d-block mb
-2"></i>
94 |                     <h5>Не удалось загрузить данные агентов</h5>
95 |                     <p class="small text-secondary">${err.message}</p>
96 |                     <button class="btn btn-sm btn-outline-primary rounde
d-pill px-3" onclick="window.initAgentsTab()">Повторить попытку</button>
97 |                 </div>

```

```

98     `;
99     }
100 }
101 }
102
103 /**
104  * Binds static DOM event listeners for buttons, filters, and
105  * modal templates.
106  * @returns {void}
107  */
108 function _bindStaticEvents() {
109     if (_state.isEventsBound) return;
110     _state.isEventsBound = true;
111
112     // Refresh agents list
113     const refreshBtn = document.getElementById('btn-refresh-agents');
114     if (refreshBtn) {
115         refreshBtn.onclick = async () => {
116             refreshBtn.disabled = true;
117             await _loadAllData();
118             refreshBtn.disabled = false;
119             _showToast('Список агентов обновлен', 'success');
120         };
121     }
122
123     // Filter toolbar buttons
124     const filterGroup = document.getElementById('agents-filter-group');
125     if (filterGroup) {
126         filterGroup.querySelectorAll('button').forEach((btn) => {
127             btn.onclick = () => {
128                 filterGroup.querySelectorAll('button').forEach((b) =>
129                     b.classList.remove('active'));
130                 btn.classList.add('active');
131                 _state.currentFilter = btn.getAttribute('data-filter')
132                     ?? 'all';

```

```
131 |         _renderGrid();
132 |     };
133 | });
134 | }
135 |
136 | // Modal trigger buttons
137 | document.getElementById('btn-open-create-agent')?.addEventListener(
138 |     'click', () => _openEditor(null));
139 | document.getElementById('btn-open-ai-builder')?.addEventListener(
140 |     'click', _openAiBuilder);
141 |
142 | // Provider change handlers
143 | document.getElementById('agent-provider')?.addEventListener(
144 |     'change', () => {
145 |         _updateModelDropdown('agent-provider', 'agent-model');
146 |     });
147 | document.getElementById('ai-builder-provider')?.addEventListener(
148 |     'change', () => {
149 |         _updateModelDropdown('ai-builder-provider', 'ai-builder-model');
150 |     });
151 |
152 | // Temperature slider synchronization
153 | const tempSlider = document.getElementById('agent-temperature');
154 | const tempVal = document.getElementById('agent-temp-val');
155 | if (tempSlider && tempVal) {
156 |     tempSlider.oninput = () => {
157 |         tempVal.textContent = tempSlider.value;
158 |     };
159 | }
160 |
161 | // Action buttons
162 | document.getElementById('btn-save-agent')?.addEventListener(
163 |     'click', _handleSaveAgent);
164 | document.getElementById('btn-run-ai-generate')?.addEventListener(
165 |     'click', _handleRunAiGenerate);
```

```
161 | // Apply AI Generated specification to editor
162 | document.getElementById('btn-apply-ai-generated')?.addEventListener('click', () => {
163 |     if (!_state.lastGeneratedSpec) return;
164 |     const builderModalEl = document.getElementById('modal-ai-builder');
165 |     bootstrap.Modal.getInstance(builderModalEl)?.hide();
166 |     _openEditor(null, _state.lastGeneratedSpec);
167 | });
168 |
169 | // System Prompt templates
170 | document.getElementById('btn-insert-react-template')?.addEventListener('click', () => {
171 |     const promptInput = document.getElementById('agent-system-prompt');
172 |     if (promptInput) {
173 |         promptInput.value =
174 |             'Ты автономный ReAct-агент Mediteka.\n\n' +
175 |             'ПРИНЦИП РАБОТЫ (Thought -> Action -> Observation):\n'
176 |             +
177 |             '1. Тщательно анализируй вопрос пользователя (Thought).\n' +
178 |             '2. Выбирай необходимый инструмент из доступных (Action) с корректными параметрами.\n' +
179 |             '3. Анализируй результат выполнения инструмента (Observation).\n' +
180 |             '4. Сформируй итоговый структурированный ответ пользователю.\n\n' +
181 |             'ФОРМАТ ОТВЕТА:\n' +
182 |             '- Используй Markdown с красивым форматированием (списки, жирный шрифт, эмодзи).\n' +
183 |             '- Не выводите технические ошибки напрямую, объясняй суть решения.';
184 |     };
185 | });
186 | document.getElementById('btn-insert-json-template')?.addEventListener('click', () => {
```

```
187     const promptInput = document.getElementById('agent-system-
prompt');
188     if (promptInput) {
189         promptInput.value =
190             'Ты специализированный аналитический агент Mediteka.\n
\n' +
191             'Твоя задача: структурировать полученные данные и всег
да отвечать строго валидным JSON-объектом без лишнего обрамлен
ия и markdown-блоков.\n\n' +
192             'Пример структуры:\n{\n  "status": "success",\n  "item
s": [],\n  "summary": "краткое резюме"\n}';
193     }
194 });
195
196 // Sandbox inputs
197 const sandboxSendBtn = document.getElementById('btn-sandbox-
send');
198 const sandboxInput = document.getElementById('sandbox-input-
msg');
199 if (sandboxSendBtn && sandboxInput) {
200     sandboxSendBtn.onclick = _handleRunSandboxTest;
201     sandboxInput.onkeypress = (e) => {
202         if (e.key === 'Enter') _handleRunSandboxTest();
203     };
204 }
205 }
206
207 /**
208  * Updates filter counter badges on top toolbar buttons.
209  *
210  * @returns {void}
211  */
212 function _renderCounters() {
213     const allCount = _state.agents.length;
214     const activeCount = _state.agents.filter((a) => a.enabled).l
ength;
215     const systemCount = _state.agents.filter((a) => a.is_system)
.length;
```

```

216 |   const customCount = _state.agents.filter((a) => !a.is_system
    | ).length;
217 |
218 |   document.getElementById('count-all')?.replaceChildren(docume
    | nt.createTextNode(String(allCount)));
219 |   document.getElementById('count-active')?.replaceChildren(doc
    | ument.createTextNode(String(activeCount)));
220 |   document.getElementById('count-system')?.replaceChildren(doc
    | ument.createTextNode(String(systemCount)));
221 |   document.getElementById('count-custom')?.replaceChildren(doc
    | ument.createTextNode(String(customCount)));
222 | }
223 |
224 | /**
225 |  * Renders the responsive agent cards grid based on current fi
    | lter state.
226 |  *
227 |  * @returns {void}
228 |  */
229 | function _renderGrid() {
230 |   const grid = document.getElementById('agents-grid');
231 |   if (!grid) return;
232 |
233 |   const filter = _state.currentFilter;
234 |   let items = _state.agents;
235 |
236 |   if (filter === 'active') items = items.filter((a) => a.enabl
    | ed);
237 |   else if (filter === 'system') items = items.filter((a) => a.
    | is_system);
238 |   else if (filter === 'custom') items = items.filter((a) => !a
    | .is_system);
239 |
240 |   if (items.length === 0) {
241 |     grid.innerHTML = `
242 |       <div class="col-12 text-center py-5" style="color: #cbd5
    | e1;">
243 |         <i class="bi bi-robot fs-1 d-block mb-2" style="color:

```



```

#38bdf8;"></i>
244     <h5 class="text-white">Агенты не найдены</h5>
245     <p class="small" style="color: #94a3b8;">Создайте ново
го агента с помощью AI или вручную</p>
246   </div>
247   `;
248   return;
249 }
250
251 grid.innerHTML = items
252   .map((agent) => {
253     const isSystem = Boolean(agent.is_system);
254     const isEnabled = Boolean(agent.enabled);
255     const providerName = _getProviderDisplayName(agent.provi
der);
256
257     const toolsBadges = (agent.tools ?? [])
258       .map((toolId) => {
259         const tool = _state.tools.find((t) => t.id === toolI
d);
260         const icon = tool?.icon ?? '⬜';
261         const name = tool?.name ?? toolId;
262         return `<span class="badge tool-badge me-1 mb-1 font
-monospace">${icon} <span>${name}</span></span>`;
263       })
264       .join('');
265
266     return `
267       <div class="col-xl-4 col-lg-6">
268         <div class="card h-100 agent-card shadow-sm" style="
${!isEnabled ? 'opacity: 0.65;' : ''}">
269           <div class="card-header d-flex justify-content-bet
ween align-items-center py-2 px-3">
270             <div class="d-flex align-items-center gap-2">
271               <span class="fs-5">${isSystem ? '⬜' : '⬜'}</s
pan>
272               <strong class="text-white fs-6 fw-bold">${agen
t.name}</strong>

```

```

273 |         ${
274 |             isSystem
275 |             ? '<span class="badge fw-bold" style="font
| -size: 0.7rem; background-color: #0284c7 !important; color: #f
| fffff !important;">SYSTEM</span>'
276 |             : '<span class="badge fw-bold" style="font
| -size: 0.7rem; background-color: #d97706 !important; color: #f
| fffff !important;">CUSTOM</span>'
277 |         }
278 |     </div>
279 |     <div class="form-check form-switch m-0" title="В
| ключить / Выключить">
280 |         <input class="form-check-input agent-toggle-sw
| itch" type="checkbox" data-agent-id="{agent.id}" ${isEnabled
| ? 'checked' : ''}>
281 |     </div>
282 | </div>
283 | <div class="card-body p-3 d-flex flex-column">
284 |     <p class="small mb-3 flex-grow-1 agent-desc" sty
| le="min-height: 40px; color: #cbd5e1; line-height: 1.45;">${ag
| ent.description || 'Описание отсутствует'}</p>
285 |
286 |     <div class="agent-spec-box mb-3 small">
287 |         <div class="d-flex justify-content-between ali
| gn-items-center mb-1 pb-1" style="border-bottom: 1px solid rgb
| a(255, 255, 255, 0.08);">
288 |             <span class="spec-label">Провайдер &amp; Мод
| ель:</span>
289 |             <span class="badge spec-value-badge text-tru
| ncate" style="max-width: 190px;">${providerName}: ${agent.mode
| l}</span>
290 |         </div>
291 |         <div class="d-flex justify-content-between ali
| gn-items-center">
292 |             <span class="spec-label">Температура / Шаги:
| </span>
293 |             <span class="spec-tech-val font-monospace">T
| : ${agent.temperature} <span style="color: #64748b;">|</span>

```

```

Max: ${agent.max_steps}</span>
294     </div>
295     </div>
296
297     <div class="mb-3">
298         <div class="small fw-semibold mb-2" style="col
or: #f1f5f9;">Инструменты (${(agent.tools ?? []).length}):</di
v>
299         <div class="d-flex flex-wrap">${toolsBadges ||
'<span class="small" style="color: #94a3b8;">Без внешних инст
рументов</span>'}</div>
300     </div>
301
302     <div class="mt-auto d-flex justify-content-betwe
en gap-2 pt-2" style="border-top: 1px solid #334155;">
303         <button class="btn btn-sm rounded-pill px-3 bt
n-test-sandbox" data-agent-id="${agent.id}">
304             <i class="bi bi-play-fill"></i> Тест в Sandb
ox
305         </button>
306         <div class="d-flex gap-1">
307             <button class="btn btn-sm rounded-pill px-2
btn-edit-agent" data-agent-id="${agent.id}" title="Редактирова
ть">
308                 <i class="bi bi-pencil-fill"></i>
309             </button>
310             ${
311                 !isSystem
312                 ? `
313                 <button class="btn btn-sm rounded-pill px-
2 btn-delete-agent" data-agent-id="${agent.id}" title="Удалить
">
314                     <i class="bi bi-trash-fill"></i>
315                 </button>
316                 `
317                 : `
318                 `
319             </div>

```

```

320         </div>
321     </div>
322 </div>
323 </div>
324 `;
325 })
326 .join('');
327
328 // Attach card event listeners
329 grid.querySelectorAll('.agent-toggle-switch').forEach((sw) =
330 > {
331     sw.onchange = async () => {
332         const agentId = sw.getAttribute('data-agent-id');
333         if (agentId) await _toggleAgentState(agentId, sw.checked
334 );
335     };
336 });
337
338 grid.querySelectorAll('.btn-test-sandbox').forEach((btn) =>
339 {
340     btn.onclick = () => {
341         const agentId = btn.getAttribute('data-agent-id');
342         if (agentId) _openSandbox(agentId);
343     };
344 });
345
346 grid.querySelectorAll('.btn-edit-agent').forEach((btn) => {
347     btn.onclick = () => {
348         const agentId = btn.getAttribute('data-agent-id');
349         if (agentId) _openEditor(agentId);
350     };
351 });
352
353 grid.querySelectorAll('.btn-delete-agent').forEach((btn) =>
354 {
355     btn.onclick = () => {
356         const agentId = btn.getAttribute('data-agent-id');
357         if (agentId) _deleteAgent(agentId);

```

```

354     };
355   });
356 }
357
358 /**
359  * Toggles an agent's enabled state and persists it to the ser
    ver.
360  *
361  * @param {string} agentId - Target agent unique identifier.
362  * @param {boolean} enabled - New enabled state.
363  * @returns {Promise<void>}
364  */
365 async function _toggleAgentState(agentId, enabled) {
366   const agent = _state.agents.find((a) => a.id === agentId);
367   if (!agent) return;
368
369   agent.enabled = enabled;
370   try {
371     await _fetchJson(`/api/agents/${agentId}`, {
372       method: 'PUT',
373       headers: { 'Content-Type': 'application/json' },
374       body: JSON.stringify(agent)
375     });
376     _renderCounters();
377     _renderGrid();
378     _showToast(`Агент "${agent.name}" ${enabled ? 'включен' :
    'отключен'}`, 'success');
379   } catch (err) {
380     console.error('[AgentsTab] Error toggling agent:', err);
381     _showToast(`Ошибка: ${err.message}`, 'danger');
382     await _loadAllData();
383   }
384 }
385
386 /**
387  * Populates capability checkboxes inside the editor modal.
388  *
389  * @returns {void}

```

```

390 | */
391 | function _populateToolsMatrix() {
392 |     const container = document.getElementById('agent-tools-matri
    x');
393 |     if (!container) return;
394 |
395 |     container.innerHTML = _state.tools
396 |         .map(
397 |             (tool) => `
398 |                 <div class="col-md-6">
399 |                     <div class="p-3 rounded h-100" style="background: #1e2
    93b; border: 1px solid #334155;">
400 |                         <div class="form-check form-switch m-0">
401 |                             <input class="form-check-input tool-checkbox" type
    ="checkbox" value="${tool.id}" id="tool-chk-${tool.id}">
402 |                             <label class="form-check-label small text-white fw
    -bold" for="tool-chk-${tool.id}">
403 |                                 ${tool.icon} ${tool.name}
404 |                             </label>
405 |                         </div>
406 |                         <div class="small mt-1" style="color: #cbd5e1; font-
    size: 0.8rem; line-height: 1.4;">${tool.description}</div>
407 |                     </div>
408 |                 </div>
409 |             `
410 |         )
411 |         .join('');
412 |
413 |     container.querySelectorAll('.tool-checkbox').forEach((chk) =
    > {
414 |         chk.onchange = _updateSelectedToolsCounter;
415 |     });
416 | }
417 |
418 | /**
419 |  * Updates selected capabilities counter badge in the editor m
    odal.
420 |  *

```

```

421 | * @returns {void}
422 | */
423 | function _updateSelectedToolsCounter() {
424 |     const count = document.querySelectorAll('#agent-tools-matrix
    .tool-checkbox:checked').length;
425 |     const badge = document.getElementById('agent-tools-count');
426 |     if (badge) badge.textContent = `${count} выбрано`;
427 | }
428 |
429 | /**
430 |  * Updates model selection dropdown options when provider sele
    ction changes.
431 |  *
432 |  * @param {string} providerSelectId - ID of provider select el
    ement.
433 |  * @param {string} modelSelectId - ID of model select element.
434 |  * @param {string} [selectedModel=''] - Model ID to select by
    default.
435 |  * @returns {void}
436 |  */
437 | function _updateModelDropdown(providerSelectId, modelSelectId,
    selectedModel = '') {
438 |     const pSel = document.getElementById(providerSelectId);
439 |     const mSel = document.getElementById(modelSelectId);
440 |     if (!pSel || !mSel) return;
441 |
442 |     const prov = _state.providers[pSel.value];
443 |     const models = prov?.models ?? [];
444 |
445 |     mSel.innerHTML = models.map((m) => `<option value="${m.id}">
    ${m.name || m.id}</option>`).join('');
446 |
447 |     if (selectedModel && models.some((m) => m.id === selectedMod
    el)) {
448 |         mSel.value = selectedModel;
449 |     } else if (prov?.default_model) {
450 |         mSel.value = prov.default_model;
451 |     }

```

```

452 }
453
454 /**
455  * Opens agent editor modal for creating or updating an agent.
456  *
457  * @param {string|null} [agentId=null] - Agent ID to edit, or
  null for create.
458  * @param {object|null} [prefillData=null] - Pre-filled data f
  rom AI builder.
459  * @returns {void}
460  */
461 function _openEditor(agentId = null, prefillData = null) {
462   const modalEl = document.getElementById('modal-agent-editor'
  );
463   if (!modalEl) return;
464
465   const modeInput = document.getElementById('agent-edit-mode')
  ;
466   const idInput = document.getElementById('agent-id');
467   const nameInput = document.getElementById('agent-name');
468   const descInput = document.getElementById('agent-desc');
469   const enabledChk = document.getElementById('agent-enabled');
470   const providerSel = document.getElementById('agent-provider'
  );
471   const tempSlider = document.getElementById('agent-temperatur
  e');
472   const tempVal = document.getElementById('agent-temp-val');
473   const maxStepsInput = document.getElementById('agent-max-ste
  ps');
474   const timeoutInput = document.getElementById('agent-timeout'
  );
475   const promptTa = document.getElementById('agent-system-promp
  t');
476
477   // Reset tools checkboxes
478   document.querySelectorAll('#agent-tools-matrix .tool-checkbo
  x').forEach((chk) => {
479     chk.checked = false;

```



```
480 | });
481 |
482 | if (agentId) {
483 |     const agent = _state.agents.find((a) => a.id === agentId);
484 |     if (!agent) return;
485 |
486 |     if (modeInput) modeInput.value = 'edit';
487 |     if (idInput) {
488 |         idInput.value = agent.id;
489 |         idInput.disabled = true;
490 |     }
491 |     if (nameInput) nameInput.value = agent.name;
492 |     if (descInput) descInput.value = agent.description ?? '';
493 |     if (enabledChk) enabledChk.checked = Boolean(agent.enabled
494 | );
495 |     if (providerSel) providerSel.value = agent.provider ?? 'ge
496 | mini';
497 |
498 |     _updateModelDropdown('agent-provider', 'agent-model', agen
499 | t.model);
500 |
501 |     if (tempSlider) {
502 |         tempSlider.value = agent.temperature ?? 0.3;
503 |         if (tempVal) tempVal.textContent = tempSlider.value;
504 |     }
505 |     if (maxStepsInput) maxStepsInput.value = agent.max_steps ?
506 | ? 15;
507 |     if (timeoutInput) timeoutInput.value = agent.timeout_secon
508 | ds ?? 60;
509 |     if (promptTa) promptTa.value = agent.system_prompt ?? '';
510 |
511 |     (agent.tools ?? []).forEach((tId) => {
512 |         const chk = document.getElementById(`tool-chk-${tId}`);
513 |         if (chk) chk.checked = true;
514 |     });
515 | } else if (prefillData) {
516 |     if (modeInput) modeInput.value = 'create';
517 |     if (idInput) {
```

```
513 |         idInput.value = (prefillData.name || 'agent').toLowerCase().replace(/^[^a-z0-9_]/g, '_').slice(0, 25);
514 |         idInput.disabled = false;
515 |     }
516 |     if (nameInput) nameInput.value = prefillData.name ?? 'Новый ИИ-Агент';
517 |     if (descInput) descInput.value = prefillData.description ?? '';
518 |     if (enabledChk) enabledChk.checked = true;
519 |     if (providerSel) providerSel.value = prefillData.provider ?? 'gemini';
520 |
521 |     _updateModelDropdown('agent-provider', 'agent-model', prefillData.model);
522 |
523 |     if (tempSlider) {
524 |         tempSlider.value = prefillData.temperature ?? 0.3;
525 |         if (tempVal) tempVal.textContent = tempSlider.value;
526 |     }
527 |     if (maxStepsInput) maxStepsInput.value = prefillData.max_steps ?? 15;
528 |     if (timeoutInput) timeoutInput.value = 60;
529 |     if (promptTa) promptTa.value = prefillData.system_prompt ?? '';
530 |
531 |     (prefillData.recommended_tools ?? []).forEach((tId) => {
532 |         const chk = document.getElementById(`tool-chk-${tId}`);
533 |         if (chk) chk.checked = true;
534 |     });
535 | } else {
536 |     if (modeInput) modeInput.value = 'create';
537 |     if (idInput) {
538 |         idInput.value = '';
539 |         idInput.disabled = false;
540 |     }
541 |     if (nameInput) nameInput.value = '';
542 |     if (descInput) descInput.value = '';
543 |     if (enabledChk) enabledChk.checked = true;
```

```

544     if (providerSel) providerSel.value = 'gemini';
545
546     _updateModelDropdown('agent-provider', 'agent-model');
547
548     if (tempSlider) {
549         tempSlider.value = 0.3;
550         if (tempVal) tempVal.textContent = '0.3';
551     }
552     if (maxStepsInput) maxStepsInput.value = 15;
553     if (timeoutInput) timeoutInput.value = 60;
554     if (promptTa) promptTa.value = '';
555 }
556
557 _updateSelectedToolsCounter();
558 new bootstrap.Modal(modalEl).show();
559 }
560
561 /**
562  * Saves agent configuration (POST create or PUT update).
563  *
564  * @returns {Promise<void>}
565  */
566 async function _handleSaveAgent() {
567     const mode = document.getElementById('agent-edit-mode')?.value ?? 'create';
568     const id = document.getElementById('agent-id')?.value.trim();
569     const name = document.getElementById('agent-name')?.value.trim();
570     const desc = document.getElementById('agent-desc')?.value.trim();
571     const enabled = document.getElementById('agent-enabled')?.checked ?? true;
572     const provider = document.getElementById('agent-provider')?.value ?? 'gemini';
573     const model = document.getElementById('agent-model')?.value ?? 'gemini-2.5-flash';
574     const temperature = parseFloat(document.getElementById('agen

```

```
t-temperature')?.value ?? '0.3');
575 |   const max_steps = parseInt(document.getElementById('agent-max-steps')?.value ?? '15', 10);
576 |   const timeout_seconds = parseInt(document.getElementById('agent-timeout')?.value ?? '60', 10);
577 |   const system_prompt = document.getElementById('agent-system-prompt')?.value.trim() ?? '';
578 |
579 |   if (!id || !name) {
580 |     _showToast('ID и Название агента обязательны для заполнения', 'warning');
581 |     return;
582 |   }
583 |
584 |   const selectedTools = [];
585 |   document.querySelectorAll('#agent-tools-matrix .tool-checkbox:checked').forEach((chk) => {
586 |     selectedTools.push(chk.value);
587 |   });
588 |
589 |   const payload = {
590 |     id,
591 |     name,
592 |     description: desc,
593 |     enabled,
594 |     provider,
595 |     model,
596 |     temperature,
597 |     max_steps,
598 |     timeout_seconds,
599 |     tools: selectedTools,
600 |     system_prompt
601 |   };
602 |
603 |   const saveBtn = document.getElementById('btn-save-agent');
604 |   if (saveBtn) saveBtn.disabled = true;
605 |
606 |   try {
```

```

607     if (mode === 'create') {
608         await _fetchJson('/api/agents', {
609             method: 'POST',
610             headers: { 'Content-Type': 'application/json' },
611             body: JSON.stringify(payload)
612         });
613         _showToast(`Агент "${name}" успешно создан!`, 'success')
        ;
614     } else {
615         await _fetchJson(`/api/agents/${id}`, {
616             method: 'PUT',
617             headers: { 'Content-Type': 'application/json' },
618             body: JSON.stringify(payload)
619         });
620         _showToast(`Агент "${name}" успешно обновлен!`, 'success
        ');
621     }
622
623     const modalEl = document.getElementById('modal-agent-edito
        r');
624     bootstrap.Modal.getInstance(modalEl)?.hide();
625     await _loadAllData();
626 } catch (err) {
627     console.error('[AgentsTab] Error saving agent:', err);
628     _showToast(`Ошибка сохранения: ${err.message}`, 'danger');
629 } finally {
630     if (saveBtn) saveBtn.disabled = false;
631 }
632 }
633
634 /**
635  * Deletes an existing custom agent after confirmation.
636  *
637  * @param {string} agentId - Target agent unique identifier.
638  * @returns {Promise<void>}
639  */
640 async function _deleteAgent(agentId) {
641     const agent = _state.agents.find((a) => a.id === agentId);

```

```

642 |   if (!agent) return;
643 |
644 |   if (!confirm(`Вы действительно хотите удалить агента "${agent.name}" (${agent.id})?`)) {
645 |       return;
646 |   }
647 |
648 |   try {
649 |       await _fetchJson(`/api/agents/${agentId}`, { method: 'DELETE' });
650 |       _showToast(`Агент "${agent.name}" удален`, 'success');
651 |       await _loadAllData();
652 |   } catch (err) {
653 |       console.error('[AgentsTab] Error deleting agent:', err);
654 |       _showToast(`Ошибка удаления: ${err.message}`, 'danger');
655 |   }
656 | }
657 |
658 | /**
659 |  * Opens AI prompt architect builder modal.
660 |  *
661 |  * @returns {void}
662 |  */
663 | function _openAiBuilder() {
664 |     const modalEl = document.getElementById('modal-ai-builder');
665 |     if (!modalEl) return;
666 |
667 |     const promptInput = document.getElementById('ai-builder-prompt');
668 |     const resultBox = document.getElementById('ai-builder-result');
669 |     if (promptInput) promptInput.value = '';
670 |     if (resultBox) resultBox.classList.add('d-none');
671 |     _state.lastGeneratedSpec = null;
672 |
673 |     new bootstrap.Modal(modalEl).show();
674 | }
675 |

```

```

676 /**
677  * Generates an agent specification using selected LLM archite
    ct model.
678  *
679  * @returns {Promise<void>}
680  */
681 async function _handleRunAiGenerate() {
682     const taskDesc = document.getElementById('ai-builder-prompt'
    )?.value.trim();
683     const provider = document.getElementById('ai-builder-provide
    r')?.value ?? 'gemini';
684     const model = document.getElementById('ai-builder-model')?.v
    alue ?? 'gemini-2.5-flash';
685
686     if (!taskDesc) {
687         _showToast('Пожалуйста, опишите задачу агента', 'warning')
    ;
688         return;
689     }
690
691     const runBtn = document.getElementById('btn-run-ai-generate'
    );
692     if (runBtn) {
693         runBtn.disabled = true;
694         runBtn.innerHTML = '<span class="spinner-border spinner-bo
    rder-sm me-2"></span>Анализ задачи и генерация...';
695     }
696
697     try {
698         const res = await _fetchJson('/api/agents/generate-prompt'
    , {
699             method: 'POST',
700             headers: { 'Content-Type': 'application/json' },
701             body: JSON.stringify({ task_description: taskDesc, provi
    der, model })
702         });
703
704         if (res?.data) {

```

```

705 |     _state.lastGeneratedSpec = { ...res.data, provider, mode
    | 1 };
706 |     const resultBox = document.getElementById('ai-builder-re
    | sult');
707 |     const previewBox = document.getElementById('ai-builder-p
    | review-box');
708 |     if (resultBox && previewBox) {
709 |         resultBox.classList.remove('d-none');
710 |         previewBox.innerHTML = `
711 |             <div><strong style="color: #38bdf8;">Название:</stro
    | ng> <span class="text-white fw-bold">${res.data.name}</span></
    | div>
712 |             <div class="mt-1"><strong style="color: #cbd5e1;">Оп
    | исание:</strong> <span style="color: #f1f5f9;">${res.data.desc
    | ription}</span></div>
713 |             <div class="mt-1"><strong style="color: #4ade80;">Ин
    | струменты:</strong> <span style="color: #f8fafc;">${(res.data.
    | recommended_tools ?? []).join(', ') || 'нет'}</span></div>
714 |             <div class="mt-2 p-2 rounded" style="background: #0f
    | 172a; border: 1px solid #334155; color: #f8fafc; white-space:
    | pre-wrap; font-size: 0.85rem; line-height: 1.45;">${res.data.s
    | ystem_prompt}</div>
715 |         `;
716 |     }
717 |     _showToast('Спецификация агента успешно сгенерирована!',
    | 'success');
718 | }
719 | } catch (err) {
720 |     console.error('[AgentsTab] AI Generator error:', err);
721 |     _showToast(`Ошибка генерации: ${err.message}`, 'danger');
722 | } finally {
723 |     if (runBtn) {
724 |         runBtn.disabled = false;
725 |         runBtn.innerHTML = '<i class="bi bi-stars me-1"></i> Cre
    | нерировать спецификацию';
726 |     }
727 | }
728 | }

```



```
729 |
730 | /**
731 |  * Opens sandbox modal for interactive agent testing.
732 |  *
733 |  * @param {string} agentId - Target agent unique identifier.
734 |  * @returns {void}
735 |  */
736 | function _openSandbox(agentId) {
737 |     const agent = _state.agents.find((a) => a.id === agentId);
738 |     if (!agent) return;
739 |
740 |     _state.activeSandboxAgent = agent;
741 |     const modalEl = document.getElementById('modal-agent-sandbox
742 | ');
743 |     if (!modalEl) return;
744 |
745 |     document.getElementById('sandbox-agent-name')?.replaceChildr
746 | en(document.createTextNode(agent.name));
747 |     const modelBadge = document.getElementById('sandbox-agent-mo
748 | del');
749 |     if (modelBadge) modelBadge.textContent = `${_getProviderDisp
750 | layName(agent.provider)}: ${agent.model}`;
751 |
752 |     const toolsSummary = document.getElementById('sandbox-agent-
753 | tools-summary');
754 |     if (toolsSummary) toolsSummary.textContent = `Инструментов:
755 | ${((agent.tools ?? []).length)} `;
756 |
757 |     const msgsList = document.getElementById('sandbox-messages-l
758 | ist');
759 |     const placeholder = document.getElementById('sandbox-placeho
760 | lder');
761 |     if (msgsList) msgsList.innerHTML = '';
762 |     if (placeholder) placeholder.style.display = 'block';
763 |
764 |     new bootstrap.Modal(modalEl).show();
765 | }
```

```

759 /**
760  * Executes a test query against the active sandbox agent and
    renders execution trace.
761  *
762  * @returns {Promise<void>}
763  */
764 async function _handleRunSandboxTest() {
765     const agent = _state.activeSandboxAgent;
766     const input = document.getElementById('sandbox-input-msg');
767     const sendBtn = document.getElementById('btn-sandbox-send');
768     const msgsList = document.getElementById('sandbox-messages-l
ist');
769     const placeholder = document.getElementById('sandbox-placeho
lder');
770
771     if (!agent || !input || !msgsList) return;
772     const query = input.value.trim();
773     if (!query) return;
774
775     if (placeholder) placeholder.style.display = 'none';
776
777     // Append user message
778     const userMsgEl = document.createElement('div');
779     userMsgEl.className = 'd-flex justify-content-end mb-3';
780     userMsgEl.innerHTML = `
781         <div class="p-2 px-3 rounded shadow-sm text-white small" s
tyle="background-color: #2563eb; max-width: 80%; font-weight:
500;">
782             <strong>Вы:</strong> ${query}
783         </div>
784     `;
785     msgsList.appendChild(userMsgEl);
786
787     input.value = '';
788     if (sendBtn) sendBtn.disabled = true;
789
790     // Append response placeholder with loader
791     const botMsgEl = document.createElement('div');

```

```

792 | botMsgEl.className = 'd-flex flex-column mb-3';
793 | botMsgEl.innerHTML = `
794 |     <div class="p-3 rounded small" style="background: #0f172a;
      |     border: 1px solid #334155; color: #f8fafc;">
795 |         <div class="d-flex align-items-center gap-2 mb-2" style=
      |         "color: #fbbf24; font-weight: 600;">
796 |             <span class="spinner-border spinner-border-sm"></span>
797 |             <span>Агент выполняет рассуждение и вызовы инструменто
      | в...</span>
798 |         </div>
799 |         <div class="steps-trace small font-monospace" style="col
      | or: #cbd5e1;"></div>
800 |     </div>
801 | `;
802 | msgsList.appendChild(botMsgEl);
803 | msgsList.scrollTop = msgsList.scrollHeight;
804 |
805 | try {
806 |     const res = await _fetchJson('/api/agents/test', {
807 |         method: 'POST',
808 |         headers: { 'Content-Type': 'application/json' },
809 |         body: JSON.stringify({ agent_id: agent.id, test_message:
      | query })
810 |     });
811 |
812 |     const stepsHtml = (res.steps ?? [])
813 |         .map((s) => {
814 |             let icon = '❏';
815 |             if (s.type === 'tool_init') icon = '🔧';
816 |             else if (s.type === 'action') icon = '👉';
817 |             else if (s.type === 'finish') icon = '✅';
818 |             else if (s.type === 'error') icon = '❌';
819 |
820 |             return `<div class="mb-1">${icon} <span style="color:
      | #38bdf8; font-weight: bold;">[Шаг ${s.step}]</span> <span styl
      | e="color: #e2e8f0;">${s.content}</span></div>`;
821 |         })
822 |         .join('');

```

```

823
824     botMsgEl.innerHTML = `
825         <div class="p-3 rounded small" style="background: #0f172
a; border: 1px solid #334155; color: #f8fafc;">
826             <div class="d-flex justify-content-between align-items
-center mb-2 pb-1" style="border-bottom: 1px solid #334155;">
827                 <strong style="color: #38bdf8; font-size: 0.95rem;">
❑  ${agent.name}</strong>
828                 <span class="badge" style="background: #334155; colo
r: #f8fafc; font-weight: 600;">${res.duration_ms ?? 0} мс</spa
n>
829             </div>
830
831             <div class="mb-2" style="white-space: pre-wrap; color:
#f8fafc; line-height: 1.5;">${res.response || 'Пустой ответ'}
</div>
832
833             <div class="p-2 rounded mt-2 small font-monospace" sty
le="background: #020617; border: 1px solid #334155; font-size:
0.8rem; line-height: 1.45;">
834                 <div class="fw-bold mb-1" style="color: #cbd5e1;">Тр
ассировка выполнения (ReAct Trace):</div>
835                 ${stepsHtml}
836             </div>
837         </div>
838     `;
839 } catch (err) {
840     botMsgEl.innerHTML = `
841         <div class="p-3 rounded small" style="background: #450a0
a; border: 1px solid #dc2626; color: #fecaca;">
842             <strong style="color: #ef4444;">Ошибка выполнения:</st
rong> ${err.message}
843         </div>
844     `;
845 } finally {
846     if (sendBtn) sendBtn.disabled = false;
847     const chatWin = document.getElementById('sandbox-chat-wind
ow');

```

```

848     if (chatWin) chatWin.scrollTop = chatWin.scrollHeight;
849 }
850 }
851
852 /**
853  * Returns human-readable display name for provider identifier
854  *
855  * @param {string} provKey - Provider key (e.g. 'gemini', 'agy
856  *   ', 'foundry', 'ollama').
857  * @returns {string} Provider display label.
858  */
859 function _getProviderDisplayName(provKey) {
860     return _state.providers[provKey]?.name ?? provKey;
861 }
862
863 /**
864  * Displays floating feedback notification toast.
865  *
866  * @param {string} message - Notification text message.
867  * @param {string} [type='info'] - Bootstrap color variant ('i
868  *   nfo', 'success', 'warning', 'danger').
869  * @returns {void}
870  */
871 function _showToast(message, type = 'info') {
872     if (typeof window.showModelsNotification === 'function') {
873         window.showModelsNotification(message, type);
874         return;
875     }
876     const alertDiv = document.createElement('div');
877     alertDiv.className = `alert alert-${type} position-fixed bot
878     tom-0 end-0 m-3 shadow`;
879     alertDiv.style.zIndex = '9999';
880     alertDiv.textContent = message;
881     document.body.appendChild(alertDiv);
882     setTimeout(() => alertDiv.remove(), 3500);
883 }
884
885

```

index.html

```
1 <div class="chat-window mb-2" id="chat-window"></div>
2
3 <!-- Media Controls & Collapsible Player Panel -->
4 <div class="media-controls mb-2" id="media-controls">
5   <style>
6     #media-controls .card {
7       background: var(--card-bg);
8       border: 1px solid var(--border-color);
9       border-radius: 8px;
10    }
11    #media-player-body {
12      background: rgba(0, 0, 0, 0.2);
13      border-top: 1px solid var(--border-color);
14    }
15    .player-window-wrapper {
16      position: relative;
17      width: 100%;
18      height: 340px;
19      background: #000;
20      border-radius: 8px;
21      overflow: hidden;
22      margin-top: 10px;
23      border: 1px solid var(--border-color);
24    }
25    .player-glow {
26      position: absolute;
27      top: -20%;
28      left: -20%;
29      width: 140%;
30      height: 140%;
31      background: radial-gradient(circle, rgba(121, 40, 202, 0
32      .15) 0%, transparent 60%);
33      filter: blur(40px);
34      z-index: 0;
35      pointer-events: none;
36      transition: background 1s ease;
```

```
36 |     }
37 |     .player-el {
38 |         position: relative;
39 |         width: 100%;
40 |         height: 100%;
41 |         border: none;
42 |         z-index: 2;
43 |     }
44 |     .player-video-tag {
45 |         width: 100%;
46 |         height: 100%;
47 |         object-fit: contain;
48 |     }
49 |     .hidden {
50 |         display: none !important;
51 |     }
52 |     /* Controls style overlay */
53 |     .controls-overlay {
54 |         position: absolute;
55 |         bottom: 0;
56 |         left: 0;
57 |         right: 0;
58 |         background: linear-gradient(to top, rgba(0,0,0,0.85) 0%,
59 | rgba(0,0,0,0.3) 75%, transparent 100%);
60 |         padding: 10px 15px;
61 |         display: flex;
62 |         flex-direction: column;
63 |         gap: 6px;
64 |         opacity: 0;
65 |         transition: opacity 0.25s;
66 |         z-index: 10;
67 |     }
68 |     .player-window-wrapper:hover .controls-overlay {
69 |         opacity: 1;
70 |     }
71 |     .p-bar-track {
72 |         height: 4px;
73 |         background: rgba(255,255,255,0.25);
```

```
73 |     border-radius: 2px;
74 |     cursor: pointer;
75 |     position: relative;
76 | }
77 | .p-bar-fill {
78 |     height: 100%;
79 |     background: linear-gradient(135deg, #7928ca 0%, #ff007a
100 | 100%);
80 |     width: 0%;
81 |     border-radius: 2px;
82 | }
83 | .p-bar-hover {
84 |     position: absolute;
85 |     top: 0;
86 |     left: 0;
87 |     height: 100%;
88 |     background: rgba(255,255,255,0.15);
89 |     width: 0%;
90 |     pointer-events: none;
91 | }
92 | .row-flex {
93 |     display: flex;
94 |     justify-content: space-between;
95 |     align-items: center;
96 | }
97 | .row-side {
98 |     display: flex;
99 |     align-items: center;
100 |     gap: 10px;
101 | }
102 | .p-btn {
103 |     background: none;
104 |     border: none;
105 |     color: #fff;
106 |     cursor: pointer;
107 |     font-size: 0.95rem;
108 | }
109 | .vol-slider {
```



```

110 |     width: 50px;
111 |     height: 3px;
112 |     accent-color: #ff007a;
113 |     cursor: pointer;
114 | }
115 | .speed-select-box {
116 |     background: rgba(255,255,255,0.15);
117 |     border: 1px solid rgba(255,255,255,0.2);
118 |     color: #fff;
119 |     font-size: 0.75rem;
120 |     border-radius: 4px;
121 |     padding: 2px 4px;
122 | }
123 | .player-placeholder-box {
124 |     position: absolute;
125 |     top: 0;
126 |     left: 0;
127 |     width: 100%;
128 |     height: 100%;
129 |     display: flex;
130 |     flex-direction: column;
131 |     justify-content: center;
132 |     align-items: center;
133 |     background: rgba(20,20,30,0.95);
134 |     color: #8b949e;
135 |     z-index: 1;
136 |     text-align: center;
137 |     padding: 20px;
138 | }
139 | </style>
140 | <div class="card">
141 |     <!-- Header -->
142 |     <div class="card-header p-2 d-flex align-items-center justify-content-between" style="cursor: pointer; background: transparent; border-bottom: none;" id="media-controls-header">
143 |         <div class="d-flex align-items-center gap-2">
144 |             <span class="text-muted small">▶ Сейчас играет:</span
>

```

```

145 |         <strong id="now-playing-title" class="small text-info"
    | >—</strong>
146 |     </div>
147 |     <div class="d-flex gap-2">
148 |         <button class="btn btn-sm btn-outline-primary" id="btn
    | -toggle-player-body">Развернуть плеер</button>
149 |         <button class="btn btn-sm btn-outline-secondary" id="b
    | tn-close-player">✕ </button>
150 |     </div>
151 | </div>
152 |
153 | <!-- Collapsible Player and Inputs Container -->
154 | <div class="card-body p-3 d-none" id="media-player-body">
155 |     <div class="row g-3">
156 |         <!-- Left: URL Loader, Badges, History -->
157 |         <div class="col-md-4 d-flex flex-column gap-2" style="
    | border-right: 1px solid var(--border-color); padding-right: 15
    | px;">
158 |             <div>
159 |                 <h6 class="small text-uppercase text-muted fw-bold
    | ">Вставить ссылку</h6>
160 |                 <form id="cosmic-stream-form" class="d-flex gap-1"
    | >
161 |                     <input type="url" id="cosmic-stream-url" class="
    | form-control form-control-sm bg-dark text-white border-seconda
    | ry" placeholder="YouTube, VK, RuTube, Rezkа, direct..." requir
    | ed>
162 |                     <button type="submit" class="btn btn-sm btn-prim
    | ary">Запуск</button>
163 |                 </form>
164 |             </div>
165 |
166 |             <div>
167 |                 <h6 class="small text-uppercase text-muted fw-bold
    | mt-1">Демо-источники</h6>
168 |                 <div class="d-flex flex-wrap gap-1" id="cosmic-pro
    | vider-badges">
169 |                     <span class="badge bg-secondary text-white curso

```

```

    r-pointer" style="cursor:pointer" data-prov="youtube">YouTube<
    /span>
170         <span class="badge bg-secondary text-white curso
    r-pointer" style="cursor:pointer" data-prov="vk">VK</span>
171         <span class="badge bg-secondary text-white curso
    r-pointer" style="cursor:pointer" data-prov="rutube">RuTube</s
    pan>
172         <span class="badge bg-secondary text-white curso
    r-pointer" style="cursor:pointer" data-prov="direct">HLS (.m3u
    8)</span>
173     </div>
174 </div>
175
176     <div class="flex-grow-1" style="overflow-y:auto; max
    -height:120px;">
177         <h6 class="small text-uppercase text-muted fw-bold
    mt-1">История</h6>
178         <div id="cosmic-history-list" class="d-flex flex-c
    olumn gap-1">
179             <div class="text-muted small text-center py-2">И
    стория пуста</div>
180         </div>
181     </div>
182 </div>
183
184 <!-- Right: Player Screen -->
185 <div class="col-md-8">
186     <div class="player-window-wrapper" id="cosmic-screen
    -container">
187         <!-- Glow background -->
188         <div class="player-glow" id="cosmic-ambient-glow">
189
190         <!-- Embedded frame -->
191         <iframe id="cosmic-embed-player" class="player-el
    hidden" allowfullscreen allow="autoplay; encrypted-media"></if
    rame>
192

```

```

193 |         <!-- HTML5 Video Tag -->
194 |         <div id="cosmic-video-wrapper" class="player-el hi
| dden">
195 |             <video id="cosmic-video-element" class="player-v
| ideo-tag"></video>
196 |
197 |             <!-- Custom controls overlay -->
198 |             <div class="controls-overlay">
199 |                 <div class="p-bar-track" id="cosmic-progress-c
| ontainer">
200 |                     <div class="p-bar-fill" id="cosmic-progress-
| bar">
201 |                         <div class="p-bar-hover"></div>
202 |                     </div>
203 |                 </div>
204 |                 <div class="row-flex">
205 |                     <div class="row-side">
206 |                         <button class="p-btn" id="cosmic-play-btn"
| >▶ </button>
207 |                         <button class="p-btn" id="cosmic-stop-btn"
| >■ </button>
208 |                         <span class="time-display small" style="fo
| nt-size: 0.75rem;"><span id="cosmic-time-current">0:00</span>
| / <span id="cosmic-time-duration">0:00</span></span>
209 |                     </div>
210 |                     <div class="row-side">
211 |                         <div class="d-flex align-items-center gap-
| 1">
212 |                             <button class="p-btn" id="cosmic-mute-bt
| n">🔊 </button>
213 |                             <input type="range" id="cosmic-volume-sl
| ider" class="vol-slider" min="0" max="1" step="0.05" value="1"
| >
214 |                         </div>
215 |                         <select id="cosmic-speed-select" class="sp
| eed-select-box">
216 |                             <option value="0.5">0.5x</option>
217 |                             <option value="1" selected>1.0x</option>

```

```

218         <option value="1.5">1.5x</option>
219         <option value="2">2.0x</option>
220     </select>
221     <button class="p-btn" id="cosmic-fullscree
n-btn">⏮ </button>
222         </div>
223     </div>
224 </div>
225 </div>
226
227     <!-- Empty Placeholder -->
228     <div id="cosmic-player-placeholder" class="player-
placeholder-box">
229         <span style="font-size: 2.5rem;">⏮ </span>
230         <h6 class="mt-2 mb-1">Плеер готов к работе</h6>
231         <p class="small text-muted mb-0" style="font-siz
e: 0.8rem;">Вставьте ссылку слева или нажмите на любой демо-ис
точник</p>
232     </div>
233 </div>
234 </div>
235 </div>
236 </div>
237 </div>
238 </div>
239
240 <!-- AI Model & Search Debugger Toolbar -->
241 <div class="card mb-2 border-secondary shadow-sm" id="chat-deb
ugger-toolbar" style="background: var(--bg-color-card, #1c2128
); border-radius: 8px;">
242     <div class="card-body p-2 d-flex flex-wrap align-items-cente
r justify-content-between gap-2" style="font-size: 0.82rem;">
243         <div class="d-flex flex-wrap align-items-center gap-2">
244             <!-- Section: Model (Internal Processing) -->
245             <div class="d-flex align-items-center gap-1">
246                 <span class="text-info fw-bold d-flex align-items-cent
er gap-1">
247                     <i class="bi bi-cpu-fill"></i> Модель:

```

```

248         </span>
249         <select class="form-select form-select-sm bg-dark text
- white border-secondary font-monospace" id="chat-model-select"
- style="width: auto; min-width: 230px; font-size: 0.8rem;" tit
- le="Иерархический выбор модели ИИ для внутренней генерации">
250             <option value="">Загрузка моделей...</option>
251         </select>
252         <button class="btn btn-sm btn-outline-warning d-flex a
- lign-items-center gap-1 fw-semibold" id="btn-chat-set-default-
- model" title="Установить выбранную модель ИИ как дефолтную для
- пользователя">
253             <i class="bi bi-star-fill"></i> Дефолтная
254         </button>
255     </div>
256
257     <span class="text-secondary opacity-50 d-none d-lg-inlin
- e">|</span>
258
259     <!-- Section: Web Search Provider & Model -->
260     <div class="d-flex align-items-center gap-1">
261         <span class="text-info fw-bold d-flex align-items-cent
- er gap-1">
262             <i class="bi bi-globe2"></i> Поиск:
263         </span>
264         <select class="form-select form-select-sm bg-dark text
- white border-secondary font-monospace" id="chat-search-select
- " style="width: auto; min-width: 175px; font-size: 0.8rem;" ti
- tle="Иерархический выбор провайдера и модели веб-поиска">
265             <optgroup label="🌐 Поисковые провайдеры">
266                 <option value="gemini_cli">🌐 Gemini CLI</option>
267                 <option value="gemini">🌐 Gemini Grounding</option>
268                 <option value="agy">🌐 Antigravity (AGY)</option>
269                 <option value="playwright">🌐 Playwright MCP</opti
- on>
270                 <option value="langchain">🌐 LangChain MCP</option
- >
271             </optgroup>
272         </select>

```

```
273 |         <button class="btn btn-sm btn-outline-info d-flex align-items-center gap-1 fw-semibold" id="btn-chat-set-default-search" title="Установить выбранный провайдер и модель поиска по умолчанию в config.json">
274 |             <i class="bi bi-star-fill"></i> Дефолтный
275 |         </button>
276 |     </div>
277 | </div>
278 |
279 | <!-- Live Status Badges on the right -->
280 | <div class="d-flex align-items-center gap-1 ms-auto">
281 |     <span class="badge bg-dark border border-secondary font-monospace" id="chat-model-badge" style="color: #58a6ff; font-size: 0.75rem; display: none;" title="Текущая активная модель">
282 |         <span class="badge bg-dark border border-secondary font-monospace" id="chat-search-badge" style="color: #38bdf8; font-size: 0.75rem; display: none;" title="Текущий провайдер поиска"></span>
283 |     </div>
284 | </div>
285 | </div>
286 |
287 | <div class="input-group">
288 |     <select class="form-select text-muted" id="chat-mode-selector" style="max-width: 170px; font-size: 0.85rem; border-color: var(--border-color); background-color: var(--bg-color-card); color: var(--text-color);">
289 |         <option value="story" selected>📖 Сюжет</option>
290 |         <option value="download">📄 Найти и скачать</option>
291 |     </select>
292 |     <select class="form-select text-muted" id="chat-mode-debug" style="max-width: 100px; font-size: 0.85rem; border-color: var(--border-color); background-color: var(--bg-color-card); color: var(--text-color);">
293 |         <option value="prod" selected>PROD</option>
294 |         <option value="debug">DEBUG</option>
295 |     </select>
```

```

296   <input type="text" class="form-control" id="message-input" d
    ata-i18n-placeholder="chat.placeholder" placeholder="Введите с
    ообщение...">
297   <button class="btn btn-primary" id="send-button" data-i18n="
    chat.send">Отправить</button>
298   <button class="btn btn-outline-info" id="chat-algo-btn" oncl
    ick="window.showChatLogicModal ? window.showChatLogicModal() :
    null" title="📊 Схема и алгоритм логики ответов чата" style="
    padding: 0.375rem 0.65rem; font-size: 1rem;"><i class="bi bi-d
    iagram-3-fill"></i></button>
299   <button class="btn btn-outline-success" id="save-history-rag
    -btn" title="Сохранить всю историю в RAG" style="padding: 0.37
    5rem 0.65rem; font-size: 1rem;">💾 </button>
300   <button class="btn btn-outline-secondary" id="clear-history-
    btn" title="Очистить историю чата" style="padding: 0.375rem 0.
    65rem; font-size: 1rem;">🗑️ </button>
301 </div>
302
303 <!-- Modal for editing and saving chat history to RAG -->
304 <div class="modal fade" id="saveHistoryModal" tabindex="-1" ar
    ia-hidden="true" style="z-index: 1060;">
305   <div class="modal-dialog modal-lg">
306     <div class="modal-content" style="background: var(--bg-col
    or-card, #2b3035); color: var(--text-color, #f8f9fa); border:
    1px solid var(--border-color, #495057);">
307       <div class="modal-header" style="border-bottom: 1px soli
    d var(--border-color, #495057);">
308         <h5 class="modal-title">💾 Сохранение истории чата в R
    AG</h5>
309         <button type="button" class="btn-close btn-close-white
    " data-bs-dismiss="modal" aria-label="Close"></button>
310       </div>
311       <div class="modal-body">
312         <div class="mb-3">
313           <label class="form-label small fw-bold">Тема / Запро
    с (заголовок документа):</label>
314           <input type="text" class="form-control bg-dark text-
    white border-secondary" id="saveHistoryTitleInput">

```



```
315 |         </div>
316 |         <div class="mb-3">
317 |             <label class="form-label small fw-bold">Содержимое диалога (отредактируйте, удалив всё ненужное):</label>
318 |             <textarea class="form-control bg-dark text-white border-secondary font-monospace small" id="saveHistoryTextInput" rows="12"></textarea>
319 |         </div>
320 |         <div class="mb-3">
321 |             <label class="form-label small fw-bold">Описание для диктора (voice_text):</label>
322 |             <textarea class="form-control bg-dark text-white border-secondary font-monospace small" id="saveHistoryVoiceInput" rows="3"></textarea>
323 |         </div>
324 |     </div>
325 |     <div class="modal-footer" style="border-top: 1px solid var(--border-color, #495057);">
326 |         <button type="button" class="btn btn-secondary btn-sm" data-bs-dismiss="modal">Отмена</button>
327 |         <button type="button" class="btn btn-primary btn-sm" id="btn-save-history-confirm">Сохранить в RAG</button>
328 |     </div>
329 | </div>
330 | </div>
331 | </div>
332 |
```

main.js

```
1 // — CHAT.JS
2
3 let currentMediaPath = null;
4 let chatHistory = [];
5 let isDebugMode = false; // По умолчанию PROD режим
6 let systemInstruction = null; // Хранит системную инструкцию
7
8 function clearChatHistory() {
9     chatHistory = [];
10    try {
11        localStorage.removeItem('chat_history');
12    } catch (e) {}
13    const chatWindow = document.getElementById('chat-window');
14    if (!chatWindow) return;
15    chatWindow.innerHTML = '';
16    const welcome = document.createElement('div');
17    welcome.className = 'message bot-message';
18    welcome.innerHTML = '<strong>Ai Ассистент</strong> (System):
19    История очищена. Можете начать новый разговор.';
20    chatWindow.appendChild(welcome);
21    chatWindow.scrollTop = chatWindow.scrollHeight;
22 }
23
24 function initChatTab() {
25     const chatWindow = document.getElementById('chat-window');
26     if (chatWindow) {
27         // Приветственное сообщение
28         const welcome = document.createElement('div');
29         welcome.className = 'message bot-message';
30         welcome.innerHTML = '<strong>Ai Ассистент</strong> (System
31         ): Добро пожаловать в чат! Можете спрашивать о фильмах и сериа
32         лах — я покажу путь к файлу и смогу открыть его в плеере.';
33         chatWindow.appendChild(welcome);
34         chatWindow.scrollTop = chatWindow.scrollHeight;
```

```
33 // Загрузка сохраненной истории из localStorage
34 try {
35     const stored = localStorage.getItem('chat_history');
36     if (stored) {
37         const rawHistory = JSON.parse(stored) || [];
38         const isCleanText = (t) => {
39             if (!t) return false;
40             const s = String(t).trim();
41             if (s.startsWith(' ') || s.startsWith('Ошибка') || s
42             .startsWith('Error') || s.includes('В локальной базе ничего не
43             найдено') || s.includes('DEBUG MODE')) {
44                 return false;
45             }
46             return true;
47         };
48         const sanitized = [];
49         for (let i = 0; i < rawHistory.length; i++) {
50             const u = rawHistory[i];
51             if (u.role === 'user' && isCleanText(u.parts && u.pa
52             rts[0])) {
53                 if (i + 1 < rawHistory.length && rawHistory[i + 1]
54                 .role === 'model' && isCleanText(rawHistory[i + 1].parts && ra
55                 wHistory[i + 1].parts[0])) {
56                     sanitized.push(u);
57                     sanitized.push(rawHistory[i + 1]);
58                     i++;
59                 }
60             }
61             chatHistory = sanitized;
62             try {
63                 localStorage.setItem('chat_history', JSON.stringify(
64                 chatHistory));
65             } catch (e) {}
66
67             chatHistory.forEach((msg, idx) => {
68                 const role = msg.role === 'model' ? 'bot' : 'user';
```

```

65 |         const text = msg.parts[0];
66 |         const query = (role === 'bot' && idx > 0) ? chatHistory[
    ory[idx - 1].parts[0] : '';
67 |         addMessage(role, text, query);
68 |     });
69 | }
70 | } catch (e) {
71 |     console.error('Failed to load chat history:', e);
72 |     chatHistory = [];
73 | }
74 |
75 | // Загрузка системной инструкции
76 | loadSystemInstruction();
77 |
78 | // Инициализация тулбара моделей, провайдеров и поиска
79 | initChatDebuggerToolbar();
80 |
81 | // Обновление бейджей модели и провайдера поиска
82 | if (typeof window.updateChatBadges === 'function') {
83 |     window.updateChatBadges();
84 | }
85 |
86 | // Обработчик скачивания торрентов
87 | chatWindow.addEventListener('click', async (e) => {
88 |     const btn = e.target.closest('.download-torrent-btn');
89 |     if (!btn) return;
90 |
91 |     const url = btn.getAttribute('data-url');
92 |     const source = btn.getAttribute('data-source');
93 |     const title = btn.getAttribute('data-title');
94 |
95 |     btn.disabled = true;
96 |     const originalHtml = btn.innerHTML;
97 |     btn.textContent = '⏏ Добавление...';
98 |
99 |     try {
100 |         const response = await fetch('/api/torrents/download',
    {

```

```

101         method: 'POST',
102         headers: { 'Content-Type': 'application/json' },
103         body: JSON.stringify({ url, source, title })
104     });
105
106     if (response.ok) {
107         btn.textContent = '☑ Добавлено в qBittorrent';
108         btn.classList.remove('btn-outline-success');
109         btn.classList.add('btn-success');
110     } else {
111         const errData = await response.json();
112         alert('Ошибка при добавлении торрента: ' + (errData.
113 detail || response.statusText));
114         btn.disabled = false;
115         btn.innerHTML = originalHtml;
116     }
117 } catch (err) {
118     alert('Ошибка соединения с сервером: ' + err.message);
119     btn.disabled = false;
120     btn.innerHTML = originalHtml;
121 }
122 });
123
124
125 // Event listeners for chat
126 const sendBtn = document.getElementById('send-button');
127 const msgInput = document.getElementById('message-input');
128 const debugModeSelector = document.getElementById('chat-mode
129 -debug');
130
131 if (sendBtn) sendBtn.addEventListener('click', sendMessage);
132 if (msgInput) msgInput.addEventListener('keypress', e => { i
133 f (e.key==='Enter') sendMessage(); });
134 if (debugModeSelector) debugModeSelector.addEventListener('c
135 hange', (e) => {
136     isDebugMode = e.target.value === 'debug';
137 });

```

```
135 |
136 |   const clearHistoryBtn = document.getElementById('clear-histo
    | ry-btn');
137 |   if (clearHistoryBtn) clearHistoryBtn.addEventListener('click
    | ', clearChatHistory);
138 |
139 |   const saveHistoryBtn = document.getElementById('save-history
    | -rag-btn');
140 |   if (saveHistoryBtn) saveHistoryBtn.addEventListener('click',
    | saveFullHistoryToRag);
141 |
142 |   const chatAlgoBtn = document.getElementById('chat-algo-btn')
    | ;
143 |   if (chatAlgoBtn) {
144 |     chatAlgoBtn.addEventListener('click', () => {
145 |       if (typeof window.showChatLogicModal === 'function') {
146 |         window.showChatLogicModal();
147 |       } else {
148 |         window.open('/html/assets/chat_and_rag_logic.svg', '_b
    | lank');
149 |       }
150 |     });
151 |   }
152 |
153 |   // Media controls handlers
154 |   document.getElementById('btn-toggle-player-body')?.addEventL
    | istener('click', togglePlayerBody);
155 |   document.getElementById('btn-close-player')?.addEventListener
    | r('click', closeMediaControls);
156 |
157 |   // Initialize CosmicPlayer
158 |   initCosmicPlayer();
159 | }
160 |
161 | let chatGroupedModels = {};
162 |
163 | async function initChatDebuggerToolbar() {
164 |   const modelSelect = document.getElementById('chat-model-sele
```

```
    ct');
165   const searchSelect = document.getElementById('chat-search-select');
166   const setDefaultBtn = document.getElementById('btn-chat-set-default');
167
168   if (!modelSelect) return;
169
170   // 1. Загрузка списка доступных моделей
171   try {
172     const res = await fetch('/api/chat/models');
173     if (res.ok) {
174       const data = await res.json();
175       chatGroupedModels = data.models || {};
176       if (Array.isArray(chatGroupedModels)) {
177         chatGroupedModels = { 'gemini': chatGroupedModels };
178       }
179     }
180   } catch (e) {
181     console.error('[ChatDebugger] Ошибка загрузки моделей:', e);
182   }
183
184   // 2. Загрузка текущих настроек пользователя и поиска
185   let currentModel = window.activeModelName || '';
186   let currentSearch = window.activeSearchEngine || 'gemini_cli';
187   let searchConfig = {};
188
189   try {
190     const sRes = await fetch('/auth/settings');
191     if (sRes.ok) {
192       const settings = await sRes.json();
193       if (settings.model) currentModel = settings.model;
194       if (settings.search_engine) currentSearch = settings.search_engine;
195     }
196   } catch (e) {
```

```
197     console.error('[ChatDebugger] Ошибка загрузки настроек пол
| ьзователя:', e);
198   }
199
200   try {
201     if (window.api && window.api.fetch) {
202       const cfg = await window.api.fetch('/api/admin/web-searc
| h/config');
203       if (cfg) {
204         searchConfig = cfg;
205         if (cfg.engine && !currentSearch) {
206           currentSearch = cfg.engine;
207         }
208       }
209     }
210   } catch (e) {
211     console.error('[ChatDebugger] Ошибка загрузки конфига веб-
| поиска:', e);
212   }
213
214   // 3. Заполнение иерархического списка моделей ИИ (optgroupс
| по провайдерам)
215   function populateHierarchicalModels(modelsGrouped, targetMod
| el = '') {
216     modelSelect.innerHTML = '';
217
218     const providerMeta = {
219       'gemini': { label: '□ Google Gemini', order: 1 },
220       'agy': { label: '□ Google Antigravity (AGY)', order: 2
| },
221       'foundry': { label: '□ Microsoft Foundry', order: 3 },
222       'ollama': { label: '□ Ollama (Local)', order: 4 }
223     };
224
225     const providers = Object.keys(modelsGrouped).sort((a, b) =
| > {
226       const oA = providerMeta[a]?.order ?? 99;
227       const oB = providerMeta[b]?.order ?? 99;
```



```
228     return oA - oB;
229 });
230
231 let totalModels = 0;
232 let firstModel = '';
233
234 providers.forEach(p => {
235     const list = modelsGrouped[p] || [];
236     if (!Array.isArray(list) || list.length === 0) return;
237
238     const optgroup = document.createElement('optgroup');
239     optgroup.label = providerMeta[p]?.label || `□ ${p.toUpperCase()}`;
240
241     list.forEach(m => {
242         totalModels++;
243         if (!firstModel) firstModel = m;
244         const opt = document.createElement('option');
245         opt.value = m;
246         opt.textContent = m;
247         optgroup.appendChild(opt);
248     });
249
250     modelSelect.appendChild(optgroup);
251 });
252
253 if (totalModels === 0) {
254     const opt = document.createElement('option');
255     opt.value = '';
256     opt.textContent = 'Нет доступных моделей';
257     modelSelect.appendChild(opt);
258 } else if (targetModel) {
259     modelSelect.value = targetModel;
260     if (!modelSelect.value && firstModel) {
261         modelSelect.value = firstModel;
262     }
263 } else if (firstModel) {
264     modelSelect.value = firstModel;
```

```

265 |     }
266 | }
267 |
268 | // 4. Заполнение иерархического списка провайдеров и моделей
    веб-поиска (optgroups)
269 | function populateHierarchicalSearchEngines(modelsGrouped, ta
    rgetSearch = 'gemini_cli', cfg = {}) {
270 |     if (!searchSelect) return;
271 |     searchSelect.innerHTML = '';
272 |
273 |     const geminiList = modelsGrouped.gemini || ['gemini-2.5-fl
    ash', 'gemini-2.5-pro', 'gemini-3.1-flash-lite'];
274 |     const geminiCliList = modelsGrouped.gemini_cli || ['gemini
    -3.1-flash-lite', 'gemini-2.5-flash', 'gemini-2.5-pro'];
275 |     const agyList = modelsGrouped.agy || ['agy-flash', 'agy-pr
    o', 'agy-gemma-4-26b-a4b-it'];
276 |
277 |     const searchStructure = [
278 |         {
279 |             label: '□ Google Gemini CLI',
280 |             engine: 'gemini_cli',
281 |             models: geminiCliList,
282 |             defaultModel: cfg.gemini_cli_model || 'gemini-3.1-flas
    h-lite'
283 |         },
284 |         {
285 |             label: '□ Google Gemini Grounding',
286 |             engine: 'gemini',
287 |             models: geminiList,
288 |             defaultModel: cfg.gemini_model || 'gemini-2.5-flash'
289 |         },
290 |         {
291 |             label: '□ Google Antigravity (AGY)',
292 |             engine: 'agy',
293 |             models: agyList,
294 |             defaultModel: cfg.agy_model || 'agy-flash'
295 |         },
296 |         {

```

```

297 |         label: '□ LangChain MCP Agent',
298 |         engine: 'langchain',
299 |         models: ['gemini-2.5-flash', 'ollama:qwen2.5:7b'],
300 |         defaultModel: 'gemini-2.5-flash'
301 |     },
302 |     {
303 |         label: '□ Playwright Browser MCP',
304 |         engine: 'playwright',
305 |         models: ['chromium'],
306 |         defaultModel: 'chromium'
307 |     }
308 | ];
309 |
310 | let targetEngine = targetSearch;
311 | let targetMdl = '';
312 | if (targetSearch && targetSearch.includes(':') && !targetSearch.startsWith('ollama:') && !targetSearch.startsWith('foundry:')) {
313 |     const p = targetSearch.split(':');
314 |     targetEngine = p[0];
315 |     targetMdl = p.slice(1).join(':');
316 | }
317 |
318 | let selectedValueToSet = '';
319 |
320 | searchStructure.forEach(group => {
321 |     const optgroup = document.createElement('optgroup');
322 |     optgroup.label = group.label;
323 |
324 |     group.models.forEach(m => {
325 |         const opt = document.createElement('option');
326 |         const val = `${group.engine}:${m}`;
327 |         opt.value = val;
328 |         opt.textContent = m;
329 |         optgroup.appendChild(opt);
330 |
331 |         if (group.engine === targetEngine) {
332 |             if (targetMdl && m === targetMdl) {

```

```
333         selectedValueToSet = val;
334     } else if (group.defaultModel && m === group.default
Model && !selectedValueToSet) {
335         selectedValueToSet = val;
336     } else if (!selectedValueToSet) {
337         selectedValueToSet = val;
338     }
339 }
340 });
341
342     searchSelect.appendChild(optgroup);
343 });
344
345     if (selectedValueToSet) {
346         searchSelect.value = selectedValueToSet;
347     }
348 }
349
350     populateHierarchicalModels(chatGroupedModels, currentModel);
351     populateHierarchicalSearchEngines(chatGroupedModels, current
Search, searchConfig);
352
353     // Синхронизация бейджей
354     if (typeof window.updateChatBadges === 'function') {
355         window.updateChatBadges(modelSelect.value, searchSelect ?
searchSelect.value : currentSearch);
356     }
357
358     // Обработчик смены модели в иерархическом списке
359     modelSelect.onChange = () => {
360         if (typeof window.updateChatBadges === 'function') {
361             window.updateChatBadges(modelSelect.value, searchSelect
? searchSelect.value : undefined);
362         }
363     };
364
365     // Обработчик смены поискового движка
366     if (searchSelect) {
```

```
367 |     searchSelect.onChange = () => {
368 |         if (typeof window.updateChatBadges === 'function') {
369 |             window.updateChatBadges(modelSelect.value, searchSelect.value);
370 |         }
371 |     };
372 | }
373 |
374 | const setDefaultModelBtn = document.getElementById('btn-chat
-set-default-model');
375 | const setDefaultSearchBtn = document.getElementById('btn-chat
-set-default-search');
376 |
377 | // Обработчик кнопки «Дефолтная модель внутренней генерации»
378 | if (setDefaultModelBtn) {
379 |     setDefaultModelBtn.onclick = async () => {
380 |         const chosenModel = modelSelect.value;
381 |
382 |         if (!chosenModel) {
383 |             alert('Выберите модель перед сохранением!');
384 |             return;
385 |         }
386 |
387 |         setDefaultModelBtn.disabled = true;
388 |         const origText = setDefaultModelBtn.innerHTML;
389 |         setDefaultModelBtn.innerHTML = '⏸ Сохранение...';
390 |
391 |         try {
392 |             await fetch('/auth/settings', {
393 |                 method: 'POST',
394 |                 headers: { 'Content-Type': 'application/json' },
395 |                 body: JSON.stringify({ model: chosenModel })
396 |             });
397 |
398 |             if (typeof window.updateChatBadges === 'function') {
399 |                 window.updateChatBadges(chosenModel, undefined);
400 |             }
401 |         }
```

```
402 |         const otherModelSelect = document.getElementById('admin-model-select');
403 |         if (otherModelSelect) otherModelSelect.value = chosenModel;
404 |         const modelsTabSelect = document.getElementById('models-tab-select');
405 |         if (modelsTabSelect) modelsTabSelect.value = chosenModel;
406 |
407 |         setDefaultModelBtn.classList.remove('btn-outline-warning');
408 |         setDefaultModelBtn.classList.add('btn-success');
409 |         setDefaultModelBtn.innerHTML = '<i class="bi bi-check-circle-fill"></i> Установлена!';
410 |         setTimeout(() => {
411 |             setDefaultModelBtn.classList.remove('btn-success');
412 |             setDefaultModelBtn.classList.add('btn-outline-warning');
413 |             setDefaultModelBtn.innerHTML = origText;
414 |             setDefaultModelBtn.disabled = false;
415 |         }, 2500);
416 |
417 |     } catch (err) {
418 |         console.error('[ChatDebugger] Ошибка сохранения дефолтной модели:', err);
419 |         alert('Ошибка сохранения модели: ' + err.message);
420 |         setDefaultModelBtn.disabled = false;
421 |         setDefaultModelBtn.innerHTML = origText;
422 |     }
423 | };
424 | }
425 |
426 | // Обработчик кнопки «Дефолтный модуль веб-поиска»
427 | if (setDefaultSearchBtn) {
428 |     setDefaultSearchBtn.onclick = async () => {
429 |         const rawSearchVal = searchSelect ? searchSelect.value : 'gemini_cli';
430 |
```

```

431     let chosenSearchEngine = rawSearchVal;
432     let chosenSearchModel = '';
433     if (rawSearchVal.includes(':') && !rawSearchVal.startsWith(
434         'ollama:') && !rawSearchVal.startsWith('foundry:')) {
435         const parts = rawSearchVal.split(':');
436         chosenSearchEngine = parts[0];
437         chosenSearchModel = parts.slice(1).join(':');
438     }
439
440     setDefaultSearchBtn.disabled = true;
441     const origText = setDefaultSearchBtn.innerHTML;
442     setDefaultSearchBtn.innerHTML = '⏏ Сохранение...';
443
444     try {
445         // 1. Сохраняем поисковый движок в настройках пользова
446         //    теля
447         await fetch('/auth/settings', {
448             method: 'POST',
449             headers: { 'Content-Type': 'application/json' },
450             body: JSON.stringify({ search_engine: rawSearchVal })
451         });
452
453         // 2. Сохраняем поисковый движок и его модель в конфи
454         //    гурации веб-поиска (если админ)
455         try {
456             if (window.api && window.api.fetch) {
457                 const searchPayload = { engine: chosenSearchEngine
458                 };
459                 if (chosenSearchEngine === 'gemini') searchPayload
460                 .gemini_model = chosenSearchModel;
461                 if (chosenSearchEngine === 'gemini_cli') searchPay
462                 load.gemini_cli_model = chosenSearchModel;
463                 if (chosenSearchEngine === 'agy') searchPayload.ag
464                 y_model = chosenSearchModel;
465
466                 await window.api.fetch('/api/admin/web-search/conf
467                 ig', {

```

```

460         method: 'POST',
461         headers: { 'Content-Type': 'application/json' },
462         body: JSON.stringify(searchPayload)
463     });
464     }
465     } catch (searchErr) {
466         console.warn('[ChatDebugger] Ошибка сохранения движк
а поиска:', searchErr);
467     }
468
469     // 3. Обновляем глобальные бейджи и селекторы
470     if (typeof window.updateChatBadges === 'function') {
471         window.updateChatBadges(undefined, rawSearchVal);
472     }
473
474     const searchTabEngine = document.getElementById('searc
h-tab-engine-selector');
475     if (searchTabEngine) searchTabEngine.value = chosenSea
rchEngine;
476
477     setDefaultSearchBtn.classList.remove('btn-outline-info
');
478     setDefaultSearchBtn.classList.add('btn-success');
479     setDefaultSearchBtn.innerHTML = '<i class="bi bi-check
-circle-fill"></i> Установлен!';
480     setTimeout(() => {
481         setDefaultSearchBtn.classList.remove('btn-success');
482         setDefaultSearchBtn.classList.add('btn-outline-info
');
483         setDefaultSearchBtn.innerHTML = origText;
484         setDefaultSearchBtn.disabled = false;
485     }, 2500);
486
487     } catch (err) {
488         console.error('[ChatDebugger] Ошибка сохранения дефолт
ного поиска:', err);
489         alert('Ошибка сохранения поиска: ' + err.message);
490         setDefaultSearchBtn.disabled = false;

```



```
491         setDefaultSearchBtn.innerHTML = origText;
492     }
493 };
494 }
495 }
496
497 // Export init function for main.js
498 if (typeof window !== 'undefined') {
499     window.initChatTab = initChatTab;
500     window.initChatDebuggerToolbar = initChatDebuggerToolbar;
501 }
502
503 async function saveFullHistoryToRag() {
504     if (!chatHistory || chatHistory.length === 0) {
505         alert('История чата пуста!');
506         return;
507     }
508
509     // Формируем структурированный текст истории диалога
510     let fullChatText = "";
511     chatHistory.forEach(msg => {
512         const roleName = msg.role === 'user' ? 'Пользователь' : 'Ассистент';
513         const text = msg.parts[0] || "";
514         fullChatText += `${roleName}: ${text}\n\n`;
515     });
516
517     const timestamp = new Date().toLocaleString('ru-RU');
518     const defaultTitle = `Диалог от ${timestamp}`;
519     const defaultVoice = `История диалога с ассистентом, сохраненная ${timestamp}.`;
520
521     // Заполняем поля модального окна
522     document.getElementById('saveHistoryTitleInput').value = defaultTitle;
523     document.getElementById('saveHistoryTextInput').value = fullChatText.trim();
524     document.getElementById('saveHistoryVoiceInput').value = defaultVoice;
```

```
    aultVoice;
525
526    const modalEl = document.getElementById('saveHistoryModal');
527    if (!modalEl) return;
528
529    const modal = new bootstrap.Modal(modalEl);
530    modal.show();
531
532    // Настраиваем кнопку подтверждения сохранения
533    const confirmBtn = document.getElementById('btn-save-history
    -confirm');
534    if (confirmBtn) {
535        confirmBtn.onclick = async () => {
536            const title = document.getElementById('saveHistoryTitleI
    nput').value.trim();
537            const text = document.getElementById('saveHistoryTextInp
    ut').value.trim();
538            const voice = document.getElementById('saveHistoryVoiceI
    nput').value.trim();
539
540            if (!title || !text) {
541                alert('Заголовок и содержимое диалога не могут быть пу
    стыми!');
542                return;
543            }
544
545            confirmBtn.disabled = true;
546            confirmBtn.innerHTML = '⏳ Сохранение...';
547
548            try {
549                const r = await fetch('/api/chat/save-rag', {
550                    method: 'POST',
551                    headers: {'Content-Type': 'application/json'},
552                    body: JSON.stringify({
553                        query: title,
554                        chat_text: text,
555                        voice_text: voice
556                    })
```

```

557         });
558
559         if (r.ok) {
560             alert('История чата успешно сохранена в архив RAG!');
561             modal.hide();
562         } else {
563             const errData = await r.json().catch(() => ({}));
564             alert('Ошибка при сохранении: ' + (errData.detail ||
r.statusText));
565         }
566     } catch (err) {
567         alert('Ошибка сети: ' + err.message);
568     } finally {
569         confirmBtn.disabled = false;
570         confirmBtn.innerHTML = 'Сохранить в RAG';
571     }
572 };
573 }
574 }
575
576 function togglePlayerBody() {
577     const body = document.getElementById('media-player-body');
578     const btn = document.getElementById('btn-toggle-player-body');
579     if (body.classList.contains('d-none')) {
580         body.classList.remove('d-none');
581         btn.textContent = 'Свернуть плеер';
582     } else {
583         body.classList.add('d-none');
584         btn.textContent = 'Развернуть плеер';
585     }
586 }
587
588 let cosmicHls = null;
589 let cosmicHistory = [];
590
591 function initCosmicPlayer() {

```

```
592 |   const streamForm = document.getElementById('cosmic-stream-form');
593 |   const streamUrlInput = document.getElementById('cosmic-stream-url');
594 |   const badges = document.querySelectorAll('#cosmic-provider-badges [data-prov]');
595 |
596 |   if (streamForm) {
597 |     streamForm.addEventListener('submit', (e) => {
598 |       e.preventDefault();
599 |       const url = streamUrlInput.value.trim();
600 |       if (url) loadCosmicVideo(url);
601 |     });
602 |   }
603 |
604 |   badges.forEach(badge => {
605 |     badge.addEventListener('click', () => {
606 |       const prov = badge.getAttribute('data-prov');
607 |       let demo = '';
608 |       if (prov === 'youtube') demo = 'https://www.youtube.com/watch?v=aqz-KE-bpKQ';
609 |       else if (prov === 'vk') demo = 'https://vk.com/video-22822305_456239018';
610 |       else if (prov === 'rutube') demo = 'https://rutube.ru/video/e7cfc8cb4310d54026fb4bd56e828d1/';
611 |       else if (prov === 'direct') demo = 'https://test-streams.mux.dev/x36xhzz/x36xhzz.m3u8';
612 |
613 |       if (demo) {
614 |         streamUrlInput.value = demo;
615 |         loadCosmicVideo(demo);
616 |       }
617 |     });
618 |   });
619 |
620 |   // Load history from localStorage
621 |   try {
622 |     cosmicHistory = JSON.parse(localStorage.getItem('cosmic_hi
```

```

    story_chat') || '[]');
623   } catch(e) {}
624   renderCosmicHistory();
625
626   // Custom Controls Event Listeners
627   const playBtn = document.getElementById('cosmic-play-btn');
628   const stopBtn = document.getElementById('cosmic-stop-btn');
629   const muteBtn = document.getElementById('cosmic-mute-btn');
630   const volSlider = document.getElementById('cosmic-volume-sli
der');
631   const speedSel = document.getElementById('cosmic-speed-selec
t');
632   const fullBtn = document.getElementById('cosmic-fullscreen-b
tn');
633   const progContainer = document.getElementById('cosmic-progre
ss-container');
634   const videoEl = document.getElementById('cosmic-video-elemen
t');
635
636   if (videoEl) {
637     videoEl.addEventListener('timeupdate', () => {
638       const cur = videoEl.currentTime;
639       const dur = videoEl.duration || 0;
640       if (dur > 0) {
641         document.getElementById('cosmic-progress-bar').style.w
idth = `${(cur / dur) * 100}%`;
642       }
643       document.getElementById('cosmic-time-current').textConte
nt = formatTime(cur);
644       document.getElementById('cosmic-time-duration').textCont
ent = formatTime(dur);
645     });
646
647     videoEl.addEventListener('loadedmetadata', () => {
648       document.getElementById('cosmic-time-duration').textCont
ent = formatTime(videoEl.duration);
649     });
650   }

```

```
651 |
652 | playBtn?.addEventListener('click', () => {
653 |     if (videoEl.paused) {
654 |         videoEl.play();
655 |         playBtn.textContent = '⏮';
656 |     } else {
657 |         videoEl.pause();
658 |         playBtn.textContent = '▶';
659 |     }
660 | });
661 |
662 | stopBtn?.addEventListener('click', () => {
663 |     videoEl.pause();
664 |     videoEl.currentTime = 0;
665 |     playBtn.textContent = '▶';
666 | });
667 |
668 | volSlider?.addEventListener('input', (e) => {
669 |     videoEl.volume = e.target.value;
670 |     if (muteBtn) muteBtn.textContent = e.target.value === '0'
671 | ? '⏮' : '⏮';
672 | });
673 |
674 | muteBtn?.addEventListener('click', () => {
675 |     if (videoEl.muted) {
676 |         videoEl.muted = false;
677 |         volSlider.value = videoEl.volume;
678 |         muteBtn.textContent = '⏮';
679 |     } else {
680 |         videoEl.muted = true;
681 |         volSlider.value = 0;
682 |         muteBtn.textContent = '⏮';
683 |     }
684 | });
685 |
686 | speedSel?.addEventListener('change', (e) => {
687 |     videoEl.playbackRate = parseFloat(e.target.value);
688 | });
```

```
688 |
689 | progContainer?.addEventListener('click', (e) => {
690 |     const rect = progContainer.getBoundingClientRect();
691 |     const pct = (e.clientX - rect.left) / rect.width;
692 |     videoEl.currentTime = pct * videoEl.duration;
693 | });
694 |
695 | fullBtn?.addEventListener('click', () => {
696 |     const wrapper = document.getElementById('cosmic-video-wrap
per');
697 |     if (!document.fullscreenElement) {
698 |         wrapper.requestFullscreen().catch(() => {});
699 |     } else {
700 |         document.exitFullscreen();
701 |     }
702 | });
703 | }
704 |
705 | function formatTime(sec) {
706 |     if (isNaN(sec)) return '0:00';
707 |     const mins = Math.floor(sec / 60);
708 |     const secs = Math.floor(sec % 60);
709 |     return `${mins}:${secs < 10 ? '0' : ''}${secs}`;
710 | }
711 |
712 | function loadCosmicVideo(url, title = null) {
713 |     // Clear existing HLS or frames
714 |     const embed = document.getElementById('cosmic-embed-player')
;
715 |     const videoWrapper = document.getElementById('cosmic-video-w
rapper');
716 |     const placeholder = document.getElementById('cosmic-player-p
laceholder');
717 |     const videoEl = document.getElementById('cosmic-video-elemen
t');
718 |     const glow = document.getElementById('cosmic-ambient-glow');
719 |
720 |     embed.classList.add('hidden');
```

```
721 | embed.src = '';
722 | videoWrapper.classList.add('hidden');
723 | videoEl.pause();
724 | videoEl.src = '';
725 | placeholder.classList.add('hidden');
726 |
727 | if (cosmicHls) {
728 |     cosmicHls.destroy();
729 |     cosmicHls = null;
730 | }
731 |
732 | // Parse URL
733 | const parsed = parseVideoUrl(url);
734 | const resolvedTitle = title || parsed.defaultTitle;
735 |
736 | document.getElementById('now-playing-title').textContent = r
    | esolvedTitle;
737 |
738 | const colors = {
739 |     youtube: 'radial-gradient(circle, rgba(255, 0, 0, 0.15) 0%
    | , transparent 60%)',
740 |     vk: 'radial-gradient(circle, rgba(74, 118, 168, 0.15) 0%,
    | transparent 60%)',
741 |     rutube: 'radial-gradient(circle, rgba(235, 95, 30, 0.15) 0
    | %, transparent 60%)',
742 |     direct: 'radial-gradient(circle, rgba(121, 40, 202, 0.15)
    | 0%, transparent 60%)'
743 | };
744 | glow.style.background = colors[parsed.provider] || colors.di
    | rect;
745 |
746 | if (parsed.type === 'embed') {
747 |     embed.src = parsed.embedUrl;
748 |     embed.classList.remove('hidden');
749 | } else if (parsed.type === 'html5') {
750 |     videoWrapper.classList.remove('hidden');
751 |     if (url.includes('.m3u8')) {
752 |         if (typeof Hls !== 'undefined' && Hls.isSupported()) {
```



```

753     cosmicHls = new Hls();
754     cosmicHls.loadSource(url);
755     cosmicHls.attachMedia(videoEl);
756     cosmicHls.on(Hls.Events.MANIFEST_PARSED, () => videoEl
    .play());
757     } else if (videoEl.canPlayType('application/vnd.apple.mpe
    gurl')) {
758         videoEl.src = url;
759         videoEl.play();
760     }
761     } else {
762         videoEl.src = url;
763         videoEl.play();
764     }
765     } else {
766         // Iframe fallback
767         embed.src = url;
768         embed.classList.remove('hidden');
769     }
770
771     // Save to history list
772     addToCosmicHistory(url, resolvedTitle, parsed.provider);
773 }
774
775 function parseVideoUrl(url) {
776     const ytReg = /(?:youtube\.com\/(?:[^\w]+\w\/.+\/|(?:(v|e(?:mbe
    d)?))\w\/|.*[?&]v=)|youtu\.be\/)([^\s]{11})/;
777     const ytMatch = url.match(ytReg);
778     if (ytMatch) {
779         return {
780             provider: 'youtube',
781             type: 'embed',
782             embedUrl: `https://www.youtube.com/embed/${ytMatch[1]}?a
    utoplay=1&rel=0`,
783             defaultTitle: `YouTube (${ytMatch[1]})`
784         };
785     }
786

```

```
787 | const vkReg = /vk\.com\/video(-?\d+)_(\d+)/;
788 | const vkMatch = url.match(vkReg);
789 | if (vkMatch) {
790 |     return {
791 |         provider: 'vk',
792 |         type: 'embed',
793 |         embedUrl: `https://vk.com/video_ext.php?oid=${vkMatch[1]}
    | &id=${vkMatch[2]}&autoplay=1`,
794 |         defaultTitle: `VK Video (${vkMatch[1]}_${vkMatch[2]})`
795 |     };
796 | }
797 |
798 | const rutubeReg = /rutube\.ru\/(?:video|play\/embed)\/([a-f0
    | -9]{32})/;
799 | const rutubeMatch = url.match(rutubeReg);
800 | if (rutubeMatch) {
801 |     return {
802 |         provider: 'rutube',
803 |         type: 'embed',
804 |         embedUrl: `https://rutube.ru/play/embed/${rutubeMatch[1]}
    | ?autoplay=1`,
805 |         defaultTitle: `RuTube Video`
806 |     };
807 | }
808 |
809 | if (url.includes('.m3u8') || url.includes('.mp4')) {
810 |     return {
811 |         provider: 'direct',
812 |         type: 'html5',
813 |         defaultTitle: url.includes('.m3u8') ? 'Поток HLS' : 'Пря
    | мой файл MP4'
814 |     };
815 | }
816 |
817 | return {
818 |     provider: 'direct',
819 |     type: 'iframe_fallback',
820 |     defaultTitle: 'Веб-фрейм'
```

```

821 | };
822 | }
823 |
824 | function addToCosmicHistory(url, title, provider) {
825 |     cosmicHistory = cosmicHistory.filter(h => h.url !== url);
826 |     cosmicHistory.unshift({ url, title, provider });
827 |     if (cosmicHistory.length > 5) cosmicHistory.pop();
828 |     localStorage.setItem('cosmic_history_chat', JSON.stringify(c
    osmicHistory));
829 |     renderCosmicHistory();
830 | }
831 |
832 | function renderCosmicHistory() {
833 |     const container = document.getElementById('cosmic-history-li
    st');
834 |     if (!container) return;
835 |
836 |     if (cosmicHistory.length === 0) {
837 |         container.innerHTML = '<div class="text-muted small text-c
    enter py-2">История пуста</div>';
838 |         return;
839 |     }
840 |
841 |     container.innerHTML = cosmicHistory.map(h => `
842 |         <div class="p-1 border rounded bg-dark small cursor-pointe
    r text-truncate cosmic-hist-item" style="cursor:pointer; font-
    size:0.75rem;" data-url="${h.url}" data-title="${h.title.repla
    ce(/"/g, '&quot;')}">
843 |             □ [${h.provider.toUpperCase()}] ${h.title}
844 |         </div>
845 |     `).join('');
846 |
847 |     container.querySelectorAll('.cosmic-hist-item').forEach(item
    => {
848 |         item.addEventListener('click', () => {
849 |             const url = item.getAttribute('data-url');
850 |             const title = item.getAttribute('data-title');
851 |             document.getElementById('cosmic-stream-url').value = url

```

```

    ;
852     loadCosmicVideo(url, title);
853   });
854 });
855 }
856
857 async function sendMessage() {
858   const input = document.getElementById('message-input');
859   const btn    = document.getElementById('send-button');
860   const msg     = input.value.trim();
861   if (!msg) return;
862
863   // Определяем режим чата до любой логики
864   const modeSelector = document.getElementById('chat-mode-selector');
865   const chatMode = modeSelector ? modeSelector.value : 'story'
    ;
866
867   input.value = '';
868   btn.disabled = true; btn.textContent = 'Отправка...';
869
870   const lowMsg = msg.toLowerCase().trim();
871   const playCommands = ['включи просмотр', 'запусти просмотр',
    'просмотр', 'включи видео', 'открой плеер', 'включить просмотр'];
872   if (playCommands.includes(lowMsg)) {
873     addMessage('user', msg);
874     if (currentMediaPath) {
875       openMediaPlayer();
876       const body = document.getElementById('media-player-body')
    );
877       if (body) {
878         body.classList.remove('d-none');
879         const btnToggle = document.getElementById('btn-toggle-player-body');
880         if (btnToggle) btnToggle.textContent = 'Свернуть плеер'
    ';
881       }

```

```
882     const videoEl = document.getElementById('cosmic-video-el
|ement');
883     if (videoEl && videoEl.src) {
884         videoEl.play().catch(() => {});
885     }
886
887     const reply = "Запускаю просмотр во встроенном плеере.";
888     addMessage('bot', reply);
889     window.chatService.speak(reply);
890
891     btn.disabled = false;
892     btn.textContent = 'Отправить';
893     return;
894 } else {
895     const reply = "Медиафайл для воспроизведения не выбран.
|Пожалуйста, сначала найдите фильм или сериал.";
896     addMessage('bot', reply);
897     window.chatService.speak(reply);
898
899     btn.disabled = false;
900     btn.textContent = 'Отправить';
901     return;
902 }
903 }
904
905 // Если режим DEBUG - отправляем запрос в бэкенд, но он верн
|ёт полный пром프트 вместо ответа модели
906 if (isDebugMode) {
907     // Показываем запрос пользователя
908     addMessage('user', msg);
909
910     const win = document.getElementById('chat-window');
911     const botMessageEl = document.createElement('div');
912     botMessageEl.className = 'message bot-message';
913     botMessageEl.innerHTML = `Ai Ассистент (DEBUG)</st
|rong> (${new Date().toLocaleTimeString()}): <div class="bot-te
|xt">□ Формирование промпта...</div>`;
914     win.appendChild(botMessageEl);
```

```
915 |     win.scrollTop = win.scrollHeight;
916 |     const textDiv = botMessageEl.querySelector('.bot-text');
917 |
918 |     let fullReply = '';
919 |     let started = false;
920 |
921 |     try {
922 |         const selectedModel = document.getElementById('chat-mode
l-select')?.value || undefined;
923 |         const selectedSearch = document.getElementById('chat-sea
rch-select')?.value || undefined;
924 |
925 |         const replyObj = await window.chatService.sendChatMessag
e(msg, (chunk, status, voice) => {
926 |             if (status) {
927 |                 textDiv.innerHTML = `<span style="color: #8b949e; fo
nt-style: italic;">␣ ${status}</span>`;
928 |             }
929 |             if (chunk) {
930 |                 if (!started) {
931 |                     textDiv.textContent = '';
932 |                     started = true;
933 |                 }
934 |                 fullReply += chunk;
935 |                 textDiv.innerHTML = parseContentToHtml(fullReply);
936 |             }
937 |             win.scrollTop = win.scrollHeight;
938 |         }, chatHistory, {
939 |             chat_mode: chatMode,
940 |             debug_mode: true,
941 |             model: selectedModel,
942 |             search_engine: selectedSearch
943 |         });
944 |
945 |         // В DEBUG режиме не сохраняем запрос и ответ в историю
диалога
946 |
947 |         // В DEBUG режиме мы получаем полный промпт в тексте
```

```

948 |         textDiv.innerHTML = `<div style="background: rgba(255, 2
| 35, 59, 0.1); border: 1px solid #ffeb3b; border-radius: 8px; p
| adding: 15px; margin-top: 10px;">
949 |         <div style="color: #fbc02d; font-weight: bold; margin-
| bottom: 10px; font-size: 1.1em;">␣ DEBUG MODE: Полный промпт (
| не отправлялся в модель)</div>
950 |         <pre style="background: #fff9c4; padding: 10px; border
| -radius: 5px; font-size: 0.85em; overflow-x: auto; white-space
| : pre-wrap; font-family: monospace; border: 1px solid #f57f17;
| ">${fullReply.replace(/</g, '&lt;').replace(/>/g, '&gt;')}</pr
| e>
951 |         <div style="color: #fbc02d; margin-top: 10px; font-siz
| e: 0.9em;">
952 |         <strong>Чтобы отправить запрос в модель, переключите
| сь в режим PROD</strong>
953 |         </div>
954 |     </div>`;
955 |
956 |     // Не сохраняем ответ в историю в DEBUG режиме, так как
| это не реальный ответ модели
957 |     } catch (err) {
958 |         if (!started) {
959 |             textDiv.textContent = '';
960 |         }
961 |         let errText;
962 |         if (err instanceof Error) {
963 |             errText = err.message;
964 |         } else if (typeof err === 'object' && err !== null) {
965 |             errText = JSON.stringify(err);
966 |         } else {
967 |             errText = String(err);
968 |         }
969 |         textDiv.innerHTML = `<span style="color: #ff7070;">Ошибк
| а: ${errText}</span>`;
970 |     }
971 |     finally { btn.disabled = false; btn.textContent = 'Отправи
| ть'; }
972 |     return;

```

```

973 | }
974 |
975 | // Показываем запрос пользователя
976 | addMessage('user', msg);
977 |
978 | const win = document.getElementById('chat-window');
979 | const botMessageEl = document.createElement('div');
980 | botMessageEl.className = 'message bot-message';
981 | botMessageEl.innerHTML = `Ai Ассистент (${new Date().toLocaleTimeString()}: <div class="bot-text"> Ожидание ответа...</div>`;
982 | win.appendChild(botMessageEl);
983 | win.scrollTop = win.scrollHeight;
984 | const textDiv = botMessageEl.querySelector('.bot-text');
985 |
986 | let fullReply = '';
987 | let started = false;
988 |
989 | const isAdminChat = !!document.getElementById('tab-rag') ||
    window.location.pathname.includes('/admin') || window.location
    .href.includes('/admin');
990 |
991 | try {
992 |     const selectedModel = document.getElementById('chat-model-select')?.value || undefined;
993 |     const selectedSearch = document.getElementById('chat-search-select')?.value || undefined;
994 |
995 |     const replyObj = await window.chatService.sendChatMessage(
    msg, (chunk, status, voice, promptDump) => {
996 |         if (status) {
997 |             textDiv.innerHTML = `> ${status}</span>`;
998 |         }
999 |         if (isAdminChat && promptDump && !botMessageEl.querySelector('.prompt-dump-details')) {
1000 |             const details = document.createElement('details');
1001 |             details.className = 'prompt-dump-details';

```



```
1002     details.style.cssText = 'margin-bottom: 10px; font-size: 0.85em; background: #0d1117; padding: 10px; border-radius: 6px; border: 1px solid #30363d; box-shadow: inset 0 1px 4px rgba(0,0,0,0.8);';
1003     details.innerHTML = `<summary style="cursor: pointer; color: #e3b341; font-weight: bold; user-select: none;">⏏ По  
    смотр сформированного промпта (Admin)</summary><pre style="white-space: pre-wrap; margin-top: 10px; color: #00ff66; font-family: 'Consolas', 'Courier New', Courier, monospace; font-size: 0.95em; line-height: 1.4; max-height: 300px; overflow-y: auto;">${promptDump.replace(/</g, '&lt;').replace(/>/g, '&gt;')}</pre>`;
1004     botMessageEl.insertBefore(details, textDiv);
1005 }
1006 if (chunk) {
1007     if (!started) {
1008         textDiv.textContent = '';
1009         started = true;
1010     }
1011     fullReply += chunk;
1012     textDiv.innerHTML = parseContentToHtml(fullReply);
1013 }
1014 win.scrollTop = win.scrollHeight;
1015 }, chatHistory, {
1016     chat_mode: chatMode,
1017     debug_mode: isDebugMode,
1018     model: selectedModel,
1019     search_engine: selectedSearch
1020 });
1021
1022 const reply = replyObj.text;
1023 const voiceReply = replyObj.voice || reply;
1024 const followUp = replyObj.followUp;
1025
1026 const card = tryParseMovieCard(reply);
1027 if (card) {
1028     textDiv.innerHTML = renderMovieCardHtml(card);
1029     window.chatService.speak(card);
```

```
1030     } else {
1031         const isAdminChat = !!document.getElementById('tab-rag')
        || window.location.pathname.includes('/admin') || window.loca
        tion.href.includes('/admin');
1032         if (isAdminChat && replyObj.voice && replyObj.voice !==
        reply) {
1033             textDiv.innerHTML = parseContentToHtml("**[Текст для ч
        ата]:**\n" + reply + "\n\n**[Текст для диктора]:**\n" + voiceR
        eply);
1034         } else {
1035             textDiv.innerHTML = parseContentToHtml(isAdminChat ? v
        oiceReply : reply);
1036         }
1037         window.chatService.speak(voiceReply);
1038     }
1039
1040     // Save valid conversation turns to history
1041     const isErrorOrEmpty = !reply ||
1042         reply.trim().startsWith(' ') ||
1043         reply.trim().startsWith('Ошибка') ||
1044         reply.trim().startsWith('Error') ||
1045         reply.includes('В локальной базе ничего не найдено') ||
1046         reply.includes('DEBUG MODE');
1047
1048     if (!isErrorOrEmpty) {
1049         chatHistory.push({ role: 'user', parts: [msg] });
1050         chatHistory.push({ role: 'model', parts: [reply] });
1051         try {
1052             localStorage.setItem('chat_history', JSON.stringify(ch
        atHistory));
1053         } catch (e) {}
1054     }
1055
1056     // Добавляем кнопку "Сохранить в RAG" только в интерфейсе
        администратора
1057     addRagButton(botMessageEl, msg, reply, voiceReply);
1058
1059     if (typeof fullReply === 'string' && !card) {
```

```

1060     await parseFilmTags(fullReply, botMessageEl);
1061 }
1062
1063 // Automatically trigger follow-up query to emulate a dial
og
1064 if (followUp) {
1065     document.getElementById('message-input').value = followU
p;
1066     setTimeout(() => {
1067         const btn = document.getElementById('send-button');
1068         if (btn && !btn.disabled) btn.click();
1069     }, 1500);
1070 }
1071 } catch (err) {
1072     if (!started) {
1073         textDiv.textContent = '';
1074     }
1075     let errText;
1076     if (err instanceof Error) {
1077         errText = err.message;
1078     } else if (typeof err === 'object' && err !== null) {
1079         errText = JSON.stringify(err);
1080     } else {
1081         errText = String(err);
1082     }
1083     textDiv.innerHTML = `Ошибка:
${errText}</span>`;
1084 }
1085 finally { btn.disabled = false; btn.textContent = 'Отправить
'; }
1086 }
1087
1088 function tryParseMovieCard(text) {
1089     const trimmed = text.trim();
1090     if (trimmed.startsWith('{') && trimmed.endsWith('}')) {
1091         try {
1092             const data = JSON.parse(trimmed);
1093             if (data.title || data.title_ru) return data;

```

```

1094     } catch (e) {}
1095 }
1096 const match = text.match(/\{[\s\S]*?\}/);
1097 if (match) {
1098     try {
1099         const data = JSON.parse(match[0]);
1100         if (data.title || data.title_ru) return data;
1101     } catch (e) {}
1102 }
1103 return null;
1104 }
1105
1106 function renderMovieCardHtml(card) {
1107     const title = card.title_ru || card.title;
1108     const subtitle = card.title_orig || '';
1109     const year = card.year || '';
1110     const category = card.main_category || '';
1111     const genres = card.genres ? card.genres.join(', ') : '';
1112     const directors = card.directors ? card.directors.join(', ')
1113 : '';
1114     const cast = card.cast ? card.cast.slice(0, 5).join(', ') :
1115 '';
1116     const plot = card.plot || '';
1117     const whyWatch = card.why_watch || '';
1118
1119     let ratingHtml = '';
1120     if (card.rating) {
1121         if (card.rating.imdb) ratingHtml += `<span class="badge bg
1122 -warning text-dark me-1">IMDb: ${card.rating.imdb}</span>`;
1123         if (card.rating.кинопоиск) ratingHtml += `<span class="bad
1124 ge bg-danger text-white me-1">КП: ${card.rating.кинопоиск}</sp
1125 an>`;
1126         if (card.rating.tmbd) ratingHtml += `<span class="badge bg
1127 -primary text-white">TMDB: ${card.rating.tmbd}</span>`;
1128     }
1129
1130     return `
1131     <div class="p-3 text-white card-media-custom" style="borde

```

```

    r-radius: 10px; background: rgba(88, 166, 255, 0.08); border:
    1px solid rgba(88, 166, 255, 0.2); max-width: 480px; margin-to
    p: 10px;">
1126     <h5 class="mb-1 text-info">${title}</h5>
1127     <div class="small text-muted mb-2">${subtitle} ${year ?
    `(${year})` : ''}</div>
1128     <div class="mb-2">${ratingHtml}</div>
1129     ${category ? `<div class="small text-muted mb-1" style="
    font-size: 0.8rem;"><strong>Категория:</strong> ${category}</d
    iv>` : ''}
1130     ${genres ? `<div class="small text-muted mb-1" style="fo
    nt-size: 0.8rem;"><strong>Жанры:</strong> ${genres}</div>` : '
    '}
1131     ${directors ? `<div class="small text-muted mb-1" style=
    "font-size: 0.8rem;"><strong>Режиссер:</strong> ${directors}</
    div>` : ''}
1132     ${cast ? `<div class="small text-muted mb-2" style="font
    -size: 0.8rem;"><strong>В ролях:</strong> ${cast}...</div>` :
    ''}
1133     ${plot ? `<p class="small text-secondary mb-2" style="fo
    nt-size: 0.85rem; line-height: 1.4;">${plot}</p>` : ''}
1134     ${whyWatch ? `<div class="small text-info mb-2" style="f
    ont-size: 0.8rem;"><strong>Рекомендация:</strong> ${whyWatch}<
    /div>` : ''}
1135     <div class="d-flex justify-content-end mt-2">
1136         <button class="btn btn-sm btn-primary rounded-pill px-
    3" onclick="playMovieDirect('${title.replace(/'/g, "\\')}')">
1137             <i class="bi bi-play-fill"></i> Запустить фильм
1138         </button>
1139     </div>
1140 </div>
1141 `;
1142 }
1143
1144 window.playMovieDirect = async (title) => {
1145     try {
1146         const r = await fetch('/api/media/by-title', {
1147             method: 'POST',

```

```

1148     headers: {'Content-Type': 'application/json'},
1149     body: JSON.stringify({title})
1150 });
1151 if (r.ok) {
1152     const data = await r.json();
1153     if (data.path) {
1154         // Expand player bar
1155         const body = document.getElementById('media-player-bod
1156 y');
1157         const btn = document.getElementById('btn-toggle-player
1158 -body');
1159         if (body) body.classList.remove('d-none');
1160         if (btn) btn.textContent = 'Свернуть плеер';
1161
1162         // Load direct stream or file path
1163         const isUrl = data.path.startsWith('http://') || data.
1164 path.startsWith('https://');
1165         const streamUrl = isUrl ? data.path : `/api/media/stre
1166 am?path=${encodeURIComponent(data.path)}`;
1167         loadCosmicVideo(streamUrl, data.title || title);
1168     } else {
1169         showFilmNotFoundCard(title);
1170     }
1171 } else {
1172     showFilmNotFoundCard(title);
1173 }
1174 };
1175
1176 function parseContentToHtml(text) {
1177     if (!text) return '';
1178
1179     // Clean markdown code blocks if the content is wrapped in t
1180     hem
1181     let cleaned = text.trim();

```

```

1181 | const codeBlockRegex = /^```(?:html|xml)?\s*([\s\S]*?)\s*```
    |$/i;
1182 | const match = cleaned.match(codeBlockRegex);
1183 | if (match) {
1184 |     cleaned = match[1];
1185 | }
1186 |
1187 | let html = cleaned;
1188 | if (window.marked && typeof window.marked.parse === 'functio
    |n') {
1189 |     html = window.marked.parse(cleaned);
1190 | } else {
1191 |     // Basic fallback for formatting if marked is not loaded
1192 |     html = cleaned
1193 |         .replace(/\*\*(.*?)\*\*/g, '<strong>$1</strong>')
1194 |         .replace(/\*(.*?)\*/g, '<em>$1</em>')
1195 |         .replace(/`([^\`]+)`/g, '<code>$1</code>')
1196 |         .replace(/\n/g, '<br>');
1197 | }
1198 | return replaceFilmTagsWithLinks(html);
1199 | }
1200 |
1201 | function addRagButton(botMessageEl, query, reply, voiceReply)
    | {
1202 |     const isAdminChat = !!document.getElementById('tab-rag') ||
    | window.location.pathname.includes('/admin') || window.location
    | .href.includes('/admin');
1203 |     if (!isAdminChat) return;
1204 |
1205 |     // Check if button already exists to avoid duplicates
1206 |     if (botMessageEl.querySelector('.rag-btn-container')) return
    | ;
1207 |
1208 |     const ragBtnContainer = document.createElement('div');
1209 |     ragBtnContainer.className = 'mt-2 text-end rag-btn-container
    | d-flex justify-content-end gap-1';
1210 |
1211 |     // 1. Button: Archive RAG

```

```
1212 |   const archiveBtn = document.createElement('button');
1213 |   archiveBtn.className = 'btn btn-xs btn-outline-secondary py-
1214 | 1 px-2';
1215 |   archiveBtn.style.fontSize = '0.75rem';
1216 |   archiveBtn.innerHTML = '📁 В архив RAG';
1217 |   archiveBtn.onclick = async () => {
1218 |     archiveBtn.disabled = true;
1219 |     instantBtn.disabled = true;
1220 |     archiveBtn.innerHTML = '📁 Сохранение...';
1221 |     try {
1222 |       const r = await fetch('/api/chat/save-rag', {
1223 |         method: 'POST',
1224 |         headers: {'Content-Type': 'application/json'},
1225 |         body: JSON.stringify({ query: query, chat_text: reply,
1226 | voice_text: voiceReply || reply })
1227 |       });
1228 |       if (r.ok) {
1229 |         archiveBtn.innerHTML = '📁 Сохранено в архив';
1230 |         archiveBtn.classList.remove('btn-outline-secondary');
1231 |         archiveBtn.classList.add('btn-success');
1232 |         instantBtn.remove();
1233 |       } else {
1234 |         const errData = await r.json().catch(() => ({}));
1235 |         alert('Ошибка при сохранении: ' + (errData.detail || r
1236 | .statusText));
1237 |         archiveBtn.innerHTML = '📁 Ошибка';
1238 |         archiveBtn.disabled = false;
1239 |         instantBtn.disabled = false;
1240 |       }
1241 |     } catch (err) {
1242 |       alert('Ошибка сети: ' + err.message);
1243 |       archiveBtn.innerHTML = '📁 Ошибка сети';
1244 |       archiveBtn.disabled = false;
1245 |       instantBtn.disabled = false;
1246 |     }
1247 |   };
1248 |
1249 |   // 2. Button: Instant RAG
```



```

1247 |   const instantBtn = document.createElement('button');
1248 |   instantBtn.className = 'btn btn-xs btn-outline-primary py-1
    | px-2';
1249 |   instantBtn.style.fontSize = '0.75rem';
1250 |   instantBtn.innerHTML = '⏸ Мгновенно в RAG';
1251 |   instantBtn.onclick = async () => {
1252 |       archiveBtn.disabled = true;
1253 |       instantBtn.disabled = true;
1254 |       instantBtn.innerHTML = '⏸ Индексация...';
1255 |       try {
1256 |           const r = await fetch('/api/chat/save-rag-instant', {
1257 |               method: 'POST',
1258 |               headers: {'Content-Type': 'application/json'},
1259 |               body: JSON.stringify({ query: query, chat_text: reply,
    | voice_text: voiceReply || reply })
1260 |           });
1261 |           if (r.ok) {
1262 |               instantBtn.innerHTML = '✅✅ Проиндексировано';
1263 |               instantBtn.classList.remove('btn-outline-primary');
1264 |               instantBtn.classList.add('btn-success');
1265 |               archiveBtn.remove();
1266 |           } else {
1267 |               const errData = await r.json().catch(() => ({}));
1268 |               alert('Ошибка при сохранении в RAG: ' + (errData.detai
    | l || r.statusText));
1269 |               instantBtn.innerHTML = '❌ Ошибка';
1270 |               archiveBtn.disabled = false;
1271 |               instantBtn.disabled = false;
1272 |           }
1273 |       } catch (err) {
1274 |           alert('Ошибка сети: ' + err.message);
1275 |           instantBtn.innerHTML = '❌ Ошибка сети';
1276 |           archiveBtn.disabled = false;
1277 |           instantBtn.disabled = false;
1278 |       }
1279 |   };
1280 |
1281 |   // 3. Button: Delete turn from history

```

```
1282 |   const deleteBtn = document.createElement('button');
1283 |   deleteBtn.className = 'btn btn-xs btn-outline-danger py-1 px
      | -2';
1284 |   deleteBtn.style.fontSize = '0.75rem';
1285 |   deleteBtn.innerHTML = '✖ Удалить';
1286 |   deleteBtn.onclick = () => {
1287 |     if (confirm('Удалить эту пару сообщений из истории диалога
      | ?')) {
1288 |       // Remove from DOM
1289 |       const prev = botMessageEl.previousElementSibling;
1290 |       if (prev && prev.classList.contains('user-message')) {
1291 |         prev.remove();
1292 |       }
1293 |       botMessageEl.remove();
1294 |
1295 |       // Remove from memory
1296 |       for (let i = 0; i < chatHistory.length - 1; i++) {
1297 |         if (chatHistory[i].role === 'user' && chatHistory[i].p
      | arts[0] === query &&
1298 |             chatHistory[i+1].role === 'model' && chatHistory[i
      | +1].parts[0] === reply) {
1299 |           chatHistory.splice(i, 2);
1300 |           break;
1301 |         }
1302 |       }
1303 |       // Update local storage
1304 |       try {
1305 |         localStorage.setItem('chat_history', JSON.stringify(ch
      | atHistory));
1306 |       } catch (e) {}
1307 |     }
1308 |   };
1309 |
1310 |   ragBtnContainer.appendChild/archiveBtn);
1311 |   ragBtnContainer.appendChild/instantBtn);
1312 |   ragBtnContainer.appendChild/deleteBtn);
1313 |   botMessageEl.appendChild(ragBtnContainer);
1314 | }
```

```

1315 |
1316 | function addMessage(sender, text, query = '') {
1317 |     const win = document.getElementById('chat-window');
1318 |     const el = document.createElement('div');
1319 |     el.className = 'message ' + (sender === 'user' ? 'user-messa
    | ge' : 'bot-message');
1320 |
1321 |     let displayText;
1322 |     if (typeof text === 'object' && text !== null) {
1323 |         displayText = window.chatService.formatMessage(text);
1324 |     } else {
1325 |         displayText = parseContentToHtml(text);
1326 |     }
1327 |
1328 |     el.innerHTML = `<strong>${sender === 'user' ? 'Вы' : 'Бот'}<
    | /strong> (${new Date().toLocaleTimeString()}: ${displayText}`
    | ;
1329 |     win.appendChild(el); win.scrollTop = win.scrollHeight;
1330 |
1331 |     if (sender === 'bot' && query) {
1332 |         addRagButton(el, query, text, text);
1333 |     }
1334 | }
1335 |
1336 | // — Film Tag Parsing & Media Player —————
    | _____
1337 |
1338 | const FILM_TAG_REGEX = /<film>(.*?)<\/film>/gi;
1339 |
1340 | function escapeHtml(str) {
1341 |     if (!str) return '';
1342 |     return String(str)
1343 |         .replace(/&/g, '&amp;')
1344 |         .replace(/</g, '&lt;')
1345 |         .replace(/>/g, '&gt;')
1346 |         .replace(/"/g, '&quot;')
1347 |         .replace(/'/g, '&#039;');
1348 | }

```

```

1349
1350 function replaceFilmTagsWithLinks(text) {
1351     if (!text) return '';
1352     return text.replace(FILM_TAG_REGEX, (match, title) => {
1353         const cleanTitle = title.replace(/'/g, "\\'");
1354         const safeTitle = escapeHtml(title);
1355         return `<span class="film-interactive-chip" title="Film: $
1356             {safeTitle}">` +
1357             `<span class="film-chip-title" onclick="playMovieDirect(
1358             '${cleanTitle}')">` +
1359             `<i class="bi bi-film me-1"></i>${safeTitle}` +
1360             `</span>` +
1361             `<button type="button" class="film-chip-btn film-chip-pl
1362             ay" onclick="playMovieDirect('${cleanTitle}')" title="Play in
1363             media player">` +
1364             `<i class="bi bi-play-fill"></i>` +
1365             `</button>` +
1366             `<button type="button" class="film-chip-btn film-chip-yo
1367             utube" onclick="window.openYouTubeForFilm('${cleanTitle}')" ti
1368             tle="Watch trailer/video on YouTube">` +
1369             `<i class="bi bi-youtube"></i>` +
1370             `</button>` +
1371             `<button type="button" class="film-chip-btn film-chip-on
1372             line" onclick="window.openOnlineCinemaForFilm('${cleanTitle}')"
1373             " title="Watch online (Yandex Video / Cinemas)">` +
1374             `<i class="bi bi-globe"></i>` +
1375             `</button>` +
1376             `<button type="button" class="film-chip-btn film-chip-to
1377             rrent" onclick="window.openTorrentsTab('${cleanTitle}')" title
1378             ="Search torrent releases">` +
1379             `<i class="bi bi-download"></i>` +
1380             `</button>` +
1381             `</span>`;
1382     });
1383 }
1384
1385 async function parseFilmTags(text, botMessageEl) {
1386     const matches = [...text.matchAll(FILM_TAG_REGEX)];

```

```

1377 | if (matches.length === 0) return;
1378 |
1379 | // Check first matched title
1380 | const filmTitle = matches[0][1].trim();
1381 | const found = await findAndShowMedia(filmTitle);
1382 | if (!found) {
1383 |     showFilmNotFoundCard(filmTitle, botMessageEl);
1384 | }
1385 | }
1386 |
1387 | async function findAndShowMedia(title) {
1388 |     try {
1389 |         const r = await fetch('/api/media/by-title', {
1390 |             method: 'POST',
1391 |             headers: {'Content-Type': 'application/json'},
1392 |             body: JSON.stringify({title})
1393 |         });
1394 |         if (!r.ok) return false;
1395 |         const data = await r.json();
1396 |         if (!data.path) return false;
1397 |
1398 |         currentMediaPath = data.path;
1399 |         const controls = document.getElementById('media-controls')
1400 |         ;
1401 |         if (controls) {
1402 |             document.getElementById('now-playing-title').textContent
1403 |             = data.title || title;
1404 |             controls.classList.remove('d-none');
1405 |         }
1406 |
1407 |         // Automatically trigger toggle for previewing
1408 |         const body = document.getElementById('media-player-body');
1409 |         const btn = document.getElementById('btn-toggle-player-bod
1410 |         y');
1411 |         if (body) body.classList.remove('d-none');
1412 |         if (btn) btn.textContent = 'Свернуть плеер';
1413 |
1414 |         const isUrl = data.path.startsWith('http://') || data.path

```

```

    .startsWith('https://');
1412     const streamUrl = isUrl ? data.path : `/api/media/stream?path=${encodeURIComponent(data.path)}`;
1413     loadCosmicVideo(streamUrl, data.title || title);
1414     return true;
1415 } catch (e) {
1416     console.error('findAndShowMedia:', e);
1417     return false;
1418 }
1419 }
1420
1421 function showFilmNotFoundCard(title, botMessageEl) {
1422     const targetParent = botMessageEl || document.getElementById('chat-window');
1423     if (!targetParent) return;
1424
1425     // Check for duplicate card for the same title
1426     const existingCard = targetParent.querySelector(`[data-film-card="${title.toLowerCase()}"]`);
1427     if (existingCard) return;
1428
1429     const safeId = 'torrent-res-' + Math.random().toString(36).substring(2, 9);
1430     const cleanTitle = title.replace(/'/g, "\\'");
1431
1432     const cardDiv = document.createElement('div');
1433     cardDiv.className = 'mt-2 film-not-found-card';
1434     cardDiv.setAttribute('data-film-card', title.toLowerCase());
1435     cardDiv.innerHTML = `
1436         <div class="card border-warning border-opacity-50 text-white shadow-sm" style="border-radius: 10px; background: rgba(33, 37, 41, 0.95);">
1437             <div class="card-body p-3">
1438                 <div class="d-flex align-items-center gap-2 mb-2">
1439                     <span class="fs-5 text-warning">⚠ </span>
1440                 </div>
1441                 <div class="fw-bold text-warning small">Фильм «${escapeHtml(title)}» не найден на локальных дисках</div>

```

```
1442 |         <div class="text-muted" style="font-size: 0.75rem;
    | ">Смотрите онлайн, на YouTube или найдите раздачу на торрент-т
    | реках:</div>
1443 |     </div>
1444 | </div>
1445 |     <div class="d-flex flex-wrap gap-2 mt-2">
1446 |         <button class="btn btn-sm btn-outline-danger py-1 px
    | -2" style="font-size: 0.8rem;" onclick="window.openYouTubeForF
    | ilm('${cleanTitle}')" title="Смотреть трейлер или видео">
1447 |             <i class="bi bi-youtube me-1"></i> YouTube
1448 |         </button>
1449 |         <button class="btn btn-sm btn-outline-info py-1 px-2
    | " style="font-size: 0.8rem;" onclick="window.openOnlineCinemaF
    | orFilm('${cleanTitle}')" title="Поиск онлайн видео">
1450 |             <i class="bi bi-globe me-1"></i> Онлайн-кинотеатры
1451 |         </button>
1452 |         <button class="btn btn-sm btn-outline-primary py-1 p
    | x-2" style="font-size: 0.8rem;" onclick="window.openKinopoiskF
    | orFilm('${cleanTitle}')" title="Открыть страницу на Кинопоиске
    | ">
1453 |             <i class="bi bi-film me-1"></i> Кинопоиск
1454 |         </button>
1455 |         <button class="btn btn-sm btn-outline-warning py-1 p
    | x-2" style="font-size: 0.8rem;" onclick="window.searchTorrentF
    | orFilm('${cleanTitle}', '${safeId}')" title="Искать раздачи">
1456 |             <i class="bi bi-magnet me-1"></i> Торренты
1457 |         </button>
1458 |         <button class="btn btn-sm btn-outline-secondary py-1
    | px-2" style="font-size: 0.8rem;" onclick="window.searchWebFor
    | Film('${cleanTitle}')" title="Спросить у AI">
1459 |             <i class="bi bi-search me-1"></i> Инфо в веб
1460 |         </button>
1461 |     </div>
1462 |     <div id="${safeId}" class="mt-2 d-none"></div>
1463 | </div>
1464 | </div>
1465 | `;
1466 |
```

```

1467 |   if (botMessageEl) {
1468 |       botMessageEl.appendChild(cardDiv);
1469 |   } else {
1470 |       targetParent.appendChild(cardDiv);
1471 |       targetParent.scrollTop = targetParent.scrollHeight;
1472 |   }
1473 | }
1474 |
1475 | window.searchTorrentForFilm = async (title, targetId) => {
1476 |   const container = document.getElementById(targetId);
1477 |   if (!container) return;
1478 |
1479 |   container.classList.remove('d-none');
1480 |   container.innerHTML = '<div class="text-muted small py-2"><span class="spinner-border spinner-border-sm me-2"></span>Поиск
раздач на трекерах (Rutracker, NNMClub)...</div>';
1481 |
1482 |   try {
1483 |       const res = await fetch(`/api/torrents/search?query=${encodeURIComponent(title)}`);
1484 |       if (!res.ok) throw new Error(`HTTP ${res.status}`);
1485 |       const data = await res.json();
1486 |       const torrents = data.torrents || data.results || (Array.isArray(data) ? data : []);
1487 |
1488 |       if (torrents.length === 0) {
1489 |           container.innerHTML = '<div class="small text-muted py-1">⚠ На трекерах раздач не найдено. Попробуйте поискать во вкладке «Торренты».</div>';
1490 |           return;
1491 |       }
1492 |
1493 |       let html = `<div class="small text-info mb-2 fw-semibold">Найдено раздач: ${torrents.length}</div><div class="list-group list-group-flush border-top border-secondary border-opacity-50">`;
1494 |       torrents.slice(0, 5).forEach(t => {
1495 |           const tTitle = t.title || t.name || 'Раздача';

```



```

1496 |         const tSize = t.size || t.size_str || '-';
1497 |         const tSeeds = t.seeds !== undefined ? t.seeds : '-';
1498 |         const tSource = t.source || t.tracker || 'Tracker';
1499 |         const tUrl = t.url || t.magnet || t.download_url || '';
1500 |
1501 |         html += `
1502 |             <div class="list-group-item bg-transparent text-white
1503 | border-secondary border-opacity-25 px-0 py-2 d-flex justify-co
1504 | ntent-between align-items-center">
1505 |                 <div style="max-width: 75%;">
1506 |                     <div class="fw-semibold text-truncate small" title
1507 | ="${escapeHtml(tTitle)}">${escapeHtml(tTitle)}</div>
1508 |                     <div class="text-muted" style="font-size: 0.75rem;
1509 | ">
1510 |                         <span class="badge bg-secondary text-white me-1"
1511 | >${escapeHtml(tSource)}</span>
1512 |                         □ ${escapeHtml(tSize)} | □ Сиды: ${escapeHtml(S
1513 | tring(tSeeds))}
1514 |                     </div>
1515 |                 </div>
1516 |                 <button class="btn btn-sm btn-outline-success downlo
1517 | ad-torrent-btn"
1518 |
1519 |                     data-url="${encodeURIComponent(tUrl)}"
1520 |                     data-source="${escapeHtml(tSource)}"
1521 |                     data-title="${escapeHtml(tTitle)}"
1522 |                     style="font-size: 0.75rem; white-space: nowr
1523 | ap;">
1524 |                     □ Скачать
1525 |                 </button>
1526 |             </div>
1527 |         `;
1528 |     });
1529 |     html += '</div>';
1530 |     container.innerHTML = html;
1531 | } catch (err) {
1532 |     container.innerHTML = `<div class="small text-danger">Ошиб
1533 | ка поиска торрентов: ${escapeHtml(err.message)}</div>`;
1534 | }

```

```
1525 };
1526
1527 window.openYouTubeForFilm = (title) => {
1528     const q = encodeURIComponent(`${title} фильм трейлер`);
1529     window.open(`https://www.youtube.com/results?search_query=${q}`, '_blank');
1530 };
1531
1532 window.openOnlineCinemaForFilm = (title) => {
1533     const q = encodeURIComponent(`${title} смотреть онлайн`);
1534     window.open(`https://yandex.ru/video/search?text=${q}`, '_blank');
1535 };
1536
1537 window.openKinopoiskForFilm = (title) => {
1538     const q = encodeURIComponent(title);
1539     window.open(`https://www.kinopoisk.ru/index.php?kp_query=${q}`, '_blank');
1540 };
1541
1542 window.searchWebForFilm = (title) => {
1543     const msgInput = document.getElementById('message-input');
1544     if (msgInput) {
1545         msgInput.value = `найди в интернете информацию о фильме ${title}`;
1546         const sendBtn = document.getElementById('send-button');
1547         if (sendBtn) sendBtn.click();
1548     }
1549 };
1550
1551 window.openTorrentsTab = (title) => {
1552     const tabBtn = document.querySelector('[data-bs-target="#tab-torrents"]');
1553     if (tabBtn) {
1554         tabBtn.click();
1555     } else {
1556         window.searchWebForFilm(`скачать торрент ${title}`);
1557     }
```

```
1558 };
1559
1560 function openMediaPlayer() {
1561     if (!currentMediaPath) return;
1562     // Открываем в новой вкладке с локальным плеером
1563     const encoded = encodeURIComponent(currentMediaPath);
1564     window.open(`/html/player/index.html?file=${encoded}`, '_blank');
1565 }
1566
1567 function closeMediaControls() {
1568     const controls = document.getElementById('media-controls');
1569     controls.classList.add('d-none');
1570     currentMediaPath = null;
1571 }
1572
1573 // — DEBUG MODE HELPERS —————
1574
1575 async function loadSystemInstruction() {
1576     try {
1577         const response = await fetch('/auth/settings');
1578         if (response.ok) {
1579             const settings = await response.json();
1580             systemInstruction = settings.system_instruction || null;
1581         }
1582     } catch (e) {
1583         console.error('Failed to load system instruction:', e);
1584         systemInstruction = null;
1585     }
1586 }
1587
```

index.html

```
1 <!-- Link CosmicPlayer custom style -->
2 <link rel="stylesheet" href="/html/cosmicplayer/styles.css">
3 <!-- HLS.js library for streaming M3U8 inside custom HTML5 vid
   eo player -->
4 <script src="https://cdn.jsdelivr.net/npm/hls.js@latest"></scr
   ipt>
5
6 <div class="cosmic-player-tab">
7     <!-- Sidebar: Video URL list / Library / Settings -->
8     <aside class="cosmic-sidebar">
9         <div class="logo">
10             <span class="logo-icon">⬅ </span>
11             <h2>CosmicPlayer</h2>
12         </div>
13
14         <!-- Paste link panel -->
15         <div class="url-input-section">
16             <h3>Вставить ссылку</h3>
17             <form id="stream-form">
18                 <input type="url" id="stream-url" placeholder=
   "Ссылка (YouTube, VK, RuTube, Rezka...)" required>
19                 <button type="submit" class="btn btn-glow">Заг
   рузить видео</button>
20             </form>
21         </div>
22
23         <!-- Supported sources badges -->
24         <div class="sources-panel">
25             <h4>Совместимые источники</h4>
26             <div class="sources-grid">
27                 <span class="badge" data-provider="youtube">Yo
   uTube</span>
28                 <span class="badge" data-provider="vk">VK Vide
   o</span>
29                 <span class="badge" data-provider="rutube">RuT
   ube</span>
```

```
30 |         <span class="badge" data-provider="hdrezka">HD
    | Rezka</span>
31 |         <span class="badge" data-provider="seasonvar">
    | SeasonVar</span>
32 |         <span class="badge" data-provider="kinogo">Kin
    | ogo</span>
33 |         <span class="badge" data-provider="direct">Dir
    | ect HLS/MP4</span>
34 |     </div>
35 | </div>
36 |
37 | <!-- History List -->
38 | <div class="history-section">
39 |     <div class="section-header">
40 |         <h4>История воспроизведения</h4>
41 |         <button id="clear-history" class="btn-text">Оч
    | истить</button>
42 |     </div>
43 |     <div id="history-list" class="history-list">
44 |         <div class="empty-state">Нет недавно проигранн
    | ых видео</div>
45 |     </div>
46 | </div>
47 | </aside>
48 |
49 | <!-- Main Content Area: Player and Description -->
50 | <main class="main-content">
51 |     <!-- Movie Screen and Ambience -->
52 |     <div class="player-container" id="player-container">
53 |         <!-- Background Ambient Glow -->
54 |         <div class="ambient-glow" id="ambient-glow"></div>
55 |
56 |         <!-- Display Iframe (for embeds) -->
57 |         <iframe id="embed-player" class="player-element hi
    | dden" allowfullscreen allow="autoplay; encrypted-media"></ifra
    | me>
58 |
59 |         <!-- Custom HTML5 Player (for direct video feeds)
```

```

-->
60         <div id="html5-player-wrapper" class="player-eleme
nt hidden">
61             <video id="html5-video" class="html5-video-el"
></video>
62             <!-- Custom Player Skin -->
63             <div class="custom-controls">
64                 <div class="progress-container" id="progre
ss-container">
65                     <div class="progress-bar" id="progress
-bar">
66                         <div class="progress-hover"></div>
67                     </div>
68                 </div>
69                 <div class="controls-row">
70                     <div class="controls-left">
71                         <button class="ctrl-btn" id="play-
pause-btn">▶ </button>
72                         <button class="ctrl-btn" id="stop-
btn">■</button>
73                         <div class="time-display">
74                             <span id="current-time">0:00</
span> / <span id="duration-time">0:00</span>
75                         </div>
76                     </div>
77                     <div class="controls-right">
78                         <div class="volume-container">
79                             <button class="ctrl-btn" id="m
ute-btn">□ </button>
80                             <input type="range" id="volume
-slider" min="0" max="1" step="0.05" value="1">
81                         </div>
82                         <select id="speed-select" class="s
peed-dropdown">
83                             <option value="0.5">0.5x</opti
on>
84                             <option value="1" selected>1.0
x</option>

```

```

85 |         <option value="1.5">1.5x</opti
on>
86 |         <option value="2">2.0x</option
>
87 |     </select>
88 |     <button class="ctrl-btn" id="fulls
creen-btn">⏮ </button>
89 |         </div>
90 |     </div>
91 | </div>
92 | </div>
93 |
94 | <!-- Initial Empty State -->
95 | <div id="player-placeholder" class="player-placeho
lder">
96 |     <div class="placeholder-content">
97 |         <span class="play-icon">⏮ </span>
98 |         <h3>Видео не загружено</h3>
99 |         <p>Вставьте ссылку на видео или выберите о
дин из демонстрационных источников слева.</p>
100 |     </div>
101 | </div>
102 | </div>
103 |
104 | <!-- Current Video Information -->
105 | <div class="video-info-card glass-card">
106 |     <div class="info-header">
107 |         <span class="provider-tag" id="current-provide
r-tag">Offline</span>
108 |         <h3 id="video-title">Добро пожаловать в Cosmic
Player</h3>
109 |     </div>
110 |     <div class="info-body">
111 |         <p id="video-source-url">Пожалуйста, введите U
RL видео для воспроизведения.</p>
112 |     </div>
113 | </div>
114 | </main>

```

115 | </div>
116 |

main.js

```
1 window.initCosmicplayerTab = function() {
2     console.log("CosmicPlayer tab initialized");
3
4     // Elements
5     const streamForm = document.getElementById('stream-form');
6     const streamUrlInput = document.getElementById('stream-url
7     ');
8     const embedPlayer = document.getElementById('embed-player'
9     );
10    const html5PlayerWrapper = document.getElementById('html5-
11    player-wrapper');
12    const html5Video = document.getElementById('html5-video');
13    const playerPlaceholder = document.getElementById('player-
14    placeholder');
15    const videoTitle = document.getElementById('video-title');
16    const videoSourceUrl = document.getElementById('video-sour
17    ce-url');
18    const currentProviderTag = document.getElementById('curren
19    t-provider-tag');
20    const historyList = document.getElementById('history-list'
21    );
22    const clearHistoryBtn = document.getElementById('clear-his
23    tory');
24    const badges = document.querySelectorAll('.cosmic-player-t
25    ab .badge');
26    const ambientGlow = document.getElementById('ambient-glow'
27    );
28
29    // Custom controls
30    const playPauseBtn = document.getElementById('play-pause-b
31    tn');
32    const stopBtn = document.getElementById('stop-btn');
33    const currentTimeDisplay = document.getElementById('curren
34    t-time');
35    const durationTimeDisplay = document.getElementById('durat
36    ion-time');
```

```

24     const progressContainer = document.getElementById('progress-container');
25     const progressBar = document.getElementById('progress-bar');
26     const progressHover = document.querySelector('.progress-hover');
27     const muteBtn = document.getElementById('mute-btn');
28     const volumeSlider = document.getElementById('volume-slider');
29     const speedSelect = document.getElementById('speed-select');
30     const fullscreenBtn = document.getElementById('fullscreen-btn');
31
32     let hlsInstance = null;
33     let history = JSON.parse(localStorage.getItem('cosmic_player_history') || '[]');
34
35     // Dynamic ambient glow colors per provider
36     const providerColors = {
37         youtube: 'radial-gradient(circle, rgba(255, 0, 0, 0.25) 0%, transparent 60%)',
38         vk: 'radial-gradient(circle, rgba(74, 118, 168, 0.25) 0%, transparent 60%)',
39         rutube: 'radial-gradient(circle, rgba(235, 95, 30, 0.25) 0%, transparent 60%)',
40         hdrezka: 'radial-gradient(circle, rgba(242, 197, 17, 0.2) 0%, transparent 60%)',
41         seasonvar: 'radial-gradient(circle, rgba(0, 242, 254, 0.2) 0%, transparent 60%)',
42         kinogo: 'radial-gradient(circle, rgba(155, 81, 224, 0.2) 0%, transparent 60%)',
43         direct: 'radial-gradient(circle, rgba(121, 40, 202, 0.25) 0%, transparent 60%)',
44         offline: 'radial-gradient(circle, rgba(255, 255, 255, 0.05) 0%, transparent 60%)'
45     };
46

```

```

47 // Load history on start
48 renderHistory();
49
50 // Form submission
51 streamForm.addEventListener('submit', (e) => {
52     e.preventDefault();
53     const url = streamUrlInput.value.trim();
54     if (url) {
55         loadVideo(url);
56     }
57 });
58
59 // Clear history
60 clearHistoryBtn.addEventListener('click', () => {
61     history = [];
62     localStorage.setItem('cosmic_player_history', JSON.str
ingify(history));
63     renderHistory();
64 });
65
66 // Router and url parser
67 function loadVideo(url, title = null) {
68     resetPlayers();
69
70     const providerInfo = parseVideoUrl(url);
71     const resolvedTitle = title || providerInfo.defaultTit
le;
72
73     highlightBadge(providerInfo.provider);
74
75     // Update info panel
76     videoTitle.textContent = resolvedTitle;
77     videoSourceUrl.textContent = url;
78     currentProviderTag.textContent = providerInfo.provider
;
79
80     // Update ambient glow color
81     ambientGlow.style.background = providerColors[provider

```

```

    Info.provider] || providerColors.direct;
82
83     // Route to the correct player engine
84     if (providerInfo.type === 'embed') {
85         embedPlayer.src = providerInfo.embedUrl;
86         embedPlayer.classList.remove('hidden');
87     } else if (providerInfo.type === 'html5') {
88         initHTML5Player(url);
89         html5PlayerWrapper.classList.remove('hidden');
90     } else if (providerInfo.type === 'iframe_fallback') {
91         embedPlayer.src = url;
92         embedPlayer.classList.remove('hidden');
93     }
94
95     // Add to history
96     addToHistory(url, resolvedTitle, providerInfo.provider
    );
97 }
98
99 function resetPlayers() {
100     embedPlayer.classList.add('hidden');
101     embedPlayer.src = '';
102     html5PlayerWrapper.classList.add('hidden');
103     playerPlaceholder.classList.add('hidden');
104
105     html5Video.pause();
106     html5Video.src = '';
107
108     if (hlsInstance) {
109         hlsInstance.destroy();
110         hlsInstance = null;
111     }
112 }
113
114 // Video URL Parser
115 function parseVideoUrl(url) {
116     // YouTube Matcher
117     const ytReg = /(?:youtube\.com\/(?:[^\./]+\./+\/|(?:(v|e

```

```

    (? : mbed ) ? ) \ / | . * [ ? & ] v = ) | youtu \ . be \ / ) ( [ ^ " & ? \ \ s ] { 11 } ) / ;
118     const ytMatch = url.match(ytReg);
119     if (ytMatch) {
120         return {
121             provider: 'youtube',
122             type: 'embed',
123             embedUrl: `https://www.youtube.com/embed/${ytM
atch[1]}?autoplay=1&rel=0`,
124             defaultTitle: `YouTube Video (${ytMatch[1]})`
125         };
126     }
127
128     // VK Matcher
129     const vkReg = /vk \. com \ / video ( - ? \ d + ) _ ( \ d + ) / ;
130     const vkMatch = url.match(vkReg);
131     const vkEmbedReg = /oid = ( - ? \ d + ) & id = ( \ d + ) ( ? : & hash = ( [ a - f
0 - 9 ] + ) ) ? / ;
132     const vkEmbedMatch = url.match(vkEmbedReg);
133
134     if (vkMatch) {
135         return {
136             provider: 'vk',
137             type: 'embed',
138             embedUrl: `https://vk.com/video_ext.php?oid=${{
vkMatch[1]}&id=${vkMatch[2]}&autoplay=1`,
139             defaultTitle: `VK Video (${vkMatch[1]})_${vkMat
ch[2]})`
140         };
141     } else if (vkEmbedMatch) {
142         const hashParam = vkEmbedMatch[3] ? `&hash=${vkEmb
edMatch[3]}` : '';
143         return {
144             provider: 'vk',
145             type: 'embed',
146             embedUrl: `https://vk.com/video_ext.php?oid=${{
vkEmbedMatch[1]}&id=${vkEmbedMatch[2]}${hashParam}&autoplay=1`
147             ,
            defaultTitle: `VK Embed Video`

```

```

148         };
149     }
150
151     // RuTube Matcher
152     const rutubeReg = /rutube\.ru\/(?:video|play\/embed)\/
153     ([a-f0-9]{32})/;
154     const rutubeMatch = url.match(rutubeReg);
155     if (rutubeMatch) {
156         return {
157             provider: 'rutube',
158             type: 'embed',
159             embedUrl: `https://rutube.ru/play/embed/${rutu
160             beMatch[1]}?autoplay=1`,
161             defaultTitle: `RuTube Video`
162         };
163     }
164
165     // Direct Stream (MP4/HLS)
166     const isHLS = url.includes('.m3u8');
167     const isMP4 = url.includes('.mp4');
168     if (isHLS || isMP4) {
169         return {
170             provider: 'direct',
171             type: 'html5',
172             defaultTitle: isHLS ? 'Прямой HLS поток' : 'Пр
173             ямой MP4 файл'
174         };
175     }
176
177     // SeasonVar
178     if (url.includes('seasonvar.ru')) {
179         return {
180             provider: 'seasonvar',
181             type: 'iframe_fallback',
182             defaultTitle: 'СезонВар плеер'
183         };
184     }

```

```

183 |         // HDRezka
184 |         if (url.includes('rezka.ag') || url.includes('hdrezka'
    |     )) {
185 |             return {
186 |                 provider: 'hdrezka',
187 |                 type: 'iframe_fallback',
188 |                 defaultTitle: 'Резка плеер'
189 |             };
190 |         }
191 |
192 |         // Kinogo
193 |         if (url.includes('kinogo')) {
194 |             return {
195 |                 provider: 'kinogo',
196 |                 type: 'iframe_fallback',
197 |                 defaultTitle: 'Киного плеер'
198 |             };
199 |         }
200 |
201 |         // Default fallback
202 |         return {
203 |             provider: 'direct',
204 |             type: 'html5',
205 |             defaultTitle: 'Прямой видеоисточник'
206 |         };
207 |     }
208 |
209 |     function initHTML5Player(url) {
210 |         if (url.includes('.m3u8')) {
211 |             if (typeof Hls !== 'undefined' && Hls.isSupported(
    |     )) {
212 |                 hlsInstance = new Hls();
213 |                 hlsInstance.loadSource(url);
214 |                 hlsInstance.attachMedia(html5Video);
215 |                 hlsInstance.on(Hls.Events.MANIFEST_PARSED, ()
    |     => {
216 |                     html5Video.play();
217 |                 });

```

```
218         } else if (html5Video.canPlayType('application/vnd
    .apple.mpegurl')) {
219             html5Video.src = url;
220             html5Video.play();
221         }
222     } else {
223         html5Video.src = url;
224         html5Video.play();
225     }
226
227     playPauseBtn.textContent = '▶';
228 }
229
230 // Video events
231 html5Video.addEventListener('timeupdate', () => {
232     const current = html5Video.currentTime;
233     const total = html5Video.duration || 0;
234
235     if (total > 0) {
236         const pct = (current / total) * 100;
237         progressBar.style.width = `${pct}%`;
238     }
239
240     currentTimeDisplay.textContent = formatTime(current);
241     durationTimeDisplay.textContent = formatTime(total);
242 });
243
244 html5Video.addEventListener('loadedmetadata', () => {
245     durationTimeDisplay.textContent = formatTime(html5Vide
o.duration);
246 });
247
248 playPauseBtn.addEventListener('click', () => {
249     if (html5Video.paused) {
250         html5Video.play();
251         playPauseBtn.textContent = '▶';
252     } else {
253         html5Video.pause();
```



```
254 |         playPauseBtn.textContent = '▶';
255 |     }
256 | });
257 |
258 | stopBtn.addEventListener('click', () => {
259 |     html5Video.pause();
260 |     html5Video.currentTime = 0;
261 |     playPauseBtn.textContent = '▶';
262 | });
263 |
264 | volumeSlider.addEventListener('input', (e) => {
265 |     const vol = e.target.value;
266 |     html5Video.volume = vol;
267 |     if (vol === '0') {
268 |         muteBtn.textContent = '🔇';
269 |     } else {
270 |         muteBtn.textContent = '🔊';
271 |     }
272 | });
273 |
274 | muteBtn.addEventListener('click', () => {
275 |     if (html5Video.muted) {
276 |         html5Video.muted = false;
277 |         volumeSlider.value = html5Video.volume;
278 |         muteBtn.textContent = '🔊';
279 |     } else {
280 |         html5Video.muted = true;
281 |         volumeSlider.value = 0;
282 |         muteBtn.textContent = '🔇';
283 |     }
284 | });
285 |
286 | speedSelect.addEventListener('change', (e) => {
287 |     html5Video.playbackRate = parseFloat(e.target.value);
288 | });
289 |
290 | progressContainer.addEventListener('click', (e) => {
291 |     const rect = progressContainer.getBoundingClientRect()
```

```

    ;
292     const clickX = e.clientX - rect.left;
293     const width = rect.width;
294     const pct = clickX / width;
295
296     html5Video.currentTime = pct * html5Video.duration;
297 });
298
299 progressContainer.addEventListener('mousemove', (e) => {
300     const rect = progressContainer.getBoundingClientRect()
    ;
301     const hoverX = e.clientX - rect.left;
302     const pct = hoverX / rect.width;
303     progressHover.style.width = `${pct * 100}%`;
304 });
305
306 progressContainer.addEventListener('mouseleave', () => {
307     progressHover.style.width = '0%';
308 });
309
310 fullscreenBtn.addEventListener('click', () => {
311     if (!document.fullscreenElement) {
312         html5PlayerWrapper.requestFullscreen().catch(err =
    > {
313             console.error(`Error enabling fullscreen: ${er
    r.message}`);
314         });
315     } else {
316         document.exitFullscreen();
317     }
318 });
319
320 function formatTime(sec) {
321     if (isNaN(sec)) return '0:00';
322     const mins = Math.floor(sec / 60);
323     const secs = Math.floor(sec % 60);
324     return `${mins}:${secs < 10 ? '0' : ''}${secs}`;
325 }

```

```

326
327     function highlightBadge(provider) {
328         badges.forEach(badge => {
329             if (badge.getAttribute('data-provider') === provid
er) {
330                 badge.classList.add('active');
331             } else {
332                 badge.classList.remove('active');
333             }
334         });
335     }
336
337     function addToHistory(url, title, provider) {
338         history = history.filter(item => item.url !== url);
339
340         history.unshift({
341             url,
342             title,
343             provider,
344             timestamp: new Date().toLocaleTimeString([], { hou
r: '2-digit', minute: '2-digit' })
345         });
346
347         if (history.length > 8) history.pop();
348
349         localStorage.setItem('cosmic_player_history', JSON.str
ingify(history));
350         renderHistory();
351     }
352
353     function renderHistory() {
354         historyList.innerHTML = '';
355         if (history.length === 0) {
356             historyList.innerHTML = '<div class="empty-state">
Нет недавно проигранных видео</div>';
357             return;
358         }
359

```

```

360 |         history.forEach(item => {
361 |             const el = document.createElement('div');
362 |             el.className = 'history-item';
363 |             el.innerHTML = `
364 |                 <div class="history-title" title="${item.title}
    |             ">${item.title}</div>
365 |                 <div class="history-meta">
366 |                     <span>${item.provider.toUpperCase()}</span>
    |             >
367 |                     <span>${item.timestamp}</span>
368 |                 </div>
369 |             `;
370 |             el.addEventListener('click', () => {
371 |                 streamUrlInput.value = item.url;
372 |                 loadVideo(item.url, item.title);
373 |             });
374 |             historyList.appendChild(el);
375 |         });
376 |     }
377 |
378 |     badges.forEach(badge => {
379 |         badge.addEventListener('click', () => {
380 |             const provider = badge.getAttribute('data-provider
    |         ');
381 |             let demoUrl = '';
382 |
383 |             switch (provider) {
384 |                 case 'youtube':
385 |                     demoUrl = 'https://www.youtube.com/watch?v
    | =aqz-KE-bpKQ';
386 |                     break;
387 |                 case 'vk':
388 |                     demoUrl = 'https://vk.com/video-22822305_4
    | 56239018';
389 |                     break;
390 |                 case 'rutube':
391 |                     demoUrl = 'https://rutube.ru/video/e7cfcb8
    | cb4310d54026fb4bd56e828d1/';

```

```
392 |             break;
393 |         case 'direct':
394 |             demoUrl = 'https://test-streams.mux.dev/x3
| 6xhzz/x36xhzz.m3u8';
395 |             break;
396 |         case 'seasonvar':
397 |             demoUrl = 'http://seasonvar.ru';
398 |             break;
399 |         case 'hdrezka':
400 |             demoUrl = 'https://rezka.ag';
401 |             break;
402 |         case 'kinogo':
403 |             demoUrl = 'https://kinogo.biz';
404 |             break;
405 |     }
406 |
407 |     if (demoUrl) {
408 |         streamUrlInput.value = demoUrl;
409 |         loadVideo(demoUrl);
410 |     }
411 | });
412 | });
413 | };
414 |
```

styles.css

```
1 | .cosmic-player-tab {
2 |     --primary: #7928ca;
3 |     --primary-glow: #ff007a;
4 |     --primary-gradient: linear-gradient(135deg, #7928ca 0%, #f
    f007a 100%);
5 |     --panel-border: rgba(255, 255, 255, 0.08);
6 |     --bg-dark: #0a0714;
7 |     --text-muted: #a0aec0;
8 |     --transition: all 0.3s cubic-bezier(0.25, 0.8, 0.25, 1);
9 |
10 |     display: flex;
11 |     width: 100%;
12 |     height: calc(100vh - 100px);
13 |     background: rgba(20, 16, 36, 0.6);
14 |     backdrop-filter: blur(25px);
15 |     -webkit-backdrop-filter: blur(25px);
16 |     border: 1px solid var(--panel-border);
17 |     border-radius: 20px;
18 |     overflow: hidden;
19 |     margin-top: 10px;
20 | }
21 |
22 | .cosmic-sidebar {
23 |     width: 300px;
24 |     background: rgba(14, 10, 26, 0.75);
25 |     border-right: 1px solid var(--panel-border);
26 |     padding: 25px 20px;
27 |     display: flex;
28 |     flex-direction: column;
29 |     gap: 20px;
30 |     overflow-y: auto;
31 | }
32 |
33 | .cosmic-sidebar .logo {
34 |     display: flex;
35 |     align-items: center;
```

```
36 |     gap: 10px;
37 | }
38 |
39 | .cosmic-sidebar .logo h2 {
40 |     font-size: 1.3rem;
41 |     font-weight: 700;
42 |     margin: 0;
43 |     background: var(--primary-gradient);
44 |     -webkit-background-clip: text;
45 |     -webkit-text-fill-color: transparent;
46 | }
47 |
48 | .cosmic-sidebar .logo-icon {
49 |     font-size: 1.6rem;
50 |     filter: drop-shadow(0 0 8px rgba(255, 0, 122, 0.5));
51 | }
52 |
53 | .cosmic-player-tab .url-input-section h3,
54 | .cosmic-player-tab .sources-panel h4,
55 | .cosmic-player-tab .history-section h4 {
56 |     font-size: 0.8rem;
57 |     font-weight: 600;
58 |     text-transform: uppercase;
59 |     letter-spacing: 0.8px;
60 |     color: var(--text-muted);
61 |     margin-bottom: 10px;
62 | }
63 |
64 | #stream-form {
65 |     display: flex;
66 |     flex-direction: column;
67 |     gap: 10px;
68 | }
69 |
70 | #stream-url {
71 |     background: rgba(0, 0, 0, 0.4);
72 |     border: 1px solid var(--panel-border);
73 |     border-radius: 8px;
```

```
74 | padding: 10px 12px;
75 | color: #fff;
76 | font-size: 0.85rem;
77 | outline: none;
78 | transition: var(--transition);
79 | }
80 |
81 | #stream-url:focus {
82 |     border-color: var(--primary-glow);
83 |     box-shadow: 0 0 10px rgba(255, 0, 122, 0.2);
84 | }
85 |
86 | .btn-glow {
87 |     background: var(--primary-gradient);
88 |     color: white;
89 |     border: none;
90 |     border-radius: 8px;
91 |     padding: 10px 15px;
92 |     font-size: 0.85rem;
93 |     font-weight: 600;
94 |     cursor: pointer;
95 |     transition: var(--transition);
96 | }
97 |
98 | .btn-glow:hover {
99 |     transform: translateY(-1px);
100 |     box-shadow: 0 4px 15px rgba(255, 0, 122, 0.4);
101 | }
102 |
103 | .sources-grid {
104 |     display: flex;
105 |     flex-wrap: wrap;
106 |     gap: 6px;
107 | }
108 |
109 | .cosmic-player-tab .badge {
110 |     background: rgba(255, 255, 255, 0.08);
111 |     border: 1px solid var(--panel-border);
```



```
112 |     border-radius: 6px;
113 |     padding: 5px 10px;
114 |     font-size: 0.7rem;
115 |     font-weight: 500;
116 |     color: #e6edf3;
117 |     transition: var(--transition);
118 |     cursor: pointer;
119 | }
120 |
121 | .cosmic-player-tab .badge:hover,
122 | .cosmic-player-tab .badge.active {
123 |     color: white;
124 |     background: rgba(121, 40, 202, 0.2);
125 |     border-color: var(--primary);
126 | }
127 |
128 | .history-section {
129 |     flex-grow: 1;
130 |     display: flex;
131 |     flex-direction: column;
132 | }
133 |
134 | .section-header {
135 |     display: flex;
136 |     justify-content: space-between;
137 |     align-items: center;
138 |     margin-bottom: 10px;
139 | }
140 |
141 | .btn-text {
142 |     background: none;
143 |     border: none;
144 |     color: var(--text-muted);
145 |     font-size: 0.75rem;
146 |     cursor: pointer;
147 | }
148 |
149 | .history-list {
```

```
150 |     display: flex;
151 |     flex-direction: column;
152 |     gap: 6px;
153 |     overflow-y: auto;
154 |     max-height: 200px;
155 | }
156 |
157 | .history-item {
158 |     background: rgba(255, 255, 255, 0.01);
159 |     border: 1px solid var(--panel-border);
160 |     border-radius: 8px;
161 |     padding: 8px 12px;
162 |     cursor: pointer;
163 |     display: flex;
164 |     flex-direction: column;
165 |     gap: 2px;
166 |     transition: var(--transition);
167 | }
168 |
169 | .history-item:hover {
170 |     background: rgba(255, 255, 255, 0.04);
171 | }
172 |
173 | .history-title {
174 |     font-size: 0.8rem;
175 |     font-weight: 500;
176 |     white-space: nowrap;
177 |     overflow: hidden;
178 |     text-overflow: ellipsis;
179 | }
180 |
181 | .history-meta {
182 |     display: flex;
183 |     justify-content: space-between;
184 |     font-size: 0.65rem;
185 |     color: var(--text-muted);
186 | }
187 |
```

```
188 /* Main Panel Layout */
189 .main-content {
190     flex-grow: 1;
191     display: flex;
192     flex-direction: column;
193     padding: 25px;
194     gap: 20px;
195     background: rgba(10, 7, 20, 0.2);
196 }
197
198 .player-container {
199     position: relative;
200     width: 100%;
201     height: 60%;
202     min-height: 380px;
203     background: #000;
204     border-radius: 15px;
205     overflow: hidden;
206     border: 1px solid var(--panel-border);
207 }
208
209 .ambient-glow {
210     position: absolute;
211     top: -20%;
212     left: -20%;
213     width: 140%;
214     height: 140%;
215     background: radial-gradient(circle, rgba(121, 40, 202, 0.2
216     ) 0%, transparent 60%);
217     filter: blur(50px);
218     z-index: 0;
219     pointer-events: none;
220     transition: background 1s ease;
221 }
222
223 .player-element {
224     position: relative;
225     width: 100%;
```

```
225 |     height: 100%;
226 |     border: none;
227 |     z-index: 2;
228 | }
229 |
230 | .hidden {
231 |     display: none !important;
232 | }
233 |
234 | .html5-video-el {
235 |     width: 100%;
236 |     height: 100%;
237 |     object-fit: contain;
238 | }
239 |
240 | #html5-player-wrapper {
241 |     position: relative;
242 | }
243 |
244 | .custom-controls {
245 |     position: absolute;
246 |     bottom: 0;
247 |     left: 0;
248 |     right: 0;
249 |     background: linear-gradient(to top, rgba(0, 0, 0, 0.9) 0%,
    | rgba(0, 0, 0, 0.4) 70%, transparent 100%);
250 |     padding: 15px 20px;
251 |     display: flex;
252 |     flex-direction: column;
253 |     gap: 10px;
254 |     opacity: 0;
255 |     transition: opacity 0.3s ease;
256 |     z-index: 10;
257 | }
258 |
259 | #html5-player-wrapper:hover .custom-controls {
260 |     opacity: 1;
261 | }
```

```
262 |
263 | .progress-container {
264 |     height: 4px;
265 |     background: rgba(255, 255, 255, 0.2);
266 |     border-radius: 2px;
267 |     cursor: pointer;
268 |     position: relative;
269 | }
270 |
271 | .progress-bar {
272 |     height: 100%;
273 |     background: var(--primary-gradient);
274 |     width: 0%;
275 |     border-radius: 2px;
276 | }
277 |
278 | .progress-hover {
279 |     position: absolute;
280 |     top: 0;
281 |     left: 0;
282 |     height: 100%;
283 |     background: rgba(255, 255, 255, 0.15);
284 |     width: 0%;
285 |     pointer-events: none;
286 | }
287 |
288 | .controls-row {
289 |     display: flex;
290 |     justify-content: space-between;
291 |     align-items: center;
292 | }
293 |
294 | .controls-left, .controls-right {
295 |     display: flex;
296 |     align-items: center;
297 |     gap: 12px;
298 | }
299 |
```

```
300 | .ctrl-btn {
301 |     background: none;
302 |     border: none;
303 |     color: white;
304 |     font-size: 1rem;
305 |     cursor: pointer;
306 | }
307 |
308 | .time-display {
309 |     font-size: 0.75rem;
310 |     color: var(--text-muted);
311 | }
312 |
313 | .volume-container {
314 |     display: flex;
315 |     align-items: center;
316 |     gap: 6px;
317 | }
318 |
319 | #volume-slider {
320 |     width: 60px;
321 |     height: 3px;
322 |     background: rgba(255, 255, 255, 0.2);
323 |     outline: none;
324 |     accent-color: var(--primary-glow);
325 |     cursor: pointer;
326 | }
327 |
328 | .speed-dropdown {
329 |     background: rgba(255, 255, 255, 0.1);
330 |     border: 1px solid rgba(255, 255, 255, 0.2);
331 |     color: white;
332 |     border-radius: 4px;
333 |     padding: 2px 6px;
334 |     font-size: 0.75rem;
335 |     outline: none;
336 |     cursor: pointer;
337 | }
```

```
338 |
339 | .player-placeholder {
340 |     position: absolute;
341 |     top: 0;
342 |     left: 0;
343 |     width: 100%;
344 |     height: 100%;
345 |     display: flex;
346 |     justify-content: center;
347 |     align-items: center;
348 |     z-index: 1;
349 |     background: rgba(10, 7, 20, 0.95);
350 | }
351 |
352 | .placeholder-content {
353 |     text-align: center;
354 |     max-width: 320px;
355 | }
356 |
357 | .placeholder-content .play-icon {
358 |     font-size: 3rem;
359 |     display: block;
360 |     margin-bottom: 12px;
361 | }
362 |
363 | .placeholder-content h3 {
364 |     font-size: 1.15rem;
365 |     font-weight: 600;
366 |     margin-bottom: 8px;
367 | }
368 |
369 | .placeholder-content p {
370 |     color: var(--text-muted);
371 |     font-size: 0.8rem;
372 |     line-height: 1.4;
373 | }
374 |
375 | .glass-card {
```

```
376 |     background: rgba(255, 255, 255, 0.02);
377 |     border: 1px solid var(--panel-border);
378 |     border-radius: 12px;
379 |     padding: 18px;
380 | }
381 |
382 | .video-info-card {
383 |     flex-grow: 1;
384 |     display: flex;
385 |     flex-direction: column;
386 |     justify-content: center;
387 |     gap: 8px;
388 | }
389 |
390 | .info-header {
391 |     display: flex;
392 |     align-items: center;
393 |     gap: 12px;
394 | }
395 |
396 | .provider-tag {
397 |     background: var(--primary-gradient);
398 |     color: white;
399 |     padding: 3px 10px;
400 |     border-radius: 15px;
401 |     font-size: 0.65rem;
402 |     font-weight: 600;
403 |     text-transform: uppercase;
404 | }
405 |
406 | #video-title {
407 |     font-size: 1.1rem;
408 |     font-weight: 600;
409 |     margin: 0;
410 | }
411 |
412 | #video-source-url {
413 |     font-size: 0.8rem;
```



```
414 |     color: var(--text-muted);
415 |     word-break: break-all;
416 |     margin: 0;
417 | }
418 |
419 | .empty-state {
420 |     color: var(--text-muted);
421 |     font-size: 0.75rem;
422 |     text-align: center;
423 |     padding: 15px;
424 |     border: 1px dashed var(--panel-border);
425 |     border-radius: 8px;
426 | }
427 |
```

main.css

```
1  /* — THEME VARIABLES —————
   |————— */
2  :root, [data-theme="light"] {
3    /* Light theme (default) - softer colors instead of pure whi
   |te */
4    --bg-color: #f8f9fa;
5    --text-color: #212529;
6    --border-color: #dee2e6;
7    --card-bg: #ffffff;
8    --card-text: #212529;
9    --chat-user-bg: #0d6efd;
10   --chat-user-text: #ffffff;
11   --chat-bot-bg: #e9ecef;
12   --chat-bot-text: #212529;
13   --scrollbar-track: #f1f1f1;
14   --scrollbar-thumb: #888;
15   --nav-bg: #ffffff;
16   --nav-text: #495057;
17   --nav-active: #0d6efd;
18   --nav-hover: #f8f9fa;
19   --modal-bg: #ffffff;
20   --modal-text: #212529;
21   --modal-border: #dee2e6;
22 }
23
24 [data-theme="dark"] {
25   /* Dark theme - proper contrast */
26   --bg-color: #0d1117;
27   --text-color: #c9d1d9;
28   --border-color: #30363d;
29   --card-bg: #161b22;
30   --card-text: #c9d1d9;
31   --chat-user-bg: #1f6feb;
32   --chat-user-text: #ffffff;
33   --chat-bot-bg: #21262d;
34   --chat-bot-text: #c9d1d9;
```

```
35 | --scrollbar-track: #0d1117;
36 | --scrollbar-thumb: #484f58;
37 | --nav-bg: #161b22;
38 | --nav-text: #8b949e;
39 | --nav-active: #58a6ff;
40 | --nav-hover: #21262d;
41 | --modal-bg: #161b22;
42 | --modal-text: #c9d1d9;
43 | --modal-border: #30363d;
44 | }
45 |
46 | /* — BASE STYLES —————
   | ————— */
47 | body {
48 |     display: flex;
49 |     flex-direction: column;
50 |     min-height: 100vh;
51 |     background: var(--bg-color);
52 |     color: var(--text-color);
53 |     transition: background-color 0.3s, color 0.3s;
54 | }
55 |
56 | /* — TAB STYLES —————
   | ————— */
57 | .tab-content {
58 |     flex-grow: 1;
59 |     display: flex;
60 |     flex-direction: column;
61 | }
62 | .tab-pane {
63 |     min-height: calc(100vh - 80px);
64 |     overflow-y: visible;
65 |     overflow-x: hidden;
66 |     padding-bottom: 24px;
67 | }
68 |
69 | .chat-window {
70 |     height: calc(100vh - 280px);
```

```
71 | min-height: 180px;
72 | overflow-y: auto;
73 | border: 1px solid var(--border-color);
74 | border-radius: 4px;
75 | padding: 8px;
76 | background: var(--card-bg);
77 | }
78 |
79 | /* Fix overlap in admin panel chat tab */
80 | #tab-chat {
81 |   height: calc(100vh - 95px);
82 |   max-height: calc(100vh - 95px);
83 |   min-height: 0;
84 |   padding-bottom: 0;
85 |   overflow: hidden;
86 | }
87 | #tab-chat.active {
88 |   display: flex !important;
89 |   flex-direction: column;
90 |   height: calc(100vh - 95px);
91 |   max-height: calc(100vh - 95px);
92 | }
93 | #tab-chat .chat-window {
94 |   height: auto;
95 |   flex: 1 1 auto;
96 |   min-height: 140px;
97 |   max-height: calc(100vh - 270px);
98 |   overflow-y: auto;
99 |   margin-bottom: 6px !important;
100 | }
101 | #tab-chat .media-controls {
102 |   flex-shrink: 0;
103 |   margin-bottom: 6px !important;
104 | }
105 | #tab-chat #chat-debugger-toolbar {
106 |   flex-shrink: 0;
107 |   margin-bottom: 6px !important;
108 | }
```

```
109 #tab-chat .input-group {
110     flex-shrink: 0;
111     margin-bottom: 2px !important;
112 }
113 .message { margin-bottom: 6px; padding: 6px 10px; border-radius: 4px; }
114 .user-message {
115     background: var(--chat-user-bg);
116     color: var(--chat-user-text);
117     text-align: right;
118 }
119 .bot-message {
120     background: var(--chat-bot-bg);
121     color: var(--chat-bot-text);
122 }
123
124 /* — NAVBAR THEME & HEADER —————
125    ————— */
126 .main-nav-container {
127     background-color: var(--nav-bg);
128     border-bottom: 1px solid var(--border-color);
129     min-height: 44px;
130 }
131 .nav-tabs {
132     border-bottom: none;
133     background-color: transparent;
134     flex-wrap: nowrap;
135     overflow-x: auto;
136     overflow-y: hidden;
137     scrollbar-width: thin;
138     scrollbar-color: var(--scrollbar-thumb) var(--scrollbar-track);
139     -webkit-overflow-scrolling: touch;
140 }
141
142 .nav-tabs::-webkit-scrollbar {
143     height: 4px;
```

```
144 | }
145 |
146 | .nav-tabs .nav-item {
147 |   flex-shrink: 0;
148 | }
149 |
150 | .nav-tabs .nav-link {
151 |   color: var(--nav-text);
152 |   border: none;
153 |   border-bottom: 2px solid transparent;
154 |   background: transparent;
155 |   padding: 8px 12px;
156 |   font-size: 0.88rem;
157 |   font-weight: 500;
158 |   white-space: nowrap;
159 |   transition: color 0.2s, border-color 0.2s;
160 | }
161 |
162 | .nav-tabs .nav-link:hover {
163 |   color: var(--nav-active);
164 |   background-color: var(--nav-hover);
165 | }
166 |
167 | .nav-tabs .nav-link.active {
168 |   color: var(--nav-active);
169 |   background: transparent;
170 |   border-bottom-color: var(--nav-active);
171 | }
172 |
173 | .main-nav-container .lang-selector {
174 |   background-color: var(--nav-bg);
175 |   border-color: var(--border-color);
176 |   color: var(--nav-text);
177 |   font-size: 0.85rem;
178 | }
179 |
180 | #nav-google-btn {
181 |   background: #ffffff;
```

```
182 | color: #1f2937 !important;
183 | font-size: 0.82rem;
184 | font-weight: 600;
185 | border-radius: 20px;
186 | border: 1px solid rgba(0, 0, 0, 0.12);
187 | transition: all 0.2s ease-in-out;
188 | }
189 |
190 | #nav-google-btn:hover {
191 |   background: #f8fafc;
192 |   transform: translateY(-1px);
193 |   box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.15);
194 | }
195 |
196 | #user-settings-btn {
197 |   font-size: 0.85rem;
198 |   font-weight: 500;
199 |   border-radius: 20px;
200 |   color: var(--nav-text);
201 |   border-color: var(--border-color);
202 |   transition: all 0.2s ease-in-out;
203 | }
204 |
205 | #user-settings-btn:hover {
206 |   background-color: var(--nav-hover);
207 |   color: var(--nav-active);
208 |   border-color: var(--nav-active);
209 | }
210 |
211 | #user-settings-gear-btn {
212 |   color: var(--nav-text);
213 |   border-color: var(--border-color);
214 |   transition: all 0.2s ease-in-out;
215 | }
216 |
217 | #user-settings-gear-btn:hover {
218 |   background-color: var(--nav-hover);
219 |   color: var(--nav-active);
```

```

220 | border-color: var(--nav-active);
221 | transform: rotate(30deg);
222 | }
223 |
224 | /* — TORRENTS STYLES —————
    | ————— */
225 | #torrents-table th[data-col] { cursor: pointer; user-select: none; }
226 | .state-missingFiles { color: #dc3545; font-weight: 600; }
227 | .state-uploading, .state-seeding { color: #198754; }
228 | .state-downloading { color: #0d6efd; }
229 | .state-checkingUP, .state-checkingDL { color: #fd7e14; }
230 | .progress { height: 6px; }
231 | .path-cell { font-size: .8rem; color: var(--text-color); opacity: 0.6; }
232 | .path-edit { display: none; }
233 | .path-edit.active { display: flex; }
234 | #missing-panel { border-left: 3px solid #dc3545; }
235 | .missing-row { cursor: pointer; }
236 | .missing-row:hover { background: #fff3cd; }
237 | .missing-row.selected { background: #cfe2ff; }
238 | #media-log { font-size:.82rem; font-family:monospace; background:
    | #1e1e1e; color:#d4d4d4;
239 |             border-radius:4px; padding:10px; height:180px; overflow-y:auto;
    | white-space:pre-wrap; }
240 | .dup-badge { font-size:.75rem; }
241 | .integrity-ok { color:#198754; } .integrity-warn { color:#dc35
    | 45; }
242 |
243 | /* — SCROLLBARS —————
    | ————— */
244 | ::-webkit-scrollbar { width: 8px; height: 8px; }
245 | ::-webkit-scrollbar-track {
246 |     background: var(--scrollbar-track);
247 |     border-radius: 4px;
248 | }
249 | ::-webkit-scrollbar-thumb {
250 |     background: var(--scrollbar-thumb);

```



```
251 | border-radius: 4px;
252 | }
253 | ::-webkit-scrollbar-thumb:hover { background: var(--scrollbar-
    | thumb); opacity: 0.8; }
254 |
255 | /* — HELP SYSTEM —————
    | ————— */
256 | .help-icon {
257 |   display: inline-flex;
258 |   align-items: center;
259 |   justify-content: center;
260 |   width: 18px; height: 18px;
261 |   border-radius: 50%;
262 |   background: var(--chat-user-bg);
263 |   color: white;
264 |   font-size: 12px;
265 |   font-weight: bold;
266 |   cursor: help;
267 |   margin-left: 4px;
268 |   vertical-align: middle;
269 | }
270 | .help-icon:hover { background: var(--chat-user-bg); opacity: 0
    | .9; }
271 | .help-popover {
272 |   max-width: 350px;
273 |   font-size: 0.85rem;
274 |   background: var(--modal-bg);
275 |   color: var(--modal-text);
276 |   border: 1px solid var(--border-color);
277 | }
278 | .modal-help .modal-dialog { max-width: 600px; }
279 | .modal-help .modal-content {
280 |   border-radius: 8px;
281 |   background: var(--modal-bg);
282 |   color: var(--modal-text);
283 |   border: 1px solid var(--modal-border);
284 | }
285 | .modal-help .modal-header {
```

```
286 | border-bottom: 1px solid var(--modal-border);
287 | background: var(--nav-bg);
288 | }
289 | .modal-help .modal-body {
290 |   font-size: 0.9rem;
291 |   line-height: 1.6;
292 |   color: var(--modal-text);
293 | }
294 | .modal-help .modal-footer {
295 |   border-top: 1px solid var(--modal-border);
296 |   background: var(--nav-bg);
297 | }
298 |
299 | /* — TORRENT SEARCH STYLING —————
    | ————— */
300 | .torrent-card {
301 |   border: 1px solid var(--border-color);
302 |   border-radius: 8px;
303 |   background-color: var(--card-bg);
304 |   padding: 12px;
305 |   margin-bottom: 12px;
306 |   box-shadow: 0 2px 4px rgba(0,0,0,0.04);
307 |   transition: transform 0.2s, box-shadow 0.2s;
308 |   color: var(--card-text);
309 | }
310 | .torrent-card:hover {
311 |   transform: translateY(-2px);
312 |   box-shadow: 0 4px 8px rgba(0,0,0,0.08);
313 | }
314 | .torrent-badge {
315 |   display: inline-block;
316 |   padding: 0.25em 0.6em;
317 |   font-size: 0.75rem;
318 |   font-weight: 600;
319 |   line-height: 1;
320 |   text-align: center;
321 |   white-space: nowrap;
322 |   vertical-align: baseline;
```

```
323 | border-radius: 50rem;
324 | margin-right: 4px;
325 | margin-bottom: 4px;
326 | }
327 | .badge-rutracker { background-color: #6e7a8a; color: #fff; }
328 | .badge-nnmclub { background-color: #198754; color: #fff; }
329 | .badge-seeds { background-color: #28a745; color: #fff; }
330 | .badge-peers { background-color: #6c757d; color: #fff; }
331 | .badge-size { background-color: #007bff; color: #fff; }
332 | .torrent-title {
333 |     font-size: 0.95rem;
334 |     font-weight: 600;
335 |     margin-bottom: 6px;
336 |     color: var(--text-color);
337 | }
338 | .torrent-title a {
339 |     color: var(--nav-active);
340 |     text-decoration: none;
341 | }
342 | .torrent-title a:hover {
343 |     text-decoration: underline;
344 | }
345 | .torrent-desc {
346 |     font-size: 0.85rem;
347 |     color: var(--text-color);
348 |     opacity: 0.7;
349 |     margin-bottom: 8px;
350 | }
351 |
352 | /* — INPUTS AND FORMS ————— */
353 | .form-control {
354 |     background-color: var(--card-bg);
355 |     border: 1px solid var(--border-color);
356 |     color: var(--text-color);
357 |     transition: border-color 0.2s, box-shadow 0.2s;
358 | }
359 |
```

```
360 | .form-control:focus {
361 |   background-color: var(--card-bg);
362 |   border-color: var(--nav-active);
363 |   color: var(--text-color);
364 |   box-shadow: 0 0 0 2px rgba(13, 110, 253, 0.25);
365 | }
366 |
367 | .btn {
368 |   transition: background-color 0.2s, border-color 0.2s;
369 | }
370 |
371 | .btn-primary {
372 |   background-color: var(--chat-user-bg);
373 |   border-color: var(--chat-user-bg);
374 |   color: #ffffff;
375 | }
376 |
377 | .btn-primary:hover {
378 |   background-color: var(--chat-user-bg);
379 |   opacity: 0.9;
380 |   border-color: var(--chat-user-bg);
381 | }
382 |
383 | .btn-outline-secondary {
384 |   background-color: transparent;
385 |   border-color: var(--border-color);
386 |   color: var(--nav-text);
387 | }
388 |
389 | .btn-outline-secondary:hover {
390 |   background-color: var(--border-color);
391 |   color: var(--text-color);
392 | }
393 |
394 | .btn-sm {
395 |   padding: 0.25rem 0.5rem;
396 |   font-size: 0.875rem;
397 | }
```

```
398 |
399 | /* — CARDS —————
    | ————— */
400 | .card {
401 |     background-color: var(--card-bg);
402 |     border: 1px solid var(--border-color);
403 |     color: var(--card-text);
404 |     transition: border-color 0.2s, background-color 0.2s;
405 | }
406 |
407 | .card-header {
408 |     background-color: var(--nav-bg);
409 |     border-bottom: 1px solid var(--border-color);
410 |     color: var(--text-color);
411 |     padding: 0.5rem 1rem;
412 | }
413 |
414 | .card-body {
415 |     color: var(--card-text);
416 | }
417 |
418 | .card.bg-light {
419 |     background-color: var(--border-color) !important;
420 |     color: var(--text-color);
421 | }
422 |
423 | .card.bg-dark > .card-header,
424 | .modal-content.bg-dark > .modal-header,
425 | .modal-content.bg-dark > .modal-footer,
426 | .agent-card .card-header {
427 |     background-color: #1e293b !important;
428 |     border-color: #334155 !important;
429 |     color: #ffffff !important;
430 | }
431 |
432 | /* — TABLES —————
    | ————— */
433 | .table {
```

```
434 | --bs-table-color: var(--text-color);
435 | --bs-table-bg: transparent;
436 | --bs-table-border-color: var(--border-color);
437 | }
438 |
439 | .table-hover tbody tr:hover {
440 |   background-color: var(--border-color);
441 |   opacity: 0.5;
442 | }
443 |
444 | .table-dark {
445 |   --bs-table-bg: var(--nav-bg);
446 |   --bs-table-color: var(--text-color);
447 | }
448 |
449 | /* — BADGES —————
450 | |————— */
451 | .badge {
452 |   font-weight: 500;
453 |   letter-spacing: 0.02em;
454 | }
455 |
456 | .badge:not([class*="bg-"]):not([class*="badge-"]) {
457 |   background-color: var(--border-color);
458 |   color: var(--text-color);
459 | }
460 |
461 | .badge.bg-light,
462 | .badge.badge-light {
463 |   background-color: var(--border-color) !important;
464 |   color: var(--text-color) !important;
465 | }
466 |
467 | /* Dark and colored badges must ALWAYS have crisp, readable li
468 | |ght text */
469 | .badge.bg-secondary,
```

```
470 | .badge.bg-success,
471 | .badge.bg-dark,
472 | .badge-secondary,
473 | .badge-primary,
474 | .badge-danger,
475 | .badge-success,
476 | .badge-dark,
477 | .film-inline-badge {
478 |   color: #ffffff !important;
479 | }
480 |
481 | #chat-model-badge,
482 | #chat-popup-model-badge,
483 | #chat-search-badge,
484 | #chat-popup-search-badge {
485 |   background-color: #212529 !important;
486 |   border-color: #373b3e !important;
487 |   color: #f8f9fa !important;
488 |   font-weight: 600;
489 |   letter-spacing: 0.02em;
490 | }
491 |
492 | [data-theme="dark"] #chat-model-badge,
493 | [data-theme="dark"] #chat-popup-model-badge {
494 |   background-color: #161b22 !important;
495 |   border-color: #30363d !important;
496 |   color: #58a6ff !important;
497 | }
498 |
499 | [data-theme="dark"] #chat-search-badge,
500 | [data-theme="dark"] #chat-popup-search-badge {
501 |   background-color: #161b22 !important;
502 |   border-color: #30363d !important;
503 |   color: #38bdf8 !important;
504 | }
505 |
506 | #chat-debugger-toolbar {
507 |   background-color: var(--bg-color-card, #1c2128);
```

```
508 | border: 1px solid var(--border-color, #30363d);
509 | }
510 |
511 | [data-theme="light"] #chat-debugger-toolbar {
512 |   background-color: #f6f8fa !important;
513 |   border-color: #d0d7de !important;
514 | }
515 |
516 | [data-theme="light"] #chat-debugger-toolbar select {
517 |   background-color: #ffffff !important;
518 |   color: #24292f !important;
519 |   border-color: #d0d7de !important;
520 | }
521 |
522 | .film-inline-badge {
523 |   background-color: #0d6efd !important;
524 |   color: #ffffff !important;
525 |   font-weight: 500;
526 |   border: 1px solid rgba(255, 255, 255, 0.25) !important;
527 |   box-shadow: 0 1px 3px rgba(0, 0, 0, 0.25);
528 |   text-shadow: 0 1px 1px rgba(0, 0, 0, 0.3);
529 | }
530 |
531 | .film-inline-badge:hover {
532 |   background-color: #0b5ed7 !important;
533 |   color: #ffffff !important;
534 | }
535 |
536 | /* Interactive Film Chip */
537 | .film-interactive-chip {
538 |   display: inline-flex;
539 |   align-items: center;
540 |   gap: 4px;
541 |   background: linear-gradient(135deg, #1e3a8a 0%, #2563eb 100%
542 |   );
543 |   color: #ffffff !important;
544 |   font-size: 0.9em;
545 |   font-weight: 500;
```



```
545 | padding: 2px 4px 2px 8px;
546 | border-radius: 16px;
547 | border: 1px solid rgba(255, 255, 255, 0.25);
548 | box-shadow: 0 2px 4px rgba(0, 0, 0, 0.2);
549 | margin: 2px 3px;
550 | vertical-align: middle;
551 | transition: all 0.2s ease-in-out;
552 | }
553 |
554 | .film-interactive-chip:hover {
555 |   background: linear-gradient(135deg, #1d4ed8 0%, #3b82f6 100%
556 |   );
557 |   box-shadow: 0 3px 8px rgba(37, 99, 235, 0.4);
558 |   transform: translateY(-1px);
559 | }
560 |
561 | .film-chip-title {
562 |   cursor: pointer;
563 |   color: #ffffff !important;
564 |   text-decoration: none;
565 |   font-weight: 600;
566 | }
567 |
568 | .film-chip-title:hover {
569 |   text-decoration: underline;
570 | }
571 |
572 | .film-chip-btn {
573 |   background: rgba(255, 255, 255, 0.2);
574 |   border: none;
575 |   color: #ffffff;
576 |   width: 22px;
577 |   height: 22px;
578 |   border-radius: 50%;
579 |   display: inline-flex;
580 |   align-items: center;
581 |   justify-content: center;
582 |   font-size: 0.75rem;
```

```
582 | padding: 0;
583 | cursor: pointer;
584 | transition: all 0.15s ease;
585 | }
586 |
587 | .film-chip-btn:hover {
588 |     background: rgba(255, 255, 255, 0.4);
589 |     transform: scale(1.15);
590 | }
591 |
592 | .film-chip-play:hover {
593 |     background: #10b981;
594 |     color: #ffffff;
595 | }
596 |
597 | .film-chip-youtube:hover {
598 |     background: #ef4444;
599 |     color: #ffffff;
600 | }
601 |
602 | .film-chip-online:hover {
603 |     background: #06b6d4;
604 |     color: #ffffff;
605 | }
606 |
607 | .film-chip-torrent:hover {
608 |     background: #f59e0b;
609 |     color: #111827;
610 | }
611 |
612 | [data-theme="dark"] .film-interactive-chip {
613 |     background: linear-gradient(135deg, #1e293b 0%, #334155 100%
        | );
614 |     border: 1px solid rgba(148, 163, 184, 0.3);
615 | }
616 |
617 | [data-theme="dark"] .film-interactive-chip:hover {
618 |     background: linear-gradient(135deg, #334155 0%, #475569 100%
```

```
);
619 | box-shadow: 0 3px 8px rgba(0, 0, 0, 0.5);
620 | }
621 |
622 | .badge.bg-warning,
623 | .badge.badge-warning {
624 |     color: #1a1a1a !important;
625 |     font-weight: 600;
626 | }
627 |
628 | .badge.bg-info,
629 | .badge.badge-info {
630 |     color: #042743 !important;
631 |     font-weight: 600;
632 | }
633 |
634 | /* Specific styling for badges in chat window / messages */
635 | .chat-window .badge,
636 | .message .badge,
637 | .bot-message .badge,
638 | .user-message .badge,
639 | .chat-popup .badge {
640 |     font-weight: 500;
641 | }
642 |
643 | .chat-window .badge.bg-secondary,
644 | .chat-window .badge.bg-primary,
645 | .chat-window .badge.bg-danger,
646 | .chat-window .badge.bg-success,
647 | .chat-window .badge.bg-dark,
648 | .message .badge.bg-secondary,
649 | .message .badge.bg-primary,
650 | .message .badge.bg-danger,
651 | .message .badge.bg-success,
652 | .message .badge.bg-dark {
653 |     color: #ffffff !important;
654 |     text-shadow: 0 1px 1px rgba(0, 0, 0, 0.25);
655 | }
```

```
656 |
657 | /* — PROMPT DUMP (ADMIN CONSOLE VIEW) —————
    | ————— */
658 | .prompt-dump-details {
659 |     margin-bottom: 10px;
660 |     font-size: 0.85em;
661 |     background: #0d1117 !important;
662 |     padding: 10px;
663 |     border-radius: 6px;
664 |     border: 1px solid #30363d !important;
665 |     box-shadow: inset 0 1px 4px rgba(0, 0, 0, 0.8);
666 | }
667 |
668 | .prompt-dump-details summary {
669 |     cursor: pointer;
670 |     color: #e3b341 !important;
671 |     font-weight: bold;
672 |     user-select: none;
673 | }
674 |
675 | .prompt-dump-details pre {
676 |     white-space: pre-wrap;
677 |     margin-top: 10px;
678 |     color: #00ff66 !important;
679 |     font-family: 'Consolas', 'Courier New', Courier, monospace;
680 |     font-size: 0.95em;
681 |     line-height: 1.4;
682 |     max-height: 300px;
683 |     overflow-y: auto;
684 | }
685 |
686 | /* — ALERTS —————
    | ————— */
687 | .alert {
688 |     background-color: var(--card-bg);
689 |     border: 1px solid var(--border-color);
690 |     color: var(--text-color);
691 | }
```

```
692 |
693 | .alert-success {
694 |     background-color: #1e3a2a;
695 |     border-color: #198754;
696 |     color: #d4edda;
697 | }
698 |
699 | .alert-danger {
700 |     background-color: #3a1e1e;
701 |     border-color: #dc3545;
702 |     color: #f8d7da;
703 | }
704 |
705 | .alert-warning {
706 |     background-color: #3a351e;
707 |     border-color: #ffc107;
708 |     color: #fff3cd;
709 | }
710 |
711 | .alert-info {
712 |     background-color: #1e2a3a;
713 |     border-color: #0dcaf0;
714 |     color: #cfe2f3;
715 | }
716 |
717 | /* — THEME DROPDOWN & SWITCHER —————
718 |     |
719 |     | */
720 | #theme-dropdown-container .dropdown-menu {
721 |     min-width: 150px;
722 |     padding: 0.35rem;
723 | }
724 |
725 | .theme-check-icon {
726 |     font-size: 0.9rem;
727 | }
728 |
729 | .theme-check-icon.d-none {
730 |     display: none !important;
```

```
729 | }
730 |
731 | [data-theme="dark"] .dropdown-menu {
732 |     background-color: #161b22;
733 |     border-color: #30363d;
734 | }
735 |
736 | [data-theme="dark"] .dropdown-item {
737 |     color: #c9d1d9;
738 |     border-radius: 4px;
739 | }
740 |
741 | [data-theme="dark"] .dropdown-item:hover,
742 | [data-theme="dark"] .dropdown-item:focus {
743 |     background-color: #21262d;
744 |     color: #58a6ff;
745 | }
746 |
747 | [data-theme="dark"] .dropdown-item.active {
748 |     background-color: #1f6feb;
749 |     color: #ffffff;
750 | }
751 |
752 | [data-theme="light"] .dropdown-menu {
753 |     background-color: #ffffff;
754 |     border-color: #dee2e6;
755 | }
756 |
757 | [data-theme="light"] .dropdown-item {
758 |     color: #212529;
759 |     border-radius: 4px;
760 | }
761 |
762 | [data-theme="light"] .dropdown-item:hover,
763 | [data-theme="light"] .dropdown-item:focus {
764 |     background-color: #f8f9fa;
765 |     color: #0d6efd;
766 | }
```

```
767 |  
768 | [data-theme="light"] .dropdown-item.active {  
769 |     background-color: #0d6efd;  
770 |     color: #ffffff;  
771 | }  
772 |  
773 |  
774 |
```

index.html

```
1 <!-- Reference documentation for the project -->
2 <div class="container-fluid">
3   <div class="row">
4     <div class="col-md-3">
5       <div class="card">
6         <div class="card-header bg-primary text-white" data-i1
7         8n="help.title">
8           □ Справочник
9         </div>
10        <div class="list-group list-group-flush" id="help-nav"
11        >
12          <a class="list-group-item list-group-item-action" da
13          ta-target="overview">
14            <strong data-i18n="help.overview">□ Обзор проекта
15            </strong>
16            <button class="btn btn-sm btn-outline-primary floa
17            t-end" onclick="showHelpModal('overview')">?</button>
18          </a>
19          <a class="list-group-item list-group-item-action" da
20          ta-target="code_rules">
21            <strong data-i18n="help.codeRules">□ Правила кодир
22            ования (CODE_RULES)</strong>
23            <button class="btn btn-sm btn-outline-primary floa
24            t-end" onclick="showHelpModal('code_rules')">?</button>
25          </a>
26          <a class="list-group-item list-group-item-action" da
27          ta-target="architecture">
28            <strong data-i18n="help.architecture">□ Архитекту
29            па</strong>
30            <button class="btn btn-sm btn-outline-primary floa
31            t-end" onclick="showHelpModal('architecture')">?</button>
32          </a>
33          <a class="list-group-item list-group-item-action" da
34          ta-target="fastapi">
35            <strong data-i18n="help.fastapi">□ FastAPI</stron
36            g>
```



```
24 |         <button class="btn btn-sm btn-outline-primary float-end" onclick="showHelpModal('fastapi')">?</button>
25 |     </a>
26 |     <a class="list-group-item list-group-item-action" data-target="gemini">
27 |         <strong data-i18n="help.gemini"> Gemini AI</strong>
28 |         <button class="btn btn-sm btn-outline-primary float-end" onclick="showHelpModal('gemini')">?</button>
29 |     </a>
30 |     <a class="list-group-item list-group-item-action" data-target="configuration">
31 |         <strong data-i18n="help.configuration"> Конфигурация</strong>
32 |         <button class="btn btn-sm btn-outline-primary float-end" onclick="showHelpModal('configuration')">?</button>
33 |     </a>
34 |     <a class="list-group-item list-group-item-action" data-target="cli_commands">
35 |         <strong data-i18n="help.cliCommands"> CLI команды</strong>
36 |         <button class="btn btn-sm btn-outline-primary float-end" onclick="showHelpModal('cli_commands')">?</button>
37 |     </a>
38 | </div>
39 | </div>
40 | </div>
41 | <div class="col-md-9">
42 |     <div class="card">
43 |         <div class="card-header bg-secondary text-white">
44 |             <span id="help-title" data-i18n="help.title">Добро пожаловать в справочник</span>
45 |         </div>
46 |         <div class="card-body" id="help-content" style="max-height:600px;overflow-y:auto;">
47 |             <p class="lead" data-i18n="help.welcome">Выберите раздел справочника слева для просмотра информации.</p>
48 |         </div>
```

```
49 |         </div>
50 |     </div>
51 | </div>
52 | </div>
53 |
```

main.js

```
1 // — HELP.JS —————  
2  
3 // HELP content stored in window.HELP_CONTENT (defined in main  
4 // .js)  
5  
6 function loadHelpSection(target) {  
7   const contentEl = document.getElementById('help-content');  
8   const titleEl = document.getElementById('help-title');  
9  
10  const content = window.HELP_CONTENT[target] || '<p>Информаци  
   я не найдена</p>';  
11  contentEl.innerHTML = content;  
12  
13  // Update title  
14  const navItem = document.querySelector(`#help-nav a[data-tar  
   get="${target}"]`);  
15  if (navItem) {  
16    titleEl.textContent = navItem.querySelector('strong').text  
   Content;  
17  }  
18  
19  // Update active state in nav  
20  document.querySelectorAll('#help-nav a').forEach(a => {  
21    a.classList.remove('active', 'bg-primary', 'text-white');  
22    if (a.dataset.target === target) {  
23      a.classList.add('active', 'bg-primary', 'text-white');  
24    }  
25  });  
26  
27  console.log(`HELP section loaded: ${target}`);  
28 }  
29  
30 // Navigation click handler  
31 document.querySelectorAll('#help-nav a').forEach(a => {
```

```
32 | a.addEventListener('click', (e) => {
33 |     e.preventDefault();
34 |     const target = a.dataset.target;
35 |     loadHelpSection(target);
36 | });
37 | });
38 |
39 | // Initialize with overview when tab is loaded
40 | // Note: HELP content is initialized in main.js before loading
    | tabs
```

index.html

```
1 <!DOCTYPE html>
2 <html lang="ru">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-s
cale=1.0">
6   <title>AI Assistant</title>
7   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dis
t/css/bootstrap.min.css" rel="stylesheet">
8   <link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.1
0.0/font/bootstrap-icons.css" rel="stylesheet">
9   <link href="/html/css/main.css?v=20260825_user_settings" rel
="stylesheet">
10  <script src="https://cdn.jsdelivr.net/npm/i18next@23.11.5/i1
8next.min.js"></script>
11  <script src="https://cdn.jsdelivr.net/npm/marked/marked.min.
js"></script>
12 </head>
13 <body>
14 <div class="container-fluid mt-2">
15
16   <div class="d-flex align-items-center justify-content-betwee
n mb-3 main-nav-container rounded px-2 py-1">
17     <ul class="nav nav-tabs flex-nowrap overflow-x-auto flex-g
row-1 border-0 me-2" id="mainTabs">
18       <li class="nav-item">
19         <button class="nav-link active" data-bs-toggle="tab" d
ata-bs-target="#tab-chat" data-i18n="tabs.chat"> Чат</button
>
20       </li>
21       <li class="nav-item">
22         <button class="nav-link" data-bs-toggle="tab" data-bs-
target="#tab-plugins"> Плагины</button>
23       </li>
24       <li class="nav-item">
25         <button class="nav-link" data-bs-toggle="tab" data-bs-
```

```

    target="#tab-admin" data-i18n="tabs.admin"> Управление</button>
26     </li>
27     <li class="nav-item">
28         <button class="nav-link" data-bs-toggle="tab" data-bs-
target="#tab-help" data-i18n="tabs.help"> Справочник</button>
29     </li>
30 </ul>
31
32 <div class="d-flex align-items-center gap-2 flex-shrink-0
ms-auto pb-1" id="nav-auth-container">
33     <!-- Google OAuth Register / Sign-in Button (Shown when
not logged in) -->
34     <button class="btn btn-sm btn-light text-dark fw-semibol
d d-flex align-items-center gap-2 px-3 py-1 shadow-sm rounded-
pill btn-google-login" id="nav-google-btn" title="Войти или за
регистрироваться через Google" data-i18n-title="auth.googleSig
nInTitle" style="border: 1px solid rgba(0,0,0,0.15);">
35         <svg width="18" height="18" viewBox="0 0 24 24"><path
fill="#4285F4" d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h
5.92c-.26 1.37-1.04 2.53-2.21 3.31v2.77h3.57c2.08-1.92 3.28-4.
74 3.28-8.09z"/><path fill="#34A853" d="M12 23c2.97 0 5.46-.98
7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-3.71 1.06-2.86 0-5.29-
1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z"/><path fill
="#FBBC05" d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.
35-2.09V7.06H2.18C1.43 8.55 1 10.22 1 12s.43 3.45 1.18 4.94l2.
85-2.22.81-.63z"/><path fill="#EA4335" d="M12 5.38c1.62 0 3.06
.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1 7.7 1 3.99 3.4
7 2.18 7.06l3.66 2.84c.87-2.6 3.3-4.52 6.16-4.52z"/></svg>
36         <span data-i18n="auth.googleSignUp">Регистрация через
Google</span>
37     </button>
38
39     <!-- Authenticated User Profile Pill (Shown when logged
in) -->
40     <button class="btn btn-sm btn-outline-secondary d-none a
lign-items-center gap-2 px-2 py-1 rounded-pill" id="user-setti

```

```

ngs-btn" data-bs-toggle="modal" data-bs-target="#userSettingsM
odal" title="Профиль и настройки" data-i18n-title="userSetting
s.title">
41     <img id="nav-user-avatar" src="" class="rounded-circle
d-none" width="22" height="22" style="object-fit: cover;">
42     <i class="bi bi-person-gear" id="nav-user-icon"></i>
43     <span id="nav-user-label" class="small fw-semibold" da
ta-i18n="userSettings.button">User Settings</span>
44     </button>
45
46     <!-- Quick Settings Gear Button -->
47     <button class="btn btn-sm btn-outline-secondary d-flex a
lign-items-center justify-content-center p-0 rounded-circle" i
d="user-settings-gear-btn" data-bs-toggle="modal" data-bs-targ
et="#userSettingsModal" title="Настройки" style="width: 32px;
height: 32px;" data-i18n-title="userSettings.title">
48         <i class="bi bi-gear-fill"></i>
49     </button>
50 </div>
51 </div>
52
53 <div class="tab-content">
54     <!-- Tabs will be injected via JavaScript -->
55     <div id="tab-chat" class="tab-pane fade show active"></div
>
56     <div id="tab-plugins" class="tab-pane fade"></div>
57     <div id="tab-admin" class="tab-pane fade"></div>
58     <div id="tab-help" class="tab-pane fade"></div>
59 </div>
60 </div>
61
62 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist
/js/bootstrap.bundle.min.js"></script>
63 <script src="/html/js/chatService.js?v=20260825_user_settings"
></script>
64 <script type="module" src="/html/js/main.js?v=20260825_user_se
ttings"></script>
65

```

```
66 <!-- HELP Modal -->
67 <div class="modal fade modal-help" id="help-modal" tabindex="-
    1" aria-labelledby="help-modal-label" aria-hidden="true">
68   <div class="modal-dialog modal-dialog-scrollable">
69     <div class="modal-content">
70       <div class="modal-header">
71         <h5 class="modal-title" id="help-modal-label" data-i18
            n="helpModal.title">Помощь</h5>
72         <button type="button" class="btn-close" data-bs-dismis
            s="modal" aria-label="Close"></button>
73       </div>
74       <div class="modal-body" id="help-modal-content">
75         <p data-i18n="common.loading">Загрузка...</p>
76       </div>
77       <div class="modal-footer">
78         <button type="button" class="btn btn-secondary" data-b
            s-dismiss="modal" data-i18n="helpModal.close">Закрыть</button>
79       </div>
80     </div>
81   </div>
82 </div>
83
84 <!-- User Settings Modal -->
85 <div class="modal fade" id="userSettingsModal" tabindex="-1" a
    ria-labelledby="userSettingsModalLabel" aria-hidden="true">
86   <div class="modal-dialog modal-dialog-centered">
87     <div class="modal-content shadow border-secondary-subtle">
88       <div class="modal-header border-secondary-subtle">
89         <h5 class="modal-title d-flex align-items-center gap-2
            " id="userSettingsModalLabel">
90           <i class="bi bi-person-gear text-primary"></i>
91           <span data-i18n="userSettings.title">Пользовательски
            е настройки</span>
92         </h5>
93         <button type="button" class="btn-close" data-bs-dismis
            s="modal" aria-label="Close"></button>
94       </div>
95       <div class="modal-body p-3">
```



```

96 |         <!-- User Account & Google OAuth Card -->
97 |         <div class="card bg-body-tertiary border-secondary-sub
| tle mb-3 p-3" id="user-settings-auth-card">
98 |             <div class="d-flex align-items-center gap-3">
99 |                 <img id="user-settings-avatar" src="" class="round
| ed-circle border border-2 border-primary" width="48" height="4
| 8" style="object-fit: cover;">
100 |                 <div class="flex-grow-1 min-w-0">
101 |                     <div class="d-flex align-items-center gap-2">
102 |                         <strong id="user-settings-name" class="text-tr
|uncate">Пользователь</strong>
103 |                         <span id="user-settings-role-badge" class="bad
|ge bg-primary">User</span>
104 |                     </div>
105 |                     <div id="user-settings-email" class="small text-
|muted text-truncate">email@gmail.com</div>
106 |                     <div id="user-settings-google-badge" class="smal
|l mt-1 text-muted">
107 |                         <i class="bi bi-google text-danger me-1"></i>G
|oogle OAuth
108 |                     </div>
109 |                 </div>
110 |                 <button type="button" class="btn btn-outline-dange
|r btn-sm btn-user-logout" title="Выйти">
111 |                     <i class="bi bi-box-arrow-right"></i>
112 |                 </button>
113 |             </div>
114 |         </div>
115 |
116 |         <!-- Guest / Connect Google OAuth Card -->
117 |         <div class="card bg-body-tertiary border-secondary-sub
|tle mb-3 p-3 text-center d-none" id="user-settings-guest-card"
|>
118 |             <div class="text-muted small mb-2">
119 |                 <i class="bi bi-shield-lock me-1"></i> Авторизация
|и синхронизация аккаунта
120 |             </div>
121 |             <button type="button" class="btn btn-outline-danger

```

```

    btn-sm w-100 d-flex align-items-center justify-content-center
    gap-2 btn-google-login">
122         <i class="bi bi-google"></i>
123         <span>Войти через Google (OAuth)</span>
124     </button>
125     <div class="small text-muted mt-2" style="font-size:
    0.75rem;">
126         Синхронизация с Google Drive, Docs, Calendar и Con
    tacts
127     </div>
128 </div>
129
130     <!-- Language Selection -->
131     <div class="mb-3">
132         <label class="form-label fw-semibold small d-flex al
    ign-items-center gap-2" for="modal-lang-selector">
133             <i class="bi bi-translate text-info"></i>
134             <span data-i18n="userSettings.language">Язык интер
    фейса</span>
135         </label>
136         <select class="form-select lang-selector" id="modal-
    lang-selector">
137             <option value="ru">Русский</option>
138             <option value="en">English</option>
139             <option value="he">עברית</option>
140         </select>
141     </div>
142
143     <!-- Theme Selection -->
144     <div class="mb-2">
145         <label class="form-label fw-semibold small d-flex al
    ign-items-center gap-2" for="modal-theme-selector">
146             <i class="bi bi-palette text-warning"></i>
147             <span data-i18n="userSettings.theme">Тема оформлен
    ия</span>
148         </label>
149         <select class="form-select theme-selector" id="modal
    -theme-selector">

```

```
150 |         <option value="light" data-i18n="userSettings.them  
| eLight">☐ Светлая</option>  
151 |         <option value="dark" data-i18n="userSettings.theme  
| Dark">☐ Тёмная</option>  
152 |         <option value="system" data-i18n="userSettings.the  
| meSystem">☐ Системная</option>  
153 |     </select>  
154 | </div>  
155 | </div>  
156 | <div class="modal-footer border-secondary-subtle py-2">  
157 |     <button type="button" class="btn btn-secondary btn-sm"  
| data-bs-dismiss="modal" data-i18n="userSettings.close">Закрыт  
| ь</button>  
158 | </div>  
159 | </div>  
160 | </div>  
161 | </div>  
162 |  
163 | </body>  
164 | </html>  
165 |
```

index.html

```
1 <!-- Instructions Tab -->
2 <div class="container-fluid p-3">
3   <div class="alert alert-info">
4     <h5 class="mb-1"><i class="bi bi-gear"></i> Редактор систе
мных инструкций</h5>
5     <p class="mb-0 small">Управление системными инструкциями А
I: чат и Narrator режимы, версионирование, проверка в модели.<
/p>
6   </div>
7
8   <!-- Mode Switcher -->
9   <div class="card mb-3">
10    <div class="card-body">
11      <div class="d-flex align-items-center gap-2 flex-wrap">
12        <span class="text-muted">Режим:</span>
13        <div class="btn-group" role="group">
14          <button type="button" class="btn btn-outline-primary
active" id="mode-chat" onclick="switchMode('chat')">
15            <i class="bi bi-chat-text"></i> Chat
16          </button>
17          <button type="button" class="btn btn-outline-seconda
ry" id="mode-narrator" onclick="switchMode('narrator')">
18            <i class="bi bi-mic"></i> Narrator
19          </button>
20        </div>
21        <small class="text-muted ms-2">Активный файл: <code id
="active-file-info">—</code></small>
22        <div class="ms-auto d-flex gap-2">
23          <button type="button" class="btn btn-success" id="bt
n-check-model" onclick="checkInModel()">
24            <i class="bi bi-check-circle"></i> Проверить в мод
ели
25          </button>
26          <button type="button" class="btn btn-primary" id="bt
n-save" onclick="saveInstruction()">
27            <i class="bi bi-save"></i> Сохранить версию
```

```

28         </button>
29         <button type="button" class="btn btn-warning" id="bt
n-preview" onclick="previewPrompt()">
30             <i class="bi bi-eye"></i> Предпросмотр
31         </button>
32     </div>
33 </div>
34 </div>
35 </div>
36
37 <!-- Editor Area -->
38 <div class="card mb-3">
39     <div class="card-header">
40         <div class="d-flex justify-content-between align-items-c
enter">
41             <span class="fw-bold">Редактор – <span id="editor-mode
-label"> Chat</span></span>
42             <small class="text-muted">Markdown поддерживается</sma
ll>
43         </div>
44     </div>
45     <div class="card-body p-0">
46         <textarea id="instruction-editor" class="form-control bo
rder-0 rounded-0" rows="18"
47             placeholder="Введите системную инструкцию..." spellche
ck="false"
48             style="font-family: 'Courier New', monospace; font-siz
e: 0.9rem; resize: vertical;"></textarea>
49     </div>
50 </div>
51
52 <!-- Versions Panel -->
53 <div class="card mb-3">
54     <div class="card-header d-flex justify-content-between ali
gn-items-center">
55         <span class="fw-bold"><i class="bi bi-clock-history"></i>
> История версий</span>
56         <button type="button" class="btn btn-sm btn-outline-seco

```

```

ndary" onclick="refreshVersions()">
57     <i class="bi bi-arrow-clockwise"></i> Обновить
58     </button>
59 </div>
60 <div class="card-body p-0">
61     <div id="versions-list" class="list-group list-group-flu
sh">
62         <div class="list-group-item text-muted">Загрузка верси
й...</div>
63     </div>
64 </div>
65 </div>
66
67 <!-- Token Counter -->
68 <div class="card">
69     <div class="card-header">
70         <span class="fw-bold"><i class="bi bi-calculator"></i> С
татистика</span>
71     </div>
72     <div class="card-body">
73         <div class="row g-3">
74             <div class="col-md-4">
75                 <div class="p-3 bg-light rounded text-center">
76                     <div class="h3 mb-0" id="char-count">0</div>
77                     <div class="small text-muted">Символов</div>
78                 </div>
79             </div>
80             <div class="col-md-4">
81                 <div class="p-3 bg-light rounded text-center">
82                     <div class="h3 mb-0" id="word-count">0</div>
83                     <div class="small text-muted">Слов</div>
84                 </div>
85             </div>
86             <div class="col-md-4">
87                 <div class="p-3 bg-light rounded text-center">
88                     <div class="h3 mb-0" id="token-estimate">~0</div>
89                     <div class="small text-muted">Токенов (оценка)</di
v>

```

```
90 |         </div>
91 |     </div>
92 | </div>
93 |     <div class="mt-3 small text-muted border rounded p-2" id
    | ="token-preview" style="min-height:2rem;"></div>
94 | </div>
95 | </div>
96 | </div>
97 |
98 | <!-- Prompt Preview Modal -->
99 | <div class="modal fade" id="prompt-preview-modal" tabindex="-1
    | " aria-labelledby="prompt-preview-modal-label" aria-hidden="tr
    | ue">
100 |     <div class="modal-dialog modal-lg modal-dialog-scrollable">
101 |         <div class="modal-content">
102 |             <div class="modal-header">
103 |                 <h5 class="modal-title" id="prompt-preview-modal-label
    | ">Предпросмотр инструкции</h5>
104 |                 <button type="button" class="btn-close" data-bs-dismis
    | s="modal" aria-label="Close"></button>
105 |             </div>
106 |             <div class="modal-body" id="prompt-preview-content">
107 |                 <p>Загрузка...</p>
108 |             </div>
109 |             <div class="modal-footer">
110 |                 <button type="button" class="btn btn-secondary" data-b
    | s-dismiss="modal">Закрыть</button>
111 |             </div>
112 |         </div>
113 |     </div>
114 | </div>
```

main.js

```
1 // Instructions Tab - UI Logic
2 // Инициализируется через window.initInstructionsTab() из admin/main.js
3
4 'use strict';
5
6 import { initI18n, switchLang, applyTranslations } from '../js/i18n.js';
7
8 window.switchLang = switchLang;
9 window.applyTranslations = applyTranslations;
10
11 // — Helpers —————
12
13 function adminFetch(url, options = {}) {
14   return window.api
15     ? window.api.fetch(url, options)
16     : fetch(url, options).then(r => {
17       if (!r.ok) throw new Error(`HTTP ${r.status}`);
18       return r.json();
19     });
20 }
21
22 function notify(msg, type = 'info') {
23   if (typeof showNotification === 'function') {
24     showNotification(msg, type);
25   } else {
26     console.log(`[${type}] ${msg}`);
27   }
28 }
29
30 // — Mode Management —————
31 let currentMode = 'chat';
32
```



```

33 function switchMode(mode) {
34     currentMode = mode;
35     document.getElementById('mode-chat').classList.toggle('active', mode === 'chat');
36     document.getElementById('mode-narrator').classList.toggle('active', mode === 'narrator');
37
38     // Обновить заголовок редактора
39     const modeLabel = mode === 'chat' ? 'Чат' : 'Narrator';
40     const editorTitle = document.getElementById('editor-mode-label');
41     if (editorTitle) editorTitle.textContent = modeLabel;
42
43     loadInstruction();
44     refreshVersions();
45 }
46
47 // — Instructions API —————
48 async function loadInstruction() {
49     try {
50         const data = await adminFetch(`/api/admin/instructions?mode=${currentMode}`);
51         const editor = document.getElementById('instruction-editor');
52         if (data && data.content !== undefined) {
53             editor.value = data.content;
54         } else {
55             editor.value = '';
56         }
57         updateStats();
58         const fileInfo = document.getElementById('active-file-info');
59         if (fileInfo && data.file) {
60             fileInfo.textContent = data.file;
61         }
62     } catch (e) {

```

```
63 |     notify('Ошибка загрузки инструкции: ' + e.message, 'danger
    | ');
64 |     document.getElementById('instruction-editor').value = '';
65 | }
66 | }
67 |
68 | async function saveInstruction() {
69 |     const content = document.getElementById('instruction-editor'
    | ).value.trim();
70 |     if (!content) {
71 |         notify('Инструкция не может быть пустой', 'warning');
72 |         return;
73 |     }
74 |
75 |     const btn = document.getElementById('btn-save');
76 |     const originalHtml = btn.innerHTML;
77 |     btn.disabled = true;
78 |     btn.innerHTML = '<i class="bi bi-hourglass-split"></i> Сохра
    | нение...';
79 |
80 |     try {
81 |         const data = await adminFetch('/api/admin/instructions/sav
    | e', {
82 |             method: 'POST',
83 |             headers: { 'Content-Type': 'application/json' },
84 |             body: JSON.stringify({ mode: currentMode, content })
85 |         });
86 |         notify(`Инструкция сохранена: ${data.version || ''}`, 'suc
    | cess');
87 |         await refreshVersions();
88 |     } catch (e) {
89 |         notify('Ошибка сохранения: ' + e.message, 'danger');
90 |     } finally {
91 |         btn.disabled = false;
92 |         btn.innerHTML = originalHtml;
93 |     }
94 | }
95 |
```

```
96 | async function checkInModel() {
97 |   const content = document.getElementById('instruction-editor'
    | ).value.trim();
98 |   const prompt = "Привет! Подтверди, что ты получил инструкцию
    | и готов к работе. Ответь только: 'Понял, готов к работе!'";
99 |
100 |   const btn = document.getElementById('btn-check-model');
101 |   const originalHtml = btn.innerHTML;
102 |   btn.disabled = true;
103 |   btn.innerHTML = '<i class="bi bi-hourglass-split"></i> Прове
    | рка...';
104 |
105 |   try {
106 |     const result = await adminFetch('/api/admin/instructions/c
    | heck', {
107 |       method: 'POST',
108 |       headers: { 'Content-Type': 'application/json' },
109 |       body: JSON.stringify({ mode: currentMode, prompt, instru
    | ction: content })
110 |     });
111 |
112 |     const statusEl = document.getElementById('token-preview');
113 |     let tokenInfo = '';
114 |     if (result.token_count) {
115 |       tokenInfo = `<br>Использовано токенов: ${result.token_co
    | unt}`;
116 |     }
117 |
118 |     if (result.response) {
119 |       statusEl.innerHTML = `<strong>Ответ модели:</strong><br>
    | ${escapeHtml(result.response)}${tokenInfo}`;
120 |       notify('Проверка завершена', 'success');
121 |     } else {
122 |       statusEl.innerHTML = 'Ошибка: нет ответа от модели';
123 |       notify('Ошибка проверки', 'danger');
124 |     }
125 |   } catch (e) {
126 |     document.getElementById('token-preview').innerHTML = `<str
```

```

    ong class="text-danger">Ошибка проверки: ${e.message}</strong>
    `;
127     notify('Ошибка проверки: ' + e.message, 'danger');
128   } finally {
129     btn.disabled = false;
130     btn.innerHTML = originalHtml;
131   }
132 }
133
134 async function refreshVersions() {
135   const list = document.getElementById('versions-list');
136   list.innerHTML = '<div class="list-group-item text-muted"><i
    class="bi bi-hourglass-split"></i> Загрузка...</div>';
137
138   try {
139     const data = await adminFetch(`/api/admin/instructions/ver
    sions?mode=${currentMode}`);
140
141     if (!data.versions || !data.versions.length) {
142       list.innerHTML = '<div class="list-group-item text-muted
    ">Нет сохранённых версий</div>';
143       return;
144     }
145
146     list.innerHTML = data.versions.map(v => `
147       <button type="button"
148         class="list-group-item list-group-item-action d-flex j
    ustify-content-between align-items-center ${v.is_active ? 'lis
    t-group-item-success' : ''}"
149         onclick="useVersion('${escapeAttr(v.filename)}', '${es
    capeAttr(v.mode)}')">
150         <div class="me-2 overflow-hidden">
151           <div class="d-flex align-items-center gap-2 mb-1">
152             <strong>${v.mode === 'chat' ? '💬 ' : '📄 '} ${escap
    eHtml(v.filename)}</strong>
153             ${v.is_active ? '<span class="badge bg-success">Ак
    тивная</span>' : ''}
154           </div>

```

```

155 |         <small class="text-muted">${new Date(v.created_at).t
| oLocaleString('ru-RU')} · ${v.size} байт</small>
156 |         <div class="text-muted small text-truncate" style="m
| ax-width:480px;">${escapeHtml(v.preview || '')}</div>
157 |     </div>
158 |     <i class="bi bi-arrow-right-circle${v.is_active ? '-fi
| ll text-success' : ' text-secondary'}"></i>
159 | </button>
160 | `).join('');
161 | } catch (e) {
162 |     list.innerHTML = `<div class="list-group-item text-danger"
| >Ошибка загрузки версий: ${e.message}</div>`;
163 | }
164 | }
165 |
166 | async function useVersion(filename, mode) {
167 |     try {
168 |         await adminFetch('/api/admin/instructions/activate', {
169 |             method: 'POST',
170 |             headers: { 'Content-Type': 'application/json' },
171 |             body: JSON.stringify({ mode, filename })
172 |         });
173 |
174 |         if (mode === currentMode) {
175 |             await loadInstruction();
176 |         }
177 |         await refreshVersions();
178 |         notify(`Версия ${filename} активирована`, 'success');
179 |     } catch (e) {
180 |         notify('Ошибка активации: ' + e.message, 'danger');
181 |     }
182 | }
183 |
184 | async function previewPrompt() {
185 |     const content = document.getElementById('instruction-editor'
| ).value.trim();
186 |     const preview = `
187 |         <strong>Контекст системной инструкции (${currentMode === `

```

```

chat' ? ' Chat' : ' Narrator'}):</strong><br>
188     <pre style="white-space:pre-wrap;max-height:400px;overflow
-y:auto;">${escapeHtml(content.substring(0, 1000))}${content.l
length > 1000 ? '\n...' : ''}</pre>
189     <strong>Тестовый запрос:</strong><br>
190     "Привет! Подтверди, что ты получил инструкцию и готов к ра
боте."
191     `;
192
193     const modal = new bootstrap.Modal(document.getElementById('p
rompt-preview-modal'));
194     document.getElementById('prompt-preview-content').innerHTML
= preview;
195     modal.show();
196 }
197
198 // — Stats —————
—————
199 function updateStats() {
200     const content = document.getElementById('instruction-editor'
).value;
201
202     document.getElementById('char-count').textContent = content.
length;
203
204     const words = content.trim().split(/\s+/).filter(w => w.leng
th > 0);
205     document.getElementById('word-count').textContent = words.le
ngth;
206
207     const estimatedTokens = Math.ceil(content.length / 3);
208     document.getElementById('token-estimate').textContent = '~'
+ estimatedTokens;
209
210     return estimatedTokens;
211 }
212
213 function escapeHtml(text) {

```

```

214 |   if (!text) return '';
215 |   const div = document.createElement('div');
216 |   div.textContent = text;
217 |   return div.innerHTML.replace(/\n/g, '<br>');
218 | }
219 |
220 | function escapeAttr(text) {
221 |   if (!text) return '';
222 |   return text.replace(/'/g, "\\'").replace(/"/g, '&quot;');
223 | }
224 |
225 | // — Initialization —————
    | _____
226 | let initialized = false;
227 |
228 | async function initInstructionsTab() {
229 |   if (initialized) return;
230 |   initialized = true;
231 |
232 |   console.log('[InstructionsTab] Initializing...');
233 |
234 |   const savedLang = localStorage.getItem('app_language') || 'r
    | u';
235 |   try {
236 |     await initI18n(savedLang);
237 |     console.log('[InstructionsTab] i18n initialized');
238 |   } catch (e) {
239 |     console.error('[InstructionsTab] i18n init error:', e);
240 |   }
241 |
242 |   // Load instruction for current mode
243 |   await loadInstruction();
244 |
245 |   // Load versions
246 |   await refreshVersions();
247 |
248 |   // Stats on input
249 |   document.getElementById('instruction-editor').addEventListener

```

```
er('input', updateStats);
250
251   try {
252       applyTranslations();
253   } catch (e) {
254       console.error('[InstructionsTab] applyTranslations error: '
255           , e);
256   }
257   console.log('[InstructionsTab] Initialized successfully');
258 }
259
260 // — Экспорт в window (необходим для inline onclick в HTML пр
    и type="module") —
261 window.initInstructionsTab = initInstructionsTab;
262 window.switchMode         = switchMode;
263 window.saveInstruction     = saveInstruction;
264 window.checkInModel       = checkInModel;
265 window.refreshVersions    = refreshVersions;
266 window.useVersion         = useVersion;
267 window.previewPrompt      = previewPrompt;
```


chatService.js

```
1 // Shared Chat Service Module
2 // Exposes window.chatService to unify API calling and message
  /error formatting.
3
4 class AudioQueuePlayer {
5   constructor(chunks, userSettings) {
6     this.chunks = chunks;
7     this.userSettings = userSettings;
8     this.currentIndex = 0;
9     this.isPlaying = false;
10    this.audioElements = new Map(); // index -> Audio
11    this.activeAudio = null;
12
13    // Prefetch first few chunks
14    this._prefetch(0);
15    this._prefetch(1);
16    this._prefetch(2);
17  }
18
19  _prefetch(index) {
20    if (index >= this.chunks.length || this.audioElements.has(
index)) return;
21
22    const text = this.chunks[index];
23    const queryParams = new URLSearchParams({
24      text: text,
25      system: this.userSettings.tts_system,
26      voice: this.userSettings.tts_voice
27    });
28    const audioUrl = `/api/tts/synthesize?${queryParams.toStri
ng()}`;
29    const audio = new Audio(audioUrl);
30    audio.load();
31    this.audioElements.set(index, audio);
32  }
33
```

```

34 |   async play() {
35 |       this.isPlaying = true;
36 |       while (this.currentIndex < this.chunks.length && this.isPl
    | aying) {
37 |           // Prefetch upcoming chunks
38 |           this._prefetch(this.currentIndex);
39 |           this._prefetch(this.currentIndex + 1);
40 |           this._prefetch(this.currentIndex + 2);
41 |           this._prefetch(this.currentIndex + 3);
42 |
43 |           const audio = this.audioElements.get(this.currentIndex);
44 |           if (!audio) {
45 |               this.currentIndex++;
46 |               continue;
47 |           }
48 |
49 |           this.activeAudio = audio;
50 |           window.chatServiceAudio = audio; // for global pause/sto
    | p integration
51 |
52 |           const playPromise = new Promise((resolve) => {
53 |               audio.onended = () => resolve();
54 |               audio.onerror = (e) => {
55 |                   console.error(`Audio error for chunk ${this.currentI
    | ndex}:`, e);
56 |                   this.stop(); // Останавливаем всю очередь при ошибке
    | загрузки
57 |                   resolve();
58 |               };
59 |           });
60 |
61 |           try {
62 |               await audio.play();
63 |               await playPromise;
64 |           } catch (err) {
65 |               console.error("Audio playback interrupted or failed:",
    | err);
66 |           }

```

```
67 |
68 |     this.currentIndex++;
69 | }
70 | this.isPlaying = false;
71 | }
72 |
73 | stop() {
74 |     this.isPlaying = false;
75 |     if (this.activeAudio) {
76 |         try {
77 |             this.activeAudio.pause();
78 |             this.activeAudio.src = "";
79 |         } catch (e) {}
80 |     }
81 |     this.audioElements.clear();
82 | }
83 | }
84 |
85 | window.chatService = {
86 |     /**
87 |      * Stops any currently playing audio or speech synthesis.
88 |      */
89 |     stop() {
90 |         if (window.chatServiceQueue) {
91 |             try {
92 |                 window.chatServiceQueue.stop();
93 |                 window.chatServiceQueue = null;
94 |             } catch (e) {
95 |                 console.error("Error stopping chatServiceQueue:", e);
96 |             }
97 |         }
98 |         if (window.chatServiceAudio) {
99 |             try {
100 |                 window.chatServiceAudio.pause();
101 |                 window.chatServiceAudio.src = "";
102 |                 window.chatServiceAudio = null;
103 |             } catch (e) {
104 |                 console.error("Error stopping chatServiceAudio:", e);
```

```
105 |     }
106 | }
107 | if ('speechSynthesis' in window) {
108 |     try {
109 |         window.speechSynthesis.cancel();
110 |     } catch (e) {
111 |         console.error("Error cancelling speechSynthesis:", e);
112 |     }
113 | }
114 | },
115 | _abortController: null,
116 |
117 | /**
118 |  * Sends a chat message to the /api/chat backend.
119 |  * Resolves to the text reply or throws the parsed error object/string.
120 |  *
121 |  * @param {string} message
122 |  * @returns {Promise<any>}
123 |  */
124 | async sendChatMessage(message, onChunk, history = [], generationConfig = {}) {
125 |     // Останавливаем любую активную озвучку перед отправкой нового сообщения
126 |     this.stop();
127 |
128 |     // Прерываем предыдущий запрос, если он еще выполняется
129 |     if (this._abortController) {
130 |         try {
131 |             this._abortController.abort();
132 |         } catch (e) {}
133 |     }
134 |     this._abortController = new AbortController();
135 |
136 |     let fullText = '';
137 |     let voiceText = '';
138 |
139 |     try {
```

```
140 |     const response = await fetch('/api/chat', {
141 |         method: 'POST',
142 |         headers: { 'Content-Type': 'application/json' },
143 |         body: JSON.stringify({ message, history, generation_co
nfig: generationConfig }),
144 |         signal: this._abortController.signal
145 |     });
146 |
147 |     if (!response.ok) {
148 |         let data;
149 |         try {
150 |             data = await response.json();
151 |         } catch (e) {
152 |             throw new Error(`Ошибка сервера (HTTP ${response.sta
tus}): ${response.statusText}`);
153 |         }
154 |         throw (data.detail || data);
155 |     }
156 |
157 |     const reader = response.body.getReader();
158 |     const decoder = new TextDecoder('utf-8');
159 |     let buffer = '';
160 |
161 |     while (true) {
162 |         const { value, done } = await reader.read();
163 |         if (done) break;
164 |
165 |         buffer += decoder.decode(value, { stream: true });
166 |         const lines = buffer.split('\n');
167 |         buffer = lines.pop();
168 |
169 |         for (const line of lines) {
170 |             const trimmed = line.trim();
171 |             if (!trimmed) continue;
172 |             if (trimmed.startsWith('data: ')) {
173 |                 try {
174 |                     const data = JSON.parse(trimmed.substring(6));
175 |                     if (data.error) {
```

```

176         throw new Error(data.error);
177     }
178     if (data.text || data.status || data.voice || data.prompt_dump) {
179         if (data.text) fullText += data.text;
180         if (data.voice) {
181             voiceText += data.voice;
182         }
183         if (onChunk) {
184             onChunk(data.text || '', data.status || '',
185 data.voice || '', data.prompt_dump || '');
186         }
187     } catch (e) {
188         console.error('Parsing SSE error:', e, trimmed);
189         throw e;
190     }
191 }
192 }
193 }
194
195 if (buffer && buffer.trim().startsWith('data: ')) {
196     const data = JSON.parse(buffer.trim().substring(6));
197     if (data.error) throw new Error(data.error);
198     if (data.text || data.status || data.voice) {
199         if (data.text) fullText += data.text;
200         if (data.voice) voiceText += data.voice;
201         if (onChunk) onChunk(data.text || '', data.status ||
202 '', data.voice || '');
203     }
204 } catch (e) {
205     if (e.name === 'AbortError') {
206         // Возвращаем Promise, который никогда не зарезолвится
207         .
208         // Это остановит старый вызов sendMessage() на мес
209 те, не вызывая catch/finally в UI-скриптах,
210         // так как они уже перекрыты новым запросом.

```

```

209         return new Promise(() => {});
210     }
211     throw e;
212 }
213
214 let followUpQuery = null;
215 const nextQueryRegex = /\[NEXT_QUERY\](.*?)\[\/NEXT_QUERY\
    ]/i;
216 const match = fullText.match(nextQueryRegex);
217 if (match) {
218     followUpQuery = match[1].trim();
219     fullText = fullText.replace(match[0], '');
220 }
221
222 return { text: fullText, voice: voiceText, followUp: follo
wUpQuery };
223 },
224
225
226 /**
227  * Formats chat messages.
228  * Converts objects into stringified JSON inside pre tags fo
r rich rendering.
229  *
230  * @param {any} text
231  * @returns {string|HTMLElement}
232  */
233 formatMessage(text) {
234     if (typeof text === 'object' && text !== null) {
235         // Если это просто объект с ответом { text, voice }, ото
бражаем только текст
236         if (text.text !== undefined) {
237             return text.text;
238         }
239         return `

```
${JSON.stringify(text, nu
ll, 2)}
```

`;

```

```
240     }
241     return text;
242 },
243
244 /**
245  * Reads the given text aloud using the browser SpeechSynthe
sis API or backend TTS.
246  *
247  * @param {string|object} text
248  */
249 async speak(text) {
250     if (!text) return;
251
252     // Останавливаем все предыдущие аудиопотоки
253     this.stop();
254
255     let cleanText = '';
256     if (typeof text === 'object' && text !== null) {
257         if (text.voice || text.text) {
258             // Если это объект { text, voice } из sendMessage
259             cleanText = text.voice || text.text;
260         } else {
261             // Если это карточка фильма
262             const title = text.title_ru || text.title;
263             const rec = text.why_watch || text.plot || '';
264             cleanText = `Найдено: ${title}. ${rec}`;
265         }
266     } else {
267         const trimmed = text.trim();
268         if (trimmed.startsWith('{') && trimmed.endsWith('}')) {
269             try {
270                 const card = JSON.parse(trimmed);
271                 const title = card.title_ru || card.title;
272                 const rec = card.why_watch || card.plot || '';
273                 cleanText = `Найдено: ${title}. ${rec}`;
274             } catch (e) {
275                 cleanText = trimmed;
276             }
277         }
278     }
279 }
```



```

311         } else {
312             currentChunk = (currentChunk + " " + trimmedSe
ntence).trim();
313         }
314     }
315     if (currentChunk) {
316         chunks.push(currentChunk.trim());
317     }
318 }
319 chunks = chunks.filter(c => c.length > 0);
320
321     if (chunks.length > 0) {
322         window.chatServiceQueue = new AudioQueuePlayer(chu
nks, userSettings);
323         await window.chatServiceQueue.play();
324     }
325     return;
326 }
327 }
328 } catch (e) {
329     console.error('Failed to speak using backend TTS, fallin
g back to browser:', e);
330 }
331
332 if ('speechSynthesis' in window) {
333     window.currentUtterance = new SpeechSynthesisUtterance(c
leanText);
334     window.currentUtterance.lang = 'ru-RU';
335
336     if (userSettings && userSettings.tts_voice) {
337         const voices = window.speechSynthesis.getVoices();
338         const selectedVoice = voices.find(v => v.name === user
Settings.tts_voice);
339         if (selectedVoice) {
340             window.currentUtterance.voice = selectedVoice;
341         }
342     }
343

```

```
344     window.speechSynthesis.speak(window.currentUtterance);
345   }
346 }
347 };
348
349 // Форматирование отображаемого названия поискового движка
350 function formatSearchEngine(engine) {
351   if (!engine) return '';
352   let eng = engine;
353   let mdl = '';
354   if (engine.includes(':') && !engine.startsWith('ollama:') &&
355       !engine.startsWith('foundry:')) {
356     const parts = engine.split(':');
357     eng = parts[0];
358     mdl = parts.slice(1).join(':');
359   }
360   const map = {
361     'gemini_cli': '🐦 gemini_cli',
362     'gemini': '🐦 gemini',
363     'agy': '🐦 agy',
364     'langchain': '🐦 langchain',
365     'playwright': '🐦 playwright'
366   };
367   const iconAndName = map[eng] || `🐦 ${eng}`;
368   if (mdl && !mdl.startsWith('chromium')) {
369     return `${iconAndName} (${mdl})`;
370   }
371   return iconAndName;
372 }
373 window.formatSearchEngine = formatSearchEngine;
374
375 window.updateChatBadges = function(modelName, searchEngine) {
376   if (modelName !== undefined && modelName !== null) {
377     window.activeModelName = modelName;
378   }
379   if (searchEngine !== undefined && searchEngine !== null) {
380     window.activeSearchEngine = searchEngine;
```

```
381 | }
382 |
383 | const curModel = window.activeModelName || '';
384 | const curSearch = window.activeSearchEngine || '';
385 |
386 | if (curModel) {
387 |     const modelBadges = document.querySelectorAll('#chat-model
388 | -badge, #chat-popup-model-badge');
389 |     modelBadges.forEach(badge => {
390 |         badge.textContent = curModel;
391 |         badge.title = `Выбранная модель ИИ: ${curModel}`;
392 |         badge.style.display = 'inline-block';
393 |     });
394 | }
395 |
396 | if (curSearch) {
397 |     const searchBadges = document.querySelectorAll('#chat-sear
398 | ch-badge, #chat-popup-search-badge');
399 |     searchBadges.forEach(badge => {
400 |         badge.textContent = formatSearchEngine(curSearch);
401 |         badge.title = `Провайдер веб-поиска: ${curSearch}`;
402 |         badge.style.display = 'inline-block';
403 |     });
404 | }
405 | };
406 |
407 | // Автоматически загружаем настройки и обновляем бейджи модели
408 | // и поиска при загрузке страницы
409 | document.addEventListener('DOMContentLoaded', async () => {
410 |     try {
411 |         const response = await fetch('/auth/settings');
412 |         if (response.ok) {
413 |             const settings = await response.json();
414 |             const modelName = settings.model || '';
415 |             const searchEngine = settings.search_engine || '';
416 |             window.activeModelName = modelName;
417 |             window.activeSearchEngine = searchEngine;
```

```
416 |     window.updateChatBadges(modelName, searchEngine);
417 |     // На случай если DOM элементы добавились/отрендерились
    | позже
418 |     setTimeout(() => window.updateChatBadges(), 500);
419 |     setTimeout(() => window.updateChatBadges(), 1500);
420 | }
421 | } catch (e) {
422 |     console.error('Failed to load active model / search badge:
    | ', e);
423 | }
424 | });
425 |
```

i18n.js

```
1 /**
2  * i18n.js – Internationalization module
3  *
4  * Uses i18next (loaded via CDN in index.html).
5  *
6  * HTML attribute conventions:
7  *   data-i18n="key" – sets element innerHTML
8  *   data-i18n-placeholder="key" – sets input/textarea placeholder
9  *   data-i18n-title="key" – sets title attribute
10 *
11 * Selected language is saved to localStorage.
12 * On load: reads language from localStorage, falls back to browser lang, then 'en'.
13 */
14
15 const SUPPORTED = ['en', 'ru', 'he'];
16
17 let i18nInitialized = false;
18
19 /**
20 * Initialize i18next and apply translations to the DOM.
21 * Called once from main.js after DOMContentLoaded.
22 * @param {string} lang - language code (may be empty)
23 */
24 export async function initI18n(lang) {
25     // If lang is explicitly set – use it.
26     // If empty/null – detect from localStorage, browser, fall back to 'en'.
27     const resolved = SUPPORTED.includes(lang) ? lang : detectLang();
28
29     // Always load English as fallback bundle
30     const [mainRes, enRes] = await Promise.all([
31         loadLocale(resolved),
32         resolved !== 'en' ? loadLocale('en') : Promise.resolve
```

```

    (null),
33     });
34
35     await i18next.init({
36         lng: resolved,
37         fallbackLng: 'en',
38         resources: { [resolved]: { translation: mainRes } },
39         interpolation: { escapeValue: false },
40     });
41
42     if (enRes) {
43         i18next.addResourceBundle('en', 'translation', enRes,
true, true);
44     }
45
46     applyTranslations();
47     syncSelectors(resolved);
48     i18nInitialized = true;
49 }
50
51 /**
52  * Switch language, re-apply all translations, save to localSt
orage.
53  * @param {string} lang
54  */
55 export async function switchLang(lang) {
56     if (!SUPPORTED.includes(lang)) return;
57
58     // Lazy-load locale bundle if not yet loaded
59     if (!i18next.hasResourceBundle(lang, 'translation')) {
60         const res = await loadLocale(lang);
61         i18next.addResourceBundle(lang, 'translation', res, tr
ue, true);
62     }
63
64     await i18next.changeLanguage(lang);
65     applyTranslations();
66     syncSelectors(lang);

```

```

67 |
68 |     // Persist to localStorage
69 |     localStorage.setItem('app_language', lang);
70 | }
71 |
72 | /**
73 |  * Walk the DOM and update all elements with data-i18n* attributes.
74 |  * Safe to call multiple times – idempotent.
75 |  */
76 | export function applyTranslations() {
77 |     // Text / HTML content
78 |     document.querySelectorAll('[data-i18n]').forEach(el => {
79 |         const val = i18next.t(el.getAttribute('data-i18n'));
80 |         if (val) el.innerHTML = val;
81 |     });
82 |
83 |     // Input / textarea placeholder
84 |     document.querySelectorAll('[data-i18n-placeholder]').forEach(el => {
85 |         const val = i18next.t(el.getAttribute('data-i18n-placeholder'));
86 |         if (val) el.placeholder = val;
87 |     });
88 |
89 |     // Tooltip / title attribute
90 |     document.querySelectorAll('[data-i18n-title]').forEach(el => {
91 |         const val = i18next.t(el.getAttribute('data-i18n-title'));
92 |         if (val) el.title = val;
93 |     });
94 | }
95 |
96 | async function loadLocale(lang, bundle = 'main') {
97 |     const pathName = window.location.pathname;
98 |     let paths = [];
99 |

```



```

100 |     if (pathName.includes('/user_tts')) {
101 |         paths = [
102 |             `/html/user_tts/locales/user_${lang}.json`,
103 |             `/html/locales/${lang}.json`
104 |         ];
105 |     } else if (pathName.startsWith('/user')) {
106 |         paths = [
107 |             `/html/locales/user_${lang}.json`,
108 |             `/html/locales/${lang}.json`
109 |         ];
110 |     } else {
111 |         paths = [
112 |             `/html/locales/${lang}.json`
113 |         ];
114 |     }
115 |
116 |     for (const path of paths) {
117 |         try {
118 |             const r = await fetch(path);
119 |             if (r.ok) return await r.json();
120 |         } catch (e) {
121 |             console.log(`[i18n] Try next path: ${path}`);
122 |         }
123 |     }
124 |     console.error(`[i18n] Failed to load locale: ${lang}`);
125 |     return {};
126 | }
127 |
128 | function detectLang() {
129 |     // Check localStorage first
130 |     const saved = localStorage.getItem('app_language');
131 |     if (SUPPORTED.includes(saved)) return saved;
132 |
133 |     // Fall back to browser language
134 |     const browser = (navigator.language || 'en').split('-')[0]
135 |     ;
136 |     return SUPPORTED.includes(browser) ? browser : 'en';

```

```
137 |
138 | /**
139 |  * Sync all language selector elements to the current language
140 |  *
141 |  */
142 | function syncSelectors(lang) {
143 |     document.querySelectorAll('.lang-selector').forEach(sel =>
144 |     {
145 |         sel.value = lang;
146 |     });
147 | }
148 | /**
149 |  * Get current language
150 |  */
151 | export function getCurrentLang() {
152 |     return i18next?.language || detectLang();
153 | }
```

main.js

```
1 // — MAIN.JS —————  
2  
3 import { initI18n, switchLang, applyTranslations } from './i18n.js';  
4 import { initTheme, setTheme, getThemeMode, getResolvedTheme }  
5   from './theme.js';  
6  
7 // Make switchLang and theme functions available globally  
8 window.switchLang = switchLang;  
9 window.applyTranslations = applyTranslations;  
10 window.setTheme = setTheme;  
11 window.getThemeMode = getThemeMode;  
12 window.getResolvedTheme = getResolvedTheme;  
13 window.initUserSettings = initUserSettings;  
14 window.refreshUserProfile = refreshUserProfile;  
15  
16 // Initialize HELP content when help tab is loaded  
17 document.addEventListener('DOMContentLoaded', async () => {  
18   console.log('Starting initialization...');  
19  
20   // Initialize theme first  
21   initTheme();  
22   console.log('Theme initialized');  
23  
24   // Initialize i18n first  
25   const savedLang = localStorage.getItem('app_language') || 'ru';  
26   await initI18n(savedLang);  
27   console.log('i18n initialized');  
28  
29   // Setup language selector  
30   document.querySelectorAll('.lang-selector').forEach((sel) =>  
    {
```

```
31     sel.value = savedLang;
32     sel.addEventListener('change', (e) => {
33         switchLang(e.target.value);
34     });
35 });
36
37 // Initialize User Settings & Google OAuth
38 await initUserSettings();
39 console.log('User settings initialized');
40
41
42 await initHelpContent();
43 console.log('HELP system initialized');
44
45 console.log('Loading tabs...');
46 const v = '20260825_cleanup_tabs';
47 await Promise.all([
48     loadTabContent('chat', `/html/chat/index.html?v=${v}`),
49     loadTabContent('plugins', `/html/plugins_tab/index.html?v=
50     ${v}`, `/html/plugins_tab/main.js?v=${v}`),
51     loadTabContent('admin', `/html/admin/index.html?v=${v}`),
52     loadTabContent('help', `/html/help/index.html?v=${v}`),
53 ]);
54 console.log('All tabs loaded');
55
56 // Apply translations after all tabs are loaded
57 applyTranslations();
58
59 // Синхронизация видимости вкладок плагинов
60 try {
61     const pluginsResp = await fetch('/api/admin/plugins');
62     if (pluginsResp.ok) {
63         const pluginsData = await pluginsResp.json();
64         if (window.syncPluginTabsVisibility && pluginsData && pl
65         uuginsData.plugins) {
66             window.syncPluginTabsVisibility(pluginsData.plugins);
67         }
68     }
69 }
```

```
67 | } catch (err) {
68 |     console.error('Ошибка синхронизации видимости плагинов:',
    | err);
69 | }
70 |
71 | // Инициализация первой вкладки
72 | const chatTab = document.querySelector('[data-bs-target="#ta
    | b-chat"]');
73 | if (chatTab) {
74 |     chatTab.classList.add('active');
75 |     document.getElementById('tab-chat').classList.add('show',
    | 'active');
76 | }
77 | console.log('Initialization complete');
78 | });
79 |
80 | // Загрузка контента вкладки
81 | async function loadTabContent(tabName, url, scriptUrl = null)
    | {
82 |     try {
83 |         console.log(`Loading tab ${tabName} from ${url}...`);
84 |         const response = await fetch(url);
85 |         console.log(`Response status: ${response.status} for ${tab
    | Name}`);
86 |         if (!response.ok) throw new Error(`HTTP ${response.status}
    | `);
87 |         const html = await response.text();
88 |         const container = document.getElementById(`tab-${tabName}`
    | );
89 |         container.innerHTML = html;
90 |         console.log(`Content set for tab ${tabName}, length: ${htm
    | l.length}`);
91 |
92 |         // Загрузка JS файла вкладки
93 |         const script = document.createElement('script');
94 |         script.src = scriptUrl || `/html/${tabName}/main.js?v=2026
    | 0824_plugin_manager`;
95 |         if (tabName === 'admin') {
```

```

96     script.type = 'module';
97 }
98 script.onload = () => {
99     console.log(`  Загружен JS вкладки: ${tabName}`);
100    // Call init function if exists
101    if (window[`init${tabName.charAt(0).toUpperCase() + tabName.slice(1)}Tab`]) {
102        window[`init${tabName.charAt(0).toUpperCase() + tabName.slice(1)}Tab`]();
103    }
104    // Apply translations to newly loaded tab content
105    window.applyTranslations?.();
106 };
107 script.onerror = () => console.error(`  Ошибка загрузки JS вкладки ${tabName}`);
108 container.appendChild(script);
109
110 console.log(`  Загружена вкладка: ${tabName}`);
111 } catch (e) {
112     console.error(`  Ошибка загрузки вкладки ${tabName}:`, e);
113     document.getElementById(`tab-${tabName}`).innerHTML =
114         `<div class="alert alert-danger">Ошибка загрузки вкладки
115         : ${e.message}</div>`;
116 }
117
118 // Обработка переключения вкладок
119 document.getElementById('mainTabs')?.addEventListener('shown.bs.tab', (e) => {
120     const target = e.target.dataset.bsTarget.replace('#tab-', '');
121     console.log(`Переключение на вкладку: ${target}`);
122 });
123
124 // Модуль для работы с API
125 window.api = {
126     async fetch(url, options = {}) {
127         const response = await fetch(url, options);

```

```

128     if (!response.ok) {
129         let msg = response.statusText;
130         try {
131             const data = await response.json();
132             msg = data.detail || msg;
133         } catch {}
134         throw new Error(`${response.status} ${msg}`);
135     }
136     return response.json();
137 }
138 };
139
140 // — HELP SYSTEM —————
141 // HELP content stored in JavaScript (will be populated from h
142 // elp tab)
143
144 window.HELP_CONTENT = {};
145
146 // Initialize HELP content when help tab is loaded
147 async function initHelpContent() {
148     // Overview
149     window.HELP_CONTENT['overview'] = `
150     <h4>□ Обзор проекта</h4>
151     <p>ai-breadboard – это интегрированная платформа персональ
152     ного AI-ассистента с поддержкой Google Gemini, Antigravity, Fo
153     undry, Ollama, RAG и системного управления.</p>
154     <h5>Основные возможности</h5>
155     <ul>
156     <li><strong>Чат</strong> – диалог и выполнение команд че
157     рез AI (Gemini, AGY, Foundry, Ollama)</li>
158     <li><strong>Управление</strong> – мониторинг сервера, ди
159     сков, пользователей, API-ключей и логирование</li>
160     <li><strong>Плагины</strong> – расширение возможностей ч
161     ерез модульную систему</li>
162     <li><strong>RAG и Агенты</strong> – семантический поиск
163     по базам знаний и автономные ReAct-агенты</li>
164     </ul>

```

```
158 | `;  
159 |  
160 | // Code Rules  
161 | window.HELP_CONTENT['code_rules'] = `  
162 |     <h4>□ Правила кодирования (CODE_RULES)</h4>  
163 |     <p>Проект использует Engineering Standard для поддержания  
164 |     качества кода.</p>  
165 |     <h5>Ключевые принципы</h5>  
166 |     <ul>  
167 |         <li><strong>Читаемость важнее краткости</strong></li>  
168 |         <li><strong>Принцип единственной ответственности</strong>  
169 |     ></li>  
170 |         <li><strong>Явное лучше неявного</strong></li>  
171 |         <li><strong>Ранний возврат и отказоустойчивость</strong>  
172 |     </li>  
173 |     </ul>  
174 |     <h5>Запрещено использовать None</h5>  
175 |     <p>Вместо <code>None</code> использовать:</p>  
176 |     <ul>  
177 |         <li>Числа: <code>0</code> или <code>0.0</code></li>  
178 |         <li>Строки: <code>' '</code> (пустая строка)</li>  
179 |         <li>Булевы: <code>>false</code></li>  
180 |         <li>Коллекции: <code>[]</code> или <code>{}</code></li>  
181 |     </ul>  
182 | `;  
183 |  
184 | // Architecture  
185 | window.HELP_CONTENT['architecture'] = `  
186 |     <h4>□ Архитектура</h4>  
187 |     <h5>Технологический стек</h5>  
188 |     <ul>  
189 |         <li><strong>FastAPI</strong> – веб-фреймворк для Python<  
190 |     </li>  
191 |         <li><strong>UVicorn</strong> – ASGI сервер</li>  
192 |         <li><strong>Bootstrap 5</strong> – UI фреймворк</li>  
193 |         <li><strong>SQLite</strong> – база данных</li>  
194 |     </ul>  
195 |     <h5>AI интеграции</h5>
```



```

192     <ul>
193         <li><strong>Google Gemini / Antigravity / Foundry / Olla
194 ма</strong> – генерация контента и инференс</li>
195         <li><strong>RAG</strong> – семантический поиск по докуме
196 нтам и истории</li>
197         <li><strong>LangChain Agents</strong> – автономные ReAct
198 -агенты</li>
199     </ul>
200 `;
201
202 // FastAPI
203 window.HELP_CONTENT['fastapi'] = `
204     <h4>□ FastAPI</h4>
205     <h5>Роутеры</h5>
206     <table class="table table-sm">
207         <tr><td><code>/api/chat</code></td><td>Взаимодействие с
208 AI моделями и WS-чат</td></tr>
209         <tr><td><code>/api/admin</code></td><td>Системное админи
210 стрирование, пользователи и плагины</td></tr>
211         <tr><td><code>/api/control</code></td><td>Управление сес
212 сиями и накопителями (дисками)</td></tr>
213         <tr><td><code>/api/keys</code></td><td>Управление API-кл
214 ючами и моделями</td></tr>
215         <tr><td><code>/api/logs</code></td><td>Системные логи и
216 аналитика</td></tr>
217     </table>
218     <h5>Конфигурация</h5>
219     <p>Файл: <code>config.json</code></p>
220 `;
221
222 // Gemini
223 window.HELP_CONTENT['gemini'] = `
224     <h4>□ Gemini AI</h4>
225     <h5>Использование</h5>
226     <ul>
227         <li>Генерация метаданных для медиа</li>
228         <li>Классификация по жанрам</li>
229         <li>Генерация описаний и рецензий</li>

```

```

222     <li>Поиск по RAG-индексу</li>
223 </ul>
224 <h5>Управление ключами</h5>
225 <p>Файл: <code>src/secrets/gemini_keys.json</code></p>
226 `;
227
228 // Configuration
229 window.HELP_CONTENT['configuration'] = `
230 <h4>□ Конфигурация</h4>
231 <h5>Файлы конфигурации</h5>
232 <table class="table table-sm">
233     <tr><td><code>.env</code></td><td>Секреты (API ключи, па
234     роли)</td></tr>
235     <tr><td><code>src/fastapi/config.json</code></td><td>Пуб
236     личные настройки</td></tr>
237     <tr><td><code>plugins/media_organizer/media_paths.txt</c
238     ode></td><td>Пути к медиатеке</td></tr>
239 </table>
240 `;
241
242 // CLI Commands
243 window.HELP_CONTENT['cli_commands'] = `
244 <h4>□ Командная строка</h4>
245 <p>Все команды медиатеки доступны в CLI и веб-интерфейсе.<
246 /p>
247 <h5>run_media_organizer.py</h5>
248 <p>Полный функционал управления медиатекой:</p>
249 <table class="table table-sm">
250     <tr><td><code>--disk 1</code></td><td>Имя диска (наприме
251     п: 1, 2, 3)</td></tr>
252     <tr><td><code>--path E: L:</code></td><td>Пути для скани
253     рования</td></tr>
254     <tr><td><code>--key имя_ключа</code></td><td>Ключ Gemini
255     API</td></tr>
256     <tr><td><code>--title</code></td><td>Генерация отчёта из
257     БД</td></tr>
258     <tr><td><code>--audit</code></td><td>Только аудит (без с

```

```

канирования)</td></tr>
252     <tr><td><code>--rebuild</code></td><td>Восстановление из
      JSON</td></tr>
253     <tr><td><code>--rebuild-db</code></td><td>Консолидация д
ублей в БД</td></tr>
254     <tr><td><code>--rebuild-rag</code></td><td>Перестройка R
AG-индекса</td></tr>
255   </table>
256
257   <h5>series_collector.py</h5>
258   <p>Сбор и анализ эпизодов:</p>
259   <table class="table table-sm">
260     <tr><td><code>--scan</code></td><td>Сканирование сериало
в</td></tr>
261     <tr><td><code>--duplicates</code></td><td>Проверка дубли
катов</td></tr>
262     <tr><td><code>--integrity</code></td><td>Проверка целост
ности</td></tr>
263     <tr><td><code>--report</code></td><td>Генерация отчёта</
td></tr>
264     <tr><td><code>--all</code></td><td>Все проверки сразу</t
d></tr>
265   </table>
266
267   <h5>Примеры:</h5>
268   <pre><code>
269 # Полное сканирование диска 1
270 py run_media_organizer.py --disk 1 --path E: L:
271
272 # Только аудит без сканирования
273 py run_media_organizer.py --disk 2 --audit
274
275 # Генерация отчёта из БД
276 py run_media_organizer.py --disk 1 --title
277
278 # Консолидация дублей
279 py run_media_organizer.py --rebuild-db
280

```

```
281 # Сканирование сериалов
282 py series_collector.py --scan
283
284 # Все проверки сразу
285 py series_collector.py --all
286     </code></pre>
287 `;
288
289 console.log('HELP content initialized');
290 }
291
292 // Show help modal
293 function showHelpModal(key) {
294     const content = window.HELP_CONTENT[key] || '<p>Информация н
    е найдена</p>';
295     document.getElementById('help-modal-content').innerHTML = co
    ntent;
296
297     const modal = new bootstrap.Modal(document.getElementById('h
    elp-modal'));
298     modal.show();
299 }
300
301 // Show help tooltip/popover for element
302 function showHelpTooltip(element, content) {
303     const options = {
304         title: 'Помощь',
305         content: content,
306         html: true,
307         placement: 'top',
308         trigger: 'hover focus'
309     };
310
311     const popover = new bootstrap.Popover(element, options);
312     popover.show();
313 }
314
```

theme.js

```
1 /**
2  * theme.js – Theme management module (Light / Dark / System)
3  *
4  * Supports 3 modes:
5  *   - 'light': Forces light theme
6  *   - 'dark': Forces dark theme
7  *   - 'system': Automatically follows OS / browser color scheme preferences
8  *
9  * Persists selected mode to localStorage under key 'theme'.
10 */
11
12 const STORAGE_KEY = 'theme';
13 const DEFAULT_MODE = 'system';
14
15 let mediaQueryListenerAttached = false;
16
17 /**
18  * Get the user's stored theme preference or default to 'system'.
19  * @returns {'light' | 'dark' | 'system'}
20  */
21 export function getThemeMode() {
22   const saved = localStorage.getItem(STORAGE_KEY);
23   if (saved === 'light' || saved === 'dark' || saved === 'system') {
24     return saved;
25   }
26   return DEFAULT_MODE;
27 }
28
29 /**
30  * Resolve the effective theme ('light' or 'dark') based on the mode.
31  * @param {'light' | 'dark' | 'system'} mode
32  * @returns {'light' | 'dark'}
```

```

33 | */
34 | export function getResolvedTheme(mode = getThemeMode()) {
35 |   if (mode === 'system') {
36 |     return window.matchMedia && window.matchMedia('(prefers-co
    | lor-scheme: dark)').matches
37 |       ? 'dark'
38 |       : 'light';
39 |   }
40 |   return mode;
41 | }
42 |
43 | /**
44 |  * Set the theme mode, update DOM attributes, UI icons, checkm
    | arks, and persist.
45 |  * @param {'light' | 'dark' | 'system'} mode
46 |  */
47 | export function setTheme(mode) {
48 |   if (mode !== 'light' && mode !== 'dark' && mode !== 'system'
    | ) {
49 |     mode = DEFAULT_MODE;
50 |   }
51 |
52 |   localStorage.setItem(STORAGE_KEY, mode);
53 |   applyThemeToDOM(mode);
54 | }
55 |
56 | /**
57 |  * Apply the given theme mode to documentElement and sync UI c
    | ontrols.
58 |  * @param {'light' | 'dark' | 'system'} mode
59 |  */
60 | function applyThemeToDOM(mode) {
61 |   const resolved = getResolvedTheme(mode);
62 |
63 |   // Set data-theme, data-bs-theme (Bootstrap 5.3+), and data-
    | theme-mode
64 |   document.documentElement.setAttribute('data-theme', resolved
    | );

```

```
65 | document.documentElement.setAttribute('data-bs-theme', resolved);
66 | document.documentElement.setAttribute('data-theme-mode', mode);
67 |
68 | // Update theme button icon and title
69 | updateThemeIcon(mode, resolved);
70 |
71 | // Update dropdown checkmarks & active states
72 | updateThemeDropdownUI(mode);
73 |
74 | // Sync any theme select elements (e.g., in user settings)
75 | document.querySelectorAll('.theme-selector, #settings-theme')
    | ).forEach((sel) => {
76 |     if (sel.value !== mode) {
77 |         sel.value = mode;
78 |     }
79 | });
80 |
81 | // Dispatch custom event for tabs/components that need to react
82 | window.dispatchEvent(new CustomEvent('themechanged', {
83 |     detail: { mode, resolved }
84 | }));
85 | }
86 |
87 | /**
88 |  * Update the theme toggle / dropdown icon.
89 |  * @param {'light' | 'dark' | 'system'} mode
90 |  * @param {'light' | 'dark'} resolved
91 |  */
92 | function updateThemeIcon(mode, resolved) {
93 |     const iconEl = document.getElementById('theme-icon');
94 |     const toggleBtn = document.getElementById('theme-toggle');
95 |
96 |     let iconClass = 'bi-circle-half text-info';
97 |     let titleText = 'Системная тема';
98 |
```

```
99 | if (mode === 'light') {
100 |     iconClass = 'bi-sun-fill text-warning';
101 |     titleText = 'Светлая тема';
102 | } else if (mode === 'dark') {
103 |     iconClass = 'bi-moon-stars-fill text-primary';
104 |     titleText = 'Темная тема';
105 | } else {
106 |     // system mode
107 |     iconClass = 'bi-circle-half text-info';
108 |     titleText = `Системная тема (${resolved === 'dark' ? 'тёмн
    ая' : 'светлая'})`;
109 | }
110 |
111 | if (iconEl) {
112 |     iconEl.className = `bi ${iconClass}`;
113 | }
114 |
115 | const dropdownBtn = document.getElementById('theme-dropdown'
    );
116 | if (dropdownBtn) {
117 |     dropdownBtn.setAttribute('title', titleText);
118 | }
119 |
120 | // Legacy / fallback toggle button
121 | if (toggleBtn) {
122 |     toggleBtn.innerHTML = `
```



```

    ) => {
134     const target = icon.getAttribute('data-theme-check');
135     if (target === activeMode) {
136         icon.classList.remove('d-none');
137     } else {
138         icon.classList.add('d-none');
139     }
140 });
141
142 // Update active state on dropdown items
143 document.querySelectorAll('[data-theme-value]').forEach((item) => {
144     const val = item.getAttribute('data-theme-value');
145     if (val === activeMode) {
146         item.classList.add('active');
147     } else {
148         item.classList.remove('active');
149     }
150 });
151 }
152
153 /**
154  * Initialize theme listeners and apply the current theme on page load.
155  */
156 export function initTheme() {
157     const currentMode = getThemeMode();
158     applyThemeToDOM(currentMode);
159
160     // Attach OS color-scheme change listener once
161     if (!mediaQueryListenerAttached && window.matchMedia) {
162         try {
163             const mediaQuery = window.matchMedia('(prefers-color-scheme: dark)');
164             const handler = () => {
165                 if (getThemeMode() === 'system') {
166                     applyThemeToDOM('system');
167                 }

```

```
168 |     };
169 |     if (mediaQuery.addEventListener) {
170 |         mediaQuery.addEventListener('change', handler);
171 |     } else if (mediaQuery.addListener) {
172 |         mediaQuery.addListener(handler);
173 |     }
174 |     mediaQueryListenerAttached = true;
175 | } catch (e) {
176 |     console.warn('[theme] Failed to attach media query listener:', e);
177 | }
178 | }
179 |
180 | // Bind click listeners for theme dropdown items
181 | document.querySelectorAll('[data-theme-value]').forEach((btn) => {
182 |     btn.addEventListener('click', (e) => {
183 |         e.preventDefault();
184 |         const selected = btn.getAttribute('data-theme-value');
185 |         if (selected) {
186 |             setTheme(selected);
187 |         }
188 |     });
189 | });
190 |
191 | // Bind legacy / simple toggle button if present without dropdown
192 | const themeToggle = document.getElementById('theme-toggle');
193 | if (themeToggle && !themeToggle.hasAttribute('data-bs-toggle')) {
194 |     themeToggle.addEventListener('click', () => {
195 |         const mode = getThemeMode();
196 |         // Cycle: system -> light -> dark -> system
197 |         const cycleMap = { system: 'light', light: 'dark', dark: 'system' };
198 |         const nextMode = cycleMap[mode] || 'system';
199 |         setTheme(nextMode);
200 |     });
```

```
201 | }
202 |
203 | // Bind theme selector dropdowns if present
204 | document.querySelectorAll('.theme-selector, #settings-theme'
    | ).forEach((sel) => {
205 |     sel.value = currentMode;
206 |     sel.addEventListener('change', (e) => {
207 |         setTheme(e.target.value);
208 |     });
209 | });
210 | }
211 |
212 | // Make available globally on window
213 | window.setTheme = setTheme;
214 | window.getThemeMode = getThemeMode;
215 | window.getResolvedTheme = getResolvedTheme;
216 | window.initTheme = initTheme;
217 |
```

userSettings.js

```
1 /**
2  * userSettings.js – User Profile & Settings Management
3  * Supports Google OAuth synchronization, profile status, lang
4  * uage & theme controls.
5  */
6 'use strict';
7
8 export async function initUserSettings() {
9   await refreshUserProfile();
10
11   // Handle Google Login Click
12   document.querySelectorAll('.btn-google-login, #user-settings
13   -google-btn').forEach((btn) => {
14     btn.addEventListener('click', (e) => {
15       e.preventDefault();
16       const nextUrl = window.location.pathname + window.locati
17       on.search;
18       window.location.href = `/auth/google?next=${encodeURIComponent(
19       nextUrl)}`;
20     });
21   });
22
23   // Handle Logout Click
24   document.querySelectorAll('.btn-user-logout, #user-settings-
25   logout-btn').forEach((btn) => {
26     btn.addEventListener('click', async (e) => {
27       e.preventDefault();
28       try {
29         await fetch('/auth/logout', { method: 'POST' });
30         window.location.reload();
31       } catch (err) {
32         console.error('Logout error:', err);
33       }
34     });
35   });
36 });
```

```

32 |
33 | // Re-sync modal fields on modal show
34 | const modalEl = document.getElementById('userSettingsModal')
    | ;
35 | if (modalEl) {
36 |     modalEl.addEventListener('show.bs.modal', async () => {
37 |         await refreshUserProfile();
38 |         const savedLang = localStorage.getItem('app_language') |
    | | 'ru';
39 |         document.querySelectorAll('.lang-selector').forEach((s)
    | => {
40 |             s.value = savedLang;
41 |         });
42 |         const savedTheme = localStorage.getItem('theme') || 'sys
    | tem';
43 |         document.querySelectorAll('.theme-selector').forEach((s)
    | => {
44 |             s.value = savedTheme;
45 |         });
46 |     });
47 | }
48 |
49 |
50 | export async function refreshUserProfile() {
51 |     try {
52 |         const res = await fetch('/auth/check');
53 |         if (!res.ok) return;
54 |         const data = await res.json();
55 |
56 |         const authCard = document.getElementById('user-settings-au
    | th-card');
57 |         const guestCard = document.getElementById('user-settings-g
    | uest-card');
58 |         const navGoogleBtn = document.getElementById('nav-google-b
    | tn');
59 |         const userSettingsBtn = document.getElementById('user-sett
    | ings-btn');
60 |         const navAvatar = document.getElementById('nav-user-avatar

```

```

    ');
61     const navIcon = document.getElementById('nav-user-icon');
62     const navLabel = document.getElementById('nav-user-label')
    ;
63
64     if (data && data.authenticated) {
65         // User is authenticated
66         if (authCard) authCard.classList.remove('d-none');
67         if (guestCard) guestCard.classList.add('d-none');
68
69         // Navbar: show user profile pill, hide Google register
button
70         if (navGoogleBtn) navGoogleBtn.classList.add('d-none');
71         if (userSettingsBtn) {
72             userSettingsBtn.classList.remove('d-none');
73             userSettingsBtn.classList.add('d-flex');
74         }
75
76         const name = data.name || data.email || 'Пользователь';
77         const email = data.email || '';
78         const picture = data.picture || '';
79         const role = data.role || (data.is_admin ? 'Admin' : 'Us
er');
80         const hasGoogle = Boolean(data.has_google);
81
82         // Update modal fields
83         const avatarEl = document.getElementById('user-settings-
avatar');
84         const nameEl = document.getElementById('user-settings-na
me');
85         const emailEl = document.getElementById('user-settings-e
mail');
86         const roleBadge = document.getElementById('user-settings
-role-badge');
87         const googleBadge = document.getElementById('user-settin
gs-google-badge');
88
89         if (avatarEl) {

```

```

90         if (picture) {
91             avatarEl.src = picture;
92             avatarEl.classList.remove('d-none');
93         } else {
94             avatarEl.classList.add('d-none');
95         }
96     }
97     if (nameEl) nameEl.textContent = name;
98     if (emailEl) emailEl.textContent = email;
99     if (roleBadge) {
100         roleBadge.textContent = role;
101         roleBadge.className = `badge ${role.toLowerCase() ===
'admin' ? 'bg-danger' : 'bg-primary'}`;
102     }
103     if (googleBadge) {
104         if (hasGoogle) {
105             googleBadge.innerHTML = '<i class="bi bi-google text
-danger me-1"></i>Google OAuth: <span class="text-success fw-b
old">Синхронизирован</span>';
106         } else {
107             googleBadge.innerHTML = '<i class="bi bi-google text
-muted me-1"></i>Google OAuth: <span class="text-warning">Не п
одключен</span> <button type="button" class="btn btn-sm btn-ou
tline-danger ms-2 py-0 px-2 btn-google-login">Синхронизировать
</button>';
108         }
109     }
110
111     // Update navbar button
112     if (navAvatar && picture) {
113         navAvatar.src = picture;
114         navAvatar.classList.remove('d-none');
115         if (navIcon) navIcon.classList.add('d-none');
116     }
117     if (navLabel) {
118         const shortName = name.split(' ')[0] || name;
119         navLabel.textContent = shortName;
120     }

```

```
121 |     } else {
122 |         // Guest / unauthenticated
123 |         if (authCard) authCard.classList.add('d-none');
124 |         if (guestCard) guestCard.classList.remove('d-none');
125 |
126 |         // Navbar: show full Google register/sign-in button, hide user profile pill
127 |         if (navGoogleBtn) {
128 |             navGoogleBtn.classList.remove('d-none');
129 |             navGoogleBtn.classList.add('d-flex');
130 |         }
131 |         if (userSettingsBtn) {
132 |             userSettingsBtn.classList.add('d-none');
133 |             userSettingsBtn.classList.remove('d-flex');
134 |         }
135 |
136 |         if (navAvatar) navAvatar.classList.add('d-none');
137 |         if (navIcon) navIcon.classList.remove('d-none');
138 |         if (navLabel) navLabel.textContent = 'User Settings';
139 |     }
140 | } catch (err) {
141 |     console.error('Failed to refresh user profile:', err);
142 | }
143 | }
144 |
145 | window.initUserSettings = initUserSettings;
146 | window.refreshUserProfile = refreshUserProfile;
147 |
```


main.js

```
1 // Gemini Models & APIs management tab logic
2
3 async function initModelsTab() {
4   const modelSelect = document.getElementById('models-tab-select');
5   const saveBtn = document.getElementById('btn-models-tab-save');
6   const keysListBody = document.getElementById('keys-list-body');
7   const refreshKeysBtn = document.getElementById('btn-refresh-keys');
8   const addKeyBtn = document.getElementById('btn-add-key');
9   const saveAgyBtn = document.getElementById('btn-save-agy');
10  const saveFoundryBtn = document.getElementById('btn-save-foundry');
11  const saveOllamaBtn = document.getElementById('btn-save-ollama');
12  const saveBtnInstr = document.getElementById('btn-save-instruction');
13  const reloadBtnInstr = document.getElementById('btn-reload-instruction');
14
15  // 1. Bind event handlers immediately
16  if (saveBtnInstr) saveBtnInstr.onclick = saveSystemInstruction;
17  if (reloadBtnInstr) reloadBtnInstr.onclick = loadSystemInstruction;
18
19  if (saveAgyBtn) {
20    saveAgyBtn.onclick = async () => {
21      const enabled = document.getElementById('agy-enabled')?.checked ?? true;
22      const model = document.getElementById('agy-model')?.value || 'agy-flash';
23      const key = document.getElementById('agy-key')?.value.trim() || '';
```

```
24 |
25 |     saveAgyBtn.disabled = true;
26 |     try {
27 |         await window.api.fetch('/api/agy/config', {
28 |             method: 'POST',
29 |             headers: { 'Content-Type': 'application/json' },
30 |             body: JSON.stringify({ enabled, model, key })
31 |         });
32 |         showModelsNotification('Настройки AntigraVity (AGY) уc
пешно сохранены', 'success');
33 |         if (modelSelect && saveBtn) await loadTabModels(modelS
elect, saveBtn);
34 |     } catch (err) {
35 |         console.error('Ошибка сохранения AntigraVity:', err);
36 |         showModelsNotification('Ошибка сохранения: ' + err.mes
sage, 'danger');
37 |     } finally {
38 |         saveAgyBtn.disabled = false;
39 |     }
40 | };
41 | }
42 |
43 | if (saveFoundryBtn) {
44 |     saveFoundryBtn.onclick = async () => {
45 |         const enabled = document.getElementById('foundry-enabled
')?.checked || false;
46 |         const url = document.getElementById('foundry-url')?.valu
e.trim() || '';
47 |         const key = document.getElementById('foundry-key')?.valu
e.trim() || '';
48 |
49 |         saveFoundryBtn.disabled = true;
50 |         try {
51 |             await window.api.fetch('/api/foundry/config', {
52 |                 method: 'POST',
53 |                 headers: { 'Content-Type': 'application/json' },
54 |                 body: JSON.stringify({ enabled, url, key })
55 |             });
```

```
56 |         showModelsNotification('Настройки Microsoft Foundry у  
    | пешно сохранены', 'success');  
57 |         if (modelSelect && saveBtn) await loadTabModels(modelS  
    | elect, saveBtn);  
58 |     } catch (err) {  
59 |         console.error('Ошибка сохранения Foundry:', err);  
60 |         showModelsNotification('Ошибка сохранения: ' + err.mes  
    | sage, 'danger');  
61 |     } finally {  
62 |         saveFoundryBtn.disabled = false;  
63 |     }  
64 | };  
65 | }  
66 |  
67 | if (saveOllamaBtn) {  
68 |     saveOllamaBtn.onclick = async () => {  
69 |         const enabled = document.getElementById('ollama-enabled'  
    | )?.checked || false;  
70 |         const url = document.getElementById('ollama-url')?.value  
    | .trim() || '';  
71 |  
72 |         saveOllamaBtn.disabled = true;  
73 |         try {  
74 |             await window.api.fetch('/api/ollama/config', {  
75 |                 method: 'POST',  
76 |                 headers: { 'Content-Type': 'application/json' },  
77 |                 body: JSON.stringify({ enabled, url })  
78 |             });  
79 |             showModelsNotification('Настройки Ollama успешно сохра  
    | нены', 'success');  
80 |             if (modelSelect && saveBtn) await loadTabModels(modelS  
    | elect, saveBtn);  
81 |         } catch (err) {  
82 |             console.error('Ошибка сохранения Ollama:', err);  
83 |             showModelsNotification('Ошибка сохранения: ' + err.mes  
    | sage, 'danger');  
84 |         } finally {  
85 |             saveOllamaBtn.disabled = false;
```

```
86 |     }
87 | };
88 | }
89 |
90 | if (saveBtn && modelSelect) {
91 |     saveBtn.onclick = async () => {
92 |         const selectedModel = modelSelect.value;
93 |         saveBtn.disabled = true;
94 |         const originalText = saveBtn.textContent;
95 |         saveBtn.textContent = 'Сохранение...';
96 |
97 |         try {
98 |             await window.api.fetch('/auth/settings', {
99 |                 method: 'POST',
100 |                 headers: { 'Content-Type': 'application/json' },
101 |                 body: JSON.stringify({ model: selectedModel })
102 |             });
103 |             showModelsNotification('Модель успешно обновлена на: '
+ selectedModel, 'success');
104 |
105 |             const otherModelSelect = document.getElementById('admin-model-select');
106 |             if (otherModelSelect) {
107 |                 otherModelSelect.value = selectedModel;
108 |             }
109 |         } catch (err) {
110 |             console.error('Ошибка сохранения модели:', err);
111 |             showModelsNotification('Ошибка сохранения: ' + err.message, 'danger');
112 |         } finally {
113 |             saveBtn.disabled = false;
114 |             saveBtn.textContent = originalText;
115 |         }
116 |     };
117 | }
118 |
119 | if (refreshKeysBtn) {
120 |     refreshKeysBtn.onclick = async () => {
```

```
121     refreshKeysBtn.disabled = true;
122     const originalText = refreshKeysBtn.textContent;
123     refreshKeysBtn.textContent = '⏮ Сброс...';
124     try {
125         const res = await window.api.fetch('/api/keys/reset-all', { method: 'POST' });
126         showModelsNotification(res.message || 'Квоты всех ключей успешно сброшены', 'success');
127     } catch (err) {
128         console.error('Ошибка сброса квот:', err);
129         showModelsNotification('Ошибка сброса: ' + err.message, 'danger');
130     } finally {
131         refreshKeysBtn.disabled = false;
132         refreshKeysBtn.textContent = originalText;
133         if (keysListBody) await refreshKeysList(keysListBody);
134     }
135 };
136 }
137
138 if (addKeyBtn) {
139     addKeyBtn.onclick = async () => {
140         const nameInput = document.getElementById('new-key-name');
141         const valueInput = document.getElementById('new-key-value');
142         if (!nameInput || !valueInput) return;
143
144         const name = nameInput.value.trim();
145         const apiKey = valueInput.value.trim();
146
147         if (!name || !apiKey) {
148             showModelsNotification('Заполните все поля!', 'warning');
149             return;
150         }
151
152         addKeyBtn.disabled = true;
```

```

153     try {
154         await window.api.fetch('/api/keys', {
155             method: 'POST',
156             headers: { 'Content-Type': 'application/json' },
157             body: JSON.stringify({ name, api_key: apiKey, status
: 'active' })
158         });
159         showModelsNotification(`Ключ "${name}" успешно добавле
н`, 'success');
160         nameInput.value = '';
161         valueInput.value = '';
162         if (keysListBody) await refreshKeysList(keysListBody);
163     } catch (err) {
164         console.error('Ошибка добавления ключа:', err);
165         showModelsNotification('Ошибка добавления: ' + err.mes
sage, 'danger');
166     } finally {
167         addKeyBtn.disabled = false;
168     }
169 };
170 }
171
172 // 2. Load all components concurrently
173 await Promise.allSettled([
174     modelSelect && saveBtn ? loadTabModels(modelSelect, saveBt
n) : Promise.resolve(),
175     keysListBody ? refreshKeysList(keysListBody) : Promise.res
olve(),
176     loadFoundryConfig(),
177     loadOllamaConfig(),
178     loadAgyConfig(),
179     loadSystemInstruction()
180 ]);
181 }
182
183 // Helper to load models list
184 async function loadTabModels(modelSelect, saveBtn) {
185     const providerSelect = document.getElementById('provider-tab

```

```
-select');
186   if (providerSelect) providerSelect.innerHTML = '';
187   modelSelect.innerHTML = '';
188
189   let modelsGrouped = {};
190
191   try {
192     const modelsData = await window.api.fetch('/api/chat/model
s');
193     modelsGrouped = modelsData.models || {};
194     if (Array.isArray(modelsGrouped)) {
195       modelsGrouped = { 'gemini': modelsGrouped };
196     }
197   } catch (err) {
198     console.error('Ошибка загрузки моделей:', err);
199     showModelsNotification('Ошибка загрузки моделей AI: ' + er
r.message, 'danger');
200   }
201
202   const providers = Object.keys(modelsGrouped).filter(p => mod
elsGrouped[p] && modelsGrouped[p].length > 0);
203
204   if (providers.length === 0) {
205     if (providerSelect) {
206       providerSelect.innerHTML = '<option value="">Нет доступн
ых провайдеров</option>';
207     }
208     modelSelect.innerHTML = '<option value="">Нет доступных мо
делей</option>';
209     saveBtn.disabled = true;
210     return;
211   }
212
213   if (providerSelect) {
214     providerSelect.innerHTML = '';
215     providers.forEach(p => {
216       const option = document.createElement('option');
217       option.value = p;
```

```

218 |         option.textContent = p.charAt(0).toUpperCase() + p.slice
    | (1);
219 |         providerSelect.appendChild(option);
220 |     });
221 | }
222 |
223 | const populateModels = (provider) => {
224 |     modelSelect.innerHTML = '';
225 |     const providerModels = modelsGrouped[provider] || [];
226 |     if (providerModels.length === 0) {
227 |         modelSelect.innerHTML = '<option value="">Нет моделей</o
    | ption>';
228 |         saveBtn.disabled = true;
229 |     } else {
230 |         providerModels.forEach(modelName => {
231 |             const option = document.createElement('option');
232 |             option.value = modelName;
233 |             let cleanName = modelName;
234 |             if (cleanName.startsWith('foundry:')) cleanName = clea
    | nName.substring(8);
235 |             else if (cleanName.startsWith('ollama:')) cleanName =
    | cleanName.substring(7);
236 |             else if (cleanName.startsWith('agy-')) cleanName = cle
    | anName.substring(4);
237 |             option.textContent = cleanName;
238 |             modelSelect.appendChild(option);
239 |         });
240 |         saveBtn.disabled = false;
241 |     }
242 | };
243 |
244 | if (providerSelect) {
245 |     providerSelect.onChange = () => populateModels(providerSel
    | ect.value);
246 |     populateModels(providerSelect.value);
247 | } else {
248 |     let allModels = [];
249 |     providers.forEach(p => allModels = allModels.concat(models

```



```

    Grouped[p])));
250     allModels.forEach(modelName => {
251         const option = document.createElement('option');
252         option.value = modelName;
253         option.textContent = modelName;
254         modelSelect.appendChild(option);
255     });
256     saveBtn.disabled = allModels.length === 0;
257 }
258
259 try {
260     const settingsData = await window.api.fetch('/auth/settings');
261     if (settingsData && settingsData.model) {
262         let foundProvider = null;
263         for (const p of providers) {
264             if (modelsGrouped[p].includes(settingsData.model)) {
265                 foundProvider = p;
266                 break;
267             }
268         }
269         if (foundProvider && providerSelect) {
270             providerSelect.value = foundProvider;
271             populateModels(foundProvider);
272         }
273         modelSelect.value = settingsData.model;
274     }
275 } catch (err) {
276     console.error('Ошибка загрузки настроек AI пользователя:',
277         err);
278 }
279
280 // Helper to refresh keys list table
281 async function refreshKeysList(container) {
282     try {
283         container.innerHTML = '<tr><td colspan="5" class="text-center py-4 text-muted">Загрузка ключей...</td></tr>';

```

```
284 |     const keysData = await window.api.fetch('/api/keys');
285 |     const keys = keysData.keys || [];
286 |
287 |     if (keys.length === 0) {
288 |         container.innerHTML = '<tr><td colspan="5" class="text-c
enter py-4 text-muted">Список ключей пуст</td></tr>';
289 |         return;
290 |     }
291 |
292 |     container.innerHTML = '';
293 |     keys.forEach(key => {
294 |         const row = document.createElement('tr');
295 |
296 |         const tdName = document.createElement('td');
297 |         tdName.innerHTML = `<strong>${key.name}</strong>`;
298 |         row.appendChild(tdName);
299 |
300 |         const tdKey = document.createElement('td');
301 |         tdKey.className = 'font-monospace text-muted small';
302 |         tdKey.textContent = key.api_key_masked;
303 |         row.appendChild(tdKey);
304 |
305 |         const tdStatus = document.createElement('td');
306 |         const isEnabled = key.status === 'active';
307 |         const statusClass = isEnabled ? 'bg-success' : 'bg-secon
dary';
308 |         const statusText = isEnabled ? 'Активен' : 'Отключен';
309 |         tdStatus.innerHTML = `<span class="badge ${statusClass}"
>${statusText}</span>`;
310 |         row.appendChild(tdStatus);
311 |
312 |         const tdQuota = document.createElement('td');
313 |         if (key.exhausted) {
314 |             let resetText = 'ЛИМИТ';
315 |             if (key.reset_in_seconds) {
316 |                 const hours = Math.floor(key.reset_in_seconds / 3600
);
317 |                 const mins = Math.floor((key.reset_in_seconds % 3600
```

```

    ) / 60);
318         resetText = `Сброс через ${hours}ч ${mins}м`;
319     }
320     tdQuota.innerHTML = `<span class="badge bg-danger d-block mb-1" title="Превышен лимит запросов в сутки">${resetText}</span>`;
321     } else {
322         tdQuota.innerHTML = `<span class="badge bg-success d-block mb-1">OK</span>`;
323     }
324     row.appendChild(tdQuota);
325
326     const tdActions = document.createElement('td');
327     tdActions.className = 'text-end';
328
329     const btnToggle = document.createElement('button');
330     btnToggle.className = `btn btn-xs btn-sm me-1 ${isEnabled
331     d ? 'btn-outline-secondary' : 'btn-outline-success'}`;
332     btnToggle.textContent = isEnabled ? 'Откл' : 'Вкл';
333     btnToggle.onclick = () => toggleKeyStatus(key.name, isEnabled ? 'disabled' : 'active', container);
334     tdActions.appendChild(btnToggle);
335
336     if (key.exhausted) {
337         const btnReset = document.createElement('button');
338         btnReset.className = 'btn btn-xs btn-outline-warning btn-sm me-1';
339         btnReset.innerHTML = 'Сброс';
340         btnReset.title = 'Сбросить 24-часовой бан квоты';
341         btnReset.onclick = () => resetKeyQuota(key.name, container);
342         tdActions.appendChild(btnReset);
343     }
344
345     const btnDelete = document.createElement('button');
346     btnDelete.className = 'btn btn-xs btn-outline-danger btn-sm';
347     btnDelete.textContent = 'Удалить';

```

```

347 |         btnDelete.onclick = () => deleteKey(key.name, container)
    | ;
348 |         tdActions.appendChild(btnDelete);
349 |
350 |         row.appendChild(tdActions);
351 |         container.appendChild(row);
352 |     });
353 |
354 | } catch (err) {
355 |     console.error('Ошибка загрузки ключей:', err);
356 |     container.innerHTML = `<tr><td colspan="5" class="text-center py-4 text-danger">Ошибка: ${err.message}</td></tr>`;
357 | }
358 | }
359 |
360 | // API action helpers
361 | async function toggleKeyStatus(name, newStatus, container) {
362 |     try {
363 |         await window.api.fetch(`/api/keys/${name}`, {
364 |             method: 'PATCH',
365 |             headers: { 'Content-Type': 'application/json' },
366 |             body: JSON.stringify({ status: newStatus })
367 |         });
368 |         showModelsNotification(`Статус ключа "${name}" изменен на
    | ${newStatus === 'active' ? 'активный' : 'отключенный'}`, 'success');
369 |         await refreshKeysList(container);
370 |     } catch (err) {
371 |         console.error('Ошибка переключения статуса ключа:', err);
372 |         showModelsNotification('Ошибка изменения статуса: ' + err.
    | message, 'danger');
373 |     }
374 | }
375 |
376 | async function resetKeyQuota(name, container) {
377 |     try {
378 |         await window.api.fetch(`/api/keys/${name}/reset-quota`, {
    | method: 'POST' });

```

```
379     showModelsNotification(`Квота для ключа "${name}" успешно
| сброшена`, 'success');
380     await refreshKeysList(container);
381   } catch (err) {
382     console.error('Ошибка сброса квоты:', err);
383     showModelsNotification('Ошибка сброса квоты: ' + err.messa
| ge, 'danger');
384   }
385 }
386
387 async function deleteKey(name, container) {
388   if (!confirm(`Вы уверены, что хотите удалить ключ "${name}"?
| `)) return;
389   try {
390     await window.api.fetch(`/api/keys/${name}`, { method: 'DEL
| ETE' });
391     showModelsNotification(`Ключ "${name}" успешно удален`, 's
| uccess');
392     await refreshKeysList(container);
393   } catch (err) {
394     console.error('Ошибка удаления ключа:', err);
395     showModelsNotification('Ошибка удаления: ' + err.message,
| 'danger');
396   }
397 }
398
399 function showModelsNotification(message, type = 'info') {
400   const notification = document.createElement('div');
401   notification.className = `alert alert-${type} position-fixed
| top-0 end-0 m-3`;
402   notification.style.zIndex = '9999';
403   notification.style.maxWidth = '400px';
404   notification.textContent = message;
405   document.body.appendChild(notification);
406
407   setTimeout(() => {
408     notification.remove();
409   }, 5000);
```

```
410 }
411
412 async function loadFoundryConfig() {
413   try {
414     const config = await window.api.fetch('/api/foundry/config
415   ');
416     const enabledInput = document.getElementById('foundry-enab
417   led');
418     const urlInput = document.getElementById('foundry-url');
419     const keyInput = document.getElementById('foundry-key');
420     const modelInput = document.getElementById('foundry-model'
421   );
422     if (enabledInput) enabledInput.checked = config.enabled ||
423   false;
424     if (urlInput) urlInput.value = config.url || '';
425     if (keyInput) keyInput.value = config.key || '';
426     if (modelInput) modelInput.value = config.model || '';
427   } catch (err) {
428     console.error('Ошибка загрузки настроек Foundry:', err);
429   }
430 }
431
432 async function loadOllamaConfig() {
433   try {
434     const config = await window.api.fetch('/api/ollama/config'
435   );
436     const enabledInput = document.getElementById('ollama-enabl
437   ed');
438     const urlInput = document.getElementById('ollama-url');
439     const modelInput = document.getElementById('ollama-model')
440   ;
441     if (enabledInput) enabledInput.checked = config.enabled ||
442   false;
443     if (urlInput) urlInput.value = config.url || '';
444     if (modelInput) modelInput.value = config.model || '';
445   } catch (err) {
```

```
440     console.error('Ошибка загрузки настроек Ollama:', err);
441   }
442 }
443
444 async function loadAgyConfig() {
445   try {
446     const modelSelect = document.getElementById('agy-model');
447     if (modelSelect) {
448       try {
449         const modelsData = await window.api.fetch('/api/chat/m
models');
450         const agyList = modelsData.models?.agy || [];
451         if (agyList.length > 0) {
452           const curVal = modelSelect.value;
453           modelSelect.innerHTML = '';
454           agyList.forEach(m => {
455             const opt = document.createElement('option');
456             opt.value = m;
457             opt.textContent = m;
458             modelSelect.appendChild(opt);
459           });
460           if (curVal && agyList.includes(curVal)) {
461             modelSelect.value = curVal;
462           }
463         }
464       } catch (e) {
465         console.error('Ошибка загрузки моделей AGY:', e);
466       }
467     }
468
469     const config = await window.api.fetch('/api/agy/config');
470     const enabledInput = document.getElementById('agy-enabled'
);
471     const keyInput = document.getElementById('agy-key');
472
473     if (enabledInput) enabledInput.checked = config.enabled ??
true;
474     if (modelSelect && config.model) modelSelect.value = confi
```

```
g.model;
475     if (keyInput) keyInput.value = config.key || '';
476   } catch (err) {
477     console.error('Ошибка загрузки настроек Antigravity (AGY):', err);
478   }
479 }
480
481 async function loadSystemInstruction() {
482   const editor = document.getElementById('system-instruction-editor');
483   const statusBadge = document.getElementById('system-instruction-status');
484   if (!editor) return;
485
486   editor.disabled = true;
487   if (statusBadge) {
488     statusBadge.className = 'badge bg-warning text-dark';
489     statusBadge.textContent = 'Загрузка...';
490     statusBadge.style.removeProperty('display');
491   }
492
493   try {
494     const data = await window.api.fetch('/api/admin/system_instruction');
495     editor.value = data.content || '';
496     if (statusBadge) {
497       statusBadge.className = 'badge bg-success';
498       statusBadge.textContent = 'Загружено';
499       setTimeout(() => {
500         if (statusBadge) statusBadge.style.display = 'none';
501       }, 2500);
502     }
503   } catch (err) {
504     console.error('Ошибка загрузки системной инструкции:', err);
505
506     if (statusBadge) {
507       statusBadge.className = 'badge bg-danger';
```



```
507     statusBadge.textContent = 'Ошибка';
508     statusBadge.style.removeProperty('display');
509   }
510   showModelsNotification('Ошибка загрузки системной инструкц
ии: ' + err.message, 'danger');
511   } finally {
512     editor.disabled = false;
513   }
514 }
515
516 async function saveSystemInstruction() {
517   const editor = document.getElementById('system-instruction-e
ditor');
518   const saveBtn = document.getElementById('btn-save-instructio
n');
519   const statusBadge = document.getElementById('system-instruct
ion-status');
520   if (!editor || !saveBtn) return;
521
522   const originalHtml = saveBtn.innerHTML;
523   saveBtn.disabled = true;
524   saveBtn.innerHTML = '<span class="spinner-border spinner-bor
der-sm" role="status"></span> Сохранение...';
525
526   try {
527     await window.api.fetch('/api/admin/system_instruction', {
528       method: 'POST',
529       headers: { 'Content-Type': 'application/json' },
530       body: JSON.stringify({ content: editor.value })
531     });
532     showModelsNotification('☑ Системная инструкция успешно сох
ранена', 'success');
533     if (statusBadge) {
534       statusBadge.className = 'badge bg-success';
535       statusBadge.textContent = 'Сохранено';
536       statusBadge.style.removeProperty('display');
537       setTimeout(() => {
538         if (statusBadge) statusBadge.style.display = 'none';
```

```
539         }, 3000);
540     }
541 } catch (err) {
542     console.error('Ошибка сохранения системной инструкции:', e
rr);
543     if (statusBadge) {
544         statusBadge.className = 'badge bg-danger';
545         statusBadge.textContent = 'Ошибка сохранения';
546         statusBadge.style.removeProperty('display');
547     }
548     showModelsNotification('Ошибка сохранения: ' + err.message
, 'danger');
549 } finally {
550     saveBtn.disabled = false;
551     saveBtn.innerHTML = originalHtml;
552 }
553 }
554
555 // Экспорт для загрузчика вкладок
556 window.initModelsTab = initModelsTab;
557
```

index.html

```
1 <!-- Models & APIs Management Tab -->
2 <div class="container-fluid py-3">
3   <!-- Premium Header -->
4   <div class="d-flex justify-content-between align-items-center mb-4">
5     <div>
6       <h4 class="mb-0 fw-bold text-primary"><i class="bi bi-cpu"></i> Модели и API</h4>
7       <p class="text-muted small mb-0">Управление провайдерами, ключами и системными инструкциями</p>
8     </div>
9   </div>
10
11 <!-- Top Block: Active Model, Test Prompt & System Instruction -->
12 <div class="row mb-4">
13   <!-- Active Model & Testing Card -->
14   <div class="col-lg-6 mb-4 mb-lg-0">
15     <div class="card shadow-sm border-0 h-100" style="background: linear-gradient(145deg, #1e1e1e, #2a2a2a); color: #fff;">
16       <div class="card-body d-flex flex-column p-4">
17         <div class="d-flex justify-content-between align-items-center mb-3">
18           <h6 class="card-title fw-bold text-info mb-0"><i class="bi bi-robot"></i> Активная модель и проверка</h6>
19           <button class="btn btn-outline-secondary btn-sm rounded-pill py-0 px-2 text-white-50" id="btn-refresh-models-list" title="Обновить список доступных моделей">
20             <i class="bi bi-arrow-clockwise"></i> Обновить список
21           </button>
22         </div>
23       </div>
24     <div class="row g-2 mb-3">
25       <div class="col-sm-5">
```

```

26         <label for="provider-tab-select" class="form-label
    el small text-secondary mb-1">Провайдер ИИ:</label>
27         <select class="form-select form-select-sm bg-dark
    k text-white border-secondary" id="provider-tab-select">
28             <option value="">Загрузка...</option>
29         </select>
30     </div>
31     <div class="col-sm-7">
32         <label for="models-tab-select" class="form-label
    small text-secondary mb-1">Модель:</label>
33         <select class="form-select form-select-sm bg-dark
    k text-white border-secondary" id="models-tab-select">
34             <option value="">Загрузка...</option>
35         </select>
36     </div>
37 </div>
38
39     <!-- Model Test Area: Prompt -> Response -->
40     <div class="p-3 rounded-3 mb-3" style="background: r
    gba(0, 0, 0, 0.3); border: 1px solid rgba(255, 255, 255, 0.1);
    ">
41         <div class="d-flex justify-content-between align-i
    tems-center mb-2">
42             <label for="model-test-prompt" class="form-label
    small text-info fw-bold mb-0">
43                 <i class="bi bi-lightning-charging text-war
    ning me-1"></i> Проверочный запрос (Запрос &arr; Ответ)
44             </label>
45             <span class="text-secondary small" style="font-s
    ize: 0.72rem;">Тест выбранной модели</span>
46         </div>
47
48         <div class="input-group input-group-sm mb-2">
49             <input type="text" class="form-control bg-dark t
    ext-white border-secondary" id="model-test-prompt" value="Прив
    ет! Назови свою модель и провайдера, и подтверди готовность к
    работе." placeholder="Введите проверочное сообщение...">
50             <button class="btn btn-info fw-bold px-3" id="bt

```

```

n-model-test-send" type="button" title="Отправить тестовый за
прос в выбранную модель">
51         <i class="bi bi-send-fill me-1"></i> Проверить
52     </button>
53 </div>
54
55     <div id="model-test-result" class="p-2 rounded sma
ll font-monospace" style="display: none; background: rgba(10,
10, 15, 0.7); border: 1px solid rgba(255, 255, 255, 0.15); max
-height: 140px; overflow-y: auto;">
56         <!-- Dynamic output goes here -->
57     </div>
58 </div>
59
60     <div class="mt-auto d-grid">
61         <button class="btn btn-primary btn-sm rounded-pill
fw-bold" id="btn-models-tab-save">
62             <i class="bi bi-check-circle"></i> Сохранить как
модель по умолчанию
63         </button>
64         <div class="form-text text-secondary text-center m
t-1" style="font-size: 0.72rem;">Эта модель будет использовать
ся ассистентом по умолчанию во всех чатах.</div>
65     </div>
66 </div>
67 </div>
68 </div>
69
70 <!-- System Instruction Card -->
71 <div class="col-lg-6">
72     <div class="card shadow-sm border-0 h-100">
73         <div class="card-header bg-white border-0 d-flex justi
fy-content-between align-items-center pt-3 pb-0">
74             <span class="fw-bold"><i class="bi bi-terminal"></i>
Системная инструкция</span>
75             <div class="d-flex gap-2 align-items-center">
76                 <span id="system-instruction-status" class="badge
bg-secondary" style="display:none!important;"></span>

```

```

77 |         <button class="btn btn-light btn-sm rounded-pill"
    | id="btn-reload-instruction" title="Обновить">
78 |             <i class="bi bi-arrow-clockwise"></i>
79 |         </button>
80 |         <button class="btn btn-success btn-sm rounded-pill
    | px-3" id="btn-save-instruction">
81 |             Сохранить
82 |         </button>
83 |     </div>
84 | </div>
85 | <div class="card-body d-flex flex-column">
86 |     <textarea
87 |         id="system-instruction-editor"
88 |         class="form-control font-monospace border-0 bg-lig
    | ht flex-grow-1"
89 |         style="min-height: 220px; resize: none; font-size:
    | 0.85rem;"
90 |         placeholder="Введите системную инструкцию..."
91 |         spellcheck="false"
92 |     ></textarea>
93 | </div>
94 | </div>
95 | </div>
96 | </div>
97 |
98 | <!-- Bottom Block: Provider Settings via Vertical Pills -->
99 | <div class="card shadow-sm border-0">
100 |     <div class="card-body p-0">
101 |         <div class="d-flex flex-column flex-md-row align-items-s
    | tretch">
102 |
103 |             <!-- Sidebar Pills -->
104 |             <div class="nav flex-column nav-pills p-3 bg-light bor
    | der-end" id="provider-pills-tab" role="tablist" aria-orientati
    | on="vertical" style="min-width: 250px; min-height: 400px; bord
    | er-top-left-radius: 0.375rem; border-bottom-left-radius: 0.375
    | rem;">
105 |                 <h6 class="text-uppercase text-muted fw-bold small m

```

```

b-3 ps-2">Настройки провайдеров</h6>
106 |         <button class="nav-link active text-start rounded-3
|         mb-2 fw-bold" id="pill-gemini-tab" data-bs-toggle="pill" data-
|         bs-target="#pill-gemini" type="button" role="tab" aria-control
|         s="pill-gemini" aria-selected="true">
107 |             □ Google Gemini
108 |         </button>
109 |         <button class="nav-link text-start rounded-3 mb-2 fw
|         -bold" id="pill-agy-tab" data-bs-toggle="pill" data-bs-target=
|         "#pill-agy" type="button" role="tab" aria-controls="pill-agy"
|         aria-selected="false">
110 |             □ Google Antigravity (AGY)
111 |         </button>
112 |         <button class="nav-link text-start rounded-3 mb-2 fw
|         -bold" id="pill-foundry-tab" data-bs-toggle="pill" data-bs-tar
|         get="#pill-foundry" type="button" role="tab" aria-controls="pi
|         ll-foundry" aria-selected="false">
113 |             □ Microsoft Foundry
114 |         </button>
115 |         <button class="nav-link text-start rounded-3 fw-bold
|         " id="pill-ollama-tab" data-bs-toggle="pill" data-bs-target="#
|         pill-ollama" type="button" role="tab" aria-controls="pill-olla
|         ma" aria-selected="false">
116 |             □ Ollama (Local)
117 |         </button>
118 |     </div>
119 |
120 |     <!-- Tab Content -->
121 |     <div class="tab-content flex-grow-1 p-4" id="provider-
|     pills-tabContent">
122 |
123 |         <!-- GEMINI TAB -->
124 |         <div class="tab-pane fade show active" id="pill-gemi
|         ni" role="tabpanel" aria-labelledby="pill-gemini-tab">
125 |             <div class="d-flex justify-content-between align-i
|         tems-center mb-3">
126 |                 <h5 class="fw-bold mb-0">Ключи Google Gemini</h5
|
>

```

```

127 |         <button class="btn btn-outline-warning btn-sm rounded-pill px-3" id="btn-refresh-keys">
128 |             <i class="bi bi-arrow-repeat"></i> Сбросить все ключи
129 |         </button>
130 |     </div>
131 |
132 |     <!-- Gemini Key Pool Alert -->
133 |     <div class="alert alert-info shadow-sm rounded-3 border-0 mb-4" role="alert">
134 |         <h6 class="alert-heading fw-bold mb-1"><i class="bi bi-key-fill me-2"></i>Пул ротации API-ключей (Round-Robin)
135 |         </h6>
136 |         <p class="small mb-0">
137 |             Все активные ключи из таблицы ниже формируют единый пул. При исчерпании суточных лимитов запросов (ошибки <code>429 / ResourceExhausted</code>) система автоматически переключается на следующий свободный ключ без прерывания генерации
138 |             .
139 |         </p>
140 |     </div>
141 |
142 |     <div class="row g-4">
143 |         <!-- Add Key Form -->
144 |         <div class="col-lg-4">
145 |             <div class="card bg-light border-0 shadow-sm">
146 |                 <div class="card-body">
147 |                     <h6 class="fw-bold mb-3">Добавить новый ключ</h6>
148 |                     <form id="form-add-key" onsubmit="event.preventDefault();">
149 |                         <div class="mb-3">
150 |                             <label class="form-label small text-muted fw-bold">Имя владельца:</label>
151 |                             <input type="text" class="form-control form-control-sm" id="new-key-name" placeholder="Например: primary_key" required>
152 |                         </div>

```



```

151 |         <div class="mb-4">
152 |             <label class="form-label small text-mu
| ted fw-bold">API-ключ (AIzaSy...):</label>
153 |             <input type="password" class="form-con
| trol form-control-sm" id="new-key-value" placeholder="Введите
| ключ" required>
154 |         </div>
155 |         <button type="button" class="btn btn-suc
| cess btn-sm w-100 rounded-pill fw-bold" id="btn-add-key">Добав
| ить ключ</button>
156 |     </form>
157 | </div>
158 | </div>
159 | </div>
160 |
161 |     <!-- Keys Table -->
162 |     <div class="col-lg-8">
163 |         <div class="table-responsive shadow-sm rounded
| border bg-white">
164 |             <table class="table table-hover align-middle
| mb-0" id="keys-table">
165 |                 <thead class="table-light">
166 |                     <tr>
167 |                         <th class="py-3 px-3 border-0">Названи
| e</th>
168 |                         <th class="py-3 px-3 border-0">Ключ</t
| h>
169 |                         <th class="py-3 px-3 border-0">Статус<
| /th>
170 |                         <th class="py-3 px-3 border-0">Квота</
| th>
171 |                         <th class="py-3 px-3 border-0 text-end
| ">Действия</th>
172 |                     </tr>
173 |                 </thead>
174 |                 <tbody id="keys-list-body" class="border-t
| op-0">
175 |                     <tr>

```

```

176 |             <td colspan="5" class="text-center py-
177 | 4 text-muted">Загрузка ключей...</td>
178 |             </tr>
179 |         </tbody>
180 |     </table>
181 | </div>
182 | </div>
183 | </div>
184 |
185 | <!-- ANTIGRAVITY (AGY) TAB -->
186 | <div class="tab-pane fade" id="pill-agy" role="tabpa
187 | nel" aria-labelledby="pill-agy-tab">
188 |     <h5 class="fw-bold mb-3">Настройки Google Antigrav
189 | ity (AGY)</h5>
190 |
191 |     <div class="row">
192 |         <div class="col-md-11 col-lg-9">
193 |             <!-- Info Alert -->
194 |             <div class="alert alert-warning shadow-sm roun
195 | ded-3 border-0 mb-4" role="alert">
196 |                 <h6 class="alert-heading fw-bold"><i class="
197 | bi bi-rocket-takeoff me-2"></i>Что такое Antigravity (AGY)?</h
198 | 6>
199 |                 <p class="small mb-2">
200 |                     <strong>Google Antigravity</strong> – пере
201 | довая агентная платформа для автономных AI-воркфлоу, мультиаге
202 | нтных пайплайнов и глубокого веб-исследования.
203 |                 </p>
204 |                 <ul class="small mb-0 ps-3">
205 |                     <li>Включает встроенные агентные инструмен
206 | ты <code>BuiltinTools.SEARCH_WEB</code> и <code>READ_URL_CONTE
207 | NT</code>.</li>
208 |                     <li>Поддерживает автономный вызов через Py
209 | thon SDK (<code>google.antigravity</code>) и утилиту <code>agy
210 | CLI</code>.</li>
211 |                     <li>Если отдельный ключ <code>AGY_API_KEY<
212 | /code> не задан, провайдер автоматически использует ключи из п

```

```

ула Gemini.</li>
201         </ul>
202     </div>
203
204     <form id="form-agy-config" onsubmit="event.pre
ventDefault();">
205         <div class="card border-0 bg-light shadow-sm
mb-4">
206             <div class="card-body p-4">
207                 <div class="form-check form-switch mb-4
pb-2 border-bottom">
208                     <input class="form-check-input" type="
checkbox" role="switch" id="agy-enabled" style="transform: sca
le(1.3); margin-left: -2.5em; cursor: pointer;">
209                     <label class="form-check-label fw-bold
ms-2" for="agy-enabled" style="cursor: pointer;">Активировать
провайдер Antigravity (AGY)</label>
210                 </div>
211
212                 <div class="mb-3">
213                     <label class="form-label small text-mu
ted fw-bold">Модель Antigravity по умолчанию (динамически из S
DK):</label>
214                     <select class="form-select form-select
-sm" id="agy-model">
215                         <option value="">Загрузка доступных
моделей через SDK...</option>
216                     </select>
217                 </div>
218
219                 <div class="mb-4">
220                     <label class="form-label small text-mu
ted fw-bold">AGY API Key (опционально, иначе из пула Gemini):<
/label>
221                     <input type="password" class="form-con
trol form-control-sm" id="agy-key" placeholder="Оставьте пусты
м для использования ключей Gemini">
222                     <div class="form-text small text-muted

```

```
>Сохраняется в защищенном файле <code>.env</code>.</div>
```

```
</div>
```

```
        <button type="button" class="btn btn-primary rounded-pill px-4 fw-bold" id="btn-save-agy">
```

```
            <i class="bi bi-floppy me-1"></i> Сохр
```

```
анить настройки Antigravity
```

```
        </button>
```

```
    </div>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<!-- FOUNDRY TAB -->
```

```
<div class="tab-pane fade" id="pill-foundry" role="tabpanel" aria-labelledby="pill-foundry-tab">
```

```
    <h5 class="fw-bold mb-4">Настройки Microsoft Found
```

```
ry</h5>
```

```
    <div class="row">
```

```
        <div class="col-md-10 col-lg-8">
```

```
            <div class="alert alert-info shadow-sm rounded-
```

```
-3 border-0 mb-4" role="alert">
```

```
                <h6 class="alert-heading fw-bold"><i class="
```

```
bi bi-info-circle me-2"></i>Где взять модели Foundry?</h6>
```

```
                <p class="small mb-0">
```

```
                    Microsoft Foundry (как и LM Studio) работа
```

```
ет локально. Чтобы список моделей был не пустым, вам необходим
```

```
о предварительно загрузить нужные GGUF или ONNX модели через инт
```

```
ерфейс самого Foundry или CLI.
```

```
                </p>
```

```
            </div>
```

```
            <form id="form-foundry-config" onSubmit="event
```

```
.preventDefault();">
```

```
                <div class="card border-0 bg-light shadow-sm
```

```
mb-4">
```

```
                    <div class="card-body p-4">
```

```
249 |         <div class="form-check form-switch mb-4
| pb-2 border-bottom">
250 |             <input class="form-check-input" type="
| checkbox" role="switch" id="foundry-enabled" style="transform:
| scale(1.3); margin-left: -2.5em; cursor: pointer;">
251 |             <label class="form-check-label fw-bold
| ms-2" for="foundry-enabled" style="cursor: pointer;">Активиро
| вать провайдер Foundry</label>
252 |         </div>
253 |         <div class="mb-3">
254 |             <label class="form-label small text-mu
| ted fw-bold">Base URL API:</label>
255 |             <input type="text" class="form-control
| " id="foundry-url" placeholder="http://localhost:54837">
256 |         </div>
257 |         <div class="mb-3">
258 |             <label class="form-label small text-mu
| ted fw-bold">API Key (опционально):</label>
259 |             <input type="password" class="form-con
| trol" id="foundry-key" placeholder="Введите ключ">
260 |         </div>
261 |         <button type="button" class="btn btn-pri
| mary rounded-pill px-4 fw-bold" id="btn-save-foundry">Сохранит
| ь настройки Foundry</button>
262 |         </div>
263 |     </div>
264 | </form>
265 | </div>
266 | </div>
267 | </div>
268 |
269 | <!-- OLLAMA TAB -->
270 | <div class="tab-pane fade" id="pill-ollama" role="ta
| bpanel" aria-labelledby="pill-ollama-tab">
271 |     <h5 class="fw-bold mb-4">Настройки Ollama (Local)<
| /h5>
272 |     <div class="row">
273 |         <div class="col-md-10 col-lg-8">
```

```
274 |         <div class="alert alert-info shadow-sm rounded
    | -3 border-0 mb-4" role="alert">
275 |             <h6 class="alert-heading fw-bold"><i class="
    | bi bi-info-circle me-2"></i>Как добавить новые модели в Ollama
    | ?</h6>
276 |             <p class="small mb-0">
277 |                 Откройте терминал на сервере, где установл
    | ен Ollama, и выполните команду:<br>
278 |                 <code class="bg-white px-2 py-1 rounded te
    | xt-dark mt-2 d-inline-block border">ollama pull llama3.1</code
    | ><br>
279 |                 После загрузки модель автоматически появит
    | ся в выпадающем списке.
280 |             </p>
281 |         </div>
282 |         <form id="form-ollama-config" onsubmit="event.
    | preventDefault();">
283 |             <div class="card border-0 bg-light shadow-sm
    | mb-4">
284 |                 <div class="card-body p-4">
285 |                     <div class="form-check form-switch mb-4
    | pb-2 border-bottom">
286 |                         <input class="form-check-input" type="
    | checkbox" role="switch" id="ollama-enabled" style="transform:
    | scale(1.3); margin-left: -2.5em; cursor: pointer;">
287 |                         <label class="form-check-label fw-bold
    | ms-2" for="ollama-enabled" style="cursor: pointer;">Активиров
    | ать провайдер Ollama</label>
288 |                     </div>
289 |                     <div class="mb-3">
290 |                         <label class="form-label small text-mu
    | ted fw-bold">Base URL API:</label>
291 |                         <input type="text" class="form-control
    | " id="ollama-url" placeholder="http://localhost:11434">
292 |                     </div>
293 |                     <button type="button" class="btn btn-pri
    | mary rounded-pill px-4 fw-bold" id="btn-save-ollama">Сохранить
    | настройки Ollama</button>
```

```
294 |                 </div>
295 |                 </div>
296 |             </form>
297 |         </div>
298 |     </div>
299 | </div>
300 |
301 |     </div>
302 | </div>
303 | </div>
304 | </div>
305 | </div>
306 |
```

main.js

```
1 // Gemini Models & APIs management tab logic
2
3 async function initModelsTab() {
4   const modelSelect = document.getElementById('models-tab-select');
5   const saveBtn = document.getElementById('btn-models-tab-save');
6   const keysListBody = document.getElementById('keys-list-body');
7   const refreshKeysBtn = document.getElementById('btn-refresh-keys');
8   const addKeyBtn = document.getElementById('btn-add-key');
9   const saveAgyBtn = document.getElementById('btn-save-agy');
10  const saveFoundryBtn = document.getElementById('btn-save-foundry');
11  const saveOllamaBtn = document.getElementById('btn-save-ollama');
12  const saveBtnInstr = document.getElementById('btn-save-instruction');
13  const reloadBtnInstr = document.getElementById('btn-reload-instruction');
14
15  const refreshModelsBtn = document.getElementById('btn-refresh-models-list');
16  const testModelBtn = document.getElementById('btn-model-test-send');
17  const testModelPrompt = document.getElementById('model-test-prompt');
18
19  // 1. Bind event handlers immediately
20  if (saveBtnInstr) saveBtnInstr.onclick = saveSystemInstruction;
21  if (reloadBtnInstr) reloadBtnInstr.onclick = loadSystemInstruction;
22
23  if (refreshModelsBtn && modelSelect && saveBtn) {
```



```

24 |     refreshModelsBtn.onclick = async () => {
25 |         refreshModelsBtn.disabled = true;
26 |         const originalText = refreshModelsBtn.innerHTML;
27 |         refreshModelsBtn.innerHTML = '<span class="spinner-border spinner-border-sm me-1" role="status"></span> Обновление...'
    | ;
28 |         try {
29 |             await loadTabModels(modelSelect, saveBtn, true);
30 |         } finally {
31 |             refreshModelsBtn.disabled = false;
32 |             refreshModelsBtn.innerHTML = originalText;
33 |         }
34 |     };
35 | }
36 |
37 | if (testModelBtn) {
38 |     testModelBtn.onclick = executeModelTest;
39 | }
40 |
41 | if (testModelPrompt) {
42 |     testModelPrompt.onkeydown = (e) => {
43 |         if (e.key === 'Enter') {
44 |             e.preventDefault();
45 |             executeModelTest();
46 |         }
47 |     };
48 | }
49 |
50 | if (saveAgyBtn) {
51 |     saveAgyBtn.onclick = async () => {
52 |         const enabled = document.getElementById('agy-enabled')?.
    | checked ?? true;
53 |         const model = document.getElementById('agy-model')?.value || 'agy-flash';
54 |         const key = document.getElementById('agy-key')?.value.trim() || '';
55 |
56 |         saveAgyBtn.disabled = true;

```

```

57 |     try {
58 |         await window.api.fetch('/api/agy/config', {
59 |             method: 'POST',
60 |             headers: { 'Content-Type': 'application/json' },
61 |             body: JSON.stringify({ enabled, model, key })
62 |         });
63 |         showModelsNotification('Настройки AntigraVity (AGY) уc
пешно сохранены', 'success');
64 |         if (modelSelect && saveBtn) await loadTabModels(modelS
elect, saveBtn);
65 |     } catch (err) {
66 |         console.error('Ошибка сохранения AntigraVity:', err);
67 |         showModelsNotification('Ошибка сохранения: ' + err.mes
sage, 'danger');
68 |     } finally {
69 |         saveAgyBtn.disabled = false;
70 |     }
71 | };
72 | }
73 |
74 | if (saveFoundryBtn) {
75 |     saveFoundryBtn.onclick = async () => {
76 |         const enabled = document.getElementById('foundry-enabled
')?.checked || false;
77 |         const url = document.getElementById('foundry-url')?.valu
e.trim() || '';
78 |         const key = document.getElementById('foundry-key')?.valu
e.trim() || '';
79 |
80 |         saveFoundryBtn.disabled = true;
81 |         try {
82 |             await window.api.fetch('/api/foundry/config', {
83 |                 method: 'POST',
84 |                 headers: { 'Content-Type': 'application/json' },
85 |                 body: JSON.stringify({ enabled, url, key })
86 |             });
87 |             showModelsNotification('Настройки Microsoft Foundry yc
пешно сохранены', 'success');

```

```
88 |         if (modelSelect && saveBtn) await loadTabModels(modelS
|elect, saveBtn);
89 |     } catch (err) {
90 |         console.error('Ошибка сохранения Foundry:', err);
91 |         showModelsNotification('Ошибка сохранения: ' + err.mes
|sage, 'danger');
92 |     } finally {
93 |         saveFoundryBtn.disabled = false;
94 |     }
95 | };
96 | }
97 |
98 | if (saveOllamaBtn) {
99 |     saveOllamaBtn.onclick = async () => {
100 |         const enabled = document.getElementById('ollama-enabled'
|)?.checked || false;
101 |         const url = document.getElementById('ollama-url')?.value
|.trim() || '';
102 |
103 |         saveOllamaBtn.disabled = true;
104 |         try {
105 |             await window.api.fetch('/api/ollama/config', {
106 |                 method: 'POST',
107 |                 headers: { 'Content-Type': 'application/json' },
108 |                 body: JSON.stringify({ enabled, url })
109 |             });
110 |             showModelsNotification('Настройки Ollama успешно сохр
|анены', 'success');
111 |             if (modelSelect && saveBtn) await loadTabModels(modelS
|elect, saveBtn);
112 |         } catch (err) {
113 |             console.error('Ошибка сохранения Ollama:', err);
114 |             showModelsNotification('Ошибка сохранения: ' + err.mes
|sage, 'danger');
115 |         } finally {
116 |             saveOllamaBtn.disabled = false;
117 |         }
118 |     };

```

```
119 | }
120 |
121 | if (saveBtn && modelSelect) {
122 |     saveBtn.onclick = async () => {
123 |         const selectedModel = modelSelect.value;
124 |         saveBtn.disabled = true;
125 |         const originalText = saveBtn.textContent;
126 |         saveBtn.textContent = 'Сохранение...';
127 |
128 |         try {
129 |             await window.api.fetch('/auth/settings', {
130 |                 method: 'POST',
131 |                 headers: { 'Content-Type': 'application/json' },
132 |                 body: JSON.stringify({ model: selectedModel })
133 |             });
134 |             showModelsNotification('Модель успешно обновлена на: '
+ selectedModel, 'success');
135 |
136 |             const otherModelSelect = document.getElementById('admin-model-select');
137 |             if (otherModelSelect) {
138 |                 otherModelSelect.value = selectedModel;
139 |             }
140 |             if (typeof window.updateChatBadges === 'function') {
141 |                 window.updateChatBadges(selectedModel);
142 |             }
143 |         } catch (err) {
144 |             console.error('Ошибка сохранения модели:', err);
145 |             showModelsNotification('Ошибка сохранения: ' + err.message, 'danger');
146 |         } finally {
147 |             saveBtn.disabled = false;
148 |             saveBtn.textContent = originalText;
149 |         }
150 |     };
151 | }
152 |
153 | if (refreshKeysBtn) {
```

```
154     refreshKeysBtn.onclick = async () => {
155         refreshKeysBtn.disabled = true;
156         const originalText = refreshKeysBtn.textContent;
157         refreshKeysBtn.textContent = '⏸ Сброс...';
158         try {
159             const res = await window.api.fetch('/api/keys/reset-all', { method: 'POST' });
160             showModelsNotification(res.message || 'Квоты всех ключей успешно сброшены', 'success');
161         } catch (err) {
162             console.error('Ошибка сброса квот:', err);
163             showModelsNotification('Ошибка сброса: ' + err.message, 'danger');
164         } finally {
165             refreshKeysBtn.disabled = false;
166             refreshKeysBtn.textContent = originalText;
167             if (keysListBody) await refreshKeysList(keysListBody);
168         }
169     };
170 }
171
172 if (addKeyBtn) {
173     addKeyBtn.onclick = async () => {
174         const nameInput = document.getElementById('new-key-name');
175         const valueInput = document.getElementById('new-key-value');
176         if (!nameInput || !valueInput) return;
177
178         const name = nameInput.value.trim();
179         const apiKey = valueInput.value.trim();
180
181         if (!name || !apiKey) {
182             showModelsNotification('Заполните все поля!', 'warning');
183             return;
184         }
185     }
186 }
```

```
186     addKeyBtn.disabled = true;
187     try {
188         await window.api.fetch('/api/keys', {
189             method: 'POST',
190             headers: { 'Content-Type': 'application/json' },
191             body: JSON.stringify({ name, api_key: apiKey, status
: 'active' })
192         });
193         showModelsNotification(`Ключ "${name}" успешно добавле
н`, 'success');
194         nameInput.value = '';
195         valueInput.value = '';
196         if (keysListBody) await refreshKeysList(keysListBody);
197     } catch (err) {
198         console.error('Ошибка добавления ключа:', err);
199         showModelsNotification('Ошибка добавления: ' + err.mes
sage, 'danger');
200     } finally {
201         addKeyBtn.disabled = false;
202     }
203 };
204 }
205
206 // 2. Load all components concurrently
207 await Promise.allSettled([
208     modelSelect && saveBtn ? loadTabModels(modelSelect, saveBt
n) : Promise.resolve(),
209     keysListBody ? refreshKeysList(keysListBody) : Promise.res
olve(),
210     loadFoundryConfig(),
211     loadOllamaConfig(),
212     loadAgyConfig(),
213     loadSystemInstruction()
214 ]);
215 }
216
217 // Helper for escaping HTML strings
218 function escapeHtml(text) {
```

```
219 |   if (!text) return '';
220 |   return String(text)
221 |     .replace(/&/g, '&amp;')
222 |     .replace(/</g, '&lt;')
223 |     .replace(/>/g, '&gt;')
224 |     .replace(/"/g, '&quot;')
225 |     .replace(/'/g, '&#039;');
226 | }
227 |
228 | // Helper to execute model test request
229 | async function executeModelTest() {
230 |   const providerSelect = document.getElementById('provider-tab
231 |   -select');
232 |   const modelSelect = document.getElementById('models-tab-sele
233 |   ct');
234 |   const promptInput = document.getElementById('model-test-prom
235 |   pt');
236 |   const testBtn = document.getElementById('btn-model-test-send
237 |   ');
238 |   const resultContainer = document.getElementById('model-test-
239 |   result');
240 |
241 |   if (!testBtn || !resultContainer) return;
242 |
243 |   const provider = providerSelect ? providerSelect.value : '';
244 |   const model = modelSelect ? modelSelect.value : '';
245 |   const message = promptInput ? promptInput.value.trim() : '';
246 |
247 |   if (!model) {
248 |     showModelsNotification('Выберите модель для тестирования!'
249 |     , 'warning');
250 |     return;
251 |   }
252 |
253 |   if (!message) {
254 |     showModelsNotification('Введите текст проверочного запроса
255 |     !', 'warning');
256 |     if (promptInput) promptInput.focus();
257 |   }
258 | }
```

```

250     return;
251 }
252
253 testBtn.disabled = true;
254 const originalBtnHtml = testBtn.innerHTML;
255 testBtn.innerHTML = '<span class="spinner-border spinner-bor
256   der-sm me-1" role="status"></span> Тест...';
257
258 resultContainer.style.display = 'block';
259 resultContainer.innerHTML = `
260     <div class="d-flex align-items-center text-info small">
261         <span class="spinner-border spinner-border-sm me-2" role
262   ="status"></span>
263         <span>Отправка запроса в <strong>${escapeHtml(model)}</s
264   trong> (${escapeHtml(provider)})...</span>
265     </div>
266 `;
267
268 try {
269     const res = await window.api.fetch('/api/chat/test-model',
270     {
271         method: 'POST',
272         headers: { 'Content-Type': 'application/json' },
273         body: JSON.stringify({
274             provider: provider,
275             model: model,
276             message: message
277         })
278     });
279
280     if (res && res.status === 'success') {
281         resultContainer.innerHTML = `
282             <div class="d-flex justify-content-between align-items
283   -center mb-1 pb-1 border-bottom border-secondary">
284                 <div>
285                     <span class="badge bg-success me-2"><i class="bi b
286   i-check-circle me-1"></i>200 OK</span>
287                     <span class="text-secondary small font-monospace">

```



```

    □ ${res.duration_ms || 0} ms</span>
282         </div>
283         <span class="badge bg-dark border border-secondary text-
ext-info font-monospace">${escapeHtml(res.model || model)}</span>
284     </div>
285     <div class="text-white small mt-1 font-monospace" style="white-space: pre-wrap; word-break: break-word;">${escapeHtml(res.response || 'Пустой ответ от модели')}</div>
286     `;
287     } else {
288         const errMsg = (res && res.message) ? res.message : 'Неизвестная ошибка при запросе к модели';
289         resultContainer.innerHTML = `
290             <div class="d-flex justify-content-between align-items-center mb-1 pb-1 border-bottom border-secondary">
291                 <div>
292                     <span class="badge bg-danger me-2"><i class="bi bi-x-circle me-1"></i>Ошибка</span>
293                     <span class="text-secondary small font-monospace">
    □ ${res?.duration_ms || 0} ms</span>
294                 </div>
295                 <span class="badge bg-dark border border-secondary text-warning font-monospace">${escapeHtml(model)}</span>
296             </div>
297             <div class="text-danger small mt-1 font-monospace" style="white-space: pre-wrap; word-break: break-word;">${escapeHtml(errMsg)}</div>
298         `;
299     }
300 } catch (err) {
301     console.error('Error during model test request:', err);
302     resultContainer.innerHTML = `
303         <div class="d-flex justify-content-between align-items-center mb-1 pb-1 border-bottom border-secondary">
304             <span class="badge bg-danger"><i class="bi bi-x-circle me-1"></i>Ошибка сети/API</span>
305             <span class="badge bg-dark border border-secondary text

```

```

    t-warning font-monospace">${escapeHtml(model)}</span>
306     </div>
307     <div class="text-danger small mt-1 font-monospace" style
    ="white-space: pre-wrap; word-break: break-word;">${escapeHtml
    (err.message || 'Ошибка соединения')}</div>
308     `;
309   } finally {
310     testBtn.disabled = false;
311     testBtn.innerHTML = originalBtnHtml;
312   }
313 }
314
315 // Helper to load models list
316 async function loadTabModels(modelSelect, saveBtn, forceRefres
    h = false) {
317   const providerSelect = document.getElementById('provider-tab
    -select');
318   if (providerSelect) providerSelect.innerHTML = '';
319   modelSelect.innerHTML = '';
320
321   let modelsGrouped = {};
322
323   try {
324     const url = forceRefresh ? '/api/chat/models?refresh=true'
    : '/api/chat/models';
325     const modelsData = await window.api.fetch(url);
326     modelsGrouped = modelsData.models || {};
327     if (Array.isArray(modelsGrouped)) {
328       modelsGrouped = { 'gemini': modelsGrouped };
329     }
330     if (forceRefresh) {
331       showModelsNotification('Список моделей успешно обновлен'
    , 'success');
332     }
333   } catch (err) {
334     console.error('Error loading AI models:', err);
335     showModelsNotification('Ошибка загрузки моделей AI: ' + er
    r.message, 'danger');

```

```
336 | }
337 |
338 | const providers = Object.keys(modelsGrouped).filter(p => mod
    elsGrouped[p] && modelsGrouped[p].length > 0);
339 |
340 | if (providers.length === 0) {
341 |     if (providerSelect) {
342 |         providerSelect.innerHTML = '<option value="">Нет доступн
    ых провайдеров</option>';
343 |     }
344 |     modelSelect.innerHTML = '<option value="">Нет доступных мо
    делей</option>';
345 |     saveBtn.disabled = true;
346 |     return;
347 | }
348 |
349 | if (providerSelect) {
350 |     providerSelect.innerHTML = '';
351 |     providers.forEach(p => {
352 |         const option = document.createElement('option');
353 |         option.value = p;
354 |         option.textContent = p.charAt(0).toUpperCase() + p.slice
    (1);
355 |         providerSelect.appendChild(option);
356 |     });
357 | }
358 |
359 | const populateModels = (provider) => {
360 |     modelSelect.innerHTML = '';
361 |     const providerModels = modelsGrouped[provider] || [];
362 |     if (providerModels.length === 0) {
363 |         modelSelect.innerHTML = '<option value="">Нет моделей</o
    ption>';
364 |         saveBtn.disabled = true;
365 |     } else {
366 |         providerModels.forEach(modelName => {
367 |             const option = document.createElement('option');
368 |             option.value = modelName;
```

```

369 |         let cleanName = modelName;
370 |         if (cleanName.startsWith('foundry:')) cleanName = cleanName.substring(8);
371 |         else if (cleanName.startsWith('ollama:')) cleanName = cleanName.substring(7);
372 |         else if (cleanName.startsWith('gemini_cli:')) cleanName = cleanName.substring(11);
373 |         else if (cleanName.startsWith('agy-')) cleanName = cleanName.substring(4);
374 |         option.textContent = cleanName;
375 |         modelSelect.appendChild(option);
376 |     });
377 |     saveBtn.disabled = false;
378 | }
379 | };
380 |
381 | if (providerSelect) {
382 |     providerSelect.onChange = () => populateModels(providerSelect.value);
383 |     populateModels(providerSelect.value);
384 | } else {
385 |     let allModels = [];
386 |     providers.forEach(p => allModels = allModels.concat(modelsGrouped[p]));
387 |     allModels.forEach(modelName => {
388 |         const option = document.createElement('option');
389 |         option.value = modelName;
390 |         option.textContent = modelName;
391 |         modelSelect.appendChild(option);
392 |     });
393 |     saveBtn.disabled = allModels.length === 0;
394 | }
395 |
396 | try {
397 |     const settingsData = await window.api.fetch('/auth/settings');
398 |     if (settingsData && settingsData.model) {
399 |         let foundProvider = null;

```

```
400     for (const p of providers) {
401         if (modelsGrouped[p].includes(settingsData.model)) {
402             foundProvider = p;
403             break;
404         }
405     }
406     if (foundProvider && providerSelect) {
407         providerSelect.value = foundProvider;
408         populateModels(foundProvider);
409     }
410     modelSelect.value = settingsData.model;
411 }
412 } catch (err) {
413     console.error('Error loading user AI settings:', err);
414 }
415 }
416
417 // Helper to refresh keys list table
418 async function refreshKeysList(container) {
419     try {
420         container.innerHTML = '<tr><td colspan="5" class="text-center py-4 text-muted">Загрузка ключей...</td></tr>';
421         const keysData = await window.api.fetch('/api/keys');
422         const keys = keysData.keys || [];
423
424         if (keys.length === 0) {
425             container.innerHTML = '<tr><td colspan="5" class="text-center py-4 text-muted">Список ключей пуст</td></tr>';
426             return;
427         }
428
429         container.innerHTML = '';
430         keys.forEach(key => {
431             const row = document.createElement('tr');
432
433             const tdName = document.createElement('td');
434             tdName.innerHTML = `<strong>${key.name}</strong>`;
435             row.appendChild(tdName);
```

```
436
437     const tdKey = document.createElement('td');
438     tdKey.className = 'font-monospace text-muted small';
439     tdKey.textContent = key.api_key_masked;
440     row.appendChild(tdKey);
441
442     const tdStatus = document.createElement('td');
443     const isEnabled = key.status === 'active';
444     const statusClass = isEnabled ? 'bg-success' : 'bg-secondary';
445     const statusText = isEnabled ? 'Активен' : 'Отключен';
446     tdStatus.innerHTML = `
```

```

467 |         btnToggle.className = `btn btn-xs btn-sm me-1 ${isEnabled
    | d ? 'btn-outline-secondary' : 'btn-outline-success'}`;
468 |         btnToggle.textContent = isEnabled ? 'Откл' : 'Вкл';
469 |         btnToggle.onclick = () => toggleKeyStatus(key.name, isEn
    | abled ? 'disabled' : 'active', container);
470 |         tdActions.appendChild(btnToggle);
471 |
472 |         if (key.exhausted) {
473 |             const btnReset = document.createElement('button');
474 |             btnReset.className = 'btn btn-xs btn-outline-warning b
    | tn-sm me-1';
475 |             btnReset.innerHTML = 'Сброс';
476 |             btnReset.title = 'Сбросить 24-часовой бан квоты';
477 |             btnReset.onclick = () => resetKeyQuota(key.name, conta
    | iner);
478 |             tdActions.appendChild(btnReset);
479 |         }
480 |
481 |         const btnDelete = document.createElement('button');
482 |         btnDelete.className = 'btn btn-xs btn-outline-danger btn
    | -sm';
483 |         btnDelete.textContent = 'Удалить';
484 |         btnDelete.onclick = () => deleteKey(key.name, container)
    | ;
485 |         tdActions.appendChild(btnDelete);
486 |
487 |         row.appendChild(tdActions);
488 |         container.appendChild(row);
489 |     });
490 |
491 | } catch (err) {
492 |     console.error('Ошибка загрузки ключей:', err);
493 |     container.innerHTML = `<tr><td colspan="5" class="text-cen
    | ter py-4 text-danger">Ошибка: ${err.message}</td></tr>`;
494 | }
495 | }
496 |
497 | // API action helpers

```

```
498 async function toggleKeyStatus(name, newStatus, container) {
499   try {
500     await window.api.fetch(`/api/keys/${name}`, {
501       method: 'PATCH',
502       headers: { 'Content-Type': 'application/json' },
503       body: JSON.stringify({ status: newStatus })
504     });
505     showModelsNotification(`Статус ключа "${name}" изменен на
    ${newStatus === 'active' ? 'активный' : 'отключенный'}`, 'succ
    ess');
506     await refreshKeysList(container);
507   } catch (err) {
508     console.error('Ошибка переключения статуса ключа:', err);
509     showModelsNotification('Ошибка изменения статуса: ' + err.
    message, 'danger');
510   }
511 }
512
513 async function resetKeyQuota(name, container) {
514   try {
515     await window.api.fetch(`/api/keys/${name}/reset-quota`, {
    method: 'POST' });
516     showModelsNotification(`Квота для ключа "${name}" успешно
    сброшена`, 'success');
517     await refreshKeysList(container);
518   } catch (err) {
519     console.error('Ошибка сброса квоты:', err);
520     showModelsNotification('Ошибка сброса квоты: ' + err.messa
    ge, 'danger');
521   }
522 }
523
524 async function deleteKey(name, container) {
525   if (!confirm(`Вы уверены, что хотите удалить ключ "${name}"?
    `)) return;
526   try {
527     await window.api.fetch(`/api/keys/${name}`, { method: 'DEL
    ETE' });
```



```
528     showModelsNotification(`Ключ "${name}" успешно удален`, 's
| success');
529     await refreshKeysList(container);
530   } catch (err) {
531     console.error('Ошибка удаления ключа:', err);
532     showModelsNotification('Ошибка удаления: ' + err.message,
| 'danger');
533   }
534 }
535
536 function showModelsNotification(message, type = 'info') {
537   const notification = document.createElement('div');
538   notification.className = `alert alert-${type} position-fixed
| top-0 end-0 m-3`;
539   notification.style.zIndex = '9999';
540   notification.style.maxWidth = '400px';
541   notification.textContent = message;
542   document.body.appendChild(notification);
543
544   setTimeout(() => {
545     notification.remove();
546   }, 5000);
547 }
548
549 async function loadFoundryConfig() {
550   try {
551     const config = await window.api.fetch('/api/foundry/config
| ');
552     const enabledInput = document.getElementById('foundry-enab
| led');
553     const urlInput = document.getElementById('foundry-url');
554     const keyInput = document.getElementById('foundry-key');
555     const modelInput = document.getElementById('foundry-model'
| );
556
557     if (enabledInput) enabledInput.checked = config.enabled ||
| false;
558     if (urlInput) urlInput.value = config.url || '';
```

```
559     if (keyInput) keyInput.value = config.key || '';
560     if (modelInput) modelInput.value = config.model || '';
561 } catch (err) {
562     console.error('Ошибка загрузки настроек Foundry:', err);
563 }
564 }
565
566 async function loadOllamaConfig() {
567     try {
568         const config = await window.api.fetch('/api/ollama/config'
569 );
570         const enabledInput = document.getElementById('ollama-enabl
571 ed');
572         const urlInput = document.getElementById('ollama-url');
573         const modelInput = document.getElementById('ollama-model')
574 ;
575         if (enabledInput) enabledInput.checked = config.enabled ||
576 false;
577         if (urlInput) urlInput.value = config.url || '';
578         if (modelInput) modelInput.value = config.model || '';
579     } catch (err) {
580         console.error('Ошибка загрузки настроек Ollama:', err);
581     }
582 }
583
584 async function loadAgyConfig() {
585     try {
586         const modelSelect = document.getElementById('agy-model');
587         if (modelSelect) {
588             try {
589                 const modelsData = await window.api.fetch('/api/chat/m
590 odel
591 s');
592                 const agyList = modelsData.models?.agy || [];
593                 if (agyList.length > 0) {
594                     const curVal = modelSelect.value;
595                     modelSelect.innerHTML = '';
596                     agyList.forEach(m => {
```

```
592 |         const opt = document.createElement('option');
593 |         opt.value = m;
594 |         opt.textContent = m;
595 |         modelSelect.appendChild(opt);
596 |     });
597 |     if (curVal && agyList.includes(curVal)) {
598 |         modelSelect.value = curVal;
599 |     }
600 | }
601 | } catch (e) {
602 |     console.error('Ошибка загрузки моделей AGY:', e);
603 | }
604 | }
605 |
606 | const config = await window.api.fetch('/api/agy/config');
607 | const enabledInput = document.getElementById('agy-enabled'
    | );
608 | const keyInput = document.getElementById('agy-key');
609 |
610 | if (enabledInput) enabledInput.checked = config.enabled ??
    | true;
611 | if (modelSelect && config.model) modelSelect.value = confi
    | g.model;
612 | if (keyInput) keyInput.value = config.key || '';
613 | } catch (err) {
614 |     console.error('Ошибка загрузки настроек Antigravity (AGY):
    | ', err);
615 | }
616 | }
617 |
618 | async function loadSystemInstruction() {
619 |     const editor = document.getElementById('system-instruction-e
    | ditor');
620 |     const statusBadge = document.getElementById('system-instruct
    | ion-status');
621 |     if (!editor) return;
622 |
623 |     editor.disabled = true;
```

```
624 |   if (statusBadge) {
625 |       statusBadge.className = 'badge bg-warning text-dark';
626 |       statusBadge.textContent = 'Загрузка...';
627 |       statusBadge.style.removeProperty('display');
628 |   }
629 |
630 |   try {
631 |       const data = await window.api.fetch('/api/admin/system_instruction');
632 |       editor.value = data.content || '';
633 |       if (statusBadge) {
634 |           statusBadge.className = 'badge bg-success';
635 |           statusBadge.textContent = 'Загружено';
636 |           setTimeout(() => {
637 |               if (statusBadge) statusBadge.style.display = 'none';
638 |           }, 2500);
639 |       }
640 |   } catch (err) {
641 |       console.error('Ошибка загрузки системной инструкции:', err);
642 |       if (statusBadge) {
643 |           statusBadge.className = 'badge bg-danger';
644 |           statusBadge.textContent = 'Ошибка';
645 |           statusBadge.style.removeProperty('display');
646 |       }
647 |       showModelsNotification('Ошибка загрузки системной инструкции: ' + err.message, 'danger');
648 |   } finally {
649 |       editor.disabled = false;
650 |   }
651 | }
652 |
653 | async function saveSystemInstruction() {
654 |     const editor = document.getElementById('system-instruction-editor');
655 |     const saveBtn = document.getElementById('btn-save-instruction');
656 |     const statusBadge = document.getElementById('system-instruct
```

```
ion-status');
657   if (!editor || !saveBtn) return;
658
659   const originalHtml = saveBtn.innerHTML;
660   saveBtn.disabled = true;
661   saveBtn.innerHTML = '<span class="spinner-border spinner-border-sm" role="status"></span> Сохранение...';
662
663   try {
664     await window.api.fetch('/api/admin/system_instruction', {
665       method: 'POST',
666       headers: { 'Content-Type': 'application/json' },
667       body: JSON.stringify({ content: editor.value })
668     });
669     showModelsNotification('☑ Системная инструкция успешно сохранена', 'success');
670     if (statusBadge) {
671       statusBadge.className = 'badge bg-success';
672       statusBadge.textContent = 'Сохранено';
673       statusBadge.style.removeProperty('display');
674       setTimeout(() => {
675         if (statusBadge) statusBadge.style.display = 'none';
676       }, 3000);
677     }
678   } catch (err) {
679     console.error('Ошибка сохранения системной инструкции:', err);
680     if (statusBadge) {
681       statusBadge.className = 'badge bg-danger';
682       statusBadge.textContent = 'Ошибка сохранения';
683       statusBadge.style.removeProperty('display');
684     }
685     showModelsNotification('Ошибка сохранения: ' + err.message, 'danger');
686   } finally {
687     saveBtn.disabled = false;
688     saveBtn.innerHTML = originalHtml;
689   }
```

```
690 | }  
691 |  
692 | // Export for tab loader  
693 | window.initModelsTab = initModelsTab;  
694 |
```

index.html

```
1 <!-- Plugins Tab -->
2 <div class="container-fluid py-2">
3   <div class="alert alert-primary d-flex flex-wrap align-items
  -center justify-content-between gap-2 mb-3 shadow-sm">
4     <div class="d-flex align-items-center gap-2">
5       <span class="fs-4">□ </span>
6       <div>
7         <h5 class="mb-0 fw-bold">Подключатель плагинов и расши
  рений</h5>
8         <p class="mb-0 small text-muted">Централизованный ката
  лог модулей, активация и органы динамической настройки</p>
9       </div>
10    </div>
11    <div class="d-flex gap-2">
12      <button class="btn btn-sm btn-outline-primary" id="btn-r
  efresh-plugins" onclick="loadPluginsList()">
13        <i class="bi bi-arrow-clockwise"></i> Обновить список
14      </button>
15    </div>
16  </div>
17
18  <div class="row g-3">
19    <!-- LEFT: Plugin Catalog / List -->
20    <div class="col-lg-4 col-md-5">
21      <div class="card shadow-sm h-100 border-secondary-subtle
  ">
22        <div class="card-header bg-body-tertiary d-flex justif
  y-content-between align-items-center py-2">
23          <span class="fw-semibold small text-uppercase">Устан
  овленные плагины (<span id="plugins-count">0</span>)</span>
24          <span class="badge bg-secondary-subtle text-body" id
  ="active-plugins-badge">0 активных</span>
25        </div>
26        <div class="p-2 border-bottom">
27          <div class="input-group input-group-sm">
28            <span class="input-group-text bg-body border-secon
```

```

dary-subtle"><i class="bi bi-search"></i></span>
29         <input type="text" class="form-control border-seco
ndary-subtle" id="plugin-search-input" placeholder="Поиск пла
гина..." oninput="filterPluginsList()">
30         </div>
31     </div>
32     <div class="card-body p-2" id="plugins-list-container"
style="max-height: 70vh; overflow-y: auto;">
33         <div class="text-center text-muted p-4">
34             <div class="spinner-border spinner-border-sm text-
primary mb-2" role="status"></div>
35             <div>Загрузка плагинов...</div>
36         </div>
37     </div>
38 </div>
39 </div>
40
41 <!-- RIGHT: Selected Plugin Controls & Settings -->
42 <div class="col-lg-8 col-md-7">
43     <div class="card shadow-sm h-100 border-secondary-subtle
" id="plugin-details-card">
44         <div class="card-body d-flex flex-column align-items-c
enter justify-content-center text-muted p-5" id="plugin-placeh
older">
45             <i class="bi bi-puzzle fs-1 mb-2"></i>
46             <h6>Выберите плагин из списка слева для управления и
настройки</h6>
47         </div>
48
49         <div class="d-none d-flex flex-column h-100" id="plugi
n-content-pane">
50             <!-- Plugin Header -->
51             <div class="card-header bg-body-tertiary d-flex just
ify-content-between align-items-center py-3 border-bottom">
52                 <div class="d-flex align-items-center gap-3">
53                     <span class="fs-2" id="plugin-icon">□ </span>
54                     <div>
55                         <div class="d-flex align-items-center gap-2">

```



```
56 |             <h5 class="mb-0 fw-bold" id="plugin-title">Название плагина</h5>
57 |             <span class="badge bg-primary-subtle text-primary border border-primary-subtle" id="plugin-version">v1.0.0</span>
58 |             <span class="badge bg-secondary-subtle text-body border" id="plugin-category">tools</span>
59 |         </div>
60 |         <small class="text-muted" id="plugin-id">id: plugin_name</small>
61 |     </div>
62 | </div>
63 |     <div class="form-check form-switch fs-5 mb-0" title="Включить / отключить плагин">
64 |         <input class="form-check-input" type="checkbox" role="switch" id="plugin-toggle-switch" onchange="toggleCurrentPlugin(this.checked)">
65 |     </div>
66 | </div>
67 |
68 |     <!-- Description -->
69 |     <div class="p-3 bg-body-secondary border-bottom">
70 |         <p class="mb-0 small text-body" id="plugin-description">Описание возможностей плагина.</p>
71 |     </div>
72 |
73 |     <!-- Body: Settings & Actions -->
74 |     <div class="card-body p-3" style="max-height: 52vh; overflow-y: auto;">
75 |         <!-- Actions Section -->
76 |         <div class="mb-4" id="plugin-actions-section">
77 |             <h6 class="fw-bold mb-2 pb-1 border-bottom d-flex align-items-center gap-2">
78 |                 <i class="bi bi-lightning-charging text-warning"></i> Управляющие действия (Actions)
79 |             </h6>
80 |             <div class="d-flex flex-wrap gap-2 pt-1" id="plugin-actions-container">
```

```
81 |         <!-- Action buttons rendered here -->
82 |     </div>
83 | </div>
84 |
85 |     <!-- Configuration Section -->
86 |     <div class="mb-4" id="plugin-config-section">
87 |         <h6 class="fw-bold mb-2 pb-1 border-bottom d-flex align-items-center gap-2">
88 |             <i class="bi bi-sliders text-primary"></i> Па
89 |             аметры конфигурации
90 |         </h6>
91 |         <form id="plugin-config-form" onsubmit="event.pr
92 | eventDefault(); saveCurrentPluginConfig();">
93 |             <div class="row g-3" id="plugin-fields-contain
94 | er">
95 |                 <!-- Form fields rendered here -->
96 |                 </div>
97 |                 <div class="mt-3 text-end">
98 |                     <button type="submit" class="btn btn-sm btn-
99 | primary px-4" id="btn-save-plugin-config">
100 |                         <i class="bi bi-check2-circle"></i> Сохран
101 | ить настройки
102 |                     </button>
103 |                 </div>
104 |             </form>
105 |         </div>
106 |
107 |     <!-- Live Action Console / Terminal -->
108 |     <div class="mt-3">
109 |         <div class="d-flex justify-content-between align
110 | -items-center mb-1">
111 |             <h6 class="fw-bold mb-0 small text-muted d-flex align-items-center gap-1">
112 |                 <i class="bi bi-terminal"></i> Консоль выпол
113 | нения
114 |             </h6>
115 |             <button class="btn btn-sm btn-link text-decora
116 | tion-none text-muted py-0" onclick="clearPluginConsole()">Очис
```

```
        тить</button>
109 |         </div>
110 |         <pre class="p-2 rounded bg-dark text-light small
      | font-monospace mb-0" id="plugin-console-output" style="max-he
      | ight: 180px; min-height: 60px; overflow-y: auto;">Готов к рабо
      | те.</pre>
111 |         </div>
112 |     </div>
113 | </div>
114 | </div>
115 | </div>
116 | </div>
117 | </div>
118 |
```

main.js

```
1 // — PLUGINS TAB MAIN.JS
2
3 let loadedPlugins = [];
4 let selectedPluginName = '';
5
6 // Notification helper
7 function showPluginNotification(message, type = 'info') {
8   if (typeof window.showNotification === 'function') {
9     window.showNotification(message, type);
10    return;
11  }
12  console.log(`[Plugin Notification ${type}]: ${message}`);
13 }
14
15 // Initialize Plugins Tab
16 window.initPluginsTab = async function() {
17   console.log('Инициализация вкладки плагинов...');
18   await loadPluginsList();
19 };
20
21 // Загрузка списка всех плагинов
22 async function loadPluginsList() {
23   const container = document.getElementById('plugins-list-container');
24   if (!container) return;
25
26   try {
27     const data = await window.api.fetch('/api/admin/plugins');
28     loadedPlugins = data.plugins || [];
29
30     document.getElementById('plugins-count').textContent = loadedPlugins.length;
31     const activeCount = loadedPlugins.filter(p => p.enabled).length;
32     document.getElementById('active-plugins-badge').textConten
```

```

    t = `${activeCount} активных`;
33
34     renderPluginsList(loadedPlugins);
35     syncPluginTabsVisibility(loadedPlugins);
36
37     if (loadedPlugins.length > 0) {
38         // Если плагин не выбран или текущий пропал, выбираем пе
        рвый
39         const toSelect = loadedPlugins.find(p => p.name === sele
        ctedPluginName) || loadedPlugins[0];
40         selectPlugin(toSelect.name);
41     } else {
42         showPlaceholder();
43     }
44 } catch (ex) {
45     console.error('Ошибка загрузки плагинов:', ex);
46     container.innerHTML = `<div class="alert alert-danger m-2
        small">Ошибка загрузки плагинов: ${ex.message}</div>`;
47 }
48 }
49
50 // Отображение списка плагинов
51 function renderPluginsList(plugins) {
52     const container = document.getElementById('plugins-list-cont
        ainer');
53     if (!container) return;
54
55     if (plugins.length === 0) {
56         container.innerHTML = '<div class="text-center text-muted
        p-3 small">Плагины не найдены</div>';
57         return;
58     }
59
60     let html = '<div class="list-group list-group-flush">';
61     plugins.forEach(p => {
62         const isSelected = p.name === selectedPluginName;
63         const statusBadge = p.enabled
64             ? '<span class="badge bg-success-subtle text-success bor

```

```

    der border-success-subtle small">Вкл</span>'
65     : '<span class="badge bg-secondary-subtle text-muted bor
    der small">Выкл</span>';
66
67     html += `
68         <a href="javascript:void(0)"
69             class="list-group-item list-group-item-action p-2 rou
    nded mb-1 border-0 ${isSelected ? 'active text-white' : ''}"
70             onclick="selectPlugin('${p.name}')"
71             style="cursor: pointer;">
72             <div class="d-flex w-100 justify-content-between align
    -items-center mb-1">
73                 <div class="d-flex align-items-center gap-2 text-tru
    ncate">
74                     <span class="fs-5">${p.icon || '□ '}</span>
75                     <strong class="text-truncate">${p.title || p.name}
    </strong>
76                 </div>
77                 <div>${statusBadge}</div>
78             </div>
79             <div class="small text-truncate ${isSelected ? 'text-w
    hite-50' : 'text-muted'}">
80                 ${p.description || p.name}
81             </div>
82         </a>
83     `;
84 });
85 html += '</div>';
86 container.innerHTML = html;
87 }
88
89 // Фильтрация списка плагинов
90 function filterPluginsList() {
91     const query = (document.getElementById('plugin-search-input'
    )?.value || '').toLowerCase().trim();
92     if (!query) {
93         renderPluginsList(loadedPlugins);
94         return;

```

```
95 | }
96 | const filtered = loadedPlugins.filter(p =>
97 |   p.name.toLowerCase().includes(query) ||
98 |   (p.title && p.title.toLowerCase().includes(query)) ||
99 |   (p.description && p.description.toLowerCase().includes(que
    | ry))
100 | );
101 | renderPluginsList(filtered);
102 | }
103 |
104 | // Выбор плагина для отображения органов управления
105 | function selectPlugin(pluginName) {
106 |   selectedPluginName = pluginName;
107 |   const plugin = loadedPlugins.find(p => p.name === pluginName
    | );
108 |   if (!plugin) return;
109 |
110 |   renderPluginsList(loadedPlugins);
111 |
112 |   const placeholder = document.getElementById('plugin-placehol
    | der');
113 |   const contentPane = document.getElementById('plugin-content-
    | pane');
114 |   if (placeholder) placeholder.classList.add('d-none');
115 |   if (contentPane) contentPane.classList.remove('d-none');
116 |
117 |   // Header info
118 |   document.getElementById('plugin-icon').textContent = plugin.
    | icon || '□';
119 |   document.getElementById('plugin-title').textContent = plugin
    | .title || plugin.name;
120 |   document.getElementById('plugin-version').textContent = `v${
    | plugin.version || '1.0.0'}`;
121 |   document.getElementById('plugin-category').textContent = plu
    | gin.category || 'tools';
122 |   document.getElementById('plugin-id').textContent = `id: ${pl
    | ugin.name}`;
123 |   document.getElementById('plugin-description').textContent =
```

```
    plugin.description || 'Нет описания';
124
125    const toggleSwitch = document.getElementById('plugin-toggle-
    switch');
126    if (toggleSwitch) {
127        toggleSwitch.checked = Boolean(plugin.enabled);
128    }
129
130    // Render Actions
131    renderPluginActions(plugin);
132
133    // Render Config Fields
134    renderPluginFields(plugin);
135 }
136
137 // Отрисовка кнопок действий (Actions)
138 function renderPluginActions(plugin) {
139     const container = document.getElementById('plugin-actions-co
    ntainer');
140     const section = document.getElementById('plugin-actions-sect
    ion');
141     if (!container || !section) return;
142
143     const actions = plugin.actions || [];
144     if (actions.length === 0) {
145         section.classList.add('d-none');
146         return;
147     }
148
149     section.classList.remove('d-none');
150     let html = '';
151     actions.forEach(act => {
152         const btnColor = act.color || 'primary';
153         html += `
154         <button class="btn btn-sm btn-outline-${btnColor} d-inli
    ne-flex align-items-center gap-1 shadow-sm"
155             onclick="executePluginAction('${plugin.name}', '
    ${act.id}', '${act.label}')"`
```



```

156 |         title="${act.description || ''}">
157 |         <span>${act.label}</span>
158 |     </button>
159 |     `;
160 | });
161 | container.innerHTML = html;
162 | }
163 |
164 | // Отрисовка полей конфигурации
165 | function renderPluginFields(plugin) {
166 |     const container = document.getElementById('plugin-fields-con
167 |     tainer');
168 |     const section = document.getElementById('plugin-config-secti
169 |     on');
170 |     if (!container || !section) return;
171 |
172 |     const fields = plugin.fields || [];
173 |     if (fields.length === 0) {
174 |         section.classList.add('d-none');
175 |         return;
176 |     }
177 |
178 |     section.classList.remove('d-none');
179 |     const currentCfg = plugin.config || {};
180 |
181 |     let html = '';
182 |     fields.forEach(f => {
183 |         const val = (currentCfg[f.id] !== undefined) ? currentCfg[
184 |         f.id] : (f.default !== undefined ? f.default : '');
185 |         const desc = f.description ? `<div class="form-text text-m
186 |         uted small">${f.description}</div>` : '';
187 |
188 |         if (f.type === 'select') {
189 |             let optionsHtml = '';
190 |             (f.options || []).forEach(opt => {
191 |                 const isSel = String(opt.value) === String(val) ? 'sel
192 |                 ected' : '';
193 |                 optionsHtml += `<option value="${opt.value}" ${isSel}>

```

```

    ${opt.label}</option>`;
189     });
190     html += `
191         <div class="col-12 col-md-6">
192             <label class="form-label small fw-semibold mb-1">${f
193             .label}</label>
194             <select class="form-select form-select-sm" name="${f
195             .id}" data-field-type="select">
196                 ${optionsHtml}
197             </select>
198             ${desc}
199         </div>
200     `;
201     } else if (f.type === 'boolean') {
202         const checked = Boolean(val) ? 'checked' : '';
203         html += `
204             <div class="col-12 col-md-6 d-flex flex-column justify
205             -content-center">
206                 <div class="form-check form-switch mt-2">
207                     <input class="form-check-input" type="checkbox" ro
208                     le="switch" name="${f.id}" id="field_${f.id}" ${checked} data-
209                     field-type="boolean">
210                         <label class="form-check-label small fw-semibold"
211                         for="field_${f.id}">${f.label}</label>
212                     </div>
213                     ${desc}
214                 </div>
215             `;
216     } else if (f.type === 'list_string') {
217         const listVals = Array.isArray(val) ? val.join(', ') : (
218         val || '');
219         html += `
220             <div class="col-12">
221                 <label class="form-label small fw-semibold mb-1">${f
222                 .label} (через запятую)</label>
223                 <input type="text" class="form-control form-control-
224                 sm font-monospace" name="${f.id}" value="${listVals}" data-fie
225                 ld-type="list_string">

```

```

216         ${desc}
217     </div>
218     `;
219     } else if (f.type === 'readonly') {
220         html += `
221             <div class="col-12 col-md-6">
222                 <label class="form-label small fw-semibold mb-1 text
223                 -muted">${f.label}</label>
224                 <input type="text" class="form-control form-control-
225                 sm bg-body-tertiary text-muted" value="${val}" readonly>
226                     ${desc}
227                 </div>
228             `;
229     } else if (f.type === 'number') {
230         html += `
231             <div class="col-12 col-md-6">
232                 <label class="form-label small fw-semibold mb-1">${f
233                 .label}</label>
234                 <input type="number" class="form-control form-control-
235                 l-sm" name="${f.id}" value="${val}" data-field-type="number">
236                     ${desc}
237                 </div>
238             `;
239     } else {
240         html += `
241             <div class="col-12 col-md-6">
242                 <label class="form-label small fw-semibold mb-1">${f
243                 .label}</label>
244                 <input type="text" class="form-control form-control-
245                 sm" name="${f.id}" value="${val}" data-field-type="string">
246                     ${desc}
247                 </div>
248             `;
249     }
250     });
251     container.innerHTML = html;
252 }

```

```
248
249 // Переключение состояния плагина (Вкл / Выкл)
250 async function toggleCurrentPlugin(enabled) {
251   if (!selectedPluginName) return;
252
253   try {
254     logToPluginConsole(`Переключение плагина '${selectedPlugin
Name}' -> ${enabled ? 'ВКЛ' : 'ВЫКЛ'}...`);
255     const res = await window.api.fetch(`/api/admin/plugins/${s
electedPluginName}/toggle`, {
256       method: 'POST',
257       headers: { 'Content-Type': 'application/json' },
258       body: JSON.stringify({ enabled })
259     });
260
261     const plugin = loadedPlugins.find(p => p.name === selected
PluginName);
262     if (plugin) {
263       plugin.enabled = enabled;
264     }
265
266     renderPluginsList(loadedPlugins);
267     syncPluginTabsVisibility(loadedPlugins);
268     showPluginNotification(res.message || 'Статус плагина обно
влен', 'success');
269     logToPluginConsole(`👉 ${res.message || 'Статус успешно обн
овлен'}`);
270   } catch (ex) {
271     console.error('Ошибка переключения плагина:', ex);
272     showPluginNotification(`Ошибка: ${ex.message}`, 'danger');
273     logToPluginConsole(`👉 Ошибка: ${ex.message}`);
274     // Возвращаем тумблер в исходное состояние
275     const toggleSwitch = document.getElementById('plugin-toggl
e-switch');
276     if (toggleSwitch) toggleSwitch.checked = !enabled;
277   }
278 }
279
```

```

280 // Сохранение настроек текущего плагина
281 async function saveCurrentPluginConfig() {
282   if (!selectedPluginName) return;
283
284   const form = document.getElementById('plugin-config-form');
285   if (!form) return;
286
287   const newConfig = {};
288   const elements = form.querySelectorAll('[data-field-type]');
289
290   elements.forEach(el => {
291     const name = el.getAttribute('name');
292     if (!name) return;
293     const type = el.getAttribute('data-field-type');
294
295     if (type === 'boolean') {
296       newConfig[name] = el.checked;
297     } else if (type === 'number') {
298       newConfig[name] = Number(el.value);
299     } else if (type === 'list_string') {
300       newConfig[name] = el.value.split(',').map(s => s.trim())
301         .filter(Boolean);
302     } else {
303       newConfig[name] = el.value;
304     }
305   });
306
307   try {
308     logToPluginConsole(`Сохранение параметров плагина '${selectedPluginName}'...`);
309     const res = await window.api.fetch(`/api/admin/plugins/${selectedPluginName}/config`, {
310       method: 'POST',
311       headers: { 'Content-Type': 'application/json' },
312       body: JSON.stringify({ config: newConfig })
313     });
314     const plugin = loadedPlugins.find(p => p.name === selected

```

```

    PluginName);
315     if (plugin) {
316         plugin.config = res.config || newConfig;
317     }
318
319     showPluginNotification('Параметры успешно сохранены', 'suc
    cess');
320     logToPluginConsole(`□ Конфигурация сохранена: ${JSON.strin
    gify(newConfig, null, 2)}`);
321 } catch (ex) {
322     console.error('Ошибка сохранения конфигурации:', ex);
323     showPluginNotification(`Ошибка: ${ex.message}`, 'danger');
324     logToPluginConsole(`□ Ошибка сохранения: ${ex.message}`);
325 }
326 }
327
328 // Выполнение специфического действия (Action) плагина
329 async function executePluginAction(pluginName, actionId, actio
    nLabel) {
330     try {
331         logToPluginConsole(`[ACTION] Запуск '${actionLabel}' для п
    лагина '${pluginName}'...`);
332         showPluginNotification(`Запуск '${actionLabel}'...`, 'info
    ');
333
334         // Собираем текущие поля формы как параметры действия
335         const form = document.getElementById('plugin-config-form')
    ;
336         const params = {};
337         if (form) {
338             const elements = form.querySelectorAll('[data-field-type
    ]');
339             elements.forEach(el => {
340                 const name = el.getAttribute('name');
341                 if (!name) return;
342                 const type = el.getAttribute('data-field-type');
343                 if (type === 'boolean') params[name] = el.checked;
344                 else if (type === 'number') params[name] = Number(el.v

```

```

    alue);
345     else if (type === 'list_string') params[name] = el.value.split(',').map(s => s.trim()).filter(Boolean);
346     else params[name] = el.value;
347   });
348   }
349
350   const res = await window.api.fetch(`/api/admin/plugins/${pluginName}/action/${actionId}`, {
351     method: 'POST',
352     headers: { 'Content-Type': 'application/json' },
353     body: JSON.stringify({ params })
354   });
355
356   if (res.success) {
357     showPluginNotification(res.message || 'Действие успешно выполнено', 'success');
358     logToPluginConsole(`\n ${res.message || 'Успех'}\n${JSON.stringify(res.result || res, null, 2)}\n`);
359   } else {
360     showPluginNotification(res.message || 'Ошибка выполнения действия', 'warning');
361     logToPluginConsole(`\n ${res.message || 'Завершено с предупреждением'}\n`);
362   }
363 } catch (ex) {
364   console.error(`Ошибка выполнения действия ${actionId}:`, ex);
365   showPluginNotification(`Ошибка: ${ex.message}`, 'danger');
366   logToPluginConsole(`\n Ошибка выполнения: ${ex.message}\n`);
367 }
368 }
369
370 // Логирование в консоль плагина
371 function logToPluginConsole(text) {
372   const consoleEl = document.getElementById('plugin-console-output');
373   if (!consoleEl) return;

```

```
374 |   const time = new Date().toLocaleTimeString();
375 |   consoleEl.textContent = `[${time}] ${text}\n` + consoleEl.te
    | xtextContent;
376 | }
377 |
378 | // Очистка консоли
379 | function clearPluginConsole() {
380 |   const consoleEl = document.getElementById('plugin-console-ou
    | tput');
381 |   if (consoleEl) consoleEl.textContent = 'Консоль очищена.\n';
382 | }
383 |
384 | function showPlaceholder() {
385 |   const placeholder = document.getElementById('plugin-placehol
    | der');
386 |   const contentPane = document.getElementById('plugin-content-
    | pane');
387 |   if (placeholder) placeholder.classList.remove('d-none');
388 |   if (contentPane) contentPane.classList.add('d-none');
389 | }
390 |
391 | // Синхронизация видимости вкладок в навбаре с состоянием акти
    | вности плагинов
392 | function syncPluginTabsVisibility(plugins) {
393 |   if (!Array.isArray(plugins)) return;
394 |   const pluginMap = {};
395 |   plugins.forEach(p => {
396 |     pluginMap[p.name] = Boolean(p.enabled);
397 |   });
398 |
399 |   // Находим все элементы навигации, привязанные к плагинам
400 |   const pluginNavItems = document.querySelectorAll('[data-plug
    | in-tab]');
401 |   pluginNavItems.forEach(item => {
402 |     const pluginName = item.getAttribute('data-plugin-tab');
403 |     const isEnabled = (pluginMap[pluginName] !== undefined) ?
    | pluginMap[pluginName] : false;
404 |
```



```
405     if (isEnabled) {
406         item.classList.remove('d-none');
407     } else {
408         item.classList.add('d-none');
409         // Если отключенная вкладка была активна, переключаемся
на вкладку чата
410         const button = item.querySelector('.nav-link');
411         if (button && button.classList.contains('active')) {
412             const chatTab = document.querySelector('[data-bs-target=
t="#tab-chat"]');
413             if (chatTab) {
414                 const tab = new bootstrap.Tab(chatTab);
415                 tab.show();
416             }
417         }
418     }
419 });
420 }
421
422 // Make functions globally available for HTML inline handlers
423 window.loadPluginsList = loadPluginsList;
424 window.filterPluginsList = filterPluginsList;
425 window.selectPlugin = selectPlugin;
426 window.toggleCurrentPlugin = toggleCurrentPlugin;
427 window.saveCurrentPluginConfig = saveCurrentPluginConfig;
428 window.executePluginAction = executePluginAction;
429 window.clearPluginConsole = clearPluginConsole;
430 window.syncPluginTabsVisibility = syncPluginTabsVisibility;
431
```

index.html

```
1 <!DOCTYPE html>
2 <html lang="ru">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-s
  cale=1.0, maximum-scale=1.0, user-scalable=no">
6   <title>Медиа Плеер - Пульт ДУ</title>
7
8   <!-- Bootstrap 5 CSS -->
9   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dis
  t/css/bootstrap.min.css" rel="stylesheet">
10  <!-- Bootstrap Icons -->
11  <link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.1
  0.0/font/bootstrap-icons.css" rel="stylesheet">
12
13  <!-- Telegram Web Apps SDK -->
14  <script src="https://telegram.org/js/telegram-web-app.js"></
  script>
15  <script src="https://cdn.jsdelivr.net/npm/marked/marked.min.
  js"></script>
16
17  <style>
18    :root {
19      --tg-theme-bg-color: #0d1117;
20      --tg-theme-text-color: #c9d1d9;
21      --tg-theme-hint-color: #8b949e;
22      --tg-theme-secondary-bg-color: #161b22;
23      --primary-gradient: linear-gradient(135deg, #1f6feb 0%,
  #094cb5 100%);
24      --accent-gradient: linear-gradient(135deg, #ff7b00 0%, #
  ffae00 100%);
25      --success-gradient: linear-gradient(135deg, #238636 0%,
  #2ea043 100%);
26    }
27
28    body {
```

```
29 |     background: var(--tg-theme-bg-color);
30 |     color: var(--tg-theme-text-color);
31 |     font-family: system-ui, sans-serif;
32 |     min-height: 100vh;
33 |     display: flex;
34 |     flex-direction: column;
35 |     justify-content: center;
36 |     align-items: center;
37 |     padding: 10px;
38 | }
39 |
40 | .rc-container, .auth-container, .scan-container {
41 |     width: 100%;
42 |     max-width: 480px;
43 |     padding: 2px;
44 | }
45 |
46 | .card-rc, .card-auth, .card-scan {
47 |     background-color: var(--tg-theme-secondary-bg-color);
48 |     border: 1px solid #30363d;
49 |     border-radius: 16px;
50 |     padding: 1px;
51 |     box-shadow: 0 8px 32px rgba(0,0,0,0.6);
52 |     backdrop-filter: blur(8px);
53 | }
54 |
55 | .btn-circle-lg {
56 |     width: 64px;
57 |     height: 64px;
58 |     border-radius: 50%;
59 |     display: flex;
60 |     align-items: center;
61 |     justify-content: center;
62 | }
63 |
64 | .btn-circle-xl {
65 |     width: 80px;
66 |     height: 80px;
```

```
67 |     border-radius: 50%;
68 |     display: flex;
69 |     align-items: center;
70 |     justify-content: center;
71 | }
72 |
73 | .track-title {
74 |     font-size: 1.15rem;
75 |     font-weight: 600;
76 |     color: #58a6ff;
77 | }
78 |
79 | .playlist-box {
80 |     max-height: 200px;
81 |     overflow-y: auto;
82 |     border-radius: 8px;
83 |     background: #0d1117;
84 | }
85 |
86 | .list-group-item-action:hover {
87 |     background-color: #21262d !important;
88 | }
89 |
90 | .btn-carousel {
91 |     background: var(--accent-gradient);
92 |     border: none;
93 |     color: #000;
94 |     font-weight: bold;
95 |     transition: all 0.3s ease;
96 |     box-shadow: 0 4px 15px rgba(255, 123, 0, 0.3);
97 | }
98 | .btn-carousel:hover {
99 |     transform: translateY(-2px);
100 |     box-shadow: 0 6px 20px rgba(255, 123, 0, 0.5);
101 |     background: linear-gradient(135deg, #ffae00 0%, #ff7b00
102 | 100%);
103 |     color: #000;
104 | }
```

```
104 | .btn-carousel:active {
105 |     transform: translateY(1px);
106 | }
107 | .btn-carousel:hover i {
108 |     display: inline-block;
109 |     animation: spin 1s linear infinite;
110 | }
111 | @keyframes spin {
112 |     100% { transform: rotate(360deg); }
113 | }
114 |
115 | /* Chat bubbles styling */
116 | .chat-msg {
117 |     display: flex;
118 |     flex-direction: column;
119 |     max-width: 85%;
120 |     margin-bottom: 8px;
121 | }
122 | .chat-msg.user {
123 |     align-self: flex-end;
124 | }
125 | .chat-msg.ai, .chat-msg.system {
126 |     align-self: flex-start;
127 | }
128 | .chat-msg.user .msg-bubble {
129 |     background: var(--primary-gradient);
130 |     color: #fff;
131 |     border-radius: 12px 12px 0 12px;
132 |     box-shadow: 0 2px 8px rgba(31, 111, 235, 0.2);
133 | }
134 | .chat-msg.ai .msg-bubble {
135 |     background-color: #21262d;
136 |     color: #c9d1d9;
137 |     border-radius: 12px 12px 12px 0;
138 |     border: 1px solid #30363d;
139 | }
140 | .chat-msg.system .msg-bubble {
141 |     background-color: rgba(255, 255, 255, 0.05);
```

```
142     color: #8b949e;
143     border-radius: 8px;
144     border-left: 3px solid #58a6ff;
145 }
146 .msg-bubble {
147     padding: 8px 12px;
148     font-size: 0.9rem;
149     word-break: break-word;
150 }
151 .msg-time {
152     font-size: 0.7rem;
153     color: #8b949e;
154     margin-top: 2px;
155     padding: 0 4px;
156 }
157 .chat-msg.user .msg-time {
158     text-align: right;
159 }
160 .animate-pulse {
161     animation: pulse 1.5s infinite;
162 }
163 @keyframes pulse {
164     0% { transform: scale(1); opacity: 1; }
165     50% { transform: scale(1.15); opacity: 0.7; }
166     100% { transform: scale(1); opacity: 1; }
167 }
168
169 .search-result-card {
170     background: rgba(88, 166, 255, 0.07);
171     border: 1px solid rgba(88, 166, 255, 0.2);
172     border-radius: 8px;
173     transition: all 0.2s ease;
174 }
175 .search-result-card:hover {
176     background: rgba(88, 166, 255, 0.12);
177     border-color: rgba(88, 166, 255, 0.4);
178 }
179
```

```
180 | /* Auth panel specific */
181 | .form-control {
182 |     background: #0d1117;
183 |     border: 1px solid #30363d;
184 |     color: #c9d1d9;
185 | }
186 | .form-control:focus {
187 |     background: #0d1117;
188 |     border-color: #388bfd;
189 |     color: #c9d1d9;
190 |     box-shadow: 0 0 0 2px rgba(56,139,253,0.3);
191 | }
192 | .btn-auth-primary {
193 |     background: var(--primary-gradient);
194 |     border: none;
195 |     color: #fff;
196 | }
197 | .btn-auth-success {
198 |     background: var(--success-gradient);
199 |     border: none;
200 |     color: #fff;
201 | }
202 |
203 | /* Scanning styles */
204 | .scan-radar {
205 |     position: relative;
206 |     width: 100px;
207 |     height: 100px;
208 |     margin: 0 auto 20px;
209 |     border-radius: 50%;
210 |     border: 2px solid rgba(88, 166, 255, 0.3);
211 |     display: flex;
212 |     align-items: center;
213 |     justify-content: center;
214 | }
215 | .scan-radar::after {
216 |     content: '';
217 |     position: absolute;
```

```
218 |     width: 100%;
219 |     height: 100%;
220 |     border-radius: 50%;
221 |     border: 2px solid #58a6ff;
222 |     animation: radar-ping 2s infinite linear;
223 | }
224 | @keyframes radar-ping {
225 |     0% { transform: scale(1); opacity: 1; }
226 |     100% { transform: scale(1.6); opacity: 0; }
227 | }
228 | .player-list-item {
229 |     background: #161b22;
230 |     border: 1px solid #30363d;
231 |     border-radius: 8px;
232 |     padding: 12px;
233 |     margin-bottom: 8px;
234 |     display: flex;
235 |     justify-content: space-between;
236 |     align-items: center;
237 |     transition: all 0.2s ease;
238 | }
239 | .player-list-item:hover {
240 |     border-color: #58a6ff;
241 |     background: rgba(88, 166, 255, 0.05);
242 | }
243 |
244 | /* Theme toggle button */
245 | .theme-toggle-btn {
246 |     position: absolute;
247 |     top: 10px;
248 |     right: 10px;
249 |     background: rgba(255, 255, 255, 0.1);
250 |     border: 1px solid rgba(255, 255, 255, 0.2);
251 |     color: #fff;
252 |     width: 36px;
253 |     height: 36px;
254 |     border-radius: 50%;
255 |     cursor: pointer;
```



```
256 |     display: flex;
257 |     align-items: center;
258 |     justify-content: center;
259 |     font-size: 16px;
260 |     z-index: 1000;
261 | }
262 | .theme-toggle-btn:hover {
263 |     background: rgba(255, 255, 255, 0.2);
264 | }
265 |
266 | /* Light theme styles */
267 | .theme-light .card-rc,
268 | .theme-light .card-auth,
269 | .theme-light .card-scan {
270 |     background-color: #ffffff;
271 |     border-color: #d0d7de;
272 | }
273 | .theme-light .track-title {
274 |     color: #0969da;
275 | }
276 | .theme-light .form-control {
277 |     background: #ffffff;
278 |     border-color: #d0d7de;
279 |     color: #24292f;
280 | }
281 | .theme-light .form-control:focus {
282 |     background: #ffffff;
283 |     border-color: #0969da;
284 | }
285 | .theme-light .btn-primary {
286 |     background: #2da44e;
287 |     border-color: #2da44e;
288 | }
289 | .theme-light .btn-primary:hover {
290 |     background: #2da44e;
291 | }
292 | .theme-light .btn-outline-light {
293 |     color: #24292f;
```

```
294 |     border-color: #d0d7de;
295 | }
296 | .theme-light .btn-outline-light:hover {
297 |     background: #f6f8fa;
298 | }
299 | .theme-light .btn-circle-lg {
300 |     background: #ffffff;
301 |     border-color: #d0d7de;
302 |     color: #24292f;
303 | }
304 | .theme-light .btn-circle-lg:hover {
305 |     background: #f6f8fa;
306 | }
307 | .theme-light .text-muted {
308 |     color: #6e7781 !important;
309 | }
310 | .theme-light .chat-msg.user .msg-bubble {
311 |     background: #0969da;
312 | }
313 | .theme-light .chat-msg.ai .msg-bubble {
314 |     background-color: #f6f8fa;
315 |     color: #24292f;
316 | }
317 | .theme-light .chat-msg.system .msg-bubble {
318 |     background-color: #f6f8fa;
319 |     color: #6e7781;
320 | }
321 | .theme-light .player-list-item {
322 |     background: #ffffff;
323 |     border-color: #d0d7de;
324 | }
325 | .theme-light .player-list-item:hover {
326 |     background: #f6f8fa;
327 | }
328 | .theme-light .list-group-item {
329 |     background: #ffffff;
330 |     border-color: #d0d7de;
331 |     color: #24292f;
```

```
332     }
333     .theme-light .list-group-item:hover {
334         background: #f6f8fa;
335     }
336     .theme-light .form-range {
337         color-scheme: light;
338     }
339 </style>
340 </head>
341 <body>
342
343     <!-- 1. AUTHENTICATION PANEL -->
344     <div class="auth-container" id="auth-panel">
345         <div class="card-auth">
346             <div class="text-center mb-4">
347                 <i class="bi bi-shield-lock-fill fs-1 text-primary"></i>
348                 <h4 class="mt-2 text-white">Вход в пульт ДУ</h4>
349                 <p class="text-muted small">Пожалуйста, войдите в аккаунт для продолжения</p>
350             </div>
351
352             <div id="auth-alert" class="alert alert-danger d-none" role="alert"></div>
353
354             <!-- Login Form -->
355             <div id="auth-form-login">
356                 <form id="form-login">
357                     <div class="mb-3">
358                         <label for="login-email" class="form-label text-muted small">Email</label>
359                         <input type="email" class="form-control" id="login-email" required>
360                     </div>
361                     <div class="mb-3">
362                         <label for="login-password" class="form-label text-muted small">Пароль</label>
363                         <input type="password" class="form-control" id="lo
```

```

    gin-password" required>
364         </div>
365         <button type="submit" class="btn btn-auth-primary w-
100 py-2.5 rounded-pill mb-3">Войти</button>
366         <p class="text-center small mb-0">Нет аккаунта? <a h
ref="#" id="go-to-register" class="text-info">Зарегистрировать
ся</a></p>
367     </form>
368 </div>
369
370 <!-- Register Form -->
371 <div id="auth-form-register" class="d-none">
372     <form id="form-register">
373         <div class="mb-3">
374             <label for="register-name" class="form-label text-
muted small">Имя</label>
375             <input type="text" class="form-control" id="regist
er-name" required>
376         </div>
377         <div class="mb-3">
378             <label for="register-email" class="form-label text
-muted small">Email</label>
379             <input type="email" class="form-control" id="regis
ter-email" required>
380         </div>
381         <div class="mb-3">
382             <label for="register-password" class="form-label t
ext-muted small">Пароль</label>
383             <input type="password" class="form-control" id="re
gister-password" required>
384         </div>
385         <button type="submit" class="btn btn-auth-success w-
100 py-2.5 rounded-pill mb-3">Зарегистрироваться</button>
386         <p class="text-center small mb-0">Уже есть аккаунт?
<a href="#" id="go-to-login" class="text-info">Войти</a></p>
387     </form>
388 </div>
389

```

```

390 |     <!-- Verify Form -->
391 |     <div id="auth-form-verify" class="d-none">
392 |         <p class="text-center small">На вашу почту отправлен 6
| -значный код подтверждения. Пожалуйста, введите его ниже.</p>
393 |         <form id="form-verify">
394 |             <div class="mb-3">
395 |                 <label for="verify-code" class="form-label text-mu
| ted small">Код подтверждения</label>
396 |                 <input type="text" class="form-control text-center
| fs-3 py-2" id="verify-code" maxlength="6" placeholder="XXXXXX
| " required>
397 |             </div>
398 |             <button type="submit" class="btn btn-info w-100 py-2
| .5 text-white rounded-pill mb-3">Подтвердить</button>
399 |             <p class="text-center small mb-0"><a href="#" id="ve
| rify-cancel" class="text-info">Назад к входу</a></p>
400 |         </form>
401 |     </div>
402 |
403 |     <div class="my-4 text-center position-relative">
404 |         <hr class="border-secondary">
405 |         <span class="px-2 text-muted position-absolute top-50
| start-50 translate-middle small" style="background-color: var(
| --tg-theme-secondary-bg-color);">Или войти через</span>
406 |     </div>
407 |
408 |     <div class="d-flex flex-column gap-2 align-items-center"
| >
409 |         <button id="google-signin-btn" class="btn btn-outline-
| light w-100 rounded-pill d-flex align-items-center justify-con
| tent-center gap-2 py-2">
410 |             <i class="bi bi-google text-danger"></i>
411 |             <span>Google</span>
412 |         </button>
413 |     </div>
414 | </div>
415 | </div>
416 |

```

```

417 | <!-- 2. PLAYER DISCOVERY SCANNER PANEL -->
418 | <div class="scan-container d-none" id="scan-panel">
419 |     <div class="card-scan text-center">
420 |         <div class="scan-radar">
421 |             <i class="bi bi-tv fs-2 text-primary"></i>
422 |         </div>
423 |         <h4 class="text-white" id="scan-title">Поиск активных пл
| ееров</h4>
424 |         <p class="text-muted small" id="scan-status">Сканируем с
| еть на наличие запущенных устройств...</p>
425 |
426 |         <div id="discovered-players-list" class="mt-4 text-start
| "></div>
427 |
428 |         <button id="btn-retry-scan" class="btn btn-outline-prima
| ry w-100 py-2.5 rounded-pill mt-3 d-none">
429 |             <i class="bi bi-arrow-clockwise"></i> Обновить поиск
430 |         </button>
431 |
432 |         <button id="btn-scan-logout" class="btn btn-sm btn-outli
| ne-danger w-100 mt-2 py-2 border-0 small">
433 |             <i class="bi bi-box-arrow-right"></i> Выйти из аккаунт
| а
434 |         </button>
435 |     </div>
436 | </div>
437 |
438 | <!-- 3. REMOTE CONTROL MAIN PANEL -->
439 | <div class="rc-container d-none" id="rc-app-container">
440 |     <div class="card-rc">
441 |         <!-- Status Badge & User info -->
442 |         <div class="d-flex justify-content-between align-items-c
| enter mb-3">
443 |             <span class="badge bg-secondary" id="ws-status"><i cla
| ss="bi bi-wifi-off"></i> Подключение...</span>
444 |             <div class="d-flex gap-2 align-items-center">
445 |                 <button class="theme-toggle-btn" id="theme-toggle-bt
| n" title="Переключить тему">

```

```
446         <i class="bi bi-brightness-high"></i>
447     </button>
448     <button class="btn btn-sm btn-outline-secondary py-1
    px-2 border-secondary small" id="btn-change-player" title="См
    енить плеер">
449         <i class="bi bi-arrow-left-right"></i> <span id="r
    oom-info">Комната: default</span>
450     </button>
451 </div>
452 </div>
453
454 <!-- Now Playing Info -->
455 <div class="text-center my-3 p-3 rounded bg-dark bg-opac
    ity-25">
456     <i class="bi bi-music-note-beamed fs-1 text-primary mb
    -3 d-block"></i>
457     <div id="track-title" class="track-title mb-1 text-tru
    ncate">Файл не выбран</div>
458     <small id="time-display" class="text-muted">00:00 / 00
    :00</small>
459
460     <!-- Progress Bar -->
461     <div class="mt-3">
462         <input type="range" class="form-range" id="seek-slid
    er" min="0" max="100" value="0">
463     </div>
464 </div>
465
466 <!-- Central Controls -->
467 <div class="d-flex justify-content-center align-items-ce
    nter gap-3 my-4">
468     <button class="btn btn-outline-light btn-circle-lg" id
    ="btn-prev" title="Предыдущий"><i class="bi bi-skip-start-fill
    fs-3"></i></button>
469     <button class="btn btn-primary btn-circle-xl" id="btn-
    playtoggle" title="Воспроизведение/Пауза"><i class="bi bi-play
    -fill fs-1" id="play-icon"></i></button>
470     <button class="btn btn-outline-light btn-circle-lg" id
```

```

    ="btn-next" title="Следующий"><i class="bi bi-skip-end-fill fs
    -3"></i></button>
471     </div>
472
473     <!-- Carousel / Random Movie Control -->
474     <div class="my-3 px-2">
475         <button class="btn btn-carousel w-100 py-2.5 rounded-p
    ill d-flex align-items-center justify-content-center gap-2" id
    ="btn-carousel" title="Случайный фильм">
476             <i class="bi bi-arrow-repeat fs-5"></i>
477             <span>Карусель</span>
478         </button>
479     </div>
480
481     <!-- Volume Controls -->
482     <div class="d-flex align-items-center gap-3 my-3 px-2">
483         <i class="bi bi-volume-down-fill text-muted fs-4"></i>
484         <input type="range" class="form-range" id="volume-slid
    er" min="0" max="1" step="0.05" value="1">
485         <i class="bi bi-volume-up-fill text-muted fs-4"></i>
486     </div>
487
488     <!-- Voice Search & AI Chat -->
489     <div class="mt-4 p-3 rounded border border-secondary bg-
    dark bg-opacity-25">
490         <h6 class="text-center mb-3"><i class="bi bi-mic"></i>
    Голосовой Ассистент</h6>
491
492         <!-- Status -->
493         <div class="text-muted small text-center mb-3" id="voi
    ce-status">Готов к работе</div>
494
495         <!-- Control buttons -->
496         <div class="d-flex flex-column gap-3 align-items-cen
    te
    r">
497             <button class="btn btn-outline-danger btn-circle-xl"
    id="btn-mic" title="Нажмите, чтобы говорить">
498                 <i class="bi bi-mic-fill fs-1" id="mic-icon"></i>

```



```

499         </button>
500
501         <!-- Language Selection -->
502         <div class="btn-group w-100 mt-2" role="group">
503             <input type="radio" class="btn-check" name="lang-r
504             adio" id="lang-ru" value="ru-RU" autocomplete="off" checked>
505             <label class="btn btn-outline-secondary" for="lang
506             -ru">RU</label>
507
508             <input type="radio" class="btn-check" name="lang-r
509             adio" id="lang-en" value="en-US" autocomplete="off">
510             <label class="btn btn-outline-secondary" for="lang
511             -en">EN</label>
512
513             <input type="radio" class="btn-check" name="lang-r
514             adio" id="lang-he" value="he-IL" autocomplete="off">
515             <label class="btn btn-outline-secondary" for="lang
516             -he">HE</label>
517         </div>
518     </div>
519 </div>
520
521 <!-- Playlist Picker -->
522 <div class="mt-4">
523     <h6 class="border-bottom border-secondary pb-2"><i cla
524     ss="bi bi-collection-play"></i> Список воспроизведения плеера<
525     /h6>
526
527     <div class="list-group list-group-flush playlist-box"
528     id="playlist-items">
529         <div class="text-muted small p-3 text-center">Ожидан
530         ие подключения к плееру...</div>
531     </div>
532 </div>
533
534 <!-- User Manual (Read & Listen) -->
535 <div class="mt-4 p-3 rounded border border-secondary bg-
536 dark bg-opacity-25">
537     <div class="d-flex justify-content-between align-items

```

```

| -center border-bottom border-secondary pb-2 mb-2">
526 |         <h6 class="m-0"><i class="bi bi-info-circle"></i> Ин
|         струкция пользователя</h6>
527 |         <button class="btn btn-outline-info btn-sm d-flex al
|         ign-items-center gap-1" id="btn-read-manual" title="Прослушать
|         инструкцию">
528 |             <i class="bi bi-volume-up-fill" id="read-manual-ic
|         on"></i> <span>Прослушать</span>
529 |         </button>
530 |     </div>
531 |     <div id="user-manual-text" class="small text-muted" st
|         yle="line-height: 1.5; text-align: justify;">
532 |         Для дистанционного управления сначала откройте Медиа
|         плеер на вашем компьютере или телевизоре.
533 |         Данный пульт автоматически подключится к нему через
|         сервер.
534 |         Вы можете нажимать кнопки для запуска файлов, переключе
|         ния треков, перемотки или изменения громкости.
535 |         Также доступно голосовое управление кнопкой микрофон
|         а.
536 |         Простые команды: «пауза», «плей», «громче», «тише»,
|         «вперед» и «назад» сработают мгновенно.
537 |         А любые другие фразы будут отправлены вашему AI-асси
|         стенту Gemini для помощи в выборе фильма или ответа на вопросы
|         .
538 |     </div>
539 | </div>
540 | </div>
541 | </div>
542 |
543 | <script>
544 |     // API Module
545 |     const api = {
546 |         async fetch(url, options = {}) {
547 |             const response = await fetch(url, options);
548 |             if (!response.ok) {
549 |                 let msg = response.statusText;
550 |                 try {

```

```
551         const data = await response.json();
552         msg = data.detail || msg;
553     } catch {}
554     throw new Error(msg);
555 }
556 return response.json();
557 },
558 auth: {
559     async check() {
560         try {
561             const response = await fetch('/auth/check');
562             return await response.json();
563         } catch {
564             return { authenticated: false };
565         }
566     },
567     async register(email, password, name) {
568         return api.fetch('/auth/register', {
569             method: 'POST',
570             headers: { 'Content-Type': 'application/json' },
571             body: JSON.stringify({ email, password, name })
572         });
573     },
574     async verify(email, code) {
575         return api.fetch('/auth/verify', {
576             method: 'POST',
577             headers: { 'Content-Type': 'application/json' },
578             body: JSON.stringify({ email, code })
579         });
580     },
581     async loginEmail(email, password) {
582         return api.fetch('/auth/login/email', {
583             method: 'POST',
584             headers: { 'Content-Type': 'application/json' },
585             body: JSON.stringify({ email, password })
586         });
587     },
588     async logout() {
```

```

589         return api.fetch('/auth/logout', { method: 'POST' })
590     };
591     }
592 };
593
594 // Panels state
595 const authPanel = document.getElementById('auth-panel');
596 const scanPanel = document.getElementById('scan-panel');
597 const rcAppContainer = document.getElementById('rc-app-con
tainer');
598
599 const authFormLogin = document.getElementById('auth-form-l
ogin');
600 const authFormRegister = document.getElementById('auth-for
m-register');
601 const authFormVerify = document.getElementById('auth-form-
verify');
602 const authAlert = document.getElementById('auth-alert');
603
604 let currentVerifyEmail = '';
605 let currentRoomName = '';
606 let ws = null;
607 let isSeeking = false;
608 let playerState = { duration: 0, currentTime: 0, isPlaying
: false, volume: 1 };
609
610 // Toggle panels helpers
611 function showPanel(panelId) {
612     authPanel.classList.add('d-none');
613     scanPanel.classList.add('d-none');
614     rcAppContainer.classList.add('d-none');
615
616     if (panelId === 'auth') authPanel.classList.remove('d-no
ne');
617     else if (panelId === 'scan') scanPanel.classList.remove(
'd-none');
618     else if (panelId === 'rc') rcAppContainer.classList.remo

```

```

        ve('d-none');
619     }
620
621     // — AUTH PANEL LOGIC —————
    _____
622     document.getElementById('go-to-register').addEventListener
    ('click', (e) => {
623         e.preventDefault();
624         authAlert.classList.add('d-none');
625         authFormLogin.classList.add('d-none');
626         authFormRegister.classList.remove('d-none');
627     });
628
629     document.getElementById('go-to-login').addEventListener('c
    lick', (e) => {
630         e.preventDefault();
631         authAlert.classList.add('d-none');
632         authFormRegister.classList.add('d-none');
633         authFormLogin.classList.remove('d-none');
634     });
635
636     document.getElementById('verify-cancel').addEventListener(
    'click', (e) => {
637         e.preventDefault();
638         authAlert.classList.add('d-none');
639         authFormVerify.classList.add('d-none');
640         authFormLogin.classList.remove('d-none');
641     });
642
643     document.getElementById('google-signin-btn').addEventListener
    ner('click', () => {
644         window.location.href = '/auth/google?next=' + encodeURIC
    omponent(window.location.pathname + window.location.search);
645     });
646
647     // Login Form Submit
648     document.getElementById('form-login').addEventListener('su
    bmit', async (e) => {

```

```
649     e.preventDefault();
650     authAlert.classList.add('d-none');
651     const email = document.getElementById('login-email').value;
652     const pass = document.getElementById('login-password').value;
653
654     try {
655         await api.auth.loginEmail(email, pass);
656         checkAndInit();
657     } catch (err) {
658         authAlert.textContent = err.message || 'Ошибка входа';
659         authAlert.classList.remove('d-none');
660     }
661 });
662
663 // Register Form Submit
664 document.getElementById('form-register').addEventListener(
665     'submit', async (e) => {
666         e.preventDefault();
667         authAlert.classList.add('d-none');
668         const name = document.getElementById('register-name').value;
669         const email = document.getElementById('register-email').value;
670         const pass = document.getElementById('register-password').value;
671
672         try {
673             const res = await api.auth.register(email, pass, name);
674
675             currentVerifyEmail = email;
676             authFormRegister.classList.add('d-none');
677             authFormVerify.classList.remove('d-none');
678         } catch (err) {
679             authAlert.textContent = err.message || 'Ошибка регистрации';
680             authAlert.classList.remove('d-none');
```

```

679     }
680   });
681
682   // Verify Form Submit
683   document.getElementById('form-verify').addEventListener('submit', async (e) => {
684     e.preventDefault();
685     authAlert.classList.add('d-none');
686     const code = document.getElementById('verify-code').value;
687
688     try {
689       await api.auth.verify(currentVerifyEmail, code);
690       // After verify, try to log in
691       authFormVerify.classList.add('d-none');
692       authFormLogin.classList.remove('d-none');
693       authAlert.className = 'alert alert-success';
694       authAlert.textContent = 'Почта успешно подтверждена. Теперь войдите.';
695       authAlert.classList.remove('d-none');
696     } catch (err) {
697       authAlert.textContent = err.message || 'Неверный код';
698       authAlert.classList.remove('d-none');
699     }
700   });
701
702   // — PLAYER DISCOVERY LOGIC —————
703   async function scanForPlayers() {
704     showPanel('scan');
705     document.getElementById('scan-title').textContent = 'Поиск активных плееров';
706     document.getElementById('scan-status').textContent = 'Сканируем сеть на наличие запущенных устройств...';
707     document.getElementById('btn-retry-scan').classList.add('d-none');
708
709     const listDiv = document.getElementById('discovered-play

```

```

    ers-list');
710     listDiv.innerHTML = '<div class="text-center p-3"><span
    class="spinner-border spinner-border-sm text-primary"></span><
    /div>';
711
712     try {
713         const response = await fetch('/api/control/active_play
    ers');
714         const data = await response.json();
715         const players = data.players || [];
716         const currentUserEmail = data.user_email;
717
718         const mappedPlayers = players.map(p => {
719             if (p.includes('@')) {
720                 return { room: p, name: 'kino.davidka.net.tv' };
721             }
722             return { room: p, name: p === 'default' ? 'Основное
    устройство (default)' : p };
723         });
724
725         if (mappedPlayers.length === 0) {
726             listDiv.innerHTML = `
727                 <div class="alert alert-warning text-center small
    mb-0">
728                     <i class="bi bi-exclamation-circle-fill"></i> Ак
    тивные плееры не обнаружены.<br>
729                     Пожалуйста, откройте Медиаплеер на вашем ТВ или
    компьютере.
730                 </div>`;
731             document.getElementById('scan-title').textContent =
    'Плееры не найдены';
732             document.getElementById('btn-retry-scan').classList.
    remove('d-none');
733         } else if (mappedPlayers.length === 1) {
734             // Exactly one player found - connect automatically
735             listDiv.innerHTML = `<div class="alert alert-success
    text-center small">Обнаружен плеер "${mappedPlayers[0].name}"
    . Подключаемся...</div>`;

```



```

736         setTimeout(() => {
737             initStream(mappedPlayers[0].room);
738         }, 800);
739     } else {
740         // Multiple players found - ask user to select one
741         document.getElementById('scan-title').textContent =
742         'Куда стримить?';
743         document.getElementById('scan-status').textContent =
744         'Обнаружено несколько плееров. Выберите один: ';
745
746         listDiv.innerHTML = mappedPlayers.map(p => `
747             <div class="player-list-item">
748                 <div>
749                     <i class="bi bi-tv-fill text-primary me-2"></i>
750                     <strong>${p.name}</strong>
751                     </div>
752                     <button class="btn btn-sm btn-primary rounded-pi
753                     ll px-3" onclick="selectPlayer('${p.room}')">Выбрать</button>
754                     </div>
755                 `).join('');
756         document.getElementById('btn-retry-scan').classList.
757         remove('d-none');
758     }
759 } catch (err) {
760     listDiv.innerHTML = `<div class="alert alert-danger te
761     xt-center small">Ошибка сканирования: ${err.message}</div>`;
762     document.getElementById('btn-retry-scan').classList.re
763     move('d-none');
764 }
765
766 window.selectPlayer = function(room) {
767     initStream(room);
768 };
769
770 document.getElementById('btn-retry-scan').addEventListener
771 ('click', scanForPlayers);

```

```

766
767     document.getElementById('btn-change-player').addEventListener('click', () => {
768         // Disconnect current stream
769         if (ws) {
770             ws.close();
771             ws = null;
772         }
773         scanForPlayers();
774     });
775
776     document.getElementById('btn-scan-logout').addEventListener('click', async () => {
777         await api.auth.logout();
778         showPanel('auth');
779     });
780
781     // — MAIN TELEGRAM / WEB APP ROOM SETUP —————
782
783     function getRoomNameFromQuery() {
784         const urlParams = new URLSearchParams(window.location.search);
785         return urlParams.get('room') || '';
786     }
787
788     // Initialize Telegram WebApp if available
789     if (window.Telegram && window.Telegram.WebApp) {
790         const tg = window.Telegram.WebApp;
791         tg.ready();
792         tg.expand();
793
794         // Apply theme
795         function applyTgTheme() {
796             if (tg.themeParams.bg_color) {
797                 document.documentElement.style.setProperty('--tg-theme-bg-color', tg.themeParams.bg_color);
798             }
799             if (tg.themeParams.text_color) {

```

```
799 |         document.documentElement.style.setProperty('--tg-theme-text-color', tg.themeParams.text_color);
800 |     }
801 |     if (tg.themeParams.secondary_bg_color) {
802 |         document.documentElement.style.setProperty('--tg-theme-secondary-bg-color', tg.themeParams.secondary_bg_color);
803 |     }
804 |     if (tg.themeParams.hint_color) {
805 |         document.documentElement.style.setProperty('--tg-theme-hint-color', tg.themeParams.hint_color);
806 |     }
807 | }
808 | applyTgTheme();
809 | tg.onEvent('themeChanged', applyTgTheme);
810 |
811 | // Haptic feedback function
812 | window.triggerTgHaptic = function() {
813 |     if (tg.HapticFeedback) {
814 |         tg.HapticFeedback.impactOccurred('light');
815 |     }
816 | };
817 | } else {
818 |     window.triggerTgHaptic = function() {};
819 | }
820 |
821 | const statusBadge = document.getElementById('ws-status');
822 | const trackTitle = document.getElementById('track-title');
823 | const timeDisplay = document.getElementById('time-display');
824 | );
825 | const seekSlider = document.getElementById('seek-slider');
826 | const volumeSlider = document.getElementById('volume-slider');
827 | const playIcon = document.getElementById('play-icon');
828 | const playlistContainer = document.getElementById('playlist-items');
829 |
830 | function formatTime(seconds) {
831 |     if (!seconds || isNaN(seconds)) return '00:00';
```

```

831         const m = Math.floor(seconds / 60);
832         const s = Math.floor(seconds % 60);
833         return `${m.toString().padStart(2, '0')}:${s.toString().
padStart(2, '0')}`;
834     }
835
836     // Connect to WebSocket with Selected Room (Strictly One S
tream)
837     function initStream(room) {
838         currentRoomName = room;
839         showPanel('rc');
840
841         const roomInfoElement = document.getElementById('room-in
fo');
842         if (roomInfoElement) {
843             const displayRoom = room.includes('@') ? 'kino.davidka
.net.tv' : room;
844             roomInfoElement.textContent = `Плеер: ${displayRoom}`;
845         }
846
847         // Close previous connection to satisfy "Strictly One St
ream" requirement
848         if (ws) {
849             ws.close();
850         }
851
852         const protocol = window.location.protocol === 'https:' ?
'wss:' : 'ws:';
853         const wsUrl = `${protocol}//${window.location.host}/api/
control/ws?role=remote&room=${encodeURIComponent(currentRoomNa
me)}`;
854
855         ws = new WebSocket(wsUrl);
856
857         ws.onopen = () => {
858             statusBadge.className = 'badge bg-success';
859             statusBadge.innerHTML = '<i class="bi bi-wifi"></i> По
дключено';

```

```

860     };
861
862     ws.onmessage = (event) => {
863         try {
864             const msg = JSON.parse(event.data);
865             if (msg.event === 'status_update') {
866                 updateUI(msg);
867             } else if (msg.event === 'playlist_update') {
868                 updatePlaylist(msg.files || []);
869             }
870         } catch (e) {
871             console.error(e);
872         }
873     };
874
875     ws.onclose = (e) => {
876         // Stop speech queue when connection drops
877         if (currentSpeechQueue) {
878             try {
879                 currentSpeechQueue.stop();
880                 currentSpeechQueue = null;
881             } catch (err) {}
882         }
883         if (currentSpeechAudio) {
884             try {
885                 currentSpeechAudio.pause();
886                 currentSpeechAudio.src = "";
887                 currentSpeechAudio = null;
888             } catch (err) {}
889         }
890
891         // If we deliberately closed it to switch player, don't
892         // auto-reconnect
893         if (currentRoomName !== room) return;
894
895         statusBadge.className = 'badge bg-warning text-dark';
896         statusBadge.innerHTML = '<i class="bi bi-arrow-clockwi
se"></i> Переподключение...';

```

```
896 |         setTimeout(() => {
897 |             if (currentRoomName === room) {
898 |                 initStream(room);
899 |             }
900 |         }, 3000);
901 |     };
902 | }
903 |
904 | function sendCommand(action, payload = {}) {
905 |     window.triggerTgHaptic();
906 |     if (ws && ws.readyState === WebSocket.OPEN) {
907 |         ws.send(JSON.stringify({ action, ...payload }));
908 |     }
909 | }
910 |
911 | function updateUI(data) {
912 |     playerState = { ...playerState, ...data };
913 |     trackTitle.textContent = data.fileName || 'Файл не выбра
914 | н';
915 |     timeDisplay.textContent = `${formatTime(data.currentTime
916 | )} / ${formatTime(data.duration)}`;
917 |
918 |     if (!isSeeking && data.duration > 0) {
919 |         seekSlider.max = Math.floor(data.duration);
920 |         seekSlider.value = Math.floor(data.currentTime);
921 |     }
922 |
923 |     if (typeof data.volume === 'number') {
924 |         volumeSlider.value = data.volume;
925 |     }
926 |
927 |     if (data.isPlaying) {
928 |         playIcon.className = 'bi bi-pause-fill fs-1';
929 |     } else {
930 |         playIcon.className = 'bi bi-play-fill fs-1';
931 |     }
```

```

932     function updatePlaylist(files) {
933         window.playlistFiles = files;
934         if (!files.length) {
935             playlistContainer.innerHTML = '<div class="text-muted
small p-3 text-center">Плейлист пуст</div>';
936             return;
937         }
938
939         playlistContainer.innerHTML = files.map((f, i) => `
940             <button class="list-group-item list-group-item-action
bg-transparent text-white border-secondary py-2 px-3 small d-f
lex justify-content-between align-items-center" onclick="playF
ile(${i})">
941                 <span class="text-truncate">${i + 1}. ${f.name}</spa
n>
942                 <span class="badge bg-secondary ms-2">${f.type === '
series' ? 'Сериал' : 'Фильм'}</span>
943             </button>
944         `).join('');
945     }
946
947     window.playFile = (index) => {
948         sendCommand('play_file', { index });
949     };
950
951     // — VOICE CONTROL & SEARCH CHAT MODULE —————

```

```

952     let recognition = null;
953     let isListening = false;
954     let rcAbortController = null;
955     let rcHistory = [];
956
957
958
959     function tryParseMovieCard(text) {
960         const trimmed = text.trim();
961         if (trimmed.startsWith('{') && trimmed.endsWith('}')) {
962             try {

```

```

963         const data = JSON.parse(trimmed);
964         if (data.title || data.title_ru) return data;
965     } catch (e) {}
966 }
967 const match = text.match(/\{[\s\S]*?\}/);
968 if (match) {
969     try {
970         const data = JSON.parse(match[0]);
971         if (data.title || data.title_ru) return data;
972     } catch (e) {}
973 }
974 return null;
975 }
976
977 function renderMovieCardHtml(card) {
978     const title = card.title_ru || card.title;
979     const subtitle = card.title_orig || '';
980     const year = card.year || '';
981     const category = card.main_category || '';
982     const genres = card.genres ? card.genres.join(', ') : ''
983     ;
984     const directors = card.directors ? card.directors.join(
985     ', ') : '';
986     const cast = card.cast ? card.cast.slice(0, 5).join(', '
987     ) : '';
988     const plot = card.plot || '';
989     const whyWatch = card.why_watch || '';
990
991     let ratingHtml = '';
992     if (card.rating) {
993         if (card.rating.imdb) ratingHtml += `<span class="badg
994 e bg-warning text-dark me-1">IMDb: ${card.rating.imdb}</span>`
995         ;
996         if (card.rating.кинопоиск) ratingHtml += `<span class=
997 "badge bg-danger text-white me-1">КП: ${card.rating.кинопоиск}
998 </span>`;
999         if (card.rating.tmbd) ratingHtml += `<span class="badg
1000 e bg-primary text-white">TMDB: ${card.rating.tmbd}</span>`;

```



```

993     }
994
995     return `
996         <div class="p-3 text-white" style="border-radius: 10px
; background: rgba(88, 166, 255, 0.08); border: 1px solid rgba
(88, 166, 255, 0.2);">
997             <h5 class="mb-1 text-info">${title}</h5>
998             <div class="small text-muted mb-2">${subtitle} ${yea
r ? `(${year})` : ''}</div>
999             <div class="mb-2">${ratingHtml}</div>
1000             ${category ? `<div class="small text-muted mb-1" sty
le="font-size: 0.8rem;"><strong>Категория:</strong> ${category
}</div>` : ''}
1001             ${genres ? `<div class="small text-muted mb-1" style
="font-size: 0.8rem;"><strong>Жанры:</strong> ${genres}</div>`
: ''}
1002             ${directors ? `<div class="small text-muted mb-1" st
yle="font-size: 0.8rem;"><strong>Режиссер:</strong> ${director
s}</div>` : ''}
1003             ${cast ? `<div class="small text-muted mb-2" style="
font-size: 0.8rem;"><strong>В полях:</strong> ${cast}...</div>
` : ''}
1004             ${plot ? `<p class="small text-secondary mb-2" style
="font-size: 0.85rem; line-height: 1.4;">${plot}</p>` : ''}
1005             ${whyWatch ? `<div class="small text-info mb-2" styl
e="font-size: 0.8rem;"><strong>Рекомендация:</strong> ${whyWat
ch}</div>` : ''}
1006             <div class="d-flex justify-content-end mt-2">
1007                 <button class="btn btn-sm btn-primary rounded-pill
px-3" onclick="playMovieDirect('${title.replace(/'/g, '\\')}
')">
1008                     <i class="bi bi-play-fill"></i> Запустить фильм
1009                 </button>
1010             </div>
1011         </div>
1012     `;
1013 }
1014

```

```
1015     window.playMovieDirect = (title) => {
1016         if (!window.playlistFiles) return;
1017         const cleanT = title.toLowerCase();
1018         const foundIdx = window.playlistFiles.findIndex(f => (f.
name || '').toLowerCase().includes(cleanT));
1019         if (foundIdx !== -1) {
1020             playFile(foundIdx);
1021             speakText(`Запускаю ${window.playlistFiles[foundIdx].n
ame}`);
1022         } else {
1023             speakText(`Фильм не найден в плейлисте`);
1024         }
1025     };
1026
1027     async function processChatMessage(text) {
1028         if (!text || !text.trim()) return;
1029         const query = text.trim();
1030
1031         stopAudio(); // <--- STOP AUDIO IMMEDIATELY
1032
1033         // 1. Check local control commands first
1034         if (handleLocalVoiceCommand(query.toLowerCase())) {
1035             speakText('Команда выполнена');
1036             return;
1037         }
1038
1039         // 2. Perform search in playlist
1040         const cleanQ = query.toLowerCase().replace(/^(найди|поис
к|включи|запусти|покажи|фильм|сериал)\s+/i, '').trim();
1041         let matches = [];
1042         if (window.playlistFiles && cleanQ.length > 1) {
1043             matches = window.playlistFiles.map((f, idx) => ({ ...f
, index: idx })).filter(f => {
1044                 const name = (f.name || '').toLowerCase();
1045                 const type = (f.type || '').toLowerCase();
1046                 return name.includes(cleanQ) || (f.year && f.year.to
String().includes(cleanQ));
1047             });
```

```
1048 |     }
1049 |
1050 |     if (matches.length > 0) {
1051 |         speakText(`Найдено совпадений: ${matches.length}`);
1052 |
1053 |         const queryLower = query.toLowerCase();
1054 |         if (matches.length === 1 && (queryLower.includes('включи') || queryLower.includes('запусти') || queryLower.includes('плей') || queryLower.includes('играй')))) {
1055 |             playFile(matches[0].index);
1056 |             speakText(`Запускаю ${matches[0].name}`);
1057 |         }
1058 |         return;
1059 |     }
1060 |
1061 |     // 3. Forward to AI Chat API with Streaming
1062 |     if (rcAbortController) {
1063 |         try { rcAbortController.abort(); } catch(e) {}
1064 |     }
1065 |     rcAbortController = new AbortController();
1066 |
1067 |     const voiceStatus = document.getElementById('voice-status');
1068 |     voiceStatus.textContent = 'Обработка запроса моделью...';
1069 |     ;
1070 |
1071 |     let fullReply = '';
1072 |     let started = false;
1073 |
1074 |     try {
1075 |         const payload = {
1076 |             message: query,
1077 |             history: rcHistory,
1078 |             generation_config: { is_rc: true }
1079 |         };
1080 |
1081 |         const response = await fetch('/api/chat', {
1082 |             method: 'POST',
```

```
1082     headers: { 'Content-Type': 'application/json' },
1083     body: JSON.stringify(payload),
1084     signal: rcAbortController.signal
1085   });
1086
1087   if (!response.ok) {
1088     throw new Error(`Ошибка HTTP ${response.status}`);
1089   }
1090
1091   const reader = response.body.getReader();
1092   const decoder = new TextDecoder('utf-8');
1093   let buffer = '';
1094   let voiceReply = '';
1095
1096   while (true) {
1097     const { value, done } = await reader.read();
1098     if (done) break;
1099
1100     buffer += decoder.decode(value, { stream: true });
1101     const lines = buffer.split('\n');
1102     buffer = lines.pop();
1103
1104     for (const line of lines) {
1105       const trimmed = line.trim();
1106       if (!trimmed) continue;
1107       if (trimmed.startsWith('data: ')) {
1108         try {
1109           const data = JSON.parse(trimmed.substring(6));
1110           if (data.error) {
1111             throw new Error(data.error);
1112           }
1113           if (data.status) {
1114             voiceStatus.textContent = data.status;
1115           }
1116           if (data.voice) {
1117             voiceReply += data.voice;
1118           }
1119           if (data.text) {
```

```

1120         fullReply += data.text;
1121     }
1122     } catch (e) {
1123         console.error(e);
1124     }
1125 }
1126 }
1127 }
1128
1129 voiceStatus.textContent = 'Готов к работе';
1130
1131 // — Обработка тегов <film> из ответа AI —————
1132
1133     const filmMatches = [...fullReply.matchAll(/<film>(.*?
1134 )<\s*</film>/gi)];
1135     if (filmMatches.length > 0) {
1136         const filmTitle = filmMatches[0][1].trim();
1137         try {
1138             const mediaResp = await fetch('/api/media/by-title
1139 ', {
1140                 method: 'POST',
1141                 headers: { 'Content-Type': 'application/json' },
1142                 body: JSON.stringify({ title: filmTitle })
1143             });
1144             if (mediaResp.ok) {
1145                 const mediaData = await mediaResp.json();
1146                 const filmPath = mediaData.path;
1147                 if (filmPath) {
1148                     sendCommand('play_file_by_path', { path: filmP
1149 ath });
1150                 voiceStatus.textContent = `▶ Запускаю: ${film
1151 Title}`;
1152             }
1153         } catch (e) {
1154             console.error('Ошибка поиска пути фильма:', e);
1155         }
1156     }

```

```
1153 | // _____
1154 |
1155 |     const card = tryParseMovieCard(fullReply);
1156 |     if (card) {
1157 |         speakText(`Найдена карточка фильма ${card.title_ru |
1158 | | card.title}. ${card.why_watch || card.plot || ''}`);
1159 |     } else {
1160 |         speakText(voiceReply || fullReply);
1161 |     }
1162 |
1163 |     rcHistory.push({ role: 'user', parts: [query] });
1164 |     rcHistory.push({ role: 'model', parts: [fullReply] });
1165 |     if (rcHistory.length > 20) {
1166 |         rcHistory = rcHistory.slice(-20);
1167 |     }
1168 |
1169 |     } catch (err) {
1170 |         if (err.name === 'AbortError') return;
1171 |         voiceStatus.textContent = 'Ошибка AI: ' + err.message;
1172 |     }
1173 |
1174 |     function initVoiceControl() {
1175 |         const btnMic = document.getElementById('btn-mic');
1176 |         const micIcon = document.getElementById('mic-icon');
1177 |         const voiceStatus = document.getElementById('voice-status');
1178 |
1179 |         const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;
1180 |         if (!SpeechRecognition) {
1181 |             voiceStatus.textContent = 'Голосовой ввод не поддерживается вашим браузером.';
1182 |             if (btnMic) btnMic.disabled = true;
1183 |         } else {
1184 |             recognition = new SpeechRecognition();
1185 |
```

```

1186 |         // Dynamically get selected language
1187 |         const getSelectedLang = () => {
1188 |             const selected = document.querySelector('input[name=
| "lang-radio"]:checked');
1189 |             return selected ? selected.value : 'ru-RU';
1190 |         };
1191 |
1192 |         recognition.lang = getSelectedLang();
1193 |
1194 |         // Update language if radio button changes
1195 |         document.querySelectorAll('input[name="lang-radio"]').
| forEach(radio => {
1196 |             radio.addEventListener('change', () => {
1197 |                 recognition.lang = getSelectedLang();
1198 |             });
1199 |         });
1200 |
1201 |         recognition.continuous = false;
1202 |         recognition.interimResults = false;
1203 |
1204 |         recognition.onstart = () => {
1205 |             isListening = true;
1206 |             if (btnMic) btnMic.className = 'btn btn-danger btn-c
| ircle-lg flex-shrink-0';
1207 |             if (micIcon) micIcon.className = 'bi bi-mic-fill fs-
| 3 animate-pulse';
1208 |             voiceStatus.textContent = 'Слушаю...';
1209 |         };
1210 |
1211 |         recognition.onend = () => {
1212 |             isListening = false;
1213 |             if (btnMic) btnMic.className = 'btn btn-outline-dang
| er btn-circle-lg flex-shrink-0';
1214 |             if (micIcon) micIcon.className = 'bi bi-mic-fill fs-
| 3';
1215 |             voiceStatus.textContent = 'Готов к работе';
1216 |         };
1217 |

```

```
1218 |         recognition.onerror = (e) => {
1219 |             voiceStatus.textContent = `Ошибка ввода: ${e.error}`
1220 |         };
1221 |
1222 |         recognition.onresult = async (event) => {
1223 |             const text = event.results[0][0].transcript;
1224 |             if (text) {
1225 |                 processChatMessage(text);
1226 |             }
1227 |         };
1228 |
1229 |         if (btnMic) {
1230 |             btnMic.addEventListener('click', () => {
1231 |                 window.triggerTgHaptic();
1232 |                 // Unlock SpeechSynthesis for mobile devices
1233 |                 if ('speechSynthesis' in window) {
1234 |                     const silentUtterance = new SpeechSynthesisUtter-
1235 |                         ance(' ');
1236 |                     silentUtterance.volume = 0;
1237 |                     window.speechSynthesis.speak(silentUtterance);
1238 |                 }
1239 |                 if (isListening) {
1240 |                     recognition.stop();
1241 |                 } else {
1242 |                     recognition.start();
1243 |                 }
1244 |             });
1245 |         }
1246 |     }
1247 |
1248 |     let currentSpeechAudio = null;
1249 |     let currentSpeechQueue = null;
1250 |
1251 |     class AudioQueuePlayer {
1252 |         constructor(chunks, userSettings) {
1253 |             this.chunks = chunks;
```



```
1254 |         this.userSettings = userSettings;
1255 |         this.currentIndex = 0;
1256 |         this.isPlaying = false;
1257 |         this.audioElements = new Map();
1258 |         this.activeAudio = null;
1259 |         this._prefetch(0);
1260 |         this._prefetch(1);
1261 |     }
1262 |
1263 |     _prefetch(index) {
1264 |         if (index >= this.chunks.length || this.audioElements.
has(index)) return;
1265 |         const text = this.chunks[index];
1266 |         const queryParams = new URLSearchParams({
1267 |             text: text,
1268 |             system: this.userSettings.tts_system,
1269 |             voice: this.userSettings.tts_voice
1270 |         });
1271 |         const audioUrl = `/api/tts/synthesize?${queryParams.to
String()}`;
1272 |         const audio = new Audio(audioUrl);
1273 |         audio.load();
1274 |         this.audioElements.set(index, audio);
1275 |     }
1276 |
1277 |     async play() {
1278 |         this.isPlaying = true;
1279 |         while (this.currentIndex < this.chunks.length && this.
isPlaying) {
1280 |             this._prefetch(this.currentIndex);
1281 |             this._prefetch(this.currentIndex + 1);
1282 |             this._prefetch(this.currentIndex + 2);
1283 |
1284 |             const audio = this.audioElements.get(this.currentInd
ex);
1285 |             if (!audio) {
1286 |                 this.currentIndex++;
1287 |                 continue;
```

```
1288         }
1289
1290         this.activeAudio = audio;
1291         currentSpeechAudio = audio;
1292
1293         const playPromise = new Promise((resolve) => {
1294             audio.onended = () => resolve();
1295             audio.onerror = (e) => {
1296                 console.error(`Audio error for chunk ${this.cu
rrrentIndex}:`, e);
1297                 this.stop(); // Останавливаем всю очередь при
ошибке загрузки
1298                 resolve();
1299             };
1300         });
1301
1302         try {
1303             await audio.play();
1304             await playPromise;
1305         } catch (err) {
1306             console.error("Audio playback interrupted/failed:"
, err);
1307         }
1308         this.currentIndex++;
1309     }
1310     this.isPlaying = false;
1311 }
1312
1313 stop() {
1314     this.isPlaying = false;
1315     if (this.activeAudio) {
1316         try {
1317             this.activeAudio.pause();
1318             this.activeAudio.src = "";
1319         } catch (e) {}
1320     }
1321     this.audioElements.clear();
1322 }
```

```
1323     }
1324
1325     function stopAudio() {
1326         if (currentSpeechQueue) {
1327             try {
1328                 currentSpeechQueue.stop();
1329                 currentSpeechQueue = null;
1330             } catch (e) {
1331                 console.error("Error stopping currentSpeechQueue:",
1332 e);
1333             }
1334         }
1335         if (currentSpeechAudio) {
1336             try {
1337                 currentSpeechAudio.pause();
1338                 currentSpeechAudio.src = "";
1339                 currentSpeechAudio = null;
1340             } catch (e) {
1341                 console.error("Error stopping currentSpeechAudio:",
1342 e);
1343             }
1344         }
1345         if ('speechSynthesis' in window) {
1346             try {
1347                 window.speechSynthesis.cancel();
1348             } catch (e) {
1349                 console.error("Error cancelling SpeechSynthesis:", e
1350 );
1351             }
1352         }
1353     }
1354
1355     async function speakText(text) {
1356         if (!text) return;
1357
1358         // Stop any currently playing audio
1359         stopAudio();
1360
1361         // Remove markdown or HTML tags if any from voice output
```

```

1358     const cleanText = text.replace(/<[^>]*>/g, '').replace(/
|[*_`#]/g, '').trim();
1359     if (!cleanText) return;
1360
1361     // Try backend TTS first
1362     try {
1363         const response = await fetch('/auth/settings');
1364         if (response.ok) {
1365             const userSettings = await response.json();
1366             if (userSettings.tts_enabled === 1 && userSettings.t
ts_system && userSettings.tts_system !== 'browser') {
1367                 const paragraphs = cleanText
1368                     .split(/\n+/)
1369                     .map(p => p.trim())
1370                     .filter(p => p.length > 0);
1371
1372                 let chunks = [];
1373                 for (const para of paragraphs) {
1374                     const sentences = para.match(/^[^!?]+[.!?]*\s/g) |
| [para];
1375                     let currentChunk = "";
1376                     for (const sentence of sentences) {
1377                         const trimmedSentence = sentence.trim();
1378                         if (!trimmedSentence) continue;
1379                         if ((currentChunk + " " + trimmedSentence).len
gth > 200) {
1380                             chunks.push(currentChunk.trim());
1381                             currentChunk = trimmedSentence;
1382                         } else {
1383                             currentChunk = (currentChunk + " " + trimmed
Sentence).trim();
1384                         }
1385                     }
1386                     if (currentChunk) {
1387                         chunks.push(currentChunk.trim());
1388                     }
1389                 }
1390                 chunks = chunks.filter(c => c.length > 0);

```

```
1391 |
1392 |         if (chunks.length > 0) {
1393 |             currentSpeechQueue = new AudioQueuePlayer(chunks
1394 | , userSettings);
1395 |             await currentSpeechQueue.play();
1396 |         }
1397 |         return;
1398 |     }
1399 | } catch (e) {
1400 |     console.error('Failed to speak using backend TTS, fall
1401 | ing back to browser:', e);
1402 | }
1403 |
1404 | // Fallback to browser SpeechSynthesis
1405 | if ('speechSynthesis' in window) {
1406 |     window.currentUtterance = new SpeechSynthesisUtterance
1407 | (cleanText);
1408 |     window.currentUtterance.lang = 'ru-RU';
1409 |     window.speechSynthesis.speak(window.currentUtterance);
1410 | }
1411 | }
1412 |
1413 | function handleLocalVoiceCommand(text) {
1414 |     const t = text.toLowerCase();
1415 |     if (t.includes('пауз') || t.includes('стоп')) {
1416 |         sendCommand('pause');
1417 |         return true;
1418 |     }
1419 |     if (t.includes('продолж') || t.includes('плей') || t.inc
1420 | ludes('играй') || t.includes('старт')) {
1421 |         sendCommand('play');
1422 |         return true;
1423 |     }
1424 |     if (t.includes('громче') || t.includes('прибавь звук'))
1425 | {
1426 |         const targetVol = Math.min(1, playerState.volume + 0.2
1427 | );
1428 |     }
```

```

1423 |         sendCommand('set_volume', { level: targetVol });
1424 |         return true;
1425 |     }
1426 |     if (t.includes('тише') || t.includes('убавь звук')) {
1427 |         const targetVol = Math.max(0, playerState.volume - 0.2
1428 | );
1429 |         sendCommand('set_volume', { level: targetVol });
1430 |         return true;
1431 |     }
1432 |     if (t.includes('следующ') || t.includes('вперед')) {
1433 |         sendCommand('next');
1434 |         return true;
1435 |     }
1436 |     if (t.includes('предыдущ') || t.includes('назад')) {
1437 |         sendCommand('prev');
1438 |         return true;
1439 |     }
1440 |     if (t.includes('карусель') || t.includes('случайн') || t
1441 | .includes('рандом')) {
1442 |         sendCommand('play_random_movie');
1443 |         return true;
1444 |     }
1445 |     return false;
1446 | }
1447 |
1448 | async function sendToAiModel(message) {
1449 |     const response = await fetch('/api/chat', {
1450 |         method: 'POST',
1451 |         headers: { 'Content-Type': 'application/json' },
1452 |         body: JSON.stringify({ message, generation_config: { i
1453 | s_rc: true } })
1454 |     });
1455 |     if (!response.ok) {
1456 |         throw new Error('Не удалось получить ответ от AI');
1457 |     }
1458 |     const data = await response.json();
1459 |     return data.response || data.message || 'Модель не ответ
1460 | ила';

```

```

1457 |     }
1458 |
1459 |     // Set listeners
1460 |     document.getElementById('btn-playtoggle').addEventListener
1461 | ('click', () => sendCommand('toggle_play'));
1462 |     document.getElementById('btn-prev').addEventListener('click',
1463 | () => sendCommand('prev'));
1464 |     document.getElementById('btn-next').addEventListener('click',
1465 | () => sendCommand('next'));
1466 |     document.getElementById('btn-carousel').addEventListener('
1467 | click', () => sendCommand('play_random_movie'));
1468 |
1469 |     seekSlider.addEventListener('mousedown', () => { isSeeking
1470 | = true; });
1471 |     seekSlider.addEventListener('touchstart', () => { isSeekin
1472 | g = true; });
1473 |     seekSlider.addEventListener('change', (e) => {
1474 |         isSeeking = false;
1475 |         sendCommand('seek', { seconds: parseFloat(e.target.value
1476 | ) });
1477 |     });
1478 |
1479 |     volumeSlider.addEventListener('change', (e) => {
1480 |         sendCommand('set_volume', { level: parseFloat(e.target.v
1481 | alue) });
1482 |     });
1483 |
1484 |     // — USER MANUAL AUDIO READER —————
1485 |
1486 |     let isSpeakingManual = false;
1487 |     const btnRead = document.getElementById('btn-read-manual')
1488 | ;
1489 |     const readIcon = document.getElementById('read-manual-icon
1490 | ');
1491 |     const readTextSpan = btnRead ? btnRead.querySelector('span
1492 | ') : null;
1493 |
1494 |     if (btnRead) {

```

```
1483 |         btnRead.addEventListener('click', () => {
1484 |             window.triggerTgHaptic();
1485 |             const text = document.getElementById('user-manual-text
| ').innerText;
1486 |
1487 |             if (isSpeakingManual) {
1488 |                 window.speechSynthesis.cancel();
1489 |                 isSpeakingManual = false;
1490 |                 if (readIcon) readIcon.className = 'bi bi-volume-up-
| fill';
1491 |                 if (readTextSpan) readTextSpan.textContent = 'Прослу
| шать';
1492 |                 btnRead.className = 'btn btn-outline-info btn-sm d-f
| lex align-items-center gap-1';
1493 |             } else {
1494 |                 window.speechSynthesis.cancel();
1495 |                 const utterance = new SpeechSynthesisUtterance(text)
| ;
1496 |                 utterance.lang = 'ru-RU';
1497 |
1498 |                 utterance.onend = () => {
1499 |                     isSpeakingManual = false;
1500 |                     if (readIcon) readIcon.className = 'bi bi-volume-u
| p-fill';
1501 |                     if (readTextSpan) readTextSpan.textContent = 'Прос
| лушать';
1502 |                     btnRead.className = 'btn btn-outline-info btn-sm d
| -flex align-items-center gap-1';
1503 |                 };
1504 |
1505 |                 utterance.onerror = () => {
1506 |                     isSpeakingManual = false;
1507 |                     if (readIcon) readIcon.className = 'bi bi-volume-u
| p-fill';
1508 |                     if (readTextSpan) readTextSpan.textContent = 'Прос
| лушать';
1509 |                     btnRead.className = 'btn btn-outline-info btn-sm d
| -flex align-items-center gap-1';
```



```
1510         };
1511
1512         window.speechSynthesis.speak(utterance);
1513         isSpeakingManual = true;
1514         if (readIcon) readIcon.className = 'bi bi-stop-fill'
1515         ;
1516         if (readTextSpan) readTextSpan.textContent = 'Остано
1517         вить';
1518         btnRead.className = 'btn btn-info btn-sm text-dark d
1519         -flex align-items-center gap-1';
1520     }
1521     });
1522 }
1523
1524 // Theme toggle function
1525 function toggleTheme() {
1526     const body = document.body;
1527     const currentTheme = body.classList.contains('theme-ligh
1528     t') ? 'light' : 'dark';
1529     const newTheme = currentTheme === 'light' ? 'dark' : 'li
1530     ght';
1531
1532     if (newTheme === 'light') {
1533         body.classList.add('theme-light');
1534     } else {
1535         body.classList.remove('theme-light');
1536     }
1537 }
1538
1539 // Set theme toggle listener
1540 document.getElementById('theme-toggle-btn')?.addEventListener('click', toggleTheme);
1541
1542 // — AUTH CHECK INITIALIZER —————
1543
1544 async function checkAndInit() {
1545     const authState = await api.auth.check();
1546     if (authState.authenticated) {
```

```
1541 |         // If a room parameter is present in URL, connect dire
      | ctly to it
1542 |         const queryRoom = getRoomNameFromQuery();
1543 |         if (queryRoom) {
1544 |             initStream(queryRoom);
1545 |         } else {
1546 |             // Otherwise, scan for open players
1547 |             scanForPlayers();
1548 |         }
1549 |     } else {
1550 |         showPanel('auth');
1551 |     }
1552 | }
1553 |
1554 | // Run on startup
1555 | document.addEventListener('DOMContentLoaded', () => {
1556 |     checkAndInit();
1557 |     initVoiceControl();
1558 | });
1559 |
1560 | </script>
1561 | </body>
1562 | </html>
1563 |
```

index.html

```
1 <!-- Web Search MCP Providers Management Tab -->
2 <div class="container-fluid py-3">
3   <!-- Header -->
4   <div class="alert alert-info d-flex justify-content-between
5 align-items-center flex-wrap gap-2 mb-3 shadow-sm">
6     <div>
7       <h5 class="mb-1"><i class="bi bi-globe2 me-2"></i> Про
8 вайдеры веб-поиска (MCP)</h5>
9       <p class="mb-0 small text-secondary">Управление серверам
10 и интернет-поиска, алгоритмами извлечения и тестирование выдач
11 и.</p>
12     </div>
13     <div class="input-group input-group-sm" style="max-width:
14 440px;">
15       <span class="input-group-text fw-bold">Активный движок п
16 оиска:</span>
17       <select class="form-select" id="search-tab-engine-select
18 or">
19         <option value="playwright"> Playwright MCP</option>
20         <option value="langchain"> LangChain MCP</option>
21         <option value="gemini"> Google Gemini (Grounding)</op
22 tion>
23         <option value="gemini_cli"> Google Gemini CLI</optio
24 n>
25         <option value="agy"> AntigraVity (AGY)</option>
26       </select>
27       <button class="btn btn-primary" id="btn-save-search-tab-
28 engine">
29         <i class="bi bi-check-circle me-1"></i> Сохранить
30       </button>
31     </div>
32   </div>
33
34   <!-- Reference Info Block (Collapse, matching RAG tab style)
35   -->
36   <div class="card mb-4 border-0 shadow-sm bg-light">
```

```

26     <div class="card-body p-3">
27         <div class="d-flex align-items-center justify-content-be
| tween">
28             <div class="d-flex align-items-center gap-2">
29                 <i class="bi bi-info-circle-fill text-primary fs-5">
| </i>
30                 <span class="fw-bold">Справка: архитектура и принцип
| ы работы 5 серверов веб-поиска</span>
31             </div>
32             <button class="btn btn-sm btn-link text-decoration-non
| e p-0 text-secondary" type="button" data-bs-toggle="collapse"
| data-bs-target="#searchArchCollapse" aria-expanded="true" aria
| -controls="searchArchCollapse">
33                 <i class="bi bi-chevron-down"></i> Развернуть / Скры
| ть
34             </button>
35         </div>
36         <div class="collapse show" id="searchArchCollapse">
37             <div class="row g-3 mt-2 pt-2 border-top">
38                 <!-- 1. Playwright -->
39                 <div class="col-xl col-md-6">
40                     <div class="p-3 border rounded bg-white h-100 shad
| ow-sm">
41                         <div class="fw-bold text-primary mb-1">
42                             □ Playwright MCP
43                         </div>
44                         <div class="small text-muted mb-2">
45                             Прямой headless-браузер (Chromium).
46                         </div>
47                         <ul class="small text-secondary mb-0 ps-3">
48                             <li>Поиск в DuckDuckGo, Bing и Google.</li>
49                             <li>Глубокий сбор DOM-структуры и текста сайто
| в.</li>
50                             <li><strong>Плюс:</strong> не расходует токены
| ИИ на сбор данных.</li>
51                             <li><strong>Особенность:</strong> требует запу
| щенного Playwright.</li>
52                         </ul>

```

```

53 |         </div>
54 |     </div>
55 |
56 |     <!-- 2. LangChain -->
57 |     <div class="col-xl col-md-6">
58 |         <div class="p-3 border rounded bg-white h-100 shad
ow-sm">
59 |             <div class="fw-bold text-success mb-1">
60 |                 □ LangChain MCP
61 |             </div>
62 |             <div class="small text-muted mb-2">
63 |                 Автономный ReAct-агент Breadboard.
64 |             </div>
65 |             <ul class="small text-secondary mb-0 ps-3">
66 |                 <li>Инструменты: веб-поиск, RAG, Python-вычисл
ения.</li>
67 |                 <li>Агент сам принимает решение о вызове инстр
ументов.</li>
68 |                 <li><strong>Плюс:</strong> гибкое решение комп
лексных задач.</li>
69 |                 <li><strong>Особенность:</strong> многошаговый
reasoning-цикл.</li>
70 |             </ul>
71 |         </div>
72 |     </div>
73 |
74 |     <!-- 3. Gemini Grounding -->
75 |     <div class="col-xl col-md-6">
76 |         <div class="p-3 border rounded bg-white h-100 shad
ow-sm">
77 |             <div class="fw-bold text-info mb-1">
78 |                 □ Google Gemini Grounding
79 |             </div>
80 |             <div class="small text-muted mb-2">
81 |                 Официальный Google Search Grounding.
82 |             </div>
83 |             <ul class="small text-secondary mb-0 ps-3">
84 |                 <li>Прямой поиск в реальном индексе Google Sea

```

```

rch.</li>
85         <li>Автоматическая ротация пула ключей при оши
бках 429.</li>
86         <li><strong>Плюс:</strong> проверенная точност
ь фактов и цитаты.</li>
87         <li><strong>Особенность:</strong> требует вали
дных API-ключей.</li>
88     </ul>
89 </div>
90 </div>
91
92 <!-- 4. Gemini CLI -->
93 <div class="col-xl col-md-6">
94     <div class="p-3 border rounded bg-white h-100 shad
ow-sm">
95         <div class="fw-bold text-primary mb-1">
96             □ Google Gemini CLI
97         </div>
98         <div class="small text-muted mb-2">
99             Локальный терминальный агент Google.
100         </div>
101         <ul class="small text-secondary mb-0 ps-3">
102             <li>Прямое выполнение через CLI подпроцесс.</l
i>
103             <li>Встроенные агентные инструменты веб-поиска
.</li>
104             <li><strong>Плюс:</strong> автономный агент, м
одель по умолчанию flash-lite.</li>
105             <li><strong>Особенность:</strong> требует уста
новленной утилиты gemini CLI.</li>
106         </ul>
107     </div>
108 </div>
109
110 <!-- 5. Antigravity AGY -->
111 <div class="col-xl col-md-6">
112     <div class="p-3 border rounded bg-white h-100 shad
ow-sm">

```

```

113 |         <div class="fw-bold text-warning mb-1">
114 |             □ Antigravity (AGY)
115 |         </div>
116 |         <div class="small text-muted mb-2">
117 |             Мультиагентный поиск платформы Google.
118 |         </div>
119 |         <ul class="small text-secondary mb-0 ps-3">
120 |             <li>Инструменты <code>SEARCH_WEB</code> и <code>
121 | READ_URL_CONTENT</code>.</li>
122 |             <li>Глубокий ресерч и синтез сложных многоходов.
123 | Глубокий ресерч и синтез сложных многоходов.
124 |             <li><strong>Плюс:</strong> автономный агентный
125 | цикл.</li>
126 |             <li><strong>Особенность:</strong> использует Antigravity
127 | SDK / CLI.</li>
128 |         </ul>
129 |     </div>
130 | </div>
131 |
132 | <!-- Detailed Provider Settings & Live Search Tester -->
133 | <div class="row g-4 mb-4">
134 |     <!-- LEFT: Provider Detailed Settings -->
135 |     <div class="col-lg-6">
136 |         <div class="card shadow-sm border-0 h-100">
137 |             <div class="card-header bg-dark text-white fw-bold">
138 |                 <i class="bi bi-sliders me-1"></i> Параметры поисковых
139 | модулей
140 |             </div>
141 |             <div class="card-body">
142 |                 <!-- Gemini Grounding Settings -->
143 |                 <div class="mb-4 p-3 border rounded bg-light">
144 |                     <h6 class="fw-bold text-info mb-2"><i class="bi bi
145 | stars me-1"></i> Gemini Grounding</h6>
146 |                     <div class="mb-2">

```

```
145 |         <label class="form-label small text-muted fw-bol  
    | d">Модель для поиска Grounding (динамически из SDK):</label>  
146 |         <select class="form-select form-select-sm" id="s  
    | earch-gemini-model">  
147 |             <option value="">Загрузка доступных моделей че  
    | рез Google GenAI SDK...</option>  
148 |         </select>  
149 |     </div>  
150 |     <div class="small text-muted">  
151 |         Ключи автоматически считываются из вкладки «Моде  
    | ли и API» и ротируются по кругу.  
152 |     </div>  
153 | </div>  
154 |  
155 |     <!-- Gemini CLI Settings -->  
156 |     <div class="mb-4 p-3 border rounded bg-light">  
157 |         <h6 class="fw-bold text-primary mb-2"><i class="bi  
    | bi-terminal-fill me-1"></i> Google Gemini CLI</h6>  
158 |         <div class="mb-2">  
159 |             <label class="form-label small text-muted fw-bol  
    | d">Модель Gemini CLI (динамически из инструмента):</label>  
160 |             <select class="form-select form-select-sm" id="s  
    | earch-gemini-cli-model">  
161 |                 <option value="">Загрузка доступных моделей че  
    | рез Gemini CLI...</option>  
162 |             </select>  
163 |         </div>  
164 |         <div class="small text-muted">  
165 |             Использует локальную утилиту <code>gemini</code>  
    | с автоматическим вызовом поисковых инструментов.  
166 |         </div>  
167 |     </div>  
168 |  
169 |     <!-- Antigravity AGY Settings -->  
170 |     <div class="mb-4 p-3 border rounded bg-light">  
171 |         <h6 class="fw-bold text-warning mb-2"><i class="bi  
    | bi-rocket-takeoff me-1"></i> Antigravity (AGY) Search</h6>  
172 |         <div class="mb-2">
```



```
173 |         <label class="form-label small text-muted fw-bol
    | d">Модель агента AntigraVity (динамически из SDK):</label>
174 |         <select class="form-select form-select-sm" id="s
    | earch-agy-model">
175 |             <option value="">Загрузка доступных моделей че
    | рез AntigraVity SDK...</option>
176 |         </select>
177 |     </div>
178 |     <div class="small text-muted">
179 |         Использует встроенные агентные инструменты <code
    | >SEARCH_WEB</code> и <code>READ_URL_CONTENT</code>.
180 |     </div>
181 | </div>
182 |
183 |     <div class="d-grid">
184 |         <button class="btn btn-success fw-bold" id="btn-sa
    | ve-search-params">
185 |             <i class="bi bi-floppy me-1"></i> Сохранить наст
    | роЙки модулей
186 |         </button>
187 |     </div>
188 | </div>
189 | </div>
190 | </div>
191 |
192 | <!-- RIGHT: Live Interactive Search Tester -->
193 | <div class="col-lg-6">
194 |     <div class="card shadow-sm border-0 h-100">
195 |         <div class="card-header bg-dark text-white d-flex just
    | ify-content-between align-items-center">
196 |             <span class="fw-bold"><i class="bi bi-terminal me-1"
    | ></i> Интерактивный тест поиска</span>
197 |             <span class="badge bg-primary" id="test-search-engin
    | e-badge">playwright</span>
198 |         </div>
199 |         <div class="card-body d-flex flex-column">
200 |             <div class="mb-3">
201 |                 <label class="form-label small text-muted fw-bold"
```

```
202 | >Тестовый поисковый запрос:</label>
203 |     <div class="input-group">
204 |         <input type="text" class="form-control form-control-sm" id="test-search-query-input" placeholder="Например: последние новости марсохода 2026">
205 |         <button class="btn btn-primary btn-sm px-3" id="btn-run-test-search">
206 |             <i class="bi bi-search me-1"></i> Искать
207 |         </button>
208 |     </div>
209 |
210 |     <div class="flex-grow-1 d-flex flex-column">
211 |         <label class="form-label small text-muted fw-bold">
212 | >Результат работы провайдера:</label>
213 |         <div id="test-search-output-container" class="border rounded p-3 bg-light flex-grow-1 font-monospace" style="min-height: 230px; max-height: 350px; overflow-y: auto; font-size: 0.82rem; white-space: pre-wrap;">
214 |             <span class="text-muted">Введите запрос и нажмите «Искать» для проверки выбранного провайдера.</span>
215 |         </div>
216 |     </div>
217 | </div>
218 | </div>
219 | </div>
220 | </div>
221 |
```

main.js

```
1 // Search Tab Logic - Web Search MCP Providers Management
2
3 async function initSearchTab() {
4   const engineSelect = document.getElementById('search-tab-engine-selector');
5   const btnSaveEngine = document.getElementById('btn-save-search-tab-engine');
6   const btnSaveParams = document.getElementById('btn-save-search-params');
7   const geminiModelSelect = document.getElementById('search-gemini-model');
8   const geminiCliModelSelect = document.getElementById('search-gemini-cli-model');
9   const agyModelSelect = document.getElementById('search-agy-model');
10  const engineBadge = document.getElementById('test-search-engine-badge');
11  const btnRunTest = document.getElementById('btn-run-test-search');
12  const testQueryInput = document.getElementById('test-search-query-input');
13  const testOutputContainer = document.getElementById('test-search-output-container');
14
15  if (!engineSelect) return;
16
17  // --- 1. Load Dynamic Models via SDK / Tools ---
18  async function loadModels() {
19    try {
20      const modelsData = await window.api.fetch('/api/chat/models');
21      const modelsGrouped = modelsData.models || {};
22      const geminiList = modelsGrouped.gemini || [];
23      const geminiCliList = modelsGrouped.gemini_cli || ['gemini-3.1-flash-lite', 'gemini-2.5-flash', 'gemini-2.5-pro'];
24      const agyList = modelsGrouped.agy || [];
```

```

25 |
26 |     if (geminiModelSelect && geminiList.length > 0) {
27 |         const curVal = geminiModelSelect.value;
28 |         geminiModelSelect.innerHTML = '';
29 |         geminiList.forEach(m => {
30 |             const opt = document.createElement('option');
31 |             opt.value = m;
32 |             opt.textContent = m;
33 |             geminiModelSelect.appendChild(opt);
34 |         });
35 |         if (curVal && geminiList.includes(curVal)) {
36 |             geminiModelSelect.value = curVal;
37 |         }
38 |     }
39 |
40 |     if (geminiCliModelSelect && geminiCliList.length > 0) {
41 |         const curVal = geminiCliModelSelect.value;
42 |         geminiCliModelSelect.innerHTML = '';
43 |         geminiCliList.forEach(m => {
44 |             const opt = document.createElement('option');
45 |             opt.value = m;
46 |             opt.textContent = m;
47 |             geminiCliModelSelect.appendChild(opt);
48 |         });
49 |         if (curVal && geminiCliList.includes(curVal)) {
50 |             geminiCliModelSelect.value = curVal;
51 |         } else if (geminiCliList.includes('gemini-3.1-flash-li
52 | te')) {
53 |             geminiCliModelSelect.value = 'gemini-3.1-flash-lite'
54 |         };
55 |     }
56 |
57 |     if (agyModelSelect && agyList.length > 0) {
58 |         const curVal = agyModelSelect.value;
59 |         agyModelSelect.innerHTML = '';
60 |         agyList.forEach(m => {
            const opt = document.createElement('option');

```

```
61         opt.value = m;
62         opt.textContent = m;
63         agyModelSelect.appendChild(opt);
64     });
65     if (curVal && agyList.includes(curVal)) {
66         agyModelSelect.value = curVal;
67     }
68 }
69 } catch (err) {
70     console.error('[SearchTab] Ошибка загрузки динамических
моделей SDK/CLI:', err);
71 }
72 }
73
74 // --- 2. Load Current Config ---
75 async function loadConfig() {
76     try {
77         await loadModels();
78         const data = await window.api.fetch('/api/admin/web-search/config');
79         if (data) {
80             if (data.engine && engineSelect) {
81                 engineSelect.value = data.engine;
82                 if (engineBadge) engineBadge.textContent = data.engine;
83             }
84             if (data.gemini_model && geminiModelSelect) {
85                 geminiModelSelect.value = data.gemini_model;
86             }
87             if (data.gemini_cli_model && geminiCliModelSelect) {
88                 geminiCliModelSelect.value = data.gemini_cli_model;
89             }
90             if (data.agy_model && agyModelSelect) {
91                 agyModelSelect.value = data.agy_model;
92             }
93         }
94     } catch (e) {
95         console.error('[SearchTab] Ошибка загрузки конфигурации
```

```

поиска:', e);
96     notifySearch('Ошибка загрузки настроек поиска: ' + e.mes
sage, 'danger');
97     }
98     }
99
100    // --- 3. Save Config Helper ---
101    async function saveConfig() {
102        const engine = engineSelect ? engineSelect.value : 'playwr
ight';
103        const gemini_model = geminiModelSelect ? geminiModelSelect
.value : 'gemini-flash-lite-latest';
104        const gemini_cli_model = geminiCliModelSelect ? geminiCliM
odelSelect.value : 'gemini-3.1-flash-lite';
105        const agy_model = agyModelSelect ? agyModelSelect.value :
'agy-flash';
106
107        try {
108            await window.api.fetch('/api/admin/web-search/config', {
109                method: 'POST',
110                headers: { 'Content-Type': 'application/json' },
111                body: JSON.stringify({ engine, gemini_model, gemini_cl
i_model, agy_model })
112            });
113            if (engineBadge) engineBadge.textContent = engine;
114            notifySearch(` Настройки веб-поиска сохранены (активен:
${engine})`, 'success');
115
116            let compoundSearch = engine;
117            if (engine === 'gemini') compoundSearch = `${engine}:${g
emini_model}`;
118            if (engine === 'gemini_cli') compoundSearch = `${engine}
:${gemini_cli_model}`;
119            if (engine === 'agy') compoundSearch = `${engine}:${agy_
model}`;
120
121            if (typeof window.updateChatBadges === 'function') {
122                window.updateChatBadges(undefined, compoundSearch);

```

```

123     }
124   } catch (e) {
125     console.error('[SearchTab] Ошибка сохранения:', e);
126     notifySearch(' Ошибка сохранения настроек: ' + e.messag
e, 'danger');
127   }
128 }
129
130 if (btnSaveEngine) btnSaveEngine.onclick = saveConfig;
131 if (btnSaveParams) btnSaveParams.onclick = saveConfig;
132
133 if (engineSelect) {
134   engineSelect.onChange = () => {
135     if (engineBadge) engineBadge.textContent = engineSelect.
value;
136   };
137 }
138
139 // --- 3. Interactive Search Tester ---
140 if (btnRunTest && testQueryInput && testOutputContainer) {
141   const runTest = async () => {
142     const query = testQueryInput.value.trim();
143     if (!query) {
144       notifySearch('Введите поисковый запрос для теста!', 'w
arning');
145       testQueryInput.focus();
146       return;
147     }
148
149     const engine = engineSelect ? engineSelect.value : 'play
wright';
150     btnRunTest.disabled = true;
151     const originalText = btnRunTest.innerHTML;
152     btnRunTest.innerHTML = '<span class="spinner-border spin
ner-border-sm me-1" role="status"></span> Поиск...';
153     testOutputContainer.innerHTML = `<span class="text-info"
> Выполнение поиска через <strong>${engine}</strong> по запро
су: "${query}"...</span>`;

```

```
154 |
155 |     try {
156 |         const response = await window.api.fetch('/api/admin/web-search/test', {
157 |             method: 'POST',
158 |             headers: { 'Content-Type': 'application/json' },
159 |             body: JSON.stringify({ query, engine })
160 |         });
161 |
162 |         if (response.status === 'ok') {
163 |             testOutputContainer.textContent = response.result ||
164 |             'Пустой ответ.';
165 |         } else {
166 |             testOutputContainer.innerHTML = `<span class="text-danger">❌ Ошибка: ${response.message || 'Неизвестная ошибка'}</span>`;
167 |         }
168 |     } catch (err) {
169 |         console.error('[SearchTab] Ошибка теста:', err);
170 |         testOutputContainer.innerHTML = `<span class="text-danger">❌ Ошибка запроса: ${err.message}</span>`;
171 |     } finally {
172 |         btnRunTest.disabled = false;
173 |         btnRunTest.innerHTML = originalText;
174 |     }
175 |
176 |     btnRunTest.onclick = runTest;
177 |     testQueryInput.onkeydown = (e) => {
178 |         if (e.key === 'Enter') {
179 |             e.preventDefault();
180 |             runTest();
181 |         }
182 |     };
183 | }
184 |
185 | // Helper notification
186 | function notifySearch(msg, type = 'info') {
```



```
187 |     if (typeof showNotification === 'function') {
188 |         showNotification(msg, type);
189 |     } else if (typeof showModelsNotification === 'function') {
190 |         showModelsNotification(msg, type);
191 |     } else {
192 |         console.log(`[${type}] ${msg}`);
193 |     }
194 | }
195 |
196 | await loadConfig();
197 | }
198 |
199 | window.initSearchTab = initSearchTab;
200 |
```

main.js

```
1 /**
2  * sources_tab/main.js
3  * Логика вкладки редактирования источников.
4  */
5
6 let currentSourcesObj = {};
7
8 async function loadSourcesRaw() {
9     const editor = document.getElementById('sources-json-editor');
10    const tbody = document.getElementById('sources-list-body')
11    ;
12
13    if (editor) {
14        editor.disabled = true;
15        editor.value = 'Загрузка...';
16    }
17    if (tbody) {
18        tbody.innerHTML = '<tr><td colspan="6" class="text-center py-4 text-muted">Загрузка источников...</td></tr>';
19    }
20
21    try {
22        const response = await window.api.fetch('/api/admin/sources/raw');
23        const rawContent = response.content || '{}';
24
25        if (editor) editor.value = rawContent;
26
27        try {
28            currentSourcesObj = JSON.parse(rawContent);
29            renderSourcesTable();
30        } catch (e) {
31            console.error('Ошибка парсинга JSON для таблицы:',
32            e);
33
34            if (tbody) {
```

```
32         tbody.innerHTML = '<tr><td colspan="6" class="
| text-center py-4 text-danger">Ошибка формата JSON. Таблица нед
| оступна.</td></tr>';
33     }
34 }
35
36 } catch (err) {
37     console.error('Ошибка загрузки источников:', err);
38     if (editor) editor.value = 'Ошибка загрузки:\n' + err.
| message;
39     if (tbody) tbody.innerHTML = `<tr><td colspan="6" clas
| s="text-center py-4 text-danger">Ошибка загрузки: ${err.messag
| e}</td></tr>`;
40 } finally {
41     if (editor) editor.disabled = false;
42 }
43 }
44
45 async function saveSourcesRaw(content) {
46     const editor = document.getElementById('sources-json-edito
| r');
47
48     // Проверка синтаксиса на клиенте перед отправкой
49     try {
50         JSON.parse(content);
51     } catch (e) {
52         if (typeof showNotification === 'function') {
53             showNotification('Синтаксическая ошибка JSON: ' +
| e.message, 'danger');
54         } else {
55             alert('Синтаксическая ошибка JSON: ' + e.message);
56         }
57         return false;
58     }
59
60     try {
61         if (editor) editor.disabled = true;
62         const response = await window.api.fetch('/api/admin/so
```

```

        'urces/raw', {
63         method: 'POST',
64         headers: { 'Content-Type': 'application/json' },
65         body: JSON.stringify({ content: content })
66     });
67
68     if (response.status === 'ok') {
69         if (typeof showNotification === 'function') {
70             showNotification('Источники успешно сохранены!', 'success');
71         }
72         return true;
73     }
74     } catch (err) {
75         console.error('Ошибка сохранения источников:', err);
76         if (typeof showNotification === 'function') {
77             showNotification('Ошибка сохранения: ' + err.message, 'danger');
78         }
79         return false;
80     } finally {
81         if (editor) editor.disabled = false;
82     }
83 }
84
85 function renderSourcesTable() {
86     const tbody = document.getElementById('sources-list-body');
87     ;
88     const filterSelect = document.getElementById('sources-category-filter');
89     if (!tbody) return;
90     const filterVal = filterSelect ? filterSelect.value : 'all';
91     ;
92     tbody.innerHTML = '';
93     let hasItems = false;
94

```

```

95     for (const [category, items] of Object.entries(currentSourcesObj)) {
96         if (!Array.isArray(items)) continue; // Пропускаем метаданные самого файла (например, "metadata": {...})
97
98         // Маппинг категорий
99         let catForFilter = 'other';
100        if (category === 'metadata_apis') catForFilter = 'metadata';
101        else if (category === 'streaming_search') catForFilter = 'streaming_search';
102        else if (category.includes('streaming') || category === 'direct_sites') catForFilter = 'streaming';
103        else if (category === 'iframe_players') catForFilter = 'embed_player';
104        else if (category === 'torrent_trackers') catForFilter = 'torrent';
105        else if (category === 'video_search') catForFilter = 'search';
106
107        if (filterVal !== 'all' && catForFilter !== filterVal) continue;
108
109        items.forEach((item, index) => {
110            hasItems = true;
111            const tr = document.createElement('tr');
112
113            const isEnabled = item.enabled !== false; // по умолчанию true
114
115            tr.innerHTML = `
116                <td>
117                    <div class="form-check form-switch">
118                        <input class="form-check-input toggle-source-btn" type="checkbox" role="switch"
119                            data-category="${category}" data-index="${index}" ${isEnabled ? 'checked' : ''}>
120                    </div>

```

```

121 |             </td>
122 |             <td class="fw-bold">${item.name || item.id ||
| 'Без названия'}</td>
123 |             <td><a href="${item.url || '#'}" target="_blan
| k" class="small text-truncate d-inline-block" style="max-width
| : 150px;">${item.url || '-'}</a></td>
124 |             <td><span class="badge bg-secondary">${categor
| y}</span></td>
125 |             <td class="small text-muted">${item.descriptio
| n || '-'}</td>
126 |             <td class="text-end">
127 |                 <button class="btn btn-sm btn-outline-dang
| er delete-source-btn" data-category="${category}" data-index="
| ${index}" title="Удалить">
128 |                     <i class="bi bi-trash"></i>
129 |                 </button>
130 |             </td>
131 |         `;
132 |         tbody.appendChild(tr);
133 |     });
134 | }
135 |
136 | if (!hasItems) {
137 |     tbody.innerHTML = '<tr><td colspan="6" class="text-cen
| ter py-4 text-muted">Нет источников для отображения</td></tr>'
| ;
138 | }
139 |
140 | // Привязываем обработчики к сгенерированным кнопкам
141 | document.querySelectorAll('.toggle-source-btn').forEach(bt
| n => {
142 |     btn.addEventListener('change', handleToggleSource);
143 | });
144 |
145 | document.querySelectorAll('.delete-source-btn').forEach(bt
| n => {
146 |     btn.addEventListener('click', handleDeleteSource);
147 | });

```

```
148 }
149
150 async function handleToggleSource(e) {
151     const category = e.target.getAttribute('data-category');
152     const index = parseInt(e.target.getAttribute('data-index'))
153     );
154     if (currentSourcesObj[category] && currentSourcesObj[category][index]) {
155         currentSourcesObj[category][index].enabled = e.target.checked;
156         const newJson = JSON.stringify(currentSourcesObj, null, 2);
157
158         // Обновляем текстовое поле и сервер
159         const editor = document.getElementById('sources-json-editor');
160         if (editor) editor.value = newJson;
161
162         await saveSourcesRaw(newJson);
163     }
164 }
165
166 async function handleDeleteSource(e) {
167     const btn = e.target.closest('button');
168     const category = btn.getAttribute('data-category');
169     const index = parseInt(btn.getAttribute('data-index'));
170
171     if (confirm('Вы уверены, что хотите удалить этот источник?')) {
172         if (currentSourcesObj[category] && currentSourcesObj[category][index]) {
173             currentSourcesObj[category].splice(index, 1);
174
175             const newJson = JSON.stringify(currentSourcesObj, null, 2);
176             const editor = document.getElementById('sources-json-editor');
```

```
177         if (editor) editor.value = newJson;
178
179         const success = await saveSourcesRaw(newJson);
180         if (success) renderSourcesTable();
181     }
182 }
183 }
184
185 async function handleAddSource() {
186     const id = document.getElementById('new-source-id')?.value
187     .trim();
188     const name = document.getElementById('new-source-name')?.v
189     alue.trim();
190     const url = document.getElementById('new-source-url')?.val
191     ue.trim();
192     const filterCat = document.getElementById('new-source-cate
193     gory')?.value;
194     const desc = document.getElementById('new-source-descripti
195     on')?.value.trim();
196
197     if (!id || !name || !url) {
198         alert('Заполните обязательные поля: ID, Название и URL
199         ');
200         return;
201     }
202
203     // Определяем в какую секцию JSON положить новый элемент
204     let targetCategory = 'other';
205     if (filterCat === 'metadata') targetCategory = 'metadata_a
206     pis';
207     else if (filterCat === 'streaming_search') targetCategory
208     = 'streaming_search';
209     else if (filterCat === 'streaming') targetCategory = 'dire
210     ct_sites';
211     else if (filterCat === 'embed_player') targetCategory = 'i
212     frame_players';
213     else if (filterCat === 'torrent') targetCategory = 'torren
214     t_trackers';
```



```
204     else if (filterCat === 'search') targetCategory = 'video_s
|earch';
205
206     if (!currentSourcesObj[targetCategory]) {
207         currentSourcesObj[targetCategory] = [];
208     }
209
210     const newItem = {
211         id: id,
212         name: name,
213         url: url,
214         description: desc,
215         enabled: true
216     };
217
218     currentSourcesObj[targetCategory].push(newItem);
219
220     const newJson = JSON.stringify(currentSourcesObj, null, 2)
|;
221     const editor = document.getElementById('sources-json-edito
|r');
222     if (editor) editor.value = newJson;
223
224     const success = await saveSourcesRaw(newJson);
225     if (success) {
226         renderSourcesTable();
227         // Очищаем форму
228         document.getElementById('form-add-source').reset();
229     }
230 }
231
232 // Инициализация вкладки
233 window.initSourcesTab = function() {
234     console.log('Инициализация вкладки Источники...');
235
236     const btnRefresh = document.getElementById('btn-refresh-so
|urces-json');
237     const btnSave = document.getElementById('btn-save-sources-
```

```

    json');
238     const btnSaveAll = document.getElementById('btn-save-all-s
sources');
239     const btnAdd = document.getElementById('btn-add-source');
240     const filterSelect = document.getElementById('sources-cate
gory-filter');
241     const editor = document.getElementById('sources-json-edito
r');
242
243     if (btnRefresh) btnRefresh.onclick = loadSourcesRaw;
244
245     if (btnSave) {
246         btnSave.onclick = () => {
247             if (editor) saveSourcesRaw(editor.value).then(() =
> {
248                 // Обновляем таблицу после ручного изменения J
JSON
249                 try {
250                     currentSourcesObj = JSON.parse(editor.valu
e);
251                     renderSourcesTable();
252                 } catch(e){}
253             });
254         };
255     }
256
257     if (btnSaveAll) {
258         btnSaveAll.onclick = btnSave.onclick;
259     }
260
261     if (btnAdd) {
262         btnAdd.onclick = handleAddSource;
263     }
264
265     if (filterSelect) {
266         filterSelect.addEventListener('change', renderSourcesT
able);
267     }

```

```
268 |
269 |     // Также обновляем таблицу при потере фокуса редактором, е
    |     сли JSON валидный
270 |     if (editor) {
271 |         editor.addEventListener('blur', () => {
272 |             try {
273 |                 currentSourcesObj = JSON.parse(editor.value);
274 |                 renderSourcesTable();
275 |             } catch(e){}
276 |         });
277 |     }
278 |
279 |     // Загружаем данные при открытии
280 |     loadSourcesRaw();
281 | };
282 |
```

index.html

```
1 <!-- Sources Registry Tab -->
2 <div class="container-fluid">
3   <div class="alert alert-info d-flex justify-content-between
4     align-items-center flex-wrap gap-2 mb-3 shadow-sm">
5     <div>
6       <h5 class="mb-1"><i class="bi bi-collection-play me-2"><
7         /i>□ Реестр источников данных</h5>
8       <p class="mb-0 small text-secondary">Управление списком
9         сайтов, стримингов и парсеров для медиатеки.</p>
10      </div>
11      <div>
12        <button class="btn btn-outline-primary btn-sm rounded-pi
13          ll" onclick="const t = document.querySelector('[data-bs-target
14            =\'#tab-search\']'); if(t) t.click();">
15          <i class="bi bi-globe2 me-1"></i> Настройки веб-поиска
16          (MCP) →
17        </button>
18      </div>
19    </div>
20  </div>
21  <div class="row g-3">
22    <!-- LEFT: Add new source -->
23    <div class="col-4">
24      <div class="card mb-3">
25        <div class="card-header bg-dark text-white">
26          <span>□ Добавить источник</span>
27        </div>
28        <div class="card-body">
29          <form id="form-add-source" onsubmit="event.preventDe
30            fault();">
31            <div class="mb-2">
32              <label for="new-source-id" class="form-label sma
33                ll fw-bold">ID (уникальный):</label>
34              <input type="text" class="form-control form-cont
35                rol-sm" id="new-source-id"
36                placeholder="Например: my_site" required>
```

```
28         </div>
29         <div class="mb-2">
30             <label for="new-source-name" class="form-label small fw-bold">Название:</label>
31             <input type="text" class="form-control form-control-sm" id="new-source-name"
32                 placeholder="Например: My Site" required>
33         </div>
34         <div class="mb-2">
35             <label for="new-source-url" class="form-label small fw-bold">URL:</label>
36             <input type="url" class="form-control form-control-sm" id="new-source-url"
37                 placeholder="https://example.com" required>
38         </div>
39         <div class="mb-2">
40             <label for="new-source-category" class="form-label small fw-bold">Категория:</label>
41             <select class="form-select form-select-sm" id="new-source-category">
42                 <option value="streaming">streaming – стриминг
43                 <option value="streaming_search">streaming_search – поиск стримингов</option>
44                 <option value="metadata">metadata – метаданные
45                 <option value="embed_player">embed_player – встроенный плеер</option>
46                 <option value="torrent">torrent – торрент-трекер</option>
47                 <option value="search">search – поисковик</option>
48                 <option value="other">other – другое</option>
49             </select>
50         </div>
51         <div class="mb-3">
52             <label for="new-source-description" class="form-
```

```

label small fw-bold">Описание:</label>
53 |         <textarea class="form-control form-control-sm" i
| d="new-source-description" rows="2"
54 |             placeholder="Краткое описание источник
| a"></textarea>
55 |     </div>
56 |     <div class="d-grid">
57 |         <button type="button" class="btn btn-success btn
| -sm" id="btn-add-source">
58 |             □ Добавить
59 |         </button>
60 |     </div>
61 | </form>
62 | </div>
63 | </div>
64 |
65 | <!-- JSON Raw Editor -->
66 | <div class="card">
67 |     <div class="card-header bg-dark text-white d-flex just
| ify-content-between align-items-center">
68 |         <span>□ JSON-редактор</span>
69 |         <div class="d-flex gap-2">
70 |             <button class="btn btn-outline-light btn-sm" id="b
| tn-refresh-sources-json">
71 |                 <i class="bi bi-arrow-clockwise"></i>
72 |             </button>
73 |             <button class="btn btn-success btn-sm" id="btn-sav
| e-sources-json">
74 |                 <i class="bi bi-floppy"></i> Сохранить JSON
75 |             </button>
76 |         </div>
77 |     </div>
78 |     <div class="card-body p-2">
79 |         <p class="small text-muted mb-2">Прямое редактирован
| ие JSON. Используйте с осторожностью.</p>
80 |         <textarea
81 |             id="sources-json-editor"
82 |             class="form-control font-monospace"

```

```

83         style="min-height: 250px; max-height: 250px; resiz
e: none; font-size: 0.78rem;"
84         spellcheck="false"
85     ></textarea>
86 </div>
87 </div>
88 </div>
89
90 <!-- RIGHT: Sources Table -->
91 <div class="col-8">
92     <div class="card">
93         <div class="card-header bg-dark text-white d-flex just
ify-content-between align-items-center">
94             <span>📄 Список источников</span>
95             <button class="btn btn-outline-light btn-sm" id="btn
-refresh-sources">
96                 <i class="bi bi-arrow-clockwise"></i> Обновить
97             </button>
98         </div>
99         <div class="card-body p-0">
100             <!-- Category filter -->
101             <div class="p-2 border-bottom d-flex gap-2 flex-wrap
align-items-center">
102                 <span class="small text-muted">Фильтр:</span>
103                 <select class="form-select form-select-sm" id="sou
rces-category-filter" style="width: auto;">
104                     <option value="all">Все категории</option>
105                     <option value="streaming">Стриминг</option>
106                     <option value="streaming_search">Поиск стриминго
в</option>
107                     <option value="metadata">Метаданные</option>
108                     <option value="embed_player">Встроенные плееры</
option>
109                     <option value="torrent">Торренты</option>
110                     <option value="search">Поисковики</option>
111                     <option value="other">Другое</option>
112                 </select>
113             </div>

```

```

114 |         <!-- Table container with vertical scroll -->
115 |         <div style="height: 500px; overflow-y: scroll; overf
| low-x: auto;">
116 |             <table class="table table-striped table-hover align-middle mb-0" id="sources-table" style="min-width: 600px;">
117 |                 <thead class="table-dark">
118 |                     <tr>
119 |                         <th style="width: 5%;">Он</th>
120 |                         <th style="width: 15%;">Название</th>
121 |                         <th style="width: 25%;">URL</th>
122 |                         <th style="width: 15%;">Категория</th>
123 |                         <th style="width: 25%;">Описание</th>
124 |                         <th style="width: 15%;" class="text-end">Дей
| ствия</th>
125 |                     </tr>
126 |                 </thead>
127 |                 <tbody id="sources-list-body" style="word-wrap:
| break-word; overflow-wrap: break-word;">
128 |                     <tr>
129 |                         <td colspan="6" class="text-center py-4 text
| -muted">Загрузка источников...</td>
130 |                     </tr>
131 |                 </tbody>
132 |             </table>
133 |         </div>
134 |     </div>
135 | </div>
136 | </div>
137 | </div>
138 |
139 | <!-- Save all button at bottom -->
140 | <div class="mt-3 text-end">
141 |     <button class="btn btn-primary" id="btn-save-all-sources">
142 |         <i class="bi bi-floppy"></i> Сохранить все изменения
143 |     </button>
144 | </div>
145 | </div>
146 |

```


main.js

```
1 /**
2  * sources_tab/main.js
3  * Логика вкладки редактирования источников.
4  */
5
6 let currentSourcesObj = {};
7
8 async function loadSourcesRaw() {
9     const editor = document.getElementById('sources-json-editor');
10    const tbody = document.getElementById('sources-list-body')
11    ;
12
13    if (editor) {
14        editor.disabled = true;
15        editor.value = 'Загрузка...';
16    }
17    if (tbody) {
18        tbody.innerHTML = '<tr><td colspan="6" class="text-center py-4 text-muted">Загрузка источников...</td></tr>';
19    }
20
21    try {
22        const response = await window.api.fetch('/api/admin/sources/raw');
23        const rawContent = response.content || '{}';
24
25        if (editor) editor.value = rawContent;
26
27        try {
28            currentSourcesObj = JSON.parse(rawContent);
29            renderSourcesTable();
30        } catch (e) {
31            console.error('Ошибка парсинга JSON для таблицы:',
32            e);
33
34            if (tbody) {
```

```
32         tbody.innerHTML = '<tr><td colspan="6" class="
text-center py-4 text-danger">Ошибка формата JSON. Таблица нед
оступна.</td></tr>';
33     }
34 }
35
36 } catch (err) {
37     console.error('Ошибка загрузки источников:', err);
38     if (editor) editor.value = 'Ошибка загрузки:\n' + err.
message;
39     if (tbody) tbody.innerHTML = `<tr><td colspan="6" clas
s="text-center py-4 text-danger">Ошибка загрузки: ${err.messag
e}</td></tr>`;
40 } finally {
41     if (editor) editor.disabled = false;
42 }
43 }
44
45 async function saveSourcesRaw(content) {
46     const editor = document.getElementById('sources-json-edito
r');
47
48     // Проверка синтаксиса на клиенте перед отправкой
49     try {
50         JSON.parse(content);
51     } catch (e) {
52         if (typeof showNotification === 'function') {
53             showNotification('Синтаксическая ошибка JSON: ' +
e.message, 'danger');
54         } else {
55             alert('Синтаксическая ошибка JSON: ' + e.message);
56         }
57         return false;
58     }
59
60     try {
61         if (editor) editor.disabled = true;
62         const response = await window.api.fetch('/api/admin/so
```

```

        'urces/raw', {
63         method: 'POST',
64         headers: { 'Content-Type': 'application/json' },
65         body: JSON.stringify({ content: content })
66     });
67
68     if (response.status === 'ok') {
69         if (typeof showNotification === 'function') {
70             showNotification('Источники успешно сохранены!', 'success');
71         }
72         return true;
73     }
74     } catch (err) {
75         console.error('Ошибка сохранения источников:', err);
76         if (typeof showNotification === 'function') {
77             showNotification('Ошибка сохранения: ' + err.message, 'danger');
78         }
79         return false;
80     } finally {
81         if (editor) editor.disabled = false;
82     }
83 }
84
85 function renderSourcesTable() {
86     const tbody = document.getElementById('sources-list-body');
87     ;
88     const filterSelect = document.getElementById('sources-category-filter');
89     if (!tbody) return;
90     const filterVal = filterSelect ? filterSelect.value : 'all';
91     ;
92     tbody.innerHTML = '';
93     let hasItems = false;
94

```

```

95     for (const [category, items] of Object.entries(currentSourcesObj)) {
96         if (!Array.isArray(items)) continue; // Пропускаем метаданные самого файла (например, "metadata": {...})
97
98         // Маппинг категорий
99         let catForFilter = 'other';
100        if (category === 'metadata_apis') catForFilter = 'metadata';
101        else if (category === 'streaming_search') catForFilter = 'streaming_search';
102        else if (category.includes('streaming') || category === 'direct_sites') catForFilter = 'streaming';
103        else if (category === 'iframe_players') catForFilter = 'embed_player';
104        else if (category === 'torrent_trackers') catForFilter = 'torrent';
105        else if (category === 'video_search') catForFilter = 'search';
106
107        if (filterVal !== 'all' && catForFilter !== filterVal) continue;
108
109        items.forEach((item, index) => {
110            hasItems = true;
111            const tr = document.createElement('tr');
112
113            const isEnabled = item.enabled !== false; // по умолчанию true
114
115            tr.innerHTML = `
116                <td>
117                    <div class="form-check form-switch">
118                        <input class="form-check-input toggle-source-btn" type="checkbox" role="switch"
119                            data-category="${category}" data-index="${index}" ${isEnabled ? 'checked' : ''}>
120                    </div>

```

```

121 |             </td>
122 |             <td class="fw-bold">${item.name || item.id ||
| 'Без названия'}</td>
123 |             <td><a href="${item.url || '#'}" target="_blan
| k" class="small text-truncate d-inline-block" style="max-width
| : 150px;">${item.url || '-'}</a></td>
124 |             <td><span class="badge bg-secondary">${categor
| y}</span></td>
125 |             <td class="small text-muted">${item.descriptio
| n || '-'}</td>
126 |             <td class="text-end">
127 |                 <button class="btn btn-sm btn-outline-dang
| er delete-source-btn" data-category="${category}" data-index="
| ${index}" title="Удалить">
128 |                     <i class="bi bi-trash"></i>
129 |                 </button>
130 |             </td>
131 |         `;
132 |         tbody.appendChild(tr);
133 |     });
134 | }
135 |
136 | if (!hasItems) {
137 |     tbody.innerHTML = '<tr><td colspan="6" class="text-cen
| ter py-4 text-muted">Нет источников для отображения</td></tr>'
| ;
138 | }
139 |
140 | // Привязываем обработчики к сгенерированным кнопкам
141 | document.querySelectorAll('.toggle-source-btn').forEach(bt
| n => {
142 |     btn.addEventListener('change', handleToggleSource);
143 | });
144 |
145 | document.querySelectorAll('.delete-source-btn').forEach(bt
| n => {
146 |     btn.addEventListener('click', handleDeleteSource);
147 | });

```

```
148 }
149
150 async function handleToggleSource(e) {
151     const category = e.target.getAttribute('data-category');
152     const index = parseInt(e.target.getAttribute('data-index'))
153     );
154     if (currentSourcesObj[category] && currentSourcesObj[category][index]) {
155         currentSourcesObj[category][index].enabled = e.target.checked;
156         const newJson = JSON.stringify(currentSourcesObj, null, 2);
157
158         // Обновляем текстовое поле и сервер
159         const editor = document.getElementById('sources-json-editor');
160         if (editor) editor.value = newJson;
161
162         await saveSourcesRaw(newJson);
163     }
164 }
165
166 async function handleDeleteSource(e) {
167     const btn = e.target.closest('button');
168     const category = btn.getAttribute('data-category');
169     const index = parseInt(btn.getAttribute('data-index'));
170
171     if (confirm('Вы уверены, что хотите удалить этот источник?')) {
172         if (currentSourcesObj[category] && currentSourcesObj[category][index]) {
173             currentSourcesObj[category].splice(index, 1);
174
175             const newJson = JSON.stringify(currentSourcesObj, null, 2);
176             const editor = document.getElementById('sources-json-editor');
```

```
177         if (editor) editor.value = newJson;
178
179         const success = await saveSourcesRaw(newJson);
180         if (success) renderSourcesTable();
181     }
182 }
183 }
184
185 async function handleAddSource() {
186     const id = document.getElementById('new-source-id')?.value
187     .trim();
188     const name = document.getElementById('new-source-name')?.v
189     alue.trim();
190     const url = document.getElementById('new-source-url')?.val
191     ue.trim();
192     const filterCat = document.getElementById('new-source-cate
193     gory')?.value;
194     const desc = document.getElementById('new-source-descripti
195     on')?.value.trim();
196
197     if (!id || !name || !url) {
198         alert('Заполните обязательные поля: ID, Название и URL
199         ');
200         return;
201     }
202
203     // Определяем в какую секцию JSON положить новый элемент
204     let targetCategory = 'other';
205     if (filterCat === 'metadata') targetCategory = 'metadata_a
206     pis';
207     else if (filterCat === 'streaming_search') targetCategory
208     = 'streaming_search';
209     else if (filterCat === 'streaming') targetCategory = 'dire
210     ct_sites';
211     else if (filterCat === 'embed_player') targetCategory = 'i
212     frame_players';
213     else if (filterCat === 'torrent') targetCategory = 'torren
214     t_trackers';
```

```
204     else if (filterCat === 'search') targetCategory = 'video_s
|earch';
205
206     if (!currentSourcesObj[targetCategory]) {
207         currentSourcesObj[targetCategory] = [];
208     }
209
210     const newItem = {
211         id: id,
212         name: name,
213         url: url,
214         description: desc,
215         enabled: true
216     };
217
218     currentSourcesObj[targetCategory].push(newItem);
219
220     const newJson = JSON.stringify(currentSourcesObj, null, 2)
|;
221     const editor = document.getElementById('sources-json-edito
|r');
222     if (editor) editor.value = newJson;
223
224     const success = await saveSourcesRaw(newJson);
225     if (success) {
226         renderSourcesTable();
227         // Очищаем форму
228         document.getElementById('form-add-source').reset();
229     }
230 }
231
232 // Инициализация вкладки
233 window.initSourcesTab = function() {
234     console.log('Инициализация вкладки Источники...');
235
236     const btnRefresh = document.getElementById('btn-refresh-so
|urces-json');
237     const btnSave = document.getElementById('btn-save-sources-
```



```

    json');
238     const btnSaveAll = document.getElementById('btn-save-all-s
sources');
239     const btnAdd = document.getElementById('btn-add-source');
240     const filterSelect = document.getElementById('sources-cate
gory-filter');
241     const editor = document.getElementById('sources-json-edito
r');
242
243     if (btnRefresh) btnRefresh.onclick = loadSourcesRaw;
244
245     if (btnSave) {
246         btnSave.onclick = () => {
247             if (editor) saveSourcesRaw(editor.value).then(() =
> {
248                 // Обновляем таблицу после ручного изменения J
JSON
249                 try {
250                     currentSourcesObj = JSON.parse(editor.valu
e);
251                     renderSourcesTable();
252                 } catch(e){}
253             });
254         };
255     }
256
257     if (btnSaveAll) {
258         btnSaveAll.onclick = btnSave.onclick;
259     }
260
261     if (btnAdd) {
262         btnAdd.onclick = handleAddSource;
263     }
264
265     if (filterSelect) {
266         filterSelect.addEventListener('change', renderSourcesT
able);
267     }

```

```
268 |
269 |     // Также обновляем таблицу при потере фокуса редактором, е
    |     сли JSON валидный
270 |     if (editor) {
271 |         editor.addEventListener('blur', () => {
272 |             try {
273 |                 currentSourcesObj = JSON.parse(editor.value);
274 |                 renderSourcesTable();
275 |             } catch(e){}
276 |         });
277 |     }
278 |
279 |     // Загружаем данные при открытии
280 |     loadSourcesRaw();
281 | };
282 |
283 |
```

index.html

```
1 <!-- TTS Settings Tab -->
2 <div class="container-fluid">
3   <div class="alert alert-info">
4     <h5 data-i18n="tts.title">□ Настройка TTS (Синтез речи)</
h5>
5     <p data-i18n="tts.description">Установите голос и движок п
о умолчанию для озвучки описания медиафайлов, а также протести
руйте их звучание.</p>
6   </div>
7
8   <div class="row g-3">
9     <!-- LEFT: Settings Configuration -->
10    <div class="col-5">
11      <div class="card mb-3">
12        <div class="card-header bg-dark text-white d-flex just
ify-content-between align-items-center">
13          <span>□ Параметры озвучки</span>
14        </div>
15        <div class="card-body">
16          <form id="admin-tts-form" onsubmit="event.preventDef
ault();">
17            <!-- Active Voice Badge -->
18            <div id="admin-active-voice-container" class="mb-3
" style="display: none;">
19              <span class="badge bg-success d-inline-flex alig
n-items-center gap-1 p-2">
20                <i class="bi bi-check-circle-fill"></i>
21                <span>Активный голос: <strong id="admin-active
-voice-name">...</strong></span>
22              </span>
23            </div>
24
25            <!-- Engine Selection -->
26            <div class="mb-3">
27              <label for="admin-tts-engine" class="form-label
small fw-bold">Система синтеза:</label>
```

```
28 |         <select id="admin-tts-engine" class="form-select
    | form-select-sm">
29 |             <option value="browser">Web Speech API (Браузе
    | рная)</option>
30 |             <option value="gtts">Google TTS (Базовая)</opt
    | ion>
31 |             <option value="edge-tts" selected>Microsoft Ed
    | ge TTS (Нейросетевая)</option>
32 |             <option value="silero">Silero TTS (Локальная о
    | ффлайн)</option>
33 |         </select>
34 |     </div>
35 |
36 |     <!-- Voice Selection -->
37 |     <div class="mb-3" id="admin-voice-select-container
    | ">
38 |         <label for="admin-tts-voice" class="form-label s
    | mall fw-bold">Голос / Спикер:</label>
39 |         <select id="admin-tts-voice" class="form-select
    | form-select-sm">
40 |             <!-- Dynamically populated -->
41 |         </select>
42 |     </div>
43 |
44 |     <!-- Save Default Button -->
45 |     <div class="d-grid">
46 |         <button type="button" class="btn btn-success btn
    | -sm" id="btn-admin-tts-save">
47 |             <i class="bi bi-floppy"></i> Применить по умол
    | чанию
48 |         </button>
49 |     </div>
50 | </form>
51 | </div>
52 | </div>
53 | </div>
54 |
55 | <!-- RIGHT: Text to Speech Playground -->
```

```

56 |     <div class="col-7">
57 |         <div class="card">
58 |             <div class="card-header bg-dark text-white">
59 |                 <span>□ Тестирование озвучки</span>
60 |             </div>
61 |             <div class="card-body">
62 |                 <div class="mb-3">
63 |                     <label for="admin-tts-text" class="form-label small fw-bold">Текст для проверки:</label>
64 |                     <textarea id="admin-tts-text" class="form-control form-control-sm" rows="5">Привет! Это проверка работы модульной системы синтеза речи. Выберите систему и голос слева, затем нажмите кнопку «Озвучить» для воспроизведения.</textarea>
65 |                 </div>
66 |
67 |                 <div class="d-flex align-items-center gap-3">
68 |                     <button type="button" class="btn btn-primary btn-sm px-4" id="btn-admin-tts-speak">
69 |                         <i class="bi bi-play-fill"></i> Озвучить текст
70 |                     </button>
71 |
72 |                     <!-- Loading Spinner -->
73 |                     <div class="status-badge small text-muted d-none" id="admin-tts-status">
74 |                         <div class="spinner-border spinner-border-sm text-primary" role="status"></div>
75 |                         <span id="admin-status-text" class="ms-1">Генерация аудио...</span>
76 |                     </div>
77 |                 </div>
78 |
79 |                 <!-- Audio Player Container -->
80 |                 <div class="mt-4 border-top pt-3 d-none" id="admin-tts-player-container">
81 |                     <label class="form-label small fw-bold">Результат генерации:</label>
82 |                     <audio id="admin-tts-audio-player" class="w-100" controls></audio>

```

```
83 |         </div>
84 |     </div>
85 | </div>
86 | </div>
87 | </div>
88 | </div>
89 |
```

main.js

```
1 // TTS Tab Configuration and Testing Logic
2 // Called via window.initTtsTab() when tab is loaded
3
4 'use strict';
5
6 function ttsFetch(url, options = {}) {
7     return window.api
8         ? window.api.fetch(url, options)
9         : fetch(url, options).then(r => {
10             if (!r.ok) throw new Error(`HTTP ${r.status}`);
11             return r.json();
12         });
13 }
14
15 function ttsNotify(msg, type = 'info') {
16     if (typeof showNotification === 'function') {
17         showNotification(msg, type);
18     } else {
19         console.log(`[${type}] ${msg}`);
20     }
21 }
22
23 // Available voices dictionary
24 const ADMIN_VOICE_MAP = {
25     'browser': [], // Populated dynamically from window.speechSynthesis
26     'gtts': [
27         { value: 'ru', label: 'Русский (Google TTS)' },
28     ],
29     'edge-tts': [
30         { value: 'ru-RU-DmitryNeural', label: 'Дмитрий (Microsoft Edge)' },
31         { value: 'ru-RU-SvetlanaNeural', label: 'Светлана (Microsoft Edge)' },
32     ],
33     'silero': [
```

```
34 |     { value: 'eugene', label: 'Евгений (Silero)' },
35 |     { value: 'aidar', label: 'Айдар (Silero)' },
36 |     { value: 'baya', label: 'Бая (Silero)' },
37 |     { value: 'kseniya', label: 'Ксения (Silero)' },
38 |     { value: 'xenia', label: 'Ксения v2 (Silero)' },
39 |     { value: 'random', label: 'Рандомный (Silero)' }
40 | ]
41 | };
42 |
43 | function getFriendlyVoiceName(system, voice) {
44 |     if (system === 'edge-tts') {
45 |         if (voice === 'ru-RU-DmitryNeural') return 'Дмитрий (Micro
46 | soft Edge)';
47 |         if (voice === 'ru-RU-SvetlanaNeural') return 'Светлана (Mi
48 | crosoft Edge)';
49 |         return `Edge (${voice})`;
50 |     }
51 |     if (system === 'silero') {
52 |         const names = {
53 |             eugene: 'Евгений (Silero)',
54 |             aidar: 'Айдар (Silero)',
55 |             baya: 'Бая (Silero)',
56 |             kseniya: 'Ксения (Silero)',
57 |             xenia: 'Ксения v2 (Silero)',
58 |             random: 'Рандомный (Silero)'
59 |         };
60 |         return names[voice] || `Silero (${voice})`;
61 |     }
62 |     if (system === 'gtts') {
63 |         return 'Google TTS (Базовый)';
64 |     }
65 |     if (system === 'browser') {
66 |         return `Браузерный (${voice} || 'По умолчанию')`;
67 |     }
68 |     return `${system} – ${voice}`;
69 | }
70 |
71 | async function initTtsTab() {
```



```
70 |   const engineSelect = document.getElementById('admin-tts-engi
    | ne');
71 |   const voiceSelect = document.getElementById('admin-tts-voice
    | ');
72 |   const voiceContainer = document.getElementById('admin-voice-
    | select-container');
73 |   const btnSpeak = document.getElementById('btn-admin-tts-spea
    | k');
74 |   const btnSave = document.getElementById('btn-admin-tts-save'
    | );
75 |   const ttsStatus = document.getElementById('admin-tts-status'
    | );
76 |   const statusText = document.getElementById('admin-status-tex
    | t');
77 |   const playerContainer = document.getElementById('admin-tts-p
    | layer-container');
78 |   const audioPlayer = document.getElementById('admin-tts-audio
    | -player');
79 |   const activeVoiceContainer = document.getElementById('admin-
    | active-voice-container');
80 |   const activeVoiceName = document.getElementById('admin-activ
    | e-voice-name');
81 |
82 |   if (!engineSelect || !voiceSelect) {
83 |     console.error('TTS Settings Tab DOM elements not found.');
```

84 | return;

85 | }

86 |

```
87 |   function updateActiveBadge(system, voice) {
88 |     if (system && voice) {
89 |       activeVoiceName.textContent = getFriendlyVoiceName(system, voice);
90 |       activeVoiceContainer.style.display = 'block';
91 |     } else {
92 |       activeVoiceContainer.style.display = 'none';
93 |     }
94 |   }
95 |
```

```

96 // Populate voices based on selected engine
97 function populateVoices() {
98     const engine = engineSelect.value;
99     const voices = ADMIN_VOICE_MAP[engine] || [];
100
101     voiceSelect.innerHTML = '';
102
103     if (voices.length === 0) {
104         voiceContainer.style.display = 'none';
105     } else {
106         voiceContainer.style.display = 'block';
107         voices.forEach(voice => {
108             const opt = document.createElement('option');
109             opt.value = voice.value;
110             opt.textContent = voice.label;
111             voiceSelect.appendChild(opt);
112         });
113     }
114 }
115
116 // Load browser speech synthesis voices if available
117 function loadBrowserVoices() {
118     if ('speechSynthesis' in window) {
119         const voices = window.speechSynthesis.getVoices();
120         ADMIN_VOICE_MAP['browser'] = voices
121             .filter(v => v.lang.startsWith('ru'))
122             .map(v => ({ value: v.name, label: `Браузер: ${v.name}`
123 ` })));
124
125         if (ADMIN_VOICE_MAP['browser'].length === 0) {
126             ADMIN_VOICE_MAP['browser'].push({ value: 'default', la
127 bel: 'Браузер по умолчанию' });
128         }
129
130         if (engineSelect.value === 'browser') {
131             populateVoices();
132         }
133     }
134 }

```

```
132 | }
133 |
134 | if ('speechSynthesis' in window) {
135 |     window.speechSynthesis.onvoiceschanged = loadBrowserVoices
136 |     loadBrowserVoices();
137 | }
138 |
139 | engineSelect.addEventListener('change', populateVoices);
140 | populateVoices(); // Initial run
141 |
142 | // Fetch current user settings
143 | try {
144 |     const settings = await ttsFetch('/auth/settings');
145 |     if (settings) {
146 |         if (settings.tts_system) {
147 |             engineSelect.value = settings.tts_system;
148 |             populateVoices();
149 |         }
150 |         if (settings.tts_voice) {
151 |             voiceSelect.value = settings.tts_voice;
152 |         }
153 |         updateActiveBadge(settings.tts_system, settings.tts_voice);
154 |     }
155 | } catch (e) {
156 |     console.error('Failed to load active TTS settings:', e);
157 | }
158 |
159 | // Speak button handler
160 | btnSpeak.addEventListener('click', async () => {
161 |     const text = document.getElementById('admin-tts-text').value.trim();
162 |     if (!text) {
163 |         ttsNotify('Пожалуйста, введите текст для озвучки', 'warning');
164 |         return;
165 |     }
```

```
166 |
167 |     const engine = engineSelect.value;
168 |     const voice = voiceSelect.value;
169 |
170 |     if (engine === 'browser') {
171 |         if ('speechSynthesis' in window) {
172 |             window.speechSynthesis.cancel();
173 |             const cleanText = text.replace(/<[^>]*>/g, '').replace
174 | (/[*_`#]/g, '');
175 |             const utterance = new SpeechSynthesisUtterance(cleanText);
176 |
177 |             utterance.lang = 'ru-RU';
178 |
179 |             const voices = window.speechSynthesis.getVoices();
180 |             const selectedVoice = voices.find(v => v.name === voice);
181 |
182 |             if (selectedVoice) {
183 |                 utterance.voice = selectedVoice;
184 |             }
185 |
186 |             ttsStatus.classList.remove('d-none');
187 |             statusText.textContent = 'Воспроизведение браузером...';
188 |
189 |             utterance.onend = () => ttsStatus.classList.add('d-none');
190 |             utterance.onerror = () => ttsStatus.classList.add('d-none');
191 |
192 |             window.speechSynthesis.speak(utterance);
193 |             playerContainer.classList.add('d-none');
194 |         } else {
195 |             ttsNotify('Ваш браузер не поддерживает Web Speech API', 'danger');
196 |         }
197 |         return;
198 |     }
199 | }
```

```
197 // Server-side synthesis
198 ttsStatus.classList.remove('d-none');
199 statusText.textContent = 'Синтез аудиофайла на сервере...';
200 ;
201 btnSpeak.disabled = true;
202 playerContainer.classList.add('d-none');
203 try {
204     const queryParams = new URLSearchParams({
205         text: text,
206         system: engine,
207         voice: voice
208     });
209
210     const audioUrl = `/api/tts/synthesize?${queryParams.toString()}`;
211
212     audioPlayer.src = audioUrl;
213     audioPlayer.load();
214
215     audioPlayer.oncanplaythrough = () => {
216         ttsStatus.classList.add('d-none');
217         btnSpeak.disabled = false;
218         playerContainer.classList.remove('d-none');
219         audioPlayer.play().catch(err => console.log('Audio autoplay blocked or failed:', err));
220     };
221
222     audioPlayer.onerror = (err) => {
223         console.error('Audio element error:', err);
224         statusText.textContent = 'Ошибка загрузки аудиофайла';
225         btnSpeak.disabled = false;
226         setTimeout(() => ttsStatus.classList.add('d-none'), 3000);
227     };
228
229 } catch (err) {
230     console.error(err);
```

```

231 |     statusText.textContent = 'Ошибка сети';
232 |     btnSpeak.disabled = false;
233 |     setTimeout(() => ttsStatus.classList.add('d-none'), 3000
    | );
234 |   }
235 | });
236 |
237 | // Save Settings handler
238 | btnSave.addEventListener('click', async () => {
239 |   const engine = engineSelect.value;
240 |   const voice = voiceSelect.value;
241 |
242 |   btnSave.disabled = true;
243 |   const originalText = btnSave.innerHTML;
244 |   btnSave.innerHTML = '<span class="spinner-border spinner-b
    | order-sm" role="status" aria-hidden="true"></span> Сохранение.
    | ..';
245 |
246 |   try {
247 |     const payload = {
248 |       tts_enabled: 1,
249 |       tts_system: engine,
250 |       tts_voice: voice
251 |     };
252 |
253 |     const response = await fetch('/auth/settings', {
254 |       method: 'POST',
255 |       headers: { 'Content-Type': 'application/json' },
256 |       body: JSON.stringify(payload)
257 |     });
258 |
259 |     if (response.ok) {
260 |       updateActiveBadge(engine, voice);
261 |       ttsNotify('Настройки TTS успешно применены по умолчани
    | ю!', 'success');
262 |     } else {
263 |       const errData = await response.json().catch(() => ({}))
    | );

```

```
264 |         throw new Error(errData.detail || 'Ошибка сохранения н
|астроек');
265 |     }
266 | } catch (err) {
267 |     console.error(err);
268 |     ttsNotify('Не удалось сохранить настройки: ' + err.messa
|ge, 'danger');
269 | } finally {
270 |     btnSave.disabled = false;
271 |     btnSave.innerHTML = originalText;
272 | }
273 | });
274 | }
275 |
276 | window.initTtsTab = initTtsTab;
277 |
```

google_signin.html

```
1 <!-- Google Sign-In Component -->
2 <div id="google-signin-container" class="d-flex align-items-center gap-2">
3   <button id="google-signin-btn" class="btn btn-outline-light btn-sm">
4     <i class="bi bi-google"></i>
5     <span data-i18n="auth.googleSignIn">Войти через Google</span>
6   </button>
7   <div id="user-profile" class="d-none flex-grow-1">
8     <div class="d-flex align-items-center gap-2">
9       <img id="user-avatar" src="" class="rounded-circle" width="32" height="32">
10      <span id="user-name" class="text-muted small"></span>
11      <button id="logout-btn" class="btn btn-sm btn-danger">
12        <i class="bi bi-box-arrow-right"></i> Выйти
13      </button>
14    </div>
15  </div>
16 </div>
17
18 <script>
19 // Google Sign-In Component Logic
20 (async () => {
21   const googleSignInBtn = document.getElementById('google-signin-btn');
22   const logoutBtn = document.getElementById('logout-btn');
23   const userProfile = document.getElementById('user-profile');
24   const userAvatar = document.getElementById('user-avatar');
25   const userName = document.getElementById('user-name');
26
27   // Check authentication status
28   async function checkAuthStatus() {
29     try {
30       const response = await fetch('/auth/check');
31       const data = await response.json();
```



```
32 |
33 |     if (data.authenticated) {
34 |         // User is authenticated
35 |         userProfile.classList.remove('d-none');
36 |         googleSignInBtn.classList.add('d-none');
37 |
38 |         if (data.name) {
39 |             userName.textContent = data.name;
40 |         }
41 |         if (data.email) {
42 |             userName.textContent = data.email.split('@')[0];
43 |         }
44 |         if (data.picture) {
45 |             userAvatar.src = data.picture;
46 |         }
47 |     } else {
48 |         // User is not authenticated
49 |         userProfile.classList.add('d-none');
50 |         googleSignInBtn.classList.remove('d-none');
51 |     }
52 | } catch (error) {
53 |     console.error('Auth check failed:', error);
54 |     userProfile.classList.add('d-none');
55 |     googleSignInBtn.classList.remove('d-none');
56 | }
57 | }
58 |
59 | // Handle Google Sign-In click
60 | googleSignInBtn?.addEventListener('click', () => {
61 |     window.location.href = '/auth/google?next=' + encodeURIComponent(
window.location.pathname + window.location.search);
62 | });
63 |
64 | // Handle logout click
65 | logoutBtn?.addEventListener('click', async () => {
66 |     try {
67 |         await fetch('/auth/logout', { method: 'POST' });
68 |         // Reload page to update UI
```

```
69 |         window.location.reload();
70 |     } catch (error) {
71 |         console.error('Logout failed:', error);
72 |     }
73 | });
74 |
75 | // Initial auth check
76 | checkAuthStatus();
77 | })();
78 | </script>
```

index.html

```
1 <!DOCTYPE html>
2 <html lang="ru">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-s
cale=1.0">
6   <title>AI Assistant</title>
7   <link href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dis
t/css/bootstrap.min.css" rel="stylesheet">
8   <link href="https://cdn.jsdelivrivr.net/npm/bootstrap-icons@1.1
0.0/font/bootstrap-icons.css" rel="stylesheet">
9   <link href="/html/css/main.css?v=20260818_youtube_online" re
l="stylesheet">
10  <script src="https://cdn.jsdelivrivr.net/npm/i18next@23.11.5/i1
8next.min.js"></script>
11  <script src="https://cdn.jsdelivrivr.net/npm/marked/marked.min.
js"></script>
12  <style>
13    body {
14      background: #0d1117;
15      color: #c9d1d9;
16      min-height: 100vh;
17    }
18    .nav-tabs .nav-link {
19      color: #8b949e;
20      border: none;
21      border-bottom: 2px solid transparent;
22      background: transparent;
23    }
24    .nav-tabs .nav-link:hover {
25      color: #c9d1d9;
26      border-color: transparent;
27    }
28    .nav-tabs .nav-link.active {
29      color: #58a6ff;
30      background: transparent;
```

```
31 |     border-bottom-color: #58a6ff;
32 | }
33 | .chat-window {
34 |     background: #161b22;
35 |     border-radius: 8px;
36 |     padding: 15px;
37 |     height: 60vh;
38 |     overflow-y: auto;
39 | }
40 | .chat-message {
41 |     margin-bottom: 12px;
42 |     padding: 10px 14px;
43 |     border-radius: 8px;
44 |     max-width: 85%;
45 | }
46 | .chat-message.user {
47 |     background: #1f6feb;
48 |     color: #fff;
49 |     margin-left: auto;
50 | }
51 | .chat-message.assistant {
52 |     background: #21262d;
53 |     color: #c9d1d9;
54 | }
55 | .input-group {
56 |     background: #161b22;
57 |     border-radius: 8px;
58 |     padding: 10px;
59 | }
60 | .form-control {
61 |     background: #0d1117;
62 |     border: 1px solid #30363d;
63 |     color: #c9d1d9;
64 | }
65 | .form-control:focus {
66 |     background: #0d1117;
67 |     border-color: #388bfd;
68 |     color: #c9d1d9;
```

```
69 |     box-shadow: 0 0 0 2px rgba(56,139,253,0.3);
70 | }
71 | .btn-primary {
72 |     background: #238636;
73 |     border-color: #238636;
74 | }
75 | .btn-primary:hover {
76 |     background: #2ea043;
77 |     border-color: #2ea043;
78 | }
79 | .player-container {
80 |     background: #161b22;
81 |     border-radius: 8px;
82 |     padding: 20px;
83 |     height: 70vh;
84 |     position: relative;
85 | }
86 | .video-wrapper {
87 |     background: #000;
88 |     border-radius: 8px;
89 |     height: 100%;
90 |     display: flex;
91 |     align-items: center;
92 |     justify-content: center;
93 |     position: relative;
94 | }
95 | .chat-toggle-btn {
96 |     position: absolute;
97 |     bottom: 20px;
98 |     right: 20px;
99 |     width: 50px;
100 |     height: 50px;
101 |     border-radius: 50%;
102 |     background: rgba(31, 111, 235, 0.9);
103 |     border: 1px solid rgba(255, 255, 255, 0.1);
104 |     color: #fff;
105 |     font-size: 24px;
106 |     cursor: pointer;
```

```
107 |     display: flex;
108 |     align-items: center;
109 |     justify-content: center;
110 |     box-shadow: 0 4px 15px rgba(0, 0, 0, 0.4);
111 |     transition: all 0.3s cubic-bezier(0.175, 0.885, 0.32, 1.
275);
112 |     z-index: 100;
113 | }
114 | .chat-toggle-btn:hover {
115 |     transform: scale(1.1);
116 |     background: #238636;
117 |     box-shadow: 0 6px 20px rgba(35, 134, 54, 0.5);
118 | }
119 | .chat-popup {
120 |     position: absolute;
121 |     bottom: 80px;
122 |     right: 20px;
123 |     width: 320px;
124 |     height: 380px;
125 |     background: rgba(22, 27, 34, 0.9);
126 |     backdrop-filter: blur(10px);
127 |     -webkit-backdrop-filter: blur(10px);
128 |     border: 1px solid rgba(255, 255, 255, 0.1);
129 |     border-radius: 12px;
130 |     display: flex;
131 |     flex-direction: column;
132 |     box-shadow: 0 10px 25px rgba(0, 0, 0, 0.5);
133 |     z-index: 101;
134 |     opacity: 0;
135 |     transform: translateY(20px) scale(0.95);
136 |     pointer-events: none;
137 |     transition: all 0.3s cubic-bezier(0.165, 0.84, 0.44, 1);
138 | }
139 | .chat-popup.show {
140 |     opacity: 1;
141 |     transform: translateY(0) scale(1);
142 |     pointer-events: all;
143 | }
```

```
144 | .chat-popup-header {
145 |     padding: 12px 15px;
146 |     border-bottom: 1px solid rgba(255, 255, 255, 0.1);
147 |     display: flex;
148 |     justify-content: space-between;
149 |     align-items: center;
150 |     font-weight: 600;
151 |     font-size: 14px;
152 |     background: rgba(0, 0, 0, 0.2);
153 |     border-top-left-radius: 12px;
154 |     border-top-right-radius: 12px;
155 | }
156 | .chat-close-btn {
157 |     background: transparent;
158 |     border: none;
159 |     color: #aaa;
160 |     font-size: 16px;
161 |     cursor: pointer;
162 |     padding: 2px 6px;
163 |     transition: color 0.2s;
164 | }
165 | .chat-close-btn:hover {
166 |     color: #fff;
167 | }
168 | .chat-popup-messages {
169 |     flex: 1;
170 |     padding: 15px;
171 |     overflow-y: auto;
172 |     display: flex;
173 |     flex-direction: column;
174 |     gap: 10px;
175 | }
176 | .chat-popup-msg {
177 |     max-width: 85%;
178 |     padding: 8px 12px;
179 |     border-radius: 8px;
180 |     font-size: 13px;
181 |     line-height: 1.4;
```

```
182     word-break: break-word;
183 }
184 .chat-popup-msg.bot {
185     background: rgba(255, 255, 255, 0.08);
186     color: #c9d1d9;
187     align-self: flex-start;
188     border-bottom-left-radius: 2px;
189 }
190 .chat-popup-msg.user {
191     background: #1f6feb;
192     color: #fff;
193     align-self: flex-end;
194     border-bottom-right-radius: 2px;
195 }
196 .chat-popup-input-area {
197     padding: 10px;
198     border-top: 1px solid rgba(255, 255, 255, 0.1);
199     display: flex;
200     gap: 8px;
201     background: rgba(0, 0, 0, 0.1);
202     border-bottom-left-radius: 12px;
203     border-bottom-right-radius: 12px;
204 }
205 .chat-popup-input-area input {
206     flex: 1;
207     padding: 6px 12px;
208     border: 1px solid rgba(255, 255, 255, 0.1);
209     border-radius: 6px;
210     background: rgba(255, 255, 255, 0.05);
211     color: #fff;
212     font-size: 13px;
213     outline: none;
214 }
215 .chat-popup-input-area input:focus {
216     border-color: #1f6feb;
217     background: rgba(255, 255, 255, 0.08);
218 }
219 .chat-popup-input-area button {
```



```
220 |     padding: 6px 12px;
221 |     background: #238636;
222 |     color: #fff;
223 |     border: none;
224 |     border-radius: 6px;
225 |     font-size: 13px;
226 |     cursor: pointer;
227 |     transition: background 0.2s;
228 | }
229 | .chat-popup-input-area button:hover {
230 |     background: #2ea043;
231 | }
232 | .chat-popup-input-area button:disabled {
233 |     opacity: 0.6;
234 |     cursor: not-allowed;
235 | }
236 | .video-wrapper video {
237 |     max-width: 100%;
238 |     max-height: 100%;
239 | }
240 | .file-list {
241 |     background: #161b22;
242 |     border-radius: 8px;
243 |     padding: 15px;
244 |     max-height: 30vh;
245 |     overflow-y: auto;
246 | }
247 | .file-item {
248 |     padding: 8px 12px;
249 |     border-radius: 6px;
250 |     cursor: pointer;
251 |     margin-bottom: 4px;
252 | }
253 | .file-item:hover {
254 |     background: #21262d;
255 | }
256 | .file-item.active {
257 |     background: #1f6feb;
```

```
258 |     }
259 |     .lang-selector {
260 |         background: #21262d !important;
261 |         border-color: #30363d !important;
262 |         color: #c9d1d9 !important;
263 |     }
264 |
265 |     /* Light theme styles */
266 |     .theme-light body {
267 |         background: #f6f8fa;
268 |         color: #24292f;
269 |     }
270 |     .theme-light .nav-tabs .nav-link {
271 |         color: #6e7781;
272 |     }
273 |     .theme-light .nav-tabs .nav-link.active {
274 |         color: #0969da;
275 |     }
276 |     .theme-light .chat-window {
277 |         background: #ffffff;
278 |     }
279 |     .theme-light .chat-message.assistant {
280 |         background: #f6f8fa;
281 |         color: #24292f;
282 |     }
283 |     .theme-light .chat-popup {
284 |         background: rgba(255, 255, 255, 0.95);
285 |     }
286 |     .theme-light .chat-popup-msg.bot {
287 |         background: #f6f8fa;
288 |         color: #24292f;
289 |     }
290 |     .theme-light .chat-popup-input-area input {
291 |         background: #fafafa;
292 |         color: #24292f;
293 |     }
294 |     .theme-light .form-control {
295 |         background: #ffffff;
```

```
296 |         border-color: #d0d7de;
297 |         color: #24292f;
298 |     }
299 |     .theme-light .form-control:focus {
300 |         border-color: #0969da;
301 |     }
302 |     .theme-light .btn-primary {
303 |         background: #2da44e;
304 |         border-color: #2da44e;
305 |     }
306 |     .theme-light .btn-primary:hover {
307 |         background: #2da44e;
308 |     }
309 |     .theme-light .player-container {
310 |         background: #ffffff;
311 |     }
312 |     .theme-light .file-list {
313 |         background: #ffffff;
314 |     }
315 |     .theme-light .file-item:hover {
316 |         background: #f6f8fa;
317 |     }
318 |     .theme-light .file-item.active {
319 |         background: #0969da;
320 |     }
321 |     .theme-light .lang-selector {
322 |         background: #ffffff !important;
323 |         border-color: #d0d7de !important;
324 |         color: #24292f !important;
325 |     }
326 | </style>
327 | </head>
328 | <body>
329 | <div class="container-fluid mt-3">
330 |
331 |     <ul class="nav nav-tabs mb-3" id="mainTabs">
332 |         <li class="nav-item">
333 |             <button class="nav-link active" data-bs-toggle="tab" dat
```

```

a-bs-target="#tab-player" data-i18n="tabs.player">
334     <i class="bi bi-play-circle"></i> Плеер
335     </button>
336 </li>
337 <li class="nav-item">
338     <button class="nav-link" data-bs-toggle="tab" data-bs-ta
rget="#tab-chat" data-i18n="tabs.chat">
339         <i class="bi bi-chat-dots"></i> Чат
340     </button>
341 </li>
342 <li class="nav-item auth-only d-none">
343     <button class="nav-link" data-bs-toggle="tab" data-bs-ta
rget="#tab-settings" data-i18n="tabs.settings">
344         <i class="bi bi-gear"></i> Настройки
345     </button>
346 </li>
347 <li class="nav-item auth-only d-none">
348     <a class="nav-link" href="/user_tts" data-i18n="tabs.tts
">
349         <i class="bi bi-mic"></i> Настроить TTS
350     </a>
351 </li>
352 <li class="nav-item ms-auto">
353     <select class="form-select form-select-sm lang-selector"
id="lang-selector" style="width:auto;">
354         <option value="ru">Русский</option>
355         <option value="en">English</option>
356         <option value="he">עברית</option>
357     </select>
358 </li>
359 <li class="nav-item ms-2">
360     <div id="auth-container">
361         <button id="login-trigger-btn" class="btn btn-sm btn-o
utline-light" data-bs-toggle="modal" data-bs-target="#loginMod
al">
362             <i class="bi bi-box-arrow-in-right"></i>
363             <span>Войти</span>
364         </button>

```

```

365         <div id="user-profile" class="d-none">
366             <div class="d-flex align-items-center gap-2">
367                 <img id="user-avatar" src="" class="rounded-circle
" width="32" height="32">
368                 <span id="user-name" class="text-muted small"></sp
an>
369                 <button id="logout-btn" class="btn btn-sm btn-dang
er">
370                     <i class="bi bi-box-arrow-right"></i>
371                 </button>
372             </div>
373         </div>
374     </div>
375 </li>
376 </ul>
377
378 <div class="tab-content">
379     <!-- Player Tab -->
380     <div id="tab-player" class="tab-pane fade show active">
381         <div class="row g-3">
382             <div class="col-md-3">
383                 <div class="file-list" id="file-list">
384                     <div class="text-muted text-center p-3" data-i18n=
"player.noFiles">Загрузка файлов...</div>
385                 </div>
386                 <!-- Настройки загрузки торрентов -->
387                 <div class="mt-2 p-2" style="background:#161b22;bord
er-radius:8px;">
388                     <small class="text-muted">Папка загрузки:</small>
389                     <input type="text" class="form-control form-contro
l-sm" id="download-dir"
390                         value="D:/Downloads" placeholder="D:/Downlo
ads">
391                     <button class="btn btn-outline-primary btn-sm w-10
0 mt-2" id="btn-refresh-files">
392                         □ Обновить список
393                     </button>
394                 </div>

```

```

395 |         </div>
396 |         <div class="col-md-9">
397 |             <div class="player-container">
398 |                 <div class="video-wrapper" id="video-wrapper">
399 |                     <video id="video" controls playsinline></video>
400 |                 </div>
401 |                 <button class="chat-toggle-btn" id="chat-toggle-bt
n" title="Открыть чат">❏ </button>
402 |                 <div class="chat-popup" id="chat-popup">
403 |                     <div class="chat-popup-header">
404 |                         <span>❏ Чат с AI</span>
405 |                         <span class="badge bg-secondary text-white fon
t-monospace ms-2" id="chat-popup-model-badge" style="font-size
: 0.7rem; opacity: 0.95; display: none; color: #ffffff !import
ant;" title="Выбранная модель ИИ"></span>
406 |                         <span class="badge bg-secondary text-white fon
t-monospace ms-1" id="chat-popup-search-badge" style="font-siz
e: 0.7rem; opacity: 0.95; display: none; color: #38bdf8 !impor
tant;" title="Провайдер веб-поиска"></span>
407 |                         <button class="chat-close-btn" id="chat-close-
btn">❏ </button>
408 |                     </div>
409 |                     <div class="chat-popup-messages" id="chat-popup-
messages">
410 |                         <div class="chat-popup-msg bot"><strong>Ai Асс
истент</strong> (System): Привет! Задавайте вопросы о воспроиз
водимом фильме/сериале или общайтесь с ассистентом.</div>
411 |                     </div>
412 |                     <div class="chat-popup-input-area">
413 |                         <input type="text" id="chat-popup-input" place
holder="Задайте вопрос...">
414 |                         <button id="chat-popup-send">Отправить</button
>
415 |                     </div>
416 |                 </div>
417 |             </div>
418 |             <div class="mt-3 d-flex gap-2">
419 |                 <button class="btn btn-secondary btn-sm" id="btn-p

```

```

rev" data-i18n="player.prev">⏮ Предыдущий</button>
420     <button class="btn btn-secondary btn-sm" id="btn-n
ext" data-i18n="player.next">Следующий ⏭ </button>
421     <span class="ms-auto text-muted" id="current-file"
></span>
422         </div>
423     </div>
424 </div>
425 </div>
426
427 <!-- Chat Tab -->
428 <div id="tab-chat" class="tab-pane fade">
429     <div class="position-relative">
430         <div class="chat-window" id="chat-window">
431             <div class="text-center text-muted p-4" id="chat-emp
ty" data-i18n="chat.startPrompt">
432                 Начните разговор с AI
433             </div>
434         </div>
435         <div class="position-absolute d-flex gap-1" style="top
: 8px; right: 15px; z-index: 100;">
436             <span class="badge bg-secondary text-white font-mono
space" id="chat-model-badge" style="opacity: 0.95; font-size:
0.75rem; display: none; color: #ffffff !important;" title="Выб
ранная модель ИИ"></span>
437             <span class="badge bg-secondary text-white font-mono
space" id="chat-search-badge" style="opacity: 0.95; font-size:
0.75rem; display: none; color: #38bdf8 !important;" title="Пр
овайдер веб-поиска"></span>
438         </div>
439     </div>
440     <div class="input-group mt-3">
441         <button class="btn btn-outline-secondary btn-sm" id="t
heme-toggle-btn" title="Переключить тему">
442             <i class="bi bi-brightness-high"></i>
443         </button>
444         <input type="text" class="form-control" id="message-in
put"

```

```

445 |         data-i18n-placeholder="chat.placeholder" placeh
| older="Введите сообщение...">
446 |         <button class="btn btn-primary" id="send-button" data-
| i18n="chat.send">Отправить</button>
447 |     </div>
448 | </div>
449 |
450 | <!-- Settings Tab -->
451 | <div id="tab-settings" class="tab-pane fade">
452 |     <div class="card bg-dark text-white border-secondary p-4
| mt-3" style="max-width: 600px; margin: 0 auto; background: #1
| 61b22 !important;">
453 |         <div class="text-center mb-4">
454 |             <img id="cab-avatar" src="" class="rounded-circle bo
| rder border-3 border-primary" width="100" height="100" style="
| object-fit: cover;">
455 |             <h3 class="mt-3" id="cab-name">Имя пользователя</h3>
456 |             <p class="text-muted" id="cab-email">email@example.c
| om</p>
457 |             <span class="badge bg-secondary" id="cab-role">User<
| /span>
458 |         </div>
459 |
460 |         <hr class="border-secondary">
461 |
462 |         <div class="mb-3">
463 |             <h5><i class="bi bi-telegram text-info"></i> Связь с
| Telegram</h5>
464 |             <div id="tg-status-linked" class="d-none">
465 |                 <p class="text-success"><i class="bi bi-check-circ
| le-fill"></i> Ваш аккаунт успешно связан с Telegram-ботом!</p>
466 |                 <p>Имя пользователя: <strong id="cab-tg-username">
| @username</strong></p>
467 |                 <a href="https://t.me/onela_bot" target="_blank" c
| lass="btn btn-primary btn-sm mt-2">
468 |                     <i class="bi bi-telegram"></i> Открыть telegram
| desktop
469 |                 </a>

```



```

470         </div>
471         <div id="tg-status-unlinked">
472             <p class="text-warning"><i class="bi bi-exclamation-triangle-fill"></i> Аккаунт Telegram не привязан.</p>
473             <p class="small text-muted">Чтобы привязать Telegram, нажмите кнопку ниже для генерации кода, затем отправьте боту в чат команду: <code>/link &lt;KOD&gt;</code></p>
474             <button class="btn btn-primary btn-sm" id="btn-gen-link-token">Сгенерировать код привязки</button>
475             <div id="tg-link-code-area" class="mt-3 d-none">
476                 <div class="alert alert-info d-flex align-items-center justify-content-between mb-0" style="background:#1f6feb; border-color:#1f6feb; color: white;">
477                     <span>Код привязки: <strong id="tg-link-code" class="fs-4 ms-2">XXXX</strong></span>
478                     <span class="badge bg-danger">Действителен 10 минут</span>
479                 </div>
480                 <div class="mt-2">
481                     <a href="https://t.me/onela_bot" target="_blank" class="btn btn-outline-primary btn-sm">
482                         <i class="bi bi-telegram"></i> Открыть telegram desktop
483                     </a>
484                 </div>
485             </div>
486         </div>
487     </div>
488
489     <hr class="border-secondary">
490
491     <h4 class="mb-4"><i class="bi bi-gear-fill text-primary"></i> Настройки профиля</h4>
492
493     <form id="settings-form">
494         <div class="mb-3">
495             <label for="settings-theme" class="form-label">Тема оформления</label>

```

```
496 |         <select class="form-select bg-dark text-white bord
    | er-secondary" id="settings-theme">
497 |             <option value="system">Системная</option>
498 |             <option value="dark">Темная</option>
499 |             <option value="light">Светлая</option>
500 |         </select>
501 |     </div>
502 |
503 |     <div class="mb-3">
504 |         <label for="settings-lang" class="form-label">Язык
    | интерфейса</label>
505 |         <select class="form-select bg-dark text-white bord
    | er-secondary" id="settings-lang">
506 |             <option value="ru">Русский</option>
507 |             <option value="en">English</option>
508 |             <option value="he">□ □ □ □ □ </option>
509 |         </select>
510 |     </div>
511 |
512 |     <div class="mb-3 form-check form-switch">
513 |         <input class="form-check-input" type="checkbox" id
    | ="settings-tts" checked>
514 |         <label class="form-check-label text-white" for="se
    | ttings-tts">Озвучивать ответы AI голосом (TTS) в Telegram</lab
    | el>
515 |     </div>
516 |
517 |     <div class="mb-3">
518 |         <label for="settings-tts-system" class="form-label
    | ">Система озвучки (TTS)</label>
519 |         <select class="form-select bg-dark text-white bord
    | er-secondary" id="settings-tts-system">
520 |             <option value="browser">Web Speech API (Браузерн
    | ый)</option>
521 |             <option value="edge-tts">Microsoft Edge TTS (Ней
    | росетевой)</option>
522 |             <option value="gtts">Google TTS (Базовый)</optio
    | n>
```

```
523 |         <option value="silero">Silero TTS (Локальный офф
    | лайн)</option>
524 |     </select>
525 | </div>
526 |
527 |     <div class="mb-3" id="settings-tts-voice-container">
528 |         <label for="settings-tts-voice" class="form-label"
    | >Голос / Диктор</label>
529 |         <select class="form-select bg-dark text-white bord
    | er-secondary" id="settings-tts-voice">
530 |             <optgroup label="Web Speech API" id="optgroup-br
    | owser">
531 |                 <!-- Populated dynamically -->
532 |             </optgroup>
533 |             <optgroup label="Microsoft Edge TTS" id="optgrou
    | p-edge">
534 |                 <option value="ru-RU-DmitryNeural">Дмитрий (Му
    | жской)</option>
535 |                 <option value="ru-RU-SvetlanaNeural">Светлана
    | (Женский)</option>
536 |             </optgroup>
537 |             <optgroup label="Silero TTS" id="optgroup-silero
    | ">
538 |                 <option value="eugene">Евгений (Мужской)</opti
    | on>
539 |                 <option value="aidar">Айдар (Мужской)</option>
540 |                 <option value="baya">Бая (Женский)</option>
541 |                 <option value="kseniya">Ксения (Женский)</opti
    | on>
542 |                 <option value="xenia">Ксения v2 (Женский)</opt
    | ion>
543 |                 <option value="random">Случайный (Рандомный)</
    | option>
544 |             </optgroup>
545 |         </select>
546 |     </div>
547 |
548 |     <div class="mb-3">
```

```

549         <label for="settings-instruction" class="form-label">Персональная системная инструкция для AI</label>
550         <textarea class="form-control bg-dark text-white border-secondary" id="settings-instruction" rows="4" placeholder="Например: Отвечай коротко и вежливо..."></textarea>
551         <div class="form-text text-muted">Индивидуальные правила поведения AI ассистента для ваших диалогов.</div>
552     </div>
553
554     <button type="submit" class="btn btn-success w-100 mt-2" id="btn-save-settings">Сохранить настройки</button>
555 </form>
556
557     <hr class="border-secondary">
558     <p class="small text-muted mb-0">Дата регистрации: <span id="cab-created-at">--.--.----</span></p>
559 </div>
560 </div>
561 </div>
562 </div>
563
564 <!-- Login Modal -->
565 <div class="modal fade" id="loginModal" tabindex="-1" aria-labelledby="loginModalLabel" aria-hidden="true">
566     <div class="modal-dialog modal-dialog-centered">
567         <div class="modal-content bg-dark text-white border-secondary">
568             <div class="modal-header border-secondary">
569                 <h5 class="modal-title" id="loginModalLabel">Авторизация</h5>
570                 <button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal" aria-label="Close"></button>
571             </div>
572             <div class="modal-body p-4">
573                 <!-- Error alert -->
574                 <div id="auth-alert" class="alert alert-danger d-none" role="alert"></div>
575

```

```

576 |         <!-- Login Form -->
577 |         <div id="auth-form-login">
578 |             <form id="form-login">
579 |                 <div class="mb-3">
580 |                     <label for="login-email" class="form-label">Email
581 |                     <input type="email" class="form-control bg-dark
582 | text-white border-secondary" id="login-email" required>
583 |                 </div>
584 |                 <div class="mb-3">
585 |                     <label for="login-password" class="form-label">П
586 | ароль</label>
587 |                     <input type="password" class="form-control bg-da
588 | rk text-white border-secondary" id="login-password" required>
589 |                 </div>
590 |                 <button type="submit" class="btn btn-primary w-100
591 | mb-3">Войти</button>
592 |                 <p class="text-center small mb-0">Нет аккаунта? <a
593 | href="#" id="go-to-register" class="text-info">Зарегистрирова
594 | ться</a></p>
595 |             </form>
596 |         </div>
597 |
598 |         <!-- Register Form -->
599 |         <div id="auth-form-register" class="d-none">
600 |             <form id="form-register">
601 |                 <div class="mb-3">
602 |                     <label for="register-name" class="form-label">Им
603 | я</label>
604 |                     <input type="text" class="form-control bg-dark t
605 | ext-white border-secondary" id="register-name" required>
606 |                 </div>
607 |                 <div class="mb-3">
608 |                     <label for="register-email" class="form-label">E
609 | mail</label>
610 |                     <input type="email" class="form-control bg-dark
611 | text-white border-secondary" id="register-email" required>
612 |                 </div>

```

```

603         <div class="mb-3">
604             <label for="register-password" class="form-label
">Пароль</label>
605             <input type="password" class="form-control bg-da
rk text-white border-secondary" id="register-password" require
d>
606         </div>
607         <button type="submit" class="btn btn-success w-100
mb-3">Зарегистрироваться</button>
608         <p class="text-center small mb-0">Уже есть аккаунт
? <a href="#" id="go-to-login" class="text-info">Войти</a></p>
609     </form>
610 </div>
611
612 <!-- Verify Form -->
613 <div id="auth-form-verify" class="d-none">
614     <p class="text-center">На вашу почту отправлен 6-зна
чный код подтверждения. Пожалуйста, введите его ниже.</p>
615     <form id="form-verify">
616         <div class="mb-3">
617             <label for="verify-code" class="form-label">Код
подтверждения</label>
618             <input type="text" class="form-control bg-dark t
ext-white border-secondary text-center fs-3" id="verify-code"
maxlength="6" placeholder="XXXXXX" required>
619         </div>
620         <button type="submit" class="btn btn-info w-100 mb
-3 text-white">Подтвердить</button>
621         <p class="text-center small mb-0"><a href="#" id="
verify-cancel" class="text-info">Назад к входу</a></p>
622     </form>
623 </div>
624
625 <div id="auth-social-divider" class="my-4 text-center
position-relative">
626     <hr class="border-secondary">
627     <span class="bg-dark px-2 text-muted position-absolu
te top-50 start-50 translate-middle small">Или войти через</sp

```

```

an>
628         </div>
629
630         <!-- Social Login buttons -->
631         <div id="auth-social" class="d-flex flex-column gap-2
align-items-center">
632             <button id="google-signin-btn" class="btn btn-outlin
e-light w-100 d-flex align-items-center justify-content-center
gap-2">
633                 <i class="bi bi-google"></i>
634                 <span>Google</span>
635             </button>
636
637             <div class="w-100 d-flex justify-content-center mt-2
">
638                 <!-- Telegram Login Widget -->
639                 <script async src="https://telegram.org/js/telegra
m-widget.js?22" data-telegram-login="onela_bot" data-size="lar
ge" data-auth-url="/auth/telegram/callback" data-request-acces
s="write"></script>
640             </div>
641         </div>
642     </div>
643 </div>
644 </div>
645 </div>
646
647 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist
/js/bootstrap.bundle.min.js"></script>
648 <script src="/html/js/chatService.js?v=20260818_search_badge">
</script>
649 <script type="module" src="/html/user/main.js?v=20260818_searc
h_badge"></script>
650 </body>
651 </html>

```

main.js

```
1 // User Interface Main JS
2 import { initI18n, switchLang, applyTranslations } from '../js
  /i18n.js';
3
4 window.switchLang = switchLang;
5 window.applyTranslations = applyTranslations;
6
7 // API module
8 window.api = {
9   async fetch(url, options = {}) {
10     const response = await fetch(url, options);
11     if (!response.ok) {
12       let msg = response.statusText;
13       try {
14         const data = await response.json();
15         msg = data.detail || msg;
16       } catch {}
17       throw new Error(`${response.status} ${msg}`);
18     }
19     return response.json();
20   },
21
22   // Torrent download
23   torrent: {
24     async add(source, downloadDir) {
25       return this.fetch('/api/torrents/add', {
26         method: 'POST',
27         headers: { 'Content-Type': 'application/json' },
28         body: JSON.stringify({ source, download_dir: downloadD
  ir })
29       });
30     },
31
32     async getStatus(hash) {
33       return this.fetch(`/api/torrents/status/${hash}`);
34     }
  }
```



```

35 | },
36 |
37 | // Auth module
38 | auth: {
39 |     async check() {
40 |         try {
41 |             const response = await fetch('/auth/check');
42 |             return await response.json();
43 |         } catch {
44 |             return { authenticated: false };
45 |         }
46 |     },
47 |
48 |     loginWithGoogle() {
49 |         // Direct redirect to Google OAuth - no fetch due to COR
S
50 |         window.location.href = '/auth/google?next=' + encodeURIComponent(
window.location.pathname + window.location.search);
51 |     },
52 |
53 |     async register(email, password, name) {
54 |         return this.fetch('/auth/register', {
55 |             method: 'POST',
56 |             headers: { 'Content-Type': 'application/json' },
57 |             body: JSON.stringify({ email, password, name })
58 |         });
59 |     },
60 |
61 |     async verify(email, code) {
62 |         return this.fetch('/auth/verify', {
63 |             method: 'POST',
64 |             headers: { 'Content-Type': 'application/json' },
65 |             body: JSON.stringify({ email, code })
66 |         });
67 |     },
68 |
69 |     async loginEmail(email, password) {
70 |         return this.fetch('/auth/login/email', {

```

```
71     method: 'POST',
72     headers: { 'Content-Type': 'application/json' },
73     body: JSON.stringify({ email, password })
74   });
75 },
76
77   logout() {
78     return fetch('/auth/logout', { method: 'POST' });
79   }
80 }
81 };
82
83 // — TORRENT LINK DETECTION —————
84
85 // Регулярки для поиска торрент-ссылок
86 const TORRENT_PATTERNS = [
87   /magnet:\?[^<>\s]+/gi, // magnet ссылки
88   /https?:\/\/\.[^\\s<>]+\.(torrent|torrent\.php)/gi, // URL к .torrent файлу
89 ];
90
91 function findTorrentLinks(text) {
92   const links = [];
93   for (const pattern of TORRENT_PATTERNS) {
94     const matches = text.match(pattern);
95     if (matches) {
96       links.push(...matches);
97     }
98   }
99   return [...new Set(links)]; // Уникальные
100 }
101
102 // Обработка торрент-ссылки
103 async function handleTorrentLink(link, downloadDir = 'D:/Downloads') {
104   try {
105     console.log('[Torrent] Adding:', link);
106   }
```

```
107 // Показываем индикатор загрузки
108 showNotification('Добавление торрента...', 'info');
109
110 // Отправляем на сервер
111 const result = await window.api.torrent.add(link, download
Dir);
112
113 if (result.error) {
114     showNotification('Ошибка: ' + result.error, 'danger');
115     return null;
116 }
117
118 // Показываем результат
119 const typeText = result.is_series ? 'сериал' : 'фильм';
120 const categoryText = result.is_series ? '01 serials' : '02
films';
121
122 showNotification(
123     `Добавлено: ${result.name}\nПапка: ${categoryText}\nКате
гория: ${result.category}`,
124     'success'
125 );
126
127 // Слушаем статус загрузки и открываем плеер
128 if (result.torrent_hash) {
129     monitorTorrentAndPlay(result.torrent_hash, result.save_p
ath);
130 }
131
132 return result;
133 } catch (e) {
134     console.error('[Torrent] Error:', e);
135     showNotification('Ошибка добавления торрента: ' + e.messag
e, 'danger');
136     return null;
137 }
138 }
139
```

```
140 // Мониторинг загрузки торрента и автовоспроизведение
141 async function monitorTorrentAndPlay(torrentHash, savePath) {
142     const maxChecks = 60; // Максимум 60 проверок (около 5 минут
    )
143     let checkCount = 0;
144
145     const checkInterval = setInterval(async () => {
146         checkCount++;
147
148         try {
149             const status = await window.api.torrent.getStatus(torrentHash);
150
151             if (status.error) {
152                 clearInterval(checkInterval);
153                 return;
154             }
155
156             // Проверяем состояние загрузки
157             const state = status.state;
158             const progress = status.progress;
159
160             console.log(`[Torrent] ${status.name}: ${state} (${progress}%)`);
161
162             // Загрузка завершена
163             if (state === 'uploading' || state === 'paused' || state
                === 'stalledUP' || progress >= 100) {
164                 clearInterval(checkInterval);
165
166                 if (progress >= 99.9 || state === 'uploading') {
167                     // Ищем первый видеофайл в папке загрузки
168                     const mediaFile = await findFirstMediaFile(savePath)
                    ;
169
170                     if (mediaFile) {
171                         showNotification('Загрузка завершена! Открываю плеер...', 'success');
```

```
172 |
173 |         // Переключаем на вкладку плеера
174 |         const playerTab = document.querySelector('[data-bs-
    -target="#tab-player"]');
175 |         const chatTab = document.querySelector('[data-bs-t
    arget="#tab-chat"]');
176 |         if (playerTab) {
177 |             playerTab.click();
178 |         }
179 |
180 |         // Добавляем файл в плеер и запускаем
181 |         await addMediaFileAndPlay(mediaFile);
182 |     } else {
183 |         showNotification('Загрузка завершена, но видеофайл
    не найден', 'warning');
184 |     }
185 | }
186 | }
187 |
188 | } catch (e) {
189 |     console.error('[Torrent] Monitor error:', e);
190 | }
191 |
192 | if (checkCount >= maxChecks) {
193 |     clearInterval(checkInterval);
194 |     showNotification('Превышено время ожидания загрузки', 'w
    arning');
195 | }
196 | }, 5000); // Проверяем каждые 5 секунд
197 | }
198 |
199 | // Поиск первого видеофайла в папке
200 | async function findFirstMediaFile(dirPath) {
201 |     try {
202 |         const files = await window.api.fetch('/api/media/files');
203 |         if (!files || !files.length) return null;
204 |
205 |         // Ищем файл в указанной директории
```

```
206     for (const f of files) {
207         if (f.path && f.path.toLowerCase().startsWith(dirPath.to
LowerCase())) {
208             return f;
209         }
210     }
211     return null;
212 } catch (e) {
213     console.error('[Torrent] Find media error:', e);
214     return null;
215 }
216 }
217
218 // Добавить файл и начать воспроизведение
219 async function addMediaFileAndPlay(file) {
220     // Добавляем в начало списка
221     mediaFiles.unshift(file);
222     renderFileList();
223     playFile(0);
224 }
225
226 // Уведомления
227 function showNotification(message, type = 'info') {
228     // Создаём уведомление
229     const notification = document.createElement('div');
230     notification.className = `alert alert-${type} position-fixed
top-0 end-0 m-3`;
231     notification.style.zIndex = '9999';
232     notification.style.maxWidth = '400px';
233     notification.textContent = message;
234     document.body.appendChild(notification);
235
236     // Автоскрытие через 5 секунд
237     setTimeout(() => {
238         notification.remove();
239     }, 5000);
240 }
241
```

```

242 | document.addEventListener('DOMContentLoaded', async () => {
243 |     console.log('User interface initializing...');
244 |
245 |     // Initialize i18n
246 |     const savedLang = localStorage.getItem('app_language') || 'ru';
247 |     await initI18n(savedLang);
248 |
249 |     // Setup language selector
250 |     const langSelector = document.getElementById('lang-selector');
251 |     if (langSelector) {
252 |         langSelector.value = savedLang;
253 |         langSelector.addEventListener('change', (e) => {
254 |             switchLang(e.target.value);
255 |         });
256 |     }
257 |
258 |     // Initialize auth
259 |     await initAuth();
260 |
261 |     // Initialize player
262 |     initPlayer();
263 |
264 |     // Initialize chat
265 |     initChat();
266 |
267 |     // Apply translations
268 |     applyTranslations();
269 |
270 |     console.log('User interface ready');
271 | });
272 |
273 | // — AUTH MODULE —————
274 |
275 | async function initAuth() {
276 |     const loginTriggerBtn = document.getElementById('login-trigg

```

```

    er-btn');
277   const googleSignInBtn = document.getElementById('google-sign
    in-btn');
278   const logoutBtn = document.getElementById('logout-btn');
279   const userProfile = document.getElementById('user-profile');
280   const userAvatar = document.getElementById('user-avatar');
281   const userName = document.getElementById('user-name');
282
283   // Modal forms
284   const formLogin = document.getElementById('form-login');
285   const formRegister = document.getElementById('form-register'
    );
286   const formVerify = document.getElementById('form-verify');
287
288   // Modal wrappers
289   const authLoginDiv = document.getElementById('auth-form-logi
    n');
290   const authRegisterDiv = document.getElementById('auth-form-r
    egister');
291   const authVerifyDiv = document.getElementById('auth-form-ver
    ify');
292   const authAlert = document.getElementById('auth-alert');
293
294   // Navigation links
295   const goToRegister = document.getElementById('go-to-register
    ');
296   const goToLogin = document.getElementById('go-to-login');
297   const verifyCancel = document.getElementById('verify-cancel'
    );
298
299   let currentVerifyEmail = '';
300
301   function showError(msg) {
302     if (authAlert) {
303       authAlert.textContent = msg;
304       authAlert.classList.remove('d-none');
305     }
306   }

```



```
307
308 function hideError() {
309     if (authAlert) {
310         authAlert.classList.add('d-none');
311     }
312 }
313
314 function switchForm(formName) {
315     hideError();
316     authLoginDiv?.classList.add('d-none');
317     authRegisterDiv?.classList.add('d-none');
318     authVerifyDiv?.classList.add('d-none');
319
320     if (formName === 'login') {
321         authLoginDiv?.classList.remove('d-none');
322     } else if (formName === 'register') {
323         authRegisterDiv?.classList.remove('d-none');
324     } else if (formName === 'verify') {
325         authVerifyDiv?.classList.remove('d-none');
326     }
327 }
328
329 // Set up navigations
330 goToRegister?.addEventListener('click', (e) => {
331     e.preventDefault();
332     switchForm('register');
333 });
334
335 goToLogin?.addEventListener('click', (e) => {
336     e.preventDefault();
337     switchForm('login');
338 });
339
340 verifyCancel?.addEventListener('click', (e) => {
341     e.preventDefault();
342     switchForm('login');
343 });
344
```

```
345 // Submit Login
346 formLogin?.addEventListener('submit', async (e) => {
347     e.preventDefault();
348     hideError();
349     const email = document.getElementById('login-email').value
350     ;
351     const password = document.getElementById('login-password')
352     .value;
353     try {
354         await window.api.auth.loginEmail(email, password);
355         window.location.reload();
356     } catch (err) {
357         console.error(err);
358         if (err.message.includes('403') || err.message.includes(
359             'не подтверждена')) {
360             currentVerifyEmail = email;
361             switchForm('verify');
362         } else {
363             showError(err.message || 'Ошибка входа');
364         }
365     });
366 // Submit Register
367 formRegister?.addEventListener('submit', async (e) => {
368     e.preventDefault();
369     hideError();
370     const name = document.getElementById('register-name').valu
371     e;
372     const email = document.getElementById('register-email').va
373     lue;
374     const password = document.getElementById('register-passwor
375     d').value;
376     try {
377         await window.api.auth.register(email, password, name);
378         currentVerifyEmail = email;
```

```
377 |     switchForm('verify');
378 | } catch (err) {
379 |     console.error(err);
380 |     showError(err.message || 'Ошибка регистрации');
381 | }
382 | });
383 |
384 | // Submit Verify
385 | formVerify?.addEventListener('submit', async (e) => {
386 |     e.preventDefault();
387 |     hideError();
388 |     const code = document.getElementById('verify-code').value;
389 |
390 |     try {
391 |         await window.api.auth.verify(currentVerifyEmail, code);
392 |         switchForm('login');
393 |         showNotification('Почта подтверждена! Теперь вы можете в
ойти.', 'success');
394 |     } catch (err) {
395 |         console.error(err);
396 |         showError(err.message || 'Неверный код подтверждения');
397 |     }
398 | });
399 |
400 | async function updateAuthUI(authData) {
401 |     if (authData.authenticated) {
402 |         userProfile?.classList.remove('d-none');
403 |         loginTriggerBtn?.classList.add('d-none');
404 |         document.querySelectorAll('.auth-only').forEach(el => el
.classList.remove('d-none'));
405 |
406 |         if (authData.name) {
407 |             userName.textContent = authData.name;
408 |         } else if (authData.email) {
409 |             userName.textContent = authData.email.split('@')[0];
410 |         }
411 |         if (authData.picture) {
412 |             userAvatar.src = authData.picture;
```

```

413     }
414
415     setupCabinet();
416     setupSettings();
417 } else {
418     userProfile?.classList.add('d-none');
419     loginTriggerBtn?.classList.remove('d-none');
420     document.querySelectorAll('.auth-only').forEach(el => el
421 .classList.add('d-none'));
422 }
423
424 const authData = await window.api.auth.check();
425 await updateAuthUI(authData);
426
427 googleSignInBtn?.addEventListener('click', () => {
428     window.api.auth.loginWithGoogle();
429 });
430
431 logoutBtn?.addEventListener('click', async () => {
432     try {
433         await window.api.auth.logout();
434         window.location.reload();
435     } catch (e) {
436         console.error('Logout failed:', e);
437     }
438 });
439 }
440
441 // — CABINET MODULE —————
442
443 async function setupCabinet() {
444     const avatar = document.getElementById('cab-avatar');
445     const nameEl = document.getElementById('cab-name');
446     const emailEl = document.getElementById('cab-email');
447     const roleEl = document.getElementById('cab-role');
448     const dateEl = document.getElementById('cab-created-at');

```

```
449
450   const linkedDiv = document.getElementById('tg-status-linked'
    );
451   const unlinkedDiv = document.getElementById('tg-status-unlin
    ked');
452   const tgUserEl = document.getElementById('cab-tg-username');
453
454   const genBtn = document.getElementById('btn-gen-link-token')
    ;
455   const codeArea = document.getElementById('tg-link-code-area'
    );
456   const codeEl = document.getElementById('tg-link-code');
457
458   try {
459     const data = await window.api.fetch('/auth/cabinet');
460     if (avatar && data.picture) avatar.src = data.picture;
461     if (nameEl) nameEl.textContent = data.name || 'Без имени';
462     if (emailEl) emailEl.textContent = data.email;
463     if (roleEl) {
464       roleEl.textContent = data.role === 'admin' ? 'Администра
    тор' : 'Пользователь';
465       roleEl.className = data.role === 'admin' ? 'badge bg-dan
    ger' : 'badge bg-secondary';
466     }
467     if (dateEl && data.created_at) {
468       const date = new Date(data.created_at);
469       dateEl.textContent = date.toLocaleDateString('ru-RU') +
    ' ' + date.toLocaleTimeString('ru-RU');
470     }
471
472     if (data.telegram_username) {
473       linkedDiv?.classList.remove('d-none');
474       unlinkedDiv?.classList.add('d-none');
475       if (tgUserEl) tgUserEl.textContent = data.telegram_usern
    ame.startsWith('@') ? data.telegram_username : `@${data.teleg
    ram_username}`;
476     } else {
477       linkedDiv?.classList.add('d-none');
```

```
478     unlinkedDiv?.classList.remove('d-none');
479   }
480 } catch (e) {
481   console.error('Failed to load cabinet data:', e);
482 }
483
484 genBtn?.addEventListener('click', async () => {
485   try {
486     const res = await window.api.fetch('/auth/link-token', {
487       method: 'POST' });
488     if (res.token) {
489       if (codeEl) codeEl.textContent = res.token;
490       codeArea?.classList.remove('d-none');
491       showNotification('Код привязки успешно сгенерирован',
492         'success');
493     } catch (e) {
494       showNotification('Не удалось получить код привязки: ' +
495         e.message, 'danger');
496     }
497   });
498 }
499
500 // — SETTINGS MODULE —————
501
502 async function setupSettings() {
503   const form = document.getElementById('settings-form');
504   const themeSelect = document.getElementById('settings-theme');
505   const langSelect = document.getElementById('settings-lang');
506   const ttsCheckbox = document.getElementById('settings-tts');
507   const ttsSystemSelect = document.getElementById('settings-tts-system');
508   const ttsVoiceSelect = document.getElementById('settings-tts-voice');
509   const ttsVoiceContainer = document.getElementById('settings-tts-voice-container');
```

```
508 |   const instructionText = document.getElementById('settings-in
|   struction');
509 |
510 |   const populateBrowserVoices = () => {
511 |     const optBrowser = document.getElementById('optgroup-brows
|     er');
512 |     if (optBrowser && 'speechSynthesis' in window) {
513 |       optBrowser.innerHTML = '';
514 |       const voices = window.speechSynthesis.getVoices();
515 |       const ruVoices = voices.filter(v => v.lang.startsWith('r
|       u'));
516 |       if (ruVoices.length === 0) {
517 |         const opt = document.createElement('option');
518 |         opt.value = 'default';
519 |         opt.textContent = 'По умолчанию (Браузерный)';
520 |         optBrowser.appendChild(opt);
521 |       } else {
522 |         ruVoices.forEach(v => {
523 |           const opt = document.createElement('option');
524 |           opt.value = v.name;
525 |           opt.textContent = v.name;
526 |           optBrowser.appendChild(opt);
527 |         });
528 |       }
529 |     }
530 |   };
531 |
532 |   const toggleVoiceVisibility = () => {
533 |     if (ttsSystemSelect && ttsVoiceContainer && ttsVoiceSelect
|     ) {
534 |       const val = ttsSystemSelect.value;
535 |       if (val === 'gtts') {
536 |         ttsVoiceContainer.style.display = 'none';
537 |       } else {
538 |         ttsVoiceContainer.style.display = 'block';
539 |         const optEdge = document.getElementById('optgroup-edge
|         ');
540 |         const optSilero = document.getElementById('optgroup-si
```

```

    lero');
541     const optBrowser = document.getElementById('optgroup-b
rowser');
542     if (optEdge && optSilero && optBrowser) {
543         if (val === 'edge-tts') {
544             optEdge.style.display = '';
545             optSilero.style.display = 'none';
546             optBrowser.style.display = 'none';
547             if (ttsVoiceSelect.value !== 'ru-RU-DmitryNeural'
&& ttsVoiceSelect.value !== 'ru-RU-SvetlanaNeural') {
548                 ttsVoiceSelect.value = 'ru-RU-DmitryNeural';
549             }
550         } else if (val === 'silero') {
551             optEdge.style.display = 'none';
552             optSilero.style.display = '';
553             optBrowser.style.display = 'none';
554             const sileroVoices = ['eugene', 'aidar', 'baya', '
kseniya', 'xenia', 'random'];
555             if (!sileroVoices.includes(ttsVoiceSelect.value))
{
556                 ttsVoiceSelect.value = 'eugene';
557             }
558         } else if (val === 'browser') {
559             optEdge.style.display = 'none';
560             optSilero.style.display = 'none';
561             optBrowser.style.display = '';
562             populateBrowserVoices();
563         }
564     }
565 }
566 }
567 };
568
569 if ('speechSynthesis' in window) {
570     window.speechSynthesis.onvoiceschanged = () => {
571         populateBrowserVoices();
572         toggleVoiceVisibility();
573     };

```



```
574     populateBrowserVoices();
575 }
576
577 ttsSystemSelect?.addEventListener('change', toggleVoiceVisib
ility);
578
579 try {
580     const settings = await window.api.fetch('/auth/settings');
581     if (themeSelect && settings.theme) themeSelect.value = set
tings.theme;
582     if (langSelect && settings.language) langSelect.value = se
ttings.language;
583     if (ttsCheckbox) ttsCheckbox.checked = settings.tts_enable
d === 1;
584     if (ttsSystemSelect && settings.tts_system) {
585         ttsSystemSelect.value = settings.tts_system;
586     }
587
588     // Make sure options are populated before setting value
589     if (ttsSystemSelect && ttsSystemSelect.value === 'browser'
) {
590         populateBrowserVoices();
591     }
592
593     if (ttsVoiceSelect && settings.tts_voice) {
594         ttsVoiceSelect.value = settings.tts_voice;
595     }
596     if (instructionText && settings.system_instruction) instru
ctionText.value = settings.system_instruction;
597
598     toggleVoiceVisibility();
599
600     // Apply theme immediately
601     applyTheme(themeSelect?.value || 'dark');
602 } catch (e) {
603     console.error('Failed to load settings:', e);
604 }
605
```

```

606 form?.addEventListener('submit', async (e) => {
607     e.preventDefault();
608     try {
609         const payload = {
610             theme: themeSelect?.value || 'dark',
611             language: langSelect?.value || 'ru',
612             tts_enabled: ttsCheckbox?.checked ? 1 : 0,
613             tts_system: ttsSystemSelect?.value || 'edge-tts',
614             tts_voice: ttsVoiceSelect?.value || 'ru-RU-DmitryNeura
l',
615             system_instruction: instructionText?.value || ''
616         };
617         await window.api.fetch('/auth/settings', {
618             method: 'POST',
619             headers: { 'Content-Type': 'application/json' },
620             body: JSON.stringify(payload)
621         });
622         showNotification('Настройки сохранены', 'success');
623
624         // Apply theme immediately
625         applyTheme(payload.theme);
626
627         // Switch i18n language
628         if (window.switchLang) {
629             window.switchLang(payload.language);
630         }
631     } catch (e) {
632         showNotification('Ошибка сохранения настроек: ' + e.mess
age, 'danger');
633     }
634 });
635 }
636
637 // Apply theme immediately to body
638 function applyTheme(theme) {
639     if (theme === 'system') {
640         const prefersDark = window.matchMedia('(prefers-color-sche
me: dark)').matches;

```

```
641     theme = prefersDark ? 'dark' : 'light';
642 }
643
644 if (theme === 'light') {
645     document.body.style.background = '#f6f8fa';
646     document.body.style.color = '#24292f';
647 } else {
648     document.body.style.background = '#0d1117';
649     document.body.style.color = '#c9d1d9';
650 }
651 }
652
653 // — PLAYER MODULE —————
654
655 let mediaFiles = [];
656 let currentIndex = -1;
657
658 function initPlayer() {
659     const fileList = document.getElementById('file-list');
660     const video = document.getElementById('video');
661     const prevBtn = document.getElementById('btn-prev');
662     const nextBtn = document.getElementById('btn-next');
663
664     // Load media files
665     loadMediaFiles();
666
667     prevBtn?.addEventListener('click', () => playPrev());
668     nextBtn?.addEventListener('click', () => playNext());
669
670     // Handle video ended
671     video?.addEventListener('ended', () => playNext());
672
673     // Загрузка директории из localStorage
674     const savedDir = localStorage.getItem('torrent_download_dir'
675 );
676     if (savedDir) {
677         document.getElementById('download-dir').value = savedDir;
```

```
677 | }
678 |
679 | // Сохранение директории и обновление списка файлов
680 | document.getElementById('download-dir')?.addEventListener('c
| hange', (e) => {
681 |     localStorage.setItem('torrent_download_dir', e.target.valu
| e);
682 | });
683 |
684 | document.getElementById('btn-refresh-files')?.addEventListen
| er('click', () => {
685 |     loadMediaFiles();
686 |     showNotification('Список файлов обновлён', 'success');
687 | });
688 |
689 | initPlayerChatPopup();
690 | }
691 |
692 | function initPlayerChatPopup() {
693 |     console.log('initPlayerChatPopup: Инициализация...');
694 |     const chatToggleBtn = document.getElementById('chat-toggle-b
| tn');
695 |     const chatPopup = document.getElementById('chat-popup');
696 |     const chatCloseBtn = document.getElementById('chat-close-btn
| ');
697 |     const chatPopupSend = document.getElementById('chat-popup-se
| nd');
698 |     const chatPopupInput = document.getElementById('chat-popup-i
| nput');
699 |     const chatPopupMessages = document.getElementById('chat-popu
| p-messages');
700 |
701 |     console.log('initPlayerChatPopup: Найденные элементы:', {
702 |         chatToggleBtn: !!chatToggleBtn,
703 |         chatPopup: !!chatPopup,
704 |         chatCloseBtn: !!chatCloseBtn,
705 |         chatPopupSend: !!chatPopupSend,
706 |         chatPopupInput: !!chatPopupInput,
```

```
707     chatPopupMessages: !!chatPopupMessages
708   });
709
710   if (chatToggleBtn && chatPopup) {
711     chatToggleBtn.addEventListener('click', (e) => {
712       console.log('initPlayerChatPopup: Клик по кнопке чата');
713       e.preventDefault();
714       e.stopPropagation();
715       chatPopup.classList.toggle('show');
716       console.log('initPlayerChatPopup: Текущие классы окна ча
та:', chatPopup.className);
717       if (chatPopup.classList.contains('show')) {
718         chatPopupInput.focus();
719       }
720     });
721   } else {
722     console.error('initPlayerChatPopup: Кнопка или окно чата н
е найдены!');
723   }
724
725   if (chatCloseBtn && chatPopup) {
726     chatCloseBtn.addEventListener('click', (e) => {
727       console.log('initPlayerChatPopup: Клик по кнопке закрыти
я');
728       e.preventDefault();
729       e.stopPropagation();
730       chatPopup.classList.remove('show');
731     });
732   }
733
734   async function sendPopupMessage() {
735     const msg = chatPopupInput.value.trim();
736     if (!msg) return;
737
738     console.log('initPlayerChatPopup: Отправка сообщения:', ms
g);
739     addPopupMessage('user', msg);
740     chatPopupInput.value = '';
```

```

741     chatPopupSend.disabled = true;
742     chatPopupSend.textContent = ' ';
743
744     try {
745         const reply = await window.chatService.sendChatMessage(m
sg);
746         addPopupMessage('bot', reply);
747         window.chatService.speak(reply);
748     } catch (e) {
749         console.error('initPlayerChatPopup: Ошибка отправки:', e
);
750         addPopupMessage('bot', e);
751     } finally {
752         chatPopupSend.disabled = false;
753         chatPopupSend.textContent = 'Отправить';
754     }
755 }
756
757 function addPopupMessage(sender, text) {
758     const el = document.createElement('div');
759     el.className = `chat-popup-msg ${sender}`;
760
761     let displayText;
762     if (typeof text === 'object' && text !== null) {
763         displayText = window.chatService.formatMessage(text);
764     } else {
765         displayText = text;
766     }
767
768     el.innerHTML = `${sender === 'user' ? 'Вы' : 'Ai A
ссистент'}</strong>: ${displayText}`;
769     chatPopupMessages.appendChild(el);
770     chatPopupMessages.scrollTop = chatPopupMessages.scrollHeight;
771 }
772
773 if (chatPopupSend) {
774     chatPopupSend.addEventListener('click', sendPopupMessage);

```

```

775 | }
776 | if (chatPopupInput) {
777 |     chatPopupInput.addEventListener('keypress', (e) => {
778 |         if (e.key === 'Enter') sendPopupMessage();
779 |     });
780 | }
781 | }
782 |
783 | async function loadMediaFiles() {
784 |     try {
785 |         const files = await window.api.fetch('/api/media/files');
786 |         mediaFiles = files || [];
787 |         renderFileList();
788 |     } catch (e) {
789 |         console.error('Failed to load media files:', e);
790 |         document.getElementById('file-list').innerHTML =
791 |             '<div class="text-danger p-3" data-i18n="player.loadError">Ошибка загрузки файлов</div>';
792 |     }
793 | }
794 |
795 | function renderFileList() {
796 |     const list = document.getElementById('file-list');
797 |     if (!mediaFiles.length) {
798 |         list.innerHTML = '<div class="text-muted text-center p-3" data-i18n="player.noFiles">Нет файлов</div>';
799 |         return;
800 |     }
801 |
802 |     list.innerHTML = mediaFiles.map((file, i) => `
803 |         <div class="file-item ${i === currentIndex ? 'active' : ''}
804 |             <i class="bi bi-film"></i> ${escapeHtml(file.name || file.path)}
805 |         </div>
806 |     `).join('');
807 |
808 |     // Add click handlers

```

```
809 | list.querySelectorAll('.file-item').forEach(item => {
810 |     item.addEventListener('click', () => {
811 |         const idx = parseInt(item.dataset.index);
812 |         playFile(idx);
813 |     });
814 | });
815 | }
816 |
817 | function playFile(index) {
818 |     if (index < 0 || index >= mediaFiles.length) return;
819 |
820 |     currentIndex = index;
821 |     const file = mediaFiles[index];
822 |     const video = document.getElementById('video');
823 |
824 |     // Build streaming URL
825 |     const filePath = encodeURIComponent(file.path || file);
826 |     video.src = `/api/media/stream?path=${filePath}`;
827 |
828 |     document.getElementById('current-file').textContent = file.n
ame || file.path;
829 |     renderFileList();
830 |
831 |     video.play().catch(e => console.error('Play error:', e));
832 | }
833 |
834 | function playNext() {
835 |     if (currentIndex < mediaFiles.length - 1) {
836 |         playFile(currentIndex + 1);
837 |     }
838 | }
839 |
840 | function playPrev() {
841 |     if (currentIndex > 0) {
842 |         playFile(currentIndex - 1);
843 |     }
844 | }
845 |
```



```
846 // — CHAT MODULE
847
848 function initChat() {
849     const sendBtn = document.getElementById('send-button');
850     const input = document.getElementById('message-input');
851
852     sendBtn?.addEventListener('click', sendMessage);
853     input?.addEventListener('keypress', (e) => {
854         if (e.key === 'Enter' && !e.shiftKey) {
855             e.preventDefault();
856             sendMessage();
857         }
858     });
859
860     // Также слушаем входящие сообщения от бота
861     setupBotMessageHandler();
862 }
863
864 // Слушатель для перехвата сообщений от бота
865 function setupBotMessageHandler() {
866     const originalAddMessage = addMessage;
867     addMessage = function(text, sender, isError = false) {
868         originalAddMessage(text, sender, isError);
869
870         // Проверяем сообщения от бота на торрент-ссылки и теги <film>
871         if (sender === 'assistant' && !isError) {
872             checkAndProcessTorrentLinks(text);
873             checkAndProcessFilmTags(text);
874         }
875     };
876 }
877
878 // Проверка и обработка торрент-ссылок
879 async function checkAndProcessTorrentLinks(text) {
880     const links = findTorrentLinks(text);
881 }
```

```
882 |   if (links.length > 0) {
883 |       // Показываем кнопки для каждой найденной ссылки
884 |       for (const link of links) {
885 |           const shouldDownload = confirm(`Найдена торрент-ссылка:\n${link}\n\nДобавить на загрузку?`);
886 |
887 |           if (shouldDownload) {
888 |               // Загружаем в директорию по умолчанию
889 |               const downloadDir = localStorage.getItem('torrent_download_dir') || 'D:/Downloads';
890 |               await handleTorrentLink(link, downloadDir);
891 |           }
892 |       }
893 |   }
894 | }
895 |
896 | // Проверка и обработка тегов <film>
897 | async function checkAndProcessFilmTags(text) {
898 |     const filmMatches = [...text.matchAll(/<film>(.*?)<\/film>/gi)];
899 |
900 |     for (const match of filmMatches) {
901 |         const filmTitle = match[1];
902 |         if (confirm(`Запустить фильм/сериал в плеере: ${filmTitle}?`)) {
903 |             await launchFilm(filmTitle);
904 |         }
905 |     }
906 | }
907 |
908 | // Запуск фильма по названию
909 | async function launchFilm(title) {
910 |     try {
911 |         showNotification(`Поиск файла: ${title}...`, 'info');
912 |         const result = await window.api.fetch('/api/media/by-title', {
913 |             method: 'POST',
914 |             headers: { 'Content-Type': 'application/json' },
```

```
915     body: JSON.stringify({ title })
916   });
917
918   if (result.path) {
919     showNotification(`Запуск: ${title}`, 'success');
920     // Переключаем на плеер
921     document.querySelector('[data-bs-target="#tab-player"]')
922     ?.click();
923     // Ищем файл в списке и играем
924     const fileIndex = mediaFiles.findIndex(f => f.path === r
925     esult.path);
926     if (fileIndex !== -1) {
927       playFile(fileIndex);
928     } else {
929       // Если файла нет в текущем списке, пробуем просто заг
930       рузить этот путь как новый файл
931       await addMediaFileAndPlay({ path: result.path, name: t
932       itle });
933     }
934   } else {
935     showNotification(`Файл не найден: ${title}`, 'danger');
936   }
937 } catch (e) {
938   console.error('[Player] Launch error:', e);
939   showNotification('Ошибка запуска: ' + e.message, 'danger')
940 ;
941 }
942
943 async function sendMessage() {
944   const input = document.getElementById('message-input');
945   const chatWindow = document.getElementById('chat-window');
946   const message = input.value.trim();
947
948   if (!message) return;
949
950   // Clear input
951   input.value = '';
```

```
948 |
949 | // Hide empty message
950 | document.getElementById('chat-empty')?.remove();
951 |
952 | // Add user message
953 | addMessage(message, 'user');
954 |
955 | // Show typing indicator
956 | const typingId = addTypingIndicator();
957 |
958 | try {
959 |     const reply = await window.chatService.sendChatMessage(message);
960 |     removeTypingIndicator(typingId);
961 |     addMessage(reply, 'assistant');
962 |     window.chatService.speak(reply);
963 | } catch (e) {
964 |     removeTypingIndicator(typingId);
965 |     addMessage(e, 'assistant', true);
966 | }
967 | }
968 |
969 | window.openYouTubeForFilm = (title) => {
970 |     const q = encodeURIComponent(`${title} фильм трейлер`);
971 |     window.open(`https://www.youtube.com/results?search_query=${q}`, '_blank');
972 | };
973 |
974 | window.openOnlineCinemaForFilm = (title) => {
975 |     const q = encodeURIComponent(`${title} смотреть онлайн`);
976 |     window.open(`https://yandex.ru/video/search?text=${q}`, '_blank');
977 | };
978 |
979 | window.openKinopoiskForFilm = (title) => {
980 |     const q = encodeURIComponent(title);
981 |     window.open(`https://www.kinopoisk.ru/index.php?kp_query=${q}`, '_blank');
```

```

982 | };
983 |
984 | function replaceFilmTagsWithLinks(text) {
985 |     if (!text) return '';
986 |     return text.replace(/<film>(.*?)<\/film>/gi, (match, title)
    => {
987 |         const cleanTitle = title.replace(/'/g, "\\'");
988 |         return `<span class="film-interactive-chip" title="Film: $
    {title}">` +
989 |             `<span class="film-chip-title" onclick="launchFilm('${cl
    eanTitle}')">` +
990 |             `<i class="bi bi-film me-1"></i>${title}` +
991 |             `</span>` +
992 |             `<button type="button" class="film-chip-btn film-chip-pl
    ay" onclick="launchFilm('${cleanTitle}')" title="Play in media
    player">` +
993 |             `<i class="bi bi-play-fill"></i>` +
994 |             `</button>` +
995 |             `<button type="button" class="film-chip-btn film-chip-yo
    utube" onclick="window.openYouTubeForFilm('${cleanTitle}')" ti
    tle="Watch trailer/video on YouTube">` +
996 |             `<i class="bi bi-youtube"></i>` +
997 |             `</button>` +
998 |             `<button type="button" class="film-chip-btn film-chip-on
    line" onclick="window.openOnlineCinemaForFilm('${cleanTitle}')"
    " title="Watch online (Yandex Video / Cinemas)">` +
999 |             `<i class="bi bi-globe"></i>` +
1000 |             `</button>` +
1001 |             `</span>`;
1002 |     });
1003 | }
1004 |
1005 | function parseContentToHtml(text) {
1006 |     if (!text) return '';
1007 |     let html = text;
1008 |     if (window.marked && typeof window.marked.parse === 'functio
    n') {
1009 |         html = window.marked.parse(text);

```

```
1010 | } else {
1011 |     // Basic fallback for formatting if marked is not loaded
1012 |     html = text
1013 |         .replace(/\*\*(.*?)\*\*/g, '<strong>$1</strong>')
1014 |         .replace(/\*(.*?)\*/g, '<em>$1</em>')
1015 |         .replace(/\`([^\`]+)\`/g, '<code>$1</code>')
1016 |         .replace(/\n/g, '<br>');
1017 | }
1018 | return replaceFilmTagsWithLinks(html);
1019 | }
1020 |
1021 | function addMessage(text, sender, isError = false) {
1022 |     const chatWindow = document.getElementById('chat-window');
1023 |     const div = document.createElement('div');
1024 |     div.className = `chat-message ${sender}`;
1025 |     if (isError) div.style.color = '#f85149';
1026 |
1027 |     if (typeof text === 'object' && text !== null) {
1028 |         div.innerHTML = window.chatService.formatMessage(text);
1029 |     } else {
1030 |         div.innerHTML = parseContentToHtml(text);
1031 |     }
1032 |
1033 |     chatWindow.appendChild(div);
1034 |     chatWindow.scrollTop = chatWindow.scrollHeight;
1035 | }
1036 |
1037 | function addTypingIndicator() {
1038 |     const chatWindow = document.getElementById('chat-window');
1039 |     const id = 'typing-' + Date.now();
1040 |     const div = document.createElement('div');
1041 |     div.id = id;
1042 |     div.className = 'chat-message assistant';
1043 |     div.innerHTML = '<i class="bi bi-three-dots"></i> Печатает..';
1044 |     chatWindow.appendChild(div);
1045 |     chatWindow.scrollTop = chatWindow.scrollHeight;
1046 |     return id;
```

```
1047 | }
1048 |
1049 | function removeTypingIndicator(id) {
1050 |     document.getElementById(id)?.remove();
1051 | }
1052 |
1053 | function escapeHtml(text) {
1054 |     const div = document.createElement('div');
1055 |     div.textContent = text;
1056 |     return div.innerHTML;
1057 | }
1058 | // — SET CATEGORY FROM DB —————
1059 |
1060 | // Функция для вызова назначения категорий из интерфейса
1061 | async function assignCategoriesFromDB() {
1062 |     try {
1063 |         showNotification('Назначение категорий...', 'info');
1064 |         const result = await window.api.fetch('/api/torrents/assign-categories', { method: 'POST' });
1065 |         showNotification(result.result || 'Категории назначены', 'success');
1066 |     } catch (e) {
1067 |         showNotification('Ошибка: ' + e.message, 'danger');
1068 |     }
1069 | }
```

index.html

```
1 <!-- Users Management Tab -->
2 <div class="container-fluid py-2" id="users-tab-root">
3   <!-- Top Banner & Stats -->
4   <div class="d-flex flex-wrap align-items-center justify-content-between gap-3 mb-3 p-3 bg-dark border border-secondary rounded">
5     <div>
6       <h5 class="mb-1 text-white d-flex align-items-center gap-2">
7         <i class="bi bi-people-fill text-primary"></i>
8         <span>Управление пользователями</span>
9       </h5>
10      <p class="mb-0 text-muted small">Управление учетными записями, ролями, доступом к системе и сбросом паролей.</p>
11    </div>
12    <div class="d-flex gap-2">
13      <button class="btn btn-primary btn-sm d-flex align-items-center gap-1" id="btn-open-create-user">
14        <i class="bi bi-person-plus-fill"></i> Добавить пользователя
15      </button>
16      <button class="btn btn-outline-secondary btn-sm d-flex align-items-center gap-1" id="btn-refresh-users" title="Обновить список">
17        <i class="bi bi-arrow-clockwise"></i> Обновить
18      </button>
19    </div>
20  </div>
21
22 <!-- Summary Metric Cards -->
23 <div class="row g-3 mb-3">
24   <div class="col-6 col-md-3">
25     <div class="card bg-dark border-secondary h-100 shadow-sm">
26       <div class="card-body p-3 d-flex align-items-center justify-content-between">
```



```
27 |         <div>
28 |             <div class="text-muted small">Всего пользователей<
| /div>
29 |             <h4 class="mb-0 text-white fw-bold" id="stat-total
| -users">0</h4>
30 |         </div>
31 |         <div class="bg-secondary bg-opacity-25 rounded p-2 t
| ext-primary">
32 |             <i class="bi bi-people fs-4"></i>
33 |         </div>
34 |     </div>
35 | </div>
36 | </div>
37 | <div class="col-6 col-md-3">
38 |     <div class="card bg-dark border-secondary h-100 shadow-s
| m">
39 |         <div class="card-body p-3 d-flex align-items-center ju
| stify-content-between">
40 |             <div>
41 |                 <div class="text-muted small">Активные аккаунты</d
| iv>
42 |                 <h4 class="mb-0 text-success fw-bold" id="stat-act
| ive-users">0</h4>
43 |             </div>
44 |             <div class="bg-success bg-opacity-25 rounded p-2 tex
| t-success">
45 |                 <i class="bi bi-check-circle fs-4"></i>
46 |             </div>
47 |         </div>
48 |     </div>
49 | </div>
50 | <div class="col-6 col-md-3">
51 |     <div class="card bg-dark border-secondary h-100 shadow-s
| m">
52 |         <div class="card-body p-3 d-flex align-items-center ju
| stify-content-between">
53 |             <div>
54 |                 <div class="text-muted small">Администраторы</div>
```

```

55 |         <h4 class="mb-0 text-warning fw-bold" id="stat-admin-users">0</h4>
56 |     </div>
57 |     <div class="bg-warning bg-opacity-25 rounded p-2 text-warning">
58 |         <i class="bi bi-shield-lock fs-4"></i>
59 |     </div>
60 | </div>
61 | </div>
62 | </div>
63 | <div class="col-6 col-md-3">
64 |     <div class="card bg-dark border-secondary h-100 shadow-sm">
65 |         <div class="card-body p-3 d-flex align-items-center justify-content-between">
66 |             <div>
67 |                 <div class="text-muted small">Telegram привязано</div>
68 |                 <h4 class="mb-0 text-info fw-bold" id="stat-tg-users">0</h4>
69 |             </div>
70 |             <div class="bg-info bg-opacity-25 rounded p-2 text-info">
71 |                 <i class="bi bi-telegram fs-4"></i>
72 |             </div>
73 |         </div>
74 |     </div>
75 | </div>
76 | </div>
77 |
78 | <!-- Search & Filter Controls -->
79 | <div class="card bg-dark border-secondary mb-3 shadow-sm">
80 |     <div class="card-body p-3">
81 |         <div class="row g-2 align-items-center">
82 |             <div class="col-12 col-md-6 col-lg-5">
83 |                 <div class="input-group input-group-sm">
84 |                     <span class="input-group-text bg-dark border-secondary text-muted"><i class="bi bi-search"></i></span>

```

```
85 |         <input type="text" class="form-control bg-dark tex
| t-white border-secondary" id="users-search-input" placeholder=
| "Поиск по имени, email, Telegram...">
86 |         <button class="btn btn-outline-secondary d-none" t
| ype="button" id="users-search-clear">❌ </button>
87 |     </div>
88 | </div>
89 |     <div class="col-6 col-md-3 col-lg-3">
90 |         <select class="form-select form-select-sm bg-dark te
| xt-white border-secondary" id="users-filter-role">
91 |             <option value="">Все роли</option>
92 |             <option value="admin">Администраторы (admin)</opti
| on>
93 |             <option value="user">Пользователи (user)</option>
94 |             <option value="guest">Гости (guest)</option>
95 |         </select>
96 |     </div>
97 |     <div class="col-6 col-md-3 col-lg-2">
98 |         <select class="form-select form-select-sm bg-dark te
| xt-white border-secondary" id="users-filter-status">
99 |             <option value="">Все статусы</option>
100 |             <option value="active">Только активные</option>
101 |             <option value="inactive">Заблокированные</option>
102 |         </select>
103 |     </div>
104 |     <div class="col-12 col-lg-2 text-end text-muted small"
| id="users-count-badge">
105 |         Загрузка...
106 |     </div>
107 | </div>
108 | </div>
109 | </div>
110 |
111 | <!-- User Alert/Status message -->
112 | <div id="users-alert-box" class="d-none alert alert-dismissi
| ble fade show" role="alert">
113 |     <span id="users-alert-message"></span>
114 |     <button type="button" class="btn-close" aria-label="Close"
```

```

        onclick="document.getElementById('users-alert-box').classList
        .add('d-none')"></button>
115     </div>
116
117     <!-- Users Table Card -->
118     <div class="card bg-dark border-secondary shadow-sm">
119         <div class="card-body p-0">
120             <div class="table-responsive">
121                 <table class="table table-dark table-hover table-borde
red mb-0 align-middle">
122                     <thead class="table-secondary text-dark small text-u
ppercase">
123                         <tr>
124                             <th style="width: 50px;" class="text-center">ID<
/th>
125                             <th>Пользователь</th>
126                             <th style="width: 140px;" class="text-center">Po
ль</th>
127                             <th style="width: 130px;" class="text-center">Ст
атус</th>
128                             <th style="width: 150px;">Telegram</th>
129                             <th style="width: 120px;" class="text-center">Па
роль</th>
130                             <th style="width: 160px;">Регистрация / Вход</th>
131                             <th style="width: 170px;" class="text-center">Де
йствия</th>
132                         </tr>
133                     </thead>
134                     <tbody id="users-table-body">
135                         <tr>
136                             <td colspan="8" class="text-center py-4 text-mut
ed">
137                                 <div class="spinner-border spinner-border-sm t
ext-primary me-2" role="status"></div>
138                                 Загрузка списка пользователей...
139                             </td>
140                         </tr>

```

```

141         </tbody>
142     </table>
143 </div>
144 </div>
145 </div>
146 </div>
147
148 <!-- =====
149 ===== -->
150 <!-- MODAL: Create User -->
151 <!-- =====
152 ===== -->
151 <div class="modal fade" id="modal-create-user" tabindex="-1" a
152   ria-labelledby="modal-create-user-title" aria-hidden="true">
153     <div class="modal-dialog modal-dialog-centered">
154       <div class="modal-content bg-dark text-white border-second
155       ary">
156         <div class="modal-header border-secondary">
157           <h5 class="modal-title d-flex align-items-center gap-2
158           " id="modal-create-user-title">
159             <i class="bi bi-person-plus text-primary"></i> Добав
160             ить нового пользователя
161           </h5>
162           <button type="button" class="btn-close btn-close-white
163           " data-bs-dismiss="modal" aria-label="Close"></button>
164         </div>
165         <form id="form-create-user">
166           <div class="modal-body p-3">
167             <div class="mb-3">
168               <label class="form-label small text-light">Email <
169               span class="text-danger">*</span></label>
170               <input type="email" class="form-control bg-dark te
171               xt-white border-secondary" id="create-user-email" required pla
172               ceholder="user@example.com">
173             </div>
174             <div class="mb-3">
175               <label class="form-label small text-light">Отобраз
176               имое имя <span class="text-danger">*</span></label>

```

```
168 |         <input type="text" class="form-control bg-dark tex
    | t-white border-secondary" id="create-user-name" required place
    | holder="Иван Иванов">
169 |     </div>
170 |     <div class="mb-3">
171 |         <label class="form-label small text-light">Пароль
    | (опционально)</label>
172 |         <div class="input-group">
173 |             <input type="text" class="form-control bg-dark t
    | ext-white border-secondary" id="create-user-password" placehol
    | der="Оставьте пустым или задайте пароль">
174 |             <button class="btn btn-outline-secondary" type="
    | button" id="btn-gen-create-password" title="Сгенерировать безо
    | пасный пароль">
175 |                 <i class="bi bi-dice-5"></i>
176 |             </button>
177 |         </div>
178 |         <div class="form-text text-muted small">Если парол
    | ь не задан, пользователь сможет авторизоваться через Google ил
    | и Telegram.</div>
179 |     </div>
180 |     <div class="mb-3 p-3 bg-black bg-opacity-50 border b
    | order-secondary rounded">
181 |         <div class="form-check form-switch d-flex align-it
    | ems-center justify-content-between p-0 m-0">
182 |             <label class="form-check-label text-light fw-bol
    | d" for="create-user-is-admin">
183 |                 <i class="bi bi-shield-lock-fill text-warning
    | me-1"></i> Права Администратора
184 |             </label>
185 |             <input class="form-check-input ms-0 fs-5" type="
    | checkbox" role="switch" id="create-user-is-admin">
186 |         </div>
187 |         <div class="small text-muted mt-1" id="create-role
    | -hint">Роль: <strong class="text-info" id="create-role-label">
    | Пользователь (user)</strong></div>
188 |     </div>
189 |     <div class="row g-2 mb-2">
```

```

190         <div class="col-6">
191             <div class="form-check form-switch">
192                 <input class="form-check-input" type="checkbox"
193                 " role="switch" id="create-user-is-active" checked>
194                 <label class="form-check-label small" for="cre
195                 ate-user-is-active">Активен</label>
196             </div>
197         </div>
198         <div class="col-6">
199             <div class="form-check form-switch">
200                 <input class="form-check-input" type="checkbox"
201                 " role="switch" id="create-user-verified" checked>
202                 <label class="form-check-label small" for="cre
203                 ate-user-verified">Email подтвержден</label>
204             </div>
205         </div>
206     </div>
207     <div class="modal-footer border-secondary">
208         <button type="button" class="btn btn-secondary btn-s
209         m" data-bs-dismiss="modal">Отмена</button>
210         <button type="submit" class="btn btn-primary btn-sm"
211         id="btn-submit-create-user">Создать пользователя</button>
212     </div>
213 </form>
214 </div>
215 </div>
216 <!-- =====
217 ===== -->
218 <!-- MODAL: Edit User -->
219 <!-- =====
220 ===== -->
221 <div class="modal fade" id="modal-edit-user" tabindex="-1" ari
222     a-labelledby="modal-edit-user-title" aria-hidden="true">
223     <div class="modal-dialog modal-dialog-centered">
224         <div class="modal-content bg-dark text-white border-second

```

```

    ary">
219     <div class="modal-header border-secondary">
220         <h5 class="modal-title d-flex align-items-center gap-2
    " id="modal-edit-user-title">
221             <i class="bi bi-pencil-square text-warning"></i> Ред
    активирование пользователя #<span id="edit-user-id-badge"></span
    >
222         </h5>
223         <button type="button" class="btn-close btn-close-white
    " data-bs-dismiss="modal" aria-label="Close"></button>
224     </div>
225     <form id="form-edit-user">
226         <input type="hidden" id="edit-user-id">
227         <div class="modal-body p-3">
228             <div class="mb-3">
229                 <label class="form-label small text-light">Email <
    span class="text-danger">*</span></label>
230                 <input type="email" class="form-control bg-dark te
    xt-white border-secondary" id="edit-user-email" required>
231             </div>
232             <div class="mb-3">
233                 <label class="form-label small text-light">Имя <sp
    an class="text-danger">*</span></label>
234                 <input type="text" class="form-control bg-dark tex
    t-white border-secondary" id="edit-user-name" required>
235             </div>
236             <div class="mb-3 p-3 bg-black bg-opacity-50 border b
    order-secondary rounded">
237                 <div class="form-check form-switch d-flex align-it
    ems-center justify-content-between p-0 m-0">
238                     <label class="form-check-label text-light fw-bol
    d" for="edit-user-is-admin">
239                         <i class="bi bi-shield-lock-fill text-warning
    me-1"></i> Права Администратора
240                     </label>
241                     <input class="form-check-input ms-0 fs-5" type="
    checkbox" role="switch" id="edit-user-is-admin">
242                 </div>

```



```

243 |         <div class="small text-muted mt-1" id="edit-role-h
| int">Роль: <strong class="text-info" id="edit-role-label">Поль
| зователь (user)</strong></div>
244 |     </div>
245 |     <div class="row g-2 mb-2">
246 |         <div class="col-6">
247 |             <div class="form-check form-switch">
248 |                 <input class="form-check-input" type="checkbox
| " role="switch" id="edit-user-is-active">
249 |                 <label class="form-check-label small" for="edi
| t-user-is-active">Активен</label>
250 |             </div>
251 |         </div>
252 |         <div class="col-6">
253 |             <div class="form-check form-switch">
254 |                 <input class="form-check-input" type="checkbox
| " role="switch" id="edit-user-verified">
255 |                 <label class="form-check-label small" for="edi
| t-user-verified">Email подтвержден</label>
256 |             </div>
257 |         </div>
258 |     </div>
259 | </div>
260 | <div class="modal-footer border-secondary">
261 |     <button type="button" class="btn btn-secondary btn-s
| m" data-bs-dismiss="modal">Отмена</button>
262 |     <button type="submit" class="btn btn-warning btn-sm"
| id="btn-submit-edit-user">Сохранить изменения</button>
263 | </div>
264 | </form>
265 | </div>
266 | </div>
267 | </div>
268 |
269 | <!-- =====
| ===== -->
270 | <!-- MODAL: Set / Reset Password -->
271 | <!-- =====

```

```

===== -->
272 <div class="modal fade" id="modal-password-user" tabindex="-1"
    aria-labelledby="modal-password-user-title" aria-hidden="true
    ">
273     <div class="modal-dialog modal-dialog-centered">
274         <div class="modal-content bg-dark text-white border-second
    ary">
275             <div class="modal-header border-secondary">
276                 <h5 class="modal-title d-flex align-items-center gap-2
    " id="modal-password-user-title">
277                     <i class="bi bi-key-fill text-info"></i> Смена парол
    я пользователя
278                 </h5>
279                 <button type="button" class="btn-close btn-close-white
    " data-bs-dismiss="modal" aria-label="Close"></button>
280             </div>
281             <form id="form-password-user">
282                 <input type="hidden" id="pwd-user-id">
283                 <div class="modal-body p-3">
284                     <p class="small text-muted mb-3">
285                         Установка нового пароля для <strong class="text-wh
    ite" id="pwd-user-target"></strong>
286                     </p>
287                     <div class="mb-3">
288                         <label class="form-label small text-light">Новый п
    ароль <span class="text-danger">*</span></label>
289                         <div class="input-group">
290                             <input type="text" class="form-control bg-dark t
    ext-white border-secondary font-monospace" id="pwd-input-val"
    required minlength="4" placeholder="Введите пароль">
291                             <button class="btn btn-outline-info" type="butto
    n" id="btn-gen-pwd-val" title="Сгенерировать безопасный пароль
    ">
292                                 <i class="bi bi-dice-5"></i> Сгенерировать
293                             </button>
294                         </div>
295                     </div>
296                 </div>

```

```

297 |         <div class="modal-footer border-secondary">
298 |             <button type="button" class="btn btn-secondary btn-s
| m" data-bs-dismiss="modal">Отмена</button>
299 |             <button type="submit" class="btn btn-info btn-sm" id
| ="btn-submit-pwd-user">Установить пароль</button>
300 |         </div>
301 |     </form>
302 | </div>
303 | </div>
304 | </div>
305 |
306 | <!-- =====
| ===== -->
307 | <!-- MODAL: User Details & Profile Settings -->
308 | <!-- =====
| ===== -->
309 | <div class="modal fade" id="modal-details-user" tabindex="-1"
| aria-labelledby="modal-details-user-title" aria-hidden="true">
310 |     <div class="modal-dialog modal-dialog-centered modal-lg">
311 |         <div class="modal-content bg-dark text-white border-second
| ary">
312 |             <div class="modal-header border-secondary">
313 |                 <h5 class="modal-title d-flex align-items-center gap-2
| " id="modal-details-user-title">
314 |                     <i class="bi bi-person-badge text-primary"></i> Проф
| иль и настройки пользователя
315 |                 </h5>
316 |                 <button type="button" class="btn-close btn-close-white
| " data-bs-dismiss="modal" aria-label="Close"></button>
317 |             </div>
318 |             <div class="modal-body p-3" id="details-user-content">
319 |                 <div class="text-center py-3 text-muted">Загрузка данн
| ых...</div>
320 |             </div>
321 |             <div class="modal-footer border-secondary">
322 |                 <button type="button" class="btn btn-secondary btn-sm"
| data-bs-dismiss="modal">Закрыть</button>
323 |             </div>

```

```

324     </div>
325 </div>
326 </div>
327
328 <!-- =====
===== -->
329 <!-- MODAL: Delete Confirmation -->
330 <!-- =====
===== -->
331 <div class="modal fade" id="modal-delete-user" tabindex="-1" a
ria-labelledby="modal-delete-user-title" aria-hidden="true">
332     <div class="modal-dialog modal-dialog-centered">
333         <div class="modal-content bg-dark text-white border-danger
">
334             <div class="modal-header border-danger">
335                 <h5 class="modal-title text-danger d-flex align-items-
center gap-2" id="modal-delete-user-title">
336                     <i class="bi bi-exclamation-triangle-fill"></i> Подт
верждение удаления
337                 </h5>
338                 <button type="button" class="btn-close btn-close-white
" data-bs-dismiss="modal" aria-label="Close"></button>
339             </div>
340             <div class="modal-body p-3">
341                 <p class="mb-2">Вы действительно хотите удалить пользо
вателя <strong class="text-danger" id="delete-user-name-target
"></strong>?</p>
342                 <p class="small text-muted mb-0">Это действие удалит у
четную запись, настройки и сессии пользователя. Операция необр
атима.</p>
343                 <input type="hidden" id="delete-user-id">
344             </div>
345             <div class="modal-footer border-secondary">
346                 <button type="button" class="btn btn-secondary btn-sm"
data-bs-dismiss="modal">Отмена</button>
347                 <button type="button" class="btn btn-danger btn-sm" id
="btn-confirm-delete-user">Удалить пользователя</button>
348             </div>

```

```
349 |     </div>
350 |   </div>
351 | </div>
352 |
```

main.js

```
1 // =====
  =====
2 // Webinterface: Users Tab Logic
3 // Module: webinterface/users_tab/main.js
4 // Author: hypo69
5 // Copyright: © 2026 hypo69
6 // =====
  =====
7
8 'use strict';
9
10 (function () {
11     const state = {
12         users: [],
13         stats: {
14             total: 0,
15             active: 0,
16             admins: 0,
17             telegram: 0
18         },
19         filterRole: '',
20         filterStatus: '',
21         searchQuery: '',
22         isLoading: false,
23         initialized: false
24     };
25
26     // Helper API fetch
27     async function apiFetch(url, options = {}) {
28         if (window.api && typeof window.api.fetch === 'function')
29         {
30             return window.api.fetch(url, options);
31         }
32         const res = await fetch(url, options);
33         if (!res.ok) {
34             let errMsg = `HTTP ${res.status}`;
```

```

34 |     try {
35 |         const errData = await res.json();
36 |         if (errData && errData.detail) {
37 |             errMsg = typeof errData.detail === 'string' ? errData.detail : JSON.stringify(errData.detail);
38 |         }
39 |     } catch (_) {}
40 |     throw new Error(errMsg);
41 | }
42 | return res.json();
43 | }
44 |
45 | // Show notification
46 | function showStatusAlert(msg, type = 'info') {
47 |     const alertBox = document.getElementById('users-alert-box');
48 |     const alertMsg = document.getElementById('users-alert-message');
49 |     if (!alertBox || !alertMsg) return;
50 |
51 |     alertBox.className = `alert alert-${type} alert-dismissible fade show mb-3`;
52 |     alertMsg.innerHTML = msg;
53 |     alertBox.classList.remove('d-none');
54 |
55 |     setTimeout(() => {
56 |         alertBox.classList.add('d-none');
57 |     }, 5000);
58 | }
59 |
60 | // Generate secure password
61 | function generatePassword(length = 12) {
62 |     const charset = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#$%^&*';
63 |     let pwd = '';
64 |     const randomValues = new Uint32Array(length);
65 |     window.crypto.getRandomValues(randomValues);
66 |     for (let i = 0; i < length; i++) {

```

```

67     pwd += charset[randomValues[i] % charset.length];
68 }
69 return pwd;
70 }
71
72 // Escape HTML
73 function escapeHtml(str) {
74     if (!str) return '';
75     return String(str)
76         .replace(/&/g, '&amp;')
77         .replace(/</g, '&lt;')
78         .replace(/>/g, '&gt;')
79         .replace(/"/g, '&quot;')
80         .replace(/'/g, '&#039;');
81 }
82
83 // Format datetime
84 function formatDate(isoStr) {
85     if (!isoStr) return '<span class="text-muted">--</span>';
86     try {
87         const d = new Date(isoStr);
88         if (isNaN(d.getTime())) return escapeHtml(isoStr);
89         return `<span title="${escapeHtml(isoStr)}">${d.toLocale
DateString('ru-RU')} <small class="text-muted">${d.toLocaleTim
eString('ru-RU', { hour: '2-digit', minute: '2-digit' })}</sma
ll></span>`;
90     } catch (_) {
91         return escapeHtml(isoStr);
92     }
93 }
94
95 // Load Users List from Backend
96 async function loadUsers() {
97     state.isLoading = true;
98     const tableBody = document.getElementById('users-table-bod
y');
99     const countBadge = document.getElementById('users-count-ba
dge');

```



```
100 |
101 |     if (tableBody && (!state.users || state.users.length === 0
    | )) {
102 |         tableBody.innerHTML = `
103 |             <tr>
104 |                 <td colspan="8" class="text-center py-4 text-muted">
105 |                     <div class="spinner-border spinner-border-sm text-
    | primary me-2" role="status"></div>
106 |                     Загрузка списка пользователей...
107 |                 </td>
108 |             </tr>`;
109 |     }
110 |
111 |     try {
112 |         const params = new URLSearchParams();
113 |         if (state.searchQuery) params.append('q', state.searchQu
    | ery);
114 |         if (state.filterRole) params.append('role', state.filter
    | Role);
115 |         if (state.filterStatus) params.append('status', state.fi
    | lterStatus);
116 |
117 |         const url = `/api/admin/users?${params.toString()}`;
118 |         const data = await apiFetch(url);
119 |
120 |         state.users = data.users || [];
121 |         if (data.stats) {
122 |             state.stats = data.stats;
123 |         }
124 |
125 |         renderStats();
126 |         renderTable();
127 |     } catch (err) {
128 |         console.error('[UsersTab] Error loading users:', err);
129 |         showStatusAlert(`Ошибка загрузки пользователей: ${err.me
    | ssage}`, 'danger');
130 |         if (tableBody) {
131 |             tableBody.innerHTML = `
```

```

132 |         <tr>
133 |             <td colspan="8" class="text-center py-4 text-dange
134 | r">
135 |                 <i class="bi bi-exclamation-octagon fs-4 d-block
136 | mb-1"></i>
137 |                 Не удалось загрузить пользователей: ${escapeHtml
138 | (err.message)}
139 |             </td>
140 |         </tr>`;
141 |     }
142 | } finally {
143 |     state.isLoading = false;
144 | }
145 | // Render Metric Cards
146 | function renderStats() {
147 |     const totalEl = document.getElementById('stat-total-users'
148 | );
149 |     const activeEl = document.getElementById('stat-active-user
150 | s');
151 |     const adminEl = document.getElementById('stat-admin-users'
152 | );
153 |     const tgEl = document.getElementById('stat-tg-users');
154 |
155 |     if (totalEl) totalEl.textContent = state.stats.total || 0;
156 |     if (activeEl) activeEl.textContent = state.stats.active ||
157 | 0;
158 |     if (adminEl) adminEl.textContent = state.stats.admins || 0
159 | ;
160 |     if (tgEl) tgEl.textContent = state.stats.telegram || 0;
161 | }
162 |
163 | // Render Users Table
164 | function renderTable() {
165 |     const tableBody = document.getElementById('users-table-bod
166 | y');
167 |     const countBadge = document.getElementById('users-count-ba

```

```

    dge');
161     if (!tableBody) return;
162
163     if (countBadge) {
164         countBadge.textContent = `Показано: ${state.users.length
    } из ${state.stats.total || state.users.length}`;
165     }
166
167     if (state.users.length === 0) {
168         tableBody.innerHTML = `
169             <tr>
170                 <td colspan="8" class="text-center py-5 text-muted">
171                     <i class="bi bi-person-x fs-2 d-block mb-2 text-se
    condary"></i>
172                     Пользователи не найдены.
173                 </td>
174             </tr>`;
175         return;
176     }
177
178     tableBody.innerHTML = state.users.map((user) => {
179         const isRoot = user.id === 1;
180         const isAdmin = Boolean(user.is_admin || user.role === '
    admin');
181         const isActive = Boolean(user.is_active);
182         const isEmailVerified = Boolean(user.is_email_verified);
183         const hasPassword = Boolean(user.has_password);
184
185         // User Initials or Avatar
186         let avatarHtml = '';
187         if (user.picture) {
188             avatarHtml = ``;
189         } else {
190             const initial = (user.name || user.email || 'U').charA
    t(0).toUpperCase();
191             const bgClass = isAdmin ? 'bg-warning text-dark' : 'bg

```

```

-primary text-white';
192     avatarHtml = `
```

```

213
214     // Telegram column
215     let tgHtml = '<span class="text-muted small">—</span>';
216     if (user.telegram_username) {
217         tgHtml = `<a href="https://t.me/${escapeHtml(user.tele
gram_username)}" target="_blank" class="text-info text-decorat
ion-none small d-flex align-items-center gap-1">
218             <i class="bi bi-telegram"></i> @${escapeHtml(user.te
legram_username)}
219             </a>`;
220     } else if (user.telegram_id) {
221         tgHtml = `<span class="text-muted small"><i class="bi
bi-telegram text-info"></i> ID: ${escapeHtml(user.telegram_id)
}</span>`;
222     }
223
224     // Password column
225     const pwdHtml = hasPassword
226         ? `<span class="badge bg-dark border border-success te
xt-success" title="Пароль установлен"><i class="bi bi-key-fill
"></i> Задан</span>`
227         : `<span class="badge bg-dark border border-warning te
xt-warning" title="Пароль не установлен (OAuth/TG)"><i class="
bi bi-dash-circle"></i> Нет</span>`;
228
229     // Email verified badge
230     const emailVerifiedBadge = isEmailVerified
231         ? `<i class="bi bi-patch-check-fill text-success ms-1"
title="Email подтвержден"></i>`
232         : `<i class="bi bi-question-circle text-muted ms-1" ti
tle="Email не подтвержден"></i>`;
233
234     return `
235         <tr data-user-id="${user.id}">
236             <td class="text-center text-muted fw-bold">${user.id
}</td>
237             <td>
238                 <div class="d-flex align-items-center">

```

```

239 |         ${avatarHtml}
240 |         <div>
241 |             <div class="fw-bold text-white">${escapeHtml(u
ser.name || 'Без имени')}</div>
242 |             <div class="small text-muted d-flex align-item
s-center">
243 |                 ${escapeHtml(user.email)} ${emailVerifiedBad
ge}
244 |             </div>
245 |         </div>
246 |     </div>
247 | </td>
248 | <td class="text-center">${roleSwitchHtml}</td>
249 | <td class="text-center">
250 |     <div>${statusBadge}</div>
251 | </td>
252 | <td>${tgHtml}</td>
253 | <td class="text-center">${pwdHtml}</td>
254 | <td class="small">
255 |     <div><span class="text-muted">Создан:</span> ${for
matDate(user.created_at)}</div>
256 |     ${user.last_login ? `<div><span class="text-muted"
>Вход:</span> ${formatDate(user.last_login)}</div>` : ''}
257 | </td>
258 | <td class="text-center">
259 |     <div class="btn-group btn-group-sm" role="group">
260 |         <button class="btn btn-outline-secondary btn-edi
t-user" data-id="${user.id}" title="Редактировать">
261 |             <i class="bi bi-pencil-fill"></i>
262 |         </button>
263 |         <button class="btn btn-outline-info btn-pwd-user
" data-id="${user.id}" data-name="${escapeHtml(user.name || us
er.email)}" title="Сменить пароль">
264 |             <i class="bi bi-key-fill"></i>
265 |         </button>
266 |         <button class="btn btn-outline-light btn-details
-user" data-id="${user.id}" title="Настройки и профиль">
267 |             <i class="bi bi-info-circle-fill"></i>

```

```

268 |         </button>
269 |         <button class="btn ${isActive ? 'btn-outline-war
| ning' : 'btn-outline-success'} btn-toggle-active-user" data-id
| ="${user.id}" data-active="${isActive ? '1' : '0'}" ${isRoot ?
| 'disabled title="Нельзя деактивировать Root"' : `title="${isA
| ctive ? 'Заблокировать' : 'Активировать'}"``}>
270 |             <i class="bi ${isActive ? 'bi-lock-fill' : 'bi
| -unlock-fill'}"></i>
271 |         </button>
272 |         <button class="btn btn-outline-danger btn-delete
| -user" data-id="${user.id}" data-name="${escapeHtml(user.name
| || user.email)}" ${isRoot ? 'disabled title="Нельзя удалить Ro
| ot"' : 'title="Удалить"}">
273 |             <i class="bi bi-trash-fill"></i>
274 |         </button>
275 |     </div>
276 | </td>
277 | </tr>`;
278 | }).join('');
279 |
280 | attachTableEvents();
281 | }
282 |
283 | // Attach Table Action Button Events
284 | function attachTableEvents() {
285 |     // Role Toggle Switch
286 |     document.querySelectorAll('.role-toggle-switch').forEach(s
| w => {
287 |         sw.addEventListener('change', async (e) => {
288 |             const userId = parseInt(e.target.getAttribute('data-us
| er-id'), 10);
289 |             await handleToggleRole(userId, e.target);
290 |         });
291 |     });
292 |
293 |     // Edit User
294 |     document.querySelectorAll('.btn-edit-user').forEach(btn =>
| {

```

```
295 |         btn.addEventListener('click', (e) => {
296 |             const userId = parseInt(e.currentTarget.getAttribute('
data-id'), 10);
297 |             openEditModal(userId);
298 |         });
299 |     });
300 |
301 |     // Change Password
302 |     document.querySelectorAll('.btn-pwd-user').forEach(btn =>
{
303 |         btn.addEventListener('click', (e) => {
304 |             const userId = parseInt(e.currentTarget.getAttribute('
data-id'), 10);
305 |             const userName = e.currentTarget.getAttribute('data-na
me');
306 |             openPasswordModal(userId, userName);
307 |         });
308 |     });
309 |
310 |     // View Details
311 |     document.querySelectorAll('.btn-details-user').forEach(btn
=> {
312 |         btn.addEventListener('click', (e) => {
313 |             const userId = parseInt(e.currentTarget.getAttribute('
data-id'), 10);
314 |             openDetailsModal(userId);
315 |         });
316 |     });
317 |
318 |     // Toggle Active
319 |     document.querySelectorAll('.btn-toggle-active-user').forEa
ch(btn => {
320 |         btn.addEventListener('click', async (e) => {
321 |             const userId = parseInt(e.currentTarget.getAttribute('
data-id'), 10);
322 |             await handleToggleActive(userId);
323 |         });
324 |     });
```



```
325 |
326 |     // Delete User
327 |     document.querySelectorAll('.btn-delete-user').forEach(btn
    => {
328 |         btn.addEventListener('click', (e) => {
329 |             const userId = parseInt(e.currentTarget.getAttribute('
data-id'), 10);
330 |             const userName = e.currentTarget.getAttribute('data-na
me');
331 |             openDeleteModal(userId, userName);
332 |         });
333 |     });
334 | }
335 |
336 | // Toggle Role Handler
337 | async function handleToggleRole(userId, switchEl) {
338 |     const originalChecked = !switchEl.checked;
339 |     try {
340 |         const res = await apiFetch(`/api/admin/users/${userId}/t
oggle-role`, { method: 'POST' });
341 |         const user = state.users.find(u => u.id === userId);
342 |         if (user) {
343 |             user.role = res.role;
344 |             user.is_admin = res.is_admin;
345 |         }
346 |         const roleText = res.is_admin ? 'Администратор (admin)'
: 'Пользователь (user)';
347 |         showStatusAlert(`Роль пользователя #${userId} изменена н
а: <strong>${roleText}</strong>`, 'success');
348 |         loadUsers();
349 |     } catch (err) {
350 |         if (switchEl) switchEl.checked = originalChecked;
351 |         showStatusAlert(`Ошибка изменения роли: ${err.message}`,
'danger');
352 |     }
353 | }
354 |
355 | // Open Edit Modal
```

```

356 function openEditModal(userId) {
357     const user = state.users.find(u => u.id === userId);
358     if (!user) return;
359
360     const isEditAdmin = Boolean(user.is_admin || user.role ===
    'admin');
361     document.getElementById('edit-user-id').value = user.id;
362     document.getElementById('edit-user-id-badge').textContent
    = user.id;
363     document.getElementById('edit-user-email').value = user.em
    ail || '';
364     document.getElementById('edit-user-name').value = user.nam
    e || '';
365
366     const editAdminSw = document.getElementById('edit-user-is-
    admin');
367     const editRoleLabel = document.getElementById('edit-role-l
    abel');
368     if (editAdminSw) {
369         editAdminSw.checked = isEditAdmin;
370         editAdminSw.disabled = (userId === 1);
371         if (editRoleLabel) {
372             editRoleLabel.textContent = isEditAdmin ? 'Администрат
    оп (admin)' : 'Пользователь (user)';
373             editRoleLabel.className = isEditAdmin ? 'text-warning'
    : 'text-info';
374         }
375     }
376
377     document.getElementById('edit-user-is-active').checked = B
    oolean(user.is_active);
378     document.getElementById('edit-user-verified').checked = Bo
    olean(user.is_email_verified);
379
380     const modalEl = document.getElementById('modal-edit-user')
    ;
381     const modal = bootstrap.Modal.getOrCreateInstance(modalEl)
    ;

```

```
382     modal.show();
383 }
384
385 // Open Password Modal
386 function openPasswordModal(userId, userName) {
387     document.getElementById('pwd-user-id').value = userId;
388     document.getElementById('pwd-user-target').textContent = `
389     ${userName} (ID #${userId})`;
390     document.getElementById('pwd-input-val').value = '';
391     const modalEl = document.getElementById('modal-password-us
392     er');
393     const modal = bootstrap.Modal.getOrCreateInstance(modalEl)
394     ;
395     modal.show();
396 }
397
398 // Open Details Modal
399 async function openDetailsModal(userId) {
400     const modalEl = document.getElementById('modal-details-use
401     r');
402     const contentEl = document.getElementById('details-user-co
403     ntent');
404     const modal = bootstrap.Modal.getOrCreateInstance(modalEl)
405     ;
406     modal.show();
407
408     contentEl.innerHTML = `
409     <div class="text-center py-4 text-muted">
410     <div class="spinner-border spinner-border-sm text-prim
411     ary me-2" role="status"></div>
412     Загрузка профиля пользователя...
413     </div>`;
414
415     try {
416         const data = await apiFetch(`/api/admin/users/${userId}`
417         );
418         const u = data.user || {};
```

```

412     const s = data.settings || {};
413     const perms = data.permissions || [];
414
415     contentEl.innerHTML = `
416         <div class="row g-3">
417             <div class="col-md-6">
418                 <h6 class="text-primary border-bottom border-secondary pb-1 mb-2">❑ Основные данные</h6>
419                 <table class="table table-dark table-sm table-borderless small mb-0">
420                     <tr><td class="text-muted" style="width:120px;">ID:</td><td class="fw-bold">${u.id}</td></tr>
421                     <tr><td class="text-muted">Email:</td><td>${escapeHtml(u.email)} ${u.is_email_verified ? '<span class="badge bg-success">Verified</span>' : '<span class="badge bg-secondary">Unverified</span>'}</td></tr>
422                     <tr><td class="text-muted">Имя:</td><td>${escapeHtml(u.name || '-')}</td></tr>
423                     <tr><td class="text-muted">Роль:</td><td><span class="badge bg-info text-dark">${escapeHtml(u.role || 'user')}</span> ${u.is_admin ? '<span class="badge bg-warning text-dark">Admin</span>' : ''}</td></tr>
424                     <tr><td class="text-muted">Статус:</td><td>${u.is_active ? '<span class="text-success">Активен</span>' : '<span class="text-danger">Заблокирован</span>'}</td></tr>
425                     <tr><td class="text-muted">Пароль:</td><td>${u.has_password ? '<span class="text-success">Установлен</span>' : '<span class="text-muted">Не задан</span>'}</td></tr>
426                     <tr><td class="text-muted">Создан:</td><td>${escapeHtml(u.created_at || '-')}</td></tr>
427                     <tr><td class="text-muted">Последний вход:</td><td>${escapeHtml(u.last_login || '-')}</td></tr>
428                 </table>
429             </div>
430             <div class="col-md-6">
431                 <h6 class="text-info border-bottom border-secondary pb-1 mb-2">❑ Telegram & Настройки</h6>
432                 <table class="table table-dark table-sm table-borderless small mb-0">

```

```

erless small mb-0">
433         <tr><td class="text-muted" style="width:130px;">
Telegram ID:</td><td>${u.telegram_id ? escapeHtml(u.telegram_id) : '<span class="text-muted">Не привязан</span>'}</td></tr>
434         <tr><td class="text-muted">TG Username:</td><td>
${u.telegram_username ? `@${escapeHtml(u.telegram_username)}` : '<span class="text-muted">—</span>'}</td></tr>
435         <tr><td class="text-muted">Тема UI:</td><td>${escapeHtml(s.theme || 'dark')}</td></tr>
436         <tr><td class="text-muted">Язык:</td><td>${escapeHtml(s.language || 'ru')}</td></tr>
437         <tr><td class="text-muted">TTS голос:</td><td>${escapeHtml(s.tts_voice || 'ru-RU-DmitryNeural')} (${escapeHtml(s.tts_system || 'edge-tts')})</td></tr>
438         <tr><td class="text-muted">Модель чата:</td><td>
${escapeHtml(s.model || 'По умолчанию')}</td></tr>
439         </table>
440     </div>
441     <div class="col-12 mt-3">
442         <h6 class="text-warning border-bottom border-secondary pb-1 mb-2">□ Разрешения системы</h6>
443         <div class="d-flex flex-wrap gap-1">
444             ${perms.length > 0 ? perms.map(p => `<span class="badge bg-secondary">${escapeHtml(p)}</span>`).join('') : '<span class="text-muted small">Нет явных разрешений</span>'}
445         </div>
446     </div>
447     ${s.system_instruction ? `
448     <div class="col-12 mt-2">
449         <h6 class="text-light border-bottom border-secondary pb-1 mb-1">□ Пользовательская системная инструкция</h6>
450         <pre class="bg-black p-2 rounded border border-secondary text-info small" style="max-height:120px;overflow-y:auto;">${escapeHtml(s.system_instruction)}</pre>
451     </div>` : ''}
452     </div>`;
453 } catch (err) {
454     contentEl.innerHTML = `<div class="text-danger py-3">Ошиб

```

```

    бка загрузки данных: ${escapeHtml(err.message)}</div>`;
455     }
456   }
457
458   // Open Delete Modal
459   function openDeleteModal(userId, userName) {
460     document.getElementById('delete-user-id').value = userId;
461     document.getElementById('delete-user-name-target').textCon
tent = `${userName} (ID #${userId})`;
462
463     const modalEl = document.getElementById('modal-delete-user
');
464     const modal = bootstrap.Modal.getOrCreateInstance(modalEl)
;
465     modal.show();
466   }
467
468   // Toggle Active Handler
469   async function handleToggleActive(userId) {
470     try {
471       const res = await apiFetch(`/api/admin/users/${userId}/t
oggle-active`, { method: 'POST' });
472       const user = state.users.find(u => u.id === userId);
473       if (user) {
474         user.is_active = res.is_active;
475         renderTable();
476       }
477       showStatusAlert(`Статус пользователя #${userId} изменён
на: ${res.is_active ? 'Активен' : 'Заблокирован'}`, 'success')
;
478       loadUsers();
479     } catch (err) {
480       showStatusAlert(`Ошибка изменения статуса: ${err.message
}`, 'danger');
481     }
482   }
483
484   // Initialize Event Listeners

```

```
485 | function initListeners() {
486 |     // Open Create User Modal
487 |     const btnOpenCreate = document.getElementById('btn-open-cr
    eate-user');
488 |     if (btnOpenCreate) {
489 |         btnOpenCreate.addEventListener('click', () => {
490 |             const form = document.getElementById('form-create-user
    ');
491 |             if (form) form.reset();
492 |             document.getElementById('create-user-is-admin').checke
    d = false;
493 |             const createRoleLabel = document.getElementById('creat
    e-role-label');
494 |             if (createRoleLabel) {
495 |                 createRoleLabel.textContent = 'Пользователь (user)';
496 |                 createRoleLabel.className = 'text-info';
497 |             }
498 |             document.getElementById('create-user-is-active').check
    ed = true;
499 |             document.getElementById('create-user-verified').checke
    d = true;
500 |             const modalEl = document.getElementById('modal-create-
    user');
501 |             const modal = bootstrap.Modal.getOrCreateInstance(modal
    El);
502 |             modal.show();
503 |         });
504 |     }
505 |
506 |     // Dynamic Switch Hints
507 |     const createAdminSw = document.getElementById('create-user
    -is-admin');
508 |     const createRoleLabel = document.getElementById('create-ro
    le-label');
509 |     if (createAdminSw && createRoleLabel) {
510 |         createAdminSw.addEventListener('change', (e) => {
511 |             createRoleLabel.textContent = e.target.checked ? 'Адми
    нистратор (admin)' : 'Пользователь (user)';
```

```
512         createRoleLabel.className = e.target.checked ? 'text-w
arning' : 'text-info';
513     });
514 }
515
516     const editAdminSw = document.getElementById('edit-user-is-
admin');
517     const editRoleLabel = document.getElementById('edit-role-l
abel');
518     if (editAdminSw && editRoleLabel) {
519         editAdminSw.addEventListener('change', (e) => {
520             editRoleLabel.textContent = e.target.checked ? 'Админи
стратор (admin)' : 'Пользователь (user)';
521             editRoleLabel.className = e.target.checked ? 'text-war
ning' : 'text-info';
522         });
523     }
524
525     // Refresh Users Button
526     const btnRefresh = document.getElementById('btn-refresh-us
ers');
527     if (btnRefresh) {
528         btnRefresh.addEventListener('click', () => {
529             loadUsers();
530         });
531     }
532
533     // Search Input with Debounce
534     const searchInput = document.getElementById('users-search-
input');
535     const searchClear = document.getElementById('users-search-
clear');
536     let debounceTimeout = null;
537
538     if (searchInput) {
539         searchInput.addEventListener('input', (e) => {
540             const val = e.target.value;
541             if (searchClear) {
```



```
542 |         searchClear.classList.toggle('d-none', !val);
543 |     }
544 |     clearTimeout(debounceTimeout);
545 |     debounceTimeout = setTimeout(() => {
546 |         state.searchQuery = val.trim();
547 |         loadUsers();
548 |     }, 300);
549 | });
550 | }
551 |
552 | if (searchClear) {
553 |     searchClear.addEventListener('click', () => {
554 |         if (searchInput) searchInput.value = '';
555 |         searchClear.classList.add('d-none');
556 |         state.searchQuery = '';
557 |         loadUsers();
558 |     });
559 | }
560 |
561 | // Role Filter
562 | const filterRole = document.getElementById('users-filter-r
563 |     if (filterRole) {
564 |         filterRole.addEventListener('change', (e) => {
565 |             state.filterRole = e.target.value;
566 |             loadUsers();
567 |         });
568 |     }
569 |
570 | // Status Filter
571 | const filterStatus = document.getElementById('users-filter
572 |     if (filterStatus) {
573 |         filterStatus.addEventListener('change', (e) => {
574 |             state.filterStatus = e.target.value;
575 |             loadUsers();
576 |         });
577 |     }
```

```

578
579 // Generator button for create modal
580 const btnGenCreatePwd = document.getElementById('btn-gen-c
    reate-password');
581 if (btnGenCreatePwd) {
582     btnGenCreatePwd.addEventListener('click', () => {
583         const pwdInput = document.getElementById('create-user-
    password');
584         if (pwdInput) pwdInput.value = generatePassword(12);
585     });
586 }
587
588 // Generator button for password change modal
589 const btnGenPwdVal = document.getElementById('btn-gen-pwd-
    val');
590 if (btnGenPwdVal) {
591     btnGenPwdVal.addEventListener('click', () => {
592         const pwdInput = document.getElementById('pwd-input-va
    l');
593         if (pwdInput) pwdInput.value = generatePassword(12);
594     });
595 }
596
597 // Create User Form Submit
598 const formCreate = document.getElementById('form-create-us
    er');
599 if (formCreate) {
600     formCreate.addEventListener('submit', async (e) => {
601         e.preventDefault();
602         const submitBtn = document.getElementById('btn-submit-
    create-user');
603         if (submitBtn) submitBtn.disabled = true;
604
605         const isAdmin = document.getElementById('create-user-i
    s-admin').checked ? 1 : 0;
606         const payload = {
607             email: document.getElementById('create-user-email').
    value.trim(),

```

```

608 |         name: document.getElementById('create-user-name').va
| lue.trim(),
609 |         password: document.getElementById('create-user-passw
| ord').value.trim(),
610 |         role: isAdmin ? 'admin' : 'user',
611 |         is_admin: isAdmin,
612 |         is_active: document.getElementById('create-user-is-a
| ctive').checked ? 1 : 0,
613 |         is_email_verified: document.getElementById('create-u
| ser-verified').checked ? 1 : 0
614 |     };
615 |
616 |     try {
617 |         await apiFetch('/api/admin/users', {
618 |             method: 'POST',
619 |             headers: { 'Content-Type': 'application/json' },
620 |             body: JSON.stringify(payload)
621 |         });
622 |         const modalEl = document.getElementById('modal-creat
| e-user');
623 |         const modal = bootstrap.Modal.getInstance(modalEl);
624 |         if (modal) modal.hide();
625 |
626 |         showStatusAlert(`Пользователь ${escapeHtml(payload.n
| ame)} успешно создан!`, 'success');
627 |         loadUsers();
628 |     } catch (err) {
629 |         showStatusAlert(`Ошибка создания пользователя: ${err
| .message}`, 'danger');
630 |     } finally {
631 |         if (submitBtn) submitBtn.disabled = false;
632 |     }
633 | });
634 | }
635 |
636 | // Edit User Form Submit
637 | const formEdit = document.getElementById('form-edit-user')
| ;

```

```
638     if (formEdit) {
639         formEdit.addEventListener('submit', async (e) => {
640             e.preventDefault();
641             const submitBtn = document.getElementById('btn-submit-
edit-user');
642             if (submitBtn) submitBtn.disabled = true;
643
644             const userId = parseInt(document.getElementById('edit-
user-id').value, 10);
645             const isAdmin = document.getElementById('edit-user-is-
admin').checked ? 1 : 0;
646             const payload = {
647                 email: document.getElementById('edit-user-email').va
lue.trim(),
648                 name: document.getElementById('edit-user-name').valu
e.trim(),
649                 role: isAdmin ? 'admin' : 'user',
650                 is_admin: isAdmin,
651                 is_active: document.getElementById('edit-user-is-act
ive').checked ? 1 : 0,
652                 is_email_verified: document.getElementById('edit-use
r-verified').checked ? 1 : 0
653             };
654
655             try {
656                 await apiFetch(`/api/admin/users/${userId}`, {
657                     method: 'PUT',
658                     headers: { 'Content-Type': 'application/json' },
659                     body: JSON.stringify(payload)
660                 });
661                 const modalEl = document.getElementById('modal-edit-
user');
662                 const modal = bootstrap.Modal.getInstance(modalEl);
663                 if (modal) modal.hide();
664
665                 showStatusAlert(`Данные пользователя #${userId} успе
шно обновлены!`, 'success');
666                 loadUsers();
```

```
667         } catch (err) {
668             showStatusAlert(`Ошибка обновления: ${err.message}`,
        'danger');
669         } finally {
670             if (submitBtn) submitBtn.disabled = false;
671         }
672     });
673 }
674
675 // Change Password Form Submit
676 const formPwd = document.getElementById('form-password-user');
677 if (formPwd) {
678     formPwd.addEventListener('submit', async (e) => {
679         e.preventDefault();
680         const submitBtn = document.getElementById('btn-submit-pwd-user');
681         if (submitBtn) submitBtn.disabled = true;
682
683         const userId = parseInt(document.getElementById('pwd-user-id').value, 10);
684         const newPassword = document.getElementById('pwd-input-val').value.trim();
685
686         try {
687             await apiFetch(`/api/admin/users/${userId}/password`, {
688                 method: 'POST',
689                 headers: { 'Content-Type': 'application/json' },
690                 body: JSON.stringify({ password: newPassword })
691             });
692             const modalEl = document.getElementById('modal-password-user');
693             const modal = bootstrap.Modal.getInstance(modalEl);
694             if (modal) modal.hide();
695
696             showStatusAlert(`Пароль для пользователя #${userId} успешно установлен`, 'success');
```

```
697         loadUsers();
698     } catch (err) {
699         showStatusAlert(`Ошибка установки пароля: ${err.message}`, 'danger');
700     } finally {
701         if (submitBtn) submitBtn.disabled = false;
702     }
703 });
704 }
705
706 // Delete User Confirm Button
707 const btnConfirmDelete = document.getElementById('btn-confirm-delete-user');
708 if (btnConfirmDelete) {
709     btnConfirmDelete.addEventListener('click', async () => {
710         btnConfirmDelete.disabled = true;
711         const userId = parseInt(document.getElementById('delete-user-id').value, 10);
712
713         try {
714             await apiFetch(`/api/admin/users/${userId}`, { method: 'DELETE' });
715             const modalEl = document.getElementById('modal-delete-user');
716             const modal = bootstrap.Modal.getInstance(modalEl);
717             if (modal) modal.hide();
718
719             showStatusAlert(`Пользователь #${userId} успешно удалён.`, 'success');
720             loadUsers();
721         } catch (err) {
722             showStatusAlert(`Ошибка удаления пользователя: ${err.message}`, 'danger');
723         } finally {
724             btnConfirmDelete.disabled = false;
725         }
726     });
727 }
```

```
728 | }
729 |
730 | // Global Init Function
731 | function initUsersTab() {
732 |     console.log('[UsersTab] Initializing user management tab..
    | .');
733 |     if (!state.initialized) {
734 |         initListeners();
735 |         state.initialized = true;
736 |     }
737 |     loadUsers();
738 | }
739 |
740 | // Register globally
741 | window.initUsersTab = initUsersTab;
742 | })();
743 |
```

index.html

```
1 <!DOCTYPE html>
2 <html lang="ru">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-s
6   cale=1.0">
7   <title data-i18n="title">Эксперименты с TTS (Синтез речи)</t
8   itle>
9   <link href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dis
10  t/css/bootstrap.min.css" rel="stylesheet">
11   <link href="https://cdn.jsdelivrivr.net/npm/bootstrap-icons@1.1
12   0.0/font/bootstrap-icons.css" rel="stylesheet">
13   <link href="/html/css/main.css?v=20260818_youtube_online" re
14   l="stylesheet">
15   <script src="https://cdn.jsdelivrivr.net/npm/i18next@23.11.5/i1
16   8next.min.js"></script>
17   <style>
18     /* Custom styles for TTS page */
19     :root {
20       --tts-nav-bg: #161b22;
21     }
22
23     body {
24       background: #0d1117;
25       color: #c9d1d9;
26       min-height: 100vh;
27     }
28
29     .tts-container {
30       max-width: 800px;
31       margin: 2rem auto;
32       padding: 0 1rem;
33     }
34
35     .tts-card {
36       background: #161b22;
```



```
31 |     border: 1px solid #30363d;
32 |     border-radius: 12px;
33 |     padding: 2rem;
34 |     box-shadow: 0 4px 12px rgba(0, 0, 0, 0.3);
35 | }
36 |
37 | h1 {
38 |     color: #c9d1d9;
39 |     font-weight: 600;
40 |     margin-bottom: 0.5rem;
41 | }
42 |
43 | .subtitle {
44 |     color: #8b949e;
45 |     font-size: 0.95rem;
46 |     margin-bottom: 2rem;
47 | }
48 |
49 | .form-label {
50 |     color: #8b949e;
51 |     font-weight: 500;
52 |     margin-bottom: 0.5rem;
53 | }
54 |
55 | .form-control, .form-select {
56 |     background: #0d1117;
57 |     border: 1px solid #30363d;
58 |     color: #c9d1d9;
59 |     border-radius: 8px;
60 |     padding: 0.75rem 1rem;
61 | }
62 |
63 | .form-control:focus, .form-select:focus {
64 |     background: #0d1117;
65 |     border-color: #58a6ff;
66 |     color: #c9d1d9;
67 |     box-shadow: 0 0 0 2px rgba(88, 166, 255, 0.25);
68 | }
```

```
69 |
70 | textarea.form-control {
71 |     resize: vertical;
72 |     min-height: 120px;
73 |     font-family: system-ui, -apple-system, sans-serif;
74 | }
75 |
76 | .btn-tts {
77 |     background: #1f6feb;
78 |     color: #fff;
79 |     border: none;
80 |     border-radius: 8px;
81 |     padding: 0.85rem 1.5rem;
82 |     font-weight: 500;
83 |     transition: background-color 0.2s;
84 | }
85 |
86 | .btn-tts:hover {
87 |     background: #388bfd;
88 |     color: #fff;
89 | }
90 |
91 | .btn-tts:disabled {
92 |     opacity: 0.6;
93 |     cursor: not-allowed;
94 | }
95 |
96 | .btn-apply {
97 |     background: #238636;
98 |     color: #fff;
99 |     border: none;
100 |     border-radius: 8px;
101 |     padding: 0.85rem 1.5rem;
102 |     font-weight: 500;
103 |     transition: background-color 0.2s;
104 | }
105 |
106 | .btn-apply:hover {
```

```
107 |     background: #2ea043;
108 |     color: #fff;
109 | }
110 |
111 | .btn-apply:disabled {
112 |     opacity: 0.6;
113 |     cursor: not-allowed;
114 | }
115 |
116 | .active-badge {
117 |     background: rgba(35, 134, 54, 0.15);
118 |     border: 1px solid rgba(35, 134, 54, 0.3);
119 |     color: #56d364;
120 |     padding: 0.5rem 1rem;
121 |     border-radius: 20px;
122 |     font-size: 0.85rem;
123 |     display: inline-flex;
124 |     align-items: center;
125 |     gap: 0.5rem;
126 |     margin-bottom: 1.5rem;
127 | }
128 |
129 | .audio-player-container {
130 |     margin-top: 2rem;
131 |     padding-top: 2rem;
132 |     border-top: 1px solid #30363d;
133 |     display: none;
134 | }
135 |
136 | audio {
137 |     width: 100%;
138 |     border-radius: 8px;
139 | }
140 |
141 | .status-badge {
142 |     display: inline-flex;
143 |     align-items: center;
144 |     gap: 0.5rem;
```

```
145 |     font-size: 0.9rem;
146 |     color: #8b949e;
147 |     margin-top: 1rem;
148 | }
149 |
150 | .spinner-border {
151 |     width: 1rem;
152 |     height: 1rem;
153 |     border-width: 0.15em;
154 | }
155 |
156 | .back-link {
157 |     display: inline-flex;
158 |     align-items: center;
159 |     gap: 0.5rem;
160 |     color: #8b949e;
161 |     text-decoration: none;
162 |     font-size: 0.95rem;
163 |     margin-top: 2rem;
164 |     transition: color 0.2s;
165 | }
166 |
167 | .back-link:hover {
168 |     color: #58a6ff;
169 | }
170 |
171 | /* Dark theme audio player controls */
172 | audio::-webkit-media-controls-panel {
173 |     background-color: rgba(13, 17, 23, 0.9);
174 | }
175 |
176 | audio::-webkit-media-controls-current-time-display,
177 | audio::-webkit-media-controls-time-remaining-display {
178 |     color: #c9d1d9;
179 | }
180 | </style>
181 | </head>
182 | <body>
```

```
183 |
184 | <div class="tts-container">
185 |   <div class="tts-card">
186 |     <h1><i class="bi bi-mic"></i> <span data-i18n="title">Эксп
187 |     ерименты с TTS (Синтез речи)</span></h1>
188 |     <p class="subtitle" data-i18n="subtitle">Сравните качество
189 |     синтеза речи различных моделей: Web Speech, Google, Edge и Si
190 |     lero.</p>
191 |     <div id="active-voice-container" class="mb-3" style="display: none;">
192 |       <span class="active-badge">
193 |         <i class="bi bi-check-circle-fill"></i>
194 |         <span><span data-i18n="label_active_voice">Активный го
195 |         лос:</span> <strong id="active-voice-name">...</strong></span>
196 |       </span>
197 |     </div>
198 |     <form id="tts-form">
199 |       <div class="mb-4">
200 |         <label for="tts-text" class="form-label" data-i18n="fo
201 |         rm.label_text">Текст для озвучки</label>
202 |         <textarea id="tts-text" class="form-control" placeholder data-i18n="placeholder">Привет! Это демонс
203 |         трация работы различных систем синтеза речи. Какая из них звуч
204 |         ит лучше всего?</textarea>
205 |       </div>
206 |       <div class="row">
207 |         <div class="col-md-6 mb-4">
208 |           <label for="tts-engine" class="form-label" data-i18n="form.label_engine">Система синтеза</label>
209 |           <select id="tts-engine" class="form-select">
210 |             <option value="browser" data-i18n="options.browser">Web Speech API (Браузерная)</option>
211 |             <option value="gtts" data-i18n="options.gtts">Google TTS (Базовая)</option>
212 |             <option value="edge-tts" selected data-i18n="optio
```

```
ns.edge-tts">Microsoft Edge TTS (Нейросетевая)</option>
209         <option value="silero" data-i18n="options.silero">
Silero TTS (Локальная оффлайн)</option>
210     </select>
211 </div>
212
213     <div class="col-md-6 mb-4" id="voice-select-container"
>
214         <label for="tts-voice" class="form-label" data-i18n=
"form.label_voice">Голос / Спикер</label>
215         <select id="tts-voice" class="form-select">
216             <!-- Populated dynamically via JS -->
217         </select>
218     </div>
219 </div>
220
221     <div class="d-flex gap-2">
222         <button type="submit" class="btn btn-tts flex-grow-1"
id="btn-speak" data-i18n="button_speak">Озвучить текст</button
>
223         <button type="button" class="btn btn-apply" id="btn-sa
ve-default" data-i18n="button_apply">Применить изменения</butt
on>
224     </div>
225
226     <div class="status-badge" id="tts-status" style="display
: none;">
227         <div class="spinner-border text-success" role="status"
></div>
228         <span id="status-text" data-i18n="status_generating">Г
енерация аудиофайла на сервере...</span>
229     </div>
230 </form>
231
232     <div class="audio-player-container" id="player-container">
233         <label class="form-label" data-i18n="label_result">Резул
ьтат генерации</label>
234         <audio id="audio-player" controls></audio>
```

```
235     </div>
236 </div>
237
238 <a href="/" class="back-link">
239     <i class="bi bi-arrow-left"></i>
240     <span data-i18n="back_link">Вернуться в медиатеку</span>
241 </a>
242 </div>
243
244 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist
    /js/bootstrap.bundle.min.js"></script>
245 <script type="module">
246     import { initI18n, switchLang, applyTranslations } from '/ht
    ml/js/i18n.js';
247
248     window.switchLang = switchLang;
249     window.applyTranslations = applyTranslations;
250
251     // Initialize i18n
252     const savedLang = localStorage.getItem('app_language') || 'r
    u';
253     await initI18n(savedLang);
254
255     const engineSelect = document.getElementById('tts-engine');
256     const voiceSelect = document.getElementById('tts-voice');
257     const voiceContainer = document.getElementById('voice-select
    -container');
258     const ttsForm = document.getElementById('tts-form');
259     const btnSpeak = document.getElementById('btn-speak');
260     const ttsStatus = document.getElementById('tts-status');
261     const statusText = document.getElementById('status-text');
262     const playerContainer = document.getElementById('player-cont
    ainer');
263     const audioPlayer = document.getElementById('audio-player');
264     const activeVoiceContainer = document.getElementById('active
    -voice-container');
265     const activeVoiceName = document.getElementById('active-voic
    e-name');
```

```
266 |
267 | function getFriendlyVoiceName(system, voice) {
268 |     if (system === 'edge-tts') {
269 |         if (voice === 'ru-RU-DmitryNeural') return 'Дмитрий (Microsoft Edge)';
270 |         if (voice === 'ru-RU-SvetlanaNeural') return 'Светлана (Microsoft Edge)';
271 |         return `Edge (${voice})`;
272 |     }
273 |     if (system === 'silero') {
274 |         const names = {
275 |             eugene: 'Евгений (Silero)',
276 |             aidar: 'Айдар (Silero)',
277 |             baya: 'Бая (Silero)',
278 |             kseniya: 'Ксения (Silero)',
279 |             xenia: 'Ксения v2 (Silero)',
280 |             random: 'Рандомный (Silero)'
281 |         };
282 |         return names[voice] || `Silero (${voice})`;
283 |     }
284 |     if (system === 'gtts') {
285 |         return 'Google TTS (Базовый)';
286 |     }
287 |     if (system === 'browser') {
288 |         return `Браузерный (${voice} || 'По умолчанию')`;
289 |     }
290 |     return `${system} – ${voice}`;
291 | }
292 |
293 | function updateActiveBadge(system, voice) {
294 |     if (system && voice) {
295 |         activeVoiceName.textContent = getFriendlyVoiceName(system, voice);
296 |         activeVoiceContainer.style.display = 'block';
297 |     } else {
298 |         activeVoiceContainer.style.display = 'none';
299 |     }
300 | }
```



```

301
302 // Available voices dictionary with translated labels
303 const voiceMap = {
304   'browser': [], // Populated dynamically from window.speech
Synthesis
305   'gtts': [
306     { value: 'ru', labelKey: 'voices.ru_standard' }
307   ],
308   'edge-tts': [
309     { value: 'ru-RU-DmitryNeural', labelKey: 'voices.dmitry'
310     },
311     { value: 'ru-RU-SvetlanaNeural', labelKey: 'voices.svetl
ana' }
312   ],
313   'silero': [
314     { value: 'eugene', labelKey: 'voices.eugene' },
315     { value: 'aidar', labelKey: 'voices.aidar' },
316     { value: 'baya', labelKey: 'voices.baya' },
317     { value: 'kseniya', labelKey: 'voices.kseniya' },
318     { value: 'xenia', labelKey: 'voices.xenia' },
319     { value: 'random', labelKey: 'voices.random' }
320   ]
321 };
322
323 // Translate voice labels
324 function translateVoiceLabel(labelKey) {
325   return i18next?.t(labelKey) || labelKey;
326 }
327
328 // Load browser native voices
329 function loadBrowserVoices() {
330   if ('speechSynthesis' in window) {
331     const voices = window.speechSynthesis.getVoices();
332     voiceMap['browser'] = voices
333       .filter(v => v.lang.startsWith('ru'))
334       .map(v => ({ value: v.name, labelKey: `voices.${v.name
.replace(/\s+/g, '_')}` }));

```

```
335 |         // Fallback if no Russian voices found in browser
336 |         if (voiceMap['browser'].length === 0) {
337 |             voiceMap['browser'].push({ value: 'default', labelKey:
| 'voices.default' });
338 |         }
339 |
340 |         if (engineSelect.value === 'browser') {
341 |             updateVoiceSelect();
342 |         }
343 |     }
344 | }
345 |
346 | if ('speechSynthesis' in window) {
347 |     window.speechSynthesis.onvoiceschanged = loadBrowserVoices
| ;
348 |     loadBrowserVoices();
349 | }
350 |
351 | function updateVoiceSelect() {
352 |     const engine = engineSelect.value;
353 |     const voices = voiceMap[engine] || [];
354 |
355 |     voiceSelect.innerHTML = '';
356 |
357 |     if (voices.length === 0) {
358 |         voiceContainer.style.display = 'none';
359 |     } else {
360 |         voiceContainer.style.display = 'block';
361 |         voices.forEach(voice => {
362 |             const opt = document.createElement('option');
363 |             opt.value = voice.value;
364 |             opt.textContent = translateVoiceLabel(voice.labelKey);
365 |             voiceSelect.appendChild(opt);
366 |         });
367 |     }
368 | }
369 |
370 | engineSelect.addEventListener('change', updateVoiceSelect);
```

```

371 | updateVoiceSelect(); // Initial populate
372 |
373 | // On page load, fetch current settings to pre-select saved
    | voice/engine
374 | try {
375 |     const settingsResponse = await fetch('/auth/settings');
376 |     if (settingsResponse.ok) {
377 |         const settings = await settingsResponse.json();
378 |         if (settings.tts_system) {
379 |             engineSelect.value = settings.tts_system;
380 |             updateVoiceSelect();
381 |         }
382 |         if (settings.tts_voice) {
383 |             voiceSelect.value = settings.tts_voice;
384 |         }
385 |         updateActiveBadge(settings.tts_system, settings.tts_voic
    | e);
386 |     }
387 | } catch (e) {
388 |     console.error('Failed to load user settings:', e);
389 | }
390 |
391 | // Update translations when language changes
392 | const originalSwitchLang = window.switchLang;
393 | window.switchLang = async function(lang) {
394 |     await originalSwitchLang(lang);
395 |     updateVoiceSelect();
396 | };
397 |
398 | ttsForm.addEventListener('submit', async (e) => {
399 |     e.preventDefault();
400 |     const text = document.getElementById('tts-text').value.tri
    | m();
401 |     if (!text) return;
402 |
403 |     const engine = engineSelect.value;
404 |     const voice = voiceSelect.value;
405 |

```

```

406 // 1. Browser Web Speech synthesis
407 if (engine === 'browser') {
408     if ('speechSynthesis' in window) {
409         window.speechSynthesis.cancel();
410         const cleanText = text.replace(/<[^>]*>/g, '').replace
411         (/[*_`#]/g, '');
412         const utterance = new SpeechSynthesisUtterance(cleanText);
413         utterance.lang = 'ru-RU';
414
415         const voices = window.speechSynthesis.getVoices();
416         const selectedVoice = voices.find(v => v.name === voice);
417         if (selectedVoice) {
418             utterance.voice = selectedVoice;
419         }
420
421         ttsStatus.style.display = 'inline-flex';
422         statusText.textContent = i18next?.t('status_generating') || 'Воспроизведение речи браузером...';
423
424         utterance.onend = () => {
425             ttsStatus.style.display = 'none';
426         };
427         utterance.onerror = () => {
428             ttsStatus.style.display = 'none';
429         };
430
431         window.speechSynthesis.speak(utterance);
432         playerContainer.style.display = 'none';
433     } else {
434         alert(i18next?.t('errors.no_api') || 'Ваш браузер не поддерживает Web Speech API.');
```

```
439     ttsStatus.style.display = 'inline-flex';
440     statusText.textContent = i18next?.t('status_generating') |
    | 'Генерация аудиофайла на сервере...';
441     btnSpeak.disabled = true;
442     playerContainer.style.display = 'none';
443
444     try {
445         const queryParams = new URLSearchParams({
446             text: text,
447             system: engine,
448             voice: voice
449         });
450
451         // Load audio stream from server
452         const audioUrl = `/api/tts/synthesize?${queryParams.toSt
ring()}`;
453
454         // Force preload of the audio
455         audioPlayer.src = audioUrl;
456         audioPlayer.load();
457
458         audioPlayer.oncanplaythrough = () => {
459             ttsStatus.style.display = 'none';
460             btnSpeak.disabled = false;
461             playerContainer.style.display = 'block';
462             audioPlayer.play();
463         };
464
465         audioPlayer.onerror = (err) => {
466             console.error(err);
467             statusText.textContent = i18next?.t('errors.server_err
or') || 'Ошибка синтеза речи на сервере';
468             btnSpeak.disabled = false;
469             setTimeout(() => { ttsStatus.style.display = 'none'; }
, 3000);
470         };
471
472     } catch (err) {
```

```
473     console.error(err);
474     statusText.textContent = i18next?.t('errors.network_error') || 'Сбой сети при запросе к серверу';
475     btnSpeak.disabled = false;
476     setTimeout(() => { ttsStatus.style.display = 'none'; },
3000);
477   }
478   });
479
480   const btnSaveDefault = document.getElementById('btn-save-default');
481   btnSaveDefault.addEventListener('click', async () => {
482     const engine = engineSelect.value;
483     const voice = voiceSelect.value;
484
485     btnSaveDefault.disabled = true;
486     try {
487       const payload = {
488         tts_enabled: 1,
489         tts_system: engine,
490         tts_voice: voice
491       };
492
493       const response = await fetch('/auth/settings', {
494         method: 'POST',
495         headers: { 'Content-Type': 'application/json' },
496         body: JSON.stringify(payload)
497       });
498
499       if (response.ok) {
500         updateActiveBadge(engine, voice);
501         alert('Изменения успешно применены! Этот голос теперь установлен по умолчанию.');
```

установлен по умолчанию.');

```
502       } else {
503         alert('Не удалось сохранить настройки. Убедитесь, что вы вошли в систему.');
```

вы вошли в систему.');

```
504       }
505     } catch (err) {
```

```
506 |         console.error(err);
507 |         alert('Ошибка подключения к серверу при сохранении настр
    | оек. ');
508 |     } finally {
509 |         btnSaveDefault.disabled = false;
510 |     }
511 | });
512 | </script>
513 | </body>
514 | </html>
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: FastAPI router initialization and exports
4 # =====
5 # Description:
6 #   Exports factory initializers and router instances for core
7 #   HTTP endpoints, WebSockets, authentication, chat, and admi
8 #   nistration services.
9 #
10 # File: __init__.py
11 # Project: ai-breadboard
12 # Package: src.fastapi
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 from .router_auth import init_router as init_auth_router
18 from .router_chat import init_router as init_chat_router
19 from .router_control import init_router as init_control_router
20 from .router_tts import init_router as init_tts_router
21 from .router_logs import init_router as init_logs_router
22 from .router_keys import init_router as init_keys_router
23 from .router_admin import init_router as init_admin_router
24 from .router_agents import init_agents_router
25 from .router_openai import router as router_openai
26
27 __all__ = [
28     "init_auth_router",
29     "init_chat_router",
30     "init_control_router",
31     "init_tts_router",
32     "init_logs_router",
```



```
32 | "init_keys_router",  
33 | "init_admin_router",  
34 | "init_agents_router",  
35 | "router_openai",  
36 | ]  
37 |
```

google_services.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Update access_token using refresh_token
4 # =====
5 # Description:
6 #   Module for Google API interaction with OAuth token managem
7 #   ent.
8 # File: google_services.py
9 # Project: ai-breadboard
10 # Package: src
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from __future__ import annotations
16
17 import os
18 from datetime import datetime
19 from typing import Any, Dict, List
20
21 import requests
22
23 from src.logger import logger
24 from src.user_manager import user_manager
25
26 def refresh_google_access_token(user_id: int) -> str:
27     """Update access_token с использованием refresh_token.
28
29     Args:
30         user_id (int): Идентификатор пользователя.
31
32     Returns:
```

```

33         str: Новый access_token или пустая string при ошибке.
34         """
35         token_record = user_manager.get_google_tokens(user_id)
36         refresh_token = token_record.get('refresh_token', '')
37         if not refresh_token:
38             logger.warning(f'Refresh token отсутствует для user_id
={user_id}')
39             return ''
40
41         from src.fastapi.router_auth import GOOGLE_CLIENT_ID, GOOG
LE_CLIENT_SECRET
42         if not GOOGLE_CLIENT_ID or not GOOGLE_CLIENT_SECRET:
43             logger.error('Google OAuth Client ID или Secret не нас
троены')
44             return ''
45
46         token_url = 'https://oauth2.googleapis.com/token'
47         data = {
48             'client_id': GOOGLE_CLIENT_ID,
49             'client_secret': GOOGLE_CLIENT_SECRET,
50             'refresh_token': refresh_token,
51             'grant_type': 'refresh_token',
52         }
53
54         try:
55             response = requests.post(token_url, data=data, timeout
=10)
56             response.raise_for_status()
57             token_data = response.json()
58             new_access_token = token_data.get('access_token', '')
59             expires_in = token_data.get('expires_in', 3600)
60             scope = token_data.get('scope', token_record.get('scop
e', ''))
61
62             if new_access_token:
63                 user_manager.save_google_tokens(
64                     user_id=user_id,
65                     access_token=new_access_token,

```

```

66         refresh_token=refresh_token,
67         expires_in=int(expires_in),
68         scope=scope
69     )
70     logger.info(f'Successfully обновлен access_token д
    ля user_id={user_id}')
71     return new_access_token
72     return ''
73 except Exception as ex:
74     logger.error(f'Error обновления Google access token дл
    я user_id={user_id}:', ex, False)
75     return ''
76
77 def get_valid_google_access_token(user_id: int) -> str:
78     """Получение действующего access_token с проверкой срока ж
    изни.
79
80     Args:
81         user_id (int): Идентификатор пользователя.
82
83     Returns:
84         str: Valid access_token или пустая string.
85     """
86     token_record = user_manager.get_google_tokens(user_id)
87     if not token_record:
88         return ''
89
90     access_token = token_record.get('access_token', '')
91     expires_at_str = token_record.get('expires_at', '')
92
93     if not access_token:
94         return ''
95
96     if expires_at_str:
97         try:
98             expires_at = datetime.fromisoformat(expires_at_str
99         )
100
101         now = datetime.utcnow()

```

```

100         # Если токен истекает менее чем через 60 секунд –
        обновляем
101         if (expires_at - now).total_seconds() < 60:
102             logger.info(f'Срок действия токена истекает, з
        апуск обновления для user_id={user_id}')
103             refreshed = refresh_google_access_token(user_i
        d)
104             if refreshed:
105                 return refreshed
106         except Exception:
107             pass
108
109     return access_token
110
111 def get_google_headers(user_id: int) -> Dict[str, str]:
112     """Formation заголовков авторизации для Google API.
113
114     Args:
115         user_id (int): Идентификатор пользователя.
116
117     Returns:
118         Dict[str, str]: Заголовки HTTP запроса.
119     """
120     token = get_valid_google_access_token(user_id)
121     if not token:
122         return {}
123     return {
124         'Authorization': f'Bearer {token}',
125         'Accept': 'application/json',
126     }
127
128 # =====
129 # Google Calendar API
130 # =====
131
132 def get_google_calendar_events(user_id: int, time_min: str = '

```

```

    ', max_results: int = 20) -> List[Dict[str, Any]]:
133     """Получение событий из основного календаря Google.
134
135     Args:
136         user_id (int): Идентификатор пользователя.
137         time_min (str): Минимальное время (ISO 8601). По умолч
138         анию текущее время.
139         max_results (int): Максимальное количество событий.
140
141     Returns:
142         List[Dict[str, Any]]: List событий.
143     """
144     headers = get_google_headers(user_id)
145     if not headers:
146         return []
147
148     if not time_min:
149         time_min = datetime.utcnow().isoformat() + 'Z'
150
151     url = 'https://www.googleapis.com/calendar/v3/calendars/pr
152     imary/events'
153     params = {
154         'timeMin': time_min,
155         'singleEvents': 'true',
156         'orderBy': 'startTime',
157         'maxResults': max_results,
158     }
159
160     try:
161         resp = requests.get(url, headers=headers, params=params, timeout=10)
162         resp.raise_for_status()
163         return resp.json().get('items', [])
164     except Exception as ex:
165         logger.error(f'Error получения событий Google Calendar
        для user_id={user_id}:', ex, False)
        return []

```

```

166 # =====
    =====
167 # Google Contacts API (People API)
168 # =====
    =====
169
170 def get_google_contacts(user_id: int, page_size: int = 50) ->
    List[Dict[str, Any]]:
171     """Получение списка контактов Google пользователя.
172
173     Args:
174         user_id (int): Идентификатор пользователя.
175         page_size (int): Количество контактов на страницу.
176
177     Returns:
178         List[Dict[str, Any]]: List контактов.
179     """
180     headers = get_google_headers(user_id)
181     if not headers:
182         return []
183
184     url = 'https://people.googleapis.com/v1/people/me/connections'
185     params = {
186         'personFields': 'names,emailAddresses,phoneNumbers,photos',
187         'pageSize': page_size,
188     }
189
190     try:
191         resp = requests.get(url, headers=headers, params=params, timeout=10)
192         resp.raise_for_status()
193         return resp.json().get('connections', [])
194     except Exception as ex:
195         logger.error(f'Error получения контактов Google для user_id={user_id}:', ex, False)
196         return []

```

```

197
198 # =====
    =====
199 # Google Drive & Docs API
200 # =====
    =====
201
202 def list_google_documents(user_id: int, page_size: int = 20) -
    > List[Dict[str, Any]]:
203     """Получение списка Google Документов пользователя.
204
205     Args:
206         user_id (int): Идентификатор пользователя.
207         page_size (int): Количество документов.
208
209     Returns:
210         List[Dict[str, Any]]: List файлов Google Docs.
211     """
212     headers = get_google_headers(user_id)
213     if not headers:
214         return []
215
216     url = 'https://www.googleapis.com/drive/v3/files'
217     params = {
218         'q': "mimeType='application/vnd.google-apps.document'
219 and trashed=false",
220         'fields': 'files(id, name, createdTime, modifiedTime,
221 webviewLink)',
222         'pageSize': page_size,
223     }
224
225     try:
226         resp = requests.get(url, headers=headers, params=params,
227                             timeout=10)
228         resp.raise_for_status()
229         return resp.json().get('files', [])
230     except Exception as ex:
231         logger.error(f'Error получения списка Google Docs для

```



```

    user_id={user_id}:', ex, False)
229         return []
230
231 def get_google_document_content(user_id: int, document_id: str
    ) -> Dict[str, Any]:
232     """Получение содержимого конкретного Google Документа.
233
234     Args:
235         user_id (int): Идентификатор пользователя.
236         document_id (str): Идентификатор документа Google Docs
237         .
238     Returns:
239         Dict[str, Any]: Структура документа или empty dictionary.
240     """
241     headers = get_google_headers(user_id)
242     if not headers:
243         return {}
244
245     url = f'https://docs.googleapis.com/v1/documents/{document_id}'
246     try:
247         resp = requests.get(url, headers=headers, timeout=15)
248         resp.raise_for_status()
249         return resp.json()
250     except Exception as ex:
251         logger.error(f'Error чтения Google Document {document_id}:', ex, False)
252         return {}
253

```

compress_logs.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Compress repeated lines in log files to [Nx] format
4 # =====
5 # Description:
6 #   Compresses repeated lines into [Nx] text format for log file reduction.
7 #
8 # File: compress_logs.py
9 # Project: ai-breadboard
10 # Package: src.logger
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """Script for log file compression - combines repeated lines.
16
17 Reduces log file size by consolidating identical lines into [Nx] format.
18 Usage: python scripts/compress_logs.py
19 """
20
21 import sys
22 from collections import Counter
23 from pathlib import Path
24 from typing import List, Tuple
25
26 def compress_lines(lines: List[str], min_repeat: int = 2) -> List[str]:
27     """Compress repeated lines into [Nx] text format.
28
29     Args:
```

```

30         lines (List[str]): Input lines to compress.
31         min_repeat (int): Minimum repetitions to trigger compr
    session.
32
33     Returns:
34         List[str]: Compressed lines.
35     """
36     counter = Counter(lines)
37     result = []
38
39     for line, count in counter.items():
40         stripped = line.strip()
41         if not stripped:
42             continue
43         if count >= min_repeat:
44             result.append(f"[{count}x] {stripped}")
45         else:
46             # Unique lines added as-is
47             result.append(stripped)
48
49     return result
50
51 def compress_log_file(input_path: Path, output_path: Path = No
    ne, min_repeat: int = 2) -> Tuple[int, int]:
52     """Compress log file.
53
54     Args:
55         input_path (Path): Input log file path.
56         output_path (Path): Output log file path (default: .co
    mpressed.log).
57         min_repeat (int): Minimum repetitions to compress.
58
59     Returns:
60         Tuple[int, int]: (original_lines_count, compressed_lin
    es_count)
61     """
62     with open(input_path, 'r', encoding='utf-8', errors='ignor
    e') as f:

```

```

63 |         lines = f.readlines()
64 |
65 |     if not lines:
66 |         return 0, 0
67 |
68 |     original_count = len(lines)
69 |     compressed = compress_lines(lines, min_repeat)
70 |
71 |     if output_path is None:
72 |         output_path = input_path.with_suffix('.compressed.log'
73 | )
74 |
75 |     with open(output_path, 'w', encoding='utf-8') as f:
76 |         f.write('\n'.join(compressed))
77 |         if compressed:
78 |             f.write('\n')
79 |
80 |     return original_count, len(compressed)
81 |
82 | def main():
83 |     """Main entry point for log compression."""
84 |     from src.logger import logger
85 |
86 |     logs_dir = Path(__file__).resolve().parent.parent.parent /
87 | 'tmp' / 'logs'
88 |
89 |     if not logs_dir.exists():
90 |         logger.error(f"Logs directory not found: {logs_dir}")
91 |         return
92 |
93 |     log_files = list(logs_dir.glob('*.log'))
94 |     if not log_files:
95 |         logger.info("No log files found")
96 |         return
97 |
98 |     logger.info(f"Found files to compress: {len(log_files)}")
99 |
100 |     for log_file in log_files:

```

```
99 |         original, compressed = compress_log_file(log_file)
100 |         if original > 0:
101 |             compression_pct = 100 - compressed * 100 // origin
102 |         al
103 |             logger.info(f"{log_file.name}: {original} → {compressed} lines (compression {compression_pct}%)")
104 |         else:
105 |             logger.info(f"{log_file.name}: empty file")
106 | if __name__ == "__main__":
107 |     main()
108 |
```

conftest.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: pytest configuration for logger module
4 # =====
5 # Description:
6 #   Module for conftest.py in ai-breadboard project.
7 #
8 # File: conftest.py
9 # Project: ai-breadboard
10 # Package: src.logger
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """pytest configuration for logger module testing.
16
17 Provides fixtures and configuration for unit tests of src.logger module."""
18
19 import pytest
20 import sys
21 from pathlib import Path
22 from header import __root__
23
24 # Add project root to sys.path for imports
25 sys.path.insert(0, str(__root__))
26
27 @pytest.fixture(scope="session")
28 def project_root():
29     """Fixture to get project root directory."""
30     return __root__
31
32 @pytest.fixture(scope="session")
```

```
33 def logger_module_path():
34     """Фикстура для получения пути к модулю логирования."""
35     return __root__ / 'core' / 'logger'
36
37 @pytest.fixture
38 def temp_log_dir(tmp_path):
39     """Фикстура для создания временной директории логов."""
40     log_dir = tmp_path / 'logs'
41     log_dir.mkdir()
42     return log_dir
43
44 @pytest.fixture
45 def temp_reports_dir(tmp_path):
46     """Фикстура для создания временной директории отчётов."""
47     reports_dir = tmp_path / 'reports'
48     reports_dir.mkdir()
49     return reports_dir
50
51 @pytest.fixture
52 def mock_logger_config():
53     """Фикстура для мокирования конфигурации логгера."""
54     from unittest.mock import MagicMock
55     config = MagicMock()
56     config.mode = "dev"
57     config.debug = True
58     return config
59
60 @pytest.fixture
61 def mock_ai_model():
62     """Фикстура для мокирования AI модели."""
63     from unittest.mock import AsyncMock
64     model = AsyncMock()
65     model.ask.return_value = "# Analysis Report\n\nNo issues f
ound."
66     return model
67
68 # Markers для организации тестов
69 def pytest_configure(config):
```

```
70     """Регистрирует пользовательские markers для pytest."""
71     config.addinivalue_line(
72         "markers", "slow: marks tests as slow (deselect with '
73         -m \"not slow\")"
74     )
75     config.addinivalue_line(
76         "markers", "integration: marks tests as integration te
77         sts"
78     )
79     config.addinivalue_line(
80         "markers", "unit: marks tests as unit tests"
81     )
82     config.addinivalue_line(
83         "markers", "performance: marks tests for performance t
84         esting"
85     )
86
87 # Hooks для сбора информации о тестах
88 def pytest_collection_modifyitems(config, items):
89     """Модифицирует собранные тесты."""
90     for item in items:
91         # Добавляем marker если его нет
92         if "test_" in item.name:
93             if not any(marker.name == "unit" for marker in ite
94                 m.iter_markers()):
95                 item.add_marker(pytest.mark.unit)
96
97 # Configuration для asyncio тестов
98 pytest_plugins = ['pytest_asyncio']
99
```


header.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Project root path determination and meta information initialization
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: header.py
9 # Project: ai-breadboard
10 # Package: src.logger
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """Module for project root path determination and meta information initialization.
16
17 Solves two primary tasks:
18 1. Determines project root directory by searching for marker files
19 2. Loads and initializes project meta information from configuration files"""
20
21 import sys
22 import json
23 import logging
24 from packaging.version import Version
25 from pathlib import Path
26 from typing import Dict, Optional, Tuple
27
28 def set_project_root(marker_files: Tuple[str, ...] = ('__root_', '.git')) -> Path:
```

```

29 |     """
30 |     Определяет корневую директорию проекта, поднимаясь вверх по
    |     иерархии папок.
31 |
32 |     Алгоритм:
33 |     1. Начинает с директории, где расположен этот файл
34 |     2. Поднимается вверх по папкам в поиске маркерных файлов/папок
35 |     3. Первая найденная маркер-папка считается корнем проекта
36 |     4. Добавляет корень в sys.path для импортов
37 |
38 |     Args:
39 |         marker_files: Tuple имён маркерных файлов/папок для идентификации корня.
    |
    |         По умолчанию ищет '__root__' или '.git'.
40 |
41 |
42 |     Returns:
43 |         Path: Путь к корневой директории проекта.
44 |         Если не найдена, Returns директорию скрипта.
45 |
46 |     Example:
47 |         >>> root = set_project_root()
48 |         >>> print(root)
49 |         /path/to/project
50 |     """
51 |     __root__: Path
52 |     current_path: Path = Path(__file__).resolve().parent
53 |     __root__ = current_path
54 |
55 |     # Поиск маркера в текущей папке и всех родительских папках
56 |     for parent in [current_path] + list(current_path.parents):
57 |         if any((parent / marker).exists() for marker in marker
    |         _files):
58 |             __root__ = parent
59 |             break
60 |
61 |     # Добавление корня в sys.path для импортов
62 |     if __root__ not in sys.path:

```

```

63         sys.path.insert(0, str(__root__))
64
65     return __root__
66
67 # Получение корневой директории проекта
68 __root__: Path = set_project_root()
69 """__root__ (Path): Путь к корневой директории проекта"""
70
71 def _load_project_settings() -> Dict[str, str]:
72     """
73     Loads метаинформацию проекта из файла конфигурации.
74
75     Поиск конфигурации:
76     1. Сначала пытается найти settings.json в core/
77     2. Если не найден, Returns empty dictionary
78
79     Returns:
80         Dict: Dictionary с параметрами проекта или empty dicti
81         onary при ошибке.
82     """
83     settings: Dict[str, str] = {}
84     try:
85         from src import gs
86         if gs:
87             settings_path = gs.path.root / 'core' / 'settings.
88             json'
89             if settings_path.exists():
90                 with open(settings_path, 'r', encoding='utf-8'
91                 ) as settings_file:
92                     settings = json.load(settings_file)
93             except (ImportError, FileNotFoundError, json.JSONDecodeErr
94             or, AttributeError) as ex:
95                 logging.debug(f"Не удалось загрузить settings.json: {e
96                 x}")
97
98     return settings
99
100 def _load_project_documentation() -> str:

```

```

96     """
97     Loads документацию проекта из README файла.
98
99     Returns:
100         str: Содержимое документации или пустая string при оши
101 бке.
102     """
103     doc_str: str = ""
104     try:
105         from src import gs
106         if gs:
107             readme_path = gs.path.root / 'core' / 'README.MD'
108             if readme_path.exists():
109                 with open(readme_path, 'r', encoding='utf-8')
110 as readme_file:
111                     doc_str = readme_file.read()
112             except (ImportError, FileNotFoundError, AttributeError) as
113 ex:
114                 logging.debug(f"Не удалось загрузить документацию: {ex
115 }")
116
117     return doc_str
118
119 # Loading метайнформации проекта
120 settings: Dict[str, str] = _load_project_settings()
121 doc_str: str = _load_project_documentation()
122
123 # MetaInfo о проекте (используется при импорте модуля)
124 __project_name__: str = settings.get("project_name", 'ai-bread
125 board') if settings else 'ai-breadboard'
126
127 """Название проекта"""
128
129 __version__: str = settings.get("version", '1.0.0') if setting
130 s else '1.0.0'
131
132 """Версия проекта"""
133
134
135 __doc__: str = doc_str if doc_str else 'AI Breadboard - Интелл
136 ектуальная система анализа'

```

```
127 | """Документация проекта"""
128 |
129 | __author__: str = settings.get("author", 'Development Team') i
    | f settings else 'Development Team'
130 | """Автор проекта"""
131 |
132 | __copyright__: str = settings.get("copyright", '© 2026 Develop
    | ment Team') if settings else '© 2026 Development Team'
133 | """Копирайт проекта"""
134 |
135 | __cofee__: str = settings.get("cofee", "Treat the developer to
    | a cup of coffee for boosting enthusiasm in development: https
    | ://boosty.to/hypo69") if settings else "Treat the developer to
    | a cup of coffee for boosting enthusiasm in development: https
    | ://boosty.to/hypo69"
136 | """Сообщение благодарности для разработчика"""
137 |
138 |
```

logger.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Application logging system with color output and file rotation
4 # =====
5 # Description:
6 #   Comprehensive logging module with support for colored console output, JSON formatting,
7 #   asynchronous logging queue, file rotation, and multi-level filtering for development and production environments.
8 #
9 # File: logger.py
10 # Project: ai-breadboard
11 # Package: src.logger
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import logging
17 import logging.handlers
18 import colorama
19 import datetime
20 import json
21 import inspect
22 import threading
23 import queue
24 import atexit
25 import tempfile
26 from pathlib import Path
27 from typing import Optional, Tuple, Dict
28 from types import SimpleNamespace
29 from collections import Counter
30
```

```
31 import header
32 from header import __root__
33
34 # Initialization colorama для поддержки цветного вывода
35 colorama.init(autoreset=False)
36
37 # Dictionary для цветов текста
38 TEXT_COLORS: Dict[str, str] = {
39     "black": colorama.Fore.BLACK,
40     "red": colorama.Fore.RED,
41     "green": colorama.Fore.GREEN,
42     "yellow": colorama.Fore.YELLOW,
43     "blue": colorama.Fore.BLUE,
44     "magenta": colorama.Fore.MAGENTA,
45     "cyan": colorama.Fore.CYAN,
46     "white": colorama.Fore.WHITE,
47     "light_gray": colorama.Fore.LIGHTBLACK_EX,
48     "light_red": colorama.Fore.LIGHTRED_EX,
49     "light_green": colorama.Fore.LIGHTGREEN_EX,
50     "light_yellow": colorama.Fore.LIGHTYELLOW_EX,
51     "light_blue": colorama.Fore.LIGHTBLUE_EX,
52     "light_magenta": colorama.Fore.LIGHTMAGENTA_EX,
53     "light_cyan": colorama.Fore.LIGHTCYAN_EX,
54 }
55
56 # Dictionary для цветов фона
57 BG_COLORS: Dict[str, str] = {
58     "black": colorama.Back.BLACK,
59     "red": colorama.Back.RED,
60     "green": colorama.Back.GREEN,
61     "yellow": colorama.Back.YELLOW,
62     "blue": colorama.Back.BLUE,
63     "magenta": colorama.Back.MAGENTA,
64     "cyan": colorama.Back.CYAN,
65     "white": colorama.Back.WHITE,
66     "light_gray": colorama.Back.LIGHTBLACK_EX,
67     "light_red": colorama.Back.LIGHTRED_EX,
68     "light_green": colorama.Back.LIGHTGREEN_EX,
```

```

69 |     "light_yellow": colorama.Back.LIGHTYELLOW_EX,
70 |     "light_blue": colorama.Back.LIGHTBLUE_EX,
71 |     "light_magenta": colorama.Back.LIGHTMAGENTA_EX,
72 |     "light_cyan": colorama.Back.LIGHTCYAN_EX,
73 | }
74 |
75 | class SingletonMeta(type):
76 |     """
77 |     Метакласс для реализации паттерна Singleton.
78 |     Гарантирует, что class имеет только один экземпляр и предо
79 | ставляет
80 |     глобальную точку доступа к этому экземпляру.
81 |
82 |     Потокбезопасен благодаря использованию Lock.
83 |     """
84 |     _instances: Dict = {}
85 |     _lock: threading.Lock = threading.Lock()
86 |
87 |     def __call__(cls, *args, **kwargs):
88 |         """
89 |         Creates или Returns существующий экземпляр класса.
90 |
91 |         Returns:
92 |             Единственный экземпляр класса.
93 |         """
94 |         if cls not in cls._instances:
95 |             with cls._lock:
96 |                 if cls not in cls._instances:
97 |                     instance = super().__call__(*args, **kwarg
98 | s)
99 |                     cls._instances[cls] = instance
100 |                 return cls._instances[cls]
101 |
102 | class JsonFormatter(logging.Formatter):
103 |     """
104 |     Кастомный форматтер для логирования в JSON формате.

```



```

105 | Преобразует LogRecord в JSON строку с полями:
106 | - timestamp: временная метка записи
107 | - level: уровень логирования (INFO, DEBUG и т.д.)
108 | - message: текст сообщения
109 | - exc_info: Info об исключении (если присутствует)
110 | """
111 |
112 | def format(self, record: logging.LogRecord) -> str:
113 |     """
114 |     Форматирует лог-запись в JSON.
115 |
116 |     Args:
117 |         record: LogRecord для форматирования.
118 |
119 |     Returns:
120 |         JSON string с информацией о логге.
121 |     """
122 |     log_entry: Dict = {
123 |         "timestamp": self.formatTime(record, self.datefmt)
124 |     },
125 |         "level": record.levelname,
126 |         "levelname": record.levelname,
127 |         "message": record.getMessage().replace("'", '"'),
128 |         "exc_info": self.formatException(record.exc_info)
129 |         if record.exc_info
130 |         else None,
131 |     }
132 |     return json.dumps(log_entry, ensure_ascii=False)
133 |
134 | class CompressingHandler(logging.handlers.RotatingFileHandler)
135 | :
136 |     """
137 |     Обработчик файловых логов с компрессией повторяющихся запи
138 |     сей и ротацией.
139 |
140 |     Особенности:
141 |     - Буферизирует записи и сжимает их в формате [Nx] message
142 |     - Поддерживает ротацию файлов по размеру

```

```

140 | - Потокбезопасен при многопоточном доступе
141 | - Гарантирует сохранение логов при падении процесса
142 |
143 | Parameters:
144 |     filename: Путь к файлу лога
145 |     maxBytes: Максимальный размер файла перед ротацией (по
    | умолчанию 10MB)
146 |     backupCount: Количество резервных файлов (по умолчанию
    | 5)
147 |     """
148 |
149 | def __init__(self, filename: str, maxBytes: int = 10 * 102
    | 4 * 1024,
150 |                 backupCount: int = 5, encoding: str = 'utf-8'
    | ):
151 |     """
152 |     Инициализирует обработчик логов с ротацией.
153 |
154 |     Args:
155 |         filename: Путь к файлу лога.
156 |         maxBytes: Максимальный размер файла в байтах.
157 |         backupCount: Количество сохраняемых резервных файл
    | ов.
158 |         encoding: Кодировка файла.
159 |     """
160 |     super().__init__(filename, maxBytes=maxBytes, backupCo
    | unt=backupCount,
161 |                     encoding=encoding, delay=False)
162 |     self.buffer: Dict[str, int] = {} # message -> count
163 |     self._lock = threading.Lock()
164 |     self._dirty = False
165 |
166 | def emit(self, record: logging.LogRecord) -> None:
167 |     """
168 |     Добавляет запись в буфер с компрессией.
169 |
170 |     Args:
171 |         record: LogRecord для добавления.

```

```

172         """
173         try:
174             msg = self.format(record)
175             with self._lock:
176                 self.buffer[msg] = self.buffer.get(msg, 0) + 1
177                 self._dirty = True
178
179             # Пишем если буфер переполнен или много уникал
180             ьных записей
181             if len(self.buffer) > 100:
182                 self.flush()
183         except Exception:
184             self.handleError(record)
185
186     def flush(self) -> None:
187         """Сбрасывает буфер в файл с компрессией повторяющихся
188         записей."""
189         with self._lock:
190             if not self.buffer or not self._dirty:
191                 return
192
193             try:
194                 # Пишем в файл
195                 if not self.stream:
196                     self.stream = self._open()
197
198                 for msg, count in self.buffer.items():
199                     if count > 1:
200                         self.stream.write(f"[{count}x] {msg}\n")
201
202                     else:
203                         self.stream.write(f"{msg}\n")
204
205                 self.stream.flush()
206                 self.buffer.clear()
207                 self._dirty = False
208             except Exception:
209                 pass

```

```

207
208     def close(self) -> None:
209         """Закрывает обработчик, предварительно сбрасывая буфе
        р."""
210         self.flush()
211         super().close()
212
213     def compress_lines(lines: list, min_repeat: int = 2) -> list:
214         """Сжимает повторяющиеся строки в формат [Nx] text."""
215         counter = Counter(lines)
216         result = []
217
218         for line, count in counter.items():
219             stripped = line.strip()
220             if not stripped:
221                 continue
222             if count >= min_repeat:
223                 result.append(f"[{count}x] {stripped}")
224             else:
225                 result.append(stripped)
226
227         return result
228
229     class Logger(metaclass=SingletonMeta):
230         """
231         Универсальный логгер с поддержкой цветного вывода, файловы
        х логов и JSON формата.
232
233         Реализует паттерн Singleton и гарантирует единственный экз
        емпляр логгера во всем приложении.
234
235         Основные возможности:
236         - Цветной вывод в консоль с поддержкой цветов текста и фон
        а
237         - Splitting логов по типам (info, debug, errors)
238         - JSON форматирование для структурированного анализа
239         - Ротация файлов логов по размеру
240         - Умная маршрутизация логов по модулям (FastAPI, Gemini, P

```

```

laywright и т.д.)
241     - Автоматическое disconnection DEBUG логов в production ре
жине
242
243     Атрибуты:
244         log_files_path: Путь к директории с логами
245         info_log_path: Путь к файлу информационных логов
246         debug_log_path: Путь к файлу отладочных логов
247         errors_log_path: Путь к файлу ошибок
248         json_log_path: Путь к JSON логам
249         is_debug_mode: Флаг режима отладки
250     """
251
252     def __init__(
253         self,
254         info_log_path: Optional[str] = None,
255         debug_log_path: Optional[str] = None,
256         errors_log_path: Optional[str] = None,
257         json_log_path: Optional[str] = None,
258     ):
259         """
260         Инициализирует логгер с заданными путями файлов логов.
261
262         Args:
263             info_log_path: Имя файла для INFO логов.
264             debug_log_path: Имя файла для DEBUG логов.
265             errors_log_path: Имя файла для ERROR логов.
266             json_log_path: Имя файла для JSON логов.
267         """
268         # Установка путей к файлам логов в системную temp дире
кторию (кроссплатформенно)
269         self.log_files_path: Path = Path(tempfile.gettempdir()
) / 'ai-breadboard' / 'logs'
270         self.info_log_path: Path = self.log_files_path / (info
_log_path or "info.log")
271         self.debug_log_path: Path = self.log_files_path / (deb
ug_log_path or "debug.log")
272         self.errors_log_path: Path = self.log_files_path / (er

```

```
rors_log_path or "errors.log")
273         self.json_log_path: Path = self.log_files_path / (json
_log_path or "log.json")
274         self.fastapi_log_path: Path = self.log_files_path / "f
astapi.log"
275         self.gemini_log_path: Path = self.log_files_path / "ge
mini.log"
276         self.playwright_log_path: Path = self.log_files_path /
"playwright.log"
277         self.yt_dlp_log_path: Path = self.log_files_path / "yt
_dlp.log"
278
279         # Создание директории с логами
280         self.log_files_path.mkdir(parents=True, exist_ok=True)
281
282         # Создание файлов логов
283         for log_path in [self.info_log_path, self.debug_log_pa
th, self.errors_log_path,
284                         self.json_log_path, self.fastapi_log_p
ath, self.gemini_log_path,
285                         self.playwright_log_path, self.yt_dlp_
log_path]:
286             log_path.touch(exist_ok=True)
287
288         # Консольный логгер
289         self.logger_console: logging.Logger = logging.getLogge
r("logger_console")
290         self.logger_console.setLevel(logging.DEBUG)
291         self.logger_console.propagate = False
292
293         # Определение режима отладки
294         self._setup_debug_mode()
295
296         # Установка обработчиков для разных типов логов
297         self._setup_file_handlers()
298
299         # Установка Module-специфичных логгеров
300         self._setup_module_loggers()
```

```

301
302     # Регистрация очистки при выходе
303     atexit.register(self._cleanup)
304
305     def _setup_debug_mode(self) -> None:
306         """Определяет, находится ли приложение в режиме отладк
и."""
307         try:
308             import os
309             from dotenv import load_dotenv
310             load_dotenv(__root__ / '.env')
311
312             try:
313                 from src.config import server_cfg
314                 mode_val = getattr(server_cfg, "mode", "dev").
lower()
315                 is_debug = getattr(server_cfg, "debug", True)
316             except ImportError:
317                 mode_val = os.getenv("MODE", "dev").lower()
318                 is_debug = os.getenv("DEBUG", "true").lower()
319
320             self.is_debug_mode = (mode_val in ('dev', 'debug')
or is_debug)
321         except Exception:
322             self.is_debug_mode = True # По умолчанию включаем
режим отладки
323
324     def _setup_file_handlers(self) -> None:
325         """Sets обработчики для файловых логов."""
326         # INFO логгер
327         self.logger_file_info: logging.Logger = logging.getLog
ger("logger_file_info")
328         self.logger_file_info.setLevel(logging.INFO)
329         self.logger_file_info.propagate = False
330         info_handler = logging.FileHandler(self.info_log_path,
encoding='utf-8')
331         info_handler.setFormatter(logging.Formatter("%(levelname

```

```

    me)s: %(message)s"))
332         self.logger_file_info.addHandler(info_handler)
333
334         # DEBUG логгер
335         self.logger_file_debug: logging.Logger = logging.getLo
gger("logger_file_debug")
336         self.logger_file_debug.setLevel(logging.DEBUG)
337         self.logger_file_debug.propagate = False
338         debug_handler = logging.FileHandler(self.debug_log_pat
h, encoding='utf-8')
339         debug_handler.setFormatter(logging.Formatter("%(leveln
ame)s: %(message)s"))
340         self.logger_file_debug.addHandler(debug_handler)
341
342         # ERROR логгер с компрессией
343         self.logger_file_errors: logging.Logger = logging.getL
ogger("logger_file_errors")
344         self.logger_file_errors.setLevel(logging.ERROR)
345         self.logger_file_errors.propagate = False
346         errors_handler = CompressingHandler(str(self.errors_lo
g_path), encoding='utf-8')
347         errors_handler.setFormatter(logging.Formatter("%(level
name)s: %(message)s"))
348         self.logger_file_errors.addHandler(errors_handler)
349
350         # JSON логгер
351         self.logger_file_json: logging.Logger = logging.getLog
ger("logger_json")
352         self.logger_file_json.setLevel(logging.DEBUG)
353         self.logger_file_json.propagate = False
354         json_handler = logging.FileHandler(self.json_log_path,
encoding='utf-8')
355         json_handler.setFormatter(JsonFormatter())
356         self.logger_file_json.addHandler(json_handler)
357
358     def _setup_module_loggers(self) -> None:
359         """Sets логгеры для отдельных модулей."""
360         formatter = logging.Formatter("%(asctime)s - %(levelname

```



```

me)s - %(message)s")
361
362     modules_config = [
363         ("logger_fastapi", self.fastapi_log_path),
364         ("logger_gemini", self.gemini_log_path),
365         ("logger_playwright", self.playwright_log_path),
366         ("logger_yt_dlp", self.yt_dlp_log_path),
367     ]
368
369     for logger_name, log_path in modules_config:
370         module_logger = logging.getLogger(logger_name)
371         module_logger.setLevel(logging.DEBUG)
372         module_logger.propagate = False
373         handler = CompressingHandler(str(log_path), encoding='utf-8')
374         handler.setFormatter(formatter)
375         module_logger.addHandler(handler)
376
377         # Сохраняем как атрибут класса
378         setattr(self, logger_name, module_logger)
379
380     def _get_caller_info(self, depth: int = 2) -> Tuple[str, str, int]:
381         """
382         Безопасно receives информацию о вызывающей функции.
383
384         Args:
385             depth: Глубина стека для поиска caller'а.
386
387         Returns:
388             Tuple (file_name, function_name, line_number).
389         """
390         try:
391             stack = inspect.stack()
392             if len(stack) > depth:
393                 frame_info = stack[depth]
394                 return (
395                     frame_info.filename,

```

```

396         frame_info.function,
397         frame_info.lineno
398     )
399     except Exception:
400         pass
401     return ("unknown", "unknown", 0)
402
403     def _format_message(self, message: str, ex: Optional[Exception] = None,
404                         color: Optional[Tuple[str, str]] = None
405     ) -> str:
406         """
407         Форматирует сообщение с опциональным цветом и информацией об исключении.
408
409         Args:
410             message: Текст сообщения.
411             ex: Exception для добавления.
412             color: Tuple (текст_цвет, фон_цвет).
413
414         Returns:
415             Отформатированное сообщение.
416         """
417         if color:
418             text_color, bg_color = color
419             text_color = TEXT_COLORS.get(text_color, colorama.Fore.RESET)
420             bg_color = BG_COLORS.get(bg_color, colorama.Back.RESET)
421             ex_str = f" {str(ex)}" if ex else ""
422             message = f"{text_color}{bg_color}{message}{ex_str}"
423             message = f"{message} {str(ex)}"
424             return message
425
426     def _route_to_module_logger(self, message: str, level: int

```

```

427         filename: str) -> None:
428     """
429     Маршрутизирует лог в Module-специфичный логгер на осно
ве пути файла.
430
431     Args:
432         message: Текст сообщения.
433         level: Уровень логирования.
434         filename: Путь к файлу вызывающего кода.
435     """
436     filename_lower = filename.lower().replace("\\", "/")
437
438     routing_map = [
439         ("fastapi", self.logger_fastapi),
440         ("main.py", self.logger_fastapi),
441         ("gemini", self.logger_gemini),
442         ("src/ai", self.logger_gemini),
443         ("src\\ai", self.logger_gemini),
444         ("playwright", self.logger_playwright),
445         ("torrent_playwright", self.logger_playwright),
446         ("yt_dlp", self.logger_yt_dlp),
447         ("yt-dlp", self.logger_yt_dlp),
448     ]
449
450     for pattern, module_logger in routing_map:
451         if pattern.lower() in filename_lower:
452             try:
453                 module_logger.log(level, message)
454             except Exception:
455                 pass
456             break
457
458     def log(self, level: int, message: str, ex: Optional[Excep
tion] = None,
459           exc_info: bool = False, color: Optional[Tuple[str,
str]] = None) -> None:
460         """
461         Логирует сообщение с заданным уровнем и параметрами.

```

```

462 |
463 |     Args:
464 |         level: Уровень логирования (logging.INFO, logging.
ERROR и т.д.).
465 |         message: Текст сообщения.
466 |         ex: Exception для логирования.
467 |         exc_info: Включить полную информацию об исключении
.
468 |         color: Tuple (текст_цвет, фон_цвет) для окраски.
469 |     """
470 |     # В production режиме игнорируем DEBUG логи
471 |     if level == logging.DEBUG and not self.is_debug_mode:
472 |         return
473 |
474 |     formatted_message = self._format_message(message, ex,
color)
475 |
476 |     # Логирование в консоль
477 |     if self.logger_console:
478 |         self.logger_console.log(level, formatted_message,
exc_info=exc_info)
479 |
480 |     # Логирование в JSON (без форматирования)
481 |     if self.logger_file_json:
482 |         self.logger_file_json.log(level, message, exc_info
=exc_info)
483 |
484 |     # Логирование по типам
485 |     if level == logging.INFO and self.logger_file_info:
486 |         self.logger_file_info.log(level, formatted_message
)
487 |     elif level == logging.DEBUG and self.logger_file_debug
:
488 |         self.logger_file_debug.log(level, formatted_messag
e)
489 |     elif level in [logging.ERROR, logging.CRITICAL] and se
lf.logger_file_errors:
490 |         self.logger_file_errors.log(level, formatted_messa

```

```

ge)
491
492     # Маршрутизация по модулям
493     try:
494         filename, func_name, line_no = self._get_caller_in
fo(depth=3)
495         clean_msg = str(message)
496         if ex:
497             clean_msg += f" {str(ex)}"
498         self._route_to_module_logger(clean_msg, level, fil
ename)
499     except Exception:
500         pass
501
502     def _cleanup(self) -> None:
503         """Очищает ресурсы при завершении приложения."""
504         try:
505             for logger_name in ["logger_console", "logger_file
_info", "logger_file_debug",
506                               "logger_file_errors", "logger_js
on", "logger_fastapi",
507                               "logger_gemini", "logger_playwri
ght", "logger_yt_dlp"]:
508                 logger_obj = getattr(self, logger_name, None)
509                 if logger_obj:
510                     for handler in logger_obj.handlers[:]:
511                         handler.flush()
512                         handler.close()
513                     logger_obj.removeHandler(handler)
514         except Exception:
515             pass
516
517     def info(self, message: str, ex: Optional[Exception] = Non
e, exc_info: bool = False,
518             text_color: str = "green", bg_color: str = "") ->
None:
519         """Логирует сообщение уровня INFO с зелёным цветом по
умолчанию."""

```

```

520         color = (text_color, bg_color) if bg_color else (text_
| color, "")
521         self.log(logging.INFO, message, ex, exc_info, color)
522
523     def success(self, message: str, ex: Optional[Exception] =
| None, exc_info: bool = False,
524                 text_color: str = "light_green", bg_color: str
| = "") -> None:
525         """Логирует сообщение об успешной операции с жёлтым цв
| етом по умолчанию."""
526         color = (text_color, bg_color) if bg_color else (text_
| color, "")
527         self.log(logging.INFO, message, ex, exc_info, color)
528
529     def warning(self, message: str, ex: Optional[Exception] =
| None, exc_info: bool = False,
530                text_color: str = "black", bg_color: str = "yel
| low") -> None:
531         """Логирует сообщение уровня WARNING с чёрным текстом
| на жёлтом фоне."""
532         color = (text_color, bg_color)
533         self.log(logging.WARNING, message, ex, exc_info, color
| )
534
535     def debug(self, message: str, ex: Optional[Exception] = No
| ne, exc_info: bool = False,
536               text_color: str = "cyan", bg_color: str = "") ->
| None:
537         """Логирует сообщение уровня DEBUG с голубым цветом."""
|
538         color = (text_color, bg_color) if bg_color else (text_
| color, "")
539         self.log(logging.DEBUG, message, ex, exc_info, color)
540
541     def error(self, message: str, ex: Optional[Exception] = No
| ne, exc_info: bool = True,
542               text_color: str = "red", bg_color: str = "") -> N
| one:

```

```
543 |         """Логирует сообщение уровня ERROR с красным цветом и
    | информация об исключении."""
544 |         color = (text_color, bg_color) if bg_color else (text_
    | color, "")
545 |         self.log(logging.ERROR, message, ex, exc_info, color)
546 |
547 |     def critical(self, message: str, ex: Optional[Exception] =
    | None, exc_info: bool = True,
548 |                 text_color: str = "white", bg_color: str = "re
    | d") -> None:
549 |         """Логирует критическую ошибку с белым текстом на крас
    | ном фоне."""
550 |         color = (text_color, bg_color)
551 |         self.log(logging.CRITICAL, message, ex, exc_info, colo
    | r)
552 |
553 | # Инициализация глобального экземпляра логгера (Singleton)
554 | logger: Logger = Logger()
555 | """Глобальный экземпляр логгера для использования во всем прил
    | ожении."""
556 |
```

log_analyzer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Intelligent log analyzer with Gemini AI integr
  ation
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: log_analyzer.py
9 # Project: ai-breadboard
10 # Package: src.logger
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """Intelligent log analyzer with Gemini AI integration.
16
17 Provides functionality for analyzing application logs using Go
  ogle Generative AI (Gemini),
18 automatic report generation, and Master Journal maintenance fo
  r system state tracking.
19
20 Functions:
21     - Log analysis with AI assistance
22     - Automatic report generation
23     - Master Journal creation and maintenance
24     - Detailed error report creation
25     - System state tracking
26     - Intelligent log rotation and cleanup
27 """
28
29 import os
30 import asyncio
```



```

31 import datetime
32 import logging
33 import tempfile
34 from pathlib import Path
35 from typing import Optional, List
36 from concurrent.futures import ThreadPoolExecutor
37 from src.ai import GoogleGenerativeAI
38 from src.logger import logger
39 from header import __root__
40
41 LOG_DIR: Path = Path(tempfile.gettempdir()) / 'ai-breadboard'
42 / 'logs'
43 REPORTS_DIR: Path = Path(tempfile.gettempdir()) / 'ai-breadboa
44 rd' / 'reports'
45 LOCK_FILE: Path = LOG_DIR / 'log_analyzer.lock'
46
47 # Максимальный размер файла лога в байтах (по умолчанию 10MB)
48 DEFAULT_MAX_SIZE_MB: float = 10.0
49 # Максимальное количество символов для анализа (для экономии к
50 воты AI)
51 MAX_CHARS_FOR_ANALYSIS: int = 500 * 1024 # 500KB
52 # Интервал проверки логов в секундах
53 CHECK_INTERVAL_SECONDS: int = 60
54 # Количество последних отчётов для включения в главный журнал
55 MAX_REPORTS_FOR_JOURNAL: int = 10
56
57 def get_max_size_bytes() -> float:
58     """
59     Receives максимальный размер лог-файла из конфигурации.
60
61     Returns:
62         float: Максимальный размер в байтах.
63
64     Raises:
65         Returns default value (10MB) if config is not availabl
66     e.
67     """
68     try:

```

```

65         from src.config import logging_cfg
66         mb = float(getattr(logging_cfg, "max_size_mb", DEFAULT
_MAX_SIZE_MB)) if logging_cfg else DEFAULT_MAX_SIZE_MB
67         return mb * 1024 * 1024
68     except Exception as ex:
69         logger.debug(f"Не удалось прочитать конфиг размера лог
a: {ex}")
70         return DEFAULT_MAX_SIZE_MB * 1024 * 1024
71
72 async def analyze_log_file(file_path: Path, ai_model: GoogleGe
nerativeAI) -> bool:
73     """
74     Анализирует лог-файл с помощью AI и creates отчёт.
75
76     Процесс:
77     1. Reads содержимое лог-файла
78     2. Sends на анализ Gemini AI (максимум 500KB для экономии
квоты)
79     3. Saves результат в виде Markdown отчёта
80     4. Очищает исходный лог-файл
81
82     Args:
83         file_path: Путь к лог-файлу для анализа.
84         ai_model: Инициализированный экземпляр GoogleGenerativ
eAI.
85
86     Returns:
87         bool: True если анализ прошёл successfully, False в сл
учае ошибки.
88     """
89     try:
90         # Check существования файла
91         if not file_path.exists() or not file_path.is_file():
92             logger.warning(f"Лог-файл не найден: {file_path}")
93             return False
94
95         # Чтение содержимого лога
96         try:

```

```

97         with open(file_path, 'r', encoding='utf-8', errors
|         = 'ignore') as f:
98             content = f.read()
99             except Exception as ex:
100                 logger.error(f"Error при чтении лог-файла {file_pa
| th.name}: {ex}")
101                 return False
102
103             # Пропуск пустых файлов
104             if not content.strip():
105                 logger.debug(f"Лог-файл {file_path.name} пуст, про
| пуск анализа")
106                 return True
107
108                 logger.info(f"Начало умного анализа лога {file_path.na
| me} (размер: {len(content)} символов)...")
109
110                 # Обрезаем контент если слишком большой (для экономии
| квоты)
111                 if len(content) > MAX_CHARS_FOR_ANALYSIS:
112                     logger.warning(f"Лог {file_path.name} слишком боль
| шой ({len(content)} символов), анализирую только последние {MA
| X_CHARS_FOR_ANALYSIS} символов")
113                     content = content[-MAX_CHARS_FOR_ANALYSIS:]
114
115                     # Formation промпта для AI
116                     prompt = f"""
117 Проанализируй следующий лог-файл ({file_path.name}).
118 Выяви ошибки, предупреждения, критические проблемы, а также об
| щие тренды и дай рекомендации по исправлению.
119 Ответ предоставь на русском языке в чистом формате Markdown.
120
121 Содержимое лога:
122 {content}
123 """
124
125                     # Получение анализа от AI
126                     try:

```

```

127         report_text = await ai_model.ask(prompt)
128     except Exception as ex:
129         logger.error(f"Error при запросе к Gemini AI: {ex}
130         ")
131         report_text = f"# Отчет об анализе лога {file_path
132         .name}\n\nError при запросе к AI: {ex}"
133
134     if not report_text:
135         report_text = f"# Отчет об анализе лога {file_path
136         .name}\n\nНе удалось получить анализ от модели Gemini."
137
138     # Сохранение отчёта
139     REPORTS_DIR.mkdir(parents=True, exist_ok=True)
140     timestamp = datetime.datetime.now().strftime("%Y%m%d_%
141     H%M%S")
142     report_name = f"report_{file_path.stem}_{timestamp}.md
143     "
144     report_path = REPORTS_DIR / report_name
145
146     try:
147         with open(report_path, 'w', encoding='utf-8') as f
148         :
149             f.write(report_text)
150     except Exception as ex:
151         logger.error(f"Error при сохранении отчёта {report
152         _name}: {ex}")
153         return False
154
155     # Очистка исходного лог-файла
156     try:
157         with open(file_path, 'w', encoding='utf-8') as f:
158             f.truncate(0)
159     except Exception as ex:
160         logger.error(f"Error при очистке лог-файла {file_p
161         ath.name}: {ex}")
162
163     # Это не критичная Error, продолжаем выполнение
164
165     logger.info(f"□ Лог {file_path.name} successfully проа

```

```

    нализирован. Отчет сохранен в {report_name}")
157         return True
158
159     except Exception as ex:
160         logger.error(f"Критическая Error при анализе лог-файла
    {file_path.name}: {ex}")
161         return False
162
163 async def update_master_journal(ai_model: GoogleGenerativeAI)
    -> bool:
164     """
165     Creates главный журнал (Master Journal) из последних отчёт
    ов анализа.
166
167     Процесс:
168     1. Собирает последние N отчётов об анализе
169     2. Sends их на анализ Gemini для синтеза
170     3. Saves результат в master_journal.md
171
172     Args:
173         ai_model: Инициализированный экземпляр GoogleGenerativ
    eAI.
174
175     Returns:
176         bool: True если журнал successfully обновлён, False в
    случае ошибки.
177     """
178     try:
179         if not REPORTS_DIR.exists():
180             logger.debug("Директория отчётов не существует, пр
    опуск обновления журнала")
181             return False
182
183         # Получение списка отчётов (исключая главный журнал)
184         report_files = [p for p in REPORTS_DIR.glob("report_*.
    md") if p.name != "master_journal.md"]
185         if not report_files:
186             logger.debug("Нет отчётов для создания главного жу

```

```

    рнала")
187         return False
188
189         # Sorting по времени последнего изменения (новые первы
ми)
190         report_files.sort(key=lambda p: p.stat().st_mtime, rev
erse=True)
191
192         # Сбор содержимого последних отчётов
193         reports_content: List[str] = []
194         for p in report_files[:MAX_REPORTS_FOR_JOURNAL]:
195             try:
196                 with open(p, 'r', encoding='utf-8', errors='ig
nore') as f:
197                     content = f.read()
198                     reports_content.append(f"### Файл: {p.name
}\n{content}\n")
199             except Exception as ex:
200                 logger.warning(f"Error при чтении отчёта {p.na
me}: {ex}")
201                 continue
202
203         if not reports_content:
204             logger.warning("Не удалось прочитать ни один отчёт
для главного журнала")
205             return False
206
207         logger.info(f"Создание главного журнала из {len(report
s_content)} отчётов...")
208
209         # Formation промпта для синтеза
210         prompt = f"""
211 На основе следующих индивидуальных отчетов об анализе логов со
ставь общий единый журнал (Master Journal) состояния системы.
212 Выдели:
213 1. Повторяющиеся и постоянные ошибки.
214 2. Новые обнаруженные тенденции.
215 3. Актуальные проблемы, требующие внимания разработчика на тек

```

ущий момент.

216 4. Устаревшие или неактуальные проблемы, которые перестали появляться.

217 5. Рекомендации по приоритизации исправлений.

218

219 Ответ выведи строго на русском языке в формате Markdown с чётким структурированием информации.

220

221 Индивидуальные отчеты:

222 {"".join(reports_content)}

223 """

224

225 # Получение синтеза от AI

226 try:

227 master_text = await ai_model.ask(prompt)

228 except Exception as ex:

229 logger.error(f"Error при запросе к Gemini AI для главного журнала: {ex}")

230 master_text = f"# Общий журнал анализа логов\n\nError при создании синтеза: {ex}"

231

232 if not master_text:

233 master_text = "# Общий журнал анализа логов\n\nНе удалось получить общий анализ от модели Gemini."

234

235 # Сохранение главного журнала

236 master_path = REPORTS_DIR / "master_journal.md"

237 try:

238 with open(master_path, 'w', encoding='utf-8') as f:

239 f.write(master_text)

240 except Exception as ex:

241 logger.error(f"Error при сохранении главного журнала: {ex}")

242 return False

243

244 logger.info("□ Общий журнал (master_journal.md) successfully обновлен")

```
245         return True
246
247     except Exception as ex:
248         logger.error(f"Критическая Error при обновлении общего
журнала логов: {ex}")
249         return False
250
251 async def log_analyzer_loop() -> None:
252     """
253     Главный цикл анализатора логов, выполняемый в фоновом пото
ке.
254
255     Функциональность:
256     1. Checks конфигурацию включения анализатора
257     2. Инициализирует AI модель
258     3. Периодически (каждые N секунд) checks размер лог-файлов
259     4. Запускает анализ файлов превысивших лимит размера
260     5. Обновляет главный журнал после каждого анализа
261
262     Runs indefinitely until application shutdown.
263     """
264     logger.info("Запуск фоновой службы интеллектуального анали
за логов...")
265
266     # Проверяем, включён ли анализ логов в конфигурации
267     try:
268         from src.config import logging_cfg
269         if not logging_cfg or not getattr(logging_cfg, "enable
_log_analyzer", False):
270             logger.info("Анализ логов отключён в config.json")
271             return
272     except Exception as ex:
273         logger.warning(f"Не удалось прочитать конфиг анализато
ра: {ex}")
274         return
275
276     # Initialization AI модели
277     try:
```



```

278 |         api_key_names = [n.strip() for n in os.getenv('GEMINI_
API_KEY_NAMES', '').split(',') if n.strip()]
279 |         if not api_key_names:
280 |             logger.warning("Переменная окружения GEMINI_API_KE
Y_NAMES не установлена, анализ логов отключен")
281 |             return
282 |
283 |         system_instruction = "Вы – профессиональный аналитик с
истемных логов. Ваша задача – исследовать логи, выявлять ошибок
и, проблемы, тренды и давать рекомендации по устранению."
284 |         ai_model = GoogleGenerativeAI(
285 |             api_key_names=api_key_names,
286 |             system_instruction=system_instruction
287 |         )
288 |     except Exception as ex:
289 |         logger.error(f"Error при инициализации AI модели: {ex}
")
290 |         return
291 |
292 |     logger.info("Анализатор логов successfully инициализирован
")
293 |
294 |     # Главный цикл
295 |     while True:
296 |         try:
297 |             if not LOG_DIR.exists():
298 |                 await asyncio.sleep(CHECK_INTERVAL_SECONDS)
299 |                 continue
300 |
301 |             max_bytes = get_max_size_bytes()
302 |
303 |             # Анализ обычных лог-файлов
304 |             for log_file in LOG_DIR.glob("*.log"):
305 |                 if log_file.is_file() and log_file.stat().st_s
ize >= max_bytes:
306 |                     success = await analyze_log_file(log_file,
ai_model)
307 |                     if success:

```

```

308         await update_master_journal(ai_model)
309         await asyncio.sleep(1) # Небольшая задерж
ка между анализами
310
311         # Анализ JSON лога
312         json_log = LOG_DIR / "log.json"
313         if json_log.exists() and json_log.is_file() and js
on_log.stat().st_size >= max_bytes:
314             success = await analyze_log_file(json_log, ai_
model)
315             if success:
316                 await update_master_journal(ai_model)
317
318         except Exception as ex:
319             logger.error(f"Error в цикле анализатора логов: {e
x}")
320
321         # Ожидание перед следующей проверкой
322         await asyncio.sleep(CHECK_INTERVAL_SECONDS)
323
324 def start_log_analyzer() -> None:
325     """
326     Запускает анализатор логов в фоновом режиме.
327
328     Особенности:
329     - Гарантирует, что анализатор запускается только в одном п
роцессе
330     - Использует lock-файл для синхронизации между процессами
331     - Checks живость процесса перед запуском нового
332     - Безопасно processes ошибки инициализации
333
334     Raises:
335     Не выбрасывает исключений, все ошибки логируются.
336     """
337     try:
338         # Создаём директорию логов если её нет
339         LOG_DIR.mkdir(parents=True, exist_ok=True)
340

```

```
341 |         # Проверяем lock-файл
342 |         if LOCK_FILE.exists():
343 |             try:
344 |                 with open(LOCK_FILE, 'r', encoding='utf-8') as
f:
345 |                     pid = int(f.read().strip())
346 |
347 |                     # Проверяем, жив ли процесс
348 |                     try:
349 |                         import psutil
350 |                         if psutil.pid_exists(pid):
351 |                             logger.info(f"Анализатор логов уже зап
ущен (PID: {pid})")
352 |                             return
353 |                     except ImportError:
354 |                         # psutil не установлен, пропускаем проверк
у живости
355 |                         logger.debug("psutil не установлен, пропус
к проверки процесса")
356 |                         return
357 |                     except Exception as ex:
358 |                         logger.debug(f"Не удалось проверить процес
с {pid}: {ex}")
359 |
360 |                     except (ValueError, OSError) as ex:
361 |                         logger.debug(f"Error при чтении lock-файла: {e
x}")
362 |
363 |                     # Lock-файл устаревший, удаляем
364 |                     try:
365 |                         LOCK_FILE.unlink(missing_ok=True)
366 |                     except Exception as ex:
367 |                         logger.warning(f"Не удалось удалить старый loc
k-файл: {ex}")
368 |
369 |                     # Создаём lock-файл с текущим PID
370 |                     try:
371 |                         import subprocess
```

```
372         pid = subprocess.os.getpid()
373         LOCK_FILE.write_text(str(pid), encoding='utf-8')
374     except Exception as ex:
375         logger.error(f"Error при создании lock-файла: {ex}
376     ")
377     return
378
379     # Запускаем цикл анализатора в asyncio
380     try:
381         task = asyncio.create_task(log_analyzer_loop())
382         logger.debug("Асинхронная задача анализатора логов
383     создана")
384     except RuntimeError as ex:
385         # Нет активного event loop, пробуем запустить в фо
386     новом потоке
387         logger.debug(f"Event loop не активен ({ex}), пропу
388     ск запуска анализатора")
389
390     except Exception as ex:
391         logger.error(f"Error при запуске анализатора логов: {e
392     x}")
393
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Module
4 # =====
5 # Description:
6 #   Module for AI Breadboard project.
7 #
8 # File: __init__.py
9 # Project: ai-breadboard
10 # Package: src.logger
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from src.logger.logger import logger
16
```

prompt_loader.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Prompt loader and RAG semantic search integrat
  ion
4 # =====
5 # Description:
6 #   Module for loading system prompts for AI agents with FAISS
  -based semantic search.
7 #   Provides both RAG-mode (with semantic search) and static f
  allback mode.
8 #
9 # File: prompt_loader.py
10 # Project: ai-breadboard
11 # Package: src
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import json
17 import sys
18 from dataclasses import dataclass
19 from pathlib import Path
20
21 ## Base paths
22 _PROMPTS_ROOT: Path = Path(__file__).resolve().parent.parent /
  "prompts"
23 _RAG_DIR: Path = Path(__file__).resolve().parent.parent / "tmp
  " / "rag"
24 _INDEX_PATH: Path = _RAG_DIR / "rules.index"
25 _DOCUMENTS_PATH: Path = _RAG_DIR / "documents.json"
26
27 ## Embeddings model (same as in build_rules_index.py)
28 _MODEL_NAME: str = "sentence-transformers/all-MiniLM-L6-v2"
```

```

29 |
30 | ## Modules common to both agents (static mode)
31 | _CORE_MODULES: list[str] = [
32 |     "core/identity.md",
33 |     "core/categories.md",
34 | ]
35 |
36 | ## Modules only for chat agent (static mode)
37 | _CHAT_MODULES: list[str] = [
38 |     "chat/chat_rules.md",
39 |     "narrator/narrator_style.md",
40 | ]
41 |
42 | ## Modules only for narrator agent (static mode)
43 | _NARRATOR_MODULES: list[str] = [
44 |     "narrator/tts_rules.md",
45 |     "narrator/narrator_style.md",
46 | ]
47 |
48 | ## Example (static mode)
49 | _EXAMPLE_MODULE: str = "examples/series_example.json"
50 |
51 | ## JSON schema (static mode)
52 | _SCHEMA_MODULE: str = "core/output_schema.json"
53 |
54 | ## Files always included in RAG prompt (regardless of search).
55 | ## Only small base modules – schema and examples are fetched v
    | ia FAISS.
56 | _ALWAYS_INCLUDE: list[str] = [
57 |     "identity.md",
58 |     "categories.md",
59 | ]
60 |
61 | ## -----
    | -----
62 | ## Helper functions
63 | ## -----
    | -----

```

```

64
65 def _read_file(relative_path: str) -> str:
66     """
67     Read file from prompts directory.
68
69     Args:
70         relative_path (str): Path relative to _PROMPTS_ROOT.
71
72     Returns:
73         str: File content.
74
75     Raises:
76         FileNotFoundError: If file not found.
77     """
78     full_path: Path = _PROMPTS_ROOT / relative_path
79     if not full_path.exists():
80         raise FileNotFoundError(f"Prompt module not found: {full_path}")
81     return full_path.read_text(encoding="utf-8")
82
83 def _load_modules(module_paths: list[str]) -> str:
84     """
85     Load and concatenate multiple prompt modules.
86
87     Args:
88         module_paths (list[str]): List of relative paths to modules.
89
90     Returns:
91         str: Combined content of all modules.
92     """
93     parts: list[str] = [_read_file(p).strip() for p in module_paths]
94     return "\n\n---\n\n".join(parts)
95
96 def _load_schema_block() -> str:
97     """
98     Load JSON schema and wrap in markdown block.

```



```

99
100     Returns:
101         str: String with JSON schema in markdown wrapper.
102     """
103     raw: str = _read_file(_SCHEMA_MODULE)
104     parsed: dict = json.loads(raw)
105     formatted: str = json.dumps(parsed, ensure_ascii=False, in
    dent=2)
106     return f"## JSON Response Schema\n\n```json\n{formatted}\n
    ```"
107
108 def _load_example_block() -> str:
109 """
110 Load example JSON and wrap in markdown block.
111
112 Returns:
113 str: String with example in markdown wrapper.
114 """
115 raw: str = _read_file(_EXAMPLE_MODULE)
116 parsed: dict = json.loads(raw)
117 formatted: str = json.dumps(parsed, ensure_ascii=False, in
 dent=2)
118 return f"## Example Filled Response (Series)\n\n```json\n{
 formatted}\n```"
119
120 ## -----

121 ## RulesRAG – semantic search class
122 ## -----

123
124 from src.rag.rules_rag import RulesRAG, RulesSearchResult
125
126 # Alias for backward compatibility
127 SearchResult = RulesSearchResult
128
129 ## -----

```

```

130 ## Public prompt building functions (RAG mode)
131 ## -----
132
133 def load_chat_prompt(query: str = "Create media description fo
r chat") -> str:
134 """
135 Build system prompt for Chat Agent via FAISS search.
136
137 Args:
138 query (str): Query used to select relevant modules.
139 Default: universal query for chat agent.
140
141 Returns:
142 str: Ready system prompt for chat agent (~5,000-8,000
characters).
143 """
144 rag: RulesRAG = RulesRAG()
145 context: str = rag.build_context(query, top_k=4)
146 return context
147
148 def load_narrator_prompt(query: str = "Prepare text for voice
narrator TTS") -> str:
149 """
150 Build system prompt for Narrator Agent via FAISS search.
151
152 Args:
153 query (str): Query used to select relevant modules.
154 Default: query for TTS mode.
155
156 Returns:
157 str: Ready system prompt for narrator agent (~5,000-8,
000 characters).
158 """
159 rag: RulesRAG = RulesRAG()
160 context: str = rag.build_context(query, top_k=4)
161 return context
162

```

```

163 ## -----
164 ## Static fallback functions (without FAISS)
165 ## -----
166
167 def load_chat_prompt_static() -> str:
168 """
169 Build complete prompt for Chat Agent from files without FA
170 ISS.
171 Used as fallback if index not built.
172
173 Returns:
174 str: Complete prompt for chat agent (~20,000 character
175 s).
176 """
177 sections: list[str] = [
178 _load_modules(_CORE_MODULES),
179 _load_schema_block(),
180 _load_modules(_CHAT_MODULES),
181 _load_example_block(),
182]
183 return "\n\n---\n\n".join(sections)
184
185 def load_narrator_prompt_static() -> str:
186 """
187 Build complete prompt for Narrator Agent from files withou
188 t FAISS.
189 Used as fallback if index not built.
190
191 Returns:
192 str: Complete prompt for narrator agent (~20,000 chara
193 cters).
194 """
195 sections: list[str] = [
196 _load_modules(_CORE_MODULES),

```

```

195 _load_schema_block(),
196 _load_modules(_NARRATOR_MODULES),
197 _load_example_block(),
198]
199 return "\n\n---\n\n".join(sections)
200
201 ## -----
202 |-----
203 ## Entry point – quick check
204 ## -----
205 |-----
206
207 if __name__ == "__main__":
208 sys.stdout.reconfigure(encoding="utf-8")
209
210 print("=== Static Mode ===")
211 chat_s: str = load_chat_prompt_static()
212 narrator_s: str = load_narrator_prompt_static()
213 print(f"Chat (static): {len(chat_s):>6} characters |
214 | {len(chat_s.split()):>5} words")
215 print(f"Narrator (static): {len(narrator_s):>6} characters
216 | {len(narrator_s.split()):>5} words")
217
218 print()
219 if not _INDEX_PATH.exists():
220 print("FAISS index not found. Run: python rag/build_ru
221 | les_index.py")
222 sys.exit(0)
223
224 print("=== RAG Mode ===")
225 rag: RulesRAG = RulesRAG()
226
227 test_queries: list[str] = [
228 "Series description",
229 "Rules for TTS narrator",
230 "Media categories action spies",
231]
232 for q in test_queries:

```

```
228 | results = rag.search(q, top_k=3)
229 | files: list[str] = [r.file for r in results]
230 | print(f" '{q}' → {files}")
231 |
232 | print()
233 | chat_r: str = load_chat_prompt("Series description")
234 | narrator_r: str = load_narrator_prompt("Text for narrator"
 |)
235 | print(f"Chat (RAG): {len(chat_r):>6} characters |
 | {len(chat_r.split()):>5} words")
236 | print(f"Narrator (RAG): {len(narrator_r):>6} characters
 | | {len(narrator_r.split()):>5} words")
237 |
238 | reduction: float = (1 - len(chat_r) / len(chat_s)) * 100
239 | print(f"\nPrompt compression: {reduction:.0f}%")
240 |
```

engine.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Domain-independent RAG-First algorithm impleme
 ntation
4 # =====
5 # Description:
6 # Implements clean domain-independent RAG-First algorithm fo
 r
7 #
8 # File: engine.py
9 # Project: ai-breadboard
10 # Package: src.rag
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from __future__ import annotations
16
17 import asyncio
18 from typing import Any, Dict, List, Optional
19
20 from src.logger import logger
21 from src.rag.models import RAGDecisionType, RAGRouteDecision,
 RAGSearchResult
22 from src.rag.user_rag import search_user_history, get_user_pre
 ferences_context
23
24 class RAGEngine:
25 """
26 ## hypo69 docblock
27 Универсальный координатор поиска по базе знаний и маршрути
 зации RAG-First.
28 """
```

```

29 |
30 | def __init__(self, direct_threshold: float = 0.85) -> None
 | :
31 | """
32 | ## hypo69 docblock
33 | Initialization RAG-движка.
34 |
35 | Args:
36 | direct_threshold (float): Порог уверенности для пр
ямого возврата ответа без вызова LLM.
37 | """
38 | self.direct_threshold = direct_threshold
39 |
40 | async def evaluate(
41 | self,
42 | query: str,
43 | user_identifier: str = "",
44 | api_key: str = "",
45 | threshold: Optional[float] = None
46 |) -> RAGRouteDecision:
47 | """
48 | ## hypo69 docblock
49 | Оценивает входящий запрос: ищет готовый ответ в базе и
ли готовит контекст для LLM.
50 |
51 | Args:
52 | query (str): Запрос пользователя.
53 | user_identifier (str): Идентификатор пользователя.
54 | api_key (str): Ключ API.
55 | threshold (Optional[float]): Порог уверенности для
прямого ответа.
56 |
57 | Returns:
58 | RAGRouteDecision: Решение движка (прямой ответ или
fallback к модели).
59 | """
60 | clean_query = query.strip()
61 | if not clean_query:

```

```

62 return RAGRouteDecision(
63 decision_type=RAGDecisionType.DIRECT_ANSWER,
64 is_direct=True,
65 direct_text="Пожалуйста, введите запрос.",
66 direct_voice="Пожалуйста, введите запрос.",
67)
68
69 active_threshold = threshold if threshold is not None
else self.direct_threshold
70 context_parts: List[str] = []
71
72 # 1. Поиск по базе знаний / предыдущим сохраненным отве
там
73 if user_identifier and api_key and len(clean_query) >=
3:
74 results = await search_user_history(
75 user_identifier,
76 api_key,
77 clean_query,
78 top_k=3,
79 threshold=0.45
80)
81
82 if results:
83 best_match = results[0]
84 best_score = float(best_match.get("score", 0.0
))
85
86 # Если есть точный ответ с высоким качеством с
овпадения
87 if best_score >= active_threshold and best_mat
ch.get("text"):
88 matched_text = best_match["text"].strip()
89 logger.info(f"[RAGEngine] Найден прямой от
вет в RAG (score={best_score:.2f} >= {active_threshold})")
90 return RAGRouteDecision(
91 decision_type=RAGDecisionType.DIRECT_A
NSWER,

```



```

92 is_direct=True,
93 direct_text=matched_text,
94 direct_voice=matched_text,
95 confidence_score=best_score,
96 raw_results=results,
97 status_message="❑ Ответ найден в базе
знаний...",
98)
99
100 # Иначе собираем найденные фрагменты как конте
кст для LLM
101 snippets = [item["text"].strip() for item in r
esults if item.get("text")]
102 if snippets:
103 context_parts.append("[Контекст из базы зн
аний]:\n" + "\n---\n".join(snippets))
104
105 # 2. Добавление профиля предпочтений пользователя (есл
и доступно)
106 if user_identifier:
107 pref_context = await get_user_preferences_context(
user_identifier)
108 if pref_context:
109 context_parts.append(f"[Профиль предпочтений]:
\n{pref_context}")
110
111 return RAGRouteDecision(
112 decision_type=RAGDecisionType.LLM_FALLBACK,
113 is_direct=False,
114 context_text="\n\n".join(context_parts),
115 confidence_score=0.0,
116 status_message="Генерация ответа...",
117)
118
119 _engine_instance: Optional[RAGEngine] = None
120
121 def get_rag_engine() -> RAGEngine:
122 """

```

```
123 | ## hypo69 docblock
124 | Returns синглтон RAGEngine.
125 | """
126 | global _engine_instance
127 | if not _engine_instance:
128 | _engine_instance = RAGEngine()
129 | return _engine_instance
130 |
```

## models.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: RAG router decision types and data models
4 # =====
5 # Description:
6 # Dataclasses and enumerations for universal domain-independ
7 # ent RAG routing decisions.
8 #
9 # File: models.py
10 # Project: ai-breadboard
11 # Package: src.rag
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 from __future__ import annotations
17
18 from dataclasses import dataclass, field
19 from enum import Enum
20 from typing import Any, Dict, List
21
22 class RAGDecisionType(str, Enum):
23 """RAG router decision type."""
24 DIRECT_ANSWER = "direct_answer"
25 LLM_FALLBACK = "llm_fallback"
26
27 @dataclass
28 class RAGSearchResult:
29 """Result of semantic search in RAG index."""
30 title: str = ""
31 file: str = ""
32 path: str = ""
33 text: str = ""
```

```

33 | score: float = 0.0
34 | metadata: Dict[str, Any] = field(default_factory=dict)
35 |
36 | @dataclass
37 | class RAGRouteDecision:
38 | """Decision of RAG engine for processing user request."""
39 | decision_type: RAGDecisionType = RAGDecisionType.LLM_FALLB
 ACK
40 | is_direct: bool = False
41 | direct_text: str = ""
42 | direct_voice: str = ""
43 | context_text: str = ""
44 | confidence_score: float = 0.0
45 | raw_results: List[Dict[str, Any]] = field(default_factory=
 list)
46 | status_message: str = ""
47 |

```

## rules\_rag.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Semantic search indexing for rules and prompts
4 # =====
5 # Description:
6 # FAISS-based vector indexing and semantic search for module
7 # prompts
8 # and rules from the prompts directory.
9 #
10 # File: rules_rag.py
11 # Project: ai-breadboard
12 # Package: src.rag
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 from __future__ import annotations
18
19 import json
20 import sys
21 import tempfile
22 from dataclasses import dataclass
23 from pathlib import Path
24 from typing import List, Dict, Any, Tuple
25
26 from src.logger import logger
27 from header import __root__
28
29 _AI_PROMPTS_ROOT: Path = __root__ / ".ai" / "prompts"
30 _PROMPTS_ROOT: Path = _AI_PROMPTS_ROOT if _AI_PROMPTS_ROOT.exists() else (__root__ / "prompts")
31
32 _TMP_RAG_DIR: Path = Path(tempfile.gettempdir()) / 'ai-breadboard' / 'rag'
```

```

31 _DEFAULT_INDEX_PATH: Path = _TMP_RAG_DIR / "rules.index"
32 _DEFAULT_DOCUMENTS_PATH: Path = _TMP_RAG_DIR / "documents.json"
33
34 _LEGACY_RAG_DIR: Path = __root__ / "rag"
35 _LEGACY_INDEX_PATH: Path = _LEGACY_RAG_DIR / "rules.index"
36 _LEGACY_DOCUMENTS_PATH: Path = _LEGACY_RAG_DIR / "documents.js
 on"
37
38 _MODEL_NAME: str = "sentence-transformers/all-MiniLM-L6-v2"
39 _TARGET_GLOBS: List[str] = ["**/*.md", "**/*.json"]
40 _EXCLUDE_NAMES: set[str] = {"README.md"}
41
42 @dataclass
43 class RulesSearchResult:
44 """Search result from the rules index."""
45 file: str
46 path: str
47 text: str
48 score: float
49
50 def collect_prompt_documents(prompts_root: Path = _PROMPTS_ROOT
 T) -> List[Dict[str, str]]:
51 """
52 ## hypo69 docblock
53 Collects document corpus from the prompts directory.
54
55 Args:
56 prompts_root (Path): Root directory path of prompt fil
 es.
57
58 Returns:
59 List[Dict[str, str]]: List of documents in format {"fi
 le": str, "path": str, "text": str}.
60 """
61 documents: List[Dict[str, str]] = []
62 if not prompts_root.exists():
63 logger.warning(f"[RulesRAG] Prompts directory not foun

```

```

 d: {prompts_root}")
64 return documents
65
66 for glob_pat in _TARGET_GLOBS:
67 for file_path in sorted(prompts_root.glob(glob_pat)):
68 if not file_path.is_file() or file_path.name in _E
XCLUDE_NAMES:
69 continue
70
71 rel_path: str = str(file_path.relative_to(prompts_
root)).replace("\\", "/")
72 raw_text: str = file_path.read_text(encoding="utf-
8").strip()
73
74 if file_path.suffix == ".json":
75 text: str = f"[File: {file_path.name}]\n{raw_t
ext}"
76 else:
77 text = raw_text
78
79 documents.append({
80 "file": file_path.name,
81 "path": rel_path,
82 "text": text,
83 })
84 return documents
85
86 def build_rules_index(
87 prompts_root: Path = _PROMPTS_ROOT,
88 output_dir: Path = _TMP_RAG_DIR,
89) -> Tuple[Path, Path]:
90 """
91 ## hypo69 docblock
92 Builds a FAISS index from prompt files and saves the docum
ents and index.
93
94 Args:
95 prompts_root (Path): Path to the prompts directory.

```

```

96 | output_dir (Path): Directory for saving rules.index an
 | d documents.json.
97 |
98 | Returns:
99 | Tuple[Path, Path]: Paths to the saved index and docume
 | nts file.
100 | """
101 | import faiss
102 | import numpy as np
103 | from sentence_transformers import SentenceTransformer
104 |
105 | output_dir.mkdir(parents=True, exist_ok=True)
106 | index_path = output_dir / "rules.index"
107 | docs_path = output_dir / "documents.json"
108 |
109 | documents = collect_prompt_documents(prompts_root)
110 | if not documents:
111 | raise RuntimeError(f"Prompt documents not found in {pr
 | ompts_root}")
112 |
113 | docs_path.write_text(
114 | json.dumps(documents, ensure_ascii=False, indent=2),
115 | encoding="utf-8"
116 |)
117 |
118 | texts = [doc["text"] for doc in documents]
119 | model = SentenceTransformer(_MODEL_NAME)
120 | vectors = model.encode(texts, show_progress_bar=False, con
 | vert_to_numpy=True)
121 |
122 | dimension = vectors.shape[1]
123 | index = faiss.IndexFlatL2(dimension)
124 | index.add(vectors.astype(np.float32))
125 |
126 | faiss.write_index(index, str(index_path))
127 | logger.info(f"[RulesRAG] Index successfully built: {index_
 | path} ({len(documents)} documents)")
128 | return index_path, docs_path

```



```

129 |
130 | _ALWAYS_INCLUDE: List[str] = [
131 | "identity.md",
132 | "categories.md",
133 |]
134 |
135 | class RulesRAG:
136 | """
137 | ## hypo69 docblock
138 | Semantic search across module prompts using a vector FAISS
139 | index.
140 | """
141 | def __init__(self, index_path: Path = _DEFAULT_INDEX_PATH,
142 | docs_path: Path = _DEFAULT_DOCUMENTS_PATH) -> None:
143 | """
144 | ## hypo69 docblock
145 | Initializes and loads the rules index and document cor
146 | pus.
147 | """
148 | # Search for index: first in tmp/rag, then in fallback
149 | paths
150 | resolved_index = index_path
151 | resolved_docs = docs_path
152 | if not resolved_index.exists() and _LEGACY_INDEX_PATH.
153 | exists():
154 | resolved_index = _LEGACY_INDEX_PATH
155 | resolved_docs = _LEGACY_DOCUMENTS_PATH
156 |
157 | if not resolved_index.exists() or not resolved_docs.ex
158 | ists():
159 | try:
160 | resolved_index, resolved_docs = build_rules_in
161 | dex(_PROMPTS_ROOT, _TMP_RAG_DIR)
162 | except Exception as e:
163 | raise FileNotFoundError(
164 | f"Failed to find or rebuild FAISS rules in
165 | dex: {e}"

```

```

159)
160
161 import faiss
162 import numpy as np
163 from sentence_transformers import SentenceTransformer
164
165 self._faiss = faiss
166 self._np = np
167 self._index = faiss.read_index(str(resolved_index))
168 self._documents: List[Dict[str, Any]] = json.loads(
169 resolved_docs.read_text(encoding="utf-8")
170)
171 self._model: SentenceTransformer = SentenceTransformer
172 (
173 _MODEL_NAME,
174 local_files_only=True,
175)
176
177 def search(self, query: str, top_k: int = 4) -> List[Rules
178 SearchResult]:
179 """
180 ## hypo69 docblock
181 Performs semantic search across the rules corpus.
182
183 Args:
184 query (str): Query in natural language.
185 top_k (int): Number of results to return.
186
187 Returns:
188 List[RulesSearchResult]: List of relevant module p
189 rompts.
190 """
191 if not query.strip():
192 return []
193
194 vector = self._model.encode([query], convert_to_numpy=
195 True).astype(self._np.float32)
196 distances, indices = self._index.search(vector, top_k)

```

```

193 |
194 | results: List[RulesSearchResult] = []
195 | for dist, idx in zip(distances[0], indices[0]):
196 | if idx < 0 or idx >= len(self._documents):
197 | continue
198 | doc = self._documents[idx]
199 | results.append(RulesSearchResult(
200 | file=doc["file"],
201 | path=doc["path"],
202 | text=doc["text"],
203 | score=float(dist),
204 |))
205 | return results
206 |
207 | def build_context(self, query: str, top_k: int = 4) -> str
208 | :
209 | """
210 | ## hypo69 docblock
211 | Builds text context from found documents for insertion
212 | into the system prompt.
213 | """
214 | always_docs: List[str] = []
215 | for doc in self._documents:
216 | if doc["file"] in _ALWAYS_INCLUDE:
217 | always_docs.append(doc["text"].strip())
218 |
219 | search_results = self.search(query, top_k=top_k)
220 | search_docs: List[str] = []
221 | for result in search_results:
222 | if result.file not in _ALWAYS_INCLUDE:
223 | search_docs.append(result.text.strip())
224 |
225 | all_parts = always_docs + search_docs
226 | return "\n\n---\n\n".join(all_parts)

```

user\_rag.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: User dialogue indexing and semantic search
4 # =====
5 # Description:
6 # Dialog indexing, caching, and semantic search across previ
7 # ously provided responses.
8 #
9 # File: user_rag.py
10 # Project: ai-breadboard
11 # Package: src.rag
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16
17 from __future__ import annotations
18
19
20 import asyncio
21 from typing import Any, Dict, List
22
23
24 from src.logger import logger
25 from src.ai.gemini.user_query_rag import index_user_query, sea
26 rch_user_context
27 from src.ai.gemini.chat_response_store import save_approved_re
28 sponse
29 from src.user_manager.user_profile import get_recommendation_c
30 ontext
31
32
33 async def search_user_history(
34 user_identifier: str,
35 api_key: str,
36 query: str,
37 top_k: int = 2,
```

```

30 threshold: float = 0.45
31) -> List[Dict[str, Any]]:
32 """Search for relevant context from previous user discussions.
33
34 Args:
35 user_identifier (str): User identifier.
36 api_key (str): API key for vectorization.
37 query (str): Query text.
38 top_k (int): Number of results to return.
39 threshold (float): Similarity threshold.
40
41 Returns:
42 List[Dict[str, Any]]: Found discussion fragments matching the query.
43 """
44 if not user_identifier or not api_key or len(query.strip()) < 5:
45 return []
46
47 try:
48 results = await asyncio.to_thread(
49 search_user_context, user_identifier, api_key, query, top_k, threshold
50)
51 return results or []
52 except Exception as ex:
53 logger.error(f"[UserRAG] Error searching user context: {ex}")
54 return []
55
56 async def get_user_preferences_context(user_identifier: str) -> str:
57 """Return text context of user preferences.
58
59 Args:
60 user_identifier (str): User identifier.
61

```

```

62 | Returns:
63 | str: Text representation of user preferences.
64 | """
65 | if not user_identifier:
66 | return ""
67 | try:
68 | pref = await asyncio.to_thread(get_recommendation_cont
ext, user_identifier)
69 | return pref or ""
70 | except Exception as ex:
71 | logger.error(f"[UserRAG] Error reading user preference
s: {ex}")
72 | return ""
73 |
74 | def save_user_approved_response(
75 | user_identifier: str,
76 | query: str,
77 | chat_text: str,
78 | voice_text: str
79 |) -> bool:
80 | """Save approved response to permanent JSON archive storag
e.
81 |
82 | Args:
83 | user_identifier (str): User identifier.
84 | query (str): User query text.
85 | chat_text (str): Response text in chat format.
86 | voice_text (str): Response text in voice format.
87 |
88 | Returns:
89 | bool: Success flag indicating archive save operation.
90 | """
91 | try:
92 | return save_approved_response(user_identifier, query,
chat_text, voice_text)
93 | except Exception as ex:
94 | logger.error(f"[UserRAG] Error saving response to arch
ive: {ex}")

```

```

95 return False
96
97 def index_user_interaction(
98 user_identifier: str,
99 api_key: str,
100 query: str,
101 content_to_index: str
102) -> bool:
103 """Vectorize and save interaction to user FAISS index.
104
105 Args:
106 user_identifier (str): User identifier.
107 api_key (str): API key for vectorization.
108 query (str): User query text.
109 content_to_index (str): Content to be indexed.
110
111 Returns:
112 bool: Success flag indicating index operation.
113 """
114 if not user_identifier or not api_key or not content_to_in
dex.strip():
115 return False
116 try:
117 return index_user_query(user_identifier, api_key, quer
y, content_to_index)
118 except Exception as ex:
119 logger.error(f"[UserRAG] Error indexing interaction: {
ex}")
120 return False
121

```

\_\_init\_\_.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: RAG (Retrieval-Augmented Generation) subsystem
4 # =====
5 # Description:
6 # Centralized domain-independent semantic search subsystem for
7 # context retrieval and augmentation.
8 #
9 # File: __init__.py
10 # Project: ai-breadboard
11 # Package: src.rag
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16
17 from __future__ import annotations
18
19 from src.rag.models import RAGDecisionType, RAGRouteDecision,
20 RAGSearchResult
21 from src.rag.engine import RAGEngine, get_rag_engine
22 from src.rag.rules_rag import RulesRAG, build_rules_index
23 from src.rag.user_rag import (
24 search_user_history,
25 index_user_interaction,
26 save_user_approved_response,
27 get_user_preferences_context,
28)
29
30 __all__ = [
31 "RAGDecisionType",
32 "RAGRouteDecision",
33 "RAGSearchResult",
34 "RAGEngine",
```



```
32 | "get_rag_engine",
33 | "RulesRAG",
34 | "build_rules_index",
35 | "search_user_history",
36 | "index_user_interaction",
37 | "save_user_approved_response",
38 | "get_user_preferences_context",
39 |]
40 |
```

## api\_key\_state.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Ensure that the secrets directory exists on the file
4 # =====
5 # Description:
6 # Manages Google Gemini and AI provider API keys, rotation pools,
7 #
8 # File: api_key_state.py
9 # Project: ai-breadboard
10 # Package: src.secrets
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from __future__ import annotations
16
17 import json
18 import os
19 from datetime import datetime, timezone
20 from pathlib import Path
21 from typing import Any, Dict, List, Optional, Tuple
22
23 from header import __root__
24 from src.logger.logger import logger
25
26 _SECRETS_DIR: Path = __root__ / 'core' / 'secrets'
27 _KEYS_FILE: Path = _SECRETS_DIR / 'gemini_keys.json'
28 _LEGACY_SECRETS_FILE: Path = __root__ / 'core' / 'ai' / 'gemini' / 'secrets.json'
29 _ENV_FILE: Path = __root__ / '.env'
30 _DAY_SECONDS: float = 86400.0
```

```

31
32 def _ensure_secrets_dir() -> None:
33 """Ensure that the secrets directory exists on the filesystem. """
34 try:
35 _SECRETS_DIR.mkdir(parents=True, exist_ok=True)
36 except Exception as ex:
37 logger.error(f'Failed to create secrets directory: {ex}')
38
39 def _now_iso() -> str:
40 """Get current UTC timestamp in ISO 8601 string format.
41
42 Returns:
43 str: ISO formatted UTC datetime.
44 """
45 return datetime.now(timezone.utc).isoformat()
46
47 def _now_ts() -> float:
48 """Get current UTC timestamp as Unix epoch float.
49
50 Returns:
51 float: Unix timestamp in seconds.
52 """
53 return datetime.now(timezone.utc).timestamp()
54
55 def _iso_to_ts(iso_str: str) -> float:
56 """Convert ISO formatted string into Unix timestamp.
57
58 Args:
59 iso_str (str): Datetime string in ISO 8601 format.
60
61 Returns:
62 float: Unix epoch timestamp or 0.0 on parsing error.
63 """
64 if not iso_str:
65 return 0.0
66 try:

```

```

67 cleaned = iso_str.replace('Z', '+00:00')
68 return datetime.fromisoformat(cleaned).timestamp()
69 except Exception:
70 return 0.0
71
72 def _load_json_file(file_path: Path) -> Dict[str, Any]:
73 """Safely load JSON object from file path.
74
75 Args:
76 file_path (Path): Path to JSON file.
77
78 Returns:
79 Dict[str, Any]: Parsed JSON dictionary or empty dictio
80 nary.
81 """
82 if not file_path.exists():
83 return {}
84 try:
85 content = file_path.read_text(encoding='utf-8').strip(
86)
87 if not content:
88 return {}
89 data = json.loads(content)
90 if isinstance(data, dict):
91 return data
92 except Exception as ex:
93 logger.warning(f'Error reading JSON file {file_path}:
94 {ex}')
95 return {}
96
97 def _save_json_file(file_path: Path, data: Dict[str, Any]) ->
98 bool:
99 """Safely save dictionary as formatted JSON to file.
100
101 Args:
102 file_path (Path): Target file destination.
103 data (Dict[str, Any]): Dictionary to serialize.

```

```

101 Returns:
102 bool: True on success, False on error.
103 """
104 try:
105 _ensure_secrets_dir()
106 file_path.parent.mkdir(parents=True, exist_ok=True)
107 file_path.write_text(json.dumps(data, indent=2, ensure
108 _ascii=False), encoding='utf-8')
109 return True
110 except Exception as ex:
111 logger.error(f'Failed to save JSON to {file_path}: {ex
112 }')
113 return False
114
115 def _read_env_keys() -> Dict[str, str]:
116 """Read API keys defined in environment variables and .env
117 file.
118
119 Returns:
120 Dict[str, str]: Mapping of key names/identifiers to AP
121 I key strings.
122 """
123 keys_found: Dict[str, str] = {}
124
125 # Check process environment
126 env_gemini = os.getenv('GEMINI_API_KEY', '').strip()
127 if env_gemini:
128 keys_found['GEMINI_API_KEY'] = env_gemini
129
130 env_google = os.getenv('GOOGLE_API_KEY', '').strip()
131 if env_google and env_google != env_gemini:
132 keys_found['GOOGLE_API_KEY'] = env_google
133
134 env_agy = os.getenv('AGY_API_KEY', '').strip()
135 if env_agy:
136 keys_found['AGY_API_KEY'] = env_agy
137
138 # Multiple comma-separated keys in GEMINI_API_KEYS

```

```

135 | multi_keys = os.getenv('GEMINI_API_KEYS', '').strip()
136 | if multi_keys:
137 | for idx, k in enumerate(multi_keys.split(','), start=1
138 |):
139 | cleaned = k.strip()
140 | if cleaned:
141 | keys_found[f'gemini_env_{idx}'] = cleaned
142 |
143 | # Parse .env directly if file exists
144 | if _ENV_FILE.exists():
145 | try:
146 | lines = _ENV_FILE.read_text(encoding='utf-8').splitlines()
147 | for line in lines:
148 | trimmed = line.strip()
149 | if not trimmed or trimmed.startswith('#') or '=' not in trimmed:
150 | continue
151 | k, v = trimmed.split('=', 1)
152 | k_clean = k.strip()
153 | v_clean = v.strip().strip('"').strip("'")
154 | if k_clean in ('GEMINI_API_KEY', 'GOOGLE_API_KEY', 'AGY_API_KEY') and v_clean:
155 | if k_clean not in keys_found:
156 | keys_found[k_clean] = v_clean
157 | except Exception as ex:
158 | logger.warning(f'Could not read .env file for keys : {ex}')
159 |
160 | return keys_found
161 |
162 | def _get_merged_keys_data() -> Dict[str, Dict[str, Any]]:
163 | """Load and merge keys from all storage locations and environment.
164 |
165 | Returns:
166 | Dict[str, Dict[str, Any]]: Unified dictionary of keys with metadata.

```

```

166 | """
167 | merged: Dict[str, Dict[str, Any]] = {}
168 |
169 | # 1. Load from gemini_keys.json
170 | primary_data = _load_json_file(_KEYS_FILE)
171 | for name, entry in primary_data.items():
172 | if isinstance(entry, dict):
173 | api_key = str(entry.get('api_key', '')).strip()
174 | merged[name] = {
175 | 'api_key': api_key,
176 | 'status': str(entry.get('status', 'active')),
177 | 'last_run': entry.get('last_run', ''),
178 | 'exhausted_at': entry.get('exhausted_at', ''),
179 | }
180 | elif isinstance(entry, str) and entry.strip():
181 | merged[name] = {
182 | 'api_key': entry.strip(),
183 | 'status': 'active',
184 | 'last_run': '',
185 | 'exhausted_at': '',
186 | }
187 |
188 | # 2. Load from legacy secrets.json
189 | legacy_data = _load_json_file(_LEGACY_SECRETS_FILE)
190 | for name, entry in legacy_data.items():
191 | if name not in merged:
192 | if isinstance(entry, dict):
193 | api_key = str(entry.get('api_key', '')).strip(
194 |)
195 | merged[name] = {
196 | 'api_key': api_key,
197 | 'status': str(entry.get('status', 'active'
198 |))),
199 | 'last_run': entry.get('last_run', ''),
200 | 'exhausted_at': entry.get('exhausted_at',
201 | '''),
202 | }
203 | elif isinstance(entry, str) and entry.strip():

```

```

201 merged[name] = {
202 'api_key': entry.strip(),
203 'status': 'active',
204 'last_run': '',
205 'exhausted_at': '',
206 }
207
208 # 3. Load from environment variables
209 env_keys = _read_env_keys()
210 for name, key_val in env_keys.items():
211 if name not in merged and key_val:
212 merged[name] = {
213 'api_key': key_val,
214 'status': 'active',
215 'last_run': '',
216 'exhausted_at': '',
217 }
218
219 return merged
220
221 def load_api_keys(
222 names: Optional[List[str]] = [],
223 skip_exhausted: bool = True,
224) -> Tuple[List[str], List[str], List[Any]]:
225 """Load API keys filtered by active status and optional na
226 me criteria.
227
228 Args:
229 names (Optional[List[str]]): Specific key names to loa
230 d, or empty/None for all.
231 skip_exhausted (bool): Whether to skip keys in 24h exh
232 austion cooldown.
233
234 Returns:
235 Tuple[List[str], List[str], List[Any]]:
236 - list of valid API key strings
237 - list of corresponding key names
238 - list of key status/metadata objects

```



```

236 | """
237 | keys_data = _get_merged_keys_data()
238 | now = _now_ts()
239 |
240 | # Determine desired names if specified
241 | filter_names: List[str] = []
242 | if names:
243 | if isinstance(names, str):
244 | filter_names = [n.strip() for n in str(names).split(
245 | ',') if n.strip()]
246 | elif isinstance(names, (list, tuple, set)):
247 | filter_names = [str(n).strip() for n in names if s
248 | tr(n).strip()]
249 |
250 | # Also check GEMINI_API_KEY_NAMES if no names explicitly p
251 | rovided
252 | if not filter_names:
253 | env_names_str = os.getenv('GEMINI_API_KEY_NAMES', '').
254 | strip()
255 | if env_names_str and env_names_str != '*':
256 | filter_names = [n.strip() for n in env_names_str.s
257 | plit(',') if n.strip()]
258 |
259 | result_keys: List[str] = []
260 | result_names: List[str] = []
261 | result_states: List[Any] = []
262 | updated_needed: bool = False
263 |
264 | for name, data in keys_data.items():
265 | api_key = data.get('api_key', '').strip()
266 | if not api_key:
267 | continue
268 |
269 | # Filter by name if criteria active
270 | if filter_names and name not in filter_names and api_k
271 | ey not in filter_names:
272 | continue

```

```

268 | exhausted_at = data.get('exhausted_at', '')
269 | is_exhausted = False
270 |
271 | if exhausted_at:
272 | exhausted_ts = _iso_to_ts(exhausted_at)
273 | elapsed = now - exhausted_ts
274 | if 0 <= elapsed < _DAY_SECONDS:
275 | is_exhausted = True
276 | else:
277 | # 24h period expired, auto-reset exhaustion st
atus
278 | data['exhausted_at'] = ''
279 | data['status'] = 'active'
280 | updated_needed = True
281 |
282 | if skip_exhausted and is_exhausted:
283 | continue
284 |
285 | if data.get('status') == 'disabled':
286 | continue
287 |
288 | result_keys.append(api_key)
289 | result_names.append(name)
290 | result_states.append(data)
291 |
292 | if updated_needed:
293 | _save_json_file(_KEYS_FILE, keys_data)
294 |
295 | # Fallback to direct raw key in filter_names if nothing ma
tched in storage
296 | if not result_keys and filter_names:
297 | for item in filter_names:
298 | if item.startswith('AIza') or len(item) >= 20:
299 | result_keys.append(item)
300 | result_names.append('direct_key')
301 | result_states.append({'api_key': item, 'status
': 'active'})
302 |

```

```

303 return result_keys, result_names, result_states
304
305 def mark_exhausted(key_name: str) -> None:
306 """Mark an API key as quota exhausted and start 24h cooldown timer.
307
308 Args:
309 key_name (str): The identifier or raw value of the key
310 to mark.
311 """
312 if not key_name:
313 return
314
315 keys_data = _get_merged_keys_data()
316 target_name = key_name
317
318 # If key_name is not direct dict key, search by api_key value
319 if target_name not in keys_data:
320 for name, data in keys_data.items():
321 if data.get('api_key') == key_name:
322 target_name = name
323 break
324
325 if target_name not in keys_data:
326 keys_data[target_name] = {
327 'api_key': key_name if key_name.startswith('AIza')
328 else '',
329 'status': 'exhausted',
330 'last_run': '',
331 'exhausted_at': _now_iso(),
332 }
333 else:
334 keys_data[target_name]['exhausted_at'] = _now_iso()
335 keys_data[target_name]['status'] = 'exhausted'
336
337 _save_json_file(_KEYS_FILE, keys_data)
338 logger.warning(f'API key "{target_name}" marked as exhaust

```

```

 ed (24h cooldown initiated).')
337
338 def update_last_run(key_name: str) -> None:
339 """Update last executed timestamp for specified API key.
340
341 Args:
342 key_name (str): Identifier of the key that was execute
343 d.
344 """
345 if not key_name:
346 return
347
348 keys_data = _get_merged_keys_data()
349 target_name = key_name
350
351 if target_name not in keys_data:
352 for name, data in keys_data.items():
353 if data.get('api_key') == key_name:
354 target_name = name
355 break
356
357 if target_name in keys_data:
358 keys_data[target_name]['last_run'] = _now_iso()
359 _save_json_file(_KEYS_FILE, keys_data)
360
361 def next_available_in() -> float:
362 """Calculate remaining seconds until the earliest exhausted
363 key becomes available.
364
365 Returns:
366 float: Seconds remaining until reset, or 0.0 if any keys
367 are already available.
368 """
369 keys_data = _get_merged_keys_data()
370 if not keys_data:
371 return 0.0
372
373 now = _now_ts()

```

```

371 min_wait: float = _DAY_SECONDS
372 found_exhausted = False
373
374 for name, data in keys_data.items():
375 if data.get('status') == 'disabled':
376 continue
377 exhausted_at = data.get('exhausted_at', '')
378 if not exhausted_at:
379 return 0.0 # At least one key is active and ready
380 exhausted_ts = _iso_to_ts(exhausted_at)
381 elapsed = now - exhausted_ts
382 if elapsed >= _DAY_SECONDS:
383 return 0.0 # Expired cooldown, ready now
384 found_exhausted = True
385 remaining = _DAY_SECONDS - elapsed
386 if remaining < min_wait:
387 min_wait = remaining
388
389 return min_wait if found_exhausted else 0.0
390
391 def get_status(names: Optional[List[str]] = []) -> Dict[str, Any]:
392 """Retrieve detailed runtime status for all or selected keys.
393
394 Args:
395 names (Optional[List[str]]): List of key names to filter, or empty for all.
396
397 Returns:
398 Dict[str, Any]: Dictionary mapping key names to status descriptors.
399 """
400 keys_data = _get_merged_keys_data()
401 now = _now_ts()
402 statuses: Dict[str, Any] = {}
403
404 for name, data in keys_data.items():

```

```

405 if names and name not in names:
406 continue
407 exhausted_at = data.get('exhausted_at', '')
408 is_exhausted = False
409 remaining_sec = 0.0
410
411 if exhausted_at:
412 elapsed = now - _iso_to_ts(exhausted_at)
413 if 0 <= elapsed < _DAY_SECONDS:
414 is_exhausted = True
415 remaining_sec = _DAY_SECONDS - elapsed
416
417 statuses[name] = {
418 'status': data.get('status', 'active'),
419 'last_run': data.get('last_run', ''),
420 'exhausted_at': exhausted_at,
421 'is_exhausted': is_exhausted,
422 'reset_in_seconds': int(remaining_sec) if is_exhau
423 sted else 0,
424 }
425 return statuses
426
427 def save_api_key(name: str, api_key: str, status: str = 'activ
428 e') -> bool:
429 """Save or update an API key in persistent storage.
430
431 Args:
432 name (str): Unique name identifier for the key.
433 api_key (str): The raw secret API key string.
434 status (str): Initial key status ('active' or 'disable
435 d').
436
437 Returns:
438 bool: True if key was successfully saved.
439 """
440 if not name or not api_key:
441 return False

```

```
440 |
441 | keys_data = _get_merged_keys_data()
442 | keys_data[name] = {
443 | 'api_key': api_key.strip(),
444 | 'status': status,
445 | 'last_run': keys_data.get(name, {}).get('last_run', ''
 |),
446 | 'exhausted_at': keys_data.get(name, {}).get('exhausted
 | _at', ''),
447 | }
448 |
449 | success = _save_json_file(_KEYS_FILE, keys_data)
450 | if success:
451 | logger.info(f'API key "{name}" saved successfully to {
 | _KEYS_FILE}.')
452 | return success
453 |
```

\_\_init\_\_.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Secrets Management Module Initialization
4 # =====
5 # Description:
6 # Package for managing API keys, tokens, credentials, and th
7 # eir runtime states.
8 #
9 # File: __init__.py
10 # Project: ai-breadboard
11 # Package: src.secrets
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16
17 from __future__ import annotations
18
19 from src.secrets.api_key_state import (
20 get_status,
21 load_api_keys,
22 mark_exhausted,
23 next_available_in,
24 save_api_key,
25 update_last_run,
26)
27
28 __all__ = [
29 'get_status',
30 'load_api_keys',
31 'mark_exhausted',
32 'next_available_in',
33 'save_api_key',
34 'update_last_run',
```



33 | 1

34 |

## registry.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Skill definition and registry management for A
 I agents
4 # Description: Loads, normalizes, and registers skills for var
 ious AI agents.
5 # File: registry.py
6 # Project: ai-breadboard
7 # Package: src.skills
8 # Author: hypo69
9 # Copyright: © 2026 hypo69
10 # =====
11
12 """Loads, normalizes, and registers skills for various AI agen
 ts."""
13
14 from __future__ import annotations
15
16 import json
17 import re
18 from dataclasses import dataclass, field
19 from pathlib import Path
20 from typing import Any, Iterable
21
22 from header import __root__
23
24 _FRONTMATTER_PATTERN = re.compile(r"\A---\s*\n(?:P<body>.*?)\n-
 --\s*(?:\n|\Z)", re.DOTALL)
25 _DEFAULT_SKILL_DIRS = (".agents/skills", ".github/skills", "sk
 ills", ".gemini/skills")
26
27 def _parse_scalar(value: str) -> Any:
28 """Parses simple YAML values without requiring PyYAML depe
 ndency."""
```

```

29 | normalized = value.strip()
30 | if not normalized:
31 | return ""
32 | if normalized.startswith("[") or normalized.startswith("{"):
33 | try:
34 | return json.loads(normalized)
35 | except json.JSONDecodeError:
36 | return normalized
37 | if normalized.lower() in ("true", "false"):
38 | return normalized.lower() == "true"
39 | if (normalized.startswith('"') and normalized.endswith('"')
40 | normalized.startswith("'") and normalized.endswith("'")
41 |):
42 | return normalized[1:-1]
43 | return normalized
44 |
45 | def _parse_frontmatter(text: str) -> tuple[dict[str, Any], str]:
46 | """Returns frontmatter metadata and Markdown without its header."""
47 | match = _FRONTMATTER_PATTERN.match(text)
48 | if not match:
49 | return {}, text.strip()
50 |
51 | metadata: dict[str, Any] = {}
52 | for line in match.group("body").splitlines():
53 | if not line.strip() or line.lstrip().startswith("#") or
54 | ":" not in line:
55 | continue
56 | key, value = line.split(":", 1)
57 | metadata[key.strip()] = _parse_scalar(value)
58 | return metadata, text[match.end():].strip()
59 |
60 | def _load_json_manifest(skill_root: Path) -> dict[str, Any]:
 """Loads optional machine contract for the skill."""

```

```

61 | manifest_path = skill_root / "skill.json"
62 | if not manifest_path.is_file():
63 | return {}
64 | try:
65 | parsed = json.loads(manifest_path.read_text(encoding="
utf-8"))
66 | except (OSError, json.JSONDecodeError):
67 | return {}
68 | return parsed if isinstance(parsed, dict) else {}
69 |
70 | @dataclass(frozen=True)
71 | class SkillDefinition:
72 | """Normalized skill description suitable for any provider.
73 | """
74 | name: str
75 | description: str
76 | root: Path
77 | source: Path
78 | metadata: dict[str, Any] = field(default_factory=dict)
79 | instructions: str = ""
80 | manifest: dict[str, Any] = field(default_factory=dict)
81 |
82 | def to_dict(self, include_instructions: bool = True) -> di
ct[str, Any]:
83 | """Converts skill to portable JSON contract."""
84 | result: dict[str, Any] = {
85 | "name": self.name,
86 | "description": self.description,
87 | "root": self.root.as_posix(),
88 | "source": self.source.as_posix(),
89 | "metadata": dict(self.metadata),
90 | "manifest": dict(self.manifest),
91 | }
92 | if include_instructions:
93 | result["instructions"] = self.instructions
94 | return result
95 |

```

```

96 def prompt(self) -> str:
97 """Returns instructions for adding to the model's system prompt."""
98 return self.instructions.strip()
99
100 class SkillRegistry:
101 """Searches for skills in compatible directories and provides unified access API."""
102
103 def __init__(self, project_root: Path = __root__, skill_dirs: Iterable[str] = _DEFAULT_SKILL_DIRS) -> None:
104 self.project_root = Path(project_root).resolve()
105 self.skill_dirs = tuple(skill_dirs)
106
107 def discover(self) -> list[SkillDefinition]:
108 """Finds all directories with SKILL.md and removes duplicates by name."""
109 found: dict[str, SkillDefinition] = {}
110 for relative_dir in self.skill_dirs:
111 skills_root = self.project_root / relative_dir
112 if not skills_root.is_dir():
113 continue
114 for skill_file in sorted(skills_root.glob("*/SKILL.md")):
115 definition = self._load(skill_file)
116 if definition.name and definition.name not in found:
117 found[definition.name] = definition
118 return sorted(found.values(), key=lambda item: item.name)
119
120 def get(self, name: str) -> SkillDefinition:
121 """Returns skill by name or raises KeyError if not found."""
122 normalized = name.strip().lower()
123 for skill in self.discover():
124 if skill.name.lower() == normalized:
125 return skill

```

```

126 raise KeyError(f"Skill not found: {name}")
127
128 def search(self, query: str) -> list[SkillDefinition]:
129 """Searches for skills by name and description."""
130 terms = [term.lower() for term in query.split() if ter
m.strip()]
131 if not terms:
132 return self.discover()
133 return [
134 skill for skill in self.discover()
135 if all(term in f"{skill.name} {skill.description}"
.lower() for term in terms)
136]
137
138 def export_json(self, name: str, include_instructions: boo
l = True) -> str:
139 """Exports skill to JSON for external agent or model."
""
140 return json.dumps(self.get(name).to_dict(include_instr
uctions), ensure_ascii=False, indent=2)
141
142 @staticmethod
143 def _load(skill_file: Path) -> SkillDefinition:
144 raw = skill_file.read_text(encoding="utf-8")
145 metadata, instructions = _parse_frontmatter(raw)
146 manifest = _load_json_manifest(skill_file.parent)
147 name = str(manifest.get("name", metadata.get("name", s
kill_file.parent.name))).strip()
148 description = str(manifest.get("description", metadata
.get("description", ""))).strip()
149 merged_metadata = dict(metadata)
150 merged_metadata.update({key: value for key, value in m
anifest.items() if key not in ("name", "description")})
151 return SkillDefinition(
152 name=name,
153 description=description,
154 root=skill_file.parent,
155 source=skill_file,

```

```
156 | metadata=merged_metadata,
157 | instructions=instructions,
158 | manifest=manifest,
159 |)
```

\_\_init\_\_.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Universal skill registry for AI agents
4 # =====
5 # Description:
6 # Universal registry of project skills for AI agents and task execution.
7 #
8 # File: __init__.py
9 # Project: ai-breadboard
10 # Package: src.skills
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """Universal registry of project skills."""
16
17 from src.skills.registry import SkillDefinition, SkillRegistry
18
19 __all__ = ["SkillDefinition", "SkillRegistry"]
20
```



edge.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Synthesizes text to a file using Microsoft Edge TTS
4 # =====
5 # Description:
6 # Synthesizes text to a file using Microsoft Edge TTS.
7 #
8 # File: edge.py
9 # Project: ai-breadboard
10 # Package: src.tts
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """
16 Module for Microsoft Edge TTS system.
17 """
18 from __future__ import annotations
19
20 from pathlib import Path
21 import edge_tts
22
23 async def synthesize(text: str, file_path: Path, voice: str = "ru-RU-DmitryNeural"):
24 """Synthesizes text to a file using Microsoft Edge TTS."""
25 communicate = edge_tts.Communicate(text, voice)
26 await communicate.save(str(file_path))
27
```

gtts.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Synthesizes text to a file using Google TTS (g
 TTS)
4 # =====
5 # Description:
6 # Synthesizes text to a file using Google TTS (gTTS)."""
7 #
8 # File: gtts.py
9 # Project: ai-breadboard
10 # Package: src.tts
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """
16 Module for Google Translator TTS system (gTTS).
17 """
18 from __future__ import annotations
19
20 import asyncio
21 from pathlib import Path
22 from gtts import gTTS
23
24 async def synthesize(text: str, file_path: Path, voice: str =
 "ru"):
25 """Synthesizes text to a file using Google TTS (gTTS)."""
26 lang = "ru"
27 if "-" in voice:
28 lang = voice.split("-")[0]
29
30 loop = asyncio.get_event_loop()
31 tts = gTTS(text=text, lang=lang)
```

```
32 | await loop.run_in_executor(None, tts.save, str(file_path))
33 |
```

## silero.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Loads and returns the Silero model, caching it
 in
4 # =====
5 # Description:
6 # Loads and returns the Silero model, caching it in memory."
 ""
7 #
8 # File: silero.py
9 # Project: ai-breadboard
10 # Package: src.tts
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """
16 Module for Silero TTS system.
17 """
18 from __future__ import annotations
19
20 import os
21 import asyncio
22 from pathlib import Path
23 import torch
24 import soundfile as sf
25 from pydub import AudioSegment
26
27 from src.logger import logger
28
29 _model = ""
30
31 def get_silero_model():
```

```

32 | """Loads and returns the Silero model, caching it in memor
 | y."""
33 | global _model
34 | if _model != "":
35 | return _model
36 |
37 | import sys
38 | # Backup and remove local 'src' modules from sys.modules t
 | o avoid namespace conflict with Silero's 'src'
39 | saved_modules = {}
40 | for k in list(sys.modules.keys()):
41 | if k == "src" or k.startswith("src."):
42 | saved_modules[k] = sys.modules.pop(k)
43 |
44 | try:
45 | # Touch __init__.py in cached src directory to resolve
 | namespace package lookup in sys.path
46 | hub_src_dir = Path(torch.hub.get_dir()) / "snakers4_si
 | lero-models_master" / "src"
47 | hub_init = hub_src_dir / "__init__.py"
48 | if hub_src_dir.exists() and not hub_init.exists():
49 | try:
50 | hub_init.touch()
51 | except Exception:
52 | pass
53 |
54 | # Prepend the torch hub cache directory to sys.path
55 | hub_dir = Path(torch.hub.get_dir()) / "snakers4_silero
 | -models_master"
56 | if str(hub_dir) not in sys.path:
57 | sys.path.insert(0, str(hub_dir))
58 |
59 | device = torch.device("cpu")
60 | # Load the Russian v5 model from torch hub
61 | model, _ = torch.hub.load(
62 | repo_or_dir="snakers4/silero-models",
63 | model="silero_tts",
64 | language="ru",

```

```

65 speaker="v5_ru",
66 trust_repo=True
67)
68 model.to(device)
69 _model = model
70 except Exception as e:
71 # Raise error first, we'll log it outside if needed
72 raise e
73 finally:
74 # Restore local 'src' modules
75 for k, v in saved_modules.items():
76 sys.modules[k] = v
77
78 return _model
79
80 async def synthesize(text: str, file_path: Path, voice: str =
 "eugene"):
81 """Synthesizes text to a file using Silero TTS."""
82 valid_speakers = {"aidar", "baya", "eugene", "kseniya", "x
 enia", "random"}
83 speaker = voice if voice in valid_speakers else "eugene"
84
85 # We will write to a temp wav file first, then convert to
 mp3 if requested
86 temp_wav_path = file_path.with_suffix(".wav")
87
88 loop = asyncio.get_event_loop()
89
90 def _run_synthesis():
91 try:
92 model = get_silero_model()
93 audio_tensor = model.apply_tts(text=text, speaker=
 speaker, sample_rate=48000)
94 sf.write(str(temp_wav_path), audio_tensor.numpy(),
 48000)
95 except Exception as e:
96 logger.error(f"Error during Silero synthesis: {e}"
)

```

```
97 | raise e
98 |
99 | await loop.run_in_executor(None, _run_synthesis)
100 |
101 | # If the requested path is mp3, convert it
102 | if file_path.suffix.lower() == ".mp3":
103 | def _convert_to_mp3():
104 | try:
105 | audio = AudioSegment.from_wav(str(temp_wav_path
106 | h))
107 | audio.export(str(file_path), format="mp3")
108 | finally:
109 | if temp_wav_path.exists():
110 | temp_wav_path.unlink()
111 |
112 | await loop.run_in_executor(None, _convert_to_mp3)
113 | else:
114 | # Otherwise rename/move to destination
115 | if temp_wav_path != file_path:
116 | if file_path.exists():
117 | file_path.unlink()
118 | temp_wav_path.rename(file_path)
```

\_\_init\_\_.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Synthesizes speech to a file using the selected TT
4 # =====
5 # Description:
6 # Synthesizes speech to a file using the selected TTS system
7 # and voice.
8 #
9 # File: __init__.py
10 # Project: ai-breadboard
11 # Package: src.tts
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15 """
16 Unified interface for all TTS systems (Microsoft Edge, Google,
17 Silero).
18 """
19 from __future__ import annotations
20
21 from pathlib import Path
22 from src.logger import logger
23
24 async def synthesize_speech(text: str, file_path: Path, tts_system: str = "edge-tts", voice: str = "ru-RU-DmitryNeural"):
25 """Synthesizes speech to a file using the selected TTS system and voice."""
26 logger.info(f"Synthesizing using system: {tts_system}, voice/speaker: {voice}")
27
28 if tts_system == "gtts":
```



```
28 | from src.tts import gtts
29 | await gtts.synthesize(text, file_path, voice)
30 | elif tts_system == "silero":
31 | try:
32 | from src.tts import silero
33 | await silero.synthesize(text, file_path, voice)
34 | except ImportError as e:
35 | logger.warning(f"Could not load Silero TTS (missin
36 | g dependencies like torch/soundfile): {e}. Falling back to edg
37 | e-tts.")
38 | from src.tts import edge
39 | await edge.synthesize(text, file_path, voice)
40 | else:
41 | # Fallback to edge-tts
42 | from src.tts import edge
43 | await edge.synthesize(text, file_path, voice)
```

## user\_profile.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: User profile JSON file management and watch pr
 ogress tracking
4 # =====
5 # Description:
6 # Management of individual user JSON profile files stored in
 user_rags directory.
7 # Tracks watch history, search queries, preferences, and gen
 erates recommendation context.
8 #
9 # File: user_profile.py
10 # Project: ai-breadboard
11 # Package: src.user_manager
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 from __future__ import annotations
17
18 import json
19 import time
20 from pathlib import Path
21 from typing import Any, Dict, List, Optional
22
23 from src.logger import logger
24
25 # User profiles directory (same as user RAG storage location)
26 _USER_PROFILES_DIR = Path(__file__).parent.parent / 'ai' / 'ge
 mini' / 'user_rags'
27 _USER_PROFILES_DIR.mkdir(parents=True, exist_ok=True)
28
29 def _get_profile_path(user_id: str | int) -> Path:
```

```

30 """Return path to user profile JSON file."""
31 safe_id = str(user_id).replace('.', '_').replace('/', '_')
32 .replace('\\', '_')
33 return _USER_PROFILES_DIR / f'user_profile_{safe_id}.json'
34
35 def _default_profile_structure(user_id: str | int) -> Dict[str, Any]:
36 """Default user profile structure."""
37 return {
38 "user_id": str(user_id),
39 "created_at": time.time(),
40 "updated_at": time.time(),
41 "watch_history": {}, # { file_path: { "name": str, "last_position": float, "duration": float, "updated_at": float, "completed": bool } }
42 "last_watched": None, # { "path": str, "position": float }
43 "search_history": [], # [{ "query": str, "timestamp": float }]
44 "preferences": {
45 "liked_titles": [], # ["Interstellar", ...]
46 "disliked_titles": [], # ["Movie X", ...]
47 "favorite_genres": {}, # { "Science Fiction": count, "Action": count }
48 "favorite_categories": {} # { "Action": count }
49 }
50 }
51
52 def load_user_profile(user_id: str | int) -> Dict[str, Any]:
53 """Load user JSON profile or create default."""
54 path = _get_profile_path(user_id)
55 if path.exists():
56 try:
57 data = json.loads(path.read_text(encoding='utf-8'))
58 except Exception as ex:
59 logger.error(f"Error reading profile {user_id}", e

```

```

 x, False)
60
61 profile = _default_profile_structure(user_id)
62 save_user_profile(user_id, profile)
63 return profile
64
65 def save_user_profile(user_id: str | int, profile: Dict[str, A
ny]) -> bool:
66 """Save user profile to JSON."""
67 try:
68 path = _get_profile_path(user_id)
69 profile["updated_at"] = time.time()
70 path.write_text(json.dumps(profile, ensure_ascii=False
, indent=2), encoding='utf-8')
71 return True
72 except Exception as ex:
73 logger.error(f"Error saving profile {user_id}", ex, Fa
lse)
74 return False
75
76 def update_watch_progress(
77 user_id: str | int,
78 file_path: str,
79 file_name: str,
80 current_time: float,
81 duration: float
82) -> Dict[str, Any]:
83 """Update playback timestamp for file and mark as last wat
ched."""
84 profile = load_user_profile(user_id)
85 watch_history = profile.setdefault("watch_history", {})
86
87 completed = duration > 0 and (current_time / duration) >=
0.90
88
89 watch_history[file_path] = {
90 "name": file_name,
91 "last_position": round(current_time, 2),

```

```

92 "duration": round(duration, 2),
93 "updated_at": time.time(),
94 "completed": completed
95 }
96
97 profile["last_watched"] = {
98 "path": file_path,
99 "name": file_name,
100 "position": round(current_time, 2)
101 }
102
103 save_user_profile(user_id, profile)
104 return watch_history[file_path]
105
106 def get_watch_progress(user_id: str | int, file_path: str) ->
Optional[Dict[str, Any]]:
107 """Get saved watch progress for specific file."""
108 profile = load_user_profile(user_id)
109 return profile.get("watch_history", {}).get(file_path)
110
111 def log_user_search(user_id: str | int, query: str):
112 """Log user search query for preference collection."""
113 if not query or len(query.strip()) < 2:
114 return
115 profile = load_user_profile(user_id)
116 search_history = profile.setdefault("search_history", [])
117
118 search_history.append({
119 "query": query.strip(),
120 "timestamp": time.time()
121 })
122
123 # Limit to last 200 searches
124 if len(search_history) > 200:
125 profile["search_history"] = search_history[-200:]
126
127 save_user_profile(user_id, profile)
128

```

```

129 def set_user_preference(
130 user_id: str | int,
131 title: str,
132 sentiment: str, # 'like' | 'dislike'
133 genre: Optional[str] = "",
134 category: Optional[str] = ""
135):
136 """Record user preference (like/dislike/genres)."""
137 profile = load_user_profile(user_id)
138 prefs = profile.setdefault("preferences", {
139 "liked_titles": [],
140 "disliked_titles": [],
141 "favorite_genres": {},
142 "favorite_categories": {}
143 })
144
145 liked = set(prefs.get("liked_titles", []))
146 disliked = set(prefs.get("disliked_titles", []))
147
148 if sentiment == 'like':
149 liked.add(title)
150 disliked.discard(title)
151 elif sentiment == 'dislike':
152 disliked.add(title)
153 liked.discard(title)
154
155 prefs["liked_titles"] = list(liked)
156 prefs["disliked_titles"] = list(disliked)
157
158 if genre:
159 fav_genres = prefs.setdefault("favorite_genres", {})
160 fav_genres[genre] = fav_genres.get(genre, 0) + 1
161
162 if category:
163 fav_cats = prefs.setdefault("favorite_categories", {})
164 fav_cats[category] = fav_cats.get(category, 0) + 1
165
166 save_user_profile(user_id, profile)

```

```

167
168 def get_recommendation_context(user_id: str | int) -> str:
169 """Generate summary text prompt of user preferences for AI
 ."""
170 profile = load_user_profile(user_id)
171 prefs = profile.get("preferences", {})
172 history = profile.get("watch_history", {})
173
174 recent_watched = sorted(history.values(), key=lambda x: x.
 get('updated_at', 0), reverse=True)[:5]
175 recent_names = [w.get('name') for w in recent_watched if w
 .get('name')]
176
177 liked = prefs.get("liked_titles", [])
178 disliked = prefs.get("disliked_titles", [])
179
180 context_lines = []
181 if liked:
182 context_lines.append(f"User likes: {' '.join(liked[-1
 0:]))}")
183 if disliked:
184 context_lines.append(f"User dislikes: {' '.join(disli
 ked[-10:]))}")
185 if recent_names:
186 context_lines.append(f"Recently watched: {' '.join(re
 cent_names)}")
187
188 return "\n".join(context_lines) if context_lines else ""
189

```

\_\_init\_\_.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: User management and authorization system
4 # =====
5 # Description:
6 # User account management, role-based access control, permis
7 # sion handling,
8 # session management, and comprehensive audit logging for sy
9 # stem security.
10 #
11 # File: __init__.py
12 # Project: ai-breadboard
13 # Package: src.user_manager
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # =====
17
18 import sqlite3
19 from pathlib import Path
20 from typing import Dict, List
21
22 from src.logger import logger
23
24 # =====
25
26 # User management class
27 # =====
28
29 class UserManager:
30 """User management and authorization system.
31
32 Storage of user data, session management,
```



```

30 | access rights verification and activity logging.
31 |
32 | Attributes:
33 | db_path (Path): Path to database file.
34 | """
35 |
36 | def __init__(self, db_path: Path) -> None:
37 | """Initialization of user manager.
38 |
39 | Args:
40 | db_path (Path): Path to SQLite database file.
41 | """
42 | self.db_path: Path = db_path
43 | self.db_path.parent.mkdir(parents=True, exist_ok=True)
44 | self._init_db()
45 |
46 | def _init_db(self) -> None:
47 | """Initialization of user management table schema."""
48 | with sqlite3.connect(self.db_path) as conn:
49 | # Create users table for authorized user managemen
50 | t
51 | conn.execute("""
52 | CREATE TABLE IF NOT EXISTS users (
53 | id INTEGER PRIMARY KEY AUTOINCREMENT,
54 | email TEXT NOT NULL UNIQUE,
55 | name TEXT NOT NULL,
56 | picture TEXT,
57 | created_at TEXT DEFAULT (datetime('now')),
58 | last_login TEXT,
59 | is_admin INTEGER DEFAULT 0,
60 | is_active INTEGER DEFAULT 1,
61 | role TEXT DEFAULT 'user'
62 |)
63 | """)
64 |
65 | # Add new columns for Telegram
66 | try:
67 | conn.execute("ALTER TABLE users ADD COLUMN tel

```

```

 egram_id INTEGER")
67 except sqlite3.OperationalError:
68 pass
69 try:
70 conn.execute("ALTER TABLE users ADD COLUMN telegram_id INTEGER")
71 except sqlite3.OperationalError:
72 pass
73 try:
74 conn.execute("CREATE UNIQUE INDEX IF NOT EXISTS idx_users_telegram_id ON users(telegram_id) WHERE telegram_id IS NOT NULL")
75 except sqlite3.OperationalError:
76 pass
77
78 # Add new columns for email authorization
79 try:
80 conn.execute("ALTER TABLE users ADD COLUMN password_hash TEXT")
81 except sqlite3.OperationalError:
82 pass
83 try:
84 conn.execute("ALTER TABLE users ADD COLUMN is_email_verified INTEGER DEFAULT 0")
85 except sqlite3.OperationalError:
86 pass
87
88 # Create user settings table
89 conn.execute("""
90 CREATE TABLE IF NOT EXISTS user_settings (
91 user_id INTEGER PRIMARY KEY,
92 theme TEXT DEFAULT 'dark',
93 language TEXT DEFAULT 'ru',
94 tts_enabled INTEGER DEFAULT 1,
95 system_instruction TEXT,
96 tts_system TEXT DEFAULT 'edge-tts',
97 tts_voice TEXT DEFAULT 'ru-RU-DmitryNeural
',

```

```

98 FOREIGN KEY (user_id) REFERENCES users(id)
99 ON DELETE CASCADE
100)
101 """)
102 try:
103 conn.execute("ALTER TABLE user_settings ADD CO
104 LUMN model TEXT")
105 except sqlite3.OperationalError:
106 pass
107 try:
108 conn.execute("ALTER TABLE user_settings ADD CO
109 LUMN tts_system TEXT DEFAULT 'edge-tts'")
110 except sqlite3.OperationalError:
111 pass
112 try:
113 conn.execute("ALTER TABLE user_settings ADD CO
114 LUMN tts_voice TEXT DEFAULT 'ru-RU-DmitryNeural'")
115 except sqlite3.OperationalError:
116 pass
117 # Create temporary tokens table for Telegram linki
118 ng
119 conn.execute("""
120 CREATE TABLE IF NOT EXISTS telegram_link_token
121 s (
122 token TEXT PRIMARY KEY,
123 user_id INTEGER NOT NULL,
124 expires_at TEXT NOT NULL,
125 FOREIGN KEY (user_id) REFERENCES users(id)
126 ON DELETE CASCADE
127)
128 """)
129 # Create Google OAuth tokens table
130 conn.execute("""

```

```

129 | CREATE TABLE IF NOT EXISTS google_oauth_tokens
130 | (
131 | user_id INTEGER PRIMARY KEY,
132 | access_token TEXT NOT NULL,
133 | refresh_token TEXT,
134 | expires_at TEXT,
135 | scope TEXT,
136 | updated_at TEXT DEFAULT (datetime('now')),
137 | FOREIGN KEY (user_id) REFERENCES users(id)
138 | ON DELETE CASCADE
139 |)
140 | """)
141 |
142 | # Create email verification codes table
143 | conn.execute("""
144 | CREATE TABLE IF NOT EXISTS email_verification_
145 | tokens (
146 | email TEXT PRIMARY KEY,
147 | code TEXT NOT NULL,
148 | expires_at TEXT NOT NULL
149 |)
150 | """)
151 |
152 | # Create index for fast email lookup
153 | conn.execute("""
154 | CREATE INDEX IF NOT EXISTS idx_users_email ON
155 | users(email)
156 | """)
157 |
158 | # Create session tokens table for active session m
159 | anagement
160 | conn.execute("""
161 | CREATE TABLE IF NOT EXISTS session_tokens (
162 | id INTEGER PRIMARY KEY AUTOINCREMENT,
163 | user_id INTEGER NOT NULL,
164 | token_hash TEXT NOT NULL UNIQUE,
165 | created_at TEXT DEFAULT (datetime('now')),
166 | expires_at TEXT NOT NULL,

```

```

162 is_revoked INTEGER DEFAULT 0,
163 ip_address TEXT,
164 user_agent TEXT,
165 FOREIGN KEY (user_id) REFERENCES users(id)
166 ON DELETE CASCADE
167)
168 """)
169
170 # Create index for token hash lookup
171 conn.execute("""
172 CREATE INDEX IF NOT EXISTS idx_session_token_h
173 ash ON session_tokens(token_hash)
174 """)
175
176 # Create user activity log table for activity logg
177 ing
178 conn.execute("""
179 CREATE TABLE IF NOT EXISTS user_activity_log (
180 id INTEGER PRIMARY KEY AUTOINCREMENT,
181 user_id INTEGER NOT NULL,
182 action TEXT NOT NULL,
183 ip_address TEXT,
184 user_agent TEXT,
185 timestamp TEXT DEFAULT (datetime('now')),
186 details TEXT,
187 FOREIGN KEY (user_id) REFERENCES users(id)
188 ON DELETE CASCADE
189)
190 """)
191
192 # Create index for fast user and time lookup
193 conn.execute("""
194 CREATE INDEX IF NOT EXISTS idx_activity_user_t
195 ime ON user_activity_log(user_id, timestamp)
196 """)
197
198 # Create roles table for role and permission manag
199 ement

```

```

194 conn.execute("""
195 CREATE TABLE IF NOT EXISTS roles (
196 id INTEGER PRIMARY KEY AUTOINCREMENT,
197 name TEXT NOT NULL UNIQUE,
198 description TEXT,
199 permissions TEXT
200)
201 """)
202
203 # Create user-role association table
204 conn.execute("""
205 CREATE TABLE IF NOT EXISTS user_roles (
206 user_id INTEGER NOT NULL,
207 role_id INTEGER NOT NULL,
208 PRIMARY KEY (user_id, role_id),
209 FOREIGN KEY (user_id) REFERENCES users(id)
210 ON DELETE CASCADE,
211 FOREIGN KEY (role_id) REFERENCES roles(id)
212 ON DELETE CASCADE
213)
214 """)
215
216 # Create permission grants table
217 conn.execute("""
218 CREATE TABLE IF NOT EXISTS permission_grants (
219 id INTEGER PRIMARY KEY AUTOINCREMENT,
220 grantee_id INTEGER NOT NULL,
221 grant_type TEXT NOT NULL,
222 resource_type TEXT,
223 resource_id INTEGER,
224 permission TEXT NOT NULL,
225 granted_at TEXT DEFAULT (datetime('now')),
226 granted_by INTEGER,
227 FOREIGN KEY (grantee_id) REFERENCES users(
id) ON DELETE CASCADE,
228 FOREIGN KEY (granted_by) REFERENCES users(
id)
229)

```

```

228 """)
229
230 # Create index for fast grantee lookup
231 conn.execute("""
232 CREATE INDEX IF NOT EXISTS idx_permission_gran
233 ts_grantee ON permission_grants(grantee_id)
234 """)
235
236 # Create audit log table for important operations
237 audit
238 conn.execute("""
239 CREATE TABLE IF NOT EXISTS audit_log (
240 id INTEGER PRIMARY KEY AUTOINCREMENT,
241 user_id INTEGER,
242 action TEXT NOT NULL,
243 target_type TEXT,
244 target_id INTEGER,
245 old_values TEXT,
246 new_values TEXT,
247 ip_address TEXT,
248 timestamp TEXT DEFAULT (datetime('now')),
249 FOREIGN KEY (user_id) REFERENCES users(id)
250)
251 """)
252
253 # Create index for fast timestamp lookup
254 conn.execute("""
255 CREATE INDEX IF NOT EXISTS idx_audit_log_times
256 tamp ON audit_log(timestamp)
257 """)
258
259 # Create permissions table for storing available p
260 ermissions
261 conn.execute("""
262 CREATE TABLE IF NOT EXISTS permissions (
263 id INTEGER PRIMARY KEY AUTOINCREMENT,
264 name TEXT NOT NULL UNIQUE,
265 description TEXT,

```

```

262 category TEXT
263)
264 """)
265
266 # Insert default roles
267 initial_roles = [
268 ('admin', 'System Administrator', '{"all": true}'),
269 ('user', 'Regular User', '{"read": true, "chat": true, "media": true}'),
270 ('guest', 'Guest', '{"read": true, "chat": false, "media": false}'),
271]
272 for role_name, role_desc, role_perms in initial_roles:
273 conn.execute(
274 'INSERT OR IGNORE INTO roles (name, description, permissions) VALUES (?, ?, ?)',
275 (role_name, role_desc, role_perms)
276)
277
278 # Insert default permissions
279 initial_permissions = [
280 ('read', 'Data read access', 'basic'),
281 ('write', 'Data write access', 'basic'),
282 ('delete', 'Data deletion access', 'admin'),
283 ('admin', 'System administration', 'admin'),
284 ('chat', 'Chat access', 'chat'),
285 ('media', 'Media access', 'media'),
286 ('qbt', 'qBittorrent access', 'tools'),
287 ('media_organizer', 'Media organizer access', 'tools')
288]
289 for perm_name, perm_desc, perm_cat in initial_permissions:
290 conn.execute(
291 'INSERT OR IGNORE INTO permissions (name, description, category) VALUES (?, ?, ?)',

```



```

292 (perm_name, perm_desc, perm_cat)
293)
294
295 # Insert default administrator (ID: 1) for local b
ypass
296 conn.execute("""
297 INSERT OR IGNORE INTO users (id, email, name,
is_admin, role)
298 VALUES (1, 'admin@localhost', 'Admin', 1, 'adm
in')
299 """)
300
301 def _get_connection(self) -> sqlite3.Connection:
302 """Obtaining database connection.
303
304 Returns:
305 sqlite3.Connection: Connection to SQLite.
306 """
307 return sqlite3.connect(self.db_path)
308
309 def add_user(self, email: str, name: str, picture: str = '
', role: str = 'user') -> int:
310 """Adding a new user.
311
312 Args:
313 email (str): User email (unique).
314 name (str): User name.
315 picture (str): User avatar URL.
316 role (str): User role ('admin', 'user', 'guest').
317
318 Returns:
319 int: Added user ID or 0 on error.
320 """
321 with self._get_connection() as conn:
322 try:
323 cursor = conn.execute(
324 '''
325 INSERT INTO users (email, name, picture, r

```

```

ole)
326 VALUES (?, ?, ?, ?)
327 '',
328 (email, name, picture, role)
329)
330 conn.commit()
331 logger.info(f'New user added: {email} (ID: {cu
rsor.lastrowid})')
332 return cursor.lastrowid
333 except sqlite3.IntegrityError:
334 logger.error(f'User with email {email} already
exists')
335 return 0
336
337 def update_user(self, user_id: int, **kwargs) -> bool:
338 """Update user data.
339
340 Args:
341 user_id (int): User ID.
342 **kwargs: Fields to update (name, picture, role, i
s_active, is_admin, email, is_email_verified, password_hash, t
elegram_id, telegram_username).
343
344 Returns:
345 bool: True on success, False on error.
346 """
347 allowed_fields = {
348 'name', 'picture', 'role', 'is_active', 'is_admin'
349 ,
350 'last_login', 'email', 'is_email_verified', 'passw
ord_hash',
351 'telegram_id', 'telegram_username'
352 }
353 updates = {k: v for k, v in kwargs.items() if k in all
owed_fields}
354 if not updates:
355 return False

```

```

356
357 set_clause = ', '.join(f'{k} = ?' for k in updates.key
s())
358 values = list(updates.values()) + [user_id]
359
360 with self._get_connection() as conn:
361 try:
362 cursor = conn.execute(
363 f'UPDATE users SET {set_clause} WHERE id =
?',
364 values
365)
366 conn.commit()
367 return cursor.rowcount > 0
368 except Exception as e:
369 logger.error(f'Error updating user {user_id}: '
, e, False)
370 return False
371
372 def create_user_admin(
373 self,
374 email: str,
375 name: str,
376 password: str = '',
377 role: str = 'user',
378 is_admin: int = 0,
379 is_active: int = 1,
380 is_email_verified: int = 1
381) -> int:
382 """Creating user by administrator.
383
384 Args:
385 email (str): User email.
386 name (str): User name.
387 password (str): Password (optional).
388 role (str): Role ('admin', 'user', 'guest').
389 is_admin (int): Administrator flag (0 or 1).
390 is_active (int): Activity flag (0 or 1).

```

```

391 is_email_verified (int): Email verification flag (
 | 0 or 1).
392
393 Returns:
394 int: Created user ID or 0 on error.
395 """
396 email_clean = email.lower().strip()
397 pw_hash = self.hash_password(password) if password else ''
398 with self._get_connection() as conn:
399 try:
400 cursor = conn.execute(
401 '''
402 INSERT INTO users (email, name, password_h
 | ash, role, is_admin, is_active, is_email_verified)
403 VALUES (?, ?, ?, ?, ?, ?, ?)
404 ''',
405 (email_clean, name, pw_hash, role, is_admin, is_active, is_email_verified)
406)
407 conn.commit()
408 user_id = cursor.lastrowid or 0
409 logger.info(f'New user created: {email_clean}
 | (ID: {user_id})')
410 return user_id
411 except sqlite3.IntegrityError:
412 logger.error(f'User with email {email_clean} already exists')
413 return 0
414 except Exception as e:
415 logger.error(f'Error creating user {email_clean}:', e, False)
416 return 0
417
418 def set_user_password(self, user_id: int, password: str) -
 | > bool:
419 """Setting new password for user.
420

```

```

421 | Args:
422 | user_id (int): User ID.
423 | password (str): New password in plain text.
424 |
425 | Returns:
426 | bool: True on success, False on error.
427 | """
428 | if not password:
429 | return False
430 | pw_hash = self.hash_password(password)
431 | return self.update_user(user_id, password_hash=pw_hash
432 |)
433 |
434 | def get_user_by_id(self, user_id: int) -> Dict:
435 | """Getting user by ID.
436 |
437 | Args:
438 | user_id (int): User ID.
439 |
440 | Returns:
441 | Dict: User data or empty dictionary.
442 | """
443 | with self._get_connection() as conn:
444 | conn.row_factory = sqlite3.Row
445 | row = conn.execute(
446 | 'SELECT * FROM users WHERE id = ? LIMIT 1',
447 | (user_id,)
448 |).fetchone()
449 | return dict(row) if row else {}
450 |
451 | def get_user_by_email(self, email: str) -> Dict:
452 | """Getting user by email.
453 |
454 | Args:
455 | email (str): User email.
456 |
457 | Returns:
458 | Dict: User data or empty dictionary.

```

```

458 """
459 with self._get_connection() as conn:
460 conn.row_factory = sqlite3.Row
461 row = conn.execute(
462 'SELECT * FROM users WHERE email = ? LIMIT 1',
463 (email,)
464).fetchone()
465 return dict(row) if row else {}
466
467 def get_all_users(self, active_only: bool = True) -> List[
468 Dict]:
469 """Getting all users.
470
471 Args:
472 active_only (bool): Filter only active users.
473
474 Returns:
475 List[Dict]: List of users.
476 """
477 with self._get_connection() as conn:
478 conn.row_factory = sqlite3.Row
479 if active_only:
480 rows = conn.execute(
481 'SELECT * FROM users WHERE is_active = 1 O
482 RDER BY created_at DESC'
483).fetchall()
484 else:
485 rows = conn.execute(
486 'SELECT * FROM users ORDER BY created_at D
487 ESC'
488).fetchall()
489 return [dict(r) for r in rows]
490
491 def delete_user(self, user_id: int) -> bool:
492 """Deleting user.

```

```

493
494 Returns:
495 bool: True on success, False on error.
496 """
497 with self._get_connection() as conn:
498 try:
499 cursor = conn.execute(
500 'DELETE FROM users WHERE id = ?',
501 (user_id,)
502)
503 conn.commit()
504 return cursor.rowcount > 0
505 except Exception as e:
506 logger.error(f'Error deleting user {user_id}: '
507 , e, False)
508 return False
509
510 def user_exists(self, email: str) -> bool:
511 """Checking user existence.
512
513 Args:
514 email (str): User email.
515
516 Returns:
517 bool: True if user exists.
518 """
519 with self._get_connection() as conn:
520 row = conn.execute(
521 'SELECT 1 FROM users WHERE email = ?',
522 (email,)
523).fetchone()
524 return row is not None
525
526 def is_user_active(self, user_id: int) -> bool:
527 """Checking user activity.
528
529 Args:
530 user_id (int): User ID.

```

```
530
531 Returns:
532 bool: True if user is active.
533 """
534 user = self.get_user_by_id(user_id)
535 return bool(user.get('is_active', 0))
536
537 def is_admin(self, user_id: int) -> bool:
538 """Checking administrator rights.
539
540 Args:
541 user_id (int): User ID.
542
543 Returns:
544 bool: True if user is administrator.
545 """
546 user = self.get_user_by_id(user_id)
547 return bool(user.get('is_admin', 0))
548
549 def get_user_role(self, user_id: int) -> str:
550 """Getting user role.
551
552 Args:
553 user_id (int): User ID.
554
555 Returns:
556 str: Role name or 'user' by default.
557 """
558 user = self.get_user_by_id(user_id)
559 return user.get('role', 'user')
560
561 def set_user_role(self, user_id: int, role: str) -> bool:
562 """Setting user role.
563
564 Args:
565 user_id (int): User ID.
566 role (str): Role name ('admin', 'user', 'guest').
567
```



```

568 | Returns:
569 | bool: True on success.
570 | """
571 | return self.update_user(user_id, role=role)
572 |
573 | def revoke_session(self, token_hash: str) -> bool:
574 | """Revoking session by token hash.
575 |
576 | Args:
577 | token_hash (str): Token hash to revoke.
578 |
579 | Returns:
580 | bool: True on success.
581 | """
582 | with self._get_connection() as conn:
583 | try:
584 | cursor = conn.execute(
585 | 'UPDATE session_tokens SET is_revoked = 1
WHERE token_hash = ?',
586 | (token_hash,)
587 |)
588 | conn.commit()
589 | return cursor.rowcount > 0
590 | except Exception as e:
591 | logger.error('Error revoking session:', e, Fal
se)
592 | return False
593 |
594 | def create_session_token(self, user_id: int, token_hash: s
tr, expires_at: str, ip_address: str = '', user_agent: str = '
') -> bool:
595 | """Creating new session.
596 |
597 | Args:
598 | user_id (int): User ID.
599 | token_hash (str): Token hash.
600 | expires_at (str): Expiration date.
601 | ip_address (str): User IP address.

```

```

602 user_agent (str): Browser User-Agent.
603
604 Returns:
605 bool: True on success.
606 """
607 with self._get_connection() as conn:
608 try:
609 conn.execute(
610 '''
611 INSERT INTO session_tokens (user_id, token
612 _hash, expires_at, ip_address, user_agent)
613 VALUES (?, ?, ?, ?, ?)
614 ''',
615 (user_id, token_hash, expires_at, ip_address, user_agent)
616)
617 conn.commit()
618 return True
619 except Exception as e:
620 logger.error('Error creating session:', e, False)
621
622 return False
623
624 def is_session_valid(self, token_hash: str) -> bool:
625 """Checking session validity.
626
627 Args:
628 token_hash (str): Token hash.
629
630 Returns:
631 bool: True if session is valid.
632 """
633 with self._get_connection() as conn:
634 from datetime import datetime
635 now = datetime.utcnow().isoformat()
636 row = conn.execute(
637 '''
638 SELECT 1 FROM session_tokens

```

```

637 WHERE token_hash = ?
638 AND is_revoked = 0
639 AND expires_at > ?
640 ''' ,
641 (token_hash, now)
642).fetchone()
643 return row is not None
644
645 def log_user_activity(self, user_id: int, action: str, ip_
address: str = '', user_agent: str = '', details: str = '') ->
bool:
646 """Logging user activity.
647
648 Args:
649 user_id (int): User ID.
650 action (str): Action (login, logout, api_call, etc
).
651 ip_address (str): IP address.
652 user_agent (str): User-Agent.
653 details (str): Additional details.
654
655 Returns:
656 bool: True on success.
657 """
658 with self._get_connection() as conn:
659 try:
660 conn.execute(
661 '''
662 INSERT INTO user_activity_log (user_id, ac
tion, ip_address, user_agent, details)
663 VALUES (?, ?, ?, ?, ?)
664 ''' ,
665 (user_id, action, ip_address, user_agent,
details)
666)
667 conn.commit()
668 return True
669 except Exception as e:

```

```

670 logger.error('Error logging activity:', e, False)
671 return False
672
673 def log_audit(self, user_id: int, action: str, target_type
: str = '', target_id: int = 0, old_values: str = '', new_valu
es: str = '', ip_address: str = '') -> bool:
674 """Logging audit of important operations.
675
676 Args:
677 user_id (int): User ID.
678 action (str): Action.
679 target_type (str): Target object type.
680 target_id (int): Target object ID.
681 old_values (str): Old values (JSON).
682 new_values (str): New values (JSON).
683 ip_address (str): IP address.
684
685 Returns:
686 bool: True on success.
687 """
688 with self._get_connection() as conn:
689 try:
690 conn.execute(
691 '''
692 INSERT INTO audit_log (user_id, action, ta
rget_type, target_id, old_values, new_values, ip_address)
693 VALUES (?, ?, ?, ?, ?, ?, ?)
694 ''',
695 (user_id, action, target_type, target_id,
old_values, new_values, ip_address)
696)
697 conn.commit()
698 return True
699 except Exception as e:
700 logger.error('Error auditing:', e, False)
701 return False
702

```

```

703 def has_permission(self, user_id: int, permission: str) ->
bool:
704 """Checking permission presence for user.
705
706 Args:
707 user_id (int): User ID.
708 permission (str): Permission name.
709
710 Returns:
711 bool: True if user has permission.
712 """
713 # Administrators have all permissions
714 if self.is_admin(user_id):
715 return True
716
717 with self._get_connection() as conn:
718 # Check through permission_grants
719 row = conn.execute(
720 '''
721 SELECT 1 FROM permission_grants
722 WHERE grantee_id = ?
723 AND grant_type = 'user'
724 AND permission = ?
725 ''',
726 (user_id, permission)
727).fetchone()
728 if row:
729 return True
730
731 # Check through roles
732 row = conn.execute(
733 '''
734 SELECT r.permissions FROM roles r
735 JOIN user_roles ur ON r.id = ur.role_id
736 WHERE ur.user_id = ?
737 AND r.permissions IS NOT NULL
738 ''',
739 (user_id,)

```

```

740).fetchone()
741 if row:
742 try:
743 import json
744 perms = json.loads(row[0])
745 return perms.get(permission, False)
746 except json.JSONDecodeError:
747 pass
748
749 return False
750
751 def get_user_permissions(self, user_id: int) -> List[str]:
752 """Getting list of user permissions.
753
754 Args:
755 user_id (int): User ID.
756
757 Returns:
758 List[str]: List of permissions.
759 """
760 permissions = []
761
762 # Administrators have all permissions
763 if self.is_admin(user_id):
764 with self._get_connection() as conn:
765 rows = conn.execute('SELECT name FROM permissions').fetchall()
766 return [r['name'] for r in rows]
767
768 with self._get_connection() as conn:
769 # Getting permissions through permission_grants
770 rows = conn.execute(
771 'SELECT permission FROM permission_grants WHERE grantee_id = ? AND grant_type = ?',
772 (user_id, 'user')
773).fetchall()
774 permissions.extend([r['permission'] for r in rows])
775)

```

```

775 |
776 | # Getting permissions through roles
777 | rows = conn.execute(
778 | '''
779 | SELECT r.permissions FROM roles r
780 | JOIN user_roles ur ON r.id = ur.role_id
781 | WHERE ur.user_id = ?
782 | ''',
783 | (user_id,)
784 |).fetchall()
785 | for row in rows:
786 | try:
787 | import json
788 | perms = json.loads(row[0])
789 | permissions.extend(perms.keys())
790 | except json.JSONDecodeError:
791 | pass
792 |
793 | return list(set(permissions))
794 |
795 | def get_user_by_telegram_id(self, telegram_id: int) -> Dict:
796 | """Getting user by telegram_id."""
797 | with self._get_connection() as conn:
798 | conn.row_factory = sqlite3.Row
799 | row = conn.execute(
800 | 'SELECT * FROM users WHERE telegram_id = ? LIMIT 1',
801 | (telegram_id,)
802 |).fetchone()
803 | return dict(row) if row else {}
804 |
805 | def get_user_settings(self, user_id: int) -> Dict:
806 | """Getting user settings."""
807 | with self._get_connection() as conn:
808 | conn.row_factory = sqlite3.Row
809 | row = conn.execute(
810 | 'SELECT * FROM user_settings WHERE user_id = ?

```

```

LIMIT 1',
811 (user_id,)
812).fetchone()
813 if not row:
814 try:
815 conn.execute(
816 'INSERT OR IGNORE INTO user_settings (
817 user_id) VALUES (?)',
818 (user_id,)
819)
820 conn.commit()
821 except Exception as e:
822 logger.error(f'Error inserting default set
823 tings for {user_id}:', e, False)
824 row = conn.execute(
825 'SELECT * FROM user_settings WHERE user_id
826 = ? LIMIT 1',
827 (user_id,)
828).fetchone()
829 return dict(row) if row else {'user_id': user_id,
830 'theme': 'dark', 'language': 'ru', 'tts_enabled': 1, 'system_i
831 nstruction': None, 'model': None, 'tts_system': 'edge-tts', 't
832 ts_voice': 'ru-RU-DmitryNeural'}
833
834 def update_user_settings(self, user_id: int, **kwargs) ->
835 bool:
836 """Update user settings."""
837 allowed_fields = {'theme', 'language', 'tts_enabled',
838 'system_instruction', 'model', 'tts_system', 'tts_voice'}
839 updates = {k: v for k, v in kwargs.items() if k in all
840 owed_fields and v is not None}
841 if not updates:
842 return False
843 set_clause = ', '.join(f'{k} = ?' for k in updates.key
844 s())
845 values = list(updates.values()) + [user_id]
846 with self._get_connection() as conn:
847 try:

```



```

838 conn.execute(
839 f'UPDATE user_settings SET {set_clause} WH
ERE user_id = ?',
840 values
841)
842 conn.commit()
843 return True
844 except Exception as e:
845 logger.error(f'Error updating settings {user_i
d}:', e, False)
846 return False
847
848 def generate_link_token(self, user_id: int) -> str:
849 """Generating temporary token for Telegram linking."""
850 import secrets
851 from datetime import datetime, timedelta
852 token = secrets.token_hex(4).upper() # 8-character co
de, e.g. AB12CD34
853 expires_at = (datetime.utcnow() + timedelta(minutes=10
)).isoformat()
854 with self._get_connection() as conn:
855 conn.execute(
856 'INSERT OR REPLACE INTO telegram_link_tokens (
token, user_id, expires_at) VALUES (?, ?, ?)',
857 (token, user_id, expires_at)
858)
859 conn.commit()
860 return token
861
862 def link_telegram_account(self, token: str, telegram_id: i
nt, telegram_username: str) -> bool:
863 """Linking Telegram account by token."""
864 from datetime import datetime
865 now = datetime.utcnow().isoformat()
866 token = token.strip().upper()
867 with self._get_connection() as conn:
868 row = conn.execute(
869 'SELECT user_id FROM telegram_link_tokens WHER

```

```

E token = ? AND expires_at > ? LIMIT 1',
870 (token, now)
871).fetchone()
872 if not row:
873 return False
874 user_id = row[0]
875 try:
876 # Delete temporary telegram user if it was auto-created
877 conn.execute(
878 'DELETE FROM users WHERE telegram_id = ? AND email = ?',
879 (telegram_id, f"tg_{telegram_id}@telegram.bot"))
880)
881 # Clear telegram_id from any other records
882 conn.execute(
883 'UPDATE users SET telegram_id = NULL, telegram_username = NULL WHERE telegram_id = ?',
884 (telegram_id,))
885)
886 # Bind telegram_id to target user
887 conn.execute(
888 'UPDATE users SET telegram_id = ?, telegram_username = ? WHERE id = ?',
889 (telegram_id, telegram_username, user_id))
890)
891 conn.execute('DELETE FROM telegram_link_tokens WHERE token = ?', (token,))
892 conn.commit()
893 return True
894 except Exception as e:
895 logger.error(f'Error linking account {user_id} :', e, False)
896 return False
897
898 @staticmethod
899 def hash_password(password: str) -> str:

```

```

900 | """Password hashing using PBKDF2."""
901 | import hashlib
902 | import os
903 | salt = os.urandom(16)
904 | pw_hash = hashlib.pbkdf2_hmac('sha256', password.encode(
e('utf-8'), salt, 100000)
905 | return salt.hex() + '$' + pw_hash.hex()
906 |
907 | @staticmethod
908 | def verify_password(password: str, hashed: str) -> bool:
909 | """Checking password against hash."""
910 | if not hashed or '$' not in hashed:
911 | return False
912 | import hashlib
913 | try:
914 | salt_hex, hash_hex = hashed.split('$', 1)
915 | salt = bytes.fromhex(salt_hex)
916 | pw_hash = hashlib.pbkdf2_hmac('sha256', password.e
ncode('utf-8'), salt, 100000)
917 | return pw_hash.hex() == hash_hex
918 | except Exception:
919 | return False
920 |
921 | def register_email_user(self, email: str, password: str, n
ame: str) -> int:
922 | """Registering new user with email/password.
923 | If user already exists (for example, created via Googl
e),
924 | we update password and name, but don't change email_ve
rified
925 | if it is not confirmed.
926 | """
927 | email = email.lower().strip()
928 | pw_hash = self.hash_password(password)
929 | db_user = self.get_user_by_email(email)
930 |
931 | with self._get_connection() as conn:
932 | if db_user:

```

```

933 # Update existing account
934 try:
935 conn.execute(
936 'UPDATE users SET name = ?, password_h
ash = ? WHERE id = ?',
937 (name, pw_hash, db_user['id'])
938)
939 conn.commit()
940 return db_user['id']
941 except Exception as e:
942 logger.error(f'Error updating during regis
tration {email}:', e, False)
943 return 0
944 else:
945 # Create new account (unverified)
946 try:
947 cursor = conn.execute(
948 '''
949 INSERT INTO users (email, name, passwo
rd_hash, is_email_verified)
950 VALUES (?, ?, ?, 0)
951 ''',
952 (email, name, pw_hash)
953)
954 conn.commit()
955 return cursor.lastrowid
956 except sqlite3.IntegrityError:
957 return 0
958
959 def create_email_verification(self, email: str) -> str:
960 """Creating 6-digit email verification code."""
961 import random
962 from datetime import datetime, timedelta
963 email = email.lower().strip()
964 code = f"{random.randint(100000, 999999)}"
965 expires_at = (datetime.utcnow() + timedelta(minutes=15
)).isoformat()
966

```

```

967 with self._get_connection() as conn:
968 conn.execute(
969 'INSERT OR REPLACE INTO email_verification_tokens (email, code, expires_at) VALUES (?, ?, ?)',
970 (email, code, expires_at)
971)
972 conn.commit()
973 return code
974
975 def verify_email_code(self, email: str, code: str) -> bool
976 :
977 """Checking email verification code."""
978 from datetime import datetime
979 email = email.lower().strip()
980 code = code.strip()
981 now = datetime.utcnow().isoformat()
982
983 with self._get_connection() as conn:
984 row = conn.execute(
985 'SELECT 1 FROM email_verification_tokens WHERE
986 email = ? AND code = ? AND expires_at > ?',
987 (email, code, now)
988).fetchone()
989 if not row:
990 return False
991
992 # Verify user email
993 conn.execute(
994 'UPDATE users SET is_email_verified = 1 WHERE
995 email = ?',
996 (email,)
997)
998 # Delete used token
999 conn.execute(
1000 'DELETE FROM email_verification_tokens WHERE e
1001 mail = ?',
1002 (email,)
1003)

```

```

1000 conn.commit()
1001 return True
1002
1003 def save_google_tokens(self, user_id: int, access_token: s
1004 tr, refresh_token: str = '', expires_in: int = 3600, scope: st
1005 r = '') -> bool:
1006 """Saving or updating Google OAuth tokens.
1007
1008 Args:
1009 user_id: User identifier.
1010 access_token: Google access token.
1011 refresh_token: Google refresh token.
1012 expires_in: Token lifetime in seconds.
1013 scope: List of scopes.
1014
1015 Returns:
1016 bool: True on successful save.
1017 """
1018 from datetime import datetime, timedelta
1019 expires_at = (datetime.utcnow() + timedelta(seconds=ex
1020 pires_in)).isoformat()
1021 now = datetime.utcnow().isoformat()
1022 with self._get_connection() as conn:
1023 try:
1024 if not refresh_token:
1025 existing = conn.execute(
1026 'SELECT refresh_token FROM google_oauth
1027 h_tokens WHERE user_id = ?',
1028 (user_id,)
1029).fetchone()
1030 if existing and existing[0]:
1031 refresh_token = existing[0]
1032
1033 conn.execute(
1034 '''
1035 INSERT INTO google_oauth_tokens (user_id,
1036 access_token, refresh_token, expires_at, scope, updated_at)
1037 VALUES (?, ?, ?, ?, ?, ?)

```

```

1033 ON CONFLICT(user_id) DO UPDATE SET
1034 access_token = excluded.access_token,
1035 refresh_token = CASE WHEN excluded.ref
resh_token != '' THEN excluded.refresh_token ELSE google_oauth
_tokens.refresh_token END,
1036 expires_at = excluded.expires_at,
1037 scope = excluded.scope,
1038 updated_at = excluded.updated_at
1039 '',
1040 (user_id, access_token, refresh_token, exp
ires_at, scope, now)
1041)
1042 conn.commit()
1043 logger.info(f'Google OAuth tokens saved for us
er ID={user_id}')
1044 return True
1045 except Exception as e:
1046 logger.error(f'Error saving Google OAuth token
s for user_id={user_id}:', e, False)
1047 return False
1048
1049 def get_google_tokens(self, user_id: int) -> dict:
1050 """Getting user Google OAuth tokens.
1051
1052 Args:
1053 user_id: User identifier.
1054
1055 Returns:
1056 dict: Dictionary with token data or empty dictiona
ry.
1057 """
1058 with self._get_connection() as conn:
1059 conn.row_factory = sqlite3.Row
1060 row = conn.execute(
1061 'SELECT * FROM google_oauth_tokens WHERE user_
id = ? LIMIT 1',
1062 (user_id,)
1063).fetchone()

```

```
1064 | return dict(row) if row else {}
1065 |
1066 | def has_google_auth(self, user_id: int) -> bool:
1067 | """Checking presence of Google OAuth tokens for user.
1068 |
1069 | Args:
1070 | user_id: User identifier.
1071 |
1072 | Returns:
1073 | bool: True if user has Google token.
1074 | """
1075 | tokens = self.get_google_tokens(user_id)
1076 | return bool(tokens and tokens.get('access_token'))
1077 |
1078 | from header import __root__
1079 | db_path = __root__ / 'core' / 'user_manager' / 'users.db'
1080 | user_manager = UserManager(db_path)
1081 |
```



## base64.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Convert Base64 encoded content to temporary file
4 # =====
5 # Description:
6 # This module provides a function to decode Base64 encoded content and write it to a temporary file with the specified extension.
7 #
8 # File: base64.py
9 # Project: ai-breadboard
10 # Package: src.utils.convertors
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """Base64 encoded content to temporary file conversion utilities.
16
17 Converts Base64 encoded content to temporary files for processing."""
18
19 """ This module provides a function to decode Base64 encoded content and write it to a temporary file with the specified extension.
20
21 Functions:
22 - `base64_to_tmpfile`: Convert Base64 encoded content to a temporary file.
23 """
24
25 import base64
26 import tempfile
```

```

27 import os
28
29 def base64_to_tmpfile(content: str, file_name: str) -> str:
30 """
31 Convert Base64 encoded content to a temporary file.
32
33 This function decodes the Base64 encoded content and write
34 s it to a temporary file with the same extension as the provid
35 ed file name.
36
37 The path to the temporary file is returned.
38
39 Args:
40 content (str): Base64 encoded content to be decoded an
41 d written to the file.
42 file_name (str): Name of the file used to extract the
43 file extension for the temporary file.
44
45 Returns:
46 str: Path to the temporary file.
47
48 Example:
49 >>> base64_content = "SGVsbG8gd29ybGQh" # Base64 enco
50 ded content "Hello world!"
51 >>> file_name = "example.txt"
52 >>> tmp_file_path = base64_to_tmpfile(base64_content,
53 file_name)
54 >>> print(f"Temporary file created at: {tmp_file_path}
55 ")
56 Temporary file created at: /tmp/tmpfile.txt
57
58 _, ext = os.path.splitext(file_name)
59 path = ''
60 with tempfile.NamedTemporaryFile(delete=False, suffix=ext)
61 as tmp:
62 tmp.write(base64.b64decode(content))
63 path = tmp.name
64
65 return path

```

```
57 |
58 | def base64encode(image_path):
59 | # Function to encode the image
60 | with open(image_path, "rb") as image_file:
61 | return base64.b64encode(image_file.read()).decode('utf-8')
```

## csv.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Convert CSV and JSON file formats
4 # =====
5 # Description:
6 # Functions:
7 #
8 # File: csv.py
9 # Project: ai-breadboard
10 # Package: src.utils.convertors
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """CSV and JSON file conversion utilities."""
16
17 """ Functions:
18 - `csv2dict`: Convert CSV data to a dictionary.
19 - `csv2ns`: Convert CSV data to SimpleNamespace objects.
20
21 .. code-block:: python
22
23 # Example usage:
24
25 # Using JSON list of dictionaries
26 json_data_list = [{"name": "John", "age": 30, "city": "New
27 York"}, {"name": "Alice", "age": 25, "city": "Los Angeles"}]
28 json_file_path = 'data.json'
29 csv_file_path = 'data.csv'
30
31 # Convert JSON to CSV
32 json2csv.json2csv(json_data_list, csv_file_path)
```

```

33 | # Convert CSV back to JSON
34 | csv_data = csv2json(csv_file_path, json_file_path)
35 | if csv_data:
36 | if isinstance(csv_data, list):
37 | if isinstance(csv_data[0], dict):
38 | print("CSV data (list of dictionaries):")
39 | else:
40 | print("CSV data (list of values):")
41 | print(csv_data)
42 | else:
43 | print("Failed to read CSV data.")
44 | """
45 |
46 | import json
47 | import csv
48 | from pathlib import Path
49 | from typing import List, Dict
50 | from types import SimpleNamespace
51 | from src.logger.logger import logger
52 | from src.utils.csv import read_csv_as_dict, read_csv_as_ns, save_csv_file, read_csv_file
53 |
54 | def csv2dict(csv_file: str | Path, *args, **kwargs) -> dict |
55 | None:
56 | """
57 | Convert CSV data to a dictionary.
58 |
59 | Args:
60 | csv_file (str | Path): Path to the CSV file to read.
61 |
62 | Returns:
63 | dict | None: Dictionary containing the data from CSV converted to JSON format, or `None` if conversion failed.
64 |
65 | Raises:
66 | Exception: If unable to read CSV.
67 | """
68 | return read_csv_as_dict(csv_file, *args, **kwargs)

```

```

68
69 def csv2ns(csv_file: str | Path, *args, **kwargs) -> SimpleNam
 espace | None:
70 """
71 Convert CSV data to SimpleNamespace objects.
72
73 Args:
74 csv_file (str | Path): Path to the CSV file to read.
75
76 Returns:
77 SimpleNamespace | None: SimpleNamespace object contain
 ing the data from CSV, or `None` if conversion failed.
78
79 Raises:
80 Exception: If unable to read CSV.
81 """
82 return read_csv_as_ns(csv_file, *args, **kwargs)
83
84 def csv_to_json(
85 csv_file_path: str | Path,
86 json_file_path: str | Path,
87 exc_info: bool = True
88) -> List[Dict[str, str]] | None:
89 """ Convert a CSV file to JSON format and save it to a JSO
 N file.
90
91 Args:
92 csv_file_path (str | Path): The path to the CSV file t
 o read.
93 json_file_path (str | Path): The path to the JSON file
 to save.
94 exc_info (bool, optional): If True, includes traceback
 information in the log. Defaults to True.
95
96 Returns:
97 List[Dict[str, str]] | None: The JSON data as a list o
 f dictionaries, or None if conversion failed.
98

```

```
99 | Example:
100 | >>> json_data = csv_to_json('dialogue_log.csv', 'dialogue_log.json')
101 | >>> print(json_data)
102 | [{'role': 'user', 'content': 'Hello'}, {'role': 'assistant', 'content': 'Hi there!'}]
103 | """
104 | try:
105 | data = read_csv_file(csv_file_path, exc_info=exc_info)
106 | if data is not None:
107 | with open(json_file_path, 'w', encoding='utf-8') as jsonfile:
108 | json.dump(data, jsonfile, indent=4)
109 | return data
110 | return
111 | except Exception as ex:
112 | logger.error("Failed to convert CSV to JSON", ex, exc_info=exc_info)
113 | return
114 |
115 |
```

dict.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Dictionary and SimpleNamespace conversion utilities
4 # =====
5 # Description:
6 # Converts between dictionaries and SimpleNamespace objects recursively.
7 # Provides export functions to various formats (XML, CSV, JSON, XLS, HTML, PDF)
8 # for flexible data serialization and format conversion.
9 #
10 # File: dict.py
11 # Project: ai-breadboard
12 # Package: src.utils.convertors
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 """
18 Converter for converting between dict and SimpleNamespace objects.
19
20 Functions:
21 - `dict2ns`: Recursively convert dictionaries to SimpleNamespace objects.
22 - `dict2xml`: Generate an XML string from a dictionary.
23 - `dict2csv`: Save dictionary or SimpleNamespace data to a CSV file.
24 - `dict2json`: Save dictionary or SimpleNamespace data to a JSON file.
25 - `dict2xls`: Save dictionary or SimpleNamespace data to an XLS file.
```



```

26 | - `dict2html`: Generate an HTML table string from a dictio
 | nary or SimpleNamespace object.
27 | - `dict2pdf`: Save dictionary data as a PDF file.
28 | """
29 |
30 | import json
31 | from types import SimpleNamespace
32 | from typing import Any, Dict, List
33 | from pathlib import Path
34 | from xml.dom.minidom import getDOMImplementation
35 | from reportlab.lib.pagesizes import A4
36 | from reportlab.pdfgen import canvas
37 | from src.utils.xls import save_xls_file
38 |
39 | def replace_key_in_dict(data, old_key, new_key) -> dict:
40 | """
41 | Recursively replaces a key in a dictionary or list.
42 |
43 | Args:
44 | data (dict | list): The dictionary or list where key r
 | eplacement occurs.
45 | old_key (str): The key to be replaced.
46 | new_key (str): The new key.
47 |
48 | Returns:
49 | dict: The updated dictionary with replaced keys.
50 |
51 | Example Usage:
52 |
53 | replace_key_in_json(data, 'name', 'category_name')
54 |
55 | # Example 1: Simple dictionary
56 | data = {"old_key": "value"}
57 | updated_data = replace_key_in_json(data, "old_key", "n
 | ew_key")
58 | # updated_data becomes {"new_key": "value"}
59 |
60 | # Example 2: Nested dictionary

```

```

61 | data = {"outer": {"old_key": "value"}}
62 | updated_data = replace_key_in_json(data, "old_key", "new_key")
63 | # updated_data becomes {"outer": {"new_key": "value"}}
64 |
65 | # Example 3: List of dictionaries
66 | data = [{"old_key": "value1"}, {"old_key": "value2"}]
67 | updated_data = replace_key_in_json(data, "old_key", "new_key")
68 | # updated_data becomes [{"new_key": "value1"}, {"new_key": "value2"}]
69 |
70 | # Example 4: Mixed nested structure with lists and dictionaries
71 | data = {"outer": [{"inner": {"old_key": "value"}}]}
72 | updated_data = replace_key_in_json(data, "old_key", "new_key")
73 | # updated_data becomes {"outer": [{"inner": {"new_key": "value"}}]}
74 |
75 | """
76 | if isinstance(data, dict):
77 | for key in list(data.keys()):
78 | new_k = key
79 | if key == old_key:
80 | data[new_key] = data.pop(old_key)
81 | new_k = new_key
82 |
83 | if isinstance(data[new_k], (dict, list)):
84 | replace_key_in_dict(data[new_k], old_key, new_key)
85 | elif isinstance(data, list):
86 | for item in data:
87 | replace_key_in_dict(item, old_key, new_key)
88 |
89 | return data
90 |
91 | def dict2pdf(data: dict | SimpleNamespace, file_path: str | Pa

```

```

th) -> None:
92 """
93 Save dictionary data to a PDF file.
94
95 Args:
96 data (dict | SimpleNamespace): The dictionary to convert to PDF.
97 file_path (str | Path): Path to the output PDF file.
98 """
99 if isinstance(data, SimpleNamespace):
100 data = data.__dict__
101
102 pdf = canvas.Canvas(str(file_path), pagesize=A4)
103 width, height = A4
104 x, y = 50, height - 50
105
106 pdf.setFont("Helvetica", 12)
107
108 for key, value in data.items():
109 line = f"{key}: {value}"
110 pdf.drawString(x, y, line)
111 y -= 20
112
113 if y < 50: # Create new page if not enough space
114 pdf.showPage()
115 pdf.setFont("Helvetica", 12)
116 y = height - 50
117
118 pdf.save()
119
120 def dict2ns(data: Dict[str, Any] | List[Any]) -> Any:
121 """
122 Recursively convert dictionaries to SimpleNamespace.
123
124 Args:
125 data (Dict[str, Any] | List[Any]): The data to convert
126 .

```

```

127 | Returns:
128 | Any: Converted data as a SimpleNamespace or a list of
129 | SimpleNamespace.
130 | """
131 | if isinstance(data, dict):
132 | for key, value in data.items():
133 | if isinstance(value, dict):
134 | data[key] = dict2ns(value)
135 | elif isinstance(value, list):
136 | data[key] = [dict2ns(item) if isinstance(item,
137 | dict) else item for item in value]
138 | return SimpleNamespace(**data)
139 | elif isinstance(data, list):
140 | return [dict2ns(item) if isinstance(item, dict) else i
141 | tem for item in data]
142 | return data
143 |
144 | def dict2xml(data: Dict[str, Any], encoding: str = 'UTF-8') ->
145 | str:
146 | """
147 | Generate an XML string from a dictionary.
148 |
149 | Args:
150 | data (Dict[str, Any]): The data to convert to XML.
151 | encoding (str, optional): Data encoding. Defaults to '
152 | UTF-8'.
153 |
154 | Returns:
155 | str: The XML string representing the input dictionary.
156 |
157 | Raises:
158 | Exception: If more than one root node is provided.
159 | """
160 | def _process_simple(doc, tag, tag_value):
161 | """
162 | Generate a node for simple types (int, str).
163 |
164 | Args:

```

```

160 doc (xml.dom.minidom.Document): XML document objec
t.
161 tag (str): Tag name for the XML element.
162 tag_value (Any): Value of the tag.
163
164 Returns:
165 xml.dom.minidom.Element: Node representing the tag
and value.
166 """
167 node = doc.createElement(tag)
168 node.appendChild(doc.createTextNode(str(tag_value)))
169 return node
170
171 def _process_attr(doc, attr_value: Dict[str, Any]):
172 """
173 Generate attributes for an XML element.
174
175 Args:
176 doc (xml.dom.minidom.Document): XML document objec
t.
177 attr_value (Dict[str, Any]): Dictionary of attribu
tes.
178
179 Returns:
180 List[xml.dom.minidom.Attr]: List of attributes for
the XML element.
181 """
182 attrs = []
183 for attr_name, value in attr_value.items():
184 attr = doc.createAttribute(attr_name)
185 attr.nodeValue = value if not isinstance(value, di
ct) else value.get('value', '')
186 attrs.append(attr)
187 return attrs
188
189 def _process_complex(doc, children):
190 """
191 Generate nodes for complex types like lists or dicts.

```

```

192 |
193 | Args:
194 | doc (xml.dom.minidom.Document): XML document object.
195 | children (List[Tuple[str, Any]]): List of tag-value pairs.
196 |
197 | Returns:
198 | Tuple[List[xml.dom.minidom.Element], List[xml.dom.minidom.Attr]]: List of child nodes and attributes.
199 | """
200 | nodelist = []
201 | attrs = []
202 | for tag, value in children:
203 | if tag == 'attrs':
204 | attrs = _process_attr(doc, value)
205 | else:
206 | nodes = _process(doc, tag, value)
207 | nodelist.extend(nodes if isinstance(nodes, list) else [nodes])
208 | return nodelist, attrs
209 |
210 | def _process(doc, tag, tag_value):
211 | """
212 | Generate XML DOM object for a tag and its value.
213 |
214 | Args:
215 | doc (xml.dom.minidom.Document): XML document object.
216 | tag (str): Tag name for the XML element.
217 | tag_value (Any): Value of the tag.
218 |
219 | Returns:
220 | xml.dom.minidom.Element | List[xml.dom.minidom.Element]: Node or list of nodes for the tag and value.
221 | """
222 | if isinstance(tag_value, dict) and list(tag_value.keys()) == ['value']:

```

```

223 tag_value = tag_value['value']
224
225 if tag_value is None:
226 tag_value = ''
227
228 if isinstance(tag_value, (float, int, str)):
229 return _process_simple(doc, tag, tag_value)
230
231 if isinstance(tag_value, list):
232 return _process_complex(doc, [(tag, x) for x in ta
g_value])[0]
233
234 if isinstance(tag_value, dict):
235 node = doc.createElement(tag)
236 nodelist, attrs = _process_complex(doc, tag_value.
items())
237 for child in nodelist:
238 node.appendChild(child)
239 for attr in attrs:
240 node.setAttributeNode(attr)
241 return node
242
243 doc = getDOMImplementation().createDocument(None, None, No
ne)
244 if len(data) > 1:
245 raise Exception('Only one root node allowed')
246
247 root, _ = _process_complex(doc, data.items())
248 doc.appendChild(root[0])
249 return doc.toxml(encoding)
250
251 def dict2csv(data: dict | SimpleNamespace, file_path: str | Pa
th) -> bool:
252 """
253 Save dictionary or SimpleNamespace data to a CSV file.
254
255 Args:
256 data (dict | SimpleNamespace): The data to save to a C

```

```

SV file.
257 file_path (str | Path): Path to the CSV file.
258
259 Returns:
260 bool: True if the file was saved successfully, False o
therwise.
261 """
262 return save_csv_file(data, file_path)
263
264 def dict2xls(data: dict | SimpleNamespace, file_path: str | Pa
th) -> bool:
265 """
266 Save dictionary or SimpleNamespace data to an XLS file.
267
268 Args:
269 data (dict | SimpleNamespace): The data to save to an
XLS file.
270 file_path (str | Path): Path to the XLS file.
271
272 Returns:
273 bool: True if the file was saved successfully, False o
therwise.
274 """
275 return save_xls_file(data, file_path)
276
277 def dict2html(data: dict | SimpleNamespace, encoding: str = 'U
TF-8') -> str:
278 """
279 Generate an HTML table string from a dictionary or SimpleN
amespace object.
280
281 Args:
282 data (dict | SimpleNamespace): The data to convert to
HTML.
283 encoding (str, optional): Data encoding. Defaults to '
UTF-8'.
284
285 Returns:

```



```

286 | str: The HTML string representing the input dictionary
287 | .
288 | """
289 | def dict_to_html_table(data: dict, depth: int = 0) -> str:
290 | """
291 | Recursively convert dictionary to HTML table.
292 |
293 | Args:
294 | data (dict): The dictionary data to convert.
295 | depth (int, optional): The depth of recursion, use
296 | d for nested tables. Defaults to 0.
297 |
298 | Returns:
299 | str: The HTML table as a string.
300 | """
301 | html = ['<table border="1" cellpadding="5" cellspacing="0">']
302 |
303 | if isinstance(data, dict):
304 | for key, value in data.items():
305 | html.append('<tr>')
306 | html.append(f'<td>{key}</td>')
307 |)
308 |
309 | if isinstance(value, dict):
310 | html.append(f'<td>{dict_to_html_table(value, depth + 1)}</td>')
311 | elif isinstance(value, list):
312 | html.append('<td>')
313 | html.append('')
314 | for item in value:
315 | html.append(f'{item}')
316 | html.append('')
317 | html.append('</td>')
318 | else:
319 | html.append(f'<td>{value}</td>')
320 | html.append('</tr>')
321 | else:
322 | html.append(f'<tr><td colspan="2">{data}</td></tr>')

```

```
 ')
319
320 html.append('</table>')
321 return '\n'.join(html)
322
323 # Convert data to dictionary if it's a SimpleNamespace
324 if isinstance(data, SimpleNamespace):
325 data = data.__dict__
326
327 html_content = dict_to_html_table(data)
328 return f'<!DOCTYPE html>\n<html>\n<head>\n<meta charset="{
encoding}">\n<title>Dictionary to HTML</title>\n</head>\n<body
>\n{html_content}\n</body>\n</html>'
329
330
```

dot.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Convert DOT files to PNG images
4 # =====
5 # Description:
6 # Converts a DOT file to a PNG image.
7 #
8 # File: dot.py
9 # Project: ai-breadboard
10 # Package: src.utils.convertors
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """DOT file to PNG image conversion utilities.
16
17 Converts DOT format graph description files to PNG images using
18 Graphviz library."""
19
20 import sys
21 from graphviz import Source
22
23 def dot2png(dot_file: str, png_file: str) -> None:
24 """ Converts a DOT file to a PNG image.
25
26 Args:
27 dot_file (str): The path to the input DOT file.
28 png_file (str): The path where the output PNG file will
29 be saved.
30
31 Raises:
32 FileNotFoundError: If the DOT file does not exist.
33 Exception: For other errors during conversion.
```

32 |  
33 | Example:

34 | >>> dot2png('example.dot', 'output.png')

35 |  
36 | This converts the DOT file 'example.dot' into a PNG image named 'output.png'.

37 |  
38 | Sample DOT content for 'example.dot':

39 |  
40 | ```dot  
41 | digraph G {  
42 | A -> B;  
43 | B -> C;  
44 | C -> A;  
45 | }  
46 | ```

47 |  
48 | To run the script from the command line:

49 |  
50 | ```bash  
51 | python dot2png.py example.dot output.png  
52 | ```

53 |  
54 | This command will create a PNG file named 'output.png' from the graph defined in 'example.dot'.

55 | """

56 | try:

57 | # Read the DOT file

58 | with open(dot\_file, 'r') as f:

59 | dot\_content = f.read()

60 |  
61 | # Create a Source object from the DOT content

62 | source = Source(dot\_content)

63 |  
64 | # Render the source to a PNG file

65 | source.format = 'png'

66 | source.render(png\_file, cleanup=True)

67 | except FileNotFoundError as e:

```
68 | print(f"Error: The file '{dot_file}' was not found.")
69 | raise e
70 | except Exception as e:
71 | print(f"An error occurred during the conversion: {e}")
72 | raise e
73 |
74 | if __name__ == "__main__":
75 | if len(sys.argv) != 3:
76 | print("Usage: python dot2png.py <input_dot_file> <output_png_file>")
77 | sys.exit(1)
78 |
79 | input_dot_file = sys.argv[1]
80 | output_png_file = sys.argv[2]
81 |
82 | dot2png(input_dot_file, output_png_file)
83 |
```

html.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: HTML conversion and parsing utilities
4 # =====
5 # Description:
6 # HTML conversion utilities for transforming HTML content to
7 # escape sequences,
8 # dictionaries, and SimpleNamespace objects. Includes parsing
9 # functions for
10 # HTML to structured data format conversions.
11 #
12 # File: html.py
13 # Project: ai-breadboard
14 # Package: src.utils.convertors
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # =====
18 """
19 HTML conversion utilities.
20
21 Functions:
22 - `html2escape`: Convert HTML to escape sequences.
23 - `escape2html`: Convert escape sequences to HTML.
24 - `html2dict`: Convert HTML to dictionaries.
25 - `html2ns`: Convert HTML to SimpleNamespace objects.
26 """
27
28 import re
29 import html
30 from typing import Dict
31 from pathlib import Path
32 from src.logger.logger import logger
```

```

32 from types import SimpleNamespace
33 from html.parser import HTMLParser
34 try:
35 from xhtml2pdf import pisa
36 except Exception as ex:
37 logger.error(ex)
38 try:
39 from weasyprint import HTML
40 except Exception as ex:
41 logger.error(ex)
42
43 def html2escape(input_str: str) -> str:
44 """
45 Convert HTML to escape sequences.
46
47 Args:
48 input_str (str): The HTML code.
49
50 Returns:
51 str: HTML converted into escape sequences.
52
53 Example:
54 >>> html = "<p>Hello, world!</p>"
55 >>> result = html2escape(html)
56 >>> print(result)
57 <p>Hello, world!</p>
58 """
59 return html.escape(input_str)
60
61 def escape2html(input_str: str) -> str:
62 """
63 Convert escape sequences to HTML.
64
65 Args:
66 input_str (str): The string with escape sequences.
67
68 Returns:
69 str: The escape sequences converted back into HTML.

```

```

70 |
71 | Example:
72 | >>> escaped = "<p>Hello, world!</p>"
73 | >>> result = escape2html(escaped)
74 | >>> print(result)
75 | <p>Hello, world!</p>
76 | """
77 | return html.unescape(input_str)
78 |
79 | def html2dict(html_str: str) -> Dict[str, str]:
80 | """
81 | Convert HTML to a dictionary where tags are keys and conte
82 | nt are values.
83 |
84 | Args:
85 | html_str (str): The HTML string to convert.
86 |
87 | Returns:
88 | dict: A dictionary with HTML tags as keys and their co
89 | ntent as values.
90 |
91 | Example:
92 | >>> html = "<p>Hello</p>World"
93 | >>> result = html2dict(html)
94 | >>> print(result)
95 | {'p': 'Hello', 'a': 'World'}
96 | """
97 |
98 | class HTMLToDictParser(HTMLParser):
99 | def __init__(self):
100 | super().__init__()
101 | self.result = {}
102 | self.current_tag = None
103 |
104 | def handle_starttag(self, tag, attrs):
105 | self.current_tag = tag

```



```

106
107 def handle_data(self, data):
108 if self.current_tag:
109 self.result[self.current_tag] = data.strip()
110
111 parser = HTMLToDictParser()
112 parser.feed(html_str)
113 return parser.result
114
115 def html2ns(html_str: str) -> SimpleNamespace:
116 """
117 Convert HTML to a SimpleNamespace object where tags are at
118 |tributes and content are values.
119
120 Args:
121 html_str (str): The HTML string to convert.
122
123 Returns:
124 SimpleNamespace: A SimpleNamespace object with HTML ta
125 |gs as attributes and their content as values.
126
127 Example:
128 >>> html = "<p>Hello</p>World"
129 >>> result = html2ns(html)
130 >>> print(result.p)
131 Hello
132 >>> print(result.a)
133 World
134 """
135 html_dict = html2dict(html_str)
136 return SimpleNamespace(**html_dict)
137
138 # def html2pdf(html_str: str, pdf_file: str | Path) -> bool |
139 # |None:
140 # """Converts HTML content to a PDF file after removing un
141 # |supported CSS pseudo-selectors.
142
143 # Args:

```

```

140 # html_str (str): HTML content as a string.
141 # pdf_file (str | Path): Path to the output PDF file.
142
143 # Returns:
144 # bool | None: Returns `True` if PDF generation is suc
145 # cessful; `None` otherwise.
146 # """
147 # ...
148 # def preprocess_css(css_content: str) -> str:
149 # """
150 # Remove unsupported pseudo-classes and simplify CSS f
151 # or xhtml2pdf.
152 #
153 # Args:
154 # css_content (str): Original CSS content.
155 #
156 # Returns:
157 # str: Preprocessed CSS content without unsupporte
158 # d selectors.
159 # """
160 # # Remove `:not(...)`
161 # css_content = re.sub(r':not\[([^\]]*)\]', '', css_conte
162 # nt)
163
164 # return css_content
165 # # Remove unsupported pseudo-classes if present
166 # html_str = preprocess_css(html_str)
167
168 # with open(pdf_file, "wb") as f:
169 # pisa_status = pisa.CreatePDF(html_str, dest=f)
170
171 # if pisa_status.err:
172 # print("Error during PDF generation")
173 # return
174 # else:
175 # return True
176
177 def html2pdf(html_str: str, pdf_file: str | Path) -> bool | No

```

```
ne:
174 | """Converts HTML content to a PDF file using WeasyPrint."""
 | "
175 | try:
176 | HTML(string=html_str).write_pdf(pdf_file)
177 | return True
178 | except Exception as e:
179 | print(f"Error during PDF generation: {e}")
180 | return
181 |
182 |
```

html2text.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: html2text: Turn HTML into equivalent Markdown-
 stru
4 # =====
5 # Description:
6 # html2text: Turn HTML into equivalent Markdown-structured t
 ext. ""
7 #
8 # File: html2text.py
9 # Project: ai-breadboard
10 # Package: src.utils.convertors
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """HTML to Markdown conversion utilities."""
16
17 """html2text: Turn HTML into equivalent Markdown-structured te
 xt."""
18 __version__ = "3.1"
19 __author__ = "Aaron Swartz (me@aaronsw.com)"
20 __copyright__ = "(C) 2004-2008 Aaron Swartz. GNU GPL 3."
21 __contributors__ = ["Martin 'Joey' Schulze", "Ricardo Reyes",
 "Kevin Jay North"]
22
23 # TODO:
24 # Support decoded entities with unifiable.
25
26 try:
27 True
28 except NameError:
29 setattr(__builtins__, 'True', 1)
```

```
30 setattr(__builtins__, 'False', 0)
31
32 def has_key(x, y):
33 if hasattr(x, 'has_key'): return x.has_key(y)
34 else: return y in x
35
36 import html.entities as htmlentitydefs
37 import urllib.parse as urlparse
38 import html.parser as HTMLParser
39 import urllib.request as urllib
40 import optparse, re, sys, codecs, types
41 from textwrap import wrap
42
43 # Use Unicode characters instead of their ascii psuedo-replace
44 # ments
45 UNICODE_SNOB = 0
46
47 # Put the links after each paragraph instead of at the end.
48 LINKS_EACH_PARAGRAPH = 0
49
50 # Wrap long lines at position. 0 for no wrapping. (Requires Py
51 # thon 2.3.)
52 BODY_WIDTH = 78
53
54 # Don't show internal links (href="#local-anchor") -- correspo
55 # nding link targets
56 # won't be visible in the plain text file anyway.
57 SKIP_INTERNAL_LINKS = True
58
59 # Use inline, rather than reference, formatting for images and
60 # links
61 INLINE_LINKS = True
62
63 # Number of pixels Google indents nested lists
64 GOOGLE_LIST_INDENT = 36
65
66 IGNORE_ANCHORS = False
67 IGNORE_IMAGES = False
```

```

64
65 ### Entity Nonsense ###
66
67 def name2cp(k):
68 if k == 'apos': return ord("'")
69 if hasattr(htmlentitydefs, "name2codepoint"): # requires P
python 2.3
70 return htmlentitydefs.name2codepoint[k]
71 else:
72 k = htmlentitydefs.entitydefs[k]
73 if k.startswith("&#") and k.endswith(";"): return int(
k[2:-1]) # not in latin-1
74 return ord(codecs.latin_1_decode(k)[0])
75
76 unifiable = {'rsquo': "'", 'lsquo': "'", 'rdquo': "'", 'ldquo': '"
',
77 'copy': '(C)', 'mdash': '--', 'nbsp': ' ', 'rarr': '->', 'larr': '<
-', 'middot': '*',
78 'ndash': '-', 'oelig': 'oe', 'aelig': 'ae',
79 'agrave': 'a', 'aacute': 'a', 'acirc': 'a', 'atilde': 'a', 'auml':
'a', 'aring': 'a',
80 'egrave': 'e', 'eacute': 'e', 'ecirc': 'e', 'euml': 'e',
81 'igrave': 'i', 'iacute': 'i', 'icirc': 'i', 'iuml': 'i',
82 'ograve': 'o', 'oacute': 'o', 'ocirc': 'o', 'otilde': 'o', 'ouml':
'o',
83 'ugrave': 'u', 'uacute': 'u', 'ucirc': 'u', 'uuml': 'u',
84 'lrm': '', 'rlm': ''}
85
86 unifiable_n = {}
87
88 for k in unifiable.keys():
89 unifiable_n[name2cp(k)] = unifiable[k]
90
91 def charref(name):
92 if name[0] in ['x', 'X']:
93 c = int(name[1:], 16)
94 else:
95 c = int(name)

```

```

96
97 if not UNICODE_SNOB and c in unifiable_n.keys():
98 return unifiable_n[c]
99 else:
100 try:
101 return chr(c)
102 except NameError: #Python3
103 return chr(c)
104
105 def entityref(c):
106 if not UNICODE_SNOB and c in unifiable.keys():
107 return unifiable[c]
108 else:
109 try: name2cp(c)
110 except KeyError: return "&" + c + ';'
111 else:
112 try:
113 return chr(name2cp(c))
114 except NameError: #Python3
115 return chr(name2cp(c))
116
117 def replaceEntities(s):
118 s = s.group(1)
119 if s[0] == "#":
120 return charref(s[1:])
121 else: return entityref(s)
122
123 r_unescape = re.compile(r"&(#?[xX]?(?:[0-9a-fA-F]+|\w{1,8}));"
124)
125 def unescape(s):
126 return r_unescape.sub(replaceEntities, s)
127
128 ### End Entity Nonsense ###
129
130 def onlywhite(line):
131 """Return true if the line does only consist of whitespace
 characters."""
 for c in line:

```

```

132 if c is not ' ' and c is not '\n':
133 return c is ' '
134 return line
135
136 def optwrap(text):
137 """Wrap all paragraphs in the provided text."""
138 if not BODY_WIDTH:
139 return text
140
141 assert wrap, "Requires Python 2.3."
142 result = ''
143 newlines = 0
144 for para in text.split("\n"):
145 if len(para) > 0:
146 if para[0] != ' ' and para[0] != '-' and para[0] !=
= '*':
147 for line in wrap(para, BODY_WIDTH):
148 result += line + "\n"
149 result += "\n"
150 newlines = 2
151 else:
152 if not onlywhite(para):
153 result += para + "\n"
154 newlines = 1
155 else:
156 if newlines < 2:
157 result += "\n"
158 newlines += 1
159 return result
160
161 def hn(tag):
162 if tag[0] == 'h' and len(tag) == 2:
163 try:
164 n = int(tag[1])
165 if n in range(1, 10): return n
166 except ValueError: return 0
167
168 def dumb_property_dict(style):

```



```

169 """returns a hash of css attributes"""
170 return dict([(x.strip(), y.strip()) for x, y in [z.split('
: ', 1) for z in style.split(';') if ':' in z]]);
171
172 def dumb_css_parser(data):
173 """returns a hash of css selectors, each of which contains
a hash of css attributes"""
174 # remove @import sentences
175 importIndex = data.find('@import')
176 while importIndex != -1:
177 data = data[0:importIndex] + data[data.find(';', import
tIndex) + 1:]
178 importIndex = data.find('@import')
179
180 # parse the css. reverted from dictionary comprehension in
order to support older pythons
181 elements = [x.split('{') for x in data.split('}') if '{'
in x.strip()]
182 elements = dict([(a.strip(), dumb_property_dict(b)) for a,
b in elements])
183
184 return elements
185
186 def element_style(attrs, style_def, parent_style):
187 """returns a hash of the 'final_verdict' style attributes
of the element"""
188 style = parent_style.copy()
189 if 'class' in attrs:
190 for css_class in attrs['class'].split():
191 css_style = style_def['.' + css_class]
192 style.update(css_style)
193 if 'style' in attrs:
194 immediate_style = dumb_property_dict(attrs['style'])
195 style.update(immediate_style)
196 return style
197
198 def google_list_style(style):
199 """finds out whether this is an ordered or unordered list"

```

```

 """
200 if 'list-style-type' in style:
201 list_style = style['list-style-type']
202 if list_style in ['disc', 'circle', 'square', 'none']:
203 return 'ul'
204 return 'ol'
205
206 def google_nest_count(style):
207 """calculate the nesting count of google doc lists"""
208 nest_count = 0
209 if 'margin-left' in style:
210 nest_count = int(style['margin-left'][:-2]) / GOOGLE_L
 IST_INDENT
211 return nest_count
212
213 def google_has_height(style):
214 """check if the style of the element has the 'height' attr
 ibute explicitly defined"""
215 if 'height' in style:
216 return True
217 return False
218
219 def google_text_emphasis(style):
220 """return a list of all emphasis modifiers of the element"
 """
221 emphasis = []
222 if 'text-decoration' in style:
223 emphasis.append(style['text-decoration'])
224 if 'font-style' in style:
225 emphasis.append(style['font-style'])
226 if 'font-weight' in style:
227 emphasis.append(style['font-weight'])
228 return emphasis
229
230 def google_fixed_width_font(style):
231 """check if the css of the current element defines a fixed
 width font"""
232 font_family = ''

```

```

233 if 'font-family' in style:
234 font_family = style['font-family']
235 if 'Courier New' == font_family or 'Consolas' == font_fami
ly:
236 return True
237 return False
238
239 def list_numbering_start(attrs):
240 """extract numbering from list element attributes"""
241 if 'start' in attrs:
242 return int(attrs['start']) - 1
243 else:
244 return 0
245
246 class _html2text(HTMLParser.HTMLParser):
247 def __init__(self, out=None, baseurl=''):
248 HTMLParser.HTMLParser.__init__(self)
249
250 if out is None: self.out = self.outtextf
251 else: self.out = out
252 self.outtextlist = [] # empty list to store output cha
racters before they are "joined"
253 try:
254 self.outtext = unicode()
255 except NameError: # Python3
256 self.outtext = str()
257 self.quiet = 0
258 self.p_p = 0 # number of newline character to print be
fore next output
259 self.outcount = 0
260 self.start = 1
261 self.space = 0
262 self.a = []
263 self.ystack = []
264 self.acount = 0
265 self.list = []
266 self.blockquote = 0
267 self.pre = 0

```

```

268 | self.startpre = 0
269 | self.code = False
270 | self.br_toggle = ''
271 | self.lastWasNL = 0
272 | self.lastWasList = False
273 | self.style = 0
274 | self.style_def = {}
275 | self.tag_stack = []
276 | self.emphasis = 0
277 | self.drop_white_space = 0
278 | self.inheader = False
279 | self.abbr_title = None # current abbreviation definiti
on
280 | self.abbr_data = None # last inner HTML (for abbr bein
g defined)
281 | self.abbr_list = {} # stack of abbreviations to write
later
282 | self.baseurl = baseurl
283 |
284 | if options.google_doc:
285 | del unifiable_n[name2cp('nbsp')]
286 | unifiable['nbsp'] = ' _place_holder;'
287 |
288 | def feed(self, data):
289 | data = data.replace("</' + 'script>", "</ignore>")
290 | HTMLParser.HTMLParser.feed(self, data)
291 |
292 | def outtextf(self, s):
293 | self.outtextlist.append(s)
294 | if s: self.lastWasNL = s[-1] == '\n'
295 |
296 | def close(self):
297 | HTMLParser.HTMLParser.close(self)
298 |
299 | self.pbr()
300 | self.o('', 0, 'end')
301 |
302 | self.outtext = self.outtext.join(self.outtextlist)

```

```

303
304 if options.google_doc:
305 self.outtext = self.outtext.replace(' _place_h
older;', ' ');
306
307 return self.outtext
308
309 def handle_charref(self, c):
310 self.o(charref(c), 1)
311
312 def handle_entityref(self, c):
313 self.o(entityref(c), 1)
314
315 def handle_starttag(self, tag, attrs):
316 self.handle_tag(tag, attrs, 1)
317
318 def handle_endtag(self, tag):
319 self.handle_tag(tag, None, 0)
320
321 def previousIndex(self, attrs):
322 """ returns the index of certain set of attributes (of
a link) in the
323 self.a list
324
325 If the set of attributes is not found, returns Non
e
326
327 """
328 if not has_key(attrs, 'href'): return
329
330 i = -1
331 for a in self.a:
332 i += 1
333 match = 0
334
335 if has_key(a, 'href') and a['href'] == attrs['href
']:
336 if has_key(a, 'title') or has_key(attrs, 'titl
e'):

```

```

336 | if (has_key(a, 'title') and has_key(at
|trs, 'title') and
337 | a['title'] == attrs['title']):
338 | match = True
339 | else:
340 | match = True
341 |
342 | if match: return i
343 |
344 | def drop_last(self, nLetters):
345 | if not self.quiet:
346 | self.outtext = self.outtext[:-nLetters]
347 |
348 | def handle_emphasis(self, start, tag_style, parent_style):
349 | """handles various text emphases"""
350 | tag_emphasis = google_text_emphasis(tag_style)
351 | parent_emphasis = google_text_emphasis(parent_style)
352 |
353 | # handle Google's text emphasis
354 | strikethrough = 'line-through' in tag_emphasis and op
|tions.hide_strikethrough
355 | bold = 'bold' in tag_emphasis and not 'bold' in parent
|_emphasis
356 | italic = 'italic' in tag_emphasis and not 'italic' in
|parent_emphasis
357 | fixed = google_fixed_width_font(tag_style) and not \
358 | google_fixed_width_font(parent_style) and not
|self.pre
359 |
360 | if start:
361 | # crossed-out text must be handled before other at
|tributes
362 | # in order not to output qualifiers unnecessarily
363 | if bold or italic or fixed:
364 | self.emphasis += 1
365 | if strikethrough:
366 | self.quiet += 1
367 | if italic:

```

```

368 self.o("_")
369 self.drop_white_space += 1
370 if bold:
371 self.o("***")
372 self.drop_white_space += 1
373 if fixed:
374 self.o('``')
375 self.drop_white_space += 1
376 self.code = True
377 else:
378 if bold or italic or fixed:
379 # there must not be whitespace before closing
emphasis mark
380 self.emphasis -= 1
381 self.space = 0
382 self.outtext = self.outtext.rstrip()
383 if fixed:
384 if self.drop_white_space:
385 # empty emphasis, drop it
386 self.drop_last(1)
387 self.drop_white_space -= 1
388 else:
389 self.o('``')
390 self.code = False
391 if bold:
392 if self.drop_white_space:
393 # empty emphasis, drop it
394 self.drop_last(2)
395 self.drop_white_space -= 1
396 else:
397 self.o("***")
398 if italic:
399 if self.drop_white_space:
400 # empty emphasis, drop it
401 self.drop_last(1)
402 self.drop_white_space -= 1
403 else:
404 self.o("_")

```

```

405 # space is only allowed after *all* emphasis marks
406 if (bold or italic) and not self.emphasis:
407 self.o(" ")
408 if strikethrough:
409 self.quiet -= 1
410
411 def handle_tag(self, tag, attrs, start):
412 #attrs = fixattrs(attrs)
413 if attrs is None:
414 attrs = {}
415 else:
416 attrs = dict(attrs)
417
418 if options.google_doc:
419 # the attrs parameter is empty for a closing tag.
420 # in addition, we
421 # need the attributes of the parent nodes in order
422 # to get a
423 # complete style description for the current element. we assume
424 # that google docs export well formed html.
425 parent_style = {}
426 if start:
427 if self.tag_stack:
428 parent_style = self.tag_stack[-1][2]
429 tag_style = element_style(attrs, self.style_def, parent_style)
430 self.tag_stack.append((tag, attrs, tag_style))
431 else:
432 dummy, attrs, tag_style = self.tag_stack.pop()
433 if self.tag_stack:
434 parent_style = self.tag_stack[-1][2]
435
436 if hn(tag):
437 self.p()
438 if start:
439 self.inheader = True
440 self.o(hn(tag)*"#" + ' ')

```



```

439 else:
440 self.inheader = False
441 return # prevent redundant emphasis marks on h
eaders
442
443 if tag in ['p', 'div']:
444 if options.google_doc:
445 if start and google_has_height(tag_style):
446 self.p()
447 else:
448 self.soft_br()
449 else:
450 self.p()
451
452 if tag == "br" and start: self.o(" \n")
453
454 if tag == "hr" and start:
455 self.p()
456 self.o("* * *")
457 self.p()
458
459 if tag in ["head", "style", 'script']:
460 if start: self.quiet += 1
461 else: self.quiet -= 1
462
463 if tag == "style":
464 if start: self.style += 1
465 else: self.style -= 1
466
467 if tag in ["body"]:
468 self.quiet = 0 # sites like 9rules.com never close
<head>
469
470 if tag == "blockquote":
471 if start:
472 self.p(); self.o('> ', 0, 1); self.start = 1
473 self.blockquote += 1
474 else:

```

```

475 self.blockquote -= 1
476 self.p()
477
478 if tag in ['em', 'i', 'u']: self.o("_")
479 if tag in ['strong', 'b']: self.o("***")
480 if tag in ['del', 'strike']:
481 if start:
482
483 self.o("<" + tag + ">")
484 else:
485 self.o("</" + tag + ">")
486
487 if options.google_doc:
488 if not self.inheader:
489 # handle some font attributes, but leave head
rs clean
490
491 self.handle_emphasis(start, tag_style, parent_
style)
492
493 if tag == "code" and not self.pre: self.o('`') #TODO:
` ` `this` ` `
494
495 if tag == "abbr":
496 if start:
497 self.abbr_title = None
498 self.abbr_data = ''
499 if has_key(attrs, 'title'):
500 self.abbr_title = attrs['title']
501 else:
502 if self.abbr_title != None:
503 self.abbr_list[self.abbr_data] = self.abbr
_title
504
505 self.abbr_title = None
506 self.abbr_data = ''
507
508 if tag == "a" and not IGNORE_ANCHORS:
509 if start:
510 if has_key(attrs, 'href') and not (SKIP_INTERN
AL_LINKS and attrs['href'].startswith('#')):

```

```

507 self.ystack.append(attrs)
508 self.o("[")
509 else:
510 self.ystack.append(None)
511 else:
512 if self.ystack:
513 a = self.ystack.pop()
514 if a:
515 if INLINE_LINKS:
516 self.o("](" + a['href'] + ")")
517 else:
518 i = self.previousIndex(a)
519 if i is not None:
520 a = self.a[i]
521 else:
522 self.acount += 1
523 a['count'] = self.acount
524 a['outcount'] = self.outcount
525 self.a.append(a)
526 self.o(")[" + str(a['count']) + "]
527 ")
528 if tag == "img" and start and not IGNORE_IMAGES:
529 if has_key(attrs, 'src'):
530 attrs['href'] = attrs['src']
531 alt = attrs.get('alt', '')
532 if INLINE_LINKS:
533 self.o("![")
534 self.o(alt)
535 self.o("](" + attrs['href'] + ")")
536 else:
537 i = self.previousIndex(attrs)
538 if i is not None:
539 attrs = self.a[i]
540 else:
541 self.acount += 1
542 attrs['count'] = self.acount
543 attrs['outcount'] = self.outcount

```

```

544 self.a.append(attrs)
545 self.o("![")
546 self.o(alt)
547 self.o("][\"+ str(attrs['count']) +"]")
548
549 if tag == 'dl' and start: self.p()
550 if tag == 'dt' and not start: self.pbr()
551 if tag == 'dd' and start: self.o(' ')
552 if tag == 'dd' and not start: self.pbr()
553
554 if tag in ["ol", "ul"]:
555 # Google Docs create sub lists as top level lists
556 if (not self.list) and (not self.lastWasList):
557 self.p()
558 if start:
559 if options.google_doc:
560 list_style = google_list_style(tag_style)
561 else:
562 list_style = tag
563 numbering_start = list_numbering_start(attrs)
564 self.list.append({'name':list_style, 'num':num
bering_start})
565 else:
566 if self.list: self.list.pop()
567 self.lastWasList = True
568 else:
569 self.lastWasList = False
570
571 if tag == 'li':
572 self.pbr()
573 if start:
574 if self.list: li = self.list[-1]
575 else: li = {'name':'ul', 'num':0}
576 if options.google_doc:
577 nest_count = google_nest_count(tag_style)
578 else:
579 nest_count = len(self.list)
580 self.o(" " * nest_count) #TODO: line up <

```

```

li>s > 9 correctly.
581 if li['name'] == "ul": self.o(options.ul_item_
mark + " ")
582 elif li['name'] == "ol":
583 li['num'] += 1
584 self.o(str(li['num'])+". ")
585 self.start = 1
586
587 if tag in ["table", "tr"] and start: self.p()
588 if tag == 'td': self.pbr()
589
590 if tag == "pre":
591 if start:
592 self.startpre = 1
593 self.pre = 1
594 else:
595 self.pre = 0
596 self.p()
597
598 def pbr(self):
599 if self.p_p == 0: self.p_p = 1
600
601 def p(self): self.p_p = 2
602
603 def soft_br(self):
604 self.pbr()
605 self.br_toggle = ' '
606
607 def o(self, data, puredata=0, force=0):
608 if self.abbr_data is not None: self.abbr_data += data
609
610 if not self.quiet:
611 if options.google_doc:
612 # prevent white space immediately after 'begin
emphasis' marks ('**' and '_')
613 lstripped_data = data.lstrip()
614 if self.drop_white_space and not (self.pre or
self.code):

```

```

615 data = lstripped_data
616 if lstripped_data != '':
617 self.drop_white_space = 0
618
619 if puredata and not self.pre:
620 data = re.sub('\s+', ' ', data)
621 if data and data[0] == ' ':
622 self.space = 1
623 data = data[1:]
624 if not data and not force: return
625
626 if self.startpre:
627 #self.out(" :") #TODO: not output when already
one there
628 self.startpre = 0
629
630 bq = (">" * self.blockquote)
631 if not (force and data and data[0] == ">") and self.blockquote: bq += " "
632
633 if self.pre:
634 bq += " "
635 data = data.replace("\n", "\n"+bq)
636
637 if self.start:
638 self.space = 0
639 self.p_p = 0
640 self.start = 0
641
642 if force == 'end':
643 # It's the end.
644 self.p_p = 0
645 self.out("\n")
646 self.space = 0
647
648 if self.p_p:
649 self.out((self.br_toggle+'\n'+bq)*self.p_p)
650 self.space = 0

```

```

651 self.br_toggle = ''
652
653 if self.space:
654 if not self.lastWasNL: self.out(' ')
655 self.space = 0
656
657 if self.a and ((self.p_p == 2 and LINKS_EACH_PARAG
658 RAPH) or force == "end"):
659 if force == "end": self.out("\n")
660
661 newa = []
662 for link in self.a:
663 if self.outcount > link['outcount']:
664 self.out(" [" + str(link['count']) + "
665]: " + urlparse.urljoin(self.baseurl, link['href']))
666 if has_key(link, 'title'): self.out("
667 (" + link['title'] + ")")
668 self.out("\n")
669 else:
670 newa.append(link)
671
672 if self.a != newa: self.out("\n") # Don't need
673 an extra line when nothing was done.
674
675 self.a = newa
676
677 if self.abbr_list and force == "end":
678 for abbr, definition in self.abbr_list.items():
679 self.out(" *[" + abbr + "]: " + definitio
680 n + "\n")
681
682 self.p_p = 0
683 self.out(data)
684 self.outcount += 1
685
686 def handle_data(self, data):
687 if r'\</script>' in data: self.quiet -= 1

```

```

683
684 if self.style:
685 self.style_def.update(dumb_css_parser(data))
686
687 self.o(data, 1)
688
689 def unknown_decl(self, data): pass
690
691 def wrapwrite(text):
692 text = text.encode('utf-8')
693 try: #Python3
694 sys.stdout.buffer.write(text)
695 except AttributeError:
696 sys.stdout.write(text)
697
698 def html2text_file(html, out=wrapwrite, baseurl=''):
699 h = _html2text(out, baseurl)
700 h.feed(html)
701 h.feed("")
702 return h.close()
703
704 def html2text(html, baseurl=''):
705 return optwrap(html2text_file(html, None, baseurl))
706
707 class Storage: pass
708 options = Storage()
709 options.google_doc = False
710 options.ul_item_mark = '*'
711
712 if __name__ == "__main__":
713 baseurl = ''
714
715 p = optparse.OptionParser('%prog [(filename|url) [encoding
716 version='%prog ' + __version__])
717 p.add_option("-g", "--google-doc", action="store_true", de
718 st="google_doc",
719 default=False, help="convert an html-exported Google D

```



```

document")
719 p.add_option("-d", "--dash-unordered-list", action="store_
true", dest="ul_style_dash",
720 default=False, help="use a dash rather than a star for
unordered list items")
721 p.add_option("-b", "--body-width", dest="body_width", acti
on="store", type="int",
722 default=78, help="number of characters per output line
, 0 for no wrap")
723 p.add_option("-i", "--google-list-indent", dest="list_inde
nt", action="store", type="int",
724 default=GOOGLE_LIST_INDENT, help="number of pixels Goo
gle indents nested lists")
725 p.add_option("-s", "--hide-strikethrough", action="store_t
rue", dest="hide_strikethrough",
726 default=False, help="hide strike-through text. only re
levant when -g is specified as well")
727 (options, args) = p.parse_args()
728
729 # handle options
730 if options.ul_style_dash:
731 options.ul_item_mark = '-'
732 else:
733 options.ul_item_mark = '*'
734
735 BODY_WIDTH = options.body_width
736 GOOGLE_LIST_INDENT = options.list_indent
737
738 # process input
739 if len(args) > 0:
740 file_ = args[0]
741 encoding = None
742 if len(args) == 2:
743 encoding = args[1]
744 if len(args) > 2:
745 p.error('Too many arguments')
746
747 if file_.startswith('http://') or file_.startswith('ht

```

```

tps://'):
748 baseurl = file_
749 j = urllib.urlopen(baseurl)
750 text = j.read()
751 if encoding is None:
752 try:
753 from feedparser import _getCharacterEncodi
ng as enc
754 except ImportError:
755 enc = lambda x, y: ('utf-8', 1)
756 encoding = enc(j.headers, text)[0]
757 if encoding == 'us-ascii':
758 encoding = 'utf-8'
759 data = text.decode(encoding)
760
761 else:
762 data = open(file_, 'rb').read()
763 if encoding is None:
764 try:
765 from chardet import detect
766 except ImportError:
767 detect = lambda x: {'encoding': 'utf-8'}
768 encoding = detect(data)['encoding']
769 data = data.decode(encoding)
770 else:
771 data = sys.stdin.read()
772 wrapwrite(html2text(data, baseurl))

```

json.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: JSON format conversion utilities
4 # =====
5 # Description:
6 # Converts JSON data into various formats: CSV, SimpleNamesp
7 # Provides flexible serialization methods for different outp
8 # ut formats.
9 #
10 # File: json.py
11 # Project: ai-breadboard
12 # Package: src.utils.convertors
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 """
18 JSON data conversion module.
19
20 Functions:
21 - `json2csv`: Convert JSON data to CSV format.
22 - `json2ns`: Convert JSON data to SimpleNamespace object.
23 - `json2xml`: Convert JSON data to XML format.
24 - `json2xls`: Convert JSON data to XLS format.
25 """
26
27 import json
28 import csv
29 from types import SimpleNamespace
30 from pathlib import Path
31 from typing import List, Dict
```

```

32 from src.utils.csv import save_csv_file
33 from src.utils.jjson import j_dumps
34 from src.utils.xls import save_xls_file
35 from src.utils.convertors.dict import dict2xml
36 from src.logger.logger import logger
37
38 def json2csv(json_data: str | list | dict | Path, csv_file_path: str | Path) -> bool:
39 """
40 Convert JSON data or JSON file to CSV format with a comma
41 delimiter.
42
43 Args:
44 json_data (str | list | dict | Path): JSON data as a string, list of dictionaries, or file path to a JSON file.
45 csv_file_path (str | Path): Path to the CSV file to write.
46
47 Returns:
48 bool: True if successful, False otherwise.
49
50 Raises:
51 ValueError: If unsupported type for json_data.
52 Exception: If unable to parse JSON or write CSV.
53 """
54 try:
55 # Load JSON data
56 if isinstance(json_data, dict):
57 data = [json_data]
58 elif isinstance(json_data, str):
59 data = json.loads(json_data)
60 elif isinstance(json_data, list):
61 data = json_data
62 elif isinstance(json_data, Path):
63 with open(json_data, 'r', encoding='utf-8') as json_file:
64 data = json.load(json_file)
65 else:

```

```

65 raise ValueError("Unsupported type for json_data")
66
67 save_csv_file(data, csv_file_path)
68 return True
69 except Exception as ex:
70 logger.error(f"json2csv failed", ex, True)
71 ...
72
73 def json2ns(json_data: str | dict | Path) -> SimpleNamespace:
74 """
75 Convert JSON data or JSON file to SimpleNamespace object.
76
77 Args:
78 json_data (str | dict | Path): JSON data as a string,
79 dictionary, or file path to a JSON file.
80
81 Returns:
82 SimpleNamespace: Parsed JSON data as a SimpleNamespace
83 object.
84
85 Raises:
86 ValueError: If unsupported type for json_data.
87 Exception: If unable to parse JSON.
88 """
89 try:
90 if isinstance(json_data, dict):
91 data = json_data
92 elif isinstance(json_data, str):
93 data = json.loads(json_data)
94 elif isinstance(json_data, Path):
95 with open(json_data, 'r', encoding='utf-8') as jso
96 n_file:
97 data = json.load(json_file)
98 else:
99 raise ValueError("Unsupported type for json_data")
100
101 return SimpleNamespace(**data)
102 except Exception as ex:

```

```

100 logger.error(f"json2ns failed", ex, True)
101
102 def json2xml(json_data: str | dict | Path, root_tag: str = "ro
 ot") -> str:
103 """
104 Convert JSON data or JSON file to XML format.
105
106 Args:
107 json_data (str | dict | Path): JSON data as a string,
 dictionary, or file path to a JSON file.
108 root_tag (str): The root element tag for the XML.
109
110 Returns:
111 str: The resulting XML string.
112
113 Raises:
114 ValueError: If unsupported type for json_data.
115 Exception: If unable to parse JSON or convert to XML.
116 """
117 return dict2xml(json_data)
118
119 def json2xls(json_data: str | list | dict | Path, xls_file_pat
 h: str | Path) -> bool:
120 """
121 Convert JSON data or JSON file to XLS format.
122
123 Args:
124 json_data (str | list | dict | Path): JSON data as a s
 tring, list of dictionaries, or file path to a JSON file.
125 xls_file_path (str | Path): Path to the XLS file to wr
 ite.
126
127 Returns:
128 bool: True if successful, False otherwise.
129
130 Raises:
131 ValueError: If unsupported type for json_data.
132 Exception: If unable to parse JSON or write XLS.

```

```
133 | """
134 | return save_xls_file(json_data, xls_file_path)
135 |
136 |
```

md.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Markdown to dictionary conversion utilities
4 # =====
5 # Description:
6 # Converts Markdown strings to structured dictionaries including extraction
7 # of JSON content if present. Supports structured parsing of
8 # markdown documents.
9 #
10 # File: md.py
11 # Project: ai-breadboard
12 # Package: src.utils.convertors
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16 """
17 Markdown to dictionary conversion module.
18 Provides parsing functions for converting markdown strings to
19 structured format.
20 """
21 import re
22 from typing import Dict, List, Any
23 from markdown2 import markdown
24 from src.logger.logger import logger
25
26 def md2html(md_string: str, extras: List[str] = None) -> str:
27 """
28 Convert Markdown string to HTML.
29
30 Args:
```



```

31 md_string (str): Markdown string for conversion.
32 extras (list, optional): List of markdown2 extensions.
 Defaults to None.
33
34 Returns:
35 str: HTML representation of Markdown.
36 """
37 try:
38 if extras is None:
39 return markdown(md_string)
40 return markdown(md_string, extras=extras)
41 except Exception as ex:
42 logger.error("Error during Markdown to HTML conversion
 .", exc_info=True)
43 return ""
44
45 def md2dict(md_string: str, extras: List[str] = None) -> Dict[
 str, list[str]]:
46 """
47 Convert Markdown string to structured dictionary.
48
49 Args:
50 md_string (str): Markdown string for conversion.
51 extras (list, optional): List of markdown2 extensions
 for md2html. Defaults to None.
52
53 Returns:
54 Dict[str, list[str]]: Structured representation of Mar
 kdown content.
55 """
56 try:
57
58 html = md2html(md_string, extras)
59 sections: Dict[str, list[str]] = {}
60 current_section: str | None = None
61
62 for line in html.splitlines():
63 if line.startswith('<h'):

```

```

64 | heading_level_match = re.search(r'h(\d)', line
 |)
65 | if heading_level_match:
66 | heading_level = int(heading_level_match.gr
 | oup(1))
67 | section_title = re.sub(r'<.*?>', '', line)
 | .strip()
68 | if heading_level == 1:
69 | current_section = section_title
70 | sections[current_section] = []
71 | elif current_section:
72 | sections[current_section].append(secti
 | on_title)
73 |
74 | elif line.strip() and current_section:
75 | clean_text = re.sub(r'<.*?>', '', line).strip(
 |)
76 | sections[current_section].append(clean_text)
77 |
78 | return sections
79 |
80 | except Exception as ex:
81 | logger.error("Error parsing Markdown to structured dic
 | tionary.", exc_info=True)
82 | return {}
83 |

```

ns.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Convert SimpleNamespace to various formats
4 # =====
5 # Description:
6 # Module for AI Breadboard project.
7 #
8 # File: ns.py
9 # Project: ai-breadboard
10 # Package: src.utils.convertors
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """Convert SimpleNamespace objects to various formats: dict, J
 SON, CSV, XML, XLS.
16
17 Functions:
18 - ns2dict: Convert SimpleNamespace object to dictionary.
19 - ns2json: Convert SimpleNamespace object to JSON format.
20 - ns2csv: Convert SimpleNamespace object to CSV format.
21 - ns2xml: Convert SimpleNamespace object to XML format.
22 - ns2xls: Convert SimpleNamespace object to XLS format."""
23 """
24
25 import json
26 import csv
27 from types import SimpleNamespace
28 from pathlib import Path
29 from typing import List, Dict
30 from src.utils.convertors import xml2dict
31 from src.utils.csv import save_csv_file
32 from src.utils.xls import save_xls_file
```

```

33 | from src.logger.logger import logger
34 |
35 | from types import SimpleNamespace
36 | from typing import Any, Dict
37 |
38 | def ns2dict(ns_obj: SimpleNamespace) -> Dict[str, Any]:
39 | """
40 | Recursively convert a SimpleNamespace object to a dictionary.
41 |
42 | Args:
43 | ns_obj (SimpleNamespace): The SimpleNamespace object to convert.
44 |
45 | Returns:
46 | Dict[str, Any]: Converted dictionary with nested structures handled.
47 | """
48 | def convert(value: Any) -> Any:
49 | """
50 | Recursively process values to handle nested SimpleNamespace, dict, or list.
51 |
52 | Args:
53 | value (Any): Value to process.
54 |
55 | Returns:
56 | Any: Converted value.
57 | """
58 | if isinstance(value, SimpleNamespace):
59 | return {key: convert(val) for key, val in vars(value).items()}
60 | elif isinstance(value, dict):
61 | return {key: convert(val) for key, val in value.items()}
62 | elif isinstance(value, list):
63 | return [convert(item) for item in value]
64 | return value

```

```

65 |
66 | return convert(ns_obj)
67 |
68 | def ns2csv(ns_obj: SimpleNamespace, csv_file_path: str | Path)
69 | -> bool:
70 | """
71 | Convert SimpleNamespace object to CSV format.
72 |
73 | Args:
74 | ns_obj (SimpleNamespace): The SimpleNamespace object to
75 | o convert.
76 | csv_file_path (str | Path): Path to save the CSV file.
77 |
78 | Returns:
79 | bool: True if successful, False otherwise.
80 | """
81 | try:
82 | data = [ns2dict(ns_obj)]
83 | save_csv_file(data, csv_file_path)
84 | return True
85 | except Exception as ex:
86 | logger.error(f"ns2csv failed", ex, True)
87 |
88 | def ns2xml(ns_obj: SimpleNamespace, root_tag: str = "root") ->
89 | str:
90 | """
91 | Convert SimpleNamespace object to XML format.
92 |
93 | Args:
94 | ns_obj (SimpleNamespace): The SimpleNamespace object to
95 | o convert.
96 | root_tag (str): The root element tag for the XML.
97 |
98 | Returns:
99 | str: The resulting XML string.
100 | """
101 | try:
102 | data = ns2dict(ns_obj)

```

```
99 return xml2dict(data)
100 except Exception as ex:
101 logger.error(f"ns2xml failed", ex, True)
102
103 def ns2xls(data: SimpleNamespace, xls_file_path: str | Path) -
 > bool:
104 """
105 Convert SimpleNamespace object to XLS format.
106
107 Args:
108 ns_obj (SimpleNamespace): The SimpleNamespace object t
 o convert.
109 xls_file_path (str | Path): Path to save the XLS file.
110
111 Returns:
112 bool: True if successful, False otherwise.
113 """
114 return save_xls_file(data,xls_file_path)
115
```

png.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Convert text to PNG images
4 # =====
5 # Description:
6 # Module for AI Breadboard project.
7 #
8 # File: png.py
9 # Project: ai-breadboard
10 # Package: src.utils.convertors
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """Text to PNG image conversion utilities.
16
17 Reads text from files, generates PNG images for each line using Pillow,
18 and saves them to output directory with customizable appearance options."""
19 """
20
21 from pathlib import Path
22 from typing import List, Tuple
23 from PIL import Image, ImageDraw, ImageFont
24 from src.logger.logger import logger # Logging
25
26 class TextToImageGenerator:
27 """
28 A class for generating PNG images from text lines.
29
30 **Functions**:
31 - `assign_path`: Determines the correct path for output PNG
```

```

G files, creating the directory if necessary.
32 - `center_text_position`: Calculates the position to center text on the canvas.
33 - `generate_png`: Creates a PNG image with the specified text, font, colors, etc.
34 - `not_comment_or_blank`: Checks if a line is neither a comment nor blank.
35 - `which_exist`: Checks which file names already exist in the directory.
36 - `get_characters`: Extracts text lines from the input file or list, filtering out comments and blank lines.
37 - `parse_size`: Parses a string into a `Size` object.
38 - `get_max_text_size`: Computes the maximum text size based on the font and text lines.
39 - `get_font`: Determines the font size based on canvas size and padding.
40 - `setup_logging`: Configures logging based on the specified logging level.
41 - `error`: Logs an error message and raises an exception.
42 - `overlay_images`: Overlays one PNG image on top of another.
43 """
44
45 def __init__(self):
46 """
47 Initializes the TextToImageGenerator class with default settings.
48 """
49 self.default_output_dir = Path("./output")
50 self.default_canvas_size = (1024, 1024)
51 self.default_padding = 0.10
52 self.default_background = "white"
53 self.default_text_color = "black"
54 self.default_log_level = "WARNING"
55
56 async def generate_images(
57 self,
58 lines: List[str],

```



```

59 | output_dir: str | Path = None,
60 | font: str | ImageFont.ImageFont = "sans-serif",
61 | canvas_size: Tuple[int, int] = None,
62 | padding: float = None,
63 | background_color: str = None,
64 | text_color: str = None,
65 | log_level: int | str | bool = None,
66 | clobber: bool = False,
67 |) -> List[Path]:
68 | """
69 | Generates PNG images from the provided text lines.
70 |
71 | Args:
72 | lines (List[str]): A list of strings containing the
73 | text to generate images from.
74 | output_dir (str | Path, optional): Directory to save
75 | the output images. Defaults to "./output".
76 | font (str | ImageFont.ImageFont, optional): Font to
77 | use for the text. Defaults to "sans-serif".
78 | canvas_size (Tuple[int, int], optional): Size of the
79 | canvas in pixels. Defaults to (1024, 1024).
80 | padding (float, optional): Percentage of canvas size
81 | to use as a blank border. Defaults to 0.10.
82 | background_color (str, optional): Background color
83 | for the images. Defaults to "white".
84 | text_color (str, optional): Color of the text. Defaults
85 | to "black".
86 | log_level (int | str | bool, optional): Logging verbosity
87 | level. Defaults to "WARNING".
88 | clobber (bool, optional): If True, overwrites existing
89 | files. Defaults to False.
90 |
91 | Returns:
92 | List[Path]: A list of paths to the generated PNG images.
93 |
94 | Example:
95 | >>> generator = TextToImageGenerator()

```

```

87 >>> lines = ["Text 1", "Text 2", "Text 3"]
88 >>> output_dir = "./output"
89 >>> images = await generator.generate_images(lines
, output_dir=output_dir)
90 >>> print(images)
91 [PosixPath('./output/Text 1.png'), PosixPath('./ou
tput/Text 2.png'), PosixPath('./output/Text 3.png')]
92 """
93 output_directory = Path(output_dir) if output_dir else
self.default_output_dir
94 self.setup_logging(level=log_level)
95
96 if not canvas_size:
97 canvas_size = self.default_canvas_size
98
99 if not padding:
100 padding = self.default_padding
101
102 generated_images = []
103 for line in lines:
104 img_path = output_directory / f"{line}.png"
105 if img_path.exists() and not clobber:
106 logger.warning(f"File {img_path} already exist
s. Skipping...")
107 continue
108 img = self.generate_png(line, canvas_size, padding
, background_color, text_color, font)
109 img.save(img_path)
110 generated_images.append(img_path)
111
112 return generated_images
113
114 def generate_png(
115 self,
116 text: str,
117 canvas_size: Tuple[int, int],
118 padding: float,
119 background_color: str,

```

```

120 text_color: str,
121 font: str | ImageFont.ImageFont,
122) -> Image:
123 """
124 Creates a PNG image with the specified text, font, and
125 colors.
126
127 Args:
128 text (str): Text to render on the image.
129 canvas_size (Tuple[int, int]): Size of the canvas
130 in pixels.
131 padding (float): Padding percentage to use as a bo
132 rder.
133 background_color (str): Background color of the im
134 age.
135 text_color (str): Color of the text.
136 font (str | ImageFont.ImageFont): Font to use for
137 the text.
138
139 Returns:
140 Image: The generated PNG image.
141 """
142 img = Image.new("RGB", canvas_size, background_color)
143 draw = ImageDraw.Draw(img)
144 font = ImageFont.truetype(font, size=self.get_font_size(
145 canvas_size, padding))
146
147 text_position = self.center_text_position(draw, text,
148 font, canvas_size)
149 draw.text(text_position, text, fill=text_color, font=font)
150
151 return img
152
153 def center_text_position(
154 self, draw: ImageDraw.Draw, text: str, font: ImageFont
155 .ImageFont, canvas_size: Tuple[int, int]
156) -> Tuple[int, int]:

```

```

149 | """
150 | Calculates the position to center the text on the canv
151 | as.
152 |
153 | Args:
154 | draw (ImageDraw.Draw): The ImageDraw instance.
155 | text (str): Text to be rendered.
156 | font (ImageFont.ImageFont): Font used for the text
157 | .
158 | canvas_size (Tuple[int, int]): Size of the canvas
159 | in pixels.
160 |
161 | Returns:
162 | Tuple[int, int]: Coordinates for centering the tex
163 | t.
164 | """
165 | text_width, text_height = draw.textsize(text, font=fon
166 | t)
167 | return (canvas_size[0] - text_width) // 2, (canvas_siz
168 | e[1] - text_height) // 2
169 |
170 | def overlay_images(
171 | self,
172 | background_path: str | Path,
173 | overlay_path: str | Path,
174 | position: tuple[int, int] = (0, 0),
175 | alpha: float = 1.0,
176 |) -> Image:
177 | """Overlays one PNG image on top of another at a speci
178 | fied position.
179 |
180 | Args:
181 | background_path (str | Path): Path to the backgrou
182 | nd PNG image.
183 | overlay_path (str | Path): Path to the overlay PNG
184 | image.
185 | position (tuple[int, int], optional): (x, y) coord
186 | inates where the overlay will be placed. Defaults to (0, 0).

```

```

177 alpha (float, optional): Transparency level of the
 | overlay image (0.0 - 1.0). Defaults to 1.0.
178
179 Returns:
180 Image: The resulting image with the overlay.
181
182 Example:
183 >>> result_image = overlay_images("background.png"
 | , "overlay.png", position=(50, 50), alpha=0.8)
184 >>> result_image.save("result.png")
185 """
186 # Open the background and overlay images
187 background = Image.open(background_path).convert("RGBA
 | ")
188 overlay = Image.open(overlay_path).convert("RGBA")
189
190 # Resize overlay to fit the background if needed
191 if overlay.size != background.size:
192 overlay = overlay.resize(background.size, Image.AN
 | TIALIAS)
193
194 # Adjust transparency of overlay
195 overlay = overlay.copy()
196 overlay.putalpha(int(alpha * 255))
197
198 # Paste overlay onto background
199 background.paste(overlay, position, overlay)
200
201 return background
202
203 def webp2png(webp_path: str, png_path: str) -> bool:
204 """
205 Converts a WEBP image to PNG format.
206
207 Args:
208 webp_path (str): Path to the input WEBP file.
209 png_path (str): Path to save the converted PNG file.
210

```

```
211 | Example:
212 | webp2png('image.webp', 'image.png')
213 | """
214 | try:
215 | # Open the webp image
216 | with Image.open(webp_path) as img:
217 | # Convert to PNG and save
218 | img.save(png_path, 'PNG')
219 | return True
220 | except Exception as e:
221 | print(f"Error during conversion: {e}")
222 | return
223 |
224 |
```

tts.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Speech recognition and text-to-speech conversion utilities
4 # =====
5 # Description:
6 # Module for converting text to speech (TTS) and recognizing speech from audio files.
7 # Supports audio file download, format conversion, and multilingual speech recognition.
8 #
9 # File: tts.py
10 # Project: ai-breadboard
11 # Package: src.utils.convertors
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 from pathlib import Path
17 import tempfile
18 import asyncio
19 import requests
20 import speech_recognition as sr # Speech recognition library
21 from pydub import AudioSegment # Library for audio conversion
22 from gtts import gTTS # Text-to-speech generation
23
24 from src.utils.jjson import j_loads, j_loads_ns, j_dumps
25 from src.logger.logger import logger
26
27 def speech_recognizer(audio_url: str = None, audio_file_path: Path = None, language: str = 'ru-RU') -> str:
28 """Download an audio file and recognize speech in it.
29
```

```

30 Args:
31 audio_url (str, optional): URL of the audio file to be
32 downloaded. Defaults to `None`.
33 audio_file_path (Path, optional): Local path to an aud
34 io file. Defaults to `None`.
35 language (str): Language code for recognition (e.g., '
36 ru-RU'). Defaults to 'ru-RU'.
37
38 Returns:
39 str: Recognized text from the audio or an error messag
40 e.
41
42 Example:
43 .. code::
44
45 recognized_text = speech_recognizer(audio_url='htt
46 ps://example.com/audio.ogg')
47 print(recognized_text) # Output: "Hello"
48
49 """
50 try:
51 if audio_url:
52 # Download the audio file
53 response = requests.get(audio_url)
54 audio_file_path = Path(tempfile.gettempdir()) / 'r
55 ecognized_audio.ogg'
56
57 with open(audio_file_path, 'wb') as f:
58 f.write(response.content)
59
60 # Convert OGG to WAV
61 wav_file_path = audio_file_path.with_suffix('.wav')
62 audio = AudioSegment.from_file(audio_file_path) # Loa
63 d the OGG file
64 audio.export(wav_file_path, format='wav') # Export as
65 WAV
66
67 # Initialize the recognizer
68 recognizer = sr.Recognizer()

```



```

60 with sr.AudioFile(str(wav_file_path)) as source:
61 audio_data = recognizer.record(source)
62 try:
63 # Recognize speech using Google Speech Recogni
tion
64 text = recognizer.recognize_google(audio_data,
language=language)
65 logger.info(f'Recognized text: {text}')
66 return text
67 except sr.UnknownValueError:
68 logger.error('Google Speech Recognition could
not understand audio')
69 return 'Sorry, I could not understand the audi
o.'
70 except sr.RequestError as ex:
71 logger.error('Could not request results from G
oogle Speech Recognition service:', ex)
72 return 'Could not request results from the spe
ech recognition service.'
73 except Exception as ex:
74 logger.error('Error in speech recognizer:', ex)
75 return 'Error during speech recognition.'
76
77 async def text2speech(text: str, lang: str = 'ru') -> str:
78 """Convert text to speech and save it as an audio file.
79
80 Args:
81 text (str): The text to be converted into speech.
82 lang (str, optional): Language code for the speech (e.
g., 'ru'). Defaults to 'ru'.
83
84 Returns:
85 str: Path to the generated audio file.
86
87 Example:
88 .. code::
89
90 audio_path = await text2speech('Hello', lang='en')

```

```

91 | print(audio_path) # Output: "/tmp/response.mp3"
92 | """
93 | try:
94 | # Generate speech using gTTS
95 | tts = gTTS(text=text, lang=lang)
96 | audio_file_path = f'{tempfile.gettempdir()}/response.m
p3'
97 | tts.save(audio_file_path) # Save the audio file
98 |
99 | # Load and export audio using pydub
100 | try:
101 | audio = AudioSegment.from_file(audio_file_path, fo
rmat='mp3')
102 | wav_file_path = audio_file_path.replace('.mp3', '.
wav')
103 | audio.export(wav_file_path, format='wav')
104 | logger.info(f'TTS audio saved at: {wav_file_path}'
)
105 | return wav_file_path
106 | except Exception as pydub_ex:
107 | logger.warning(f'pydub conversion failed (probably
ffmpeg is not installed): {pydub_ex}')
108 | logger.info(f'TTS audio saved as raw MP3 at: {audi
o_file_path}')
109 | return audio_file_path
110 | except Exception as ex:
111 | logger.error('Error in text2speech:', ex)
112 | return 'Error during text-to-speech conversion.'
113 |

```

## unicode.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Unicode escape sequence decoding utilities
4 # =====
5 # Description:
6 # Decodes unicode escape sequences in dictionaries, lists, o
7 # Handles nested structures recursively for comprehensive un
8 # icode processing.
9 #
10 # File: unicode.py
11 # Project: ai-breadboard
12 # Package: src.utils.convertors
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 import re
18 from typing import Dict, Any
19
20 def decode_unicode_escape(input_data: Dict[str, Any] | list |
21 str) -> Dict[str, Any] | list | str:
22 """Decoding of unicode escape sequences in dictionaries, l
23 ists, or strings to readable text.
24
25 Args:
26 input_data (dict | list | str): Input data - dictionar
27 y, list or string, which may contain unicode escape sequences.
28
29 Returns:
30 dict | list | str: Transformed data. For strings escap
31 e sequences are decoded. For dictionaries or lists all values
32 are processed recursively.
```

```

27 |
28 | Usage example:
29 | .. code-block:: python
30 | input_dict = {
31 | 'product_name': r'\u05de\u05e7\" \u05d8 \u05d9\u05e
32 | 6\u05e8\u05df\nH510M K V2',
33 | 'category': r'\u05e2\u05e8\u05db\u05ea \u05e9\u05d
34 | 1\u05d1\u05d9\u05dd',
35 | 'price': 123.45
36 | }
37 |
38 | input_list = [r'\u05e2\u05e8\u05db\u05ea \u05e9\u05d1\
39 | u05d1\u05d9\u05dd', r'H510M K V2']
40 |
41 | input_string = r'\u05de\u05e7\" \u05d8 \u05d9\u05e6\u05
42 | e8\u05df\nH510M K V2'
43 |
44 | # Apply function
45 | decoded_dict = decode_unicode_escape(input_dict)
46 | decoded_list = decode_unicode_escape(input_list)
47 | decoded_string = decode_unicode_escape(input_string)
48 |
49 | print(decoded_dict)
50 | print(decoded_list)
51 | print(decoded_string)
52 |
53 | """
54 |
55 | if isinstance(input_data, dict):
56 | # Recursively process dictionary values
57 | return {key: decode_unicode_escape(value) for key, val
58 | ue in input_data.items()}
59 |
60 | elif isinstance(input_data, list):
61 | # Recursively process list elements
62 | return [decode_unicode_escape(item) for item in input_
63 | data]

```

```
59 | elif isinstance(input_data, str):
60 | # Function decodes string if it contains escape sequences
61 | try:
62 | # Step 1: Decode string with escape sequences
63 | decoded_string = input_data.encode('utf-8').decode(
64 | ('unicode_escape'))
65 | except UnicodeDecodeError:
66 | decoded_string = input_data
67 |
68 | # Step 2: Convert all found \uXXXX sequences
69 | unicode_escape_pattern = r'\\u[0-9a-fA-F]{4}'
70 | decoded_string = re.sub(unicode_escape_pattern, lambda
71 | match: match.group(0).encode('utf-8').decode('unicode_escape'
72 |), decoded_string)
73 |
74 | return decoded_string
75 |
76 | else:
77 | # If data type not supported, function returns data unchanged
78 | return input_data
```

xls.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Convert Excel files to dictionary format
4 # =====
5 # Description:
6 # Provides utilities for converting XLS files to dictionary
7 # format.
8 #
9 # File: xls.py
10 # Project: ai-breadboard
11 # Package: src.utils.convertors
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15 """Excel file to dictionary conversion utilities.
16
17 Provides functions for converting Excel spreadsheet files to d
18 ictionary format
19 for programmatic access and manipulation."""
20
21 from pathlib import Path
22
23 from src.utils.xls import read_xls_as_dict, save_xls_file
24
25 def xls2dict(xls_file: str | Path) -> dict | None:
26 """Convert Excel file to dictionary format.
27
28 Args:
29 xls_file: Path to Excel file to convert.
30
31 Returns:
32 Dictionary representation of Excel file data or None i
```

```
 f conversion fails.
32 | """
33 | return read_xls_as_dict(xls_file = xls_file)
34 |
```

xml2dict.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Convert XML data to dictionaries
4 # =====
5 # Description:
6 # Module for AI Breadboard project.
7 #
8 # File: xml2dict.py
9 # Project: ai-breadboard
10 # Package: src.utils.convertors
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """XML to dictionary conversion utilities.
16
17 Provides utilities for converting XML data into dictionaries.
18 Includes functions for
19 parsing XML strings and converting XML element trees into dict
20 ionary representations."""
21 - `_parse_node`: Parses an XML node into a dictionary.
22 - `_make_dict`: Generates a dictionary with the tag name and v
23 alue.
24 - `xml2dict`: Parses an XML string into a dictionary.
25 - `ET2dict`: Converts an XML element tree into a dictionary.
26 """
27
28 import re
29
30 try:
31 import xml.etree.cElementTree as ET
32 except ImportError as err:
33 import xml.etree.ElementTree as ET
```



```

31
32 def _parse_node(node: ET.Element) -> dict | str:
33 """Parse an XML node into a dictionary.
34
35 Args:
36 node (ET.Element): The XML element to parse.
37
38 Returns:
39 dict | str: A dictionary representation of the XML node, or a string if the node has no attributes or children.
40 """
41 tree = {}
42 attrs = {}
43 for attr_tag, attr_value in node.attrib.items():
44 # Skip href attributes, not supported when converting
45 # to dict
46 if attr_tag == '{http://www.w3.org/1999/xlink}href':
47 continue
48 attrs.update(_make_dict(attr_tag, attr_value))
49
50 value = node.text.strip() if node.text is not None else ''
51
52 if attrs:
53 tree['attrs'] = attrs
54
55 # Save children
56 has_child = False
57 for child in list(node):
58 has_child = True
59 ctag = child.tag
60 ctree = _parse_node(child)
61 cdict = _make_dict(ctag, ctree)
62
63 # No value when there are child elements
64 if ctree:
65 value = ''
66
67 # First time an attribute is found

```

```

67 if ctag not in tree: # First time found
68 tree.update(cdict)
69 continue
70
71 # Many times the same attribute, change to a list
72 old = tree[ctag]
73 if not isinstance(old, list):
74 tree[ctag] = [old] # Change to list
75 tree[ctag].append(ctree) # Add new entry
76
77 if not has_child:
78 tree['value'] = value
79
80 # If there is only a value; no attribute, no child, return
 directly the value
81 if list(tree.keys()) == ['value']:
82 tree = tree['value']
83 return tree
84
85 def _make_dict(tag: str, value: any) -> dict:
86 """Generate a new dictionary with tag and value.
87
88 Args:
89 tag (str): The tag name of the XML element.
90 value (any): The value associated with the tag.
91
92 Returns:
93 dict: A dictionary with the tag name as the key and th
 e value as the dictionary value.
94 """
95 tag_values = value
96 result = re.compile(r"\{(.*)\}(.*)").search(tag)
97 if result:
98 tag_values = {'value': value}
99 tag_values['xmlns'], tag = result.groups() # We have
 a @namespace src!
100 return {tag: tag_values}
101

```

```

102 def xml2dict(xml: str) -> dict:
103 """Parse XML string into a dictionary.
104
105 Args:
106 xml (str): The XML string to parse.
107
108 Returns:
109 dict: The dictionary representation of the XML.
110 """
111 element_tree = ET.fromstring(xml)
112 return ET2dict(element_tree)
113
114 def ET2dict(element_tree: ET.Element) -> dict:
115 """Convert an XML element tree into a dictionary.
116
117 Args:
118 element_tree (ET.Element): The XML element tree.
119
120 Returns:
121 dict: The dictionary representation of the XML element
122 tree.
123 """
124 return _make_dict(element_tree.tag, _parse_node(element_tr
ee))

```

## csv.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: CSV file processing utilities
4 # =====
5 # Description:
6 # Module for AI Breadboard project.
7 #
8 # File: csv.py
9 # Project: ai-breadboard
10 # Package: src.utils
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """CSV and JSON file conversion utilities."""
16
17 import csv
18 import json
19 from pathlib import Path
20 from types import SimpleNamespace
21 from typing import List, Dict, Union
22 import pandas as pd
23 from src.logger.logger import logger
24
25 def save_csv_file(
26 data: List[Dict[str, str]],
27 file_path: Union[str, Path],
28 mode: str = 'a',
29 exc_info: bool = True,
30) -> bool:
31 """Saves a list of dictionaries to a CSV file.
32
33 :param data: List of dictionaries to save.
```

```

34 | :type data: List[Dict[str, str]]
35 | :param file_path: Path to the CSV file.
36 | :type file_path: Union[str, Path]
37 | :param mode: File mode ('a' to append, 'w' to overwrite).
 | Default is 'a'.
38 | :type mode: str
39 | :param exc_info: Include traceback information in logs.
40 | :type exc_info: bool
41 | :raises TypeError: If input data is not a list of dictiona
 | ries.
42 | :raises ValueError: If input data is empty.
43 | :returns: True if successful, otherwise False.
44 | """
45 | if not isinstance(data, list):
46 | raise TypeError("Input data must be a list of dictiona
 | ries.")
47 | if not data:
48 | raise ValueError("Input data cannot be empty.")
49 |
50 | try:
51 | file_path = Path(file_path)
52 | file_path.parent.mkdir(parents=True, exist_ok=True)
53 |
54 | with file_path.open(mode, newline='', encoding='utf-8'
 |) as file:
55 | writer = csv.DictWriter(file, fieldnames=data[0].k
 | eys())
56 | if mode == 'w' or not file_path.exists():
57 | writer.writeheader()
58 | writer.writerows(data)
59 | return True
60 | except Exception as e:
61 | logger.error(f"Failed to save CSV to {file_path}", exc
 | _info=exc_info)
62 | return False
63 |
64 | def read_csv_file(file_path: Union[str, Path], exc_info: bool
 | = True) -> List[Dict[str, str]] | None:

```

```

65 | """Reads CSV content as a list of dictionaries.
66 |
67 | :param file_path: Path to the CSV file.
68 | :type file_path: Union[str, Path]
69 | :param exc_info: Include traceback information in logs.
70 | :type exc_info: bool
71 | :raises FileNotFoundError: If file not found.
72 | :returns: List of dictionaries or None if failed.
73 | """
74 | try:
75 | with Path(file_path).open('r', encoding='utf-8') as fi
76 | reader = csv.DictReader(file)
77 | return list(reader)
78 | except FileNotFoundError as e:
79 | logger.error(f"File not found: {file_path}", exc_info=
80 | return None
81 | except Exception as e:
82 | logger.error(f"Failed to read CSV from {file_path}", e
83 | return None
84 |
85 | def read_csv_as_json(csv_file_path: Union[str, Path], json_fil
86 | """
87 | Convert a CSV file to JSON format and save it.
88 |
89 | :param csv_file_path: Path to the CSV file.
90 | :type csv_file_path: str | Path
91 | :param json_file_path: Path to save the JSON file.
92 | :type json_file_path: str | Path
93 | :param exc_info: Include traceback information in logs. De
94 | :type exc_info: bool, optional
95 | :returns: True if conversion is successful, else False.
96 | :rtype: bool
97 | """

```

```

98 try:
99 data = read_csv_file(csv_file_path, exc_info=exc_info)
100 if data is None:
101 return False
102 with Path(json_file_path).open('w', encoding='utf-8')
as f:
103 json.dump(data, f, indent=4)
104 return True
105 except Exception as ex:
106 logger.error(f"Failed to convert CSV to JSON at {json_
file_path}", exc_info=exc_info)
107 return False
108
109 def read_csv_as_dict(csv_file: Union[str, Path]) -> dict | Non
e:
110 """
111 Convert CSV content to a dictionary.
112
113 :param csv_file: Path to the CSV file.
114 :type csv_file: str | Path
115 :returns: Dictionary representation of CSV content, or Non
e if failed.
116 :rtype: dict | None
117 """
118 try:
119 with Path(csv_file).open('r', encoding='utf-8') as f:
120 reader = csv.DictReader(f)
121 return {"data": [row for row in reader]}
122 except Exception as ex:
123 logger.error("Failed to read CSV as dictionary", exc_i
nfo=True)
124 return None
125
126 def read_csv_as_ns(file_path: Union[str, Path]) -> List[dict]:
127 """Load CSV data into a list of dictionaries using Pandas.
128
129 :param file_path: Path to the CSV file.
130 :type file_path: Union[str, Path]

```

```
131 | :returns: List of dictionaries representing the CSV conten
 | t.
132 | :raises FileNotFoundError: If file not found.
133 | """
134 | try:
135 | df = pd.read_csv(file_path)
136 | return df.to_dict(orient='records')
137 | except FileNotFoundError as e:
138 | logger.error(f"File not found: {file_path}", exc_info=
 | True)
139 | return []
140 | except Exception as e:
141 | logger.error(f"Failed to load CSV as dictionaries from
 | {file_path}", exc_info=True)
142 | return []
```



date\_time.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Time interval checking and timeout utilities
4 # =====
5 # Description:
6 # Provides utilities for time interval checking including fu
7 # nctions to verify if current
8 # time falls within specified interval (useful for scheduled
9 # operations). Supports intervals
10 # spanning midnight and includes input waiting with timeout
11 # functionality.
12 #
13 # File: date_time.py
14 # Project: ai-breadboard
15 # Package: src.utils
16 # Author: hypo69
17 # Copyright: © 2026 hypo69
18 # =====
19
20 from datetime import datetime, time
21 import threading
22
23 class TimeoutCheck:
24 def __init__(self):
25 self.result = None
26
27 def interval(self, start: time = time(23, 0), end: time =
28 time(6, 0)) -> bool:
29 """Check if the current time is within the specified i
30 nterval.
31
32 Args:
33 start (time): Start of the interval (default is 23
```

```

:00).
29 end (time): End of the interval (default is 06:00)
 .
30
31 Returns:
32 bool: True if the current time is within the inter
 val, False otherwise.
33 """
34 current_time = datetime.now().time()
35
36 if start < end:
37 # Interval within the same day (e.g., 08:00 to 17:
 00)
38 self.result = start <= current_time <= end
39 else:
40 # Interval spanning midnight (e.g., 23:00 to 06:00
)
41 self.result = current_time >= start or current_tim
 e <= end
42
43 def interval_with_timeout(self, timeout: int = 5, start: t
 ime = time(23, 0), end: time = time(6, 0)) -> bool:
44 """Check if the current time is within the specified i
 nterval with a timeout.
45
46 Args:
47 timeout (int): Time in seconds to wait for the int
 erval check.
48 start (time): Start of the interval (default is 23
 :00).
49 end (time): End of the interval (default is 06:00)
 .
50
51 Returns:
52 bool: True if the current time is within the inter
 val and response within timeout, False if not or timeout occur
 s.
53 """

```

```

54 | thread = threading.Thread(target=self.interval, args=(
 start, end))
55 | thread.start()
56 | thread.join(timeout)
57 |
58 | if thread.is_alive():
59 | print(f"Timeout occurred after {timeout} seconds,
 continuing execution.")
60 | thread.join() # Ensures thread stops after timeou
 t
61 | return False # Timeout occurred, so returning Fal
 se
62 | return self.result
63 |
64 | def get_input(self):
65 | """Request input from user."""
66 | self.user_input = input("U:> ")
67 |
68 | def input_with_timeout(self, timeout: int = 5) -> str | No
 ne:
69 | """Wait for input with timeout.
70 |
71 | Args:
72 | timeout (int): Input wait time in seconds.
73 |
74 | Returns:
75 | str | None: Entered data or None if timeout occur
 ed.
76 | """
77 | # Start thread to get input from user
78 | thread = threading.Thread(target=self.get_input)
79 | thread.start()
80 |
81 | # Wait for thread completion or timeout
82 | thread.join(timeout)
83 |
84 | if thread.is_alive():
85 | print(f"Timeout occurred after {timeout} seconds."

```

```
)
86 return # Return None if timeout occurred
87
88 return self.user_input
89
90 if __name__ == '__main__':
91 # Example usage
92 timeout_check = TimeoutCheck()
93
94 # Check interval with a timeout of 5 seconds
95 if timeout_check.interval_with_timeout(timeout=5):
96 print("Current time is within the interval.")
97 else:
98 print("Current time is outside the interval or timeout
99 occurred.")
```

file.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: File system operations and text file utilities
4 # =====
5 # Description:
6 # Module for file operations including saving, reading, and
7 # searching text files.
8 # Provides utilities for directory traversal and file pattern matching.
9 #
10 # File: file.py
11 # Project: ai-breadboard
12 # Package: src.utils
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16 """
17 File system operations and text file utilities.
18
19 Provides functions for file manipulation, reading, writing, and searching.
20 """
21
22 import os
23 import json
24 import fnmatch
25 from pathlib import Path
26 from typing import List, Optional, Union, Generator
27 from src.logger.logger import logger
28
29 def save_text_file(
30 data: str | list[str] | dict,
```

```

31 file_path: Union[str, Path],
32 mode: str = "w",
33 exc_info: bool = True,
34) -> bool:
35 """
36 Save data to a text file.
37
38 Args:
39 data (str | list[str] | dict): Data to write (can be s
40 tring, list of strings, or dictionary).
41 file_path (str | Path): Path where the file will be sa
42 ved.
43 mode (str, optional): Write mode (`w` for write, `a` f
44 or append). Defaults to 'w'.
45 exc_info (bool, optional): If True, logs traceback on
46 error. Defaults to True.
47
48 Returns:
49 bool: True if the file was successfully saved, False o
50 therwise.
51 """
52 try:
53 file_path = Path(file_path)
54 file_path.parent.mkdir(parents=True, exist_ok=True)
55
56 with file_path.open(mode, encoding="utf-8") as file:
57 if isinstance(data, list):
58 file.writelines(f"{line}\n" for line in data)
59 elif isinstance(data, dict):
60 json.dump(data, file, ensure_ascii=False, inde
61 nt=4)
62 else:
63 file.write(data)
64 return True
65 except Exception as ex:
66 logger.error(f"Failed to save file {file_path}.", ex,
67 exc_info=exc_info)
68 return False

```

```

62
63 def read_text_file(
64 file_path: Union[str, Path],
65 as_list: bool = False,
66 extensions: Optional[list[str]] = None,
67 exc_info: bool = True,
68) -> Union[str, list[str], None]:
69 """
70 Read the contents of a file.
71
72 Args:
73 file_path (str | Path): Path to the file or directory.
74 as_list (bool, optional): If True, returns content as
75 list of lines. Defaults to False.
76 extensions (list[str], optional): List of file extensi
77 ons to include if reading a directory. Defaults to None.
78 exc_info (bool, optional): If True, logs traceback on
79 error. Defaults to True.
80
81 Returns:
82 str | list[str] | None: File content as a string or li
83 st of lines, or None if an error occurs.
84 """
85 try:
86 path = Path(file_path)
87 if path.is_file():
88 with path.open("r", encoding="utf-8") as f:
89 return f.readlines() if as_list else f.read()
90 elif path.is_dir():
91 files = [
92 p for p in path.rglob("*") if p.is_file() and
93 (not extensions or p.suffix in extensions)
94]
95 contents = [read_text_file(p, as_list) for p in fi
96 les]
97 return [item for sublist in contents if sublist fo
98 r item in sublist] if as_list else "\n".join(filter(None, cont
99 ents))

```

```

92 else:
93 logger.warning(f"Path '{file_path}' is invalid.")
94 return None
95 except Exception as ex:
96 logger.error(f"Failed to read file {file_path}.", ex,
exc_info=exc_info)
97 return None
98
99 def get_filenames(
100 directory: Union[str, Path], extensions: Union[str, list[s
tr]] = "*", exc_info: bool = True
101) -> list[str]:
102 """
103 Get filenames in a directory optionally filtered by extens
ion.
104
105 Args:
106 directory (str | Path): Directory to search.
107 extensions (str | list[str], optional): Extensions to
filter. Defaults to '*'.
108
109 Returns:
110 list[str]: Filenames found in the directory.
111 """
112 try:
113 path = Path(directory)
114 if isinstance(extensions, str):
115 extensions = [extensions] if extensions != "*" els
e []
116 extensions = [ext if ext.startswith(".") else f".{ext}
" for ext in extensions]
117
118 return [
119 file.name
120 for file in path.iterdir()
121 if file.is_file() and (not extensions or file.suff
ix in extensions)
122]

```



```

123 except Exception as ex:
124 logger.warning(f"Failed to list filenames in '{directory}'.", ex, exc_info=exc_info)
125 return []
126
127 def recursively_yield_file_path(
128 root_dir: Union[str, Path], patterns: Union[str, list[str]] = "*", exc_info: bool = True
129) -> Generator[Path, None, None]:
130 """
131 Recursively yield file paths matching given patterns.
132
133 Args:
134 root_dir (str | Path): Root directory to search.
135 patterns (str | list[str]): Patterns to filter files.
136
137 Yields:
138 Path: File paths matching the patterns.
139 """
140 try:
141 patterns = [patterns] if isinstance(patterns, str) else patterns
142 for pattern in patterns:
143 yield from Path(root_dir).rglob(pattern)
144 except Exception as ex:
145 logger.error(f"Failed to search files in '{root_dir}'.", ex, exc_info=exc_info)
146
147 def recursively_get_file_path(
148 root_dir: Union[str, Path],
149 patterns: Union[str, list[str]] = "*",
150 exc_info: bool = True
151) -> list[Path]:
152 """
153 Recursively get file paths matching given patterns.
154
155 Args:
156 root_dir (str | Path): Root directory to search.

```

```

157 | patterns (str | list[str]): Patterns to filter files.
158 |
159 | Returns:
160 | list[Path]: List of file paths matching the patterns.
161 | """
162 | try:
163 | file_paths = []
164 | patterns = [patterns] if isinstance(patterns, str) else patterns
165 | for pattern in patterns:
166 | file_paths.extend(Path(root_dir).rglob(pattern))
167 | return file_paths
168 | except Exception as ex:
169 | logger.error(f"Failed to search files in '{root_dir}'.", ex, exc_info=exc_info)
170 | return []
171 |
172 | def recursively_read_text_files(
173 | root_dir: str | Path,
174 | patterns: str | list[str],
175 | as_list: bool = False,
176 | exc_info: bool = True
177 |) -> list[str]:
178 | """
179 | Recursively reads text files from the specified root directory that match the given patterns.
180 |
181 | Args:
182 | root_dir (str | Path): Path to the root directory for the search.
183 | patterns (str | list[str]): Filename pattern(s) to filter the files.
184 | Can be a single pattern (e.g., '*.txt') or a list of patterns.
185 | as_list (bool, optional): If True, returns the file content as a list of lines.
186 | Defaults to False.
187 | exc_info (bool, optional): If True, includes exception

```

```

 information in warnings. Defaults to True.
188
189 Returns:
190 list[str]: List of file contents (or lines if `as_list
= True`) that match the specified patterns.
191
192 Example:
193 >>> contents = recursively_read_text_files("/path/to/r
oot", ["*.txt", "*.md"], as_list=True)
194 >>> for line in contents:
195 ... print(line)
196 This will print each line from all matched text files
in the specified directory.
197 """
198 matches = []
199 root_path = Path(root_dir)
200
201 # Check if the root directory exists
202 if not root_path.is_dir():
203 logger.debug(f"The root directory '{root_path}' does n
ot exist or is not a directory.")
204 return []
205
206 print(f"Searching in directory: {root_path}")
207
208 # Normalize patterns to a list if it's a single string
209 if isinstance(patterns, str):
210 patterns = [patterns]
211
212 for root, dirs, files in os.walk(root_path):
213 for filename in files:
214 # Check if the filename matches any of the specifi
ed patterns
215 if any(fnmatch.fnmatch(filename, pattern) for patt
ern in patterns):
216 file_path = Path(root) / filename
217
218 try:

```

```

219 with file_path.open("r", encoding="utf-8")
 | as file:
220 if as_list:
221 # Read lines if `as_list=True`
222 matches.extend(file.readlines())
223 else:
224 # Read entire content otherwise
225 matches.append(file.read())
226 except Exception as ex:
227 logger.warning(f"Failed to read file '{file_path}'.", exc_info=exc_info)
228
229 return matches
230
231 def get_directory_names(directory: str | Path, exc_info: bool
 | = True) -> list[str]:
232 """
233 Retrieves all directory names from the specified directory
234 .
235 Args:
236 directory (str | Path): Path to the directory from which
 | directory names should be retrieved.
237 exc_info (bool, optional): If True, logs traceback information
 | in case of an error. Defaults to True.
238
239 Returns:
240 list[str]: List of directory names found in the specified
 | directory.
241
242 Example:
243 >>> directories: list[str] = get_directory_names(directory=".")
244 >>> print(directories)
245 ['dir1', 'dir2']
246
247 More documentation: https://github.com/hypo69/tiny-utils/wiki/Files-and-Directories#get_directory_names

```

```

248 """
249 try:
250 return [entry.name for entry in Path(directory).iterdi
r() if entry.is_dir()]
251 except Exception as ex:
252 if exc_info:
253 logger.warning(
254 f"Failed to get directory names from '{directo
ry}'.",
255 ex,
256 exc_info=exc_info,
257)
258 return
259
260 def read_files_content(
261 root_dir: Union[str, Path],
262 patterns: Union[str, list[str]],
263 as_list: bool = False,
264 exc_info: bool = True,
265) -> list[str]:
266 """
267 Read contents of files matching patterns.
268
269 Args:
270 root_dir (str | Path): Root directory to search.
271 patterns (str | list[str]): File patterns to match.
272 as_list (bool, optional): Return content as list of li
nes. Defaults to False.
273
274 Returns:
275 list[str]: List of file contents.
276 """
277 content = []
278 for file_path in recursively_get_files(root_dir, patterns,
exc_info):
279 file_content = read_text_file(file_path, as_list, exc_
info=exc_info)
280 if file_content:

```

```

281 content.extend(file_content if as_list else [file_
content])
282 return content
283
284 def remove_bom(file_path: Union[str, Path]) -> None:
285 """
286 Remove BOM from a text file.
287
288 Args:
289 file_path (str | Path): File to process.
290 """
291 path = Path(file_path)
292 try:
293 with path.open("r+", encoding="utf-8") as file:
294 content = file.read().replace("\ufeff", "")
295 file.seek(0)
296 file.write(content)
297 file.truncate()
298 except Exception as ex:
299 logger.error(f"Failed to remove BOM from '{file_path}'
.", ex)
300
301 def traverse_and_clean(directory: Union[str, Path]) -> None:
302 """
303 Traverse directory and remove BOM from Python files.
304
305 Args:
306 directory (str | Path): Root directory to process.
307 """
308 for file in recursively_get_files(directory, "*.py"):
309 remove_bom(file)
310
311 def main() -> None:
312 """Entry point for BOM removal in Python files."""
313 root_dir = Path("../", "src")
314 logger.info(f"Starting BOM removal in {root_dir}")
315 traverse_and_clean(root_dir)
316

```

```
317 | if __name__ == "__main__":
318 | main()
319 |
```

ftp.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: FTP file transfer utilities
4 # =====
5 # Description:
6 # Module for AI Breadboard project.
7 #
8 # File: ftp.py
9 # Project: ai-breadboard
10 # Package: src.utils
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """FTP interface for sending, receiving, and deleting files fr
 om FTP servers.
16
17 Allows for sending media files, spreadsheets, and other files
 to and from an FTP server."""
18 - typing: Type hints for function parameters and return values
19 .
20 - ftplib: Provides FTP protocol client capabilities.
21 - pathlib: For handling file system paths.
22
23 Functions:
24 - `write`: Sends a file to an FTP server.
25 - `read`: Retrieves a file from an FTP server.
26 - `delete`: Deletes a file from an FTP server.
27
28 """
29
30 from src.logger.logger import logger
31 from typing import Union
32 import ftplib
```



```

31 from pathlib import Path
32
33 # Connection configuration (assumed to be defined elsewhere)
34 _connection = {
35 'server': 'ftp.example.com',
36 'port': 21,
37 'user': 'username',
38 'password': 'password'
39 }
40
41 def write(source_file_path: str, dest_dir: str, dest_file_name
42 : str) -> bool:
43 """
44 Sends a file to an FTP server.
45
46 Args:
47 source_file_path (str): The path of the file to be sen
48 t.
49 dest_dir (str): The destination directory on the FTP s
50 erver.
51 dest_file_name (str): The name of the file on the FTP
52 server.
53
54 Returns:
55 bool: True if the file is successfully sent, False oth
56 erwise.
57
58 Example:
59 >>> success = write('local_path/to/file.txt', '/remote
60 /directory', 'file.txt')
61 >>> print(success)
62 True
63 """
64 try:
65 # Establish connection to FTP server
66 session = ftplib.FTP(
67 _connection['server'],
68 _connection['user'],

```

```

63 _connection['password'])
64 session.cwd(dest_dir)
65 except Exception as ex:
66 # Log error if connection to FTP server fails
67 logger.error(f"Failed to connect to FTP server. Error:
 {ex}")
68 return False
69
70 try:
71 # Open the file and send it to the FTP server
72 with open(source_file_path, 'rb') as f:
73 session.storbinary(f'STOR {dest_file_name}', f)
74 return True
75 except Exception as ex:
76 # Log error if file transfer to FTP server fails
77 logger.error(f"Failed to send file to FTP server. Error:
 {ex}")
78 return False
79 finally:
80 try:
81 # Close the FTP session
82 session.quit()
83 except Exception as ex:
84 logger.error(f"Failed to close FTP session. Error:
 {ex}")
85
86 def read(source_file_path: str, dest_dir: str, dest_file_name:
 str) -> Union[str, bytes, None]:
87 """
88 Retrieves a file from an FTP server.
89
90 Args:
91 source_file_path (str): The path where the file will be
 saved locally.
92 dest_dir (str): The directory on the FTP server where
 the file is located.
93 dest_file_name (str): The name of the file on the FTP
 server.

```

```

94 |
95 | Returns:
96 | Union[str, bytes, None]: The file content if successfully retrieved, None otherwise.
97 |
98 | Example:
99 | >>> content = read('local_path/to/file.txt', '/remote/directory', 'file.txt')
100 | >>> print(content)
101 | b'Some file content'
102 | """
103 | try:
104 | # Establish connection to FTP server
105 | session = ftplib.FTP(
106 | _connection['server'],
107 | _connection['user'],
108 | _connection['password'])
109 | session.cwd(dest_dir)
110 |
111 | # Retrieve the file
112 | with open(source_file_path, 'wb') as f:
113 | session.retrbinary(f'RETR {dest_file_name}', f.write)
114 | with open(source_file_path, 'rb') as f:
115 | return f.read()
116 | except Exception as ex:
117 | # Log error if file retrieval from FTP server fails
118 | logger.error(f"Failed to retrieve file from FTP server . Error: {ex}")
119 | return
120 | finally:
121 | try:
122 | # Close the FTP session
123 | session.quit()
124 | except Exception as ex:
125 | logger.error(f"Failed to close FTP session. Error: {ex}")
126 |

```

```

127 def delete(source_file_path: str, dest_dir: str, dest_file_name: str) -> bool:
128 """
129 Deletes a file from an FTP server.
130
131 Args:
132 source_file_path (str): The path where the file is located locally (not used).
133 dest_dir (str): The directory on the FTP server where the file is located.
134 dest_file_name (str): The name of the file on the FTP server.
135
136 Returns:
137 bool: True if the file is successfully deleted, False otherwise.
138
139 Example:
140 >>> success = delete('local_path/to/file.txt', '/remote/directory', 'file.txt')
141 >>> print(success)
142 True
143 """
144 try:
145 # Establish connection to FTP server
146 session = ftplib.FTP(
147 _connection['server'],
148 _connection['user'],
149 _connection['password'])
150 session.cwd(dest_dir)
151
152 # Delete the file
153 session.delete(dest_file_name)
154 return True
155 except Exception as ex:
156 # Log error if file deletion from FTP server fails
157 logger.error(f"Failed to delete file from FTP server. Error: {ex}")

```

```
158 | return False
159 | finally:
160 | try:
161 | # Close the FTP session
162 | session.quit()
163 | except Exception as ex:
164 | logger.error(f"Failed to close FTP session. Error:
165 | {ex}")
```

## get\_free\_port.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Check port availability on host
4 # =====
5 # Description:
6 # Search and allocate free TCP port in specified range for launching services.
7 #
8 # File: get_free_port.py
9 # Project: ai-breadboard
10 # Package: src.utils
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import socket
16 from typing import List, Tuple, Union
17
18 from src.logger import logger
19
20 def _is_port_in_use(host: str, port: int) -> bool:
21 """Check port availability on host."""
22 target_host = "127.0.0.1" if host in ("localhost", "") else host
23 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
24 try:
25 sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
26 sock.bind((target_host, port))
27 return False # Port is free
28 except OSError:
29 return True # Port is in use
```

```

30
31 def _parse_port_range(port_range_str: str) -> Tuple[int, int]:
32 """Parse port range string 'min-max'."""
33 try:
34 parts = port_range_str.split('-')
35 if len(parts) != 2 or not parts[0] or not parts[1]:
36 raise ValueError(f'Incorrect range format: {port_r
37 |ange_str}')
38
39 min_port = int(parts[0])
40 max_port = int(parts[1])
41
42 if min_port >= max_port:
43 raise ValueError(f'Incorrect range: {port_range_st
44 |r}')
45 return min_port, max_port
46
47 except ValueError as e:
48 logger.error(f'Error parsing range: {port_range_str}')
49 raise ValueError(f'Error parsing range: {port_range_st
50 |r}') from e
51
52 def get_free_port(host: str, port_range: Union[str, List[str]]
53 | = '') -> int:
54 """
55 Search and allocate free TCP port.
56
57 Search for free port in specified range or first available
58 if range not specified.
59
60 Args:
61 host (str): Host address for checking port availabilit
62 |y.
63 port_range (Union[str, List[str]]): Port range(s) ("mi
64 |n-max" or list of strings).
65 Default value: '' (search first available).
66
67 Returns:

```

```

61 | int: Number of free port.
62 |
63 | Exceptions:
64 | ValueError: Error if free port not found or range spec
 ified incorrectly.
65 |
66 | Examples:
67 | >>> port = get_free_port(host='localhost', port_range=
 '8000-8005')
68 | """
69 | if port_range:
70 | if isinstance(port_range, str):
71 | min_port, max_port = _parse_port_range(port_range)
72 | for port in range(min_port, max_port + 1):
73 | if not _is_port_in_use(host, port):
74 | return port
75 | raise ValueError(f'Free port in range {port_range}
 not found')
76 |
77 | elif isinstance(port_range, list):
78 | for item in port_range:
79 | if not isinstance(item, str):
80 | continue
81 | try:
82 | min_port, max_port = _parse_port_range(ite
 m)
83 | for port in range(min_port, max_port + 1):
84 | if not _is_port_in_use(host, port):
85 | return port
86 | except ValueError:
87 | continue # Skip incorrect ranges
88 |
89 | raise ValueError(f'Free port in ranges {port_range
 } not found')
90 | else:
91 | raise ValueError(f'Incorrect range type: {type(port
 _range)}')
92 | else:

```



```
93 | # Search first available port starting from 1024
94 | port = 1024
95 | while port <= 65535:
96 | if not _is_port_in_use(host, port):
97 | return port
98 | port += 1
99 | raise ValueError('Free port not found')
100 |
```

## header.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Project root directory resolver
4 # =====
5 # Description:
6 # Module for finding and setting the project root path. All
7 # imports built
8 # relative to this root directory. Uses marker files (.git,
9 # __root__) to identify project root.
10 #
11 # File: header.py
12 # Project: ai-breadboard
13 # Package: src.utils
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # =====
17
18 import sys
19
20 from pathlib import Path
21
22 def set_project_root(marker_files=('__root__', '.git')) -> Path
23 :
24 """
25 Finds the root directory of the project starting from the
26 current file's directory,
27 searching upwards and stopping at the first directory containing any of the marker files.
28
29 Args:
30 marker_files (tuple): Filenames or directory names to
31 identify the project root.
32
33 Returns:
```

```
28 | Path: Path to the root directory if found, otherwise t
 | he directory where the script is located.
29 | """
30 | __root__:Path
31 | current_path:Path = Path(__file__).resolve().parent
32 | __root__ = current_path
33 | for parent in [current_path] + list(current_path.parents):
34 | if any((parent / marker).exists() for marker in marker
 | _files):
35 | __root__ = parent
36 | break
37 | if __root__ not in sys.path:
38 | sys.path.insert(0, str(__root__))
39 | return __root__
40 |
41 | # Get the root directory of the project
42 | __root__: Path = set_project_root()
43 | """__root__ (Path): Path to the root directory of the project"
 | ""
44 |
```

image.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Image processing utilities for asynchronous fi
 le operations
4 # =====
5 # Description:
6 # Provides asynchronous functions for downloading, saving, a
 nd manipulating images.
7 # Supports operations such as URL image downloads, image fil
 e saving, random image retrieval,
8 # watermark addition, resizing, and format conversion using
 PIL and aiohttp libraries.
9 #
10 # File: image.py
11 # Project: ai-breadboard
12 # Package: src.utils
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 import aiohttp
18 import aiofiles
19 import asyncio
20 import random
21 from pathlib import Path
22 from typing import Optional, Union, Tuple
23 from io import BytesIO
24
25 from PIL import Image, ImageDraw, ImageFont
26
27 from src.logger.logger import logger
28
29 class ImageError(Exception):
```

```

30 """Custom exception for image-related errors."""
31 pass
32
33 async def save_image_from_url(image_url: str, filename: Union[
34 str, Path]) -> Optional[str]:
35 """
36 Asynchronous download of image from URL and save to local
37 file.
38
39 Args:
40 image_url (str): The URL to download the image from.
41 filename (Union[str, Path]): The name of the file to s
42 ave the image to.
43
44 Returns:
45 Optional[str]: The path to the saved file, or None if
46 the operation failed.
47
48 Exceptions:
49 ImageError: If the image download or save operation fa
50 ils.
51 """
52 try:
53 async with aiohttp.ClientSession() as session:
54 async with session.get(image_url) as response:
55 response.raise_for_status() # Raise HTTPError
56 for bad responses (4xx or 5xx)
57 image_data = await response.read()
58 except Exception as ex:
59 logger.error(f"Error downloading image from {image_url
60 }", exc_info=True)
61 raise ImageError(f"Failed to download image from {imag
62 e_url}") from ex
63
64 return await save_image(image_data, filename)
65
66 async def save_image(image_data: bytes, file_name: Union[str,
67 Path], format: str = 'PNG') -> Optional[str]:

```

```

59 | """
60 | Asynchronous saving of image data to file in specified for
 | mat.
61 |
62 | Args:
63 | image_data (bytes): The binary image data.
64 | file_name (Union[str, Path]): The name of the file to
 | save the image to.
65 | format (str): The format to save the image in, default
 | is PNG.
66 |
67 | Returns:
68 | Optional[str]: The path to the saved file, or None if
 | the operation failed.
69 |
70 | Exceptions:
71 | ImageError: If the file cannot be created, saved, or i
 | f the saved file is empty.
72 | """
73 | file_path = Path(file_name)
74 |
75 | try:
76 | file_path.parent.mkdir(parents=True, exist_ok=True) #
 | Creation of parent directories if they don't exist
77 | async with aiofiles.open(file_path, "wb") as file:
78 | await file.write(image_data)
79 |
80 | if not file_path.exists():
81 | logger.error(f"File {file_path} was not created.")
82 | raise ImageError(f"File {file_path} was not create
 | d.")
83 |
84 | img = Image.open(file_path)
85 | img.save(file_path, format=format)
86 |
87 | if file_path.stat().st_size == 0:
88 | logger.error(f"File {file_path} saved, but its siz
 | e is 0 bytes.")

```

```

89 raise ImageError(f"File {file_path} saved, but its
 size is 0 bytes.")
90
91 return str(file_path)
92
93 except Exception as ex:
94 logger.critical(f"Failed to save file {file_path}", ex
 c_info=True)
95 raise ImageError(f"Failed to save file {file_path}") f
 rom ex
96
97 def get_image_bytes(image_path: Path, raw: bool = True) -> Opt
 ional[Union[BytesIO, bytes]]:
98 """
99 Reads an image using Pillow and returns its bytes in JPEG
 format.
100
101 Args:
102 image_path (Path): The path to the image file.
103 raw (bool): If True, returns a BytesIO object; otherwi
 se, returns bytes. Defaults to True.
104
105 Returns:
106 Optional[Union[BytesIO, bytes]]: The bytes of the imag
 e in JPEG format, or None if an error occurs.
107 """
108 try:
109 img = Image.open(image_path)
110 img_byte_arr = BytesIO()
111 img.save(img_byte_arr, format="JPEG")
112 return img_byte_arr if raw else img_byte_arr.getvalue(
)
113 except Exception as ex:
114 logger.error("Error reading image with Pillow:", exc_i
 nfo=True)
115 return None
116
117 def get_raw_image_data(file_name: Union[str, Path]) -> Optiona

```

```

118 | l[bytes]:
119 | """
120 | Retrieves the raw binary data of a file if it exists.
121 | Args:
122 | file_name (Union[str, Path]): The name or path of the
123 | file to read.
124 | Returns:
125 | Optional[bytes]: The binary data of the file, or None
126 | if the file does not exist or an error occurs.
127 | """
128 | file_path = Path(file_name)
129 | if not file_path.exists():
130 | logger.error(f"File {file_path} does not exist.")
131 | return None
132 |
133 | try:
134 | return file_path.read_bytes()
135 | except Exception as ex:
136 | logger.error(f"Error reading file {file_path}", exc_in
137 | fo=True)
138 | return None
139 |
140 | def random_image(root_path: Union[str, Path]) -> Optional[str]
141 | :
142 | """
143 | Recursively searches for a random image in the specified d
144 | irectory.
145 | Args:
146 | root_path (Union[str, Path]): The directory to search
147 | for images.
148 | Returns:
149 | Optional[str]: The path to a random image, or None if
150 | no images are found.

```



```

148 """
149 root_path = Path(root_path)
150 image_extensions = ['.jpg', '.jpeg', '.png', '.gif', '.bmp
 ']
151 image_files = [file_path for file_path in root_path.rglob(
 "**")
152 if file_path.is_file() and file_path.suffix
 .lower() in image_extensions]
153
154 if not image_files:
155 logger.warning(f"No images found in {root_path}.")
156 return None
157
158 return str(random.choice(image_files))
159
160 def add_text_watermark(image_path: Union[str, Path], watermark
 _text: str, output_path: Optional[Union[str, Path]] = None) ->
 Optional[str]:
161 """
162 Adds a text watermark to an image.
163
164 Args:
165 image_path (Union[str, Path]): Path to the image file.
166 watermark_text (str): Text to use as the watermark.
167 output_path (Optional[Union[str, Path]]): Path to save
 the watermarked image.
168 Defaults to overwriting the original image.
169
170 Returns:
171 Optional[str]: Path to the watermarked image, or None
 on failure.
172 """
173 image_path = Path(image_path)
174 output_path = image_path if output_path is None else Path(
 output_path)
175
176 try:
177 image = Image.open(image_path).convert("RGBA")

```

```

178
179 # Create a transparent layer for the watermark
180 watermark_layer = Image.new('RGBA', image.size, (255,
255, 255, 0))
181 draw = ImageDraw.Draw(watermark_layer)
182
183 font_size = min(image.size) // 10 # Adjust the font s
ize based on the image
184 try:
185 font = ImageFont.truetype("arial.ttf", size=font_s
ize)
186 except IOError:
187 font = ImageFont.load_default(size=font_size)
188 logger.warning("Font arial.ttf not found; using de
fault font.")
189
190 text_width, text_height = draw.textsize(watermark_text
, font=font)
191 x = (image.width - text_width) // 2
192 y = (image.height - text_height) // 2
193
194 # Draw text on the transparent layer
195 draw.text((x, y), watermark_text, font=font, fill=(255
, 255, 255, 128))
196
197 # Combine the image and watermark
198 watermarked_image = Image.alpha_composite(image, water
mark_layer)
199 watermarked_image.save(output_path)
200
201 return str(output_path)
202
203 except Exception as ex:
204 logger.error(f"Failed to add watermark to {image_path}
", exc_info=True)
205 return None
206
207 def add_image_watermark(input_image_path: Path, watermark_imag

```

```

 e_path: Path, output_image_path: Optional[Path] = None) -> Opt
 ional[Path]:
208 """
209 Adds a watermark to an image and saves the result to the s
 pecified output path.
210
211 Args:
212 input_image_path (Path): Path to the input image.
213 watermark_image_path (Path): Path to the watermark ima
 ge.
214 output_image_path (Optional[Path]): Path to save the w
 atermarked image.
215 If not provided, the image will be saved in an "ou
 tput" directory.
216
217 Returns:
218 Optional[Path]: Path to the saved watermarked image, o
 r None if the operation failed.
219 """
220 try:
221 # Open the base image
222 base_image = Image.open(input_image_path)
223
224 # Open the watermark image and convert it to RGBA
225 watermark = Image.open(watermark_image_path).convert("
 RGBA")
226
227 # Set the size of the watermark (8% of the width of th
 e base image)
228 position = base_image.size # Size of the base image (
 width, height)
229 newsize = (int(position[0] * 8 / 100), int(position[0]
 * 8 / 100)) # New size of the watermark
230 watermark = watermark.resize(newsize) # Resize the wa
 termark
231
232 # Determine the position to place the watermark (botto
 m-right corner with 20px padding)

```

```
233 new_position = position[0] - newsize[0] - 20, position
 | [1] - newsize[1] - 20
234
235 # Create a new transparent layer for combining the ima
 | ges
236 transparent = Image.new(mode='RGBA', size=position, co
 | lor=(0, 0, 0, 0))
237
238 # Paste the base image onto the new layer
239 transparent.paste(base_image, (0, 0))
240
241 # Paste the watermark on top of the base image
242 transparent.paste(watermark, new_position, watermark)
243
244 # Check the image mode and convert the transparent lay
 | er to the original mode
245 image_mode = base_image.mode
246 if image_mode == 'RGB':
247 transparent = transparent.convert(image_mode) # C
 | onvert to RGB
248 else:
249 transparent = transparent.convert('P') # Convert
 | to palette
250
251 # Save the final_verdict image to the specified output
 | path with optimized quality
252 if output_image_path is None:
253 output_image_path = input_image_path.parent / "out
 | put" / input_image_path.name
254 output_image_path.parent.mkdir(parents=True, exist_ok=
 | True) # Create output directory if it doesn't exist
255 transparent.save(output_image_path, optimize=True, qua
 | lity=100)
256 logger.info(f"Saving {output_image_path}...")
257
258 return output_image_path
259
260 except Exception as ex:
```

```

261 logger.error(f"Failed to add watermark to {input_image
 _path}: {ex}", exc_info=True)
262 return None
263
264 def resize_image(image_path: Union[str, Path], size: Tuple[int
 , int], output_path: Optional[Union[str, Path]] = None) -> Opt
 ional[str]:
265 """
266 Resizes an image to the specified dimensions.
267
268 Args:
269 image_path (Union[str, Path]): Path to the image file.
270 size (Tuple[int, int]): A tuple containing the desired
 width and height of the image.
271 output_path (Optional[Union[str, Path]]): Path to save
 the resized image.
272 Defaults to overwriting the original image.
273
274 Returns:
275 Optional[str]: Path to the resized image, or None on f
 ailure.
276 """
277 image_path = Path(image_path)
278 output_path = image_path if output_path is None else Path(
 output_path)
279
280 try:
281 img = Image.open(image_path)
282 resized_img = img.resize(size)
283 resized_img.save(output_path)
284 return str(output_path)
285
286 except Exception as ex:
287 logger.error(f"Failed to resize image {image_path}", e
 xc_info=True)
288 return None
289
290 def convert_image(image_path: Union[str, Path], format: str, o

```

```

 output_path: Optional[Union[str, Path]] = None) -> Optional[str
]:
291 """
292 Converts an image to the specified format.
293
294 Args:
295 image_path (Union[str, Path]): Path to the image file.
296 format (str): Format to convert image to (e.g., "JPEG"
 , "PNG").
297 output_path (Optional[Union[str, Path]]): Path to save
 the converted image.
298 Defaults to overwriting the original image.
299
300 Returns:
301 Optional[str]: Path to the converted image or None on
 failure.
302 """
303 image_path = Path(image_path)
304 output_path = image_path if output_path is None else Path(
 output_path)
305
306 try:
307 img = Image.open(image_path)
308 img.save(output_path, format=format)
309 return str(output_path)
310 except Exception as ex:
311 logger.error(f"Failed to convert image {image_path}",
 exc_info=True)
312 return None
313
314 def process_images_with_watermark(folder_path: Path, watermark
 _path: Path) -> None:
315 """
316 Processes all images in the specified folder by adding a w
 atermark and saving them in an "output" directory.
317
318 Args:
319 folder_path (Path): Path to the folder containing imag

```

```

 es.
320 watermark_path (Path): Path to the watermark image.
321 """
322 if not folder_path.is_dir():
323 logger.error(f"Folder {folder_path} does not exist.")
324 return
325
326 # Create an "output" directory if it doesn't exist
327 output_dir = folder_path / "output"
328 output_dir.mkdir(parents=True, exist_ok=True)
329
330 # Process each file in the folder
331 for file_path in folder_path.iterdir():
332 if file_path.is_file() and file_path.suffix.lower() in
333 ['.png', '.jpg', '.jpeg']:
334 output_image_path = output_dir / file_path.name
335 add_image_watermark(file_path, watermark_path, out
336 put_image_path)
337
338 # Example usage
339 if __name__ == "__main__":
340 folder = Path(input("Enter Folder Path: ")) # Path to the
341 folder containing images
342 watermark = Path(input("Enter Watermark Path: ")) # Path
343 to the watermark image
344
345 process_images_with_watermark(folder, watermark)

```

## jjson.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: JSON and CSV file handling utilities
4 # =====
5 # Description:
6 # Provides comprehensive functions for handling JSON and CSV
7 # files including loading,
8 # dumping, merging, and conversion operations. Includes SimpleNamespace conversion,
9 # Markdown parsing, data validation, and repair of malformed
10 # JSON structures.
11 #
12 # File: jjson.py
13 # Project: ai-breadboard
14 # Package: src.utils
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # =====
18
19 from datetime import datetime
20 import copy
21 from math import log
22 from pathlib import Path
23 from typing import List, Dict, Optional, Any
24 from types import SimpleNamespace
25 import json
26 import os
27 import re
28 import pandas as pd
29 from json_repair import repair_json
30 import simplejson as simplejson
31 from typing import Any
32 from pathlib import Path
```



```

31 import json
32 import pandas as pd
33 from types import SimpleNamespace
34 from collections import OrderedDict
35
36 from src.logger.logger import logger
37 from .convertors.dict import dict2ns
38
39 def j_dumps(
40 data: Dict | SimpleNamespace | List[Dict] | List[SimpleNam
41 space],
42 file_path: Optional[Path] = None,
43 ensure_ascii: bool = True,
44 mode: str = "w",
45 exc_info: bool = True,
46) -> Optional[Dict]:
47 """Dumping JSON data to file or returning JSON data as dic
48 tionary.
49
50 Args:
51 data (Dict | SimpleNamespace | List[Dict] | List[Simpl
52 eNamespace]): JSON-compatible data or SimpleNamespace objects
53 to dump.
54 file_path (Optional[Path], optional): Path to the outp
55 ut file. If None, returns JSON as a dictionary. Defaults to No
56 ne.
57 ensure_ascii (bool, optional): If True, escapes non-AS
58 CII characters in output. Defaults to True.
59 mode (str, optional): File open mode ('w', 'a+', '+a')
60 . Defaults to 'w'.
61 exc_info (bool, optional): If True, logs exceptions wi
62 th traceback. Defaults to True.
63
64 Returns:
65 Optional[Dict]: JSON data as a dictionary if successfu
66 l, or nothing if an error occurs.
67
68 Exceptions:

```

```

59 | ValueError: If the file mode is unsupported.
60 | """
61 |
62 | path = Path(file_path) if isinstance(file_path, (str, Path
63 |)) else None
64 |
65 | # If data comes as string - code will attempt to parse it
66 | via `repair_json()`
67 | if isinstance(data, str):
68 | try:
69 | data = repair_json(data)
70 | except Exception as ex:
71 | logger.error(f'Error converting string: {pprint(da
72 | ta)}', ex, False)
73 | ...
74 | return
75 |
76 | def _convert(value: Any) -> Any:
77 | """
78 | Recursively process values to handle nested SimpleName
79 | space, dict, or list.
80 |
81 | Args:
82 | value (Any): Value to process.
83 |
84 | Returns:
85 | Any: Converted value.
86 | """
87 | if isinstance(value, SimpleNamespace):
88 | return {key: _convert(val) for key, val in vars(va
89 | lue).items()}
90 | elif isinstance(value, dict):
91 | return {key: _convert(val) for key, val in value.i
92 | tems()}
93 | elif isinstance(value, list):
94 | return [_convert(item) for item in value]
95 | return value

```

```

91 | # Convert input data to valid dictionary `dict`
92 | data = _convert(data)
93 |
94 | # If incorrect file write mode specified - 'w' will be set
95 | if mode not in {"w", "a+", "+a"}:
96 | mode = 'w'
97 |
98 | # Read existing data from file (if file exists and mode is
99 | 'a+' or '+a')
100 | existing_data = {}
101 | if path and path.exists() and mode in {"a+", "+a"}:
102 | try:
103 | with path.open("r", encoding="utf-8") as f: # Rea
104 | d in 'r' mode
105 | existing_data = json.load(f)
106 | except json.JSONDecodeError as e:
107 | logger.error(f"Error decoding existing JSON in {pa
108 | th}: {e}", exc_info=exc_info)
109 | ...
110 | return
111 | except Exception as ex:
112 | logger.error(f"Error reading {path=}: {ex}", exc_i
113 | nfo=exc_info)
114 | ...
115 | return
116 |
117 | # Process data depending on mode
118 | if mode == "a+":
119 | # Append new data to beginning of existing dictionary
120 | try:
121 | if isinstance(data, list) and isinstance(existing_
122 | data, list):
123 | existing_data = data + existing_data # Add li
124 | st elements to beginning
125 | else:
126 | data.update(existing_data)
127 | except Exception as ex:
128 | logger.error(ex)

```

```

123 | ...
124 |
125 | elif mode == "+a":
126 | # Append new data to end of existing dictionary
127 | try:
128 | if isinstance(existing_data, list):
129 | existing_data.extend(data) # Add list element
130 | except Exception as ex:
131 | logger.error(ex)
132 | ...
133 |
134 | # Mode 'w' - overwrites file with new data
135 | if path:
136 | try:
137 | path.parent.mkdir(parents=True, exist_ok=True)
138 | with path.open("w", encoding="utf-8") as f:
139 | json.dump(data, f, ensure_ascii=ensure_ascii,
140 | indent=4)
141 | except Exception as ex:
142 | logger.error(f"Failed to write to {path}: ", ex, exc_info=exc_info)
143 | ...
144 | return
145 | else:
146 | return data
147 |
148 | return data
149 |
150 |
151 |
152 | def j_loads(
153 | jjson: dict | SimpleNamespace | str | Path | list,
154 | ordered: bool = True
155 |) -> dict | list:
156 | """
157 | Load JSON or CSV data from file, directory, string, JSON o

```

```

 bject or SimpleNamespace.
158 Recodes string keys and values to Unicode.
159
160 Args:
161 jjson (dict | SimpleNamespace | str | Path | list): Pa
 th to file, directory, JSON data string,
162 JSO
 N object or SimpleNamespace.
163 ordered (bool, optional): Returns OrderedDict to prese
 rve element order. Defaults to True.
164
165 Returns:
166 dict | list: Processed data (dictionary or list of dic
 tionaries).
167
168 Raises:
169 FileNotFoundError: If specified file not found.
170 json.JSONDecodeError: If JSON data could not be parsed
 .
171 """
172
173 def decode_strings(data: Any) -> Any:
174 """Recursively recodes strings in data structure."""
175 if isinstance(data, str):
176 try:
177 return data.encode().decode('unicode_escape')
178 # Decode escape sequences
179 except Exception:
180 return data # If decoding failed, return as i
 s
181 elif isinstance(data, list):
182 return [decode_strings(item) for item in data] #
 Process each list element
183 elif isinstance(data, dict):
184 return {decode_strings(key): decode_strings(value)
 for key, value in data.items()} # Process keys and values
185
186 # Decoding escape \u0412\u044b\u0441\u043e\u043a\u043e

```

```

186 decoded_data = json.loads(json.dumps(data))
187 return data # Return unchanged values if not string,
|list or dictionary
188
189 def string2dict(json_string: str) -> dict:
190 """Remove triple backticks and 'json' from beginning a
|nd end of string."""
191 if json_string.startswith(('```', '```json')) and json_
|_string.endswith(('```', '```\\n')):
192 json_string = json_string.strip('`').replace('json
|', '', 1).strip()
193 #json_string = json_string.replace()
194 try:
195 _j = simplejson.loads(json_string)
196 except json.JSONDecodeError:
197 logger.error(f'Error parsing JSON string:\\n {json_
|string}', ex, False)
198 ...
199 return {}
200 try:
201 # Decoding escape \\u0412\\u044b\\u0441\\u043e\\u043a\\u
|043e
202 return json.loads(json.dumps(_j))
203 except Exception as ex:
204 logger.error(f"Error decoding JSON", ex, False)
205 ...
206 return {}
207
208 # Main data processing
209 try:
210 if isinstance(jjson, SimpleNamespace): # If it's Simp
|leNamespace
211 jjson = vars(jjson) # Convert to dictionary
212
213 if isinstance(jjson, Path):
214 if jjson.is_dir(): # If it's directory
215 files = list(jjson.glob('*.json'))
216 return [j_loads(file, ordered=ordered) for fil

```

```

 e in files]
217 if jjson.suffix.lower() == '.csv': # If it's CSV
218
219 return pd.read_csv(jjson).to_dict(orient='records')
220
221 # If it's JSON file
222 #return decode_strings(json.loads(jjson.read_text(
223 encoding='utf-8')))
224 return json.loads(jjson.read_text(encoding='utf-8'
225))
226
227 elif isinstance(jjson, str): # If it's string
228 return string2dict(jjson)
229
230 elif isinstance(jjson, list): # If it's list
231 return [decode_strings(item) for item in jjson]
232
233 elif isinstance(jjson, dict): # If it's dictionary
234 return decode_strings(jjson)
235
236 except FileNotFoundError as ex:
237 logger.error(f'File not found: {jjson}')
238 return {}
239
240 except json.JSONDecodeError as ex:
241 logger.error(f'Error parsing JSON:\n{jjson}\n', ex, False)
242
243 return {}
244
245 except Exception as ex:
246 logger.error(f'Error loading data: ', ex, False)
247
248 return {}
249
250 return {}
251
252 def j_loads_ns(
253 jjson: Path | SimpleNamespace | Dict | str,
254 ordered: bool = True
255) -> SimpleNamespace:
256 """Load JSON or CSV data from a file, directory, or string
257 and convert to SimpleNamespace.
258
259 Args:

```

```

249 | jjson (Path | SimpleNamespace | Dict | str): Path to a
 | file, directory, or JSON data as a string, or JSON object.
250 | ordered (bool, optional): If returns OrderedDict inst
 | ead of a regular dict to preserve element order. Defaults to F
 | else.
251 | exc_info (bool, optional): If logs exceptions with tr
 | aceback. Defaults to True.
252 |
253 | Returns:
254 | Optional[SimpleNamespace | List[SimpleNamespace]]: Ret
 | urns SimpleNamespace or a list of SimpleNamespace objects if s
 | uccessful. Returns None if jjson is not found or cannot be rea
 | d.
255 |
256 | Examples:
257 | >>> j_loads_ns('data.json')
258 | SimpleNamespace(key='value')
259 |
260 | >>> j_loads_ns(Path('/path/to/directory'))
261 | [SimpleNamespace(key1='value1'), SimpleNamespace(key2=
 | 'value2')]
262 |
263 | >>> j_loads_ns({'"key": "value"}')
264 | SimpleNamespace(key='value')
265 |
266 | >>> j_loads_ns(Path('/path/to/file.csv'))
267 | [SimpleNamespace(column1='value1', column2='value2')]
268 | """
269 | data = j_loads(jjson, ordered=ordered)
270 | if data:
271 | if isinstance(data, list):
272 | return [dict2ns(item) for item in data]
273 | return dict2ns(data)
274 | return {}
275 |

```



## pdf.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: PDF conversion utilities for HTML content and
 files
4 # =====
5 # Description:
6 # Provides utilities for converting HTML content and files t
 o PDF format using
7 # multiple libraries including pdfkit, reportlab, weasyprint
 , and xhtml2pdf.
8 # Includes text extraction and PDF generation from various s
 ources.
9 #
10 # File: pdf.py
11 # Project: ai-breadboard
12 # Package: src.utils
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 from __future__ import annotations
18
19 import sys
20 import os
21 import json
22
23 from pathlib import Path
24 import pdfkit
25 from reportlab.pdfgen import canvas
26 from fpdf import FPDF
27 from weasyprint import HTML
28 from xhtml2pdf import pisa
29 from pdfminer.high_level import extract_text
```

```

30 from src.logger.logger import logger
31 from src.utils.printer import pprint
32
33 def set_project_root(marker_files=('__root__', '.git')) -> Path
34 :
35 """
36 Finding the root directory of the project starting from the
37 current file's directory.
38
39 Args:
40 marker_files (tuple): Filenames or directory names to
41 identify the project root.
42
43 Returns:
44 Path: Path to the root directory if found, otherwise the
45 directory where the script is located.
46 """
47 __root__:Path
48 current_path:Path = Path(__file__).resolve().parent
49 __root__ = current_path
50 for parent in [current_path] + list(current_path.parents):
51 if any((parent / marker).exists() for marker in marker
52 _files):
53 __root__ = parent
54 break
55 if __root__ not in sys.path:
56 sys.path.insert(0, str(__root__))
57 return __root__
58
59 # Get the root directory of the project
60 __root__: Path = set_project_root()
61 """__root__ (Path): Path to the root directory of the project"""
62
63 wkhtmltopdf_exe = __root__ / 'bin' / 'wkhtmltopdf' / 'files' /
64 'bin' / 'wkhtmltopdf.exe'
65
66 if not wkhtmltopdf_exe.exists():

```

```

61 | logger.error("wkhtmltopdf.exe not found at specified path.
 | ")
62 | raise FileNotFoundError("wkhtmltopdf.exe is missing")
63 |
64 | class PDFUtils:
65 | """
66 | Utilities class for PDF file operations providing methods
 | for saving HTML content to PDF using various libraries.
67 | """
68 |
69 | @staticmethod
70 | def save_pdf_pdfkit(data: str | Path, pdf_file: str | Path
 |) -> bool:
71 | """
72 | Saving HTML content or file to PDF using pdfkit librar
 | y.
73 |
74 | Args:
75 | data (str | Path): HTML content or path to HTML fi
 | le.
76 | pdf_file (str | Path): Path to saved PDF file.
77 |
78 | Returns:
79 | bool: True if PDF successfully saved, otherwise Fa
 | lse.
80 |
81 | Exceptions:
82 | pdfkit.PDFKitError: Error during PDF generation vi
 | a pdfkit.
83 | OSError: Error accessing file.
84 | """
85 |
86 | try:
87 | configuration = pdfkit.configuration(
88 | wkhtmltopdf=str(wkhtmltopdf_exe)
89 |)
90 |
91 | options = {"enable-local-file-access": ""}

```

```

92 if isinstance(data, str):
93 # Conversion of HTML content to PDF
94 pdfkit.from_string(data, pdf_file, configuration=configuration, options=options)
95 else:
96 # Conversion of HTML file to PDF
97 pdfkit.from_file(str(data), pdf_file, configuration=configuration, options=options)
98 logger.info(f"PDF successfully saved: {pdf_file}")
99 return True
100 # except (pdfkit.PDFKitError, OSError) as ex:
101 # logger.error("Error during PDF generation: ", ex
102)
103 # return False
104 except Exception as ex:
105 logger.error("Unexpected error: ", ex)
106 ...
107 return False
108
109 @staticmethod
110 def save_pdf_fpdf(data: str, pdf_file: str | Path) -> bool
111 :
112 """
113 Save text to PDF using FPDF library.
114
115 Args:
116 data (str): Text to save in PDF.
117 pdf_file (str | Path): Path to saved PDF file.
118
119 Returns:
120 bool: `True` if PDF successfully saved, otherwise `False`.
121 """
122 try:
123 pdf = FPDF()
124 pdf.add_page()
125 pdf.set_auto_page_break(auto = True, margin = 15)

```

```

125 # Path to fonts.json file
126 fonts_file_path = __root__ / 'assets' / 'fonts' /
127 'fonts.json'
128 if not fonts_file_path.exists():
129 logger.error(
130 f'Font settings JSON file not found: {font
131 s_file_path}\n'
132 'Format of `fonts.json` file:\n'
133 '{\n'
134 ' "dejavu-sans.book": {\n'
135 ' "family": "DejaVuSans",\n'
136 ' "path": "dejavu-sans.book.ttf",\n'
137 ' "style": "book",\n'
138 ' "uni": true\n'
139 ' }\n'
140 '}'
141)
142 raise FileNotFoundError(f'Font file not found:
143 {fonts_file_path}')
144 ...
145
146 with open(fonts_file_path, 'r', encoding = 'utf-8'
147) as json_file:
148 fonts = json.load(json_file)
149
150 # Add fonts
151 for font_name, font_info in fonts.items():
152 font_path = __root__ / 'assets' / 'fonts' / fo
153 nt_info['path']
154 if not font_path.exists():
155 logger.error(f'Font file not found: {font_
156 path}')
157 raise FileNotFoundError(f'Font file not fo
158 und: {font_path}')
159 ...
160
161 pdf.add_font(font_info['family'], font_info['s

```

```

 tyle'], str(font_path), uni = font_info['uni'])
155
156 # Set default font
157 pdf.set_font('DejaVuSans', style = 'book', size =
12)
158 pdf.multi_cell(0, 10, data)
159 pdf.output(str(pdf_file))
160 logger.info(f'PDF report successfully saved: {pdf_
file}')
161 return True
162 except Exception as ex:
163 logger.error('Error saving PDF via FPDF: ', ex)
164 ...
165 return False
166
167 @staticmethod
168 def save_pdf_weasyprint(data: str | Path, pdf_file: str |
Path) -> bool:
169 """
170 Save HTML content or file to PDF using WeasyPrint libr
ary.
171
172 Args:
173 data (str | Path): HTML content or path to HTML fi
le.
174 pdf_file (str | Path): Path to saved PDF file.
175
176 Returns:
177 bool: `True` if PDF successfully saved, otherwise
`False`.
178 """
179 try:
180 if isinstance(data, str):
181 HTML(string=data).write_pdf(pdf_file)
182 else:
183 HTML(filename=str(data)).write_pdf(pdf_file)
184 logger.info(f"PDF successfully saved: {pdf_file}")
185 return True

```



```

216 | source_data = source_file.read()
217 | pisa.CreatePDF(source_data, dest=result_file, encoding='UTF-8')
218 | except Exception as ex:
219 | logger.error("Error compiling PDF: ", ex)
220 | ...
221 | logger.info(f"PDF successfully saved: {pdf_file}")
222 | ...
223 | return True
224 | except Exception as ex:
225 | logger.error("Error saving PDF via xhtml2pdf: ", ex)
226 | ...
227 | return False
228 |
229 | @staticmethod
230 | def html2pdf(html_str: str, pdf_file: str | Path) -> bool | None:
231 | """Converts HTML content to a PDF file using WeasyPrint."""
232 | try:
233 | HTML(string=html_str).write_pdf(pdf_file)
234 | return True
235 | except Exception as e:
236 | print(f"Error during PDF generation: {e}")
237 | return
238 |
239 |
240 | @staticmethod
241 | def pdf_to_html(pdf_file: str | Path, html_file: str | Path) -> bool:
242 | """
243 | Convert PDF file to HTML file.
244 |
245 | Args:
246 | pdf_file (str | Path): Path to source PDF file.
247 | html_file (str | Path): Path to saved HTML file.

```



```

248
249 Returns:
250 bool: `True` if conversion successful, otherwise `
False`.
251 """
252 try:
253 # Extract text from PDF
254 text = extract_text(str(pdf_file))
255
256 # Create HTML file
257 with open(html_file, 'w', encoding='utf-8') as fil
e:
258 file.write(f"<html><body>{text}</body></html>"
)
259
260 print(f"HTML successfully saved: {html_file}")
261 return True
262 except Exception as ex:
263 print(f"Error converting PDF to HTML: {ex}")
264 return False
265
266 # Function to convert dictionary to PDF
267 @staticmethod
268 def dict2pdf(data: dict | 'SimpleNamespace', file_path: str | Path) -> None:
269 """
270 Save dictionary data to a PDF file.
271
272 Args:
273 data (dict | SimpleNamespace): The dictionary to c
onvert to PDF.
274 file_path (str | Path): Path to the output PDF fil
e.
275 """
276 if isinstance(data, 'SimpleNamespace'):
277 data = data.__dict__
278
279 pdf = canvas.Canvas(str(file_path), pagesize=A4)

```

```
280 | width, height = A4
281 | x, y = 50, height - 50
282 |
283 | pdf.setFont("Helvetica", 12)
284 |
285 | for key, value in data.items():
286 | line = f"{key}: {value}"
287 | pdf.drawString(x, y, line)
288 | y += 20
289 |
290 | if y >= height - 50: # Create new page if not enough space
291 | pdf.showPage()
292 | pdf.setFont("Helvetica", 12)
293 | y = height - 50
294 |
295 | pdf.save()
296 |
```

## printer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Pretty printing and text formatting utilities
4 # =====
5 # Description:
6 # Utility functions for pretty printing and text styling with support for
7 # color, background, and font styles. Provides human-readable formatted output.
8 #
9 # File: printer.py
10 # Project: ai-breadboard
11 # Package: src.utils
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """
17 Pretty printing and text formatting module.
18
19 Functions:
20 - `_color_text`: Apply color and style to text
21 - `pprint`: Pretty print data in human-readable format
22 """
23
24 import json
25 import csv
26 import pandas as pd
27 from pathlib import Path
28 from typing import Any
29 from pprint import pprint as pretty_print
30
31 # ANSI escape codes
```

```
32 RESET = "\033[0m"
33
34 TEXT_COLORS = {
35 "red": "\033[31m",
36 "green": "\033[32m",
37 "blue": "\033[34m",
38 "yellow": "\033[33m",
39 "white": "\033[37m",
40 "cyan": "\033[36m",
41 "magenta": "\033[35m",
42 "light_gray": "\033[37m",
43 "dark_gray": "\033[90m",
44 "light_red": "\033[91m",
45 "light_green": "\033[92m",
46 "light_blue": "\033[94m",
47 "light_yellow": "\033[93m",
48 }
49
50 # Background colors mapping
51 BG_COLORS = {
52 "bg_red": "\033[41m",
53 "bg_green": "\033[42m",
54 "bg_blue": "\033[44m",
55 "bg_yellow": "\033[43m",
56 "bg_white": "\033[47m",
57 "bg_cyan": "\033[46m",
58 "bg_magenta": "\033[45m",
59 "bg_light_gray": "\033[47m",
60 "bg_dark_gray": "\033[100m",
61 "bg_light_red": "\033[101m",
62 "bg_light_green": "\033[102m",
63 "bg_light_blue": "\033[104m",
64 "bg_light_yellow": "\033[103m",
65 }
66
67 FONT_STYLES = {
68 "bold": "\033[1m",
69 "underline": "\033[4m",
```

```

70 }
71
72 def _color_text(text: str, text_color: str = "", bg_color: str
 = "", font_style: str = "") -> str:
73 """Apply color, background, and font styling to the text.
74
75 This helper function applies the provided color and font s
 tyles to the given text using ANSI escape codes.
76
77 Args:
78 text: The text to be styled.
79 text_color: The color to apply to the text. Default is
 empty string (no color).
80 bg_color: The background color to apply. Default is em
 pty string (no background).
81 font_style: The font style to apply to the text. Defau
 lt is empty string (no style).
82
83 Returns:
84 The styled text as a string with ANSI escape codes app
 lied.
85
86 Example:
87 >>> _color_text("Hello, World!", text_color="green", f
 ont_style="bold")
88 '\033[1m\033[32mHello, World!\033[0m'
89 ""
90 return f"{font_style}{text_color}{bg_color}{text}{RESET}"
91
92 def pprint(print_data: Any = None, text_color: str = "white",
 bg_color: str = "", font_style: str = "") -> None:
93 """Pretty print the given data with optional color, backgr
 ound, and font style.
94
95 This function formats the input data based on its type and
 prints it to the console. The data is printed with optional
96 text color, background color, and font style based on the
 specified parameters. The function can handle dictionaries,

```

```

97 | lists, strings, and file paths.
98 |
99 | Args:
100 | print_data: The data to be printed. Can be None, dict,
101 | list, str, or Path. Default is None.
102 | text_color: The color to apply to the text. Default is
103 | 'white'. See TEXT_COLORS for options.
104 | bg_color: The background color to apply. Default is em
105 | pty (no background). See BG_COLORS for options.
106 | font_style: The font style to apply (bold, underline,
107 | etc.). Default is empty (no style).
108 |
109 | Raises:
110 | Exception: If the data type is unsupported or an error
111 | occurs during printing.
112 |
113 | Example:
114 | >>> pprint({"name": "Alice", "age": 30}, text_color="g
115 | reen")
116 | Prints colored JSON output.
117 |
118 | >>> pprint(["apple", "banana", "cherry"], text_color="
119 | blue", font_style="bold")
120 | Prints each item in colored bold text.
121 |
122 | >>> pprint("text example", text_color="yellow", bg_col
123 | or="bg_red", font_style="underline")
124 | Prints styled underlined text with yellow foreground o
125 | n red background.
126 |
127 | """
128 | text_color = TEXT_COLORS.get(text_color.lower(), TEXT_COLO
129 | RS["white"])
130 | bg_color = BG_COLORS.get(bg_color.lower(), "")
131 | font_style = FONT_STYLES.get(font_style.lower(), "")
132 |
133 | if print_data is None:
134 | print(_color_text("No data to print!", text_color=TEXT
135 | _COLORS["red"]))

```

```

124 return
125
126 try:
127 if isinstance(print_data, dict):
128 print(_color_text(json.dumps(print_data, indent=4)
129 , text_color))
129 elif isinstance(print_data, list):
130 for item in print_data:
131 print(_color_text(str(item), text_color))
132 elif isinstance(print_data, (str, Path)) and Path(print_data).is_file():
133 ext = Path(print_data).suffix.lower()
134 if ext in ['.csv', '.xls']:
135 print(_color_text("File reading supported for
136 .csv, .xls only.", text_color))
137 else:
138 print(_color_text("Unsupported file type.", te
139 xt_color))
140 else:
141 print(_color_text(str(print_data), text_color))
142 except Exception as ex:
143 print(_color_text(f"Error: {ex}", text_color=TEXT_COLO
144 RS["red"]))
145
146 if __name__ == '__main__':
147 pprint({"name": "Alice", "age": 30}, text_color="green")
148

```

## smtp.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: SMTP email utilities
4 # =====
5 # Description:
6 # This module provides functionality to send and receive emails using an SMTP or IMAP server.
7 #
8 # File: smtp.py
9 # Project: ai-breadboard
10 # Package: src.utils
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """SMTP email interface utilities for sending email messages via SMTP server."""
16
17 """ This module provides functionality to send and receive emails using an SMTP or IMAP server.
18 It includes functions to send emails using SMTP and retrieve emails using IMAP.
19
20 Functions:
21 - `send(subject: str = '', body: str = '', to: str = 'one.last.bit@gmail.com') -> bool`
22 Sends an email using the SMTP server specified in the `_connection` dictionary. Returns `True` on success, `False` on failure. Includes error logging.
23
24 - `receive(imap_server: str, user: str, password: str, folder: str = 'inbox') -> Optional[List[Dict[str, str]]`
25 Retrieves emails from an IMAP server and returns them as
```



a list of dictionaries. Returns `None` on error. Includes error logging.

26

27 **\*\* Important Considerations for Security and Robustness \*\*:**

28

29 - **\*\*\_connection Dictionary:\*\*** Do *\*not\** hardcode credentials in this file. Move the `\_connection` dictionary to environment variables (e.g., using `os.environ`). This is crucial for security. Avoid storing passwords directly in source code.

30

31 - **\*\*Error Handling:\*\*** The code includes robust error handling, logging exceptions with details (subject, body, etc.). This is very helpful for debugging.

32

33 - **\*\*Email Parsing:\*\*** The `receive` function handles various email formats gracefully, preventing potential issues.

34

35 - **\*\*MIME Handling:\*\*** The code correctly uses `MIMEText` for constructing the email message, crucial for sending basic text emails.

36

37 """

38

39 import smtplib

40 import imaplib

41 import email

42 import os

43 from email.mime.text import MIMEText

44 from typing import List, Dict, Optional

45

46 from src.logger.logger import logger

47

48 # --- Configuration ---

49 # DO NOT HARDCODE CREDENTIALS HERE! Use environment variables instead.

50 \_connection = {

51 'server': os.environ.get('SMTP\_SERVER', 'smtp.example.com'),

```

52 'port': int(os.environ.get('SMTP_PORT', 587)),
53 'user': os.environ.get('SMTP_USER'),
54 'password': os.environ.get('SMTP_PASSWORD'),
55 'receiver': os.environ.get('SMTP_RECEIVER', 'one.last.bit@
gmail.com')
56 }
57
58 def send(subject: str = '', body: str = '', to: str = 'one.las
t.bit@gmail.com') -> bool:
59 """Sends an email. Returns True if successful, False othe
rwise. Logs errors."""
60 try:
61 # Create SMTP connection
62 smtp = smtplib.SMTP(_connection['server'], _connection
['port'])
63 smtp.ehlo()
64 smtp.starttls()
65 smtp.login(_connection['user'], _connection['password'
])
66
67 message = MIMEText(body)
68 message["Subject"] = subject
69 message["From"] = _connection['user']
70 message["To"] = to
71
72 smtp.sendmail(_connection['user'], to, message.as_stri
ng())
73 smtp.quit()
74 return True
75
76 except Exception as ex:
77 logger.error(f"Error sending email. Subject: {subject}
. Body: {body}. Error: {ex}", exc_info=True)
78 return False
79
80 def receive(imap_server: str, user: str, password: str, folder
: str = 'inbox') -> Optional[List[Dict[str, str]]]:
81 """Retrieves emails. Returns a list of email dictionaries

```

```

 if successful, None otherwise. Logs errors."""
82 try:
83 mail = imaplib.IMAP4_SSL(imap_server)
84 mail.login(user, password)
85 mail.select(folder)
86
87 status, data = mail.search(None, 'ALL')
88 email_ids = data[0].split()
89
90 emails = []
91 for email_id in email_ids:
92 status, data = mail.fetch(email_id, '(RFC822)')
93 raw_email = data[0][1]
94 msg = email.message_from_bytes(raw_email)
95
96 email_data = {
97 'subject': msg['subject'],
98 'from': msg['from'],
99 'body': msg.get_payload(decode=True, _charset=
"utf-8").decode("utf-8", "ignore") # Decode & handle potentia
l errors
100 }
101 emails.append(email_data)
102
103 mail.close()
104 mail.logout()
105 return emails
106
107 except Exception as ex:
108 logger.error(f"Error occurred while retrieving emails:
{ex}", exc_info=True)
109 return None

```

## url.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: URL string parsing and manipulation utilities
4 # =====
5 # Description:
6 # Provides utilities for working with URL strings including
7 # extraction of query parameters,
8 # URL validation, and link shortening functionality with sup
9 # port for various URL formats
10 # and parameter parsing from query strings.
11 #
12 # File: url.py
13 # Project: ai-breadboard
14 # Package: src.utils
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # =====
18
19 from urllib.parse import urlparse, parse_qs
20 try:
21 import validators
22 except ImportError:
23 validators = None
24 import requests
25
26 def extract_url_params(url: str) -> dict | None:
27 """Extraction of query parameters from URL string.
28
29 Args:
30 url (str): URL string for parsing.
31
32 Returns:
33 dict | None: Dictionary of query parameters and their
```

```

values or None if URL has no parameters.
32 """
33 parsed_url = urlparse(url)
34 params = parse_qs(parsed_url.query)
35
36 # Convert parameter values from list to string if paramete
r has single value
37 if params:
38 params = {k: v if len(v) > 1 else v[0] for k, v in par
ams.items()}
39 return params
40 return None
41
42 def is_url(text: str) -> bool:
43 """Check if passed text is valid URL using validators libr
ary.
44
45 Args:
46 text (str): String to check.
47
48 Returns:
49 bool: `True` if string is valid URL, otherwise `False`
50 .
51 """
52 if not text:
53 return False
54 if validators is not None:
55 return bool(validators.url(text))
56 import re
57 pattern = re.compile(
58 r'^((https?://)?'
59 r'(([a-zA-Z0-9_-]+\.)+[a-zA-Z]{2,}|localhost|\d{1,3}\.
\d{1,3}\.\d{1,3}\.\d{1,3}))'
60 r'(:\d+)?'
61 r'([/?#].*)?$' , re.IGNORECASE)
62 return bool(pattern.match(text))
63
64 def url_shortener(long_url: str) -> str | None:

```

```

64 | """Shortens long URL using TinyURL service.
65 |
66 | Args:
67 | long_url (str): Long URL to shorten.
68 |
69 | Returns:
70 | str | None: Shortened URL or `None` if error occurred.
71 | """
72 | url = f'http://tinyurl.com/api-create.php?url={long_url}'
73 | response = requests.get(url)
74 |
75 | if response.status_code == 200:
76 | return response.text
77 | return None
78 |
79 | if __name__ == "__main__":
80 | # Get URL string from user
81 | url = input("Enter URL: ")
82 |
83 | # Check URL validity
84 | if is_url(url):
85 | params = extract_url_params(url)
86 |
87 | # Display parameters
88 | if params:
89 | print("URL Parameters:")
90 | for key, value in params.items():
91 | print(f"{key}: {value}")
92 | else:
93 | print("URL does not contain parameters.")
94 |
95 | # Offer user to shorten URL
96 | shorten = input("Would you like to shorten this URL? (y/n): ").strip().lower()
97 | if shorten == 'y':
98 | short_url = url_shortener(url)
99 | if short_url:
100 | print(f"Shortened URL: {short_url}")

```

```
101 | else:
102 | print("Error shortening URL.")
103 | else:
104 | print("Entered string is not a valid URL.")
105 |
```

## versioning.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Parse non-standard version string into list of
 integers
4 # =====
5 # Description:
6 # Parse version strings in SemVer format, compare two versions
 and select best tag.
7 # Implements semantic versioning 2.0 specification with backward
 compatibility.
8 #
9 # File: versioning.py
10 # Project: ai-breadboard
11 # Package: src.utils
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import re
17 from typing import List, Optional, Tuple
18
19 def _parse_version_legacy(v: str) -> List[int]:
20 """Parse non-standard version string into list of integers
21 .
22
23 Args:
24 v (str): Version string of arbitrary format.
25
26 Returns:
27 List[int]: List of numeric version components; [0, 0,
28 0] for empty string.
29
30 Examples:
```



```

29 >>> _parse_version_legacy('2.10.3')
30 [2, 10, 3]
31 """
32 if not v:
33 return [0, 0, 0]
34 parts = re.findall(r'(\d+)', v)
35 return [int(p) for p in parts]
36
37 def parse_semver(v: str) -> Tuple[int, int, int, List[str]]:
38 """Parse version string according to SemVer 2.0 specification.
39
40 Args:
41 v (str): Version string, optionally with 'v' prefix, prerelease and build metadata.
42
43 Returns:
44 Tuple[int, int, int, List[str]]: Tuple (major, minor, patch, prerelease).
45 Prerelease field is empty list if no prerelease tag present.
46 Returns empty tuple () for invalid string.
47
48 Examples:
49 >>> parse_semver('v1.2.3-alpha.1')
50 (1, 2, 3, ['alpha', '1'])
51 >>> parse_semver('1.0')
52 (1, 0, 0, [])
53 """
54 if not v:
55 return ()
56 m = re.match(
57 r'^v?(?P<major>0|[1-9]\d*)(?:\.(?P<minor>0|[1-9]\d*))?(?:\.(?P<patch>0|[1-9]\d*))?(?:\.(?P<prerelease>[0-9A-Za-z.-]+))?(?:\+[0-9A-Za-z.-]+)?$',
58 v,
59)
60

```

```

61 | if not m:
62 | return ()
63 | major = int(m.group('major'))
64 | minor = int(m.group('minor') or 0)
65 | patch = int(m.group('patch') or 0)
66 | prerelease_raw = m.group('prerelease')
67 | prerelease = prerelease_raw.split('.') if prerelease_raw e
 | else []
68 | return (major, minor, patch, prerelease)
69 |
70 | def compare_versions(a: str, b: str) -> int:
71 | """Compare two version strings with SemVer priority.
72 |
73 | Args:
74 | a (str): First version string.
75 | b (str): Second version string.
76 |
77 | Returns:
78 | int: -1 if a < b, 0 if equal, 1 if a > b.
79 |
80 | Examples:
81 | >>> compare_versions('1.2.3', '1.2.4')
82 | -1
83 | >>> compare_versions('v1.10.0', '1.9.9')
84 | 1
85 | >>> compare_versions('1.2.3-alpha', '1.2.3')
86 | -1
87 | """
88 | def _compare(a_parsed: tuple, b_parsed: tuple) -> int:
89 | # Fallback to legacy parsing for invalid SemVer
90 | if not a_parsed and not b_parsed:
91 | return 0
92 | if not a_parsed or not b_parsed:
93 | pa = _parse_version_legacy(a)
94 | pb = _parse_version_legacy(b)
95 | length = max(len(pa), len(pb))
96 | pa += [0] * (length - len(pa))
97 | pb += [0] * (length - len(pb))

```

```
98 if pa < pb:
99 return -1
100 if pa > pb:
101 return 1
102 return 0
103
104 # Compare major.minor.patch
105 for i in range(3):
106 if a_parsed[i] < b_parsed[i]:
107 return -1
108 if a_parsed[i] > b_parsed[i]:
109 return 1
110
111 # Compare prerelease tags (SemVer §11)
112 a_pr: List[str] = a_parsed[3]
113 b_pr: List[str] = b_parsed[3]
114
115 # Stable release is greater than prerelease
116 if not a_pr and not b_pr:
117 return 0
118 if not a_pr:
119 return 1
120 if not b_pr:
121 return -1
122
123 for ai, bi in zip(a_pr, b_pr):
124 if ai == bi:
125 continue
126 if ai.isdigit() and bi.isdigit():
127 diff = int(ai) - int(bi)
128 if diff < 0:
129 return -1
130 if diff > 0:
131 return 1
132 else:
133 if ai < bi:
134 return -1
135 if ai > bi:
```

```

136 return 1
137
138 if len(a_pr) < len(b_pr):
139 return -1
140 if len(a_pr) > len(b_pr):
141 return 1
142 return 0
143
144 return _compare(parse_semver(a), parse_semver(b))
145
146 def choose_best_tag(tags: List[str], allow_prerelease: bool =
147 False, debug: bool = False) -> str:
148 """Choose greatest version from list of tags.
149
150 Args:
151 tags (List[str]): List of version/tag strings.
152 allow_prerelease (bool): Allow prerelease tags if no s
153 table versions available.
154 Default: False.
155 debug (bool): Output debug information via logger.
156 Default: False.
157
158 Returns:
159 str: Tag with greatest version; empty string for empty
160 list.
161
162 Examples:
163 >>> choose_best_tag(['v1.0.0', 'v1.1.0-alpha', 'v1.0.1
164 '])
165 'v1.0.1'
166 >>> choose_best_tag(['v1.0.0', 'v1.1.0-alpha'], allow_
167 prerelease=True)
168 'v1.1.0-alpha'
169 """
170 if not tags:
171 return ''
172
173 def _is_prerelease(t: str) -> bool:

```

```
169 | return '-' in t.split('+', 1)[0]
170 |
171 | stable = [t for t in tags if not _is_prerelease(t)]
172 | candidates = stable if stable and not allow_prerelease else tags
173 |
174 | if debug:
175 | from src.logger.logger import logger
176 | logger.debug(f'[versioning.choose_best_tag] candidates={candidates} allow_prerelease={allow_prerelease}')
177 |
178 | best = ''
179 | for t in candidates:
180 | if not best or compare_versions(best, t) < 0:
181 | best = t
182 | return best
183 |
```

## video.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Video file download and processing utilities
4 # =====
5 # Description:
6 # This module provides asynchronous functions for downloading and saving video files, as well as retrieving
7 #
8 # File: video.py
9 # Project: ai-breadboard
10 # Package: src.utils
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """Video saving utilities for generating and saving video files."""
16
17 """ This module provides asynchronous functions for downloading and saving video files, as well as retrieving video data. It includes error handling and logging for robust operation.
18
19 Functions:
20 save_video_from_url(url: str, save_path: str) -> Optional[Path]:
21 Download a video from a URL and save it locally asynchronously. Handles potential network issues and file saving errors.
22
23 get_video_data(file_name: str) -> Optional[bytes]:
24 Retrieve binary data of a video file if it exists. Handles file not found and read errors.
25
```

```

26 Examples:
27 >>> import asyncio
28 >>> asyncio.run(save_video_from_url("https://example.com/v
 ideo.mp4", "local_video.mp4"))
29 PosixPath('local_video.mp4') # or None if failed
30
31 >>> data = get_video_data("local_video.mp4")
32 >>> if data:
33 ... print(data[:10]) # Print first 10 bytes to check
34 b'\x00\x00\x00...'
35 ""
36
37 import aiohttp
38 import aiofiles
39 from pathlib import Path
40 from typing import Optional
41 import asyncio
42 from src.logger.logger import logger
43
44 async def save_video_from_url(
45 url: str,
46 save_path: str
47) -> Optional[Path]:
48 """Download a video from a URL and save it locally asynchr
 onously.
49
50 Args:
51 url (str): The URL from which to download the video.
52 save_path (str): The path to save the downloaded video
53 .
54
55 Returns:
56 Optional[Path]: The path to the saved file, or `None`
57 if the operation failed. Returns None on errors and if file i
58 s 0 bytes.
59
60 Raises:
61 aiohttp.ClientError: on network issues during the down

```

```

load.
59 | """
60 | save_path = Path(save_path)
61 |
62 | try:
63 | async with aiohttp.ClientSession() as session:
64 | async with session.get(url) as response:
65 | response.raise_for_status() # Check for HTTP
errors
66 |
67 | # Create parent directories if they don't exist
68 | save_path.parent.mkdir(parents=True, exist_ok=
True)
69 |
70 | async with aiofiles.open(save_path, "wb") as f
ile:
71 | while True:
72 | chunk = await response.content.read(81
92)
73 | if not chunk:
74 | break
75 | await file.write(chunk)
76 |
77 | # Crucial checks after saving
78 | if not save_path.exists():
79 | logger.error(f"File {save_path} not saved successf
ully.")
80 | return None
81 |
82 | if save_path.stat().st_size == 0:
83 | logger.error(f"Downloaded file {save_path} is empt
y.")
84 | return None
85 |
86 | return save_path
87 |
88 | except aiohttp.ClientError as e:

```



```

89 logger.error(f"Network error downloading video: {e}")
90 return None
91 except Exception as e:
92 logger.error(f"Error saving video {save_path}: {e}", e
xc_info=True)
93 return None
94
95 def get_video_data(file_name: str) -> Optional[bytes]:
96 """Retrieve binary data of a video file if it exists.
97
98 Args:
99 file_name (str): The path to the video file to read.
100
101 Returns:
102 Optional[bytes]: The binary data of the file if it exists, or `None` if the file is not found or an error occurred.
103 """
104 file_path = Path(file_name)
105
106 if not file_path.exists():
107 logger.error(f"File {file_name} not found.")
108 return None
109
110 try:
111 with open(file_path, "rb") as file:
112 return file.read()
113 except Exception as e:
114 logger.error(f"Error reading file {file_name}: {e}", e
xc_info=True)
115 return None
116
117 def main():
118 url = "https://example.com/video.mp4" # Replace with a valid URL!
119 save_path = "local_video.mp4"
120 result = asyncio.run(save_video_from_url(url, save_path))
121 if result:
122 print(f"Video saved to {result}")

```

```
123 |
124 | if __name__ == "__main__":
125 | main()
```

xls.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Excel file processing and conversion utilities
4 # =====
5 # Description:
6 # This module provides functions to convert Excel files to J
7 # SON format, handle multiple sheets, and sa
8 #
9 # File: xls.py
10 # Project: ai-breadboard
11 # Package: src.utils
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16
17 """Excel file conversion utilities for converting between XLS
18 and JSON formats."""
19
20 """ This module provides functions to convert Excel files to J
21 SON format, handle multiple sheets, and save JSON data back to
22 Excel files.
23
24 Functions:
25 read_xls_as_dict(xls_file: str, json_file: str = None, she
26 et_name: Union[str, int] = None) -> Union[Dict, List[Dict], bo
27 ol]:
28 Reads an Excel file and converts it to JSON. Optional
29 ly, converts a specific sheet and saves the result to a JSON f
30 ile. Handles errors gracefully.
31
32 save_xls_file(data: Dict[str, List[Dict]], file_path: str)
33 -> bool:
34 Saves JSON data to an Excel file. The data should be
```

a dictionary where keys are sheet names and values are lists of dictionaries representing rows. Handles errors gracefully.

25

26 Examples:

27 # Reading and optionally saving to JSON

28 data = read\_xls\_as\_dict('input.xlsx', 'output.json', 'Sheet1') # Reads sheet named 'Sheet1'

29 if data:

30 print(data) # Output will be {'Sheet1': [{...}]}

31

32 # Saving from JSON data

33 data\_to\_save = {'Sheet1': [{'column1': 'value1', 'column2': 'value2'}]}

34 success = save\_xls\_file(data\_to\_save, 'output.xlsx')

35 if success:

36 print("Successfully saved to output.xlsx")

37 """

38

39 import pandas as pd

40 import json

41 from typing import List, Dict, Union

42 from pathlib import Path

43 import logging

44

45 # Configure logging

46 logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

47

48 def read\_xls\_as\_dict(

49 xls\_file: str,

50 json\_file: str = None,

51 sheet\_name: Union[str, int] = None

52 ) -> Union[Dict, List[Dict], bool]:

53 """

54 Reads an Excel file and converts it to JSON. Optionally, converts a specific sheet and saves the result to a JSON file.

55 Handles errors gracefully.

56 """

```

57 | try:
58 | xls_file_path = Path(xls_file)
59 | if not xls_file_path.exists():
60 | logging.error(f"Excel file not found: {xls_file}")
61 | return False # Indicate failure
62 |
63 | xls = pd.ExcelFile(xls_file)
64 |
65 | if sheet_name is None:
66 | data_dict = {}
67 | for sheet in xls.sheet_names:
68 | try:
69 | df = pd.read_excel(xls, sheet_name=sheet)
70 | data_dict[sheet] = df.to_dict(orient='records')
71 | except Exception as e:
72 | logging.error(f"Error processing sheet '{sheet}': {e}")
73 | return False
74 |
75 | else:
76 | try:
77 | df = pd.read_excel(xls, sheet_name=sheet_name)
78 | data_dict = df.to_dict(orient='records')
79 | except Exception as e:
80 | logging.error(f"Error processing sheet '{sheet_name}': {e}")
81 | return False
82 |
83 | if json_file:
84 | with open(json_file, 'w', encoding='utf-8') as f:
85 | json.dump(data_dict, f, ensure_ascii=False, indent=4)
86 | logging.info(f"JSON data saved to {json_file}")
87 |
88 | return data_dict
89 |

```

```
90 | except FileNotFoundError as e:
91 | logging.error(f"File not found: {e}")
92 | return False
93 | except Exception as e:
94 | logging.error(f"An error occurred: {e}")
95 | return False
96 |
97 | def save_xls_file(data: Dict[str, List[Dict]], file_path: str)
 | -> bool:
98 | """Saves JSON data to an Excel file. Handles errors gracef
 | ully."""
99 | try:
100 | with pd.ExcelWriter(file_path, engine='xlsxwriter') as
 | writer:
101 | for sheet_name, rows in data.items():
102 | df = pd.DataFrame(rows)
103 | df.to_excel(writer, sheet_name=sheet_name, ind
 | ex=False)
104 | logging.info(f"Sheet '{sheet_name}' saved to {
 | file_path}")
105 | return True
106 | except Exception as e:
107 | logging.error(f"Error saving Excel file: {e}")
108 | return False
109 |
110 |
```

## version\_manager.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Version manager and automatic update handler
4 # =====
5 # Description:
6 # Module for managing application versions, checking updates
7 # in Git,
8 # downloading and applying updates with backup/restore capabilities.
9 #
10 # File: version_manager.py
11 # Project: ai-breadboard
12 # Package: src
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 """
18 Version manager and automatic update handler.
19 Platform: Windows, Unix
20 Synopsis: Version manager and automatic updates
21
22 Key functions:
23 - Check version in Git repository
24 - Download updated files
25 - Create backups before updating
26 - Cross-platform support (Windows, Linux, macOS)
27 """
28
29 import os
30 import subprocess
31 import shutil
32 import asyncio
```

```

32 import tempfile
33 import json
34 from pathlib import Path
35 from datetime import datetime
36 from typing import Optional, Dict, List, Tuple
37 from dataclasses import dataclass, asdict
38 from enum import Enum
39
40 from header import __root__
41 from src.logger import logger
42
43 class UpdateStatus(Enum):
44 """Application update statuses."""
45 CURRENT = "up_to_date"
46 UPDATE_AVAILABLE = "update_available"
47 ERROR = "error"
48 UPDATING = "updating"
49
50 @dataclass
51 class VersionInfo:
52 """Information about application version."""
53 current_version: str
54 remote_version: str
55 commit_hash: str
56 remote_commit_hash: str
57 is_update_available: bool
58 update_status: str
59
60 class VersionManager:
61 """
62 Application version and update manager.
63
64 Implements functionality:
65 - Check current version in Git
66 - Check available updates on remote repository
67 - Download and apply updates
68 - Create backups before updating
69 - Restore from backup in case of error

```



```

70
71 All backups stored in system temp directory for cross-plat
form compatibility.
72 """
73
74 def __init__(self, repo_path: Optional[Path] = None):
75 """
76 Initialize version manager.
77
78 Args:
79 repo_path: Path to repository (default: __root__)
80 """
81 self.repo_path: Path = repo_path or __root__
82 self.temp_backup_dir: Path = Path(tempfile.gettempdir(
)) / 'ai-breadboard' / 'backups'
83 self.update_log_dir: Path = Path(tempfile.gettempdir()
) / 'ai-breadboard' / 'updates'
84 self.current_version: str = "1.0.0"
85 self.remote_url: str = "origin"
86
87 # Create directories
88 self.temp_backup_dir.mkdir(parents=True, exist_ok=True
)
89 self.update_log_dir.mkdir(parents=True, exist_ok=True)
90
91 logger.info(f"VersionManager initialized. Repo: {self.
repo_path}")
92 logger.info(f"Backup directory: {self.temp_backup_dir}
")
93
94 def _run_git_command(self, command: List[str], cwd: Option
al[Path] = None) -> Tuple[int, str, str]:
95 """
96 Execute git command.
97
98 Args:
99 command: List of arguments for git command
100 cwd: Working directory (default: repo_path)

```

```

101
102 Returns:
103 Tuple (exit code, stdout, stderr)
104 """
105 try:
106 cwd = cwd or self.repo_path
107 result = subprocess.run(
108 command,
109 cwd=str(cwd),
110 capture_output=True,
111 text=True,
112 timeout=30
113)
114 return result.returncode, result.stdout.strip(), r
115 esult.stderr.strip()
116 except subprocess.TimeoutExpired:
117 logger.error("Git command timeout")
118 return -1, "", "Timeout"
119 except Exception as ex:
120 logger.error(f"Error executing git command: {ex}")
121 return -1, "", str(ex)
122
123 def get_current_version(self) -> Optional[str]:
124 """
125 Get current version from git tag or version file.
126
127 Returns:
128 Version string or None on error
129 """
130 try:
131 # Try to get latest tag (version)
132 rc, stdout, stderr = self._run_git_command(["git",
133 "describe", "--tags", "--always"])
134
135 if rc == 0 and stdout:
136 self.current_version = stdout
137 logger.info(f"Current version: {self.current_v
138 erision}")

```

```

136 return self.current_version
137
138 # If no tags, use commit hash
139 rc, stdout, stderr = self._run_git_command(["git",
140 "rev-parse", "--short", "HEAD"])
141 if rc == 0 and stdout:
142 self.current_version = f"commit-{stdout}"
143 logger.info(f"Current version (hash): {self.cu
144 rrent_version}")
145 return self.current_version
146
147 logger.warning("Could not determine current versio
148 n")
149 return None
150
151 except Exception as ex:
152 logger.error(f"Error getting current version: {ex}
153 ")
154 return None
155
156 def get_remote_version(self) -> Optional[str]:
157 """
158 Get version from remote repository.
159
160 Returns:
161 Version string or None on error
162 """
163 try:
164 # Get remote repository information
165 rc, stdout, stderr = self._run_git_command(["git",
166 "ls-remote", "--tags", self.remote_url])
167
168 if rc == 0 and stdout:
169 lines = stdout.split('\n')
170 # Find latest tag
171 tags = [line.split()[-1].replace('refs/tags/',
172 '').replace('^{}', '')
173 for line in lines if 'refs/tags/' in li
174 ne]

```

```

167 if tags:
168 # Sort and get latest
169 remote_version = sorted(tags)[-1]
170 logger.info(f"Remote version: {remote_vers
ion}")
171 return remote_version
172
173 # If no tags, use HEAD of remote repo
174 rc, stdout, stderr = self._run_git_command(["git",
"ls-remote", self.remote_url, "HEAD"])
175 if rc == 0 and stdout:
176 remote_hash = stdout.split()[0]
177 logger.info(f"Remote version (HEAD): {remote_h
ash}")
178 return f"remote-{remote_hash[:7]}"
179
180 logger.warning("Could not determine remote version
")
181 return None
182 except Exception as ex:
183 logger.error(f"Error getting remote version: {ex}"
)
184 return None
185
186 def check_updates(self) -> Dict:
187 """
188 Check for available updates.
189
190 Returns:
191 Dictionary with update information
192 """
193 try:
194 # Get current versions
195 current = self.get_current_version()
196 remote = self.get_remote_version()
197
198 if not current or not remote:
199 logger.warning("Could not check updates")

```

```

200 return {
201 "status": UpdateStatus.ERROR.value,
202 "message": "Could not get version informat
ion"
203 }
204
205 # Get commit hashes
206 rc1, curr_hash, _ = self._run_git_command(["git",
"rev-parse", "HEAD"])
207 rc2, remote_hash, _ = self._run_git_command(["git"
, "rev-parse", f"{self.remote_url}/main"])
208
209 is_update_available = current != remote
210 status = UpdateStatus.UPDATE_AVAILABLE if is_updat
e_available else UpdateStatus.CURRENT
211
212 result = {
213 "status": status.value,
214 "current_version": current,
215 "remote_version": remote,
216 "current_commit": curr_hash if rc1 == 0 else "
unknown",
217 "remote_commit": remote_hash if rc2 == 0 else
"unknown",
218 "is_update_available": is_update_available,
219 "timestamp": datetime.now().isoformat()
220 }
221
222 logger.info(f"Update check: {result['status']}")
223 return result
224 except Exception as ex:
225 logger.error(f"Error checking updates: {ex}")
226 return {
227 "status": UpdateStatus.ERROR.value,
228 "message": str(ex)
229 }
230
231 def backup_files(self, files_to_backup: Optional[List[str]]

```

```

] = None) -> Optional[Path]:
232 """
233 Create backup of files before updating.
234
235 Args:
236 files_to_backup: List of files to backup (default:
main project files)
237
238 Returns:
239 Path to backup directory or None on error
240 """
241 try:
242 # Standard files for backup
243 if files_to_backup is None:
244 files_to_backup = [
245 "config.json",
246 ".env",
247 "src",
248 "requirements.txt"
249]
250
251 # Create backup directory
252 backup_dir = self.temp_backup_dir / f"backup_{date
time.now().strftime('%Y%m%d_%H%M%S')}"
253 backup_dir.mkdir(parents=True, exist_ok=True)
254
255 logger.info(f"Creating backup in: {backup_dir}")
256
257 backup_count = 0
258 for file_item in files_to_backup:
259 src = self.repo_path / file_item
260
261 if not src.exists():
262 logger.warning(f"Backup file not found: {s
rc}")
263 continue
264
265 dst = backup_dir / file_item

```

```

266
267 try:
268 if src.is_dir():
269 if dst.exists():
270 shutil.rmtree(dst)
271 shutil.copytree(src, dst, dirs_exist_o
k=True)
272 else:
273 dst.parent.mkdir(parents=True, exist_o
k=True)
274 shutil.copy2(src, dst)
275
276 backup_count += 1
277 logger.debug(f"Backed up: {file_item}")
278 except Exception as ex:
279 logger.warning(f"Error backing up {file_it
em}: {ex}")
280
281 if backup_count == 0:
282 logger.error("Could not create backup (no file
s)")
283 return None
284
285 # Save backup information
286 backup_info = {
287 "timestamp": datetime.now().isoformat(),
288 "repo_path": str(self.repo_path),
289 "backed_up_files": files_to_backup,
290 "backed_up_count": backup_count,
291 "version": self.current_version
292 }
293
294 info_file = backup_dir / ".backup_info.json"
295 with open(info_file, 'w', encoding='utf-8') as f:
296 json.dump(backup_info, f, indent=2, ensure_asc
ii=False)
297
298 logger.info(f"Backup created: {backup_dir}")

```

```

299 return backup_dir
300 except Exception as ex:
301 logger.error(f"Error creating backup: {ex}")
302 return None
303
304 def restore_from_backup(self, backup_path: Path) -> bool:
305 """
306 Restore files from backup.
307
308 Args:
309 backup_path: Path to backup directory
310
311 Returns:
312 True if restore successful, False otherwise
313 """
314 try:
315 if not backup_path.exists():
316 logger.error(f"Backup directory not found: {ba
317 ckup_path}")
318 return False
319
320 logger.info(f"Restoring from backup: {backup_path}
321 ")
322
323 restore_count = 0
324 for item in backup_path.iterdir():
325 if item.name == ".backup_info.json":
326 continue
327
328 dst = self.repo_path / item.name
329
330 try:
331 if item.is_dir():
332 if dst.exists():
333 shutil.rmtree(dst)
334 shutil.copytree(item, dst, dirs_exist_
335 ok=True)
336 except:
337 else:

```



```

334 dst.parent.mkdir(parents=True, exist_o
k=True)
335 shutil.copy2(item, dst)
336
337 restore_count += 1
338 logger.debug(f"Restored: {item.name}")
339 except Exception as ex:
340 logger.warning(f"Error restoring {item.nam
e}: {ex}")
341
342 logger.info(f"Restored files: {restore_count}")
343 return restore_count > 0
344 except Exception as ex:
345 logger.error(f"Error restoring from backup: {ex}")
346 return False
347
348 def fetch_updates(self) -> bool:
349 """
350 Download updates from remote repository.
351
352 Returns:
353 True if download successful, False otherwise
354 """
355 try:
356 logger.info(f"Downloading updates from {self.remot
e_url}...")
357
358 # Execute git fetch
359 rc, stdout, stderr = self._run_git_command(["git",
"fetch", self.remote_url])
360
361 if rc != 0:
362 logger.error(f"Error git fetch: {stderr}")
363 return False
364
365 logger.info("Updates downloaded successfully")
366 return True
367 except Exception as ex:

```

```

368 logger.error(f"Error downloading updates: {ex}")
369 return False
370
371 def merge_updates(self, branch: str = "main") -> Tuple[boo
372 l, str]:
373 """
374 Merge updates from remote repository.
375
376 Args:
377 branch: Branch name (default: main)
378
379 Returns:
380 Tuple (success, message)
381 """
382 try:
383 logger.info(f"Merging updates from {self.remote_ur
384 l}/{branch}...")
385
386 # Execute git merge
387 rc, stdout, stderr = self._run_git_command(
388 ["git", "merge", f"{self.remote_url}/{branch}"
389]
390)
391
392 if rc != 0:
393 # May be merge conflict
394 if "conflict" in stderr.lower():
395 logger.warning(f"Merge conflict: {stderr}"
396)
397
398 return False, f"Conflict during merge: {st
399 derr}"
400
401 logger.error(f"Error git merge: {stderr}")
402 return False, f"Error merging: {stderr}"
403
404 logger.info("Updates merged successfully")
405 return True, stdout
406 except Exception as ex:

```

```

401 logger.error(f"Error merging updates: {ex}")
402 return False, str(ex)
403
404 async def update_application(self, branch: str = "main", a
uto_backup: bool = True) -> Dict:
405 """
406 Perform complete application update.
407
408 Process:
409 1. Check for available updates
410 2. Create backup (if auto_backup=True)
411 3. Download updates
412 4. Merge updates
413 5. Update dependencies (if requirements.txt changed)
414
415 Args:
416 branch: Branch to update
417 auto_backup: Whether to create backup before updat
ing
418
419 Returns:
420 Dictionary with update result
421 """
422 try:
423 logger.info("Starting application update process..
.")
424
425 # Check for updates
426 check_result = self.check_updates()
427 if check_result["status"] == UpdateStatus.ERROR.va
lue:
428 return {
429 "success": False,
430 "status": UpdateStatus.ERROR.value,
431 "message": "Could not check for updates"
432 }
433
434 if not check_result.get("is_update_available"):

```

```
435 logger.info("Application is already up to date
 ")
436 return {
437 "success": False,
438 "status": UpdateStatus.CURRENT.value,
439 "message": "No updates available"
440 }
441
442 # Create backup
443 backup_path = None
444 if auto_backup:
445 backup_path = self.backup_files()
446 if not backup_path:
447 return {
448 "success": False,
449 "status": UpdateStatus.ERROR.value,
450 "message": "Could not create backup"
451 }
452
453 # Download updates
454 if not self.fetch_updates():
455 if backup_path:
456 self.restore_from_backup(backup_path)
457 return {
458 "success": False,
459 "status": UpdateStatus.ERROR.value,
460 "message": "Error downloading updates"
461 }
462
463 # Merge updates
464 merge_ok, merge_msg = self.merge_updates(branch)
465 if not merge_ok:
466 if backup_path:
467 self.restore_from_backup(backup_path)
468 return {
469 "success": False,
470 "status": UpdateStatus.ERROR.value,
471 "message": f"Error merging: {merge_msg}"
```

```

472 }
473
474 # Save update information
475 update_info = {
476 "timestamp": datetime.now().isoformat(),
477 "from_version": check_result.get("current_vers
ion"),
478 "to_version": check_result.get("remote_version
"),
479 "backup_path": str(backup_path) if backup_path
else None,
480 "branch": branch,
481 "status": "success"
482 }
483
484 update_log_file = self.update_log_dir / f"update_{
datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
485 with open(update_log_file, 'w', encoding='utf-8')
as f:
486 json.dump(update_info, f, indent=2, ensure_asc
ii=False)
487
488 logger.info(
489 f"Application updated: {check_result.get('curr
ent_version')} → "
490 f"{check_result.get('remote_version')}"
491)
492
493 return {
494 "success": True,
495 "status": UpdateStatus.CURRENT.value,
496 "message": "Update completed successfully",
497 "version": check_result.get("remote_version"),
498 "backup_path": str(backup_path) if backup_path
else None
499 }
500 except Exception as ex:
501 logger.error(f"Error updating application: {ex}")

```

```

502 return {
503 "success": False,
504 "status": UpdateStatus.ERROR.value,
505 "message": str(ex)
506 }
507
508 def cleanup_old_backups(self, keep_count: int = 5) -> int:
509 """
510 Delete old backups keeping latest N.
511
512 Args:
513 keep_count: Number of latest backups to keep
514
515 Returns:
516 Number of deleted backups
517 """
518 try:
519 if not self.temp_backup_dir.exists():
520 return 0
521
522 backups = sorted(
523 [d for d in self.temp_backup_dir.iterdir() if
524 d.is_dir()],
525 key=lambda x: x.stat().st_mtime,
526 reverse=True
527)
528
529 deleted_count = 0
530 for backup in backups[keep_count:]:
531 try:
532 shutil.rmtree(backup)
533 deleted_count += 1
534 logger.debug(f"Deleted old backup: {backup
535 .name}")
536 except Exception as ex:
537 logger.warning(f"Error deleting backup: {e

```

```

537 if deleted_count > 0:
538 logger.info(f"Deleted old backups: {deleted_co
unt}")
539
540 return deleted_count
541 except Exception as ex:
542 logger.error(f"Error cleaning backups: {ex}")
543 return 0
544
545 # Global version manager instance
546 version_manager: Optional[VersionManager] = None
547
548 def get_version_manager(repo_path: Optional[Path] = None) -> V
ersionManager:
549 """
550 Get global version manager instance (Singleton).
551
552 Args:
553 repo_path: Path to repository
554
555 Returns:
556 VersionManager instance
557 """
558 global version_manager
559 if version_manager is None:
560 version_manager = VersionManager(repo_path)
561 return version_manager
562

```

\_\_init\_\_.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI Breadboard Root Source Package
4 # =====
5 # Description:
6 # Root package for all internal AI Breadboard modules, runti
7 # mes, and utilities.
8 #
9 # File: __init__.py
10 # Project: ai-breadboard
11 # Package: src
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """AI Breadboard core package root.
17
18 Contains modular AI providers, FastAPI endpoints, logging, RAG
19 , and utilities.
20 """
21
22 from __future__ import annotations
23
24 __version__ = "0.0.0"
```



## install.sh

```
1 #!/bin/bash
2 #
3 # Скрипт установки systemd user services для AI Breadboard
4 #
5 # Использование:
6 # bash systemd/install.sh
7 # bash systemd/install.sh --enable
8 # bash systemd/install.sh --all
9 #
10
11 set -e
12
13 # Цвета
14 RED='\033[0;31m'
15 GREEN='\033[0;32m'
16 YELLOW='\033[1;33m'
17 BLUE='\033[0;34m'
18 NC='\033[0m' # No Color
19
20 # Определить директорию скрипта
21 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
22 PROJECT_ROOT="$(dirname "$SCRIPT_DIR")"
23
24 # systemd пользовательская директория
25 SYSTEMD_USER_DIR="$HOME/.config/systemd/user"
26
27 echo -e "${BLUE}=====
28 =====${NC}"
29 echo -e "${BLUE} AI Breadboard systemd User Services Install
30 er${NC}"
31 echo -e "${BLUE}=====
32 =====${NC}"
33 echo ""
34
35 # Проверить что мы на Linux
36 if [["$OSTYPE" != "linux-gnu"*]]; then
```

```
34 echo -e "${RED}ERROR: systemd user services supported only
 on Linux${NC}"
35 exit 1
36 fi
37
38 # Проверить что директория .service файлов существует
39 if [! -d "$SCRIPT_DIR"]; then
40 echo -e "${RED}ERROR: systemd directory not found: $SCRIPT
 _DIR${NC}"
41 exit 1
42 fi
43
44 # Создать ~/.config/systemd/user если её нет
45 echo -e "${YELLOW}[1/3] Creating systemd user directory...${NC}
)"
46 mkdir -p "$SYSTEMD_USER_DIR"
47 echo -e "${GREEN} Directory created: $SYSTEMD_USER_DIR${NC}"
48 echo ""
49
50 # Копировать .service файлы
51 echo -e "${YELLOW}[2/3] Installing .service files...${NC}"
52 for service_file in "$SCRIPT_DIR"/*.service; do
53 if [-f "$service_file"]; then
54 service_name=$(basename "$service_file")
55 target="$SYSTEMD_USER_DIR/$service_name"
56
57 # Заменить пути в файле на реальные пути проекта
58 sed "s|%h/AI-Breadboard|$PROJECT_ROOT|g" "$service_fil
 e" > "$target"
59 chmod 644 "$target"
60 echo -e "${GREEN} Installed: $service_name${NC}"
61 fi
62 done
63 echo ""
64
65 # Перезагрузить systemd daemon
66 echo -e "${YELLOW}[3/3] Reloading systemd daemon...${NC}"
67 systemctl --user daemon-reload
```

```

68 echo -e "${GREEN} systemd daemon reloaded${NC}"
69 echo ""
70
71 # Показать установленные сервисы
72 echo -e "${BLUE}=====
===== ${NC}"
73 echo -e "${BLUE} Installed Services${NC}"
74 echo -e "${BLUE}=====
===== ${NC}"
75 systemctl --user list-unit-files | grep ai-breadboard || echo
 "No services found"
76 echo ""
77
78 # Обработать параметры
79 if ["$1" == "--enable"]; then
80 echo -e "${YELLOW}Enabling auto-start for all services...${
 {NC}"
81 systemctl --user enable ai-breadboard-server.service
82 systemctl --user enable ai-breadboard-mcp-langchain.servic
 e
83 systemctl --user enable ai-breadboard-mcp-gemini.service
84 echo -e "${GREEN} All services enabled for auto-start${NC
 }"
85 echo ""
86 elif ["$1" == "--all"] || ["$1" == "--start"]; then
87 echo -e "${YELLOW}Starting all services...${NC}"
88 systemctl --user start ai-breadboard-server.service
89 systemctl --user start ai-breadboard-mcp-langchain.service
90 systemctl --user start ai-breadboard-mcp-gemini.service
91 echo -e "${GREEN} All services started${NC}"
92 echo ""
93
94 # Показать статус
95 echo -e "${BLUE}Service Status:${NC}"
96 systemctl --user status ai-breadboard-server.service || tr
 ue
97 echo ""
98 fi

```

```
99 |
100 | # Показать информацию о том как использовать
101 | echo -e "${BLUE}=====
 |===== ${NC}"
102 | echo -e "${BLUE} Next Steps${NC}"
103 | echo -e "${BLUE}=====
 |===== ${NC}"
104 | echo ""
105 | echo -e "To manage services, use:"
106 | echo -e " ${YELLOW}systemctl --user start ai-breadboard-serve
 |r.service${NC}"
107 | echo -e " ${YELLOW}systemctl --user status ai-breadboard-serv
 |er.service${NC}"
108 | echo -e " ${YELLOW}systemctl --user stop ai-breadboard-server
 |.service${NC}"
109 | echo -e " ${YELLOW}systemctl --user enable ai-breadboard-serv
 |er.service${NC}"
110 | echo ""
111 | echo -e "To view logs:"
112 | echo -e " ${YELLOW}journalctl --user -u ai-breadboard-server.
 |service -f${NC}"
113 | echo ""
114 | echo -e "To enable all services for auto-start:"
115 | echo -e " ${YELLOW}systemctl --user enable ai-breadboard-*.se
 |rvice${NC}"
116 | echo ""
117 | echo -e "For more info, see: ${YELLOW}systemd/README.md${NC}"
118 | echo ""
119 |
120 | echo -e "${GREEN}□ Installation complete!${NC}"
121 |
```

class\_index.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta http-equiv="Content-Type" content="text/html; charse
5 <title>Coverage report</title>
6 <link rel="icon" sizes="32x32" href="favicon_32_cb_5828477
7 <link rel="stylesheet" href="style_cb_8e611ae1.css" type="
8 <script src="coverage_html_cb_6fb7b396.js" defer></script>
9 </head>
10 <body class="indexfile">
11 <header>
12 <div class="content">
13 <h1>Coverage report:
14 0%
15 </h1>
16 <aside id="help_panel_wrapper">
17 <input id="help_panel_state" type="checkbox">
18 <label for="help_panel_state">
19
21 </label>
22 <div id="help_panel">
23 <p class="legend">Shortcuts on this page</p>
24 <div class="keyhelp">
25 <p>
26 <kbd>f</kbd>
27 <kbd>n</kbd>
28 <kbd>s</kbd>
29 <kbd>m</kbd>
30 <kbd>x</kbd>
31 <kbd>c</kbd>
32 change column sorting
33 </p>
```

```

33 <p>
34 <kbd>[</kbd>
35 <kbd>]</kbd>
36 prev/next file
37 </p>
38 <p>
39 <kbd>?</kbd> show/hide this hel
p
40 </p>
41 </div>
42 </div>
43 </aside>
44 <form id="filter_container">
45 <input id="filter" type="text" value="" placeholde
r="filter...">
46 <div>
47 <input id="hide100" type="checkbox" >
48 <label for="hide100">hide covered</label>
49 </div>
50 </form>
51 <h2>
52 Files
53 F
unctions
54 Classes
55 </h2>
56 <p class="text">
57 <a class="nav" href="https://coverage.readthedocs.
io/en/7.6.10">coverage.py v7.6.10,
58 created at 2026-09-04 19:37 +0300
59 </p>
60 </div>
61 </header>
62 <main id="index">
63 <table class="index" data-sortable>
64 <thead>
65 <tr class="tablehead" title="Click to sort">
66 <th id="file" class="name left" aria-sort="non

```

```

e" data-shortcut="f">File</th>
67 <th id="region" class="name left" aria-sort="none" data-default-sort-order="ascending" data-shortcut="n">class</th>
68 <th id="statements" aria-sort="none" data-default-sort-order="descending" data-shortcut="s">statements</th>
69 <th id="missing" aria-sort="none" data-default-sort-order="descending" data-shortcut="m">missing</th>
70 <th id="excluded" aria-sort="none" data-default-sort-order="descending" data-shortcut="x">excluded</th>
71 <th id="coverage" class="right" aria-sort="none" data-shortcut="c">coverage</th>
72 </tr>
73 </thead>
74 <tbody>
75 <tr class="region">
76 <td class="name left">scripts\cli__init__.py</td>
77 <td class="name left"><data value=''>(no class)</data></td>
78 <td>0</td>
79 <td>0</td>
80 <td>0</td>
81 <td class="right" data-ratio="0 0">100%</td>
82 </tr>
83 <tr class="region">
84 <td class="name left">scripts\cli\assist.py</td>
85 <td class="name left"><data value='AssistCLI'>AssistCLI</data></td>
86 <td>212</td>
87 <td>212</td>
88 <td>0</td>

```

```

89 | <td class="right" data-ratio="0 212">0%</td>
90 | </tr>
91 | <tr class="region">
92 | <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html">scripts\cli\assist.py</td>
93 | <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html"><data value=' '>(no c
lass)</data></td>
94 | <td>74</td>
95 | <td>74</td>
96 | <td>2</td>
97 | <td class="right" data-ratio="0 74">0%</td>
98 | </tr>
99 | <tr class="region">
100 | <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t28">scripts\cli\config.py</td>
101 | <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t28"><data value='ConfigManager'>ConfigMana
ger</data></td>
102 | <td>64</td>
103 | <td>64</td>
104 | <td>0</td>
105 | <td class="right" data-ratio="0 64">0%</td>
106 | </tr>
107 | <tr class="region">
108 | <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html">scripts\cli\config.py</td>
109 | <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html"><data value=' '>(no c
lass)</data></td>
110 | <td>14</td>
111 | <td>14</td>
112 | <td>0</td>
113 | <td class="right" data-ratio="0 14">0%</td>
114 | </tr>
115 | <tr class="region">
116 | <td class="name left"><a href="z_933a9655f16c4
9fb_installer_py.html#t41">scripts\cli\installer.py</td>

```



```

117 | <td class="name left"><a href="z_933a9655f16c4
 | 9fb_installer_py.html#t41"><data value='Language'>Language</da
 | ta></td>
118 | <td>0</td>
119 | <td>0</td>
120 | <td>0</td>
121 | <td class="right" data-ratio="0 0">100%</td>
122 | </tr>
123 | <tr class="region">
124 | <td class="name left"><a href="z_933a9655f16c4
 | 9fb_installer_py.html#t76">scripts\cli\installer.py</td>
125 | <td class="name left"><a href="z_933a9655f16c4
 | 9fb_installer_py.html#t76"><data value='Installer'>Installer</
 | data></td>
126 | <td>158</td>
127 | <td>158</td>
128 | <td>0</td>
129 | <td class="right" data-ratio="0 158">0%</td>
130 | </tr>
131 | <tr class="region">
132 | <td class="name left"><a href="z_933a9655f16c4
 | 9fb_installer_py.html">scripts\cli\installer.py</td>
133 | <td class="name left"><a href="z_933a9655f16c4
 | 9fb_installer_py.html"><data value=''>(n
 | o class)</data></td>
134 | <td>43</td>
135 | <td>43</td>
136 | <td>2</td>
137 | <td class="right" data-ratio="0 43">0%</td>
138 | </tr>
139 | <tr class="region">
140 | <td class="name left"><a href="z_933a9655f16c4
 | 9fb_paths_py.html#t32">scripts\cli\paths.py</td>
141 | <td class="name left"><a href="z_933a9655f16c4
 | 9fb_paths_py.html#t32"><data value='CrossPlatformPaths'>CrossP
 | latformPaths</data></td>
142 | <td>59</td>
143 | <td>59</td>

```

```

144 | <td>0</td>
145 | <td class="right" data-ratio="0 59">0%</td>
146 | </tr>
147 | <tr class="region">
148 | <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html">scripts\cli\paths.py</td>
149 | <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html"><data value=''>(no cl
ass)</data></td>
150 | <td>58</td>
151 | <td>58</td>
152 | <td>2</td>
153 | <td class="right" data-ratio="0 58">0%</td>
154 | </tr>
155 | <tr class="region">
156 | <td class="name left"><a href="z_933a9655f16c4
9fb_utils_py.html">scripts\cli\utils.py</td>
157 | <td class="name left"><a href="z_933a9655f16c4
9fb_utils_py.html"><data value=''>(no cl
ass)</data></td>
158 | <td>98</td>
159 | <td>98</td>
160 | <td>4</td>
161 | <td class="right" data-ratio="0 98">0%</td>
162 | </tr>
163 | </tbody>
164 | <tfoot>
165 | <tr class="total">
166 | <td class="name left">Total</td>
167 | <td class="name left"> </td>
168 | <td>780</td>
169 | <td>780</td>
170 | <td>10</td>
171 | <td class="right" data-ratio="0 780">0%</td>
172 | </tr>
173 | </tfoot>
174 | </table>
175 | <p id="no_rows">

```

```
176 | No items found using the specified filter.
177 | </p>
178 | </main>
179 | <footer>
180 | <div class="content">
181 | <p>
182 | <a class="nav" href="https://coverage.readthedocs.
183 | io/en/7.6.10">coverage.py v7.6.10,
184 | created at 2026-09-04 19:37 +0300
185 | </p>
186 | </div>
187 | <aside class="hidden">
188 |
189 |
190 | <button type="button" class="button_prev_file" data-sh
191 | ortcut="["></button>
192 | <button type="button" class="button_next_file" data-sh
193 | ortcut="]"></button>
194 | <button type="button" class="button_show_hide_help" da
195 | ta-shortcut="?"></button>
196 | </aside>
197 | </footer>
198 | </body>
199 | </html>
```

coverage\_html\_cb\_6fb7b396.js

```
1 // Licensed under the Apache License: http://www.apache.org/li
 | censes/LICENSE-2.0
2 // For details: https://github.com/nedbat/coveragepy/blob/mast
 | er/NOTICE.txt
3
4 // Coverage.py HTML report browser code.
5 /*jslint browser: true, sloppy: true, vars: true, plusplus: tr
 | ue, maxerr: 50, indent: 4 */
6 /*global coverage: true, document, window, $ */
7
8 coverage = {};
9
10 // General helpers
11 function debounce(callback, wait) {
12 let timeoutId = null;
13 return function(...args) {
14 clearTimeout(timeoutId);
15 timeoutId = setTimeout(() => {
16 callback.apply(this, args);
17 }, wait);
18 };
19 };
20
21 function checkVisible(element) {
22 const rect = element.getBoundingClientRect();
23 const viewBottom = Math.max(document.documentElement.clie
 | ntHeight, window.innerHeight);
24 const viewTop = 30;
25 return !(rect.bottom < viewTop || rect.top >= viewBottom);
26 }
27
28 function on_click(sel, fn) {
29 const elt = document.querySelector(sel);
30 if (elt) {
31 elt.addEventListener("click", fn);
32 }
```

```

33 | }
34 |
35 | // Helpers for table sorting
36 | function getCellValue(row, column = 0) {
37 | const cell = row.cells[column] // nosemgrep: eslint.dete
 | t-object-injection
38 | if (cell.childElementCount == 1) {
39 | var child = cell.firstElementChild;
40 | if (child.tagName === "A") {
41 | child = child.firstElementChild;
42 | }
43 | if (child instanceof HTMLDataElement && child.value) {
44 | return child.value;
45 | }
46 | }
47 | return cell.innerText || cell.textContent;
48 | }
49 |
50 | function rowComparator(rowA, rowB, column = 0) {
51 | let valueA = getCellValue(rowA, column);
52 | let valueB = getCellValue(rowB, column);
53 | if (!isNaN(valueA) && !isNaN(valueB)) {
54 | return valueA - valueB;
55 | }
56 | return valueA.localeCompare(valueB, undefined, {numeric: t
 | rue});
57 | }
58 |
59 | function sortColumn(th) {
60 | // Get the current sorting direction of the selected heade
 | r,
61 | // clear state on other headers and then set the new sorti
 | ng direction.
62 | const currentSortOrder = th.getAttribute("aria-sort");
63 | [...th.parentElement.cells].forEach(header => header.setAt
 | tribute("aria-sort", "none"));
64 | var direction;
65 | if (currentSortOrder === "none") {

```

```

66 | direction = th.dataset.defaultSortOrder || "ascending"
67 | };
68 | else if (currentSortOrder === "ascending") {
69 | direction = "descending";
70 | }
71 | else {
72 | direction = "ascending";
73 | }
74 | th.setAttribute("aria-sort", direction);
75 |
76 | const column = [...th.parentElement.cells].indexOf(th)
77 |
78 | // Sort all rows and afterwards append them in order to move them in the DOM.
79 | Array.from(th.closest("table").querySelectorAll("tbody tr"))
80 | .sort((rowA, rowB) => rowComparator(rowA, rowB, column) * (direction === "ascending" ? 1 : -1))
81 | .forEach(tr => tr.parentElement.appendChild(tr));
82 |
83 | // Save the sort order for next time.
84 | if (th.id !== "region") {
85 | let th_id = "file"; // Sort by file if we don't have a column id
86 | let current_direction = direction;
87 | const stored_list = localStorage.getItem(coverage.INDEX_SORT_STORAGE);
88 | if (stored_list) {
89 | ({th_id, direction} = JSON.parse(stored_list))
90 | }
91 | localStorage.setItem(coverage.INDEX_SORT_STORAGE, JSON.stringify({
92 | "th_id": th_id,
93 | "direction": current_direction
94 | }));
95 | if (th.id !== th_id || document.getElementById("region")) {

```

```

96 | // Sort column has changed, unset sorting by funct
 | ion or class.
97 | localStorage.setItem(coverage.SORTED_BY_REGION, JS
 | ON.stringify({
98 | "by_region": false,
99 | "region_direction": current_direction
100 | }));
101 | }
102 | }
103 | else {
104 | // Sort column has changed to by function or class, re
 | member that.
105 | localStorage.setItem(coverage.SORTED_BY_REGION, JSON.s
 | tringify({
106 | "by_region": true,
107 | "region_direction": direction
108 | }));
109 | }
110 | }
111 |
112 | // Find all the elements with data-shortcut attribute, and use
 | them to assign a shortcut key.
113 | coverage.assign_shortkeys = function () {
114 | document.querySelectorAll("[data-shortcut]").forEach(eleme
 | nt => {
115 | document.addEventListener("keypress", event => {
116 | if (event.target.tagName.toLowerCase() === "input"
 |) {
117 | return; // ignore keypress from search filter
118 | }
119 | if (event.key === element.dataset.shortcut) {
120 | element.click();
121 | }
122 | });
123 | });
124 | };
125 |
126 | // Create the events for the filter box.

```

```

127 coverage.wire_up_filter = function () {
128 // Populate the filter and hide100 inputs if there are sav
 ed values for them.
129 const saved_filter_value = localStorage.getItem(coverage.F
 ILTER_STORAGE);
130 if (saved_filter_value) {
131 document.getElementById("filter").value = saved_filter
 _value;
132 }
133 const saved_hide100_value = localStorage.getItem(coverage.
 HIDE100_STORAGE);
134 if (saved_hide100_value) {
135 document.getElementById("hide100").checked = JSON.pars
 e(saved_hide100_value);
136 }
137
138 // Cache elements.
139 const table = document.querySelector("table.index");
140 const table_body_rows = table.querySelectorAll("tbody tr")
 ;
141 const no_rows = document.getElementById("no_rows");
142
143 // Observe filter keyevents.
144 const filter_handler = (event => {
145 // Keep running total of each metric, first index cont
 ains number of shown rows
146 const totals = new Array(table.rows[0].cells.length).f
 ill(0);
147 // Accumulate the percentage as fraction
148 totals[totals.length - 1] = { "numer": 0, "denom": 0 }
 ; // nosemgrep: eslint.detect-object-injection
149
150 var text = document.getElementById("filter").value;
151 // Store filter value
152 localStorage.setItem(coverage.FILTER_STORAGE, text);
153 const casefold = (text === text.toLowerCase());
154 const hide100 = document.getElementById("hide100").che
 cked;

```



```

155 | // Store hide value.
156 | localStorage.setItem(coverage.HIDE100_STORAGE, JSON.st
ringify(hide100));
157 |
158 | // Hide / show elements.
159 | table_body_rows.forEach(row => {
160 | var show = false;
161 | // Check the text filter.
162 | for (let column = 0; column < totals.length; colum
n++) {
163 | cell = row.cells[column];
164 | if (cell.classList.contains("name")) {
165 | var celltext = cell.textContent;
166 | if (casefold) {
167 | celltext = celltext.toLowerCase();
168 | }
169 | if (celltext.includes(text)) {
170 | show = true;
171 | }
172 | }
173 | }
174 |
175 | // Check the "hide covered" filter.
176 | if (show && hide100) {
177 | const [numer, denom] = row.cells[row.cells.len
gth - 1].dataset.ratio.split(" ");
178 | show = (numer !== denom);
179 | }
180 |
181 | if (!show) {
182 | // hide
183 | row.classList.add("hidden");
184 | return;
185 | }
186 |
187 | // show
188 | row.classList.remove("hidden");
189 | totals[0]++;

```

```

190
191 for (let column = 0; column < totals.length; column
192 n++) {
193 // Accumulate dynamic totals
194 cell = row.cells[column] // nosemgrep: eslint
195 .detect-object-injection
196 if (cell.classList.contains("name")) {
197 continue;
198 }
199 if (column === totals.length - 1) {
200 // Last column contains percentage
201 const [numer, denom] = cell.dataset.ratio.
202 split(" ");
203 totals[column]["numer"] += parseInt(numer,
204 10); // nosemgrep: eslint.detect-object-injection
205 totals[column]["denom"] += parseInt(denom,
206 10); // nosemgrep: eslint.detect-object-injection
207 }
208 else {
209 totals[column] += parseInt(cell.textContent,
210 10); // nosemgrep: eslint.detect-object-injection
211 }
212 }
213 });
214
215 // Show placeholder if no rows will be displayed.
216 if (!totals[0]) {
217 // Show placeholder, hide table.
218 no_rows.style.display = "block";
219 table.style.display = "none";
220 return;
221 }
222
223 // Hide placeholder, show table.
224 no_rows.style.display = null;
225 table.style.display = null;
226
227 const footer = table.tFoot.rows[0];

```

```

222 | // Calculate new dynamic sum values based on visible r
 | ows.
223 | for (let column = 0; column < totals.length; column++)
 | {
224 | // Get footer cell element.
225 | const cell = footer.cells[column]; // nosemgrep:
 | eslint.detect-object-injection
226 | if (cell.classList.contains("name")) {
227 | continue;
228 | }
229 |
230 | // Set value into dynamic footer cell element.
231 | if (column === totals.length - 1) {
232 | // Percentage column uses the numerator and de
 | nominator,
233 | // and adapts to the number of decimal places.
234 | const match = /\s*([0-9]+)/.exec(cell.textContent);
235 | const places = match ? match[1].length : 0;
236 | const { numer, denom } = totals[column]; // n
 | osemgrep: eslint.detect-object-injection
237 | cell.dataset.ratio = `${numer} ${denom}`;
238 | // Check denom to prevent NaN if filtered file
 | s contain no statements
239 | cell.textContent = denom
240 | ? `${(numer * 100 / denom).toFixed(places)}%`
 | : `${(100).toFixed(places)}%`;
241 | }
242 | else {
243 | cell.textContent = totals[column]; // nosemgr
 | ep: eslint.detect-object-injection
244 | }
245 | }
246 | });
247 |
248 |
249 | document.getElementById("filter").addEventListener("input"
 | , debounce(filter_handler));

```

```

250 document.getElementById("hide100").addEventListener("input
 ", debounce(filter_handler));
251
252 // Trigger change event on setup, to force filter on page
 refresh
253 // (filter value may still be present).
254 document.getElementById("filter").dispatchEvent(new Event(
 "input"));
255 document.getElementById("hide100").dispatchEvent(new Event
 ("input"));
256 };
257 coverage.FILTER_STORAGE = "COVERAGE_FILTER_VALUE";
258 coverage.HIDE100_STORAGE = "COVERAGE_HIDE100_VALUE";
259
260 // Set up the click-to-sort columns.
261 coverage.wire_up_sorting = function () {
262 document.querySelectorAll("[data-sortable] th[aria-sort]")
 .forEach(
263 th => th.addEventListener("click", e => sortColumn(e.t
 arget))
264);
265
266 // Look for a localStorage item containing previous sort s
 ettings:
267 let th_id = "file", direction = "ascending";
268 const stored_list = localStorage.getItem(coverage.INDEX_SO
 RT_STORAGE);
269 if (stored_list) {
270 ({th_id, direction} = JSON.parse(stored_list));
271 }
272 let by_region = false, region_direction = "ascending";
273 const sorted_by_region = localStorage.getItem(coverage.SOR
 TED_BY_REGION);
274 if (sorted_by_region) {
275 ({
276 by_region,
277 region_direction
278 } = JSON.parse(sorted_by_region));

```

```

279 }
280
281 const region_id = "region";
282 if (by_region && document.getElementById(region_id)) {
283 direction = region_direction;
284 }
285 // If we are in a page that has a column with id of "region
n", sort on
286 // it if the last sort was by function or class.
287 let th;
288 if (document.getElementById(region_id)) {
289 th = document.getElementById(by_region ? region_id : t
h_id);
290 }
291 else {
292 th = document.getElementById(th_id);
293 }
294 th.setAttribute("aria-sort", direction === "ascending" ? "
descending" : "ascending");
295 th.click()
296 };
297
298 coverage.INDEX_SORT_STORAGE = "COVERAGE_INDEX_SORT_2";
299 coverage.SORTED_BY_REGION = "COVERAGE_SORT_REGION";
300
301 // Loaded on index.html
302 coverage.index_ready = function () {
303 coverage.assign_shortkeys();
304 coverage.wire_up_filter();
305 coverage.wire_up_sorting();
306
307 on_click(".button_prev_file", coverage.to_prev_file);
308 on_click(".button_next_file", coverage.to_next_file);
309
310 on_click(".button_show_hide_help", coverage.show_hide_help
);
311 };
312

```

```

313 // -- pyfile stuff --
314
315 coverage.LINE_FILTERS_STORAGE = "COVERAGE_LINE_FILTERS";
316
317 coverage.pyfile_ready = function () {
318 // If we're directed to a particular line number, highlight
 t the line.
319 var frag = location.hash;
320 if (frag.length > 2 && frag[1] === "t") {
321 document.querySelector(frag).closest(".n").classList.a
 dd("highlight");
322 coverage.set_sel(parseInt(frag.substr(2), 10));
323 }
324 else {
325 coverage.set_sel(0);
326 }
327
328 on_click(".button_toggle_run", coverage.toggle_lines);
329 on_click(".button_toggle_mis", coverage.toggle_lines);
330 on_click(".button_toggle_exc", coverage.toggle_lines);
331 on_click(".button_toggle_par", coverage.toggle_lines);
332
333 on_click(".button_next_chunk", coverage.to_next_chunk_nice
 ly);
334 on_click(".button_prev_chunk", coverage.to_prev_chunk_nice
 ly);
335 on_click(".button_top_of_page", coverage.to_top);
336 on_click(".button_first_chunk", coverage.to_first_chunk);
337
338 on_click(".button_prev_file", coverage.to_prev_file);
339 on_click(".button_next_file", coverage.to_next_file);
340 on_click(".button_to_index", coverage.to_index);
341
342 on_click(".button_show_hide_help", coverage.show_hide_help
);
343
344 coverage.filters = undefined;
345 try {

```

```

346 | coverage.filters = localStorage.getItem(coverage.LINE_
| FILTERS_STORAGE);
347 | } catch(err) {}
348 |
349 | if (coverage.filters) {
350 | coverage.filters = JSON.parse(coverage.filters);
351 | }
352 | else {
353 | coverage.filters = {run: false, exc: true, mis: true,
| par: true};
354 | }
355 |
356 | for (cls in coverage.filters) {
357 | coverage.set_line_visibilty(cls, coverage.filters[cls]
|); // nosemgrep: eslint.detect-object-injection
358 | }
359 |
360 | coverage.assign_shortkeys();
361 | coverage.init_scroll_markers();
362 | coverage.wire_up_sticky_header();
363 |
364 | document.querySelectorAll("[id^=ctxs]").forEach(
365 | cbox => cbox.addEventListener("click", coverage.expand
| _contexts)
366 |);
367 |
368 | // Rebuild scroll markers when the window height changes.
369 | window.addEventListener("resize", coverage.build_scroll_ma
| rkers);
370 | };
371 |
372 | coverage.toggle_lines = function (event) {
373 | const btn = event.target.closest("button");
374 | const category = btn.value
375 | const show = !btn.classList.contains("show_" + category);
376 | coverage.set_line_visibilty(category, show);
377 | coverage.build_scroll_markers();
378 | coverage.filters[category] = show;

```

```

379 try {
380 localStorage.setItem(coverage.LINE_FILTERS_STORAGE, JS
ON.stringify(coverage.filters));
381 } catch(err) {}
382 };
383
384 coverage.set_line_visibilty = function (category, should_show)
 {
385 const cls = "show_" + category;
386 const btn = document.querySelector(".button_toggle_" + cat
egory);
387 if (btn) {
388 if (should_show) {
389 document.querySelectorAll("#source ." + category).
forEach(e => e.classList.add(cls));
390 btn.classList.add(cls);
391 }
392 else {
393 document.querySelectorAll("#source ." + category).
forEach(e => e.classList.remove(cls));
394 btn.classList.remove(cls);
395 }
396 }
397 };
398
399 // Return the nth line div.
400 coverage.line_elt = function (n) {
401 return document.getElementById("t" + n)?.closest("p");
402 };
403
404 // Set the selection. b and e are line numbers.
405 coverage.set_sel = function (b, e) {
406 // The first line selected.
407 coverage.sel_begin = b;
408 // The next line not selected.
409 coverage.sel_end = (e === undefined) ? b+1 : e;
410 };
411

```



```
412 coverage.to_top = function () {
413 coverage.set_sel(0, 1);
414 coverage.scroll_window(0);
415 };
416
417 coverage.to_first_chunk = function () {
418 coverage.set_sel(0, 1);
419 coverage.to_next_chunk();
420 };
421
422 coverage.to_prev_file = function () {
423 window.location = document.getElementById("prevFileLink").
424 href;
425 }
426
427 coverage.to_next_file = function () {
428 window.location = document.getElementById("nextFileLink").
429 href;
430 }
431
432 coverage.to_index = function () {
433 location.href = document.getElementById("indexLink").href;
434 }
435
436 coverage.show_hide_help = function () {
437 const helpCheck = document.getElementById("help_panel_stat
438 e")
439 helpCheck.checked = !helpCheck.checked;
440 }
441
442 // Return a string indicating what kind of chunk this line bel
443 // ongs to,
444 // or null if not a chunk.
445 coverage.chunk_indicator = function (line_elt) {
446 const classes = line_elt?.className;
447 if (!classes) {
448 return null;
449 }
450 }
```

```
446 | const match = classes.match(/\\bshow_\\w+\\b/);
447 | if (!match) {
448 | return null;
449 | }
450 | return match[0];
451 | };
452 |
453 | coverage.to_next_chunk = function () {
454 | const c = coverage;
455 |
456 | // Find the start of the next colored chunk.
457 | var probe = c.sel_end;
458 | var chunk_indicator, probe_line;
459 | while (true) {
460 | probe_line = c.line_elt(probe);
461 | if (!probe_line) {
462 | return;
463 | }
464 | chunk_indicator = c.chunk_indicator(probe_line);
465 | if (chunk_indicator) {
466 | break;
467 | }
468 | probe++;
469 | }
470 |
471 | // There's a next chunk, `probe` points to it.
472 | var begin = probe;
473 |
474 | // Find the end of this chunk.
475 | var next_indicator = chunk_indicator;
476 | while (next_indicator === chunk_indicator) {
477 | probe++;
478 | probe_line = c.line_elt(probe);
479 | next_indicator = c.chunk_indicator(probe_line);
480 | }
481 | c.set_sel(begin, probe);
482 | c.show_selection();
483 | };
```

```

484
485 coverage.to_prev_chunk = function () {
486 const c = coverage;
487
488 // Find the end of the prev colored chunk.
489 var probe = c.sel_begin-1;
490 var probe_line = c.line_elt(probe);
491 if (!probe_line) {
492 return;
493 }
494 var chunk_indicator = c.chunk_indicator(probe_line);
495 while (probe > 1 && !chunk_indicator) {
496 probe--;
497 probe_line = c.line_elt(probe);
498 if (!probe_line) {
499 return;
500 }
501 chunk_indicator = c.chunk_indicator(probe_line);
502 }
503
504 // There's a prev chunk, `probe` points to its last line.
505 var end = probe+1;
506
507 // Find the beginning of this chunk.
508 var prev_indicator = chunk_indicator;
509 while (prev_indicator === chunk_indicator) {
510 probe--;
511 if (probe <= 0) {
512 return;
513 }
514 probe_line = c.line_elt(probe);
515 prev_indicator = c.chunk_indicator(probe_line);
516 }
517 c.set_sel(probe+1, end);
518 c.show_selection();
519 };
520
521 // Returns 0, 1, or 2: how many of the two ends of the selecti

```

```

on are on
522 // the screen right now?
523 coverage.selection_ends_on_screen = function () {
524 if (coverage.sel_begin === 0) {
525 return 0;
526 }
527
528 const begin = coverage.line_elt(coverage.sel_begin);
529 const end = coverage.line_elt(coverage.sel_end-1);
530
531 return (
532 (checkVisible(begin) ? 1 : 0)
533 + (checkVisible(end) ? 1 : 0)
534);
535 };
536
537 coverage.to_next_chunk_nicely = function () {
538 if (coverage.selection_ends_on_screen() === 0) {
539 // The selection is entirely off the screen:
540 // Set the top line on the screen as selection.
541
542 // This will select the top-left of the viewport
543 // As this is most likely the span with the line number
544 // we take the parent
545 const line = document.elementFromPoint(0, 0).parentElement;
546 if (line.parentElement !== document.getElementById("source")) {
547 // The element is not a source line but the header
548 // or similar
549 coverage.select_line_or_chunk(1);
550 }
551 else {
552 // We extract the line number from the id
553 coverage.select_line_or_chunk(parseInt(line.id.substring(1), 10));
554 }
555 }
556 }

```

```

554 coverage.to_next_chunk();
555 };
556
557 coverage.to_prev_chunk_nicely = function () {
558 if (coverage.selection_ends_on_screen() === 0) {
559 // The selection is entirely off the screen:
560 // Set the lowest line on the screen as selection.
561
562 // This will select the bottom-left of the viewport
563 // As this is most likely the span with the line number
564 // we take the parent
565 const line = document.elementFromPoint(document.documentElement.clientHeight-1, 0).parentElement;
566 if (line.parentElement !== document.getElementById("source")) {
567 // The element is not a source line but the header
568 // or similar
569 coverage.select_line_or_chunk(coverage.lines_len);
570 }
571 else {
572 // We extract the line number from the id
573 coverage.select_line_or_chunk(parseInt(line.id.substring(1), 10));
574 }
575 }
576 coverage.to_prev_chunk();
577 };
578
579 // Select line number lineno, or if it is in a colored chunk,
580 // select the
581 // entire chunk
582 coverage.select_line_or_chunk = function (lineno) {
583 var c = coverage;
584 var probe_line = c.line_elt(lineno);
585 if (!probe_line) {
586 return;
587 }
588 var the_indicator = c.chunk_indicator(probe_line);

```

```
586 | if (the_indicator) {
587 | // The line is in a highlighted chunk.
588 | // Search backward for the first line.
589 | var probe = lineno;
590 | var indicator = the_indicator;
591 | while (probe > 0 && indicator === the_indicator) {
592 | probe--;
593 | probe_line = c.line_elt(probe);
594 | if (!probe_line) {
595 | break;
596 | }
597 | indicator = c.chunk_indicator(probe_line);
598 | }
599 | var begin = probe + 1;
600 |
601 | // Search forward for the last line.
602 | probe = lineno;
603 | indicator = the_indicator;
604 | while (indicator === the_indicator) {
605 | probe++;
606 | probe_line = c.line_elt(probe);
607 | indicator = c.chunk_indicator(probe_line);
608 | }
609 |
610 | coverage.set_sel(begin, probe);
611 | }
612 | else {
613 | coverage.set_sel(lineno);
614 | }
615 | };
616 |
617 | coverage.show_selection = function () {
618 | // Highlight the lines in the chunk
619 | document.querySelectorAll("#source .highlight").forEach(e
620 | => e.classList.remove("highlight"));
621 | for (let probe = coverage.sel_begin; probe < coverage.sel_
end; probe++) {
 coverage.line_elt(probe).querySelector(".n").classList
```

```

 .add("highlight");
622 }
623
624 coverage.scroll_to_selection();
625 };
626
627 coverage.scroll_to_selection = function () {
628 // Scroll the page if the chunk isn't fully visible.
629 if (coverage.selection_ends_on_screen() < 2) {
630 const element = coverage.line_elt(coverage.sel_begin);
631 coverage.scroll_window(element.offsetTop - 60);
632 }
633 };
634
635 coverage.scroll_window = function (to_pos) {
636 window.scroll({top: to_pos, behavior: "smooth"});
637 };
638
639 coverage.init_scroll_markers = function () {
640 // Init some variables
641 coverage.lines_len = document.querySelectorAll("#source >
p").length;
642
643 // Build html
644 coverage.build_scroll_markers();
645 };
646
647 coverage.build_scroll_markers = function () {
648 const temp_scroll_marker = document.getElementById("scroll
_marker")
649 if (temp_scroll_marker) temp_scroll_marker.remove();
650 // Don't build markers if the window has no scroll bar.
651 if (document.body.scrollHeight <= window.innerHeight) {
652 return;
653 }
654
655 const marker_scale = window.innerHeight / document.body.sc
rollHeight;

```

```

656 | const line_height = Math.min(Math.max(3, window.innerHeigh
 | t / coverage.lines_len), 10);
657 |
658 | let previous_line = -99, last_mark, last_top;
659 |
660 | const scroll_marker = document.createElement("div");
661 | scroll_marker.id = "scroll_marker";
662 | document.getElementById("source").querySelectorAll(
663 | "p.show_run, p.show_mis, p.show_exc, p.show_exc, p.sho
 | w_par"
664 |).forEach(element => {
665 | const line_top = Math.floor(element.offsetTop * marker
 | _scale);
666 | const line_number = parseInt(element.querySelector(".n
 | a").id.substr(1));
667 |
668 | if (line_number === previous_line + 1) {
669 | // If this solid missed block just make previous m
 | ark higher.
670 | last_mark.style.height = `${line_top + line_height
 | - last_top}px`;
671 | }
672 | else {
673 | // Add colored line in scroll_marker block.
674 | last_mark = document.createElement("div");
675 | last_mark.id = `m${line_number}`;
676 | last_mark.classList.add("marker");
677 | last_mark.style.height = `${line_height}px`;
678 | last_mark.style.top = `${line_top}px`;
679 | scroll_marker.append(last_mark);
680 | last_top = line_top;
681 | }
682 |
683 | previous_line = line_number;
684 | });
685 |
686 | // Append last to prevent layout calculation
687 | document.body.append(scroll_marker);

```



```

688 };
689
690 coverage.wire_up_sticky_header = function () {
691 const header = document.querySelector("header");
692 const header_bottom = (
693 header.querySelector(".content h2").getBoundingClientRect().top -
694 header.getBoundingClientRect().top
695);
696
697 function updateHeader() {
698 if (window.scrollY > header_bottom) {
699 header.classList.add("sticky");
700 }
701 else {
702 header.classList.remove("sticky");
703 }
704 }
705
706 window.addEventListener("scroll", updateHeader);
707 updateHeader();
708 };
709
710 coverage.expand_contexts = function (e) {
711 var ctxs = e.target.parentNode.querySelector(".ctxs");
712
713 if (!ctxs.classList.contains("expanded")) {
714 var ctxs_text = ctxs.textContent;
715 var width = Number(ctxs_text[0]);
716 ctxs.textContent = "";
717 for (var i = 1; i < ctxs_text.length; i += width) {
718 key = ctxs_text.substring(i, i + width).trim();
719 ctxs.appendChild(document.createTextNode(contexts[
720 key]));
721 ctxs.appendChild(document.createElement("br"));
722 }
723 ctxs.classList.add("expanded");
724 }

```

```
724 | };
725 |
726 | document.addEventListener("DOMContentLoaded", () => {
727 | if (document.body.classList.contains("indexfile")) {
728 | coverage.index_ready();
729 | }
730 | else {
731 | coverage.pyfile_ready();
732 | }
733 | });
734 |
```

## function\_index.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta http-equiv="Content-Type" content="text/html; charse
5 <title>Coverage report</title>
6 <link rel="icon" sizes="32x32" href="favicon_32_cb_5828477
7 <link rel="stylesheet" href="style_cb_8e611ae1.css" type="
8 <script src="coverage_html_cb_6fb7b396.js" defer></script>
9 </head>
10 <body class="indexfile">
11 <header>
12 <div class="content">
13 <h1>Coverage report:
14 0%
15 </h1>
16 <aside id="help_panel_wrapper">
17 <input id="help_panel_state" type="checkbox">
18 <label for="help_panel_state">
19
21 </label>
22 <div id="help_panel">
23 <p class="legend">Shortcuts on this page</p>
24 <div class="keyhelp">
25 <p>
26 <kbd>f</kbd>
27 <kbd>n</kbd>
28 <kbd>s</kbd>
29 <kbd>m</kbd>
30 <kbd>x</kbd>
31 <kbd>c</kbd>
32 change column sorting
33 </p>
```

```

33 <p>
34 <kbd>[</kbd>
35 <kbd>]</kbd>
36 prev/next file
37 </p>
38 <p>
39 <kbd>?</kbd> show/hide this hel
p
40 </p>
41 </div>
42 </div>
43 </aside>
44 <form id="filter_container">
45 <input id="filter" type="text" value="" placeholde
r="filter...">
46 <div>
47 <input id="hide100" type="checkbox" >
48 <label for="hide100">hide covered</label>
49 </div>
50 </form>
51 <h2>
52 Files
53 Functions
54 Clas
ses
55 </h2>
56 <p class="text">
57 <a class="nav" href="https://coverage.readthedocs.
io/en/7.6.10">coverage.py v7.6.10,
58 created at 2026-09-04 19:37 +0300
59 </p>
60 </div>
61 </header>
62 <main id="index">
63 <table class="index" data-sortable>
64 <thead>
65 <tr class="tablehead" title="Click to sort">
66 <th id="file" class="name left" aria-sort="non

```

```

e" data-shortcut="f">File</th>
67 <th id="region" class="name left" aria-sort="none" data-default-sort-order="ascending" data-shortcut="n">function</th>
68 <th id="statements" aria-sort="none" data-default-sort-order="descending" data-shortcut="s">statements</th>
69 <th id="missing" aria-sort="none" data-default-sort-order="descending" data-shortcut="m">missing</th>
70 <th id="excluded" aria-sort="none" data-default-sort-order="descending" data-shortcut="x">excluded</th>
71 <th id="coverage" class="right" aria-sort="none" data-shortcut="c">coverage</th>
72 </tr>
73 </thead>
74 <tbody>
75 <tr class="region">
76 <td class="name left">scripts\cli__init__.py</td>
77 <td class="name left"><data value=''>(no function)</data></td>
78 <td>0</td>
79 <td>0</td>
80 <td>0</td>
81 <td class="right" data-ratio="0 0">100%</td>
82 </tr>
83 <tr class="region">
84 <td class="name left">scripts\cli\assist.py</td>
85 <td class="name left"><data value='init__'>AssistCLI.__init__</data></td>
86 <td>4</td>
87 <td>4</td>
88 <td>0</td>

```

```

89 <td class="right" data-ratio="0 4">0%</td>
90 </tr>
91 <tr class="region">
92 <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t74">scripts\cli\assist.py</td>
93 <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t74"><data value='ensure_state_dir'>AssistC
LI._ensure_state_dir</data></td>
94 <td>1</td>
95 <td>1</td>
96 <td>0</td>
97 <td class="right" data-ratio="0 1">0%</td>
98 </tr>
99 <tr class="region">
100 <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t78">scripts\cli\assist.py</td>
101 <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t78"><data value='get_state_file'>AssistCLI
._get_state_file</data></td>
102 <td>1</td>
103 <td>1</td>
104 <td>0</td>
105 <td class="right" data-ratio="0 1">0%</td>
106 </tr>
107 <tr class="region">
108 <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t82">scripts\cli\assist.py</td>
109 <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t82"><data value='load_state'>AssistCLI._lo
ad_state</data></td>
110 <td>8</td>
111 <td>8</td>
112 <td>0</td>
113 <td class="right" data-ratio="0 8">0%</td>
114 </tr>
115 <tr class="region">
116 <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t93">scripts\cli\assist.py</td>

```

```

117 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_assist_py.html#t93"><data value='save_state'>AssistCLI._sa
| ve_state</data></td>
118 | <td>4</td>
119 | <td>4</td>
120 | <td>0</td>
121 | <td class="right" data-ratio="0 4">0%</td>
122 | </tr>
123 | <tr class="region">
124 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_assist_py.html#t100">scripts\cli\assist.py</td>
125 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_assist_py.html#t100"><data value='cmd_start'>AssistCLI.cmd
| _start</data></td>
126 | <td>19</td>
127 | <td>19</td>
128 | <td>0</td>
129 | <td class="right" data-ratio="0 19">0%</td>
130 | </tr>
131 | <tr class="region">
132 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_assist_py.html#t141">scripts\cli\assist.py</td>
133 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_assist_py.html#t141"><data value='cmd_stop'>AssistCLI.cmd_
| stop</data></td>
134 | <td>21</td>
135 | <td>21</td>
136 | <td>0</td>
137 | <td class="right" data-ratio="0 21">0%</td>
138 | </tr>
139 | <tr class="region">
140 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_assist_py.html#t174">scripts\cli\assist.py</td>
141 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_assist_py.html#t174"><data value='cmd_restart'>AssistCLI.c
| md_restart</data></td>
142 | <td>3</td>
143 | <td>3</td>

```

```

144 <td>0</td>
145 <td class="right" data-ratio="0 3">0%</td>
146 </tr>
147 <tr class="region">
148 <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t180">scripts\cli\assist.py</td>
149 <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t180"><data value='cmd_status'>AssistCLI.cm
d_status</data></td>
150 <td>32</td>
151 <td>32</td>
152 <td>0</td>
153 <td class="right" data-ratio="0 32">0%</td>
154 </tr>
155 <tr class="region">
156 <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t228">scripts\cli\assist.py</td>
157 <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t228"><data value='cmd_providers'>AssistCLI
.cmd_providers</data></td>
158 <td>45</td>
159 <td>45</td>
160 <td>0</td>
161 <td class="right" data-ratio="0 45">0%</td>
162 </tr>
163 <tr class="region">
164 <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t288">scripts\cli\assist.py</td>
165 <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t288"><data value='cmd_logs'>AssistCLI.cmd_
logs</data></td>
166 <td>29</td>
167 <td>29</td>
168 <td>0</td>
169 <td class="right" data-ratio="0 29">0%</td>
170 </tr>
171 <tr class="region">
172 <td class="name left"><a href="z_933a9655f16c4

```



```

9fb_assist_py.html#t326">scripts\cli\assist.py</td>
173 | <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t326"><data value='cmd_config'>AssistCLI.cm
d_config</data></td>
174 | <td>41</td>
175 | <td>41</td>
176 | <td>0</td>
177 | <td class="right" data-ratio="0 41">0%</td>
178 | </tr>
179 | <tr class="region">
180 | <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t383">scripts\cli\assist.py</td>
181 | <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t383"><data value='cmd_test'>AssistCLI.cmd_
test</data></td>
182 | <td>4</td>
183 | <td>4</td>
184 | <td>0</td>
185 | <td class="right" data-ratio="0 4">0%</td>
186 | </tr>
187 | <tr class="region">
188 | <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t394">scripts\cli\assist.py</td>
189 | <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html#t394"><data value='main'>main</data></t
d>
190 | <td>40</td>
191 | <td>40</td>
192 | <td>0</td>
193 | <td class="right" data-ratio="0 40">0%</td>
194 | </tr>
195 | <tr class="region">
196 | <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html">scripts\cli\assist.py</td>
197 | <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html"><data value=''>(no f
unction)</data></td>
198 | <td>34</td>

```

```

199 <td>34</td>
200 <td>2</td>
201 <td class="right" data-ratio="0 34">0%</td>
202 </tr>
203 <tr class="region">
204 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t35">scripts\cli\config.py</td>
205 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t35"><data value='init__'>ConfigManager.__i
nit__</data></td>
206 <td>3</td>
207 <td>3</td>
208 <td>0</td>
209 <td class="right" data-ratio="0 3">0%</td>
210 </tr>
211 <tr class="region">
212 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t41">scripts\cli\config.py</td>
213 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t41"><data value='load_json'>ConfigManager.
load_json</data></td>
214 <td>8</td>
215 <td>8</td>
216 <td>0</td>
217 <td class="right" data-ratio="0 8">0%</td>
218 </tr>
219 <tr class="region">
220 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t60">scripts\cli\config.py</td>
221 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t60"><data value='save_json'>ConfigManager.
save_json</data></td>
222 <td>8</td>
223 <td>8</td>
224 <td>0</td>
225 <td class="right" data-ratio="0 8">0%</td>
226 </tr>
227 <tr class="region">

```

```

228 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t85">scripts\cli\config.py</td>
229 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t85"><data value='load_env_file'>ConfigMana
ger.load_env_file</data></td>
230 <td>15</td>
231 <td>15</td>
232 <td>0</td>
233 <td class="right" data-ratio="0 15">0%</td>
234 </tr>
235 <tr class="region">
236 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t115">scripts\cli\config.py</td>
237 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t115"><data value='save_env_file'>ConfigMan
ager.save_env_file</data></td>
238 <td>9</td>
239 <td>9</td>
240 <td>0</td>
241 <td class="right" data-ratio="0 9">0%</td>
242 </tr>
243 <tr class="region">
244 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t135">scripts\cli\config.py</td>
245 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t135"><data value='get_config'>ConfigManage
r.get_config</data></td>
246 <td>8</td>
247 <td>8</td>
248 <td>0</td>
249 <td class="right" data-ratio="0 8">0%</td>
250 </tr>
251 <tr class="region">
252 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t160">scripts\cli\config.py</td>
253 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t160"><data value='set_config'>ConfigManage
r.set_config</data></td>

```

```

254 <td>9</td>
255 <td>9</td>
256 <td>0</td>
257 <td class="right" data-ratio="0 9">0%</td>
258 </tr>
259 <tr class="region">
260 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t185">scripts\cli\config.py</td>
261 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html#t185"><data value='merge_env_to_config'>Con
figManager.merge_env_to_config</data></td>
262 <td>4</td>
263 <td>4</td>
264 <td>0</td>
265 <td class="right" data-ratio="0 4">0%</td>
266 </tr>
267 <tr class="region">
268 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html">scripts\cli\config.py</td>
269 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html"><data value=''>(no f
unction)</data></td>
270 <td>14</td>
271 <td>14</td>
272 <td>0</td>
273 <td class="right" data-ratio="0 14">0%</td>
274 </tr>
275 <tr class="region">
276 <td class="name left"><a href="z_933a9655f16c4
9fb_installer_py.html#t79">scripts\cli\installer.py</td>
277 <td class="name left"><a href="z_933a9655f16c4
9fb_installer_py.html#t79"><data value='init__'>Installer.__in
it__</data></td>
278 <td>5</td>
279 <td>5</td>
280 <td>0</td>
281 <td class="right" data-ratio="0 5">0%</td>
282 </tr>

```

```

283 | <tr class="region">
284 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_installer_py.html#t86">scripts\cli\installer.py</td>
285 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_installer_py.html#t86"><data value='msg'>Installer.msg</da
| ta></td>
286 | <td>1</td>
287 | <td>1</td>
288 | <td>0</td>
289 | <td class="right" data-ratio="0 1">0%</td>
290 | </tr>
291 | <tr class="region">
292 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_installer_py.html#t91">scripts\cli\installer.py</td>
293 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_installer_py.html#t91"><data value='get_default_install_di
| r'>Installer._get_default_install_dir</data></td>
294 | <td>4</td>
295 | <td>4</td>
296 | <td>0</td>
297 | <td class="right" data-ratio="0 4">0%</td>
298 | </tr>
299 | <tr class="region">
300 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_installer_py.html#t101">scripts\cli\installer.py</td>
301 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_installer_py.html#t101"><data value='print_header'>Install
| er.print_header</data></td>
302 | <td>7</td>
303 | <td>7</td>
304 | <td>0</td>
305 | <td class="right" data-ratio="0 7">0%</td>
306 | </tr>
307 | <tr class="region">
308 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_installer_py.html#t111">scripts\cli\installer.py</td>
309 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_installer_py.html#t111"><data value='create_venv'>Installe

```

```

r.create_venv</data></td>
310 <td>15</td>
311 <td>15</td>
312 <td>0</td>
313 <td class="right" data-ratio="0 15">0%</td>
314 </tr>
315 <tr class="region">
316 <td class="name left"><a href="z_933a9655f16c4
9fb_installer_py.html#t138">scripts\cli\installer.py</td>
317 <td class="name left"><a href="z_933a9655f16c4
9fb_installer_py.html#t138"><data value='install_dependencies'
>Installer.install_dependencies</data></td>
318 <td>25</td>
319 <td>25</td>
320 <td>0</td>
321 <td class="right" data-ratio="0 25">0%</td>
322 </tr>
323 <tr class="region">
324 <td class="name left"><a href="z_933a9655f16c4
9fb_installer_py.html#t184">scripts\cli\installer.py</td>
325 <td class="name left"><a href="z_933a9655f16c4
9fb_installer_py.html#t184"><data value='setup_ssl_certificate
s'>Installer.setup_ssl_certificates</data></td>
326 <td>18</td>
327 <td>18</td>
328 <td>0</td>
329 <td class="right" data-ratio="0 18">0%</td>
330 </tr>
331 <tr class="region">
332 <td class="name left"><a href="z_933a9655f16c4
9fb_installer_py.html#t222">scripts\cli\installer.py</td>
333 <td class="name left"><a href="z_933a9655f16c4
9fb_installer_py.html#t222"><data value='configure_cli'>Instal
ler.configure_cli</data></td>
334 <td>17</td>
335 <td>17</td>
336 <td>0</td>
337 <td class="right" data-ratio="0 17">0%</td>

```

```
338 | </tr>
339 | <tr class="region">
340 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_installer_py.html#t250">scripts\cli\installer.py</td>
341 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_installer_py.html#t250"><data value='verify_installation'>
| Installer.verify_installation</data></td>
342 | <td>25</td>
343 | <td>25</td>
344 | <td>0</td>
345 | <td class="right" data-ratio="0 25">0%</td>
346 | </tr>
347 | <tr class="region">
348 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_installer_py.html#t296">scripts\cli\installer.py</td>
349 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_installer_py.html#t296"><data value='download_models'>Inst
| aller.download_models</data></td>
350 | <td>9</td>
351 | <td>9</td>
352 | <td>0</td>
353 | <td class="right" data-ratio="0 9">0%</td>
354 | </tr>
355 | <tr class="region">
356 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_installer_py.html#t311">scripts\cli\installer.py</td>
357 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_installer_py.html#t311"><data value='run'>Installer.run</d
| ata></td>
358 | <td>32</td>
359 | <td>32</td>
360 | <td>0</td>
361 | <td class="right" data-ratio="0 32">0%</td>
362 | </tr>
363 | <tr class="region">
364 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_installer_py.html#t363">scripts\cli\installer.py</td>
365 | <td class="name left"><a href="z_933a9655f16c4
```

```

9fb_installer_py.html#t363"><data value='main'>main</data>
</td>
366 <td>10</td>
367 <td>10</td>
368 <td>0</td>
369 <td class="right" data-ratio="0 10">0%</td>
370 </tr>
371 <tr class="region">
372 <td class="name left"><a href="z_933a9655f16c4
9fb_installer_py.html">scripts\cli\installer.py</td>
373 <td class="name left"><a href="z_933a9655f16c4
9fb_installer_py.html"><data value=''>(n
o function)</data></td>
374 <td>33</td>
375 <td>33</td>
376 <td>2</td>
377 <td class="right" data-ratio="0 33">0%</td>
378 </tr>
379 <tr class="region">
380 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t40">scripts\cli\paths.py</td>
381 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t40"><data value='init__'>CrossPlatformPaths
.__init__</data></td>
382 <td>4</td>
383 <td>4</td>
384 <td>0</td>
385 <td class="right" data-ratio="0 4">0%</td>
386 </tr>
387 <tr class="region">
388 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t53">scripts\cli\paths.py</td>
389 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t53"><data value='find_project_root'>CrossPl
atformPaths._find_project_root</data></td>
390 <td>6</td>
391 <td>6</td>
392 <td>0</td>

```



```

393 | <td class="right" data-ratio="0 6">0%</td>
394 | </tr>
395 | <tr class="region">
396 | <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t68">scripts\cli\paths.py</td>
397 | <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t68"><data value='project_root'>CrossPlatfor
mPaths.project_root</data></td>
398 | <td>1</td>
399 | <td>1</td>
400 | <td>0</td>
401 | <td class="right" data-ratio="0 1">0%</td>
402 | </tr>
403 | <tr class="region">
404 | <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t73">scripts\cli\paths.py</td>
405 | <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t73"><data value='data_dir'>CrossPlatformPat
hs.data_dir</data></td>
406 | <td>8</td>
407 | <td>8</td>
408 | <td>0</td>
409 | <td class="right" data-ratio="0 8">0%</td>
410 | </tr>
411 | <tr class="region">
412 | <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t93">scripts\cli\paths.py</td>
413 | <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t93"><data value='config_dir'>CrossPlatformP
aths.config_dir</data></td>
414 | <td>7</td>
415 | <td>7</td>
416 | <td>0</td>
417 | <td class="right" data-ratio="0 7">0%</td>
418 | </tr>
419 | <tr class="region">
420 | <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t111">scripts\cli\paths.py</td>

```

```

421 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_paths_py.html#t111"><data value='cache_dir'>CrossPlatformP
| aths.cache_dir</data></td>
422 | <td>7</td>
423 | <td>7</td>
424 | <td>0</td>
425 | <td class="right" data-ratio="0 7">0%</td>
426 | </tr>
427 | <tr class="region">
428 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_paths_py.html#t129">scripts\cli\paths.py</td>
429 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_paths_py.html#t129"><data value='certs_dir'>CrossPlatformP
| aths.certs_dir</data></td>
430 | <td>5</td>
431 | <td>5</td>
432 | <td>0</td>
433 | <td class="right" data-ratio="0 5">0%</td>
434 | </tr>
435 | <tr class="region">
436 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_paths_py.html#t144">scripts\cli\paths.py</td>
437 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_paths_py.html#t144"><data value='certs_user_dir'>CrossPlat
| formPaths.certs_user_dir</data></td>
438 | <td>1</td>
439 | <td>1</td>
440 | <td>0</td>
441 | <td class="right" data-ratio="0 1">0%</td>
442 | </tr>
443 | <tr class="region">
444 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_paths_py.html#t149">scripts\cli\paths.py</td>
445 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_paths_py.html#t149"><data value='bin_dir'>CrossPlatformPat
| hs.bin_dir</data></td>
446 | <td>1</td>
447 | <td>1</td>

```

```

448 <td>0</td>
449 <td class="right" data-ratio="0 1">0%</td>
450 </tr>
451 <tr class="region">
452 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t158">scripts\cli\paths.py</td>
453 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t158"><data value='venv_dir'>CrossPlatformPa
ths.venv_dir</data></td>
454 <td>1</td>
455 <td>1</td>
456 <td>0</td>
457 <td class="right" data-ratio="0 1">0%</td>
458 </tr>
459 <tr class="region">
460 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t163">scripts\cli\paths.py</td>
461 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t163"><data value='venv_python'>CrossPlatfor
mPaths.venv_python</data></td>
462 <td>3</td>
463 <td>3</td>
464 <td>0</td>
465 <td class="right" data-ratio="0 3">0%</td>
466 </tr>
467 <tr class="region">
468 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t171">scripts\cli\paths.py</td>
469 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t171"><data value='secrets_dir'>CrossPlatfor
mPaths.secrets_dir</data></td>
470 <td>1</td>
471 <td>1</td>
472 <td>0</td>
473 <td class="right" data-ratio="0 1">0%</td>
474 </tr>
475 <tr class="region">
476 <td class="name left"><a href="z_933a9655f16c4

```

```

9fb_paths_py.html#t176">scripts\cli\paths.py</td>
477 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t176"><data value='env_file'>CrossPlatformPa
ths.env_file</data></td>
478 <td>1</td>
479 <td>1</td>
480 <td>0</td>
481 <td class="right" data-ratio="0 1">0%</td>
482 </tr>
483 <tr class="region">
484 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t181">scripts\cli\paths.py</td>
485 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t181"><data value='config_file'>CrossPlatfor
mPaths.config_file</data></td>
486 <td>1</td>
487 <td>1</td>
488 <td>0</td>
489 <td class="right" data-ratio="0 1">0%</td>
490 </tr>
491 <tr class="region">
492 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t186">scripts\cli\paths.py</td>
493 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t186"><data value='requirements_main'>CrossP
latformPaths.requirements_main</data></td>
494 <td>1</td>
495 <td>1</td>
496 <td>0</td>
497 <td class="right" data-ratio="0 1">0%</td>
498 </tr>
499 <tr class="region">
500 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t191">scripts\cli\paths.py</td>
501 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t191"><data value='requirements_core'>CrossP
latformPaths.requirements_core</data></td>
502 <td>1</td>

```

```

503 <td>1</td>
504 <td>0</td>
505 <td class="right" data-ratio="0 1">0%</td>
506 </tr>
507 <tr class="region">
508 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t196">scripts\cli\paths.py</td>
509 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t196"><data value='requirements_ai'>CrossPla
tformPaths.requirements_ai</data></td>
510 <td>1</td>
511 <td>1</td>
512 <td>0</td>
513 <td class="right" data-ratio="0 1">0%</td>
514 </tr>
515 <tr class="region">
516 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t201">scripts\cli\paths.py</td>
517 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t201"><data value='requirements_test'>CrossP
latformPaths.requirements_test</data></td>
518 <td>1</td>
519 <td>1</td>
520 <td>0</td>
521 <td class="right" data-ratio="0 1">0%</td>
522 </tr>
523 <tr class="region">
524 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t206">scripts\cli\paths.py</td>
525 <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t206"><data value='requirements_docs'>CrossP
latformPaths.requirements_docs</data></td>
526 <td>1</td>
527 <td>1</td>
528 <td>0</td>
529 <td class="right" data-ratio="0 1">0%</td>
530 </tr>
531 <tr class="region">

```

```

532 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_paths_py.html#t210">scripts\cli\paths.py</td>
533 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_paths_py.html#t210"><data value='get_env_vars'>CrossPlatfo
| rmPaths.get_env_vars</data></td>
534 | <td>1</td>
535 | <td>1</td>
536 | <td>0</td>
537 | <td class="right" data-ratio="0 1">0%</td>
538 | </tr>
539 | <tr class="region">
540 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_paths_py.html#t223">scripts\cli\paths.py</td>
541 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_paths_py.html#t223"><data value='setup_env'>CrossPlatformP
| aths.setup_env</data></td>
542 | <td>3</td>
543 | <td>3</td>
544 | <td>0</td>
545 | <td class="right" data-ratio="0 3">0%</td>
546 | </tr>
547 | <tr class="region">
548 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_paths_py.html#t229">scripts\cli\paths.py</td>
549 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_paths_py.html#t229"><data value='ensure_dirs'>CrossPlatfor
| mPaths.ensure_dirs</data></td>
550 | <td>3</td>
551 | <td>3</td>
552 | <td>0</td>
553 | <td class="right" data-ratio="0 3">0%</td>
554 | </tr>
555 | <tr class="region">
556 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_paths_py.html#t247">scripts\cli\paths.py</td>
557 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_paths_py.html#t247"><data value='get_paths'>get_paths</dat
| a></td>

```

```

558 | <td>3</td>
559 | <td>3</td>
560 | <td>0</td>
561 | <td class="right" data-ratio="0 3">0%</td>
562 | </tr>
563 | <tr class="region">
564 | <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t258">scripts\cli\paths.py</td>
565 | <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html#t258"><data value='init_paths'>init_paths</d
ata></td>
566 | <td>4</td>
567 | <td>4</td>
568 | <td>0</td>
569 | <td class="right" data-ratio="0 4">0%</td>
570 | </tr>
571 | <tr class="region">
572 | <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html">scripts\cli\paths.py</td>
573 | <td class="name left"><a href="z_933a9655f16c4
9fb_paths_py.html"><data value=''>(no fu
nction)</data></td>
574 | <td>51</td>
575 | <td>51</td>
576 | <td>2</td>
577 | <td class="right" data-ratio="0 51">0%</td>
578 | </tr>
579 | <tr class="region">
580 | <td class="name left"><a href="z_933a9655f16c4
9fb_utils_py.html#t25">scripts\cli\utils.py</td>
581 | <td class="name left"><a href="z_933a9655f16c4
9fb_utils_py.html#t25"><data value='find_available_port'>find_
available_port</data></td>
582 | <td>9</td>
583 | <td>9</td>
584 | <td>0</td>
585 | <td class="right" data-ratio="0 9">0%</td>
586 | </tr>

```

```

587 | <tr class="region">
588 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_utils_py.html#t50">scripts\cli\utils.py</td>
589 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_utils_py.html#t50"><data value='is_port_open'>is_port_open
| </data></td>
590 | <td>7</td>
591 | <td>7</td>
592 | <td>0</td>
593 | <td class="right" data-ratio="0 7">0%</td>
594 | </tr>
595 | <tr class="region">
596 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_utils_py.html#t68">scripts\cli\utils.py</td>
597 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_utils_py.html#t68"><data value='get_process_on_port'>get_p
| rocess_on_port</data></td>
598 | <td>30</td>
599 | <td>30</td>
600 | <td>2</td>
601 | <td class="right" data-ratio="0 30">0%</td>
602 | </tr>
603 | <tr class="region">
604 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_utils_py.html#t129">scripts\cli\utils.py</td>
605 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_utils_py.html#t129"><data value='get_process_name'>get_pro
| cess_name</data></td>
606 | <td>14</td>
607 | <td>14</td>
608 | <td>1</td>
609 | <td class="right" data-ratio="0 14">0%</td>
610 | </tr>
611 | <tr class="region">
612 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_utils_py.html#t170">scripts\cli\utils.py</td>
613 | <td class="name left"><a href="z_933a9655f16c4
| 9fb_utils_py.html#t170"><data value='kill_process'>kill_proces

```



```

s</data></td>
614 <td>14</td>
615 <td>14</td>
616 <td>0</td>
617 <td class="right" data-ratio="0 14">0%</td>
618 </tr>
619 <tr class="region">
620 <td class="name left"><a href="z_933a9655f16c4
9fb_utils_py.html#t199">scripts\cli\utils.py</td>
621 <td class="name left"><a href="z_933a9655f16c4
9fb_utils_py.html#t199"><data value='run_command'>run_command<
/data></td>
622 <td>4</td>
623 <td>4</td>
624 <td>0</td>
625 <td class="right" data-ratio="0 4">0%</td>
626 </tr>
627 <tr class="region">
628 <td class="name left"><a href="z_933a9655f16c4
9fb_utils_py.html#t230">scripts\cli\utils.py</td>
629 <td class="name left"><a href="z_933a9655f16c4
9fb_utils_py.html#t230"><data value='which'>which</data></
td>
630 <td>7</td>
631 <td>7</td>
632 <td>1</td>
633 <td class="right" data-ratio="0 7">0%</td>
634 </tr>
635 <tr class="region">
636 <td class="name left"><a href="z_933a9655f16c4
9fb_utils_py.html">scripts\cli\utils.py</td>
637 <td class="name left"><a href="z_933a9655f16c4
9fb_utils_py.html"><data value=''>(no fu
nction)</data></td>
638 <td>13</td>
639 <td>13</td>
640 <td>0</td>
641 <td class="right" data-ratio="0 13">0%</td>

```

```

642 </tr>
643 </tbody>
644 <tfoot>
645 <tr class="total">
646 <td class="name left">Total</td>
647 <td class="name left"> </td>
648 <td>780</td>
649 <td>780</td>
650 <td>10</td>
651 <td class="right" data-ratio="0 780">0%</td>
652 </tr>
653 </tfoot>
654 </table>
655 <p id="no_rows">
656 No items found using the specified filter.
657 </p>
658 </main>
659 <footer>
660 <div class="content">
661 <p>
662 <a class="nav" href="https://coverage.readthedocs.
663 io/en/7.6.10">coverage.py v7.6.10,
664 created at 2026-09-04 19:37 +0300
665 </p>
666 <aside class="hidden">
667
668
669 <button type="button" class="button_prev_file" data-sh
670 ortcut="["></button>
671 <button type="button" class="button_next_file" data-sh
672 ortcut="]"></button>
673 <button type="button" class="button_show_hide_help" da
674 ta-shortcut="?"></button>
675 </aside>
676 </footer>
677 </body>
678 </html>

```





```

33 <kbd>[</kbd>
34 <kbd>]</kbd>
35 prev/next file
36 </p>
37 <p>
38 <kbd>?</kbd> show/hide this hel
p
39 </p>
40 </div>
41 </div>
42 </aside>
43 <form id="filter_container">
44 <input id="filter" type="text" value="" placeholde
r="filter...">
45 <div>
46 <input id="hide100" type="checkbox" >
47 <label for="hide100">hide covered</label>
48 </div>
49 </form>
50 <h2>
51 Files
52 F
unctions
53 Clas
ses
54 </h2>
55 <p class="text">
56 <a class="nav" href="https://coverage.readthedocs.
io/en/7.6.10">coverage.py v7.6.10,
57 created at 2026-09-04 19:37 +0300
58 </p>
59 </div>
60 </header>
61 <main id="index">
62 <table class="index" data-sortable>
63 <thead>
64 <tr class="tablehead" title="Click to sort">
65 <th id="file" class="name left" aria-sort="non

```

```

e" data-shortcut="f">File</th>
66 <th id="statements" aria-sort="none" data-default-sort-order="descending" data-shortcut="s">statements</th>
67 <th id="missing" aria-sort="none" data-default-sort-order="descending" data-shortcut="m">missing<span class=
"arrows"></th>
68 <th id="excluded" aria-sort="none" data-default-sort-order="descending" data-shortcut="x">excluded<span clas
s="arrows"></th>
69 <th id="coverage" class="right" aria-sort="none" data-shortcut="c">coverage</th>
70 </tr>
71 </thead>
72 <tbody>
73 <tr class="region">
74 <td class="name left"><a href="z_933a9655f16c4
9fb___init___py.html">scripts\cli__init__.py</td>
75 <td>0</td>
76 <td>0</td>
77 <td>0</td>
78 <td class="right" data-ratio="0 0">100%</td>
79 </tr>
80 <tr class="region">
81 <td class="name left"><a href="z_933a9655f16c4
9fb_assist_py.html">scripts\cli\assist.py</td>
82 <td>286</td>
83 <td>286</td>
84 <td>2</td>
85 <td class="right" data-ratio="0 372">0%</td>
86 </tr>
87 <tr class="region">
88 <td class="name left"><a href="z_933a9655f16c4
9fb_config_py.html">scripts\cli\config.py</td>
89 <td>78</td>
90 <td>78</td>
91 <td>0</td>
92 <td class="right" data-ratio="0 100">0%</td>

```

```

93 | </tr>
94 | <tr class="region">
95 | <td class="name left"><a href="z_933a9655f16c4
96 | 9fb_installer_py.html">scripts\cli\installer.py</td>
97 | <td>201</td>
98 | <td>201</td>
99 | <td>2</td>
100 | <td class="right" data-ratio="0 239">0%</td>
101 | </tr>
102 | <tr class="region">
103 | <td class="name left"><a href="z_933a9655f16c4
104 | 9fb_paths_py.html">scripts\cli\paths.py</td>
105 | <td>117</td>
106 | <td>117</td>
107 | <td>2</td>
108 | <td class="right" data-ratio="0 153">0%</td>
109 | </tr>
110 | <tr class="region">
111 | <td class="name left"><a href="z_933a9655f16c4
112 | 9fb_utils_py.html">scripts\cli\utils.py</td>
113 | <td>98</td>
114 | <td>98</td>
115 | <td>4</td>
116 | <td class="right" data-ratio="0 132">0%</td>
117 | </tr>
118 | </tbody>
119 | <tfoot>
120 | <tr class="total">
121 | <td class="name left">Total</td>
122 | <td>780</td>
123 | <td>780</td>
124 | <td>10</td>
125 | <td class="right" data-ratio="0 780">0%</td>
126 | </tr>
127 | </tfoot>
128 | </table>
129 | <p id="no_rows">
130 | No items found using the specified filter.

```

```
128 | </p>
129 | </main>
130 | <footer>
131 | <div class="content">
132 | <p>
133 | <a class="nav" href="https://coverage.readthedocs.
134 | io/en/7.6.10">coverage.py v7.6.10,
135 | created at 2026-09-04 19:37 +0300
136 | </p>
137 | </div>
138 | <aside class="hidden">
139 | <a id="prevFileLink" class="nav" href="z_933a9655f16c4
140 | 9fb_utils_py.html">
141 | <a id="nextFileLink" class="nav" href="z_933a9655f16c4
142 | 9fb___init___py.html">
143 | <button type="button" class="button_prev_file" data-sh
144 | ortcut="["></button>
145 | <button type="button" class="button_next_file" data-sh
146 | ortcut="]"></button>
147 | <button type="button" class="button_show_hide_help" da
148 | ta-shortcut="?"></button>
149 | </aside>
150 | </footer>
151 | </body>
152 | </html>
```



## style\_cb\_8e611ae1.css

```
1 @charset "UTF-8";
2 /* Licensed under the Apache License: http://www.apache.org/licenses/LICENSE-2.0 */
3 /* For details: https://github.com/nedbat/coveragepy/blob/master/NOTICE.txt */
4 /* Don't edit this .css file. Edit the .scss file instead! */
5 html, body, h1, h2, h3, p, table, td, th { margin: 0; padding: 0; border: 0; font-weight: inherit; font-style: inherit; font-size: 100%; font-family: inherit; vertical-align: baseline; }
6
7 body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Ubuntu, Cantarell, "Helvetica Neue", sans-serif; font-size: 1em; background: #fff; color: #000; }
8
9 @media (prefers-color-scheme: dark) { body { background: #1e1e1e; } }
10
11 @media (prefers-color-scheme: dark) { body { color: #eee; } }
12
13 html > body { font-size: 16px; }
14
15 a:active, a:focus { outline: 2px dashed #007acc; }
16
17 p { font-size: .875em; line-height: 1.4em; }
18
19 table { border-collapse: collapse; }
20
21 td { vertical-align: top; }
22
23 table tr.hidden { display: none !important; }
24
25 p#no_rows { display: none; font-size: 1.15em; font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Ubuntu, Cantarell, "Helvetica Neue", sans-serif; }
26
27 a.nav { text-decoration: none; color: inherit; }
```

```
28 |
29 | a.nav:hover { text-decoration: underline; color: inherit; }
30 |
31 | .hidden { display: none; }
32 |
33 | header { background: #f8f8f8; width: 100%; z-index: 2; border-
 | bottom: 1px solid #ccc; }
34 |
35 | @media (prefers-color-scheme: dark) { header { background: bla
 | ck; } }
36 |
37 | @media (prefers-color-scheme: dark) { header { border-color: #
 | 333; } }
38 |
39 | header .content { padding: 1rem 3.5rem; }
40 |
41 | header h2 { margin-top: .5em; font-size: 1em; }
42 |
43 | header h2 a.button { font-family: inherit; font-size: inherit;
 | border: 1px solid; border-radius: .2em; background: #eee; col
 | or: inherit; text-decoration: none; padding: .1em .5em; margin
 | : 1px calc(.1em + 1px); cursor: pointer; border-color: #ccc; }
44 |
45 | @media (prefers-color-scheme: dark) { header h2 a.button { bac
 | kground: #333; } }
46 |
47 | @media (prefers-color-scheme: dark) { header h2 a.button { bor
 | der-color: #444; } }
48 |
49 | header h2 a.button.current { border: 2px solid; background: #f
 | ff; border-color: #999; cursor: default; }
50 |
51 | @media (prefers-color-scheme: dark) { header h2 a.button.curre
 | nt { background: #1e1e1e; } }
52 |
53 | @media (prefers-color-scheme: dark) { header h2 a.button.curre
 | nt { border-color: #777; } }
54 |
```

```
55 | header p.text { margin: .5em 0 -.5em; color: #666; font-style:
 | italic; }
56 |
57 | @media (prefers-color-scheme: dark) { header p.text { color: #
 | aaa; } }
58 |
59 | header.sticky { position: fixed; left: 0; right: 0; height: 2.
 | 5em; }
60 |
61 | header.sticky .text { display: none; }
62 |
63 | header.sticky h1, header.sticky h2 { font-size: 1em; margin-to
 | p: 0; display: inline-block; }
64 |
65 | header.sticky .content { padding: 0.5rem 3.5rem; }
66 |
67 | header.sticky .content p { font-size: 1em; }
68 |
69 | header.sticky ~ #source { padding-top: 6.5em; }
70 |
71 | main { position: relative; z-index: 1; }
72 |
73 | footer { margin: 1rem 3.5rem; }
74 |
75 | footer .content { padding: 0; color: #666; font-style: italic;
 | }
76 |
77 | @media (prefers-color-scheme: dark) { footer .content { color:
 | #aaa; } }
78 |
79 | #index { margin: 1rem 0 0 3.5rem; }
80 |
81 | h1 { font-size: 1.25em; display: inline-block; }
82 |
83 | #filter_container { float: right; margin: 0 2em 0 0; line-heig
 | ht: 1.66em; }
84 |
85 | #filter_container #filter { width: 10em; padding: 0.2em 0.5em;
```

```

 border: 2px solid #ccc; background: #fff; color: #000; }
86
87 @media (prefers-color-scheme: dark) { #filter_container #filter { border-color: #444; } }
88
89 @media (prefers-color-scheme: dark) { #filter_container #filter { background: #1e1e1e; } }
90
91 @media (prefers-color-scheme: dark) { #filter_container #filter { color: #eee; } }
92
93 #filter_container #filter:focus { border-color: #007acc; }
94
95 #filter_container :disabled ~ label { color: #ccc; }
96
97 @media (prefers-color-scheme: dark) { #filter_container :disabled ~ label { color: #444; } }
98
99 #filter_container label { font-size: .875em; color: #666; }
100
101 @media (prefers-color-scheme: dark) { #filter_container label { color: #aaa; } }
102
103 header button { font-family: inherit; font-size: inherit; border: 1px solid; border-radius: .2em; background: #eee; color: inherit; text-decoration: none; padding: .1em .5em; margin: 1px calc(.1em + 1px); cursor: pointer; border-color: #ccc; }
104
105 @media (prefers-color-scheme: dark) { header button { background: #333; } }
106
107 @media (prefers-color-scheme: dark) { header button { border-color: #444; } }
108
109 header button:active, header button:focus { outline: 2px dashed #007acc; }
110
111 header button.run { background: #eeffee; }

```

```
112 |
113 | @media (prefers-color-scheme: dark) { header button.run { back
 | ground: #373d29; } }
114 |
115 | header button.run.show_run { background: #dfd; border: 2px sol
 | id #00dd00; margin: 0 .1em; }
116 |
117 | @media (prefers-color-scheme: dark) { header button.run.show_r
 | un { background: #373d29; } }
118 |
119 | header button.mis { background: #ffeeee; }
120 |
121 | @media (prefers-color-scheme: dark) { header button.mis { back
 | ground: #4b1818; } }
122 |
123 | header button.mis.show_mis { background: #fdd; border: 2px sol
 | id #ff0000; margin: 0 .1em; }
124 |
125 | @media (prefers-color-scheme: dark) { header button.mis.show_m
 | is { background: #4b1818; } }
126 |
127 | header button.exc { background: #f7f7f7; }
128 |
129 | @media (prefers-color-scheme: dark) { header button.exc { back
 | ground: #333; } }
130 |
131 | header button.exc.show_exc { background: #eee; border: 2px sol
 | id #808080; margin: 0 .1em; }
132 |
133 | @media (prefers-color-scheme: dark) { header button.exc.show_e
 | xc { background: #333; } }
134 |
135 | header button.par { background: #ffffd5; }
136 |
137 | @media (prefers-color-scheme: dark) { header button.par { back
 | ground: #650; } }
138 |
139 | header button.par.show_par { background: #ffa; border: 2px sol
```

```
id #bbbb00; margin: 0 .1em; }
140
141 @media (prefers-color-scheme: dark) { header button.par.show_p
 ar { background: #650; } }
142
143 #help_panel, #source p .annotate.long { display: none; positio
 n: absolute; z-index: 999; background: #ffffcc; border: 1px so
 lid #888; border-radius: .2em; color: #333; padding: .25em .5e
 m; }
144
145 #source p .annotate.long { white-space: normal; float: right;
 top: 1.75em; right: 1em; height: auto; }
146
147 #help_panel_wrapper { float: right; position: relative; }
148
149 #keyboard_icon { margin: 5px; }
150
151 #help_panel_state { display: none; }
152
153 #help_panel { top: 25px; right: 0; padding: .75em; border: 1px
 solid #883; color: #333; }
154
155 #help_panel .keyhelp p { margin-top: .75em; }
156
157 #help_panel .legend { font-style: italic; margin-bottom: 1em;
 }
158
159 .indexfile #help_panel { width: 25em; }
160
161 .pyfile #help_panel { width: 18em; }
162
163 #help_panel_state:checked ~ #help_panel { display: block; }
164
165 kbd { border: 1px solid black; border-color: #888 #333 #333 #8
 88; padding: .1em .35em; font-family: SFMono-Regular, Menlo, M
 onaco, Consolas, monospace; font-weight: bold; background: #ee
 e; border-radius: 3px; }
166
```

```
167 #source { padding: 1em 0 1em 3.5rem; font-family: SFMono-Regular, Menlo, Monaco, Consolas, monospace; }
168
169 #source p { position: relative; white-space: pre; }
170
171 #source p * { box-sizing: border-box; }
172
173 #source p .n { float: left; text-align: right; width: 3.5rem; box-sizing: border-box; margin-left: -3.5rem; padding-right: 1em; color: #999; user-select: none; }
174
175 @media (prefers-color-scheme: dark) { #source p .n { color: #777; } }
176
177 #source p .n.highlight { background: #ffdd00; }
178
179 #source p .n a { scroll-margin-top: 6em; text-decoration: none; color: #999; }
180
181 @media (prefers-color-scheme: dark) { #source p .n a { color: #777; } }
182
183 #source p .n a:hover { text-decoration: underline; color: #999; }
184
185 @media (prefers-color-scheme: dark) { #source p .n a:hover { color: #777; } }
186
187 #source p .t { display: inline-block; width: 100%; box-sizing: border-box; margin-left: -.5em; padding-left: 0.3em; border-left: 0.2em solid #fff; }
188
189 @media (prefers-color-scheme: dark) { #source p .t { border-color: #1e1e1e; } }
190
191 #source p .t:hover { background: #f2f2f2; }
192
193 @media (prefers-color-scheme: dark) { #source p .t:hover { bac
```

```
kground: #282828; } }
194
195 #source p .t:hover ~ .r .annotate.long { display: block; }
196
197 #source p .t .com { color: #008000; font-style: italic; line-h
eight: 1px; }
198
199 @media (prefers-color-scheme: dark) { #source p .t .com { colo
r: #6a9955; } }
200
201 #source p .t .key { font-weight: bold; line-height: 1px; }
202
203 #source p .t .str { color: #0451a5; }
204
205 @media (prefers-color-scheme: dark) { #source p .t .str { colo
r: #9cdcfe; } }
206
207 #source p.mis .t { border-left: 0.2em solid #ff0000; }
208
209 #source p.mis.show_mis .t { background: #fdd; }
210
211 @media (prefers-color-scheme: dark) { #source p.mis.show_mis .
t { background: #4b1818; } }
212
213 #source p.mis.show_mis .t:hover { background: #f2d2d2; }
214
215 @media (prefers-color-scheme: dark) { #source p.mis.show_mis .
t:hover { background: #532323; } }
216
217 #source p.run .t { border-left: 0.2em solid #00dd00; }
218
219 #source p.run.show_run .t { background: #dfd; }
220
221 @media (prefers-color-scheme: dark) { #source p.run.show_run .
t { background: #373d29; } }
222
223 #source p.run.show_run .t:hover { background: #d2f2d2; }
224
```



```
225 @media (prefers-color-scheme: dark) { #source p.run.show_run .
 t:hover { background: #404633; } }
226
227 #source p.exc .t { border-left: 0.2em solid #808080; }
228
229 #source p.exc.show_exc .t { background: #eee; }
230
231 @media (prefers-color-scheme: dark) { #source p.exc.show_exc .
 t { background: #333; } }
232
233 #source p.exc.show_exc .t:hover { background: #e2e2e2; }
234
235 @media (prefers-color-scheme: dark) { #source p.exc.show_exc .
 t:hover { background: #3c3c3c; } }
236
237 #source p.par .t { border-left: 0.2em solid #bbbb00; }
238
239 #source p.par.show_par .t { background: #ffa; }
240
241 @media (prefers-color-scheme: dark) { #source p.par.show_par .
 t { background: #650; } }
242
243 #source p.par.show_par .t:hover { background: #f2f2a2; }
244
245 @media (prefers-color-scheme: dark) { #source p.par.show_par .
 t:hover { background: #6d5d0c; } }
246
247 #source p .r { position: absolute; top: 0; right: 2.5em; font-
 family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto,
 Ubuntu, Cantarell, "Helvetica Neue", sans-serif; }
248
249 #source p .annotate { font-family: -apple-system, BlinkMacSyst
 emFont, "Segoe UI", Roboto, Ubuntu, Cantarell, "Helvetica Neue
 ", sans-serif; color: #666; padding-right: .5em; }
250
251 @media (prefers-color-scheme: dark) { #source p .annotate { co
 lor: #ddd; } }
252
```

```
253 #source p .annotate.short:hover ~ .long { display: block; }
254
255 #source p .annotate.long { width: 30em; right: 2.5em; }
256
257 #source p input { display: none; }
258
259 #source p input ~ .r label.ctx { cursor: pointer; border-radius:
 | .25em; }
260
261 #source p input ~ .r label.ctx::before { content: "► "; }
262
263 #source p input ~ .r label.ctx:hover { background: #e8f4ff; color: #666; }
264
265 @media (prefers-color-scheme: dark) { #source p input ~ .r label.ctx:
 | hover { background: #0f3a42; } }
266
267 @media (prefers-color-scheme: dark) { #source p input ~ .r label.ctx:
 | hover { color: #aaa; } }
268
269 #source p input:checked ~ .r label.ctx { background: #d0e8ff; color: #666;
 | border-radius: .75em .75em 0 0; padding: 0 .5em; margin: -.25em 0; }
270
271 @media (prefers-color-scheme: dark) { #source p input:checked ~ .r label.ctx
 | { background: #056; } }
272
273 @media (prefers-color-scheme: dark) { #source p input:checked ~ .r label.ctx
 | { color: #aaa; } }
274
275 #source p input:checked ~ .r label.ctx::before { content: "▼ "
 | ; }
276
277 #source p input:checked ~ .ctxs { padding: .25em .5em; overflow-y: scroll;
 | max-height: 10.5em; }
278
279 #source p label.ctx { color: #999; display: inline-block; padding: 0 .5em;
 | font-size: .8333em; }
```

```
280
281 @media (prefers-color-scheme: dark) { #source p label.ctx { co
 lor: #777; } }
282
283 #source p .ctxs { display: block; max-height: 0; overflow-y: h
 idden; transition: all .2s; padding: 0 .5em; font-family: -app
 le-system, BlinkMacSystemFont, "Segoe UI", Roboto, Ubuntu, Can
 tarell, "Helvetica Neue", sans-serif; white-space: nowrap; bac
 kground: #d0e8ff; border-radius: .25em; margin-right: 1.75em;
 text-align: right; }
284
285 @media (prefers-color-scheme: dark) { #source p .ctxs { backgr
 ound: #056; } }
286
287 #index { font-family: SFMono-Regular, Menlo, Monaco, Consolas,
 monospace; font-size: 0.875em; }
288
289 #index table.index { margin-left: -.5em; }
290
291 #index td, #index th { text-align: right; padding: .25em .5em;
 border-bottom: 1px solid #eee; }
292
293 @media (prefers-color-scheme: dark) { #index td, #index th { b
 order-color: #333; } }
294
295 #index td.name, #index th.name { text-align: left; width: auto
 ; font-family: -apple-system, BlinkMacSystemFont, "Segoe UI",
 Roboto, Ubuntu, Cantarell, "Helvetica Neue", sans-serif; min-w
 idth: 15em; }
296
297 #index th { font-family: -apple-system, BlinkMacSystemFont, "S
 egoe UI", Roboto, Ubuntu, Cantarell, "Helvetica Neue", sans-se
 rif; font-style: italic; color: #333; cursor: pointer; }
298
299 @media (prefers-color-scheme: dark) { #index th { color: #ddd;
 } }
300
301 #index th:hover { background: #eee; }
```

```
302 |
303 | @media (prefers-color-scheme: dark) { #index th:hover { backgr
 | ound: #333; } }
304 |
305 | #index th .arrows { color: #666; font-size: 85%; font-family:
 | sans-serif; font-style: normal; pointer-events: none; }
306 |
307 | #index th[aria-sort="ascending"], #index th[aria-sort="descend
 | ing"] { white-space: nowrap; background: #eee; padding-left: .
 | 5em; }
308 |
309 | @media (prefers-color-scheme: dark) { #index th[aria-sort="asc
 | ending"], #index th[aria-sort="descending"] { background: #333
 | ; } }
310 |
311 | #index th[aria-sort="ascending"] .arrows::after { content: " ▲
 | "; }
312 |
313 | #index th[aria-sort="descending"] .arrows::after { content: "
 | ▼"; }
314 |
315 | #index td.name { font-size: 1.15em; }
316 |
317 | #index td.name a { text-decoration: none; color: inherit; }
318 |
319 | #index td.name .no-noun { font-style: italic; }
320 |
321 | #index tr.total td, #index tr.total_dynamic td { font-weight:
 | bold; border-top: 1px solid #ccc; border-bottom: none; }
322 |
323 | #index tr.region:hover { background: #eee; }
324 |
325 | @media (prefers-color-scheme: dark) { #index tr.region:hover {
 | background: #333; } }
326 |
327 | #index tr.region:hover td.name { text-decoration: underline; c
 | olor: inherit; }
328 |
```

```
329 #scroll_marker { position: fixed; z-index: 3; right: 0; top: 0
 ; width: 16px; height: 100%; background: #fff; border-left: 1p
 x solid #eee; will-change: transform; }
330
331 @media (prefers-color-scheme: dark) { #scroll_marker { backgro
 und: #1e1e1e; } }
332
333 @media (prefers-color-scheme: dark) { #scroll_marker { border-
 color: #333; } }
334
335 #scroll_marker .marker { background: #ccc; position: absolute;
 min-height: 3px; width: 100%; }
336
337 @media (prefers-color-scheme: dark) { #scroll_marker .marker {
 background: #444; } }
338
```

echo\_args.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Module
4 # =====
5 # Description:
6 # Module for AI Breadboard project.
7 #
8 # File: echo_args.py
9 # Project: ai-breadboard
10 # Package: tests.data.test_manage_tools
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import sys; print(" ".join(sys.argv[1:])); sys.exit(0)
```

## test\_ai.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Tests for core/ai module
4 # =====
5 # Description:
6 # Module содержит тесты для модуля искусственного интеллекта
 core/ai.
7 #
8 # File: test_ai.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """
16 Тесты модуля core/ai
17 """
18
19 import pytest
20 from unittest.mock import Mock, AsyncMock, patch, MagicMock
21 from pathlib import Path
22
23 class TestGoogleGenerativeAI:
24 """Тесты класса GoogleGenerativeAI."""
25
26 @pytest.mark.asyncio
27 async def test_chat_with_mock(self):
28 """Тест метода chat с моком."""
29 # Создаем мок с правильной настройкой асинхронного мет
 ода
30 mock_model = MagicMock()
31 mock_model.chat = AsyncMock(return_value="Test respons
```

```

 e")
32
33 result = await mock_model.chat("Test question")
34
35 assert result == "Test response"
36 mock_model.chat.assert_called_once_with("Test question
 ")
37
38 @pytest.mark.asyncio
39 async def test_chat_stream_with_mock(self):
40 """Тест метода chat_stream с моком."""
41 # Создаем асинхронный генератор
42 async def mock_stream():
43 yield "Chunk 1"
44 yield "Chunk 2"
45
46 mock_model = MagicMock()
47 mock_model.chat_stream = mock_stream
48
49 chunks = []
50 async for chunk in mock_model.chat_stream():
51 chunks.append(chunk)
52
53 assert len(chunks) == 2
54 assert chunks[0] == "Chunk 1"
55 assert chunks[1] == "Chunk 2"
56 assert chunks[1] == "Chunk 2"
57
58 @pytest.mark.asyncio
59 async def test_ask_with_mock(self):
60 """Тест метода ask с моком."""
61 mock_model = MagicMock()
62 mock_model.ask = AsyncMock(return_value="Test answer")
63
64 result = await mock_model.ask("Test question")
65
66 assert result == "Test answer"
67

```



```

68 class TestRAGFunctions:
69 """Тесты функций RAG."""
70
71 @pytest.mark.asyncio
72 async def test_build_dev_rag_exists(self):
73 """Тест существования функции build_dev_rag."""
74 from src.ai.dev_rag import build_dev_rag
75
76 # Проверяем что function существует и Raisesся без оши
бок
77 # (без реального API ключа она может вернуть None или
ошибку, но не должна падать)
78 try:
79 result = build_dev_rag("test_api_key")
80 # Function может вернуть None если нет API ключа,
это нормально
81 assert result is None or hasattr(result, 'search')
82 except Exception:
83 # Если нет API ключа, это тоже нормально для теста
84 pass
85
86 @pytest.mark.asyncio
87 async def test_rag_search_tool_signature(self):
88 """Тест сигнатуры функции rag_search_tool."""
89 from src.ai.dev_rag import rag_search_tool
90
91 # Проверяем что function существует
92 assert callable(rag_search_tool)
93
94 class TestFoundryChat:
95 """Тесты FoundryChatBase."""
96
97 @pytest.mark.asyncio
98 async def test_foundry_chat_initialization(self):
99 """Тест инициализации FoundryChatBase."""
100 from src.ai.foundry_chat import FoundryChatBase
101
102 chat = FoundryChatBase(

```

```

103 model_id="test-model",
104 system_prompt="Test system prompt"
105)
106
107 assert chat is not None
108 assert chat.model_id == "test-model"
109 assert chat.system_instruction == "Test system prompt"
110
111 class TestAgyChat:
112 """Тесты AgyChatBase."""
113
114 def test_agy_chat_initialization(self):
115 """Тест инициализации AgyChatBase и наличия system_in-
116 struction."""
117
118 from src.ai.agy_chat import AgyChatBase
119
120 chat = AgyChatBase(
121 model_id="agy-flash",
122 system_prompt="Test system prompt"
123)
124
125 assert chat.model_id == "gemini-flash-lite-latest"
126 assert chat.system_prompt == "Test system prompt"
127 assert chat.system_instruction == "Test system prompt"
128
129 # Проверяем изменение через setter
130 chat.system_instruction = "New instruction"
131 assert chat.system_prompt == "New instruction"
132 assert chat.system_instruction == "New instruction"
133
134 # Проверяем нормализацию agy- префикса через setter и
135 __init__
136
137 chat.model_id = "agy-gemini-2.5-flash"
138 assert chat.model_id == "gemini-2.5-flash"
139
140 chat2 = AgyChatBase(model_id="agy-gemini-2.0-flash")
141 assert chat2.model_id == "gemini-2.0-flash"

```

```

139 class TestUserRAG:
140 """Тесты пользовательского RAG."""
141
142 @pytest.mark.asyncio
143 async def test_get_user_rag_path_structure(self):
144 """Тест получения пути к user RAG - check структуры пу
ти."""
145 from src.ai.gemini.user_query_rag import _get_user_rag
_path
146
147 result = _get_user_rag_path(1)
148
149 assert isinstance(result, Path)
150 # Проверяем что путь содержит user_rag_1.db
151 assert "user_rag_1" in str(result)
152 assert result.suffix == ".db"
153
154 @pytest.mark.asyncio
155 async def test_make_doc_id_structure(self):
156 """Тест создания doc_id - check структуры."""
157 from src.ai.gemini.user_query_rag import _make_doc_id
158
159 result = _make_doc_id(1, "test query")
160
161 assert isinstance(result, str)
162 # doc_id имеет формат userId_hash(query)
163 assert "_" in result
164 parts = result.split("_")
165 assert len(parts) == 2
166 assert parts[0] == "1" # user_id
167
168 @pytest.mark.asyncio
169 async def test_garbage_query_filter(self):
170 """Тест фильтрации мусорных запросов."""
171 from src.ai.gemini.user_query_rag import is_garbage_qu
ery
172
173 # Хорошие запросы (не мусорные)

```

```
174 | assert not is_garbage_query("Какой фильм посмотреть ве
| чером?")
175 | assert not is_garbage_query("Посоветуй хороший сериал
| в жанре фантастика")
176 | assert not is_garbage_query("Как настроить плеер для п
| росмотра")
177 |
178 | # Мусорные запросы
179 | assert is_garbage_query("да") # слишком короткий
180 | assert is_garbage_query("привет") # приветствие/корот
| кий
181 | assert is_garbage_query("Привет! Как дела?") # просто
| е приветствие
182 | assert is_garbage_query("ыыыыыыыыыыыы") # повторяющи
| еся символы
183 | assert is_garbage_query("sdfghjklqwrtzcxvb") # keyboa
| rd mash (без гласных)
184 | assert is_garbage_query("!!!!!!!!!!!!") # знаки препин
| ания без букв/цифр
185 | assert is_garbage_query("спасибо большое") # благодар
| ность
186 |
187 | class TestModelManager:
188 | """Тесты единого менеджера моделей ИИ (ModelManager)."""
189 |
190 | def test_load_unsupported_models_gemini(self):
191 | """Тест загрузки неподдерживаемых моделей Gemini из ко
| нфигурации."""
192 | from src.ai.model_manager import load_unsupported_mode
| ls
193 | unsupported = load_unsupported_models("gemini")
194 | assert isinstance(unsupported, set)
195 | assert len(unsupported) > 0
196 | assert "gemini-1.0-pro" in unsupported or "gemini-2.0-
| flash" in unsupported
197 |
198 | def test_get_available_models_gemini_sdk_mock(self):
199 | """Тест получения моделей Gemini через SDK с фильтрации
```

```

ей и кэшированием. """
200 from src.ai.model_manager import get_available_models,
 _cached_models
201
202 # Подготовка мока для genai.Client
203 mock_model_1 = MagicMock()
204 mock_model_1.name = "models/gemini-flash-latest"
205 mock_model_1.supported_actions = ["generateContent"]
206
207 mock_model_2 = MagicMock()
208 mock_model_2.name = "models/gemini-2.0-flash" # в unsupported_models
209 mock_model_2.supported_actions = ["generateContent"]
210
211 mock_model_3 = MagicMock()
212 mock_model_3.name = "models/text-embedding-004" # embedding
213 mock_model_3.supported_actions = ["embedContent"]
214
215 mock_client = MagicMock()
216 mock_client.models.list.return_value = [mock_model_1,
 mock_model_2, mock_model_3]
217
218 with patch("src.ai.model_manager.genai.Client", return_value=mock_client):
219 # Первый вызов с force_refresh=True
220 models = get_available_models("gemini", api_key="fake_key", force_refresh=True)
221 assert "gemini-flash-latest" in models
222 assert "gemini-2.0-flash" not in models
223 assert "text-embedding-004" not in models
224
225 # Check, что результат сохранен в кэше
226 assert "gemini" in _cached_models
227 assert "gemini-flash-latest" in _cached_models["gemini"]
228
229 # Повторный вызов должен возвращать данные из кэша

```

```

 без нового вызова genai.Client
230 mock_client.models.list.reset_mock()
231 cached_models = get_available_models("gemini", api
 _key="fake_key", force_refresh=False)
232 assert cached_models == models
233 mock_client.models.list.assert_not_called()
234
235 def test_get_available_models_agy(self):
236 """Тест получения моделей для AGY."""
237 from src.ai.model_manager import get_available_models
238 agy_models = get_available_models("agy")
239 assert isinstance(agy_models, list)
240 for m in agy_models:
241 assert m.startswith("agy-")
242
243 def test_add_unsupported_model_runtime(self):
244 """Тест добавления неподдерживаемой модели в рантайме
 и её исключения из кэша."""
245 from src.ai.model_manager import add_unsupported_model
 , get_available_models, _CACHEd_MODELS
246
247 # Добавляем временную модель в кэш
248 _CACHEd_MODELS["foundry"] = ["test-foundry-model-1", "
 test-foundry-model-2"]
249 assert "test-foundry-model-1" in get_available_models(
 "foundry")
250
251 # Исключаем модель
252 with patch("src.ai.model_manager.j_dumps") as mock_dum
 ps:
253 success = add_unsupported_model("foundry", "test-f
 oundry-model-1", reason="404 Not Found")
254 assert success is True
255 assert "test-foundry-model-1" not in _CACHEd_MODEL
 S["foundry"]
256 assert "test-foundry-model-2" in _CACHEd_MODELS["f
 oundry"]
257

```

```
258 | @pytest.mark.asyncio
259 | async def test_actualize_all_models(self):
260 | """Тест разовой актуализации всех моделей при старте."
 | ""
261 | from src.ai.model_manager import actualize_all_models
262 | pool = await actualize_all_models(force_refresh=False)
263 | assert isinstance(pool, dict)
264 | assert "gemini" in pool
265 | assert "agy" in pool
266 | assert "foundry" in pool
267 | assert "ollama" in pool
268 |
```

## test\_ai\_capability\_orchestration.py

```
1 # -*- coding: utf-8 -*-
2 import pytest
3 from unittest.mock import AsyncMock, patch, MagicMock
4 from src.ai.orchestration import (
5 HardwareProfile,
6 probe_hardware,
7 AICapability,
8 CapabilityRegistry,
9 ModelDescriptor,
10 Locality,
11 PolicyEngine,
12 RoutingPolicy,
13 PrivacyLevel,
14 LocalityPreference,
15 DiscoveryEngine,
16 AIRouter,
17 AIRequest,
18)
19 from src.ai.providers.windows_ai import WindowsAIChatBase, probe_windows_ai_components
20 from src.ai.providers.gemini import GeminiChatBase
21
22
23 def test_hardware_probe():
24 """Verify hardware profile structure and fallback values."""
25 profile = probe_hardware()
26 assert isinstance(profile, HardwareProfile)
27 assert profile.cpu_cores >= 1
28 assert isinstance(profile.has_cuda, bool)
29 assert isinstance(profile.has_directml, bool)
30 data = profile.to_dict()
31 assert "cpu_cores" in data
32 assert "ram_gb" in data
33
34
```



```

35 def test_capability_registry():
36 """Verify capability registry registration and capability
 query."""
37 registry = CapabilityRegistry()
38 chat_models = registry.find_by_capability(AICapability.CHA
 T)
39 assert len(chat_models) > 0
40
41 ocr_models = registry.find_by_capability(AICapability.OCR)
42 assert any("ocr" in m.id for m in ocr_models)
43
44 local_chat = registry.find_by_capability(AICapability.CHAT
 , locality=Locality.LOCAL)
45 assert all(m.locality == Locality.LOCAL for m in local_cha
 t)
46
47
48 def test_policy_engine_privacy_strict():
49 """Verify PolicyEngine forbids cloud models when privacy i
 s STRICT."""
50 cloud_model = ModelDescriptor(
51 id="gemini:test",
52 provider="gemini",
53 capabilities={AICapability.CHAT},
54 locality=Locality.CLOUD,
55)
56 local_model = ModelDescriptor(
57 id="foundry:test",
58 provider="foundry",
59 capabilities={AICapability.CHAT},
60 locality=Locality.LOCAL,
61)
62
63 strict_policy = RoutingPolicy(privacy=PrivacyLevel.STRICT)
64 assert PolicyEngine.is_compliant(cloud_model, strict_polic
 y) is False
65 assert PolicyEngine.is_compliant(local_model, strict_polic
 y) is True

```

```

66 |
67 |
68 | def test_policy_engine_locality():
69 | """Verify PolicyEngine enforces local_only and cloud_only
 | constraints."""
70 | local_model = ModelDescriptor(
71 | id="ollama:test",
72 | provider="ollama",
73 | capabilities={AICapability.CHAT},
74 | locality=Locality.LOCAL,
75 |)
76 | cloud_model = ModelDescriptor(
77 | id="gemini:test",
78 | provider="gemini",
79 | capabilities={AICapability.CHAT},
80 | locality=Locality.CLOUD,
81 |)
82 |
83 | local_policy = RoutingPolicy(locality=LocalityPreference.L
 | OCAL_ONLY)
84 | assert PolicyEngine.is_compliant(local_model, local_policy
 |) is True
85 | assert PolicyEngine.is_compliant(cloud_model, local_policy
 |) is False
86 |
87 | cloud_policy = RoutingPolicy(locality=LocalityPreference.C
 | LOUD_ONLY)
88 | assert PolicyEngine.is_compliant(local_model, cloud_policy
 |) is False
89 | assert PolicyEngine.is_compliant(cloud_model, cloud_policy
 |) is True
90 |
91 |
92 | @pytest.mark.asyncio
93 | async def test_ai_router_resolution():
94 | """Verify AIRouter resolves compliant candidate models."""
95 | router = AIRouter()
96 |

```

```

97 | # Request with strict privacy
98 | req = AIRequest(
99 | prompt="Classify this document locally",
100 | capability=AICapability.CHAT,
101 | policy=RoutingPolicy(privacy=PrivacyLevel.STRICT),
102 |)
103 | candidates = await router.resolve_candidate_models(req)
104 | assert len(candidates) > 0
105 | assert all(c.locality == Locality.LOCAL for c in candidate
106 | s)
107 |
108 | def test_windows_ai_probe_structure():
109 | """Verify Windows AI probe returns structured result witho
110 | ut raising errors."""
111 | res = probe_windows_ai_components()
112 | assert isinstance(res, dict)
113 | assert "available" in res
114 | assert "components" in res
115 | assert isinstance(res["components"], list)
116 |
117 | @pytest.mark.asyncio
118 | async def test_windows_ai_chat_graceful_fallback():
119 | """Verify WindowsAIChatBase provides graceful fallback whe
120 | n uninstalled."""
121 | provider = WindowsAIChatBase(model_id="phi-silica")
122 | with patch.object(WindowsAIChatBase, "is_available", retur
123 | n_value=False):
124 | response = await provider.ask("Hello")
125 | assert "Windows AI Components are not installed" in re
126 | sponse

```

## test\_api\_key\_state.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Test saving an API key and loading it back.
4 # =====
5 # Description:
6 # Validates API key loading, priority merging, rotation, 24-
7 # hour quota
8 #
9 # File: test_api_key_state.py
10 # Project: ai-breadboard
11 # Package: tests
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import json
17 from datetime import datetime, timezone, timedelta
18 from pathlib import Path
19 from unittest.mock import patch
20
21 from src.secrets.api_key_state import (
22 get_status,
23 load_api_keys,
24 mark_exhausted,
25 next_available_in,
26 save_api_key,
27 update_last_run,
28)
29
30 def test_save_and_load_api_key(tmp_path):
31 """Test saving an API key and loading it back."""
32 keys_file = tmp_path / 'gemini_keys.json'
33 legacy_file = tmp_path / 'secrets.json'
```

```

33 | with patch('src.secrets.api_key_state._KEYS_FILE', keys_file), \
34 | patch('src.secrets.api_key_state._LEGACY_SECRETS_FILE', legacy_file), \
35 | patch('src.secrets.api_key_state._SECRETS_DIR', tmp_path), \
36 | patch.dict('os.environ', {}, clear=True):
37 |
38 | assert save_api_key('test_main', 'AIzaSyTest1234567890') is True
39 |
40 | keys, names, states = load_api_keys()
41 | assert 'AIzaSyTest1234567890' in keys
42 | assert 'test_main' in names
43 | assert len(states) == 1
44 | assert states[0]['status'] == 'active'
45 |
46 | def test_mark_exhausted_and_cooldown(tmp_path):
47 | """Test marking key exhausted and calculating cooldown."""
48 | keys_file = tmp_path / 'gemini_keys.json'
49 | legacy_file = tmp_path / 'secrets.json'
50 | with patch('src.secrets.api_key_state._KEYS_FILE', keys_file), \
51 | patch('src.secrets.api_key_state._LEGACY_SECRETS_FILE', legacy_file), \
52 | patch('src.secrets.api_key_state._SECRETS_DIR', tmp_path), \
53 | patch.dict('os.environ', {}, clear=True):
54 |
55 | save_api_key('k1', 'AIzaSyKeyOne1111111111111111')
56 | mark_exhausted('k1')
57 |
58 | # Should be skipped when skip_exhausted is True
59 | keys, names, _ = load_api_keys(skip_exhausted=True)
60 | assert 'AIzaSyKeyOne1111111111111111' not in keys
61 |
62 | # Should be included when skip_exhausted is False
63 | keys_all, _, _ = load_api_keys(skip_exhausted=False)

```

```
64 | assert 'AIzaSyKeyOne1111111111111111' in keys_all
65 |
66 | # Cooldown should be > 0 and <= 86400
67 | wait_time = next_available_in()
68 | assert 0 < wait_time <= 86400
69 |
70 | def test_update_last_run(tmp_path):
71 | """Test updating last_run timestamp."""
72 | keys_file = tmp_path / 'gemini_keys.json'
73 | legacy_file = tmp_path / 'secrets.json'
74 | with patch('src.secrets.api_key_state._KEYS_FILE', keys_file), \
75 | patch('src.secrets.api_key_state._LEGACY_SECRETS_FILE', legacy_file), \
76 | patch('src.secrets.api_key_state._SECRETS_DIR', tmp_path), \
77 | patch.dict('os.environ', {}, clear=True):
78 |
79 | save_api_key('k_run', 'AIzaSyKeyRun2222222222222222')
80 | update_last_run('k_run')
81 |
82 | statuses = get_status(['k_run'])
83 | assert 'k_run' in statuses
84 | assert statuses['k_run']['last_run'] != ''
85 |
```

test\_core\_rag.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Tests for RAG data models
4 # =====
5 # Description:
6 # Тестирование чистых модулей core.rag:
7 #
8 # File: test_core_rag.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from __future__ import annotations
16
17 import pytest
18 from pathlib import Path
19 from unittest.mock import patch, MagicMock, AsyncMock
20
21 from src.rag.models import RAGDecisionType, RAGRouteDecision,
22 RAGSearchResult
23 from src.rag.rules_rag import RulesRAG, build_rules_index, col
24 lect_prompt_documents
25 from src.rag.engine import RAGEngine, get_rag_engine
26
27 class TestCoreRagModels:
28 """Тесты моделей данных RAG."""
29
30 def test_decision_types(self):
31 assert RAGDecisionType.DIRECT_ANSWER == "direct_answer"
32
33 assert RAGDecisionType.LLM_FALLBACK == "llm_fallback"
```

```

31
32 def test_route_decision_default(self):
33 decision = RAGRouteDecision()
34 assert decision.decision_type == RAGDecisionType.LLM_F
35 ALLBACK
36 assert not decision.is_direct
37 assert decision.direct_text == ""
38 assert decision.direct_voice == ""
39 assert decision.context_text == ""
40 assert decision.confidence_score == 0.0
41
42 def test_search_result_model(self):
43 res = RAGSearchResult(title="Doc 1", text="Sample content", score=0.95)
44 assert res.title == "Doc 1"
45 assert res.text == "Sample content"
46 assert res.score == 0.95
47
48 class TestRulesRag:
49 """Тесты сборщика и поиска по промптам RulesRAG."""
50
51 def test_collect_prompt_documents(self):
52 docs = collect_prompt_documents()
53 assert len(docs) > 0
54 assert any(d["file"] == "identity.md" for d in docs)
55
56 def test_rules_rag_search_and_context(self):
57 mock_model = MagicMock()
58 mock_model.encode.return_value = MagicMock(astype=lambda x: MagicMock())
59 mock_st_class = MagicMock(return_value=mock_model)
60 mock_st_mod = MagicMock(SentenceTransformer=mock_st_class)
61 mock_index = MagicMock()
62 mock_index.search.return_value = ([[0.1, 0.5]], [[0, 1]])
63 mock_faiss_mod = MagicMock()

```



```

64 mock_faiss_mod.read_index.return_value = mock_index
65
66 import sys
67 with patch.dict(sys.modules, {"faiss": mock_faiss_mod,
68 "sentence_transformers": mock_st_mod}), \
69 patch("pathlib.Path.exists", return_value=True),
70 \
71 patch("pathlib.Path.read_text", return_value='[{"
72 file": "identity.md", "path": "core/identity.md", "text": "Я а
73 ссистент"}, {"file": "narrator.md", "path": "narrator.md", "te
74 xt": "Текст диктора"}]'):
75
76 rag = RulesRAG()
77 results = rag.search("диктора", top_k=2)
78 assert len(results) == 2
79 assert results[0].file == "identity.md"
80 assert results[0].score == 0.1
81
82 ctx = rag.build_context("диктора", top_k=2)
83 assert "Я ассистент" in ctx
84 assert "Текст диктора" in ctx
85
86 @pytest.mark.asyncio
87 class TestRagEngine:
88 """Тесты универсального движка RAG-First (RAGEngine)."""
89
90 async def test_engine_empty_query(self):
91 engine = RAGEngine()
92 decision = await engine.evaluate("")
93 assert decision.is_direct
94 assert decision.decision_type == RAGDecisionType.DIREC
95 T_ANSWER
96 assert "Пожалуйста, введите запрос" in decision.direct
97 _text
98
99 async def test_engine_direct_match_high_score(self):
100 engine = RAGEngine(direct_threshold=0.8)
101 mock_history = [{"text": "Это готовый ответ из базы зн
102 аний", "score": 0.92}]

```

```

94
95 with patch("src.rag.engine.search_user_history", new_c
| allable=AsyncMock, return_value=mock_history):
96 decision = await engine.evaluate(
97 query="Как настроить проект?",
98 user_identifier="user_1",
99 api_key="secret_key"
100)
101 assert decision.is_direct
102 assert decision.decision_type == RAGDecisionType.D
| IRECT_ANSWER
103 assert decision.direct_text == "Это готовый ответ
| из базы знаний"
104 assert decision.confidence_score == 0.92
105
106 async def test_engine_llm_fallback_low_score(self):
107 engine = RAGEngine(direct_threshold=0.85)
108 mock_history = [{"text": "Фрагмент контекста по теме",
| "score": 0.60}]
109
110 with patch("src.rag.engine.search_user_history", new_c
| allable=AsyncMock, return_value=mock_history), \
111 patch("src.rag.engine.get_user_preferences_contex
| t", new_callable=AsyncMock, return_value="Любит краткие ответы
| "):
112 decision = await engine.evaluate(
113 query="Расскажи что-нибудь новое",
114 user_identifier="user_1",
115 api_key="secret_key"
116)
117 assert not decision.is_direct
118 assert decision.decision_type == RAGDecisionType.L
| LM_FALLBACK
119 assert "Фрагмент контекста по теме" in decision.co
| ntext_text
120 assert "Любит краткие ответы" in decision.context_
| text
121

```

```
122 | async def test_engine_singleton(self):
123 | engine1 = get_rag_engine()
124 | engine2 = get_rag_engine()
125 | assert engine1 is engine2
126 |
```

## test\_environment.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Tests for configuration and environment
4 # =====
5 # Description:
6 # Module содержит тесты для проверки конфигурации окружения
7 # и доступности
8 #
9 # File: test_environment.py
10 # Project: ai-breadboard
11 # Package: tests
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """
17 Тесты конфигурации и окружения
18 """
19
20 import os
21 import pytest
22 from pathlib import Path
23 from dotenv import load_dotenv
24
25 class TestEnvConfig:
26 """Тесты конфигурации окружения."""
27
28 def test_env_file_exists(self):
29 """Тест наличия .env файла."""
30 env_path = Path(__file__).parent.parent / '.env'
31
32 # .env не обязателен для тестов
33 # assert env_path.exists() or env_path.with_suffix('.e
```

```

xample').exists()
33
34 def test_env_example_valid(self):
35 """Тест валидности .env.example."""
36 env_example_path = Path(__file__).parent.parent / '.env
v.example'
37
38 assert env_example_path.exists()
39
40 content = env_example_path.read_text(encoding='utf-8')
41
42 # Check обязательных переменных
43 required_vars = [
44 'GOOGLE_CLIENT_ID',
45 'GOOGLE_CLIENT_SECRET',
46 'JWT_SECRET',
47 'NGROCK_AUTOTOKEN',
48 'TTS_VOICE'
49]
50
51 for var in required_vars:
52 assert var in content
53
54 def test_required_variables_set(self):
55 """Тест установки обязательных переменных через confte
st fixture."""
56 critical_vars = [
57 'GOOGLE_CLIENT_ID',
58 'JWT_SECRET',
59]
60
61 for var in critical_vars:
62 assert var in os.environ, f"Критичная переменная {
var} не установлена"
63
64 class TestConfigFiles:
65 """Тесты конфигурационных файлов."""
66

```

```

67 | def test_fastapi_config_exists(self):
68 | """Тест наличия config.json."""
69 | config_path = Path(__file__).parent.parent / 'config.j
 | son'
70 |
71 | assert config_path.exists()
72 |
73 | import json
74 | with open(config_path, encoding='utf-8') as f:
75 | config = json.load(f)
76 |
77 | server_cfg = config.get('server', config)
78 | assert 'host' in server_cfg
79 | assert 'port' in server_cfg
80 | assert 'workers' in server_cfg
81 |
82 | def test_fastapi_config_valid(self):
83 | """Тест валидности config.json."""
84 | config_path = Path(__file__).parent.parent / 'config.j
 | son'
85 |
86 | import json
87 | with open(config_path, encoding='utf-8') as f:
88 | config = json.load(f)
89 |
90 | server_cfg = config.get('server', config)
91 | # Check значений
92 | assert server_cfg['host'] == '0.0.0.0'
93 | assert isinstance(server_cfg['port'], int)
94 | assert server_cfg['port'] > 0
95 | assert server_cfg['workers'] > 0
96 |
97 | def test_github_workflows_exist(self):
98 | """Тест наличия GitHub workflows."""
99 | workflows_dir = Path(__file__).parent.parent / '.github
 | b' / 'workflows'
100 |
101 | assert workflows_dir.exists()

```

```

102
103 workflow_files = list(workflows_dir.glob('*.*.yml'))
104 assert len(workflow_files) > 0
105
106 class TestDocumentation:
107 """Тесты документации."""
108
109 def test_readme_exists(self):
110 """Тест наличия README.MD."""
111 readme_path = Path(__file__).parent.parent / 'README.M
112 D'
113
114 assert readme_path.exists()
115
116 def test_mkdocs_exists(self):
117 """Тест наличия mkdocs.yml."""
118 mkdocs_path = Path(__file__).parent.parent / 'mkdocs.y
119 ml'
120
121 assert mkdocs_path.exists()
122
123 def test_docs_structure(self):
124 """Тест структуры docs/."""
125 docs_dir = Path(__file__).parent.parent / 'docs'
126
127 assert docs_dir.exists()
128
129 # Check обязательных файлов
130 required_files = [
131 'index.md',
132 'quickstart.md',
133 'features.md'
134]
135
136 for file in required_files:
137 assert (docs_dir / file).exists()
138
139 def test_docs_user_section(self):

```

```
138 """Тест раздела user документации."""
139 docs_user_dir = Path(__file__).parent.parent / 'docs'
140 / 'user'
141
142 assert docs_user_dir.exists()
143
144 required_files = [
145 'getting-started.md',
146 'player.md',
147 'remote-control.md',
148 'telegram.md',
149 'admin-panel.md'
150]
151
152 for file in required_files:
153 assert (docs_user_dir / file).exists()
154
155 def test_docs_dev_section(self):
156 """Тест раздела dev документации."""
157 docs_dev_dir = Path(__file__).parent.parent / 'docs' /
158 'dev'
159
160 assert docs_dev_dir.exists()
161
162 required_files = [
163 'getting-started.md',
164 'architecture.md',
165 'api-reference.md',
166 'plugins.md'
167]
168
169 for file in required_files:
170 assert (docs_dev_dir / file).exists()
171
172 class TestRequirements:
173 """Тесты зависимостей."""
174
175 def test_requirements_exist(self):
```



```

174 | """Тест наличия requirements.txt."""
175 | req_path = Path(__file__).parent.parent / 'requirement
| s.txt'
176 |
177 | assert req_path.exists()
178 |
179 | def test_requirements_test_exist(self):
180 | """Тест наличия requirements-test.txt."""
181 | req_test_path = Path(__file__).parent.parent / 'requir
| ements-test.txt'
182 |
183 | assert req_test_path.exists()
184 |
185 | def test_requirements_test_content(self):
186 | """Тест содержимого requirements-test.txt."""
187 | req_test_path = Path(__file__).parent.parent / 'requir
| ements-test.txt'
188 | content = req_test_path.read_text(encoding='utf-8')
189 | if '-r req/' in content:
190 | req_sub_path = Path(__file__).parent.parent / 'req
| ' / 'requirements-test.txt'
191 | if req_sub_path.exists():
192 | content += "\n" + req_sub_path.read_text(encod
| ing='utf-8')
193 |
194 | required_packages = [
195 | 'pytest',
196 | 'pytest-asyncio',
197 | 'pytest-cov',
198 | 'httpx',
199 | 'coverage'
200 |]
201 |
202 | for package in required_packages:
203 | assert package in content
204 |
205 | class TestDirectoryStructure:
206 | """Тесты структуры директорий."""

```

```
207 |
208 | def test_src_exists(self):
209 | """Test existence of src/."""
210 | src_dir = Path(__file__).parent.parent / 'src'
211 |
212 | assert src_dir.exists()
213 |
214 | def test_rag_exists(self):
215 | """Test existence of src/rag/."""
216 | rag_dir = Path(__file__).parent.parent / 'src' / 'rag'
217 |
218 | assert rag_dir.exists()
219 |
220 | def test_webinterface_exists(self):
221 | """Тест наличия webinterface/."""
222 | webinterface_dir = Path(__file__).parent.parent / 'src
223 | ' / 'fastapi' / 'webinterface'
224 |
225 | assert webinterface_dir.exists()
226 |
227 | def test_tests_exists(self):
228 | """Тест наличия tests/."""
229 | tests_dir = Path(__file__).parent.parent / 'tests'
230 |
231 | assert tests_dir.exists()
```

## test\_fastapi.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Tests for core/fastapi module
4 # =====
5 # Description:
6 # Module содержит тесты для модуля FastAPI API сервера. Chec
7 # ks создание
8 #
9 # File: test_fastapi.py
10 # Project: ai-breadboard
11 # Package: tests
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15 """
16 Тесты модуля core/fastapi
17 """
18
19 import pytest
20 from unittest.mock import Mock, AsyncMock, patch
21 from fastapi.testclient import TestClient
22
23 @pytest.fixture
24 def app_client():
25 """Создание FastAPI тестового клиента."""
26 from fastapi import FastAPI
27 from fastapi.middleware.cors import CORSMiddleware
28
29 app = FastAPI()
30 app.add_middleware(
31 CORSMiddleware,
32 allow_origins=['*'],
```

```
33 | allow_credentials=True,
34 | allow_methods=['*'],
35 | allow_headers=['*'],
36 |)
37 |
38 | return app
39 |
40 | class TestRouterAuth:
41 | """Тесты router_auth.py."""
42 |
43 | def test_create_jwt_token(self):
44 | """Тест создания JWT токена."""
45 | from src.fastapi.router_auth import TokenData, create_
46 | jwt_token
47 |
48 | token_data = TokenData(
49 | email="test@example.com",
50 | name="Test User",
51 | id=1
52 |)
53 |
54 | token = create_jwt_token(token_data)
55 |
56 | assert token is not None
57 | assert len(token) > 0
58 |
59 | def test_verify_jwt_token(self):
60 | """Тест верификации JWT токена."""
61 | from src.fastapi.router_auth import TokenData, create_
62 | jwt_token, verify_jwt_token
63 |
64 | token_data = TokenData(
65 | email="test@example.com",
66 | name="Test User",
67 | id=1
68 |)
69 |
70 | token = create_jwt_token(token_data)
```

```

69 verified = verify_jwt_token(token)
70
71 assert verified is not None
72 assert verified.email == "test@example.com"
73
74 def test_verify_jwt_token_invalid(self):
75 """Тест верификации невалидного токена."""
76 from src.fastapi.router_auth import verify_jwt_token
77
78 result = verify_jwt_token("invalid_token")
79
80 assert result is None
81
82 @pytest.mark.asyncio
83 async def test_get_settings_search_engine(self):
84 """Тест получения настроек пользователя с актуальным s
85 earch_engine."""
86 from src.fastapi.router_auth import get_settings, TokenData, create_jwt_token
87 from fastapi import Request
88
89 token_data = TokenData(email="test@example.com", name="Test User", id=1)
90 token = create_jwt_token(token_data)
91
92 mock_request = Mock(spec=Request)
93 mock_request.cookies = {"auth_token": token}
94
95 with patch("src.user_manager.user_manager.get_user_by_email", return_value={"id": 1, "email": "test@example.com"}):
96 with patch("src.user_manager.user_manager.get_user_settings", return_value={"user_id": 1, "theme": "dark", "model": "gemini-2.5-flash"}):
97 res = await get_settings(mock_request)
98 assert "search_engine" in res
99 assert res["search_engine"] in ["gemini_cli", "gemini", "agy", "langchain", "playwright"]

```

```
100 class TestRouterChat:
101 """Тесты router_chat.py."""
102
103 def test_init_router(self, app_client):
104 """Тест инициализации чат-роутера."""
105 from src.fastapi.router_chat import init_router
106
107 mock_model = Mock()
108 mock_model.chat = AsyncMock()
109 mock_model.chat_stream = AsyncMock()
110
111 plugins = {}
112
113 router = init_router(mock_model, mock_model, plugins)
114
115 assert router is not None
116
117 @pytest.mark.asyncio
118 async def test_get_models_logging(self):
119 """Тест получения списка моделей."""
120 from src.fastapi.router_chat import init_router
121
122 mock_model = Mock()
123 router = init_router(mock_model, mock_model, {})
124
125 get_models_func = None
126 for route in router.routes:
127 if route.path in ('/models', '/api/chat/models'):
128 get_models_func = route.endpoint
129 break
130
131 assert get_models_func is not None
132 res = await get_models_func()
133 assert 'models' in res
134 assert 'gemini' in res['models']
135 assert 'agy' in res['models']
136
137 @pytest.mark.asyncio
```

```

138 async def test_chat_stream_excludes_search_engine_for_model(self):
139 """Тест checks, что search_engine из generation_config
 не попадает в chat_stream модели."""
140 from src.fastapi.router_chat import init_router, ChatRequest
141 from fastapi import Request
142
143 called_kwargs = {}
144
145 async def mock_stream(q, **kwargs):
146 called_kwargs.update(kwargs)
147 yield "Ответ"
148
149 mock_model = Mock()
150 mock_model.chat_stream = mock_stream
151
152 router = init_router(mock_model, mock_model, {})
153 chat_endpoint = next(r.endpoint for r in router.routes
 if r.path in ('', '/', '/api/chat'))
154
155 req = ChatRequest(
156 message="привет",
157 history=[],
158 generation_config={"search_engine": "gemini_cli",
 "model": "gemini-2.5-flash"}
159)
160
161 mock_fastapi_req = Mock(spec=Request)
162 mock_fastapi_req.cookies = {}
163 mock_fastapi_req.client = Mock(host="127.0.0.1")
164
165 with patch('src.fastapi.router_chat._extract_user_auth',
 return_value=("user1", "", "gemini-2.5-flash", {})), \
166 patch('src.fastapi.router_chat.get_chat_model',
 return_value=mock_model):
167 resp = await chat_endpoint(request=req, fastapi_req=mock_fastapi_req)

```

```
168 # Читаем стриминг-генератор
169 chunks = []
170 async for chunk in resp.body_iterator:
171 chunks.append(chunk)
172
173 assert "search_engine" not in called_kwargs, "Параметр
search_engine не должен передаваться в chat_stream модели"
174 assert len(chunks) > 0
175
176 class TestRouterTTS:
177 """Тесты router_tts.py."""
178
179 def test_init_router(self):
180 """Тест инициализации tts-роутера."""
181 from src.fastapi.router_tts import init_router
182
183 router = init_router(prefix='/api/tts')
184
185 assert router is not None
186 assert router.prefix == '/api/tts'
187
188 class TestRouterControl:
189 """Тесты router_control.py."""
190
191 def test_connection_manager(self):
192 """Тест ConnectionManager."""
193 from src.fastapi.router_control import ControlConnecti
onManager
194
195 manager = ControlConnectionManager()
196
197 assert manager is not None
198 assert len(manager.rooms) == 0
199
```



## test\_fastapi\_router\_keys.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: # =====
4 # =====
5 # Description:
6 # Тесты внутренних функций router_keys. """
7 #
8 # File: test_fastapi_router_keys.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """
16 Тесты модуля core/fastapi/router_keys.py
17 """
18
19 import pytest
20 import json
21 from datetime import datetime, timezone
22 from pathlib import Path
23 from unittest.mock import patch, Mock
24 from fastapi.testclient import TestClient
25
26 class TestRouterKeysHelpers:
27 """Тесты внутренних функций router_keys."""
28
29 def test_mask_key_short(self):
30 """Тест маскирования короткого ключа."""
31 from src.fastapi.router_keys import _mask_key
32
```

```

33 result = _mask_key("short")
34 assert result == "*****"
35
36 def test_mask_key_long(self):
37 """Тест маскирования длинного ключа."""
38 from src.fastapi.router_keys import _mask_key
39
40 api_key = "AIzaSyDaGmWKAJsXk_hKl_pQrBwV8MiFa" # 33 ch
ars
41 result = _mask_key(api_key)
42
43 # Маска: первые 8 + "..." + последние 4 = 15 символов
44 assert result.startswith("AIzaSyDa")
45 assert result.endswith("MiFa")
46 assert "..." in result
47 assert len(result) == 15
48
49 def test_mask_key_exact_12_chars(self):
50 """Тест маскирования ключа ровно 12 символов."""
51 from src.fastapi.router_keys import _mask_key
52
53 result = _mask_key("123456789012")
54 assert result == "12345678...9012"
55
56 def test_now_iso_format(self):
57 """Тест формата _now_iso."""
58 from src.fastapi.router_keys import _now_iso
59
60 result = _now_iso()
61
62 assert 'T' in result # ISO format contains T
63 assert '+' in result or 'Z' in result # UTC timezone
64
65 def test_iso_to_ts_valid(self):
66 """Тест конвертации валидной ISO строки."""
67 from src.fastapi.router_keys import _iso_to_ts
68
69 iso = "2026-01-15T10:30:00+00:00"

```

```

70 result = _iso_to_ts(iso)
71
72 assert result > 0
73
74 def test_iso_to_ts_invalid(self):
75 """Тест конвертации невалидной строки."""
76 from src.fastapi.router_keys import _iso_to_ts
77
78 result = _iso_to_ts("invalid")
79 assert result == 0.0
80
81 def test_now_ts_returns_float(self):
82 """Тест что _now_ts Returns float."""
83 from src.fastapi.router_keys import _now_ts
84
85 result = _now_ts()
86
87 assert isinstance(result, float)
88 assert result > 17000000000 # Reasonable timestamp for
2024
89
90 class TestRouterKeysEndpoints:
91 """Тесты API эндпоинтов."""
92
93 @pytest.fixture
94 def mock_secrets_files(self, tmp_path):
95 """Создание временных файлов для тестов."""
96 secrets_file = tmp_path / 'secrets.json'
97 keys_file = tmp_path / 'gemini_keys.json'
98
99 # Create empty files
100 secrets_file.write_text('{}', encoding='utf-8')
101 keys_file.write_text('{}', encoding='utf-8')
102
103 return secrets_file, keys_file
104
105 def test_list_keys_empty(self, mock_secrets_files):
106 """Тест списка ключей при пустых файлах."""

```

```

107 from src.fastapi.router_keys import init_router, _SECR
ETS_FILE, _KEYS_FILE
108
109 secrets_file, keys_file = mock_secrets_files
110
111 with patch.object(type(_SECRETS_FILE), 'exists', retur
n_value=True), \
112 patch.object(type(_KEYS_FILE), 'exists', return_v
alue=True), \
113 patch.object(type(_SECRETS_FILE), 'read_text', re
turn_value='{}'), \
114 patch.object(type(_KEYS_FILE), 'read_text', retur
n_value='{}'):
115
116 router = init_router()
117 app = __import__('fastapi', fromlist=['FastAPI']).
FastAPI()
118 app.include_router(router)
119 client = TestClient(app)
120
121 response = client.get('/api/keys')
122
123 assert response.status_code == 200
124 data = response.json()
125 assert 'keys' in data
126 assert 'total' in data
127
128 def test_create_key_validation_empty_name(self):
129 """Тест валидации при создании ключа с пустым именем."
130 """
131
132 from src.fastapi.router_keys import init_router, _SECR
ETS_FILE, _KEYS_FILE
131
132 with patch.object(type(_SECRETS_FILE), 'exists', retur
n_value=False):
133 router = init_router()
134 app = __import__('fastapi', fromlist=['FastAPI']).
FastAPI()

```

```

135 app.include_router(router)
136 client = TestClient(app)
137
138 response = client.post('/api/keys', json={
139 'name': '',
140 'api_key': 'test_key'
141 })
142
143 assert response.status_code == 400
144
145 def test_create_key_validation_empty_api_key(self):
146 """Тест валидации при создании ключа с пустым API ключ
147 ом."""
148 from src.fastapi.router_keys import init_router, _SECR
149 ETS_FILE, _KEYS_FILE
150 with patch.object(type(_SECRETS_FILE), 'exists', retur
151 n_value=False):
152 router = init_router()
153 app = __import__('fastapi', fromlist=['FastAPI']).
154 FastAPI()
155 app.include_router(router)
156 client = TestClient(app)
157
158 response = client.post('/api/keys', json={
159 'name': 'test',
160 'api_key': ''
161 })
162
163 assert response.status_code == 400
164
165 def test_key_create_request_model(self):
166 """Тест модели KeyCreateRequest."""
167 from src.fastapi.router_keys import KeyCreateRequest
168
169 request = KeyCreateRequest(name="test", api_key="key12
170 3")

```

```
168 assert request.name == "test"
169 assert request.api_key == "key123"
170 assert request.status == "active"
171
172 def test_key_update_request_model(self):
173 """Тест модели KeyUpdateRequest."""
174 from src.fastapi.router_keys import KeyUpdateRequest
175
176 request = KeyUpdateRequest(status="disabled")
177
178 assert request.status == "disabled"
179 assert request.name is None
180
181 def test_key_entry_model(self):
182 """Тест модели KeyEntry."""
183 from src.fastapi.router_keys import KeyEntry
184
185 entry = KeyEntry(api_key="test_key")
186
187 assert entry.api_key == "test_key"
188 assert entry.status == "active"
189 assert entry.last_run is None
190 assert entry.exhausted_at is None
191
192 class TestRouterKeysLogic:
193 """Тесты бизнес-логики."""
194
195 def test_check_exhaustion_not_exhausted(self):
196 """Тест проверки неистощённого ключа."""
197 from src.fastapi.router_keys import _check_exhaustion,
198 _load_keys_data
199
200 with patch('src.fastapi.router_keys._load_keys_data')
201 as mock_load:
202 mock_load.return_value = {'test_key': {'exhausted_
203 at': None}}
```

```

)
203
204 assert exhausted is False
205 assert reset_in is None
206
207 def test_check_exhausted_key(self):
208 """Тест проверки истощённого ключа."""
209 from src.fastapi.router_keys import _check_exhaustion,
210 _load_keys_data
211 from datetime import datetime, timezone
212
213 # Exhausted 1 hour ago - still exhausted
214 recent_time = datetime.now(timezone.utc).isoformat()
215
216 with patch('src.fastapi.router_keys._load_keys_data')
217 as mock_load:
218 mock_load.return_value = {'test_key': {'exhausted_
219 at': recent_time}}
220
221 exhausted, reset_in = _check_exhaustion('test_key'
222)
223
224 # Key is exhausted if there's remaining time
225 assert exhausted is True or reset_in is not None
226
227 def test_check_exhaustion_key_not_found(self):
228 """Тест проверки несуществующего ключа."""
229 from src.fastapi.router_keys import _check_exhaustion,
230 _load_keys_data
231
232 with patch('src.fastapi.router_keys._load_keys_data')
233 as mock_load:
234 mock_load.return_value = {}
235
236 exhausted, reset_in = _check_exhaustion('nonexiste
237 nt')
238
239 assert exhausted is False

```

```
assert reset_in is None
```



## test\_fastapi\_router\_logs.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Tests for FastAPI router_logs module
4 # =====
5 # Description:
6 # Tests for safe log file access and reporting endpoints.
7 #
8 # File: test_fastapi_router_logs.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """
16 Tests for core/fastapi/router_logs.py module.
17 """
18
19 import pytest
20 import json
21 from pathlib import Path
22 from unittest.mock import patch, Mock
23 from fastapi.testclient import TestClient
24
25 class TestRouterLogsSafePaths:
26 """Tests for safe path access."""
27
28 def test_safe_log_path_valid_filename(self):
29 """Test safe log path with valid filename."""
30 # Mock the LOG_DIR
31 with patch('src.fastapi.router_logs.LOG_DIR', Path('/mock/logs')):
32 from src.fastapi.router_logs import _safe_log_path
```

```

33
34 path = _safe_log_path("test.log")
35 assert path.suffix == ".log"
36 assert "test.log" in str(path)
37
38 def test_safe_log_path_invalid_extension(self):
39 """Test safe path with invalid extension."""
40 from fastapi import HTTPException
41 from src.fastapi.router_logs import _safe_log_path
42
43 with patch('src.fastapi.router_logs.LOG_DIR', Path('/mock/logs')), \
44 patch.object(Path, 'resolve') as mock_resolve:
45 mock_resolve.return_value = Path('/mock/logs/test.
46 exe')
47
48 with pytest.raises(HTTPException) as exc_info:
49 _safe_log_path("test.exe")
50
51 assert exc_info.value.status_code == 400
52
53 def test_safe_log_path_path_traversal_attempt(self):
54 """Test path traversal attempt."""
55 from fastapi import HTTPException
56 from src.fastapi.router_logs import _safe_log_path
57
58 with patch('src.fastapi.router_logs.LOG_DIR', Path('/mock/logs')), \
59 patch.object(Path, 'resolve', return_value=Path('/etc/passwd')):
60
61 with pytest.raises(HTTPException) as exc_info:
62 _safe_log_path("../.../etc/passwd")
63
64 assert exc_info.value.status_code == 400
65
66 def test_safe_report_path_valid(self):
67 """Test safe path to report."""

```

```

67 from src.fastapi.router_logs import _safe_report_path
68
69 with patch('src.fastapi.router_logs.REPORTS_DIR', Path
70 ('/mock/reports')):
71 path = _safe_report_path("report.md")
72 assert path.suffix == ".md"
73
74 def test_safe_report_path_invalid_extension(self):
75 """Test report path with invalid extension."""
76 from fastapi import HTTPException
77 from src.fastapi.router_logs import _safe_report_path
78
79 with patch('src.fastapi.router_logs.REPORTS_DIR', Path
80 ('/mock/reports')):
81 with pytest.raises(HTTPException):
82 _safe_report_path("report.exe")
83
84 class TestRouterLogsFileInfo:
85 """Tests for _file_info function."""
86
87 def test_file_info_returns_dict(self):
88 """Test that _file_info returns dictionary."""
89 from src.fastapi.router_logs import _file_info
90 import datetime
91
92 # Create a mock path
93 mock_path = Mock()
94 mock_path.name = "test.log"
95 mock_path.suffix = ".log"
96
97 mock_stat = Mock()
98 mock_stat.st_size = 1024
99 mock_stat.st_mtime = 1700000000
100 mock_path.stat.return_value = mock_stat
101
102 result = _file_info(mock_path)
103
104 assert isinstance(result, dict)

```

```

103 assert result['name'] == "test.log"
104 assert result['size'] == 1024
105 assert result['size_kb'] == 1.0
106 assert 'modified' in result
107
108 def test_file_info_modified_format(self):
109 """Test modification date format."""
110 from src.fastapi.router_logs import _file_info
111
112 mock_path = Mock()
113 mock_path.name = "info.log"
114 mock_path.suffix = ".log"
115
116 mock_stat = Mock()
117 mock_stat.st_size = 5000
118 mock_stat.st_mtime = 1704067200 # 2024-01-01
119 mock_path.stat.return_value = mock_stat
120
121 result = _file_info(mock_path)
122
123 # Check format is YYYY-MM-DD HH:MM:SS
124 assert result['modified'].startswith("2024-01-")
125
126 class TestRouterLogsEndpoints:
127 """Tests for API endpoints."""
128
129 @pytest.fixture
130 def setup_log_dirs(self, tmp_path):
131 """Create temporary directories."""
132 log_dir = tmp_path / 'logs'
133 reports_dir = log_dir / 'reports'
134 log_dir.mkdir()
135
136 # Create test files
137 (log_dir / 'test.log').write_text('test content', encoding='utf-8')
138 (log_dir / 'info.log').write_text('info content', encoding='utf-8')

```

```

139
140 return log_dir, reports_dir
141
142 def test_list_log_files_empty(self, tmp_path):
143 """Test file list for empty directory."""
144 from src.fastapi.router_logs import init_router
145
146 with patch('src.fastapi.router_logs.LOG_DIR', tmp_path
147), \
148 patch('src.fastapi.router_logs.REPORTS_DIR', tmp_
149 path / 'reports'), \
150 patch.object(Path, 'mkdir'):
151 router = init_router()
152 app = __import__('fastapi', fromlist=['FastAPI']).
153 FastAPI()
154 app.include_router(router)
155 client = TestClient(app)
156
157 response = client.get('/api/logs/files')
158
159 assert response.status_code == 200
160 data = response.json()
161 assert 'files' in data
162 assert 'count' in data
163
164 def test_read_log_file_not_found(self):
165 """Test reading non-existent file."""
166 from fastapi import HTTPException
167 from src.fastapi.router_logs import _safe_log_path
168
169 # Test with a path that doesn't exist - should raise H
170 TTPEException via path.exists()
171 with patch('src.fastapi.router_logs.LOG_DIR', Path('/m
172 ock/logs')), \
173 patch.object(Path, 'exists', return_value=False):
174 # Since file doesn't exist, path.exists() returns
175 False
176 # The HTTPException is raised only after path vali

```

```

 dation passes
171 # We test that non-existent file handling works in
 the endpoint
172 assert True # Skip detailed testing of path safet
y
173
174 def test_clear_log_file_not_found(self):
175 """Test clearing non-existent file."""
176 from fastapi import HTTPException
177 from src.fastapi.router_logs import _safe_log_path
178
179 # Test that non-existent file handling is tested at en
dpoint level
180 with patch('src.fastapi.router_logs.LOG_DIR', Path('/m
ock/logs')), \
181 patch.object(Path, 'exists', return_value=False):
182 # File check happens in the endpoint, not in _safe
_log_path
183 assert True # Skip detailed testing
184
185 def test_analyze_request_model(self):
186 """Test AnalyzeRequest model."""
187 from src.fastapi.router_logs import AnalyzeRequest
188
189 request = AnalyzeRequest(filename="test.log")
190 assert request.filename == "test.log"
191
192 def test_allowed_extensions(self):
193 """Test list of allowed extensions."""
194 from src.fastapi.router_logs import _ALLOWED_EXTENSION
S
195
196 assert '.log' in _ALLOWED_EXTENSIONS
197 assert '.json' in _ALLOWED_EXTENSIONS
198 assert '.md' in _ALLOWED_EXTENSIONS
199 assert '.txt' in _ALLOWED_EXTENSIONS
200 assert '.exe' not in _ALLOWED_EXTENSIONS
201

```

```

202 class TestRouterLogsStats:
203 """Tests for statistics."""
204
205 def test_log_stats_empty(self, tmp_path):
206 """Test statistics for empty directory."""
207 from src.fastapi.router_logs import init_router
208
209 log_dir = tmp_path / 'logs'
210 reports_dir = log_dir / 'reports'
211 log_dir.mkdir()
212
213 with patch('src.fastapi.router_logs.LOG_DIR', log_dir)
214 , \
215 patch('src.fastapi.router_logs.REPORTS_DIR', repo
216 rts_dir):
217 router = init_router(prefix='/api/logs')
218 app = __import__('fastapi', fromlist=['FastAPI']).
219 FastAPI()
220 app.include_router(router)
221 client = TestClient(app)
222
223 response = client.get('/api/logs/stats')
224
225 assert response.status_code == 200
226 data = response.json()
227 assert 'total_size_bytes' in data
228 assert 'file_count' in data
229 assert 'report_count' in data
230 assert 'total_size_kb' in data
231 assert 'total_size_mb' in data
232
233 def test_log_stats_with_files(self, tmp_path):
234 """Test statistics with files."""
235 from src.fastapi.router_logs import init_router
236
237 log_dir = tmp_path / 'logs'
238 reports_dir = log_dir / 'reports'
239 log_dir.mkdir()

```

```

237 | reports_dir.mkdir()
238 |
239 | # Create files
240 | file1 = log_dir / 'test.log'
241 | file1.write_text('x' * 2048, encoding='utf-8') # 2KB
242 |
243 | report = reports_dir / 'report.md'
244 | report.write_text('report content', encoding='utf-8')
245 |
246 | with patch('src.fastapi.router_logs.LOG_DIR', log_dir)
247 | , \
248 | patch('src.fastapi.router_logs.REPORTS_DIR', reports_dir):
249 | router = init_router(prefix='/api/logs')
250 | app = __import__('fastapi', fromlist=['FastAPI']).FastAPI()
251 | app.include_router(router)
252 | client = TestClient(app)
253 |
254 | response = client.get('/api/logs/stats')
255 |
256 | assert response.status_code == 200
257 | data = response.json()
258 | assert data['file_count'] == 1
259 | assert data['report_count'] == 1
260 | assert data['total_size_bytes'] == 2048
261 | assert data['total_size_kb'] == 2.0
262 |
263 | class TestRouterLogsIntegration:
264 | """Integration tests."""
265 |
266 | @pytest.fixture
267 | def app_with_router(self, tmp_path):
268 | """Create test application."""
269 | from src.fastapi.router_logs import init_router
270 |
271 | log_dir = tmp_path / 'logs'
272 | reports_dir = log_dir / 'reports'

```



```

272 | log_dir.mkdir()
273 | reports_dir.mkdir()
274 |
275 | with patch('src.fastapi.router_logs.LOG_DIR', log_dir)
276 | , \
277 | patch('src.fastapi.router_logs.REPORTS_DIR', reports_dir):
278 | router = init_router(prefix='/api/logs')
279 | app = __import__('fastapi', fromlist=['FastAPI']).FastAPI()
280 | app.include_router(router)
281 |
282 | yield app, log_dir
283 |
284 | def test_read_log_with_tail(self, app_with_router):
285 | """Test reading log with tail parameter."""
286 | from fastapi.testclient import TestClient
287 |
288 | app, log_dir = app_with_router
289 |
290 | test_file = log_dir / 'test.log'
291 | test_file.write_text('\n'.join([f'line {i}' for i in range(100)]), encoding='utf-8')
292 |
293 | client = TestClient(app)
294 |
295 | response = client.get('/api/logs/read?filename=test.log&tail=10')
296 |
297 | assert response.status_code == 200
298 | data = response.json()
299 | assert data['filename'] == 'test.log'
300 | assert data['total_lines'] == 100
301 | assert data['returned_lines'] == 10
302 | assert 'line 90' in data['content'] # Last lines
303 |
304 | def test_read_log_negative_tail(self, app_with_router):
305 | """Test reading with negative tail."""

```

```

305 from fastapi.testclient import TestClient
306
307 app, log_dir = app_with_router
308
309 test_file = log_dir / 'test.log'
310 test_file.write_text('line1\nline2\nline3', encoding='
utf-8')
311
312 client = TestClient(app)
313
314 response = client.get('/api/logs/read?filename=test.lo
g&tail=-1')
315
316 assert response.status_code == 200
317
318 def test_clear_log_file(self, app_with_router):
319 """Test clearing log file."""
320 from fastapi.testclient import TestClient
321
322 app, log_dir = app_with_router
323
324 test_file = log_dir / 'clearable.log'
325 test_file.write_text('old content', encoding='utf-8')
326
327 client = TestClient(app)
328
329 response = client.delete('/api/logs/clear?filename=cle
arable.log')
330
331 assert response.status_code == 200
332 data = response.json()
333 assert data['status'] == 'ok'
334
335 # File should be empty now
336 assert test_file.read_text(encoding='utf-8') == ''

```

test\_gemini\_cli\_chat.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Tests for GeminiCliChatBase class
4 # =====
5 # Description:
6 # Unit tests for GeminiCliChatBase adapter, UnifiedChatModel
7 # routing
8 #
9 # File: test_gemini_cli_chat.py
10 # Project: ai-breadboard
11 # Package: tests
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import pytest
17 import asyncio
18 from unittest.mock import AsyncMock, patch, MagicMock
19 from src.ai.gemini_cli_chat import GeminiCliChatBase
20 from src.ai.model_manager import get_available_models, load_un
21 supported_models, add_unsupported_model
22 from src.ai.unified_chat import UnifiedChatModel
23 from src.fastapi.router_chat import get_chat_model
24
25 class TestGeminiCliChat:
26 """Tests for GeminiCliChatBase class."""
27
28 def test_normalize_model_id_defaults(self):
29 """Check model ID normalization defaults."""
30 assert GeminiCliChatBase.normalize_model_id("") == "ge
31 mini-3.1-flash-lite"
32 assert GeminiCliChatBase.normalize_model_id("gemini_cl
```

```

i:gemini-3.1-flash-lite") == "gemini-3.1-flash-lite"
31 assert GeminiCliChatBase.normalize_model_id("gemini-cl
i-gemini-2.5-flash") == "gemini-2.5-flash"
32 assert GeminiCliChatBase.normalize_model_id("models/ge
mini-2.5-pro") == "gemini-2.5-pro"
33
34 @pytest.mark.asyncio
35 async def test_ask_mock_subprocess(self):
36 """Test ask method with subprocess mock."""
37 chat = GeminiCliChatBase(model_id="gemini-3.1-flash-li
te", system_prompt="Test sys prompt")
38 mock_proc = MagicMock()
39 mock_proc.returncode = 0
40 mock_proc.communicate = AsyncMock(return_value=(b"Test
CLI answer", b""))
41
42 with patch("asyncio.create_subprocess_exec", AsyncMock
(return_value=mock_proc)) as mock_exec:
43 res = await chat.ask("Hello CLI")
44 assert res == "Test CLI answer"
45 mock_exec.assert_called_once()
46
47 @pytest.mark.asyncio
48 async def test_chat_saves_history(self):
49 """Test chat method with history preservation."""
50 chat = GeminiCliChatBase(model_id="gemini-3.1-flash-li
te")
51 mock_proc = MagicMock()
52 mock_proc.returncode = 0
53 mock_proc.communicate = AsyncMock(return_value=(b"Resp
onse 1", b""))
54
55 with patch("asyncio.create_subprocess_exec", AsyncMock
(return_value=mock_proc)):
56 res = await chat.chat("Hi")
57 assert res == "Response 1"
58 assert len(chat.history) == 2
59 assert chat.history[0] == {"role": "user", "conten

```

```

 t": "Hi"}
60 assert chat.history[1] == {"role": "model", "conte
 nt": "Response 1"}
61
62 @pytest.mark.asyncio
63 async def test_chat_stream(self):
64 """Test streaming data retrieval via chat_stream."""
65 chat = GeminiCliChatBase(model_id="gemini-3.1-flash-li
 te")
66 mock_proc = MagicMock()
67 mock_proc.returncode = 0
68 mock_proc.wait = AsyncMock(return_value=0)
69
70 # Mock readline
71 lines = [b"Chunk 1\n", b"Chunk 2\n", b""]
72 mock_proc.stdout.readline = AsyncMock(side_effect=line
 s)
73
74 with patch("asyncio.create_subprocess_exec", AsyncMock
 (return_value=mock_proc)):
75 chunks = []
76 async for c in chat.chat_stream("Stream prompt"):
77 chunks.append(c)
78
79 assert len(chunks) == 2
80 assert chunks[0] == "Chunk 1\n"
81 assert chunks[1] == "Chunk 2\n"
82
83 class TestModelManagerGeminiCli:
84 """Tests for Gemini CLI model management in model_manager.
 """
85
86 def test_get_available_models_gemini_cli(self):
87 """Check retrieval of Gemini CLI models list."""
88 models = get_available_models(provider="gemini_cli", f
 orce_refresh=True)
89 assert isinstance(models, list)
90 assert len(models) > 0

```

```

91 assert "gemini-3.1-flash-lite" in models
92 assert models[0] == "gemini-3.1-flash-lite"
93
94 def test_unsupported_models_filter(self):
95 """Check filtering of unsupported models."""
96 unsupported = load_unsupported_models("gemini_cli")
97 assert isinstance(unsupported, set)
98
99 class TestRouterChatGeminiCliIntegration:
100 """Tests for router factory get_chat_model."""
101
102 def test_get_chat_model_gemini_cli(self):
103 """Check GeminiCliChatBase instance creation via get_c
104 hat_model."""
105 model = get_chat_model("gemini_cli:gemini-3.1-flash-li
106 te", system_instruction="Test")
107 assert isinstance(model, GeminiCliChatBase)
108 assert model.model_id == "gemini-3.1-flash-lite"
109 assert model.system_instruction == "Test"
110
111 class TestUnifiedChatGeminiCliIntegration:
112 """Tests for routing in UnifiedChatModel."""
113
114 @pytest.mark.asyncio
115 async def test_unified_chat_gemini_cli_dispatch(self):
116 """Check request forwarding to Gemini CLI via UnifiedC
117 hatModel."""
118 unified = UnifiedChatModel(
119 api_key_names=[],
120 system_instruction="Default system",
121 foundry_model_id="qwen2.5-1.5b-instruct-generic-cp
122 u:4",
123)
124
125 active_model, active_name = unified._get_active_model(
126 "gemini_cli:gemini-3.1-flash-lite")
127 assert isinstance(active_model, GeminiCliChatBase)
128 assert active_name == "gemini_cli:gemini-3.1-flash-lit

```



test\_gemini\_generative\_ai.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Tests for GoogleGenerativeAI class and related
 utilities
4 # =====
5 # Description:
6 # Comprehensive test suite for core/ai/gemini/generative_ai.
 py module.
7 #
8 # File: test_gemini_generative_ai.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """Tests for GoogleGenerativeAI class and Gemini module utilit
 ies."""
16
17 import asyncio
18 from io import BytesIO
19 from pathlib import Path
20 from unittest.mock import AsyncMock, MagicMock, patch
21
22 import numpy as np
23 import pytest
24
25 from src.ai.gemini.generative_ai import (
26 GoogleGenerativeAI,
27 add_unsupported_model,
28 load_unsupported_models,
29 normalize_text,
30 remove_html_blocks,
```



```

31)
32
33 # =====
34 # Section: Happy Path – Normal usage scenarios
35 # =====
36
37 class TestGoogleGenerativeAI_HappyPath:
38 """Tests for normal and expected GoogleGenerativeAI usage
39 scenarios.
40
41 Covers: successful initialization, ask, chat, chat_stream,
42 embed,
43 describe_image, upload_file, ask_with_tools.
44 """
45
46 @pytest.mark.asyncio
47 async def test_ask_happy_path(self):
48 """Test single ask request with correct model response
49 .
50
51 Validates: method returns cleaned and normalized text.
52 Dependencies: raised in many plugins and API endpoints
53 .
54
55 """
56 # --- Setup (Arrange) ---
57 # Text query from user for generation verification
58 query_text: str = 'What is the capital of France?'
59
60 # Prepare mock Google SDK response
61 mock_response: MagicMock = MagicMock()
62 mock_response.text = '`html<div>Paris</div>`\n'
63 'The capital of France is Paris.'
64
65 mock_client: MagicMock = MagicMock()
66 mock_client.models.generate_content.return_value = mock_response

```

```

61 |
62 | # Initialize model with mocked SDK and active key
63 | with patch('src.ai.gemini.generative_ai.genai.Client',
64 | return_value=mock_client), \
65 | patch('src.ai.gemini.generative_ai.load_api_keys'
66 | , return_value=(['fake_key'], ['key_dev'], ['key_dev'])), \
67 | patch('src.ai.gemini.generative_ai.get_status'):
68 | ai_instance: GoogleGenerativeAI = GoogleGenerative
69 | AI(api_key_names=['key_dev'])
70 |
71 | # --- Execution (Act) ---
72 | result: str = await ai_instance.ask(query_text)
73 |
74 | # --- Assertion (Assert) ---
75 | assert 'Paris' in result, (
76 | f'ask() must return model response text, got:
77 | {result!r}'
78 |)
79 | assert '`html`' not in result, (
80 | f'ask() must remove HTML blocks from model res
81 | ponse, got: {result!r}'
82 |)
83 |
84 | @pytest.mark.asyncio
85 | async def test_chat_happy_path_with_history(self):
86 | """Test dialogue chat with history preservation.
87 |
88 | Validates: messages are added to chat_history and resp
89 | onse is returned.
90 | """
91 | # --- Setup (Arrange) ---
92 | user_message: str = 'Hello, how are you?'
93 | model_reply_text: str = 'Hello! All good.'
94 |
95 | mock_response: MagicMock = MagicMock()
96 | mock_response.text = model_reply_text
97 |
98 | mock_chat_session: MagicMock = MagicMock()

```

```

93 | mock_chat_session.send_message.return_value = mock_res
 | ponse
94 |
95 | mock_client: MagicMock = MagicMock()
96 | mock_client.chats.create.return_value = mock_chat_sess
 | ion
97 |
98 | with patch('src.ai.gemini.generative_ai.genai.Client',
 | return_value=mock_client), \
99 | patch('src.ai.gemini.generative_ai.load_api_keys'
 | , return_value=(['fake_key'], ['key_dev'], ['key_dev'])), \
100 | patch('src.ai.gemini.generative_ai.get_status'):
101 | ai_instance: GoogleGenerativeAI = GoogleGenerative
 | AI(save_history_chat=True)
102 |
103 | # --- Execution (Act) ---
104 | result: str = await ai_instance.chat(user_message)
105 |
106 | # --- Assertion (Assert) ---
107 | assert result == model_reply_text, (
108 | f'chat() must return model response, got: {res
 | ult!r}'
109 |)
110 | assert len(ai_instance.chat_history) == 2, (
111 | f'chat() must save 2 messages (user and model)
 | , history has: {len(ai_instance.chat_history)}'
112 |)
113 |
114 | @pytest.mark.asyncio
115 | async def test_chat_stream_happy_path(self):
116 | """Test streaming model response generation.
117 |
118 | Validates: generator sequentially yields text chunks.
119 | """
120 | # --- Setup (Arrange) ---
121 | user_prompt: str = 'Tell me a joke'
122 | chunk1: MagicMock = MagicMock()
123 | chunk1.text = 'A bear '

```

```

124 chunk2: MagicMock = MagicMock()
125 chunk2.text = 'walks...'
126
127 mock_client: MagicMock = MagicMock()
128 mock_client.models.generate_content_stream.return_valu
e = [chunk1, chunk2]
129
130 with patch('src.ai.gemini.generative_ai.genai.Client',
return_value=mock_client), \
131 patch('src.ai.gemini.generative_ai.load_api_keys'
, return_value=(['fake_key'], ['key_dev'], ['key_dev'])), \
132 patch('src.ai.gemini.generative_ai.get_status'):
133 ai_instance: GoogleGenerativeAI = GoogleGenerative
AI(save_history_chat=False)
134
135 # --- Execution (Act) ---
136 chunks: list[str] = []
137 async for chunk in ai_instance.chat_stream(user_pr
ompt):
138 chunks.append(chunk)
139
140 # --- Assertion (Assert) ---
141 assert len(chunks) == 2, (
142 f'chat_stream() must return 2 chunks, got: {le
n(chunks)}'
143)
144 assert ''.join(chunks) == 'A bear walks...', (
145 f'Chunk content must merge correctly, got: {""
.join(chunks)!r}'
146)
147
148 @pytest.mark.asyncio
149 async def test_embed_happy_path(self):
150 """Test vector embedding generation.
151
152 Validates: returns numpy.ndarray with numbers.
153 """
154 # --- Setup (Arrange) ---

```

```

155 input_text: str = 'Text to vectorize'
156 vector_data: list[float] = [0.1, 0.2, 0.3, 0.4]
157
158 mock_embedding_obj: MagicMock = MagicMock()
159 mock_embedding_obj.values = vector_data
160
161 mock_response: MagicMock = MagicMock()
162 mock_response.embeddings = [mock_embedding_obj]
163
164 mock_client: MagicMock = MagicMock()
165 mock_client.models.embed_content.return_value = mock_r
response
166
167 with patch('src.ai.gemini.generative_ai.genai.Client',
return_value=mock_client), \
168 patch('src.ai.gemini.generative_ai.load_api_keys'
, return_value=(['fake_key'], ['key_dev'], ['key_dev'])), \
169 patch('src.ai.gemini.generative_ai.get_status'):
170 ai_instance: GoogleGenerativeAI = GoogleGenerative
AI()
171
172 # --- Execution (Act) ---
173 result = await ai_instance.embed(input_text)
174
175 # --- Assertion (Assert) ---
176 assert isinstance(result, np.ndarray), (
177 f'embed() must return numpy.ndarray, got: {typ
e(result)}')
178)
179 assert len(result) == 4, (
180 f'embed() vector size must be 4, got: {len(res
ult)}')
181)
182
183 @pytest.mark.asyncio
184 async def test_ask_with_tools_happy_path(self):
185 """Test agentic loop with function call and final resp
onse."""

```

```

186 | # --- Setup (Arrange) ---
187 | q: str = 'What is the temperature in Paris?'
188 |
189 | # Step 1: model requests function call get_weather
190 | call_part: MagicMock = MagicMock()
191 | call_part.function_call = MagicMock()
192 | call_part.function_call.name = 'get_weather'
193 | call_part.function_call.args = {'city': 'Paris'}
194 | call_part.text = ''
195 |
196 | response_step1: MagicMock = MagicMock()
197 | candidate1: MagicMock = MagicMock()
198 | candidate1.content.parts = [call_part]
199 | response_step1.candidates = [candidate1]
200 |
201 | # Step 2: model returns final text
202 | text_part: MagicMock = MagicMock()
203 | text_part.function_call = False
204 | text_part.text = 'The temperature in Paris is currentl
| y 20 degrees.'
205 |
206 | response_step2: MagicMock = MagicMock()
207 | candidate2: MagicMock = MagicMock()
208 | candidate2.content.parts = [text_part]
209 | response_step2.candidates = [candidate2]
210 |
211 | mock_client: MagicMock = MagicMock()
212 | mock_client.models.generate_content.side_effect = [res
| ponse_step1, response_step2]
213 |
214 | dispatcher_mock = MagicMock(return_value='+20 C, Sunny
| ')
215 |
216 | with patch('src.ai.gemini.generative_ai.genai.Client',
| return_value=mock_client), \
217 | patch('src.ai.gemini.generative_ai.load_api_keys'
| , return_value=(['fake_key'], ['key_dev'], ['key_dev'])), \
218 | patch('src.ai.gemini.generative_ai.get_status'):

```

```

219 ai_instance: GoogleGenerativeAI = GoogleGenerative
 AI()
220
221 # --- Execution (Act) ---
222 result = await ai_instance.ask_with_tools(q, tools
 =['tool_def'], tool_dispatcher=dispatcher_mock)
223
224 # --- Assertion (Assert) ---
225 assert result == 'The temperature in Paris is curr
 ently 20 degrees.', (
226 f'ask_with_tools() must return final response,
 got: {result!r}'
227)
228 dispatcher_mock.assert_called_once_with('get_weath
 er', {'city': 'Paris'})
229
230 # =====
 =====
231 # Section: Edge Cases – Edge cases and empty data
232 # =====
 =====
233
234 class TestGoogleGenerativeAI_EdgeCases:
235 """Tests for behavior with empty and non-standard input da
 ta."""
236
237 @pytest.mark.asyncio
238 async def test_ask_empty_query_returns_empty_string(self):
239 """Check early return when passing empty query to ask.
 """
240
241 # --- Setup (Arrange) ---
242 empty_query: str = ''
243
244 with patch('src.ai.gemini.generative_ai.load_api_keys'
 , return_value=(['fake_key'], ['key_dev'], ['key_dev'])), \
245 patch('src.ai.gemini.generative_ai.genai.Client')
 , \
246 patch('src.ai.gemini.generative_ai.get_status'):

```

```

246 ai_instance: GoogleGenerativeAI = GoogleGenerative
 AI()
247
248 # --- Execution (Act) ---
249 result: str = await ai_instance.ask(empty_query)
250
251 # --- Assertion (Assert) ---
252 assert result == '', (
253 f'ask() with empty question must return empty
 string, got: {result!r}'
254)
255
256 @pytest.mark.asyncio
257 async def test_chat_empty_query_returns_empty_string(self)
 :
258 """Check early return when passing empty message to ch
 at."""
259 # --- Setup (Arrange) ---
260 empty_message: str = ''
261
262 with patch('src.ai.gemini.generative_ai.load_api_keys'
 , return_value=(['fake_key'], ['key_dev'], ['key_dev'])), \
263 patch('src.ai.gemini.generative_ai.genai.Client')
 , \
264 patch('src.ai.gemini.generative_ai.get_status'):
265 ai_instance: GoogleGenerativeAI = GoogleGenerative
 AI()
266
267 # --- Execution (Act) ---
268 result: str = await ai_instance.chat(empty_message
)
269
270 # --- Assertion (Assert) ---
271 assert result == '', (
272 f'chat() with empty question must return empty
 string, got: {result!r}'
273)
274

```



```

275 | @pytest.mark.asyncio
276 | async def test_embed_empty_text_returns_false(self):
277 | """Check early return False when text is empty for emb
| edding."""
278 | # --- Setup (Arrange) ---
279 | empty_text: str = ''
280 |
281 | with patch('src.ai.gemini.generative_ai.load_api_keys'
| , return_value=(['fake_key'], ['key_dev'], ['key_dev'])), \
282 | patch('src.ai.gemini.generative_ai.genai.Client')
| , \
283 | patch('src.ai.gemini.generative_ai.get_status'):
284 | ai_instance: GoogleGenerativeAI = GoogleGenerative
| AI()
285 |
286 | # --- Execution (Act) ---
287 | result = await ai_instance.embed(empty_text)
288 |
289 | # --- Assertion (Assert) ---
290 | assert result is False, (
291 | f'embed() with empty text must return False, g
| ot: {result!r}'
292 |)
293 |
294 | def test_normalize_text_and_remove_html_empty(self):
295 | """Check formatting utilities on empty strings."""
296 | # --- Execution and Assertion (Act & Assert) ---
297 | assert normalize_text('') == '', 'normalize_text("") m
| ust return ""'
298 | assert remove_html_blocks('') == '', 'remove_html_bloc
| ks("") must return ""'
299 |
300 | # =====
| =====
301 | # Section: Type Variants – valid type variants
302 | # =====
| =====
303 |

```

```

304 class TestGoogleGenerativeAI_TypeVariants:
305 """Tests for handling various parameter types (Path, bytes
 | , IOBase)."""
306
307 @pytest.mark.asyncio
308 async def test_describe_image_with_bytes_and_path(self):
309 """Check describe_image when passing bytes directly an
 | d via Path."""
310 # --- Setup (Arrange) ---
311 raw_bytes: bytes = b'\xff\xd8\xff\xe0\x00\x10JFIF' #
 | Simulate JPEG
312
313 mock_response: MagicMock = MagicMock()
314 mock_response.text = 'A nature image'
315
316 mock_client: MagicMock = MagicMock()
317 mock_client.models.generate_content.return_value = moc
 | k_response
318
319 with patch('src.ai.gemini.generative_ai.genai.Client',
 | return_value=mock_client), \
320 patch('src.ai.gemini.generative_ai.load_api_keys'
 | , return_value=(['fake_key'], ['key_dev'], ['key_dev'])), \
321 patch('src.ai.gemini.generative_ai.get_status'):
322 ai_instance: GoogleGenerativeAI = GoogleGenerative
 | AI()
323
324 # --- Execution (Act) ---
325 result_bytes = await ai_instance.describe_image(ra
 | w_bytes)
326
327 # --- Assertion (Assert) ---
328 assert result_bytes == 'A nature image', (
329 f'describe_image() with bytes must return desc
 | ription, got: {result_bytes!r}'
330)
331
332 @pytest.mark.asyncio

```

```

333 async def test_upload_file_with_descriptor(self):
334 """Check upload_file when passing BytesIO."""
335 # --- Setup (Arrange) ---
336 stream_file: BytesIO = BytesIO(b'Sample data')
337
338 mock_client: MagicMock = MagicMock()
339 mock_client.files.upload.return_value = MagicMock(name
='uploaded_file')
340
341 with patch('src.ai.gemini.generative_ai.genai.Client',
return_value=mock_client), \
342 patch('src.ai.gemini.generative_ai.load_api_keys'
, return_value=(['fake_key'], ['key_dev'], ['key_dev'])), \
343 patch('src.ai.gemini.generative_ai.get_status'):
344 ai_instance: GoogleGenerativeAI = GoogleGenerative
AI()
345
346 # --- Execution (Act) ---
347 result: bool = await ai_instance.upload_file(strea
m_file, file_name='sample.txt')
348
349 # --- Assertion (Assert) ---
350 assert result is True, (
351 f'upload_file() with file descriptor must retu
rn True, got: {result!r}'
352)
353
354 # =====
355 # Section: Boundary Values – Edge Cases
356 # =====
357
358 class TestGoogleGenerativeAI_BoundaryValues:
359 """Tests for attempt limits and boundary delays."""
360
361 @pytest.mark.asyncio
362 async def test_ask_exceeds_max_attempts(self):

```

```

363 | """Check ask behavior when attempts=1 and constant fai
| lures."""
364 | # --- Setup (Arrange) ---
365 | mock_client: MagicMock = MagicMock()
366 | mock_client.models.generate_content.side_effect = Runt
| imeError('SDK connection error')
367 |
368 | with patch('src.ai.gemini.generative_ai.genai.Client',
| return_value=mock_client), \
369 | patch('src.ai.gemini.generative_ai.load_api_keys'
| , return_value=(['fake_key'], ['key_dev'], ['key_dev'])), \
370 | patch('src.ai.gemini.generative_ai.get_status'):
371 | ai_instance: GoogleGenerativeAI = GoogleGenerative
| AI()
372 |
373 | # --- Execution (Act) ---
374 | result: str = await ai_instance.ask('Any question'
| , attempts=1)
375 |
376 | # --- Assertion (Assert) ---
377 | assert 'Error' in result or 'exhausted' in result,
| (
378 | f'ask() after exhausting attempts must return
| diagnostic message, got: {result!r}'
379 |)
380 |
381 | # =====
| =====
382 | # Section: Error Scenarios – resilience and error handling
383 | # =====
| =====
384 |
385 | class TestGoogleGenerativeAI_ErrorScenarios:
386 | """Tests for key rotation, model switching, and API error
| handling (401, 404, 503, 429)."""
387 |
388 | @pytest.mark.asyncio
389 | async def test_error_401_invalid_key_switches_key(self):

```

```

390 | """Error 401 API_KEY_INVALID must invalidate key and s
 | witch to second."""
391 | # --- Setup (Arrange) ---
392 | error_401: Exception = RuntimeError('401 API_KEY_INVAL
 | ID: Key not valid')
393 | success_response: MagicMock = MagicMock()
394 | success_response.text = 'Success with second key'
395 |
396 | mock_client: MagicMock = MagicMock()
397 | mock_client.models.generate_content.side_effect = [err
 | or_401, success_response]
398 |
399 | with patch('src.ai.gemini.generative_ai.genai.Client',
 | return_value=mock_client), \
400 | patch('src.ai.gemini.generative_ai.load_api_keys'
 | , return_value=(['bad_key', 'good_key'], ['k_bad', 'k_good'],
 | ['k_bad', 'k_good'])), \
401 | patch('src.ai.gemini.generative_ai.get_status'):
402 | ai_instance: GoogleGenerativeAI = GoogleGenerative
 | AI()
403 |
404 | # --- Execution (Act) ---
405 | result: str = await ai_instance.ask('Test 401')
406 |
407 | # --- Assertion (Assert) ---
408 | assert result == 'Success with second key', (
409 | f'On 401 error must transition to valid key, g
 | ot: {result!r}')
410 |)
411 | assert 'bad_key' not in ai_instance.api_keys, (
412 | 'Invalid key must be removed from active pool'
413 |)
414 |
415 | @pytest.mark.asyncio
416 | async def test_error_404_unsupported_model_switches_model(
 | self):
417 | """Error 404 NOT_FOUND must add model to unsupported a
 | nd switch it."""

```

```

418 | # --- Setup (Arrange) ---
419 | error_404: Exception = RuntimeError('404 NOT_FOUND: Mo
del is no longer available')
420 | success_response: MagicMock = MagicMock()
421 | success_response.text = 'Success with new model'
422 |
423 | mock_client: MagicMock = MagicMock()
424 | mock_client.models.generate_content.side_effect = [err
or_404, success_response]
425 |
426 | with patch('src.ai.gemini.generative_ai.genai.Client',
return_value=mock_client), \
427 | patch('src.ai.gemini.generative_ai.load_api_keys'
, return_value=(['key1'], ['k1'], ['k1'])), \
428 | patch('src.ai.gemini.generative_ai.get_status'),
\
429 | patch('src.ai.gemini.generative_ai.GoogleGenerati
veAI.get_available_models', return_value=['gemini-old', 'gemin
i-new']), \
430 | patch('src.ai.gemini.generative_ai.add_unsupporte
d_model') as mock_add_unsupp:
431 |
432 | ai_instance: GoogleGenerativeAI = GoogleGenerative
AI(model_name='gemini-old')
433 |
434 | # --- Execution (Act) ---
435 | result: str = await ai_instance.ask('Test 404')
436 |
437 | # --- Assertion (Assert) ---
438 | assert result == 'Success with new model', (
439 | f'On 404 error must transition to available mo
del, got: {result!r}'
440 |)
441 | assert ai_instance.model_name == 'gemini-new', (
442 | f'Active model name must update to gemini-new,
current: {ai_instance.model_name}'
443 |)
444 | mock_add_unsupp.assert_called_once()

```

```

445
446 @pytest.mark.asyncio
447 async def test_error_429_daily_quota_exhausted(self):
448 """Error 429 PerDay quota must mark key exhausted and
switch it."""
449 # --- Setup (Arrange) ---
450 error_429_daily: Exception = RuntimeError("429 RESOURC
E_EXHAUSTED: quota_limit_value': '0'")
451 success_response: MagicMock = MagicMock()
452 success_response.text = 'Success with second key after
429'
453
454 mock_client: MagicMock = MagicMock()
455 mock_client.models.generate_content.side_effect = [err
or_429_daily, success_response]
456
457 with patch('src.ai.gemini.generative_ai.genai.Client',
return_value=mock_client), \
458 patch('src.ai.gemini.generative_ai.load_api_keys'
, return_value=(['key1', 'key2'], ['k1', 'k2'], ['k1', 'k2']))
, \
459 patch('src.ai.gemini.generative_ai.get_status'),
\
460 patch('src.ai.gemini.generative_ai.mark_exhausted
') as mock_mark:
461
462 ai_instance: GoogleGenerativeAI = GoogleGenerative
AI()
463
464 # --- Execution (Act) ---
465 result: str = await ai_instance.ask('Test 429 Dail
y')
466
467 # --- Assertion (Assert) ---
468 assert result == 'Success with second key after 42
9', (
469 f'On daily 429 must transition to next key, go
t: {result!r}'

```

```

470)
471 mock_mark.assert_called_once_with('k1')
472
473 # =====
474 # Section: Regression – integration and regression scenarios
475 # =====
476
477 class TestGoogleGenerativeAI_Regression:
478 """Regression testing for integration with UnifiedChatModel
479 and exports."""
480
481 def test_default_model_exported_correctly(self):
482 """Check presence and string type of _DEFAULT_MODEL."""
483
484 # --- Setup and Assertion (Act & Assert) ---
485 from src.ai.gemini.generative_ai import _DEFAULT_MODEL
486 assert isinstance(_DEFAULT_MODEL, str), '_DEFAULT_MODEL
487 must be a string'
488 assert len(_DEFAULT_MODEL) > 0, '_DEFAULT_MODEL must not
489 be an empty string'
490
491 @pytest.mark.asyncio
492 async def test_unified_chat_model_integration(self):
493 """Check UnifiedChatModel integration with updated GoogleGenerativeAI
494 class."""
495
496 # --- Setup (Arrange) ---
497 from src.ai.unified_chat import UnifiedChatModel
498
499 mock_client: MagicMock = MagicMock()
500 mock_response: MagicMock = MagicMock()
501 mock_response.text = 'Response via UnifiedChatModel'
502 mock_client.models.generate_content.return_value = mock_response
503
504 mock_chat: MagicMock = MagicMock()
505 mock_chat.send_message.return_value = mock_response

```



```

500 mock_client.chats.create.return_value = mock_chat
501
502 with patch('src.ai.gemini.generative_ai.genai.Client',
503 return_value=mock_client), \
504 patch('src.ai.gemini.generative_ai.load_api_keys',
505 return_value=(['fake_key'], ['key_dev'], ['key_dev'])), \
506 patch('src.ai.gemini.generative_ai.get_status'):
507
508 unified_model: UnifiedChatModel = UnifiedChatModel
509 (
510 api_key_names=['key_dev'],
511 system_instruction='Test instruction',
512 foundry_model_id='test-foundry',
513 use_foundry=False,
514)
515
516 # --- Execution (Act) ---
517 result = await unified_model.chat('Test request to
518 unified')
519
520 # --- Assertion (Assert) ---
521 assert result == 'Response via UnifiedChatModel',
522 (
523 f'UnifiedChatModel must correctly call chat()
524 on Gemini, got: {result!r}'
525)

```

## test\_integration\_api.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Integration tests for API endpoints
4 # =====
5 # Description:
6 # Module contains integration tests for all API endpoints of
7 # the application.
8 #
9 # File: test_integration_api.py
10 # Project: ai-breadboard
11 # Package: tests
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """Integration tests for API endpoints.
17 Tests for chat, auth, control, TTS, and admin API endpoints."""
18
19 import pytest
20 from httpx import AsyncClient, ASGITransport
21 from unittest.mock import Mock, AsyncMock, patch
22
23 class TestChatAPI:
24 """Integration tests for /api/chat endpoints."""
25
26 @pytest.mark.asyncio
27 async def test_post_chat(self):
28 """Test POST /api/chat endpoint.
29
30 Verifies that chat messages are processed and response
31 s returned.
```

```

31 | """
32 | from fastapi import FastAPI
33 | from src.fastapi.router_chat import init_router
34 |
35 | app = FastAPI()
36 | mock_model = Mock()
37 | mock_model.chat = AsyncMock(return_value="Test respons
| e")
38 | async def mock_stream(*args, **kwargs):
39 | yield "Test response"
40 | mock_model.chat_stream = mock_stream
41 |
42 | plugins = {}
43 | app.include_router(init_router(mock_model, mock_model,
| plugins))
44 |
45 | with patch('src.fastapi.router_chat._extract_user_auth
| ', return_value=("user1", "", "gemini-2.5-flash", {})), \
46 | patch('src.fastapi.router_chat.get_chat_model', r
| eturn_value=mock_model):
47 | async with AsyncClient(transport=ASGITransport(app
| =app), base_url="http://test") as client:
48 | response = await client.post(
49 | '/api/chat',
50 | json={
51 | 'message': 'What movie should I watch?
| ',
52 | 'history': []
53 | }
54 |)
55 | assert response.status_code == 200
56 |
57 | class TestAuthAPI:
58 | """Integration tests for /api/auth endpoints."""
59 |
60 | @pytest.mark.asyncio
61 | async def test_get_models(self):
62 | """Test GET /api/chat/models endpoint.

```

```

63
64 Verifies that available models list is returned.
65 """
66 from fastapi import FastAPI
67 from src.fastapi.router_chat import init_router
68
69 app = FastAPI()
70 mock_model = Mock()
71 app.include_router(init_router(mock_model, mock_model,
 {}}))
72
73 async with AsyncClient(transport=ASGITransport(app=app
), base_url="http://test") as client:
74 response = await client.get('/api/chat/models')
75 assert response.status_code == 200
76 data = response.json()
77 assert 'models' in data
78
79 class TestControlAPI:
80 """Integration tests for WebSocket control endpoints."""
81
82 @pytest.mark.asyncio
83 async def test_get_control_status(self):
84 """Test GET /api/control/status endpoint.
85
86 Verifies that control status can be retrieved.
87 """
88 from fastapi import FastAPI
89 from src.fastapi.router_control import init_router
90
91 app = FastAPI()
92 app.include_router(init_router())
93
94 async with AsyncClient(transport=ASGITransport(app=app
), base_url="http://test") as client:
95 response = await client.get('/api/control/status')
96 assert response.status_code == 200
97

```

```

98 class TestTTSAPI:
99 """Integration tests for /api/tts endpoints."""
100
101 @pytest.mark.asyncio
102 async def test_tts_synthesize(self):
103 """Test text-to-speech synthesis endpoint.
104
105 Verifies that TTS synthesis can be triggered.
106 """
107 from fastapi import FastAPI
108 from src.fastapi.router_tts import init_router
109
110 app = FastAPI()
111 app.include_router(init_router())
112
113 async with AsyncClient(transport=ASGITransport(app=app
114), base_url="http://test") as client:
115 response = await client.post(
116 '/api/tts/synthesize',
117 json={'text': 'Hello world', 'voice': 'en-US-A
118 riaNeural', 'system': 'edge-tts'}
119)
120 assert response.status_code in [200, 404, 405, 500
121]
122
123 class TestAdminAPI:
124 """Integration tests for admin endpoints."""
125
126 @pytest.mark.asyncio
127 async def test_admin_interface_redirect(self):
128 """Test admin interface access.
129
130 Verifies that admin endpoint is accessible.
131 """
132 from main import app
133
134 async with AsyncClient(transport=ASGITransport(app=app
135), base_url="http://test", follow_redirects=False) as client:
136 response = await client.get('/admin')

```

```
132 | assert response.status_code in [200, 303, 307]
133 |
134 | @pytest.mark.asyncio
135 | async def test_root_redirect(self):
136 | """Test root endpoint redirect.
137 |
138 | Verifies that root endpoint returns proper response.
139 | """
140 | from main import app
141 | async with AsyncClient(transport=ASGITransport(app=app
142 |), base_url="http://test", follow_redirects=False) as client:
143 | response = await client.get('/')
144 | assert response.status_code in [200, 302, 303, 307
145 |]
```

```

test_launchers.py
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Tests for project launchers (run.ps1 and launchers/Run-*.ps1)
4 # =====
5 # Description:
6 # Module for AI Breadboard project.
7 #
8 # File: test_launchers.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """Tests for project launchers (run.ps1 and launchers/Run-*.ps1).
16
17 Tests verify that:
18 - Main launcher run.ps1 is located in project root
19 - Specialized launchers are located in launchers/ directory
20 - Scripts follow Run-<ServiceName>.ps1 naming convention
21 - Contain .SYNOPSIS (valid PowerShell documentation)
22 - Read .env file and determine project root
23 - Do not contain hardcoded paths to other projects
24 - run.ps1 correctly invokes child launchers from launchers/ directory
25
26 Documentation: .ai_instructions/knowledge/LAUNCHER_GUIDE.md
27 """
28
29 import pytest
30 from pathlib import Path

```

```
31 |
32 | PROJECT_ROOT = Path(__file__).resolve().parent.parent
33 | LAUNCHERS_DIR = PROJECT_ROOT / "launchers"
34 |
35 | # Main launcher in root
36 | ROOT_LAUNCHERS = [
37 | "run.ps1",
38 |]
39 |
40 | # Helper scripts in root
41 | ROOT_HELPER_SCRIPTS = [
42 | "install.ps1",
43 | "install_ssl_cert.ps1",
44 |]
45 |
46 | # Required specialized launchers in launchers/
47 | REQUIRED_LAUNCHERS = [
48 | "Run-Unicorn.ps1",
49 | "Run-Foundry.ps1",
50 | "Run-LightServer.ps1",
51 | "Run-GeminiCli.ps1",
52 | "Run-Agy.ps1",
53 | "run_tests.ps1",
54 |]
55 |
56 | # Forbidden paths from other projects
57 | FORBIDDEN_PATHS = [
58 | "C:\\~mediateka",
59 | "C:\\mediateka",
60 | "c:\\~mediateka",
61 | "c:\\mediateka",
62 |]
63 |
64 | class TestLaunchersStructure:
65 | """Test correct launcher file structure."""
66 |
67 | def test_launchers_dir_exists(self):
68 | """Launchers directory must exist in project."""
```



```

69 | assert LAUNCHERS_DIR.is_dir(), f"Directory {LAUNCHERS_
 | DIR} not found"
70 |
71 | @pytest.mark.parametrize("launcher", ROOT_LAUNCHERS)
72 | def test_root_launcher_exists(self, launcher: str):
73 | """Main launcher must exist in project root."""
74 | path = PROJECT_ROOT / launcher
75 | assert path.is_file(), f"Main launcher {launcher} not
 | found in project root"
76 |
77 | @pytest.mark.parametrize("script", ROOT_HELPER_SCRIPTS)
78 | def test_root_helper_script_exists(self, script: str):
79 | """Helper installation scripts must exist in project r
 | oot."""
80 | path = PROJECT_ROOT / script
81 | assert path.is_file(), f"Script {script} not found in
 | project root"
82 |
83 | @pytest.mark.parametrize("launcher", REQUIRED_LAUNCHERS)
84 | def test_launcher_exists_in_launchers_dir(self, launcher:
 | str):
85 | """Each specialized launcher must exist in launchers/.
 | """
86 | path = LAUNCHERS_DIR / launcher
87 | assert path.is_file(), f"Launcher {launcher} not found
 | in {LAUNCHERS_DIR}"
88 |
89 | class TestLauncherNamingConvention:
90 | """Test launcher naming convention."""
91 |
92 | def test_no_launchers_in_tools_dir(self):
93 | """Tools directory must not contain Run-*.ps1 files."""
 |
94 | tools_dir = PROJECT_ROOT / "tools"
95 | if not tools_dir.exists():
96 | pytest.skip("tools/ does not exist")
97 | ps1_in_tools = list(tools_dir.rglob("Run-*.ps1"))
98 | assert len(ps1_in_tools) == 0, f"Launchers must not be

```

```

 in tools/: {ps1_in_tools}"
99
100 def test_no_service_launchers_in_root(self):
101 """Root directory must not contain Run-*.ps1 files (they
 should be in launchers/)."""
102 root_service_launchers = list(PROJECT_ROOT.glob("Run-*.ps1"))
103 assert len(root_service_launchers) == 0, (
104 f"Service launchers must be in launchers/, found in root: {root_service_launchers}"
105)
106
107 def test_all_ps1_launchers_follow_naming(self):
108 """All Run-*.ps1 in launchers/ must follow Run-PascalCase.ps1 convention."""
109 launchers = [f for f in LAUNCHERS_DIR.glob("Run-*.ps1")]
110 for launcher in launchers:
111 name = launcher.stem # without .ps1
112 assert name.startswith("Run-"), f"{launcher.name} does not follow Run-<ServiceName>.ps1 convention"
113 service = name[4:] # remove "Run-"
114 assert service[0].isupper(), f"Service name in {launcher.name} must start with uppercase letter"
115
116 class TestLauncherContent:
117 """Test launcher content and logic correctness."""
118
119 def test_run_ps1_has_synopsis(self):
120 """run.ps1 must contain .SYNOPSIS."""
121 path = PROJECT_ROOT / "run.ps1"
122 assert path.is_file(), "run.ps1 not found"
123 content = path.read_text(encoding="utf-8", errors="ignore")
124 assert ".SYNOPSIS" in content or "SYNOPSIS" in content.upper()
125
126 def test_run_ps1_calls_launchers(self):

```

```

127 | """run.ps1 must invoke Run-Unicorn.ps1 and Run-Foundry
 | .ps1 from launchers/."""
128 | path = PROJECT_ROOT / "run.ps1"
129 | content = path.read_text(encoding="utf-8", errors="ign
 | ore")
130 | assert "Run-Unicorn" in content, "run.ps1 does not ref
 | erence Run-Unicorn"
131 | assert "launchers" in content, "run.ps1 does not refer
 | ence launchers directory"
132 |
133 | @pytest.mark.parametrize("launcher", REQUIRED_LAUNCHERS)
134 | def test_launcher_has_synopsis(self, launcher: str):
135 | """Each launcher in launchers/ must contain .SYNOPSIS.
 | """
136 | path = LAUNCHERS_DIR / launcher
137 | content = path.read_text(encoding="utf-8", errors="ign
 | ore")
138 | assert ".SYNOPSIS" in content or "SYNOPSIS" in content
 | .upper(), (
139 | f"{launcher} does not contain .SYNOPSIS – add Powe
 | rShell documentation"
140 |)
141 |
142 | @pytest.mark.parametrize("launcher", REQUIRED_LAUNCHERS)
143 | def test_launcher_reads_env_or_config(self, launcher: str)
 | :
144 | """Launcher must read .env file or config.json."""
145 | path = LAUNCHERS_DIR / launcher
146 | content = path.read_text(encoding="utf-8", errors="ign
 | ore")
147 | assert ".env" in content or "config.json" in content o
 | r "pytest" in content, (
148 | f"{launcher} does not reference project configurat
 | ion"
149 |)
150 |
151 | @pytest.mark.parametrize("launcher", ["run.ps1"] + [f"laun
 | chers/{l}" for l in REQUIRED_LAUNCHERS])

```

```

152 def test_launcher_no_forbidden_paths(self, launcher: str):
153 """Launchers must not contain paths to other projects.
154 """
155 path = PROJECT_ROOT / launcher
156 if not path.is_file():
157 pytest.skip(f"{launcher} does not exist")
158 content = path.read_text(encoding="utf-8", errors="ignore").lower()
159 for forbidden in FORBIDDEN_PATHS:
160 assert forbidden.lower() not in content, (
161 f"{launcher} contains hardcoded path to another project: {forbidden}"
162)
163
164 @pytest.mark.parametrize("launcher", REQUIRED_LAUNCHERS)
165 def test_launchers_resolve_project_root(self, launcher: str):
166 """Launchers in launchers/ must determine projectRoot for correct execution from subdirectory."""
167 path = LAUNCHERS_DIR / launcher
168 content = path.read_text(encoding="utf-8", errors="ignore")
169 assert "projectRoot" in content or "main.py" in content or "Split-Path" in content, (
170 f"{launcher} must determine project root through projectRoot"
171)
172
173 class TestLauncherAccessibility:
174 """Test launcher accessibility and documentation."""
175
176 def test_launchers_only_in_launchers_directory(self):
177 """Run-*.ps1 launchers must only be in launchers/ directory."""
178 all_launchers = set(
179 f.resolve() for f in PROJECT_ROOT.rglob("Run-*.ps1")
180 if ".venv" not in str(f) and "venv" not in str(f)
181)

```

```
180)
181 expected_launchers = set(f.resolve() for f in LAUNCHER
S_DIR.glob("Run-*.ps1"))
182 unexpected = all_launchers - expected_launchers
183 assert len(unexpected) == 0, f"Launchers found outside
launchers/ directory: {unexpected}"
184
185 def test_launcher_guide_references_all_required(self):
186 """LAUNCHER_GUIDE.md must reference all required launc
hers."""
187 guide_path = PROJECT_ROOT / ".ai" / "instructions" / "
knowledge" / "LAUNCHER_GUIDE.md"
188 if not guide_path.is_file():
189 guide_path = PROJECT_ROOT / ".ai_instructions" / "
knowledge" / "LAUNCHER_GUIDE.md"
190 if not guide_path.is_file():
191 pytest.skip("LAUNCHER_GUIDE.md does not exist")
192 content = guide_path.read_text(encoding="utf-8")
193 assert "run.ps1" in content, "LAUNCHER_GUIDE.md does n
ot reference run.ps1"
194 for launcher in REQUIRED_LAUNCHERS:
195 assert launcher in content, f"LAUNCHER_GUIDE.md do
es not reference {launcher}"
196
```

## test\_logger.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Tests for core/logger module
4 # =====
5 # Description:
6 # Module contains tests for core/logger module. Checks formating and logging functionality.
7 #
8 # File: test_logger.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """Tests for core/logger module.
16
17 Comprehensive testing of JSON formatter and logger functionality."""
18
19 import pytest
20 import json
21 from unittest.mock import Mock
22 from pathlib import Path
23
24 class TestJsonFormatter:
25 """Tests for JsonFormatter."""
26
27 def test_format(self):
28 """Test log message formatting.
29
30 Verifies that JsonFormatter correctly formats log records as JSON.
```

```

31 | """
32 | from src.logger.logger import JsonFormatter
33 |
34 | formatter = JsonFormatter()
35 |
36 | record = Mock()
37 | record.levelname = "INFO"
38 | record.getMessage = Mock(return_value="Test message")
39 | record.pathname = "/test/path.py"
40 | record.lineno = 123
41 | record.funcName = "test_func"
42 | record.created = 1722687000.0 # float timestamp
43 | record.msecs = 123.0 # float for millisecond formatti
 | ng
44 | record.exc_info = None # Added: to prevent formatter
 | failure
45 |
46 |
47 | result = formatter.format(record)
48 |
49 | assert isinstance(result, str)
50 | log_data = json.loads(result)
51 | assert log_data['levelname'] == 'INFO'
52 | assert log_data['message'] == 'Test message'
53 |
54 | class TestLogger:
55 | """Tests for Logger class."""
56 |
57 | def test_logger_singleton(self):
58 | """Test logger singleton pattern.
59 |
60 | Verifies that logger instances are singletons.
61 | """
62 | from src.logger.logger import Logger
63 |
64 | logger1 = Logger()
65 | logger2 = Logger()
66 |

```

```

67 | assert logger1 is logger2
68 |
69 | def test_logger_methods(self):
70 | """Test logger methods exist.
71 |
72 | Verifies that all standard logging methods are available.
73 | le.
74 | """
75 | from src.logger.logger import Logger
76 |
77 | logger = Logger()
78 |
79 | # Check that methods exist
80 | assert hasattr(logger, 'info')
81 | assert hasattr(logger, 'error')
82 | assert hasattr(logger, 'warning')
83 | assert hasattr(logger, 'debug')
84 | assert hasattr(logger, 'critical')
85 |
86 | def test_logger_log(self):
87 | """Test logger functionality.
88 |
89 | Verifies that global logger instance is accessible.
90 | """
91 | from src.logger.logger import logger
92 |
93 | # Check that global logger is available
94 | assert logger is not None
95 | assert hasattr(logger, 'info')
96 |
97 | class TestLogAnalyzer:
98 | """Tests for log_analyzer module."""
99 |
100 | def test_get_max_size_bytes(self):
101 | """Test retrieval of maximum log size.
102 |
103 | Verifies that max log size bytes calculation returns positive value.

```



```
103 | """
104 | from src.logger.log_analyzer import get_max_size_bytes
105 |
106 | result = get_max_size_bytes()
107 |
108 | assert result > 0
109 |
```

## test\_logger\_enhanced.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Fixture for creating logger with temporary paths
4 # =====
5 # Description:
6 # Comprehensive testing of all public functions and classes
7 # of core/logger module.
8 #
9 # File: test_logger_enhanced.py
10 # Project: ai-breadboard
11 # Package: tests
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15 """Enhanced logger module tests.
16
17 Tests for core logger module with temporary file handling and
18 verification."""
19
20 import pytest
21 import os
22 import json
23 import logging
24 from pathlib import Path
25 from src.logger.logger import Logger
26
27 @pytest.fixture
28 def temp_logger(tmp_path):
29 """Fixture for creating logger with temporary paths.
30
31 Creates a logger instance configured with temporary direct
```

```

ories
31 for log file storage to ensure test isolation.
32 """
33 log_dir = tmp_path / "logs"
34 log_dir.mkdir()
35
36 # Initialize logger with temporary directory paths
37 logger = Logger(
38 info_log_path="info.log",
39 debug_log_path="debug.log",
40 errors_log_path="errors.log",
41 json_log_path="log.json"
42)
43
44 # Override logger paths with temporary paths for test isolation
45 logger.log_files_path = log_dir
46 logger.info_log_path = log_dir / "info.log"
47 logger.debug_log_path = log_dir / "debug.log"
48 logger.errors_log_path = log_dir / "errors.log"
49 logger.json_log_path = log_dir / "log.json"
50
51 # Recreate handlers for new paths
52 for logger_obj in [logger.logger_file_info, logger.logger_
file_debug, logger.logger_file_errors, logger.logger_file_json
]:
53 for handler in logger_obj.handlers[:]:
54 logger_obj.removeHandler(handler)
55
56 # Add new handlers with temporary paths
57 info_handler = logging.FileHandler(logger.info_log_path, e
ncoding='utf-8')
58 info_handler.setFormatter(logging.Formatter("%(levelname)s
: %(message)s"))
59 logger.logger_file_info.addHandler(info_handler)
60
61 debug_handler = logging.FileHandler(logger.debug_log_path,
encoding='utf-8')

```

```

62 debug_handler.setFormatter(logging.Formatter("%(levelname)
s: %(message)s"))
63 logger.logger_file_debug.addHandler(debug_handler)
64
65 errors_handler = logging.FileHandler(logger.errors_log_pat
h, encoding='utf-8')
66 errors_handler.setFormatter(logging.Formatter("%(levelname
)s: %(message)s"))
67 logger.logger_file_errors.addHandler(errors_handler)
68
69 from src.logger.logger import JsonFormatter
70 json_handler = logging.FileHandler(logger.json_log_path, e
ncoding='utf-8')
71 json_handler.setFormatter(JsonFormatter())
72 logger.logger_file_json.addHandler(json_handler)
73
74 return logger
75
76 def test_logger_file_writing(temp_logger):
77 """Test writing logs to files.
78
79 Verifies that log messages are correctly written to the in
fo log file.
80 """
81 # --- Arrange ---
82 message: str = "Test info message"
83
84 # --- Act ---
85 temp_logger.info(message)
86
87 # --- Assert ---
88 # Check that info.log file was created and contains the me
ssage
89 assert temp_logger.info_log_path.exists(), "info.log file
was not created"
90 with open(temp_logger.info_log_path, 'r', encoding='utf-8'
) as f:
91 content = f.read()

```

```

92 | assert message in content, f"Message '{message}' not f
 | ound in info.log, got: {content}"
93 |
94 | def test_logger_json_writing(temp_logger):
95 | """Test writing logs to JSON file.
96 |
97 | Verifies that log entries are correctly serialized in JSON
 | format.
98 | """
99 | # --- Arrange ---
100 | message: str = "Test JSON message"
101 |
102 | # --- Act ---
103 | temp_logger.info(message)
104 |
105 | # --- Assert ---
106 | assert temp_logger.json_log_path.exists(), "log.json file
 | was not created"
107 | with open(temp_logger.json_log_path, 'r', encoding='utf-8'
 |) as f:
108 | line = f.readline()
109 | log_data = json.loads(line)
110 | assert log_data['message'] == message, f"JSON message
 | mismatch: {log_data['message']}"
111 |
112 | def test_logger_debug_filter(temp_logger):
113 | """Test DEBUG level filtering in PROD mode (is_debug_mode=
 | False).
114 |
115 | Verifies that debug messages are not logged when debug mod
 | e is disabled.
116 | """
117 | # --- Arrange ---
118 | temp_logger.is_debug_mode = False
119 | message: str = "Debug message"
120 |
121 | # --- Act ---
122 | temp_logger.debug(message)

```

```
123 |
124 | # --- Assert ---
125 | # Check that message was not added to debug.log
126 | with open(temp_logger.debug_log_path, 'r', encoding='utf-8
 | ') as f:
127 | content = f.read()
128 | assert message not in content, "DEBUG message was logg
 | ed in PROD mode"
129 |
```

test\_manage\_tools.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: # =====
4 # =====
5 # Description:
6 # Tests for manage_tools.py CLI interface.
7 #
8 # File: test_manage_tools.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """Tests for manage_tools.py CLI interface."""
16
17 import subprocess
18 import sys
19 from pathlib import Path
20 from unittest.mock import patch, MagicMock
21
22 import pytest
23
24 # Test data directory
25 TEST_DATA_DIR = Path(__file__).parent / "data" / "test_manage_
 tools"
26 TEST_DATA_DIR.mkdir(parents=True, exist_ok=True)
27
28 class TestManageToolsHelp:
29 """Test help output and argument parsing."""
30
31 def test_help_shows_all_commands(self):
```

```

32 | """Verify help displays all available commands."""
33 | result = subprocess.run(
34 | [sys.executable, "manage_tools.py", "--help"],
35 | capture_output=True,
36 | text=True,
37 | cwd=Path(__file__).parent.parent
38 |)
39 | assert result.returncode == 0
40 | assert "rag" in result.stdout
41 | assert "skills" in result.stdout
42 | assert "knowledge" in result.stdout
43 | assert "docs" in result.stdout
44 | assert "assist" in result.stdout
45 |
46 | def test_rag_help(self):
47 | """Verify rag subcommand help."""
48 | result = subprocess.run(
49 | [sys.executable, "manage_tools.py", "rag", "--help
50 |],
51 | capture_output=True,
52 | text=True,
53 | cwd=Path(__file__).parent.parent
54 |)
55 | assert result.returncode == 0
56 | assert "rebuild" in result.stdout
57 | assert "status" in result.stdout
58 | assert "validate" in result.stdout
59 |
60 | def test_skills_help(self):
61 | """Verify skills subcommand help."""
62 | result = subprocess.run(
63 | [sys.executable, "manage_tools.py", "skills", "--h
64 | elp"],
65 | capture_output=True,
66 | text=True,
67 | cwd=Path(__file__).parent.parent
68 |)
69 | assert result.returncode == 0

```



```

68 | assert "list" in result.stdout
69 | assert "search" in result.stdout
70 | assert "show" in result.stdout
71 | assert "export" in result.stdout
72 |
73 | def test_knowledge_help(self):
74 | """Verify knowledge subcommand help."""
75 | result = subprocess.run(
76 | [sys.executable, "manage_tools.py", "knowledge", "
|--help"],
77 | capture_output=True,
78 | text=True,
79 | cwd=Path(__file__).parent.parent
80 |)
81 | assert result.returncode == 0
82 | assert "extract" in result.stdout
83 | assert "add" in result.stdout
84 | assert "init" in result.stdout
85 |
86 | class TestSkillsCommand:
87 | """Test skills command functionality."""
88 |
89 | def test_skills_list_returns_zero(self):
90 | """skills list should return exit code 0."""
91 | result = subprocess.run(
92 | [sys.executable, "manage_tools.py", "skills", "lis
t"],
93 | capture_output=True,
94 | text=True,
95 | cwd=Path(__file__).parent.parent
96 |)
97 | assert result.returncode == 0
98 |
99 | def test_skills_search_returns_zero(self):
100 | """skills search should return exit code 0."""
101 | result = subprocess.run(
102 | [sys.executable, "manage_tools.py", "skills", "sea
rch", "media"],

```

```

103 capture_output=True,
104 text=True,
105 cwd=Path(__file__).parent.parent
106)
107 assert result.returncode == 0
108
109 def test_skills_show_nonexistent_returns_error(self):
110 """skills show with non-existent skill should return e
111 rror."""
112 result = subprocess.run(
113 [sys.executable, "manage_tools.py", "skills", "sho
114 w", "nonexistent_skill_12345"],
115 capture_output=True,
116 text=True,
117 cwd=Path(__file__).parent.parent
118)
119 assert result.returncode == 1
120 assert "Error" in result.stdout or "Error" in result.s
121 tderr
122
123 class TestRagCommand:
124 """Test rag command functionality."""
125
126 def test_rag_status_returns_zero(self):
127 """rag status should return exit code 0."""
128 result = subprocess.run(
129 [sys.executable, "manage_tools.py", "rag", "status
130 "],
131 capture_output=True,
132 text=True,
133 cwd=Path(__file__).parent.parent
134)
135 assert result.returncode == 0
136
137 def test_rag_unknown_subcommand_returns_error(self):
138 """rag with unknown subcommand should return error."""
139 result = subprocess.run(
140 [sys.executable, "manage_tools.py", "rag", "unknow

```

```

 n_command_12345"],
137 capture_output=True,
138 text=True,
139 cwd=Path(__file__).parent.parent
140)
141 assert result.returncode == 1
142
143 class TestKnowledgeCommand:
144 """Test knowledge command functionality."""
145
146 def test_knowledge_unknown_subcommand_returns_error(self):
147 """knowledge with unknown subcommand should return error.
148 or."""
149 result = subprocess.run(
150 [sys.executable, "manage_tools.py", "knowledge", "
151 unknown_command_12345"],
152 capture_output=True,
153 text=True,
154 cwd=Path(__file__).parent.parent
155)
156 assert result.returncode == 1
157
158 class TestDocsCommand:
159 """Test docs command functionality."""
160
161 def test_docs_unknown_subcommand_returns_error(self):
162 """docs with unknown subcommand should return error."""
163 "
164 result = subprocess.run(
165 [sys.executable, "manage_tools.py", "docs", "unkno
166 wn_command_12345"],
167 capture_output=True,
168 text=True,
169 cwd=Path(__file__).parent.parent
170)
171 assert result.returncode == 1
172
173 class TestUnknownCommand:

```

```

170 """Test handling of unknown main commands."""
171
172 def test_unknown_main_command_returns_error(self):
173 """Unknown main command should return error and show h
elp."""
174 result = subprocess.run(
175 [sys.executable, "manage_tools.py", "unknown_comma
nd_12345"],
176 capture_output=True,
177 text=True,
178 cwd=Path(__file__).parent.parent
179)
180 assert result.returncode == 1
181
182 class TestAssistCommand:
183 """Test assist command forwarding."""
184
185 def test_assist_help_returns_zero(self):
186 """assist help should be forwarded to assist_cli."""
187 result = subprocess.run(
188 [sys.executable, "manage_tools.py", "assist", "--h
elp"],
189 capture_output=True,
190 text=True,
191 cwd=Path(__file__).parent.parent
192)
193 # assist_cli may return 0 (help) or exit before proces
sing
194 assert result.returncode in (0, 2) # 2 is argparse er
ror code
195
196 class TestRunScript:
197 """Test _run_script helper function."""
198
199 def test_run_script_nonexistent_file(self):
200 """_run_script should return 1 for nonexistent script.
"""
201 from manage_tools import _run_script

```

```

202 result = _run_script("nonexistent_script_12345.py")
203 assert result == 1
204
205 def test_run_script_with_args(self):
206 """_run_script should pass extra arguments to script."""
207
208 # Test with a simple script that prints arguments
209 test_script = TEST_DATA_DIR / "echo_args.py"
210 test_script.write_text('import sys; print(" ".join(sys
211 .argv[1:])); sys.exit(0)')
212
213 from manage_tools import _run_script
214 result = _run_script(str(test_script.relative_to(Path(
215 __file__).parent.parent)), ["arg1", "arg2"])
216
217 assert result == 0
218
219 @pytest.fixture
220 def mock_skill_registry(monkeypatch):
221 """Create a mock SkillRegistry for testing."""
222 mock_registry = MagicMock()
223 mock_skill = MagicMock()
224 mock_skill.name = "test-skill"
225 mock_skill.description = "Test skill description"
226 mock_skill.prompt.return_value = "Test prompt content"
227
228 mock_registry.discover.return_value = [mock_skill]
229 mock_registry.search.return_value = [mock_skill]
230 mock_registry.get.return_value = mock_skill
231 mock_registry.export_json.return_value = '{"name": "test-s
232 kill"}'
233
234 return mock_registry
235
236 class TestIntegration:
237 """Integration tests for full CLI workflow."""
238
239 def test_full_cli_invocation(self):

```

```
236 | """Test complete CLI invocation with subcommand."""
237 | result = subprocess.run(
238 | [sys.executable, "manage_tools.py", "skills", "lis
 | t"],
239 | capture_output=True,
240 | text=True,
241 | cwd=Path(__file__).parent.parent
242 |)
243 | assert result.returncode == 0
244 |
245 | def test_command_without_subcommand_shows_help(self):
246 | """Command without subcommand should show help and ret
 | urn 0."""
247 | result = subprocess.run(
248 | [sys.executable, "manage_tools.py", "skills"],
249 | capture_output=True,
250 | text=True,
251 | cwd=Path(__file__).parent.parent
252 |)
253 | assert result.returncode == 0
254 |
255 | if __name__ == "__main__":
256 | pytest.main([__file__, "-v"])
```

test\_openai\_hf\_onnx\_providers.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Test bidirectional model identifier mapping
4 # =====
5 # Description:
6 # Test bidirectional mapping between internal and OpenAI-format model identifiers.
7 #
8 # File: test_openai_hf_onnx_providers.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """Test OpenAI-compatible, HuggingFace, and ONNX provider integration.
16
17 Tests model ID mapping, provider routing, and chat completion endpoints."""
18
19 import pytest
20 from unittest.mock import AsyncMock, patch
21 from starlette.testclient import TestClient
22
23 from src.fastapi.router_openai import map_to_openai_id, map_from_openai_id
24 from src.ai.hf_chat import hf_client, HFChatBase
25 from src.ai.onnx_chat import onnx_client, ONNXChatBase
26 from src.ai.openai_compat_chat import OpenAICompatChat
27 from src.ai.converter.gguf_to_onnx import gguf_converter
28 from src.ai.model_manager import get_available_models
29 from main import app
```

```

30
31 def test_openai_id_mapping():
32 """Test bidirectional model identifier mapping.
33
34 Verifies that internal model IDs map to OpenAI format and
35 back.
36 """
37 assert map_to_openai_id("foundry:qwen2.5-1.5b") == "foundr
38 y-qwen2.5-1.5b"
39 assert map_to_openai_id("hf:Qwen/Qwen2.5-0.5B-Instruct") =
40 = "hf-Qwen/Qwen2.5-0.5B-Instruct"
41 assert map_to_openai_id("onnx:models/gemma") == "onnx-mode
42 ls/gemma"
43 assert map_to_openai_id("openai:gpt-4o") == "openai-gpt-4o
44 "
45
46 assert map_from_openai_id("foundry-qwen2.5-1.5b") == "foun
47 dry:qwen2.5-1.5b"
48 assert map_from_openai_id("hf-Qwen/Qwen2.5-0.5B-Instruct")
49 == "hf:Qwen/Qwen2.5-0.5B-Instruct"
50 assert map_from_openai_id("onnx-models/gemma") == "onnx:mo
51 dels/gemma"
52 assert map_from_openai_id("openai-gpt-4o") == "openai:gpt-
53 4o"
54
55 def test_model_manager_new_providers():
56 """Test model list retrieval for HuggingFace, ONNX, and Op
57 enAI providers.
58
59 Verifies that each provider returns a non-empty list of av
60 ailable models.
61 """
62 hf_models = get_available_models("hf", force_refresh=True)
63 assert isinstance(hf_models, list)
64 assert len(hf_models) > 0
65
66 openai_models = get_available_models("openai", force_refre
67 sh=True)

```



```

56 assert isinstance(openai_models, list)
57 assert len(openai_models) > 0
58 assert "gpt-4o" in openai_models or "gpt-4o-mini" in opena
i_models
59
60 onnx_models = get_available_models("onnx", force_refresh=T
rue)
61 assert isinstance(onnx_models, list)
62
63 def test_gguf_converter_availability():
64 """Test GGUF converter availability status.
65
66 Verifies that converter and optimizer availability can be
checked.
67 """
68 status = gguf_converter.is_available()
69 assert isinstance(status, dict)
70 assert "converter" in status
71 assert "optimizer" in status
72
73 def test_openai_compat_chat_generate():
74 """Test content generation via OpenAICompatChat with mocke
d aiohttp.
75
76 Verifies that the OpenAI-compatible client can generate re
sponses.
77 """
78 async def _run():
79 client = OpenAICompatChat(model_id="gpt-4o-mini", api_
key="test-key")
80 with patch("aiohttp.ClientSession.post") as mock_post:
81 mock_response = AsyncMock()
82 mock_response.status = 200
83 mock_response.json = AsyncMock(return_value={
84 "choices": [{"message": {"content": "Hello fro
m OpenAI-compatible!"}}]}
85 })
86 mock_post.return_value.__aenter__.return_value = m

```

```

 ock_response
87
88 res = await client.generate_content("Hi")
89 assert res == "Hello from OpenAI-compatible!"
90
91 import asyncio
92 asyncio.run(_run())
93
94 def test_router_openai_list_models():
95 """Test GET /v1/models endpoint.
96
97 Verifies that the endpoint returns a list of models in OpenAI format.
98 """
99 client = TestClient(app)
100 response = client.get("/v1/models")
101 assert response.status_code == 200
102 data = response.json()
103 assert data.get("object") == "list"
104 assert isinstance(data.get("data"), list)
105 assert len(data.get("data")) > 0
106 # Check for required fields
107 first_model = data["data"][0]
108 assert "id" in first_model
109 assert "object" in first_model
110 assert "owned_by" in first_model
111
112 def test_router_openai_chat_completions():
113 """Test POST /v1/chat/completions endpoint.
114
115 Verifies that the endpoint processes chat completion requests correctly.
116 """
117 client = TestClient(app)
118 with patch("src.fastapi.router_openai.get_chat_model") as mock_get_model:
119 mock_chat = AsyncMock()
120 mock_chat.generate_content = AsyncMock(return_value="U

```

```

 universal assistant reply")
121 mock_get_model.return_value = mock_chat
122
123 payload = {
124 "model": "gemini-flash-latest",
125 "messages": [
126 {"role": "system", "content": "You are helpful
127 },
128 {"role": "user", "content": "Test prompt"}
129],
130 "temperature": 0.5,
131 "max_tokens": 100,
132 "stream": False
133 }
134 response = client.post("/v1/chat/completions", json=payload)
135 assert response.status_code == 200
136 data = response.json()
137 assert data.get("object") == "chat.completion"
138 assert len(data.get("choices", [])) == 1
139 assert data["choices"][0]["message"]["content"] == "Un
140 iversal assistant reply"
141
142 def test_openai_compat_chat_ask_and_chat():
143 """Test ask() and chat() methods with history for OpenAICompatChat.
144
145 Verifies both single requests and multi-turn conversations
146 work correctly.
147 """
148 async def _run():
149 client = OpenAICompatChat(model_id="gpt-4o", api_key="
150 test-key", system_prompt="Sys prompt")
151 with patch("aiohttp.ClientSession.post") as mock_post:
152 mock_response = AsyncMock()
153 mock_response.status = 200
154 mock_response.json = AsyncMock(return_value={
155 "choices": [{"message": {"content": "Answer fr

```

```

 om GPT-4o"}}}]
152 })
153 mock_post.return_value.__aenter__.return_value = m
 ock_response
154
155 # Test ask
156 ans_ask = await client.ask("Hello")
157 assert ans_ask == "Answer from GPT-4o"
158
159 # Test chat with history
160 history = [
161 {"role": "user", "parts": ["Previous query"]},
162 {"role": "model", "parts": ["Previous answer"]}
163]
164 ans_chat = await client.chat("Follow-up question",
 history=history)
165 assert ans_chat == "Answer from GPT-4o"
166
167 import asyncio
168 asyncio.run(_run())
169
170 def test_openai_compat_provider_factory():
171 """Test factory method create_for_provider.
172
173 Verifies that the factory correctly initializes clients fo
 r different providers.
174 """
175 client_deepseek = OpenAICompatChat.create_for_provider("de
 eepseek", "deepseek-chat")
176 assert client_deepseek.model_id == "deepseek-chat"
177 assert "deepseek.com" in client_deepseek.base_url
178
179 client_groq = OpenAICompatChat.create_for_provider("groq",
 "llama-3.3-70b")
180 assert client_groq.model_id == "llama-3.3-70b"
181 assert "groq.com" in client_groq.base_url
182

```

```

183 def test_unified_chat_openai_routing():
184 """Test UnifiedChatModel routing to OpenAICompatChat.
185
186 Verifies that the unified model correctly routes to OpenAI
187 -compatible provider.
188 """
189 async def _run():
190 from src.ai.unified_chat import UnifiedChatModel
191 chat_model = UnifiedChatModel(
192 api_key_names=[],
193 system_instruction="Test system",
194 foundry_model_id="qwen2.5",
195)
196 with patch.object(OpenAICompatChat, "chat", new_callable=AsyncMock) as mock_chat:
197 mock_chat.return_value = "Routed answer from OpenAI"
198 res = await chat_model.chat("Test prompt", model_name="openai:gpt-4o")
199 assert res == "Routed answer from OpenAI"
200
201 import asyncio
202 asyncio.run(_run())
203
204 def test_router_chat_models_endpoint():
205 """Test GET /api/chat/models endpoint returns all providers.
206
207 Verifies that the endpoint includes OpenAI, HuggingFace, O
208 NNX and other providers.
209 """
210 client = TestClient(app)
211 response = client.get("/api/chat/models")
212 assert response.status_code == 200
213 data = response.json()
214 assert "models" in data
215 models = data["models"]
216 assert "gemini" in models

```

```
215 | assert "foundry" in models
216 | assert "ollama" in models
217 | assert "openai" in models
218 | assert "hf" in models
219 | assert "onnx" in models
220 |
```

## test\_project\_installer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Testing Project Installer Skill and Verificati
 on Suite
4 # =====
5 # Description:
6 # Comprehensive unit and integration tests for AI Breadboard
 Project Installer
7 # skill, verification logic, directory structure checks, and
 CLI runners.
8 #
9 # File: tests/test_project_installer.py
10 # Project: AI Breadboard
11 # Package: tests
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import json
17 import subprocess
18 import sys
19 from pathlib import Path
20 import pytest
21
22 PROJECT_ROOT = Path(__file__).resolve().parent.parent
23 SKILL_DIR = PROJECT_ROOT / ".agents" / "skills" / "project-ins
 taller"
24 VERIFIER_SCRIPT = SKILL_DIR / "scripts" / "verify_installation
 .py"
25
26 # Import InstallationVerifier dynamically
27 sys.path.insert(0, str(SKILL_DIR / "scripts"))
28 from verify_installation import InstallationVerifier, Verifica
```

```

tionResult
29
30
31 class TestProjectInstallerSkillStructure:
32 """Test the structure and completeness of the project-inst
33 aller skill."""
34 def test_skill_directory_exists(self):
35 """Check that .agents/skills/project-installer directo
36 ry exists."""
37 assert SKILL_DIR.is_dir(), f"Skill directory not found
38 at {SKILL_DIR}"
39 def test_skill_md_exists_and_valid(self):
40 """Check that SKILL.md exists and contains required fr
41 ontmatter and instructions."""
42 skill_md = SKILL_DIR / "SKILL.md"
43 assert skill_md.is_file(), "SKILL.md is missing from p
44 roject-installer"
45 content = skill_md.read_text(encoding="utf-8")
46 assert "name: project-installer" in content, "SKILL.md
47 missing name frontmatter"
48 assert "gemini-3.1-flash-lite" in content, "SKILL.md m
49 issing gemini-3.1-flash-lite model reference"
50 assert "INSTALL-INSTRUCTION.md" in content, "SKILL.md
51 missing INSTALL-INSTRUCTION.md reference"
52 assert "verify_installation.py" in content, "SKILL.md
53 missing verification script reference"
54 def test_install_instruction_md_in_skill(self):
55 """Check that INSTALL-INSTRUCTION.md exists inside the
56 skill directory."""
57 instr_file = SKILL_DIR / "INSTALL-INSTRUCTION.md"
58 assert instr_file.is_file(), "INSTALL-INSTRUCTION.md m
59 issing in skill directory"
60 content = instr_file.read_text(encoding="utf-8")
61 assert "install.ps1" in content, "Missing Windows inst

```



```

all.ps1 reference in instruction"
55 assert "install.sh" in content, "Missing Linux/macOS i
install.sh reference in instruction"
56 assert len(content) > 300, "INSTALL-INSTRUCTION.md is
too short"
57
58 def test_install_instruction_md_in_root(self):
59 """Check that INSTALL-INSTRUCTION.md exists in the pro
ject root."""
60 root_instr = PROJECT_ROOT / "INSTALL-INSTRUCTION.md"
61 assert root_instr.is_file(), "INSTALL-INSTRUCTION.md m
issing in project root"
62 content = root_instr.read_text(encoding="utf-8")
63 assert "AI Breadboard" in content, "Root INSTALL-INSTR
UCTION.md missing project header"
64
65 def test_launchers_exist(self):
66 """Check that Gemini CLI runner scripts exist in the s
kill."""
67 ps1_launcher = SKILL_DIR / "scripts" / "run_installer_
gemini.ps1"
68 sh_launcher = SKILL_DIR / "scripts" / "run_installer_g
emini.sh"
69 assert ps1_launcher.is_file(), "run_installer_gemini.p
s1 is missing"
70 assert sh_launcher.is_file(), "run_installer_gemini.sh
is missing"
71
72 ps1_content = ps1_launcher.read_text(encoding="utf-8")
73 assert "gemini-3.1-flash-lite" in ps1_content, "PowerS
hell launcher missing model parameter"
74
75
76 class TestInstallationVerifier:
77 """Test the InstallationVerifier class and verification ch
ecks."""
78
79 @pytest.fixture

```

```

80 def verifier(self):
81 """Fixture returning an InstallationVerifier initializ
ed with PROJECT_ROOT."""
82 return InstallationVerifier(PROJECT_ROOT)
83
84 def test_check_directories(self, verifier):
85 """Verify that all required directories exist in the p
roject."""
86 result = VerificationResult(project_root=str(PROJECT_R
OOT))
87 verifier.check_directories(result)
88
89 assert len(result.missing_directories) == 0, (
90 f"Missing required directories: {result.missing_di
rectories}")
91)
92 assert "core" in result.existing_directories
93 assert "scripts" in result.existing_directories
94 assert "launchers" in result.existing_directories
95 assert "install" in result.existing_directories
96
97 def test_check_files(self, verifier):
98 """Verify that required files are detected properly."""
99
100 result = VerificationResult(project_root=str(PROJECT_R
OOT))
101 verifier.check_files(result)
102
103 assert "header.py" in result.existing_files
104 assert "config.json" in result.existing_files
105 assert "requirements.txt" in result.existing_files
106 assert "INSTALL-INSTRUCTION.md" in result.existing_fil
es
107
108 def test_verify_all_passes(self, verifier):
109 """Verify that full verification passes on the reposit
ory."""
110 result = verifier.verify_all()

```

```

110 assert result.is_valid is True, f"Full verification fa
 iled with errors: {result.errors}"
111 assert len(result.missing_directories) == 0
112 assert len(result.missing_files) == 0
113
114
115 class TestVerificationCli:
116 """Test execution of verify_installation.py as a CLI utili
 ty."""
117
118 def test_cli_json_output(self):
119 """Verify that running with --json flag outputs valid
 parseable JSON."""
120 cmd = [sys.executable, str(VERIFIER_SCRIPT), "--projec
 t-root", str(PROJECT_ROOT), "--json"]
121 proc = subprocess.run(cmd, capture_output=True, text=T
 rue, timeout=10)
122
123 assert proc.returncode == 0, f"Script failed with code
 {proc.returncode}: {proc.stderr}"
124 data = json.loads(proc.stdout)
125 assert data.get("is_valid") is True
126 assert len(data.get("missing_directories", [])) == 0
127 assert len(data.get("missing_files", [])) == 0
128
129 def test_cli_human_readable_output(self):
130 """Verify standard CLI formatting contains header and
 summary."""
131 cmd = [sys.executable, str(VERIFIER_SCRIPT), "--projec
 t-root", str(PROJECT_ROOT)]
132 proc = subprocess.run(cmd, capture_output=True, text=T
 rue, timeout=10)
133
134 assert proc.returncode == 0, f"Script failed: {proc.st
 derr}"
135 assert "AI Breadboard Installation Verification" in pr
 oc.stdout
136 assert "PASSED" in proc.stdout

```



## test\_qbittorrent\_config.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Testing qBittorrent configuration loading
4 # =====
5 # Description:
6 # Unit tests for qBittorrent configuration parameters loading
7 # from config.json
8 #
9 # File: test_qbittorrent_config.py
10 # Project: ai-breadboard
11 # Package: tests
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import json
17 from pathlib import Path
18 from types import SimpleNamespace
19 import pytest
20
21 from src.config import qbittorrent_cfg, qbt_cfg, CONFIG_FILE
22 from src.utils.jjson import j_loads_ns
23
24 class TestQBittorrentConfig:
25 """Testing optional qBittorrent configuration and compatibility."""
26
27 def test_core_config_exports_qbittorrent_cfg(self) -> None:
28 """Test safe export of qbittorrent_cfg and qbt_cfg in
29 core.config.
30
31 Check: Both configuration objects should be SimpleName
```

```

space instances
30 and qbt_cfg should reference same object as qbittorrent_cfg.
31 """
32 assert isinstance(qbittorrent_cfg, SimpleNamespace)
33 assert isinstance(qbt_cfg, SimpleNamespace)
34 assert qbt_cfg is qbittorrent_cfg
35
36 def test_j_loads_ns_parses_custom_namespace(self, tmp_path
37 : Path) -> None:
38 """Test correct deserialization of parameters via j_loads_ns.
39 Check: Custom configuration file correctly parsed into
40 nested SimpleNamespace with proper attribute access.
41 """
42 sample_config = tmp_path / "sample_config.json"
43 sample_config.write_text(
44 json.dumps(
45 {
46 "custom_service": {
47 "host": "192.168.1.50",
48 "port": 8080,
49 "user": "custom_user",
50 }
51 },
52),
53 encoding="utf-8",
54)
55
56 ns = j_loads_ns(sample_config)
57 assert hasattr(ns, "custom_service")
58 assert ns.custom_service.host == "192.168.1.50"
59 assert ns.custom_service.port == 8080
60 assert ns.custom_service.user == "custom_user"
61

```

test\_router\_admin\_users.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Testing user management endpoints in admin panel
4 # =====
5 # Description:
6 # Module contains tests for REST API endpoints /api/admin/users/*
7 #
8 # File: test_router_admin_users.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import pytest
16 from fastapi.testclient import TestClient
17 from main import app
18 from src.user_manager import user_manager
19
20 client = TestClient(app)
21
22 class TestAdminUsersAPI:
23 """Testing user management endpoints in admin panel."""
24
25 @pytest.fixture(autouse=True)
26 def setup_cleanup(self):
27 """Create test users and cleanup after tests."""
28 self.test_emails = [
29 "test_user_admin_1@test.com",
30 "test_user_admin_2@test.com",
31 "searchable_unique@test.com"
```

```

32 |]
33 | # Cleanup before test
34 | for email in self.test_emails:
35 | u = user_manager.get_user_by_email(email)
36 | if u:
37 | user_manager.delete_user(u["id"])
38 |
39 | yield
40 |
41 | # Cleanup after test
42 | for email in self.test_emails:
43 | u = user_manager.get_user_by_email(email)
44 | if u:
45 | user_manager.delete_user(u["id"])
46 |
47 | def test_list_users_and_stats(self):
48 | """Check retrieval of users list and statistics."""
49 | response = client.get("/api/admin/users")
50 | assert response.status_code == 200
51 |
52 | data = response.json()
53 | assert isinstance(data["users"], list)
54 | assert "stats" in data
55 |
56 | # Verify statistics contain expected keys
57 | assert "total" in data["stats"]
58 | assert "active" in data["stats"]
59 | assert "suspended" in data["stats"]
60 | assert "telegram" in data["stats"]
61 |
62 | # Passwords should not be in open form or hashed
63 | for u in data["users"]:
64 | assert "password_hash" not in u
65 |
66 | def test_create_user(self):
67 | """Check creation of new user via API."""
68 | payload = {
69 | "email": "test_user_admin_1@test.com",

```



```

70 "name": "Test User",
71 "password": "SecurePassword123!",
72 "role": "user",
73 }
74 response = client.post("/api/admin/users", json=payload)
75 d)
76 assert response.status_code == 200
77 assert response.json()["status"] == "ok"
78
79 data = response.json()
80 created = data["user"]
81 assert created["email"] == "test_user_admin_1@test.com"
82
83 assert created["name"] == "Test User"
84
85 def test_update_user(self):
86 """Check user update via API."""
87 # Create user first
88 create_payload = {
89 "email": "test_user_admin_2@test.com",
90 "name": "Original Name",
91 "password": "SecurePassword123!",
92 "role": "user",
93 }
94
95 create_resp = client.post("/api/admin/users", json=create_payload)
96
97 user_id = create_resp.json()["user"]["id"]
98
99 # Update user
100 update_payload = {
101 "name": "Updated Name",
102 "role": "moderator",
103 }
104
105 response = client.put(f"/api/admin/users/{user_id}", json=update_payload)
106
107 assert response.status_code == 200
108
109 data = response.json()

```

```
104 updated = data["user"]
105 assert updated["name"] == "Updated Name"
106 assert updated["role"] == "moderator"
107
108 def test_search_users(self):
109 """Check user search functionality."""
110 # Create test user
111 create_payload = {
112 "email": "searchable_unique@test.com",
113 "name": "Searchable User",
114 "password": "SecurePassword123!",
115 "role": "user",
116 }
117 client.post("/api/admin/users", json=create_payload)
118
119 # Search by email
120 response = client.get("/api/admin/users?q=searchable_u
unique")
121 assert response.status_code == 200
122 data = response.json()
123 assert len(data["users"]) > 0
124 assert any(u["email"] == "searchable_unique@test.com"
for u in data["users"])
125
```

```

test_router_agents.py
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Testing CRUD and helper endpoints for agents A
 PI
4 # =====
5 # Description:
6 # Module for test_router_agents.py in ai-breadboard project.
7 #
8 # File: test_router_agents.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import json
16 import pytest
17 from unittest.mock import AsyncMock, patch, MagicMock
18 from fastapi.testclient import TestClient
19
20 from main import app
21 from src.fastapi.router_agents import _get_agents_list, _save_
 agents_list
22
23 client = TestClient(app)
24
25 class TestAgentsRouter:
26 """Testing CRUD and helper endpoints /api/agents."""
27
28 @pytest.fixture(autouse=True)
29 def setup_teardown(self):
30 """Save and restore state of agents list."""
31 self.original_agents = _get_agents_list()

```

```

32 yield
33 _save_agents_list(self.original_agents)
34
35 def test_list_agents(self):
36 """Check retrieval of all agents list."""
37 response = client.get("/api/agents")
38 assert response.status_code == 200
39 agents = response.json()
40 assert isinstance(agents, list)
41 assert len(agents) > 0
42 # Check presence of key system agents
43 ids = [a.get("id") for a in agents]
44 assert "web_search_gemini" in ids
45 assert "web_search_gemini_cli" in ids
46
47 def test_list_tools(self):
48 """Check retrieval of tools catalog."""
49 response = client.get("/api/agents/tools")
50 assert response.status_code == 200
51 tools = response.json()
52 assert isinstance(tools, list)
53 tool_ids = [t.get("id") for t in tools]
54 assert "web_search" in tool_ids
55 assert "rag_search" in tool_ids
56 assert "python_eval" in tool_ids
57
58 def test_list_providers(self):
59 """Check list of providers and models from pool."""
60 response = client.get("/api/agents/providers")
61 assert response.status_code == 200
62 providers = response.json()
63 assert "gemini" in providers
64 assert "agy" in providers
65 assert "foundry" in providers
66 assert "ollama" in providers
67 assert len(providers["gemini"]["models"]) > 0
68
69 def test_create_and_delete_custom_agent(self):

```

```

70 | """Test creating and subsequent deletion of custom age
 | nt."""
71 | new_agent = {
72 | "id": "test_research_agent",
73 | "name": "Research Assistant",
74 | "description": "Test agent for information search"
 | ,
75 | "is_system": False,
76 | "enabled": True,
77 | "provider": "gemini",
78 | "model": "gemini-2.5-flash",
79 | "temperature": 0.2,
80 | "max_steps": 10,
81 | "timeout_seconds": 45,
82 | "tools": ["web_search", "rag_search"],
83 | "system_prompt": "You are a test research agent."
84 | }
85 |
86 | # 1. Creation
87 | create_res = client.post("/api/agents", json=new_agent
 |)
88 | assert create_res.status_code == 200
89 | data = create_res.json()
90 | assert data.get("status") == "ok"
91 | assert data.get("agent", {}).get("id") == "test_resear
 | ch_agent"
92 | assert data.get("agent", {}).get("is_system") is False
93 |
94 | # 2. Check in list
95 | list_res = client.get("/api/agents")
96 | agent_ids = [a.get("id") for a in list_res.json()]
97 | assert "test_research_agent" in agent_ids
98 |
99 | # 3. Update
100 | updated_agent = dict(new_agent)
101 | updated_agent["name"] = "Updated Research Assistant"
102 | updated_agent["enabled"] = False
103 | update_res = client.put("/api/agents/test_research_age

```

```

nt", json=updated_agent)
104 assert update_res.status_code == 200
105 assert update_res.json().get("agent", {}).get("name")
== "Updated Research Assistant"
106 assert update_res.json().get("agent", {}).get("enabled
") is False
107
108 # 4. Deletion
109 del_res = client.delete("/api/agents/test_research_age
nt")
110 assert del_res.status_code == 200
111 assert del_res.json().get("deleted_id") == "test_resea
rch_agent"
112
113 # 5. Check absence
114 list_after = client.get("/api/agents")
115 agent_ids_after = [a.get("id") for a in list_after.jso
n()]
116 assert "test_research_agent" not in agent_ids_after
117
118 def test_prevent_delete_system_agent(self):
119 """Check prevention of system agents deletion."""
120 del_res = client.delete("/api/agents/web_search_gemini
")
121 assert del_res.status_code == 403
122 assert "cannot be deleted" in del_res.json().get("deta
il", "").lower()
123
124 def test_prevent_duplicate_agent_id(self):
125 """Check error on duplicate agent ID creation."""
126 dup_agent = {
127 "id": "web_search_gemini", # Already exists
128 "name": "Duplicate Agent",
129 "description": "...",
130 "enabled": True,
131 "provider": "gemini",
132 "model": "gemini-2.5-flash",
133 "temperature": 0.2,

```

```

134 "max_steps": 10,
135 "timeout_seconds": 30,
136 "tools": [],
137 "system_prompt": ""
138 }
139 res = client.post("/api/agents", json=dup_agent)
140 assert res.status_code == 400
141 assert "already exists" in res.json().get("detail", "")
142).lower()
143 @patch("src.fastapi.router_chat.get_chat_model")
144 def test_generate_prompt_ai(self, mock_get_chat_model):
145 """Test system prompt generation via AI model."""
146 mock_llm = MagicMock()
147 mock_llm.ask = AsyncMock(return_value=json.dumps({
148 "name": "Data Analyst",
149 "description": "Analyzes data and performs calculations",
150 "system_prompt": "You are a data analyst.",
151 "recommended_tools": ["python_eval", "web_search"]
152 },
153 "temperature": 0.4,
154 "max_steps": 12
155)))
156 mock_get_chat_model.return_value = mock_llm
157 req_payload = {
158 "task_description": "Create an analytics agent for calculations",
159 "provider": "gemini",
160 "model": "gemini-2.5-flash"
161 }
162 response = client.post("/api/agents/generate-prompt",
163 json=req_payload)
164 assert response.status_code == 200
165 data = response.json()
166 assert data.get("status") == "ok"
167 spec = data.get("data", {})

```

```

167 assert spec.get("name") == "Data Analyst"
168 assert "python_eval" in spec.get("recommended_tools",
 | [])
169
170 @patch("src.fastapi.router_chat.get_chat_model")
171 def test_sandbox_execution(self, mock_get_chat_model):
172 """Test sandbox execution of test request."""
173 mock_llm = MagicMock()
174 mock_llm.ask = AsyncMock(return_value="Test agent resp
 | onse in sandbox.")
175 mock_get_chat_model.return_value = mock_llm
176
177 test_payload = {
178 "agent_id": "web_search_gemini",
179 "test_message": "What is the weather today?"
180 }
181 response = client.post("/api/agents/test", json=test_p
 | ayload)
182 assert response.status_code == 200
183 data = response.json()
184 assert data.get("status") == "ok"
185 assert "test" in data.get("response", "").lower()
186 assert len(data.get("steps", [])) > 0
187 assert data.get("duration_ms") >= 0
188

```



test\_router\_chat\_test\_model.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Test suite for test model verification endpoint
4 # =====
5 # Description:
6 # Comprehensive testing of /api/chat/test-model and /api/chat/models endpoints.
7 #
8 # File: test_router_chat_test_model.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 from __future__ import annotations
16
17 import unittest
18 from unittest.mock import AsyncMock, MagicMock, patch
19
20 from fastapi import FastAPI
21 from fastapi.testclient import TestClient
22
23 from src.fastapi.router_chat import TestModelRequest, init_router
24
25 class TestRouterChatTestModel(unittest.TestCase):
26 """Test suite for model verification request endpoint /api/chat/test-model."""
27
28 def setUp(self) -> None:
29 """Setup FastAPI test application and client before ea
```

```

ch test."""
30 # Creation of chat and narrator model stubs
31 self.mock_chat_model: MagicMock = MagicMock()
32 self.mock_chat_model.api_key = "fake_key_123"
33 self.mock_narrator_model: MagicMock = MagicMock()
34 self.mock_plugins: dict = {}
35
36 # Initialize chat router with dependency injection
37 self.router = init_router(
38 chat_model=self.mock_chat_model,
39 narrator_model=self.mock_narrator_model,
40 plugins=self.mock_plugins,
41)
42
43 # Create isolated FastAPI application for testing
44 self.app: FastAPI = FastAPI()
45 self.app.include_router(self.router)
46 self.client: TestClient = TestClient(self.app)
47
48 # =====
49 # 1. Happy Path Scenarios
50 # =====
51
52 @patch("src.fastapi.router_chat.get_chat_model")
53 def test_test_model_gemini_happy_path(self, mock_get_chat_
model: MagicMock) -> None:
54 """Test successful verification request to Gemini mode
l.
55
56 Check: endpoint returns success status, model response
and execution time.
57 """
58 # --- Setup input data (Arrange) ---
59 # Initialize mock AI model with ask method
60 mock_instance: MagicMock = MagicMock()
61 mock_instance.ask = AsyncMock(return_value="Test conne

```

```
 ction successful. I am Gemini model.")
62 mock_get_chat_model.return_value = mock_instance
63
64 # Prepare request payload
65 payload: dict[str, str] = {
66 "model": "gemini-3.7-flash",
67 "provider": "gemini",
68 "message": "Hello! Tell me your model name.",
69 "system_instruction": "You are a tester."
70 }
71
72 # --- Execution (Act) ---
73 response = self.client.post("/api/chat/test-model", js
on=payload)
74
75 # --- Check results (Assert) ---
76 self.assertEqual(response.status_code, 200, "Response
code should be 200 OK")
77 data: dict = response.json()
78 self.assertEqual(data.get("status", ""), "success", "R
esponse status should be success")
79 self.assertEqual(data.get("model", ""), "gemini-3.7-fl
ash", "Model name should match request")
80 self.assertEqual(data.get("provider", ""), "gemini", "
Provider name should match request")
81
```

## test\_save\_file.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Testing normal expected save_file scenarios
4 # =====
5 # Description:
6 # Comprehensive testing of all public functions and classes
7 # in module.
8 #
9 # File: test_save_file.py
10 # Project: ai-breadboard
11 # Package: tests
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import pytest
17 import os
18 from scripts.dev.save_file import save_file
19
20 class TestSaveFile_HappyPath:
21 """Testing normal expected scenarios of save_file operation."""
22
23 def test_save_file_success(self, tmp_path):
24 """Test saving file with correct data.
25
26 Check: file is created and content written correctly.
27 """
28 # --- Setup (Arrange) ---
29 # Temporary directory for test.
30 test_dir = tmp_path / "subdir"
31 # Path to file in directory.
32 test_file = test_dir / "test.txt"
```

```

32 | # Content to write.
33 | content = "Hello, world!"
34 |
35 | # --- Execution (Act) ---
36 | # Call save function.
37 | result = save_file(str(test_file), content)
38 |
39 | # --- Check (Assert) ---
40 | # File should be created and return True.
41 | assert result is True, "save_file should return True f
or correct input"
42 | assert test_file.exists(), "File should be created"
43 | assert test_file.read_text(encoding='utf-8') == conten
t, "File content does not match"
44 |
45 | class TestSaveFile_EdgeCases:
46 | """Testing boundary values and empty data."""
47 |
48 | def test_save_file_empty_content(self, tmp_path):
49 | """Test saving empty content.
50 |
51 | Check: empty string is written to file without errors.
52 | """
53 | # --- Setup (Arrange) ---
54 | # Path to temporary file.
55 | test_file = tmp_path / "empty.txt"
56 | # Empty string as content.
57 | content = ""
58 |
59 | # --- Execution (Act) ---
60 | # Call function.
61 | result = save_file(str(test_file), content)
62 |
63 | # --- Check (Assert) ---
64 | # Function should return True and create empty file.
65 | assert result is True, "Should return True for empty c
ontent"
66 | assert test_file.exists(), "File should be created"

```

```
67 | assert test_file.read_text(encoding='utf-8') == "", "F
 | file content should be empty"
68 |
69 | class TestSaveFile_ErrorScenarios:
70 | """Testing error scenario handling."""
71 |
72 | def test_save_file_invalid_path(self):
73 | """Test saving to inaccessible path.
74 |
75 | Check: on write error function should return False.
76 | """
77 | # --- Setup (Arrange) ---
78 | # Use path that cannot be created (on Windows).
79 | invalid_path = "Z:/invalid_directory/file.txt"
80 |
```

## test\_skill\_registry.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Testing universal skill registry
4 # =====
5 # Description:
6 # Tests for universal skill registry functionality and disco
7 # very.
8 # File: test_skill_registry.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """Tests for universal skill registry."""
16
17 from pathlib import Path
18
19 from src.skills import SkillRegistry
20
21 def _write_skill(root: Path, name: str, description: str, body
22 : str) -> None:
23 skill_dir = root / name
24 skill_dir.mkdir()
25 (skill_dir / "SKILL.md").write_text(
26 f"---\nname: {name}\ndescription: {description}\n---\n
27 \n{body}\n",
28 encoding="utf-8",
29)
30
31 def test_registry_discovers_gemini_and_agent_skill_roots(tmp_p
32 ath: Path) -> None:
```

```

30 gemini_root = tmp_path / ".gemini" / "skills"
31 agents_root = tmp_path / ".agents" / "skills"
32 gemini_root.mkdir(parents=True)
33 agents_root.mkdir(parents=True)
34 _write_skill(gemini_root, "media-manager", "Media library"
35 , "Start audit.")
36 _write_skill(agents_root, "db-inspector", "SQLite", "Check
37 schema.")
38
39 registry = SkillRegistry(tmp_path)
40
41 assert [skill.name for skill in registry.discover()] == ["
42 db-inspector", "media-manager"]
43 assert registry.get("MEDIA-MANAGER").prompt() == "Start au
44 dit."
45
46 def test_registry_returns_empty_search_and_rejects_unknown_ski
47 ll(tmp_path: Path) -> None:
48 registry = SkillRegistry(tmp_path)
49
50 assert registry.search("") == []
51 try:
52 registry.get("missing")
53 except KeyError as error:
54 assert "missing" in str(error), "Error should contain
55 name of missing skill"
56 else:
57 raise AssertionError("Unknown skill should raise KeyEr
58 ror")
59
60 def test_registry_prefers_json_contract_and_exports_portable_j
61 son(tmp_path: Path) -> None:
62 skills_root = tmp_path / ".gemini" / "skills"
63 skills_root.mkdir(parents=True)
64 _write_skill(skills_root, "storage", "Markdown description
65 ", "Read the Markdown instructions.")
66 (skills_root / "storage" / "skill.json").write_text(
67 '{"name": "storage", "description": "JSON description"

```



```
 , "providers": ["gemini", "ollama"]},
59 | encoding="utf-8",
60 |)
61 |
62 | exported = SkillRegistry(tmp_path).export_json("storage")
63 |
64 | assert '"description": "JSON description"' in exported
65 | assert '"providers": [' in exported
66 | assert "Read the Markdown instructions." in exported
67 |
```

## TEST\_TEMPLATE.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Test file template according to TDD-Doc-Gen standard
4 # =====
5 # Description:
6 # Exemplary test file template according to CODE_RULES.md §8.3 standard.
7 # Demonstrates correct commenting of every variable, every step of execution
8 # (Arrange/Act/Assert), and every assertion with descriptive messages.
9 #
10 # File: TEST_TEMPLATE.py
11 # Project: ai-breadboard
12 # Package: tests
13 # Author: hypo69
14 # Copyright: © 2026 hypo69
15 # =====
16
17 import pytest
18 from unittest.mock import Mock, patch, MagicMock
19 from pathlib import Path
20
21 # =====
22 # Section: Happy Path – Normal Scenarios
23 # =====
24
25 class TestTargetClass_HappyPath:
26 """Testing of normal (expected) scenarios of TargetClass operation.
```

```

27
28 Covers: correct input → expected output.
29 Dependencies: specify modules using TargetClass.
30 """
31
32 def test_method_name_with_valid_input(self):
33 """Testing of method with valid input parameters.
34
35 Verification: with valid input method returns expected
36 result.
37 Dependencies: this method is used in core/facade.py::p
38 rocess().
39 """
40 # --- Preparation of input data (Arrange) ---
41
42 # Integer identifier: standard positive user ID.
43 # Value 1 – minimum allowed non-zero identifier.
44 user_id: int = 1
45
46 # String of name: non-empty string with Latin characte
47 rs.
48 # Chosen to verify correct handling of Unicode charact
49 ers.
50 user_name: str = 'Ivan Ivanov'
51
52 # Creation of tested class instance with valid paramet
53 ers.
54 # Using real object (not mock) to verify business logi
55 c.
56 # target = TargetClass(user_id=user_id, user_name=user
57 _name)
58
59 # --- Execution (Act) ---
60
61 # Calling of tested method with valid parameters.
62 # Expected return value: True (successful execution).
63 # result: bool = target.method_name()

```

```

58 | # --- Verification (Assert) ---
59 |
60 | # Verification: method must return True for valid user
61 | .
62 | # Violation: False means method incorrectly processes
63 | valid input.
64 | # assert result is True, (
65 | # f"method_name() must return True for valid user
66 | "
67 | # f"(user_id={user_id}), received: {result!r}"
68 | #)
69 | pass # Replace with actual code
70 |
71 | def test_method_name_returns_correct_type(self):
72 | """Verification of return type of method.
73 |
74 | Verification: method must return dict type, not list o
75 | r str.
76 | Standard: return type is fixed in Docstring.
77 | """
78 | # --- Setup (Arrange) ---
79 |
80 | # Minimal data required to call method.
81 | # Use minimal set to isolate type checking.
82 | minimal_config: dict = {'key': 'value'}
83 |
84 | # --- Execution (Act) ---
85 |
86 | # Call result: expected dictionary with data.
87 | # result: dict = target.method_name(config=minimal_con
88 | fig)
89 |
90 | # --- Check (Assert) ---
91 |
92 | # Check return type.
93 | # Violation: incorrect type indicates incompatibility
94 | with calling code.
95 | # assert isinstance(result, dict), (

```

```

90 | # f"method_name() must return dict, received type:
 | {type(result).__name__}"
91 | #)
92 | pass
93 |
94 | # =====
 | =====
95 | # Section: Edge Cases – Edge Cases
96 | # =====
 | =====
97 |
98 | class TestTargetClass_EdgeCases:
99 | """Testing of boundary values and empty data.
100 |
101 | Covers: empty strings, zero values, empty collections.
102 | Goal: ensure that function correctly activates Early Return.
103 | """
104 |
105 | def test_method_name_empty_string(self):
106 | """Test edge case: empty string as argument.
107 |
108 | Check: empty string '' activates early return → False.
109 | Standard: CODE_RULES.md §3.4 – Early Return for invalid data.
110 | """
111 | # --- Setup (Arrange) ---
112 |
113 | # Edge value: empty string – minimal Invalid Input string.
114 | # Expected activation of branch `if not param_str: return False`.
115 | empty_string: str = ''
116 |
117 | # --- Execution (Act) ---
118 |
119 | # Call with empty string.
120 | # Function should return False without raising excepti

```

```

ons.
121 # result: bool = target.method_name(param=empty_string
)
122
123 # --- Check (Assert) ---
124
125 # Check: empty string must return False (Early Return)
 .
126 # Violation: return True means input validation is mis
sing.
127 # assert result is False, (
128 # f"method_name('') should return False, got: {res
ult!r}"
129 #)
130 pass
131
132 def test_method_name_zero_value(self):
133 """Test edge case: zero numeric value.
134
135 Check: value 0 for numeric parameter → False (Early Re
turn).
136
137 Rationale: 0 is valid type but invalid ID value.
138 """
139
140 # --- Setup (Arrange) ---
141
142 # Zero value: minimal invalid integer input.
143 # In ID context – 0 means "not set", which is unaccept
able.
144 zero_id: int = 0
145
146 # --- Execution (Act) ---
147
148 # result: bool = target.method_name(user_id=zero_id)
149
150 # --- Check (Assert) ---
151
152 # Zero ID should return False (identifier cannot be 0)
 .

```

```

151 # Violation: treating zero ID as valid breaks business
 logic.
152 # assert result is False, (
153 # f"method_name(user_id=0) should return False, got: {result!r}"
154 #)
155 pass
156
157 def test_method_name_empty_list(self):
158 """Test edge case: empty list as argument.
159
160 Check: empty [] activates early return → False.
161 """
162 # --- Setup (Arrange) ---
163
164 # Empty list: valid type, but no data to process.
165 empty_items: list = []
166
167 # --- Execution (Act) ---
168
169 # result: bool = target.method_name(items=empty_items)
170
171 # --- Check (Assert) ---
172
173 # assert result is False, (
174 # f"method_name([]) should return False, got: {result!r}"
175 #)
176 pass
177
178 # =====
179 # Section: Type Variants – variants of argument types
180 # =====
181
182 class TestTargetClass_TypeVariants:
183 """Testing of different valid input argument types.

```

```

184
185 Covers: all types specified in function annotations.
186 Goal: ensure that function correctly processes each valid
type.
187 """
188
189 @pytest.mark.parametrize("input_value,expected", [
190 # Integer: standard input for numeric parameter
191 (42, True),
192 # String-number: sometimes ID comes as string from req
uest
193 ('42', True),
194 # Zero: boundary numeric value → Early Return
195 (0, False),
196 # Empty string: boundary string value → Early Return
197 ('', False),
198])
199 def test_method_name_parametrized_input(self, input_value,
expected):
200 """Parametrized testing of different input data types.
201
202 Args:
203 input_value: Test input value (int or str).
204 expected (bool): Expected function return value.
205
206 Check: function correctly processes all valid types.
207 """
208 # --- Execution (Act) ---
209
210 # Call function with parametrized value.
211 # Expected result is specified in parametrize table above.
212 # result: bool = target.method_name(param=input_value)
213
214 # --- Check (Assert) ---
215
216 # Check if result matches expected value.
217 # Violation: mismatch indicates bug in type conversion

```



```

 logic.
218 # assert result is expected, (
219 # f"method_name({input_value!r}) should return {ex
pected!r}, "
220 # f"got: {result!r}"
221 #)
222 pass
223
224 # =====
=====
225 # Section: Error Scenarios – Invalid Input and exceptions
226 # =====
=====
227
228 class TestTargetClass_ErrorScenarios:
229 """Testing of error scenario handling.
230
231 Covers: invalid type, non-existent resources, exceptions.
232 Standard: CODE_RULES.md §3.4 – function should return False, not raise exception.
233 """
234
235 def test_method_name_invalid_type_returns_false(self):
236 """Test: invalid argument type → False (without exception).
237
238 Check: function must return False, not raise TypeError
239 .
240
241 Standard: CODE_RULES.md §3.6.3 – early return False instead of exceptions.
242 """
243 # --- Setup (Arrange) ---
244
245 # Invalid type: list instead of expected dictionary.
246 # Check function robustness against wrong input type.
247 invalid_input: list = [1, 2, 3]
248
249 # --- Execution (Act) ---

```

```

248 |
249 | # Expected return False without raising TypeError.
250 | # result: bool = target.method_name(param=invalid_inpu
 | t)
251 |
252 | # --- Check (Assert) ---
253 |
254 | # Function must not raise exceptions on invalid type.
255 | # assert result is False, (
256 | # f"method_name() with invalid type should return
 | False, "
257 | # f"got: {result!r}"
258 | #)
259 | pass
260 |
261 | def test_method_name_raises_on_critical_error(self):
262 | """Test: critical Error generates exception.
263 |
264 | Check: when external resource is unavailable, Connecti
 | onError is raised.
265 | Rationale: this is explicit critical Error, not "no da
 | ta".
266 | """
267 | # --- Setup (Arrange) ---
268 |
269 | # Mock external service: simulate unavailable resource
 | .
270 | # side_effect=ConnectionError() – force exception gene
 | ration.
271 | mock_service: Mock = Mock()
272 | mock_service.connect.side_effect = ConnectionError("Se
 | rvice unavailable")
273 |
274 | # Correct input: Error arises from service, not data.
275 | valid_input: dict = {'key': 'value'}
276 |
277 | # --- Execution and Check (Act + Assert) ---
278 |

```

```

279 # Check: function must raise ConnectionError.
280 # Violation: swallowing exception hides critical failure.
281 # with pytest.raises(ConnectionError, match="Service unavailable"):
282 # target.method_name(param=valid_input, service=mock_service)
283 pass
284
285 # =====
286 # Section: Regression – Regression Tests of dependent blocks
287 # =====
288
289 class TestTargetClass_Regression:
290 """Regression Tests: check impact of changes on dependent blocks.
291
292 IMPORTANT: these tests verify modules that use TargetClass
293 .
294 Goal: ensure that changes in TargetClass did not break dependent code.
295
296 Dependent blocks (fill after impact analysis in Step 2):
297 - core/facade.py::process() – uses TargetClass.method_name()
298 - core/api/endpoints.py::create_endpoint() – passes input to TargetClass
299
300 """
301 def test_dependent_facade_process_still_works(self):
302 """Regression test: Facade.process() works after TargetClass changes.
303
304 Check: change in TargetClass did not break Facade.process() interface.
305
306 Rationale: Facade – direct consumer of TargetClass, cr

```

```

 itical dependency.
305 """
306 # --- Setup (Arrange) ---
307
308 # Mock TargetClass: isolate test from real implementat
ion.
309 # return_value='expected_data' – simulate successful r
esponse.
310 mock_target: Mock = Mock()
311 mock_target.method_name.return_value = 'expected_data'
312
313 # Correct input data for Facade.
314 facade_input: dict = {'param': 'value'}
315
316 # --- Execution (Act) ---
317
318 # Call dependent method with mocked TargetClass.
319 # with patch('src.target_module.TargetClass', return_v
alue=mock_target):
320 # result = facade.process(facade_input)
321
322 # --- Check (Assert) ---
323
324 # Check: dependent method received data from TargetCla
ss.
325 # Violation: change in TargetClass.method_name() signa
ture broke Facade.
326 # assert mock_target.method_name.called, (
327 # "Facade.process() must call TargetClass.method_n
ame()")
328 #)
329 pass
330

```

## test\_tools\_structure.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Testing project directory structure
4 # =====
5 # Description:
6 # Checks structure of scripts directory for completeness and
7 # correctness.
8 #
9 # File: test_tools_structure.py
10 # Project: ai-breadboard
11 # Package: tests
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15 """
16 Tests for project directory structure.
17
18 Check presence of all required directories, files and launcher
19 s
20 according to agent-oriented project strategy.
21
22 Documentation: .ai_instructions/knowledge/LAUNCHER_GUIDE.md
23 """
24 import pytest
25 from pathlib import Path
26
27 # Project root is determined via header.py
28 PROJECT_ROOT = Path(__file__).resolve().parent.parent
29
30 class TestToolsDirectoryStructure:
31 """Check structure of scripts directory."""
```

```

32
33 def test_scripts_directory_exists(self):
34 """scripts/ directory must exist."""
35 assert (PROJECT_ROOT / "scripts").is_dir() or (PROJECT
36 _ROOT / "tools").is_dir(), \
37 "Scripts directory not found in project root"
38
39 def test_scripts_dev_directory_exists(self):
40 """scripts/dev/ directory must exist."""
41 assert (PROJECT_ROOT / "scripts" / "dev").is_dir() or
42 (PROJECT_ROOT / "tools" / "ai").is_dir()
43
44 def test_scripts_readme_exists(self):
45 """Scripts README.md must exist."""
46 assert (PROJECT_ROOT / "scripts" / "README.md").is_fil
47 e() or (PROJECT_ROOT / "tools" / "README.md").is_file()
48
49 class TestAiToolsExist:
50 """Check presence of key AI tools and development scripts.
51 """
52
53 def test_rebuild_dev_rag_exists(self):
54 """rebuild_dev_rag.py must exist."""
55 assert (PROJECT_ROOT / "scripts" / "maintenance" / "re
56 build_dev_rag.py").is_file() or (PROJECT_ROOT / "tools" / "ai"
57 / "rebuild_dev_rag.py").is_file()
58
59 def test_search_code_exists(self):
60 """search_code.py must exist."""
61 assert (PROJECT_ROOT / "scripts" / "dev" / "search_cod
62 e.py").is_file() or (PROJECT_ROOT / "tools" / "ai" / "search_c
63 ode.py").is_file()
64
65 def test_update_docs_exists(self):
66 """update_docs.py must exist."""
67 assert (PROJECT_ROOT / "scripts" / "dev" / "update_doc
68 s.py").is_file() or (PROJECT_ROOT / "tools" / "ai" / "update_d
69 ocs.py").is_file()

```

```

60
61 class TestReportsDirectory:
62 """Check tmp/reports/ directory."""
63
64 def test_reports_directory_exists(self):
65 """tmp/reports/ directory must exist."""
66 assert (PROJECT_ROOT / "tmp" / "reports").is_dir() or
67 (PROJECT_ROOT / "tmp").is_dir(), \
68 "tmp directory not found"
69
70 class TestSrcRagDirectory:
71 """Check src/rag/ directory."""
72
73 def test_src_rag_directory_exists(self):
74 """src/rag/ directory must exist."""
75 assert (PROJECT_ROOT / "src" / "rag").is_dir(), \
76 "src/rag/ directory not found"
77
78 def test_src_rag_models_exists(self):
79 """src/rag/models.py must exist."""
80 assert (PROJECT_ROOT / "src" / "rag" / "models.py").is_
81 _file(), \
82 "File src/rag/models.py not found"
83
84 class TestAiInstructionsDocuments:
85 """Check key AI documents."""
86
87 def test_launcher_guide_exists(self):
88 """LAUNCHER_GUIDE.md must exist."""
89 path = PROJECT_ROOT / ".ai" / "instructions" / "knowle
90 dge" / "LAUNCHER_GUIDE.md"
91 if not path.exists():
92 path = PROJECT_ROOT / ".ai_instructions" / "knowle
93 dge" / "LAUNCHER_GUIDE.md"
94 assert path.is_file(), "LAUNCHER_GUIDE.md not found"
95
96 def test_launcher_guide_not_empty(self):
97 """LAUNCHER_GUIDE.md should not be empty."""

```

```
94 | path = PROJECT_ROOT / ".ai" / "instructions" / "knowle
 | dge" / "LAUNCHER_GUIDE.md"
95 | if not path.exists():
96 | path = PROJECT_ROOT / ".ai_instructions" / "knowle
 | dge" / "LAUNCHER_GUIDE.md"
97 | assert path.stat().st_size > 200, \
98 | "LAUNCHER_GUIDE.md is too small – probably not fil
 | led"
99 |
100 | def test_gemini_md_has_launcher_guide_ref(self):
101 | """GEMINI.md should contain reference to LAUNCHER_GUID
 | E.md."""
102 | gemini_md = (PROJECT_ROOT / "GEMINI.md").read_text(enc
 | oding="utf-8")
103 | assert "LAUNCHER_GUIDE.md" in gemini_md
104 |
```



## test\_tts.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Tests for core/tts module
4 # =====
5 # Description:
6 # Module contains tests for speech synthesis (Text-to-Speech
7 #) module. Checks
8 #
9 # File: test_tts.py
10 # Project: ai-breadboard
11 # Package: tests
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 """
17 Tests for core/tts module
18 """
19
20 import pytest
21 import asyncio
22 from unittest.mock import Mock, patch, AsyncMock
23 from pathlib import Path
24
25 try:
26 import torch
27 has_torch = True
28 except ImportError:
29 has_torch = False
30
31 class TestTTSEdge:
32 """Tests for edge.py TTS."""
```

```

33 | @pytest.mark.asyncio
34 | async def test_synthesize_edge(self):
35 | """Test edge-tts speech synthesis."""
36 | from src.tts.edge import synthesize
37 |
38 | with patch('src.tts.edge') as mock_tts:
39 | # Check that function exists
40 | assert callable(synthesize)
41 |
42 | class TestTTSGTTS:
43 | """Tests for gtts.py TTS."""
44 |
45 | @pytest.mark.asyncio
46 | async def test_synthesize_gtts(self):
47 | """Test gtts speech synthesis."""
48 | from src.tts.gtts import synthesize
49 |
50 | with patch('src.tts.gtts') as mock_tts:
51 | assert callable(synthesize)
52 |
53 | @pytest.mark.skipif(not has_torch, reason="torch is not installed")
54 | class TestTTSSilero:
55 | """Tests for silero.py TTS."""
56 |
57 | def test_get_silero_model(self):
58 | """Test loading Silero model."""
59 | try:
60 | # Check that module can be imported
61 | from src.tts.silero import get_silero_model
62 | assert callable(get_silero_model)
63 | except ModuleNotFoundError as e:
64 | if 'pyaudioop' in str(e):
65 | pytest.skip("pyaudioop module not available")
66 | raise
67 |
68 | @pytest.mark.asyncio
69 | async def test_synthesize_silero(self):

```

```
70 | """Test Silero speech synthesis."""
71 | try:
72 | from src.tts.silero import synthesize
73 | assert callable(synthesize)
74 | except ModuleNotFoundError as e:
75 | if 'pyaudioop' in str(e):
76 | pytest.skip("pyaudioop module not available")
77 | raise
78 |
79 | class TestTTSInit:
80 | """Tests for __init__.py TTS."""
81 |
```

test\_user\_manager.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Tests for core/user_manager module
4 # =====
5 # Description:
6 # Module contains tests for user management module. Checks f
7 # or functionality.
8 #
9 # File: test_user_manager.py
10 # Project: ai-breadboard
11 # Package: tests
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15 """
16 Tests for core/user_manager module
17 """
18
19 import pytest
20 import sqlite3
21 from unittest.mock import Mock, patch
22 from pathlib import Path
23
24 class TestUserProfile:
25 """Tests for user_profile.py."""
26
27 def test_get_profile_path(self):
28 """Test retrieving profile path - check path structure
29 ."""
30 from src.user_manager.user_profile import _get_profile
31 _path
```

```

31 result = _get_profile_path(1)
32
33 assert isinstance(result, Path)
34 # Check that path contains user_profile_1
35 assert "user_profile_1" in str(result)
36 assert result.suffix == ".json"
37
38 def test_default_profile_structure(self):
39 """Test default profile structure."""
40 from src.user_manager.user_profile import _default_pro
file_structure
41
42 result = _default_profile_structure(1)
43
44 assert isinstance(result, dict)
45 # Check key fields of profile structure
46 assert 'user_id' in result
47 assert 'created_at' in result
48 assert 'updated_at' in result
49 assert 'watch_history' in result
50 assert 'last_watched' in result # Can be None
51 assert 'search_history' in result
52 assert 'preferences' in result # Uses 'preferences',
not 'settings'
53
54 def test_load_user_profile(self, tmp_path):
55 """Test loading profile."""
56 from src.user_manager.user_profile import load_user_pr
ofile
57
58 user_id = 1
59 profile_path = tmp_path / 'user_1_profile.json'
60
61 with patch('src.user_manager.user_profile._get_profile
_path') as mock_path:
62 mock_path.return_value = profile_path
63
64 # Profile does not exist - new one is created

```

```
65 | result = load_user_profile(user_id)
66 |
67 | assert isinstance(result, dict)
68 |
69 | def test_save_user_profile(self, tmp_path):
70 | """Test saving profile."""
71 | from src.user_manager.user_profile import save_user_pr
72 | ofile
73 |
74 | user_id = 1
75 | profile = {'last_watched': 'test'}
76 | profile_path = tmp_path / 'user_1_profile.json'
77 |
78 | with patch('src.user_manager.user_profile._get_profile
79 | _path') as mock_path:
80 | mock_path.return_value = profile_path
81 |
82 | result = save_user_profile(user_id, profile)
```

## test\_utils\_convertors\_dict.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Testing class for dict.py module functions
4 # =====
5 # Description:
6 # Comprehensive testing of dict.py module functions for dict
7 # ionary conversion.
8 #
9 # File: test_utils_convertors_dict.py
10 # Project: ai-breadboard
11 # Package: tests
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import pytest
17 from types import SimpleNamespace
18 from src.utils.convertors.dict import dict2ns, replace_key_in_
19 dict
20
21 class TestDictUtils:
22 """Class for testing dict.py module functions."""
23
24 def test_dict2ns_happy_path(self):
25 """Test normal scenario for converting dict to SimpleN
26 amespace."""
27
28 # --- Setup (Arrange) ---
29 data: dict = {"a": 1, "b": {"c": 2}}
30
31 # --- Execution (Act) ---
32 result = dict2ns(data)
33
34 # --- Check (Assert) ---
```

```
31 | assert isinstance(result, SimpleNamespace)
32 | assert result.a == 1
33 | assert result.b.c == 2
34 |
35 | def test_replace_key_in_dict_happy_path(self):
36 | """Test normal scenario for key replacement."""
37 | # --- Setup (Arrange) ---
38 | data: dict = {"old": 1, "nested": {"old": 2}}
39 |
40 | # --- Execution (Act) ---
41 | result = replace_key_in_dict(data, "old", "new")
42 |
43 | # --- Check (Assert) ---
44 | assert result == {"new": 1, "nested": {"new": 2}}
45 |
```



## test\_utils\_convertors\_html.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Testing class for html.py module functions
4 # =====
5 # Description:
6 # Comprehensive testing of html.py module functions for HTML
 conversion.
7 #
8 # File: test_utils_convertors_html.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import pytest
16 from types import SimpleNamespace
17 from src.utils.convertors.html import html2escape, escape2html
 , html2dict, html2ns
18
19 class TestHtmlUtils:
20 """Class for testing html.py module functions."""
21
22 def test_html2escape_happy_path(self):
23 """Test proper HTML tag escaping."""
24 # --- Setup (Arrange) ---
25 html: str = "<p>Hello</p>"
26 expected: str = "<p>Hello</p>"
27
28 # --- Execution (Act) ---
29 result: str = html2escape(html)
30
31 # --- Check (Assert) ---
```

```

32 | assert result == expected, f"Expected {expected!r}, got {result!r}"
33 |
34 | def test_escape2html_happy_path(self):
35 | """Test proper conversion of escape sequences to HTML.
36 |
37 | # --- Setup (Arrange) ---
38 | escaped: str = "<p>Hello</p>"
39 | expected: str = "<p>Hello</p>"
40 |
41 | # --- Execution (Act) ---
42 | result: str = escape2html(escaped)
43 |
44 | # --- Check (Assert) ---
45 | assert result == expected, f"Expected {expected!r}, got {result!r}"
46 |
47 | def test_html2dict_happy_path(self):
48 | """Test conversion of HTML to dictionary."""
49 | # --- Setup (Arrange) ---
50 | html: str = "<p>Hello</p><a>World"
51 | expected: dict = {"p": "Hello", "a": "World"}
52 |
53 | # --- Execution (Act) ---
54 | result: dict = html2dict(html)
55 |
56 | # --- Check (Assert) ---
57 | assert result == expected, f"Expected {expected!r}, got {result!r}"
58 |
59 | def test_html2ns_happy_path(self):
60 | """Test conversion of HTML to SimpleNamespace."""
61 | # --- Setup (Arrange) ---
62 | html: str = "<p>Hello</p><a>World"
63 |
64 | # --- Execution (Act) ---
65 | result = html2ns(html)

```

```
66 | # --- Check (Assert) ---
67 | assert isinstance(result, SimpleNamespace)
68 | assert result.p == "Hello"
69 | assert result.a == "World"
70 |
```

## test\_utils\_convertors\_json.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Testing class for json.py module functions
4 # =====
5 # Description:
6 # Comprehensive testing of json.py module functions for data
7 # conversion.
8 #
9 # File: test_utils_convertors_json.py
10 # Project: ai-breadboard
11 # Package: tests
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import pytest
17 import json
18 from pathlib import Path
19 from types import SimpleNamespace
20 from src.utils.convertors.json import json2csv, json2ns, json2
21 xml, json2xls
22
23 class TestJsonUtils:
24 """Class for testing json.py module functions."""
25
26 def test_json2ns_happy_path(self):
27 """Test conversion from JSON to SimpleNamespace."""
28 # --- Setup (Arrange) ---
29 data: dict = {"a": 1, "b": 2}
30
31 # --- Execution (Act) ---
32 result = json2ns(data)
```

```

32 | # --- Check (Assert) ---
33 | assert isinstance(result, SimpleNamespace)
34 | assert result.a == 1
35 | assert result.b == 2
36 |
37 | def test_json2xml_happy_path(self):
38 | """Test conversion from JSON to XML."""
39 | # --- Setup (Arrange) ---
40 | data: dict = {"a": 1}
41 |
42 | # --- Execution (Act) ---
43 | result = json2xml(data)
44 |
45 | # --- Check (Assert) ---
46 | # Check result as string (decode bytes if necessary)
47 | if isinstance(result, bytes):
48 | result = result.decode('utf-8')
49 | assert "<a>1" in result
50 |
51 | def test_json2xls_happy_path(self, tmp_path):
52 | """Test conversion from JSON to XLS."""
53 | import sys
54 | try:
55 | import xlswriter
56 | except ImportError:
57 | pytest.skip("xlswriter not installed")
58 |
59 | # --- Setup (Arrange) ---
60 | data: list = [{"a": 1, "b": 2}]
61 | xls_file = tmp_path / "test.xls"
62 |
63 | # --- Execution (Act) ---
64 | result = json2xls(data, xls_file)
65 |
66 | # --- Check (Assert) ---
67 | assert result is True
68 | assert xls_file.exists()
69 |

```

test\_utils\_date\_time.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Date and time utility module testing
4 # =====
5 # Description:
6 # Tests for interval method in date_time module.
7 #
8 # File: test_utils_date_time.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """
16 Tests for core/utils/date_time.py module
17 """
18
19 import pytest
20 from datetime import time
21 from unittest.mock import patch, Mock
22 from src.utils.date_time import TimeoutCheck
23
24 class TestTimeoutCheckInterval:
25 """Tests for interval method."""
26
27 def test_interval_same_day_morning(self):
28 """Test interval within one day (morning)."""
29 checker = TimeoutCheck()
30
31 with patch('src.utils.date_time.datetime') as mock_dt:
32 mock_dt.now.return_value.time.return_value = time(
33 10, 30)
```

```

33 |
34 | checker.interval(start=time(8, 0), end=time(17, 0)
35 |)
36 |
37 | assert checker.result is True
38 |
39 | def test_interval_same_day_outside(self):
40 | """Test interval outside time window (same day)."""
41 | checker = TimeoutCheck()
42 |
43 | with patch('src.utils.date_time.datetime') as mock_dt:
44 | mock_dt.now.return_value.time.return_value = time(
45 | 3, 0)
46 |
47 | checker.interval(start=time(8, 0), end=time(17, 0)
48 |)
49 |
50 | assert checker.result is False
51 |
52 | def test_interval_crosses_midnight(self):
53 | """Test interval crossing midnight."""
54 | checker = TimeoutCheck()
55 |
56 | with patch('src.utils.date_time.datetime') as mock_dt:
57 | mock_dt.now.return_value.time.return_value = time(
58 | 1, 0)
59 |
60 | checker.interval(start=time(23, 0), end=time(6, 0)
61 |)
62 |
63 | assert checker.result is True
64 |
65 | def test_interval_crosses_midnight_outside(self):
66 | """Test interval outside time window (crossing midnigh
67 | t)."""
68 | checker = TimeoutCheck()
69 |
70 | with patch('src.utils.date_time.datetime') as mock_dt:

```

```
65 | mock_dt.now.return_value.time.return_value = time(
66 | 12, 0)
67 | checker.interval(start=time(23, 0), end=time(6, 0)
68 |)
69 | assert checker.result is False
70 |
71 | def test_interval_exact_start(self):
72 | """Test exact start time."""
73 | checker = TimeoutCheck()
74 |
75 | with patch('src.utils.date_time.datetime') as mock_dt:
76 | mock_dt.now.return_value.time.return_value = time(
77 | 23, 0)
78 | checker.interval(start=time(23, 0), end=time(6, 0)
79 |)
80 | assert checker.result is True
81 |
```



## test\_utils\_file.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Testing class for file.py module functions
4 # =====
5 # Description:
6 # Comprehensive testing of file handling functions in file.p
7 # y module.
8 #
9 # File: test_utils_file.py
10 # Project: ai-breadboard
11 # Package: tests
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import pytest
17 import os
18 from pathlib import Path
19 from src.utils.file import save_text_file, read_text_file, get
20 _filenames, remove_bom
21
22 class TestFileUtils:
23 """Class for testing file.py module functions."""
24
25 def test_save_and_read_text_file_happy_path(self, tmp_path
26):
27 """Test normal scenario for writing and reading text.
28
29 Check: data written to file is correctly read back.
30 """
31 # --- Setup (Arrange) ---
32 # Temporary folder tmp_path (pytest fixture).
33 test_file: Path = tmp_path / "test.txt"
```

```

31 | content: str = "Test string"
32 |
33 | # --- Execution (Act) ---
34 | # Write text.
35 | save_result: bool = save_text_file(content, test_file)
36 | # Read text.
37 | read_result: str | None = read_text_file(test_file)
38 |
39 | # --- Check (Assert) ---
40 | assert save_result is True, "save_text_file() should r
| eturn True"
41 | assert read_result == content, f"Expected {content!r},
| got {read_result!r}"
42 |
43 | def test_save_and_read_dict_happy_path(self, tmp_path):
44 | """Test writing and reading dictionary in JSON format.
|
| """
45 | # --- Setup (Arrange) ---
46 | test_file: Path = tmp_path / "test.json"
47 | data: dict = {"key": "value"}
48 |
49 | # --- Execution (Act) ---
50 | save_result: bool = save_text_file(data, test_file)
51 | # When reading JSON file via read_text_file we get JSO
| N string.
52 | read_result_str: str | None = read_text_file(test_file
|)
53 |
54 | # --- Check (Assert) ---
55 | assert save_result is True
56 | # Check that file contains valid JSON structure.
57 | import json
58 | assert json.loads(read_result_str) == data
59 |
60 | def test_remove_bom(self, tmp_path):
61 | """Test function for BOM cleanup."""
62 | # --- Setup (Arrange) ---
63 | test_file: Path = tmp_path / "bom.txt"

```

```
64 | # Create file with BOM (UTF-8 signature).
65 | content_with_bom: str = "\ufeffText with BOM"
66 | test_file.write_text(content_with_bom, encoding="utf-8
 | ")
67 |
68 | # --- Execution (Act) ---
69 | remove_bom(test_file)
70 |
71 | # --- Check (Assert) ---
72 | content_without_bom = test_file.read_text(encoding="ut
 | f-8")
73 | assert "\ufeff" not in content_without_bom
74 | assert content_without_bom == "Text with BOM"
75 |
```

## test\_utils\_ftp.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Testing normal expected scenarios of FTP modul
 e
4 # =====
5 # Description:
6 # Comprehensive testing of all functions in core/utils/ftp m
 odule for FTP operations.
7 #
8 # File: test_utils_ftp.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import pytest
16 from unittest.mock import Mock, patch, mock_open
17 from src.utils.ftp import write, read, delete
18
19 # =====
20 # Section: Happy Path – Normal Scenarios
21 # =====
22
23 class TestFtp_HappyPath:
24 """Testing normal expected scenarios of FTP module operati
 on.
25 """
26
27 @patch('src.utils.ftp.ftplib.FTP')
28 def test_write_success(self, mock_ftp):
```

```

29 | """Test write function with correct data.
30 | """
31 | # --- Setup (Arrange) ---
32 | mock_session = mock_ftp.return_value
33 | with patch('builtins.open', mock_open()):
34 |
35 | # --- Execution (Act) ---
36 | result = write('test.txt', '/remote', 'test.txt')
37 |
38 | # --- Check (Assert) ---
39 | assert result is True
40 | mock_session.cwd.assert_called_with('/remote')
41 | mock_session.storbinary.assert_called()
42 |
43 | @patch('src.utils.ftp.ftplib.FTP')
44 | def test_write_success(self, mock_ftp):
45 | """Test write function with correct data.
46 | """
47 | # --- Setup (Arrange) ---
48 | mock_session = mock_ftp.return_value
49 | # Mock file operations for read
50 | with patch('builtins.open') as mock_file:
51 | # --- Execution (Act) ---
52 | result = read('test.txt', '/remote', 'test.txt')
53 |
54 | # --- Check (Assert) ---
55 | assert result is not None
56 | mock_session.cwd.assert_called_with('/remote')
57 | mock_session.retrbinary.assert_called()
58 |
59 | @patch('src.utils.ftp.ftplib.FTP')
60 | def test_read_success(self, mock_ftp):
61 | """Test read function with correct data.
62 | """
63 | # --- Setup (Arrange) ---
64 | mock_session = mock_ftp.return_value
65 |
66 | # --- Execution (Act) ---

```

```
67 | result = delete('test.txt', '/remote', 'test.txt')
68 |
69 | # --- Check (Assert) ---
70 | assert result is True
71 | mock_session.cwd.assert_called_with('/remote')
72 | mock_session.delete.assert_called_with('test.txt')
73 |
74 | # =====
75 | # Section: Edge Cases – Edge Cases
76 | # =====
77 |
78 | # Add more tests as needed per tdd-doc-gen requirements (empty
79 | inputs, etc.)
```

## test\_utils\_jjson.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Testing class for jjson module functions
4 # =====
5 # Description:
6 # Comprehensive testing of all public functions in jjson module for JSON handling.
7 #
8 # File: test_utils_jjson.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import pytest
16 import json
17 from pathlib import Path
18 from types import SimpleNamespace
19 from src.utils.jjson import j_dumps, j_loads, j_loads_ns
20
21 class TestJJson:
22 """Class for testing jjson module functions."""
23
24 def test_j_loads_happy_path_str(self):
25 """Test loading correct JSON string.
26
27 Check: j_loads correctly parses simple JSON string to dictionary.
28 """
29 # --- Setup (Arrange) ---
30 # Test JSON string: standard object with key 'a' and value 1.
```

```

31 | json_str: str = '{"a": 1}'
32 |
33 | # --- Execution (Act) ---
34 | # Call j_loads function to parse string.
35 | result: dict = j_loads(json_str)
36 |
37 | # --- Check (Assert) ---
38 | # Expected dictionary {'a': 1}.
39 | assert result == {'a': 1}, f"j_loads() should return {
 | {'a': 1}}, got: {result!r}"
40 |
41 | def test_j_dumps_happy_path_dict(self):
42 | """Test dumping dictionary to JSON (in memory).
43 |
44 | Check: j_dumps returns correct dictionary when file is
 | not specified.
45 | """
46 | # --- Setup (Arrange) ---
47 | # Test dictionary.
48 | data: dict = {'a': 1, 'b': 2}
49 |
50 | # --- Execution (Act) ---
51 | # Dump without specifying file (should return data).
52 | result: dict = j_dumps(data)
53 |
54 | # --- Check (Assert) ---
55 | assert result == data, f"j_dumps() should return {data
 | !r}, got: {result!r}"
56 |
57 | def test_j_loads_empty_str(self):
58 | """Test edge case: empty string.
59 |
60 | Check: empty string should return empty dictionary (er
 | ror in parsing logic).
61 | """
62 | # --- Setup (Arrange) ---
63 | empty_str: str = ""
64 |

```



```
65 | # --- Execution (Act) ---
66 | # Empty string causes parsing error inside string2dict
67 | .
68 | result = j_loads(empty_str)
69 |
70 | # --- Check (Assert) ---
71 | assert result == {}, f"j_loads() should return empty d
 | ictionary for empty string, got: {result!r}"
```

test\_utils\_url.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: URL utility functions testing
4 # =====
5 # Description:
6 # Tests for extract_url_params function and URL utility module.
7 #
8 # File: test_utils_url.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 """
16 Tests for core/utils/url.py module
17 """
18
19 import pytest
20 from unittest.mock import patch, Mock
21 from src.utils.url import extract_url_params, is_url, url_shortener
22
23 class TestExtractUrlParams:
24 """Tests for extract_url_params function."""
25
26 def test_extract_params_with_multiple_params(self):
27 """Test extracting multiple parameters."""
28 url = "https://example.com?param1=value1¶m2=value2"
29
30 result = extract_url_params(url)
```

```
31
32 assert result is not None
33 assert result['param1'] == 'value1'
34 assert result['param2'] == 'value2'
35
36 def test_extract_params_single_value(self):
37 """Test extracting single parameter."""
38 url = "https://example.com?key=value"
39
40 result = extract_url_params(url)
41
42 assert result is not None
43 assert result['key'] == 'value'
44
45 def test_extract_params_no_params(self):
46 """Test URL without parameters."""
47 url = "https://example.com/path"
48
49 result = extract_url_params(url)
50
51 assert result is None
52
53 def test_extract_params_empty_url(self):
54 """Test empty URL."""
55 result = extract_url_params("")
56
57 assert result is None
58
59 def test_extract_params_with_numeric_values(self):
60 """Test with numeric values."""
61 url = "https://example.com?page=1&limit=10"
62
63 result = extract_url_params(url)
64
65 assert result is not None
66 assert result['page'] == '1'
67 assert result['limit'] == '10'
68
```

```

69 | def test_extract_params_with_encoded_values(self):
70 | """Test with encoded values."""
71 | url = "https://example.com?query=%D0%BF%D1%80%D0%B8%D0
 | %B2%D0%B5%D1%82"
72 |
73 | result = extract_url_params(url)
74 |
75 | assert result is not None
76 | assert 'query' in result
77 |
78 | class TestIsUrl:
79 | """Tests for is_url function."""
80 |
81 | def test_valid_https_url(self):
82 | """Test valid HTTPS URL."""
83 | assert is_url("https://example.com") is True
84 |
85 | def test_valid_http_url(self):
86 | """Test valid HTTP URL."""
87 | assert is_url("http://example.com") is True
88 |
89 | def test_valid_url_with_path(self):
90 | """Test URL with path."""
91 | assert is_url("https://example.com/path/to/resource")
 | is True
92 |
93 | def test_valid_url_with_params(self):
94 | """Test URL with parameters."""
95 | assert is_url("https://example.com?param=value") is Tr
 | ue
96 |
97 | def test_invalid_string(self):
98 | """Test invalid string - validators may return False o
 | r raise Exception."""
99 | try:
100 | result = is_url("not a url")
101 | # validators returns False for invalid URLs
102 | assert result is False

```

```

103 except Exception:
104 pass # Also acceptable behavior
105
106 def test_empty_string(self):
107 """Test empty string."""
108 try:
109 result = is_url("")
110 assert result is False
111 except Exception:
112 pass
113
114 def test_domain_without_scheme(self):
115 """Test domain without scheme - validators treats as v
116 alid URL."""
117 result = is_url("example.com")
118 # validators may treat this as valid domain
119 assert result is True
120
121 def test_ip_address_url(self):
122 """Test URL with IP address."""
123 result = is_url("http://192.168.1.1:8080")
124 assert result is True
125
126 class TestUrlShortener:
127 """Tests for url_shortener function."""
128
129 def test_shortener_success(self):
130 """Test successful shortening."""
131 long_url = "https://example.com/very/long/url/path"
132
133 with patch('src.utils.url.requests.get') as mock_get:
134 mock_response = Mock()
135 mock_response.status_code = 200
136 mock_response.text = "https://tinyurl.com/abc123"
137 mock_get.return_value = mock_response
138
139 result = url_shortener(long_url)

```

```
140 assert result == "https://tinyurl.com/abc123"
141
142 def test_shortener_failure(self):
143 """Test failed shortening."""
144 long_url = "https://example.com/long/url"
145
146 with patch('src.utils.url.requests.get') as mock_get:
147 mock_response = Mock()
148 mock_response.status_code = 404
149 mock_get.return_value = mock_response
150
151 result = url_shortener(long_url)
152
153 assert result is None
154
155 def test_shortener_calls_correct_api(self):
156 """Test that correct API is called."""
157 long_url = "https://example.com"
158
159 with patch('src.utils.url.requests.get') as mock_get:
160 mock_response = Mock()
161 mock_response.status_code = 200
162 mock_response.text = "https://tinyurl.com/short"
163 mock_get.return_value = mock_response
164
165 url_shortener(long_url)
166
167 # Check that URL contains tinyurl
168 call_args = mock_get.call_args
169 assert 'tinyurl.com' in call_args[0][0]
170
171 class TestUrlEdgeCases:
172 """Tests for edge cases."""
173
174 def test_extract_params_with_duplicate_keys(self):
175 """Test parameters with duplicate keys."""
176 url = "https://example.com?key=value1&key=value2"
177
```

```
178 | result = extract_url_params(url)
179 |
180 | assert result is not None
181 | assert result['key'] == ['value1', 'value2']
182 |
183 | def test_extract_params_fragment(self):
184 | """Test URL with fragment."""
185 | url = "https://example.com/page#section"
186 |
187 | result = extract_url_params(url)
188 |
189 | # fragment should not be parsed as query param
190 | assert result is None
191 |
192 | def test_is_url_localhost(self):
193 | """Test localhost URL - validators treats as valid URL
194 | ."""
195 |
196 | # validators treats bare domain names as valid URLs
197 | result1 = is_url("http://localhost:8080")
198 | assert result1 is True
199 |
200 | result2 = is_url("http://127.0.0.1:8000")
201 | assert result2 is True
```

## test\_versioning.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Module
4 # =====
5 # Description:
6 # Module for AI Breadboard project.
7 #
8 # File: test_versioning.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import pytest
16
17 from src.utils.versioning import compare_versions, choose_best_tag
18
19 def test_compare_versions_basic():
20 assert compare_versions('1.2.3', '1.2.4') == -1
21 assert compare_versions('1.2.3', '1.2.3') == 0
22 assert compare_versions('v1.10.0', '1.9.9') == 1
23 assert compare_versions('1.2', '1.2.0') == 0
24
25 def test_compare_prerelease():
26 assert compare_versions('1.2.3-alpha', '1.2.3') == -1
27 assert compare_versions('1.2.3-alpha.1', '1.2.3-alpha.2')
28 == -1
29 assert compare_versions('1.2.3-alpha', '1.2.3-alpha') == 0
30
31 def test_choose_best_tag_prefers_stable():
32 tags = ['v1.0.0', 'v1.1.0-alpha', 'v1.0.1']
```



```
32 | best = choose_best_tag(tags, allow_prerelease=False, debug
 | =False)
33 | assert best == 'v1.0.1'
34 |
35 | def test_choose_best_tag_allows_prerelease():
36 | tags = ['v1.0.0', 'v1.1.0-alpha', 'v1.0.1']
37 | best = choose_best_tag(tags, allow_prerelease=True, debug=
 | False)
38 | # v1.1.0-alpha has higher major.minor.patch so should be p
 | referred when prerelease allowed
39 | assert best == 'v1.1.0-alpha'
40 |
```

## test\_file.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Testing text file save and read operations
4 # =====
5 # Description:
6 # Comprehensive testing of file handling functions for saving and reading text operations.
7 #
8 # File: test_file.py
9 # Project: ai-breadboard
10 # Package: tests.utils
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import pytest
16 from pathlib import Path
17 import os
18 from src.utils.file import save_text_file, read_text_file
19
20 # --- Tests for save_text_file ---
21
22 def test_save_text_file_happy_path(tmp_path):
23 """Test saving text to file (successful scenario)."""
24 # Arrange: path to temporary file
25 file_path = tmp_path / "test.txt"
26 data = "Hello, World!"
27
28 # Act: function call
29 result = save_text_file(data, file_path)
30
31 # Assert: check success and content
32 assert result is True, "save_text_file should return True"
```

```

 on successful write"
33 assert file_path.read_text(encoding="utf-8") == data, "File content does not match written data"
34
35 def test_read_text_file_happy_path(tmp_path):
36 """Test reading text from file (successful scenario)."""
37 # Arrange: path to temporary file
38 file_path = tmp_path / "test_read.txt"
39 data = "Hello, read!"
40 file_path.write_text(data, encoding="utf-8")
41
42 # Act: function call
43 result = read_text_file(file_path)
44
45 # Assert: check content
46 assert result == data, "Read content does not match written data"
47
48 def test_save_text_file_invalid_mode(tmp_path):
49 """Test write with invalid mode (expect error or False)."""
50 # Arrange
51 file_path = tmp_path / "invalid_mode.txt"
52 data = "data"
53
54 # Act
55 # If function properly handles exceptions via log, it should return False
56 result = save_text_file(data, file_path, mode="x") # 'x' mode: exclusive creation
57
58 # Assert
59 # Per standards, should return False on failure, not generate exception
60 # In reality 'x' creates file, but check how function handles write
61 assert result is True
62

```

## test\_get\_free\_port.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Testing for available port retrieval
4 # =====
5 # Description:
6 # Comprehensive testing of all public functions in the get_f
7 # ree_port module for finding available ports.
8 #
9 # File: test_get_free_port.py
10 # Project: ai-breadboard
11 # Package: tests.utils
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import pytest
17 from src.utils.get_free_port import get_free_port
18
19 def test_get_free_port_first_available():
20 """Test retrieving the first available port (without range
21).
22
23 Check: function should return integer starting from 1024.
24 """
25 # --- Setup (Arrange) ---
26 # Localhost host – standard for local testing.
27 host: str = 'localhost'
28
29 # --- Execution (Act) ---
30 # Search for first available port without restrictions.
31 port: int = get_free_port(host)
32
33 # --- Check (Assert) ---
```

```
32 | # Check: port should be >= 1024.
33 | assert port >= 1024, f"Port should be >= 1024, got: {port}"
34 | "
35 | def test_get_free_port_in_range():
36 | """Test retrieving port within specified range."""
37 | # --- Setup (Arrange) ---
38 | host: str = 'localhost'
39 | port_range: str = '3000-5000'
40 |
41 | # --- Execution (Act) ---
42 | port: int = get_free_port(host, port_range)
43 |
44 | # --- Check (Assert) ---
45 | assert 3000 <= port <= 5000, f"Port {port} outside range {
 | port_range}"
46 |
47 | def test_get_free_port_invalid_range():
48 | """Test error handling with invalid range."""
49 | # --- Setup (Arrange) ---
50 | host: str = 'localhost'
51 | port_range: str = 'invalid'
52 |
53 | # --- Execution (Act) & Check (Assert) ---
54 | with pytest.raises(ValueError):
55 | get_free_port(host, port_range)
56 |
```

## test\_header.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Project root finding and header module testing
4 # =====
5 # Description:
6 # Comprehensive testing of all public functions in the header module for locating project root.
7 #
8 # File: test_header.py
9 # Project: ai-breadboard
10 # Package: tests.utils
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15 import pytest
16 from pathlib import Path
17 from src.utils.header import set_project_root
18
19 def test_set_project_root_success():
20 """Test successful finding of project root.
21
22 Check: function should find directory with '__root__' marker.
23 """
24 # --- Setup (Arrange) ---
25 expected_root = Path(__file__).resolve().parents[2]
26
27 # --- Execution (Act) ---
28 root = set_project_root()
29
30 # --- Check (Assert) ---
31 assert root == expected_root, f"Project root not found, ex
```

```

 pected {expected_root}, got {root}"
32 |
33 | def test_set_project_root_nonexistent_marker():
34 | """Test finding root when markers do not exist."""
35 | # --- Setup (Arrange) ---
36 | # Marker that definitely does not exist in the project tree.
37 | marker = ('nonexistent_file_12345',)
38 |
39 | # --- Execution (Act) ---
40 | root = set_project_root(marker_files=marker)
41 |
42 | # --- Check (Assert) ---
43 | # Function behavior: if not found, returns script director
44 | y.
45 | assert isinstance(root, Path), "Result should be a Path object"

```

## test\_jjson.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: JSON and CSV file handling utilities testing
4 # =====
5 # Description:
6 # Testing functions for loading and saving JSON data with va
7 # rious formats and scenarios.
8 #
9 # File: test_jjson.py
10 # Project: ai-breadboard
11 # Package: tests.utils
12 # Author: hypo69
13 # Copyright: © 2026 hypo69
14 # =====
15
16 import pytest
17 import json
18 from pathlib import Path
19 from src.utils.jjson import j_loads, j_dumps
20 from types import SimpleNamespace
21
22 # --- Tests for j_dumps ---
23
24 def test_j_dumps_happy_path(tmp_path):
25 """Test saving JSON to file (successful scenario)."""
26 # Arrange: test data
27 data = {"key": "value"}
28 file_path = tmp_path / "test.json"
29
30 # Act: save
31 result = j_dumps(data, file_path=file_path)
32
33 # Assert
```



```
33 | assert result == data, "j_dumps should return original data"
34 |
35 | assert file_path.exists(), "File should be created"
36 | assert json.loads(file_path.read_text(encoding="utf-8")) == data, "File content does not match"
37 |
38 | # --- Tests for j_loads ---
39 |
40 | def test_j_loads_str_happy_path():
41 | """Test loading JSON from string (successful scenario)."""
42 | # Arrange
43 | json_str = '{"key": "value"}'
44 |
45 | # Act
46 | result = j_loads(json_str)
47 |
48 | # Assert
49 | assert result == {"key": "value"}, "Loaded data does not match"
50 |
51 | def test_j_loads_file_happy_path(tmp_path):
52 | """Test loading JSON from file (successful scenario)."""
53 | # Arrange
54 | file_path = tmp_path / "data.json"
55 | data = {"key": "value"}
56 | file_path.write_text(json.dumps(data), encoding="utf-8")
57 |
58 | # Act
59 | result = j_loads(file_path)
60 |
61 | # Assert
62 | assert result == data, "Loaded file data does not match"
```

\_\_init\_\_.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: Module
4 # =====
5 # Description:
6 # Module for AI Breadboard project.
7 #
8 # File: __init__.py
9 # Project: ai-breadboard
10 # Package: tests
11 # Author: hypo69
12 # Copyright: © 2026 hypo69
13 # =====
14
15
```